book 9

practical data structures and algorithms

an engineering handbook in seven languages

Raeliosol

version 7.0, compiled 10 2026

Contents

1. complexity analysis and honest benchmarking 9
   1.1. counting operations 9
   1.2. binary search, probed 14
   1.3. counting allocations 19
   1.4. reading a cost claim 26
   1.5. across the seven languages 27
2. arrays, spans, memory layout 28
   2.1. row major order 28
   2.2. the cache cost model 32
   2.3. spans over arrays 36
   2.4. bounds and safety 45
   2.5. across the seven languages 49
3. dynamic arrays and amortization 51
   3.1. the vector 51
   3.2. the doubling argument 56
   3.3. list versus linked, the decision 62
   3.4. across the seven languages 63
4. linked structures 64
   4.1. the singly linked list 64
   4.2. floyd's cycle detection 70
   4.3. the sentinel ring 74
   4.4. when to reach for links 81
   4.5. across the seven languages 82
5. stacks, queues, deques 83
   5.1. the stack 83
   5.2. the ring buffer 87
   5.3. bracket matching 93
   5.4. shunting yard 97
   5.5. the deque 104
   5.6. across the seven languages 110
6. hashing 112
   6.1. the hash function 112
   6.2. open addressing 117
   6.3. separate chaining 124
   6.4. the hash set 130
   6.5. across the seven languages 134
7. trees 136
   7.1. the search tree invariant 136
   7.2. deletion, the three cases 146
   7.3. avl, the rotation reflex 151
   7.4. what the bcl ships 159
   7.5. across the seven languages 160
8. heaps and priority queues 162
   8.1. the array heap 162
   8.2. heapsort 171
   8.3. streaming top-k 175
   8.4. the k-way merge 181
   8.5. the bcl priority queue 187

8.6. across the seven languages ... 188
9. tries and suffix structures ... 190
9.1. the byte-wise trie ... 190
9.2. the prefix scan ... 200
9.3. the suffix array ... 204
9.4. substring search by binary search ... 213
9.5. across the seven languages ... 220
10. graphs ... 222
10.1. representation ... 222
10.2. breadth first ... 227
10.3. depth first ... 231
10.4. topological order ... 239
10.5. connectivity ... 243
10.6. across the seven languages ... 250
11. shortest paths ... 252
11.1. dijkstra ... 252
11.2. bellman-ford ... 260
11.3. 0-1 bfs ... 265
11.4. d'esopo-pape ... 266
11.5. floyd-warshall ... 267
11.6. a star ... 270
11.7. across the seven languages ... 278
12. minimum spanning trees and flows ... 279
12.1. union-find ... 279
12.2. kruskal ... 285
12.3. prim ... 289
12.4. maximum flow, edmonds-karp ... 295
12.5. max-flow min-cut, verified ... 305
12.6. across the seven languages ... 310
13. sorting ... 312
13.1. insertion, the baseline ... 312
13.2. merge sort, with inversions ... 319
13.3. quicksort, defended ... 323
13.4. quickselect ... 331
13.5. counting and radix ... 337
13.6. the stability probe ... 344
13.7. across the seven languages ... 352
14. searching and answer spaces ... 353
14.1. boundaries, not hits ... 353
14.2. rotated arrays ... 359
14.3. the answer space ... 364
14.4. kadane's scan ... 371
14.5. mex ... 376
14.6. across the seven languages ... 382
15. string matching ... 383
15.1. the naive baseline ... 383
15.2. kmp ... 387
15.3. horspool ... 394
15.4. rabin-karp ... 402
15.5. aho-corasick ... 410
15.6. across the seven languages ... 418

16. number theory and modular arithmetic 419
16.1. gcd, lcm, and the binary variant 419
16.2. the sieve of eratosthenes 427
16.3. factorization by trial division 433
16.4. modular exponentiation 438
16.5. modular inverses 444
16.6. the chinese remainder theorem 450
16.7. across the seven languages 455
17. dynamic programming 456
17.1. the three fibonaccis 456
17.2. grid paths 464
17.3. the zero one knapsack 469
17.4. sequences and their tables 477
17.5. coin change, the greedy killer 483
17.6. longest increasing subsequence 489
17.7. bitmask subsets 497
17.8. across the seven languages 504
18. greedy and backtracking 505
18.1. activity selection and rooms 505
18.2. huffman 508
18.3. the jump game 518
18.4. backtracking 524
18.5. across the seven languages 532
19. advanced dynamic programming 534
19.1. the bounded knapsack 534
19.2. the sliding window shortcut 542
19.3. the grouped knapsack 549
19.4. convex hull trick 558
19.5. longest increasing subsequence 567
19.6. subsets as state, tsp by held-karp 575
19.7. digit dp, counting under a bound 582
19.8. interval dp, palindromes 588
19.9. across the seven languages 594
20. pruning and branch and bound 595
20.1. the pruning map, one grid three ways 595
20.2. n-queens, pruned by three arrays 603
20.3. branch and bound, the knapsack with a bound 609
20.4. alpha-beta, the game tree pruned by a window 617
20.5. across the seven languages 624
21. range queries 625
21.1. prefix sums 625
21.2. the fenwick tree 632
21.3. the segment tree with lazy propagation 638
21.4. the sparse table 646
21.5. sliding window maxima 653
21.6. next greater, both sides 660
21.7. across the seven languages 668
22. coordinate compression and intervals 669
22.1. coordinate compression 669
22.2. merging intervals 673
22.3. counting overlaps 676

22.4. the difference array ... 683
22.5. across the seven languages ... 687
23. computational geometry ... 689
23.1. the orientation predicate ... 689
23.2. the convex hull ... 696
23.3. closest pair ... 703
23.4. the sweep line ... 711
23.5. across the seven languages ... 720
24. lattice geometry and grid algorithms ... 721
24.1. shoelace area and pick's theorem ... 721
24.2. even-odd ray casting ... 728
24.3. flood fill ... 735
24.4. neighbor deltas and 3d walks ... 740
24.5. across the seven languages ... 747
25. practical structures ... 748
25.1. the bloom filter ... 748
25.2. the count-min sketch ... 755
25.3. lru and lfu caches ... 760
25.4. the skip list ... 771
25.5. across the seven languages ... 781
26. parsing and asts ... 782
26.1. the lexer ... 782
26.2. recursive descent ... 790
26.3. evaluating the ast ... 798
26.4. across the seven languages ... 804
27. an opcode machine ... 806
27.1. the instruction pointer loop ... 806
27.2. parameter modes ... 812
27.3. jumps and comparisons ... 818
27.4. across the seven languages ... 824
28. advanced number theory ... 825
28.1. the linear sieve and multiplicative functions ... 825
28.2. deterministic miller-rabin and pollard rho ... 832
28.3. divisor counts, divisor sums, and the phi identities ... 841
28.4. discrete logs and primitive roots ... 848
28.5. garner's algorithm and factorials mod p ... 854
28.6. linear diophantine equations ... 861
28.7. gray code and balanced ternary ... 869
28.8. continued fractions, convergents, and the stern-brocot tree ... 877
28.9. across the seven languages ... 885
29. fast big arithmetic ... 887
29.1. the dynamic bignum core ... 887
29.2. karatsuba multiplication ... 895
29.3. exact convolution with two primes ... 903
29.4. complex fft, plain and split ... 911
29.5. long division, knuth algorithm d ... 919
29.6. polynomials over one prime ... 927
29.7. across the seven languages ... 935
30. balanced trees and order statistics ... 936
30.1. the treap ... 936
30.2. the splay tree ... 944

30.3. meldable heaps ... 945
30.4. order statistics ... 952
30.5. across the seven languages ... 956
31. sqrt decomposition and offline queries ... 957
31.1. sqrt decomposition ... 957
31.2. mo's algorithm ... 961
31.3. the minimum stack and queue ... 968
31.4. the li chao tree ... 974
31.5. the disjoint sparse table ... 979
31.6. rollback dsu ... 986
31.7. offline dynamic connectivity ... 992
31.8. across the seven languages ... 996
32. dynamic programming ii ... 998
32.1. divide and conquer dp ... 998
32.2. the knuth optimization ... 1002
32.3. broken profile and the largest zero submatrix ... 1007
32.4. slope trick ... 1013
32.5. value iteration and mdps ... 1017
32.6. small-to-large merging ... 1022
32.7. across the seven languages ... 1023
33. string algorithms ii ... 1025
33.1. the z-function ... 1025
33.2. manacher's two passes ... 1033
33.3. the suffix automaton ... 1040
33.4. the suffix tree ... 1048
33.5. lyndon factorization, least rotation, and repetitions ... 1056
33.6. across the seven languages ... 1061
34. linear algebra ... 1062
34.1. gaussian elimination over a prime field ... 1062
34.2. the fraction-free determinant and rank ... 1068
34.3. cramer on the small systems ... 1076
34.4. matrix exponentiation ... 1082
34.5. kirchhoff's matrix-tree count ... 1091
34.6. hensel lifting over prime powers ... 1096
34.7. across the seven languages ... 1102
35. combinatorics ... 1103
35.1. binomial tables on three roads ... 1103
35.2. catalan numbers ... 1111
35.3. stars and bars with bounds ... 1115
35.4. the inclusion-exclusion principle ... 1120
35.5. burnside's lemma and polya enumeration ... 1126
35.6. generating k-subsets ... 1131
35.7. sperner's theorem and antichains ... 1138
35.8. the frobenius problem ... 1145
35.9. the josephus problem ... 1151
35.10. fifteen-puzzle solvability ... 1155
35.11. placing bishops on a chessboard ... 1155
35.12. balanced bracket sequences ... 1163
35.13. counting labeled graphs ... 1171
35.14. across the seven languages ... 1176
36. numerical methods ... 1178

36.1. ternary search ... 1178
36.2. golden-section search ... 1186
36.3. newton's method ... 1190
36.4. simpson integration ... 1196
36.5. simulated annealing ... 1204
36.6. LP duality and dual weights ... 1205
36.7. across the seven languages ... 1213
37. computational geometry ii ... 1214
37.1. line and circle intersections, tangents ... 1214
37.2. the length of a union of segments ... 1222
37.3. point in a convex polygon ... 1228
37.4. minkowski sums ... 1235
37.5. rotating calipers ... 1243
37.6. half-plane intersection ... 1251
37.7. the minimum enclosing circle ... 1259
37.8. voronoi cells and delaunay edges ... 1267
37.9. the bentley-ottmann sweep ... 1275
37.10. planar faces and point location ... 1283
37.11. the manhattan transform ... 1284
37.12. across the seven languages ... 1289
38. network flows ii ... 1291
38.1. dinic's algorithm ... 1291
38.2. push-relabel ... 1299
38.3. minimum-cost maximum flow ... 1307
38.4. flows with demands ... 1315
38.5. global minimum cut, stoer-wagner ... 1322
38.6. kuhn's bipartite matching ... 1331
38.7. the hungarian algorithm ... 1336
38.8. assignment problems ... 1344
38.9. hall's theorem ... 1347
38.10. across the seven languages ... 1354
39. graph connectivity and decomposition ... 1355
39.1. strongly connected components ... 1355
39.2. 2-sat ... 1363
39.3. bridges and articulation points ... 1368
39.4. eulerian paths ... 1377
39.5. lowest common ancestors ... 1385
39.6. heavy-light decomposition ... 1391
39.7. centroid decomposition ... 1399
39.8. functional graphs ... 1406
39.9. the 2-core ... 1414
39.10. prufer codes and the second-best mst ... 1421
39.11. strong orientation ... 1427
39.12. across the seven languages ... 1428
40. game theory and scheduling ... 1430
40.1. sprague-grundy and nim ... 1430
40.2. games on arbitrary graphs ... 1437
40.3. scheduling ... 1445
40.4. across the seven languages ... 1454
41. local search and metaheuristics ... 1456
41.1. local search: the frame ... 1456

41.2. hill climbing and plateaus ... 1461
41.3. tsp neighborhoods: swap, insert, 2-opt ... 1467
41.4. simulated annealing and the metropolis rule ... 1474
41.5. cooling schedules, restarts, reheating ... 1480
41.6. tabu search and tenure ... 1487
41.7. aspiration and candidate lists ... 1495
41.8. genetic algorithms: encoding and selection ... 1499
41.9. crossover, mutation, ox ... 1507
41.10. ant colony optimization and the pheromone model ... 1511
41.11. aco variants ... 1520
41.12. stopping: portfolios and restart budgets ... 1527
41.13. across the seven languages ... 1534
42. approximation algorithms ... 1535
42.1. np-hardness as an engineering boundary ... 1535
42.2. the approximation ratio ... 1539
42.3. vertex cover by maximal matching ... 1544
42.4. set cover by greedy ln n ... 1549
42.5. metric tsp by double-tree ... 1555
42.6. christofides 1.5x ... 1564
42.7. knapsack greedy 1/2 and the ptas ... 1570
42.8. fptas by profit scaling ... 1577
42.9. bin packing first fit ... 1586
42.10. local search as an approximation engine ... 1593
42.11. inapproximability: a reader's map ... 1600
42.12. choosing the right surrender ... 1600
42.13. across the seven languages ... 1602
43. capstone: storage engine toolkit ... 1604
43.1. the integrity layer ... 1604
43.2. the write-ahead log ... 1606
43.3. the memtable ... 1608
43.4. the sorted run ... 1610
43.5. compaction ... 1616
43.6. the engine ... 1618
43.7. cross-examined against sqlite ... 1622
43.8. htmx, taught before it is used ... 1625
43.9. the dashboard ... 1625
44. appendices: topic matrix and sources ... 1629
44.1. topic coverage ... 1629
44.2. the seven sample trees ... 1636
44.3. pinned sources ... 1641

1. complexity analysis and honest benchmarking

Every later chapter claims a cost: hash tables say constant average, balanced trees say logarithmic, sorts say linearithmic. This chapter builds the two instruments those claims must survive, operation counting for structural costs and allocation counting for memory costs, and shows why wall clock time is the last thing to trust in a test.

1.1. counting operations

Big-O is a statement about operation counts as n grows, so the honest test counts operations. A counter with a reset, plus closed-form models the instrumented run can be checked against. C counts the copies directly, one growth policy at a time.

The dry run: the fixture is doubling growth from capacity 1, the step-4 contrast from capacity 4, and the all-pairs loop, asserted by the C, Java, JavaScript, and Lua suites with the same numbers, Python walks the same counters with step 1, and C# and Go state the closed forms and hold counted runs to them.

1. Nine pushes from capacity 1 grow at sizes 1, 2, 4, 8, and the counter folds 1 + 2 = 3, 3 + 4 = 7, 7 + 8 = 15.
2. Thirty-one pushes add one more growth: 15 + 16 = 31 = 2^5 - 1, copies exactly equal to pushes.
3. The closed form 2^m - 1 holds at every size: the C# suite pins the doubling count at 1, 2, 10, 13 for n = 2, 3, 1024, 4097 and keeps every total under 2n, and Java's Allocs pins the same ladder from n = 1, 0 doublings, under the same 2n line.
4. The quadratic counter compares each pair once: ten items give 10 × 9 / 2 = 45 pairs, the insertion sort worst case of chapter 13.
5. The amortized contrast, amortized meaning the average cost over the sequence and chapter 3′s subject: 128 pushes from capacity 4 copy 1984 elements under step-4 growth against 124 under doubling, and 1984 > 4 × 128 = 512 breaks any constant per-append budget.

push	buffer full at	copied	total so far	capacity after
2	1	1	1	2
3	2	2	3	4
5	4	4	7	8
9	8	8	15	16
17	16	16	31	32

The 1984 against 124 is the contrast the meters pin, and the listings below count the same copies in seven languages.

```c
// elements copied while pushing n items into an array that doubles
static unsigned long long copies_doubling(unsigned long long n,
                                          unsigned long long cap0) {
  unsigned long long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap *= 2;
    }
    size++;
  }
  return copies;
}

// elements copied when capacity grows by a fixed step instead of doubling
static unsigned long long copies_step(unsigned long long n,
                                      unsigned long long cap0,
                                      unsigned long long step) {
  unsigned long long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap += step;
    }
    size++;
  }
  return copies;
}
```

listing 1: c, copy totals under doubling and constant-step growth (source: dsa/samples-c/src/Ch01/countops.c, lines 17-44)

The C suite pins the contrast from its main: 128 pushes into a capacity 4 buffer copy 1984 elements when capacity grows by 4 each time, and 124 when it doubles. Go keeps the counted doubling loop beside the closed form that predicts it:

```go
func DoublingCopies(n int) int64 {
  var c Counter
  capacity := 1
  for i := 0; i < n; i++ {
    if i == capacity {
      c.Add(int64(capacity))
      capacity *= 2
    }
  }
  return c.Total()
}

// DoublingCopiesClosedForm predicts that copy total. The last growth
// happens when appending item 2^(m-1), so the copied blocks sum to
// 2^m - 1 where 2^m is the smallest power of two ≥ n.
```

listing 2: go, the counted doubling loop and the closed form that predicts it (source: dsa/samples-go/ch01/countops.go, lines 31-45)

The doubling model is the whole amortization argument of chapter 3 in one formula: growing an array by doubling copies `2^m - 1` elements over n appends, which is less than 2n, so appends are constant on average even though every m-th one is a full copy. The quadratic model is the number of inverted pairs, the exact worst-case comparison count of insertion sort in chapter 13.

The other five languages count the same copies with their own instruments, each checked against a closed form by its suite. Java mirrors C's loops in long arithmetic straight from its main, and C# factors the idea into a reusable counter plus closed-form cost models:

```
// elements copied while pushing n items into an array that doubles
static long copiesDoubling(long n, long cap0) {
  long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap *= 2;
    }
    size++;
  }
  return copies;
}

// elements copied when capacity grows by a fixed step instead of doubling
static long copiesStep(long n, long cap0, long step) {
  long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap += step;
    }
    size++;
  }
  return copies;
}

// comparisons made by a quadratic all-pairs loop
static long pairComparisons(long n) {
  long cmp = 0;
  for (long i = 0; i < n; i++)
    for (long j = i + 1; j < n; j++)
      cmp++;
  return cmp;
}
```

listing 3: java, the three counters in long arithmetic, the same 1984 against 124 pinned from main (source: dsa/samples-java/src/Ch01/Countops.java, lines 18-51)

```
/// <summary>Operation counter: the honest way to test complexity without timers.</summary>
public sealed class Counter
{
    public long Count { get; private set; }

    public void Inc(long by = 1) => Count += by;

    public long Reset()
    {
        long old = Count;
        Count = 0;
        return old;
    }
}

/// <summary>Closed-form cost models the tests verify against instrumented runs.</summary>
public static class Costs
{
    // doublings needed so capacity starting at 1 reaches or exceeds n
    public static int Doublings(int n) => n <= 1 ? 0 : (int)Math.Ceiling(Math.Log2(n));

    // total elements copied by doubling growth across n appends: 2^m - 1, m = doublings
    public static long DoublingCopies(int n)
    {
        int m = Doublings(n);
        return (1L << m) - 1;
    }

    // linear work: n units
    public static long Linear(int n) => n;

    // triangular quadratic work: n(n-1)/2, the insertion sort worst case
    public static long Quadratic(int n) => (long)n * (n - 1) / 2;
}
```

listing 4: c#, the counter and the closed-form cost models (source: dsa/samples/src/Ch01/Analysis.cs, lines 3-36)

```javascript
export function copiesDoubling(n, cap0) {
  let cap = cap0;
  let size = 0;
  let copies = 0;
  while (size < n) {
    if (size === cap) {
      copies += cap;
      cap *= 2;
    }
    size++;
  }
  return copies;
}

// capacity grows by a fixed step instead of doubling
export function copiesStep(n, cap0, step) {
  let cap = cap0;
  let size = 0;
  let copies = 0;
  while (size < n) {
    if (size === cap) {
      copies += cap;
      cap += step;
    }
    size++;
  }
  return copies;
}
```

listing 5: javascript, both growth policies as plain functions (source: dsa/samples-js/src/ch01-countops.mjs, lines 7-34)

```python
def copies_doubling(items, cap0):
    # append `items` values into a buffer that doubles when it fills
    cap, size, copies = cap0, 0, 0
    for _ in range(items):
        if size == cap:
            copies += cap  # every stored element rides along on the grow
            cap *= 2
        size += 1
    return copies


def copies_step(items, cap0, step):
    # the same appends into a buffer that only ever grows by `step` slots
    cap, size, copies = cap0, 0, 0
    for _ in range(items):
        if size == cap:
            copies += cap
            cap += step
        size += 1
    return copies
```

listing 6: python, the two counters with the grow spelled out in comments (source: dsa/samples-py/src/Ch01/countops.py, lines 14-33)

```lua
local function copies_doubling(n, cap0)
  local cap, size, copies = cap0, 0, 0
  while size < n do
    if size == cap then
      copies = copies + cap
      cap = cap * 2
    end
    size = size + 1
  end
  return copies
end

-- elements copied when capacity grows by a fixed step instead of doubling
local function copies_step(n, cap0, step)
  local cap, size, copies = cap0, 0, 0
  while size < n do
    if size == cap then
      copies = copies + cap
      cap = cap + step
    end
    size = size + 1
  end
  return copies
end
```

listing 7: lua, the two counters as local functions (source: dsa/samples-lua/ch01_countops.lua, lines 7-30)

Measured across the suites: the counted loops cost 15 lines in Go, 28 in C, 34 in Java, 28 in JavaScript, 20 in Python, and 24 in Lua. The fixture families differ on purpose. C, Java, JavaScript, and Lua grow from capacity 4 with step 4, pinning 128 pushes at 1984 copied elements against doubling's 124. Python grows from capacity 1 and shows step 1 paying the triangular 45 for 10 pushes. C# and Go state the closed forms, 2^m - 1 and n(n-1)/2, and hold the counted runs to them, and Java spans both lanes across its files, Countops in C's family and Allocs carrying the closed forms. Every family agrees on the anchor that matters: doubling's copy total stays linear in n.

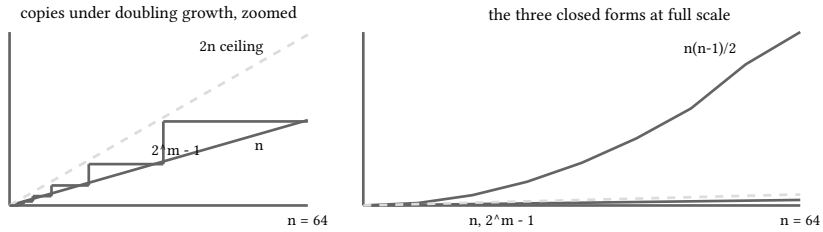


figure 1: the counter instruments the run, the closed forms predict it, and the three growth curves carry the shapes

1.2. binary search, probed

Binary search's logarithm is verifiable, not just quotable. Instrumenting the loop to count probes turns the claim into an inequality every index must satisfy. C hands the count back through an out parameter.

The dry run: the fixture is 1024 sorted slots, and all seven suites pin the same anchors, worst present key 11 probes, absence below the range 10, absence above it 11.

1. The first probe always lands at the floor midpoint of 0..1023, 0 + (1023 - 0) / 2 = 511, and key 511 hits on probe one.
2. Key 767 pays two: lo climbs to 512 and the next midpoint is 512 + (1023 - 512) / 2 = 767. C, JavaScript, and Lua pin both paths.
3. Absence below every key, target −5, halves downward through mids 511, 255, 127, ..., 1, 0, each the floor of a halved interval, and the interval empties after 10 probes.

4. Absence above, target 5000, climbs through 511, 767, 895, ..., 1022, 1023 and pays 11, one more, because the rounded-down midpoint descends faster than it ascends.
5. Every present key resolves within 11 = ceil(log2 1024) + 1 probes, and exactly one key pays the full 11, pinned by the Python suite.

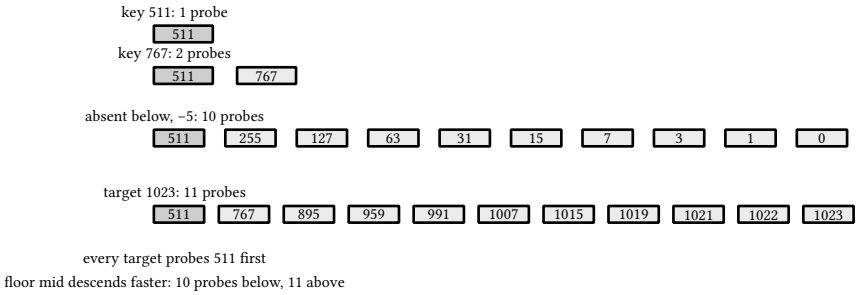


figure 2: the probe midpoints in visit order, one lane per target, the shared first probe shaded

The 10 against 11 is the anchored asymmetry, and the listings below carry the instrument in seven languages.

```c
// floor-mid search: index of t or -1, *probes counts midpoint evaluations
static long long probe_search(const long long *a, long long len, long long t,
                              long long *probes) {
  long long lo = 0, hi = len - 1;
  *probes = 0;
  while (lo ≤ hi) {
    long long mid = lo + (hi - lo) / 2;
    (*probes)++;
    if (a[mid] == t)
      return mid;
    if (a[mid] < t)
      lo = mid + 1;
    else
      hi = mid - 1;
  }
  return -1;
}
```

listing 8: c, floor-mid search with the probe count as an out parameter (source: dsa/samples-c/src/Ch01/probebin.c, lines 17-33)

```go
func BinaryProbe(slots int, target int) int {
  lo, hi := 0, slots-1
  probes := 0
  for lo <= hi {
    mid := lo + (hi-lo)/2
    probes++
    switch {
    case mid == target:
      return probes
    case mid < target:
      lo = mid + 1
    default:
      hi = mid - 1
    }
  }
  return probes // target absent: the loop exhausted the range
}

// WorstPresentProbes returns the largest probe count over every
// present target in 0..slots-1.
```

listing 9: go, the probed loop and the worst-case sweep it feeds (source: dsa/samples-go/ch01/probebin.go, lines 6-25)

The other five languages carry the same instrument, Java handing the count back as a record where C uses an out parameter:

```java
  record Probe(long index, long probes) {}

  // floor-mid search: index of t or -1, probes counts midpoint evaluations
  static Probe probeSearch(long[] a, long t) {
    long lo = 0, hi = a.length - 1, probes = 0;
    while (lo ≤ hi) {
      long mid = lo + (hi - lo) / 2;
      probes++;
      if (a[(int) mid] == t)
        return new Probe(mid, probes);
      if (a[(int) mid] < t)
        lo = mid + 1;
      else
        hi = mid - 1;
    }
    return new Probe(-1, probes);
  }
```

listing 10: java, floor-mid search returning a record that pairs index with probes (source: dsa/samples-java/src/Ch01/Probebin.java, lines 18-34)

```csharp
/// <summary>Binary search instrumented to count probes, used to pin logarithmic behavior.</summary>
public static class Search
{
    public static int CountProbes(int n, int targetIndex)
    {
        int probes = 0;
        int lo = 0, hi = n - 1;
        while (lo ≤ hi)
        {
            probes++;
            int mid = lo + (hi - lo) / 2;
            if (mid == targetIndex)
                return probes;
            if (mid < targetIndex)
                lo = mid + 1;
            else
                hi = mid - 1;
        }
        return probes; // targetIndex out of range: probes taken to prove absence
    }

    public static int WorstCase(int n)
    {
        int worst = 0;
        for (int i = 0; i < n; i++)
            worst = Math.Max(worst, CountProbes(n, i));
        return worst;
    }
}
```

listing 11: c#, probe counting search, worst case over all indices (source: dsa/samples/src/Ch01/Analysis.cs, lines 38-66)

On 1024 slots every position resolves within 11 probes, the ceiling of the base-2 logarithm plus one, and some position needs all 11. The suite pins a subtler fact too, one that interview answers usually miss:

NOTE: floor mid makes absence asymmetric

The classic loop computes `mid = lo + (hi - lo) / 2`, rounding down. Proving a target below the range is absent takes one probe less than proving absence above it, 10 versus 11 on 1024 slots, because the rounded-down midpoint descends faster than it ascends. The test `Absent_target_costs_no_more_than_the_worst_case` pins both numbers.

The other three languages carry the same instrument:

```javascript
// floor-mid search: { index, probes } on a hit, { index: -1, probes } on
// a miss. probes counts midpoint evaluations
export function probeSearch(a, t) {
  let lo = 0;
  let hi = a.length - 1;
  let probes = 0;
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    probes++;
    if (a[mid] === t) return { index: mid, probes };
    if (a[mid] < t) lo = mid + 1;
    else hi = mid - 1;
  }
  return { index: -1, probes };
}
```

listing 12: javascript, the probe count returned in an object (source: dsa/samples-js/src/ch01-probebin.mjs, lines 5-19)

```python
def probe_count(keys, target):
    lo, hi, probes = 0, len(keys) - 1, 0
    while lo ≤ hi:
        probes += 1
        mid = (lo + hi) // 2  # floor midpoint, never an overflow in python
        if keys[mid] == target:
            return probes
        if keys[mid] < target:
            lo = mid + 1
        else:
            hi = mid - 1
    return probes
```

listing 13: python, floor midpoint with no overflow rewrite needed (source: dsa/samples-py/src/Ch01/probebin.py, lines 14-25)

```lua
local function probe_search(a, t)
  local lo, hi = 1, #a
  local probes = 0
  while lo ≤ hi do
    local mid = lo + (hi - lo) // 2
    probes = probes + 1
    if a[mid] == t then
      return mid, probes
    elseif a[mid] < t then
      lo = mid + 1
    else
      hi = mid - 1
    end
  end
  return nil, probes
end
```

listing 14: lua, two return values over one-based bounds (source: dsa/samples-lua/ch01_probebin.lua, lines 7-22)

All seven suites pin the same anchors on 1024 slots: the worst present key costs 11 probes, absence below the range costs 10, absence above costs 11. C, Java, JavaScript, and Lua pin the path as well, first probe at 511, second at 767. Python pins that exactly one key pays the full 11, and adds an 8-slot array that resolves its low extreme in 3 probes and an absent key in 4. Go keeps the instrument inside one function that reports only what the sweep

needs. The return shapes differ, an out parameter, a record, an object, two values, an early return, and the anchors do not move.

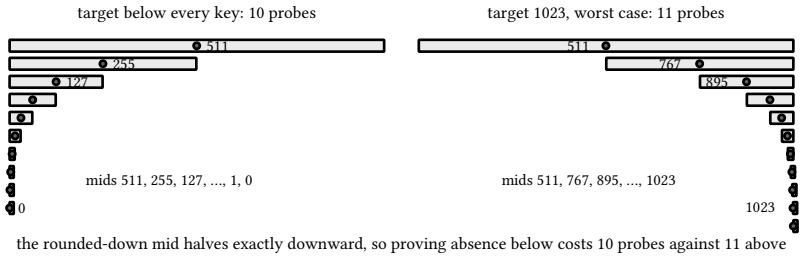


figure 3: binary search on 1024 slots, each probe halves the surviving range

1.3. counting allocations

Managed allocations are countable exactly where the runtime reports them. `GC.GetAllocatedBytesForCurrentThread()` reports bytes allocated on the current thread since its start, so differencing it around an action measures that action's allocation with no sampling error. The other trees count what they can see, C its own malloc calls, Go AllocsPerRun, Java the substring call whose fresh string it cannot avoid, Python tracemalloc snapshots. Time has jitter, thread switches, frequency scaling. Bytes do not.

The dry run: the fixtures are the headers id+721 and seq+999 plus the malformed id+7x2, asserted by all seven suites on the same parse values. Each tree meters the allocation lane its own honest way, and no lane fakes a count.

1. Parse the digits both ways everywhere: the substring road copies the tail after the separator into fresh storage and parses the copy, the walk road folds the same characters in place: 0 × 10 + 7 = 7, 7 × 10 + 2 = 72, 72 × 10 + 1 = 721.
2. Both parsers return 721 in every suite, and the six new trees pin seq+999 to 999 both ways, C# exercises that header through its meter harness.
3. The walk rejects id+7x2 in every tree, as an exception, a nonzero exit code, or a thrown error per the tree's own mechanism.
4. C counts its own mallocs, 1 for the substring road against 0 for the walk.
5. Go pins AllocsPerRun at exactly 1 against 0, with a documented sink that keeps the digit string live so the substring road honestly escapes to the heap instead of staying on the stack.
6. Java counts the substring road's unavoidable fresh strings, exactly 1 per parse, and the in-place walk leaves the counter untouched.
7. C# differences the thread's allocated byte count around each parse, strictly above 0 for the substring road and exactly 0 for the walk.
8. Python compares tracemalloc snapshots and asserts the slice road allocates strictly more than the walk, an inequality, because interpreter bookkeeping makes exact byte equality tree-specific. JavaScript and Lua assert the parse values and the rejection only.

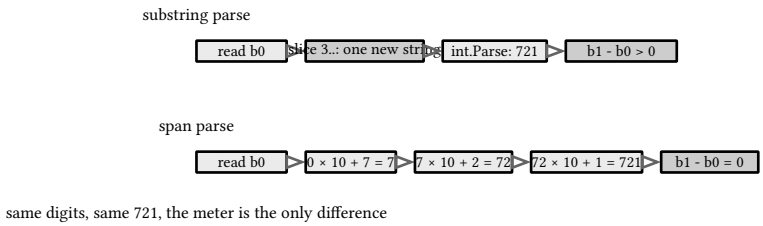


figure 4: the two parses as step chains, one allocation box against three in-place folds

The strictly positive against exactly 0 is the pinned pair, and the listings below are the meters that produced it, one per tree:

```c
static long alloc_calls = 0;

static void *counted_malloc(size_t bytes) {
  alloc_calls++;
  return malloc(bytes);
}

// the substring road: the tail after '+' is copied into a fresh buffer,
// one allocation per parse, then read back from the copy
static int parse_substring(const char *header) {
  const char *tail = strchr(header, '+') + 1;
  size_t len = strlen(tail);
  char *copy = counted_malloc(len + 1);
  memcpy(copy, tail, len + 1);
  int value = (int)strtol(copy, NULL, 10);
  free(copy);
  return value;
}

// the walk road: digits read in place, no allocation at all
static int parse_walk(const char *header, int *ok) {
  const char *p = strchr(header, '+') + 1;
  int value = 0;
  *ok = 1;
  for (; *p != '\0'; p++) {
    if (*p < '0' || *p > '9') {
      *ok = 0;
      return 0;
    }
    value = value * 10 + (*p - '0');
  }
  return value;
}
```

listing 15: c, counted mallocs split the substring road from the walk (source: dsa/samples-c/src/Ch01/allocs.c, lines 53-85)

```go
// digitsSink keeps the materialized digit string live. Go can stack
// allocate a substring that never escapes, so the sink is what makes
// the substring road honestly pay its one heap allocation, the same
// object a C# Substring always creates.
var digitsSink string

// ParseSubstring extracts the number after the first '+' by building
// the digit string first: one allocation for the copy, exactly what
// testing.AllocsPerRun sees.
func ParseSubstring(header string) (int, error) {
  plus := strings.IndexByte(header, '+')
  if plus < 0 {
    return 0, errNoSeparator
  }
  digitsSink = strings.Clone(header[plus+1:])
  return strconv.Atoi(digitsSink)
}

// ParseWalk parses the same header in place: it walks the characters
// after the separator one at a time and allocates nothing. A non digit
// there is an error, not a zero.
func ParseWalk(header string) (int, error) {
  plus := strings.IndexByte(header, '+')
  if plus < 0 {
    return 0, errNoSeparator
  }
  value := 0
  for i := plus + 1; i < len(header); i++ {
    digit := int(header[i]) - '0'
    if digit < 0 || digit > 9 {
      return 0, errNotADigit
    }
    value = value*10 + digit
  }
  return value, nil
}
```

listing 16: go, the sink that keeps the substring allocation observable, then both parses (source: dsa/samples-go/ch01/allocs.go, lines 84-119)

The other five languages carry the same two roads with the meter each runtime can honestly support:

```
  static long allocCalls = 0;

  // the substring road: the tail after '+' is copied into a fresh string,
  // one allocation per parse, then read back from the copy
  static int parseSubstring(String header) {
    allocCalls++; // substring cannot avoid the fresh copy
    return Integer.parseInt(header.substring(header.indexOf('+') + 1));
  }

  // the walk road: digits read in place off the original, no allocation
  static int parseWalk(String header) {
    int value = 0;
    for (int i = header.indexOf('+') + 1; i < header.length(); i++) {
      int digit = header.charAt(i) - '0';
      if (digit < 0 || digit > 9)
        throw new IllegalArgumentException("non digit after separator");
      value = value * 10 + digit;
    }
    return value;
  }
```

listing 17: java, the substring road's one fresh string counted by hand, the walk reads in place (source: dsa/samples-java/src/Ch01/Allocs.java, lines 60-79)

```
/// <summary>Allocation counting and a stopwatch wrapper, the two honest bench primitives.</summary>
public static class Bench
{
    public static long AllocatedBytes(Action action)
    {
        long before = GC.GetAllocatedBytesForCurrentThread();
        action();
        return GC.GetAllocatedBytesForCurrentThread() - before;
    }

    public static (long Micros, long Bytes) Run(Action action)
    {
        var watch = System.Diagnostics.Stopwatch.StartNew();
        long bytes = AllocatedBytes(action);
        watch.Stop();
        return (watch.Elapsed.Ticks / (TimeSpan.TicksPerMillisecond / 1000), bytes);
    }

    // allocation-heavy parse: every Substring is a new string
    public static int ParseSubstring(string header)
    {
        int plus = header.IndexOf('+');
        return int.Parse(header[(plus + 1)..]);
    }

    // allocation-free parse: the span walks the same chars in place
    public static int ParseSpan(string header)
    {
        ReadOnlySpan<char> chars = header.AsSpan();
        int plus = chars.IndexOf('+');
        int value = 0;
        for (int i = plus + 1; i < chars.Length; i++)
        {
            int digit = chars[i] - '0';
            if (digit is < 0 or > 9)
                throw new FormatException("non digit after separator");
            value = value * 10 + digit;
        }
        return value;
    }
}
```

listing 18: c#, allocation counting, the stopwatch wrapper, two parses (source: dsa/samples/src/Ch01/Analysis.cs, lines 68-108)

```javascript
// the allocation-heavy road: slice allocates a new string, then parseInt
// walks it. js cannot count allocations honestly, so the value is the claim
export function parseSubstring(header) {
  const plus = header.indexOf("+");
  return Number.parseInt(header.slice(plus + 1), 10);
}

// the walk: no intermediate string, digits consumed in place, any other
// character is an error
export function parseWalk(header) {
  let i = header.indexOf("+") + 1;
  if (i === 0) throw new Error("missing separator");
  let value = 0;
  for (; i < header.length; i++) {
    const digit = header.charCodeAt(i) - 48;
    if (digit < 0 || digit > 9) throw new Error("non digit after separator");
    value = value * 10 + digit;
  }
  return value;
}
```

listing 19: javascript, the slice road and the in-place walk, values asserted only (source: dsa/samples-js/src/ch01-allocs.mjs, lines 53-72)

```python
def parse_slice(header):
    # the substring road: the tail after + is copied out as a fresh string
    plus = header.index("+")
    return int(header[plus + 1 :])


def parse_walk(header):
    # the in-place road: index arithmetic over the same chars, no tail copy
    plus = header.index("+")
    value = 0
    for i in range(plus + 1, len(header)):
        digit = ord(header[i]) - ord("0")
        if digit < 0 or digit > 9:
            raise ValueError("non digit after separator")
        value = value * 10 + digit
    return value
```

listing 20: python, the slice road and the in-place walk (source: dsa/samples-py/src/Ch01/allocs.py, lines 46-61)

```lua
-- the substring road: every sub result is a fresh string
local function parse_substring(header)
  local plus = header:find("+", 1, true)
  return tonumber(header:sub(plus + 1))
end

-- the walk road: bytes consumed in place, no substring ever built
local function parse_walk(header)
  local plus = header:find("+", 1, true)
  local value = 0
  for i = plus + 1, #header do
    local digit = header:byte(i) - 48
    if digit < 0 or digit > 9 then
      error("non digit after separator")
    end
    value = value * 10 + digit
  end
  return value
end
```

listing 21: lua, substring against byte walk, values and rejection only (source: dsa/samples-lua/ch01_allocs.lua, lines 50-68)

Measured across the suites: both roads parse id+721 to 721 in all seven languages and seq+999 to 999 in the six new trees, and every walk rejects id+7x2. The meter lanes stay honest per tree. C counts mallocs, 1 against 0. Go pins AllocsPerRun at exactly 1 against 0. Java counts the substring road's fresh string as one call the walk never makes. C# differences the GC byte counter, strictly positive against exactly 0. Python's tracemalloc shows the slice road allocating strictly more than the walk. JavaScript and Lua stop at the values, nothing they carry exposes a countable allocation meter.

That contrast, allocation as a cost with an exact meter where the runtime offers one, recurs through this book whenever a data structure is asked whether it really avoids copying.

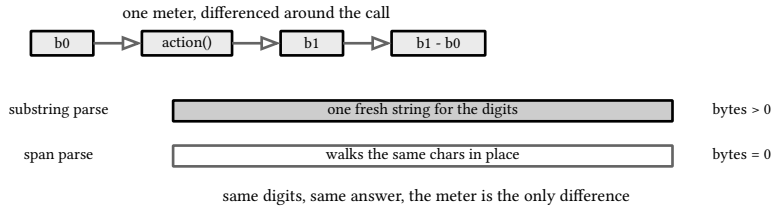


figure 5: the allocation meter differenced around the call, substring against span

WARNING: what this harness is not

This is a teaching harness. Production measurement belongs in BenchmarkDotNet, which handles warmup iterations, outlier filtering, and statistical comparison, and whose docs are listed in the sources line. `Stopwatch` plus allocation counting earns its place in a test suite because its assertions are deterministic, not because it is a rigorous benchmark.

1.4. reading a cost claim

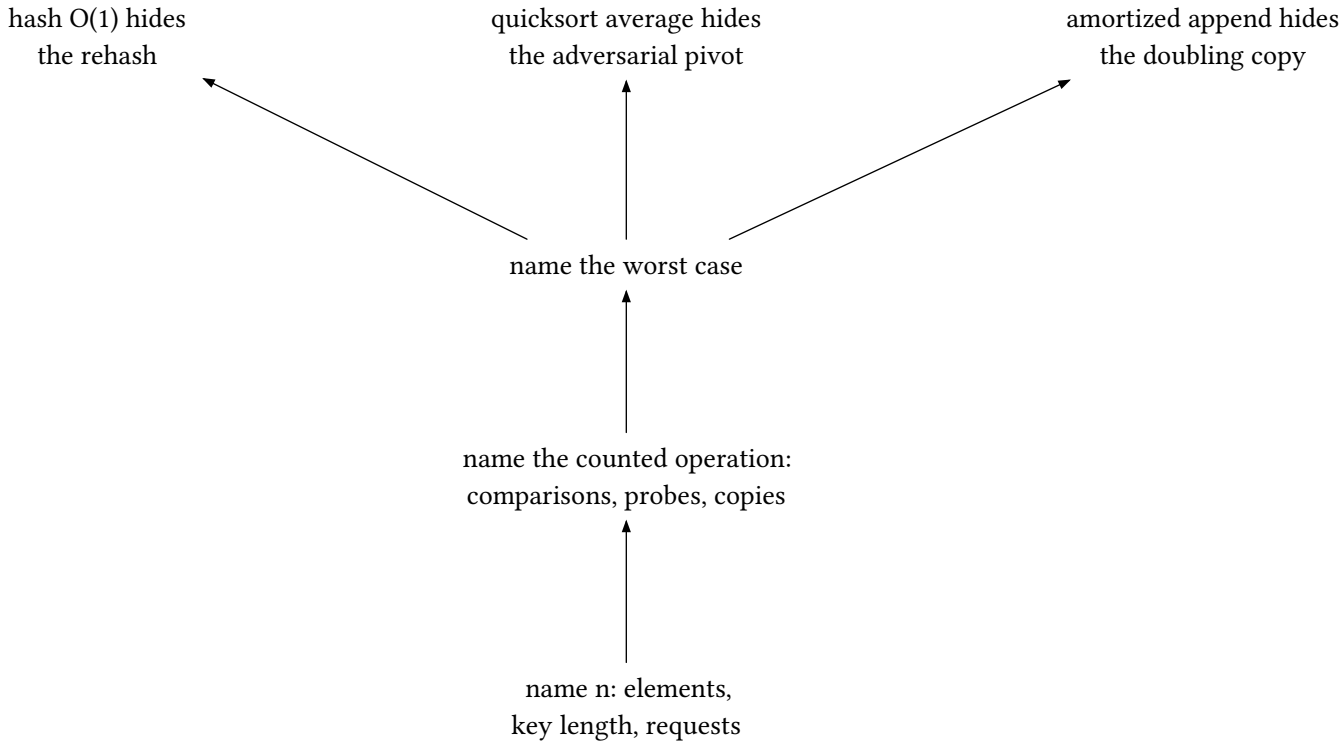


figure 6: reading a cost claim, three questions and the three famous hides

Three questions turn a claim like “O(1) average” into something checkable. What is n, the element count, the key length, the request rate. What is the operation being counted, comparisons, probes, bytes copied. And what does the worst case look like, because the hash table’s average hides the rehash, the quicksort’s average hides the adversarial pivot, and the amortized append hides the doubling copy. The rest of the book answers those questions per structure, with a counter where a skeptic can put a breakpoint.

The dry run: no suite carries this walk, the numbers are hand-derived, and the claim on trial is that appending to a growable array costs O(1) on average.

1. Name n: the element count, one hundred appends into an array born at capacity 1.
2. Name the counted operation: elements physically copied by growth, not milliseconds.
3. Name the worst case: the 65th append lands on a full buffer of 64 and copies all 64, one call hiding 64 copies.
4. The total for the run: growths at sizes 1, 2, 4, 8, 16, 32, 64 copy 1 + 2 + 4 + 8 + 16 + 32 + 64 = 127 = 2^7 - 1.
5. The average the claim means: 127 / 100 = 1.27 copies per append, inside 2 × 100 = 200 for the whole run.

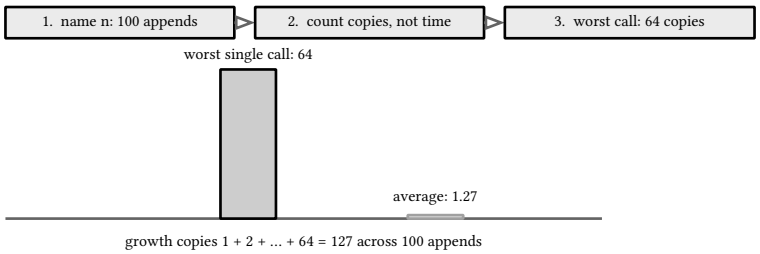


figure 7: the three questions applied to one claim, the worst call against the average it hides

The 64-copy call inside the 1.27 average is what the three questions expose, and chapter 3 runs the same argument on real meters.

1.5. across the seven languages

The build sizes count non-comment source lines over this chapter's featured files. Where a language bundles its checks into the same file the count carries them, and where the suite lives in a separate test project it does not, which is most of the C and Lua gap:

language	build SLOC	dependency	boundary note
c	257	libc only	checks share the file with main, 66 of them, counters are unsigned long long
go	151	errors, strconv, strings	the closed form shifts 1 << m inside int64, a documented sink keeps the substring road's one allocation observable
java	250	jdk 27 stdlib	no malloc to hook and no exposed allocation byte counter, the meter hand-counts the substring road's one fresh string, long exact since every count sits far below 2^63
c#	85	bcl only	gc byte counting stays the finest meter of the seven, each tree asserts what its runtime honestly exposes
javascript	97	node stdlib	every count sits far below 2^53, Number is exact with no boundary in play, the parse lanes assert values only
python	173	stdlib only	native ints, the floor midpoint needs no overflow-safe rewrite, tracemalloc compares the parse roads by inequality
lua	248	lib.lua harness	1-based indexing moves the bounds, floor division has its own operator, checks ride in the module

sources: learn.microsoft.com, `GC.GetAllocatedBytesForCurrentThread` and `Stopwatch` api pages, `Memory<T>` and `Span<T>` usage guidelines, benchmarkdotnet docs at benchmarkdotnet.org, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 10 tests in chapter 1 of the samples suite. The seven-language layer verifies the same way: 3 C programs with 66 embedded checks under `make verify-c`, 17 Go tests, the java runner's 66 Ch01 checks over 3 files under `run-java-samples`, 20 `node --test` cases, 51 Python checks across 3 files, and 19 Lua checks under `run.lua`.

2. arrays, spans, memory layout

The array is the base structure every later one is judged against: contiguous, indexed in constant time, brutal to grow. This chapter pins its layout facts, row-major order, cache line sharing, bounds checks, and the span vocabulary that works on arrays without copying them.

2.1. row major order

Every language in this book that stores a matrix as one block stores it the same way, consecutive rows, so cell (r, c) lives at flat offset `r * cols + c`. C proves the layout with pointer arithmetic and prices both walks with a one-line cache simulator.

The dry run: the fixture is the 4x4 grid holding r × 4 + c + 1, walked in both orders, asserted by the C, Java, JavaScript, and Lua suites. C# flattens a 2x3 grid instead, Go prices a 4x4 of int64 four elements per line, and Python builds a two-slot toy cache.

1. Cell (2, 3) lives at flat offset 2 × 4 + 3 = 11, the address law the suites check cell by cell.
2. The row walk visits offsets 0 through 15 in storage order, one slot per step.
3. The column walk starts 0, 4, 8, 12: column neighbors sit 4 slots, 16 bytes, apart.
4. Its fifth visit wraps from slot 12 back to slot 1: 1 - 12 = −11.
5. Both orders touch every cell exactly once, and both sums land 136, pinned twice by each suite.

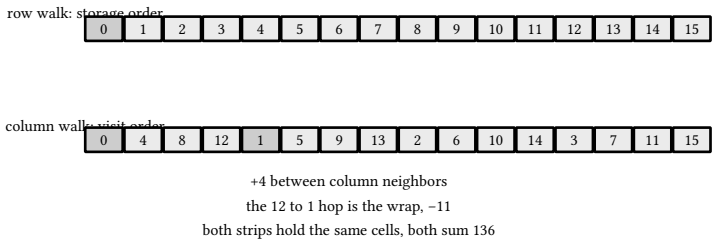


figure 8: both walks as visit sequences over the same sixteen slots, the column hops stride and wrap

Both walks land the same 136, and the listings below walk both orders in seven languages.

```c
// count 64-byte line touches for a walk, keeping only the last line warm
// (16 ints per line)
static long long line_touches(int rows, int cols, int row_major) {
  long long last = -1, touches = 0;
  for (int i = 0; i < rows * cols; i++) {
    int r = row_major ? i / cols : i % rows;
    int c = row_major ? i % cols : i / rows;
    long long idx = (long long)r * cols + c;
    long long line = idx / 16;
    if (line != last) {
      touches++;
      last = line;
    }
  }
  return touches;
}
```

listing 22: c, line touches counted for either walk order (source: dsa/samples-c/src/Ch02/rowmajor.c, lines 17-32)

```go
func WalkRow(m []int64) (sum int64, touches int) {
  lastLine := -1
  for i, v := range m {
    sum += v
    line := i / LineInts
    if line != lastLine {
      touches++
      lastLine = line
    }
  }
  return sum, touches
}

// WalkCol walks the same matrix column by column. Every step jumps a
// full row (cols elements) of stride, landing in a fresh line each
// time: cols times the touches of the row walk on this fixture.
func WalkCol(m []int64, cols int, rows int) (sum int64, touches int) {
  for c := 0; c < cols; c++ {
    for r := 0; r < rows; r++ {
      i := r*cols + c
      sum += m[i]
      touches++ // stride cols leaves the previous line behind
    }
  }
  return sum, touches
}
```

listing 23: go, both walks over a flattened matrix, touches counted per stride (source: dsa/samples-go/ch02/rowmajor.go, lines 25-50)

Row-major and column-major summation return the same value, the test asserts it, and yet they are not the same code. The order in which memory is touched differs, which is invisible to correctness and decisive for speed. That gap is where this chapter goes next.

Five more languages pin the same walk pair, Java with no two dimensional pointer arithmetic to lean on, the flatten law stated over the explicit flat index:

```java
  // count 64-byte line touches for a walk, keeping only the last line warm
  // (16 ints per line)
  static long lineTouches(int rows, int cols, boolean rowMajor) {
    long last = -1, touches = 0;
    for (int i = 0; i < rows * cols; i++) {
      int r = rowMajor ? i / cols : i % rows;
      int c = rowMajor ? i % cols : i / rows;
      long idx = (long) r * cols + c;
      long line = idx / 16;
      if (line != last) {
        touches++;
        last = line;
      }
    }
    return touches;
  }
```

listing 24: java, the walk simulator over the explicit flat index, touches counted for either order (source: dsa/samples-java/src/Ch02/Rowmajor.java, lines 20-35)

.NET stores a two dimensional `int[,]` the same way. Flatten and unflatten are that arithmetic with no library help:

```csharp
/// <summary>Row-major layout: the compiler-visible shape of a two dimensional array.</summary>
public static class Matrix
{
    public static int[] Flatten(int[,] grid)
    {
        int rows = grid.GetLength(0), cols = grid.GetLength(1);
        var flat = new int[rows * cols];
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                flat[r * cols + c] = grid[r, c];
        return flat;
    }

    public static int[,] Unflatten(int[] flat, int rows, int cols)
    {
        var grid = new int[rows, cols];
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                grid[r, c] = flat[r * cols + c];
        return grid;
    }

    // column walk in row-major storage jumps cols * 4 bytes between touches
    public static long SumRowMajor(int[,] grid)
    {
        long sum = 0;
        for (int r = 0; r < grid.GetLength(0); r++)
            for (int c = 0; c < grid.GetLength(1); c++)
                sum += grid[r, c];
        return sum;
    }

    public static long SumColumnMajor(int[,] grid)
    {
        long sum = 0;
        for (int c = 0; c < grid.GetLength(1); c++)
            for (int r = 0; r < grid.GetLength(0); r++)
                sum += grid[r, c];
        return sum;
    }
}
```

listing 25: c#, row-major flatten, unflatten, both traversal orders (source: dsa/samples/src/Ch02/Arrays.cs, lines 3-44)

```
export function walkOffsets(rows, cols) {
  const rowOrder = [];
  const colOrder = [];
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < cols; c++) rowOrder.push(flatIndex(r, c, cols));
  for (let c = 0; c < cols; c++)
    for (let r = 0; r < rows; r++) colOrder.push(flatIndex(r, c, cols));
  return { rowOrder, colOrder };
}

// count 64-byte line touches for a walk (16 ints per line), keeping only
// the last line warm
export function lineTouches(rows, cols, rowMajor) {
  let last = -1;
  let touches = 0;
  const total = rows * cols;
  for (let i = 0; i < total; i++) {
    const r = rowMajor ? Math.floor(i / cols) : i % rows;
    const c = rowMajor ? i % cols : Math.floor(i / rows);
    const line = Math.floor(flatIndex(r, c, cols) / 16);
    if (line !== last) {
      touches++;
      last = line;
    }
  }
  return touches;
}
```

listing 26: javascript, both offset orders plus the line simulator (source: dsa/samples-js/src/ch02-rowmajor.mjs, lines 12-38)

```
def offset(r, c):
    return r * COLS + c  # the row-major address formula


def walk_misses(offsets, slots=2, els_per_line=4):
    # a direct-mapped toy cache: `slots` lines, each holding 4 elements
    live = {}
    misses = 0
    for off in offsets:
        line = off // els_per_line
        if live.get(line % slots) != line:
            live[line % slots] = line
            misses += 1
    return misses
```

listing 27: python, the address formula and a two-slot toy cache (source: dsa/samples-py/src/Ch02/rowmajor.py, lines 21-34)

```lua
local function line_touches(rows, cols, row_major)
  local last, touches = -1, 0
  for i = 0, rows * cols - 1 do
    local r = row_major and (i // cols) or (i % rows)
    local c = row_major and (i % cols) or (i // rows)
    local line = (r * cols + c) // 16
    if line ~= last then
      touches = touches + 1
      last = line
    end
  end
  return touches
end
```

listing 28: lua, the same line simulator in floor-division arithmetic (source: dsa/samples-lua/ch02_rowmajor.lua, lines 8-20)

On the 64x64 fixture C, Java, JavaScript, and Lua price the row walk at 256 line touches and the column walk at 4096. Go works a 4x4 of int64 values with a four-element stand-in line, 4 touches for the row walk against 16 for the column walk, and checks both sums against an independent closed form. Python builds a two-slot direct-mapped toy cache, 4 misses on the row walk, 16 on the column walk, and 4 again once four slots make it fully associative. C# counts analytically, 4096 sequential ints fill exactly 256 lines.

2.2. the cache cost model

A cache line is 64 bytes on current x64 and arm hardware, 16 ints. Walking ints sequentially touches one line per 16 elements. Walking with a stride of 16 ints or more touches one line per element, a 16 fold inflation in memory traffic for the same arithmetical work. The cost model makes that countable without timing anything.

The dry run: the fixture is 4096 sequential ints and strided walks of 1000 touches, asserted by all seven suites on the same integers. Every tree computes the model in integer ceiling arithmetic, no float touches the state anywhere.

1. Sequential ints pack 4096 × 4 = 16384 bytes, and 16384 / 64 = 256 lines for the whole grid.
2. Forty ints span 160 bytes and straddle a third line: 3, packing has a ragged tail.
3. Stride 16 ints: 16 × 4 = 64 bytes per hop, a full line, so every touch lands on a fresh line and 1000 touches buy 1000 lines.
4. Stride 8 ints: 8 × 4 = 32 bytes, two touches share each line, and 1000 / 2 = 500.
5. Stride 1 degenerates to the sequential count, the suites assert both counters agree at 4096.
6. A 64x64 column walk is the degenerate stride, 64 ints = 256 bytes per hop: 64 touches, 64 lines, no sharing at all.

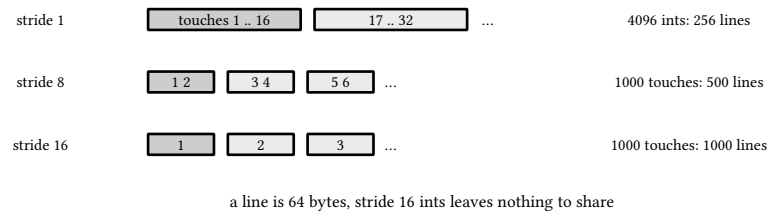


figure 9: which touches share a line at each stride, the pinned totals on the right

The 256 against the 1000-line degenerate stride are both pinned, and the listings below are the counters that compute them:

```c
enum { LINE_BYTES = 64, INT_BYTES = 4 };

static long ceil_div(long a, long b) { return (a + b - 1) / b; }

// lines touched by count sequential ints starting line aligned
static long lines_sequential(long count) {
  return ceil_div(count * INT_BYTES, LINE_BYTES);
}

// lines touched by count ints picked stride ints apart
static long lines_strided(long count, long stride) {
  long stride_bytes = stride * INT_BYTES;
  if (stride_bytes ≥ LINE_BYTES)
    return count; // one line per touch, no sharing at all
  long per_line = ceil_div(LINE_BYTES, stride_bytes);
  return ceil_div(count, per_line);
}
```

listing 29: c, the two line counters over one integer ceiling helper (source: dsa/samples-c/src/Ch02/cache.c, lines 17-33)

```go
// The cache model in two constants: one line holds 64 bytes, one int
// element occupies 4 of them.
const (
  LineBytes = 64
  IntBytes  = 4
)

// ceilDiv is (a + b - 1) / b on non negative ints: integer ceiling
// arithmetic only, no floats anywhere in this model.
func ceilDiv(a, b int) int {
  return (a + b - 1) / b
}

// LinesTouchedSequential counts cache lines touched by reading n ints
// packed back to back from a line aligned start: ceil(4n / 64).
func LinesTouchedSequential(n int) int {
  return ceilDiv(n*IntBytes, LineBytes)
}

// LinesTouchedStrided counts lines touched by n reads spaced strideElems
// apart. Once a stride spans a whole line every touch lands on a fresh
// line; below that, ceil(64 / 4s) touches share each line.
func LinesTouchedStrided(n, strideElems int) int {
  strideBytes := strideElems * IntBytes
  if strideBytes ≥ LineBytes {
    return n
  }
  perLine := ceilDiv(LineBytes, strideBytes)
  return ceilDiv(n, perLine)
}
```

listing 30: go, both counters over the line and int constants (source: dsa/samples-go/ch02/cache.go, lines 3-32)

The other five languages carry the same two counters as pure integer arithmetic:

```
  static final long LINE_BYTES = 64, INT_BYTES = 4;

  static long ceilDiv(long a, long b) {
    return (a + b - 1) / b;
  }

  // lines touched by count sequential ints starting line aligned
  static long linesSequential(long count) {
    return ceilDiv(count * INT_BYTES, LINE_BYTES);
  }

  // lines touched by count ints picked stride ints apart
  static long linesStrided(long count, long stride) {
    long strideBytes = stride * INT_BYTES;
    if (strideBytes ≥ LINE_BYTES)
      return count; // one line per touch, no sharing at all
    long perLine = ceilDiv(LINE_BYTES, strideBytes);
    return ceilDiv(count, perLine);
  }
```

listing 31: java, the ceiling helper and both counters in long arithmetic (source: dsa/samples-java/src/Ch02/Cache.java, lines 18-36)

```
public static class Cache
{
    public const int LineBytes = 64;
    public const int IntBytes = 4;

    // n sequential ints pack ceil(n * 4 / 64) lines when they start line aligned
    public static long LinesTouchedSequential(int n) =>
        (long)Math.Ceiling((double)n * IntBytes / LineBytes);

    // same n ints touched with a stride: every touch lands on a fresh line once stride ≥ 16 ints
    public static long LinesTouchedStrided(int n, int strideElems)
    {
        long strideBytes = (long)strideElems * IntBytes;
        if (strideBytes ≥ LineBytes)
            return n; // one line per touch, no sharing at all
        // touches per line = how many stride steps fit before crossing a boundary:
        // with s < 16 ints, ceil(64 / (s * 4)) touches share each line
        double perLine = Math.Ceiling((double)LineBytes / strideBytes);
        return (long)Math.Ceiling(n / perLine);
    }
}
```

listing 32: c#, deterministic cache line counts for sequential and strided walks (source: dsa/samples/src/Ch02/Arrays.cs, lines 46-67)

```javascript
export const LINE_BYTES = 64;
export const INT_BYTES = 4;

function ceilDiv(a, b) {
  return Math.floor((a + b - 1) / b);
}

// n sequential ints pack ceil(n * 4 / 64) lines when they start line aligned
export function linesTouchedSequential(n) {
  return ceilDiv(n * INT_BYTES, LINE_BYTES);
}

// the same n ints touched with a stride in elements: every touch lands on a
// fresh line once the stride spans a whole line, otherwise ceil(64 / 4s)
// touches share each line
export function linesTouchedStrided(n, strideElems) {
  const strideBytes = strideElems * INT_BYTES;
  if (strideBytes ≥ LINE_BYTES) return n;
  const perLine = ceilDiv(LINE_BYTES, strideBytes);
  return ceilDiv(n, perLine);
}
```

listing 33: javascript, both counters, the floor of the padded numerator (source: dsa/samples-js/src/ch02-cache.mjs, lines 4-24)

```python
LINE_BYTES = 64
INT_BYTES = 4


def lines_sequential(count):
    # count ints at 4 bytes pack into ceil(4n/64) lines, line aligned start
    return (count * INT_BYTES + LINE_BYTES - 1) // LINE_BYTES


def lines_strided(count, stride):
    stride_bytes = stride * INT_BYTES
    if stride_bytes ≥ LINE_BYTES:
        return count  # one line per touch, no sharing at all
    # touches per line: how many stride steps fit before a boundary crossing
    per_line = (LINE_BYTES + stride_bytes - 1) // stride_bytes
    return (count + per_line - 1) // per_line
```

listing 34: python, both counters in floor division (source: dsa/samples-py/src/Ch02/cache.py, lines 14-29)

```lua
local LINE_BYTES, INT_BYTES = 64, 4

-- ceil(a / b) on integers: add b - 1 before flooring
local function ceil_div(a, b)
  return (a + b - 1) // b
end

-- n sequential ints starting line aligned pack into ceil(4n / 64) lines
local function lines_sequential(n)
  return ceil_div(n * INT_BYTES, LINE_BYTES)
end

-- n ints touched every stride elements: past 16 ints every touch is a
-- fresh line, below that ceil(64 / (4s)) touches share each line
local function lines_strided(n, stride)
  local stride_bytes = stride * INT_BYTES
  if stride_bytes ≥ LINE_BYTES then return n end
  local per_line = ceil_div(LINE_BYTES, stride_bytes)
  return ceil_div(n, per_line)
end
```

listing 35: lua, both counters over the floor division operator (source: dsa/samples-lua/ch02_cache.lua, lines 6-25)

Measured across the suites: all seven pin 256 lines for 4096 sequential ints, 3 for 40, 1000 lines for 1000 touches at stride 16, 500 at stride 8, and the column walk of the 64x64 grid at 64 lines against the row walk. The six new trees also pin the small counts, 4 lines for 64 ints and 1 for one, with stride 1 equal to the sequential count everywhere. The C, Java, and Lua files add property lanes, line counts never dip below the sequential packing, never exceed one line per touch, and never shrink as the stride grows. Every ceiling is (a + b - 1) / b integer division.

The column walk of a 64x64 row-major grid strides one row per step, 64 ints or 256 bytes: 64 lines touched for 64 additions, against 4 lines for the row walk. This is why the standard advice holds, traverse multidimensional data in storage order, and why a jagged `int[][]`, one row per inner array, localizes that damage better than `int[,]` when rows are processed independently. The BCL sorts jagged arrays per row cheaply for the same reason.

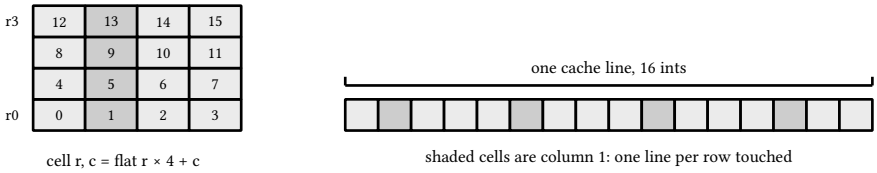


figure 10: row-major layout, the flat strip, and what a column walk costs

NOTE: the model counts lines, not nanoseconds

`LinesTouchedStrided` is a cost model, a deterministic function used in tests, not a cache simulator. Real caches prefetch sequential streams, so the measured speed gap is smaller than the line-count gap. The counts are still the right thing to assert because they are exact and directionally honest.

2.3. spans over arrays

The span is the same contiguous memory without a promise of ownership. Slicing takes a view, copying goes through it, and the line-walking count from the cache model becomes a zero allocation loop.

The dry run: the fixture is a 200 byte buffer seeded with 0xAB at offsets 0, 64, 128, 192, and 199, walked by CountByLine, asserted by all seven suites on the same values. The span is per-tree vocabulary, a pointer plus length struct in C, real slices in Go, a record over array, offset, and length in Java, subarray windows in JavaScript,

explicit (start, length) helpers in Python, offset math in Lua, and the out of range lane uses each tree's own refusal mechanism.

1. The first window slices bytes 0..63 and finds the seed at offset 0: one hit.
2. The walk advances one line at a time: windows 64..127 and 128..191 each add one hit at their first byte, three so far.
3. The tail window is 192..199, eight bytes: both 192 and 199 hit, 3 + 2 = 5.
4. The C# meter differenced around the whole walk reports exactly 0 bytes allocated, no window ever copies, the other trees carry no countable meter and claim nothing.
5. MiddleSlice(10, 4) then hands back 10, 11, 12, 13, a view materialized only by the final ToArray.

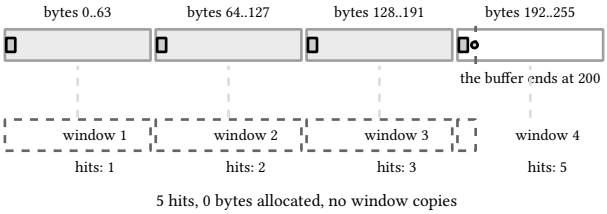


figure 11: the walk over the 200 byte buffer, four windows, the running hits under each

The 5 hits on 0 allocated bytes is the pinned pair, and the listings below are the walks themselves.

```c
// bounds guard: out of range start or length is an error, the analog of
// the c# IndexOutOfRangeException row
static int slice(const span_i s, int start, int count, span_i *out) {
  if (start < 0 || count < 0 || start + count > s.len)
    return 1;
  out->data = s.data + start;
  out->len = count;
  return 0;
}

static int *doubled(span_i s) {
  int *result = malloc((size_t)s.len * sizeof(int));
  for (int i = 0; i < s.len; i++)
    result[i] = s.data[i] * 2;
  return result;
}

static long sum_span(span_i s) {
  long sum = 0;
  for (int i = 0; i < s.len; i++)
    sum += s.data[i];
  return sum;
}

// walk the buffer one cache line at a time, no allocation for the chunks
static int count_by_line(span_b b, uint8_t needle) {
  int hits = 0, off = 0;
  while (off < b.len) {
    int take = b.len - off < LINE_BYTES ? b.len - off : LINE_BYTES;
    for (int i = 0; i < take; i++)
      if (b.data[off + i] == needle)
        hits++;
    off += take;
  }
  return hits;
}

static int *copy_span(span_i s) {
  int *target = malloc((size_t)s.len * sizeof(int));
  memcpy(target, s.data, (size_t)s.len * sizeof(int));
  return target;
}

static void fold(span_i s, long *sum, int *count) {
  *sum = 0;
  *count = s.len;
  for (int i = 0; i < s.len; i++)
    *sum += s.data[i];
}
```

listing 36: c, a pointer plus length span, the bounds guard, the line walk, fold (source: dsa/samples-c/src/Ch02/spans.c, lines 42-90)

```go
// CountByLine walks the buffer one cache line at a time and counts
// needle bytes, the chunk loop carrying (offset, length) pairs instead
// of allocating sub slices.
func CountByLine(buffer []byte, needle byte) int {
  hits := 0
  for start := 0; start < len(buffer); start += LineBytes {
    end := start + LineBytes
    if end > len(buffer) {
      end = len(buffer)
    }
    for _, b := range buffer[start:end] {
      if b == needle {
        hits++
      }
    }
  }
  return hits
}

// CopyViaSlice copies the source into a fresh slice: same contents,
// distinct object.
func CopyViaSlice(source []int) []int {
  target := make([]int, len(source))
  copy(target, source)
  return target
}

// MiddleSlice extracts length elements starting at start, with an
// explicit range check: an out of range request is a panic, the same
// contract the C# IndexOutOfRangeException row pins.
func MiddleSlice(source []int, start, length int) []int {
  if start < 0 || length < 0 || start+length > len(source) {
    panic("slice range out of bounds")
  }
  out := make([]int, length)
  copy(out, source[start:start+length])
  return out
}

// Fold reduces a slice to its sum and count in one walk.
func Fold(source []int) (sum int, count int) {
  for _, v := range source {
    sum += v
  }
  return sum, len(source)
}
```

listing 37: go, real slices, the line walk, the panicking middle slice, fold (source: dsa/samples-go/ch02/spans.go, lines 22-67)

The other five languages walk the same windows with their own slice vocabulary:

```
record Span(int[] data, int off, int len) {}

static Span spanOf(int[] data) {
  return new Span(data, 0, data.length);
}

// bounds guard: out of range start or length yields null, the analog of
// the c# IndexOutOfRangeException row
static Span slice(Span s, int start, int count) {
  if (start < 0 || count < 0 || start + count > s.len())
    return null;
  return new Span(s.data(), s.off() + start, count);
}

static int[] doubled(Span s) {
  int[] result = new int[s.len()];
  for (int i = 0; i < s.len(); i++)
    result[i] = s.data()[s.off() + i] * 2;
  return result;
}

static long sumSpan(Span s) {
  long sum = 0;
  for (int i = 0; i < s.len(); i++)
    sum += s.data()[s.off() + i];
  return sum;
}

// walk the buffer one cache line at a time, no allocation for the chunks
static int countByLine(byte[] b, byte needle) {
  int hits = 0, off = 0;
  while (off < b.length) {
    int take = Math.min(b.length - off, LINE_BYTES);
    for (int i = 0; i < take; i++)
      if (b[off + i] == needle)
        hits++;
    off += take;
  }
  return hits;
}

record Fold(long sum, int count) {}

static Fold fold(Span s) {
  long sum = 0;
  for (int i = 0; i < s.len(); i++)
    sum += s.data()[s.off() + i];
  return new Fold(sum, s.len());
}
```

listing 38: java, a record span over array, offset, length, the null-refusing guard, the line walk, fold (source: dsa/samples-java/src/Ch02/Spans.java, lines 22-70)

```
public static class Slices
{
    public static int[] Doubled(ReadOnlySpan<int> source)
    {
        var result = new int[source.Length];
        for (int i = 0; i < source.Length; i++)
            result[i] = source[i] * 2;
        return result;
    }

    public static long Sum(ReadOnlySpan<int> source)
    {
        long sum = 0;
        foreach (int v in source)
            sum += v;
        return sum;
    }

    // walk the buffer one cache line at a time, no allocation for the chunks
    public static int CountByLine(byte[] buffer, byte needle)
    {
        int hits = 0;
        ReadOnlySpan<byte> span = buffer;
        while (span.Length > 0)
        {
            ReadOnlySpan<byte> line = span[..Math.Min(Cache.LineBytes, span.Length)];
            foreach (byte b in line)
                if (b == needle)
                    hits++;
            span = span[line.Length..];
        }
        return hits;
    }

    public static int[] CopyViaSpan(int[] source)
    {
        var target = new int[source.Length];
        source.AsSpan().CopyTo(target);
        return target;
    }

    public static int[] MiddleSlice(int[] source, int start, int length) =>
        source.AsSpan(start, length).ToArray();

    public static (int Sum, int Count) Fold(ReadOnlySpan<int> source)
    {
        int sum = 0;
        foreach (int v in source)
            sum += v;
        return (sum, source.Length);
    }
}
```

listing 39: c#, slice views, the allocation free line walk, span copies (source: dsa/samples/src/Ch02/Arrays.cs, lines 69-120)

```
// walk the buffer one cache line at a time, each chunk a subarray window,
// no per-chunk allocation
export function countByLine(buffer, needle) {
  let hits = 0;
  for (let off = 0; off < buffer.length; off += LINE_BYTES) {
    const line = buffer.subarray(off, Math.min(off + LINE_BYTES, buffer.length));
    for (const b of line) if (b === needle) hits++;
  }
  return hits;
}

// a copy is a distinct object with equal contents
export function copySlice(source) {
  const target = new Array(source.length);
  for (let i = 0; i < source.length; i++) target[i] = source[i];
  return target;
}

// explicit (start, length) window with checked bounds, the mirror of the
// C# IndexOutOfRangeException row: out of range is an error, never undefined
export function middleSlice(source, start, length) {
  if (start < 0 || length < 0 || start + length > source.length) {
    throw new RangeError("slice out of range");
  }
  const out = new Array(length);
  for (let i = 0; i < length; i++) out[i] = source[start + i];
  return out;
}

export function fold(source) {
  let total = 0;
  for (const v of source) total += v;
  return { sum: total, count: source.length };
}
```

listing 40: javascript, subarray windows and explicit bounds, the line walk, fold (source: dsa/samples-js/src/ch02-spans.mjs, lines 19-52)

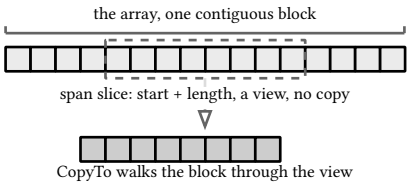
```
def doubled(src):
    out = [0] * len(src)
    for i, v in enumerate(src):
        out[i] = v * 2
    return out


def fold(src):
    total = 0
    for v in src:
        total += v
    return total, len(src)


def middle_slice(src, start, length):
    if start < 0 or length < 0 or start + length > len(src):
        raise IndexError("slice out of range")
    out = [0] * length
    for i in range(length):
        out[i] = src[start + i]
    return out


def count_by_line(buffer, needle):
    # walk one cache line at a time: no chunk is ever copied out
    hits = 0
    for off in range(0, len(buffer), LINE_BYTES):
        end = min(off + LINE_BYTES, len(buffer))
        for i in range(off, end):
            if buffer[i] == needle:
                hits += 1
    return hits
```

listing 41: python, explicit start and length helpers, the line walk, fold (source: dsa/samples-py/src/Ch02/spans.py, lines 17-48)

```lua
-- walk the buffer one cache line at a time, chunk by chunk
local function count_by_line(buf, needle)
  local hits, at = 0, 1
  while at ≤ #buf do
    local stop = math.min(at + LINE_BYTES - 1, #buf)
    for i = at, stop do
      if buf[i] == needle then hits = hits + 1 end
    end
    at = stop + 1
  end
  return hits
end

-- a copy is a distinct table with equal contents
local function copy_of(src)
  local out = {}
  for i = 1, #src do out[i] = src[i] end
  return out
end

-- start and length are 0-based offsets over src, bounds checked by hand
local function middle_slice(src, start, length)
  if start < 0 or length < 0 or start + length > #src then
    error("slice out of range")
  end
  local out = {}
  for k = 0, length - 1 do out[k + 1] = src[start + k + 1] end
  return out
end

-- fold returns the sum and the count as two values
local function fold(src)
  return sum(src), #src
end
```

listing 42: lua, offset math with hand checked bounds, the line walk, fold (source: dsa/samples-lua/ch02_spans.lua, lines 20-53)

Measured across the suites: doubled turns 1, 2, 3 into 2, 4, 6 with the input unchanged, the fold of 0..14 reads (105, 15), the line walk lands 5 hits on the seeded buffer, the copy round trip equals its source as a distinct object, and the (10, 4) window of 0..99 hands back 10, 11, 12, 13. The error lane refuses out of range start and length pairs per tree, a return code in C, a panic in Go, a null in Java, a throw in JavaScript and Python, an error in Lua. `CountByLine` walks a 200 byte buffer 64 bytes at a time and only C# proves zero allocation while doing it, the chapter 1 meter put to work. `CopyViaSpan` is `Array.Copy` in modern dress, same semantics, no overlap allowed between source and target views.



CountByLine reads the same window 64 bytes at a time, zero allocated

figure 12: the span as a window over the array block, slicing copies nothing, CopyTo goes through

2.4. bounds and safety

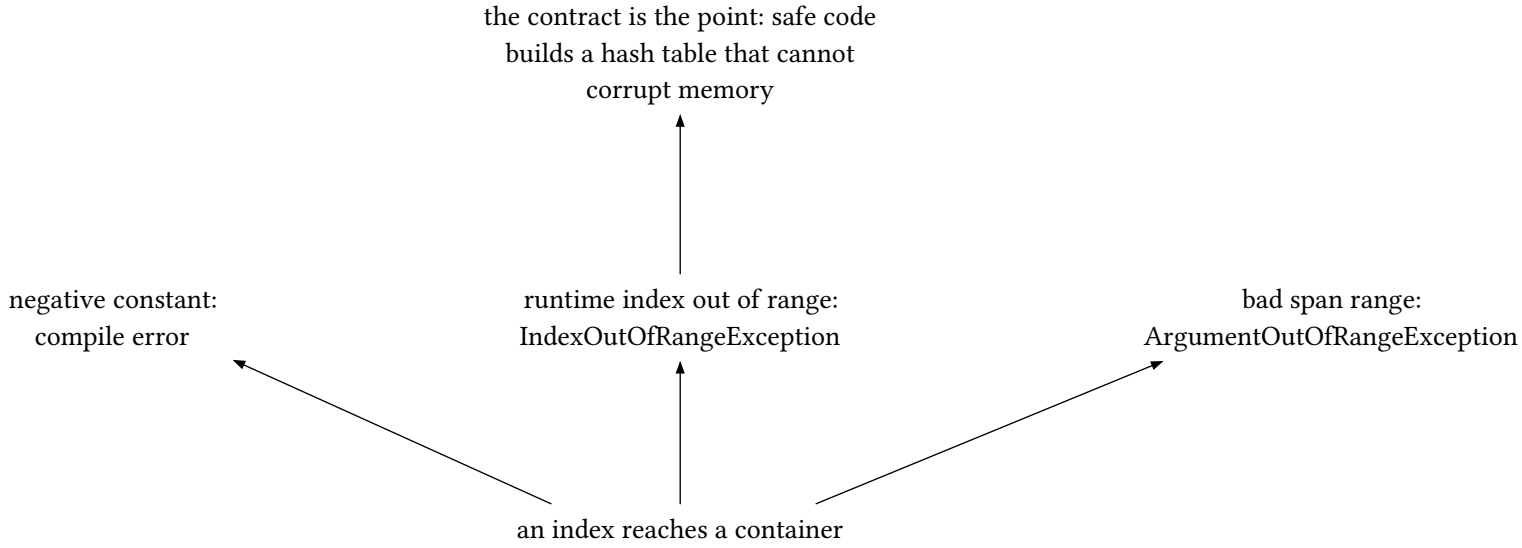


figure 13: bounds checks as a gate, three refusals and what the contract buys

The runtime checks every array index against the length. A negative index is a compile error when the compiler can see the constant and an `IndexOutOfRangeException` when it cannot, and the test pins the runtime side with an index smuggled through a local. `Span` carries the same discipline, `AsSpan(start, length)` throws `ArgumentOutOfRangeException` on bad ranges. These checks are not overhead to curse at: they are why this book can build a hash table in safe code that never corrupts memory, in any language with the same guarantee.

All seven suites build the guard by hand, a checked accessor plus an insertion-point search. For five of them it is necessity, their runtimes wrap, clamp, or trust the index, while C# and Java mirror the check their runtimes already run. C returns a refusal code before the read is ever formed.

The dry run: the fixture is the sorted run 1, 3, 5, 7 behind the checked accessor and the insertion point, asserted by the C#, C, Java, JavaScript, and Lua suites. Go pins the same semantics over 10, 20, 30, and Python's gate exists to refuse its own wrap habits.

1. The gate passes 2, 0 <= 2 < 4, and the read forms: 5. Index 0 returns 1 the same way.
2. The gate refuses 4, one past the end, and −1 and 1000000 with it: no read is ever formed outside the array.
3. The insertion point of 4 starts lo = 0, hi = 4: midpoint 2 holds 5, not below 4, so hi = 2.
4. Midpoint 1 holds 3 < 4, so lo = 2, and lo == hi = 2: absent 4 belongs at index 2.
5. The neighbors land 0 at 0, 8 past the end at 4, and duplicate 5 on its twin at 2.
6. InsertAt(2, 4) shifts 5 and 7 right one and lands 1, 3, 4, 5, 7 at length 5.

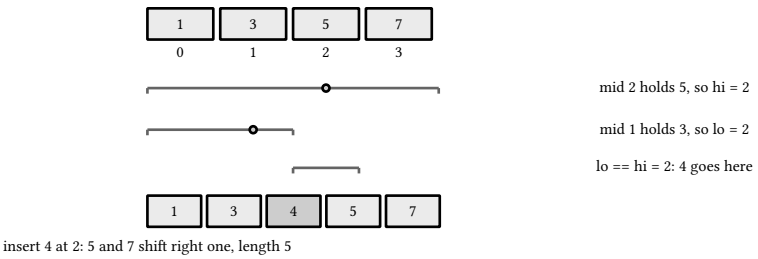


figure 14: the insertion point of 4 as a narrowing bracket over the run, then the tail shift

Absent 4 lands at index 2 in every suite that runs the fixture, and the listings below build the guard seven ways.

```c
// returns 0 without touching memory when i is out of range
static int at(const int *a, int len, long long i, int *out) {
  if (i < 0 || i ≥ len)
    return 0;
  *out = a[i];
  return 1;
}

// first index whose value is ≥ t; len when t is past the end
static int lower_bound(const int *a, int len, int t) {
  int lo = 0, hi = len;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}
```

listing 43: c, checked accessor and insertion point, no undefined access (source: dsa/samples-c/src/Ch02/bounds.c, lines 17-36)

```go
func GetChecked(m []int, i int) (int, error) {
  if i < 0 || i ≥ len(m) {
    return 0, fmt.Errorf("index %d out of range [0,%d)", i, len(m))
  }
  return m[i], nil
}

// FindSlot returns the insertion point for x in a sorted slice: the
// first index whose value is ≥ x, len(m) when x is past the end. The
// failed-search path still hands back useful information.
func FindSlot(sorted []int, x int) int {
  lo, hi := 0, len(sorted)
  for lo < hi {
    mid := lo + (hi-lo)/2
    if sorted[mid] < x {
      lo = mid + 1
    } else {
      hi = mid
    }
  }
  return lo
}
```

listing 44: go, a returned error that names the range, and the insertion point (source: dsa/samples-go/ch02/bounds.go, lines 9-30)

```java
static final int ABSENT = Integer.MIN_VALUE;

// returns ABSENT without touching memory when i is out of range for len
static int at(int[] a, int len, long i) {
  if (i < 0 || i ≥ len)
    return ABSENT;
  return a[(int) i];
}

static boolean present(int v) {
  return v ≠ ABSENT;
}

// first index whose value is ≥ t; length when t is past the end
static int lowerBound(int[] a, int len, int t) {
  int lo = 0, hi = len;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}

// insert v at pos into a with room, shifting the tail right
static int insertAt(int[] a, int len, int pos, int v) {
  for (int i = len; i > pos; i--)
    a[i] = a[i - 1];
  a[pos] = v;
  return len + 1;
}
```

listing 45: java, the refusing accessor over the logical length, lowerBound, the shift insert (source: dsa/samples-java/src/Ch02/Bounds.java, lines 20-52)

```
    /// <summary>Checked read: false, no touch, when i is out of range.</summary>
    public static bool TryAt(int[] a, int i, out int value)
    {
        if (i < 0 || i ≥ a.Length)
        {
            value = 0;
            return false;
        }
        value = a[i]; // the read only ever forms after the gate passed
        return true;
    }

    /// <summary>
    /// First index whose value is >= t; the window length when t is past
    /// the end. Duplicate values land on their leftmost twin.
    /// </summary>
    public static int LowerBound(ReadOnlySpan<int> a, int t)
    {
        int lo = 0, hi = a.Length;
        while (lo < hi)
        {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < t)
                lo = mid + 1;
            else
                hi = mid;
        }
        return lo;
```

listing 46: c#, refusal accessor and insertion point, the runtime check underneath (source: dsa/samples/src/Ch02/Bounds.cs, lines 9-36)

```
export function checkedAt(a, i) {
  if (!Number.isInteger(i) || i < 0 || i ≥ a.length) return undefined;
  return a[i];
}

// first index whose value is ≥ t; a.length when t is past the end
export function lowerBound(a, t) {
  let lo = 0;
  let hi = a.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] < t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}
```

listing 47: javascript, gate first, read second, insertion point by binary search (source: dsa/samples-js/src/ch02-bounds.mjs, lines 7-22)

```python
def safe_get(lst, i):
    # an explicit bounds gate: no wraparound, no exception, sentinel outside
    if 0 ≤ i < len(lst):
        return lst[i]
    return None


def strict_insert(lst, i, x):
    # list.insert silently clamps far positions; this one refuses them
    if not 0 ≤ i ≤ len(lst):
        raise ValueError(f"insertion point {i} outside 0..{len(lst)}")
    lst.insert(i, x)
```

listing 48: python, a gate against negative wrap and clamping insert (source: dsa/samples-py/src/Ch02/bounds.py, lines 25-36)

```lua
local function at(a, i)
  if i < 0 or i ≥ #a then return nil end
  return a[i + 1]
end

-- first index whose value is ≥ t; #a when t is past the end
local function lower_bound(a, t)
  local lo, hi = 1, #a + 1
  while lo < hi do
    local mid = lo + (hi - lo) // 2
    if a[mid] < t then lo = mid + 1 else hi = mid end
  end
  return lo - 1
end
```

listing 49: lua, nil refusal over one-based tables (source: dsa/samples-lua/ch02_bounds.lua, lines 7-20)

Anchors: C, C#, Java, JavaScript, and Lua run the insertion-point search over 1, 3, 5, 7 and pin absent 4 at index 2, 0 at 0, 8 past the end at 4, and duplicate 5 on its twin at 2. Go pins the same semantics over 10, 20, 30 with a five-case table. Python's fixture is its own warning, negative indices wrap and the builtin insert clamps far positions, so its gate refuses both habits. JavaScript's accessor also rejects 1.5, an integer check C does not need. C#'s runtime check stays the default guard, its tests pin the array and span exceptions alongside the same anchors. Java's fixture pads the run with spare slots, which splits the boundary in two: the gate refuses index 4 against the logical length while the JVM's own check still accepts the slot, and the raw read throws only past the physical end, both sides asserted.

2.5. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	279	libc only	pointer arithmetic proves the layout, refusal codes instead of exceptions, 52 checks in 4 files
go	121	fmt for errors	no unchecked mode exists, the choice is a panic or a returned error
java	273	jdk 27 stdlib	the jvm checks the physical length while the gate guards the logical one, the padded fixture asserts both boundaries, the span is a record that refuses with null

language	build SLOC	dependency	boundary note
c#	132	bcl only	the accessor mirrors the guard the runtime already runs, the span walk's zero-allocation proof is the chapter 1 meter at work
javascript	95	node stdlib	the gate rejects non-integer indices too, insertAt rebuilds through spread
python	177	stdlib only	negative indices wrap by design, the gate exists to refuse that habit
lua	257	lib.lua harness	1-based tables shift the insertion-point bounds and its return by one

sources: learn.microsoft.com, single dimensional arrays, multidimensional arrays, jagged arrays, `Span<T>` and `ReadOnlySpan<T>` api pages, `Array.Copy`, `Memory<T>` and `Span<T>` usage guidelines, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 18 tests in chapter 2 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 52 embedded checks, 16 Go tests, the java runner's 54 Ch02 checks over 4 files under `run-java-samples`, 18 `node --test` cases, 40 Python checks across 4 files, and 21 Lua checks under `run.lua`.

3. dynamic arrays and amortization

`List<T>` is the most used container in the language and almost nobody can say what its growth curve costs. This chapter rebuilds it with the copy counter from chapter 1 wired in, so the amortized claim becomes an assertion instead of folklore.

3.1. the vector

A growable array is a fixed array plus a count plus a growth policy. Every operation is array work with one twist, growing reallocates and copies. C keeps the whole anatomy in one struct and grows with a hand copy loop.

The dry run: the fixture is 33 pushes into a vector born at capacity 4, snapshotted at every growth, asserted by the C, Java, JavaScript, and Lua suites. Go stops the ladder at 32 pushes and 28 copies, Python at 16 and 12.

1. Pushes 1 through 4 fill the first block: size 4, capacity 4, no copies.
2. Push 5 finds the buffer full and doubles: 4 elements ride into the capacity 8 block, cumulative 4.
3. Push 9 doubles 8 to 16: 4 + 8 = 12.
4. Push 17 doubles 16 to 32: 12 + 16 = 28.
5. Push 33 doubles 32 to 64: 28 + 32 = 60, capacity 64 holding size 33.
6. Born at capacity 1 instead, 31 pushes grow at 1, 2, 4, 8, 16 and copy 1 + 2 + 4 + 8 + 16 = 31 = 2^5 - 1, copies equal to pushes.

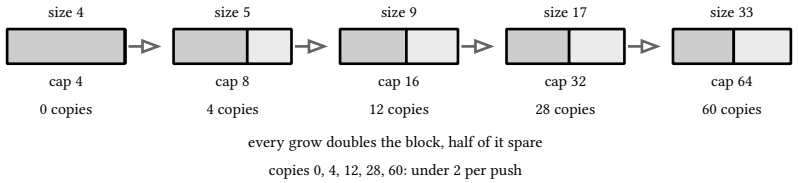


figure 15: the growth ladder as five buffer snapshots, the live prefix shaded, the copy counter under each

The 60 copies over 33 pushes and the 2^5 - 1 case are pinned, and the listings below build the vector seven ways.

```c
typedef struct {
  int *data;
  unsigned long long size, cap, copies, first_cap;
} Vec;

static void vec_init(Vec *v, unsigned long long first_cap) {
  v->data = NULL;
  v->size = v->cap = v->copies = 0;
  v->first_cap = first_cap;
}

static void vec_push(Vec *v, int x) {
  if (v->size == v->cap) {
    unsigned long long nc = v->cap ? v->cap * 2 : v->first_cap;
    int *nd = malloc(nc * sizeof *nd);
    for (unsigned long long i = 0; i < v->size; i++)
      nd[i] = v->data[i];
    v->copies += v->size;
    free(v->data);
    v->data = nd;
    v->cap = nc;
  }
  v->data[v->size++] = x;
}
```

listing 50: c, the struct, the init, and the push with its grow path (source: dsa/samples-c/src/Ch03/vector.c, lines 18-41)

```go
func (v *Vector) grow() {
  newCap := cap(v.items) * 2
  grown := make([]int, len(v.items), newCap)
  copy(grown, v.items)
  v.copies += int64(len(v.items))
  v.caps = append(v.caps, newCap)
  v.items = grown
}

// Push appends x, doubling the backing array when it is full.
func (v *Vector) Push(x int) {
  if len(v.items) == cap(v.items) {
    v.grow()
  }
  v.items = append(v.items, x)
}
```

listing 51: go, grow doubles and records, push rides on append (source: dsa/samples-go/ch03/vector.go, lines 23-39)

Six more languages build the same structure, Java hand-rolling the vec class because `ArrayList` would hide the growth this chapter teaches:

```
  static final class Vec {
    int[] data = new int[0];
    long size, cap, copies, firstCap;

    Vec(long firstCap) {
      this.firstCap = firstCap;
    }

    void push(int x) {
      if (size == cap) {
        long nc = cap != 0 ? cap * 2 : firstCap;
        int[] fresh = new int[(int) nc];
        for (int i = 0; i < size; i++)
          fresh[i] = data[i];
        copies += size;
        data = fresh;
        cap = nc;
      }
      data[(int) size++] = x;
    }

    int pop() {
      return data[(int) --size];
    }

    void insertAt(long pos, int x) {
      push(x);
      for (long i = size - 1; i > pos; i--) {
        int t = data[(int) i];
        data[(int) i] = data[(int) (i - 1)];
        data[(int) (i - 1)] = t;
      }
    }
  }
```

listing 52: java, a hand-rolled vec class, push, pop, insert-at with the counted grow (source: dsa/samples-java/src/Ch03/Vector.java, lines 19-52)

The C# version is generic and bounds checked:

```
public sealed class Vector<T>
{
    private T[] _items;
    private int _count;

    public Vector(int capacity = 4)
    {
        _items = new T[capacity];
        _count = 0;
    }

    public int Count => _count;
    public int Capacity => _items.Length;

    /// <summary>Total elements physically copied by growth since construction.</summary>
    public long CopiedOnGrow { get; private set; }

    /// <summary>Number of Grow calls, each one an O(n) event.</summary>
    public int GrowCalls { get; private set; }

    public T this[int index]
    {
        get
        {
            if ((uint)index >= (uint)_count)
                throw new ArgumentOutOfRangeException(nameof(index));
            return _items[index];
        }
        set
        {
            if ((uint)index >= (uint)_count)
                throw new ArgumentOutOfRangeException(nameof(index));
            _items[index] = value;
        }
    }

    public void Add(T item)
    {
        if (_count == _items.Length)
            Grow();
        _items[_count++] = item;
    }
```

listing 53: c#, the vector, counted growth, bounds checked indexer, add with grow on demand (source: dsa/samples/src/Ch03/Dynamic.cs, lines 4-45)

Insertion and removal at an interior index shift the tail one way or the other, so they cost proportional to the distance to the end. The version in the listing keeps that visible with `Array.Copy` doing the shift, exactly what `List<T>.Insert` does internally:

```
    // insert shifts the tail right: O(n - index)
    public void Insert(int index, T item)
    {
        if ((uint)index > (uint)_count)
            throw new ArgumentOutOfRangeException(nameof(index));
        if (_count == _items.Length)
            Grow();
        Array.Copy(_items, index, _items, index + 1, _count - index);
        _items[index] = item;
        _count++;
    }

    // remove shifts the tail left: O(n - index)
    public void RemoveAt(int index)
    {
        if ((uint)index ≥ (uint)_count)
            throw new ArgumentOutOfRangeException(nameof(index));
        Array.Copy(_items, index + 1, _items, index, _count - index - 1);
        _count--;
        _items[_count] = default!;
    }

    public void TrimExcess() => Array.Resize(ref _items, Math.Max(_count, 1));
```

listing 54: c#, interior insert and remove shift the tail, trim shrinks storage (source: dsa/samples/src/Ch03/Dynamic.cs, lines 46-69)

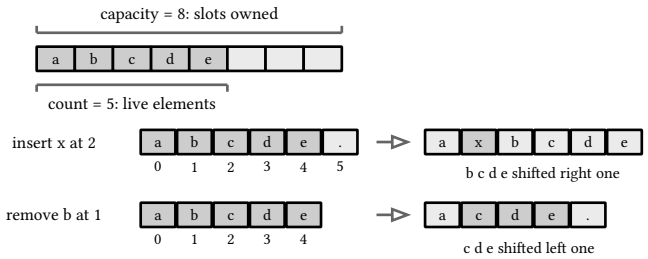


figure 16: the vector, a fixed array plus count plus capacity, and the tail shifts interior edits cost

Three more languages build the same structure:

```
push(x) {
  if (this.#size === this.#cap) {
    const next = this.#cap === 0 ? this.#firstCap : this.#cap * 2;
    const grown = new Array(next);
    for (let i = 0; i < this.#size; i++) grown[i] = this.#data[i];
    this.copies += this.#size;
    this.#data = grown;
    this.#cap = next;
  }
  this.#data[this.#size++] = x;
  return this;
}
```

listing 55: javascript, the push method and its grow path (source: dsa/samples-js/src/ch03-vector.mjs, lines 24-35)

```python
class Vector:
    def __init__(self):
        self.cap = 4
        self.size = 0
        self.buf = [None] * self.cap
        self.copies = 0
        self.caps = [4]

    def _grow(self):
        self.copies += self.cap
        self.buf = self.buf + [None] * self.cap
        self.cap *= 2
        self.caps.append(self.cap)

    def push(self, value):
        if self.size == self.cap:
            self._grow()
        self.buf[self.size] = value
        self.size += 1
```

listing 56: python, the class through push, ladder recorded (source: dsa/samples-py/src/Ch03/vector.py, lines 14-32)

```lua
local function vec_init(first_cap)
  return {data = {}, size = 0, cap = 0, copies = 0, first_cap = first_cap}
end

local function vec_push(v, x)
  if v.size == v.cap then
    local nc = v.cap > 0 and v.cap * 2 or v.first_cap
    local nd = {}
    for i = 1, v.size do nd[i] = v.data[i] end
    v.copies = v.copies + v.size
    v.data = nd
    v.cap = nc
  end
  v.size = v.size + 1
  v.data[v.size] = x
end
```

listing 57: lua, init and push over a table that copies anyway (source: dsa/samples-lua/ch03_vector.lua, lines 6-21)

The capacity ladder is the shared anchor. C, Java, JavaScript, and Lua snapshot every growth, capacity 4, 8, 16, 32, 64 at sizes 4, 5, 9, 17, 33 with cumulative copies 0, 4, 12, 28, 60, and all four pin the first-capacity-1 case where 31 pushes copy exactly 2^5 - 1. Go pins capacities 4, 8, 16, 32 with 28 copies over 32 pushes. Python pins caps 4, 8, 16 with 12 copies at 9 pushes, its insert riding spare capacity with no growth. Interior insert differs in mechanism, C, Java, JavaScript, and Lua push then bubble back, Go, Python, and C# shift the tail, and every suite pins the same resulting order.

3.2. the doubling argument

Chapter 1 stated the identity, this chapter runs it. C runs the two policies side by side, one counter each.

The dry run: the fixture is the side-by-side table from capacity 4, row by row, asserted by the C, Java, JavaScript, and Lua suites. C# exhibits 4096 appends, Python records a cost per append, Go cross-checks both totals.

1. One push and four pushes copy nothing: both policies still inside the first block.
2. Push 5 grows both: 4 elements copy, 4 against 4.

3. Push 17: doubling has grown at 4, 8, 16 for 4 + 8 + 16 = 28, step growth at 4, 8, 12, 16 for 4 + 8 + 12 + 16 = 40.
4. Push 31: doubling holds 28, its next growth waits for size 32, while step pays three more growths and reaches 112.
5. Push 65: doubling 124 against step 544, the last row.
6. Between the rows the bound breaks: at 128 appends step pays 1984 against doubling's 124, and 1984 > 4 × 128 = 512.
7. The C# exhibit scales the ladder: 4096 appends grow 12 times and copy 2^12 - 1 = 4095 elements, under one copy per append.

pushes	doubling copies	step +4 copies
1	0	0
4	0	0
5	4	4
17	28	40
31	28	112
65	124	544

The 4095 copies under one per append is the exhibit the tests pin exactly, and the listings below run the race in seven languages.

```c
static unsigned long long copies_doubling(unsigned long long n,
                                          unsigned long long cap0) {
  unsigned long long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap *= 2;
    }
    size++;
  }
  return copies;
}

static unsigned long long copies_step(unsigned long long n,
                                      unsigned long long cap0,
                                      unsigned long long step) {
  unsigned long long cap = cap0, size = 0, copies = 0;
  while (size < n) {
    if (size == cap) {
      copies += cap;
      cap += step;
    }
    size++;
  }
  return copies;
}
```

listing 58: c, both counters, doubling against constant step (source: dsa/samples-c/src/Ch03/growth.c, lines 17-42)

```go
func GrowConstant(n int) int64 {
  var copies int64
  capacity := GrowthStartCap
  for i := 0; i < n; i++ {
    if i == capacity {
      copies += int64(capacity)
      capacity += GrowthStartCap
    }
  }
  return copies
}

// GrowDouble appends n items, doubling capacity each time the buffer
// fills. Each copied block is smaller than the growth that follows it,
// so the total stays under 2n: amortized O(1) per append.
func GrowDouble(n int) int64 {
  var copies int64
  capacity := GrowthStartCap
  for i := 0; i < n; i++ {
    if i == capacity {
      copies += int64(capacity)
      capacity *= 2
    }
  }
  return copies
}
```

listing 59: go, both policies as plain counted loops (source: dsa/samples-go/ch03/growth.go, lines 9-34)

The other five languages run the same race, Java as two plain static loops over long:

```java
  static long copiesDoubling(long n, long cap0) {
    long cap = cap0, size = 0, copies = 0;
    while (size < n) {
      if (size == cap) {
        copies += cap;
        cap *= 2;
      }
      size++;
    }
    return copies;
  }

  static long copiesStep(long n, long cap0, long step) {
    long cap = cap0, size = 0, copies = 0;
    while (size < n) {
      if (size == cap) {
        copies += cap;
        cap += step;
      }
      size++;
    }
    return copies;
  }
```

listing 60: java, both counters as plain static loops over long (source: dsa/samples-java/src/Ch03/Growth.java, lines 18-40)

The C# growth routine is two counters and a resize:

```
    private void Grow()
    {
        GrowCalls++;
        CopiedOnGrow += _items.Length;
        Array.Resize(ref _items, _items.Length * 2);
    }
}
```

listing 61: c#, grow doubles and records the copy cost (source: dsa/samples/src/Ch03/Dynamic.cs, lines 71-78)

The exhibit is the head to head: doubling against growing by a constant step, same appends, counted copies:

```
public static class Amortization
{
    public sealed record GrowthReport(int Appends, int GrowCalls, long Copied)
    {
        public double CopiesPerAppend => (double)Copied / Appends;
    }

    public static GrowthReport Doubling(int appends)
    {
        var v = new Vector<int>(1);
        for (int i = 0; i < appends; i++)
            v.Add(i);
        return new GrowthReport(appends, v.GrowCalls, v.CopiedOnGrow);
    }

    // the anti pattern: grow by a constant, every append can copy the world
    public static GrowthReport ByConstant(int appends, int step)
    {
        int capacity = 1, count = 0;
        long copied = 0;
        int grows = 0;
        for (int i = 0; i < appends; i++)
        {
            if (count == capacity)
            {
                grows++;
                copied += capacity;
                capacity += step;
            }
            count++;
        }
        return new GrowthReport(appends, grows, copied);
    }
}
```

listing 62: c#, the two growth policies compared on the meter (source: dsa/samples/src/Ch03/Dynamic.cs, lines 80-113)

For 4096 appends doubling grows 12 times and copies 4095 elements, under one copy per append on average. Growing by a fixed step copies the triangular number, over 500 copies per append at the same size, and the gap widens forever. The tests pin both totals exactly. This is what amortized analysis means: an operation's average cost over any sequence, not its worst single call, and the average is what a caller actually experiences.

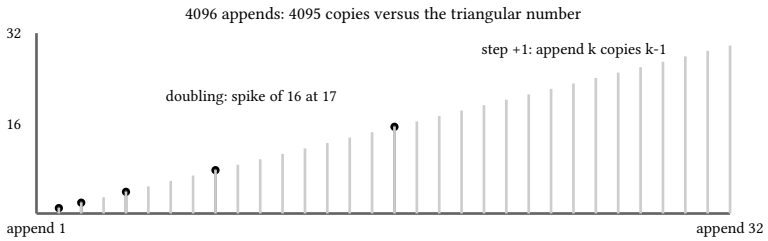


figure 17: copies per append on the meter, doubling spikes at powers of two while the constant step pays more every call

Three more languages run the same race:

```javascript
function doublingCopies(n, cap0) {
  let cap = cap0;
  let copies = 0;
  for (let size = 0; size < n; size++) {
    if (size === cap) {
      copies += cap;
      cap *= 2;
    }
  }
  return copies;
}

function stepCopies(n, cap0, step) {
  let cap = cap0;
  let copies = 0;
  for (let size = 0; size < n; size++) {
    if (size === cap) {
      copies += cap;
      cap += step;
    }
  }
  return copies;
}
```

listing 63: javascript, the two copy counters behind the table (source: dsa/samples-js/src/ch03-growth.mjs, lines 17-39)

```python
def copies_per_append(items, cap0, grow):
    cap, size, per = cap0, 0, []
    for _ in range(items):
        if size == cap:
            per.append(cap)  # the append that fills the buffer pays a copy
            cap = grow(cap)
        else:
            per.append(0)
        size += 1
    return per


def double(cap):
    return cap * 2


def add_four(cap):
    return cap + 4
```

listing 64: python, a cost recorded per append, the policy as a function (source: dsa/samples-py/src/Ch03/growth.py, lines 14-31)

```lua
local function copies_doubling(n, cap0)
  local cap, size, copies = cap0, 0, 0
  while size < n do
    if size == cap then
      copies = copies + cap
      cap = cap * 2
    end
    size = size + 1
  end
  return copies
end

local function copies_step(n, cap0, step)
  local cap, size, copies = cap0, 0, 0
  while size < n do
    if size == cap then
      copies = copies + cap
      cap = cap + step
    end
    size = size + 1
  end
  return copies
end
```

listing 65: lua, both counters in while loops (source: dsa/samples-lua/ch03_growth.lua, lines 6-28)

The numbers line up wherever the topic runs. C, Java, JavaScript, and Lua pin the side-by-side table, 28 against 40 copies at 17 appends, 124 against 544 at 65, and the bound, doubling never above 4 copies per append up to 200 while step growth pays 1984 for 128 appends. Python records a cost per append, 4 then 8 on the doubling ladder, 144 total for 36 step appends, and 612 at 72. Go runs both policies as plain loops with the totals cross-checked in its tests. C# exhibits 4096 appends, 12 grow calls, 4095 copied elements against a step policy past 500 copies per append.

NOTE: when the average is a lie

Amortization is honest only when the caller cannot observe the expensive call. A realtime or latency-sensitive path can feel every doubling of a large list, a hundreds of megabyte array copying itself is a pause. The standard mitigations are capacity pre-reservation, `List<T>` accepts it in the constructor, and pooled buffers, `ArrayPool<T>.Rent`, which trades the copy for reuse discipline.

3.3. list versus linked, the decision

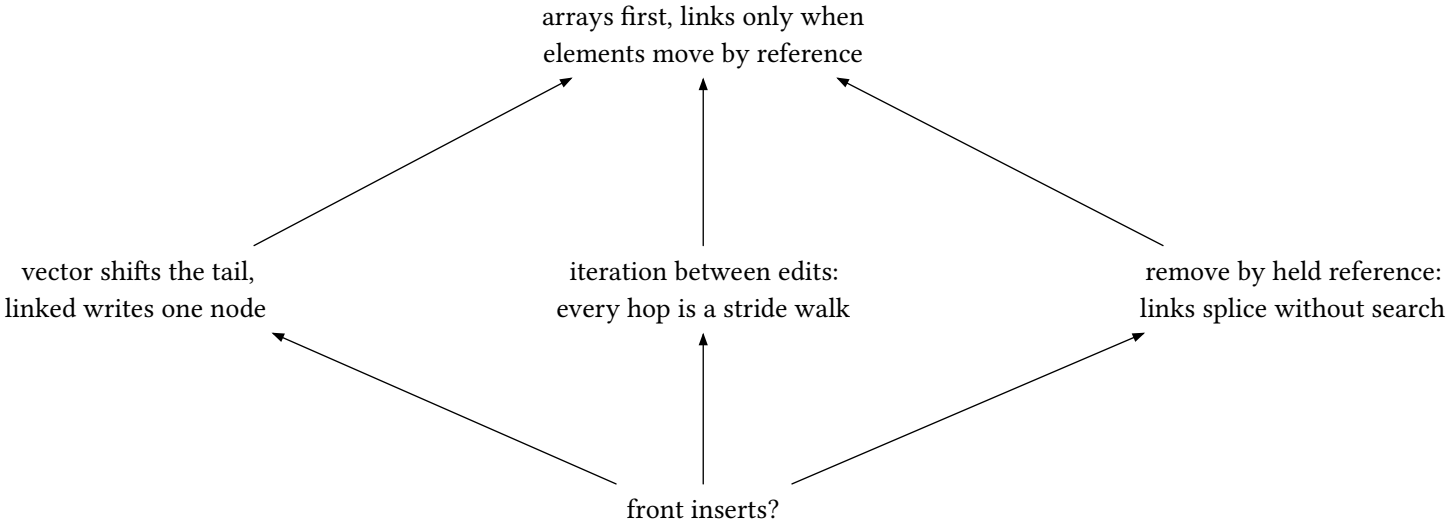


figure 18: list versus linked, where each structure actually wins

Front insertion is the vector's weak spot and the linked list's non-event: 500 front inserts into a vector shift the tail 500 times, while `AddFirst` on a linked list touches 1 node. The test suite runs both. The honest summary of the whole trade:

WARNING: the constant factors do not cancel

A linked node is a separate allocation with two references of overhead and every hop is a pointer chase, chapter 2's strided walk at stride one node. In practice a contiguous vector beats a linked list even for the linked list's nominal wins unless removals happen through held node references rather than searches. BCL guidance says the same: `LinkedList<T>` is rarely the right answer, and the next chapter's ring exists as much to show why as to show how.

The dry run: no suite pins these shift counts, the numbers are hand-derived, and the C# suite runs the 500-insert original and pins its final order.

1. The fixture: five front inserts of 4, 3, 2, 1, 0 into a vector with capacity to spare, grow noise removed the way the test removes it.
2. Insert 4 shifts nothing, 3 shifts one element, 2 shifts two, 1 shifts three, 0 shifts four: 0 + 1 + 2 + 3 + 4 = 10 shifted elements.
3. The linked list pays the same five inserts at one node write each, 5 writes, and no element ever moves.
4. Scaled to the test's 500 inserts, the vector shifts 499 × 500 / 2 = 124750 elements against 500 node writes.
5. The order is pinned at both ends: slot 0 reads 499 and slot 499 reads 0.

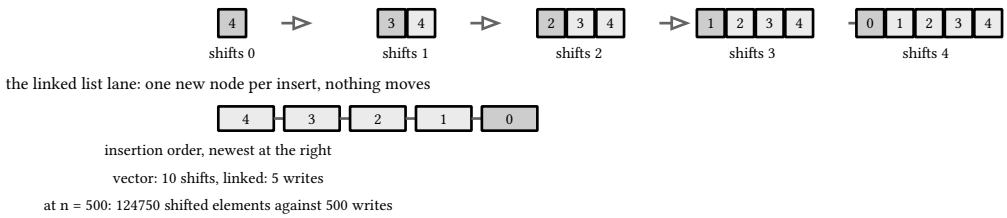


figure 19: five front inserts, the vector's growing shift against the linked list's flat one node per insert

The 124750 against 500 is the trade at the test's scale, arrays first until elements move by reference.

3.4. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	138	libc + malloc	the grow path hand copies and frees, copy totals in unsigned long long
go	76	fmt for errors	append does the copy, the count still records len at each grow
java	127	jdk 27 stdlib	ArrayList left out because it hides the taught growth, the vec is a static class with long counters, insert rides push then bubble
c#	92	bcl only	generic vector, the (uint) cast folds the negative check into the bound
javascript	79	node stdlib	private fields hold the state, push returns this for chaining
python	103	stdlib only	dead slots hold None, pop clears the slot it releases
lua	125	lib.lua harness	tables grow natively, this build copies anyway to keep the count honest

sources: learn.microsoft.com, `List<T>` api page including the growth remarks, `ArrayPool<T>`, `Array.Resize`, capacity management guidance in the performance docs, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 9 tests in chapter 3 of the samples suite. The seven-language layer verifies the same way: 2 C programs under `make verify-c`, 8 Go tests, the java runner's 31 Ch03 checks over 2 files under `run-java-samples`, 6 `node --test` cases, 20 Python checks across 2 files, and 8 Lua checks under `run.lua`.

4. linked structures

Pointer-linked nodes trade locality for flexibility: constant time insertion and removal anywhere, once you hold the node. This chapter builds the singly linked list, the in-place reversal, Floyd's cycle detection, and the sentinel ring that `LinkedList<T>` is, then says plainly when not to use any of it.

4.1. the singly linked list

Each node is a value and one forward reference. C makes that literal, a struct with one pointer, drawn from a static pool so the fixture never calls malloc.

The dry run: the fixture is the C# list built by three AddFirst calls and unspliced at 2, plus the abcdef reversal, asserted by the C# suite; the other six languages push 1 through 5 and pin the same flip on integers.

1. AddFirst(1), AddFirst(2), AddFirst(3) each point a fresh node at the old head: the read is 3, 2, 1, count 3.
2. Remove(2) walks to the predecessor 3 and unsplices with two writes: the read is 3, 1, while Remove(99) walks off the tail and returns false.
3. The reversal fixture builds f e d c b a the same way, one AddFirst per letter of abcdef.
4. The three pointers run prev = null, cur = f: each pass stashes next, flips cur onto prev, and slides, handing one letter to prev.
5. Six passes later the read is a b c d e f, the flip, and the degenerates hold: Reverse(null) returns null, one node ends with a null tail.

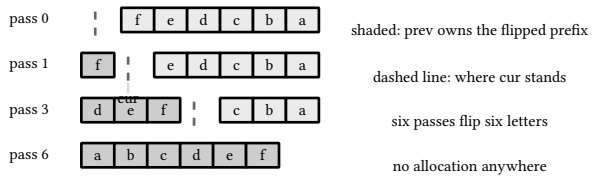


figure 20: the reversal as passes, the flipped prefix growing one letter at a time while the remainder waits

The 3, 2, 1 into 3, 1 and the abcdef round trip are the C# pins, and the listings below build the spine in seven languages.

```c
static Node *push_front(Node *head, int v) {
  Node *p = node_new(v);
  p->next = head;
  return p;
}

static Node *reverse(Node *head) {
  Node *prev = NULL;
  while (head) {
    Node *nx = head->next;
    head->next = prev;
    prev = head;
    head = nx;
  }
  return prev;
}

static Node *find(Node *head, int v) {
  for (; head; head = head->next)
    if (head->val == v)
      return head;
  return NULL;
}
```

listing 66: c, push, reverse, find, and to-array over a node pool (source: dsa/samples-c/src/Ch04/slist.c, lines 31-53)

```go
func (l *SList) PushFront(v int) {
  l.head = &Node{Value: v, next: l.head}
  l.n++
}

// Reverse unlinks the list and relinks it backwards with three
// pointers: prev holds the already-reversed prefix, cur the node being
// moved, nxt saves the rest before the link is overwritten.
func (l *SList) Reverse() {
  var prev *Node
  cur := l.head
  for cur != nil {
    nxt := cur.next
    cur.next = prev
    prev = cur
    cur = nxt
  }
  l.head = prev
}
```

listing 67: go, push front and the three pointer reverse as methods (source: dsa/samples-go/ch04/slist.go, lines 18-36)

Six more languages carry the same spine, Java spelling the C pointers as references with null for NULL:

```
  static final class Node {
    int val;
    Node next;
  }

  static Node pushFront(Node head, int v) {
    Node p = new Node();
    p.val = v;
    p.next = head;
    return p;
  }

  static Node reverse(Node head) {
    Node prev = null;
    while (head != null) {
      Node nx = head.next;
      head.next = prev;
      prev = head;
      head = nx;
    }
    return prev;
  }

  static Node find(Node head, int v) {
    for (; head != null; head = head.next)
      if (head.val == v)
        return head;
    return null;
  }

  static int toArray(Node head, int[] out) {
    int k = 0;
    for (; head != null; head = head.next)
      out[k++] = head.val;
    return k;
  }
```

listing 68: java, push front, three-pointer reverse, find, to-array over reference nodes (source: dsa/samples-java/src/Ch04/Slist.java, lines 18-53)

Removal unsplices by finding the predecessor, a search followed by a two-line fixup, the shape the C# class carries:

```
public sealed class Singly<T>
{
    public sealed class Node(T value)
    {
        public T Value { get; set; } = value;
        public Node? Next { get; set; }
    }

    private Node? _head;
    public int Count { get; private set; }

    public void AddFirst(T value)
    {
        _head = new Node(value) { Next = _head };
        Count++;
    }

    // O(n): find the node holding the value, then unsplice it
    public bool Remove(T value)
    {
        Node? prev = null;
        for (Node? cur = _head; cur ≠ null; (prev, cur) = (cur, cur.Next))
        {
            if (EqualityComparer<T>.Default.Equals(cur.Value, value))
            {
                if (prev == null)
                    _head = cur.Next;
                else
                    prev.Next = cur.Next;
                Count--;
                return true;
            }
        }
        return false;
    }

    public IEnumerable<T> Items()
    {
        for (Node? n = _head; n ≠ null; n = n.Next)
            yield return n.Value;
    }
```

listing 69: c#, add first, remove by scan, iterate (source: dsa/samples/src/Ch04/Linked.cs, lines 4-44)

Reversal is the canonical three pointer dance, `prev`, `cur`, `next`, one pass, no allocation, and it works on any head someone hands you:

```
    // three pointer reversal in place, O(n) time O(1) space
    public static Node? Reverse(Node? head)
    {
        Node? prev = null;
        while (head != null)
        {
            Node? next = head.Next;
            head.Next = prev;
            prev = head;
            head = next;
        }
        return prev;
    }

    public void ReverseInPlace() => _head = Reverse(_head);
```

listing 70: c#, in-place reversal, forward walk (source: dsa/samples/src/Ch04/Linked.cs, lines 46-61)

Three more languages carry the same spine:

```
export function pushFront(head, v) {
  const node = new Node(v);
  node.next = head;
  return node;
}

// the three-pointer reverse: prev, cur, next
export function reverse(head) {
  let prev = null;
  while (head) {
    const next = head.next;
    head.next = prev;
    prev = head;
    head = next;
  }
  return prev;
}
```

listing 71: javascript, push front and reverse as free functions over nodes (source: dsa/samples-js/src/ch04-slist.mjs, lines 11-27)

```python
def push_front(head, value):
    node = Node(value)
    node.next = head
    return node


def to_list(head):
    out = []
    while head:
        out.append(head.value)
        head = head.next
    return out


def reverse(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next  # the third pointer, saved before the twist
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

listing 72: python, push, walk out, reverse with the third pointer saved (source: dsa/samples-py/src/Ch04/slist.py, lines 21-43)

```lua
local function push_front(head, v)
  return {val = v, next = head}
end

local function reverse(head)
  local prev = nil
  while head do
    local nx = head.next
    head.next = prev
    prev = head
    head = nx
  end
  return prev
end

local function find(head, v)
  while head do
    if head.val == v then return head end
    head = head.next
  end
  return nil
```

listing 73: lua, each node a table with a next field (source: dsa/samples-lua/ch04_slist.lua, lines 5-25)

Every suite pins the reversal. C, Java, JavaScript, and Lua push 1 through 5, read the list back as 5 4 3 2 1, and read 1 2 3 4 5 after the flip. Go reverses a 3 node list to 3 2 1. Python pushes 5 4 3 2 1 so the list reads 1 through 5, flips it twice, and checks the round trip. Node shapes differ, a static pool in C, a heap node behind a reference in Java, `__slots__` in Python, a fresh table per node in Lua, and the reversal loop stays within a dozen lines in all seven.

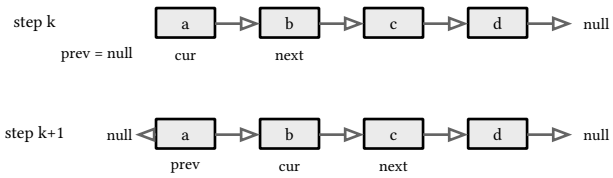


figure 21: the three pointer reversal, one iteration: stash next, flip cur onto prev, slide both

NOTE: the interview shape of reversal

Draw three boxes, prev, cur, next, and move them one node at a time: stash `next`, flip `cur` onto `prev`, slide both forward. Every test here is that loop with a different assertion, including the empty list, `Reverse(null)` returns null, and the single node, which must end with a null tail.

4.2. floyd's cycle detection

Can a fast and a slow pointer find a loop without a hash set? Yes, and the reason is arithmetic: if the list has a cycle of length k, once both pointers are inside it, the gap between them shrinks by one every step modulo k, so it must pass through zero. C runs both races, one to detect, one from the head to find the entry.

The dry run: the fixture is the rho 1, 2, 3, 4, 5 with the tail bent back onto 3, pinned by the C, Java, JavaScript, and Lua suites at entry 3, mu 2, lambda 3; Go pins the length, Python bends onto 2 instead, and C# flips a 10 node chain from acyclic to cyclic.

1. Both pointers start at the head 1: slow steps one node per turn, fast two.
2. Turn 1: slow sits at 2, fast has already reached 3, inside the loop.
3. Turn 2: slow reaches 3, the entry, while fast reaches 5, two ahead.
4. Turn 3: slow hops to 4 while fast hops 5 to 3 to 4: the meet, on 4.
5. The entry race walks head and meet in lockstep, 1 against 4, 2 against 5, then both land on 3: the entry, mu = 2.
6. Once around from 3 the loop reads 4, 5, back to 3: lambda = 3.

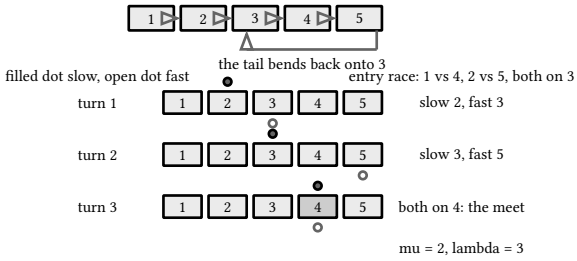


figure 22: the rho 1 to 5 with the tail bent onto 3, the race positions turn by turn, then the lockstep walk to the entry

The meet at 4 with mu 2 and lambda 3 is pinned four suites deep, and the listings below run both races in seven languages.

```c
static int has_cycle_meet(Node *head, Node **meet) {
  Node *slow = head, *fast = head;
  while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
    if (slow == fast) {
      *meet = slow;
      return 1;
    }
  }
  return 0;
}

static Node *cycle_start(Node *head, Node *meet) {
  while (head != meet) {
    head = head->next;
    meet = meet->next;
  }
  return head;
}

static int cycle_len(Node *start) {
  int k = 1;
  Node *p = start->next;
  while (p != start) {
    k++;
    p = p->next;
  }
  return k;
}
```

listing 74: c, meet detection, entry race, cycle length (source: dsa/samples-c/src/Ch04/slist.c, lines 62-91)

```go
func HasCycle(head *Node) (bool, *Node) {
  slow, fast := head, head
  for fast != nil && fast.next != nil {
    slow = slow.next
    fast = fast.next.next
    if slow == fast {
      return true, slow
    }
  }
  return false, nil
}
```

listing 75: go, has cycle returns the meeting node for the second race (source: dsa/samples-go/ch04/slist.go, lines 81-91)

The other five languages run the same races, Java walking reference equality for the meet:

```
  static Node hasCycleMeet(Node head) {
    Node slow = head, fast = head;
    while (fast != null && fast.next != null) {
      slow = slow.next;
      fast = fast.next.next;
      if (slow == fast)
        return slow;
    }
    return null;
  }

  static Node cycleStart(Node head, Node meet) {
    while (head != meet) {
      head = head.next;
      meet = meet.next;
    }
    return head;
  }

  static int cycleLen(Node start) {
    int k = 1;
    Node p = start.next;
    while (p != start) {
      k++;
      p = p.next;
    }
    return k;
  }
```

listing 76: java, meet detection, entry race, cycle length over reference nodes (source: dsa/samples-java/src/Ch04/Slist.java, lines 55-82)

```
    public static bool HasCycle(Node head)
    {
        Node? slow = head, fast = head.Next;
        while (fast != null && fast.Next != null)
        {
            if (slow == fast)
                return true;
            slow = slow!.Next;
            fast = fast.Next.Next;
        }
        return false;
    }

    public Node? Head => _head;

    public static Node? NextOf(Node? n) => n?.Next;
}
```

listing 77: c#, tortoise and hare, O(1) space (source: dsa/samples/src/Ch04/Linked.cs, lines 63-79)

The test builds a 10 node chain, splices the tail onto the third node, and asserts the detection flips. The same machinery extends to finding the cycle's entry point, the point where the distances aligned, and to finding the middle of a list in one pass.

```javascript
// floyd: slow steps one, fast steps two. on a cycle they meet
export function hasCycleMeet(head) {
  let slow = head;
  let fast = head;
  while (fast && fast.next) {
    slow = slow.next;
    fast = fast.next.next;
    if (slow === fast) return { has: true, meet: slow };
  }
  return { has: false, meet: null };
}

// after the meet, walking head and meet in lockstep finds the entry
export function cycleStart(head, meet) {
  while (head !== meet) {
    head = head.next;
    meet = meet.next;
  }
  return head;
}
```

listing 78: javascript, meet, entry race, and cycle length (source: dsa/samples-js/src/ch04-slist.mjs, lines 40-59)

```python
def floyd(head):
    # slow walks one hop, fast two; on a cycle they must meet, and a race
    # from head and the meeting spot then lands on the cycle entry
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow is fast:
            entry = head
            while entry is not slow:
                entry = entry.next
                slow = slow.next
            return True, entry
    return False, None
```

listing 79: python, both races folded into one function (source: dsa/samples-py/src/Ch04/slist.py, lines 54-67)

```lua
-- floyd: returns the meeting node inside the cycle or nil
local function has_cycle_meet(head)
  local slow, fast = head, head
  while fast and fast.next do
    slow = slow.next
    fast = fast.next.next
    if slow == fast then return slow end
  end
  return nil
end

local function cycle_start(head, meet)
  while head ~= meet do
    head = head.next
    meet = meet.next
  end
  return head
end

local function cycle_len(start)
  local k, p = 1, start.next
  while p ~= start do
    k = k + 1
    p = p.next
  end
  return k
end
```

listing 80: lua, meet, entry, and length over table nodes (source: dsa/samples-lua/ch04_slist.lua, lines 37-63)

The rho fixtures differ where the suites chose them. C, Java, JavaScript, and Lua bend 1 2 3 4 5 back onto the node holding 3 and pin the entry, mu 2, lambda 3. Go bends the same list and pins the cycle length 3. Python bends the tail onto 2, entry value 2, cycle length 4. C# splices a 10 node chain onto the third node and asserts the detection flips. The argument is identical everywhere, the second race from the head lands on the entry because both walkers then sit the same distance above it.

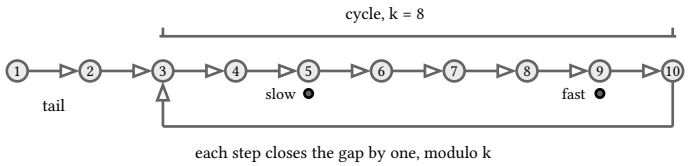


figure 23: floyd's rho, once both pointers are inside the cycle the gap closes by one per step

4.3. the sentinel ring

The production shape of a doubly linked list is a ring around a sentinel node. The sentinel deletes every null check: the list is never empty, it always contains the sentinel, and `_sentinel.Next` is the first element, `_sentinel.Prev` the last. In C the same ring is a file-scope sentinel plus four splices.

The dry run: the fixture is the C# ring across its three shapes, both-end order, position-free removal, and the version counter, asserted by the C# suite; C, Java, JavaScript, and Lua push 1, 0, 2, 9 and read 9 0 1 2 instead.

1. AddLast b, AddLast c, AddFirst a: forward reads a b c, backward reads c b a, count 3, the sentinel invisible at both ends.
2. The int ring AddLast 1, 2, 3 hands back the node references, then Remove(b) splices 1 straight onto 3 with no traversal: 1, 3.
3. Remove(a) and Remove(c) drain it, forward reads empty, and AddLast(9) still lands: the sentinel never left.

4. MoveToFront(x) unsplices and reinserts at the head in two constant-time splices: x y z reads x y z.
5. Every structural change bumps the version, v0 + 1 on the add, v0 + 2 after the remove, so in-flight enumeration is invalidated.

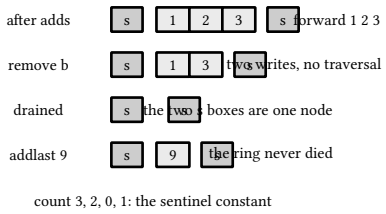


figure 24: the int ring run, add three, unsplice the middle, drain, reinsert, the same sentinel node at both ends of every frame

The 1, 3 unsplice, the 9 after the drain, and the version bumps are the C# pins, and the listings below build the ring in seven languages.

```c
static void ring_init(void) {
  sentinel.next = &sentinel;
  sentinel.prev = &sentinel;
  pool_n = 0;
}

static RNode *node_new(int v) {
  RNode *p = &pool[pool_n++];
  p->val = v;
  return p;
}

static void push_back(int v) {
  RNode *p = node_new(v);
  p->next = &sentinel;
  p->prev = sentinel.prev;
  sentinel.prev->next = p;
  sentinel.prev = p;
}

static void push_front(int v) {
  RNode *p = node_new(v);
  p->prev = &sentinel;
  p->next = sentinel.next;
  sentinel.next->prev = p;
  sentinel.next = p;
}

static void remove_node(RNode *p) {
  p->prev->next = p->next;
  p->next->prev = p->prev;
}
```

listing 81: c, ring init, both-end pushes, remove by node (source: dsa/samples-c/src/Ch04/ring.c, lines 26-57)

```go
func (r *Ring) splice(n, at *RingNode) {
  n.prev = at.prev
  n.next = at
  at.prev.next = n
  at.prev = n
}

// PushFront inserts x right after the sentinel, at the front.
func (r *Ring) PushFront(v int) *RingNode {
  n := &RingNode{Value: v}
  r.splice(n, r.root.next)
  r.n++
  return n
}
```

listing 82: go, one splice helper feeds both pushes (source: dsa/samples-go/ch04/ring.go, lines 25-38)

The other five rings, Java keeping the sentinel a real node that never becomes null:

```
  static RNode sentinel;

  static void ringInit() {
    sentinel = new RNode();
    sentinel.next = sentinel;
    sentinel.prev = sentinel;
  }

  static RNode nodeNew(int v) {
    RNode p = new RNode();
    p.val = v;
    return p;
  }

  static void pushBack(int v) {
    RNode p = nodeNew(v);
    p.next = sentinel;
    p.prev = sentinel.prev;
    sentinel.prev.next = p;
    sentinel.prev = p;
  }

  static void pushFront(int v) {
    RNode p = nodeNew(v);
    p.prev = sentinel;
    p.next = sentinel.next;
    sentinel.next.prev = p;
    sentinel.next = p;
  }

  static void removeNode(RNode p) {
    p.prev.next = p.next;
    p.next.prev = p.prev;
  }

  static int iterate(int[] out) {
    int k = 0;
    for (RNode p = sentinel.next; p != sentinel; p = p.next)
      out[k++] = p.val;
    return k;
  }
```

listing 83: java, the sentinel node, both-end pushes, remove by node, iterate (source: dsa/samples-java/src/Ch04/Ring.java, lines 24-64)

The C# class wraps the identical shape:

```
public sealed class LinkedRing<T>
{
    public sealed class Node(T value)
    {
        public T Value { get; set; } = value;
        public Node Prev { get; internal set; } = null!;
        public Node Next { get; internal set; } = null!;
    }

    private readonly Node _sentinel = new(default!);
    private int _version; // mutated on every structural change

    public LinkedRing() => _sentinel.Prev = _sentinel.Next = _sentinel;

    public int Count { get; private set; }

    public int Version => _version;

    public Node AddLast(T value)
    {
        Node node = new(value);
        InsertBefore(_sentinel, node);
        return node;
    }

    public Node AddFirst(T value)
    {
        Node node = new(value);
        InsertBefore(_sentinel.Next, node);
        return node;
    }
```

listing 84: c#, the ring with sentinel, both-end insertion (source: dsa/samples/src/Ch04/Linked.cs, lines 82-113)

Removal by node reference is the structure's whole value: the node knows both neighbors, no traversal happens. The test removes a middle node, then the first, then the last, then re-adds after emptying, and the sentinel keeps every case identical. The version counter is the enumerator-safety discipline `LinkedList<T>` also carries, structural changes invalidate in-flight iteration:

```
    public void Remove(Node node)
    {
        node.Prev.Next = node.Next;
        node.Next.Prev = node.Prev;
        Count--;
        _version++;
    }

    private void InsertBefore(Node at, Node node)
    {
        node.Prev = at.Prev;
        node.Next = at;
        at.Prev.Next = node;
        at.Prev = node;
        Count++;
        _version++;
    }

    public IEnumerable<T> Forward()
    {
        for (Node? n = _sentinel.Next; n ≠ _sentinel; n = n.Next)
            yield return n.Value;
    }

    public IEnumerable<T> Backward()
    {
        for (Node? n = _sentinel.Prev; n ≠ _sentinel; n = n.Prev)
            yield return n.Value;
    }

    public void MoveToFront(Node node)
    {
        Remove(node);
        InsertBefore(_sentinel.Next, node);
    }
}
```

listing 85: position-free removal, both directions, move to front (source: dsa/samples/src/Ch04/Linked.cs, lines 115-151)

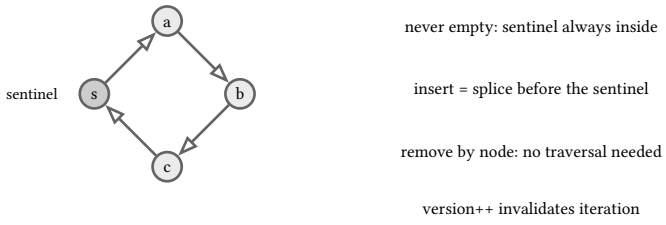


figure 25: the sentinel ring, never empty, both-end inserts and neighbor-known removal

`MoveToFront` is the LRU cache operation in miniature, remove plus reinsert at the head, both constant, which is why LRU caches are built as a hash map into a ring of nodes, chapter 6 meets chapter 4.

Three more rings:

```javascript
  pushBack(v) {
    const p = new RingNode(v);
    const tail = this.#sentinel.prev;
    p.prev = tail;
    p.next = this.#sentinel;
    tail.next = p;
    this.#sentinel.prev = p;
    return p;
  }

  pushFront(v) {
    const p = new RingNode(v);
    const head = this.#sentinel.next;
    p.next = head;
    p.prev = this.#sentinel;
    head.prev = p;
    this.#sentinel.next = p;
    return p;
  }

  remove(p) {
    p.prev.next = p.next;
    p.next.prev = p.prev;
    return this;
  }
```

listing 86: javascript, the ring class with both pushes and remove (source: dsa/samples-js/src/ch04-ring.mjs, lines 16-40)

```python
class Ring:
    def __init__(self):
        self.sentinel = RingNode()
        self.count = 0

    def _insert_after(self, at, node):
        node.next = at.next
        node.prev = at
        at.next.prev = node
        at.next = node
        self.count += 1

    def push_back(self, value):
        self._insert_after(self.sentinel.prev, RingNode(value))

    def push_front(self, value):
        self._insert_after(self.sentinel, RingNode(value))

    def remove(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev
        self.count -= 1
```

listing 87: python, insert after, both pushes, local remove (source: dsa/samples-py/src/Ch04/ring.py, lines 23-44)

```lua
local function ring_init()
  local s = {val = "*sentinel*"}
  s.next, s.prev = s, s
  return s
end

local function push_back(s, v)
  local p = {val = v, next = s, prev = s.prev}
  s.prev.next = p
  s.prev = p
end

local function push_front(s, v)
  local p = {val = v, prev = s, next = s.next}
  s.next.prev = p
  s.next = p
end

local function remove_node(p)
  p.prev.next = p.next
  p.next.prev = p.prev
end
```

listing 88: lua, the sentinel table and its four splices (source: dsa/samples-lua/ch04_ring.lua, lines 6-27)

C, Java, JavaScript, and Lua pin the same both-end fixture, pushes 1, 0, 2, 9 land as 9 0 1 2, the middle 1 unlinks to 9 0 2, and after a full drain the sentinel points at itself while remaining a valid node. Go pins pushes to 0 1 2, removal to 2 3, and a reinsert after drain to 9. Python walks both directions, 0 1 2 3 forward and its mirror backward, removes a middle node to 0 1 3, and proves the emptied ring folds onto the sentinel. C# adds what the others leave out, a version counter and `MoveToFront`, the LRU splice.

4.4. when to reach for links

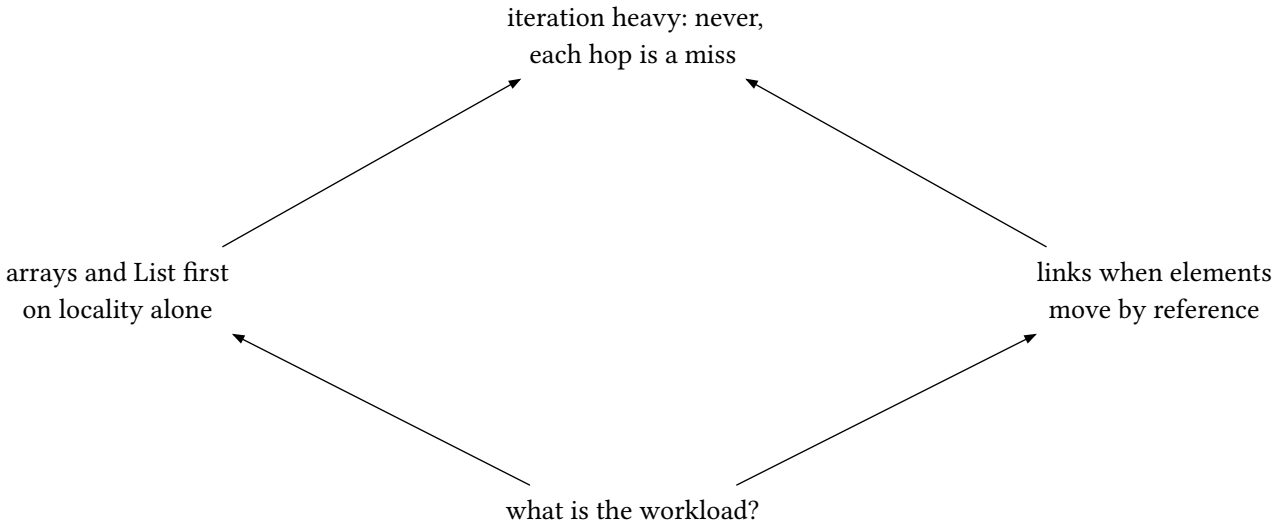


figure 26: when to reach for links, the honest ranking

The honest ranking in this language: arrays and `List<T>` first, the constant factors of locality win almost everything. `LinkedList<T>` when elements must be removed or moved by reference from several places at once, an LRU, a scheduler wheel, a free list. Never for iteration heavy or search heavy work, every hop is a cache miss that chapter 2 already priced. The BCL itself uses linked structures sparingly, and the chapters ahead quietly prefer arrays with indices as node handles, a heap in an array, a union-find in arrays, a trie as a flat map, for exactly this reason.

The dry run: no suite carries this walk, the numbers are hand-derived on chapter 2's line model, and the workload on trial is one pass over 2^20 = 1048576 elements followed by three removals.

1. The array lane packs 16 ints to a 64 byte line, so the pass touches 1048576 / 16 = 65536 lines.
2. The linked lane gives every node its own allocation, a value plus a next pointer, and the allocator scatters them: 1048576 hops, one line each.
3. The ratio 1048576 / 65536 = 16 is chapter 2's stride-16 degenerate case, which is why iteration-heavy work never gets links.
4. The one win: removing 3 held nodes from a 1000 element vector, at the front, the middle, the end, shifts 999 + 499 + 0 = 1498 elements, the ring splices each in 2 writes, 3 × 2 = 6, no search.
5. The ranking falls out: arrays first, links when elements move by reference, never when the loop dominates the cost.

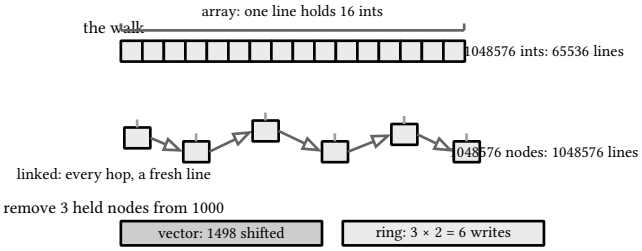


figure 27: the walk price, packed array lanes against scattered nodes, then the removal contrast running the other way

The 16 to 1 walk price against the 6 writes is the whole trade in two numbers, arrays first unless elements move by reference.

4.5. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	208	libc only	nodes come from static pools, no malloc anywhere in the fixtures, 21 checks in 2 files
go	127	none	unexported next pointers keep the list encapsulated, Tail and NodeAfter exist for the tests
java	202	jdk 27 stdlib	references spell the c pointers and null spells NULL, reference equality is the cycle meet, the sentinel stays a real node that never becomes null
c#	122	bcl only	generic rings with a version counter, the only suite that invalidates in-flight iteration
javascript	99	node stdlib	identity comparison is the cycle test, === on node references
python	144	stdlib only	`__slots__` nodes, is comparison for the meet, floyd folds both races into one function
lua	165	lib.lua harness	a node is a plain table, the sentinel carries a label value

sources: learn.microsoft.com, `LinkedList<T>` api page including the sentinel-based implementation remarks, `LinkedListNode<T>`, enumeration and thread safety notes, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 9 tests in chapter 4 of the samples suite. The seven-language layer verifies the same way: 2 C programs with 21 embedded checks, 7 Go tests, the java runner's 21 Ch04 checks over 2 files under `run-java-samples`, 7 `node --test` cases, 16 Python checks across 2 files, and 9 Lua checks under `run.lua`.

5. stacks, queues, deques

Two disciplines organize more algorithms than any other pair: last in first out and first in first out. Both are array exercises, and both earn their keep in the classic algorithms this chapter runs: bracket matching and the shunting yard.

5.1. the stack

A stack is a vector with one legal entry point. Push, pop, peek, and an honest underflow, C first with refusal codes.

The dry run: the fixture is the 1, 2, 3 run, asserted by the C# suite, with the 1000 push growth exhibit in C# and the capacity 8 refusal in C, Java, JavaScript, and Lua.

1. Push 1, Push 2, Push 3 write at the count: the count is 3 and the top is 3.
2. Pop returns 3, then 2, then 1, the count 0 after the drain.
3. Pop or Peek on the empty stack throws, no value fabricated.
4. The growth exhibit: 1000 pushes hold count 1000 and Pop returns 999, chapter 3's doubling underneath.
5. The bounded contrast: C, Java, JavaScript, and Lua cap the stack at 8 and the ninth push refuses where C# grows.

op	slots after	count	returns
push 1	1	1	
push 2	1 2	2	
push 3	1 2 3	3	
pop	1 2	2	3
pop	1	1	2
pop		0	1

The 3, 2, 1 drain and the 999 off the 1000 push run are pinned, and the listings below build the stack seven ways.

```c
static int push(char c) {
  if (sp == CAP)
    return 0;
  st[sp++] = c;
  return 1;
}

static int pop(char *out) {
  if (sp == 0)
    return 0;
  *out = st[--sp];
  return 1;
}

static int peek(char *out) {
  if (sp == 0)
    return 0;
  *out = st[sp - 1];
  return 1;
}
```

listing 89: c, push, pop, peek over a bounded char array (source: dsa/samples-c/src/Ch05/stack.c, lines 21-40)

```
func (s *Stack) Push(v int) { s.items = append(s.items, v) }

// Pop removes and returns the top element.
func (s *Stack) Pop() (int, error) {
  if len(s.items) == 0 {
    return 0, fmt.Errorf("pop from empty stack")
  }
  v := s.items[len(s.items)-1]
  s.items = s.items[:len(s.items)-1]
  return v, nil
}

// Peek returns the top element without removing it.
func (s *Stack) Peek() (int, error) {
  if len(s.items) == 0 {
    return 0, fmt.Errorf("peek at empty stack")
  }
  return s.items[len(s.items)-1], nil
}
```

listing 90: go, push, pop, peek over a slice head (source: dsa/samples-go/ch05/stack.go, lines 12-30)

The call protocol is the useful part. Anything naturally nested, calls, scopes, undo entries, xml depth, becomes a stack the moment reversal matters: the most recent opener must close first.

```
  static final int CAP = 8;

  static char[] st = new char[CAP];
  static int sp;

  static boolean push(char c) {
    if (sp == CAP)
      return false;
    st[sp++] = c;
    return true;
  }

  static Character pop() {
    if (sp == 0)
      return null;
    return st[--sp];
  }

  static Character peek() {
    if (sp == 0)
      return null;
    return st[sp - 1];
  }
```

listing 91: java, the bounded stack, pops and peeks surface as null on underflow (source: dsa/samples-java/src/Ch05/Stack.java, lines 18-40)

The C# class grows instead of refusing at the top end:

```
public sealed class ArrayStack<T>
{
    private T[] _items = new T[4];

    public int Count { get; private set; }

    public void Push(T item)
    {
        if (Count == _items.Length)
            Array.Resize(ref _items, _items.Length * 2);
        _items[Count++] = item;
    }

    public T Pop()
    {
        if (Count == 0)
            throw new InvalidOperationException("stack is empty");
        T item = _items[--Count];
        _items[Count] = default!;
        return item;
    }

    public T Peek() => Count == 0
        ? throw new InvalidOperationException("stack is empty")
        : _items[Count - 1];
}
```

listing 92: c#, push, pop, peek, the empty-stack errors (source: dsa/samples/src/Ch05/Buffers.cs, lines 4-30)

```
  push(c) {
    if (this.#slots.length === this.#cap) return false;
    this.#slots.push(c);
    return true;
  }

  // null on underflow, never a fabricated value
  pop() {
    if (this.#slots.length === 0) return null;
    return this.#slots.pop();
  }

  peek() {
    return this.#slots.length === 0 ? null : this.#slots[this.#slots.length - 1];
  }
```

listing 93: javascript, push, pop, peek with null on underflow (source: dsa/samples-js/src/ch05-stack.mjs, lines 12-26)

```python
class Stack:
    def __init__(self):
        self.items = []

    def push(self, value):
        self.items.append(value)

    def pop(self):
        if not self.items:
            raise IndexError("pop on empty stack")
        return self.items.pop()

    def peek(self):
        if not self.items:
            raise IndexError("peek on empty stack")
        return self.items[-1]

    def __len__(self):
        return len(self.items)
```

listing 94: python, the stack class with IndexError refusals (source: dsa/samples-py/src/Ch05/stack.py, lines 13-31)

```lua
local function push(st, c)
  if st.sp == CAP then return false end
  st.sp = st.sp + 1
  st.slots[st.sp] = c
  return true
end

local function pop(st)
  if st.sp == 0 then return nil end
  local c = st.slots[st.sp]
  st.slots[st.sp] = nil
  st.sp = st.sp - 1
  return c
end

local function peek(st)
  if st.sp == 0 then return nil end
  return st.slots[st.sp]
end
```

listing 95: lua, push, pop, peek, nil slots released on pop (source: dsa/samples-lua/ch05_stack.lua, lines 11-29)

Underflow refuses everywhere, never fabricates a value. C returns refusal codes, Go returns errors, C# throws InvalidOperationException, Python raises IndexError, and Java, JavaScript, and Lua return null and nil. The top end differs: C, Java, JavaScript, and Lua run bounded stacks that also refuse at capacity 8, while C#, Go, and Python grow without a bound, chapter 3's doubling doing the work.

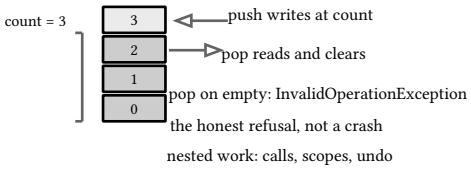


figure 28: the stack, one legal entry point: push writes at the count, pop reads and clears it

5.2. the ring buffer

A queue on a naive array dequeues by shifting everything left, which is quadratic churn. The ring keeps two cursors in a fixed array and lets arithmetic do the moving. The head is where the next dequeue reads, the tail is where the next enqueue writes, and both wrap with a modulo. C keeps the whole story in 28 lines, grow included.

The dry run: the fixture is Ring(4) across the seam, asserted by the C# suite; C, Java, JavaScript, and Lua pin the same layout 5 6 3 4 and the 4 element grow copy.

1. Enqueue 1, 2, 3, 4 fills to capacity: count 4, capacity still 4, head and tail both back at slot 0.
2. Dequeue returns 1 then 2, and the head walks to slot 2.
3. Enqueue 5 lands in slot 0, the wrap, and 6 in slot 1: the physical layout reads 5 6 3 4.
4. The drain returns 3, 4, 5, 6: fifo order survives the seam.
5. The growth exhibit: 50 enqueues into Ring(2) climb the capacity 2 to 64 and still drain 0 through 49 in order.

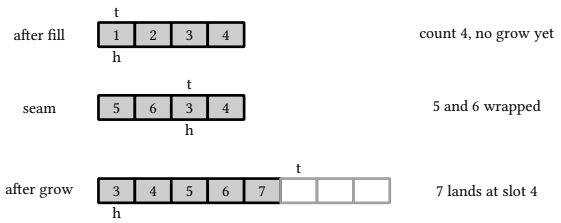


figure 29: the seam run as three layouts: full and level, split across the seam, then the grow rebased to slot 0

The 5 6 3 4 seam and the in-order drain are pinned, and the listings below build the ring in seven languages.

```c
static void grow(RQ *r) {
  int ns[8];
  for (int i = 0; i < r->count; i++)
    ns[i] = r->slots[(r->head + i) % r->cap];
  r->copies += (unsigned long long)r->count;
  for (int i = 0; i < r->count; i++)
    r->slots[i] = ns[i];
  r->head = 0;
  r->tail = r->count % (r->cap * 2);
  r->cap *= 2;
}

static void enqueue(int v) {
  if (q.count == q.cap)
    grow(&q);
  q.slots[q.tail] = v;
  q.tail = (q.tail + 1) % q.cap;
  q.count++;
}

static int dequeue(int *out) {
  if (q.count == 0)
    return 0;
  *out = q.slots[q.head];
  q.head = (q.head + 1) % q.cap;
  q.count--;
  return 1;
}
```

listing 96: c, grow rebases to slot 0, enqueue and dequeue wrap (source: dsa/samples-c/src/Ch05/ringqueue.c, lines 30-57)

```
func (q *RingQueue) Enqueue(v int) {
  if q.n == len(q.buf) {
    grown := make([]int, len(q.buf)*2)
    for i := 0; i < q.n; i++ {
      grown[i] = q.buf[(q.head+i)%len(q.buf)]
    }
    q.growCopy += int64(q.n)
    q.buf = grown
    q.head, q.tail = 0, q.n
    q.lastGrew = true
  } else {
    q.lastGrew = false
  }
  q.buf[q.tail] = v
  q.tail = (q.tail + 1) % len(q.buf)
  q.n++
}
```

listing 97: go, enqueue with the grow inlined and counted (source: dsa/samples-go/ch05/ringqueue.go, lines 24-40)

The other five rings keep the seam handling visible, Java unwinding `(_head + i) % cap` back to slot zero in its grow:

```
  static final class RQ {
    int[] slots = new int[8];
    int head, tail, count, cap;
    long copies;
  }

  static RQ q = new RQ();

  static void init(int cap) {
    q = new RQ();
    q.cap = cap;
  }

  static void grow() {
    int[] ns = new int[8];
    for (int i = 0; i < q.count; i++)
      ns[i] = q.slots[(q.head + i) % q.cap];
    q.copies += q.count;
    for (int i = 0; i < q.count; i++)
      q.slots[i] = ns[i];
    q.head = 0;
    q.tail = q.count % (q.cap * 2);
    q.cap *= 2;
  }

  static void enqueue(int v) {
    if (q.count == q.cap)
      grow();
    q.slots[q.tail] = v;
    q.tail = (q.tail + 1) % q.cap;
    q.count++;
  }

  static Integer dequeue() {
    if (q.count == 0)
      return null;
    int v = q.slots[q.head];
    q.head = (q.head + 1) % q.cap;
    q.count--;
    return v;
  }
```

listing 98: java, the ring state, the seam-unwinding grow, enqueue and dequeue (source: dsa/samples-java/src/Ch05/Ringqueue.java, lines 17-57)

```
public sealed class Ring<T>
{
    private T[] _items;
    private int _head; // next dequeue position
    private int _count;

    public Ring(int capacity = 8)
    {
        if (capacity < 1)
            throw new ArgumentOutOfRangeException(nameof(capacity));
        _items = new T[capacity];
    }
```

```
    public int Count => _count;
    public int Capacity => _items.Length;

    public void Enqueue(T item)
    {
        if (_count == _items.Length)
            Grow();
        _items[(_head + _count) % _items.Length] = item;
        _count++;
    }

    public T Dequeue()
    {
        if (_count == 0)
            throw new InvalidOperationException("ring is empty");
        T item = _items[_head];
        _items[_head] = default!;
        _head = (_head + 1) % _items.Length;
        _count--;
        return item;
    }

    public bool TryDequeue(out T item)
    {
        if (_count == 0)
        {
            item = default!;
            return false;
        }
        item = Dequeue();
        return true;
    }

    private void Grow()
    {
        var bigger = new T[_items.Length * 2];
        for (int i = 0; i < _count; i++)
            bigger[i] = _items[(_head + i) % _items.Length];
        _head = 0;
        _items = bigger;
    }
}
```

listing 99: c#, the circular buffer, wraparound, grow in order (source: dsa/samples/src/Ch05/Buffers.cs, lines 32-87)

The tests walk the boundary cases: filling exactly to capacity without growing, the wrap after dequeueing from a full ring, and growth preserving fifo order when the logical sequence is split across the array seam. That seam handling in `Grow`, copying `(_head + i) % len` back to slot zero, is the one subtle line in the structure, and the interleaved test exists to keep it honest.

```
  // grow copies the logical order, head first, back to slot 0
  #grow() {
    const next = new Array(this.#cap * 2);
    for (let i = 0; i < this.#count; i++)
      next[i] = this.#slots[(this.#head + i) % this.#cap];
    this.copies += this.#count;
    this.#slots = next;
    this.#head = 0;
    this.#tail = this.#count;
    this.#cap *= 2;
  }

  enqueue(v) {
    if (this.#count === this.#cap) this.#grow();
    this.#slots[this.#tail] = v;
    this.#tail = (this.#tail + 1) % this.#cap;
    this.#count++;
    return this;
  }

  // null on underflow, never a fabricated value
  dequeue() {
    if (this.#count === 0) return null;
    const v = this.#slots[this.#head];
    this.#head = (this.#head + 1) % this.#cap;
    this.#count--;
    return v;
  }
}
```

listing 100: javascript, private grow, enqueue, dequeue (source: dsa/samples-js/src/ch05-ringqueue.mjs, lines 37-65)

```python
    def enqueue(self, value):
        if self.count == len(self.buf):
            self._grow()
        self.buf[self.tail] = value
        self.tail = (self.tail + 1) % len(self.buf)
        self.count += 1

    def dequeue(self):
        if self.count == 0:
            raise IndexError("dequeue on empty queue")
        value = self.buf[self.head]
        self.buf[self.head] = None
        self.head = (self.head + 1) % len(self.buf)
        self.count -= 1
        return value

    def _grow(self):
        fresh = [None] * (len(self.buf) * 2)
        for i in range(self.count):
            # the wrap seam unwinds: live items re-land from slot 0
            fresh[i] = self.buf[(self.head + i) % len(self.buf)]
        self.buf = fresh
        self.head = 0
        self.tail = self.count
        self.growths += 1
```

listing 101: python, enqueue, dequeue, the seam-unwinding grow (source: dsa/samples-py/src/Ch05/ringqueue.py, lines 22-46)

```lua
local function grow(q)
  local ns = {}
  for i = 0, q.count - 1 do
    ns[i + 1] = q.slots[(q.head + i) % q.cap + 1]
  end
  q.copies = q.copies + q.count
  for i = 0, q.count - 1 do q.slots[i + 1] = ns[i + 1] end
  q.head = 0
  q.tail = q.count % (q.cap * 2)
  q.cap = q.cap * 2
end

local function enqueue(q, v)
  if q.count == q.cap then grow(q) end
  q.slots[q.tail + 1] = v
  q.tail = (q.tail + 1) % q.cap
  q.count = q.count + 1
end

local function dequeue(q)
  if q.count == 0 then return nil end
  local v = q.slots[q.head + 1]
  q.slots[q.head + 1] = nil
  q.head = (q.head + 1) % q.cap
  q.count = q.count - 1
  return v
end
```

listing 102: lua, grow, enqueue, dequeue over 1-based slots (source: dsa/samples-lua/ch05_ringqueue.lua, lines 9-35)

The seam fixture pins identically in C, Java, JavaScript, and Lua: capacity 4, dequeue two, enqueue two, physical layout 5 6 3 4 with head at slot 2, then growth copies 3 4 5 6 back to slot 0, 7 lands at slot 4, and the copy counter reads 4. Python runs the same story with letters, c d e f g across the seam and head 0 tail 5 after growth. Go pins the seam positions and the grow copy count in its tests. C# does the same rebase inside `Grow`, one door over.

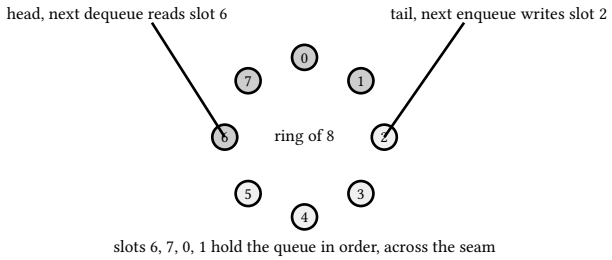


figure 30: the ring, four live slots split across the seam

The BCL `Queue<T>` and the producer consumer `System.Threading.Channels` are this shape plus locking discipline. The `Channels` version is the async answer when producers and consumers are tasks, and the ring here is its single threaded ancestor.

5.3. bracket matching

The canonical stack algorithm. Openers push, closers pop and compare, and any mismatch or leftover opener fails.

The dry run: the fixture is the theory table over “([]{})”, “([)]”, “(((“, “)”, “”, and “a[b]{c}(d)”, asserted by the C# suite; the other six pin “([)]” false and “{[()]}” true instead.

1. (opens the run: the stack holds (.
2. [pushes, then] pops it and the pair matches: back to (.
3. { pushes, then } pops it and matches: back to (.
4.) pops the last (and matches: the stack empties, the string balances.
5. The crossed “([)]” dies one char early,) expects (on top and finds [, and “(((“ fails on the leftover openers, while “” never pushes and the non-brackets in “a[b]{c}(d)” ride along.

char	action	stack after	verdict
(	push	(	
[	push	([	
]	pop, match	(	
{	push	({	
}	pop, match	(	
)	pop, match		balanced

The crossed false against the nested true is pinned by every suite, and the listings below build the matcher seven ways.

```c
static int opens(char c) { return c == '(' || c == '[' || c == '{'; }

static char closer_for(char c) {
  if (c == '(')
    return ')';
  if (c == '[')
    return ']';
  return '}';
}

static int balanced(const char *s) {
  sp = 0;
  for (const char *p = s; *p; p++) {
    if (opens(*p)) {
      if (!push(*p))
        return 0;
      continue;
    }
    char top;
    if (!pop(&top) || closer_for(top) != *p)
      return 0;
  }
  return sp == 0;
}
```

listing 103: c, the matcher over the same three primitives (source: dsa/samples-c/src/Ch05/stack.c, lines 42-65)

```go
func BracketMatch(s string) bool {
  var opens Stack // indices would work; rune values suffice here
  for _, r := range s {
    switch r {
    case '(', '[', '{':
      opens.Push(int(r))
    case ')', ']', '}':
      top, err := opens.Pop()
      if err != nil {
        return false // closer with nothing open
      }
      if top != int(openOf(r)) {
        return false // wrong kind of closer
      }
    }
  }
  return opens.Len() == 0
}
```

listing 104: go, bracket match on the slice-backed stack (source: dsa/samples-go/ch05/stack.go, lines 38-55)

```
  static boolean opens(char c) {
    return c == '(' || c == '[' || c == '{';
  }

  static char closerFor(char c) {
    if (c == '(')
      return ')';
    if (c == '[')
      return ']';
    return '}';
  }

  static boolean balanced(String s) {
    sp = 0;
    for (int i = 0; i < s.length(); i++) {
      char c = s.charAt(i);
      if (opens(c)) {
        if (!push(c))
          return false;
        continue;
      }
      Character top = pop();
      if (top == null || closerFor(top) != c)
        return false;
    }
    return sp == 0;
  }
```

listing 105: java, opens and closerFor, every non-opener pops as a closer (source: dsa/samples-java/src/Ch05/Stack.java, lines 42-68)

```
public static class Expression
{
    public static bool IsBalanced(string text)
    {
        var open = new ArrayStack<char>();
        var match = new Dictionary<char, char> { [')'] = '(', [']'] = '[', ['}'] = '{' };
        foreach (char c in text)
        {
            if (c is '(' or '[' or '{')
                open.Push(c);
            else if (match.TryGetValue(c, out char need))
            {
                if (open.Count == 0 || open.Pop() != need)
                    return false;
            }
        }
        return open.Count == 0;
    }
```

listing 106: c#, nested delimiters, dictionary of expectations (source: dsa/samples/src/Ch05/Buffers.cs, lines 89-106)

```javascript
const OPENS = new Set(["(", "[", "{"]);
const CLOSER_FOR = { "(": ")", "[": "]", "{": "}" };

// match brackets left to right: every closer must pair the stack top, an
// empty stack refuses a closer, and the stack must drain exactly empty
export function balanced(s) {
  const st = new Stack();
  for (const c of s) {
    if (OPENS.has(c)) {
      if (!st.push(c)) return false;
      continue;
    }
    const top = st.pop();
    if (top === null || CLOSER_FOR[top] !== c) return false;
  }
  return st.size === 0;
}
```

listing 107: javascript, opener sets and closer maps (source: dsa/samples-js/src/ch05-stack.mjs, lines 33-49)

```python
PAIRS = {")": "(", "]": "[", "}": "{"}


def balanced(text):
    stack = Stack()
    for ch in text:
        if ch in "([{":
            stack.push(ch)
        elif ch in ")]}":
            if len(stack) == 0 or stack.pop() != PAIRS[ch]:
                return False
    return len(stack) == 0
```

listing 108: python, the pairs map and the scan (source: dsa/samples-py/src/Ch05/stack.py, lines 34-45)

```lua
local OPENS = {["("] = true, ["["] = true, ["{"] = true}
local CLOSER = {["("] = ")", ["["] = "]", ["{"] = "}"}

local function balanced(s)
  local st = stack_new()
  for i = 1, #s do
    local c = s:sub(i, i)
    if OPENS[c] then
      if not push(st, c) then return false end
    else
      local top = pop(st)
      if top == nil or CLOSER[top] ~= c then return false end
    end
  end
  return st.sp == 0
end
```

listing 109: lua, table lookups for openers and closers (source: dsa/samples-lua/ch05_stack.lua, lines 31-46)

Every suite pins "([])" false and the empty string true. C, Java, JavaScript, Lua, and Python pin "{[()]}" true, C#'s positive fixture is "([]{})". One semantic fork the suites each state: C#, Go, and Python ignore non-bracket characters and pin fixtures that use them, while the C, Java, JavaScript, and Lua matchers treat every non-opener as a closer, so foreign input fails there.

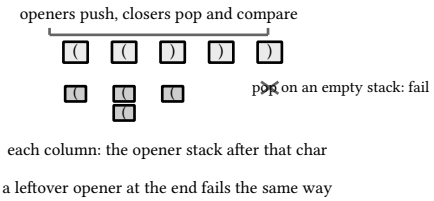


figure 31: bracket matching, the scan with the opener stack under each step

5.4. shunting yard

Dijkstra's algorithm converts infix to postfix with one operator stack and one output list. Precedence decides when to pop, and the paren pops until its partner.

The dry run: the fixture is "3 + 4 * 2" through the yard and back, asserted by the C# suite, with the associativity pin "8 - 3 - 2" beside it.

1. The yard passes 3, 4, and 2 to the output, stacks + , and * climbs on top of it, precedence 2 over 1.
2. The flush pops * then + , and the postfix reads 3 4 2 * +, pinned as a string.
3. The evaluation pass pushes 3, 4, 2, then * pops twice: 4 × 2 = 8, and 8 goes back on.
4. 1. pops 8 and 3: 3 + 8 = 11, the pinned answer.
5. The associativity pin: the second minus of "8 - 3 - 2", precedence equal at 1, pops the first, the output reads 8 3 - 2 -, and the evaluation runs 8 - 3 = 5 then 5 - 2 = 3, the (8 - 3) - 2 grouping.

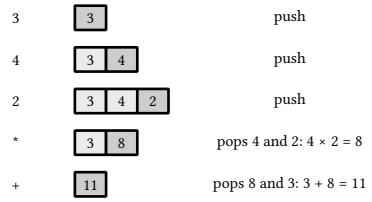


figure 32: the evaluation pass over 3 4 2 * +, the operand stack after each token, two pops and a push per operator

The 11 and the associativity 3 are the pinned pair, and the listings below run the yard in seven languages.

```
static int to_postfix(void) {
  int out = 0, sp = 0;
  char ops[64];
  for (int i = 0; i < tn; i++) {
    Tok t = toks[i];
    if (t.kind == NUM) {
      rpn[out++] = t;
    } else if (t.kind == OP) {
      while (sp > 0 && ops[sp - 1] != '(' && prec(ops[sp - 1]) >= prec(t.op))
        rpn[out++] = (Tok){OP, 0, ops[--sp]};
      ops[sp++] = t.op;
    } else if (t.kind == LP) {
      ops[sp++] = '(';
    } else {
      while (sp > 0 && ops[sp - 1] != '(')
        rpn[out++] = (Tok){OP, 0, ops[--sp]};
      sp--; // discard the '('
    }
  }
  while (sp > 0)
    rpn[out++] = (Tok){OP, 0, ops[--sp]};
  return out;
}
```

listing 110: c, the yard over char and token arrays (source: dsa/samples-c/src/Ch05/shunting.c, lines 61-83)

```
func ToPostfix(infix string) ([]string, error) {
  var out []string
  var ops []string
  tokens, err := tokenize(infix)
  if err ≠ nil {
    return nil, err
  }
  for _, tok := range tokens {
    switch tok {
    case "(":
      ops = append(ops, tok)
    case ")":
      for len(ops) > 0 && ops[len(ops)-1] ≠ "(" {
        out = append(out, ops[len(ops)-1])
        ops = ops[:len(ops)-1]
      }
      if len(ops) == 0 {
        return nil, fmt.Errorf("unbalanced ) in %q", infix)
      }
      ops = ops[:len(ops)-1] // drop the (
    case "+", "-", "*", "/":
      for len(ops) > 0 && ops[len(ops)-1] ≠ "(" &&
        precedence(ops[len(ops)-1]) ≥ precedence(tok) {
        out = append(out, ops[len(ops)-1])
        ops = ops[:len(ops)-1]
      }
      ops = append(ops, tok)
    default:
      out = append(out, tok) // a number
    }
  }
  for len(ops) > 0 {
    if ops[len(ops)-1] == "(" {
      return nil, fmt.Errorf("unbalanced ( in %q", infix)
    }
    out = append(out, ops[len(ops)-1])
    ops = ops[:len(ops)-1]
  }
  return out, nil
}
```

listing 111: go, the yard with error returns for bad input (source: dsa/samples-go/ch05/shunting.go, lines 25-64)

The other five yards, Java evaluating through a switch expression over a record token:

```
static int prec(char op) {
  return (op == '+' || op == '-') ? 1 : 2;
}

// dijkstra shunting yard: numbers pass through, operators pop equal or
// higher precedence (left associative), parens group
static Tok[] rpn = new Tok[64];

static int toPostfix() {
  int out = 0, sp = 0;
  char[] ops = new char[64];
  for (int i = 0; i < tn; i++) {
    Tok t = toks[i];
    if (t.kind() == NUM) {
      rpn[out++] = t;
    } else if (t.kind() == OP) {
      while (sp > 0 && ops[sp - 1] != '(' && prec(ops[sp - 1]) >= prec(t.op()))
        rpn[out++] = new Tok(OP, 0, ops[--sp]);
      ops[sp++] = t.op();
    } else if (t.kind() == LP) {
      ops[sp++] = '(';
    } else {
      while (sp > 0 && ops[sp - 1] != '(')
        rpn[out++] = new Tok(OP, 0, ops[--sp]);
      sp--; // discard the '('
    }
  }
  while (sp > 0)
    rpn[out++] = new Tok(OP, 0, ops[--sp]);
  return out;
}

static long eval(int len) {
  long[] st = new long[64];
  int sp = 0;
  for (int i = 0; i < len; i++) {
    if (rpn[i].kind() == NUM) {
      st[sp++] = rpn[i].num();
      continue;
    }
    long b = st[--sp], a = st[--sp];
    st[sp++] = switch (rpn[i].op()) {
      case '+' -> a + b;
      case '-' -> a - b;
      case '*' -> a * b;
      default -> a / b;
    };
  }
  return st[0];
}

static long calc(String s) {
  lex(s);
  return eval(toPostfix());
}
```

listing 112: java, precedence, the yard, and the value stack through a switch expression (source: dsa/samples-java/src/Ch05/Shunting.java, lines 49-103)

```
    public static string ToPostfix(string infix)
    {
        var output = new List<string>();
        var ops = new ArrayStack<char>();
        var precedence = new Dictionary<char, int> { ['+'] = 1, ['-'] = 1, ['*'] = 2, ['/'] = 2 };
        foreach (string token in infix.Split(' ', StringSplitOptions.RemoveEmptyEntries))
        {
            if (token.Length == 1 && precedence.ContainsKey(token[0]))
            {
                char op = token[0];
                // left associative: pop same or higher precedence first
                while (ops.Count > 0 && precedence.TryGetValue(ops.Peek(), out int p) && p ≥
precedence[op])
                    output.Add(ops.Pop().ToString());
                ops.Push(op);
            }
            else if (token == "(")
                ops.Push('(');
            else if (token == ")")
            {
                while (ops.Peek() ≠ '(')
                    output.Add(ops.Pop().ToString());
                ops.Pop(); // discard the paren
            }
            else
                output.Add(token);
        }
        while (ops.Count > 0)
            output.Add(ops.Pop().ToString());
        return string.Join(' ', output);
    }

    public static long EvalPostfix(string postfix)
    {
        var values = new ArrayStack<long>();
        foreach (string token in postfix.Split(' ', StringSplitOptions.RemoveEmptyEntries))
        {
            if (token.Length == 1 && "+-*/".Contains(token[0]))
            {
                long b = values.Pop(), a = values.Pop();
                values.Push(token[0] switch
                {
                    '+' ⇒ a + b,
                    '-' ⇒ a - b,
                    '*' ⇒ a * b,
                    '/' ⇒ a / b,
                    _ ⇒ throw new InvalidOperationException($"bad operator {token[0]}"),
                });
            }
            else
                values.Push(long.Parse(token));
        }
        return values.Pop();
    }
}
```

listing 113: c#, infix to postfix, then postfix evaluation (source: dsa/samples/src/Ch05/Buffers.cs, lines 108-161)

The left associativity rule is where implementations quietly differ: for equal precedence the loop must pop the stacked operator first, so `8 - 3 - 2` groups as `(8 - 3) - 2` and evaluates to 3, which the test pins. Evaluation itself is a second stack pass, operands push, operators pop twice, compute, push. Reverse polish is trivial to evaluate and hard to get wrong, which is why some calculators, hp's classic rpn line, exposed it directly.

Four more yards:

```
export function toPostfix(s) {
  const toks = lex(s);
  const out = [];
  const ops = [];
  for (const t of toks) {
    if (t.kind === "num") out.push(t);
    else if (t.kind === "op") {
      while (
        ops.length &&
        ops[ops.length - 1] =/= "(" &&
        PREC[ops[ops.length - 1]] ≥ PREC[t.op]
      )
        out.push({ kind: "op", op: ops.pop() });
      ops.push(t.op);
    } else if (t.kind === "(") {
      ops.push("(");
    } else {
      while (ops.length && ops[ops.length - 1] =/= "(")
        out.push({ kind: "op", op: ops.pop() });
      ops.pop(); // discard the "("
    }
  }
  while (ops.length) out.push({ kind: "op", op: ops.pop() });
  return out;
}
```

listing 114: javascript, the yard over kind-tagged tokens (source: dsa/samples-js/src/ch05-shunting.mjs, lines 27-51)

```
def to_postfix(expr):
    # single-character tokens: digits and + - * / ( )
    out, ops = [], []
    for ch in expr:
        if ch.isdigit():
            out.append(ch)
        elif ch in PREC:
            # ≥ keeps same-precedence operators left associative
            while ops and ops[-1] in PREC and PREC[ops[-1]] ≥ PREC[ch]:
                out.append(ops.pop())
            ops.append(ch)
        elif ch == "(":
            ops.append(ch)
        elif ch == ")":
            while ops[-1] =/= "(":
                out.append(ops.pop())
            ops.pop()
    while ops:
        out.append(ops.pop())
    return "".join(out)
```

listing 115: python, the yard in single-character tokens (source: dsa/samples-py/src/Ch05/shunting.py, lines 17-36)

```
local function to_postfix(toks)
  local out, ops = {}, {}
  for _, t in ipairs(toks) do
    if t.kind == "num" then
      out[#out + 1] = t
    elseif t.kind == "op" then
      while #ops > 0 and ops[#ops] ~= "(" and prec(ops[#ops]) ≥ prec(t.op) do
        out[#out + 1] = {kind = "op", op = table.remove(ops)}
      end
      ops[#ops + 1] = t.op
    elseif t.kind == "lp" then
      ops[#ops + 1] = "("
    else
      while #ops > 0 and ops[#ops] ~= "(" do
        out[#out + 1] = {kind = "op", op = table.remove(ops)}
      end
      table.remove(ops) -- discard the "("
    end
  end
  while #ops > 0 do
    out[#out + 1] = {kind = "op", op = table.remove(ops)}
  end
  return out
end
```

listing 116: lua, the yard with table.remove as the pop (source: dsa/samples-lua/ch05_shunting.lua, lines 37-60)

The associativity anchor holds in all seven: 8 - 3 - 2 emits 8 3 - 2 - and evaluates to 3. C, Go, and C# pin the rendered string directly, Java, JavaScript, and Lua render through the same helper, Python pins 83-2- in single-character form. Division truncates toward zero in C, Go, Java, C#, and JavaScript; Lua and Python floor, and their fixtures stay positive and exact so the difference never shows. Go's tokenizer is the only one that rejects unexpected characters with an error.

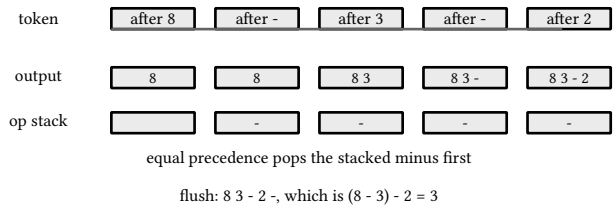


figure 33: shunting yard on 8 - 3 - 2, the equal precedence pop keeps it left associative

5.5. the deque

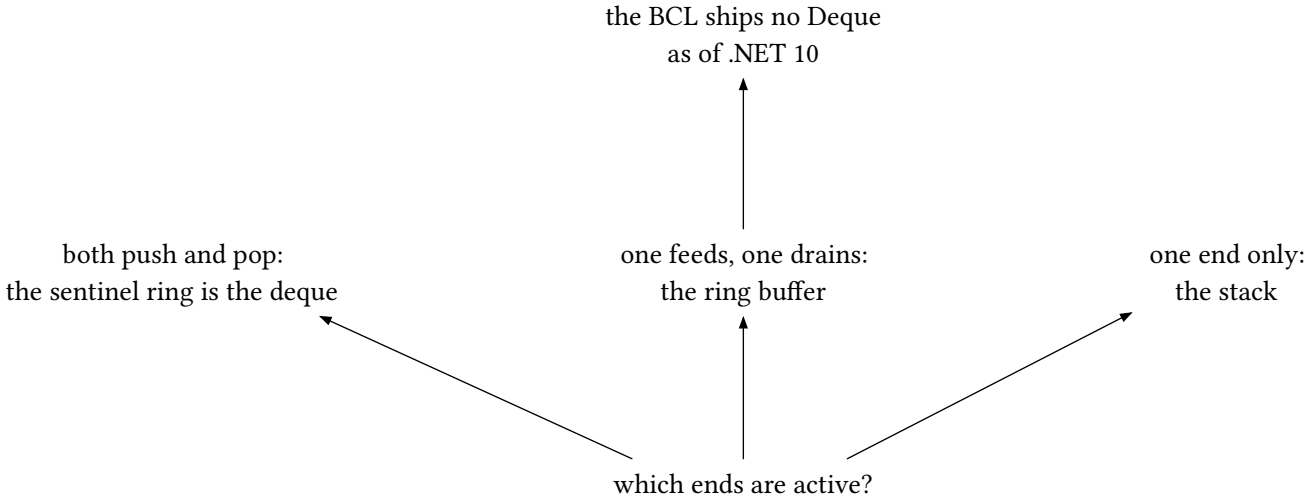


figure 34: the deque selection rule, which ends are active

A deque, double ended queue, is what you get when both ends accept push and pop. Chapter 4's sentinel ring already is one: `AddFirst`, `AddLast`, remove at either end, all constant. Notably the BCL has no general `Deque<T>` as of .NET 10, the generic collections page lists `Queue<T>`, `Stack<T>`, `LinkedList<T>`, and `PriorityQueue` but no deque, so the ring from chapter 4 is also the local answer when both ends are active. The selection rule falls out of this chapter's mechanics: deque when both ends are active, ring when one end feeds and one drains, stack when only one end exists at all.

The dry run: the fixture is RingDeque(8) across its four C# tests, asserted by the C# suite, with C, Java, JavaScript, and Lua running the same sequence over checked slots.

1. PushBack 1, 2, 3 fill slots 0, 1, 2, then PushFront 0 steps the head back to slot 7: the read is 0, 1, 2, 3 at count 4.
2. The rotate step, its own fixture rebuilt from 0,1,2,3, pops the front 0 and pushes it at the back: 1, 2, 3, 0.
3. TryPopBack reads slot 2 and returns 3, TryPopFront reads slot 7 and returns 0: count 2.
4. PushFront 9 wraps the head from slot 0 back to slot 7: the read is 9, 1, 2.
5. The empty deque refuses at both ends, and after PushBack 5 the back pop returns 5, then refuses again at 0.

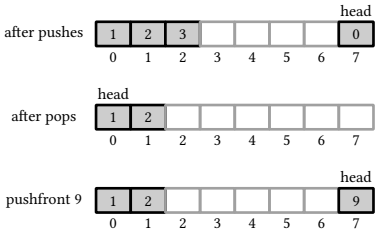


figure 35: the deque run over eight slots, the head stepping backward past slot 0 while the values stay put

The 9, 1, 2 wrap and the both-end refusals are pinned, and the listings below build the deque seven ways.

All seven languages build the ring deque by hand, C first:

```c
static void push_back(int v) {
  slots[(head + count) % CAP] = v;
  count++;
}

static void push_front(int v) {
  head = (head + CAP - 1) % CAP;
  slots[head] = v;
  count++;
}

static int pop_front(int *out) {
  if (count == 0)
    return 0;
  *out = slots[head];
  head = (head + 1) % CAP;
  count--;
  return 1;
}

static int pop_back(int *out) {
  if (count == 0)
    return 0;
  *out = slots[(head + count - 1) % CAP];
  count--;
  return 1;
}
```

listing 117: c, all four end operations over one head and a count (source: dsa/samples-c/src/Ch05/deque.c, lines 23-49)

```go
func (d *Deque) PopBack() (int, error) {
  if d.n == 0 {
    return 0, fmt.Errorf("pop back from empty deque")
  }
  v := d.buf[(d.head+d.n-1)%len(d.buf)]
  d.n--
  return v, nil
}

// Rotate moves the back element to the front, one step of a rotation.
func (d *Deque) Rotate() {
  if d.n < 2 {
    return
  }
  back, _ := d.PopBack()
  d.PushFront(back)
}
```

listing 118: go, pop back and the rotate step (source: dsa/samples-go/ch05/deque.go, lines 61-77)

```
  static final int CAP = 8;

  static int[] slots = new int[CAP];
  static int head, count;

  static void init() {
    head = count = 0;
  }

  static void pushBack(int v) {
    slots[(head + count) % CAP] = v;
    count++;
  }

  static void pushFront(int v) {
    head = (head + CAP - 1) % CAP;
    slots[head] = v;
    count++;
  }

  static Integer popFront() {
    if (count == 0)
      return null;
    int v = slots[head];
    head = (head + 1) % CAP;
    count--;
    return v;
  }

  static Integer popBack() {
    if (count == 0)
      return null;
    int v = slots[(head + count - 1) % CAP];
    count--;
    return v;
  }

  // rotate one step left: front leaves and re-enters at the back
  static void rotateLeft1() {
    pushBack(popFront());
  }
```

listing 119: java, all four end operations over one head and a count, rotate as front to back (source: dsa/samples-java/src/Ch05/Deque.java, lines 18-58)

```
    public void PushBack(int v)
    {
        _slots[(_head + _count) % _slots.Length] = v;
        _count++;
    }

    public void PushFront(int v)
    {
        _head = (_head + _slots.Length - 1) % _slots.Length;
        _slots[_head] = v;
        _count++;
    }

    public bool TryPopFront(out int v)
    {
        if (_count == 0)
        {
            v = 0;
            return false;
        }
        v = _slots[_head];
        _head = (_head + 1) % _slots.Length;
        _count--;
        return true;
    }

    public bool TryPopBack(out int v)
    {
        if (_count == 0)
        {
            v = 0;
            return false;
        }
        v = _slots[(_head + _count - 1) % _slots.Length];
        _count--;
        return true;
    }

    /// <summary>Rotate one step left: the front leaves and re-enters at the back.</summary>
    public void RotateLeftOne()
    {
        if (TryPopFront(out int v))
            PushBack(v);
    }
```

listing 120: c#, both pushes, both pops, and the one-step rotate (source: dsa/samples/src/Ch05/Deque.cs, lines 18-61)

```
  pushBack(v) {
    this.#slots[(this.#head + this.#count) % this.#slots.length] = v;
    this.#count++;
    return this;
  }

  pushFront(v) {
    this.#head = (this.#head + this.#slots.length - 1) % this.#slots.length;
    this.#slots[this.#head] = v;
    this.#count++;
    return this;
  }

  popFront() {
    if (this.#count === 0) return null;
    const v = this.#slots[this.#head];
    this.#head = (this.#head + 1) % this.#slots.length;
    this.#count--;
    return v;
  }

  popBack() {
    if (this.#count === 0) return null;
    const v = this.#slots[(this.#head + this.#count - 1) % this.#slots.length];
    this.#count--;
    return v;
  }
```

listing 121: javascript, both pushes and both pops (source: dsa/samples-js/src/ch05-deque.mjs, lines 17-43)

```
    def push_back(self, value):
        self._make_room()
        self.buf[self._at(self.count)] = value
        self.count += 1

    def push_front(self, value):
        self._make_room()
        self.head = (self.head - 1) % len(self.buf)
        self.buf[self.head] = value
        self.count += 1

    def pop_back(self):
        if self.count == 0:
            raise IndexError("pop_back on empty deque")
        self.count -= 1
        value = self.buf[self._at(self.count)]
        self.buf[self._at(self.count)] = None
        return value

    def pop_front(self):
        if self.count == 0:
            raise IndexError("pop_front on empty deque")
        value = self.buf[self.head]
        self.buf[self.head] = None
        self.head = (self.head + 1) % len(self.buf)
        self.count -= 1
        return value

    def rotate(self, k=1):
        # positive k carries tail items around to the front
        for _ in range(k % self.count if self.count else 0):
            self.push_front(self.pop_back())
```

listing 122: python, both ends, the room check, and rotate (source: dsa/samples-py/src/Ch05/deque.py, lines 32-63)

```lua
local function push_back(q, v)
  q.slots[(q.head + q.count) % CAP + 1] = v
  q.count = q.count + 1
end

local function push_front(q, v)
  q.head = (q.head + CAP - 1) % CAP
  q.slots[q.head + 1] = v
  q.count = q.count + 1
end

local function pop_front(q)
  if q.count == 0 then return nil end
  local v = q.slots[q.head + 1]
  q.slots[q.head + 1] = nil
  q.head = (q.head + 1) % CAP
  q.count = q.count - 1
  return v
end

local function pop_back(q)
  if q.count == 0 then return nil end
  local v = q.slots[(q.head + q.count - 1) % CAP + 1]
  q.slots[(q.head + q.count - 1) % CAP + 1] = nil
  q.count = q.count - 1
  return v
end
```

listing 123: lua, the four operations, modulo over 1-based slots (source: dsa/samples-lua/ch05_deque.lua, lines 11-37)

C, C#, Java, JavaScript, and Lua share one fixture family: 0 1 2 3 after both-end pushes, one rotate step turns it into 1 2 3 0, both ends pop, and an empty deque refuses at both ends. Python rotates the other way, pop back into push front, pins a rotate by the full length as the identity, and watches growth run 2 to 4 to 8 under front pushes. Go's Rotate moves the back to the front, Python's direction, and its ring grows on demand like its queue. The BCL still ships no `Deque<T>`, so C# builds the ring by hand too, chapter 4's sentinel ring in miniature.

5.6. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	384	libc only	four programs over static arrays, bounded stacks refuse at capacity 8, refusal codes instead of exceptions
go	293	fmt, strconv, strings	the tokenizer rejects bad characters with errors, EvalInfix refuses division by zero
java	390	jdk 27 stdlib	ArrayDeque left out, the ring is hand-rolled, pops surface as null where c used a status flag, yard tokens are records evaluated through a switch expression, division truncates toward zero like c
c#	197	bcl only	the frozen yard files plus the ring deque, no bcl deque exists, TryPop out-parameters refuse underflow
javascript	185	node stdlib	Math.trunc keeps division toward zero like c, tokens carry a kind field

language	build SLOC	dependency	boundary note
python	252	stdlib only	single-character tokens keep the yard small, fixtures keep division exact so floor never shows
lua	326	lib.lua harness	1-based slots shift every modulo expression by one

sources: learn.microsoft.com, `Queue<T>` and `Stack<T>` api pages including the circular-array remarks, generic collections in .NET listing the shipped container set, `System.Threading.Channels` overview, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 16 tests in chapter 5 of the samples suite. The seven-language layer verifies the same way: 4 C programs under `make verify-c`, 12 Go tests, the java runner's 62 Ch05 checks over 4 files under `run-java-samples`, 15 `node --test` cases, 42 Python checks across 4 files, and 14 Lua checks under `run.lua`.

6. hashing

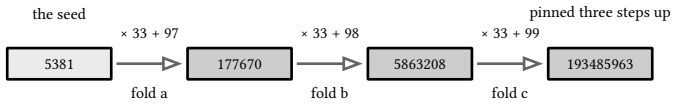
The hash table is the structure the language leans on hardest: `Dictionary<K,V>` backs half of ordinary code. This chapter builds the two families from scratch, open addressing and chaining, around a visible hash function, and prices the trade each one makes.

6.1. the hash function

FNV-1a is small enough to hold in your head and good enough to build on: xor the byte into the accumulator, multiply by the prime, repeat. The constants are definitional, the offset basis and the 1099511628211 prime. C carries both families, FNV-1a and djb2, in 24 lines.

The dry run: the fixtures are the reference vectors and the spread check, asserted by the C# suite, and the djb2 ladder with its arithmetic spelled out, asserted by the C, Java, and JavaScript suites; all seven languages pin the same digests.

1. The empty string hashes to the offset basis itself, cbf29ce484222325, “a” to af63dc4c8601ec8c, “foobar” to 85944171f73967e8.
2. djb2 over “a” is one step: 5381 × 33 + 97 = 177670.
3. “ab” folds b in, 177670 × 33 + 98 = 5863208, and “abc” one more, 5863208 × 33 + 99 = 193485963.
4. The spread check: 1000 sequential int keys over 64 buckets, mean 1000 / 64 = 15.625, and no bucket reaches twice that, 31.25.
5. FNV-1a wraps its accumulator mod 2^64 at every multiply, which is why JavaScript runs it on BigInt while Java’s long wraps exactly like C’s unsigned long long, and djb2 stays small at these lengths and needs no ring yet.



fnv-1a: the same loop shape, xor then multiply, wrapping mod 2^64

its three vectors pin in all seven suites

figure 36: the djb2 ladder over a, ab, abc: one multiply-add per folded byte, every value pinned by the suites

The three vectors and the three ladder values are pinned, and the listings below hash in seven languages.

```c
static unsigned long long fnv1a(const char *s) {
  unsigned long long h = 14695981039346656037ULL;
  while (*s) {
    h ^= (unsigned char)*s++;
    h *= 1099511628211ULL;
  }
  return h;
}

// incremental update: hash "foo" then "bar" must equal hashing "foobar"
static unsigned long long fnv1a_update(unsigned long long h, const char *s) {
  while (*s) {
    h ^= (unsigned char)*s++;
    h *= 1099511628211ULL;
  }
  return h;
}

static unsigned long long djb2(const char *s) {
  unsigned long long h = 5381;
  while (*s)
    h = h * 33 + (unsigned char)*s++;
  return h;
}
```

listing 124: c, fnv-1a, the incremental update, and djb2 (source: dsa/samples-c/src/Ch06/fnv.c, lines 17-40)

```go
func FNV1a64(s string) uint64 {
  h := fnvOffset64
  for i := 0; i < len(s); i++ {
    h ^= uint64(s[i])
    h *= fnvPrime64
  }
  return h
}

// DJB2 is the other classic: multiply the state by 33 (shift 5 and
// add, historically) and add the byte.
func DJB2(s string) uint64 {
  h := uint64(5381)
  for i := 0; i < len(s); i++ {
    h = h*33 + uint64(s[i])
  }
  return h
}
```

listing 125: go, fnv-1a and djb2 over native uint64 (source: dsa/samples-go/ch06/fnv.go, lines 14-31)

The other five languages pin the same digests:

```
  static final long FNV_OFFSET = Long.parseUnsignedLong("14695981039346656037");
  static final long FNV_PRIME = 1099511628211L;

  static long fnv1a(String s) {
    long h = FNV_OFFSET;
    for (int i = 0; i < s.length(); i++) {
      h ^= s.charAt(i);
      h *= FNV_PRIME;
    }
    return h;
  }

  // incremental update: hash "foo" then "bar" must equal hashing "foobar"
  static long fnv1aUpdate(long h, String s) {
    for (int i = 0; i < s.length(); i++) {
      h ^= s.charAt(i);
      h *= FNV_PRIME;
    }
    return h;
  }

  static long djb2(String s) {
    long h = 5381;
    for (int i = 0; i < s.length(); i++)
      h = h * 33 + s.charAt(i);
    return h;
  }
```

listing 126: java, the basis rides parseUnsignedLong, fnv-1a, the incremental update, djb2 (source: dsa/samples-java/src/Ch06/Fnv.java, lines 21-47)

The C# class is generic over the input shape:

```c#
public static class Fnv1a
{
    public const ulong OffsetBasis = 0xcbf29ce484222325UL;
    public const ulong Prime = 0x100000001b3UL;

    public static ulong Of(ReadOnlySpan<byte> bytes)
    {
        ulong hash = OffsetBasis;
        foreach (byte b in bytes)
        {
            hash ^= b;
            hash *= Prime;
        }
        return hash;
    }

    public static ulong Of(string text) =>
        Of(System.Text.Encoding.UTF8.GetBytes(text));

    // numeric keys hash over their fixed width little endian bytes,
    // the same trick a storage engine uses for binary keys
    public static ulong Of(int value)
    {
        Span<byte> bytes = stackalloc byte[4];
        System.Buffers.Binary.BinaryPrimitives.WriteInt32LittleEndian(bytes, value);
        return Of(bytes);
    }
}
```

listing 127: c#, fnv 1a over bytes, strings, and fixed width ints (source: dsa/samples/src/Ch06/Hashing.cs, lines 4-32)

The int overload hashes the four little endian bytes of the value, which is how binary keys must be handled, hash the bytes not the decimal spelling. The test pins the published reference vectors, the empty string hashes to the offset basis itself, and checks the spread: 1000 sequential ints into 64 buckets with no bucket beyond twice the mean.

```javascript
const FNV_OFFSET = 0xcbf29ce484222325n;
const FNV_PRIME = 0x100000001b3n;
const MASK64 = 0xffffffffffffffffn;

export function fnv1a64(s) {
  let h = FNV_OFFSET;
  for (const c of s) {
    h ^= BigInt(c.charCodeAt(0));
    h = BigInt.asUintN(64, h * FNV_PRIME);
  }
  return h;
}
```

listing 128: javascript, fnv-1a on BigInt with a 64-bit ring (source: dsa/samples-js/src/ch06-fnv.mjs, lines 7-18)

```python
def fnv1a(text):
    h = FNV_OFFSET
    for byte in text.encode():
        h ^= byte  # xor the byte in first: that is the -1a twist
        h = (h * FNV_PRIME) & MASK
    return h


def fnv1a_round(h, ch):
    # one byte of the loop, exposed so composition can be probed
    h ^= ord(ch)
    return (h * FNV_PRIME) & MASK


def djb2(text):
    h = 5381
    for ch in text:
        h = (h * 33 + ord(ch)) & MASK
    return h
```

listing 129: python, fnv-1a with an explicit 64-bit mask, djb2 beside it (source: dsa/samples-py/src/Ch06/fnv.py, lines 18-36)

```lua
local function hex64(s)
  local acc = 0
  for i = 1, #s do
    local d = tonumber(s:sub(i, i), 16)
    acc = acc * 16 + d
  end
  return acc
end

-- FNV-1a 64: xor the byte, then multiply by the prime, wrapping mod 2^64
local function fnv1a(s, h)
  h = h or hex64("cbf29ce484222325") -- the offset basis
  for i = 1, #s do
    h = h ~ s:byte(i)
    h = h * 0x100000001b3 -- the 64-bit prime
  end
  return h
end

-- djb2: h = h*33 + c, small fixtures never wrap
local function djb2(s)
  local h = 5381
  for i = 1, #s do
    h = h * 33 + s:byte(i)
  end
  return h
end
```

listing 130: lua, a hex parser feeding the wrapped accumulator (source: dsa/samples-lua/ch06_fnv.lua, lines 8-34)

The reference vectors hold in all seven: the empty string hashes to the offset basis, “a” to af63dc4c8601ec8c, “foobar” to 85944171f73967e8, and djb2 climbs 5381, 177670, 5863208, 193485963. How each language survives the multiply is the real story. C’s unsigned long long wraps silently and the vectors fit anyway. Go’s uint64 multiply is native. Java’s long multiplies wrap mod 2^64 the same way, and only the decimal offset basis 14695981039346656037 exceeds `Long.MAX_VALUE`, so it rides `Long.parseUnsignedLong` in and `Long.toUnsignedString` back

out, the suite's one unsigned-law site, no ordering of two hashes ever needs `compareUnsigned`. Python masks with `(1 << 64) - 1` after every step. Lua's integers are signed 64-bit, so the pinned values arrive through a hex parser and foobar reads negative once its top bit sets. JavaScript is where Number stops: past 2^53 the double loses low bits, so fnv runs on BigInt with `asUintN(64)` as the ring, the first appearance of a boundary chapter 16 returns to.

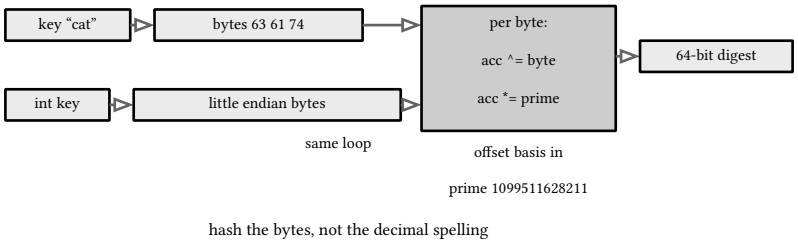


figure 37: fnv 1a, xor the byte, multiply by the prime, offset basis in, 64 bit digest out

NOTE: what object.GetHashCode gives you

The runtime hash for strings is randomized per process, `BitConverter.ToString` of it differs between runs, which protects against hash flooding and means you must never persist it. Ints hash to themselves, so sequential int keys collide only modulo capacity, exactly what the probe tests below exercise. A production table under adversarial keys needs a keyed hash, .NET's `System.IO.Hashing.XxHash3` for speed or an SipHash style for resistance.

6.2. open addressing

One array, entries live in it, collisions walk to the next slot. The linear probe is the simplest walk, and the table counts every probe so the cost claims stay testable. C keeps the two anchored pieces small, the rehash that purges tombstones and the probe-counted get.

The dry run: the fixture is the djb2 keys "a" through "i" over a capacity 8 table, traced by the C, Java, JavaScript, and Lua suites; C# counts probes over 4096 int keys instead and Python walks ordinals.

1. djb2 of one character is 5381 × 33 + code, so "a" hashes to 177670, and mod 8 the seven keys home to 6, 7, 0, 1, 2, 3, 4: one probe each, no collision yet.
2. The seventh insert crosses the load line, used 7 > 8 × 3 / 4 = 6, and the rehash doubles the table to 16 slots.
3. Homes mod 16 shift to 6 through 12: the block re-lands contiguous, still one probe per key.
4. Deleting "b" parks a tombstone at slot 7, and proving "b" absent walks the whole block, 7 probes, to the empty slot 13.
5. "h" homes to 13 and "i" to 14, one probe each, and every survivor still answers.

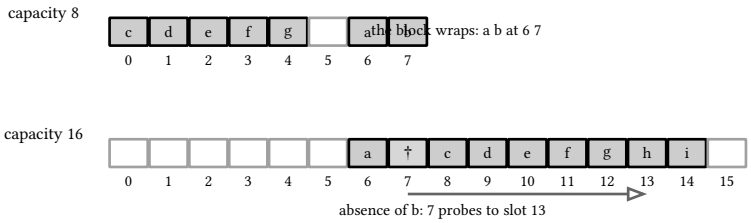


figure 38: the table at both sizes: the block already wrapped at capacity 8, then rebased to 6 through 12 at 16 with the tombstone and the absence walk

The 7 probe absence walk behind the tombstone is the pin that keeps deletion honest, and the listings below build the table in seven languages.

```c
static void rehash(void) {
  Slot old[32];
  int oldcap = cap, oldused = used;
  for (int i = 0; i < oldcap; i++)
    old[i] = tab[i];
  cap *= 2;
  used = tombs = 0;
  for (int i = 0; i < cap; i++)
    tab[i].state = EMPTY;
  for (int i = 0; i < oldcap; i++)
    if (old[i].state == USED)
      raw_put(old[i].key, old[i].val);
  used = oldused;
  rehashes++;
}

// linear probe get: returns 1 and fills val/probes/slot on a hit
static int map_get(const char *k, int *val, int *probes, int *slot) {
  int i = (int)(djb2(k) % (unsigned long long)cap);
  *probes = 0;
  for (;;) {
    (*probes)++;
    if (tab[i].state == EMPTY)
      return 0;
    if (tab[i].state == USED && strcmp(tab[i].key, k) == 0) {
      *val = tab[i].val;
      *slot = i;
      return 1;
    }
    i = (i + 1) % cap;
  }
}
```

listing 131: c, the rehash rebases live keys, get counts its probes (source: dsa/samples-c/src/Ch06/openaddr.c, lines 61-92)

```
func (m *OpenMap) Put(k string, v int) {
  i := int(m.hash(k)) & (len(m.keys) - 1)
  firstTomb := -1
  for {
    switch m.state[i] {
    case slotUsed:
      if m.keys[i] == k {
        m.vals[i] = v
        return
      }
    case slotTombstone:
      if firstTomb < 0 {
        firstTomb = i
      }
    case slotEmpty:
      at := i
      if firstTomb ≥ 0 {
        at = firstTomb // bury the key in the tombstone
      }
      m.keys[at], m.vals[at], m.state[at] = k, v, slotUsed
      if firstTomb ≥ 0 {
        m.tombs--
      }
      m.n++
      m.maybeRehash()
      return
    }
    i = (i + 1) & (len(m.keys) - 1)
  }
}
```

listing 132: go, put reuses the first tombstone and rehashes at three quarters (source: dsa/samples-go/ch06/openaddr.go, lines 61-90)

The other five builders, Java handing the probe count back inside a lookup record:

```
  // rehash drops tombstones and doubles capacity
  static void rehash() {
    Slot[] old = new Slot[32];
    for (int i = 0; i < cap; i++)
      old[i] = tab[i];
    int oldUsed = used;
    cap *= 2;
    used = tombs = 0;
    for (int i = 0; i < cap; i++)
      tab[i] = new Slot();
    for (int i = 0; i < cap / 2; i++)
      if (old[i].state == USED)
        rawPut(old[i].key, old[i].val);
    used = oldUsed;
    rehashes++;
  }

  record Lookup(boolean hit, int val, int probes, int slot) {}

  // linear probe get: hit with val/probes/slot, or miss with the probe
  // count spent
  static Lookup mapGet(String k) {
    int i = (int) (djb2(k) % cap);
    int probes = 0;
    for (;;) {
      probes++;
      if (tab[i].state == EMPTY)
        return new Lookup(false, 0, probes, -1);
      if (tab[i].state == USED && tab[i].key.equals(k))
        return new Lookup(true, tab[i].val, probes, i);
      i = (i + 1) % cap;
    }
  }
```

listing 133: java, the rehash rebases live keys, the lookup record carrying probes and slot (source: dsa/samples-java/src/Ch06/Openaddr.java, lines 53-85)

```
public sealed class OpenAddressing<K, V> where K : notnull
{
    private const byte Empty = 0, Used = 1, Dead = 2;

    private K[] _keys = new K[8];
    private V?[] _values = new V?[8];
    private byte[] _states = new byte[8];
    private readonly IEqualityComparer<K> _cmp = EqualityComparer<K>.Default;

    public int Count { get; private set; }
    public int Capacity => _keys.Length;

    /// <summary>Total probes across all operations, the cost meter.</summary>
    public long TotalProbes { get; private set; }

    public V this[K key]
    {
        get
        {
            int i = Find(key);
```

```
            if (i < 0)
                throw new KeyNotFoundException($"no key {key}");
            return _values[i]!;
        }
        set
        {
            int i = Find(key);
            if (i ≥ 0)
            {
                _values[i] = value;
                return;
            }
            Insert(key, value);
        }
    }

    public bool TryGet(K key, out V value)
    {
        int i = Find(key);
        if (i < 0)
        {
            value = default!;
            return false;
        }
        value = _values[i]!;
        return true;
    }

    public bool Remove(K key)
    {
        int i = Find(key);
        if (i < 0)
            return false;
        _states[i] = Dead; // tombstone: the probe chain must keep walking
        _keys[i] = default!;
        _values[i] = default;
        Count--;
        return true;
    }

    public bool ContainsKey(K key) => Find(key) ≥ 0;

    private int Find(K key)
    {
        int mask = _keys.Length - 1;
        int i = (int)(uint)_cmp.GetHashCode(key) & mask;
        for (int step = 0; step < _keys.Length; step++)
        {
            TotalProbes++;
            byte state = _states[i];
            if (state == Empty)
                return -1; // an empty slot ends every chain
            if (state == Used && _cmp.Equals(_keys[i], key))
                return i;
            i = (i + 1) & mask;
        }
```

```
        return -1;
    }
```

listing 134: c#, linear probing with states, probe counting, the finder (source: dsa/samples/src/Ch06/Hashing.cs, lines 34-112)

Deletion is the subtle part. Emptying a slot would break the probe chain of anything inserted past it, so removal writes a tombstone, a dead marker the finder walks through but the inserter reuses. The test kills a middle key and checks both neighbors stay reachable. Load is kept under 0.7 counting tombstones, because probe cost climbs steeply as the table fills, and the rehash rebuilds from live entries only, so tombstones vanish on resize:

```
    private void Insert(K key, V value)
    {
        if ((Count + 1) * 10 ≥ _keys.Length * 7) // 0.7 load, tombstones included
            Rehash(_keys.Length * 2);
        int mask = _keys.Length - 1;
        int i = (int)(uint)_cmp.GetHashCode(key) & mask;
        while (_states[i] == Used)
        {
            TotalProbes++;
            i = (i + 1) & mask;
        }
        TotalProbes++;
        _states[i] = Used;
        _keys[i] = key;
        _values[i] = value;
        Count++;
    }

    private void Rehash(int size)
    {
        var oldKeys = _keys;
        var oldValues = _values;
        var oldStates = _states;
        _keys = new K[size];
        _values = new V?[size];
        _states = new byte[size];
        Count = 0; // Insert increments it back
        for (int i = 0; i < oldKeys.Length; i++)
        {
            if (oldStates[i] == Used)
                Insert(oldKeys[i], oldValues[i]!);
        }
        // dead slots simply vanish
    }
}
```

listing 135: insert with load check, rehash purges tombstones (source: dsa/samples/src/Ch06/Hashing.cs, lines 113-148)

The probe meter backs the constant-time claim with a number: 4096 sequential int keys at 0.7 load average under 10 probes per operation in the test, far from the worst case and stable. The runtime `Dictionary` works differently: separate chaining, an int bucket array holding the first index of a chain threaded through one flat entries array by `next` links, bucket picked by a modulo of the hash, no probe walk anywhere in its source. Open addressing here is the deliberate choice, a table whose collision cost is countable.

Three more builders:

```
  put(key, v) {
    let i = this.#slotOf(key);
    let firstTomb = -1;
    while (this.#state[i] !== EMPTY) {
      if (this.#state[i] === TOMB && firstTomb < 0) firstTomb = i;
      if (this.#state[i] === USED && this.#keys[i] === key) {
        this.#vals[i] = v;
        return this;
      }
      i = (i + 1) % this.#cap;
    }
    if (firstTomb ≥ 0) {
      this.#place(firstTomb, key, v);
      this.#tombs--;
    } else {
      this.#place(i, key, v);
    }
    this.#used++;
    if (this.#used + this.#tombs > Math.floor((this.#cap * 3) / 4))
      this.#rehash();
    return this;
  }
```

listing 136: javascript, put with tombstone reuse and the load check (source: dsa/samples-js/src/ch06-openaddr.mjs, lines 89-110)

```
    def put(self, key, value):
        if (self.live + self.tombs) * 10 > self.cap * 7:
            self._rehash()  # its internal puts do not count as our probes
        self.last_probes = 0
        i = self._home(key)
        while True:
            self.last_probes += 1
            slot = self.slots[i]
            if slot is None:
                self.slots[i] = (key, value)
                self.live += 1
                return
            if slot is TOMB:
                self.tombs -= 1
                self.slots[i] = (key, value)
                self.live += 1
                return
            if slot[0] == key:
                self.slots[i] = (key, value)
                return
            i = (i + 1) % self.cap
```

listing 137: python, put counts its own probes through tombstones (source: dsa/samples-py/src/Ch06/openaddr.py, lines 28-48)

```lua
local function map_put(m, k, v)
  local i = djb2(k) % m.cap
  local first_tomb = nil
  while m.state[i] ~= "empty" do
    if m.state[i] == "tomb" and first_tomb == nil then first_tomb = i end
    if m.state[i] == "used" and m.key[i] == k then
      m.val[i] = v
      return
    end
    i = (i + 1) % m.cap
  end
  local slot = first_tomb or i
  m.key[slot], m.val[slot] = k, v
  m.state[slot] = "used"
  m.used = m.used + 1
  if first_tomb then m.tombs = m.tombs - 1 end
  if m.used + m.tombs > m.cap * 3 // 4 then rehash(m) end
end
```

listing 138: lua, put over string states, rehash past three quarters (source: dsa/samples-lua/ch06_openaddr.lua, lines 52-69)

C, Java, JavaScript, and Lua trace the same fixture: djb2 lands “a” through “g” in one contiguous block, every insert costs one probe, the seventh crosses the load line and rehashes to 16 slots, the block re-lands at slots 6 through 12, deleting “b” leaves a tombstone that costs 7 probes to prove absence, and “h” and “i” settle one past the block at 13 and 14. Python homes by ordinal sum and pins its own walks, “i” resting at slot 4 after four probes, a tombstone recycled by the next put, the 6-in-8 edge doubling to 16. Go injects the hash so its tests can force collisions, and masks instead of dividing. C# counts every probe across the run, 4096 int keys averaging under 10 probes per operation at 0.7 load.

6.3. separate chaining

Buckets of linked nodes instead of walking the array. Deletion is trivial, no tombstones exist, and the worst case is a bucket that grew long rather than a table-wide stall. C appends at the bucket tail and counts compares on the walk.

The dry run: the fixture is the crafted key set “a” through “i”, traced by the C, Java, JavaScript, and Lua suites; C# reads the chain gauge over 102 keys plus the adversarial constant hash, and Python buckets by key length.

1. djb2 mod 8 sends “a” through “h” to 6, 7, 0, 1, 2, 3, 4, 5, and “i” wraps back to 6, behind “a”.
2. Nine puts leave the bucket lengths reading 1 1 1 1 1 1 2 1: one bucket of two, none empty.
3. The walk to “i” compares two nodes, “a” first, against one compare for “a” itself.
4. Deleting the chain head “a” unsplices locally: bucket 6 holds “i” alone and the walk drops to one compare.
5. The C# gauge holds the longest chain at or under 8 over 102 keys, and the constant-hash keys collapse into one chain of 5, correct but linear.

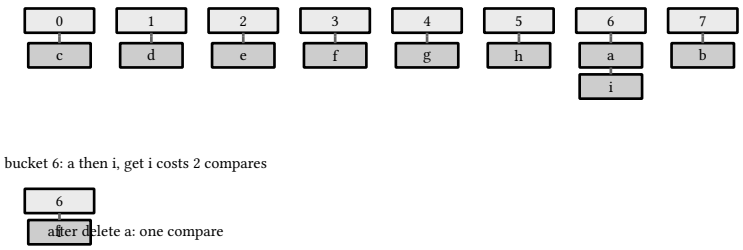


figure 39: the nine keys over eight buckets, “i” chaining behind “a” at bucket 6, and the head delete dropping the walk

The length vector and the two-into-one compare drop are pinned, and the listings below chain in seven languages.

```c
static void map_put(const char *k, int v) {
  CNode *p = &pool[pool_n++];
  copy_str(p->key, k);
  p->val = v;
  p->next = NULL;
  int b = bucket_of(k);
  if (!buckets[b]) {
    buckets[b] = p;
    return;
  }
  CNode *t = buckets[b];
  while (t->next)
    t = t->next;
  t->next = p;
}

// returns compare count: how many nodes the chain walked
static int map_get(const char *k, int *val) {
  int cmp = 0;
  for (CNode *p = buckets[bucket_of(k)]; p; p = p->next) {
    cmp++;
    if (strcmp(p->key, k) == 0) {
      *val = p->val;
      return cmp;
    }
  }
  return -cmp;
}
```

listing 139: c, tail append, the compare-counted get (source: dsa/samples-c/src/Ch06/chainmap.c, lines 50-77)

```go
func (m *ChainMap) Put(k string, v int) {
  b := &m.buckets[m.bucketOf(k)]
  for i := range *b {
    if (*b)[i].key == k {
      (*b)[i].val = v
      return
    }
  }
  *b = append(*b, chainPair{k, v})
}
```

listing 140: go, chains as slices, overwrite in place (source: dsa/samples-go/ch06/chainmap.go, lines 27-36)

The same interface in the other five languages, Java's get returning the compare count, negative on a miss:

```
  static void mapPut(String k, int v) {
    CNode p = new CNode();
    p.key = k;
    p.val = v;
    pool[poolN++] = p;
    int b = bucketOf(k);
    if (buckets[b] == null) {
      buckets[b] = p;
      return;
    }
    CNode t = buckets[b];
    while (t.next != null)
      t = t.next;
    t.next = p;
  }

  // returns compare count: how many nodes the chain walked, negative on a
  // miss with the walk count carried
  static int mapGet(String k, int[] val) {
    int cmp = 0;
    for (CNode p = buckets[bucketOf(k)]; p != null; p = p.next) {
      cmp++;
      if (p.key.equals(k)) {
        val[0] = p.val;
        return cmp;
      }
    }
    return -cmp;
  }
```

listing 141: java, tail append over pooled nodes, the get that returns the compare count (source: dsa/samples-java/src/Ch06/Chainmap.java, lines 44-72)

```
public sealed class Chaining<K, V> where K : notnull
{
    private sealed class Node(K key, V value)
    {
        public K Key = key;
        public V Value = value;
        public Node? Next;
    }

    private Node?[] _buckets = new Node?[8];
    private readonly IEqualityComparer<K> _cmp = EqualityComparer<K>.Default;

    public int Count { get; private set; }
    public int BucketCount => _buckets.Length;

    public int LongestChain
    {
        get
        {
            int longest = 0;
            foreach (Node? head in _buckets)
            {
                int len = 0;
                for (Node? n = head; n != null; n = n.Next)
```

```
                len++;
            longest = Math.Max(longest, len);
        }
        return longest;
    }
}

public V this[K key]
{
    get
    {
        int b = Bucket(key);
        for (Node? n = _buckets[b]; n ≠ null; n = n.Next)
            if (_cmp.Equals(n.Key, key))
                return n.Value;
        throw new KeyNotFoundException($"no key {key}");
    }
    set
    {
        int b = Bucket(key);
        for (Node? n = _buckets[b]; n ≠ null; n = n.Next)
        {
            if (_cmp.Equals(n.Key, key))
            {
                n.Value = value;
                return;
            }
        }
        _buckets[b] = new Node(key, value) { Next = _buckets[b] };
        Count++;
        if (Count > _buckets.Length)
            Resize(_buckets.Length * 2);
    }
}

public bool TryGet(K key, out V value)
{
    int b = Bucket(key);
    for (Node? n = _buckets[b]; n ≠ null; n = n.Next)
    {
        if (_cmp.Equals(n.Key, key))
        {
            value = n.Value;
            return true;
        }
    }
    value = default!;
    return false;
}

private int Bucket(K key) ⇒
    (int)(uint)_cmp.GetHashCode(key) & (_buckets.Length - 1);

private void Resize(int size)
{
    var old = _buckets;
    _buckets = new Node?[size];
```

```
        foreach (Node? head in old)
        {
            for (Node? n = head; n != null;)
            {
                Node? next = n.Next;
                int b = Bucket(n.Key);
                n.Next = _buckets[b];
                _buckets[b] = n;
                n = next;
            }
        }
    }
}

/// <summary>Spread checks: a hash is only as good as its distribution.</summary>
```

listing 142: c#, chaining with longest chain stat, resize at load 1 (source: dsa/samples/src/Ch06/Hashing.cs, lines 150-245)

The `LongestChain` property is the health gauge, the test inserts 102 keys and asserts no chain passed 8. The adversarial test goes further: a key type whose `GetHashCode` returns a constant forces every key into one chain, correctness survives, five keys, chain of five, and the structure degrades into an unsorted list. That is the hash table's contract: average constant, worst case linear, and the worst case is one bad hash away.

```
  put(key, v) {
    this.#chains[bucketOf(key, this.#buckets)].push({ key, val: v });
    return this;
  }

  // compare count: how many nodes the chain walked, negated on a miss
  get(key) {
    let cmp = 0;
    for (const node of this.#chains[bucketOf(key, this.#buckets)]) {
      cmp++;
      if (node.key === key) return { hit: true, value: node.val, compares: cmp };
    }
    return { hit: false, value: undefined, compares: -cmp };
  }
```

listing 143: javascript, array chains with a compare-counted get (source: dsa/samples-js/src/ch06-chainmap.mjs, lines 16-29)

```python
class ChainMapTable:
    def __init__(self, cap=4):
        self.buckets = [[] for _ in range(cap)]

    def _bucket(self, key):
        return self.buckets[len(key) % len(self.buckets)]  # length hash

    def put(self, key, value):
        chain = self._bucket(key)
        for i, (k, _) in enumerate(chain):
            if k == key:
                chain[i] = (key, value)
                return
        chain.append((key, value))

    def get(self, key, default=None):
        for k, v in self._bucket(key):
            if k == key:
                return v
        return default
```

listing 144: python, bucket by key length, the honest worst case on display (source: dsa/samples-py/src/Ch06/chainmap.py, lines 13-32)

```lua
local function map_put(m, k, v)
  local b = m[bucket_of(k)]
  b[#b + 1] = {key = k, val = v}
end

-- returns val and the compare count, or nil and -walked
local function map_get(m, k)
  local cmp = 0
  for _, node in ipairs(m[bucket_of(k)]) do
    cmp = cmp + 1
    if node.key == k then return node.val, cmp end
  end
  return nil, -cmp
end
```

listing 145: lua, tail append and the counted walk (source: dsa/samples-lua/ch06_chainmap.lua, lines 25-38)

C, Java, JavaScript, and Lua share the crafted key set, “a” through “i”, whose djb2 values are consecutive mod 8, so “i” chains behind “a” in bucket 6, the length vector reads 1 1 1 1 1 1 2 1, and deleting the chain head drops “i” from two compares to one. Python hashes by key length, pins cat and dog sharing bucket 3 against bird alone, then piles four five-letter keys into one bucket and watches every lookup still answer. Go keeps buckets as slices and reports the maximum chain. C# carries the adversarial case the others skip, a constant `GetHashCode` forcing one chain of five.

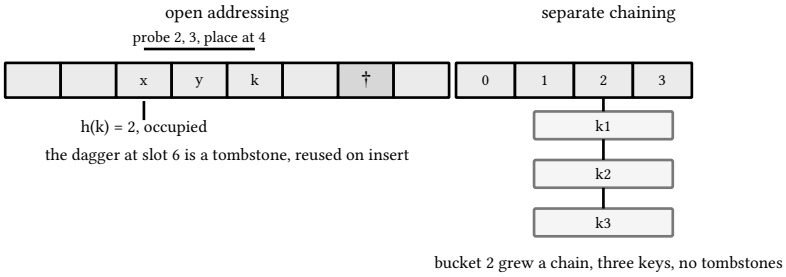


figure 40: the two collision families, walk the array or hang a chain

WARNING: the contract on the key

A mutable key that changes after insertion is lost, its bucket is where the old hash says, not the new one. .NET documents this for `Dictionary`: do not mutate a key while it is in the table. The pairing rule is equally binding, equal keys must return equal hashes, so `Equals` and `GetHashCode` must be overridden together or never, which is why records derive both and hand-written overrides get both.

6.4. the hash set

A set is the map with the value column deleted, membership only. All seven build it on the open addressing core, C first with the Knuth multiplicative mix.

The dry run: the fixture is the integer sets 1 2 3 4 against 3 4 5 6, asserted by the C# suite and pinned identically in C, Java, JavaScript, and Lua; Go and Python run string sets instead.

1. Set a adds 1, 2, 3, 4: four members. Set b adds 3, 4, 5, 6: four members.
2. Membership gates both ways: a holds 1 and refuses 5 and 0, b refuses 1 and holds 6.
3. Re-adding 3 to a leaves the count at 4: the probe found it live and stopped.
4. Union adds every member of both sides, 4 + 4 = 8 adds landing 6 distinct, 1 through 6.
5. Intersect keeps only the overlap, 3 and 4, and the disjoint probe against {10} lands empty, count 0.

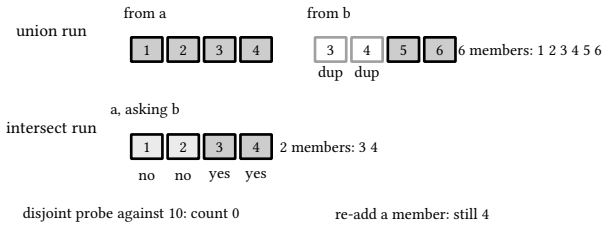


figure 41: the two set operations as runs, union swallowing the duplicates it re-adds, intersect asking the other side member by member

The 6 member union and the 3, 4 intersect are pinned, and the listings below build the set seven ways.

```c
static void set_add(Set *s, int k) {
  int i = (int)(mix(k) % CAP);
  while (s->state[i] && s->slots[i] != k)
    i = (i + 1) % CAP;
  if (!s->state[i]) {
    s->slots[i] = k;
    s->state[i] = 1;
    s->used++;
  }
}

static int set_has(const Set *s, int k) {
  int i = (int)(mix(k) % CAP);
  while (s->state[i]) {
    if (s->slots[i] == k)
      return 1;
    i = (i + 1) % CAP;
  }
  return 0;
}
```

listing 146: c, add and contains over the multiply-probe core (source: dsa/samples-c/src/Ch06/hashset.c, lines 35-54)

```
func Union(a, b *Set) *Set {
  out := NewSet(a.m.Capacity()+b.m.Capacity(), a.m.hash)
  for _, k := range a.Keys() {
    out.Add(k)
  }
  for _, k := range b.Keys() {
    out.Add(k)
  }
  return out
}

// Intersect returns the members of both a and b.
func Intersect(a, b *Set) *Set {
  out := NewSet(a.m.Capacity(), a.m.hash)
  for _, k := range a.Keys() {
    if b.Contains(k) {
      out.Add(k)
    }
  }
  return out
}
```

listing 147: go, union and intersect over the shared open map (source: dsa/samples-go/ch06/hashset.go, lines 46-66)

```
  // knuth multiplicative mix over the low 32 bits, unsigned
  static long mix(int k) {
    return (k & 0xFFFFFFFFL) * 2654435761L;
  }

  static void setInit(Set s) {
    s.used = 0;
    Arrays.fill(s.state, 0);
  }

  static void setAdd(Set s, int k) {
    int i = (int) (mix(k) % CAP);
    while (s.state[i] != 0 && s.slots[i] != k)
      i = (i + 1) % CAP;
    if (s.state[i] == 0) {
      s.slots[i] = k;
      s.state[i] = 1;
      s.used++;
    }
  }

  static boolean setHas(Set s, int k) {
    int i = (int) (mix(k) % CAP);
    while (s.state[i] != 0) {
      if (s.slots[i] == k)
        return true;
      i = (i + 1) % CAP;
    }
    return false;
  }
```

listing 148: java, the knuth mix over the masked low 32 bits, add and contains (source: dsa/samples-java/src/Ch06/Hashset.java, lines 25-54)

```
    public void Add(int k)
    {
        int i = Mix(k);
        while (_used[i] && _slots[i] != k)
            i = (i + 1) % Cap;
        if (!_used[i])
        {
            _slots[i] = k;
            _used[i] = true;
            Count++;
        }
    }

    public bool Contains(int k)
    {
        int i = Mix(k);
        while (_used[i])
        {
            if (_slots[i] == k)
                return true;
            i = (i + 1) % Cap;
        }
        return false;
    }

    /// <summary>Union is add-everything from both sides.</summary>
    public static OpenHashSet Union(OpenHashSet a, OpenHashSet b)
    {
        var out_ = new OpenHashSet();
        foreach (int k in a.Values())
            out_.Add(k);
        foreach (int k in b.Values())
            out_.Add(k);
        return out_;
    }

    /// <summary>Intersect is add only what the other set already holds.</summary>
    public static OpenHashSet Intersect(OpenHashSet a, OpenHashSet b)
    {
        var out_ = new OpenHashSet();
        foreach (int k in a.Values())
            if (b.Contains(k))
                out_.Add(k);
        return out_;
    }
```

listing 149: c#, add and contains over the same mix, union and intersect on top (source: dsa/samples/src/Ch06/HashSet.cs, lines 17-61)

```
  add(k) {
    let i = mix(k) % CAP;
    while (this.#state[i] && this.#slots[i] !== k) i = (i + 1) % CAP;
    if (!this.#state[i]) {
      this.#slots[i] = k;
      this.#state[i] = 1;
      this.#used++;
    }
    return this;
  }

  has(k) {
    let i = mix(k) % CAP;
    while (this.#state[i]) {
      if (this.#slots[i] === k) return true;
      i = (i + 1) % CAP;
    }
    return false;
  }

  // sorted dump for pinning
  values() {
    const out = [];
    for (let i = 0; i < CAP; i++) if (this.#state[i]) out.push(this.#slots[i]);
    return out.sort((a, b) => a - b);
  }
```

listing 150: javascript, add, has, and the sorted dump (source: dsa/samples-js/src/ch06-hashset.mjs, lines 22-47)

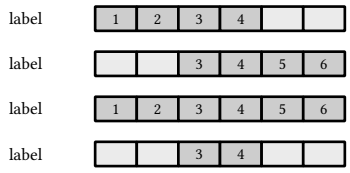
```
    def add(self, key):
        i = self._home(key)
        while self.slots[i] is not None:
            if self.slots[i] == key:
                return False  # already a member
            i = (i + 1) % self.cap
        self.slots[i] = key
        self.count += 1
        return True

    def contains(self, key):
        i = self._home(key)
        while self.slots[i] is not None:
            if self.slots[i] == key:
                return True
            i = (i + 1) % self.cap
        return False

    def values(self):
        return sorted(s for s in self.slots if s is not None)
```

listing 151: python, add, contains, values sorted for pinning (source: dsa/samples-py/src/Ch06/hashset.py, lines 22-41)

```lua
local function set_add(s, k)
  local i = mix(k)
  while s.state[i] == "used" and s.slots[i] ~= k do i = (i + 1) % CAP end
  if s.state[i] ~= "used" then
    s.slots[i], s.state[i] = k, "used"
    s.used = s.used + 1
  end
end

local function set_has(s, k)
  local i = mix(k)
  while s.state[i] == "used" do
    if s.slots[i] == k then return true end
    i = (i + 1) % CAP
  end
  return false
end
```

listing 152: lua, add and has over string states (source: dsa/samples-lua/ch06_hashset.lua, lines 15-31)

The BCL's own HashSet<T> stays the production answer, and the C# suite builds the same core by hand beside its map families. The seven hand builds carry the point, one probe loop with one comparison per slot, union is add-everything while intersect is add-if-the-other-has-it. The integer fixtures pin identically in C, C#, Java, JavaScript, and Lua: 1 2 3 4 against 3 4 5 6 unions to 1 through 6 and intersects to 3 4, a disjoint pair intersects to empty, and re-adding a member is a no-op. Python and Go run string sets, a b c against b c x, union a b c x, intersect b c, Go sorting keys before compare so order never depends on the table. JavaScript's mix keeps the product under 2^53 with a comment at the multiply.



union adds everything, intersect adds only what the other set already holds

figure 42: two sets over the same probe core, union keeps every member, intersect keeps the overlap

6.5. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	419	libc, qsort for dumps	unsigned long long wraps silently and the pinned vectors fit anyway, string keys live in fixed 8-byte slot fields
go	228	slices for sorting	the map's hash is injected, tests force collisions on purpose
java	430	jdk 27 stdlib	long wraps mod 2^64 like c, the offset basis 14695981039346656037 rides parseUnsignedLong and renders via toUnsignedString, no compareUnsigned anywhere because no two hashes are ever ordered
c#	283	bcl only	int keys hash their little endian bytes, the int-key spread check, the open set rides the same knuth mix
javascript	200	node stdlib	fnv runs on BigInt with asUintN(64), the book's first 2^53 boundary

language	build SLOC	dependency	boundary note
python	264	stdlib only	a sentinel tombstone object marks deleted slots, identity-checked
lua	345	lib.lua harness	signed int64 wrap reads negative once the top bit sets, pinned vectors arrive as hex strings

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` remarks that it is a hash table whose retrieval speed tracks hash quality, `GetHashCode` guidelines, `Object.GetHashCode` remarks on mutable keys, `System.IO.Hashing` namespace overview, accessed 2026-09-08, dotnet/runtime `Dictionary.cs` source for the bucket and entry layout, accessed 2026-09-10. Sample behavior verified by `make verify-csharp`, 15 tests in chapter 6 of the samples suite. The seven-language layer verifies the same way: 4 C programs under `make verify-c`, 13 Go tests, the java runner's 63 Ch06 checks over 4 files under `run-java-samples`, 15 `node --test` cases, 44 Python checks across 4 files, and 17 Lua checks under `run.lua`.

7. trees

The binary search tree is the first structure whose performance is a property of its shape rather than its code. This chapter builds it, breaks it with sorted input, and repairs it with rotations, then maps the territory onto what the BCL actually ships.

7.1. the search tree invariant

Everything left of a node is smaller, everything right is larger, so a lookup is a guided binary search that costs one comparison per level. C carries the core in two functions, insert by recursion, find by loop.

The dry run: the fixture is 50, 30, 70, 20, 40, 60, 80 plus the sorted storm 0 through 99, asserted by the C# suite, while C, Java, JavaScript, and Lua insert 5, 3, 8, 1, 4, 7, 9, 2, 6 for the same contract.

1. Insert 50 at the root, then 30 left and 70 right, one comparison each.
2. 20 pays two, 20 < 50 then 20 < 30, and 40 pays the same pair the other way, 40 < 50 then 40 > 30, both landing under 30.
3. 60 and 80 mirror the right side at two comparisons each, and the seven keys close a perfect tree of height 3.
4. Inorder reads 20, 30, 40, 50, 60, 70, 80, sorted out of input that arrived level by level.
5. Rebuilt with sorted keys 0 through 99 the same inserts chain rightward to height 100, and TryGet(99) compares against every node on the chain, 99 + 1 = 100, the pinned meter readout, while the same 100 keys shuffled stay under height 20.

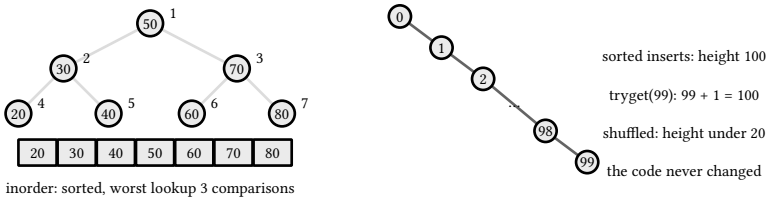


figure 43: the fixture build with insertion order numbered, then the meter walking the sorted chain

The 3 comparisons of the balanced build against the 100 on the meter is the section in two numbers, and the listings below carry both shapes.

```c
static BNode *insert(BNode *t, int v) {
  if (!t)
    return bnew(v);
  if (v < t->val)
    t->left = insert(t->left, v);
  else if (v > t->val)
    t->right = insert(t->right, v);
  return t;
}

static BNode *find(BNode *t, int v) {
  for (; t; t = v < t->val ? t->left : t->right)
    if (t->val == v)
      return t;
  return NULL;
}
```

listing 153: c, insert by recursion, find by loop (source: dsa/samples-c/src/Ch07/bst.c, lines 33-48)

```
func insert(n *BSTNode, v int) *BSTNode {
  if n == nil {
    return &BSTNode{Val: v}
  }
  if v < n.Val {
    n.Left = insert(n.Left, v)
  } else if v > n.Val {
    n.Right = insert(n.Right, v)
  }
  return n
}

// Search reports whether v is present.
func (t *BST) Search(v int) bool {
  n := t.root
  for n ≠ nil {
    switch {
    case v == n.Val:
      return true
    case v < n.Val:
      n = n.Left
    default:
      n = n.Right
    }
  }
  return false
}
```

listing 154: go, insert and search as package functions (source: dsa/samples-go/ch07/bst.go, lines 26-52)

Six more languages build the same tree, Java relinking each level through the subtree its recursion returns:

```
  static BNode insert(BNode t, int v) {
    if (t == null)
      return bnew(v);
    if (v < t.val)
      t.left = insert(t.left, v);
    else if (v > t.val)
      t.right = insert(t.right, v);
    return t;
  }

  static BNode find(BNode t, int v) {
    for (; t ≠ null; t = v < t.val ? t.left : t.right)
      if (t.val == v)
        return t;
    return null;
  }
```

listing 155: java, insert by recursion relinking through return values, find by loop (source: dsa/samples-java/src/Ch07/Bst.java, lines 31-46)

The C# class carries a comparison meter for the same reason the hash table carried a probe meter:

```
public sealed class Bst<K, V> where K : IComparable<K>
{
    public sealed class Node(K key, V value)
    {
        public K Key = key;
        public V Value = value;
        public Node? Left, Right;
    }

    private Node? _root;
    private readonly IComparer<K> _cmp = Comparer<K>.Default;

    public int Count { get; private set; }

    /// <summary>Comparisons of the last operation, the depth meter.</summary>
    public long LastComparisons { get; private set; }

    public V this[K key]
    {
        get
        {
            Node? node = Find(key);
            if (node == null)
                throw new KeyNotFoundException($"no key {key}");
            return node.Value;
        }
        set
        {
            LastComparisons = 0;
            _root = Insert(_root, key, value);
        }
    }
```

listing 156: c#, the bst with a comparison meter on every operation (source: dsa/samples/src/Ch07/Trees.cs, lines 4-35)

```
export function insert(t, v) {
  if (!t) return new BNode(v);
  if (v < t.val) t.left = insert(t.left, v);
  else if (v > t.val) t.right = insert(t.right, v);
  return t;
}

export function find(t, v) {
  while (t) {
    if (t.val === v) return t;
    t = v < t.val ? t.left : t.right;
  }
  return null;
}
```

listing 157: javascript, insert and find as free functions (source: dsa/samples-js/src/ch07-bst.mjs, lines 13-26)

```python
def insert(root, value):
    if root is None:
        return BNode(value)
    if value < root.value:
        root.left = insert(root.left, value)
    elif value > root.value:
        root.right = insert(root.right, value)
    return root


def search(root, value):
    while root:
        if root.value == value:
            return root
        root = root.left if value < root.value else root.right
    return None
```

listing 158: python, insert and search over slots nodes (source: dsa/samples-py/src/Ch07/bst.py, lines 23-38)

```lua
local function insert(t, v)
  if not t then return {val = v} end
  if v < t.val then
    t.left = insert(t.left, v)
  elseif v > t.val then
    t.right = insert(t.right, v)
  end
  return t
end

local function find(t, v)
  while t do
    if t.val == v then return t end
    t = v < t.val and t.left or t.right
  end
  return nil
end
```

listing 159: lua, insert and find over plain tables (source: dsa/samples-lua/ch07_bst.lua, lines 6-22)

Every suite pins the same contract: inorder comes back sorted no matter the insertion order, hits land on their node, misses return nothing. C, Java, JavaScript, and Lua insert 5 3 8 1 4 7 9 2 6 and read 1 through 9 back. Python inserts seven keys and deletes its way down to an empty tree. Go guards insert and delete with a search first, the only suite whose public methods refuse duplicates by design. C# is the only one that counts comparisons while it walks.

Insertion walks the same path and attaches a leaf where it falls off. The three traversals are one recursion with the visit in different positions, and level order is chapter 5's queue doing breadth first over a tree:

```c
static void pre(N *t, int *out, int *k) {
  if (!t)
    return;
  out[(*k)++] = t->val;
  pre(t->left, out, k);
  pre(t->right, out, k);
}

static void in(N *t, int *out, int *k) {
  if (!t)
    return;
  in(t->left, out, k);
  out[(*k)++] = t->val;
  in(t->right, out, k);
}

static void post(N *t, int *out, int *k) {
  if (!t)
    return;
  post(t->left, out, k);
  post(t->right, out, k);
  out[(*k)++] = t->val;
}

// level order with an array queue
static int level(N *t, int *out) {
  N *q[7];
  int head = 0, tail = 0, k = 0;
  q[tail++] = t;
  while (head < tail) {
    N *cur = q[head++];
    out[k++] = cur->val;
    if (cur->left)
      q[tail++] = cur->left;
    if (cur->right)
      q[tail++] = cur->right;
  }
  return k;
}
```

listing 160: c, three recursive walks and the array-queue level order (source: dsa/samples-c/src/Ch07/traverse.c, lines 39-77)

```go
func LevelOrder(n *BSTNode) []int {
  var out []int
  if n == nil {
    return out
  }
  queue := []*BSTNode{n}
  for len(queue) > 0 {
    cur := queue[0]
    queue = queue[1:]
    out = append(out, cur.Val)
    if cur.Left != nil {
      queue = append(queue, cur.Left)
    }
    if cur.Right != nil {
      queue = append(queue, cur.Right)
    }
  }
  return out
}
```

listing 161: go, level order through a slice queue (source: dsa/samples-go/ch07/traverse.go, lines 37-55)

The other five languages walk the same fixture, Java's level order advancing a head index over one array:

```
  static int pre(N t, int[] out, int k) {
    if (t == null)
      return k;
    out[k++] = t.val;
    k = pre(t.left, out, k);
    return pre(t.right, out, k);
  }

  static int in(N t, int[] out, int k) {
    if (t == null)
      return k;
    k = in(t.left, out, k);
    out[k++] = t.val;
    return in(t.right, out, k);
  }

  static int post(N t, int[] out, int k) {
    if (t == null)
      return k;
    k = post(t.left, out, k);
    k = post(t.right, out, k);
    out[k++] = t.val;
    return k;
  }

  // level order with an array queue
  static int level(N t, int[] out) {
    N[] q = new N[7];
    int head = 0, tail = 0, k = 0;
    q[tail++] = t;
    while (head < tail) {
      N cur = q[head++];
      out[k++] = cur.val;
      if (cur.left != null)
        q[tail++] = cur.left;
      if (cur.right != null)
        q[tail++] = cur.right;
    }
    return k;
  }
```

listing 162: java, three recursive walks and the array-queue level order over a head index (source: dsa/samples-java/src/Ch07/Traverse.java, lines 41-80)

Inorder is the payoff: it enumerates keys sorted regardless of shape, which is why a tree can back an ordered dictionary and a hash table cannot.

```
    public List<K> InOrder() =>
        InOrderOf(_root, new List<K>(Count));

    public List<K> PreOrder() => PreOrderOf(_root, new List<K>(Count));

    public List<K> PostOrder() => PostOrderOf(_root, new List<K>(Count));

    public List<K> LevelOrder()
    {
        var result = new List<K>(Count);
        var queue = new Queue<Node?>();
        queue.Enqueue(_root);
        while (queue.Count > 0)
        {
            Node? node = queue.Dequeue();
            if (node == null)
                continue;
            result.Add(node.Key);
            queue.Enqueue(node.Left);
            queue.Enqueue(node.Right);
        }
        return result;
    }
```

listing 163: c#, inorder, preorder, postorder, and the queue-driven level order (source: dsa/samples/src/Ch07/Trees.cs, lines 62-84)

```
export function preorder(t) {
  if (!t) return [];
  return [t.val, ...preorder(t.left), ...preorder(t.right)];
}

export function inorder(t) {
  if (!t) return [];
  return [...inorder(t.left), t.val, ...inorder(t.right)];
}

export function postorder(t) {
  if (!t) return [];
  return [...postorder(t.left), ...postorder(t.right), t.val];
}

// level order with an array queue
export function levelOrder(t) {
  const out = [];
  const q = t ? [t] : [];
  while (q.length) {
    const cur = q.shift();
    out.push(cur.val);
    if (cur.left) q.push(cur.left);
    if (cur.right) q.push(cur.right);
  }
  return out;
}
```

listing 164: javascript, the four walks as spread-recursive functions (source: dsa/samples-js/src/ch07-traverse.mjs, lines 15-41)

```python
def preorder(node):
    if node is None:
        return []
    return [node.value, *preorder(node.left), *preorder(node.right)]


def inorder(node):
    if node is None:
        return []
    return [*inorder(node.left), node.value, *inorder(node.right)]


def postorder(node):
    if node is None:
        return []
    return [*postorder(node.left), *postorder(node.right), node.value]


def level_order(root):
    out, queue, head = [], [root], 0
    while head < len(queue):  # a list with a walking head: a plain queue
        node = queue[head]
        head += 1
        out.append(node.value)
        for child in (node.left, node.right):
            if child:
                queue.append(child)
    return out
```

listing 165: python, three recursions and a walking-head queue (source: dsa/samples-py/src/Ch07/traverse.py, lines 30-57)

```lua
local function pre(t, out)
  if not t then return end
  out[#out + 1] = t.val
  pre(t.left, out)
  pre(t.right, out)
end

local function ino(t, out)
  if not t then return end
  ino(t.left, out)
  out[#out + 1] = t.val
  ino(t.right, out)
end

local function post(t, out)
  if not t then return end
  post(t.left, out)
  post(t.right, out)
  out[#out + 1] = t.val
end

-- level order with an array queue
local function level(t)
  local out = {}
  local q = {t}
  local head = 1
  while head ≤ #q do
    local cur = q[head]
    head = head + 1
    out[#out + 1] = cur.val
    if cur.left then q[#q + 1] = cur.left end
    if cur.right then q[#q + 1] = cur.right end
  end
  return out
end
```

listing 166: lua, the four walks over table nodes (source: dsa/samples-lua/ch07_traverse.lua, lines 29-63)

The 7-node fixture over 1 through 7 is the anchor, and all seven pin all four sequences: preorder 4 2 1 3 6 5 7, inorder 1 through 7, postorder 1 3 2 5 7 6 4, level order 4 2 6 1 3 5 7. The implementations differ only in queue discipline, C, Java, Lua, and Python advance a head index over one array, Go reslices the front away, and JavaScript shifts the front element, the same fifo three ways.

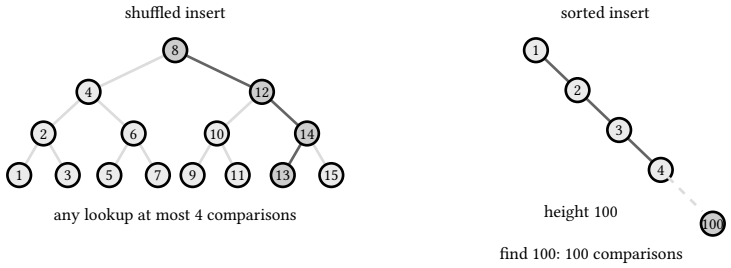


figure 44: the same keys twice, shuffled input stays a shallow guided search, sorted input degenerates into a chain

WARNING: the shape is the input's fault

Insert 100 keys in sorted order and the tree is a linked list, height 100, the last lookup takes 100 comparisons, both pinned by tests. Insert the same keys shuffled and height lands under 20. Nothing in the code changed.

> An unbalanced bst promises logarithmic search only for random input, and sorted input is common enough in real systems, timestamps, ids, auto-increment counters, that the promise must be engineered.

7.2. deletion, the three cases

Removing a leaf drops it. Removing a node with one child splices the child up. Removing a node with two children swaps in the successor, the leftmost node of the right subtree, which preserves the invariant because it is the smallest key greater than the removed one.

The dry run: the fixture is the eight keys 50, 30, 70, 20, 40, 60, 80, 35, asserted by the C# suite, while C, Java, JavaScript, and Lua run 5, 3, 8, 1, 4, 7, 9, 2, 6 and erase the leaf 2, the one-child 7, and the two-child root 5.

1. 35 arrives last and walks deepest: 35 < 50, 35 > 30, 35 < 40, three comparisons to its slot under 40.
2. Remove(20) drops a leaf, and inorder reads 30, 35, 40, 50, 60, 70, 80.
3. Remove(30) splices its only subtree up, 40 with 35 below takes the slot: 35, 40, 50, 60, 70, 80.
4. Remove(50) is the two-child root, and the successor is the leftmost of the right subtree, 60.
5. 60 copies in and leaves a slot where it had no left child, the one-child case by construction, and inorder lands 35, 40, 60, 70, 80 at count 5.
6. Remove(999) finds nothing and the count stays 5.

remove	case	inorder after	count
20	leaf	30 35 40 50 60 70 80	7
30	one child	35 40 50 60 70 80	6
50	two children	35 40 60 70 80	5
999	absent	35 40 60 70 80	5

The count resting at 5 after three removals is the pinned landing, and the listings below carry all three cases.

```c
static BNode *erase(BNode *t, int v) {
  if (!t)
    return NULL;
  if (v < t->val) {
    t->left = erase(t->left, v);
    return t;
  }
  if (v > t->val) {
    t->right = erase(t->right, v);
    return t;
  }
  if (!t->left)
    return t->right;
  if (!t->right)
    return t->left;
  // two children: pull the inorder successor's value up, delete it
  BNode *s = t->right;
  while (s->left)
    s = s->left;
  t->val = s->val;
  t->right = erase(t->right, s->val);
  return t;
}
```

listing 167: c, the three-case erase with the successor swap (source: dsa/samples-c/src/Ch07/bst.c, lines 50-72)

```go
func del(n *BSTNode, v int) *BSTNode {
  if n == nil {
    return nil
  }
  switch {
  case v < n.Val:
    n.Left = del(n.Left, v)
  case v > n.Val:
    n.Right = del(n.Right, v)
  case n.Left == nil:
    return n.Right
  case n.Right == nil:
    return n.Left
  default:
    succ := n.Right
    for succ.Left ≠ nil {
      succ = succ.Left
    }
    n.Val = succ.Val
    n.Right = del(n.Right, succ.Val)
  }
  return n
}
```

listing 168: go, deletion as one switch over the three cases (source: dsa/samples-go/ch07/bst.go, lines 66-88)

```java
  static BNode erase(BNode t, int v) {
    if (t == null)
      return null;
    if (v < t.val) {
      t.left = erase(t.left, v);
      return t;
    }
    if (v > t.val) {
      t.right = erase(t.right, v);
      return t;
    }
    if (t.left == null)
      return t.right;
    if (t.right == null)
      return t.left;
    // two children: pull the inorder successor's value up, delete it
    BNode s = t.right;
    while (s.left ≠ null)
      s = s.left;
    t.val = s.val;
    t.right = erase(t.right, s.val);
    return t;
  }
```

listing 169: java, the three-case erase with the successor swap, relinked through return values (source: dsa/samples-java/src/Ch07/Bst.java, lines 48-70)

```
    private Node? Find(K key)
    {
        LastComparisons = 0;
        Node? node = _root;
```

```
        while (node != null)
        {
            LastComparisons++;
            int c = _cmp.Compare(key, node.Key);
            if (c == 0)
                return node;
            node = c < 0 ? node.Left : node.Right;
        }
        return null;
    }

    private Node Insert(Node? node, K key, V value)
    {
        if (node == null)
        {
            Count++;
            return new Node(key, value);
        }
        LastComparisons++;
        int c = _cmp.Compare(key, node.Key);
        if (c == 0)
        {
            node.Value = value;
            return node;
        }
        if (c < 0)
            node.Left = Insert(node.Left, key, value);
        else
            node.Right = Insert(node.Right, key, value);
        return node;
    }

    // the three cases: leaf, one child, two children via successor swap
    private Node? RemoveNode(Node? node, K key, ref bool removed)
    {
        if (node == null)
            return null;
        int c = _cmp.Compare(key, node.Key);
        if (c < 0)
        {
            node.Left = RemoveNode(node.Left, key, ref removed);
            return node;
        }
        if (c > 0)
        {
            node.Right = RemoveNode(node.Right, key, ref removed);
            return node;
        }
        removed = true;
        if (node.Left == null)
            return node.Right;
        if (node.Right == null)
            return node.Left;
        // two children: pull the successor (leftmost of right) up
        Node successor = node.Right;
        while (successor.Left != null)
            successor = successor.Left;
```

```
        (node.Key, node.Value) = (successor.Key, successor.Value);
        K sk = successor.Key;
        node.Right = RemoveNode(node.Right, sk, ref removed);
        return node;
    }
```

listing 170: c#, find and insert, then the three-case removal via successor swap (source: dsa/samples/src/Ch07/Trees.cs, lines 86-151)

```
export function erase(t, v) {
  if (!t) return null;
  if (v < t.val) {
    t.left = erase(t.left, v);
    return t;
  }
  if (v > t.val) {
    t.right = erase(t.right, v);
    return t;
  }
  if (!t.left) return t.right;
  if (!t.right) return t.left;
  // two children: pull the inorder successor's value up, delete it below
  let s = t.right;
  while (s.left) s = s.left;
  t.val = s.val;
  t.right = erase(t.right, s.val);
  return t;
}
```

listing 171: javascript, erase with the successor pulled up (source: dsa/samples-js/src/ch07-bst.mjs, lines 28-46)

```
def delete(root, value):
    if root is None:
        return None
    if value < root.value:
        root.left = delete(root.left, value)
        return root
    if value > root.value:
        root.right = delete(root.right, value)
        return root
    if root.left is None:
        return root.right  # zero or one child: promote the survivor
    if root.right is None:
        return root.left
    succ = root.right  # two children: splice in the inorder successor
    while succ.left:
        succ = succ.left
    root.value = succ.value
    root.right = delete(root.right, succ.value)
    return root
```

listing 172: python, delete with the survivor promoted (source: dsa/samples-py/src/Ch07/bst.py, lines 41-59)

```lua
local function erase(t, v)
  if not t then return nil end
  if v < t.val then
    t.left = erase(t.left, v)
    return t
  end
  if v > t.val then
    t.right = erase(t.right, v)
    return t
  end
  if not t.left then return t.right end
  if not t.right then return t.left end
  -- two children: pull the inorder successor's value up, delete it
  local s = t.right
  while s.left do s = s.left end
  t.val = s.val
  t.right = erase(t.right, s.val)
  return t
end
```

listing 173: lua, erase over table nodes (source: dsa/samples-lua/ch07_bst.lua, lines 24-42)

C, Java, JavaScript, and Lua run the fixture 5 3 8 1 4 7 9 2 6 and pin the same three removals, the leaf 2, the one-child node 7, and the two-child root 5 whose successor 6 takes over, with inorder sorted after every step. Python works seven keys and drains to an empty tree, its two-child case swapping 8's successor 9. Go deletes through one switch and replays the leaf, one-child, and successor cases in its tests. The successor swap hides a subtlety every listing covers: after copying the successor into the node, the code must remove the successor from the right subtree, where it is guaranteed to have no left child, a one-child case by construction.

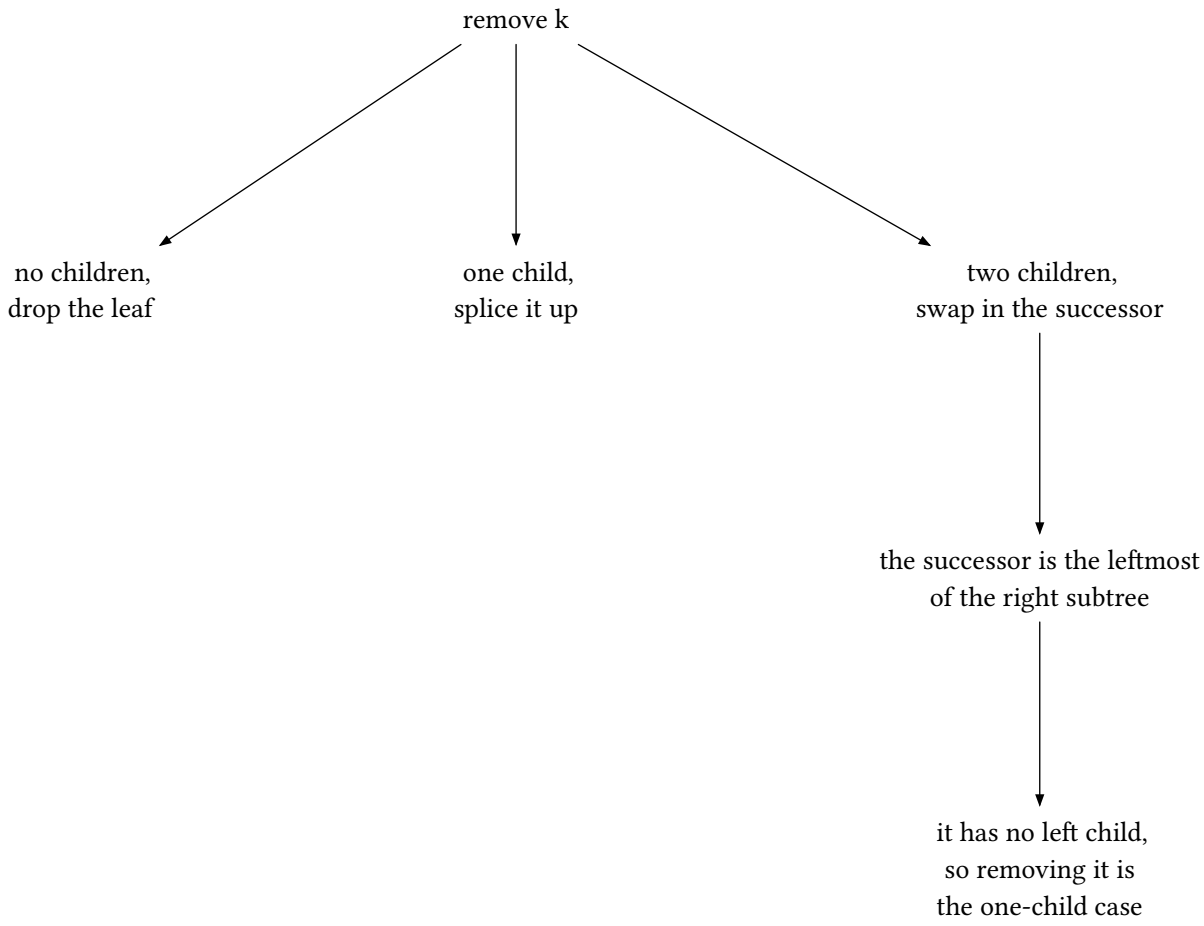


figure 45: removal, three cases by child count, the successor swap ends in the one-child case

7.3. avl, the rotation reflex

The AVL tree keeps the height difference of every node's children at most 1 and repairs violations on the way back up the insertion path. Four shapes exist, two single rotations and two doubles, and the repair is the same decision everywhere, look at the balance of the heavy child. C folds the decision into the insert itself.

The dry run: the fixture is sorted inserts 0 through 999 plus the four three-key shapes, asserted by the C# suite, while C, Java, JavaScript, and Lua pin the same shapes and ascending 1 through 15 at height 4.

1. Sorted keys are the storm: the plain tree above chains 100 of them to height 100.
2. The first repair lands at key 2, where 0, 1, 2 is a right-right stick: one left rotation roots 1 with children 0 and 2, height 2.
3. Every insert unwinds its path and rotates wherever the child heights differ by 2, and the imbalance probe reads at most 1 for the whole run.
4. 1000 keys finish at height 12 or under, 2^10 = 1024 leaves head room in 12 levels.
5. The four shapes 3 2 1, 1 2 3, 3 1 2, and 1 3 2 each end height 2, the left-left case a perfect tree with imbalance 0.
6. Rotations above zero and inorder 0 through 999 close the run: sorted input forced the repairs and still lost.

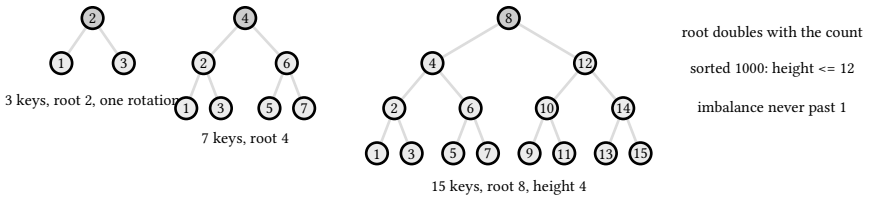


figure 46: ascending keys, one repair per boundary, the root doubling as the count grows: 2, 4, 8

The 12 over 1000 sorted keys against the plain tree's 100 over 100 is the repair doing its job, and the listings below meter it.

```c
static ANode *insert(ANode *t, int v) {
  if (!t)
    return anew(v);
  if (v < t->val)
    t->left = insert(t->left, v);
  else if (v > t->val)
    t->right = insert(t->right, v);
  else
    return t;
  fix(t);
  int b = balance_of(t);
  if (b > 1 && v < t->left->val) // LL: one right rotation
    return rot_right(t);
  if (b < -1 && v > t->right->val) // RR: one left rotation
    return rot_left(t);
  if (b > 1) { // LR: left at the child, then right here
    t->left = rot_left(t->left);
    return rot_right(t);
  }
  if (b < -1) { // RL: right at the child, then left here
    t->right = rot_right(t->right);
    return rot_left(t);
  }
  return t;
}
```

listing 174: c, avl insert with the four rotation cases inline (source: dsa/samples-c/src/Ch07/balance.c, lines 60-84)

```go
func AVLInsert(n *AVLNode, v int) *AVLNode {
  if n == nil {
    return &AVLNode{Val: v, height: 1}
  }
  if v < n.Val {
    n.Left = AVLInsert(n.Left, v)
  } else if v > n.Val {
    n.Right = AVLInsert(n.Right, v)
  } else {
    return n // duplicate: no change, nothing to rebalance
  }
  avlUpdate(n)
  return avlRebalance(n)
}

func avlRebalance(n *AVLNode) *AVLNode {
  b := avlBalance(n)
  switch {
  case b > 1 && avlBalance(n.Left) ≥ 0:
    return rotateRight(n) // LL
  case b > 1:
    n.Left = rotateLeft(n.Left) // LR
    return rotateRight(n)
  case b < -1 && avlBalance(n.Right) <= 0:
    return rotateLeft(n) // RR
  case b < -1:
    n.Right = rotateRight(n.Right) // RL
    return rotateLeft(n)
  }
  return n
}
```

listing 175: go, insert then rebalance by the child's balance factor (source: dsa/samples-go/ch07/balance.go, lines 75-105)

The same reflex in the other five languages, Java choosing the case by the inserted value against the child's key:

```
  static int ht(ANode t) { return t == null ? 0 : t.h; }

  static void fix(ANode t) {
    int l = ht(t.left), r = ht(t.right);
    t.h = Math.max(l, r) + 1;
  }

  static int balanceOf(ANode t) {
    return ht(t.left) - ht(t.right);
  }

  static ANode rotRight(ANode y) {
    ANode x = y.left;
    y.left = x.right;
    x.right = y;
    fix(y);
    fix(x);
    return x;
  }

  static ANode rotLeft(ANode x) {
    ANode y = x.right;
    x.right = y.left;
    y.left = x;
    fix(x);
    fix(y);
    return y;
  }

  static ANode insert(ANode t, int v) {
    if (t == null)
      return anew(v);
    if (v < t.val)
      t.left = insert(t.left, v);
    else if (v > t.val)
      t.right = insert(t.right, v);
    else
      return t;
    fix(t);
    int b = balanceOf(t);
    if (b > 1 && v < t.left.val) // LL: one right rotation
      return rotRight(t);
    if (b < -1 && v > t.right.val) // RR: one left rotation
      return rotLeft(t);
    if (b > 1) { // LR: left at the child, then right here
      t.left = rotLeft(t.left);
      return rotRight(t);
    }
    if (b < -1) { // RL: right at the child, then left here
      t.right = rotRight(t.right);
      return rotLeft(t);
    }
    return t;
  }
```

listing 176: java, cached heights, both rotations, avl insert with the four cases inline (source: dsa/samples-java/src/Ch07/Balance.java, lines 30-83)

```
public sealed class Avl<K, V> where K : IComparable<K>
{
    private sealed class Node(K key, V value)
    {
        public K Key = key;
        public V Value = value;
        public Node? Left, Right;
        public int Height = 1;
    }

    private Node? _root;
    public int Count { get; private set; }

    /// <summary>Rotations performed since construction, the balance meter.</summary>
    public int Rotations { get; private set; }

    public V this[K key]
    {
        get
        {
            Node? node = Find(_root, key);
            if (node == null)
                throw new KeyNotFoundException($"no key {key}");
            return node.Value;
        }
        set => _root = Insert(_root, key, value);
    }

    public int Height() => HeightOf(_root);

    /// <summary>Worst root-to-leaf height difference, must stay at most 1.</summary>
    public int MaxImbalance()
    {
        int worst = 0;
        void Walk(Node? node)
        {
            if (node == null)
                return;
            worst = Math.Max(worst, Math.Abs(Balance(node)));
            Walk(node.Left);
            Walk(node.Right);
        }
        Walk(_root);
        return worst;
    }

    public List<K> InOrder()
    {
        var result = new List<K>(Count);
        void Walk(Node? node)
        {
            if (node == null)
                return;
            Walk(node.Left);
            result.Add(node.Key);
            Walk(node.Right);
        }
```

```
        Walk(_root);
        return result;
    }

    private static Node? Find(Node? node, K key)
    {
        while (node != null)
        {
            int c = key.CompareTo(node.Key);
            if (c == 0)
                return node;
            node = c < 0 ? node.Left : node.Right;
        }
        return null;
    }

    private Node Insert(Node? node, K key, V value)
    {
        if (node == null)
        {
            Count++;
            return new Node(key, value);
        }
        int c = key.CompareTo(node.Key);
        if (c == 0)
        {
            node.Value = value;
            return node;
        }
        if (c < 0)
            node.Left = Insert(node.Left, key, value);
        else
            node.Right = Insert(node.Right, key, value);
        return Rebalance(node);
    }
```

listing 177: c#, the avl with rotation counter and imbalance probe (source: dsa/samples/src/Ch07/Trees.cs, lines 188-280)

A rotation moves a child up and a subtree across, three pointer assignments plus two height updates, and the invariant check is pure arithmetic on cached heights:

```
    private Node Rebalance(Node node)
    {
        UpdateHeight(node);
        int balance = Balance(node);
        if (balance > 1) // left heavy
        {
            if (Balance(node.Left!) < 0)
            {
                node.Left = RotateLeft(node.Left!); // left-right case
                Rotations++;
            }
            return RotateRight(node);
        }
        if (balance < -1) // right heavy
        {
            if (Balance(node.Right!) > 0)
            {
                node.Right = RotateRight(node.Right!); // right-left case
                Rotations++;
            }
            return RotateLeft(node);
        }
        return node;
    }

    private Node RotateRight(Node pivot)
    {
        Rotations++;
        Node newRoot = pivot.Left!;
        pivot.Left = newRoot.Right;
        newRoot.Right = pivot;
        UpdateHeight(pivot);
        UpdateHeight(newRoot);
        return newRoot;
    }

    private Node RotateLeft(Node pivot)
    {
        Rotations++;
        Node newRoot = pivot.Right!;
        pivot.Right = newRoot.Left;
        newRoot.Left = pivot;
        UpdateHeight(pivot);
        UpdateHeight(newRoot);
        return newRoot;
    }

    private static int HeightOf(Node? node) => node?.Height ?? 0;

    private static int Balance(Node node) => HeightOf(node.Left) - HeightOf(node.Right);

    private static void UpdateHeight(Node node) =>
        node.Height = 1 + Math.Max(HeightOf(node.Left), HeightOf(node.Right));
}
```

listing 178: c#, rebalance, the double-rotation cases, both rotations (source: dsa/samples/src/Ch07/Trees.cs, lines 281-334)

The tests pin the headline numbers: 1000 sorted inserts leave height at most 12 where the plain tree reaches 1000, the imbalance probe never exceeds 1, and the rotation counter proves sorted input is what forces rotations. The four shape tests insert three keys in each order, two single and two double rotation cases, and all end height 2.

```
export function avlInsert(t, v) {
  if (!t) return { val: v, h: 1, left: null, right: null };
  if (v < t.val) t.left = avlInsert(t.left, v);
  else if (v > t.val) t.right = avlInsert(t.right, v);
  else return t;
  fix(t);
  const b = balanceOf(t);
  if (b > 1 && v < t.left.val) return rotRight(t); // LL
  if (b < -1 && v > t.right.val) return rotLeft(t); // RR
  if (b > 1) {
    t.left = rotLeft(t.left); // LR
    return rotRight(t);
  }
  if (b < -1) {
    t.right = rotRight(t.right); // RL
    return rotLeft(t);
  }
  return t;
}
```

listing 179: javascript, avl insert with the case labels inline (source: dsa/samples-js/src/ch07-balance.mjs, lines 31-49)

```
def insert(node, value):
    if node is None:
        return AvlNode(value)
    if value < node.value:
        node.left = insert(node.left, value)
    else:
        node.right = insert(node.right, value)
    update(node)
    bf = balance_factor(node)
    if bf > 1 and value < node.left.value:  # left-left: one right rotation
        return rotate_right(node)
    if bf < -1 and value > node.right.value:  # right-right: one left rotation
        return rotate_left(node)
    if bf > 1:  # left-right: rotate the child left, then this node right
        node.left = rotate_left(node.left)
        return rotate_right(node)
    if bf < -1:  # right-left: rotate the child right, then this node left
        node.right = rotate_right(node.right)
        return rotate_left(node)
    return node
```

listing 180: python, insert with balance factors read off cached heights (source: dsa/samples-py/src/Ch07/balance.py, lines 53-72)

```lua
local function insert(t, v)
  if not t then return {val = v, h = 1} end
  if v < t.val then
    t.left = insert(t.left, v)
  elseif v > t.val then
    t.right = insert(t.right, v)
  else
    return t
  end
  fix(t)
  local b = balance_of(t)
  if b > 1 and v < t.left.val then return rot_right(t) end -- LL
  if b < -1 and v > t.right.val then return rot_left(t) end -- RR
  if b > 1 then -- LR: left at the child, then right here
    t.left = rot_left(t.left)
    return rot_right(t)
  end
  if b < -1 then -- RL: right at the child, then left here
    t.right = rot_right(t.right)
    return rot_left(t)
  end
  return t
end
```

listing 181: lua, insert with the four cases commented (source: dsa/samples-lua/ch07_balance.lua, lines 36-58)

The four shapes pin identically in C, Java, JavaScript, and Lua: 3 2 1, 1 2 3, 3 1 2, and 1 3 2 all collapse to the root 2 with children 1 and 3, and ascending 1 through 15 leaves height 4 rooted at 8. Python runs the four shapes plus the classic 10 20 30 40 50 25 sequence rooting at 30 with height 3. The case choice reads two routes to the same answer: Go asks the heavy child for its balance factor, while C, Java, JavaScript, Lua, and Python compare the inserted value against the child's key. C# meters rotations, the only balance factor the others leave uncounted.

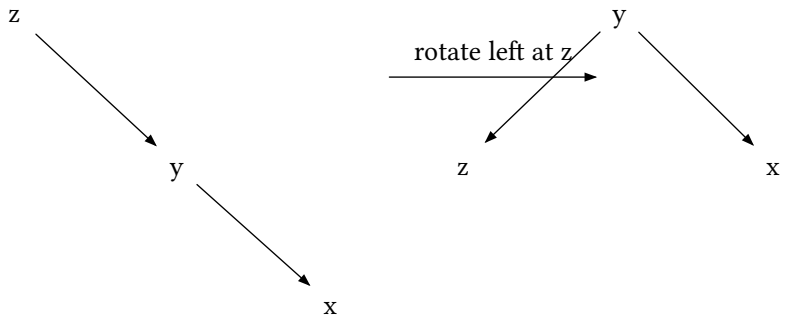


figure 47: the rotation reflex, the heavy stick before and after one left rotation

7.4. what the bcl ships

structure	inside	point read	ranges
SortedDictionary	red-black tree	O(log n)	yes
SortedList	sorted array pair	O(log n)	yes
Dictionary	bucket chains	O(1)	no

point lookups want the hash

ranges and neighbors want a tree

figure 48: what the bcl ships, red-black recoloring lazier than avl, sortedlist a sorted array pair with o(n) inserts, hash for point lookups

`SortedDictionary<K,V>` is a red-black tree, the same invariant engineered with recoloring instead of strict heights, and `SortedSet` is the same tree keyed by values. Red-black rebalances less eagerly than AVL, fewer rotations per

insert at the cost of slightly taller trees, which is the standard production trade. `SortedList<K,V>` is not a tree at all, it is a sorted array pair with binary search, fast reads and O(n) inserts, right for read-mostly tables. The rule: need ordering, ranges, predecessor and successor queries, use a tree, need only point lookups, chapter 6's hash wins on constants.

The dry run: no suite carries the shipped containers, the numbers are hand-derived, and the trade on trial is SortedList's shifts against the tree's writes.

1. Name the workload first: point lookups only point at the hash, ranges and neighbors at a tree, read-mostly at the sorted array pair.
2. Descending inserts 40, 30, 20, 10 into SortedList each binary search to slot 0 and shift the tail: 0 + 1 + 2 + 3 = 6 element moves for 4 keys.
3. The same four into SortedDictionary are 4 node writes, and no element ever shifts, shape is the rebalancer's problem.
4. Point reads stay cheap on both sorted shapes: 4 keys resolve in at most 3 probes, 1000 keys in at most 10, 2^10 = 1024.
5. The range 20 through 40 is a walk between two boundary searches on the tree, a trip the hash table cannot offer at any price.

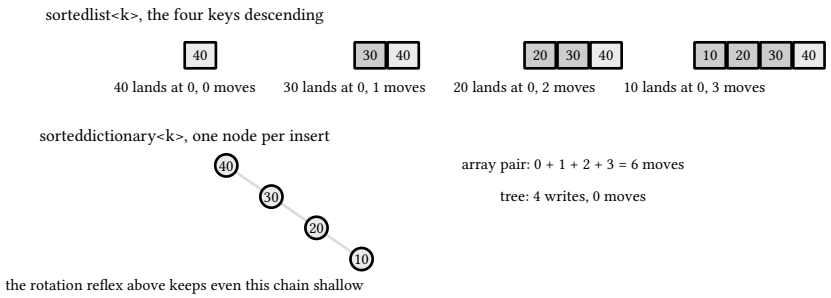


figure 49: four descending inserts, the array pair shifting its whole tail while the tree writes one node at a time

The 6 against the 4 is the table above in one run, read-mostly buys the array pair and anything that moves wants the tree.

7.5. across the seven languages

Build sizes count non-comment source lines over the featured files; bundled checks count where the language puts them in the same file:

language	build SLOC	dependency	boundary note
c	309	libc only	nodes come from static pools, recursion over pointers, 31 checks in 3 files
go	232	none	unexported fields keep the height private, rebalance reads the child's balance factor
java	306	jdk 27 stdlib	TreeMap exists in the stdlib and goes unused, insert and erase relink through return values over one shared static root, the avl case reads the inserted value against the child's key
c#	293	bcl only	generic trees with comparison and rotation meters, the only suite that counts either
javascript	119	node stdlib	spread recursion builds new arrays at every level, fine for fixtures, garbage at scale
python	208	stdlib only	`__slots__` nodes, drains the whole tree through all three delete shapes

language	build SLOC	dependency	boundary note
lua	248	lib.lua harness	a node is a table with nil-able links, and and or pick the walk direction

sources: learn.microsoft.com, `SortedDictionary<TKey,TValue>` remarks on the red-black implementation, `SortedList<TKey,TValue>` remarks on the sorted-array layout and operation costs, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 10 tests in chapter 7 of the samples suite. The seven-language layer verifies the same way: 3 C programs with 31 embedded checks, 8 Go tests, the java runner's 31 Ch07 checks over 3 files under `run-java-samples`, 6 `node --test` cases, 24 Python checks across 3 files, and 14 Lua checks under `run.lua`.

8. heaps and priority queues

A priority queue answers “smallest next” without keeping everything sorted. The binary heap gets that from partial order in an array: the parent is never larger than either child, and the tree lives entirely in index arithmetic, children at `2i+1` and `2i+2`, parent at `(i-1)/2`. Every language in this chapter builds it by hand, the matrix rule, even where a shipped priority queue sits one import away.

8.1. the array heap

Add appends at the end and sifts up, swapping with the parent until order holds. Extract takes the root, moves the last element there, and sifts down against the smaller child.

The dry run: the fixture is 5, 3, 8, 1, 9, 2, 7 drained to sorted order plus the descending storm, asserted by the C# suite, while C, Java, JavaScript, and Lua push 5, 3, 8, 1, 9, 2 and pop 1, 2, 3, 5, 8, 9 with the property re-asserted after every operation.

1. Add 5, then 3: 3 < 5 swaps up, the array reads 3, 5.
2. Add 8 under the root: 8 > 3 stays put, 3, 5, 8.
3. Add 1: 1 < 5 swaps, then 1 < 3 swaps, two hops to the root: 1, 3, 8, 5.
4. Add 9: its parent 3 is smaller, it stays. Add 2: 2 < 8 swaps once, 2 > 1 stops: 1, 3, 2, 5, 9, 8.
5. Add 7: parent 2 is smaller, it stays, and the heap closes at 1, 3, 2, 5, 9, 8, 7, valid at every step.
6. ExtractMin takes 1 and moves the tail 7 to the root: 7 swaps with the smaller child 2, leaving 2, 3, 7, 5, 9, 8, and the drain runs 1, 2, 3, 5, 7, 8, 9 down to count 0.
7. Descending 99 to 0 makes every add sift to the root: the swap meter reads at least 100, peek 0.

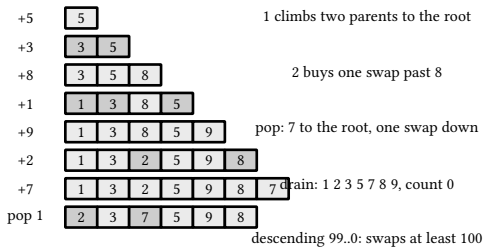


figure 50: the build as one row per arrival, the cells each sift touched shaded, then the first extraction

The drain landing 1, 2, 3, 5, 7, 8, 9 from shuffled input is the pinned pair, and the listings below meter every swap of it.

```c
static void sift_up(int i) {
  while (i > 0 && h[(i - 1) / 2] > h[i]) {
    swap((i - 1) / 2, i);
    i = (i - 1) / 2;
  }
}

static void sift_down(int i) {
  for (;;) {
    int l = 2 * i + 1, r = 2 * i + 2, m = i;
    if (l < hn && h[l] < h[m])
      m = l;
    if (r < hn && h[r] < h[m])
      m = r;
    if (m == i)
      return;
    swap(i, m);
    i = m;
  }
}

static void push(int v) {
  h[hn++] = v;
  sift_up(hn - 1);
}

static int peek(void) { return h[0]; }

static int pop(void) {
  int top = h[0];
  h[0] = h[--hn];
  sift_down(0);
  return top;
}

static int heap_ok(void) {
  for (int i = 1; i < hn; i++)
    if (h[(i - 1) / 2] > h[i])
      return 0;
  return 1;
}
```

listing 182: c, both sifts, push, peek, pop, and the invariant checker (source: dsa/samples-c/src/Ch08/heap.c, lines 26-66)

```go
func (h *MinHeap) Push(x int) {
  h.items = append(h.items, x)
  i := len(h.items) - 1
  for i > 0 {
    parent := (i - 1) / 2
    if h.items[parent] <= h.items[i] {
      break
    }
    h.items[parent], h.items[i] = h.items[i], h.items[parent]
    i = parent
  }
}

// Pop removes the root, moves the last element into the hole, and
// sifts it down against its smaller child.
func (h *MinHeap) Pop() (int, error) {
  if len(h.items) == 0 {
    return 0, fmt.Errorf("pop from empty heap")
  }
  root := h.items[0]
  last := len(h.items) - 1
  h.items[0] = h.items[last]
  h.items = h.items[:last]
  h.siftDown(0, last)
  return root, nil
}

func (h *MinHeap) siftDown(i, n int) {
  for {
    left, right := 2*i+1, 2*i+2
    smallest := i
    if left < n && h.items[left] < h.items[smallest] {
      smallest = left
    }
    if right < n && h.items[right] < h.items[smallest] {
      smallest = right
    }
    if smallest == i {
      return
    }
    h.items[i], h.items[smallest] = h.items[smallest], h.items[i]
    i = smallest
  }
}
```

listing 183: go, push sifts up, pop moves the tail to the root and sifts down, errors as values (source: dsa/samples-go/ch08/heap.go, lines 14-57)

```
static void siftUp(int i) {
  while (i > 0 && h[(i - 1) / 2] > h[i]) {
    swap((i - 1) / 2, i);
    i = (i - 1) / 2;
  }
}

static void siftDown(int i) {
  for (;;) {
    int l = 2 * i + 1, r = 2 * i + 2, m = i;
    if (l < hn && h[l] < h[m])
      m = l;
    if (r < hn && h[r] < h[m])
      m = r;
    if (m == i)
      return;
    swap(i, m);
    i = m;
  }
}

static void push(int v) {
  h[hn++] = v;
  siftUp(hn - 1);
}

static int peek() {
  return h[0];
}

static int pop() {
  int top = h[0];
  h[0] = h[--hn];
  siftDown(0);
  return top;
}

static boolean heapOk() {
  for (int i = 1; i < hn; i++)
    if (h[(i - 1) / 2] > h[i])
      return false;
  return true;
}
```

listing 184: java, both sifts, push, peek, pop, and the invariant checker over one static array (source: dsa/samples-java/src/Ch08/Heap.java, lines 27-69)

```
public sealed class BinaryHeap<T> where T : IComparable<T>
{
    private readonly List<T> _items = new();

    public int Count => _items.Count;

    /// <summary>Swap count, the sift meter.</summary>
    public long Swaps { get; private set; }

    public void Add(T item)
    {
        _items.Add(item);
        SiftUp(_items.Count - 1);
    }

    public T PeekMin() => _items.Count == 0
        ? throw new InvalidOperationException("heap is empty")
        : _items[0];

    public T ExtractMin()
    {
        if (_items.Count == 0)
            throw new InvalidOperationException("heap is empty");
        T min = _items[0];
        int last = _items.Count - 1;
        _items[0] = _items[last];
        _items.RemoveAt(last);
        if (_items.Count > 0)
            SiftDown(0);
        return min;
    }

    public bool IsValid()
    {
        for (int i = 0; i < _items.Count; i++)
        {
            int left = 2 * i + 1, right = 2 * i + 2;
            if (left < _items.Count && _items[left].CompareTo(_items[i]) < 0)
                return false;
            if (right < _items.Count && _items[right].CompareTo(_items[i]) < 0)
                return false;
        }
        return true;
    }

    public List<T> DrainSorted()
    {
        var result = new List<T>(_items.Count);
        while (_items.Count > 0)
            result.Add(ExtractMin());
        return result;
    }
```

listing 185: c#, add, peek, extract, the invariant checker, drain to sorted (source: dsa/samples/src/Ch08/Heaps.cs, lines 4-56)

```
    private void SiftUp(int i)
    {
        while (i > 0)
        {
            int parent = (i - 1) / 2;
            if (_items[i].CompareTo(_items[parent]) ≥ 0)
                break;
            (_items[i], _items[parent]) = (_items[parent], _items[i]);
            Swaps++;
            i = parent;
        }
    }

    private void SiftDown(int i)
    {
        while (true)
        {
            int left = 2 * i + 1, right = 2 * i + 2, smallest = i;
            if (left < _items.Count && _items[left].CompareTo(_items[smallest]) < 0)
                smallest = left;
            if (right < _items.Count && _items[right].CompareTo(_items[smallest]) < 0)
                smallest = right;
            if (smallest == i)
                break;
            (_items[i], _items[smallest]) = (_items[smallest], _items[i]);
            Swaps++;
            i = smallest;
        }
    }
}
```

listing 186: c#, sift up against the parent, sift down against the smaller child (source: dsa/samples/src/Ch08/Heaps.cs, lines 57-86)

```
  #siftUp(i) {
    while (i > 0 && this.#h[Math.floor((i - 1) / 2)] > this.#h[i]) {
      this.#swap(Math.floor((i - 1) / 2), i);
      i = Math.floor((i - 1) / 2);
    }
  }

  #siftDown(i) {
    for (;;) {
      const l = 2 * i + 1;
      const r = 2 * i + 2;
      let m = i;
      if (l < this.#h.length && this.#h[l] < this.#h[m]) m = l;
      if (r < this.#h.length && this.#h[r] < this.#h[m]) m = r;
      if (m === i) return;
      this.#swap(i, m);
      i = m;
    }
  }

  push(v) {
    this.#h.push(v);
    this.#siftUp(this.#h.length - 1);
    return this;
  }

  pop() {
    const top = this.#h[0];
    this.#h[0] = this.#h[this.#h.length - 1];
    this.#h.pop();
    this.#siftDown(0);
    return top;
  }
```

listing 187: javascript, private sift up and sift down behind push and pop (source: dsa/samples-js/src/ch08-heap.mjs, lines 20-52)

```
    def push(self, value):
        self.a.append(value)
        i = len(self.a) - 1
        while i > 0:  # sift up: swap past any larger parent
            parent = (i - 1) // 2
            if self.a[parent] ≤ self.a[i]:
                break
            self.a[parent], self.a[i] = self.a[i], self.a[parent]
            i = parent

    def pop(self):
        if not self.a:
            raise IndexError("pop on empty heap")
        top = self.a[0]
        last = self.a.pop()
        if self.a:
            self.a[0] = last  # the tail takes the root, then sifts down
            self._sift_down(0)
        return top

    def _sift_down(self, i):
        size = len(self.a)
        while True:
            left, right, small = 2 * i + 1, 2 * i + 2, i
            if left < size and self.a[left] < self.a[small]:
                small = left
            if right < size and self.a[right] < self.a[small]:
                small = right
            if small == i:
                return
            self.a[i], self.a[small] = self.a[small], self.a[i]
            i = small

    def is_heap(self):
        return all(self.a[(i - 1) // 2] ≤ self.a[i] for i in range(1, len(self.a)))
```

listing 188: python, the MinHeap class, push, pop, and the invariant check (source: dsa/samples-py/src/Ch08/heap.py, lines 17-51)

```lua
function Heap.push(h, v)
  local a = h.a
  a[#a + 1] = v
  local i = #a
  while i > 1 and a[i // 2] > a[i] do
    a[i // 2], a[i] = a[i], a[i // 2]
    i = i // 2
  end
end

function Heap.peek(h)
  return h.a[1]
end

function Heap.pop(h)
  local a = h.a
  local top = a[1]
  a[1] = a[#a]
  a[#a] = nil
  local i, n = 1, #a
  while true do
    local l, r, m = 2 * i, 2 * i + 1, i
    if l ≤ n and a[l] < a[m] then m = l end
    if r ≤ n and a[r] < a[m] then m = r end
    if m == i then break end
    a[i], a[m] = a[m], a[i]
    i = m
  end
  return top
end
```

listing 189: lua, the same heap one index over, parent at i div 2, children at 2i (source: dsa/samples-lua/ch08_heap.lua, lines 19-48)

Both sifts are logarithmic because the tree is complete, its height is the base-2 log of the count. The swap meter tells the story the tests check: descending input makes every add sift to the root, the maximum work, while the heap stays valid throughout. Note what the heap is not: only the root is guaranteed minimal. The invariant checkers walk parent against child, a full sort is not maintained, and iteration order is meaningless. Lua shifts every formula by one because its arrays start at 1, parent at `i//2`, children at `2i` and `2i+1`, the one translation that touches every line.

Measured across the suites: the C, Java, JavaScript, and Lua files push 5 3 8 1 9 2 and pop 1 2 3 5 8 9 with the property re-asserted after every operation, 29 checks on the C side. Python pins 5 1 4 2 8 0 to 0 1 2 4 5 8 and refuses the empty pop with IndexError. Go pushes a shuffled 1 to 9 and checks the property on a snapshot after each push and pop, and the C# class drains its 7-value fixture to 1 2 3 5 7 8 9 while metering swaps, 100 descending adds cost at least 100.

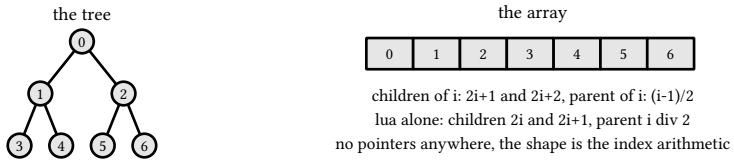


figure 51: one heap twice, the tree and the array it lives in

8.2. heapsort

Heapsort is the heap wearing a sort's clothes: heapify the array in place from the back, then repeatedly swap the maximum to the end and re-sift the shrunk heap. Every version here builds a max-heap so the root is the maximum.

The dry run: the fixture is the theory row 5, 3, 8, 1, 9, 2, 7 sorted in place, asserted by the C# suite against the ordered copy, while C, Java, JavaScript, and Lua sort the pi fixture 3, 1, 4, 1, 5, 9, 2, 6, 5 and Python returns its build peak of 9.

1. Heapify walks backward from index 2: node 8 already dominates its children 2 and 7, no swap.
2. Index 1: 3 swaps with its larger child 9, index 0: 5 swaps with 9, and the max-heap reads 9, 5, 8, 1, 3, 2, 7.
3. The first extraction parks 9 at slot 6; the tail 7 sifts once, past 8, leaving 8, 5, 7, 1, 3, 2.
4. 8 parks, and the tail 2 swaps past 7: 7, 5, 2, 1, 3. Then 7 parks, and the tail 3 swaps past 5: 5, 3, 2, 1.
5. 5, 3, and 2 park in order, each sift touching at most two nodes, and the array ends 1, 2, 3, 5, 7, 8, 9.
6. The theory's other rows arrive already sorted and reversed and land the same place, and the sift never fires on 100 equal sevens.

step	array	parked tail
input	5 3 8 1 9 2 7	
heapify	9 5 8 1 3 2 7	
9 out	8 5 7 1 3 2	9
8 out	7 5 2 1 3	8 9
7 out	5 3 2 1	7 8 9
5 out	3 1 2	5 7 8 9
3 out	2 1	3 5 7 8 9
2 out	1	2 3 5 7 8 9

The parked tail swallowing the array until only 1 survives is the whole sort, and the listings below run it on every shape.

```c
static void sift_down(int *a, int len, int i) {
  for (;;) {
    int l = 2 * i + 1, r = 2 * i + 2, m = i;
    if (l < len && a[l] > a[m])
      m = l;
    if (r < len && a[r] > a[m])
      m = r;
    if (m == i)
      return;
    swap(&a[i], &a[m]);
    i = m;
  }
}

static void heapsort(int *a, int len) {
  for (int i = len / 2 - 1; i ≥ 0; i--)
    sift_down(a, len, i);
  for (int end = len - 1; end > 0; end--) {
    swap(&a[0], &a[end]);
    sift_down(a, end, 0);
  }
}
```

listing 190: c, max-heap sift-down, heapify backward, extract to the end (source: dsa/samples-c/src/Ch08/heapsort.c, lines 24-45)

```go
func Heapsort(a []int) {
  n := len(a)
  for i := n/2 - 1; i ≥ 0; i-- {
    maxSiftDown(a, i, n)
  }
  for end := n - 1; end > 0; end-- {
    a[0], a[end] = a[end], a[0]
    maxSiftDown(a, 0, end)
  }
}

func maxSiftDown(a []int, i, n int) {
  for {
    left, right := 2*i+1, 2*i+2
    largest := i
    if left < n && a[left] > a[largest] {
      largest = left
    }
    if right < n && a[right] > a[largest] {
      largest = right
    }
    if largest == i {
      return
    }
    a[i], a[largest] = a[largest], a[i]
    i = largest
  }
}
```

listing 191: go, heapsort and its max-heap sift-down (source: dsa/samples-go/ch08/heapsort.go, lines 7-34)

```java
  // sift_down over a[0..len) treating it as a max-heap
  static void siftDown(int[] a, int len, int i) {
    for (;;) {
      int l = 2 * i + 1, r = 2 * i + 2, m = i;
      if (l < len && a[l] > a[m])
        m = l;
      if (r < len && a[r] > a[m])
        m = r;
      if (m == i)
        return;
      swap(a, i, m);
      i = m;
    }
  }

  static void heapsort(int[] a, int len) {
    for (int i = len / 2 - 1; i ≥ 0; i--)
      siftDown(a, len, i);
    for (int end = len - 1; end > 0; end--) {
      swap(a, 0, end);
      siftDown(a, end, 0);
    }
  }
```

listing 192: java, max-heap sift-down, heapify backward, extract to the end (source: dsa/samples-java/src/Ch08/Heapsort.java, lines 24-46)

```
public static class HeapSort
{
    // in-place heapsort: heapify the back, then repeatedly swap root to end
    public static void Sort<T>(T[] items) where T : IComparable<T>
    {
        int n = items.Length;
        for (int i = n / 2 - 1; i ≥ 0; i--)
            SiftDown(items, i, n);
        for (int end = n - 1; end > 0; end--)
        {
            (items[0], items[end]) = (items[end], items[0]);
            SiftDown(items, 0, end);
        }
    }

    private static void SiftDown<T>(T[] items, int i, int size) where T : IComparable<T>
    {
        while (true)
        {
            int left = 2 * i + 1, right = 2 * i + 2, largest = i;
            if (left < size && items[left].CompareTo(items[largest]) > 0)
                largest = left;
            if (right < size && items[right].CompareTo(items[largest]) > 0)
                largest = right;
            if (largest == i)
                break;
            (items[i], items[largest]) = (items[largest], items[i]);
            i = largest;
        }
    }
}
```

listing 193: c#, in-place heapsort, heapify then extract to the back (source: dsa/samples/src/Ch08/Heaps.cs, lines 88-118)

```
function siftDown(a, len, i) {
  for (;;) {
    const l = 2 * i + 1;
    const r = 2 * i + 2;
    let m = i;
    if (l < len && a[l] > a[m]) m = l;
    if (r < len && a[r] > a[m]) m = r;
    if (m === i) return;
    swap(a, i, m);
    i = m;
  }
}

export function heapsort(a) {
  for (let i = Math.floor(a.length / 2) - 1; i ≥ 0; i--)
    siftDown(a, a.length, i);
  for (let end = a.length - 1; end > 0; end--) {
    swap(a, 0, end);
    siftDown(a, end, 0);
  }
  return a;
}
```

listing 194: javascript, heapsort over the in-place max-heap (source: dsa/samples-js/src/ch08-heapsort.mjs, lines 10-31)

```python
def sift_down(a, i, size):
    # max-heap discipline: the larger child climbs past a smaller parent
    while True:
        left, right, big = 2 * i + 1, 2 * i + 2, i
        if left < size and a[left] > a[big]:
            big = left
        if right < size and a[right] > a[big]:
            big = right
        if big == i:
            return
        a[i], a[big] = a[big], a[i]
        i = big


def heapsort(a):
    size = len(a)
    if size == 0:
        return None
    for i in range(size // 2 - 1, -1, -1):  # build: sift the inner nodes
        sift_down(a, i, size)
    peak = a[0]  # remember the build-phase maximum before extracting
    for end in range(size - 1, 0, -1):
        a[0], a[end] = a[end], a[0]  # the root joins the sorted suffix
        sift_down(a, 0, end)
    return peak
```

listing 195: python, sift-down, heapify, extract, the build peak returned (source: dsa/samples-py/src/Ch08/heapsort.py, lines 15-39)

```lua
local function sift_down(a, lo, hi, i)
  while true do
    local l, r, m = 2 * i, 2 * i + 1, i
    if l ≤ hi and a[l] > a[m] then m = l end
    if r ≤ hi and a[r] > a[m] then m = r end
    if m == i then return end
    a[i], a[m] = a[m], a[i]
    i = m
  end
end

local function heapsort(a)
  local n = #a
  for i = n // 2, 1, -1 do sift_down(a, 1, n, i) end
  for last = n, 2, -1 do
    a[1], a[last] = a[last], a[1]
    sift_down(a, 1, last - 1, 1)
  end
  return a
end
```

listing 196: lua, 1-based heapsort, heapify from the middle down (source: dsa/samples-lua/ch08_heapsort.lua, lines 7-26)

It is the honest sort of this book's chapter 13 lineup: worst case linearithmic with no quicksort-style adversarial collapse, entirely in place with no merge-sort auxiliaries, and slower than both on typical data because sift-down touches memory with poor locality. Nobody allocates: each version repairs the one array it was handed.

Measured across the suites: C, Java, JavaScript, and Lua sort the pi fixture 3 1 4 1 5 9 2 6 5 to 1 1 2 3 4 5 5 6 9 and prove the permutation with sum and xor, the two 1s and two 5s counted one by one. Go runs five shapes from reversed to empty against slices.Sort ground truth plus a count map, Python returns the build-phase maximum

on its 7 2 9 4 3 1 fixture, 9, before leaving 1 2 3 4 7 9 behind and cross-checking the permutation with Counter, and C# presses 2000 values drawn from 50 keys plus an all-equal array.

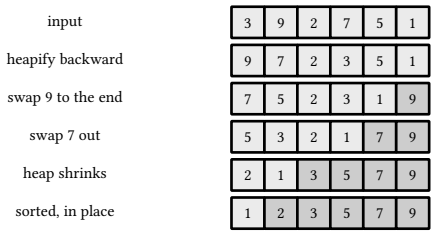


figure 52: heapsort, heapify walks backward from the middle, then each swap parks the maximum at the shrinking end

8.3. streaming top-k

Keep the k largest of a stream without storing it all: a min-heap capped at k, where the root is the entry ticket. A newcomer below the root changes nothing, a newcomer above it evicts the root and sifts down, so the heap always holds the best k so far and its root is the running k-th largest. Cost is log k per value and k of memory, the shape every leaderboard, heavy-hitter counter, and recommendation pre-pass reduces to.

The dry run: the fixture is the stream 4, 9, 1, 7, 3, 8, 6, 5, 2 at k = 3, asserted by the C# suite, while Java pins the same 9 8 7 with the equal-value, short-stream, and k = 1 edge laws, Go pins the same 9 8 7 on its own shuffle, and Python meters four skips and one eviction on its own stream.

1. Fill on 4, 9, 1: the heap reads 1, 9, 4 with 1 at the root, the entry ticket now that the heap is full.
2. Feed 7: 7 > 1 evicts the root, and 7 sifts past 4 into the heap 4, 9, 7, ticket 4.
3. Feed 3: 3 < 4, skip, nothing moves.
4. Feed 8: 8 > 4 evicts, 8 sifts past 7, heap 7, 9, 8, ticket 7.
5. Feed 6, 5, 2: all below 7, three skips.
6. Count 3 with the threshold at 7, and draining largest first reads 9, 8, 7, the k-th largest settled at 7.
7. The pinned edge laws: equal values never displace the ticket, 7, 7, 7, 7 at k = 2 keeps 7, 7, a short stream keeps everything, and k = 1 is a running maximum at threshold 9.

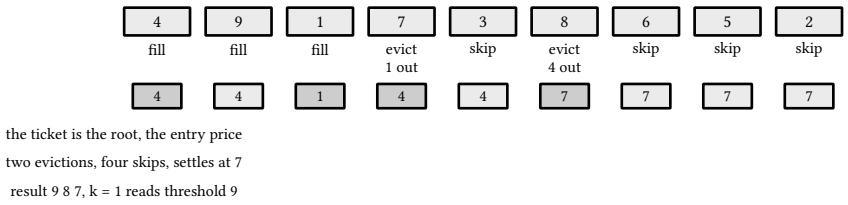


figure 53: the stream as nine arrivals, the verdict and the running ticket under each

The ticket settling at 7 with the drain reading 9, 8, 7 is the pinned landing, and the listings below cap every stream the same way.

```c
static void topk_begin(int kk) {
  k = kk;
  hn = 0;
}

static void topk_feed(int v) {
  if (hn < k) {
    h[hn++] = v;
    sift_up(hn - 1);
    return;
  }
  if (v > h[0]) {
    h[0] = v;
    sift_down(0);
  }
}

// drain and sort descending
static int topk_result(int *out) {
  for (int i = 0; i < hn; i++)
    out[i] = h[i];
  for (int i = 1; i < hn; i++)
    for (int j = i; j > 0 && out[j - 1] < out[j]; j--) {
      int t = out[j - 1];
      out[j - 1] = out[j];
      out[j] = t;
    }
  return hn;
}
```

listing 197: c, begin, feed with fill and evict, drain descending (source: dsa/samples-c/src/Ch08/topk.c, lines 45-73)

```go
func (t *TopK) Add(v int) {
  if t.h.Len() < t.k {
    t.h.Push(v)
    return
  }
  weakest, err := t.h.Peek()
  if err != nil || v <= weakest {
    return // does not displace any survivor
  }
  t.h.Pop()
  t.h.Push(v)
}

// Best drains the survivors largest first.
func (t *TopK) Best() []int {
  out := make([]int, 0, t.h.Len())
  for v, err := t.h.Pop(); err == nil; v, err = t.h.Pop() {
    out = append(out, v)
  }
  // popped ascending: reverse once instead of re-heapifying a max
  for i, j := 0, len(out)-1; i < j; i, j = i+1, j-1 {
    out[i], out[j] = out[j], out[i]
  }
  return out
}
```

listing 198: go, add against the weakest survivor, best drains largest first (source: dsa/samples-go/ch08/topk.go, lines 17-41)

```
static void topkBegin(int kk) {
  k = kk;
  hn = 0;
}

static void topkFeed(int v) {
  if (hn < k) {
    h[hn++] = v;
    siftUp(hn - 1);
    return;
  }
  if (v > h[0]) {
    h[0] = v;
    siftDown(0);
  }
}

// drain and sort descending
static int topkResult(int[] out) {
  for (int i = 0; i < hn; i++)
    out[i] = h[i];
  for (int i = 1; i < hn; i++)
    for (int j = i; j > 0 && out[j - 1] < out[j]; j--) {
      int t = out[j - 1];
      out[j - 1] = out[j];
      out[j] = t;
    }
  return hn;
}
```

listing 199: java, begin, feed with fill and evict, drain descending (source: dsa/samples-java/src/Ch08/Topk.java, lines 46-74)

```
    /// <summary>The heap minimum: the running k-th largest once the heap is full.</summary>
    public int Threshold => _heap[0];

    public void Feed(int v)
    {
        if (_n < _heap.Length)
        {
            _heap[_n++] = v;
            SiftUp(_n - 1);
            return;
        }
        if (v > _heap[0])
        {
            _heap[0] = v;
            SiftDown(0);
        }
    }

    /// <summary>The survivors, largest first.</summary>
    public int[] Result()
    {
        var out_ = new int[_n];
        Array.Copy(_heap, out_, _n);
        Array.Sort(out_, (a, b) => b.CompareTo(a));
        return out_;
    }
```

listing 200: c#, feed with fill and evict, the threshold getter, survivors descending (source: dsa/samples/src/Ch08/TopK.cs, lines 16-42)

```
  feed(v) {
    if (this.#h.length < this.#k) {
      this.#h.push(v);
      let i = this.#h.length - 1;
      while (i > 0 && this.#h[Math.floor((i - 1) / 2)] > this.#h[i]) {
        [this.#h[Math.floor((i - 1) / 2)], this.#h[i]] = [
          this.#h[i],
          this.#h[Math.floor((i - 1) / 2)],
        ];
        i = Math.floor((i - 1) / 2);
      }
      return this;
    }
    if (v > this.#h[0]) {
      this.#h[0] = v;
      this.#siftDown();
    }
    return this;
  }

  // the heap minimum is the k-th largest so far
  get threshold() {
    return this.#h[0];
  }
```

listing 201: javascript, feed with inline sifts, the threshold getter (source: dsa/samples-js/src/ch08-topk.mjs, lines 29-52)

```python
class TopK:
    def __init__(self, k):
        self.k = k
        self.heap = []  # a min-heap of the current best k
        self.skips = 0
        self.evictions = 0

    def feed(self, value):
        if len(self.heap) < self.k:
            self.heap.append(value)
            sift_up(self.heap)
            return
        if value ≤ self.heap[0]:
            self.skips += 1  # cannot beat the k-th best
            return
        self.heap[0] = value  # evict the root, sift the newcomer down
        sift_down(self.heap, len(self.heap))
        self.evictions += 1

    def best(self):
        return sorted(self.heap, reverse=True)
```

listing 202: python, TopK with skip and eviction meters (source: dsa/samples-py/src/Ch08/topk.py, lines 37-57)

```lua
function TopK.feed(t, v)
  local h = t.h
  if #h < t.k then
    h[#h + 1] = v
    sift_up(h, #h)
    return
  end
  if v > h[1] then
    h[1] = v
    sift_down(h, 1)
  end
end

-- drain and sort descending
function TopK.result(t)
  local out = {table.unpack(t.h)}
  for i = 2, #out do
    local j = i
    while j > 1 and out[j - 1] < out[j] do
      out[j - 1], out[j] = out[j], out[j - 1]
      j = j - 1
    end
  end
  return out
end
```

listing 203: lua, feed, evict at the root, result sorted descending (source: dsa/samples-lua/ch08_topk.lua, lines 31-55)

Measured across the suites: C, C#, Java, JavaScript, and Lua stream 4 9 1 7 3 8 6 5 2 with k = 3 and pin 9 8 7, the root settling at 7, the k-th largest so far. Go pins the same 9 8 7 over its own shuffled 1 to 9, and Python's tracker meters the mechanics on 8 3 9 1 7 2 6 5, four skips and one eviction, the root sequence 3 3 7 7 7 7. Three properties recur everywhere: a value equal to the threshold never displaces it, a stream shorter than k keeps everything, and k = 1 degrades to a running maximum.

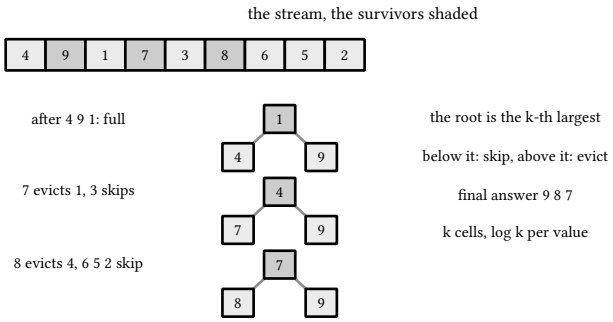


figure 54: streaming top-k, a min-heap of size k, the root is the entry ticket, evictions only from above

8.4. the k-way merge

Merge k sorted runs with a heap of cursors: each cursor holds the next unconsumed element of its run, the heap orders cursors by that element, and each extraction yields the global minimum then reinserts the cursor if its run has more.

The dry run: the fixture is the tie pair, both runs holding 1 2 3, asserted by all seven suites, and the six new trees add the pinned deterministic set, the four-run grid, the empty and lone runs, and the uneven tails, each checked by an oracle lane against the sorted concatenation of the runs.

1. Two cursors open in the heap, one per run, each holding its run's head, and ties break to the lower run index.
2. Both hold 1: run 0 wins the tie, emits 1, and its cursor reenters holding 2.
3. Run 1's 1 is now the minimum: it emits, and the cursors hold 2 and 2.
4. The 2s and 3s follow the same way, run 0 always first at each tie: 1 1 2 2 3 3, stability pinned.
5. An empty run never enters the heap: no runs emit nothing, and a lone run of 4 5 emits 4 5 unchanged.
6. Scaled up on the C# side, 4 seeded runs of 200 values merge to exactly the sort of the 800-value concatenation, 4 x 200 = 800. The seeded random lane stays C# only, the other trees pin deterministic fixtures instead.

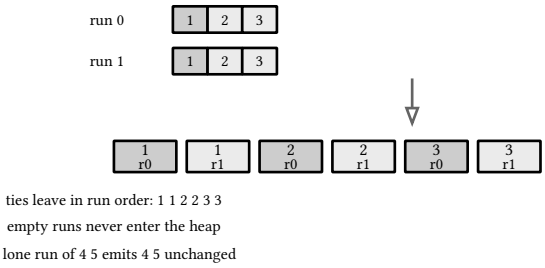


figure 55: the tie fixture as an emission sequence, run 0 first whenever the values tie

The 1 1 2 2 3 3 with run 0 leading every tie is the pinned order, and the listings below are the merges that produced it.

```c
// merge nruns sorted runs into out, returns the output length
static int kway_merge(const int *const *runs, const int *lens, int nruns,
                      int *out) {
  heap_size = 0;
  for (int r = 0; r < nruns; r++) {
    if (lens[r] == 0)
      continue; // empty runs never enter the heap
    cursor head = {runs[r][0], r, 0};
    heap_push(head);
  }
  int m = 0;
  while (heap_size > 0) {
    cursor min = heap_pop();
    out[m++] = min.value;
    if (min.pos + 1 < lens[min.run]) {
      cursor next = {runs[min.run][min.pos + 1], min.run, min.pos + 1};
      heap_push(next);
    }
  }
  return m;
}

static int cmp_int(const void *a, const void *b) {
  int x = *(const int *)a, y = *(const int *)b;
  return (x > y) - (x < y);
}

// oracle: the merged output equals the sorted concatenation of the runs
static int matches_sorted(const int *const *runs, const int *lens, int nruns,
                          const int *out, int m) {
  int all[MAX_OUT], k = 0;
  for (int r = 0; r < nruns; r++)
    for (int i = 0; i < lens[r]; i++)
      all[k++] = runs[r][i];
  if (k != m)
    return 0;
  qsort(all, (size_t)k, sizeof(int), cmp_int);
  return memcmp(all, out, (size_t)k * sizeof(int)) == 0;
}
```

listing 204: c, the cursor merge loop and its sorted-concatenation oracle (source: dsa/samples-c/src/Ch08/kway.c, lines 76-114)

```go
// KWay merges sorted runs into one sorted output. The heap holds one
// cursor per non exhausted run, keyed by value then run index, so ties
// keep run order and empty runs never enter.
func KWay(runs [][]int) []int {
  var heap cursorHeap
  for i, run := range runs {
    if len(run) > 0 {
      heap.Push(mergeCursor{value: run[0], run: i, next: 1})
    }
  }
  var out []int
  for heap.Len() > 0 {
    min := heap.Pop()
    out = append(out, min.value)
    if min.next < len(runs[min.run]) {
      heap.Push(mergeCursor{
        value: runs[min.run][min.next],
        run:   min.run,
        next:  min.next + 1,
      })
    }
  }
  return out
}
```

listing 205: go, k-way over the cursor heap, ties keyed by run index (source: dsa/samples-go/ch08/kway.go, lines 61-84)

The other five languages build the same merge on the chapter's own heap, Java's cursors records breaking ties by run index:

```
  static int kwayMerge(int[][] runs, int[] lens, int nruns, int[] out) {
    heapSize = 0;
    for (int r = 0; r < nruns; r++) {
      if (lens[r] == 0)
        continue; // empty runs never enter the heap
      heapPush(new Cursor(runs[r][0], r, 0));
    }
    int m = 0;
    while (heapSize > 0) {
      Cursor min = heapPop();
      out[m++] = min.value();
      if (min.pos() + 1 < lens[min.run()])
        heapPush(new Cursor(runs[min.run()][min.pos() + 1], min.run(), min.pos() + 1));
    }
    return m;
  }

  // oracle: the merged output equals the sorted concatenation of the runs
  static boolean matchesSorted(int[][] runs, int[] lens, int nruns, int[] out, int m) {
    int[] all = new int[MAX_OUT];
    int k = 0;
    for (int r = 0; r < nruns; r++)
      for (int i = 0; i < lens[r]; i++)
        all[k++] = runs[r][i];
    if (k != m)
      return false;
    Arrays.sort(all, 0, k);
    return Arrays.equals(Arrays.copyOf(all, k), Arrays.copyOf(out, k));
  }
```

listing 206: java, the cursor merge loop over record cursors and its sorted-concatenation oracle (source: dsa/samples-java/src/Ch08/Kway.java, lines 71-99)

```
/// <summary>K-way merge: the heap algorithm the capstone compaction runs.</summary>
public static class Merge
{
    public static IEnumerable<T> KWay<T>(params IEnumerable<T>[] runs)
        where T : IComparable<T> =>
        KWayInner(runs);

    public static IEnumerable<T> KWayInner<T>(IEnumerable<T>[] runs)
        where T : IComparable<T>
    {
        var heap = new BinaryHeap<Cursor<T>>();
        for (int i = 0; i < runs.Length; i++)
        {
            var cursor = new Cursor<T>(runs[i].GetEnumerator(), i);
            if (cursor.MoveNext())
                heap.Add(cursor);
        }
        while (heap.Count > 0)
        {
            Cursor<T> min = heap.ExtractMin();
            yield return min.Current!;
            if (min.MoveNext())
                heap.Add(min);
        }
    }

    private sealed class Cursor<T>(IEnumerator<T> source, int run)
        : IComparable<Cursor<T>> where T : IComparable<T>
    {
        private readonly IEnumerator<T> _source = source;
        private readonly int _run = run;
        public T? Current { get; private set; }

        public int CompareTo(Cursor<T>? other)
        {
            int c = Current!.CompareTo(other!.Current!);
            return c != 0 ? c : _run.CompareTo(other._run); // ties keep run order
        }

        public bool MoveNext()
        {
            if (_source.MoveNext())
            {
                Current = _source.Current;
                return true;
            }
            return false;
        }
    }
}
```

listing 207: c#, k-way merge over cursors, run-index tie break (source: dsa/samples/src/Ch08/Heaps.cs, lines 120-169)

```javascript
export function kwayMerge(runs) {
  const heap = [];
  for (let i = 0; i < runs.length; i++) {
    if (runs[i].length > 0) heapPush(heap, [runs[i][0], i, 0]);
  }
  const out = [];
  while (heap.length > 0) {
    const [, run, pos] = heapPop(heap);
    out.push(runs[run][pos]);
    if (pos + 1 < runs[run].length) {
      heapPush(heap, [runs[run][pos + 1], run, pos + 1]);
    }
  }
  return out;
}
```

listing 208: javascript, the merge loop over value, run, position cursors (source: dsa/samples-js/src/ch08-kway.mjs, lines 47-61)

```python
def kway(runs):
    heap = CursorHeap()
    pos = [0] * len(runs)
    for run, data in enumerate(runs):
        if data:  # empty runs never enter the heap
            heap.push((data[0], run))
    out = []
    while len(heap):
        value, run = heap.pop()
        out.append(value)
        pos[run] += 1
        if pos[run] < len(runs[run]):
            heap.push((runs[run][pos[run]], run))  # else: exhausted, stays out
    return out


FIXTURES = [
    [[1, 2, 3], [1, 2, 3]],
    [[1, 4, 7, 10], [2, 5, 8, 11], [3, 6, 9, 12], [0, 13]],
    [[], [4, 5]],
    [[], []],
    [[7]],
    [[1, 10, 100], [2, 20], [3]],
]
EXPECTED = [
    [1, 1, 2, 2, 3, 3],
    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13],
    [4, 5],
    [],
    [7],
    [1, 2, 3, 10, 20, 100],
]
```

listing 209: python, the cursor heap merge with its pinned fixtures (source: dsa/samples-py/src/Ch08/kway.py, lines 57-88)

```
  for r, run in ipairs(runs) do
    if #run > 0 then push({run[1], r, 1}) end -- empty runs never enter
  end
  local out = {}
  while #heap > 0 do
    local cur = pop()
    out[#out + 1] = cur[1]
    local run = runs[cur[2]]
    if cur[3] < #run then -- the run still has elements
      push({run[cur[3] + 1], cur[2], cur[3] + 1})
    end
  end
  return out
end

-- in-test oracle: merging must equal sorting the concatenation
local function sorted_concat(runs)
  local all = {}
  for _, run in ipairs(runs) do
    for _, v in ipairs(run) do all[#all + 1] = v end
  end
  table.sort(all)
  return all
end
```

listing 210: lua, the merge loop and the sorted-concatenation oracle (source: dsa/samples-lua/ch08_kway.lua, lines 42-65)

Measured across the suites: the tie pair drains to 1 1 2 2 3 3 with run 0 leading every tie in all seven languages. The six new trees also merge the four-run grid to 0 through 13, pass the empty runs and the lone run straight through with empty input to empty output, and interleave the uneven tails to 1 2 3 10 20 100, each fixture checked against the sorted concatenation. C# runs that same oracle on its seeded four-run lane.

The cost is n log k, not n log n, k being the run count, and the tie break on run index makes the merge stable, equal elements leave in run order, which every suite pins. This exact routine is the capstone's compaction pass, merging sorted segment files, and it is also how external database sorts are built. The comparators put value first and run index second everywhere, and the heaps are the chapter's own binary heap, no shipped priority queue anywhere.

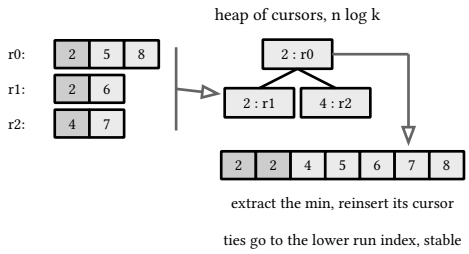


figure 56: the k-way merge, a heap of cursors over sorted runs, extraction yields the global minimum

8.5. the bcl priority queue

aspect	chapter 8 heap	priorityqueue<t>
children	2	4, one less sift level
enqueuedequeue	two calls	one fused sift
decrease key	not built in	remove then reenqueue, O(n)
ties	no promise	no fifo guarantee

figure 57: the shipped heap against the teaching one, wider nodes, fused sifts, remove emulating decrease key

`PriorityQueue<TElement,TPriority>` is an array-backed quaternary min-heap, four children per node for one less level of sift. It dequeues the lowest priority value first and does not guarantee fifo among equals. Two fused operations exist for the hot loop, `EnqueueDequeue` adds then extracts in one sift and `DequeueEnqueue` extracts then adds, both documented as cheaper than the two calls they replace.

NOTE: decrease-key, emulated in .NET 9

Array heaps cannot update a buried element's priority without a position index, so `PriorityQueue` gained `Remove` in .NET 9: a linear scan that extracts the element, then you reenqueue it at the new priority. It is O(n), and the docs say plainly that this unblocks dijkstra-style algorithms for education and prototyping where the asymptote does not hurt. The tests exercise it as the documented emulation. Serious graph work keeps its own heap with a position map, which is what chapter 11's dijkstra does.

The dry run: the fixture is the shipped-queue trio asserted by the C# suite, priorities 10, 1, 5 through dequeue, then 1 and 3 under one fused call, then b at 20 among 10, 20, 30.

1. Enqueue low at 10, high at 1, mid at 5: three nodes fill two levels, a quaternary root holds 4 children, 1 + 4 = 5 slots before a third level exists.
2. Dequeue drains by priority: high, mid, low.
3. EnqueueDequeue(2, 2) on a queue holding 1 and 3 adds 2 and extracts the minimum in one sift: it returns 1.
4. The queue then dequeues 2 and 3, the fused call replacing two full operations.
5. Remove emulates decrease-key: with a at 10, b at 20, c at 30, Remove("b") scans, extracts b, reports oldPriority 20, and reenqueue at 1 puts b at the front.
6. Dequeue returns b first, and Remove("missing") returns false.

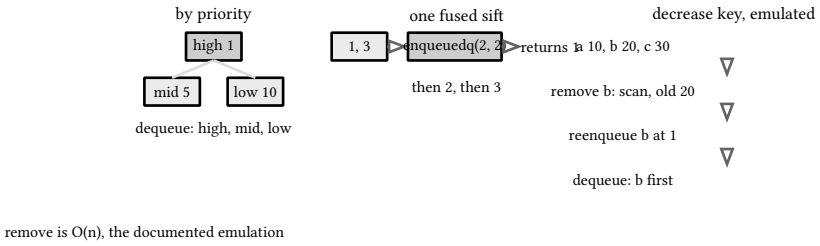


figure 58: the shipped queue in three moments, priority order out, the fused sift, the remove-then-reenqueue path

The b leaving first after its 20 became 1 is the pinned emulation, the trade the table above prices at O(n).

8.6. across the seven languages

The build sizes count non-comment source lines over this chapter's four featured files per language. Where a language bundles its checks into the same file the count carries them, and no shipped priority queue appears anywhere in the featured code:

language	build SLOC	dependency	boundary note
c	417	libc only	fixed static capacity, 64 heap slots, 16 top-k slots, 8 merge cursors, checks share the file with main, 65 of them
go	190	fmt for errors	container/heap named as the stdlib counterpart and left unused, tests in separate files, errors returned not thrown, 12 tests
java	405	jdk 27 stdlib	java.util.PriorityQueue named as the stdlib counterpart and left unused, heaps over one static array, cursors as records breaking ties by run index, 65 checks in 4 files
c#	201	bcl only	the k-way merge and the streaming top-k ride the file set, 17 tests
javascript	180	node stdlib	private class fields over one array, tests in test/, 14 tests

language	build SLOC	dependency	boundary note
python	256	stdlib only	heapq named as the stdlib counterpart and left unused, native ints, empty pop raises IndexError, 43 checks
lua	310	lib.lua harness	1-based heaps shift every index formula, checks ride in the module, 18 of them

sources: learn.microsoft.com, `PriorityQueue<TElement,TPriority>` class remarks on the quaternary heap and fifo caveats, `EnqueueDequeue` and `DequeueEnqueue` method pages, `Remove` method page, what's new in .NET 9 libraries for the decrease-key notes, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 17 tests in chapter 8 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 65 embedded checks under `make verify-c`, 12 Go tests, the java runner's 65 Ch08 checks over 4 files under `run-java-samples`, 14 `node --test` cases, 43 Python checks across 4 files, and 18 Lua checks under `run.lua`.

9. tries and suffix structures

Hash tables answer point queries. Tries answer queries about structure: which keys share this prefix, how many, what stored key matches the longest run of this input. The cost is per byte walked, never a comparison between whole keys, and no hash can do any of it.

9.1. the byte-wise trie

Each node is a map from one byte to the next node, and a key is the path spelling its bytes. End markers turn shared paths into distinct keys. Several of the seven builds carry a subtree size on every node, which is what makes count queries constant after the walk.

The dry run: the fixture is cat, car, cart, asserted by the C# suite, while C, JavaScript, and Lua build cat car cart dog done do and walk the same queries, and Java walks a four-key cat car cart dog with the size counters on every node.

1. Insert cat: root grows c, a, t, the end marker lands on t, count 1.
2. Insert car: the walk reuses c and a, extends one r, marker on r, count 2.
3. Insert cart: the walk reuses c, a, r and the marker already there, extends one t, count 3.
4. Contains walks without inserting: car and cart hit their markers, ca stops at a node holding no marker, carts runs past the last node.
5. Remove(“car”) clears the marker on r and keeps the node, cart’s path still passes through: count 2, and a second Remove(“car”) returns false.
6. The size counters track the same story on ab, abc: SizeOfPrefix of a reads 2, and Remove(ab) drops it to 1.

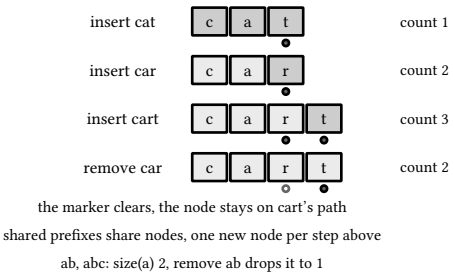


figure 59: the three inserts as path frames, shared nodes reused and new nodes shaded, then the removal that keeps the path

The count landing at 2 with cart intact is the pinned pair, and the listings below build and unbuild the same paths.

```c
static void trie_insert(const char *w) {
  TNode *t = root;
  t->words_below++;
  for (const char *p = w; *p; p++) {
    int c = *p - 'a';
    if (!t->ch[c])
      t->ch[c] = tnew();
    t = t->ch[c];
    t->words_below++;
  }
  t->word = 1;
}

static TNode *walk(const char *pfx) {
  TNode *t = root;
  for (const char *p = pfx; *p && t; p++)
    t = t->ch[*p - 'a'];
  return t;
}

// exact word search, not a prefix hit
static int trie_search(const char *w) {
  TNode *t = walk(w);
  return t && t->word;
}

static int trie_starts_with(const char *pfx) { return walk(pfx) != NULL; }

static int trie_count_under(const char *pfx) {
  TNode *t = walk(pfx);
  return t ? t->words_below : 0;
}
```

listing 211: c, a 26-wide child array from a static pool, insert, walk, the three queries (source: dsa/samples-c/src/Ch09/trie.c, lines 40-71)

```go
func (t *Trie) Insert(word string) {
  n := t.root
  for i := 0; i < len(word); i++ {
    c := word[i]
    next, ok := n.children[c]
    if !ok {
      next = &TrieNode{children: map[byte]*TrieNode{}}
      n.children[c] = next
    }
    n = next
  }
  if !n.word {
    n.word = true
    t.n++
  }
}

// Search reports whether word was stored whole. A stored prefix of a
// longer word does not count.
func (t *Trie) Search(word string) bool {
  n := t.walk(word)
  return n ≠ nil && n.word
}

// StartsWith reports whether any stored word begins with prefix.
func (t *Trie) StartsWith(prefix string) bool {
  return t.CountPrefix(prefix) > 0
}

// CountPrefix counts stored words under prefix, prefix included.
func (t *Trie) CountPrefix(prefix string) int {
  n := t.walk(prefix)
  if n == nil {
    return 0
  }
  return countWords(n)
}
```

listing 212: go, insert, search, starts-with, count by recursion (source: dsa/samples-go/ch09/trie.go, lines 23-59)

```
  static final class TNode {
    boolean word; // set when a word ends here
    int wordsBelow; // words in the subtree rooted here
    TNode[] ch = new TNode[26];
  }

  static TNode root;

  static void trieInit() {
    root = new TNode();
  }

  static void trieInsert(String w) {
    TNode t = root;
    t.wordsBelow++;
    for (int i = 0; i < w.length(); i++) {
      int c = w.charAt(i) - 'a';
      if (t.ch[c] == null)
        t.ch[c] = new TNode();
      t = t.ch[c];
      t.wordsBelow++;
    }
    t.word = true;
  }

  static TNode walk(String pfx) {
    TNode t = root;
    for (int i = 0; i < pfx.length() && t != null; i++)
      t = t.ch[pfx.charAt(i) - 'a'];
    return t;
  }

  // exact word search, not a prefix hit
  static boolean trieSearch(String w) {
    TNode t = walk(w);
    return t != null && t.word;
  }

  static boolean trieStartsWith(String pfx) {
    return walk(pfx) != null;
  }

  static int trieCountUnder(String pfx) {
    TNode t = walk(pfx);
    return t == null ? 0 : t.wordsBelow;
  }
```

listing 213: java, 26-wide child arrays, insert with subtree counters, the three queries (source: dsa/samples-java/src/Ch09/Trie.java, lines 17-62)

The C# class carries the full contract:

```
public sealed class Trie
{
    private sealed class Node
    {
        public readonly Dictionary<byte, Node> Children = new();
        public bool IsEnd;
        public int Size; // subtree key count, maintained on insert and remove
    }

    private readonly Node _root = new();

    public int Count { get; private set; }

    public void Insert(ReadOnlySpan<byte> key)
    {
        Node node = WalkOrExtend(_root, key);
        if (!node.IsEnd)
        {
            node.IsEnd = true;
            Count++;
            BumpPrefix(key, 1);
        }
    }

    public bool Remove(ReadOnlySpan<byte> key)
    {
        Node? node = TryWalk(_root, key);
        if (node is not { IsEnd: true })
            return false;
        node.IsEnd = false;
        Count--;
        BumpPrefix(key, -1);
        return true;
    }

    public bool Contains(ReadOnlySpan<byte> key) =>
        TryWalk(_root, key) is { IsEnd: true };

    public long SizeOfPrefix(ReadOnlySpan<byte> prefix) =>
        TryWalk(_root, prefix)?.Size ?? 0;

    /// <summary>All keys that start with the prefix, in sorted order.</summary>
    public List<byte[]> KeysWithPrefix(ReadOnlySpan<byte> prefix)
    {
        var results = new List<byte[]>();
        Node? start = TryWalk(_root, prefix);
        if (start == null)
            return results;
        Collect(start, prefix.ToArray(), results);
        return results;
    }
```

listing 214: c#, insert, remove, contains, size-of-prefix, keys-with-prefix (source: dsa/samples/src/Ch09/Tries.cs, lines 4-54)

Insertion walks or extends the path, then bumps the size counters down that path, one line each way. `KeysWithPrefix` walks to the prefix node and collects everything below it:

```
    public byte[]? LongestPrefixOf(ReadOnlySpan<byte> query)
    {
        Node node = _root;
        byte[]? best = node.IsEnd ? Array.Empty<byte>() : null;
        var walked = new List<byte>();
        foreach (byte b in query)
        {
            if (!node.Children.TryGetValue(b, out Node? next))
                break;
            walked.Add(b);
            node = next;
            if (node.IsEnd)
                best = walked.ToArray();
        }
        return best;
    }

    // the two walk contracts live in the return types: extending
    // always lands on a node, a lookup walk may not
    private static Node WalkOrExtend(Node root, ReadOnlySpan<byte> key)
    {
        Node node = root;
        foreach (byte b in key)
        {
            if (!node.Children.TryGetValue(b, out Node? next))
            {
                next = new Node();
                node.Children[b] = next;
            }
            node = next;
        }
        return node;
    }

    private static Node? TryWalk(Node root, ReadOnlySpan<byte> key)
    {
        Node node = root;
        foreach (byte b in key)
        {
            if (!node.Children.TryGetValue(b, out Node? next))
                return null;
            node = next;
        }
        return node;
    }

    // increment Size on every node of the key path, including the root
    private void BumpPrefix(ReadOnlySpan<byte> key, int delta)
    {
        Node node = _root;
        node.Size += delta;
        foreach (byte b in key)
        {
            node = node.Children[b];
            node.Size += delta;
        }
    }
```

```
    private static void Collect(Node node, byte[] soFar, List<byte[]> into)
    {
        if (node.IsEnd)
            into.Add(soFar);
        // dictionary iteration is by key order here because byte keys hash
        // to themselves in the default comparer, but sort for the guarantee
        foreach (var (b, child) in node.Children.OrderBy(kv => kv.Key))
            Collect(child, [.. soFar, b], into);
    }
}
```

listing 215: c#, longest matching prefix, the two walk contracts, size maintenance, sorted collect (source: dsa/samples/src/Ch09/Tries.cs, lines 57-124)

Three more languages build the same paths:

```
  insert(w) {
    let t = this.#root;
    t.wordsBelow++;
    for (const c of w) {
      if (!t.ch[c]) t.ch[c] = { word: false, wordsBelow: 0, ch: {} };
      t = t.ch[c];
      t.wordsBelow++;
    }
    t.word = true;
    return this;
  }

  #walk(pfx) {
    let t = this.#root;
    for (const c of pfx) {
      if (!t.ch[c]) return null;
      t = t.ch[c];
    }
    return t;
  }

  // exact word search, not a prefix hit
  search(w) {
    const t = this.#walk(w);
    return t !== null && t.word;
  }

  startsWith(pfx) {
    return this.#walk(pfx) !== null;
  }

  // the subtree root under pfx, or null: the raw node for scans
  nodeAt(pfx) {
    return this.#walk(pfx);
  }

  countUnder(pfx) {
    const t = this.#walk(pfx);
    return t ? t.wordsBelow : 0;
  }
```

listing 216: javascript, insert with per-node counters, the walk, search and count (source: dsa/samples-js/src/ch09-trie.mjs, lines 7-46)

```
    def insert(self, word):
        node = self.root
        path = [node]
        for ch in word:
            if ch not in node.children:
                node.children[ch] = TrieNode()
            node = node.children[ch]
            path.append(node)
        if not node.is_word:
            for p in path:  # the count climbs back up the spine
                p.words_below += 1
        node.is_word = True

    def _walk(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return None
        return node

    def search(self, word):
        node = self._walk(word)
        return node is not None and node.is_word

    def starts_with(self, prefix):
        return self._walk(prefix) is not None

    def count_under(self, prefix):
        node = self._walk(prefix)
        return 0 if node is None else node.words_below
```

listing 217: python, slotted nodes, idempotent insert, walk, the queries (source: dsa/samples-py/src/Ch09/trie.py, lines 26-56)

```lua
local function trie_insert(root, w)
  local t = root
  t.words_below = t.words_below + 1
  for i = 1, #w do
    local c = w:sub(i, i)
    if not t.ch[c] then t.ch[c] = trie_new() end
    t = t.ch[c]
    t.words_below = t.words_below + 1
  end
  t.word = true
end

local function walk(root, pfx)
  local t = root
  local i = 1
  while t and i ≤ #pfx do
    t = t.ch[pfx:sub(i, i)]
    i = i + 1
  end
  return t
end

-- exact word search, not a prefix hit
local function trie_search(root, w)
  local t = walk(root, w)
  return t ~= nil and t.word or false
end

local function trie_starts_with(root, pfx)
  return walk(root, pfx) ~= nil
end

local function trie_count_under(root, pfx)
  local t = walk(root, pfx)
  return t and t.words_below or 0
end
```

listing 218: lua, one-character string keys, insert, walk, the queries (source: dsa/samples-lua/ch09_trie.lua, lines 9-44)

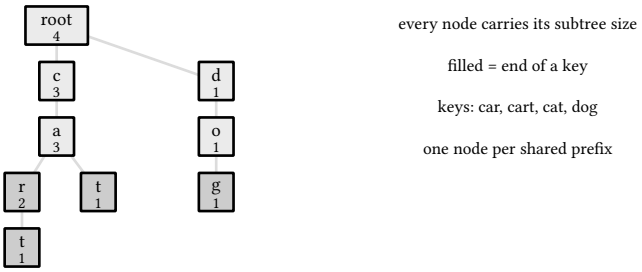


figure 60: the byte-wise trie, shared prefixes share nodes, every node carries its subtree size

`LongestPrefixOf` is the router's question, the longest stored route matching an address, answered by walking the query and remembering the last end marker seen. The tests pin all the boundary behaviors: removing `car` leaves `cart` intact, the empty key is legal, and binary keys work byte for byte with no text encoding involved, which matters because the capstone indexes arbitrary byte keys exactly this way.

NOTE: one node per distinct prefix

> A trie of n keys of length L costs at most n times L nodes and usually far less when keys share prefixes. The cost of the whole family is memory shape: a node with 256 byte children as an array is fast and enormous, as a dictionary it is compact with one more indirection. The version here uses the dictionary form and the collect sorts children explicitly rather than trusting dictionary iteration order, a lesson from book 5′s collection semantics.

9.2. the prefix scan

The count says how many, the scan says which: enumerate every stored key under a prefix, in lexicographic order, without sorting anything after the fact. The walk descends to the prefix node, then emits depth-first with children visited in ascending order, so the output is sorted by construction and a word always precedes its own extensions.

The dry run: the fixture is banana, band, bandana, bar, bee, asserted by the C# suite, while C, Java, JavaScript, and Lua scan cat car cart dog done do and Go pins app apple apron under ap.

1. Build the five keys; nothing is sorted anywhere yet.
2. The scan under ban walks root, b, a, n and lands on the ban node.
3. Below it the children split a, carrying banana, and d, carrying band and bandana, and a sorts first.
4. The depth-first walk emits banana off the a branch, then band and bandana off the d branch, a word before its own extension.
5. The output reads banana, band, bandana with no sort pass, the pinned order.
6. SizeOfPrefix reads cached counters on the same walk: ban 3, zz 0, and b at 5, the whole vocabulary.

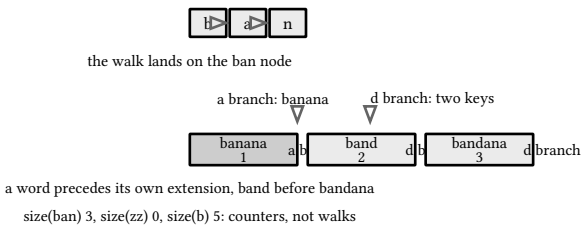


figure 61: the scan under ban, the walk to the prefix node, then the depth-first emission with the a branch ahead of d

The banana, band, bandana order off scrambled input is the pinned scan, and the listings below emit it without sorting anything.

```c
// depth-first from a node; a-to-z child order yields lexicographic output
static void emit(TNode *t, int depth) {
  if (t->word) {
    path[depth] = 0;
    copy_str(results[rn++], path);
  }
  for (int c = 0; c < 26; c++)
    if (t->ch[c]) {
      path[depth] = (char)('a' + c);
      emit(t->ch[c], depth + 1);
    }
}

static int scan(const char *pfx) {
  TNode *t = root;
  for (const char *p = pfx; *p && t; p++)
    t = t->ch[*p - 'a'];
  rn = 0;
  if (!t)
    return 0;
  copy_str(path, pfx);
  emit(t, (int)strlen(pfx));
  return rn;
}
```

listing 219: c, depth-first emit over the a-to-z child array, scan walks to the prefix (source: dsa/samples-c/src/Ch09/prefixscan.c, lines 62-85)

```go
func (t *Trie) KeysWithPrefix(prefix string) []string {
  var out []string
  n := t.walk(prefix)
  if n == nil {
    return out
  }
  var scan func(*TrieNode, []byte)
  scan = func(n *TrieNode, path []byte) {
    if n.word {
      out = append(out, string(path))
    }
    edges := make([]byte, 0, len(n.children))
    for c := range n.children {
      edges = append(edges, c)
    }
    slices.Sort(edges)
    for _, c := range edges {
      scan(n.children[c], append(path, c))
    }
  }
  scan(n, []byte(prefix))
  return out
}
```

listing 220: go, the closure scan, edges sorted so map order cannot leak (source: dsa/samples-go/ch09/prefixscan.go, lines 9-31)

```
// depth-first from a node; a-to-z child order yields lexicographic output
static void emit(TNode t) {
  if (t.word)
    results.add(path.toString());
  for (int c = 0; c < 26; c++)
    if (t.ch[c] != null) {
      path.append((char) ('a' + c));
      emit(t.ch[c]);
      path.deleteCharAt(path.length() - 1);
    }
}

static List<String> scan(String pfx) {
  TNode t = root;
  for (int i = 0; i < pfx.length() && t != null; i++)
    t = t.ch[pfx.charAt(i) - 'a'];
  results = new ArrayList<>();
  if (t == null)
    return results;
  path = new StringBuilder(pfx);
  emit(t);
  return results;
}
```

listing 221: java, depth-first emit over the a-to-z child array, a shared StringBuilder path (source: dsa/samples-java/src/Ch09/Prefixscan.java, lines 42-64)

```
  /// <summary>All keys that start with the prefix, in sorted order.</summary>
  public List<byte[]> KeysWithPrefix(ReadOnlySpan<byte> prefix)
  {
      var results = new List<byte[]>();
      Node? start = TryWalk(_root, prefix);
      if (start == null)
          return results;
      Collect(start, prefix.ToArray(), results);
      return results;
  }
```

listing 222: c#, keys-with-prefix walks then collects (source: dsa/samples/src/Ch09/Tries.cs, lines 45-54)

```
  private static void Collect(Node node, byte[] soFar, List<byte[]> into)
  {
      if (node.IsEnd)
          into.Add(soFar);
      // dictionary iteration is by key order here because byte keys hash
      // to themselves in the default comparer, but sort for the guarantee
      foreach (var (b, child) in node.Children.OrderBy(kv => kv.Key))
          Collect(child, [.. soFar, b], into);
  }
```

listing 223: c#, the recursive collect, children sorted for the guarantee (source: dsa/samples/src/Ch09/Tries.cs, lines 115-123)

```javascript
export function scanTrie(trie, pfx) {
  const root = trie.nodeAt(pfx);
  const out = [];
  if (!root) return out;
  const emit = (node, path) => {
    if (node.word) out.push(path);
    for (const c of Object.keys(node.ch).sort()) emit(node.ch[c], path + c);
  };
  emit(root, pfx);
  return out;
}
```

listing 224: javascript, the emit closure over sorted object keys (source: dsa/samples-js/src/ch09-prefixscan.mjs, lines 7-17)

```python
    def scan(self, prefix):
        node = self.root
        for ch in prefix:
            node = node.children.get(ch)
            if node is None:
                return []
        out = []

        def descend(node, acc):
            if node.is_word:
                out.append(prefix + acc)
            for ch in sorted(node.children):  # alphabetical child order
                descend(node.children[ch], acc + ch)

        descend(node, "")
        return out
```

listing 225: python, walk, then descend with sorted children (source: dsa/samples-py/src/Ch09/prefixscan.py, lines 31-46)

```lua
local function emit(t, path, results)
  if t.word then results[#results + 1] = table.concat(path) end
  local kids = {}
  for c in pairs(t.ch) do kids[#kids + 1] = c end
  table.sort(kids)
  for _, c in ipairs(kids) do
    path[#path + 1] = c
    emit(t.ch[c], path, results)
    path[#path] = nil
  end
end

local function scan(root, pfx)
  local t = root
  for i = 1, #pfx do
    t = t and t.ch[pfx:sub(i, i)] or nil
  end
  local results = {}
  if not t then return results end
  local path = {}
  for i = 1, #pfx do path[i] = pfx:sub(i, i) end
  emit(t, path, results)
  return results
end
```

listing 226: lua, emit with a shared path buffer, sorted child keys (source: dsa/samples-lua/ch09_prefixscan.lua, lines 21-44)

Measured across the suites: C, Java, JavaScript, and Lua scan a build inserted as cat car cart dog done do and pin car cart cat under ca, do dog done under do, and all six keys sorted under the empty prefix, the insertion order deliberately scrambled against the output. Python scans banana band bandana bin bard, ban yields banana band bandana, and Go pins app apple apron under ap on its own fixture. The order is structural everywhere: ascending children make the depth-first walk lexicographic, and no suite calls a sort on its result.

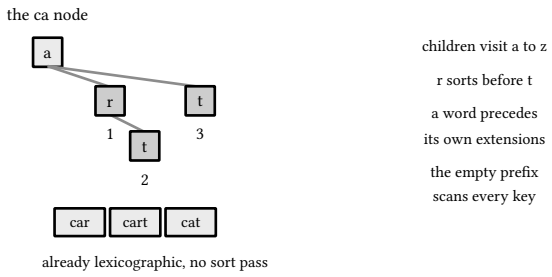


figure 62: the prefix scan under ca, depth-first with children visited a to z emits car, cart, cat

9.3. the suffix array

The complementary structure indexes one text's suffixes rather than many keys. Sorting the start positions of all suffixes lexicographically gives an array where any substring query becomes a range of adjacent suffixes.

The dry run: the fixtures are banana, aaaa, abab, and mississippi, asserted by all seven suites on the same two arrays. The tie contract runs the whole table on repeated characters, a before aa, and the Kasai walk is checked against direct adjacent-pair comparison everywhere.

1. The six suffixes by start: 0 banana, 1 anana, 2 nana, 3 ana, 4 na, 5 a.
2. Sorted lexicographically the starts read 5, 3, 1, 0, 4, 2: a, ana, anana, banana, na, nana, the pinned array.
3. Kasai walks text order, not sorted order: at start 0, banana's sorted predecessor anana shares nothing, h = 0.
4. At start 1, anana meets its predecessor ana and the loop climbs a, n, a: h = 3, then h− carries 2 forward.

5. At start 2 the carried 2 is already the whole answer against na, and at start 3 the carried 1 is the answer against a: no climbing, h− each time.
6. At start 4, na meets banana, nothing shared, and start 5′s suffix is first in the array with no predecessor.
7. The lcp lands 1, 3, 0, 0, 2 in sorted order, pinned, and the h walk 0, 3, 2, 1, 0 climbs once and only steps down.

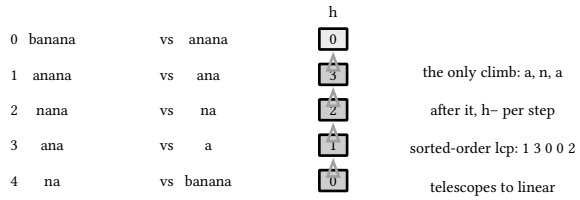


figure 63: kasai's walk in text order, h climbing once to 3 then stepping down 2, 1, 0, each carried value already the answer

The 1, 3, 0, 0, 2 is the pinned array and the climb-then-step-down is the reason it costs one pass, the listings below carry both.

```c
// full lexicographic suffix order: characters to the end, then length,
// so on repeated characters the shorter suffix with an equal prefix
// sorts first
static int cmp_suffix(const void *pa, const void *pb) {
  int i = *(const int *)pa, j = *(const int *)pb;
  int len = (int)strlen(text);
  int li = len - i, lj = len - j;
  int m = li < lj ? li : lj;
  int c = memcmp(text + i, text + j, (size_t)m);
  if (c != 0)
    return c;
  return li - lj;
}

// starting positions of the suffixes in lexicographic order
static void build_array(const char *t, int *sa, int len) {
  text = t;
  for (int i = 0; i < len; i++)
    sa[i] = i;
  qsort(sa, (size_t)len, sizeof(int), cmp_suffix);
}

// kasai: the lcp of adjacent sorted suffixes can only drop by one per
// character step, so a left-to-right walk with the h-1 invariant is
// linear in total
static void build_lcp(const char *t, const int *sa, int *lcp, int len) {
  int rank[MAX_TEXT];
  for (int i = 0; i < len; i++)
    rank[sa[i]] = i;
  for (int i = 0; i + 1 < len; i++)
    lcp[i] = 0;
  int h = 0;
  for (int i = 0; i < len; i++) {
    if (rank[i] > 0) {
      int j = sa[rank[i] - 1];
      while (i + h < len && j + h < len && t[i + h] == t[j + h])
        h++;
      lcp[rank[i] - 1] = h;
      if (h > 0)
        h--;
    } else {
      h = 0;
    }
  }
}
```

listing 227: c, the length-aware suffix comparator, the sorted array, kasai (source: dsa/samples-c/src/Ch09/suffixarray.c, lines 23-67)

```go
// BuildSuffixArray returns the starting positions of the suffixes of
// text in lexicographic order, built by sorting indices with suffix
// comparison: honest O(n^2 log n) worst case, fine for teaching sizes.
func BuildSuffixArray(text string) []int {
  order := make([]int, len(text))
  for i := range order {
    order[i] = i
  }
  slices.SortFunc(order, func(i, j int) int {
    switch {
    case suffixLess(text, i, j):
      return -1
    case suffixLess(text, j, i):
      return 1
    }
    return 0
  })
  return order
}

// BuildLcp runs Kasai's algorithm: adjacent suffixes in sorted order
// share LCP that can only drop by one per character step, so a single
// walk over positions in text order computes every entry in linear
// total time.
func BuildLcp(text string, suffixArray []int) []int {
  n := len(text)
  rank := make([]int, n)
  for pos, suffix := range suffixArray {
    rank[suffix] = pos
  }
  lcp := make([]int, max(0, n-1))
  h := 0
  for i := 0; i < n; i++ {
    if rank[i] > 0 {
      j := suffixArray[rank[i]-1]
      for i+h < n && j+h < n && text[i+h] == text[j+h] {
        h++
      }
      lcp[rank[i]-1] = h
      if h > 0 {
        h-- // the invariant that makes it linear
      }
    } else {
      h = 0
    }
  }
  return lcp
}
```

listing 228: go, index sort by full suffix order, kasai's walk (source: dsa/samples-go/ch09/suffixarray.go, lines 23-70)

The other five languages build the same two arrays:

```java
  static int cmpSuffix(String t, int i, int j) {
    int len = t.length();
    int li = len - i, lj = len - j;
    int m = Math.min(li, lj);
    for (int k = 0; k < m; k++) {
      int c = t.charAt(i + k) - t.charAt(j + k);
      if (c != 0)
        return c;
    }
    return li - lj;
  }

  // starting positions of the suffixes in lexicographic order
  static int[] buildArray(String t) {
    text = t;
    int len = t.length();
    Integer[] idx = new Integer[len];
    for (int i = 0; i < len; i++)
      idx[i] = i;
    Arrays.sort(idx, (a, b) -> cmpSuffix(t, a, b));
    int[] sa = new int[len];
    for (int i = 0; i < len; i++)
      sa[i] = idx[i];
    return sa;
  }

  // kasai: the lcp of adjacent sorted suffixes can only drop by one per
  // character step, so a left-to-right walk with the h-1 invariant is
  // linear in total
  static int[] buildLcp(String t, int[] sa) {
    int len = t.length();
    int[] rank = new int[len];
    for (int i = 0; i < len; i++)
      rank[sa[i]] = i;
    int[] lcp = new int[len];
    int h = 0;
    for (int i = 0; i < len; i++) {
      if (rank[i] > 0) {
        int j = sa[rank[i] - 1];
        while (i + h < len && j + h < len && t.charAt(i + h) == t.charAt(j + h))
          h++;
        lcp[rank[i] - 1] = h;
        if (h > 0)
          h--;
      } else {
        h = 0;
      }
    }
    return lcp;
  }
```

listing 229: java, the length-aware suffix comparator, the sorted array, kasai (source: dsa/samples-java/src/Ch09/Suffixarray.java, lines 24-74)

```
public static class Suffix
{
    // suffix array of text: starting positions of suffixes in lexicographic
    // order, built by sorting indices with span comparison, honest O(n^2 log n)
    // worst case, fine for teaching sizes
    public static int[] BuildArray(string text)
    {
        int n = text.Length;
        var order = Enumerable.Range(0, n).ToArray();
        Array.Sort(order, (i, j) =>
            text.AsSpan(i).SequenceCompareTo(text.AsSpan(j)));
        return order;
    }

    // kasai's algorithm: adjacent suffixes in sorted order share LCP that
    // can only drop by one per character step, so a right-to-left walk
    // computes every entry in linear total time
    public static int[] BuildLcp(string text, int[] suffixArray)
    {
        int n = text.Length;
        var rank = new int[n];
        for (int i = 0; i < n; i++)
            rank[suffixArray[i]] = i;

        var lcp = new int[Math.Max(0, n - 1)];
        int h = 0;
        for (int i = 0; i < n; i++)
        {
            if (rank[i] > 0)
            {
                int j = suffixArray[rank[i] - 1];
                while (i + h < n && j + h < n && text[i + h] == text[j + h])
                    h++;
                lcp[rank[i] - 1] = h;
                if (h > 0)
                    h--; // the invariant that makes it linear
            }
            else
                h = 0;
        }
        return lcp;
    }

    /// <summary>All start positions of pattern, in suffix order, via binary search.</summary>
```

listing 230: c#, suffix array by sorted indices, kasai lcp (source: dsa/samples/src/Ch09/Tries.cs, lines 127-170)

```
function compareSuffixes(text, i, j) {
  const n = text.length;
  while (i < n && j < n) {
    const a = text.charCodeAt(i);
    const b = text.charCodeAt(j);
    if (a !== b) return a - b;
    i++;
    j++;
  }
  // the exhausted side is a prefix of the other and sorts first
  return i === n && j < n ? -1 : 1;
}

export function buildSuffixArray(text) {
  const order = Array.from({ length: text.length }, (_, i) => i);
  order.sort((i, j) => compareSuffixes(text, i, j));
  return order;
}

// kasai: adjacent suffixes in sorted order share lcp that can only drop by
// one per character step, so one right-to-left sweep is linear in total
export function buildLcp(text, suffixArray) {
  const n = text.length;
  const rank = new Array(n);
  for (let i = 0; i < n; i++) rank[suffixArray[i]] = i;

  const lcp = new Array(Math.max(0, n - 1));
  let h = 0;
  for (let i = 0; i < n; i++) {
    if (rank[i] > 0) {
      const j = suffixArray[rank[i] - 1];
      while (i + h < n && j + h < n && text[i + h] === text[j + h]) h++;
      lcp[rank[i] - 1] = h;
      if (h > 0) h--; // the invariant that makes it linear
    } else {
      h = 0;
    }
  }
  return lcp;
}
```

listing 231: javascript, code-unit suffix compare, the sorted array, kasai (source: dsa/samples-js/src/ch09-suffixarray.mjs, lines 5-44)

```
def build_array(text):
    # suffix start positions in lexicographic order; the key compares to
    # the end, so among equal prefixes the longer suffix sorts later
    return sorted(range(len(text)), key=lambda i: text[i:])


def build_lcp(text, suffix_array):
    # kasai: neighbors in sorted order share an lcp that drops by at most
    # one per position step, integers only, no substring is ever built
    size = len(text)
    rank = [0] * size
    for pos, start in enumerate(suffix_array):
        rank[start] = pos
    lcp = [0] * max(0, size - 1)
    h = 0
    for i in range(size):
        if rank[i] > 0:
            j = suffix_array[rank[i] - 1]
            while i + h < size and j + h < size and text[i + h] == text[j + h]:
                h += 1
            lcp[rank[i] - 1] = h
            if h:
                h -= 1  # the invariant that makes the walk linear
        else:
            h = 0
    return lcp


FIXTURES = [
    ("banana", [5, 3, 1, 0, 4, 2], [1, 3, 0, 0, 2]),
    ("aaaa", [3, 2, 1, 0], [1, 2, 3]),
    ("abab", [2, 0, 3, 1], [2, 0, 1]),
    (
        "mississippi",
        [10, 7, 4, 1, 0, 9, 8, 6, 3, 5, 2],
        [1, 1, 4, 0, 0, 1, 0, 2, 1, 3],
    ),
]
for text, sa_want, lcp_want in FIXTURES:
    sa = build_array(text)
    check(sa == sa_want, f"the suffix array of {text!r}")
    check(build_lcp(text, sa) == lcp_want, f"kasai's lcp of {text!r}")
```

listing 232: python, the suffix key sort, kasai, and the fixture checks (source: dsa/samples-py/src/Ch09/suffixarray.py, lines 14-55)

```lua
-- true when the suffix at start i sorts before the suffix at start j
local function suffix_less(text, i, j)
  local n = #text
  while i < n and j < n do
    local bi, bj = text:byte(i + 1), text:byte(j + 1)
    if bi ~= bj then return bi < bj end
    i, j = i + 1, j + 1
  end
  return i ≥ n and j < n -- i ran out first: a proper prefix sorts earlier
end

-- starting positions of the suffixes in lexicographic order
local function build_array(text)
  local order = {}
  for i = 0, #text - 1 do order[i + 1] = i end
  table.sort(order, function(i, j) return suffix_less(text, i, j) end)
  return order
end

-- kasai: adjacent suffixes in sorted order share lcp that can only drop
-- by one per character step, so one right-to-left sweep suffices
local function build_lcp(text, sa)
  local n = #text
  local rank = {}
  for k = 1, n do rank[sa[k] + 1] = k - 1 end
  local lcp = {}
  local h = 0
  for i = 0, n - 1 do
    local r = rank[i + 1]
    if r > 0 then
      local j = sa[r] -- the suffix one slot earlier in sorted order
      while i + h < n and j + h < n
        and text:byte(i + h + 1) == text:byte(j + h + 1) do
        h = h + 1
      end
      lcp[r] = h
      if h > 0 then h = h - 1 end -- the invariant that makes it linear
    else
      h = 0
    end
  end
  return lcp
end
```

listing 233: lua, byte-wise suffix order, the array, kasai over 0-based starts (source: dsa/samples-lua/ch09_suffixarray.lua, lines 7-49)

Measured across the suites: banana lands 5, 3, 1, 0, 4, 2 with lcp 1, 3, 0, 0, 2 in all seven languages. The six new trees also pin aaaa at 3, 2, 1, 0 with lcp 1, 2, 3 plus the abab and mississippi tables, and every tree recomputes each adjacent-pair lcp by direct character comparison and asserts it equals Kasai's output, the h-1 invariant on trial. The comparators compare characters to the end and then length, so on repeated characters the shorter suffix with an equal prefix sorts first.

Every build here sorts indices with full suffix comparison, honest about its worst case, quadratic comparisons, right for teaching sizes, while production builds use prefix doubling or SA-IS for linear time. Kasai's LCP is the beautiful part: when moving from one suffix to its neighbor in text order, the shared prefix with the previous sorted neighbor can only shrink by one character, so the total work telescopes to linear. The suites brute-force every adjacent pair and check the whole array.

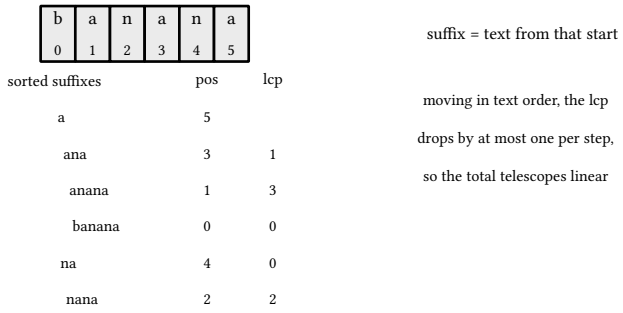


figure 64: the suffix array of banana, start positions in sorted order, adjacent lcp values from kasai's walk

9.4. substring search by binary search

With the suffix array, finding a pattern is two binary searches over suffixes. This version uses one lower bound plus a scan, bounded by the run of suffixes that start with the pattern.

The dry run: the fixtures are ana, the whole word, nab, and the empty pattern over banana, aa over aaaa, and the, th, xyz over the long text, asserted by all seven suites. Hits come out in suffix order everywhere, never ascending position, and the hit set is checked against a naive contains scan.

1. The lower bound halves 0..6: mid 3 holds banana, not less than ana, mid 1 holds ana itself, mid 0 holds a, less.
2. Three probes land lo at row 1, the first suffix that does not sort before ana.
3. The scan walks the run: ana at start 3 matches, anana at start 1 matches, banana diverges on the first character and stops it.
4. The occurrences read 3 then 1, ordered 1 3, pinned.
5. banana lands its own row exactly at 0, and nab slides between na and nana to an empty run, no hits.
6. The empty pattern prefixes every suffix and collects all six starts, 0 through 5.

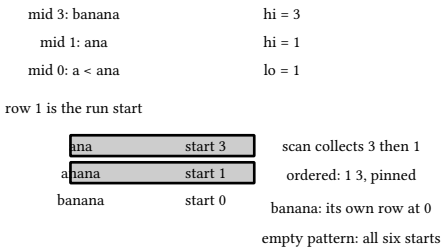


figure 65: pattern ana end to end, three probes of the lower bound, the run of two suffixes, the starts collected

The 1 3 for ana is the pinned find, and the listings below run the same two phases on the longer text against IndexOf.

```c
// three way compare of the suffix at suf against the pattern: a suffix
// that is a proper prefix of the pattern sorts first
static int suffix_cmp(const char *t, int len, int suf, const char *p, int m) {
  int shared = len - suf < m ? len - suf : m;
  int c = memcmp(t + suf, p, (size_t)shared);
  if (c != 0)
    return c;
  return (len - suf) - m;
}

static int starts_with(const char *t, int len, int suf, const char *p, int m) {
  if (len - suf < m)
    return 0;
  return memcmp(t + suf, p, (size_t)m) == 0;
}

// all start positions of the pattern, in suffix order
static int find_all(const char *t, const int *sa, int len, const char *p,
                    int *hits) {
  int m = (int)strlen(p);
  int lo = 0, hi = len;
  while (lo < hi) {
    int mid = (lo + hi) / 2;
    if (suffix_cmp(t, len, sa[mid], p, m) < 0)
      lo = mid + 1;
    else
      hi = mid;
  }
  int count = 0;
  for (int i = lo; i < len; i++) {
    if (!starts_with(t, len, sa[i], p, m))
      break;
    hits[count++] = sa[i];
  }
  return count;
}
```

listing 234: c, the bounded suffix compare, the lower bound, the run (source: dsa/samples-c/src/Ch09/suffixsearch.c, lines 41-76)

```go
// hasPrefix reports whether the suffix of text at pos starts with
// pattern.
func hasPrefix(text string, pos int, pattern string) bool {
  if pos+len(pattern) > len(text) {
    return false
  }
  for k := range pattern {
    if text[pos+k] != pattern[k] {
      return false
    }
  }
  return true
}

// suffixBefore reports whether the suffix at pos sorts strictly
// before pattern lexicographically.
func suffixBefore(text string, pos int, pattern string) bool {
  suffix := text[pos:]
  for k := 0; k < len(suffix) && k < len(pattern); k++ {
    if suffix[k] != pattern[k] {
      return suffix[k] < pattern[k]
    }
  }
  // the shorter one is the smaller: a proper prefix sorts first
  return len(suffix) < len(pattern)
}

// FindAll returns every start position of pattern in text by binary
// search over the suffix array: the lower bound of suffixes strictly
// less than the pattern, then the run of suffixes that carry the
// pattern as prefix. Emission order is suffix order, positions are
// not sorted, and the empty pattern legally returns the whole array.
func FindAll(text string, suffixArray []int, pattern string) []int {
  n := len(suffixArray)
  lo, hi := 0, n
  for lo < hi {
    mid := lo + (hi-lo)/2
    if suffixBefore(text, suffixArray[mid], pattern) {
      lo = mid + 1
    } else {
      hi = mid
    }
  }
  var hits []int
  for i := lo; i < n; i++ {
    if !hasPrefix(text, suffixArray[i], pattern) {
      break
    }
    hits = append(hits, suffixArray[i])
  }
  return hits
}
```

listing 235: go, suffix versus pattern, the lower bound, the prefix run (source: dsa/samples-go/ch09/suffixsearch.go, lines 3-54)

The other five languages run the same two phases:

```
  static int suffixCmp(String t, int suf, String p) {
    int len = t.length(), m = p.length();
    int shared = Math.min(len - suf, m);
    for (int k = 0; k < shared; k++) {
      int c = t.charAt(suf + k) - p.charAt(k);
      if (c != 0)
        return c;
    }
    return (len - suf) - m;
  }

  static boolean startsWith(String t, int suf, String p) {
    int len = t.length(), m = p.length();
    if (len - suf < m)
      return false;
    for (int k = 0; k < m; k++)
      if (t.charAt(suf + k) != p.charAt(k))
        return false;
    return true;
  }

  // all start positions of the pattern, in suffix order
  static int findAll(String t, int[] sa, String p, int[] hits) {
    int len = t.length(), m = p.length();
    int lo = 0, hi = len;
    while (lo < hi) {
      int mid = (lo + hi) / 2;
      if (suffixCmp(t, sa[mid], p) < 0)
        lo = mid + 1;
      else
        hi = mid;
    }
    int count = 0;
    for (int i = lo; i < len; i++) {
      if (!startsWith(t, sa[i], p))
        break;
      hits[count++] = sa[i];
    }
    return count;
  }

  // the emitted order equals the pinned expectation, the hit set equals
```

listing 236: java, the suffix compare, the lower bound, the run with the empty pattern legal (source: dsa/samples-java/src/Ch09/Suffixsearch.java, lines 46-87)

```
    public static List<int> FindAll(string text, int[] suffixArray, string pattern)
    {
        var hits = new List<int>();
        int n = suffixArray.Length;

        int LowerBound(Func<int, bool> suffixIsLess)
        {
            int lo = 0, hi = n;
            while (lo < hi)
            {
                int mid = (lo + hi) / 2;
                if (suffixIsLess(mid))
                    lo = mid + 1;
                else
                    hi = mid;
            }
            return lo;
        }

        int start = LowerBound(i => text.AsSpan(suffixArray[i]).SequenceCompareTo(
            pattern) < 0);

        for (int i = start; i < n; i++)
        {
            if (!text.AsSpan(suffixArray[i]).StartsWith(pattern))
                break;
            hits.Add(suffixArray[i]);
        }
        return hits;
    }
}
```

listing 237: c#, lower bound over suffixes, then the prefix-matching run (source: dsa/samples/src/Ch09/Tries.cs, lines 171-201)

```
// compare the suffix at si with the pattern: negative when the suffix is
// strictly less, 0 when the pattern is a prefix of the suffix (or equal)
function suffixVsPattern(text, si, pattern) {
  const n = text.length;
  const m = pattern.length;
  let k = 0;
  while (si + k < n && k < m) {
    const d = text.charCodeAt(si + k) - pattern.charCodeAt(k);
    if (d !== 0) return d;
    k++;
  }
  return k < m ? -1 : 0;
}

export function findAll(text, suffixArray, pattern) {
  const n = suffixArray.length;
  let lo = 0;
  let hi = n;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (suffixVsPattern(text, suffixArray[mid], pattern) < 0) lo = mid + 1;
    else hi = mid;
  }
  const hits = [];
  for (let i = lo; i < n; i++) {
    if (suffixVsPattern(text, suffixArray[i], pattern) !== 0) break;
    hits.push(suffixArray[i]);
  }
  return hits;
}
```

listing 238: javascript, the suffix compare, the lower bound, the run scan (source: dsa/samples-js/src/ch09-suffixsearch.mjs, lines 5-34)

```
def find_all(text, suffix_array, pattern):
    # lower bound over suffixes strictly less than the pattern
    lo, hi = 0, len(suffix_array)
    while lo < hi:
        mid = (lo + hi) // 2
        if text[suffix_array[mid] :] < pattern:
            lo = mid + 1
        else:
            hi = mid
    hits = []
    for i in range(lo, len(suffix_array)):
        if not text[suffix_array[i] :].startswith(pattern):
            break  # the prefix-matching run ends at the first miss
        hits.append(suffix_array[i])
    return hits


BANANA = build_array("banana")
check(find_all("banana", BANANA, "ana") == [3, 1], "ana hits suffix order [3, 1]")
check(find_all("banana", BANANA, "banana") == [0], "the whole text hits once")
check(find_all("banana", BANANA, "nab") == [], "nab hits nothing")
check(
    find_all("banana", BANANA, "") == [5, 3, 1, 0, 4, 2],
    "the empty pattern returns the whole suffix array",
)
AAAA = build_array("aaaa")
check(find_all("aaaa", AAAA, "aa") == [2, 1, 0], "aa hits [2, 1, 0] on aaaa")
```

listing 239: python, the lower bound and run over slice compares, fixtures pinned (source: dsa/samples-py/src/Ch09/suffixsearch.py, lines 18-44)

```lua
-- true when the whole suffix at start compares less than the pattern
local function suffix_less_than(text, start, pattern)
  for k = 0, #pattern - 1 do
    local b = text:byte(start + k + 1)
    if b == nil then return true end -- suffix ran out: proper prefix
    local pb = pattern:byte(k + 1)
    if b ~= pb then return b < pb end
  end
  return false -- equal across the pattern: not less
end

local function find_all(text, sa, pattern)
  local n = #sa
  local lo, hi = 1, n + 1
  while lo < hi do -- lower bound over suffixes strictly less
    local mid = lo + (hi - lo) // 2
    if suffix_less_than(text, sa[mid], pattern) then lo = mid + 1 else hi = mid end
  end
  local hits = {}
  for k = lo, n do
    local p = sa[k]
    if text:sub(p + 1, p + #pattern) ~= pattern then break end
    hits[#hits + 1] = p
  end
  return hits
end
```

listing 240: lua, suffix less than pattern, the lower bound, the run (source: dsa/samples-lua/ch09_suffixsearch.lua, lines 22-47)

Each probe compares the pattern against a suffix, so a search costs pattern length times log of text length. Measured across the suites: ana emits 3 then 1 for the pinned suffix order and the empty pattern returns the whole array in all seven languages, and the six new trees also pin aaaa's double a at 2, 1, 0 and the long text's 41-entry array with the runs for the and th in suffix order. Every tree checks the hit set against a naive scan, IndexOf in C#, while the emission order stays suffix order, and the degenerate empty pattern, which prefixes every suffix, is legal input everywhere.

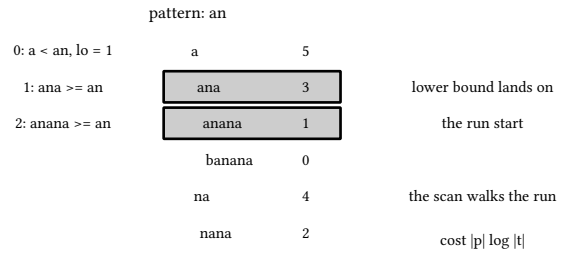


figure 66: substring search as a lower bound over the sorted suffixes, the run that starts with the pattern

What each structure buys: the trie for many keys and prefix questions, the suffix array for one long text and substring questions. Databases build b-tree range indexes, routers build tries, search engines and genome tools live in suffix arrays, and the capstone's prefix scans ride the trie in this listing.

9.5. across the seven languages

The build sizes count non-comment source lines over this chapter's four featured files per language. Every child map here is the language's own table type, no trie shipped from anyone's stdlib:

language	build SLOC	dependency	boundary note
c	395	libc only	26-wide child arrays from a 64-node static pool, lower-case words, checks share the file with main, 45 of them
go	183	slices for sorting	map of byte to node, counts by recursion, duplicate inserts ignored by Len, 13 tests
java	365	jdk 27 stdlib	26-wide child arrays like c with subtree counters on every node, a duplicate insert counts again, the empty-pattern search pinned at the whole array by a naive-scan oracle
c#	165	bcl only	byte-wise Dictionary children, remove supported, subtree sizes, the suffix array rides the same file, 11 tests
javascript	108	node stdlib	object-as-map children with per-node counters, a duplicate insert counts again, 17 tests
python	222	stdlib only	slotted nodes, dict children, the insert climbs the spine only on first store, 62 checks
lua	338	lib.lua harness	table children keyed by one-character strings, a duplicate insert counts again, 20 checks

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` iteration semantics, `MemoryExtensions.SequenceCompareTo` and `StartsWith` span pages, accessed 2026-09-08, plus the kasai and suffix array literature cited in the chapter text. Sample behavior verified by `make verify-csharp`, 11 tests in chapter 9 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 45 embedded checks under `make verify-c`, 13 Go tests, the java runner's 45 Ch09 checks over 4 files under `run-java-samples`, 17 `node --test` cases, 62 Python checks across 4 files, and 20 Lua checks under `run.lua`.

10. graphs

Graphs are the general shape under maps, schedules, dependencies, and networks: vertices and edges, directed or not, weighted or not. This chapter builds the adjacency list representation and the four traversals everything else composes from, once per language.

10.1. representation

A dictionary from vertex to neighbor list handles sparse graphs, arbitrary labels, and growth. Insertion order in the lists becomes deterministic traversal order, which is what lets tests pin exact sequences.

The dry run: the fixture is the labeled store a to b and c, b to d, c to d, d to e, asserted by the C# suite, and the integer suites carry the four-edge fixture 0-1, 0-2, 1-2, 2-3 with degrees 2 2 3 1 and the handshake 8.

1. The first call, a with b and c, opens a's list at b then c and opens empty lists for the two new names, three vertices after one call.
2. The next calls grow one list each and open one name each: b gains d, c gains d, d gains e and opens e, five vertices and five edges.
3. The reads pin both ends: a's neighbors return b then c in insertion order, e's return nothing.
4. Out-degrees read 2, 1, 1, 1, 0 and sum 2 + 1 + 1 + 1 + 0 = 5, the edge count the suite pins.
5. In-degrees read 0, 1, 1, 2, 1 and sum 0 + 1 + 1 + 2 + 1 = 5: every arc leaves one list and lands in exactly one other.

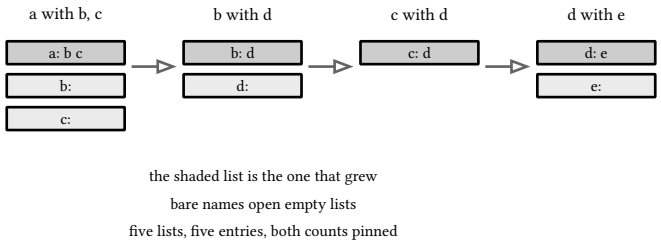


figure 67: the store growing one call at a time, the grown list shaded, bare names opening empty

The 5 and 5 are the pinned counts, and the listings below build the store seven ways.

```c
typedef struct {
  int adj[V][V];
  int deg[V];
  int indeg[V];
  int directed;
} Graph;

static void build(Graph *g, int directed) {
  g->directed = directed;
  for (int v = 0; v < V; v++)
    g->deg[v] = g->indeg[v] = 0;
  for (int i = 0; i < E; i++) {
    int u = edges[i][0], v = edges[i][1];
    g->adj[u][g->deg[u]++] = v;
    if (!directed)
      g->adj[v][g->deg[v]++] = u;
    else
      g->indeg[v]++;
  }
}

static int deg_ok(const Graph *g, const int *want, int indeg) {
  for (int v = 0; v < V; v++)
    if ((indeg ? g->indeg[v] : g->deg[v]) != want[v])
      return 0;
  return 1;
}
```

listing 241: c, fixed V-by-V lists with degrees and in-degrees, one build for both readings (source: dsa/samples-c/src/Ch10/build.c, lines 21-47)

```go
func BuildUndirected(n int, edges []Edge) [][]int {
  adj := make([][]int, n)
  for _, e := range edges {
    adj[e.From] = append(adj[e.From], e.To)
    adj[e.To] = append(adj[e.To], e.From)
  }
  return adj
}

// BuildDirected returns an adjacency list with each edge only on the
// source vertex.
func BuildDirected(n int, edges []Edge) [][]int {
  adj := make([][]int, n)
  for _, e := range edges {
    adj[e.From] = append(adj[e.From], e.To)
  }
  return adj
}
```

listing 242: go, undirected and directed builders over edge structs (source: dsa/samples-go/ch10/build.go, lines 10-27)

```
  static final class Graph {
    int[][] adj = new int[V][V];
    int[] deg = new int[V];
    int[] indeg = new int[V];
    boolean directed;
  }

  static Graph build(boolean directed) {
    Graph g = new Graph();
    g.directed = directed;
    for (int[] e : edges) {
      int u = e[0], v = e[1];
      g.adj[u][g.deg[u]++] = v;
      if (!directed)
        g.adj[v][g.deg[v]++] = u;
      else
        g.indeg[v]++;
    }
    return g;
  }
```

listing 243: java, fixed V-by-V lists with degrees and in-degrees, one build for both readings (source: dsa/samples-java/src/Ch10/Build.java, lines 21-40)

```
public sealed class Graph
{
    private readonly Dictionary<string, List<string>> _adj = new();

    public IReadOnlyList<string> Neighbors(string v) =>
        _adj.TryGetValue(v, out List<string>? list) ? list : Array.Empty<string>();

    public IEnumerable<string> Vertices => _adj.Keys;

    public int VertexCount => _adj.Count;

    public int EdgeCount => _adj.Values.Sum(l => l.Count);

    public Graph Add(string from, params string[] tos)
    {
        if (!_adj.TryGetValue(from, out List<string>? list))
            _adj[from] = list = new List<string>();
        foreach (string to in tos)
        {
            list.Add(to);
            _adj.TryAdd(to, new List<string>());
        }
        return this;
    }
```

listing 244: c#, adjacency lists, fluent adds, counts (source: dsa/samples/src/Ch10/Graphs.cs, lines 4-28)

```
export function buildUndirected(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    adj[u].push(v);
    adj[v].push(u);
  }
  return adj;
}

export function buildDirected(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) adj[u].push(v);
  return adj;
}

export function degrees(adj) {
  return adj.map((list) => list.length);
}

export function outDegrees(adj) {
  return adj.map((list) => list.length);
}

export function inDegrees(adj) {
  const indeg = new Array(adj.length).fill(0);
  for (const list of adj) for (const v of list) indeg[v]++;
  return indeg;
}
```

listing 245: javascript, the two builders and the degree counters (source: dsa/samples-js/src/ch10-build.mjs, lines 4-31)

```python
def build_undirected(edges, vertices):
    adj = [[] for _ in range(vertices)]
    for u, v in edges:
        adj[u].append(v)  # each edge lands in both lists
        adj[v].append(u)
    return adj


def build_directed(edges, vertices):
    adj = [[] for _ in range(vertices)]
    for u, v in edges:
        adj[u].append(v)  # one way only
    return adj


def degrees(adj):
    return [len(row) for row in adj]


def in_out_degrees(edges, vertices):
    indeg = [0] * vertices
    outdeg = [0] * vertices
    for u, v in edges:
        outdeg[u] += 1
        indeg[v] += 1
    return indeg, outdeg
```

listing 246: python, both builders, degrees, in and out split (source: dsa/samples-py/src/Ch10/build.py, lines 16-41)

```lua
local function build(directed)
  local g = {adj = {}, deg = {}, indeg = {}, directed = directed}
  for v = 0, V - 1 do
    g.adj[v], g.deg[v], g.indeg[v] = {}, 0, 0
  end
  for _, e in ipairs(EDGES) do
    local u, v = e[1], e[2]
    g.adj[u][g.deg[u] + 1] = v
    g.deg[u] = g.deg[u] + 1
    if not directed then
      g.adj[v][g.deg[v] + 1] = u
      g.deg[v] = g.deg[v] + 1
    else
      g.indeg[v] = g.indeg[v] + 1
    end
  end
  return g
end
```

listing 247: lua, one table graph, directed flag picks the reading (source: dsa/samples-lua/ch10_build.lua, lines 8-25)

Dense graphs belong in matrices instead, `bool[n, n]` or weights, where the edge test is O(1) but storage is quadratic and iteration over neighbors touches every vertex. The sparse list wastes nothing on absent edges, the same array-versus-hash trade as chapter 6 in another coat.

Measured across the suites: C, Go, Java, JavaScript, and Lua build the same four edges, 0-1, 0-2, 1-2, 2-3, and pin identical reads, undirected degrees 2 2 3 1 with the handshake sum 8, directed out 2 1 1 0 against in 0 1 2 1. Python pins its own edge set with the same handshake discipline, and the C# suite pins counts on its string-labeled store.

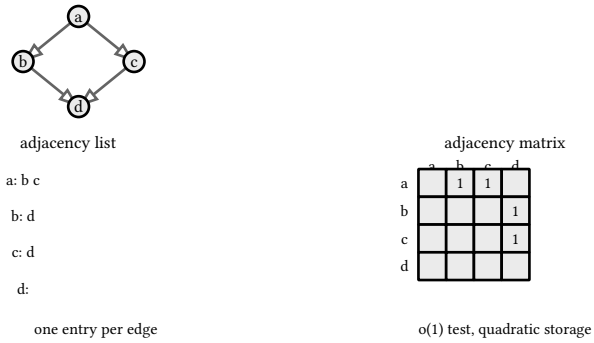


figure 68: one graph, two representations, the list against the matrix

10.2. breadth first

Breadth first is a queue: it finishes an entire distance layer before the next, so the first time it marks a vertex is along a shortest path in edge count. Distances and a parent chain fall out of the same loop, and walking parents backward rebuilds the path.

The dry run: the fixture is the same labeled store walked from a, asserted by the C# suite, and the integer suites pin the same layers on the diamond with the rebuilt path 0 1 3 4.

1. The queue opens with a alone, and the first pop labels its two neighbors, b at 0 + 1 = 1 and c at 1, both waiting behind it.
2. Pop b: d is unseen, takes 1 + 1 = 2 with parent b, and joins the queue behind c.
3. Pop c: its edge into d arrives too late, d already carries 2, and the strict test writes nothing.
4. Pop d: e takes 2 + 1 = 3. Pop e: no neighbors, the queue empties.
5. The order reads a b c d e and the distances 0 1 1 2 3, both asserted by the suite.
6. Parents walk backward from e through d and b to a, so the shortest path to e reads a b d e at three edges.

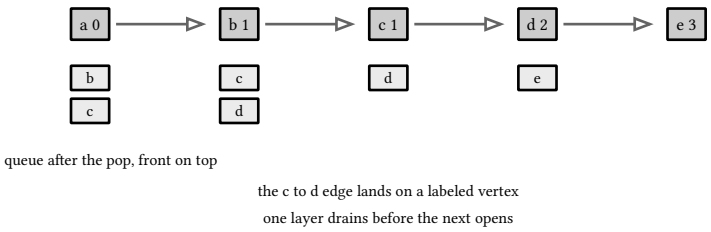


figure 69: the run as five pops, the queue waiting under each, every new label landing one layer down

The 0 1 1 2 3 with the order a b c d e is the pinned pair, and the listings below run the queue in seven languages.

```c
static void bfs(int start) {
  int queue[V], head = 0, tail = 0;
  on = 0;
  for (int v = 0; v < V; v++) {
    dist[v] = INF;
    parent[v] = -1;
  }
  dist[start] = 0;
  queue[tail++] = start;
  while (head < tail) {
    int u = queue[head++];
    order[on++] = u;
    for (int i = 0; i < deg[u]; i++) {
      int v = adj[u][i];
      if (dist[v] == INF) {
        dist[v] = dist[u] + 1;
        parent[v] = u;
        queue[tail++] = v;
      }
    }
  }
}
```

listing 248: c, the ring-buffer queue, distances, parents, visit order (source: dsa/samples-c/src/Ch10/bfs.c, lines 27-48)

```go
func BFS(adj [][]int, start int) (dist []int, parent []int) {
  dist = make([]int, len(adj))
  parent = make([]int, len(adj))
  for i := range dist {
    dist[i], parent[i] = -1, -1
  }
  dist[start] = 0
  queue := []int{start}
  for len(queue) > 0 {
    v := queue[0]
    queue = queue[1:]
    for _, n := range adj[v] {
      if dist[n] == -1 {
        dist[n] = dist[v] + 1
        parent[n] = v
        queue = append(queue, n)
      }
    }
  }
  return dist, parent
}
```

listing 249: go, slice-as-queue, −1 marks unreachable (source: dsa/samples-go/ch10/bfs.go, lines 7-27)

The other five run the queue, Java walking a head cursor over one array queue:

```
  static void bfs(int start) {
    int[] queue = new int[V];
    int head = 0, tail = 0;
    on = 0;
    for (int v = 0; v < V; v++) {
      dist[v] = INF;
      parent[v] = -1;
    }
    dist[start] = 0;
    queue[tail++] = start;
    while (head < tail) {
      int u = queue[head++];
      order[on++] = u;
      for (int i = 0; i < deg[u]; i++) {
        int v = adj[u][i];
        if (dist[v] == INF) {
          dist[v] = dist[u] + 1;
          parent[v] = u;
          queue[tail++] = v;
        }
      }
    }
  }
```

listing 250: java, the array queue with a head cursor, distances, parents, visit order (source: dsa/samples-java/src/Ch10/Bfs.java, lines 29-51)

```
    public (List<string> Order, Dictionary<string, int> Distance) Bfs(string source)
    {
        var order = new List<string>();
        var distance = new Dictionary<string, int> { [source] = 0 };
        var queue = new Queue<string>();
        queue.Enqueue(source);
        while (queue.Count > 0)
        {
            string v = queue.Dequeue();
            order.Add(v);
            foreach (string n in Neighbors(v))
            {
                if (distance.TryAdd(n, distance[v] + 1))
                    queue.Enqueue(n);
            }
        }
        return (order, distance);
    }
```

listing 251: c#, bfs with a distance map, TryAdd as the seen test (source: dsa/samples/src/Ch10/Graphs.cs, lines 30-47)

```javascript
export function bfs(adj, start) {
  const dist = new Array(adj.length).fill(-1);
  const parent = new Array(adj.length).fill(-1);
  dist[start] = 0;
  const queue = [start];
  for (let head = 0; head < queue.length; head++) {
    const u = queue[head];
    for (const v of adj[u]) {
      if (dist[v] === -1) {
        dist[v] = dist[u] + 1;
        parent[v] = u;
        queue.push(v);
      }
    }
  }
  return { dist, parent };
}

// walk parents backward from dst, or null when dst is unreachable
export function pathTo(parent, src, dst) {
  if (parent[dst] === -1 && dst !== src) return null;
  const path = [];
  for (let v = dst; v !== -1; v = parent[v]) path.push(v);
  path.reverse();
  return path[0] === src ? path : null;
}
```

listing 252: javascript, head index walks the array queue, path rebuild (source: dsa/samples-js/src/ch10-bfs.mjs, lines 4-29)

```python
def bfs(adj, start):
    dist = [None] * len(adj)
    parent = [None] * len(adj)
    dist[start] = 0
    queue, head = [start], 0
    while head < len(queue):
        u = queue[head]
        head += 1
        for v in adj[u]:
            if dist[v] is None:
                dist[v] = dist[u] + 1
                parent[v] = u
                queue.append(v)
    return dist, parent


def path_to(parent, target):
    hop, chain = target, []
    while hop is not None:
        chain.append(hop)
        hop = parent[hop]
    return chain[::-1]
```

listing 253: python, none for unreached, the parent chain reversed (source: dsa/samples-py/src/Ch10/bfs.py, lines 23-44)

```lua
local function bfs(g, start)
  local dist, parent, order = {}, {}, {}
  local queue = {start}
  local head = 1
  dist[start] = 0
  parent[start] = -1
  while head ≤ #queue do
    local u = queue[head]
    head = head + 1
    order[#order + 1] = u
    for _, v in ipairs(g.adj[u]) do
      if dist[v] == nil then
        dist[v] = dist[u] + 1
        parent[v] = u
        queue[#queue + 1] = v
      end
    end
  end
  return dist, parent, order
end
```

listing 254: lua, 1-based queue with a head cursor (source: dsa/samples-lua/ch10_bfs.lua, lines 16-35)

Measured across the suites: C, Java, and Lua walk an undirected diamond plus tail and Go and JavaScript its directed twin, and all five pin the same numbers, distances 0 1 1 2 3, parents with vertex 3 discovered through 1, and the rebuilt path 0 1 3 4. Python runs a six-vertex fixture, distances 0 1 1 2 3 3 with the path to 5 reading 0 1 3 5, and checks both lanes to a tie. The C# suite walks its labeled graph by layers.

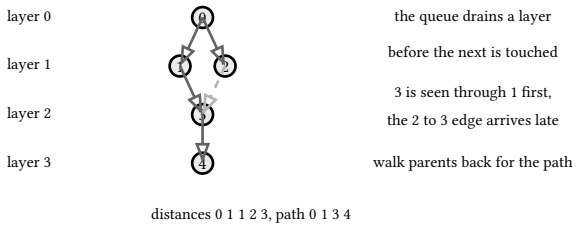


figure 70: breadth first on the diamond plus tail, one layer per row, the parent chain is a shortest path

10.3. depth first

Depth first is a stack, recursion here: it dives along one path until it dead ends, then unwinds. The iterative twin pushes neighbors reversed so its pop order matches the recursive walk, and the recursion state, gray on the path, black finished, is what cycle detection reads.

The dry run: the fixture is the same labeled store dove from a, asserted by the C# suite, and the integer suites pin the matching preorder 0 1 3 4 2 with the iterative twin agreeing.

1. The recursion opens a and follows its first neighbor b, insertion order decides, then b's only neighbor d, then d's only neighbor e: the dive runs a b d e before any unwind.
2. e has no neighbors and finishes, and the unwind closes d, then b.
3. Back at a the second neighbor c opens, and its edge into d meets a finished vertex, a cross edge, not a cycle.
4. c finishes, then a, and the preorder reads a b d e c, asserted.
5. The forest on two separate arcs, a to b and x to y, stamps the finish order b a y x, each child closing before its parent.
6. The verdicts: a self loop and a two vertex cycle both flag, the diamond's two paths to one vertex do not.

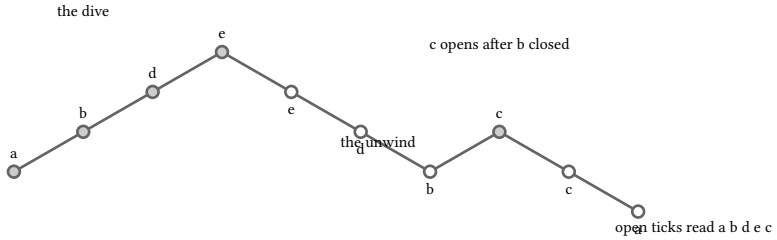


figure 71: the recursion as a depth profile, one tick per event, shaded dots are opens, hollow dots are finishes

The preorder a b d e c is the pinned walk, and the listings below carry the stack in seven languages.

```c
static void dfs_rec(int u) {
  color[u] = 1; // gray: on the current path
  order[on++] = u;
  for (int i = 0; i < deg[u]; i++) {
    int v = adj[u][i];
    if (color[v] == 1)
      saw_cycle = 1; // back edge into the active path
    else if (color[v] == 0)
      dfs_rec(v);
  }
  color[u] = 2;
}

// explicit stack, neighbors pushed reversed so the pop order matches the
// recursive walk
static void dfs_iter(int start) {
  int seen[V] = {0};
  int stack[16], sp = 0;
  on = 0;
  stack[sp++] = start;
  while (sp) {
    int u = stack[--sp];
    if (seen[u])
      continue;
    seen[u] = 1;
    order[on++] = u;
    for (int i = deg[u] - 1; i ≥ 0; i--) {
      int v = adj[u][i];
      if (!seen[v])
        stack[sp++] = v;
    }
  }
}
```

listing 255: c, recursive three-color dfs and the explicit-stack twin (source: dsa/samples-c/src/Ch10/dfs.c, lines 25-57)

```go
func DFSRecursive(adj [][]int, start int) []int {
  seen := make([]bool, len(adj))
  var order []int
  var visit func(int)
  visit = func(v int) {
    seen[v] = true
    order = append(order, v)
    for _, n := range adj[v] {
      if !seen[n] {
        visit(n)
      }
    }
  }
  visit(start)
  return order
}

// DFSIterative replaces recursion with an explicit stack. Neighbors
// push in reverse so the pops follow adjacency-list order, matching
// the recursive preorder.
func DFSIterative(adj [][]int, start int) []int {
  seen := make([]bool, len(adj))
  var order []int
  stack := []int{start}
  for len(stack) > 0 {
    v := stack[len(stack)-1]
    stack = stack[:len(stack)-1]
    if seen[v] {
      continue
    }
    seen[v] = true
    order = append(order, v)
    for i := len(adj[v]) - 1; i ≥ 0; i-- {
      if !seen[adj[v][i]] {
        stack = append(stack, adj[v][i])
      }
    }
  }
  return order
}
```

listing 256: go, recursive and iterative dfs, reversed pushes match the preorder (source: dsa/samples-go/ch10/dfs.go, lines 6-45)

The other five carry the stack, Java's iterative twin pushing neighbors reversed to match the recursive preorder:

```
  static void dfsRec(int u) {
    color[u] = 1; // gray: on the current path
    order[on++] = u;
    for (int i = 0; i < deg[u]; i++) {
      int v = adj[u][i];
      if (color[v] == 1)
        sawCycle = true; // back edge into the active path
      else if (color[v] == 0)
        dfsRec(v);
    }
    color[u] = 2;
  }

  // explicit stack, neighbors pushed reversed so the pop order matches the
  // recursive walk
  static void dfsIter(int start) {
    int[] seen = new int[V];
    int[] stack = new int[16];
    int sp = 0;
    on = 0;
    stack[sp++] = start;
    while (sp > 0) {
      int u = stack[--sp];
      if (seen[u] != 0)
        continue;
      seen[u] = 1;
      order[on++] = u;
      for (int i = deg[u] - 1; i >= 0; i--) {
        int v = adj[u][i];
        if (seen[v] == 0)
          stack[sp++] = v;
      }
    }
  }
```

listing 257: java, recursive three-color dfs and the explicit-stack twin (source: dsa/samples-java/src/Ch10/Dfs.java, lines 30-63)

```
    public List<string> Dfs(string source)
    {
        var order = new List<string>();
        var seen = new HashSet<string>();
        void Visit(string v)
        {
            if (!seen.Add(v))
                return;
            order.Add(v);
            foreach (string n in Neighbors(v))
                Visit(n);
        }
        Visit(source);
        return order;
    }
```

listing 258: c#, recursive dfs over the seen set (source: dsa/samples/src/Ch10/Graphs.cs, lines 50-64)

The forest version stamps discovery and finish ticks on every vertex, which topological sort and cycle detection both read:

```
    public Stamps DfsForest()
    {
        var times = new Dictionary<string, (int, int)>();
        var finishOrder = new List<string>();
        int tick = 0;
        void Visit(string v)
        {
            int discover = ++tick;
            foreach (string n in Neighbors(v))
            {
                if (!times.ContainsKey(n))
                    Visit(n);
            }
            times[v] = (discover, ++tick);
            finishOrder.Add(v);
        }
        foreach (string v in _adj.Keys)
        {
            if (!times.ContainsKey(v))
                Visit(v);
        }
        return new Stamps(times, finishOrder);
    }
```

listing 259: c#, dfs forest with discover and finish times (source: dsa/samples/src/Ch10/Graphs.cs, lines 71-94)

The alternative order is DFS finish order reversed, free from the stamps the forest already computes. Cycle detection needs the open versus done distinction: a back edge into a vertex still on the recursion stack is a cycle, a cross edge into a finished vertex is not. The diamond test proves the difference, two paths to one vertex are not a cycle:

```
    public bool HasCycle()
    {
        const int open = 1, done = 2;
        var state = new Dictionary<string, int>();
        bool DFS(string v)
        {
            state[v] = open;
            foreach (string n in Neighbors(v))
            {
                if (state.TryGetValue(n, out int s))
                {
                    if (s == open)
                        return true; // edge into an open vertex is a back edge
                }
                else if (DFS(n))
                    return true;
            }
            state[v] = done;
            return false;
        }
        return _adj.Keys.Any(v => !state.ContainsKey(v) && DFS(v));
    }
```

listing 260: c#, back edge detection with open and done states (source: dsa/samples/src/Ch10/Graphs.cs, lines 122-144)

Three more languages carry the walk:

```
export function dfsRecursive(adj, start) {
  const order = [];
  const color = new Array(adj.length).fill(0); // 0 white 1 gray 2 black
  let sawCycle = false;
  const visit = (u) => {
    color[u] = 1; // gray: on the current path
    order.push(u);
    for (const v of adj[u]) {
      if (color[v] === 1) sawCycle = true; // back edge into the active path
      else if (color[v] === 0) visit(v);
    }
    color[u] = 2;
  };
  visit(start);
  return { order, sawCycle };
}

// explicit stack, neighbors pushed reversed so the pop order matches the
// recursive walk
export function dfsIterative(adj, start) {
  const seen = new Array(adj.length).fill(false);
  const order = [];
  const stack = [start];
  while (stack.length) {
    const u = stack.pop();
    if (seen[u]) continue;
    seen[u] = true;
    order.push(u);
    for (let i = adj[u].length - 1; i ≥ 0; i--) {
      const v = adj[u][i];
      if (!seen[v]) stack.push(v);
    }
  }
  return order;
}
```

listing 261: javascript, three-color walk with the cycle flag, stack twin (source: dsa/samples-js/src/ch10-dfs.mjs, lines 4-38)

```python
def dfs_recursive(adj, start):
    seen, order = set(), []

    def visit(u):
        seen.add(u)
        order.append(u)
        for v in adj[u]:
            if v not in seen:
                visit(v)

    visit(start)
    return order


def dfs_iterative(adj, start):
    order, visited = [], set()
    stack = [start]
    while stack:
        u = stack.pop()
        if u in visited:
            continue
        visited.add(u)
        order.append(u)
        for v in reversed(adj[u]):  # reversed push = sorted visit order
            if v not in visited:
                stack.append(v)
    return order
```

listing 262: python, recursive and iterative over sorted adjacency (source: dsa/samples-py/src/Ch10/dfs.py, lines 24-50)

```lua
local function dfs_rec(g, start)
  local order, saw_cycle = {}, false
  local color = {}
  local function visit(u)
    color[u] = 1 -- gray: on the current path
    order[#order + 1] = u
    for _, v in ipairs(g.adj[u] or {}) do
      if color[v] == 1 then
        saw_cycle = true -- back edge into the active path
      elseif color[v] == nil then
        visit(v)
      end
    end
    color[u] = 2
  end
  visit(start)
  return order, saw_cycle
end

-- explicit stack, neighbors pushed reversed so the pop order matches the
-- recursive walk
local function dfs_iter(g, start)
  local order = {}
  local seen = {}
  local stack = {start}
  while #stack > 0 do
    local u = table.remove(stack)
    if not seen[u] then
      seen[u] = true
      order[#order + 1] = u
      local adj = g.adj[u] or {}
      for i = #adj, 1, -1 do
        if not seen[adj[i]] then stack[#stack + 1] = adj[i] end
      end
    end
  end
  return order
end
```

listing 263: lua, nested functions carry the walk state (source: dsa/samples-lua/ch10_dfs.lua, lines 17-54)

Measured across the suites: C, Go, Java, JavaScript, and Lua walk the same dag, 0 to 1 and 2, both into 3, then 4, and pin the preorder 0 1 3 4 2 with the iterative walk matching the recursive one exactly, the same 2 3 4 from a mid start, and a back edge 2 to 1 plus a self loop both flagged. Python sorts each adjacency list and walks an undirected fixture instead, 0 1 3 4 5 2, with the cycle test reading the came-from parent. The C# suite pins `a b d e c` on its labeled graph.

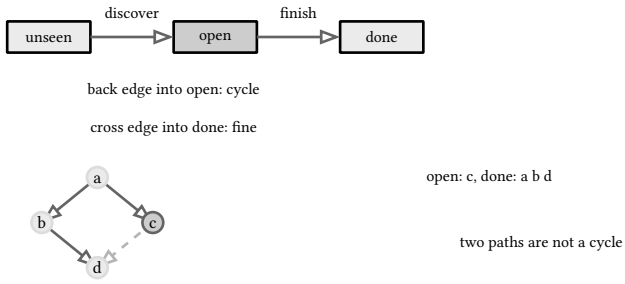


figure 72: the three dfs states, a back edge into open is the cycle, a cross edge into done is not

10.4. topological order

A topological order is a linear layout where every edge points forward. Kahn's algorithm repeatedly removes a zero in-degree vertex, which must exist in a dag, and queues the vertices it releases. A queue that drains before the whole graph is the cycle verdict.

The dry run: the fixture is the same labeled dag, asserted by the C# suite edge by edge, and C, Java, Lua, and Python pin the exact order 4 5 2 0 3 1 on the classic six-vertex fixture.

1. The five arcs leave in-degrees a 0, b 1, c 1, d 2, e 1, and the sum 0 + 1 + 1 + 2 + 1 = 5 matches the edge count.
2. Only a sits at zero, so the ready queue opens with a alone.
3. Pop a: b and c both drop to 0 and enqueue in neighbor order, b then c.
4. Pop b: d drops 2 to 1. Pop c: d hits 0 and joins.
5. Pop d releases e, pop e ends the drain, 5 of 5 emitted in the order a b c d e with every edge pointing forward.
6. On the arcs a to b, b to c, c back to a the in-degrees read 1 1 1, the queue never opens, and the sort throws.

pop	a	b	c	d	e	ready queue
	0	1	1	2	1	a
a				2	1	b c
b				1	1	c
c				0	1	d
d					0	e

The 5 of 5 drain against the empty refusal is the pinned pair, and the listings below drain the queue in seven languages.

```c
// returns the number of nodes emitted; < V means a cycle blocked the rest
static int toposort(int *out) {
  int work[V], qh = 0, qt = 0, k = 0;
  for (int v = 0; v < V; v++)
    if (indeg[v] == 0)
      work[qt++] = v;
  while (qh < qt) {
    int u = work[qh++];
    out[k++] = u;
    for (int i = 0; i < deg[u]; i++) {
      int v = adj[u][i];
      if (--indeg[v] == 0)
        work[qt++] = v;
    }
  }
  return k;
}
```

listing 264: c, the in-degree queue, emitted count below V means a cycle (source: dsa/samples-c/src/Ch10/topo.c, lines 25-41)

```go
func KahnTopo(adj [][]int) ([]int, bool) {
  indeg := InDegrees(adj)
  var queue []int
  for v, d := range indeg {
    if d == 0 {
      queue = append(queue, v)
    }
  }
  var order []int
  for len(queue) > 0 {
    v := queue[0]
    queue = queue[1:]
    order = append(order, v)
    for _, n := range adj[v] {
      indeg[n]--
      if indeg[n] == 0 {
        queue = append(queue, n)
      }
    }
  }
  return order, len(order) == len(adj)
}
```

listing 265: go, kahn over in-degrees, drained flag returned (source: dsa/samples-go/ch10/topo.go, lines 6-27)

```java
  static int toposort(int[] out) {
    int[] work = new int[V];
    int qh = 0, qt = 0, k = 0;
    for (int v = 0; v < V; v++)
      if (indeg[v] == 0)
        work[qt++] = v;
    while (qh < qt) {
      int u = work[qh++];
      out[k++] = u;
      for (int i = 0; i < deg[u]; i++) {
        int v = adj[u][i];
        if (--indeg[v] == 0)
          work[qt++] = v;
      }
    }
    return k;
  }
```

listing 266: java, the in-degree queue, emitted count below V means a cycle (source: dsa/samples-java/src/Ch10/Topo.java, lines 28-44)

```
    public List<string> TopologicalSort()
    {
        var inDegree = _adj.Keys.ToDictionary(v => v, _ => 0);
        foreach (List<string> edges in _adj.Values)
            foreach (string to in edges)
                inDegree[to]++;

        var ready = new Queue<string>(
            inDegree.Where(kv => kv.Value == 0).Select(kv => kv.Key));
        var order = new List<string>();

        while (ready.Count > 0)
        {
            string v = ready.Dequeue();
            order.Add(v);
            foreach (string n in Neighbors(v))
                if (--inDegree[n] == 0)
                    ready.Enqueue(n);
        }

        if (order.Count != _adj.Count)
            throw new InvalidOperationException("graph has a cycle");
        return order;
    }
```

listing 267: c#, kahn's algorithm, the cycle refusal (source: dsa/samples/src/Ch10/Graphs.cs, lines 96-120)

```
export function kahnTopo(adj) {
  const indeg = new Array(adj.length).fill(0);
  for (const list of adj) for (const v of list) indeg[v]++;
  const order = [];
  const queue = [];
  for (let v = 0; v < adj.length; v++) if (indeg[v] === 0) queue.push(v);
  for (let head = 0; head < queue.length; head++) {
    const u = queue[head];
    order.push(u);
    for (const v of adj[u]) if (--indeg[v] === 0) queue.push(v);
  }
  return { order, ok: order.length === adj.length };
}
```

listing 268: javascript, head cursor walks the ready queue (source: dsa/samples-js/src/ch10-topo.mjs, lines 5-17)

```python
def kahn(vertices, edges):
    adj = [[] for _ in range(vertices)]
    indeg = [0] * vertices
    for u, v in edges:
        adj[u].append(v)
        indeg[v] += 1
    ready = [v for v in range(vertices) if indeg[v] == 0]  # ascending ids
    order = []
    head = 0
    while head < len(ready):
        u = ready[head]
        head += 1
        order.append(u)
        for v in adj[u]:
            indeg[v] -= 1
            if indeg[v] == 0:
                ready.append(v)
    return order, len(order) == vertices
```

listing 269: python, ascending-id ready list, ok flag (source: dsa/samples-py/src/Ch10/topo.py, lines 14-31)

```lua
local function toposort(g)
  local out, work = {}, {}
  local head = 1
  for v = 0, 5 do
    if (g.indeg[v] or 0) == 0 then work[#work + 1] = v end
  end
  while head ≤ #work do
    local u = work[head]
    head = head + 1
    out[#out + 1] = u
    for _, v in ipairs(g.adj[u] or {}) do
      g.indeg[v] = g.indeg[v] - 1
      if g.indeg[v] == 0 then work[#work + 1] = v end
    end
  end
  return out
end
```

listing 270: lua, 1-based work queue, count-out contract (source: dsa/samples-lua/ch10_topo.lua, lines 17-33)

Measured across the suites: C, Java, Lua, and Python run the classic six-vertex fixture, 5 to 2 and 0, 4 to 0 and 1, 2 to 3, 3 to 1, and pin the exact order 4 5 2 0 3 1 under the fifo ready queue seeded in ascending id order, with two cycles holding all but the chain 2 3 hostage in C, Java, and Lua and a three-cycle draining to nothing in Python. Go and JavaScript pin the forced order 0 1 2 3 on the diamond, check every edge points forward on the dag, and flag the cycle by an empty drain. The C# suite pins its order and throws on the cycle.

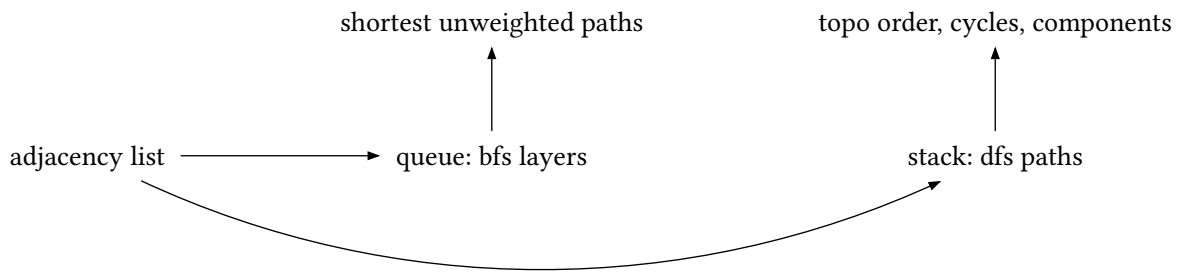


figure 73: graph traversal pick order

10.5. connectivity

Components under the undirected view are flood fills, one per unvisited vertex. The method builds the symmetric adjacency on the fly rather than mutating the directed store.

The dry run: the fixture is the mixed store a to b to c, d to e, and f alone, asserted by all seven suites. The six new trees pin the exact ids in vertex first-seen order and run a union-find oracle over the same edges, and their isolation lane adds z with no edges.

1. Stored directed, the graph holds three arcs, a to b, b to c, d to e, with f isolated.
2. Symmetrized on the fly each arc lands in two lists: b's reads a then c, every other touched vertex reads one neighbor, f none.
3. The fill from a takes b, and from b the walk sees a already labeled and takes c: a, b, c share id 0.
4. The scan resumes at d, fills e with it as id 1, and finishes at f alone as id 2.
5. Three fills cover 3 + 2 + 1 = 6 vertices, and the suite pins the three distinct ids with d and f apart from each other and from a.
6. The one-way check: a store holding only a to b still fills as one island, the directed arc joins its endpoints the moment direction is ignored.

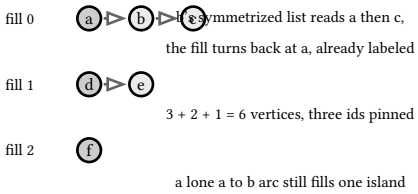


figure 74: the three fills in run order, one recursion per island, the shaded vertex where each fill starts

The three ids over six vertices are the pinned count, and the listings below are the flood fills that produced them.

```c
// flood fill over both directions of every directed edge, stack based,
// ids assigned in vertex insertion order
static int components(const graph *g, int *comp) {
  int nbr[MAX_V][MAX_V], nnbr[MAX_V];
  for (int v = 0; v < g->nverts; v++) {
    nnbr[v] = 0;
    comp[v] = -1;
  }
  for (int e = 0; e < g->nedges; e++) {
    nbr[g->from[e]][nnbr[g->from[e]]++] = g->to[e];
    nbr[g->to[e]][nnbr[g->to[e]]++] = g->from[e];
  }
  int id = 0;
  for (int v = 0; v < g->nverts; v++) {
    if (comp[v] >= 0)
      continue;
    int stack[MAX_V], top = 0;
    comp[v] = id;
    stack[top++] = v;
    while (top > 0) {
      int u = stack[--top];
      for (int k = 0; k < nnbr[u]; k++) {
        int w = nbr[u][k];
        if (comp[w] < 0) {
          comp[w] = id;
          stack[top++] = w;
        }
      }
    }
    id++;
  }
  return id;
}
```

listing 271: c, symmetrize both directions, stack fill, ids in insertion order (source: dsa/samples-c/src/Ch10/components.c, lines 50-82)

```go
// Components labels each vertex with its undirected component id:
// the flood runs over both directions of every directed edge, and ids
// are assigned in vertex first-seen order.
func (g *NameGraph) Components() map[string]int {
  sym := make(map[string][]string)
  addPair := func(a, b string) {
    sym[a] = append(sym[a], b)
  }
  for _, from := range g.names {
    for _, to := range g.adj[from] {
      addPair(from, to)
      addPair(to, from)
    }
  }
  component := make(map[string]int)
  id := 0
  var flood func(v string)
  flood = func(v string) {
    component[v] = id
    for _, n := range sym[v] {
      if _, seen := component[n]; !seen {
        flood(n)
      }
    }
  }
  for _, v := range g.names {
    if _, seen := component[v]; !seen {
      flood(v)
      id++
    }
  }
  return component
}
```

listing 272: go, recursive flood over the symmetrized map, ids in name order (source: dsa/samples-go/ch10/components.go, lines 37-69)

The other five languages fill the same islands, Java's component file switching to string labels over a first-seen id map:

```
  static void addEdge(Graph g, String a, String b) {
    g.from[g.nedges] = vertexId(g, a);
    g.to[g.nedges] = vertexId(g, b);
    g.nedges++;
  }

  // flood fill over both directions of every directed edge, stack based,
  // ids assigned in vertex insertion order
  static int components(Graph g, int[] comp) {
    int[][] nbr = new int[MAX_V][MAX_V];
    int[] nnbr = new int[MAX_V];
    for (int v = 0; v < g.nverts; v++)
      comp[v] = -1;
    for (int e = 0; e < g.nedges; e++) {
      nbr[g.from[e]][nnbr[g.from[e]]++] = g.to[e];
      nbr[g.to[e]][nnbr[g.to[e]]++] = g.from[e];
    }
    int id = 0;
    for (int v = 0; v < g.nverts; v++) {
      if (comp[v] ≥ 0)
        continue;
      int[] stack = new int[MAX_V];
      int top = 0;
      comp[v] = id;
      stack[top++] = v;
      while (top > 0) {
        int u = stack[--top];
        for (int k = 0; k < nnbr[u]; k++) {
          int w = nbr[u][k];
          if (comp[w] < 0) {
            comp[w] = id;
            stack[top++] = w;
          }
        }
      }
      id++;
    }
    return id;
  }
```

listing 273: java, symmetrize both directions, stack fill, ids in first-seen order (source: dsa/samples-java/src/Ch10/Components.java, lines 43-81)

```
    public Dictionary<string, int> Components()
    {
        var undirected = new Dictionary<string, List<string>>();
        foreach ((string from, List<string> tos) in _adj)
        {
            foreach (string to in tos)
            {
                AddPair(undirected, from, to);
                AddPair(undirected, to, from);
            }
        }

        var component = new Dictionary<string, int>();
        int id = 0;
        void Flood(string v)
        {
            component[v] = id;
            if (undirected.TryGetValue(v, out List<string>? neighbors))
                foreach (string n in neighbors)
                    if (!component.ContainsKey(n))
                        Flood(n);
        }
        foreach (string v in _adj.Keys)
        {
            if (!component.ContainsKey(v))
            {
                Flood(v);
                id++;
            }
        }
        return component;
    }

    private static void AddPair(Dictionary<string, List<string>> map, string a, string b)
    {
        if (!map.TryGetValue(a, out List<string>? list))
            map[a] = list = new List<string>();
        list.Add(b);
    }
}
```

listing 274: c#, undirected flood fill components (source: dsa/samples/src/Ch10/Graphs.cs, lines 146-185)

```
  // name -> component id, ids assigned in vertex first-seen order
  components() {
    const undirected = new Map();
    const addPair = (a, b) => {
      if (!undirected.has(a)) undirected.set(a, []);
      undirected.get(a).push(b);
    };
    for (const [from, tos] of this.#adj) {
      for (const to of tos) {
        addPair(from, to);
        addPair(to, from);
      }
    }

    const component = new Map();
    let id = 0;
    const flood = (v) => {
      component.set(v, id);
      for (const n of undirected.get(v) ?? []) {
        if (!component.has(n)) flood(n);
      }
    };
    for (const v of this.#adj.keys()) {
      if (!component.has(v)) {
        flood(v);
        id++;
      }
    }
    return component;
  }
```

listing 275: javascript, the flood over Map adjacency, first-seen ids (source: dsa/samples-js/src/ch10-components.mjs, lines 27-56)

```python
class Graph:
    def __init__(self):
        self.order = []  # vertices in insertion order, ids stay deterministic
        self.adj = {}

    def _ensure(self, v):
        if v not in self.adj:
            self.adj[v] = []
            self.order.append(v)

    def add_edge(self, a, b):
        self._ensure(a)
        self._ensure(b)
        self.adj[a].append(b)

    def add_vertex(self, v):
        self._ensure(v)

    def components(self):
        undirected = {v: [] for v in self.order}
        for v in self.order:
            for w in self.adj[v]:
                undirected[v].append(w)
                undirected[w].append(v)
        comp = {}
        next_id = 0
        for start in self.order:
            if start in comp:
                continue
            comp[start] = next_id
            flood = [start]  # depth first over the symmetrized adjacency
            while flood:
                v = flood.pop()
                for w in undirected[v]:
                    if w not in comp:
                        comp[w] = next_id
                        flood.append(w)
            next_id += 1
        return comp
```

listing 276: python, the insertion-order store and the depth-first fill (source: dsa/samples-py/src/Ch10/components.py, lines 14-52)

```lua
-- flood fill over the symmetrized adjacency, ids in first-seen order
function g.components()
  local und = {}
  for _, from in ipairs(order) do
    for _, to in ipairs(adj[from]) do
      und[from] = und[from] or {}
      und[to] = und[to] or {}
      und[from][#und[from] + 1] = to
      und[to][#und[to] + 1] = from
    end
  end
  local comp, next_id = {}, 0
  local function flood(v)
    comp[v] = next_id
    for _, n in ipairs(und[v] or {}) do
      if comp[n] == nil then flood(n) end
    end
  end
  for _, v in ipairs(order) do
    if comp[v] == nil then
      flood(v)
      next_id = next_id + 1
    end
  end
  return comp, next_id
end
```

listing 277: lua, the flood over symmetrized tables, ids in order (source: dsa/samples-lua/ch10_components.lua, lines 38-63)

Measured across the suites: all seven agree a, b, c share one id, d and e a second, f a third, 3 components over 6 vertices, and that a single directed edge still joins its endpoints when direction is ignored. The six new trees pin the exact numbering, 0, 1, 2 in first-seen order, check the union-find oracle on every pair and on the count, and run the isolation lane, z with no edges stays untouched by any traversal from a. That is the connectivity question for undirected reachability, answered by ignoring direction.

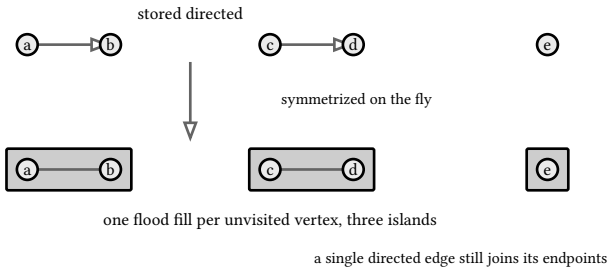


figure 75: connectivity, the directed store viewed symmetric, one flood fill per unvisited vertex

Everything ahead composes these four primitives: Dijkstra is BFS with a heap instead of a fifo, chapter 11, Kruskal and Prim build spanning trees from component walks and heap orders, chapter 12, and the capstone's segment merge graph is a dag by construction.

10.6. across the seven languages

The build sizes count non-comment source lines over this chapter's five featured files per language. The C# file carries the labeled graph with its components walk and the forest stamps, and the integer-vertex suites keep drivers in the featured file where the language has no separate test project:

language	build SLOC	dependency	boundary note
c	451	libc only	fixed V-by-V adjacency with degree counters, both readings from one build, checks share the file with main, 62 of them
go	227	none	integer vertices, tests in separate files, HasCycleDirected returns the three-color verdict, 15 tests
java	469	jdk 27 stdlib	fixed V-by-V adjacency like c, one build for both readings, the components file alone switches to string labels over a first-seen id map
c#	160	bcl only	string labels and Dictionary adjacency, forest stamps and components in the same file, 11 tests
javascript	183	node stdlib	array-of-arrays adjacency, head cursor instead of dequeue, 16 tests
python	276	stdlib only	sorted adjacency pins the walk order, undirected cycle rule reads the parent, 43 checks
lua	415	lib.lua harness	adjacency grown by assignment, 1-based queues, checks ride in the module, 21 of them

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` and `Queue<T>` pages used by the implementation, `HashSet<T>` set operations, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 11 tests in chapter 10 of the samples suite. The seven-language layer verifies the same way: 5 C programs with 62 embedded checks under `make verify-c`, 15 Go tests, the java runner's 62 Ch10 checks over 5 files under `run-java-samples`, 16 `node --test` cases, 43 Python checks across 5 files, and 21 Lua checks under `run.lua`.

11. shortest paths

Chapter 10′s BFS finds shortest paths when every edge counts as one. Weights break it: three cheap edges can beat one expensive edge. This chapter builds the weighted answer for every input class, non-negative weights, negative edges, all pairs, and grids with a goal. Two refinements of the single-source pair follow the classics, 0-1 bfs for weights capped at one and d'esopo-pape as a queue-driven bellman-ford.

11.1. dijkstra

BFS with a priority queue instead of a fifo: always settle the unsettled vertex with the smallest known distance, relax its outgoing edges, done when the heap empties.

The dry run: the fixture is the labeled weighted graph, a to b at 4 and c at 2, b to c at 5 and d at 10, c to e at 3, d to f at 11, e to d at 4, asserted by the C# suite, and the integer suites carry their own fixtures, 0 3 1 4 7 and 0 7 9 20 20 11.

1. Seed a at 0 and relax its arcs: b takes 4, c takes 2, and c is the smaller offer waiting in the heap.
2. Settle c at 2: e takes 2 + 3 = 5.
3. Settle b at 4: the arc into d offers 4 + 10 = 14, d's first label.
4. Settle e at 5: the route through e offers 5 + 4 = 9, under 14, and d's label drops.
5. Settle d at 9: f takes 9 + 11 = 20, and f settles last when the heap empties.
6. Distances read a 0, b 4, c 2, d 9, e 5, f 20, the path to f walks a c e d f, all asserted, and bellman-ford agrees everywhere on the same graph.

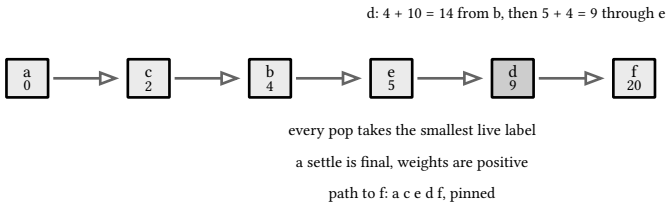


figure 76: the settle order as one chain, smallest live label first, d's label dropping once before it settles

The 9 through c and e against the 14 direct is the trap the suite pins, and the listings below settle it in seven languages.

```c
static void hpush(int key, int v) {
  heap[hn].key = key;
  heap[hn].v = v;
  int i = hn++;
  while (i > 0 && heap[(i - 1) / 2].key > heap[i].key) {
    hswap((i - 1) / 2, i);
    i = (i - 1) / 2;
  }
}

static HN hpop(void) {
  HN top = heap[0];
  heap[0] = heap[--hn];
  int i = 0;
  for (;;) {
    int l = 2 * i + 1, r = 2 * i + 2, m = i;
    if (l < hn && heap[l].key < heap[m].key)
      m = l;
    if (r < hn && heap[r].key < heap[m].key)
      m = r;
    if (m == i)
      break;
    hswap(i, m);
    i = m;
  }
  return top;
}
```

listing 278: c, the (dist, vertex) min-heap, push and pop (source: dsa/samples-c/src/Ch11/dijkstra.c, lines 49-75)

```c
static void dijkstra(int src) {
  for (int v = 0; v < V; v++) {
    dist[v] = INF;
    pred[v] = -1;
  }
  dist[src] = 0;
  hn = 0;
  hpush(0, src);
  while (hn) {
    HN cur = hpop();
    int u = cur.v;
    if (cur.key != dist[u])
      continue; // stale lazy-deletion entry
    for (int i = 0; i < deg[u]; i++) {
      int v = adj[u][i], nd = dist[u] + w[u][i];
      if (nd < dist[v]) {
        dist[v] = nd;
        pred[v] = u;
        hpush(nd, v);
      }
    }
  }
}
```

listing 279: c, dijkstra over the heap, stale entries skipped (source: dsa/samples-c/src/Ch11/dijkstra.c, lines 79-101)

```
type item struct {
  vertex int
  dist   int64
}

type distQueue []item

func (q distQueue) Len() int           { return len(q) }
func (q distQueue) Less(i, j int) bool { return q[i].dist < q[j].dist }
func (q distQueue) Swap(i, j int)      { q[i], q[j] = q[j], q[i] }
func (q *distQueue) Push(x any)        { *q = append(*q, x.(item)) }
func (q *distQueue) Pop() any {
  old := *q
  last := old[len(old)-1]
  *q = old[:len(old)-1]
  return last
}

// Dijkstra fills the shortest distances from source, with -1 marking
// unreachable vertices. Negative weights are not allowed under this
// discipline.
func Dijkstra(adj [][]WEdge, source int) []int64 {
  const inf = int64(1) << 62
  dist := make([]int64, len(adj))
  for i := range dist {
    dist[i] = inf
  }
  dist[source] = 0
  q := &distQueue{{vertex: source, dist: 0}}
  heap.Init(q)
  for q.Len() > 0 {
    cur := heap.Pop(q).(item)
    if cur.dist > dist[cur.vertex] {
      continue // stale entry: something shorter already landed
    }
    for _, e := range adj[cur.vertex] {
      if next := cur.dist + e.Weight; next < dist[e.To] {
        dist[e.To] = next
        heap.Push(q, item{vertex: e.To, dist: next})
      }
    }
  }
  for i, d := range dist {
    if d == inf {
      dist[i] = -1
    }
  }
  return dist
}
```

listing 280: go, the container/heap adapter and the relax loop (source: dsa/samples-go/ch11/dijkstra.go, lines 16-64)

This chapter is the one place the matrix lets the shipped queue in: the heap is a supporting container here, not the lesson. C and Lua hand-build one in the same file, and Java does too even though its `PriorityQueue` sits one import away, the heap entries and the stale-entry skip are the taught mechanism, so it stays hand-rolled. JavaScript carries a private heap class, Go wires container/heap through a five-method adapter, Python rides heapq, and the C# code

keeps its indexed heap with a true decrease-key while the BCL PriorityQueue plus its O(n) Remove stands as the shipped counterpart, chapter 8's callout:

```
  static int[] dist = new int[V], pred = new int[V];

  static void dijkstra(int src) {
    for (int v = 0; v < V; v++) {
      dist[v] = INF;
      pred[v] = -1;
    }
    dist[src] = 0;
    hn = 0;
    hpush(0, src);
    while (hn > 0) {
      HN cur = hpop();
      int u = cur.v();
      if (cur.key() != dist[u])
        continue; // stale lazy-deletion entry
      for (int i = 0; i < deg[u]; i++) {
        int v = adj[u][i], cand = dist[u] + w[u][i];
        if (cand < dist[v]) {
          dist[v] = cand;
          pred[v] = u;
          hpush(cand, v);
        }
      }
    }
  }

  // cross-check: relax every edge V-1 times; returns false when every
```

listing 281: java, dijkstra over the hand-built heap, stale entries skipped, PriorityQueue left out on purpose (source: dsa/samples-java/src/Ch11/Dijkstra.java, lines 78-104)

```
    public static Result Dijkstra(Dictionary<string, List<Edge>> graph, string source)
    {
        var distance = new Dictionary<string, long>();
        var previous = new Dictionary<string, string?>();
        foreach (string v in graph.Keys)
        {
            distance[v] = long.MaxValue;
            previous[v] = null;
        }
        distance[source] = 0;

        var heap = new IndexedHeap();
        heap.PushOrUpdate(source, 0);
        while (heap.PopMin(out object? raw, out long dist))
        {
            string vertex = (string)raw!;
            if (dist > distance[vertex])
                continue; // stale entry, the lazy deletion discipline
            foreach (Edge edge in graph[vertex])
            {
                long candidate = dist + edge.Weight;
                if (candidate < distance[edge.To])
                {
                    distance[edge.To] = candidate;
                    previous[edge.To] = vertex!;
                    heap.PushOrUpdate(edge.To, candidate);
                }
            }
        }
        return new Result(distance, previous);
    }
```

listing 282: c#, dijkstra with an indexed heap, stale entry skipping (source: dsa/samples/src/Ch11/Paths.cs, lines 25-56)

The heap holds one entry per improvement, so a vertex can appear with several priorities and the stale ones are skipped on pop, the lazy deletion discipline, which is also how the BCL-only version is typically written given chapter 8's O(n) `Remove`. The indexed heap here carries a real decrease-key instead, and both runtimes are logarithmic per edge. The test graph pins the classic trap: the path through c and e at total 9 beats the direct a b d route at 14, and the reconstruction walks the predecessor chain backwards.

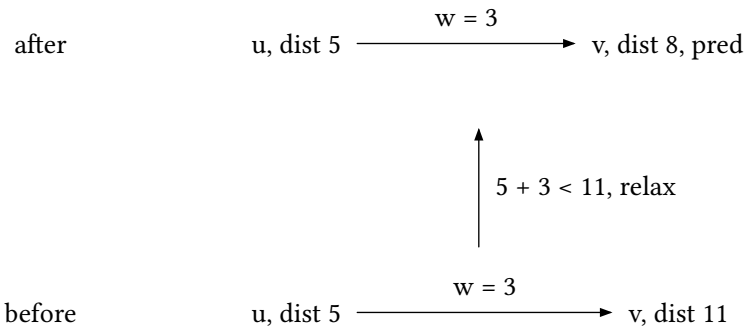


figure 77: one relax operation, before above, after below

WARNING: the non-negative precondition is binding

Dijkstra's correctness proof needs every edge weight at least zero, because it commits a vertex the moment it is popped and never revisits. A negative edge can unlock a shorter path after the commitment. The tests show the algorithm silently returning suboptimal answers on such graphs, which is why bellman-ford exists.

Four more languages settle their own fixtures:

```
class LocalMinHeap {
  #h = [];

  push(key, v) {
    this.#h.push({ key, v });
    let i = this.#h.length - 1;
    while (i > 0 && this.#h[Math.floor((i - 1) / 2)].key > this.#h[i].key) {
      [this.#h[Math.floor((i - 1) / 2)], this.#h[i]] = [
        this.#h[i],
        this.#h[Math.floor((i - 1) / 2)],
      ];
      i = Math.floor((i - 1) / 2);
    }
  }

  pop() {
    const top = this.#h[0];
    this.#h[0] = this.#h[this.#h.length - 1];
    this.#h.pop();
    let i = 0;
    for (;;) {
      const l = 2 * i + 1;
      const r = 2 * i + 2;
      let m = i;
      if (l < this.#h.length && this.#h[l].key < this.#h[m].key) m = l;
      if (r < this.#h.length && this.#h[r].key < this.#h[m].key) m = r;
      if (m === i) break;
      [this.#h[i], this.#h[m]] = [this.#h[m], this.#h[i]];
      i = m;
    }
    return top;
  }

  get size() {
    return this.#h.length;
  }
}
```

listing 283: javascript, the local min-heap kept inline on purpose (source: dsa/samples-js/src/ch11-dijkstra.mjs, lines 5-41)

```javascript
export function dijkstra(adj, src) {
  const dist = new Array(adj.length).fill(-1);
  const pred = new Array(adj.length).fill(-1);
  dist[src] = 0;
  const heap = new LocalMinHeap();
  heap.push(0, src);
  while (heap.size) {
    const { key, v: u } = heap.pop();
    if (key !== dist[u]) continue; // stale lazy-deletion entry
    for (const { to, weight } of adj[u]) {
      const nd = dist[u] + weight;
      if (dist[to] === -1 || nd < dist[to]) {
        dist[to] = nd;
        pred[to] = u;
        heap.push(nd, to);
      }
    }
  }
  return { dist, pred };
}
```

listing 284: javascript, dijkstra with −1 for unreachable (source: dsa/samples-js/src/ch11-dijkstra.mjs, lines 45-64)

```python
def dijkstra(adj, source):
    # adj[u] holds (weight, v) pairs; -1 marks unreachable, like the go
    # and c listings
    dist = [-1] * len(adj)
    dist[source] = 0
    heap = [(0, source)]
    while heap:
        d, u = heappop(heap)
        if d > dist[u]:  # a stale entry: something shorter already landed
            continue
        for w, v in adj[u]:
            nd = d + w
            if dist[v] == -1 or nd < dist[v]:
                dist[v] = nd
                heappush(heap, (nd, v))
    return dist
```

listing 285: python, heapq as the supporting container (source: dsa/samples-py/src/Ch11/dijkstra.py, lines 15-30)

```lua
local function hpush(h, key, v)
  h[#h + 1] = {key, v}
  local i = #h
  while i > 1 and h[i // 2][1] > h[i][1] do
    h[i // 2], h[i] = h[i], h[i // 2]
    i = i // 2
  end
end

local function hpop(h)
  local top = h[1]
  h[1] = h[#h]
  h[#h] = nil
  local i, n = 1, #h
  while true do
    local l, r, m = 2 * i, 2 * i + 1, i
    if l ≤ n and h[l][1] < h[m][1] then m = l end
    if r ≤ n and h[r][1] < h[m][1] then m = r end
    if m == i then break end
    h[i], h[m] = h[m], h[i]
    i = m
  end
  return top[1], top[2]
end
```

listing 286: lua, the hand-built heap of key-value pairs (source: dsa/samples-lua/ch11_dijkstra.lua, lines 24-47)

```lua
local function dijkstra(adj, src)
  local dist, pred = {}, {}
  for v = 0, V - 1 do
    dist[v] = INF
    pred[v] = -1
  end
  dist[src] = 0
  local h = {}
  hpush(h, 0, src)
  while #h > 0 do
    local key, u = hpop(h)
    if key == dist[u] then -- skip stale lazy-deletion entries
      for _, e in ipairs(adj[u]) do
        local v, nd = e[1], dist[u] + e[2]
        if nd < dist[v] then
          dist[v] = nd
          pred[v] = u
          hpush(h, nd, v)
        end
      end
    end
  end
  return dist, pred
end
```

listing 287: lua, dijkstra skipping stale lazy-deletion entries (source: dsa/samples-lua/ch11_dijkstra.lua, lines 51-74)

Measured across the suites: Go, JavaScript, and Python share one fixture where the direct 0 to 1 edge of weight 4 loses to the two-hop route of 3, pinning 0 3 1 4 7 from the source and −1 2 0 3 6 from vertex 2. C, Java, and Lua share the classic 6-vertex undirected fixture, pinning 0 7 9 20 20 11 from vertex 0, the whole predecessor chain, 11 12 13 for vertices 0 1 3 from source 5, and a bellman-ford cross-check over the same edges. The frozen C# suite

pins its own labeled graph, 9 through c and e against 14 direct. Nobody here does duplicate work: every version pushes on each improvement and skips the stale pop.

11.2. bellman-ford

Relax every edge, n minus 1 rounds. Nothing clever, entirely general: negative edges allowed, and one more full pass that still improves anything proves a negative cycle.

The dry run: the fixture is the negative-edge store, s to a at 4, s to b at 5, a to c at −3, b to a at −1, asserted by the C# suite, and the integer suites carry their own, 0 1 −1 0 in Go, JavaScript, and Python.

1. Round 1 opens with the source arcs: a takes 0 + 4 = 4, b takes 0 + 5 = 5.
2. The negative arc follows in store order: c takes 4 - 3 = 1.
3. The last arc offers the detour through b into a, 5 - 1 = 4, which ties the incumbent 4, and the strict test refuses the write.
4. Round 2 relaxes all four arcs again, nothing improves, and the quiet round exits early, two of the three possible rounds run.
5. The verdict pass changes nothing, no negative cycle, and the suite pins a at 4 and c at 1.
6. On the cycle fixture the arcs 1, −2, −2 loop at 1 - 2 - 2 = −3, the verdict pass still improves, and the call throws.

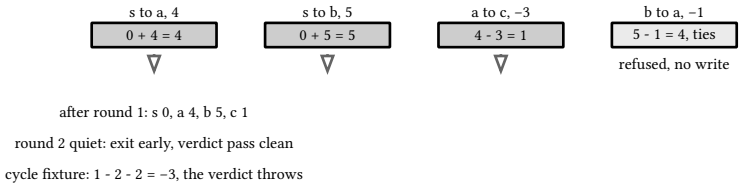


figure 78: round 1 as four arcs in store order, three writes and one refused tie, the quiet round underneath

The 4 and the 1 beside the refused tie are the pinned triple, and the listings below run the rounds in seven languages.

```c
static int bellman(const int edges[][3], int e, int src) {
  for (int v = 0; v < V; v++)
    dist[v] = INF;
  dist[src] = 0;
  for (int round = 0; round < V - 1; round++) {
    int changed = 0;
    for (int i = 0; i < e; i++) {
      int u = edges[i][0], v = edges[i][1], w = edges[i][2];
      if (dist[u] != INF && dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        changed = 1;
      }
    }
    if (!changed)
      break;
  }
  for (int i = 0; i < e; i++) {
    int u = edges[i][0], v = edges[i][1], w = edges[i][2];
    if (dist[u] != INF && dist[u] + w < dist[v])
      return 0; // a V-th relaxation succeeded
  }
  return 1;
}
```

listing 288: c, the rounds, the quiet-round exit, the V-th pass verdict (source: dsa/samples-c/src/Ch11/bellman.c, lines 22-44)

```go
func BellmanFord(n int, edges []BellEdge, source int) (dist []int64, ok bool) {
  const inf = int64(1) << 62
  dist = make([]int64, n)
  for i := range dist {
    dist[i] = inf
  }
  dist[source] = 0
  for round := 0; round < n-1; round++ {
    changed := false
    for _, e := range edges {
      if dist[e.From] == inf {
        continue
      }
      if next := dist[e.From] + e.Weight; next < dist[e.To] {
        dist[e.To] = next
        changed = true
      }
    }
    if !changed {
      break // already converged: the extra rounds cannot help
    }
  }
  for _, e := range edges {
    if dist[e.From] == inf {
      continue
    }
    if dist[e.From]+e.Weight < dist[e.To] {
      return dist, false // still improving: negative cycle
    }
  }
  for i, d := range dist {
    if d == inf {
      dist[i] = -1
    }
  }
  return dist, true
}
```

listing 289: go, negative weights in, negative cycles reported, −1 for unreachable (source: dsa/samples-go/ch11/bellman.go, lines 15-51)

```
  static boolean bellman(int[][] edges, int src) {
    for (int v = 0; v < V; v++)
      dist[v] = INF;
    dist[src] = 0;
    for (int round = 0; round < V - 1; round++) {
      boolean changed = false;
      for (int[] e : edges) {
        int u = e[0], v = e[1], w = e[2];
        if (dist[u] != INF && dist[u] + w < dist[v]) {
          dist[v] = dist[u] + w;
          changed = true;
        }
      }
      if (!changed)
        break;
    }
    for (int[] e : edges) {
      int u = e[0], v = e[1], w = e[2];
      if (dist[u] != INF && dist[u] + w < dist[v])
        return false; // a V-th relaxation succeeded
    }
    return true;
  }
```

listing 290: java, the rounds, the quiet-round exit, the V-th pass verdict (source: dsa/samples-java/src/Ch11/Bellman.java, lines 22-44)

```
    public static Result BellmanFord(Dictionary<string, List<Edge>> graph, string source)
    {
        var distance = new Dictionary<string, long>();
        var previous = new Dictionary<string, string?>();
        foreach (string v in graph.Keys)
        {
            distance[v] = long.MaxValue;
            previous[v] = null;
        }
        distance[source] = 0;

        var edges = graph.SelectMany(kv => kv.Value.Select(e => (From: kv.Key, e.To, e.Weight)))
            .ToList();

        for (int round = 0; round < graph.Count - 1; round++)
        {
            bool changed = false;
            foreach ((string from, string to, long w) in edges)
            {
                if (distance[from] == long.MaxValue)
                    continue;
                if (distance[from] + w < distance[to])
                {
                    distance[to] = distance[from] + w;
                    previous[to] = from;
                    changed = true;
                }
            }
            if (!changed)
                break; // converged early
        }

        foreach ((string from, string to, long w) in edges)
        {
            if (distance[from] != long.MaxValue && distance[from] + w < distance[to])
                throw new InvalidOperationException("negative weight cycle reachable");
        }
        return new Result(distance, previous);
    }
```

listing 291: c#, relaxation rounds, early exit on convergence, the negative cycle check (source: dsa/samples/src/Ch11/Paths.cs, lines 58-96)

The n-1 bound is the depth of the longest possible shortest path, a simple chain, and the early exit means nice graphs finish in a few rounds. The equality test runs dijkstra and bellman-ford on the same positive graph and asserts identical distances everywhere, two independent implementations agreeing.

```javascript
export function bellmanFord(n, edges, src) {
  const dist = new Array(n).fill(Infinity);
  dist[src] = 0;
  for (let round = 0; round < n - 1; round++) {
    let changed = false;
    for (const [u, v, w] of edges) {
      if (dist[u] === Infinity) continue;
      if (dist[v] === Infinity || dist[u] + w < dist[v]) {
        dist[v] = dist[u] + w;
        changed = true;
      }
    }
    if (!changed) break;
  }
  for (const [u, v, w] of edges) {
    if (dist[u] !== Infinity && dist[u] + w < dist[v])
      return { dist: dist.map((d) => (d === Infinity ? -1 : d)), ok: false };
  }
  return { dist: dist.map((d) => (d === Infinity ? -1 : d)), ok: true };
}
```

listing 292: javascript, infinity internally so a real −1 distance survives (source: dsa/samples-js/src/ch11-bellman.mjs, lines 8-27)

```python
def bellman_ford(vertices, edges, source):
    # returns (dist, ok): dist[v] is None for unreachable, ok is False
    # when a reachable negative cycle exists
    dist = [INF] * vertices
    dist[source] = 0
    for _ in range(vertices - 1):  # longest simple path has V-1 edges
        changed = False
        for u, v, w in edges:
            if dist[u] is not None and (dist[v] is None or dist[u] + w < dist[v]):
                dist[v] = dist[u] + w
                changed = True
        if not changed:
            break
    for u, v, w in edges:  # the V-th pass: improvement means a cycle
        if dist[u] is not None and (dist[v] is None or dist[u] + w < dist[v]):
            return dist, False
    return dist, True
```

listing 293: python, none for unreachable, ok flag on the cycle (source: dsa/samples-py/src/Ch11/bellman.py, lines 17-33)

```lua
local function bellman(edges, src)
  local dist = {}
  for v = 0, 4 do dist[v] = INF end
  dist[src] = 0
  for _ = 1, 4 do
    local changed = false
    for _, e in ipairs(edges) do
      local u, v, w = e[1], e[2], e[3]
      if dist[u] ~= INF and dist[u] + w < dist[v] then
        dist[v] = dist[u] + w
        changed = true
      end
    end
    if not changed then break end
  end
  for _, e in ipairs(edges) do
    local u, v, w = e[1], e[2], e[3]
    if dist[u] ~= INF and dist[u] + w < dist[v] then
      return false, dist -- a V-th relaxation succeeded
    end
  end
  return true, dist
end
```

listing 294: lua, the same contract, two return values (source: dsa/samples-lua/ch11_bellman.lua, lines 9-31)

Measured across the suites: Go, JavaScript, and Python share the negative-edge fixture and pin 0 1 −1 0, the −2 edge making vertex 2 cheaper through 1, refuse the 1 2 1 cycle, and ignore a cycle the source cannot reach, 0 5 −1 −1 −1 in Go and JavaScript. C, Java, and Lua share their own fixture, distances 0 1 2 3 with vertex 4 stranded, the −3 cycle detected, and the same cycle invisible when only an unreachable vertex can see it. Zero-weight edges relax everywhere.

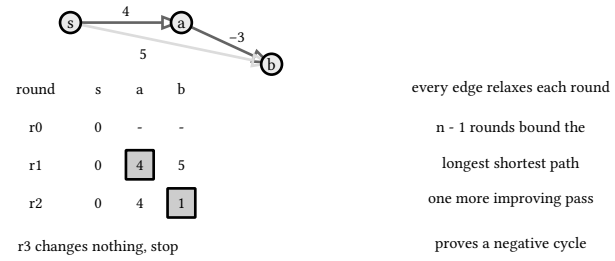


figure 79: bellman-ford, every edge relaxes each round, a quiet round exits early, one more improving pass proves a negative cycle

11.3. 0-1 bfs

Dijkstra pays for a heap because arbitrary weights create arbitrary candidate priorities. Cap every weight at 0 or 1 and the priorities collapse onto two consecutive values, which a plain deque serves: pop the front vertex, relax its edges, and when a distance improves, push that vertex to the front for weight 0 and to the back for weight 1, with the source seeded at distance 0 up front. Chapter 10's fifo queue wearing dijkstra's discipline, and nothing else in the machinery changes.

The safety argument is the monotone frontier. The deque always reads non-decreasing from front to back, and the values inside span at most one step. A weight 0 relaxation offers dist[u], never smaller than the value just popped, so a front push cannot unsort the row. A weight 1 relaxation offers dist[u] + 1, at least as large as anything still waiting, so a back push cannot unsort it either. That sorted order is exactly what dijkstra buys with the heap, so pops settle vertices in non-decreasing distance order and every pop is final. Pushes stay linear too: a vertex first

labeled across a 1 edge can drop exactly one more value through a 0 edge from the same distance class and never again, so each vertex enters the deque at most twice and the whole run costs O(V + E).

Where it earns its keep is modeling. Any state graph where some moves are free and the rest cost one, doors you walk through versus locks you pay to open, is a 0-1 graph, and distances come out linear where dijkstra would pay E log V and bellman-ford V E. Two buckets indexed by the current distance work just as well, the deque is the common spelling.

The dry run: no suite carries this facet, the numbers are hand-derived, and the fixture is a five-vertex digraph reading s to a at 1, s to c at 1, s to b at 0, b to c at 0, c to d at 0, a to d at 1.

1. The source seeds the front at 0, and the first pop reads its three arcs: a and c take 1 and enter the back, b takes 0 + 0 = 0 and enters the front, the row reads b a c.
2. Pop b at 0: c's label drops 1 to 0 + 0 = 0 across the free arc, a front push, and c now waits twice.
3. Pop c at 0: d takes 0 + 0 = 0 at the front, the row reads d a c.
4. Pop d at 0, then a at 1: a's paid arc offers d 1 + 1 = 2 against the settled 0 and is refused.
5. The second c pops at the stale 1, dist says 0, and is skipped.
6. Six pops, six pushes over five vertices and six arcs, and the distances land s 0, b 0, c 0, d 0, a 1: the free corridor reached d before the paid arc reached a.

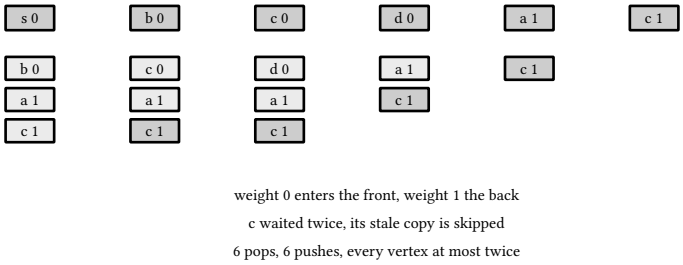


figure 80: the run as one column per pop, the deque waiting under each, the stale second c shaded

The four zeros against the lone 1 is the run's landing, corridor first, paid arc behind.

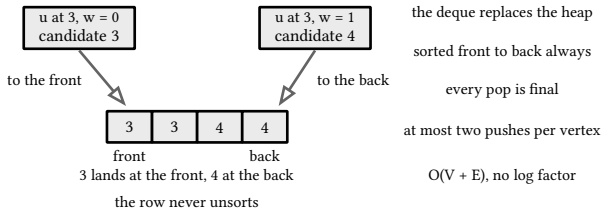


figure 81: 0-1 bfs, the deque stays sorted by distance, weight 0 pushes the front, weight 1 pushes the back, every pop is final

This facet ships prose only this edition: the wave scoped it as an extension with no new suite, and the dijkstra listings above already pin the machinery it reuses.

11.4. d'esopo-pape

Bellman-ford relaxes every edge every round because it cannot tell which vertices still matter. D'esopo-pape, a label-correcting refinement, keeps a worklist instead: only a vertex whose label moved goes back to work, and only its edges get rescanned. The queue discipline is the whole idea. A vertex queued for the first time enters the back. A vertex already waiting is not duplicated, its label improves where it sits. A vertex that had already finished and improves again jumps to the front, the bet is that its label is still moving and its neighbors should hear the news early.

It stays bellman-ford where it counts: negative edges are fine, and a reachable negative cycle still runs forever, so the same cycle guard or an iteration cap belongs in any production version. The worst case also gets worse: adversarial orders exist where the front-jump reactivations multiply into exponential time, worth knowing before reaching for this under a judge's time limit. The received experience, passed along by the cp-algorithms article without a benchmark behind it, is that it usually runs fast, often ahead of dijkstra, and the same label-correcting

loop over a plain fifo queue is better known as spfa, the shortest path faster algorithm. Treat the speed as folklore: often true, never promised.

The dry run: no suite carries this facet, the numbers are hand-derived, and the fixture is a five-vertex digraph reading s to t at 10, s to a at 1, s to b at 4, a to b at 0, b to c at 2, c to t at 1.

1. Pop s: three first-time labels enter the back in arc order, t at 0 + 10 = 10, a at 1, b at 4, the queue reads t a b.
2. Pop t at 10: it finishes, and with no arc out of t the label looks final.
3. Pop a at 1: b is still waiting, the free arc drops its label to 1 + 0 = 1 in place, no second copy enters.
4. Pop b at 1: c takes 1 + 2 = 3 at the back.
5. Pop c at 3: t's label drops to 3 + 1 = 4, and because t already finished it jumps the front, the queue reads t alone.
6. Pop t again at 4: the route s a b c t stands against the 10 direct, and t is the one vertex the worklist ran twice.

pop	event	queue after	labels
s	t, a, b first time, to the back	t a b	t 10, a 1, b 4
t	finishes at 10	a b	unchanged
a	b improves while waiting	b	b 1
b	c first time, to the back	c	c 3
c	t improves, jumps the front	t	t 4
t	runs again, label final		t 4

The 4 against the 10 with one vertex run twice is the whole trade, and the bellman-ford listings above carry the loop this facet refines.

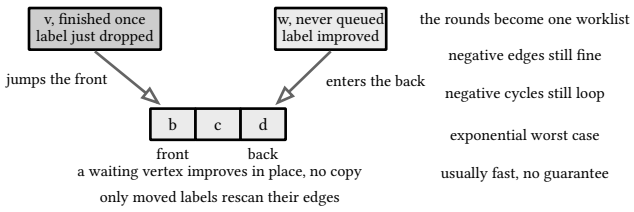


figure 82: d'esopo-pape, one worklist replaces the rounds, first discovery enters the back, a finished vertex that improves again jumps the front, waiting vertices are not duplicated

The bellman-ford listings above already pin the machinery this facet refines.

11.5. floyd-warshall

All pairs shortest paths by dynamic programming over intermediate vertices: `dist[i, j]` is the best route using only the first k vertices as way points, and k grows in the outermost loop.

The dry run: the fixture is the same labeled weights loaded into a 6 by 6 matrix, asserted by the C# suite against the dijkstra row, and C and Lua pin a 7-arc digraph whose row 0 reads 0 3 5 6.

1. The load writes the direct arcs, so row a opens 0 4 2 with the far cells unreachable.
2. k = b: d reads 4 + 10 = 14 through b, the two-hop route.
3. k = c: e reads 2 + 3 = 5.
4. k = d: the e row buys the arc to f at 4 + 11 = 15, and row a's f reads 14 + 11 = 25.
5. k = e: d drops to 5 + 4 = 9 and f drops to 5 + 15 = 20, both through e.
6. Row a finishes 0 4 2 9 5 20, every cell equal to the dijkstra single-source answers, the cross-check the suite asserts.

k	writes	row a after
load	the direct arcs	0 4 2 inf inf inf
b	d = 4 + 10	0 4 2 14 inf inf
c	e = 2 + 3	0 4 2 14 5 inf

k	writes	row a after
d	e to f = 4 + 11, a to f = 14 + 11	0 4 2 14 5 25
e	d = 5 + 4, f = 5 + 15	0 4 2 9 5 20

The 9 and the 20 in the finished row are the cells the suite pins, and the listings below close the loop in seven languages.

```
static void load(const int edges[][3], int e) {
  for (int i = 0; i < V; i++)
    for (int j = 0; j < V; j++)
      d[i][j] = i == j ? 0 : INF;
  for (int i = 0; i < e; i++)
    d[edges[i][0]][edges[i][1]] = edges[i][2];
}

static void floyd(void) {
  for (int k = 0; k < V; k++)
    for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
        if (d[i][k] + d[k][j] < d[i][j])
          d[i][j] = d[i][k] + d[k][j];
}
```

listing 295: c, matrix load, the k-outermost triple loop (source: dsa/samples-c/src/Ch11/floyd.c, lines 21-35)

```
const InfWeight = int64(1) << 61

// FloydWarshall computes all pairs shortest paths by letting every
// vertex k in turn act as a shortcut: dist[i][j] = min(dist[i][j],
// dist[i][k]+dist[k][j]). Negative cycles are out of scope here.
func FloydWarshall(adj [][]int64) [][]int64 {
  n := len(adj)
  dist := make([][]int64, n)
  for i := range dist {
    dist[i] = append([]int64(nil), adj[i]...)
  }
  for k := 0; k < n; k++ {
    for i := 0; i < n; i++ {
      for j := 0; j < n; j++ {
        through := dist[i][k] + dist[k][j]
        if through < dist[i][j] {
          dist[i][j] = through
        }
      }
    }
  }
  return dist
}
```

listing 296: go, closure over a copied matrix, 2^61 as infinity (source: dsa/samples-go/ch11/floyd.go, lines 5-27)

```
  static void load(int[][] edges) {
    for (int i = 0; i < V; i++)
      for (int j = 0; j < V; j++)
        d[i][j] = i == j ? 0 : INF;
    for (int[] e : edges)
      d[e[0]][e[1]] = e[2];
  }

  static void floyd() {
    for (int k = 0; k < V; k++)
      for (int i = 0; i < V; i++)
        for (int j = 0; j < V; j++)
          if (d[i][k] + d[k][j] < d[i][j])
            d[i][j] = d[i][k] + d[k][j];
  }
```

listing 297: java, matrix load, the k-outermost triple loop (source: dsa/samples-java/src/Ch11/Floyd.java, lines 21-35)

```
    public static long[,] FloydWarshall(long[,] adjacency)
    {
        int n = adjacency.GetLength(0);
        const long inf = long.MaxValue / 4;
        var dist = new long[n, n];
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                dist[i, j] = i == j ? 0 : adjacency[i, j] == 0 ? inf : adjacency[i, j];

        for (int k = 0; k < n; k++)
            for (int i = 0; i < n; i++)
                for (int j = 0; j < n; j++)
                    if (dist[i, k] + dist[k, j] < dist[i, j])
                        dist[i, j] = dist[i, k] + dist[k, j];
        return dist;
    }
```

listing 298: c#, three nested loops, the k in the middle of the recurrence outside (source: dsa/samples/src/Ch11/Paths.cs, lines 99-114)

Cubic time and quadratic space buys every pair's distance at once, which wins over n dijkstra runs when the graph is dense and small. The test cross-checks the first row against dijkstra's single source answers.

```
export function floydWarshall(d) {
  const n = d.length;
  const out = d.map((row) => [...row]);
  for (let k = 0; k < n; k++)
    for (let i = 0; i < n; i++)
      for (let j = 0; j < n; j++) {
        if (out[i][k] + out[k][j] < out[i][j])
          out[i][j] = out[i][k] + out[k][j];
      }
  return out;
}
```

listing 299: javascript, copy in, relax, return (source: dsa/samples-js/src/ch11-floyd.mjs, lines 4-14)

```python
def floyd_warshall(adj):
    size = len(adj)
    dist = [row[:] for row in adj]  # work on a copy, the input survives
    for k in range(size):
        for i in range(size):
            for j in range(size):
                if dist[i][k] + dist[k][j] < dist[i][j]:
                    dist[i][j] = dist[i][k] + dist[k][j]
    return dist
```

listing 300: python, the same triple loop on a copy (source: dsa/samples-py/src/Ch11/floyd.py, lines 16-24)

```lua
local function floyd(d)
  for k = 0, 3 do
    for i = 0, 3 do
      for j = 0, 3 do
        if d[i][k] + d[k][j] < d[i][j] then
          d[i][j] = d[i][k] + d[k][j]
        end
      end
    end
  end
  return d
end
```

listing 301: lua, nested numeric for loops over the table matrix (source: dsa/samples-lua/ch11_floyd.lua, lines 19-30)

Measured across the suites: Go, JavaScript, and Python pin the full closure of a 4-node matrix where 0 to 3 improves 10 to 9 through 1 and 2, Python adding a dense triangle whose 6 relaxes to 5 and a dijkstra sweep that agrees with every cell. C, Java, and Lua share a 7-edge digraph, pin all 16 cells, row 0 reading 0 3 5 6, check the triangle inequality over the finished closure, and keep an unreachable pair at infinity. Seven listings, one loop nest, k on the outside in every one.

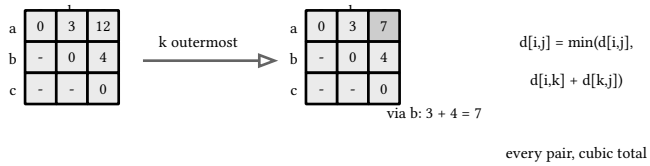


figure 83: floyd-warshall, k grows in the outermost loop, every pair's distance at cubic total

11.6. a star

Dijkstra explores by distance from the source. A star explores by distance plus an estimate to the goal, and when the estimate never overestimates, admissible, the first pop of the goal is still optimal.

The dry run: the fixture is the 5 by 5 grid walled across row 2 with the gap at column 4, searched from the corner to the corner, asserted by the C# suite, and the integer suites pin expansion counts under documented tie-breaks.

1. The heuristic at the start reads 4 + 0 = 4, an honest lower bound, and the wall kills every 4-step route.
2. The search moves right along row 0, each step buying one g and paying one h: f climbs 1 + 5 = 6, 2 + 6 = 8, 3 + 7 = 10.
3. The gap column tops out at 4 + 8 = 12, and that cell is the turn.
4. Down the gap: 5 + 7, 6 + 6, 7 + 5, every f 12, with the middle cell the pinned detour point.
5. The run turns left along row 4: 9 + 3, 10 + 2, 11 + 1, each left step spends one g and refunds one h, f holds to the goal.
6. The goal pops at g = 12 over 13 cells, both pinned, and the open 8 by 8 lands 7 + 7 = 14 over 15 cells the same way.

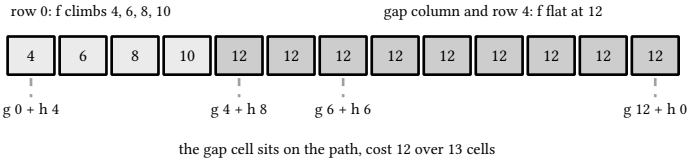


figure 84: f along the 13-cell detour, climbing by two per right step, flat 12 from the gap home

The 12 around the wall and the 14 across the open grid are the pinned pair, and the listings below search in seven languages.

```c
static int astar(void) {
  int gr = rows - 1, gc = cols - 1;
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++) {
      g[r][c] = -1;
      closed_[r][c] = 0;
      par[r][c] = -1;
    }
  hn = 0;
  seqc = 0;
  visited = 0;
  g[0][0] = 0;
  hpush(manhattan(0, 0), manhattan(0, 0), 0, 0);
  while (hn) {
    HN cur = hpop();
    if (closed_[cur.r][cur.c])
      continue;
    closed_[cur.r][cur.c] = 1;
    visited++;
    if (cur.r == gr && cur.c == gc)
      return g[gr][gc];
    for (int d = 0; d < 4; d++) {
      int nr = cur.r + DR[d], nc = cur.c + DC[d];
      if (!in_bounds(nr, nc) || closed_[nr][nc])
        continue;
      int ng = g[cur.r][cur.c] + 1;
      if (g[nr][nc] < 0 || ng < g[nr][nc]) {
        g[nr][nc] = ng;
        par[nr][nc] = cur.r * cols + cur.c;
        int h = manhattan(nr, nc);
        hpush(ng + h, h, nr, nc);
      }
    }
  }
  return -1;
}
```

listing 302: c, the search loop, closed set, tie-broken heap pushes (source: dsa/samples-c/src/Ch11/astar.c, lines 90-125)

```
  bestG := map[cell]int64{start: 0}
  parent := map[cell]cell{start: start}
  open := &openQueue{{at: start, g: 0, f: manhattan(start, goal)}}
  var counter int64
  for open.Len() > 0 {
    cur := heap.Pop(open).(openEntry)
    if cur.g > bestG[cur.at] {
      continue // superseded by a cheaper route
    }
    expanded++
    if cur.at == goal {
      for at := goal; ; at = parent[at] {
        path = append(path, at)
        if at == start {
          break
        }
      }
      for i, j := 0, len(path)-1; i < j; i, j = i+1, j-1 {
        path[i], path[j] = path[j], path[i]
      }
      return path, expanded
    }
    for _, d := range [4][2]int{{0, 1}, {1, 0}, {0, -1}, {-1, 0}} {
      next := cell{cur.at.r + d[0], cur.at.c + d[1]}
      if next.r < 0 || next.r ≥ rows || next.c < 0 || next.c ≥ cols {
        continue
      }
      if blocked[next.r][next.c] {
        continue
      }
      nextG := cur.g + 1
      if seen, ok := bestG[next]; ok && seen <= nextG {
        continue
      }
      bestG[next] = nextG
      parent[next] = cur.at
      counter++
      heap.Push(open, openEntry{
        at:      next,
        g:       nextG,
        f:       nextG + manhattan(next, goal),
        counter: counter,
      })
    }
  }
  return nil, expanded
}
```

listing 303: go, open queue by f then insertion order, parent rebuild (source: dsa/samples-go/ch11/astar.go, lines 54-100)

```
  static int astar() {
    int gr = rows - 1, gc = cols - 1;
    for (int r = 0; r < rows; r++)
      for (int c = 0; c < cols; c++) {
        g[r][c] = -1;
        closedSet[r][c] = 0;
        par[r][c] = -1;
      }
    hn = 0;
    seqc = 0;
    visited = 0;
    g[0][0] = 0;
    hpush(manhattan(0, 0), manhattan(0, 0), 0, 0);
    while (hn > 0) {
      HN cur = hpop();
      if (closedSet[cur.r()][cur.c()] != 0)
        continue;
      closedSet[cur.r()][cur.c()] = 1;
      visited++;
      if (cur.r() == gr && cur.c() == gc)
        return g[gr][gc];
      for (int d = 0; d < 4; d++) {
        int nr = cur.r() + DR[d], nc = cur.c() + DC[d];
        if (!inBounds(nr, nc) || closedSet[nr][nc] != 0)
          continue;
        int ng = g[cur.r()][cur.c()] + 1;
        if (g[nr][nc] < 0 || ng < g[nr][nc]) {
          g[nr][nc] = ng;
          par[nr][nc] = cur.r() * cols + cur.c();
          int h = manhattan(nr, nc);
          hpush(ng + h, h, nr, nc);
        }
      }
    }
    return -1;
  }
```

listing 304: java, the search loop, closed set, tie-broken heap pushes over the hand-built heap (source: dsa/samples-java/src/Ch11/Astar.java, lines 88-124)

```
    public static (long Cost, List<(int Row, int Col)> Path) AStarGrid(
        bool[,] blocked, (int Row, int Col) start, (int Row, int Col) goal)
    {
        int rows = blocked.GetLength(0), cols = blocked.GetLength(1);
        long Manhattan((int r, int c) a) =>
            Math.Abs(a.r - goal.Row) + Math.Abs(a.c - goal.Col);

        var open = new IndexedHeap();
        var gScore = new Dictionary<(int, int), long> { [start] = 0 };
        var cameFrom = new Dictionary<(int, int), (int, int)>();
        open.PushOrUpdate((start.Row, start.Col), Manhattan(start));

        while (open.PopMin(out object? raw, out long f))
        {
            var current = ((int Row, int Col))raw!;
            if (current == goal)
            {
                var path = new List<(int, int)> { current };
                while (cameFrom.TryGetValue(current, out (int, int) prev))
                {
                    current = prev;
                    path.Add(current);
                }
                path.Reverse();
                return (gScore[goal], path);
            }
            foreach ((int dr, int dc) in new[] { (-1, 0), (1, 0), (0, -1), (0, 1) })
            {
                int nr = current.Row + dr, nc = current.Col + dc;
                if (nr < 0 || nr ≥ rows || nc < 0 || nc ≥ cols || blocked[nr, nc])
                    continue;
                long tentative = gScore[current] + 1;
                var next = (nr, nc);
                if (tentative < gScore.GetValueOrDefault(next, long.MaxValue))
                {
                    gScore[next] = tentative;
                    cameFrom[next] = current;
                    open.PushOrUpdate(next, tentative + Manhattan(next));
                }
            }
        }
        throw new InvalidOperationException("no path to the goal");
    }

    /// <summary>Binary heap keyed by object identity-ish keys with decrease-key.</summary>
```

listing 305: c#, a star on a grid with the manhattan heuristic (source: dsa/samples/src/Ch11/Paths.cs, lines 117-161)

Manhattan distance is admissible for unit-cost grid steps because no route can beat the straight-line block count. The random-grid test asserts the deeper property, a star's cost equals the BFS distance on one hundred twenty random walls, optimal search without exploring the whole maze. A heavier point: the same code with the heuristic set to zero is dijkstra, and a heuristic that overestimates trades optimality for speed, which is exactly the dial game developers turn.

```
export function aStarGrid(blocked, start, goal) {
  const rows = blocked.length;
  const cols = blocked[0].length;
  const inBounds = (r, c) =>
    r ≥ 0 && r < rows && c ≥ 0 && c < cols && !blocked[r][c];
  const h = (r, c) => Math.abs(goal.r - r) + Math.abs(goal.c - c);

  const g = Array.from({ length: rows }, () => new Array(cols).fill(-1));
  const closed = Array.from({ length: rows }, () => new Array(cols).fill(false));
  const par = Array.from({ length: rows }, () => new Array(cols).fill(null));
  let expanded = 0;
  g[start.r][start.c] = 0;
  const heap = new AStarHeap();
  heap.push({ f: h(start.r, start.c), r: start.r, c: start.c });
  while (heap.size) {
    const cur = heap.pop();
    if (closed[cur.r][cur.c]) continue;
    closed[cur.r][cur.c] = true;
    expanded++;
    if (cur.r === goal.r && cur.c === goal.c) {
      const path = [];
      let at = { r: cur.r, c: cur.c };
      while (at) {
        path.push(at);
        at = par[at.r][at.c];
      }
      return { path: path.reverse(), expanded };
    }
    for (let d = 0; d < 4; d++) {
      const nr = cur.r + DR[d];
      const nc = cur.c + DC[d];
      if (!inBounds(nr, nc) || closed[nr][nc]) continue;
      const ng = g[cur.r][cur.c] + 1;
      if (g[nr][nc] === -1 || ng < g[nr][nc]) {
        g[nr][nc] = ng;
        par[nr][nc] = { r: cur.r, c: cur.c };
        const hh = h(nr, nc);
        heap.push({ f: ng + hh, r: nr, c: nc });
      }
    }
  }
  return { path: null, expanded };
}
```

listing 306: javascript, the grid walk over the private heap (source: dsa/samples-js/src/ch11-astar.mjs, lines 55-97)

```python
def astar_grid(blocked, start, goal):
    # blocked is a list of rows of booleans; every move costs 1
    rows, cols = len(blocked), len(blocked[0])
    tick = 0  # a counter keeps heap ties FIFO and deterministic

    def h(r, c):
        return abs(goal[0] - r) + abs(goal[1] - c)

    g = {start: 0}
    parent = {start: None}
    heap = [(h(*start), tick, start)]
    tick += 1
    expanded = 0
    while heap:
        _, _, cell = heappop(heap)
        expanded += 1
        if cell == goal:
            path = []
            while cell is not None:
                path.append(cell)
                cell = parent[cell]
            return path[::-1], expanded
        r, c = cell
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if not (0 <= nr < rows and 0 <= nc < cols) or blocked[nr][nc]:
                continue
            nxt, ng = (nr, nc), g[cell] + 1
            if nxt not in g or ng < g[nxt]:
                g[nxt] = ng
                parent[nxt] = cell
                heappush(heap, (ng + h(nr, nc), tick, nxt))
                tick += 1
    return None, expanded
```

listing 307: python, heapq with a tick counter for fifo ties (source: dsa/samples-py/src/Ch11/astar.py, lines 15-48)

```
function astar.run(rows, cols, wall)
  local gr, gc = rows - 1, cols - 1
  local function manhattan(r, c)
    return (gr - r) + (gc - c)
  end
  local g, closed, par = {}, {}, {}
  local h, seq, visited = {}, 0, 0
  g[0] = 0 -- cells keyed r*cols+c
  hpush(h, {f = manhattan(0, 0), h = manhattan(0, 0), seq = 0, r = 0, c = 0})
  while #h > 0 do
    local cur = hpop(h)
    local k = cur.r * cols + cur.c
    if not closed[k] then
      closed[k] = true
      visited = visited + 1
      if cur.r == gr and cur.c == gc then return g[k], visited, par end
      for d = 1, 4 do
        local nr, nc = cur.r + DR[d], cur.c + DC[d]
        if nr ≥ 0 and nr < rows and nc ≥ 0 and nc < cols
          and not wall[nr * cols + nc] then
          local nk = nr * cols + nc
          if not closed[nk] then
            local ng = g[k] + 1
            if g[nk] == nil or ng < g[nk] then
              g[nk] = ng
              par[nk] = k
              local hh = manhattan(nr, nc)
              seq = seq + 1
              hpush(h, {f = ng + hh, h = hh, seq = seq, r = nr, c = nc})
            end
          end
        end
      end
    end
  end
  return nil, visited, par
end
```

listing 308: lua, cells keyed r*cols+c, (f, h, seq) heap (source: dsa/samples-lua/ch11_astar.lua, lines 46-82)

Measured across the suites: C, Java, and Lua pin 7 expansions on an open 4 by 4 and 5 around a center-walled 3 by 3, under a documented tie-break, f first, then smaller h, then insertion order, with neighbors offered right down left up. Go, JavaScript, and Python use a 3 by 3 where every f ties at 4, so all 9 cells expand before the goal pops, and the wall fixture pins the 7-cell detour path, Go and Python counting 7 expansions while JavaScript bounds expansions by the grid size. Expansion counts are tie-break facts, not constants: reorder the neighbors and the counts move, the path lengths do not.

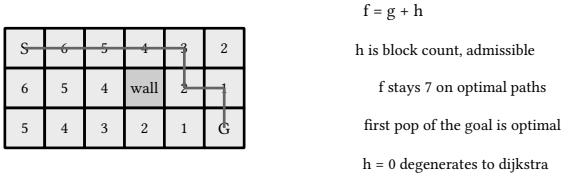


figure 85: a star on the grid, f = g + h, the manhattan heuristic never overestimates so the goal's first pop is optimal

11.7. across the seven languages

The build sizes count non-comment source lines over this chapter's four featured files per language. This is the matrix's one container exception: the priority queue under dijkstra and a star is the shipped one where a language ships it, hand-built where it does not:

language	build SLOC	dependency	boundary note
c	380	libc only	hand heaps in the dijkstra and astar files, 64 and 128 slot arrays, checks share the file with main, 55 of them
go	199	container/heap	the chapter exception in force, adapters per file, −1 marks unreachable, 11 tests
java	390	jdk 27 stdlib	PriorityQueue left unused, the exception declined because the heap entries and the stale-entry skip are the taught mechanism, hand heaps of record pairs, INF a billion
c#	218	bcl only	frozen suite, indexed heap with decrease-key in the same file, PriorityQueue named as the shipped twin, 10 tests
javascript	165	node stdlib	private local heaps inline by design, Infinity internally in bellman-ford so −1 distances survive, 12 tests
python	213	heapq	heapq as the supporting container, tick counters pin tie order, 26 checks
lua	350	lib.lua harness	hand heaps with 1-based index math, maxinteger as infinity, checks ride in the module, 16 of them

sources: learn.microsoft.com, `Dictionary<TKey,TValue>`, `HashSet<T>`, and `Queue<T>` pages used by the implementation, accessed 2026-09-08. The two prose-only extensions draw on cp-algorithms, the articles 0-1 BFS at https://cp-algorithms.com/graph/01_bfs.html and D'Esopo-Pape algorithm at https://cp-algorithms.com/graph/desopo_pape.html, both accessed 2026-09-20 under cc by-sa 4.0, taught here in our own words. Sample behavior verified by `make verify-csharp`, 10 tests in chapter 11 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 55 embedded checks under `make verify-c`, 11 Go tests, the java runner's 52 Ch11 checks over 4 files under `run-java-samples`, 12 `node --test` cases, 26 Python checks across 4 files, and 16 Lua checks under `run.lua`.

12. minimum spanning trees and flows

Two more greedy classics. Spanning trees connect everything for least total weight, and maximum flow asks how much can travel from a source to a sink through capacity-limited pipes. Both are solved by repeatedly taking what the current state allows.

12.1. union-find

The supporting actor for both chapters ahead of time: disjoint sets with two optimizations, path compression in find and union by rank. Both keep trees effectively flat.

The dry run: the fixture is the C# four-label script a through d and the 50-node chain, asserted by the C# suite; the siblings run their own scripts, C, Java, and Lua on 10 vertices, the rest on smaller unions.

1. Four Add calls stand a, b, c, and d each as its own root: SetCount reads 4.
2. Union(a, b) fuses the first pair, the equal ranks growing the survivor to rank 1: SetCount 4 - 1 = 3.
3. Union(c, d) drops the count to 2, with Connected(a, b) true and Connected(a, c) false across the divide.
4. Union(b, d) bridges the two trees and Connected(a, c) flips true transitively: SetCount 1.
5. A second pair x, y joins once, then the repeat Union(y, x) refuses: the count stays at 1.
6. The chain v0 through v49, unioned upward, sends Find("v49") climbing to v0, and the rehang pins the walked path one hop from the root.

action	SetCount	reads
add a, b, c, d	4	four roots, ranks 0
union(a, b)	3	survivor rank 1
union(c, d)	2	connected(a, b) true
union(b, d)	1	connected(a, c) true
union(x, y), then union(y, x)	1	second refuses, count holds
find(v49)	1	v0, the path rehung

The 4 to 1 count and the v49 rehang are the C# pins, and the listings below build the sets in seven languages.

```c
static int find(int v) {
  while (par[v] != v) {
    par[v] = par[par[v]]; // path halving compresses as it walks
    v = par[v];
  }
  return v;
}

static int union_(int a, int b) {
  a = find(a);
  b = find(b);
  if (a == b)
    return 0;
  if (rnk[a] < rnk[b]) {
    int t = a;
    a = b;
    b = t;
  }
  par[b] = a;
  if (rnk[a] == rnk[b])
    rnk[a]++;
  return 1;
}

static int components(void) {
  int c = 0;
  for (int v = 0; v < V; v++)
    if (find(v) == v)
      c++;
  return c;
}
```

listing 309: c, path halving in find, rank in union, the component count (source: dsa/samples-c/src/Ch12/dsu.c, lines 28-58)

```go
func (d *DSU) Find(x int) int {
  root := x
  for d.parent[root] ≠ root {
    root = d.parent[root]
  }
  for d.parent[x] ≠ root { // second pass: compression
    d.parent[x], x = root, d.parent[x]
  }
  return root
}

// Union merges the two sets. It reports false when they already
// shared a representative. The shallower tree hangs under the deeper
// one, so ranks never grow without reason.
func (d *DSU) Union(a, b int) bool {
  ra, rb := d.Find(a), d.Find(b)
  if ra == rb {
    return false
  }
  switch {
  case d.rank[ra] < d.rank[rb]:
    d.parent[ra] = rb
  case d.rank[ra] > d.rank[rb]:
    d.parent[rb] = ra
  default:
    d.parent[rb] = ra
    d.rank[ra]++
  }
  d.count--
  return true
}
```

listing 310: go, two-pass find with the compression rehang, rank-ordered union (source: dsa/samples-go/ch12/dsu.go, lines 28-58)

```
  static int find(int v) {
    while (par[v] != v) {
      par[v] = par[par[v]]; // path halving compresses as it walks
      v = par[v];
    }
    return v;
  }

  static boolean unionSet(int a, int b) {
    a = find(a);
    b = find(b);
    if (a == b)
      return false;
    if (rnk[a] < rnk[b]) {
      int t = a;
      a = b;
      b = t;
    }
    par[b] = a;
    if (rnk[a] == rnk[b])
      rnk[a]++;
    return true;
  }
```

listing 311: java, path halving in find, rank in union, the component count (source: dsa/samples-java/src/Ch12/Dsu.java, lines 29-52)

```
public sealed class DisjointSet
{
    private readonly Dictionary<string, string> _parent = new();
    private readonly Dictionary<string, int> _rank = new();

    public int SetCount { get; private set; }

    public void Add(string v)
    {
        if (_parent.ContainsKey(v))
            return;
        _parent[v] = v;
        _rank[v] = 0;
        SetCount++;
    }

    public string Find(string v)
    {
        Add(v);
        string root = v;
        while (_parent[root] != root)
            root = _parent[root];

        // path compression: rehang every node on the path directly at root
        string cur = v;
        while (_parent[cur] != root)
        {
            string next = _parent[cur];
            _parent[cur] = root;
            cur = next;
        }
        return root;
    }

    public bool Union(string a, string b)
    {
        string ra = Find(a), rb = Find(b);
        if (ra == rb)
            return false; // already one set, this edge would close a cycle
        if (_rank[ra] < _rank[rb])
            (ra, rb) = (rb, ra);
        _parent[rb] = ra;
        if (_rank[ra] == _rank[rb])
            _rank[ra]++;
        SetCount--;
        return true;
    }

    public bool Connected(string a, string b) => Find(a) == Find(b);
}
```

listing 312: c#, disjoint sets with compression and rank, cycle reporting unions (source: dsa/samples/src/Ch12/Flows.cs, lines 4-53)

Union returns false when both ends are already connected, which is precisely the cycle test kruskal needs, and the amortized cost with both optimizations is inverse Ackermann, effectively constant for any addressable n.

```
  find(v) {
    while (this.#parent[v] !== v) {
      this.#parent[v] = this.#parent[this.#parent[v]]; // path halving
      v = this.#parent[v];
    }
    return v;
  }

  // true when two components merged, false when already joined
  union(a, b) {
    a = this.find(a);
    b = this.find(b);
    if (a === b) return false;
    if (this.#rank[a] < this.#rank[b]) [a, b] = [b, a];
    this.#parent[b] = a;
    if (this.#rank[a] === this.#rank[b]) this.#rank[a]++;
    return true;
  }

  count() {
    let c = 0;
    for (let v = 0; v < this.#parent.length; v++)
      if (this.find(v) === v) c++;
```

listing 313: javascript, halving find, rank union, parent exposed for the proof (source: dsa/samples-js/src/ch12-dsu.mjs, lines 13-36)

```
    def find(self, x):
        root = x
        while self.parent[root] != root:
            root = self.parent[root]
        while self.parent[x] != root:  # compression: rehang on the root
            self.parent[x], x = root, self.parent[x]
        return root

    def union(self, a, b):
        ra, rb = self.find(a), self.find(b)
        if ra == rb:
            return False  # already one set
        if self.rank[ra] < self.rank[rb]:  # taller tree absorbs
            ra, rb = rb, ra
        self.parent[rb] = ra
        if self.rank[ra] == self.rank[rb]:
            self.rank[ra] += 1
        return True

    def components(self):
        return len({self.find(i) for i in range(len(self.parent))})
```

listing 314: python, find walks then rehangs, union by rank, component count (source: dsa/samples-py/src/Ch12/dsu.py, lines 18-38)

```
end

local function find(d, v)
  while d.par[v] ~= v do
    d.par[v] = d.par[d.par[v]] -- path halving compresses as it walks
    v = d.par[v]
  end
  return v
end

-- returns true when two sets merged, false when already joined
local function union(d, a, b)
  a = find(d, a)
  b = find(d, b)
  if a == b then return false end
  if d.rnk[a] < d.rnk[b] then a, b = b, a end
  d.par[b] = a
  if d.rnk[a] == d.rnk[b] then d.rnk[a] = d.rnk[a] + 1 end
  return true
end

local function components(d)
```

listing 315: lua, halving find over plain tables, union by rank (source: dsa/samples-lua/ch12_dsu.lua, lines 15-36)

Measured across the suites: C, Java, and Lua share one 10-vertex script, 4 components after six unions, a redundant union returning false and changing nothing, and a find that leaves 8 pointing straight at its root. JavaScript chains 4 into 1 to make the compression visible and checks the rank grew, while Go and Python pin counts through their own scripts with the same refusal. The accelerations differ in flavor: C, Java, JavaScript, and Lua halve the path as they walk, the C# and Go finds walk to the root then rehang the whole path, and Python ships both, a rehang in the standalone file and a halving copy inside its kruskal below.

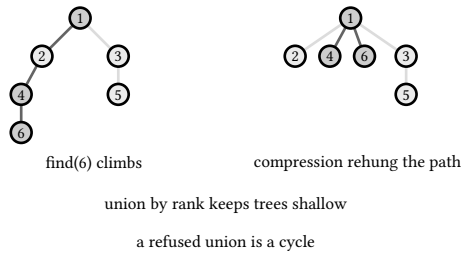


figure 86: union-find, find compresses the walked path, union by rank refuses the cycle-closing edge

12.2. kruskal

Kruskal sorts all edges and adds each unless it closes a cycle, the DSU from the section above answering the cycle question in near constant time.

The dry run: the fixture is the C# weighted graph, vertices a through d over edges a-b 1, a-c 4, b-c 2, b-d 6, c-d 3; Go, JavaScript, and Python run a 5-edge graph to the same total and C, Java, and Lua a 6-edge one at 13.

1. OrderBy weight lines the candidates up: a-b at 1, b-c at 2, c-d at 3, a-c at 4, b-d at 6.
2. a-b joins a and b, then b-c pulls c into that component: two edges down, d still alone.
3. c-d spans the last vertex: three edges, the n - 1 = 4 - 1 = 3 a tree of 4 vertices needs, and the loop breaks.
4. a-c at 4 would close the a-b-c triangle and b-d at 6 meets an old friend: both unions refuse.
5. The pinned reads: weights 1, 2, 3 in pick order, tree.Count 3, and 1 + 2 + 3 = 6 total.

weight	edge	union	tree so far	total
1	a-b	joins	a-b	1
2	b-c	joins	a-b, b-c	3
3	c-d	joins	a-b, b-c, c-d	6
4	a-c	refuses	unchanged	6
6	b-d	refuses	unchanged	6

The refused 4 and 6 beside the pinned total 6 are the C# shape, and the listings below sort and union in seven languages.

```
static int cmp_edge(const void *x, const void *y) {
  const int *a = x, *b = y; // rows are {u, v, w}
  return (a[2] > b[2]) - (a[2] < b[2]);
}

typedef struct {
  int u, v, w;
} Chosen;

// returns the number of edges taken
static int kruskal(const int edges[E][3], Chosen *out, int *total) {
  int sorted[E][3];
  for (int i = 0; i < E; i++)
    for (int j = 0; j < 3; j++)
      sorted[i][j] = edges[i][j];
  qsort(sorted, E, sizeof sorted[0], cmp_edge);
  for (int v = 0; v < V; v++)
    par[v] = v;
  int k = 0;
  *total = 0;
  for (int i = 0; i < E && k < V - 1; i++)
    if (union_(sorted[i][0], sorted[i][1])) {
      out[k].u = sorted[i][0];
      out[k].v = sorted[i][1];
      out[k].w = sorted[i][2];
      *total += sorted[i][2];
      k++;
    }
  return k;
}
```

listing 316: c, sort by weight, union accepts, the spanning count (source: dsa/samples-c/src/Ch12/kruskal.c, lines 40-69)

```go
func KruskalMST(n int, edges []UndEdge) (tree []UndEdge, total int64) {
  sorted := slices.Clone(edges)
  slices.SortFunc(sorted, func(a, b UndEdge) int {
    return cmp.Compare(a.Weight, b.Weight)
  })
  dsu := NewDSU(n)
  for _, e := range sorted {
    if dsu.Union(e.A, e.B) {
      tree = append(tree, e)
      total += e.Weight
      if len(tree) == n-1 {
        break // spanning: nothing left to connect
      }
    }
  }
  return tree, total
}
```

listing 317: go, slices sort, dsu unions, early break at spanning (source: dsa/samples-go/ch12/kruskal.go, lines 18-34)

```java
  record Chosen(int u, int v, int w) {}

  record Tree(Chosen[] edges, int total) {}

  static Tree kruskal(int[][] edges) {
    int[][] sorted = edges.clone();
    Arrays.sort(sorted, (a, b) -> Integer.compare(a[2], b[2]));
    for (int v = 0; v < V; v++)
      par[v] = v;
    Chosen[] out = new Chosen[V - 1];
    int k = 0, total = 0;
    for (int i = 0; i < E && k < V - 1; i++)
      if (unionSet(sorted[i][0], sorted[i][1])) {
        out[k++] = new Chosen(sorted[i][0], sorted[i][1], sorted[i][2]);
        total += sorted[i][2];
      }
    return new Tree(Arrays.copyOf(out, k), total);
  }
```

listing 318: java, sorted edges over the inline dsu, records for the chosen tree (source: dsa/samples-java/src/Ch12/Kruskal.java, lines 39-56)

```
    public static List<Edge> Kruskal(IEnumerable<string> vertices, List<Edge> edges)
    {
        var dsu = new DisjointSet();
        foreach (string v in vertices)
            dsu.Add(v);

        var tree = new List<Edge>();
        foreach (Edge edge in edges.OrderBy(e => e.Weight))
        {
            if (dsu.Union(edge.From, edge.To))
                tree.Add(edge);
            if (tree.Count == vertices.Count() - 1)
                break;
        }
        return tree;
    }
```

listing 319: c#, kruskal over sorted edges, stop at n-1 (source: dsa/samples/src/Ch12/Flows.cs, lines 60-75)

```
  const dsu = new DSU(n);
  const tree = [];
  let total = 0;
  for (const e of sorted) {
    if (tree.length === n - 1) break;
    if (dsu.union(e[0], e[1])) {
      tree.push(e);
      total += e[2];
    }
  }
  return { tree, total };
}
```

listing 320: javascript, sorted copy, tree in acceptance order (source: dsa/samples-js/src/ch12-kruskal.mjs, lines 10-22)

```
def kruskal(vertices, edges):
    # edges are (weight, u, v); output is the mst edge list and its total
    picked = []
    total = 0
    dsu = DSU(vertices)
    for w, u, v in sorted(edges):
        if dsu.union(u, v):  # an edge inside one component would cycle
            picked.append((u, v, w))
            total += w
    return picked, total
```

listing 321: python, sorted (weight, u, v) triples over the inline dsu (source: dsa/samples-py/src/Ch12/kruskal.py, lines 36-45)

```
end

-- returns the chosen edges and the total weight
local function kruskal(edges)
  local sorted = {table.unpack(edges)}
  table.sort(sorted, function(x, y) return x[3] < y[3] end)
  local par = {}
  for v = 0, V - 1 do par[v] = v end
  local chosen, total = {}, 0
  for _, e in ipairs(sorted) do
    if #chosen == V - 1 then break end
    if union(par, e[1], e[2]) then
      chosen[#chosen + 1] = e
      total = total + e[3]
    end
  end
  return chosen, total
end
```

listing 322: lua, table.sort, union-accept, total accumulated (source: dsa/samples-lua/ch12_kruskal.lua, lines 22-40)

Kruskal is edge-oriented and shines on sparse graphs, and the cut property is why it is correct: every light edge across a partition of the components belongs to some minimum tree.

Measured across the suites: Go, JavaScript, and Python share a 5-edge fixture and pin total 6 with the tree 0-1, 1-2, 2-3 in pick order, the heavy edges of weight 4 and 5 refused, and a disconnected graph left as a forest. C, Java, and Lua share a 6-edge fixture, total 13 from picks 1 2 4 6, C and Lua proving the spanning property by transitive closure over the chosen edges. The frozen C# suite pins n-1 edges of one total across fixed and twenty random weighted graphs.

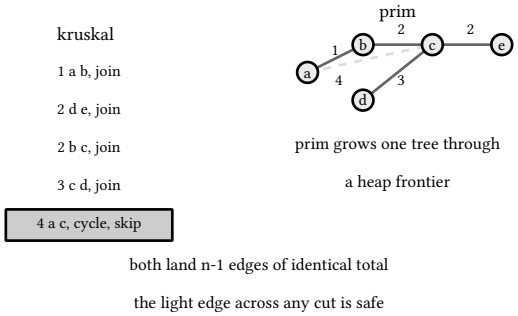


figure 87: kruskal against prim on one graph, sorted unions past cycle closers against one tree through a heap frontier

12.3. prim

Prim grows one tree outward from a start vertex, always absorbing the cheapest edge that crosses from inside to outside. The frozen C# version drives a binary heap frontier of candidate edges, the other six scan for the smallest key, quadratic on purpose and clearer than heap machinery at teaching sizes.

The dry run: the fixture is the same weighted graph grown from a through the C# heap frontier, the total asserted equal to kruskal's; the six siblings scan for the smallest key instead, Go, JavaScript, and Python pinning the same 6 from every start.

1. The heap opens with a's two crossings, a-b at 1 and a-c at 4.
2. ExtractMin pops a-b 1 and b joins: b's crossings b-c 2 and b-d 6 land beside the aging a-c 4.
3. ExtractMin pops b-c 2 and c joins: c's crossing c-d 3 undercuts the stale a-c 4 and the waiting b-d 6.
4. ExtractMin pops c-d 3 and d joins: the tree holds its n - 1 = 3 edges and the count check ends the run.

5. The stale a-c 4 and the outranked b-d 6 die in the heap unvisited, and the total reads 1 + 2 + 3 = 6, kruskal's number, the pinned equality.

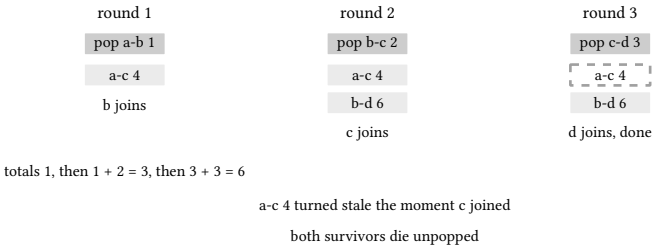


figure 88: the frontier round by round, the popped winner above the heap it came from, the stale entries never popped

The 6 matching kruskal on the same graph is the pinned pair, and the listings below grow the tree in seven languages.

```c
static int prim(int start) {
  for (int v = 0; v < V; v++) {
    key[v] = INF;
    par[v] = -1;
    taken[v] = 0;
  }
  key[start] = 0;
  int total = 0;
  for (int round = 0; round < V; round++) {
    // pick the untaken vertex with the smallest key
    int u = -1;
    for (int v = 0; v < V; v++)
      if (!taken[v] && (u < 0 || key[v] < key[u]))
        u = v;
    taken[u] = 1;
    total += key[u];
    for (int v = 0; v < V; v++)
      if (!taken[v] && adjm[u][v] < key[v]) {
        key[v] = adjm[u][v];
        par[v] = u;
      }
  }
  return total;
}
```

listing 323: c, key array over the adjacency matrix, min-scan pick (source: dsa/samples-c/src/Ch12/prim.c, lines 37-60)

```go
func PrimMST(n int, edges []UndEdge, start int) (total int64, ok bool) {
  adj := make([][]UndEdge, n)
  for _, e := range edges {
    adj[e.A] = append(adj[e.A], e)
    adj[e.B] = append(adj[e.B], e)
  }
  const inf = int64(1) << 62
  inTree := make([]bool, n)
  best := make([]int64, n) // cheapest edge weight reaching v
  for v := range best {
    best[v] = inf
  }
  best[start] = 0
  for added := 0; added < n; added++ {
    v := -1
    for u := 0; u < n; u++ {
      if !inTree[u] && (v == -1 || best[u] < best[v]) {
        v = u
      }
    }
    if v == -1 || best[v] == inf {
      return total, false // some vertex unreachable: forest, not tree
    }
    inTree[v] = true
    total += best[v]
    for _, e := range adj[v] {
      other := e.A
      if other == v {
        other = e.B
      }
      if !inTree[other] && e.Weight < best[other] {
        best[other] = e.Weight
      }
    }
  }
  return total, true
}
```

listing 324: go, best-key scan over adjacency lists, forest flag (source: dsa/samples-go/ch12/prim.go, lines 7-43)

```
  static int[] key = new int[V], par = new int[V], taken = new int[V];

  static int prim(int start) {
    for (int v = 0; v < V; v++) {
      key[v] = INF;
      par[v] = -1;
      taken[v] = 0;
    }
    key[start] = 0;
    int total = 0;
    for (int round = 0; round < V; round++) {
      // pick the untaken vertex with the smallest key
      int u = -1;
      for (int v = 0; v < V; v++)
        if (taken[v] == 0 && (u < 0 || key[v] < key[u]))
          u = v;
      taken[u] = 1;
      total += key[u];
      for (int v = 0; v < V; v++)
        if (taken[v] == 0 && adjm[u][v] < key[v]) {
          key[v] = adjm[u][v];
          par[v] = u;
        }
    }
    return total;
  }
```

listing 325: java, key, parent, and taken arrays over the matrix, min-scan pick (source: dsa/samples-java/src/Ch12/Prim.java, lines 34-59)

```
public static List<Edge> Prim(Dictionary<string, List<Edge>> adjacency, string start)
{
    var inTree = new HashSet<string> { start };
    var heap = new DsaBook.Samples.Ch08.BinaryHeap<PrimEdge>();
    foreach (Edge e in adjacency[start])
        heap.Add(new PrimEdge(e.From, e.To, e.Weight));

    var tree = new List<Edge>();
    while (heap.Count > 0 && inTree.Count < adjacency.Count)
    {
        PrimEdge min = heap.ExtractMin();
        if (inTree.Contains(min.To))
            continue; // stale frontier entry
        inTree.Add(min.To);
        tree.Add(new Edge(min.From, min.To, min.Weight));
        foreach (Edge e in adjacency[min.To])
            if (!inTree.Contains(e.To))
                heap.Add(new PrimEdge(min.To, e.To, e.Weight));
    }
    return tree;
}

private sealed record PrimEdge(string From, string To, long Weight)
    : IComparable<PrimEdge>
{
    public int CompareTo(PrimEdge? other) =>
        Weight.CompareTo(other!.Weight) is int c && c != 0
            ? c
            : string.CompareOrdinal(From + To, other!.From + other!.To);
}
```

listing 326: c#, prim through the heap frontier, tie break on names (source: dsa/samples/src/Ch12/Flows.cs, lines 78-107)

```
  const adjm = Array.from({ length: n }, () => new Array(n).fill(INF));
  for (let i = 0; i < n; i++) adjm[i][i] = 0;
  for (const [a, b, w] of edges) {
    adjm[a][b] = w;
    adjm[b][a] = w;
  }
  const key = new Array(n).fill(INF);
  const parent = new Array(n).fill(-1);
  const taken = new Array(n).fill(false);
  key[start] = 0;
  let total = 0;
  for (let round = 0; round < n; round++) {
    // pick the untaken vertex with the smallest key
    let u = -1;
    for (let v = 0; v < n; v++)
      if (!taken[v] && (u < 0 || key[v] < key[u])) u = v;
    if (key[u] === INF) return { total, ok: false, parent };
    taken[u] = true;
    total += key[u];
    for (let v = 0; v < n; v++)
      if (!taken[v] && adjm[u][v] < key[v]) {
        key[v] = adjm[u][v];
        parent[v] = u;
      }
  }
  return { total, ok: true, parent };
}
```

listing 327: javascript, key and parent arrays over the matrix (source: dsa/samples-js/src/ch12-prim.mjs, lines 8-35)

```
def prim(adj, start):
    # adj[u] holds (v, w); returns the picked edges and the total weight
    in_tree = {start}
    picked = []
    total = 0
    while True:
        best = None  # the cheapest edge from inside to outside
        for u in sorted(in_tree):
            for v, w in sorted(adj[u]):
                if v not in in_tree and (best is None or w < best[0]):
                    best = (w, u, v)
        if best is None:
            return picked, total
        w, u, v = best
        in_tree.add(v)
        picked.append((u, v, w))
        total += w
```

listing 328: python, cheapest crossing edge from the sorted inside set (source: dsa/samples-py/src/Ch12/prim.py, lines 13-29)

```lua
-- returns the total weight and fills par with each vertex's tree parent
local function prim(adjm, start, par)
  local key, taken = {}, {}
  for v = 0, V - 1 do
    key[v] = INF
    par[v] = -1
    taken[v] = false
  end
  key[start] = 0
  local total = 0
  for _ = 0, V - 1 do
    -- pick the untaken vertex with the smallest key
    local u = -1
    for v = 0, V - 1 do
      if not taken[v] and (u < 0 or key[v] < key[u]) then u = v end
    end
    taken[u] = true
    total = total + key[u]
    for v = 0, V - 1 do
      if not taken[v] and adjm[u][v] < key[v] then
        key[v] = adjm[u][v]
        par[v] = u
      end
    end
  end
  return total
end
```

listing 329: lua, key, taken, parent, the min-scan loop (source: dsa/samples-lua/ch12_prim.lua, lines 27-54)

Measured across the suites: Go, JavaScript, and Python land the same 6 as their kruskal from every start vertex, Python pinning the pick order and the rerouted picks from vertex 1, all three flagging a forest when a vertex is unreachable. C, Java, and Lua share their 6-edge fixture, total 13 with parents pinned from vertex 0 and the same 13 from starts 4 and 2 under a different parent shape. The frozen C# suite pins equal totals against kruskal on its random graphs.

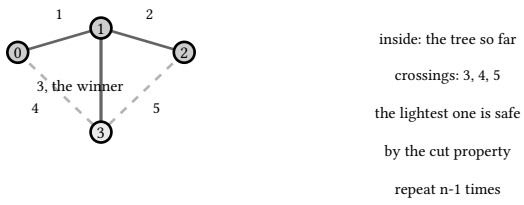


figure 89: prim mid-growth, the cheapest edge crossing from the tree to the outside wins

NOTE: spanning trees are not shortest path trees

The MST minimizes total edge weight, dijkstra minimizes per-vertex distance, and the two answers genuinely differ on shared inputs. A delivery network laying cable wants the MST, a navigation app routing each vehicle wants shortest paths. Chapter 11's tree and this chapter's tree are different objects with different optimality claims, a standard interview trap.

12.4. maximum flow, edmonds-karp

Ford-Fulkerson repeatedly pushes flow along any augmenting path. Edmonds-Karp fixes the path choice to BFS, shortest augmenting path first, which bounds the whole algorithm polynomially. The residual graph is the

machinery: every edge carries capacity minus used flow, and pushing flow creates reverse residual capacity, the undo permission that lets later paths reroute earlier greed.

The dry run: the fixture is the C# textbook network, s to a at 10, s to b at 5, a to b at 15, a to t at 10, b to t at 10; the siblings run smaller nets, Go, JavaScript, and Python pinning flow 5 and C, Java, and Lua the classic 6-vertex 23.

1. The first BFS fans out from s, a and b join the wave, and t is reached through a: the path s, a, t runs at bottleneck min(10, 10) = 10.
2. The push fills s-a and a-t to 10 each, both residuals dropping to 10 - 10 = 0.
3. The second BFS finds s-a spent, the wave reaches b, and the path s, b, t runs at bottleneck min(5, 10) = 5.
4. The total climbs 10 + 5 = 15, s-b sits saturated, and the third BFS stalls at the source with both exits at residual 0.
5. The per-edge reads land s-a 10, s-b 5, a-t 10, b-t 5, and the cross a-b 0, unneeded, with the flow value 15 pinned.
6. The all-capacity-1 trap graph lands flow 2 whatever order the paths take, the undo the reverse residual sells.

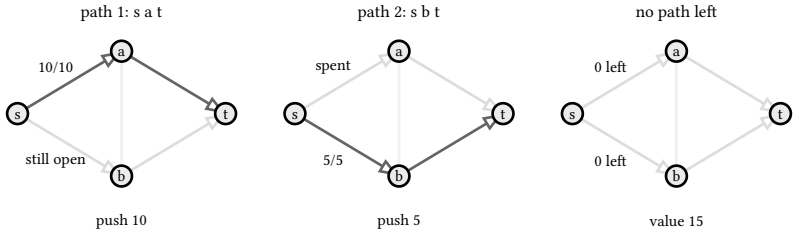


figure 90: the run as three bfs frames, two augmenting paths then the stall at a saturated source

The 15 with the cross edge flat at 0 is the pinned read, and the listings below push through the residual graph in seven languages.

```c
// bfs the shortest augmenting path; fills prev node's chosen edge
static int bfs(int *prev_edge) {
  int q[V], qh = 0, qt = 0, seen[V] = {0};
  seen[S] = 1;
  q[qt++] = S;
  while (qh < qt) {
    int u = q[qh++];
    for (int i = 0; i < deg[u]; i++) {
      int e = adj[u][i], v = endpoint(e);
      if (!seen[v] && resid[e] > 0) {
        seen[v] = 1;
        prev_edge[v] = e;
        if (v == T)
          return 1;
        q[qt++] = v;
      }
    }
  }
  return 0;
}

static int maxflow(void) {
  build();
  int flow = 0, prev_edge[V];
  while (bfs(prev_edge)) {
    int push = 1 << 30;
    for (int v = T; v != S; v = endpoint(prev_edge[v] ^ 1))
      if (resid[prev_edge[v]] < push)
        push = resid[prev_edge[v]];
    for (int v = T; v != S; v = endpoint(prev_edge[v] ^ 1)) {
      resid[prev_edge[v]] -= push;
      resid[prev_edge[v] ^ 1] += push;
    }
    flow += push;
  }
  return flow;
}
```

listing 330: c, bfs over arc pairs, bottleneck push, xor to flip an arc (source: dsa/samples-c/src/Ch12/maxflow.c, lines 47-83)

```go
func (f *FlowNet) MaxFlowBFS(source, sink int) int64 {
  if source == sink {
    return 0
  }
  var total int64
  for {
    // BFS for a path of positive residual capacity
    parent := make([]int, f.n)
    for i := range parent {
      parent[i] = -1
    }
    parent[source] = source
    queue := []int{source}
    for len(queue) > 0 && parent[sink] == -1 {
      v := queue[0]
      queue = queue[1:]
      for to := 0; to < f.n; to++ {
        if parent[to] == -1 && f.cap[v][to] > 0 {
          parent[to] = v
          queue = append(queue, to)
        }
      }
    }
    if parent[sink] == -1 {
      return total // no augmenting path left
    }
    bottleneck := int64(1) << 62
    for v := sink; v ≠ source; v = parent[v] {
      bottleneck = min(bottleneck, f.cap[parent[v]][v])
    }
    for v := sink; v ≠ source; v = parent[v] {
      f.cap[parent[v]][v] -= bottleneck
      f.cap[v][parent[v]] += bottleneck
    }
    total += bottleneck
  }
}
```

listing 331: go, matrix residual, bfs parent chain, bottleneck update (source: dsa/samples-go/ch12/maxflow.go, lines 35-71)

```
  static int endpoint(int e) {
    return (e & 1) == 0 ? EF[e / 2] : ET[e / 2];
  }

  // bfs the shortest augmenting path; fills prev node's chosen edge
  static boolean bfs(int[] prevEdge) {
    int[] q = new int[V];
    int qh = 0, qt = 0;
    int[] seen = new int[V];
    seen[S] = 1;
    q[qt++] = S;
    while (qh < qt) {
      int u = q[qh++];
      for (int i = 0; i < deg[u]; i++) {
        int e = adj[u][i], v = endpoint(e);
        if (seen[v] == 0 && resid[e] > 0) {
          seen[v] = 1;
          prevEdge[v] = e;
          if (v == T)
            return true;
          q[qt++] = v;
        }
      }
    }
    return false;
  }

  static int maxflow() {
    build();
    int flow = 0;
    int[] prevEdge = new int[V];
    while (bfs(prevEdge)) {
      int push = 1 << 30;
      for (int v = T; v != S; v = endpoint(prevEdge[v] ^ 1))
        if (resid[prevEdge[v]] < push)
          push = resid[prevEdge[v]];
      for (int v = T; v != S; v = endpoint(prevEdge[v] ^ 1)) {
        resid[prevEdge[v]] -= push;
        resid[prevEdge[v] ^ 1] += push;
      }
      flow += push;
    }
    return flow;
  }
```

listing 332: java, bfs over arc pairs, endpoint flips an arc by its bit, the bottleneck push (source: dsa/samples-java/src/Ch12/Maxflow.java, lines 44-87)

```
public static class Flow
{
    public sealed record Edge(string From, string To, long Capacity);

    // returns flow value and the per-edge flow map (positive = forward use)
    public static (long Value, Dictionary<(string, string), long> Used) MaxFlow(
        IEnumerable<Edge> edges, string source, string sink)
    {
        var capacity = new Dictionary<(string, string), long>();
```

```
        var adjacency = new Dictionary<string, List<string>>();

        void Ensure(string v) => adjacency.TryAdd(v, new List<string>());
        Ensure(source);
        Ensure(sink);
        foreach (Edge e in edges)
        {
            // parallel edges accumulate, and the residual graph needs both
            // directions even where no reverse edge exists
            capacity[(e.From, e.To)] = capacity.GetValueOrDefault((e.From, e.To)) + e.Capacity;
            capacity.TryAdd((e.To, e.From), 0);
            Ensure(e.From);
            Ensure(e.To);
            if (!adjacency[e.From].Contains(e.To))
                adjacency[e.From].Add(e.To);
            if (!adjacency[e.To].Contains(e.From))
                adjacency[e.To].Add(e.From);
        }

        var used = new Dictionary<(string, string), long>();
        long total = 0;

        while (BfsFindAugmentingPath(adjacency, capacity, used, source, sink,
                   out List<string> path))
        {
            // bottleneck: least residual capacity along the path
            long bottleneck = long.MaxValue;
            for (int i = 0; i < path.Count - 1; i++)
            {
                long residual = capacity[(path[i], path[i + 1])] -
                                used.GetValueOrDefault((path[i], path[i + 1]));
                bottleneck = Math.Min(bottleneck, residual);
            }

            for (int i = 0; i < path.Count - 1; i++)
            {
                used[(path[i], path[i + 1])] =
                    used.GetValueOrDefault((path[i], path[i + 1])) + bottleneck;
                used[(path[i + 1], path[i])] =
                    used.GetValueOrDefault((path[i + 1], path[i])) - bottleneck;
            }
            total += bottleneck;
        }

        var usedForward = used.Where(kv => kv.Value > 0)
            .ToDictionary(kv => kv.Key, kv => kv.Value);
        return (total, usedForward);
    }

    private static bool BfsFindAugmentingPath(
        Dictionary<string, List<string>> adjacency,
        Dictionary<(string, string), long> capacity,
        Dictionary<(string, string), long> used,
        string source, string sink, out List<string> path)
    {
        var parent = new Dictionary<string, string> { [source] = source };
        var queue = new Queue<string>();
```

```
        queue.Enqueue(source);
        while (queue.Count > 0)
        {
            string v = queue.Dequeue();
            foreach (string n in adjacency[v])
            {
                long residual = capacity[(v, n)] - used.GetValueOrDefault((v, n));
                if (residual > 0 && parent.TryAdd(n, v))
                    queue.Enqueue(n);
            }
        }

        if (!parent.ContainsKey(sink))
        {
            path = new List<string>();
            return false;
        }
        path = new List<string> { sink };
        while (path[^1] != source)
            path.Add(parent[path[^1]]);
        path.Reverse();
        return true;
    }
```

listing 333: c#, residual capacities, bottleneck pushes, the bfs augmenting path (source: dsa/samples/src/Ch12/Flows.cs, lines 111-199)

The reverse-edge trick is the part worth staring at, and the test named for it builds the classic trap graph where the first greedy path through the cross edge must later be partially undone. Flow conservation and capacity limits are asserted per edge on every test.

```
    const queue = [s];
    for (let head = 0; head < queue.length; head++) {
      const u = queue[head];
      for (const arc of this.#adj[u]) {
        const v = this.#endpoint(arc);
        if (!seen[v] && this.#cap[arc] > 0) {
          seen[v] = true;
          prevArc[v] = arc;
          if (v === t) return true;
          queue.push(v);
        }
      }
    }
    return false;
  }

  maxFlowBFS(s, t) {
    let flow = 0;
    const prevArc = new Array(this.#adj.length).fill(-1);
    while (this.#bfs(s, t, prevArc)) {
      let push = Infinity;
      for (let v = t; v !== s; v = this.#endpoint(prevArc[v] ^ 1))
        push = Math.min(push, this.#cap[prevArc[v]]);
      for (let v = t; v !== s; v = this.#endpoint(prevArc[v] ^ 1)) {
        this.#cap[prevArc[v]] -= push;
        this.#cap[prevArc[v] ^ 1] += push;
      }
      flow += push;
    }
    return flow;
  }

  // residual reachability from s: cut edges in input order
  minCut(s) {
    const reach = new Array(this.#adj.length).fill(false);
    reach[s] = true;
    const queue = [s];
    for (let head = 0; head < queue.length; head++) {
      const u = queue[head];
      for (const arc of this.#adj[u]) {
        const v = this.#endpoint(arc);
```

listing 334: javascript, arc pairs, private bfs, the push loop (source: dsa/samples-js/src/ch12-maxflow.mjs, lines 30-70)

```
    def max_flow(self, source, sink):
        self.cap = [row[:] for row in self.orig]
        total = 0
        while True:
            parent = [None] * self.size
            parent[source] = source
            queue, head = [source], 0
            while head < len(queue) and parent[sink] is None:
                u = queue[head]
                head += 1
                for v in range(self.size):
                    if self.cap[u][v] > 0 and parent[v] is None:
                        parent[v] = u
                        queue.append(v)
            if parent[sink] is None:
                reach = [u for u in range(self.size) if parent[u] is not None]
                self.reachable = set(reach)
                return total
            path = []
            v = sink
            while v ≠ source:
                path.append((parent[v], v))
                v = parent[v]
            bottleneck = min(self.cap[u][v] for u, v in path)
            for u, v in path:
                self.cap[u][v] -= bottleneck  # forward spends, backward earns
                self.cap[v][u] += bottleneck
            total += bottleneck
```

listing 335: python, residual copy, bfs, the path bottleneck (source: dsa/samples-py/src/Ch12/maxflow.py, lines 23-50)

```lua
local function bfs(resid, adj, prev_edge)
  local seen = {[S] = true}
  local queue = {S}
  local head = 1
  while head ≤ #queue do
    local u = queue[head]
    head = head + 1
    for _, e in ipairs(adj[u]) do
      local v = endpoint(e)
      if not seen[v] and resid[e + 1] > 0 then
        seen[v] = true
        prev_edge[v] = e
        if v == SINK then return true end
        queue[#queue + 1] = v
      end
    end
  end
  return false
end

-- returns the flow and leaves the final residual graph in resid, adj
local function maxflow()
  local resid, adj = build()
  local flow = 0
  local prev_edge = {}
  while bfs(resid, adj, prev_edge) do
    local push = math.maxinteger
    local v = SINK
    while v ~= S do
      local e = prev_edge[v]
      if resid[e + 1] < push then push = resid[e + 1] end
      v = endpoint(e ~ 1)
    end
    v = SINK
    while v ~= S do
      local e = prev_edge[v]
      resid[e + 1] = resid[e + 1] - push
      resid[(e ~ 1) + 1] = resid[(e ~ 1) + 1] + push
      v = endpoint(e ~ 1)
    end
    flow = flow + push
  end
  return flow, resid, adj
end
```

listing 336: lua, 1-based arc ids, bfs, the two push walks (source: dsa/samples-lua/ch12_maxflow.lua, lines 37-81)

Measured across the suites: Go, JavaScript, and Python share a 5-edge network and pin flow 5 with the min cut at the saturated source edges 0-1 and 0-2, plus a narrow middle edge bounding a second fixture at 4. C, Java, and Lua share the classic 6-vertex network, flow 23, per-edge flows 12 11 12 0 11 0 19 7 4, conservation checked at every internal vertex, and the residual-reachable set 0 1 2 4 cutting at capacity 23. The frozen C# suite pins the trap graph and asserts conservation and capacity per edge.

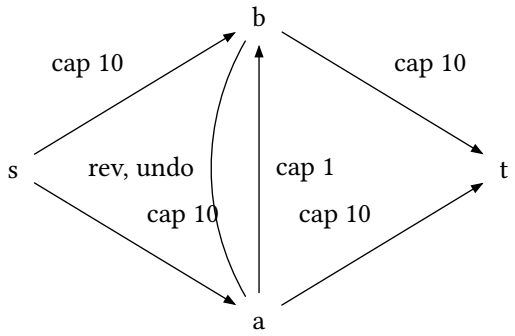


figure 91: the trap graph, one augmenting path, and the reverse edge it opens

12.5. max-flow min-cut, verified

The theorem says the maximum flow equals the minimum capacity of any s-t cut. The reachable set in the final residual graph is one side of a minimum cut, saturated across the boundary.

The dry run: the fixtures are identical literals in all seven suites. The textbook net, s-a 10, s-b 5, a-b 15, a-t 10, b-t 10, lands flow 15 with the per-edge use 10, 5, 10, 5 and the cross flat at 0, cut side {s}, cut capacity 15. The all-ones undo net lands flow 2 with every flow inside its capacity. The parallel lane is two edges (s, t, 3) and (s, t, 4) accumulating to value 7. The verification lane recomputes the residual-reachable cut and asserts its capacity equals the flow value, with conservation at every internal vertex.

1. The residual walk seeds the reachable set with s and queues it.
2. Both exits are spent, s-a at 10 - 10 = 0 and s-b at 5 - 5 = 0, so neither a nor b ever joins: the reachable set is s alone, t outside.
3. The cut sums the original capacities crossing that boundary, s-a at 10 and s-b at 5: 10 + 5 = 15, asserted equal to the flow value 15, the theorem checked by construction.
4. The two-pipe graph stacks parallel s-t edges of 3 and 4 into one capacity of 3 + 4 = 7, and the flow lands 7, the accumulation pinned.

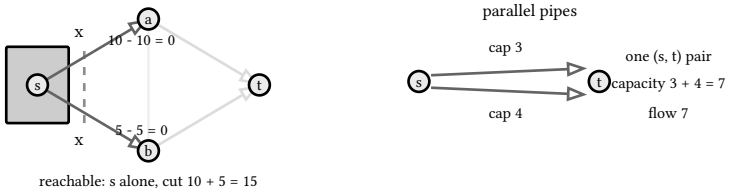


figure 92: the residual probe stalling at a lone reachable source, and the two parallel pipes merging into one capacity

The 15 against 15 and the two-pipe 7 are the pins, and the listings below are the seven readers that compute them.

```c
// residual reachability from s after the flow: the min cut source side
static void cut_side(const network *g, int s, int *reach) {
  int queue[MAX_V], qh = 0, qt = 0;
  for (int v = 0; v < g->nverts; v++)
    reach[v] = 0;
  reach[s] = 1;
  queue[qt++] = s;
  while (qh < qt) {
    int v = queue[qh++];
    for (int k = 0; k < g->nadj[v]; k++) {
      int w = g->adj[v][k];
      if (g->cap[v][w] - g->used[v][w] > 0 && !reach[w]) {
        reach[w] = 1;
        queue[qt++] = w;
      }
    }
  }
}

// original capacity leaving the reachable set
static long cut_capacity(const network *g, const int *reach) {
  long total = 0;
  for (int e = 0; e < g->nedges; e++)
    if (reach[g->efrom[e]] && !reach[g->eto[e]])
      total += g->ecap[e];
  return total;
}
```

listing 337: c, residual reachability from s, then the cut capacity over the original edge records (source: dsa/samples-c/src/Ch12/mincut.c, lines 116-142)

```go
// MinCutSides returns the vertices still reachable from the source in
// the final residual graph: the source side of a minimum cut.
func MinCutSides(edges []FlowEdge, used map[[2]string]int64, source string) map[string]bool {
  n := buildFlowNetwork(edges)
  reachable := map[string]bool{source: true}
  queue := []string{source}
  for len(queue) > 0 {
    v := queue[0]
    queue = queue[1:]
    for _, to := range n.adjacency[v] {
      if !reachable[to] && n.residual(v, to, used) > 0 {
        reachable[to] = true
        queue = append(queue, to)
      }
    }
  }
  return reachable
}
```

listing 338: go, the min cut side rebuilt from the residual graph (source: dsa/samples-go/ch12/mincut.go, lines 120-137)

```
  // residual reachability from s after the flow: the min cut source side
  static void cutSide(Network g, int s, int[] reach) {
    int[] queue = new int[MAX_V];
    int qh = 0, qt = 0;
    for (int v = 0; v < g.nverts; v++)
      reach[v] = 0;
    reach[s] = 1;
    queue[qt++] = s;
    while (qh < qt) {
      int v = queue[qh++];
      for (int k = 0; k < g.nadj[v]; k++) {
        int w = g.adj[v][k];
        if (g.cap[v][w] - g.used[v][w] > 0 && reach[w] == 0) {
          reach[w] = 1;
          queue[qt++] = w;
        }
      }
    }
  }

  // original capacity leaving the reachable set
  static long cutCapacity(Network g, int[] reach) {
    long total = 0;
    for (int e = 0; e < g.nedges; e++)
      if (reach[g.efrom[e]] == 1 && reach[g.ehead[e]] == 0)
        total += g.ecap[e];
    return total;
  }
```

listing 339: java, residual reachability from s, then the cut capacity over the original edge records (source: dsa/samples-java/src/Ch12/Mincut.java, lines 116-143)

```
    public static HashSet<string> MinCutSides(
        IEnumerable<Edge> edges, Dictionary<(string, string), long> used,
        string source)
    {
        var adjacency = new Dictionary<string, List<string>>();
        var capacity = new Dictionary<(string, string), long>();
        foreach (Edge e in edges)
        {
            capacity[(e.From, e.To)] = capacity.GetValueOrDefault((e.From, e.To)) + e.Capacity;
            if (!adjacency.TryGetValue(e.From, out List<string>? fromList))
                adjacency[e.From] = fromList = new List<string>();
            fromList.Add(e.To);
            if (!adjacency.TryGetValue(e.To, out List<string>? toList))
                adjacency[e.To] = toList = new List<string>();
            toList.Add(e.From);
        }

        var reachable = new HashSet<string> { source };
        var queue = new Queue<string>();
        queue.Enqueue(source);
        while (queue.Count > 0)
        {
            string v = queue.Dequeue();
            foreach (string n in adjacency[v])
            {
                long residual = capacity.GetValueOrDefault((v, n)) -
                                used.GetValueOrDefault((v, n));
                if (residual > 0 && reachable.Add(n))
                    queue.Enqueue(n);
            }
        }
        return reachable;
    }
}
```

listing 340: c#, residual reachability gives the min cut side (source: dsa/samples/src/Ch12/Flows.cs, lines 202-235)

The test computes the cut capacity crossing that partition and asserts it equals the flow value, the theorem checked by construction rather than quoted. Parallel edges accumulate into one capacity, which the two-pipe test pins.

```javascript
// the cut side: vertices still reachable from the source in the final
// residual graph. its outgoing capacity equals the flow value
export function minCutSides(edges, used, source) {
  const adjacency = new Map();
  const capacity = new Map();
  for (const e of edges) {
    const fwd = pair(e.from, e.to);
    capacity.set(fwd, (capacity.get(fwd) ?? 0) + e.capacity);
    if (!adjacency.has(e.from)) adjacency.set(e.from, []);
    adjacency.get(e.from).push(e.to);
    if (!adjacency.has(e.to)) adjacency.set(e.to, []);
    adjacency.get(e.to).push(e.from);
  }

  const reachable = new Set([source]);
  const queue = [source];
  while (queue.length > 0) {
    const v = queue.shift();
    for (const n of adjacency.get(v) ?? []) {
      const key = pair(v, n);
      const residual = capacity.get(key) - (used.get(key) ?? 0);
      if (residual > 0 && !reachable.has(n)) {
        reachable.add(n);
        queue.push(n);
      }
    }
  }
  return reachable;
}
```

listing 341: javascript, reachability over positive residual, the cut side as a set (source: dsa/samples-js/src/ch12-mincut.mjs, lines 84-112)

```python
def min_cut_side(edges, used, source):
    # the reachable set in the final residual graph
    capacity = {}
    adjacency = {}
    for a, b, cap in edges:
        capacity[(a, b)] = capacity.get((a, b), 0) + cap
        adjacency.setdefault(a, []).append(b)
        adjacency.setdefault(b, []).append(a)
    reachable = {source}
    queue = deque([source])
    while queue:
        v = queue.popleft()
        for w in adjacency[v]:
            residual = capacity.get((v, w), 0) - used.get((v, w), 0)
            if residual > 0 and w not in reachable:
                reachable.add(w)
                queue.append(w)
    return reachable
```

listing 342: python, the reachable set walked off the final used map (source: dsa/samples-py/src/Ch12/mincut.py, lines 68-85)

```
    local bottleneck = math.maxinteger
    for i = #path, 2, -1 do
      local r = residual(path[i], path[i - 1])
      if r < bottleneck then bottleneck = r end
    end
    for i = #path, 2, -1 do
      bump(path[i], path[i - 1], bottleneck)
      bump(path[i - 1], path[i], -bottleneck)
    end
    total = total + bottleneck
  end

  -- min cut side: residual reachability from the source
  local reach = {[source] = true}
  local queue, head = {source}, 1
  while head ≤ #queue do
    local v = queue[head]
    head = head + 1
    for _, n in ipairs(adj[v]) do
      if not reach[n] and residual(v, n) > 0 then
        reach[n] = true
        queue[#queue + 1] = n
      end
    end
  end
  return total, used, cap, adj, order, reach
end
```

listing 343: lua, the bottleneck pushes, then the reach walk inside the same pass (source: dsa/samples-lua/ch12_mincut.lua, lines 60-86)

Measured across the suites: the three networks pin identically everywhere, flow 15 with the cut side {s} of capacity 15, flow 2 on the all-ones undo net, and 7 on the parallel pair, the merged arc carrying all of it. C sums the cut capacity over the original edge records and proves conservation as a signed out-sum of zero per vertex, Java doing the same over string labels with parallel arcs accumulated into one matrix cell, Python re-derives cut and conservation from the used map alone, Go and JavaScript return the reachable set for their tests to check, and Lua folds the reach walk into the max-flow function, returning the set beside the flow value.

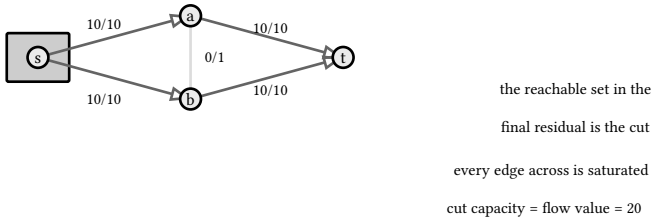


figure 93: max-flow min-cut by construction, the residual reachable set is the source side, its boundary is saturated

12.6. across the seven languages

The build sizes count non-comment source lines. The first table covers the chapter's four original featured files per language, the kruskal files folding their own DSU copy in C, Java, Python, and Lua where the language has one file per topic:

language	build SLOC	dependency	boundary note
c	335	libc only	static arrays and qsort, arc pairs as 2i and 2i+1, checks share the file with main, 72 of them

language	build SLOC	dependency	boundary note
go	198	slices	slices sort and clone, one DSU type shared by the package, prim min-scan by design, 11 tests
java	349	jdk 27 stdlib	one file per topic with the dsu inlined where kruskal needs it, records for the chosen tree, arcs flipped by their bit, 47 checks in 4 files
c#	201	bcl only	string labels, prim rides the chapter 8 heap with a name tie break, min cut in the same file, 10 tests
javascript	145	node stdlib	arc-pair FlowNet with private fields, the dsu exports its parent for the compression proof, 11 tests
python	220	stdlib only	two dsu flavors, rehang and halving, residual copy per run, 32 checks
lua	347	lib.lua harness	1-based arc ids offset by one, xor flips arcs, maxinteger as the initial bottleneck, 15 checks

The min-cut section lands as its own file in the six sibling trees, the C# side staying the reader inside the shared flow class:

language	build SLOC	dependency	boundary note
c	190	libc only	fixed arrays, the edge records kept whole, cut capacity summed over them, conservation as a signed out-sum
go	117	slices	pair-keyed maps, the cut side returned as a set for the tests to verify against the flow
java	190	jdk 27 stdlib	string labels over a first-seen name table, parallel arcs accumulated into one capacity cell, cut capacity and conservation asserted, 21 checks
c#	33	bcl only	the MinCutSides reader inside the shared flow class, the flow it reads sits above it in the same file
javascript	95	node stdlib	pair keys joined on a nul byte, the used map reports positive forward entries only
python	112	stdlib only	deque bfs, the checks re-derive cut capacity and conservation from the used map
lua	153	lib.lua harness	the reach walk folded into the max-flow pass, maxinteger as the initial bottleneck, 5 check rows

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` and `HashSet<T>` pages used by the implementation, `Enumerable.OrderBy` remarks, accessed 2026-09-08. Sample behavior verified by `make verify-csharp`, 10 tests in chapter 12 of the samples suite. The seven-language layer verifies the same way: 5 C programs with 93 embedded checks under `make verify-c`, 15 Go tests, the java runner's 68 Ch12 checks over 5 files under `run-java-samples`, 16 `node --test` cases, 49 Python checks across 5 files, and 20 Lua checks under `run.lua`.

13. sorting

Sorting is where complexity analysis meets constant factors and input distributions. This chapter builds the canonical sorts with comparison counters attached, then measures what actually distinguishes them: adaptivity, stability, worst cases, and the no-comparison radix path.

13.1. insertion, the baseline

Shift each element left until it lands. Quadratic worst case, linear on nearly sorted input, stable by construction.

The dry run: the comparison counts are the contract, identical literals in all seven suites. Sorted 0 through 4999 costs exactly 4999 comparisons with zero shifts, 5, 4, 3, 2, 1 costs 10, 3, 1, 2 costs 3, 2, 1, 3, 1, 2 costs 7, and 4, 2, 2, 1 costs 6, every output sorted. The bound lane holds n-1 through n(n-1)/2 on a shuffled 500 from each tree's own deterministic shuffle, and the Go and Java lanes return the (comparisons, shifts) pair so their zero-shift rows assert the adaptive side directly. The C# meter keeps its sorted, shuffled, and binary runs, binary insertion held under 140000 probes.

1. The adaptive fixture is 0 through 4999, already ascending.
2. Each pass compares the newcomer once against its left neighbor, smaller, and stops: 5000 - 1 = 4999 comparisons with zero shifts, both asserted exactly.
3. The 500-shuffle bounds the meter, at least the n - 1 = 499 floor and at most 500 × 499 / 2 = 124750, the triangular price of the fully reversed shape.
4. Binary insertion takes 10000 ascending keys and halves for the slot instead of walking, about 13.3 probes per element, the meter held under 14 × 10000 = 140000.
5. The shifts survive the trade unchanged, only the search turned logarithmic.

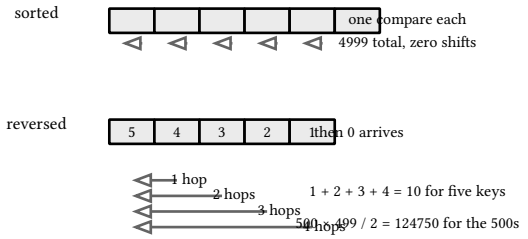


figure 94: the walk length per element, flat one-hop walks on sorted input against the growing staircase of the reversed shape

The exact 4999 with zero shifts is the pinned pair, and the listings below are the metered runs that produce it seven ways.

```c
// sorts in place, returns comparisons, reports shifts through the out
// parameter: the count increments once per while-iteration including
// the breaking comparison
static long insertion(int *items, int len, long *shifts) {
  long comparisons = 0;
  *shifts = 0;
  for (int i = 1; i < len; i++) {
    int key = items[i];
    int j = i - 1;
    while (j ≥ 0) {
      comparisons++;
      if (items[j] ≤ key)
        break; // equal keys stop the walk: stable and adaptive
      items[j + 1] = items[j];
      (*shifts)++;
      j--;
    }
    items[j + 1] = key;
  }
  return comparisons;
}
```

listing 344: c, the counted walk, shifts reported through an out parameter (source: dsa/samples-c/src/Ch13/insertion.c, lines 21-41)

```go
// Insertion sorts in place and reports its comparison and shift
// counts. The inner loop compares a[j] ≤ key and breaks on it, the
// break included in the count: that is what makes the sort stable,
// adaptive, and gives these pinned counts.
func Insertion(a []int) (comparisons int64, shifts int64) {
  for i := 1; i < len(a); i++ {
    key := a[i]
    j := i - 1
    for j ≥ 0 {
      comparisons++
      if a[j] <= key {
        break
      }
      a[j+1] = a[j]
      shifts++
      j--
    }
    a[j+1] = key
  }
  return comparisons, shifts
}

// DeterministicShuffle permutes a deterministic LCG stream: the same
// constants in every run, so the bound lane below is reproducible.
func DeterministicShuffle(n int) []int {
  a := make([]int, n)
  for i := range a {
    a[i] = i
  }
  state := uint64(0x9E3779B97F4A7C15)
  for i := n - 1; i > 0; i-- {
    state = state*6364136223846793005 + 1442695040888963407
    j := int((state >> 33) % uint64(i+1))
    a[i], a[j] = a[j], a[i]
  }
  return a
}
```

listing 345: go, comparisons and shifts returned as a pair, the pinned lcg shuffle beside it (source: dsa/samples-go/ch13/insertion.go, lines 3-39)

```java
record Result(long comparisons, long shifts) {}

// sorts in place, returns comparisons and shifts: the shift count
// increments once per while-iteration including the breaking comparison
static Result insertion(int[] items) {
  long comparisons = 0, shifts = 0;
  for (int i = 1; i < items.length; i++) {
    int key = items[i];
    int j = i - 1;
    while (j ≥ 0) {
      comparisons++;
      if (items[j] ≤ key)
        break; // equal keys stop the walk: stable and adaptive
      items[j + 1] = items[j];
      shifts++;
      j--;
    }
    items[j + 1] = key;
  }
  return new Result(comparisons, shifts);
}
```

listing 346: java, comparisons and shifts returned as a record, the coprime-multiplier shuffle inline (source: dsa/samples-java/src/Ch13/Insertion.java, lines 18-38)

```
    public static long Insertion<T>(T[] a) where T : IComparable<T>
    {
        long comparisons = 0;
        for (int i = 1; i < a.Length; i++)
        {
            T key = a[i];
            int j = i - 1;
            while (j ≥ 0)
            {
                comparisons++;
                if (a[j].CompareTo(key) ≤ 0)
                    break;
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
        return comparisons;
    }

    /// <summary>Binary insertion: same shifts, logarithmic search for the slot.</summary>
    public static long BinaryInsertion<T>(T[] a) where T : IComparable<T>
    {
        long comparisons = 0;
        for (int i = 1; i < a.Length; i++)
        {
            T key = a[i];
            int lo = 0, hi = i;
            while (lo < hi)
            {
                int mid = (lo + hi) / 2;
                comparisons++;
                if (a[mid].CompareTo(key) ≤ 0)
                    lo = mid + 1;
                else
                    hi = mid;
            }
            for (int j = i; j > lo; j--)
                a[j] = a[j - 1];
            a[lo] = key;
        }
        return comparisons;
    }
```

listing 347: c#, insertion sort, then binary insertion with logarithmic slot search (source: dsa/samples/src/Ch13/Sorting.cs, lines 7-49)

The comparison counters make both natures visible in tests: sorted input costs exactly n-1 comparisons with zero shifts, the adaptive best case, and binary insertion cuts the search to logarithmic while keeping the same shifts. Insertion sort is not a museum piece, it is the finisher inside quicksort below and inside many library sorts, because small ranges are where it wins.

```javascript
// ch13: insertion sort, the quadratic baseline. the comparison count is
// the meter: it increments once per while-iteration including the
// comparison that breaks on a[j] ≤ key, which is what makes the count
// adaptive on nearly sorted input

export function insertionSort(a, meter = null) {
  let comparisons = 0;
  for (let i = 1; i < a.length; i++) {
    const key = a[i];
    let j = i - 1;
    while (j ≥ 0) {
      comparisons++;
      if (a[j] ≤ key) break;
      if (meter =/= null) meter.shifts++;
      a[j + 1] = a[j];
      j--;
    }
    a[j + 1] = key;
  }
  return comparisons;
}
```

listing 348: javascript, the meter object carries the shift count (source: dsa/samples-js/src/ch13-insertion.mjs, lines 1-21)

```python
def insertion(a):
    comparisons = 0
    for i in range(1, len(a)):
        key = a[i]
        j = i - 1
        while j ≥ 0:
            comparisons += 1  # counted even when the probe breaks the loop
            if a[j] ≤ key:
                break
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key
    return comparisons


def det_shuffle(items, seed):
    # the pinned 64-bit lcg from the wave brief, fisher-yates downstream
    x = seed
    out = list(items)
    for i in range(len(out) - 1, 0, -1):
        x = (6364136223846793005 * x + 1442695040888963407) % (1 << 64)
        j = x % (i + 1)
        out[i], out[j] = out[j], out[i]
    return out
```

listing 349: python, the counted walk, the 64-bit lcg feeding the bound lane (source: dsa/samples-py/src/Ch13/insertion.py, lines 14-37)

```lua
-- sorts in place, returns comparisons and shifts
local function insertion(a)
  local comparisons, shifts = 0, 0
  for i = 2, #a do
    local key = a[i]
    local j = i - 1
    while j ≥ 1 do
      comparisons = comparisons + 1
      if a[j] ≤ key then break end -- the ≤ break keeps it adaptive
      a[j + 1] = a[j]
      shifts = shifts + 1
      j = j - 1
    end
    a[j + 1] = key
  end
  return comparisons, shifts
end

local function sorted_copy(a)
  local copy = {}
  for i = 1, #a do copy[i] = a[i] end
  table.sort(copy)
  return copy
end

-- deterministic shuffle: a plain lcg feeds fisher-yates, integers only
local function shuffled(n)
  local a = {}
  for i = 1, n do a[i] = i - 1 end
  local x = 12345
  for i = n, 2, -1 do
    x = (1103515245 * x + 12345) % (1 << 31)
    local j = 1 + x % i
    a[i], a[j] = a[j], a[i]
  end
  return a
end
```

listing 350: lua, the pair returned, table.sort as the bound lane's ground truth (source: dsa/samples-lua/ch13_insertion.lua, lines 6-42)

Measured across the suites: all seven pin the counted ladder, 4999 on the sorted 5000 with the array left untouched, 10 on the reversed five, 3 on 3, 1, 2, 7 on 2, 1, 3, 1, 2, and 6 on 4, 2, 2, 1, the equal break stopping each duplicate walk early. The bound lane holds on each tree's own deterministic shuffle, Go and Python sharing the pinned 64-bit lcg constants, Lua a 31-bit one, and C and Java a coprime multiplier permutation built inline. The meter rides differently per tree: Go, Java, and Lua return comparisons and shifts as a pair, Java's as a record, C writes the shifts through an out parameter, JavaScript bumps an optional meter object, and Python and C# return the comparison count alone.

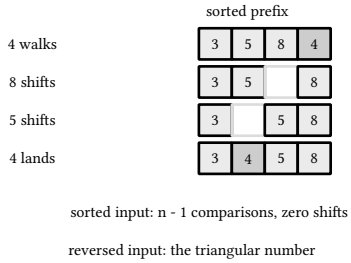


figure 95: insertion sort, one element walks the sorted prefix leftward, shifting as it goes

13.2. merge sort, with inversions

Split, sort halves, merge. Stable, linearithmic, guaranteed, at the price of the buffer. The merge is also a measuring instrument: every time an element is taken from the right half, it stands inverted with every remaining element of the left half, so the merge counts inversions exactly.

The dry run: the fixtures are the C# reversed and sorted hundreds plus the interleaved tag set, asserted by the C# suite; C, Java, and Lua walk a 5-pair set to ids 4 1 3 0 2 and Go, JavaScript, and Python pin the same tie rule on interleaved ids.

1. The reversed fixture, 99 down to 0, holds every pair inverted: the counter lands 100 × 99 / 2 = 4950.
2. At the top merge run 99..50 meets run 49..0, and every right take charges the whole remaining left run: 50 × 50 = 2500 cross pairs counted here, the halves carrying 4950 - 2500 = 2450 down.
3. The sorted twin, 0 through 99, takes only from the left: 0 inversions, asserted equal.
4. The stability fixture interleaves (2, a), (1, b), (2, c), (1, d), (2, e), (1, f), and the halves sort to 1 b 2 a 2 c against 1 d 1 f 2 e.
5. The merge reads the equal 1s as a tie and keeps the left, b before d, then f, and the 2s ride through a c e: the pinned order b d f a c e.

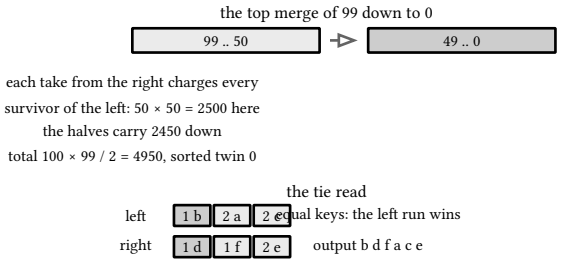


figure 96: the top merge of the reversed run with the whole-run charge, and the tie read that keeps the left half first

The 4950 against the sorted 0 and the tag order b d f a c e are the C# pins, and the listings below merge in seven languages.

```c
static void merge(Pair *a, int lo, int mid, int hi) {
  int i = lo, j = mid, k = lo;
  while (i < mid && j < hi)
    tmp[k++] = a[i].key ≤ a[j].key ? a[i++] : a[j++];
  while (i < mid)
    tmp[k++] = a[i++];
  while (j < hi)
    tmp[k++] = a[j++];
  for (int t = lo; t < hi; t++)
    a[t] = tmp[t];
}

static void mergesort(Pair *a, int lo, int hi) {
  if (hi - lo < 2)
    return;
  int mid = lo + (hi - lo) / 2;
  mergesort(a, lo, mid);
  mergesort(a, mid, hi);
  merge(a, lo, mid, hi);
}
```

listing 351: c, stable merge of key and id pairs, halves to tmp and back (source: dsa/samples-c/src/Ch13/mergesort.c, lines 23-42)

```go
// Mergesort sorts pairs by key, recursively. The merge takes from the
// left run on ties, which is exactly what stability means.
func Mergesort(pairs []Pair) {
  if len(pairs) <= 1 {
    return
  }
  mid := len(pairs) / 2
  Mergesort(pairs[:mid])
  Mergesort(pairs[mid:])
  merged := make([]Pair, 0, len(pairs))
  left, right := pairs[:mid], pairs[mid:]
  for len(left) > 0 && len(right) > 0 {
    if left[0].Key <= right[0].Key { // ≤ is the stability decision
      merged = append(merged, left[0])
      left = left[1:]
    } else {
      merged = append(merged, right[0])
      right = right[1:]
    }
  }
  merged = append(merged, left...)
  merged = append(merged, right...)
  copy(pairs, merged)
}
```

listing 352: go, the left run wins ties, appended into one buffer (source: dsa/samples-go/ch13/mergesort.go, lines 10-33)

```java
  // stable merge: on equal keys the left-run element wins
  static void merge(Pair[] a, int lo, int mid, int hi) {
    int i = lo, j = mid, k = lo;
    while (i < mid && j < hi)
      tmp[k++] = a[i].key() ≤ a[j].key() ? a[i++] : a[j++];
    while (i < mid)
      tmp[k++] = a[i++];
    while (j < hi)
      tmp[k++] = a[j++];
    for (int t = lo; t < hi; t++)
      a[t] = tmp[t];
  }

  static void mergesort(Pair[] a, int lo, int hi) {
    if (hi - lo < 2)
      return;
    int mid = lo + (hi - lo) / 2;
    mergesort(a, lo, mid);
    mergesort(a, mid, hi);
    merge(a, lo, mid, hi);
  }
```

listing 353: java, stable merge of record pairs, ties from the left, halves to tmp and back (source: dsa/samples-java/src/Ch13/Mergesort.java, lines 21-41)

```
    /// <summary>Merge sort: stable, linearithmic, n/2 extra space. Counts comparisons and inversions.</
summary>
    public static (long Comparisons, long Inversions) Merge<T>(T[] a)
        where T : IComparable<T>
    {
        var buffer = new T[a.Length];
        return MergeSort(a, buffer, 0, a.Length - 1);
    }

    private static (long, long) MergeSort<T>(T[] a, T[] buffer, int lo, int hi)
        where T : IComparable<T>
    {
        if (lo ≥ hi)
            return (0, 0);
        int mid = lo + (hi - lo) / 2;
        (long c1, long i1) = MergeSort(a, buffer, lo, mid);
        (long c2, long i2) = MergeSort(a, buffer, mid + 1, hi);
        (long c3, long i3) = MergeHalves(a, buffer, lo, mid, hi);
        return (c1 + c2 + c3, i1 + i2 + i3);
    }

    private static (long, long) MergeHalves<T>(T[] a, T[] buffer, int lo, int mid, int hi)
        where T : IComparable<T>
    {
        Array.Copy(a, lo, buffer, lo, hi - lo + 1);
        long comparisons = 0, inversions = 0;
        int i = lo, j = mid + 1, k = lo;
        while (i ≤ mid && j ≤ hi)
        {
            comparisons++;
            // taking from the right half only when strictly smaller keeps it stable
            if (buffer[j].CompareTo(buffer[i]) < 0)
            {
                inversions += mid - i + 1; // every remaining left element was inverted
                a[k++] = buffer[j++];
            }
            else
            {
                a[k++] = buffer[i++];
            }
        }
        while (i ≤ mid)
            a[k++] = buffer[i++];
        while (j ≤ hi)
            a[k++] = buffer[j++];
        return (comparisons, inversions);
    }
```

listing 354: c#, merge sort returning comparisons and the exact inversion count (source: dsa/samples/src/Ch13/Sorting.cs, lines 51-96)

The reversed-array test asserts the triangular number, every pair inverted, and the sorted-array test asserts zero. Inversion counting is the canonical divide and conquer that is not a sort but rides one, and the stability test proves the strict-less rule in the merge keeps equal keys in input order.

```
  const tmp = [];
  let i = lo;
  let j = mid;
  while (i < mid && j < hi)
    tmp.push(a[i].key ≤ a[j].key ? a[i++] : a[j++]); // left wins ties
  while (i < mid) tmp.push(a[i++]);
  while (j < hi) tmp.push(a[j++]);
  for (let t = lo; t < hi; t++) a[t] = tmp[t - lo];
}

// sorts pairs { key, id } in place and returns them
export function mergesort(a, lo = 0, hi = a.length) {
  if (hi - lo < 2) return a;
  const mid = lo + Math.floor((hi - lo) / 2);
  mergesort(a, lo, mid);
  mergesort(a, mid, hi);
  merge(a, lo, mid, hi);
  return a;
}
```

listing 355: javascript, tmp array per merge, ties from the left (source: dsa/samples-js/src/ch13-mergesort.mjs, lines 5-23)

```
def merge(left, right):
    out, i, j = [], 0, 0
    while i < len(left) and j < len(right):
        if left[i][0] ≤ right[j][0]:  # ≤ keeps the left run first on ties
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out


def mergesort(pairs):
    if len(pairs) ≤ 1:
        return list(pairs)
    mid = len(pairs) // 2
    return merge(mergesort(pairs[:mid]), mergesort(pairs[mid:]))
```

listing 356: python, merge over (key, tag) tuples, the tie keeps the left run (source: dsa/samples-py/src/Ch13/mergesort.py, lines 13-33)

```lua
local function merge(a, lo, mid, hi, tmp)
  local i, j, k = lo, mid + 1, lo
  while i ≤ mid and j ≤ hi do
    if a[i].key ≤ a[j].key then
      tmp[k] = a[i]; i = i + 1
    else
      tmp[k] = a[j]; j = j + 1
    end
    k = k + 1
  end
  while i ≤ mid do tmp[k] = a[i]; i = i + 1; k = k + 1 end
  while j ≤ hi do tmp[k] = a[j]; j = j + 1; k = k + 1 end
  for t = lo, hi do a[t] = tmp[t] end
end

local function mergesort(a, lo, hi, tmp)
  lo, hi = lo or 1, hi or #a
  tmp = tmp or {}
  if hi - lo < 1 then return a end
  local mid = lo + (hi - lo) // 2
  mergesort(a, lo, mid, tmp)
  mergesort(a, mid + 1, hi, tmp)
  merge(a, lo, mid, hi, tmp)
  return a
end
```

listing 357: lua, inclusive bounds, one shared tmp (source: dsa/samples-lua/ch13_mergesort.lua, lines 6-30)

Measured across the suites: every version sorts tagged pairs and pins the tie rule. C, Java, and Lua feed (5,0) (3,1) (5,2) (3,3) (1,4) and land ids 4 1 3 0 2, an all-equal run riding through untouched. Python pins (2,a) (1,b) (2,c) (1,d) to b d a c, Go and JavaScript pin the same shape with numeric ids, and the frozen C# suite counts comparisons and inversions, the reversed fixture hitting the triangular number. The tie decision is one character everywhere, less-or-equal, and the left run wins.

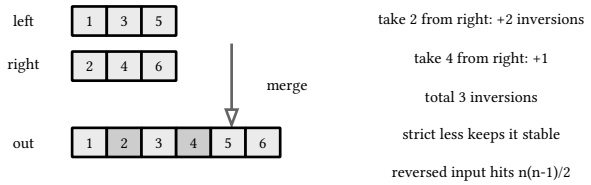


figure 97: merge sort counting inversions, every take from the right charges the remaining left half

13.3. quicksort, defended

Hoare partition with median-of-three pivots, an insertion cutoff for small ranges, and a tail loop instead of the right recursion.

The dry run: the fixture is the C# meter over 20000 sorted keys plus the four adversarial shapes at 2000; the siblings sort smaller fixtures, C, Java, JavaScript, and Lua on nine keys and Go and Python against library ground truth.

1. The metered run is 0 through 19999, already sorted, the shape a first-element pivot murders: peeling one element per partition costs 20000 × 19999 / 2 = 199990000 comparisons.
2. Median of three probes the ends and the middle, 0, 9999, and 19999, sorts the trio in place, and pivots on 9999, the true median: the split lands nearly even.
3. The range halves per level, depth about 14, ranges under 8 fall to insertion, and the meter stops under 2000000, the pinned bound.

4. The same defense holds at 2000 on random, sorted, reversed, and all-equal input, hoare's crossing cursors splitting the repeated pivots between the sides instead of peeling them.

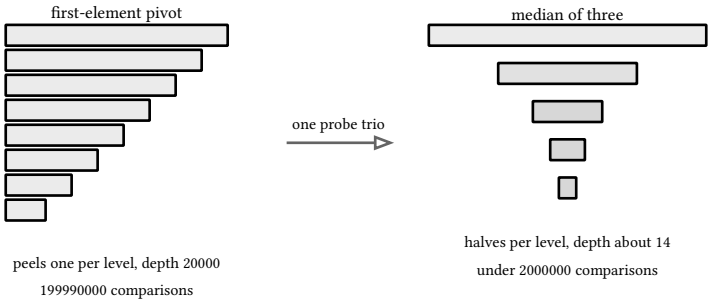


figure 98: the depth profile on sorted input, the one-per-level peel of a first-element pivot against the halving of median of three

Staying under 2000000 where the naive run pays 199990000 is the pinned gap, and the listings below partition in seven languages.

```c
static int hoare(int *a, int lo, int hi, int pivot) {
  int i = lo - 1, j = hi + 1;
  for (;;) {
    do
      i++;
    while (a[i] < pivot);
    do
      j--;
    while (a[j] > pivot);
    if (i ≥ j)
      return j;
    swap(&a[i], &a[j]);
  }
}

static void quicksort(int *a, int lo, int hi) {
  if (lo ≥ hi)
    return;
  int mid = hoare(a, lo, hi, a[lo]);
  quicksort(a, lo, mid);
  quicksort(a, mid + 1, hi);
}

// median-of-three: the pivot value is the median of first, middle, last
static void quicksort_m3(int *a, int lo, int hi) {
  if (lo ≥ hi)
    return;
  int mid = lo + (hi - lo) / 2;
  int pivot = a[lo] < a[mid]
                ? (a[mid] < a[hi] ? a[mid] : (a[lo] < a[hi] ? a[hi] : a[lo]))
                : (a[lo] < a[hi] ? a[lo] : (a[mid] < a[hi] ? a[hi] : a[mid]));
  int p = hoare(a, lo, hi, pivot);
  quicksort_m3(a, lo, p);
  quicksort_m3(a, p + 1, hi);
}
```

listing 358: c, plain hoare, then the median-of-three variant (source: dsa/samples-c/src/Ch13/quicksort.c, lines 24-58)

```
  x, y, z := a[lo], a[mid], a[hi]
  if x > y {
    x, y = y, x
  }
  if y > z {
    y, z = z, y
    if x > y {
      y = x
    }
  }
  return y
}

// hoarePartition sweeps two indices toward each other and swaps
// out-of-place pairs. It returns j: everything in a[lo..j] is ≤ the
// pivot and everything in a[j+1..hi] is ≥ it.
func hoarePartition(a []int, lo, hi int) int {
  pivot := MedianOfThree(a, lo, hi)
  i, j := lo-1, hi+1
  for {
    for {
      i++
      if a[i] ≥ pivot {
        break
      }
    }
    for {
      j--
      if a[j] <= pivot {
        break
      }
    }
    if i ≥ j {
      return j
    }
    a[i], a[j] = a[j], a[i]
  }
}
```

listing 359: go, median of three by sorting three values, the hoare sweep (source: dsa/samples-go/ch13/quicksort.go, lines 24-62)

```java
// classic hoare partition around a pivot value
static int hoare(int[] a, int lo, int hi, int pivot) {
  int i = lo - 1, j = hi + 1;
  for (;;) {
    do
      i++;
    while (a[i] < pivot);
    do
      j--;
    while (a[j] > pivot);
    if (i ≥ j)
      return j;
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
  }
}

static void quicksort(int[] a, int lo, int hi) {
  if (lo ≥ hi)
    return;
  int mid = hoare(a, lo, hi, a[lo]);
  quicksort(a, lo, mid);
  quicksort(a, mid + 1, hi);
}

// median-of-three: the pivot value is the median of first, middle, last
static void quicksortM3(int[] a, int lo, int hi) {
  if (lo ≥ hi)
    return;
  int mid = lo + (hi - lo) / 2;
  int pivot = a[lo] < a[mid]
      ? (a[mid] < a[hi] ? a[mid] : (a[lo] < a[hi] ? a[hi] : a[lo]))
      : (a[lo] < a[hi] ? a[lo] : (a[mid] < a[hi] ? a[hi] : a[mid]));
  int p = hoare(a, lo, hi, pivot);
  quicksortM3(a, lo, p);
  quicksortM3(a, p + 1, hi);
}
```

listing 360: java, plain hoare on the first element, then the median-of-three variant (source: dsa/samples-java/src/Ch13/Quicksort.java, lines 18-55)

```java
  /// <summary>Quicksort, Hoare partition, median of three, insertion cutoff for tiny ranges.</summary>
  public static long Quick<T>(T[] a) where T : IComparable<T>
  {
      long comparisons = 0;
      QuickSort(a, 0, a.Length - 1, ref comparisons);
      return comparisons;
  }

  private static void QuickSort<T>(T[] a, int lo, int hi, ref long comparisons)
      where T : IComparable<T>
  {
      while (lo < hi)
      {
          if (hi - lo < 8) // small range: insertion is faster in cache
          {
```

```
            comparisons += InsertionRange(a, lo, hi);
            return;
        }
        int p = HoarePartition(a, lo, hi, ref comparisons);
        QuickSort(a, lo, p, ref comparisons);
        lo = p + 1; // tail loop instead of right recursion
    }
}

private static int HoarePartition<T>(T[] a, int lo, int hi, ref long comparisons)
    where T : IComparable<T>
{
    // median of three pivots defends the worst case on sorted input
    int mid = lo + (hi - lo) / 2;
    if (Less(a[mid], a[lo], ref comparisons))
        (a[lo], a[mid]) = (a[mid], a[lo]);
    if (Less(a[hi], a[lo], ref comparisons))
        (a[lo], a[hi]) = (a[hi], a[lo]);
    if (Less(a[hi], a[mid], ref comparisons))
        (a[mid], a[hi]) = (a[hi], a[mid]);

    T pivot = a[mid];
    int i = lo - 1, j = hi + 1;
    while (true)
    {
        do
        {
            i++;
            comparisons++;
        } while (a[i].CompareTo(pivot) < 0);

        do
        {
            j--;
            comparisons++;
        } while (a[j].CompareTo(pivot) > 0);

        if (i ≥ j)
            return j;
        (a[i], a[j]) = (a[j], a[i]);
    }
}

private static bool Less<T>(T a, T b, ref long comparisons)
    where T : IComparable<T>
{
    comparisons++;
    return a.CompareTo(b) < 0;
}

private static long InsertionRange<T>(T[] a, int lo, int hi) where T : IComparable<T>
{
    long comparisons = 0;
    for (int i = lo + 1; i ≤ hi; i++)
    {
        T key = a[i];
        int j = i - 1;
```

```
            while (j ≥ lo && a[j].CompareTo(key) > 0)
            {
                comparisons++;
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
        return comparisons;
    }
```

listing 361: c#, quicksort driver, median-of-three hoare partition, the small-range cutoff (source: dsa/samples/src/Ch13/Sorting.cs, lines 98-179)

The defenses matter because naive quicksort is quadratic on sorted input, first-element pivot plus already ordered data means every partition peels off one element. The test pins the fix at the meter: twenty thousand sorted elements sort under two million comparisons, where the naive version would burn about two hundred million. The all-equal test matters too, Hoare handles repeated pivots without crossing into quadratic behavior, where Lomuto's scheme does not.

```
  let j = hi + 1;
  for (;;) {
    do i++;
    while (a[i] < pivot);
    do j--;
    while (a[j] > pivot);
    if (i ≥ j) return j;
    swap(a, i, j);
  }
}

export function quicksort(a, lo = 0, hi = a.length - 1) {
  if (lo ≥ hi) return a;
  const mid = hoare(a, lo, hi, a[lo]);
  quicksort(a, lo, mid);
  quicksort(a, mid + 1, hi);
  return a;
}

// median of first, middle, last: the exported probe for the pivot choice
export function medianOfThree(a, lo = 0, hi = a.length - 1) {
  const mid = lo + Math.floor((hi - lo) / 2);
```

listing 362: javascript, hoare around a pivot value, first element plain (source: dsa/samples-js/src/ch13-quicksort.mjs, lines 12-33)

```
    i, j = lo - 1, hi + 1
    while True:
        i += 1
        while a[i] < pivot:
            i += 1
        j -= 1
        while a[j] > pivot:
            j -= 1
        if i ≥ j:
            return j
        a[i], a[j] = a[j], a[i]  # swaps land small left, large right


def quicksort(a, lo=0, hi=None, median_of_three=False):
    if hi is None:
        hi = len(a) - 1
    if lo ≥ hi:
        return a
    if median_of_three:
        mid = (lo + hi) // 2
        trio = sorted((a[lo], a[mid], a[hi]))
        pivot = trio[1]  # the median value, not a position
    else:
        pivot = a[(lo + hi) // 2]
    cut = hoare_partition(a, lo, hi, pivot)
    quicksort(a, lo, cut, median_of_three)
    quicksort(a, cut + 1, hi, median_of_three)
    return a
```

listing 363: python, hoare partition, pivot from the middle by default (source: dsa/samples-py/src/Ch13/quicksort.py, lines 14-43)

```lua
local function hoare(a, lo, hi, pivot)
  local i, j = lo - 1, hi + 1
  while true do
    repeat i = i + 1 until a[i] ≥ pivot
    repeat j = j - 1 until a[j] ≤ pivot
    if i ≥ j then return j end
    a[i], a[j] = a[j], a[i]
  end
end

local function quicksort(a, lo, hi)
  lo, hi = lo or 1, hi or #a
  if lo ≥ hi then return a end
  local mid = hoare(a, lo, hi, a[lo])
  quicksort(a, lo, mid)
  quicksort(a, mid + 1, hi)
  return a
end

-- median-of-three: the pivot value is the median of first, middle, last
local function pivot_m3(a, lo, hi)
  local x, y, z = a[lo], a[lo + (hi - lo) // 2], a[hi]
  if x < y then
    if y < z then return y end
    return x < z and z or x
  end
  if x < z then return x end
  return y < z and z or y
end

local function quicksort_m3(a, lo, hi)
  lo, hi = lo or 1, hi or #a
  if lo ≥ hi then return a end
  local mid = hoare(a, lo, hi, pivot_m3(a, lo, hi))
  quicksort_m3(a, lo, mid)
  quicksort_m3(a, mid + 1, hi)
  return a
end
```

listing 364: lua, repeat-until cursors, plain and median-of-three (source: dsa/samples-lua/ch13_quicksort.lua, lines 7-45)

Measured across the suites: C, Java, JavaScript, and Lua sort 5 2 8 1 9 3 7 4 6 to 1 through 9 under plain Hoare and again under median-of-three, duplicates 3 1 2 3 1 landing 1 1 2 3 3, and prove the permutation with sum and xor. Go runs five fixtures including the sorted-input worst case against slices.Sort ground truth and unit-tests the median probe itself, and Python runs seven fixtures from negatives to all-equal with sorted() as the cross-check, its default pivot the middle value rather than the first. The frozen C# suite is the only one that meters comparisons.

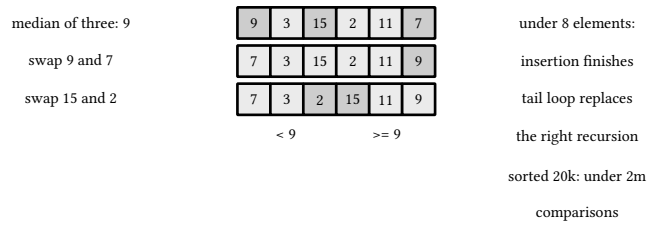


figure 99: quicksort defended, median of three, hoare cursors meet in the middle, small ranges fall to insertion

WARNING: quicksort is not stable

> Partitioning moves elements past each other without regard to original order, so equal keys can swap positions. .NET's `Array.Sort` is an introsort, quicksort with a heapsort fallback, and is documented unstable. When stability matters the BCL answer is LINQ's `OrderBy`, documented stable, at the cost of a full buffer. The suite pins both behaviors.

13.4. quickselect

The quicksort partition already knows how to split a range around a pivot. Selection is the question that needs only one half of it: to find the k-th smallest, partition once, keep the side that contains the k-th slot, and drop the rest on the floor. Hoare's split with the same median-of-three pivot as the defended quicksort above serves unchanged, and each round costs one pass over the surviving range, so the expected total is n plus n over 2 plus n over 4, a geometric sum landing at expected O(n), with the quadratic worst case stated and unfixed, the same risk profile as the sort.

The dry run: the fixture is 7, 1, 5, 3, 9, 2, 8, 6, 4 with k = 5 plus the duplicate and all-equal sets, asserted by the C# suite and pinned the same way in all six siblings.

1. Round 1 on the 9-wide range: median of three probes 7, 9, and 4, parks 9 at the far end, and pivots on 7, the hoare sweep swaps the pivot with 6 and the cursors cross at split 5.
2. The left side holds 5 - 0 + 1 = 6 cells, k = 5 fits inside, so hi tightens to 5 and k keeps its numbering.
3. Round 2 sees a 6-wide range, under the cutoff of 8: insertion sorts 4, 1, 5, 3, 6, 2 in place and the slot reads a[4] = 5, the median.
4. The right side 8, 7, 9 is dropped on the floor unread, the three elements the run never touches.
5. The duplicates 3, 1, 3, 2, 3 ride the insertion lane whole: k = 2 reads a[1] = 2, k = 4 reads a[3] = 3, and the all-equal 5s answer 5 for every k, the trap that demands a shrinking range.

round	range	action	k	lands
1	0..8	pivot 7, split at 5	5	left keeps it
2	0..5	insertion finish	5	a[4] = 5
dups	0..4	insertion finish	2	a[1] = 2
dups	0..4	insertion finish	4	a[3] = 3
all-equal	0..3	insertion finish	1..4	5 every time

The median 5 read off the 9-wide fixture is the pinned answer, and the listings below select in seven languages.

```c
// hoare partition around the pivot value; returns the split index
static int hoare(int *a, int lo, int hi, int pivot) {
  int i = lo - 1, j = hi + 1;
  for (;;) {
    do {
      i++;
    } while (a[i] < pivot);
    do {
      j--;
    } while (a[j] > pivot);
    if (i ≥ j)
      return j;
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
  }
}

// the k-th smallest (1-based k)
static int quickselect(int *a, int lo, int hi, int k) {
  while (lo < hi) {
    int pivot = median3(a, lo, hi);
    int j = hoare(a, lo, hi, pivot);
    // 0-based left count j - lo + 1: k-th slot on the left stays left
    if (k - 1 ≤ j)
      hi = j;
    else
      lo = j + 1;
  }
  return a[lo];
}
```

listing 365: c, hoare around the median-of-three pivot, then the single-sided loop (source: dsa/samples-c/src/Ch13/quickselect.c, lines 43-73)

```go
// QuickSelect returns the k-th smallest element of a (1-based k),
// partitioning a in place with the house median-of-three pivot and
// recursing into the single side that holds the k-th slot. Expected
// O(n), worst O(n^2). Hoare's split keeps both sides non-empty, so
// the all-equal input still shrinks the range and terminates.
func QuickSelect(a []int, k int) int {
  lo, hi := 0, len(a)-1
  for {
    if lo ≥ hi {
      return a[lo]
    }
    cut := hoarePartition(a, lo, hi)
    if k-1 <= cut {
      hi = cut
    } else {
      lo = cut + 1
    }
  }
}
```

listing 366: go, the whole loop over the shared hoare partition (source: dsa/samples-go/ch13/quickselect.go, lines 3-21)

```java
// hoare partition around the pivot value; returns the split index
static int hoare(int[] a, int lo, int hi, int pivot) {
  int i = lo - 1, j = hi + 1;
  for (;;) {
    do {
      i++;
    } while (a[i] < pivot);
    do {
      j--;
    } while (a[j] > pivot);
    if (i ≥ j)
      return j;
    int t = a[i];
    a[i] = a[j];
    a[j] = t;
  }
}

// the k-th smallest (1-based k)
static int quickselect(int[] a, int lo, int hi, int k) {
  while (lo < hi) {
    int pivot = median3(a, lo, hi);
    int j = hoare(a, lo, hi, pivot);
    // 0-based left count j - lo + 1: k-th slot on the left stays left
    if (k - 1 ≤ j)
      hi = j;
    else
      lo = j + 1;
  }
  return a[lo];
}
```

listing 367: java, hoare around the median-of-three pivot, then the single-sided loop (source: dsa/samples-java/src/Ch13/Quickselect.java, lines 43-73)

```
    /// <summary>
    /// Quickselect: hoare partition with the same median-of-three pivot as the
    /// quicksort listing, recursing into the single side holding the k-th slot
    /// (1-based k). Expected O(n), worst O(n^2); all-equal input still shrinks the
    /// range because the partition index always moves.
    /// </summary>
    public static int QuickSelect(int[] a, int k)
    {
        if (k < 1 || k > a.Length)
            throw new ArgumentOutOfRangeException(nameof(k), "k must lie in [1, length]");
        int lo = 0;
        int hi = a.Length - 1;
        while (true)
        {
            if (hi - lo < 8)
            {
                InsertionRange(a, lo, hi); // tiny range: sort and read the slot
                return a[lo + k - 1];
            }
            long comparisons = 0;
            int p = HoarePartition(a, lo, hi, ref comparisons);
            int leftSize = p - lo + 1;
            if (k ≤ leftSize)
                hi = p; // the k-th slot is inside the left side
            else
            {
                lo = p + 1;
                k -= leftSize; // renumber inside the right side
            }
        }
    }
```

listing 368: c#, the small-range insertion finish, the k renumbered inside the right side (source: dsa/samples/src/Ch13/Sorting.cs, lines 217-247)

```
function hoare(a, lo, hi, pivot) {
  let i = lo - 1;
  let j = hi + 1;
  for (;;) {
    do i++;
    while (a[i] < pivot);
    do j--;
    while (a[j] > pivot);
    if (i ≥ j) return j;
    [a[i], a[j]] = [a[j], a[i]];
  }
}

export function quickselect(a, k, lo = 0, hi = a.length - 1) {
  // k is 1-based within a[lo..hi]; hoare leaves every left element
  // ≤ pivot and every right element ≥ pivot, so the k-th slot lives
  // entirely on one side
  if (lo ≥ hi) return a[lo];
  const mid = hoare(a, lo, hi, medianOfThree(a, lo, hi));
  const leftSize = mid - lo + 1;
  if (k ≤ leftSize) return quickselect(a, k, lo, mid);
  return quickselect(a, k - leftSize, mid + 1, hi);
}
```

listing 369: javascript, the partition imported from the quicksort module, the recursion (source: dsa/samples-js/src/ch13-quickselect.mjs, lines 10-32)

```
def hoare_partition(a, lo, hi, pivot):
    i, j = lo - 1, hi + 1
    while True:
        i += 1
        while a[i] < pivot:
            i += 1
        j -= 1
        while a[j] > pivot:
            j -= 1
        if i ≥ j:
            return j  # both sides still shrink: the all-equal trap
        a[i], a[j] = a[j], a[i]  # swaps land small left, large right


def quickselect(a, k, lo=0, hi=None):
    # a[k] of the sorted order, 1-based k, in place
    if hi is None:
        hi = len(a) - 1
    while True:
        if lo ≥ hi:
            return a[lo]
        mid = (lo + hi) // 2
        trio = sorted((a[lo], a[mid], a[hi]))  # the house pivot rule
        cut = hoare_partition(a, lo, hi, trio[1])
        if k - 1 ≤ cut:  # keep the side holding the k-th slot
            hi = cut
        else:
            lo = cut + 1
```

listing 370: python, the hoare loop, the trio-sorted pivot, the kept side (source: dsa/samples-py/src/Ch13/quickselect.py, lines 15-42)

```lua
local function hoare(a, lo, hi, pivot)
  local i, j = lo - 1, hi + 1
  while true do
    repeat i = i + 1 until a[i] ≥ pivot
    repeat j = j - 1 until a[j] ≤ pivot
    if i ≥ j then return j end
    a[i], a[j] = a[j], a[i]
  end
end

local function pivot_m3(a, lo, hi)
  local x, y, z = a[lo], a[lo + (hi - lo) // 2], a[hi]
  if x < y then
    if y < z then return y end
    return x < z and z or x
  end
  if x < z then return x end
  return y < z and z or y
end

-- k is the 1-based slot. partitions in place (the array is partially
-- permuted afterwards, a permutation of the input), returns a[k]
local function quickselect(a, k)
  local lo, hi = 1, #a
  while lo < hi do
    local mid = hoare(a, lo, hi, pivot_m3(a, lo, hi))
    if k ≤ mid then hi = mid else lo = mid + 1 end
  end
  return a[lo]
end
```

listing 371: lua, the pivot rule in-file, the loop in 1-based indices (source: dsa/samples-lua/ch13_quickselect.lua, lines 9-38)

The equal-keys trap is the one that bites: an all-equal array must still shrink the range every round or the loop never ends. Hoare's split guarantees it here, the crossing cursors always leave at least one element on each side, and the three-way-partition alternative that handles repeated keys by grouping them is the standard note. The fixture families pin it: 7, 1, 5, 3, 9, 2, 8, 6, 4 answers k = 1 through 9 with 1 through 9 against the sorted ground truth, k = 5 reading the median 5; the duplicates 3, 1, 3, 2, 3 answer k = 2 with 2 and k = 4 with 3; all-equal 5, 5, 5, 5 answers 5 for every k and a single 42 answers itself; and a seeded lcg permutation of 40 elements matches sorted[k-1] on five sampled k. The permutation's lcg is 32-bit unsigned c: Java's int multiply wraps identically, so only the mod needs `Integer.remainderUnsigned`.

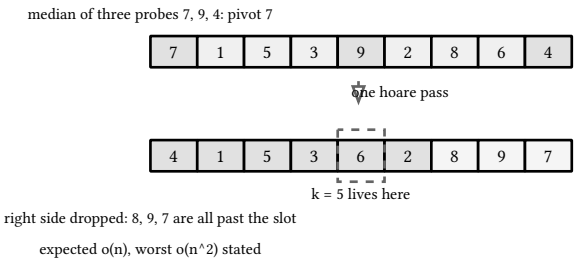


figure 100: one partition of the fixture array around the median of three, the k = 5 slot shaded on the kept side

The application is general technique, the offline k-th smallest, and the online upgrade that answers rank and select repeatedly under inserts is the fenwick machine of book 9, ch 30 (practical data structures and algorithms, balanced trees and order statistics).

13.5. counting and radix

Comparison sorts have a floor, log n factorial comparisons. Counting sort steps around the floor by tallying keys, and radix sort stacks stable counting passes least significant digit up. Each pass must be stable or the previous pass's work dies.

The dry run: the fixtures are the sign-flip ladder and the two radix runs, asserted by the C# suite; C, Go, Java, JavaScript, and Lua pin the counting fixture 4 2 2 8 3 3 1 instead and Python sorts digit tuples.

1. The flip is one xor of bit 63, and the ladder reads RadixKey(min) < RadixKey(-1) < RadixKey(0) < RadixKey(1) < RadixKey(max): unsigned byte order now equals signed long order.
2. The 5000-value fixture spans −1000000 to 1000000, and the byte passes land exactly Order()'s result, negatives first.
3. Each pass tallies 256 buckets and skips free when counts[0] equals the length, every key agreeing on that byte.
4. The agreeing fixture's 1000 keys live in 0 through 9999: after the flip byte 7 is uniformly 128 and bytes 6 down to 2 uniformly 0, so six of the eight passes skip.
5. Everything left rides the low two bytes, 8 - 6 = 2 passes of real work, the couple of passes the small magnitudes cost.

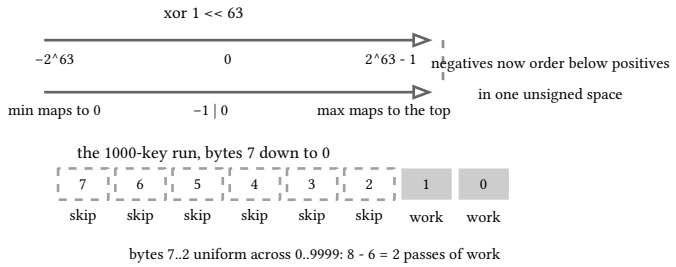


figure 101: the sign flip bending the signed line into unsigned order, then the eight byte passes with six skipped free

The ladder order and the two working passes are the C# pins, and the listings below bucket in seven languages.

```c
static void counting_sort(int *a, int len, int max) {
  int count[16] = {0}, out[16];
  for (int i = 0; i < len; i++)
    count[a[i]]++;
  for (int v = 1; v ≤ max; v++)
    count[v] += count[v - 1];
  // walk backward so equal keys keep their order
  for (int i = len - 1; i ≥ 0; i--)
    out[--count[a[i]]] = a[i];
  for (int i = 0; i < len; i++)
    a[i] = out[i];
}

typedef struct {
  int key, tag;
} Pair;

// one stable counting pass on the given decimal digit of the key
static void count_digit(Pair *a, int len, int div) {
  int count[10] = {0};
  Pair out[16];
  for (int i = 0; i < len; i++)
    count[a[i].key / div % 10]++;
  for (int d = 1; d < 10; d++)
    count[d] += count[d - 1];
  for (int i = len - 1; i ≥ 0; i--)
    out[--count[a[i].key / div % 10]] = a[i];
  for (int i = 0; i < len; i++)
    a[i] = out[i];
}

static void radix_lsd(Pair *a, int len) {
  int max = 0;
  for (int i = 0; i < len; i++)
    if (a[i].key > max)
      max = a[i].key;
  for (int div = 1; div ≤ max; div *= 10)
    count_digit(a, len, div);
}
```

listing 372: c, counting sort, one stable digit pass, radix driver (source: dsa/samples-c/src/Ch13/counting.c, lines 18-56)

```
  for _, v := range a {
    counts[v]++
  }
  out := make([]int, 0, len(a))
  for v, c := range counts {
    for ; c > 0; c-- {
      out = append(out, v)
    }
  }
  return out
}

// RadixRecord carries a 16-bit key and the original position, so the
// stability of the radix passes is observable.
type RadixRecord struct {
  Key uint16
  ID  int
}

// RadixSortLSD sorts records by their 16-bit key in two stable 8-bit
// counting passes: least significant byte first. Because each pass is
// stable, the second pass never disturbs the order the first pass
// established within equal high bytes.
func RadixSortLSD(records []RadixRecord) {
  countingPass := func(shift uint) {
    const bucket = 256
    counts := make([]int, bucket)
    for _, r := range records {
      counts[(r.Key>>shift)&0xff]++
    }
    starts := make([]int, bucket)
    total := 0
    for b := 0; b < bucket; b++ {
      starts[b] = total
      total += counts[b]
    }
    out := make([]RadixRecord, len(records))
    for _, r := range records { // scan in order: this is the stability
      b := (r.Key >> shift) & 0xff
      out[starts[b]] = r
      starts[b]++
    }
    copy(records, out)
  }
  countingPass(0)
  countingPass(8)
}
```

listing 373: go, tally-and-emit counting, two stable byte passes over 16-bit keys (source: dsa/samples-go/ch13/counting.go, lines 8-55)

```
// stable counting sort of ints in [0, max]
static void countingSort(int[] a, int max) {
  int[] count = new int[16], out = new int[16];
  for (int v : a)
    count[v]++;
  for (int v = 1; v ≤ max; v++)
    count[v] += count[v - 1];
  // walk backward so equal keys keep their order
  for (int i = a.length - 1; i ≥ 0; i--)
    out[--count[a[i]]] = a[i];
  System.arraycopy(out, 0, a, 0, a.length);
}

record Pair(int key, int tag) {}

// one stable counting pass on the given decimal digit of the key
static void countDigit(Pair[] a, int div) {
  int[] count = new int[10];
  Pair[] out = new Pair[16];
  for (Pair p : a)
    count[p.key() / div % 10]++;
  for (int d = 1; d < 10; d++)
    count[d] += count[d - 1];
  for (int i = a.length - 1; i ≥ 0; i--)
    out[--count[a[i].key() / div % 10]] = a[i];
  System.arraycopy(out, 0, a, 0, a.length);
}

static void radixLsd(Pair[] a) {
  int max = 0;
  for (Pair p : a)
    if (p.key() > max)
      max = p.key();
  for (int div = 1; div ≤ max; div *= 10)
    countDigit(a, div);
}
```

listing 374: java, backward fill for stability, decimal radix over record pairs (source: dsa/samples-java/src/Ch13/Counting.java, lines 18-53)

```
    /// <summary>LSD radix sort on bytes for longs: no comparisons at all.</summary>
    public static void RadixLsd(long[] a)
    {
        const int radixBits = 8;
        const int buckets = 1 << radixBits;
        const int passes = 64 / radixBits;

        // move into the sign-flipped key space where unsigned byte order
        // equals signed value order, then back at the end
        var keys = new ulong[a.Length];
        for (int i = 0; i < a.Length; i++)
            keys[i] = RadixKey(a[i]);
        var temp = new ulong[a.Length];

        for (int pass = 0, shift = 0; pass < passes; pass++, shift += radixBits)
        {
            var counts = new int[buckets];
            foreach (ulong k in keys)
                counts[(int)(k >> shift & (buckets - 1))]++;
            if (counts[0] == a.Length)
                continue; // this byte position is uniform
            int sum = 0;
            for (int b = 0; b < buckets; b++)
                (counts[b], sum) = (sum, sum + counts[b]);
            foreach (ulong k in keys)
                temp[counts[(int)(k >> shift & (buckets - 1))]++] = k;
            (keys, temp) = (temp, keys);
        }

        for (int i = 0; i < a.Length; i++)
            a[i] = (long)(keys[i] ^ (1UL << 63)); // unflip the key space
    }

    // flip the sign bit so unsigned byte order equals signed long order
    public static ulong RadixKey(long v) => (ulong)v ^ (1UL << 63);
```

listing 375: c#, lsd radix on bytes, sign-bit key flip, the uniform-pass skip (source: dsa/samples/src/Ch13/Sorting.cs, lines 181-215)

Two subtleties are pinned. Signed longs do not sort correctly as raw unsigned bytes, negatives would land after positives, so the key flips the sign bit into an order where unsigned comparison equals signed order, and flips back at the end. And a pass whose bucket count says every element shares the byte is skipped for free, which is why small-magnitude ints finish in a couple of passes.

```
  for (const v of a) count[v]++;
  for (let v = 1; v ≤ max; v++) count[v] += count[v - 1];
  // walk backward so equal keys keep their order
  for (let i = a.length - 1; i ≥ 0; i--) out[--count[a[i]]] = a[i];
  return out;
}

// one stable counting pass on byte (key >> shift) & 0xff
function countByte(a, shift) {
  const count = new Array(256).fill(0);
  const out = new Array(a.length);
  for (const r of a) count[(r.key >> shift) & 0xff]++;
  for (let d = 1; d < 256; d++) count[d] += count[d - 1];
  for (let i = a.length - 1; i ≥ 0; i--) {
    out[--count[(a[i].key >> shift) & 0xff]] = a[i];
  }
  return out;
}

// lsd over 16-bit keys: low byte, then high byte. keys must fit 16 bits
export function radixSortLSD(records) {
  let out = records;
  for (const shift of [0, 8]) out = countByte(out, shift);
  for (let i = 0; i < records.length; i++) records[i] = out[i];
  return records;
}
```

listing 376: javascript, backward fill for stability, byte radix (source: dsa/samples-js/src/ch13-counting.mjs, lines 9-35)

```
    # records: (key, tag) pairs with keys in a small nonnegative range
    if not records:
        return []
    counts = [0] * (max(key(r) for r in records) + 1)
    for r in records:
        counts[key(r)] += 1
    for i in range(1, len(counts)):
        counts[i] += counts[i - 1]  # prefix sums give output addresses
    out = [None] * len(records)
    for r in reversed(records):  # reversed fill keeps equal keys stable
        counts[key(r)] -= 1
        out[counts[key(r)]] = r
    return out


def radix_sort(records):
    # two-digit keys as (hi, lo) tuples: sort lo first, then hi
    lo_sorted = counting_sort(records, key=lambda r: r[0][1])
    return counting_sort(lo_sorted, key=lambda r: r[0][0])
```

listing 377: python, counting sort keyed by a function, radix over digit tuples (source: dsa/samples-py/src/Ch13/counting.py, lines 15-34)

```lua
local function counting_sort(a, max)
  local count = {}
  for v = 0, max do count[v] = 0 end
  for _, x in ipairs(a) do count[x] = count[x] + 1 end
  for v = 1, max do count[v] = count[v] + count[v - 1] end
  local out = {}
  -- walk backward so equal keys keep their order
  for i = #a, 1, -1 do
    count[a[i]] = count[a[i]] - 1
    out[count[a[i]] + 1] = a[i]
  end
  return out
end

-- one stable counting pass on the given decimal digit of the key
local function count_digit(a, div)
  local count = {}
  for d = 0, 9 do count[d] = 0 end
  for _, p in ipairs(a) do
    local d = p.key // div % 10
    count[d] = count[d] + 1
  end
  for d = 1, 9 do count[d] = count[d] + count[d - 1] end
  local out = {}
  for i = #a, 1, -1 do
    local d = a[i].key // div % 10
    count[d] = count[d] - 1
    out[count[d] + 1] = a[i]
  end
  return out
end

local function radix_lsd(a)
  local max = 0
  for _, p in ipairs(a) do
    if p.key > max then max = p.key end
  end
  local div = 1
  while div ≤ max do
    a = count_digit(a, div)
    div = div * 10
  end
  return a
end
```

listing 378: lua, backward walk with 1-based out slots, decimal radix (source: dsa/samples-lua/ch13_counting.lua, lines 7-50)

Measured across the suites: C, Go, Java, JavaScript, and Lua all counting-sort 4 2 2 8 3 3 1 to 1 2 2 3 3 4 8. The radix half splits by family: C, Java, and Lua run the classic 3-digit fixture 329 457 657 839 436 720 355 to 329 355 436 457 657 720 839 with tags surviving in order, Go and JavaScript run 16-bit keys in two stable byte passes, and Python sorts (hi, lo) digit tuples, probing the low pass alone to show it leaves the high digits alone. The frozen C# suite radixes longs by bytes with the sign flip. Stability is load-bearing: least-significant-first only works because each pass preserves the one before it.

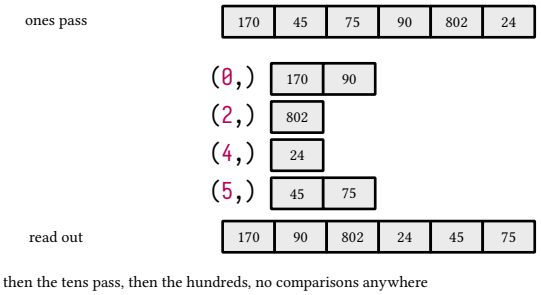


figure 102: radix pass by pass, counting buckets on the low digit, read out, repeat on the next

NOTE: an honest finding about the comparer contract

`Array.Sort` documents an `ArgumentException` for comparers that return inconsistent results. Probing net10.0 shows neither a flipping comparer nor one that reports equal elements as greater triggers it at sizes 10 through 1000, the sort completes and the output is an unspecified order, with the multiset intact. The documented check is not a safety net to rely on, the test suite records the probing result rather than the documented promise.

13.6. the stability probe

Stability is observable only when keys repeat and records carry identity. The probe sorts tagged pairs twice, once with the stable merge from this chapter, once with a deliberately unstable contrast, and reads the tags: equal keys must keep their input order under the first and are free to lose it under the second.

The dry run: the fixture is the C, Java, and Lua pair set (2, 0), (2, 1), (1, 2) through both sorts, the contrast this section teaches, with Go, JavaScript, and Python on three equal 2s and the C# suite pinning the stable half alone, OrderBy reading b d a c.

1. The set plants two equal 2s, ids 0 and 1, bracketing a 1 at id 2.
2. The stable merge outputs the 1 first, then reads the equal keys as a tie and keeps the left: ids 2 0 1, pinned.
3. Selection sort scans for the minimum, finds the 1 at index 2, and swaps it home in one long jump: (1, 2), (2, 1), (2, 0), id 0 riding the swap back past id 1.
4. Keys still read 1 2 2 ascending in both rows and only the equal pair moved: ids 2 1 0, the lost promise.
5. The longer set (3, 0), (1, 1), (3, 2), (3, 3) keeps ids 1 0 2 3 under both sorts, its only swap jumping over no equal: instability is a standing risk, not a certainty.

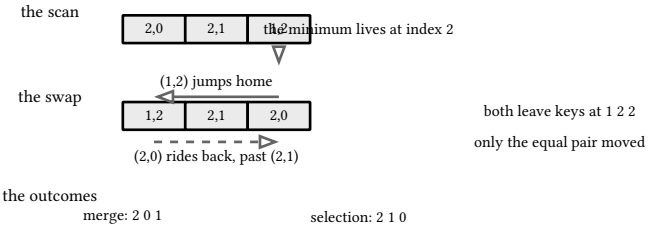


figure 103: the long swap of selection sort carrying id 0 back past id 1, the one move the stable merge never makes

The 2 0 1 against 2 1 0 contrast is the pinned pair, and the listings below probe it in seven languages.

```c
static void mmerge(Pair *a, int lo, int mid, int hi) {
  int i = lo, j = mid, k = lo;
  while (i < mid && j < hi)
    tmp[k++] = a[i].key ≤ a[j].key ? a[i++] : a[j++];
  while (i < mid)
    tmp[k++] = a[i++];
  while (j < hi)
    tmp[k++] = a[j++];
  for (int t = lo; t < hi; t++)
    a[t] = tmp[t];
}

static void msort(Pair *a, int lo, int hi) {
  if (hi - lo < 2)
    return;
  int mid = lo + (hi - lo) / 2;
  msort(a, lo, mid);
  msort(a, mid, hi);
  mmerge(a, lo, mid, hi);
}

// selection sort: find the min, swap it into place. the long-distance
// swap is what breaks stability on equal keys
static void selsort(Pair *a, int len) {
  for (int i = 0; i < len; i++) {
    int m = i;
    for (int j = i + 1; j < len; j++)
      if (a[j].key < a[m].key)
        m = j;
    if (m ≠ i) {
      Pair t = a[i];
      a[i] = a[m];
      a[m] = t;
    }
  }
}
```

listing 379: c, the stable merge against selection sort's long swap (source: dsa/samples-c/src/Ch13/stability.c, lines 23-58)

```
  n := len(recs)
  for i := n/2 - 1; i ≥ 0; i-- {
    siftRecs(recs, i, n)
  }
  for end := n - 1; end > 0; end-- {
    recs[0], recs[end] = recs[end], recs[0]
    siftRecs(recs, 0, end)
  }
}

func siftRecs(recs []Pair, i, n int) {
  for {
    left, right := 2*i+1, 2*i+2
    largest := i
    if left < n && recs[left].Key > recs[largest].Key {
      largest = left
    }
    if right < n && recs[right].Key > recs[largest].Key {
      largest = right
    }
    if largest == i {
      return
    }
    recs[i], recs[largest] = recs[largest], recs[i]
    i = largest
  }
}

// SortStable wraps the chapter's mergesort for records.
func SortStable(recs []Pair) { Mergesort(recs) }
```

listing 380: go, heapsort over records as the unstable contrast, two wrappers (source: dsa/samples-go/ch13/stability.go, lines 8-38)

```
static void mmerge(Pair[] a, int lo, int mid, int hi) {
  int i = lo, j = mid, k = lo;
  while (i < mid && j < hi)
    tmp[k++] = a[i].key() ≤ a[j].key() ? a[i++] : a[j++];
  while (i < mid)
    tmp[k++] = a[i++];
  while (j < hi)
    tmp[k++] = a[j++];
  for (int t = lo; t < hi; t++)
    a[t] = tmp[t];
}

static void msort(Pair[] a, int lo, int hi) {
  if (hi - lo < 2)
    return;
  int mid = lo + (hi - lo) / 2;
  msort(a, lo, mid);
  msort(a, mid, hi);
  mmerge(a, lo, mid, hi);
}

// selection sort: find the min, swap it into place. the long-distance
// swap is what breaks stability on equal keys
static void selsort(Pair[] a) {
  for (int i = 0; i < a.length; i++) {
    int m = i;
    for (int j = i + 1; j < a.length; j++)
      if (a[j].key() < a[m].key())
        m = j;
    if (m ≠ i) {
      Pair t = a[i];
      a[i] = a[m];
      a[m] = t;
    }
  }
}
```

listing 381: java, the stable merge against selection sort's long swap (source: dsa/samples-java/src/Ch13/Stability.java, lines 21-57)

```
    private static (long, long) MergeHalves<T>(T[] a, T[] buffer, int lo, int mid, int hi)
        where T : IComparable<T>
    {
        Array.Copy(a, lo, buffer, lo, hi - lo + 1);
        long comparisons = 0, inversions = 0;
        int i = lo, j = mid + 1, k = lo;
        while (i ≤ mid && j ≤ hi)
        {
            comparisons++;
            // taking from the right half only when strictly smaller keeps it stable
            if (buffer[j].CompareTo(buffer[i]) < 0)
            {
                inversions += mid - i + 1; // every remaining left element was inverted
                a[k++] = buffer[j++];
            }
            else
            {
                a[k++] = buffer[i++];
            }
        }
        while (i ≤ mid)
            a[k++] = buffer[i++];
        while (j ≤ hi)
            a[k++] = buffer[j++];
        return (comparisons, inversions);
    }
```

listing 382: c#, the merge halves, strict less is the stable decision (source: dsa/samples/src/Ch13/Sorting.cs, lines 71-96)

```
function mmerge(a, lo, mid, hi) {
  const tmp = [];
  let i = lo;
  let j = mid;
  while (i < mid && j < hi)
    tmp.push(a[i].key ≤ a[j].key ? a[i++] : a[j++]);
  while (i < mid) tmp.push(a[i++]);
  while (j < hi) tmp.push(a[j++]);
  for (let t = lo; t < hi; t++) a[t] = tmp[t - lo];
}

export function sortStable(a, lo = 0, hi = a.length) {
  if (hi - lo < 2) return a;
  const mid = lo + Math.floor((hi - lo) / 2);
  sortStable(a, lo, mid);
  sortStable(a, mid, hi);
  mmerge(a, lo, mid, hi);
  return a;
}

// heapsort over pairs keyed by .key with strict >: equal keys never stop
// the sift, so they ride down and come back out reversed
function siftDown(a, len, i) {
  for (;;) {
    const l = 2 * i + 1;
    const r = 2 * i + 2;
    let m = i;
    if (l < len && a[l].key > a[m].key) m = l;
    if (r < len && a[r].key > a[m].key) m = r;
    if (m === i) return;
    [a[i], a[m]] = [a[m], a[i]];
    i = m;
  }
}

export function sortUnstable(a) {
  for (let i = Math.floor(a.length / 2) - 1; i ≥ 0; i--)
    siftDown(a, a.length, i);
  for (let end = a.length - 1; end > 0; end--) {
    [a[0], a[end]] = [a[end], a[0]];
    siftDown(a, end, 0);
  }
  return a;
}
```

listing 383: javascript, stable merge sort, heapsort walking equals out reversed (source: dsa/samples-js/src/ch13-stability.mjs, lines 5-48)

```
    while i < len(left) and j < len(right):
        if left[i][0] ≤ right[j][0]:  # the tie goes to the left run
            out.append(left[i])
            i += 1
        else:
            out.append(right[j])
            j += 1
    out.extend(left[i:])
    out.extend(right[j:])
    return out


def mergesort(pairs):
    if len(pairs) ≤ 1:
        return list(pairs)
    mid = len(pairs) // 2
    return merge(mergesort(pairs[:mid]), mergesort(pairs[mid:]))


def sift_down(a, i, size):
    while True:
        left, right, big = 2 * i + 1, 2 * i + 2, i
        if left < size and a[left][0] > a[big][0]:
            big = left
        if right < size and a[right][0] > a[big][0]:
            big = right
        if big == i:
            return
        a[i], a[big] = a[big], a[i]  # distance swap: stability dies here
        i = big


def heapsort(pairs):
    size = len(pairs)
    for i in range(size // 2 - 1, -1, -1):
        sift_down(pairs, i, size)
    for end in range(size - 1, 0, -1):
        pairs[0], pairs[end] = pairs[end], pairs[0]
        sift_down(pairs, 0, end)
```

listing 384: python, tagged tuples through merge and heapsort (source: dsa/samples-py/src/Ch13/stability.py, lines 15-54)

```lua
local function msort(a)
  local tmp = {}
  local function rec(lo, hi)
    if hi - lo < 1 then return end
    local mid = lo + (hi - lo) // 2
    rec(lo, mid)
    rec(mid + 1, hi)
    -- stable merge: on equal keys the left-run element wins
    local i, j, k = lo, mid + 1, lo
    while i ≤ mid and j ≤ hi do
      if a[i].key ≤ a[j].key then
        tmp[k] = a[i]; i = i + 1
      else
        tmp[k] = a[j]; j = j + 1
      end
      k = k + 1
    end
    while i ≤ mid do tmp[k] = a[i]; i = i + 1; k = k + 1 end
    while j ≤ hi do tmp[k] = a[j]; j = j + 1; k = k + 1 end
    for t = lo, hi do a[t] = tmp[t] end
  end
  rec(1, #a)
  return a
end

-- selection sort: find the min, swap it into place. the long-distance
-- swap is what breaks stability on equal keys
local function selsort(a)
  for i = 1, #a do
    local m = i
    for j = i + 1, #a do
      if a[j].key < a[m].key then m = j end
    end
    if m ~= i then a[i], a[m] = a[m], a[i] end
  end
  return a
end
```

listing 385: lua, nested msort, selection sort contrast (source: dsa/samples-lua/ch13_stability.lua, lines 6-42)

Measured across the suites: C, Java, and Lua feed (2,0) (2,1) (1,2), the stable merge emits ids 2 0 1 while selection sort emits 2 1 0, its long swap pulling id 0 past id 1, and a longer fixture shows the contrast is a standing risk rather than a certainty. Go, JavaScript, and Python feed three equal 2s and a single 1, keep ids 0 1 2 under the stable merge, and watch heapsort walk them out reversed. Both sorts land ascending keys either way: instability is not a wrong answer, it is a lost promise, the one `Array.Sort` documents in the callout above.

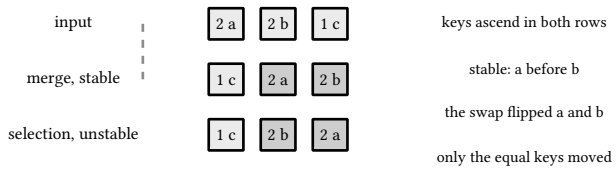


figure 104: the stability probe, one fixture, two sorts, only the equal keys move

The selection rule the whole chapter earns: default to the library sort, `Array.Sort` for primitives and sizes, `OrderBy` when stability matters, radix when keys are dense integers and n is large, insertion when the range is small or nearly ordered. The capstone's compaction does no sorting at all, it merges already sorted runs, chapter 8's k-way merge, which is the third way ordering appears in systems.

13.7. across the seven languages

The build sizes count non-comment source lines. The first table covers the chapter's five original featured files per language, quicksort, mergesort, counting, the stability probe, and quickselect, the C# file carrying the insertion pair and the metered quicksort too, one file for the whole chapter:

language	build SLOC	dependency	boundary note
c	433	libc only	static tmp buffers, one file per sort, plain and median-of-three quicksort side by side, checks share the file with main, 53 of them
go	162	slices	slices.Sort as ground truth only, radix over uint16 keys, quickselect over the shared partition, tests in separate files, 13 tests
java	413	jdk 27 stdlib	one file per sort, plain and median-of-three quicksort side by side, records carry the tags, decimal radix, the shuffle lcg's mod rides Integer.remainderUnsigned, 38 checks in 5 files
c#	212	bcl only	comparison meters and inversion counting throughout, insertion cutoff inside quicksort and quickselect, 16 tests
javascript	160	node stdlib	destructured swaps, radix over 16-bit keys in two byte passes, quickselect importing the quicksort pivot, 13 tests
python	297	stdlib only	sorted() as ground truth only, tagged tuples make stability visible, the seeded permutation property, 45 checks
lua	353	lib.lua harness	repeat-until hoare cursors, decimal radix base 10, checks ride in the module, 19 of them

The counted insertion baseline lands as its own file in the six sibling trees, the C# lane keeping its generic pair inside the chapter's one file:

language	build SLOC	dependency	boundary note
c	74	libc only	long counters, shifts through an out parameter, the bound permutation built inline from the coprime multiplier 31
go	31	slices	the pair (comparisons, shifts) returned, DeterministicShuffle feeding the bound lane
java	73	jdk 27 stdlib	long counters in a record pair, the bound permutation built inline from the same coprime multiplier 31, 14 checks
c#	41	bcl only	the generic insertion pair, plain and binary, over IComparable, the comparison count as the meter
javascript	16	node stdlib	an optional meter object carries the shift count, the smallest build of the set
python	61	stdlib only	comparisons alone returned, the 64-bit lcg shuffle beside it, shuffled slices checked too
lua	90	lib.lua harness	1-based walk, the pair returned, table.sort as the bound lane's ground truth

sources: learn.microsoft.com, `Array.Sort` remarks including the introsort and instability notes, `Enumerable.OrderBy` stability remarks, `Array.Fill`, accessed 2026-09-08, plus probing of the runtime itself for the comparer finding, and cp-algorithms, "K-th order statistic in O(N)", cp-algorithms.com/sequences/k-th.html, accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Sample behavior verified by `make verify-csharp`, 16 tests in chapter 13 of the samples suite. The seven-language layer verifies the same way: 6 C programs with 67 embedded checks under `make verify-c`, 16 Go tests, the java runner's 52 Ch13 checks over 6 files under `run-java-samples`, 19 `node --test` cases, 60 Python checks across 6 files, and 26 Lua checks under `run.lua`.

14. searching and answer spaces

Binary search is one idea with a dozen correct-looking wrong implementations. This chapter builds the boundary forms that are always right, the rotated variant, and the search-over-answers pattern that turns binary search on arrays into binary search on any monotone predicate. The string matchers live next door in chapter 15.

14.1. boundaries, not hits

Searching for "an element equal to x" is the fragile framing. The robust one asks for the first index where the array element is at least the target, lower bound, or strictly greater, upper bound. Everything else is arithmetic on those two.

The dry run: the fixture is 1 3 3 3 5 7 9 9, asserted by the C# suite, while the integer suites walk the six-element cut 1 3 3 3 5 7 and Go its 1 2 2 2 5 7 twin.

1. Lower bound of 3 opens the half-open 0..8 and probes slot 4: the 5 there is not below 3, so hi closes to 4 and the upper half is gone.
2. Slots 2 and 1 both hold 3, still not below, hi walks to 2 then 1, and slot 0′s 1 finally moves lo to 1: the walk returns 1, the first slot at least 3.
3. Upper bound keeps equals on lo's side: slots 2 and 3 pass 3 and lo walks 3 then 4, three probes closing at 4, the first greater.
4. Count is the subtraction, 4 - 1 = 3, leftmost reads the 1 back, and rightmost reads 4 - 1 = 3.
5. The absent 4 inserts at 4, between the 3s and the 5, the off-end keys 0 and 100 read back 0 and 8, and leftmost of 8 and rightmost of 2 both return −1.

walk	lo	hi	mid	a[mid]	move
lower	0	8	4	5, not below 3	hi = 4
lower	0	4	2	3, not below 3	hi = 2
lower	0	2	1	3, not below 3	hi = 1
lower	0	1	0	1, below 3	lo = 1
upper	0	8	4	5, above 3	hi = 4
upper	0	4	2	3, equals	lo = 3
upper	3	4	3	3, equals	lo = 4

The 1 and the 4 around the count 3 are the pinned triple, and the listings below run the same two loops in seven languages.

```
  int lo = 0, hi = len;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}

static int upper_bound(const int *a, int len, int t) {
  int lo = 0, hi = len;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] ≤ t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}

static int count_eq(const int *a, int len, int t) {
  return upper_bound(a, len, t) - lower_bound(a, len, t);
}

int main(void) {
```

listing 386: c, the two half-open walks, count by subtraction (source: dsa/samples-c/src/Ch14/bounds.c, lines 18-45)

```
func LowerBound(a []int, t int) int {
  lo, hi := 0, len(a)
  for lo < hi {
    mid := lo + (hi-lo)/2
    if a[mid] < t {
      lo = mid + 1
    } else {
      hi = mid
    }
  }
  return lo
}

// UpperBound returns the first index whose value is > t: the right
// insertion point of t.
func UpperBound(a []int, t int) int {
  lo, hi := 0, len(a)
  for lo < hi {
    mid := lo + (hi-lo)/2
    if a[mid] <= t {
      lo = mid + 1
    } else {
      hi = mid
    }
  }
  return lo
}

// CountOf counts entries equal to t: the distance between the two
// bounds, zero when t is absent.
func CountOf(a []int, t int) int {
  return UpperBound(a, t) - LowerBound(a, t)
}

// InsertionPoint returns where t would slot into a to keep it sorted:
```

listing 387: go, the same pair, insertion point as an alias (source: dsa/samples-go/ch14/bounds.go, lines 8-42)

```java
static int lowerBound(int[] a, int t) {
  int lo = 0, hi = a.length;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] < t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}

static int upperBound(int[] a, int t) {
  int lo = 0, hi = a.length;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] ≤ t)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}

static int countEq(int[] a, int t) {
  return upperBound(a, t) - lowerBound(a, t);
}
```

listing 388: java, the two half-open walks, count by subtraction, the linear partition sweep (source: dsa/samples-java/src/Ch14/Bounds.java, lines 18-44)

```
public static class BinarySearches
{
    // first index where a[i] ≥ target: the insertion point
    public static int LowerBound(int[] a, int target)
    {
        int lo = 0, hi = a.Length;
        while (lo < hi)
        {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] < target)
                lo = mid + 1;
            else
                hi = mid;
        }
        return lo;
    }

    // first index where a[i] > target
    public static int UpperBound(int[] a, int target)
    {
        int lo = 0, hi = a.Length;
        while (lo < hi)
        {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] ≤ target)
                lo = mid + 1;
            else
                hi = mid;
        }
        return lo;
    }

    public static int CountOf(int[] a, int target) =>
        UpperBound(a, target) - LowerBound(a, target);

    public static int Leftmost(int[] a, int target)
    {
        int i = LowerBound(a, target);
        return i < a.Length && a[i] == target ? i : -1;
    }

    public static int Rightmost(int[] a, int target)
    {
        int i = UpperBound(a, target) - 1;
        return i ≥ 0 && a[i] == target ? i : -1;
    }
```

listing 389: c#, lower and upper bound, count and leftmost and rightmost on top (source: dsa/samples/src/Ch14/Searching.cs, lines 4-49)

The duplicate-heavy test pins the payoff: count of a value is upper minus lower, no scans. Both loops return `lo` when the target is absent, which is exactly the insertion point, so “not found” carries useful information instead of a bare −1.

```javascript
export function lowerBound(a, t) {
  let lo = 0;
  let hi = a.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] < t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}

export function upperBound(a, t) {
  let lo = 0;
  let hi = a.length;
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] ≤ t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}

export function countOf(a, t) {
  return upperBound(a, t) - lowerBound(a, t);
}

export function insertionPoint(a, t) {
  return lowerBound(a, t);
}
```

listing 390: javascript, floor-mid walks, count and insertion point (source: dsa/samples-js/src/ch14-bounds.mjs, lines 5-33)

```python
def lower_bound(a, target):
    lo, hi = 0, len(a)  # half-open: the answer lives in 0..len
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] < target:
            lo = mid + 1  # everything up through mid is too small
        else:
            hi = mid
    return lo


def upper_bound(a, target):
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        if a[mid] ≤ target:
            lo = mid + 1  # mid itself is not past the target
        else:
            hi = mid
    return lo
```

listing 391: python, the half-open interval 0..len (source: dsa/samples-py/src/Ch14/bounds.py, lines 13-34)

```lua
local function lower_bound(a, t)
  local lo, hi = 1, #a + 1
  while lo < hi do
    local mid = lo + (hi - lo) // 2
    if a[mid] < t then lo = mid + 1 else hi = mid end
  end
  return lo - 1 -- 0-based insertion index
end

local function upper_bound(a, t)
  local lo, hi = 1, #a + 1
  while lo < hi do
    local mid = lo + (hi - lo) // 2
    if a[mid] ≤ t then lo = mid + 1 else hi = mid end
  end
  return lo - 1
end
```

listing 392: lua, 1-based walk, answer shifted back to 0-based (source: dsa/samples-lua/ch14_bounds.lua, lines 6-23)

Measured across the suites: C, Java, Lua, and Python share the fixture 1 3 3 3 5 7 and pin the run of 3s at lower 1, upper 4, count 3, the absent 4 inserting at 4 and out-of-range keys at 0 and 6, C, Java, and Lua sweeping every target against a linear partition check. Go runs the same boundary story on 1 2 2 2 5 7, three copies of 2, and JavaScript cross-checks both bounds against linear scans target by target. The frozen C# suite adds leftmost and rightmost on top of the same subtraction. Every loop is the same half-open walk, one comparison character apart.

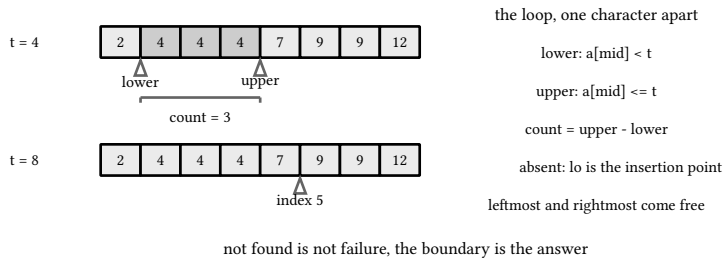


figure 105: the boundary loops on one duplicate heavy array, lower is the first at least, upper the first greater, count is the difference

WARNING: the three ways this loop goes wrong

First, `mid = (lo + hi) / 2` overflows for large arrays in languages with fixed ints, hence the `lo + (hi - lo) / 2` form everywhere here. Second, terminating with `lo > hi` inside an equality test misses the boundary semantics. Third, off-by-one in the loop body's ≤ versus < is the difference between lower and upper bound, which is why both exist explicitly.

14.2. rotated arrays

A sorted array rotated at an unknown pivot still has one sorted half at every step, and knowing which half the target belongs to keeps the search logarithmic.

The dry run: the fixture is 5 7 9 1 3 3, the sorted 1 3 3 5 7 9 rotated by 3, asserted by the C# suite, while the integer suites pin the same rule on 4 5 6 7 0 1 2.

1. The hunt for 1 opens 0..5 and probes slot 2: a[2] = 9, and a[lo] <= a[mid] names the left half 5 7 9 sorted, with 1 outside it, so lo jumps to 3.
2. Slot 4 reads 3, the left half sorts again as 1 3, and this time 1 sits inside it, so hi drops to 3.
3. Slot 3 reads 1, the hit at the pinned index.

4. The family around it: the duplicate 3 lands at 4 one probe later, the leading 5 lands at slot 0 two probes in, and the absent 42 runs the interval empty for −1.
5. The unrotated 1 2 3 4 5 walks slots 2, 3, 4 to the 5 at index 4, rotation zero degenerating into the plain boundary walk above.

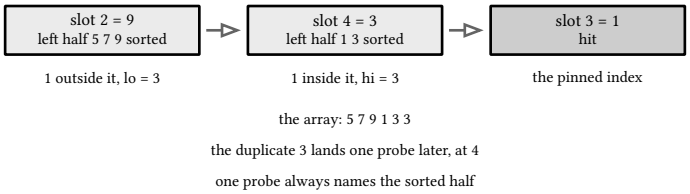


figure 106: the hunt for 1 as three probes, each naming the sorted half before the interval moves

The 3 for the 1 with the 4 one probe behind is the pinned pair, and the listings below cover both halves in seven languages.

```
  int lo = 0, hi = len - 1;
  while (lo ≤ hi) {
    int mid = lo + (hi - lo) / 2;
    if (a[mid] == t)
      return mid;
    if (a[lo] ≤ a[mid]) { // left half is sorted
      if (t ≥ a[lo] && t < a[mid])
        hi = mid - 1;
      else
        lo = mid + 1;
    } else { // right half is sorted
      if (t > a[mid] && t ≤ a[hi])
        lo = mid + 1;
      else
        hi = mid - 1;
    }
  }
  return -1;
}

int main(void) {
  static const int a[7] = {4, 5, 6, 7, 0, 1, 2};

  // every present key maps to its index
  CHECK(search(a, 7, 4) == 0);
```

listing 393: c, which half is sorted, the target inside it or the other (source: dsa/samples-c/src/Ch14/rotated.c, lines 18-42)

```go
func SearchRotated(a []int, t int) int {
  lo, hi := 0, len(a)-1
  for lo <= hi {
    mid := lo + (hi-lo)/2
    if a[mid] == t {
      return mid
    }
    if a[lo] <= a[mid] { // left half [lo..mid] is sorted
      if a[lo] <= t && t < a[mid] {
        hi = mid - 1 // t lies inside the sorted half
      } else {
        lo = mid + 1
      }
    } else { // right half [mid..hi] is sorted
      if a[mid] < t && t <= a[hi] {
        lo = mid + 1
      } else {
        hi = mid - 1
      }
    }
  }
  return -1
```

listing 394: go, endpoint comparison picks the sorted half (source: dsa/samples-go/ch14/rotated.go, lines 6-27)

```java
  static int search(int[] a, int t) {
    int lo = 0, hi = a.length - 1;
    while (lo ≤ hi) {
      int mid = lo + (hi - lo) / 2;
      if (a[mid] == t)
        return mid;
      if (a[lo] ≤ a[mid]) { // left half is sorted
        if (t ≥ a[lo] && t < a[mid])
          hi = mid - 1;
        else
          lo = mid + 1;
      } else { // right half is sorted
        if (t > a[mid] && t ≤ a[hi])
          lo = mid + 1;
        else
          hi = mid - 1;
      }
    }
    return -1;
  }
```

listing 395: java, which half is sorted, the target inside it or the other (source: dsa/samples-java/src/Ch14/Rotated.java, lines 18-37)

```
    public static int Rotated(int[] a, int target)
    {
        int lo = 0, hi = a.Length - 1;
        while (lo ≤ hi)
        {
            int mid = lo + (hi - lo) / 2;
            if (a[mid] == target)
                return mid;
            if (a[lo] ≤ a[mid]) // left half is sorted
            {
                if (a[lo] ≤ target && target < a[mid])
                    hi = mid - 1;
                else
                    lo = mid + 1;
            }
            else // right half is sorted
            {
                if (a[mid] < target && target ≤ a[hi])
                    lo = mid + 1;
                else
                    hi = mid - 1;
            }
        }
        return -1;
    }
```

listing 396: c#, the rotated walk, both halves covered (source: dsa/samples/src/Ch14/Searching.cs, lines 52-76)

```
export function searchRotated(a, t) {
  let lo = 0;
  let hi = a.length - 1;
  while (lo ≤ hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (a[mid] === t) return mid;
    if (a[lo] ≤ a[mid]) {
      // left half is sorted
      if (t ≥ a[lo] && t < a[mid]) hi = mid - 1;
      else lo = mid + 1;
    } else {
      // right half is sorted
      if (t > a[mid] && t ≤ a[hi]) lo = mid + 1;
      else hi = mid - 1;
    }
  }
  return -1;
}
```

listing 397: javascript, the same rule, strict tests on both sides (source: dsa/samples-js/src/ch14-rotated.mjs, lines 5-23)

```python
def search_rotated(a, target):
    lo, hi = 0, len(a) - 1
    while lo ≤ hi:
        mid = (lo + hi) // 2
        if a[mid] == target:
            return mid
        if a[lo] ≤ a[mid]:  # the left half runs clean
            if a[lo] ≤ target < a[mid]:
                hi = mid - 1
            else:
                lo = mid + 1
        else:  # the right half runs clean
            if a[mid] < target ≤ a[hi]:
                lo = mid + 1
            else:
                hi = mid - 1
    return -1


A = [4, 5, 6, 7, 0, 1, 2]
check(search_rotated(A, 0) == 4, "the pivot point index pins")
check(search_rotated(A, 4) == 0, "the first element is findable")
```

listing 398: python, chained comparisons name the sorted half (source: dsa/samples-py/src/Ch14/rotated.py, lines 12-34)

```lua
local function search(a, t)
  local lo, hi = 1, #a
  while lo ≤ hi do
    local mid = lo + (hi - lo) // 2
    if a[mid] == t then return mid - 1 end
    if a[lo] ≤ a[mid] then -- left half is sorted
      if t ≥ a[lo] and t < a[mid] then
        hi = mid - 1
      else
        lo = mid + 1
      end
    else -- right half is sorted
      if t > a[mid] and t ≤ a[hi] then
        lo = mid + 1
      else
        hi = mid - 1
      end
    end
  end
  return -1
end

return {
```

listing 399: lua, 1-based indices, answer shifted back (source: dsa/samples-lua/ch14_rotated.lua, lines 5-28)

Measured across the suites: C, Go, Java, JavaScript, Lua, and Python all pin indices on 4 5 6 7 0 1 2, every member at its slot, 0 at index 4, the absent 3, 8, and −1 refused, then sweep rotation zero, rotated by one, rotated by four, and the single element. The frozen C# suite pins its own fixture under the same rule. The discipline is one line everywhere: a[lo] <= a[mid] names the sorted half, and the target either lies inside it or forces the other.

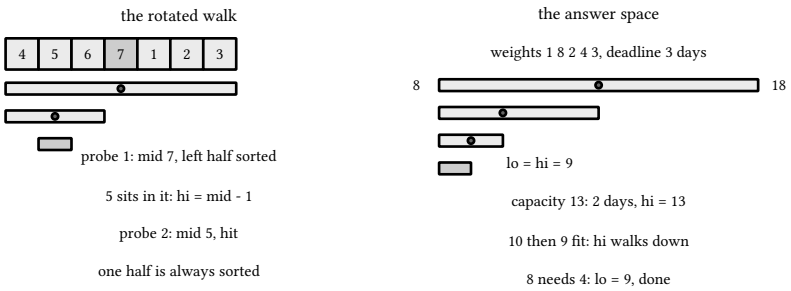


figure 107: the two logarithmic walks, the rotated array asks which half is sorted, the answer space binary searches a monotone predicate

14.3. the answer space

The pattern that extends binary search beyond arrays: when a predicate is monotone in some quantity, false then true as the quantity grows, binary search finds the transition. The package-shipping question asks for the minimum daily capacity that finishes within a deadline. The answer is not in any array, it is a number whose feasibility predicate is monotone.

The dry run: the C# fixture is the weights 1 through 10 against a 5-day deadline, asserted at 15, and the integer suites carry the chapter's cross-language anchor, 1 8 2 4 3 in 3 days landing 9.

1. The bracket opens at the heaviest package, lo = 10, and closes at the total, hi = 55, the one-day answer the suite pins beside it.
2. Probe 10 + (55 - 10) / 2 = 32: day one holds 1 through 7, the rest takes one more day, 2 days, feasible, hi = 32.
3. Probe 10 + (32 - 10) / 2 = 21: day one fills exactly on 1 + 2 + 3 + 4 + 5 + 6 = 21, two more days finish, hi = 21.
4. Probe 10 + (21 - 10) / 2 = 15: the days pack 1 through 5, then 6 and 7, then 8, 9, 10 alone, five days, exactly the deadline, and hi = 15.
5. Probes 12 and 14 both break on the same edge, 10 + 5 = 15 over either cap, and each needs a sixth day: lo walks 13 then 15 and the bisect closes on 15, the pinned answer.
6. The edges beside it: one day forces the full 55, ten days drop the answer to the heaviest package, 10.

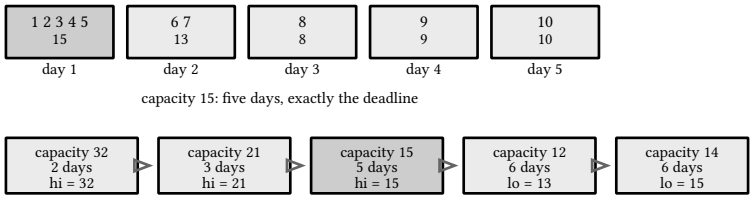


figure 108: the feasible packing at capacity 15, five day bins with their loads, the probe walk that closed on it underneath

The 15 as the first feasible capacity is the pinned landing, and the listings below bisect the same predicate in seven languages.

```c
static int days_needed(int cap) {
  int days = 1, load = 0;
  for (int i = 0; i < 5; i++) {
    if (W[i] > cap)
      return 1000; // a single weight overflows any day
    if (load + W[i] > cap) {
      days++;
      load = W[i];
    } else {
      load += W[i];
    }
  }
  return days;
}

static int feasible(int cap) { return days_needed(cap) ≤ 3; }

// smallest capacity in [lo, hi] with feasible(cap), lo infeasible-safe
static int bisect(int lo, int hi) {
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid))
      hi = mid;
    else
      lo = mid + 1;
  }
  return lo;
}
```

listing 400: c, the greedy day count, the feasibility test, the bisect (source: dsa/samples-c/src/Ch14/answerspace.c, lines 20-47)

```
  load := 0
  for _, w := range weights {
    if load+w > capacity {
      days++
      load = w
    } else {
      load += w
    }
  }
  return days
}

// MinDailyCapacity binary searches the answer space instead of a
// sorted array: the smallest capacity under which the weights ship in
// at most `days` days. The search window is bracketed by the heaviest
// single package and the total weight.
func MinDailyCapacity(weights []int, days int) int {
  lo, hi := 0, 0
  for _, w := range weights {
    lo = max(lo, w)
    hi += w
  }
  for lo < hi {
    mid := lo + (hi-lo)/2
    if DaysNeeded(weights, mid) <= days {
      hi = mid // feasible: try cheaper
    } else {
      lo = mid + 1 // infeasible: pay more
    }
  }
  return lo
}
```

listing 401: go, days needed, min capacity over the bracket (source: dsa/samples-go/ch14/answerspace.go, lines 9-41)

```
// greedy day count at a given capacity
static int daysNeeded(int cap) {
  int days = 1, load = 0;
  for (int w : W) {
    if (w > cap)
      return 1000; // a single weight overflows any day
    if (load + w > cap) {
      days++;
      load = w;
    } else {
      load += w;
    }
  }
  return days;
}

static boolean feasible(int cap) {
  return daysNeeded(cap) ≤ 3;
}

// smallest capacity in [lo, hi] with feasible(cap), lo infeasible-safe
static int bisect(int lo, int hi) {
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (feasible(mid))
      hi = mid;
    else
      lo = mid + 1;
  }
  return lo;
}
```

listing 402: java, the greedy day count, the feasibility test, the bisect, 1000 as the overflow sentinel (source: dsa/samples-java/src/Ch14/Answerspace.java, lines 20-50)

```
    public static int MinCapacity(int[] weights, int days)
    {
        int lo = weights.Max(), hi = weights.Sum();
        while (lo < hi)
        {
            int mid = lo + (hi - lo) / 2;
            if (DaysNeeded(weights, mid) ≤ days)
                hi = mid;
            else
                lo = mid + 1;
        }
        return lo;
    }

    private static int DaysNeeded(int[] weights, int capacity)
    {
        int days = 1, load = 0;
        foreach (int w in weights)
        {
            if (load + w > capacity)
            {
                days++;
                load = 0;
            }
            load += w;
        }
        return days;
    }
```

listing 403: c#, the bracket max-to-total, the bisect on feasibility (source: dsa/samples/src/Ch14/Searching.cs, lines 79-106)

```
  for (const w of weights) {
    if (w > cap) return Infinity; // a single weight overflows any day
    if (load + w > cap) {
      days++;
      load = w;
    } else {
      load += w;
    }
  }
  return days;
}

export function feasible(weights, cap, limit) {
  return daysNeeded(weights, cap) ≤ limit;
}

// smallest capacity in [lo, hi] with feasible(cap)
export function bisect(weights, lo, hi, limit) {
  while (lo < hi) {
    const mid = lo + Math.floor((hi - lo) / 2);
    if (feasible(weights, mid, limit)) hi = mid;
    else lo = mid + 1;
  }
  return lo;
}

// the answer brackets are [max weight, total]
export function minDailyCapacity(weights, days) {
  const maxw = Math.max(...weights);
  const total = weights.reduce((a, b) => a + b, 0);
  return bisect(weights, maxw, total, days);
}
```

listing 404: javascript, infinity marks an overflowing weight, the bisect (source: dsa/samples-js/src/ch14-answerspace.mjs, lines 9-40)

```python
def days_needed(weights, capacity):
    days, load = 1, 0
    for w in weights:
        if load + w > capacity:  # today is full, tomorrow starts
            days += 1
            load = w
        else:
            load += w
    return days


check(days_needed(WEIGHTS, 9) == 2, "capacity 9 packs 1+8 | 2+4+3: two days")
check(days_needed(WEIGHTS, 8) == 4, "capacity 8 splits into 4 days")
check(days_needed(WEIGHTS, 17) == 2, "capacity 17 fits 1+8+2+4 | 3")
check(days_needed(WEIGHTS, 1) == 5, "capacity 1 forces one package a day")


def min_daily_capacity(weights, days):
    lo, hi = max(weights), sum(weights)  # the feasible interval brackets
    while lo < hi:
        mid = (lo + hi) // 2
        if days_needed(weights, mid) ≤ days:  # feasible: try smaller
            hi = mid
        else:
            lo = mid + 1
    return lo
```

listing 405: python, day counts pinned around the answer, the bisect (source: dsa/samples-py/src/Ch14/answerspace.py, lines 16-42)

```lua
local function days_needed(cap)
  local days, load = 1, 0
  for _, w in ipairs(W) do
    if w > cap then return 1000 end -- a single weight overflows any day
    if load + w > cap then
      days = days + 1
      load = w
    else
      load = load + w
    end
  end
  return days
end

local function feasible(cap)
  return days_needed(cap) ≤ 3
end

-- smallest capacity in [lo, hi] with feasible(cap)
local function bisect(lo, hi)
  while lo < hi do
    local mid = lo + (hi - lo) // 2
    if feasible(mid) then hi = mid else lo = mid + 1 end
  end
  return lo
end
```

listing 406: lua, 1000 as the overflow sentinel, the bisect (source: dsa/samples-lua/ch14_answerspace.lua, lines 9-35)

Measured across the suites: this is the chapter's cross-language anchor, weights 1 8 2 4 3 in 3 days, and every language lands 9. The day counts agree around it, 4 days at capacity 8 and 2 at 9, the bracket opens at the heaviest single weight, 8, and closes at the total, 18, and the monotonicity check, feasible once means feasible forever, runs in suite after suite. Python adds the edges: 5 days drops the answer to 8, one day forces the full 18.

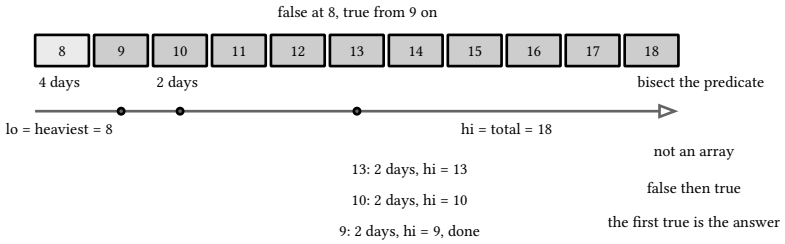


figure 109: the answer space, a monotone predicate over capacities, false then true, bisection finds the transition

14.4. kadane's scan

The maximum subarray question is a search over spans, and the one-pass answer keeps exactly two numbers: the best subarray sum ending at the current index, extended or restarted, and the global best. If the running sum plus the new element is worse than the element alone, the old prefix was dead weight and the span restarts here, otherwise it extends. The champion's start and end ride along, restarted and advanced with the sums, so the winning span falls out of the same pass.

The dry run: the fixture is −2, 1, −3, 4, −1, 2, 1, −5, 4, asserted by every suite against a double-loop brute, the one fixture all seven share.

1. The scan seeds on −2, and the first arrival already restarts: −2 + 1 = −1 loses to the 1 alone, so the span reopens at index 1 with best 1.
2. The −3 drags the ending sum to 1 - 3 = −2 without touching the best.
3. The 4 restarts again, −2 + 4 = 2 under the 4 alone, and the champion opens at index 3 with best 4.
4. Three extends follow, 4 - 1 = 3, 3 + 2 = 5, 5 + 1 = 6, the best steps up twice and the champion holds 3 through 6.
5. The tail only moves the ending sum, 6 - 5 = 1 then 1 + 4 = 5, and the answer lands (best, start, end) at (6, 3, 6), pinned.
6. The edges ride the same loop: −3, −1, −2 keeps −1 at index 1 with the empty-allowed 0, a single 5 is its own span, and 2, 3, 1 spans everything at 6.

i	decision	ending here	best	champion
1	restart, −2 + 1 = −1	1	1	1..1
2	extend, 1 - 3 = −2	−2	1	1..1
3	restart, −2 + 4 = 2	4	4	3..3
4	extend, 4 - 1 = 3	3	4	3..3
5	extend, 3 + 2 = 5	5	5	3..5
6	extend, 5 + 1 = 6	6	6	3..6
7	extend, 6 - 5 = 1	1	6	3..6
8	extend, 1 + 4 = 5	5	6	3..6

The 6 over 3 through 6 with the brute agreeing is the pinned champion, and the listings below run the same scan in seven languages.

```c
static void kadane(const int *a, int len) {
  int cur = a[0], cs = 0;
  best = a[0];
  bs = be = 0;
  for (int i = 1; i < len; i++) {
    if (cur + a[i] < a[i]) { // restart beats extend
      cur = a[i];
      cs = i;
    } else {
      cur += a[i];
    }
    if (cur > best) {
      best = cur;
      bs = cs;
      be = i;
    }
  }
}
```

listing 407: c, the loop, restart beats extend, the champion span tracked (source: dsa/samples-c/src/Ch14/kadane.c, lines 21-38)

```go
func Kadane(a []int) (int, int, int) {
  best, cur := a[0], a[0]
  bs, be, cs := 0, 0, 0
  for i := 1; i < len(a); i++ {
    if cur < 0 { // restart beats extending
      cur = a[i]
      cs = i
    } else {
      cur += a[i]
    }
    if cur > best { // a new champion span
      best = cur
      bs, be = cs, i
    }
  }
  return best, bs, be
}

// KadaneEmptyAllowed is the variant where the empty subarray counts:
// max with 0 of the nonempty best, 0 on empty input.
func KadaneEmptyAllowed(a []int) int {
  if len(a) == 0 {
    return 0
  }
  best, _, _ := Kadane(a)
  if best < 0 {
    return 0
  }
  return best
}
```

listing 408: go, the scan and the empty-allowed wrapper (source: dsa/samples-go/ch14/kadane.go, lines 6-35)

```java
static int best, bs, be;

static void kadane(int[] a) {
  int cur = a[0], cs = 0;
  best = a[0];
  bs = be = 0;
  for (int i = 1; i < a.length; i++) {
    if (cur + a[i] < a[i]) { // restart beats extend
      cur = a[i];
      cs = i;
    } else {
      cur += a[i];
    }
    if (cur > best) {
      best = cur;
      bs = cs;
      be = i;
    }
  }
}
```

listing 409: java, the loop, restart beats extend, the champion span tracked in statics (source: dsa/samples-java/src/Ch14/Kadane.java, lines 20-39)

```csharp
  public static (long Best, int Start, int End) MaxSubarray(long[] a)
  {
      long best = a[0];
      long current = a[0];
      int bestStart = 0;
      int bestEnd = 0;
      int currentStart = 0;
      for (int i = 1; i < a.Length; i++)
      {
          if (current + a[i] < a[i])
          {
              current = a[i]; // restart beats extending
              currentStart = i;
          }
          else
              current += a[i];
          if (current > best)
          {
              best = current;
              bestStart = currentStart;
              bestEnd = i;
          }
      }
      return (best, bestStart, bestEnd);
  }

  /// <summary>The empty-allowed variant: the empty subarray scores 0.</summary>
  public static long MaxSubarrayEmptyAllowed(long[] a) => Math.Max(0, MaxSubarray(a).Best);
```

listing 410: c#, the same loop, the empty-allowed variant one max away (source: dsa/samples/src/Ch14/Searching.cs, lines 116-143)

```
export function kadaneBest(a) {
  let best = a[0];
  let cur = a[0];
  let bs = 0;
  let be = 0;
  let cs = 0;
  for (let i = 1; i < a.length; i++) {
    if (cur + a[i] < a[i]) {
      cur = a[i]; // restart here
      cs = i;
    } else {
      cur += a[i]; // extend the running subarray
    }
    if (cur > best) {
      best = cur;
      bs = cs;
      be = i;
    }
  }
  return { best, start: bs, end: be };
}
```

listing 411: javascript, the whole scan, best, start, end out (source: dsa/samples-js/src/ch14-kadane.mjs, lines 6-26)

```python
def kadane(a):
    # nonempty, with index recovery: returns (best, (start, end))
    best = a[0]
    current = a[0]
    best_start = best_end = current_start = 0
    for i in range(1, len(a)):
        if current + a[i] < a[i]:  # restart beats extending
            current = a[i]
            current_start = i
        else:
            current += a[i]
        if current > best:  # a new champion span
            best = current
            best_start = current_start
            best_end = i
    return best, (best_start, best_end)


def kadane_brute(a):  # the double-loop oracle
    best = None
    span = None
    for i in range(len(a)):
        running = 0
        for j in range(i, len(a)):
            running += a[j]
            if best is None or running > best:
                best = running
                span = (i, j)
    return best, span


def kadane_empty_allowed(a):  # max(0, best): the empty subarray counts
    return max(0, kadane(a)[0]) if a else 0
```

listing 412: python, the scan, the brute oracle, the empty-allowed wrapper (source: dsa/samples-py/src/Ch14/kadane.py, lines 15-47)

```lua
local function kadane(a)
  local best, cur = a[1], a[1]
  local bs, be, cs = 0, 0, 0
  for i = 2, #a do
    if cur + a[i] < a[i] then
      cur = a[i] -- restart: the old prefix only dragged it down
      cs = i - 1
    else
      cur = cur + a[i]
    end
    if cur > best then
      best, bs, be = cur, cs, i - 1
    end
  end
  return best, bs, be
end
```

listing 413: lua, the same loop over 1-based indices (source: dsa/samples-lua/ch14_kadane.lua, lines 8-23)

The nonempty variant is the pinned one, the empty subarray scores nothing, and the empty-allowed variant is the max of the best with 0, stated rather than shipped twice. The classic fixture −2, 1, −3, 4, −1, 2, 1, −5, 4 peaks at 6 over indices 3 through 6, and every suite brute-checks the champion span with the double loop before pinning

it. All-negative input keeps the nonempty best, −1 at index 1 of −3, −1, −2, and the empty-allowed variant reads 0. A single 5 is its own span, and 2, 3, 1 spans everything at 6.

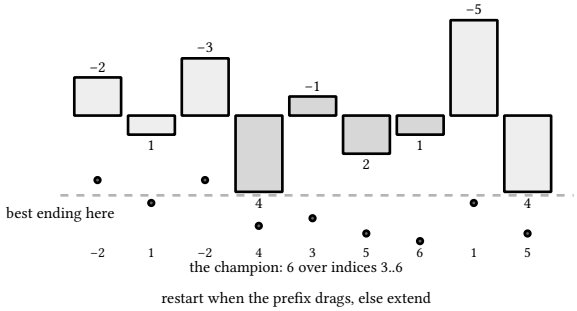


figure 110: the fixture as bars with the winning span shaded and the running best ending here drawn beneath

The application is general technique, the inner loop of every best-interval question.

14.5. mex

The minimal excluded value asks which non-negative integer a list fails to contain, and the answer never exceeds the list length: n values can cover at most n slots, so a seen array of n + 1 booleans catches every candidate that could bind. One marking pass, negatives skipped since they exclude nothing, and one first-false scan close it in O(n) with no allocation beyond the array. The prefix form answers the streaming question, mex of every prefix, with a marker that only ever advances: mark each arrival, walk the marker past the newly filled gap, record.

The dry run: the fixtures are the family the whole chapter pins, 0, 1, 2, 4, 5 with its neighbors, asserted by the C# suite and carried by the integer suites.

1. The anchor 0, 1, 2, 4, 5 allocates six slots, n + 1, and the marking pass fills five of them, every slot but 3.
2. The first-false scan steps over 0, 1, 2, all true, and stops on the hole: mex 3, pinned.
3. The neighbors move the hole: 0, 0, 1, 1 marks two slots and stops at 2, and 2, 0, 1 fills the first three for 3.
4. The negative −7 fails the bounds test and never marks, so −7, 0, 1 reads 2, the same answer as 0, 1 alone.
5. The full house 0 through 5 fills all six slots and the scan runs off the end at 6, the k of 0 through k - 1, pinned at k = 6.
6. The streaming twin over 0, 2, 1, 0, 5 runs 1, 1, 3, 3, 3, the marker stepping to 1, stalling, then jumping to 3 on the third arrival, pinned.

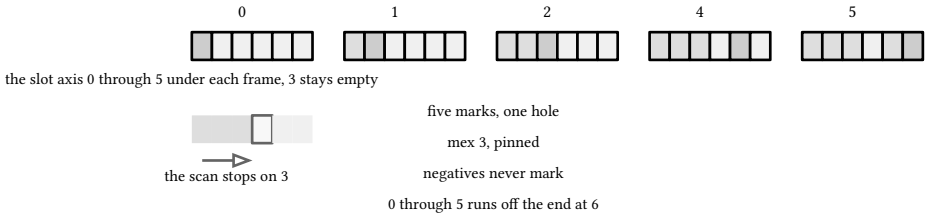


figure 111: the flat walk, five arrivals darkening six slots, the first-false scan stopping on the hole

The 3 beside the off-the-end 6 is the pinned pair, and the listings below run both forms in seven languages.

```c
static int mex_of(const int *a, int len) {
  memset(seen, 0, sizeof(int) * (size_t)(len + 1));
  for (int i = 0; i < len; i++)
    if (a[i] ≥ 0 && a[i] ≤ len)
      seen[a[i]] = 1;
  for (int x = 0; x ≤ len; x++)
    if (!seen[x])
      return x;
  return len + 1; // unreachable: seen has len + 1 slots
}

// the prefix table: mex of a[0..i] for every i
static void prefix_mex(const int *a, int len, int *out) {
  memset(seen, 0, sizeof(seen));
  int cur = 0;
  for (int i = 0; i < len; i++) {
    if (a[i] ≥ 0 && a[i] ≤ len)
      seen[a[i]] = 1;
    while (seen[cur])
      cur++;
    out[i] = cur;
  }
}
```

listing 414: c, the seen array and first-false scan, the prefix table below (source: dsa/samples-c/src/Ch14/mex.c, lines 24-46)

```go
func Mex(a []int) int {
  seen := make([]bool, len(a)+1)
  for _, v := range a {
    if v ≥ 0 && v <= len(a) {
      seen[v] = true
    }
  }
  for slot, flag := range seen {
    if !flag {
      return slot
    }
  }
  return len(a) + 1
}

// PrefixMexTable builds the incremental prefix-mex table as values
// stream in: a candidate marker that only ever advances.
func PrefixMexTable(a []int) []int {
  table := make([]int, 0, len(a))
  present := map[int]bool{}
  candidate := 0
  for _, v := range a {
    if v ≥ 0 {
      present[v] = true
    }
    for present[candidate] {
      candidate++
    }
    table = append(table, candidate)
  }
  return table
}
```

listing 415: go, the scan, the prefix table over a presence map (source: dsa/samples-go/ch14/mex.go, lines 7-38)

```
static int mexOf(int[] a) {
  int len = a.length;
  Arrays.fill(seen, 0, len + 1, 0);
  for (int v : a)
    if (v ≥ 0 && v ≤ len)
      seen[v] = 1;
  for (int x = 0; x ≤ len; x++)
    if (seen[x] == 0)
      return x;
  return len + 1; // unreachable: seen has len + 1 slots
}

// the prefix table: mex of a[0..i] for every i
static int[] prefixMex(int[] a) {
  int len = a.length;
  int[] out = new int[len];
  Arrays.fill(seen, 0);
  int cur = 0;
  for (int i = 0; i < len; i++) {
    if (a[i] ≥ 0 && a[i] ≤ len)
      seen[a[i]] = 1;
    while (seen[cur] ≠ 0)
      cur++;
    out[i] = cur;
  }
  return out;
}
```

listing 416: java, the seen array and first-false scan, the advancing-marker prefix table (source: dsa/samples-java/src/Ch14/Mex.java, lines 24-50)

```
    public static long Of(long[] values)
    {
        var seen = new bool[values.Length + 1];
        foreach (long v in values)
            if (v ≥ 0 && v < seen.Length)
                seen[(int)v] = true;
        int x = 0;
        while (x < seen.Length && seen[x])
            x++;
        return x;
    }

    /// <summary>Prefix mex table: mex of each prefix, no allocation beyond seen.</summary>
    public static long[] PrefixTable(long[] values)
    {
        var seen = new bool[values.Length + 1];
        var table = new long[values.Length];
        int mex = 0;
        for (int i = 0; i < values.Length; i++)
        {
            long v = values[i];
            if (v ≥ 0 && v < seen.Length)
                seen[(int)v] = true;
            while (mex < seen.Length && seen[mex])
                mex++;
            table[i] = mex;
        }
        return table;
    }
```

listing 417: c#, both forms, the marker advancing inside the stream loop (source: dsa/samples/src/Ch14/Searching.cs, lines 153-181)

```
export function mex(a) {
  const seen = new Array(a.length + 1).fill(false);
  for (const x of a) {
    if (x ≥ 0 && x ≤ a.length) seen[x] = true;
  }
  let m = 0;
  while (m ≤ a.length && seen[m]) m += 1;
  return m;
}

export function prefixMexTable(a) {
  const seen = new Set();
  const table = [];
  for (const x of a) {
    seen.add(x);
    let y = 0;
    while (seen.has(y)) y += 1;
    table.push(y);
  }
  return table;
}
```

listing 418: javascript, both forms, the set-backed prefix table (source: dsa/samples-js/src/ch14-mex.mjs, lines 7-27)

```python
def mex(a):
    seen = [False] * (len(a) + 1)  # allocation stays at the seen array
    for v in a:
        if 0 ≤ v ≤ len(a):  # negatives skip, big values cannot bind
            seen[v] = True
    for slot, flag in enumerate(seen):  # the first-false scan
        if not flag:
            return slot
    return len(a) + 1


def prefix_mex_table(a):
    table = []
    present = set()
    candidate = 0
    for v in a:  # the moving marker never walks back
        if v ≥ 0:
            present.add(v)
        while candidate in present:
            candidate += 1
        table.append(candidate)
    return table
```

listing 419: python, the seen scan, the prefix table with its moving marker (source: dsa/samples-py/src/Ch14/mex.py, lines 15-36)

```lua
local function mex(a)
  local n = #a
  -- slot v + 1 holds value v; mex of n values is at most n
  local seen = {}
  for i = 1, n + 1 do seen[i] = false end
  for _, v in ipairs(a) do
    if v ≥ 0 and v ≤ n then seen[v + 1] = true end
  end
  local m = 0
  while seen[m + 1] do m = m + 1 end
  return m
end

-- out[i] = mex of a[1..i], the running first-false scan
local function prefix_mex(a)
  local seen, out = {}, {}
  for i, v in ipairs(a) do
    if v ≥ 0 then seen[v + 1] = true end
    local m = 0
    while seen[m + 1] do m = m + 1 end
    out[i] = m
  end
  return out
end
```

listing 420: lua, slot v + 1 holds value v, both forms (source: dsa/samples-lua/ch14_mex.lua, lines 8-31)

The pinned family reads: mex of the empty list is 0, of 0 is 1, of 1, 2, 3 is 0, of 0, 1, 2, 4, 5 is 3, of 0, 0, 1, 1 is 2, of −7, 0, 1 is 2 with the negative skipped, and of 2, 0, 1 is 3. The prefix table of 0, 2, 1, 0, 5 runs 1, 1, 3, 3, 3, the marker stepping twice and stalling. The edges: mex of 0 through k minus 1 is k itself, pinned at k = 6, an empty stream has no prefixes, and negatives never move the marker.

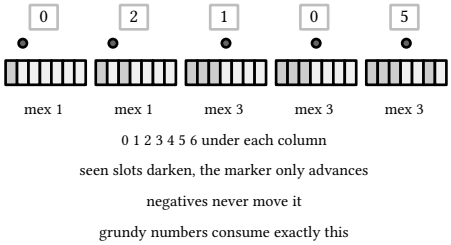


figure 112: the value axis 0 through 6 under the prefix fixture, seen slots darkening and the mex marker stepping

The application is general technique, and the grundy numbers of book 9, ch 40 (practical data structures and algorithms, game theory and scheduling) are the named kin: a game position's value is the mex of its options, computed by this scan.

14.6. across the seven languages

The build sizes count non-comment source lines over this chapter's five featured files per language, the three search topics plus the two scan extensions. The C# suite packs all five topics into one file:

language	build SLOC	dependency	boundary note
c	319	libc only	overflow sentinels 1000 for days, the kadane brute rides along, checks share the file with main, 70 of them
go	143	none	insertion point as a named alias, degenerate rotations swept, the empty-allowed kadane wrapper, tests in separate files, 14 tests
java	338	jdk 27 stdlib	one file per topic, the bounds partition-swept linearly, 1000 as the overflow sentinel, the kadane brute a record pair, 63 checks in 5 files
c#	153	bcl only	leftmost and rightmost ride the bounds, kadane and mex as static classes, 10 tests
javascript	113	node stdlib	Infinity marks an overflowing weight, floor midpoints, the two scans as small modules, 14 tests
python	224	stdlib only	chained comparisons in the rotated walk, the double-loop kadane oracle, 50 checks
lua	284	lib.lua harness	1-based walks shifted back to 0-based answers, slot v + 1 holds value v in mex, checks ride in the module, 18 of them

sources: learn.microsoft.com, `Array.BinarySearch` documentation as the boundary vocabulary reference, accessed 2026-09-08, and cp-algorithms, "Search the subsegment with the maximum/minimum sum", cp-algorithms.com/others/maximum_average_segment.html, and "MEX task (Minimal Excluded element in an array)", cp-algorithms.com/sequences/mex.html, both accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Sample behavior verified by `make verify-csharp`, 10 tests in chapter 14 of the samples suite. The seven-language layer verifies the same way: 5 C programs with 70 embedded checks under `make verify-c`, 14 Go tests, the java runner's 63 Ch14 checks over 5 files under `run-java-samples`, 14 `node --test` cases, 50 Python checks across 5 files, and 18 Lua checks under `run.lua`.

15. string matching

Finding a pattern in text has a naive quadratic answer and a family of linear and near-linear machines that each precompute something about the pattern: borders, skip tables, rolling hashes, or a whole automaton. This chapter builds them in order, seven languages deep, ending with the multi-pattern machine that scans once for every pattern at once.

15.1. the naive baseline

The quadratic matcher every clever machine is measured against. Every alignment is tried, every character compared from the front, and the cost is exactly countable: a window that matches costs the pattern length, a window that fails on the first character costs one. On abab in abababab that is three full matches at 4 comparisons plus two immediate mismatches, 14 total, and the overlapping hits at 0, 2, and 4 all surface because nothing is ever skipped.

The dry run: the C# fixture is abc over abcabcabc with the meter pinned at 13, and C, JavaScript, and Lua pin the same rule on the overlapping anchor, abab in abababab at 14.

1. Seven windows open over abcabcabc.
2. Windows 0, 3, and 6 read abc exactly, three comparisons each, 3 + 3 + 3 = 9, hits at 0, 3, and 6.
3. Windows 1, 2, 4, and 5 fail on the first character, b or c against a, one comparison each, 4 in total.
4. The meter closes at 9 + 4 = 13, pinned.
5. The near-miss twin abc over abababab pays 3 + 1 + 3 + 1 + 3 + 1 = 12, two characters agreeing wherever the text alternates and the third never, also pinned.
6. The overlap anchor abab in abababab collects 0, 2, and 4 with nothing skipped, and the ground-truth tests replay `IndexOf` and `str.find` in the C# and Python suites.

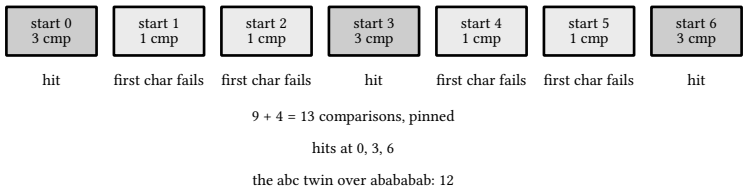


figure 113: the run as one card per window, the three hits shaded, the cost written on each

The 13 with its 12 twin is the pinned meter pair, and the listings below count the same walk in seven languages.

```c
static int naive(const char *text, const char *pat, int *hits) {
  size_t tn = strlen(text), pn = strlen(pat);
  int k = 0;
  comparisons = 0;
  for (size_t i = 0; i + pn ≤ tn; i++) {
    size_t j = 0;
    while (j < pn) {
      comparisons++;
      if (text[i + j] ≠ pat[j])
        break;
      j++;
    }
    if (j == pn)
      hits[k++] = (int)i;
  }
  return k;
}
```

listing 421: c, every alignment tried, comparisons counted globally (source: dsa/samples-c/src/Ch15/naive.c, lines 20-36)

```go
// NaiveFindAll reports every index where pat starts inside text,
// overlapping matches included. It is the ground truth the other
// matchers answer to.
func NaiveFindAll(text, pat string) []int {
  var hits []int
  if len(pat) == 0 || len(pat) > len(text) {
    return hits
  }
  for i := 0; i+len(pat) <= len(text); i++ {
    if text[i:i+len(pat)] == pat {
      hits = append(hits, i)
    }
  }
  return hits
}
```

listing 422: go, whole-window substring compare, the ground truth list (source: dsa/samples-go/ch15/naive.go, lines 5-19)

```java
  // every start index where the pattern matches, overlaps included
  static List<Integer> naiveFindAll(String text, String pat) {
    List<Integer> hits = new ArrayList<>();
    comparisons = 0;
    int n = text.length(), m = pat.length();
    for (int i = 0; i + m ≤ n; i++) {
      int j = 0;
      while (j < m) {
        comparisons++;
        if (text.charAt(i + j) ≠ pat.charAt(j))
          break;
        j++;
      }
      if (j == m)
        hits.add(i);
    }
    return hits;
  }

  // java's own scanner as ground truth, the str.find of this language.
  // indexOf clamps an empty needle's fromIndex to length instead of
  // ending the scan, so this referee is only used for non-empty patterns
  static List<Integer> indexOfAll(String text, String pat) {
    List<Integer> out = new ArrayList<>();
    int at = text.indexOf(pat);
    while (at ≥ 0) {
      out.add(at);
      at = text.indexOf(pat, at + 1);
    }
    return out;
  }
```

listing 423: java, charAt walk with the meter running, indexOf the referee for non-empty needles (source: dsa/samples-java/src/Ch15/Naive.java, lines 21-51)

```
    /// <summary>All start offsets where the pattern matches, overlaps included.</summary>
    public static List<int> Find(string text, string pattern)
    {
        var hits = new List<int>();
        if (pattern.Length == 0) // same convention as Matching.Kmp
            return hits;
        for (int i = 0; i + pattern.Length ≤ text.Length; i++)
        {
            int j = 0;
            while (j < pattern.Length && text[i + j] == pattern[j])
                j++;
            if (j == pattern.Length)
                hits.Add(i);
        }
        return hits;
    }

    /// <summary>Comparisons spent by the naive scan, char-by-char.</summary>
    public static long Comparisons(string text, string pattern)
    {
        long count = 0;
        for (int i = 0; i + pattern.Length ≤ text.Length; i++)
        {
            int j = 0;
            while (j < pattern.Length && text[i + j] == pattern[j])
            {
                count++;
                j++;
            }
            if (j < pattern.Length)
                count++; // the mismatching character also cost one comparison
        }
        return count;
```

listing 424: c#, the hit list, then the same walk with the comparison count exposed (source: dsa/samples/src/Ch15/Naive.cs, lines 6-38)

```javascript
// every start index where the pattern matches, overlaps included
export function naiveFindAll(text, pat) {
  const hits = [];
  if (pat.length === 0 || pat.length > text.length) return hits;
  for (let i = 0; i + pat.length ≤ text.length; i++) {
    let j = 0;
    while (j < pat.length && text[i + j] === pat[j]) j++;
    if (j === pat.length) hits.push(i);
  }
  return hits;
}

// the same walk with the comparison count exposed
export function naiveStats(text, pat) {
  const hits = [];
  let comparisons = 0;
  for (let i = 0; i + pat.length ≤ text.length; i++) {
    let j = 0;
    while (j < pat.length) {
      comparisons++;
      if (text[i + j] =/= pat[j]) break;
      j++;
    }
    if (j === pat.length) hits.push(i);
  }
  return { hits, comparisons };
}
```

listing 425: javascript, the plain walk and the counted walk side by side (source: dsa/samples-js/src/ch15-naive.mjs, lines 4-30)

```python
def naive_find_all(text, pattern):
    hits = []
    for start in range(len(text) - len(pattern) + 1):
        i = 0
        while i < len(pattern) and text[start + i] == pattern[i]:
            i += 1
        if i == len(pattern):
            hits.append(start)
    return hits
```

listing 426: python, the two-loop scan, empty pattern matches everywhere (source: dsa/samples-py/src/Ch15/naive.py, lines 13-21)

```lua
-- every start index where the pattern matches, overlaps included
local function naive(text, pat)
  local hits = {}
  comparisons = 0
  for i = 0, #text - #pat do
    local j = 0
    while j < #pat do
      comparisons = comparisons + 1
      if text:byte(i + j + 1) ~= pat:byte(j + 1) then break end
      j = j + 1
    end
    if j == #pat then hits[#hits + 1] = i end
  end
  return hits
end
```

listing 427: lua, byte compares over 0-based windows of a 1-based string (source: dsa/samples-lua/ch15_naive.lua, lines 7-21)

Measured across the suites: every language pins the overlapping anchor, `abab` in `abababab` at 0, 2, and 4. The comparison meter splits three ways. C, Java, JavaScript, and Lua pin the 14 for that fixture, C# pins the same counting rule on its own fixtures, 13 for `abc` in `abcabcabc` and 12 for `abc` in `abababab`, and Go compares whole substrings so it pins positions only. Python cross-checks every hit list against `str.find` advanced one past each hit, the same ground-truth move the C# suite makes with `IndexOf`, and Java does it with `indexOf` over non-empty needles only, because `indexOf` clamps an empty needle's start to the text length instead of ending the scan.

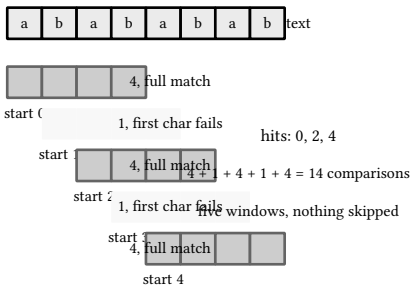


figure 114: the naive scan over abababab for abab, matching windows shaded, the comparison ledger under each

15.2. kmp

The naive matcher restarts from scratch after every mismatch. KMP precomputes, for every prefix of the pattern, the longest proper border, prefix that is also suffix, and uses it to never move the text pointer backwards.

The dry run: the fixtures are the pinned tables of `ababa`, `aaaab`, and `abc` from the C# suite, and the scan `abab` over `abababab` that every suite carries.

1. The table of `ababa` builds in one pass: b against a misses for 0, then a, b, a each extend the border one more, 1, 2, 3, the last aba equal to the first.
2. `aaaab` climbs 1, 2, 3 and falls to 0 at the b through the long fallback chain, and `abc` never leaves 0, both pinned.
3. The scan climbs the matched length to 4 by index 3, reports hit 0, then sets it to the border of `abab`, the 2 the pinned table carries one slot before its own 3, not 0.
4. Two characters lift it back to 4 at index 5, hit 2, it falls to 2 again, and the same pair lifts it to hit 4 at index 7.
5. The text pointer read 8 cells, one comparison each, and never moved backward: the hits read 0, 2, 4, pinned.

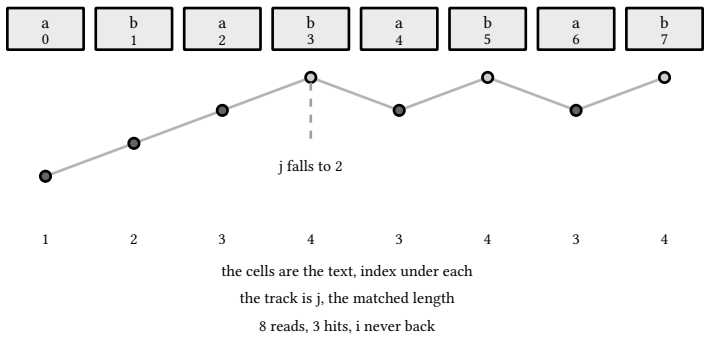


figure 115: the matched length under each text cell, hitting 4 three times, falling to the border 2 after each hit

The 0, 2, 4 off eight forward reads is the pinned scan, and the listings below build the table in seven languages.

```c
// fail[i] = length of the longest proper prefix of pat[0..i] that is also
// a suffix of it
static void failure_table(const char *pat, int *fail) {
  size_t pn = strlen(pat);
  fail[0] = 0;
  size_t k = 0;
  for (size_t i = 1; i < pn; i++) {
    while (k && pat[i] != pat[k])
      k = (size_t)fail[k - 1];
    if (pat[i] == pat[k])
      k++;
    fail[i] = (int)k;
  }
}

static int kmp(const char *text, const char *pat, int *hits) {
  int fail[64];
  failure_table(pat, fail);
  size_t tn = strlen(text);
  int k = 0, matched = 0;
  for (size_t i = 0; i < tn; i++) {
    while (matched && text[i] != pat[matched])
      matched = fail[matched - 1];
    if (text[i] == pat[matched])
      matched++;
    if (matched == (int)strlen(pat)) {
      hits[k++] = (int)(i + 1 - strlen(pat));
      matched = fail[matched - 1];
    }
  }
  return k;
}
```

listing 428: c, the failure table, then the single-pass scan (source: dsa/samples-c/src/Ch15/kmp.c, lines 18-49)

```go
// FailureTable computes fail[i]: the length of the longest proper
// prefix of pat[:i+1] that is also its suffix. It is the plumbing KMP
// uses to never back up inside the text.
func FailureTable(pat string) []int {
  fail := make([]int, len(pat))
  k := 0
  for i := 1; i < len(pat); i++ {
    for k > 0 && pat[i] ≠ pat[k] {
      k = fail[k-1]
    }
    if pat[i] == pat[k] {
      k++
    }
    fail[i] = k
  }
  return fail
}

// KMPFindAll scans text once. On a mismatch the pattern slides by the
// failure table instead of the text cursor moving back.
func KMPFindAll(text, pat string) []int {
  var hits []int
  if len(pat) == 0 || len(pat) > len(text) {
    return hits
  }
  fail := FailureTable(pat)
  k := 0 // chars of pat matched so far
  for i := 0; i < len(text); i++ {
    for k > 0 && text[i] ≠ pat[k] {
      k = fail[k-1]
    }
    if text[i] == pat[k] {
      k++
    }
    if k == len(pat) {
      hits = append(hits, i-k+1)
      k = fail[k-1] // keep matching: overlaps survive
    }
  }
  return hits
}
```

listing 429: go, the table builder and the scan that never backs up (source: dsa/samples-go/ch15/kmp.go, lines 3-43)

```java
// fail[i] = length of the longest proper prefix of pat[0..i] that is
// also a suffix of it
static int[] failureTable(String pat) {
  int m = pat.length();
  int[] fail = new int[m];
  int k = 0;
  for (int i = 1; i < m; i++) {
    while (k > 0 && pat.charAt(i) != pat.charAt(k))
      k = fail[k - 1];
    if (pat.charAt(i) == pat.charAt(k))
      k++;
    fail[i] = k;
  }
  return fail;
}

static List<Integer> kmpFindAll(String text, String pat) {
  if (pat.isEmpty())
    return idxRange(text.length() + 1);
  int[] fail = failureTable(pat);
  List<Integer> hits = new ArrayList<>();
  int k = 0;
  for (int i = 0; i < text.length(); i++) {
    while (k > 0 && text.charAt(i) != pat.charAt(k))
      k = fail[k - 1];
    if (text.charAt(i) == pat.charAt(k))
      k++;
    if (k == pat.length()) {
      hits.add(i + 1 - pat.length());
      k = fail[k - 1]; // the border lets the window slide on
    }
  }
  return hits;
}
```

listing 430: java, the border table, the scan, the hit slides k to the border (source: dsa/samples-java/src/Ch15/Kmp.java, lines 20-53)

```
public static class Matching
{
    /// <summary>KMP failure function: longest proper prefix that is also a suffix.</summary>
    public static int[] KmpTable(string pattern)
    {
        var table = new int[pattern.Length];
        int len = 0;
        for (int i = 1; i < pattern.Length;)
        {
            if (pattern[i] == pattern[len])
                table[i++] = ++len;
            else if (len > 0)
                len = table[len - 1]; // fall back, do not retry from zero
            else
                table[i++] = 0;
        }
        return table;
    }

    public static List<int> Kmp(string text, string pattern)
    {
        var hits = new List<int>();
        if (pattern.Length == 0)
            return hits;
        int[] table = KmpTable(pattern);
        int j = 0;
        for (int i = 0; i < text.Length; i++)
        {
            while (j > 0 && text[i] != pattern[j])
                j = table[j - 1];
            if (text[i] == pattern[j])
                j++;
            if (j == pattern.Length)
            {
                hits.Add(i - j + 1);
                j = table[j - 1]; // keep going, overlapping matches included
            }
        }
        return hits;
    }
```

listing 431: c#, the failure table, then the linear scan (source: dsa/samples/src/Ch15/Strings.cs, lines 4-44)

```javascript
// fail[i] = length of the longest proper prefix of pat[0..i] that is
// also a suffix of it
export function failureTable(pat) {
  const fail = new Array(pat.length).fill(0);
  let k = 0;
  for (let i = 1; i < pat.length; i++) {
    while (k && pat[i] !== pat[k]) k = fail[k - 1];
    if (pat[i] === pat[k]) k++;
    fail[i] = k;
  }
  return fail;
}

export function kmpFindAll(text, pat) {
  const fail = failureTable(pat);
  const hits = [];
  let matched = 0;
  for (let i = 0; i < text.length; i++) {
    while (matched && text[i] !== pat[matched]) matched = fail[matched - 1];
    if (text[i] === pat[matched]) matched++;
    if (matched === pat.length) {
      hits.push(i + 1 - pat.length);
      matched = fail[matched - 1];
    }
  }
  return hits;
}
```

listing 432: javascript, the border fallback chain and the scan (source: dsa/samples-js/src/ch15-kmp.mjs, lines 5-31)

```python
def failure_table(pattern):
    fail = [0] * len(pattern)
    k = 0  # length of the current border
    for i in range(1, len(pattern)):
        while k and pattern[i] != pattern[k]:
            k = fail[k - 1]  # fall back to the shorter border
        if pattern[i] == pattern[k]:
            k += 1
        fail[i] = k
    return fail


def kmp_find_all(text, pattern):
    if not pattern:
        return list(range(len(text) + 1))
    fail = failure_table(pattern)
    hits, k = [], 0
    for i, ch in enumerate(text):
        while k and ch != pattern[k]:
            k = fail[k - 1]
        if ch == pattern[k]:
            k += 1
        if k == len(pattern):
            hits.append(i - k + 1)
            k = fail[k - 1]  # the border lets the window slide on
    return hits
```

listing 433: python, the table, then the scan with the border sliding on (source: dsa/samples-py/src/Ch15/kmp.py, lines 14-39)

```lua
-- fail[i] = length of the longest proper prefix of pat[1..i] that is also
-- a suffix of it (1-based, fail[1] = 0)
local function failure_table(pat)
  local fail = {0}
  local k = 0
  for i = 2, #pat do
    while k > 0 and pat:byte(i) ~= pat:byte(k + 1) do
      k = fail[k]
    end
    if pat:byte(i) == pat:byte(k + 1) then k = k + 1 end
    fail[i] = k
  end
  return fail
end

local function kmp(text, pat)
  local fail = failure_table(pat)
  local hits = {}
  local matched = 0
  for i = 1, #text do
    while matched > 0 and text:byte(i) ~= pat:byte(matched + 1) do
      matched = fail[matched]
    end
    if text:byte(i) == pat:byte(matched + 1) then matched = matched + 1 end
    if matched == #pat then
      hits[#hits + 1] = i - #pat -- 0-based start
      matched = fail[matched]
    end
  end
  return hits
end
```

listing 434: lua, byte-indexed borders on a 1-based pattern (source: dsa/samples-lua/ch15_kmp.lua, lines 6-36)

The table test uses ababa, borders 0 0 1 2 3, and aaaab, the all-same prefix that makes the fallback chain long. The matching test overlaps: abab in abababab hits at 0, 2, and 4, and the ground-truth test cross-checks against `IndexOf` scanning.

Every suite pins the same two anchors: the ababa table 0 0 1 2 3 and the overlapping hits 0, 2, 4. The ground-truth partner differs by ecosystem, C# and Python replay `IndexOf` and `str.find`, C, Java, and Lua carry a private naive counter, and Go and JavaScript hold the scan against table-driven window lists. After a hit every version sets the matched length to its own border rather than zero, which is what keeps the overlapping occurrences arriving.

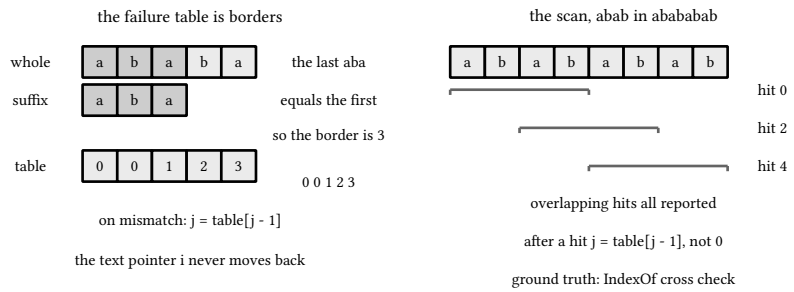


figure 116: kmp, the failure table is borders and the text pointer never moves backward

15.3. horspool

Horspool is Boyer-Moore reduced to one table: align at the end, compare backwards, and on mismatch skip by the table entry of the window's last character. Rare tail characters skip whole pattern lengths.

The dry run: the C# anchor returns first indexes only, 10 for `ababd` in its text, so the walk follows the C suite's skip fixture, `abc` over `abdbcabc`, every shift and comparison pinned, with the Python and Lua twins carrying the same numbers.

1. The table charges `a` 2 and `b` 1, absent characters the whole 3, the same rule as the pinned `abab` twin, `a` 1, `b` 2, else 4.
2. Window one reads `abd` against `abc`, the tail d fails c at one comparison, and d is absent: skip 3, the pinned first shift.
3. Window two reads `bca`, the tail a fails c, one comparison again, and `a` charges 2, the pinned second shift.
4. Window three reads `abc` backwards, c, b, a, three comparisons, the hit at 5 against the text edge.
5. The meter closes at 1 + 1 + 3 = 5, one hit, both pinned.
6. The overlap twin `abab` over `abababab` matches all three alignments and shifts by the b rule twice, both 2, reporting 0, 2, 4 at 12 comparisons, also pinned.

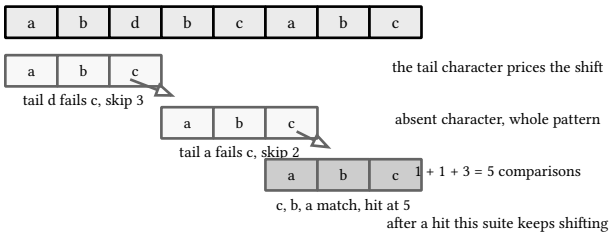


figure 117: the run as three window frames over the text, two tail mismatches, the skips 3 and 2 landing the hit

The 5 comparisons with shifts 3 then 2 is the pinned ledger, and the listings below carry the after-hit split in seven languages.

```c
static void skip_table(const char *pat, int *skip) {
  size_t pn = strlen(pat);
  for (int c = 0; c < 256; c++)
    skip[c] = (int)pn;
  for (size_t i = 0; i + 1 < pn; i++) // the last char never shifts
    skip[(unsigned char)pat[i]] = (int)(pn - 1 - i);
}

static int horspool(const char *text, const char *pat, int *hits) {
  int skip[256];
  skip_table(pat, skip);
  size_t tn = strlen(text), pn = strlen(pat);
  int k = 0;
  comparisons = 0;
  nshifts = 0;
  size_t pos = 0;
  while (pos + pn ≤ tn) {
    // compare from the tail
    size_t j = pn;
    while (j > 0) {
      comparisons++;
      if (text[pos + j - 1] ≠ pat[j - 1])
        break;
      j--;
    }
    if (j == 0) {
      hits[k++] = (int)pos;
      j = 1; // mismatch-style exit so the shift uses the last char
    }
    unsigned char tail = (unsigned char)text[pos + pn - 1];
    if (pos + pn < tn) {
      shifts[nshifts++] = skip[tail];
      pos += (size_t)skip[tail];
    } else {
      break;
    }
  }
  return k;
}
```

listing 435: c, the skip table, tail-aligned compare, shifts recorded (source: dsa/samples-c/src/Ch15/horspool.c, lines 20-58)

```go
func BadCharTable(pat string) []int {
  const alphabet = 256
  shift := make([]int, alphabet)
  for i := range shift {
    shift[i] = len(pat)
  }
  for i := 0; i < len(pat)-1; i++ {
    shift[pat[i]] = len(pat) - 1 - i // rightmost occurrence wins
  }
  return shift
}

// HorspoolStats reports every hit position, how many window
// alignments the scan examined, and how many character comparisons it
// spent. Each alignment compares from the pattern tail leftward.
func HorspoolStats(text, pat string) (hits []int, alignments, compares int) {
  if len(pat) == 0 || len(pat) > len(text) {
    return nil, 0, 0
  }
  shift := BadCharTable(pat)
  at := 0
  for at+len(pat) <= len(text) {
    alignments++
    i := len(pat) - 1
    for ; i ≥ 0; i-- {
      compares++
      if text[at+i] ≠ pat[i] {
        break
      }
    }
    if i < 0 {
      hits = append(hits, at) // whole pattern matched
      at += len(pat)
      continue
    }
    at += shift[text[at+len(pat)-1]]
  }
  return hits, alignments, compares
}
```

listing 436: go, bad character table, then the stats-bearing scan (source: dsa/samples-go/ch15/horspool.go, lines 8-46)

```java
static void skipTable(String pat, int[] skip) {
  int m = pat.length();
  Arrays.fill(skip, m);
  for (int i = 0; i + 1 < m; i++) // the last char never shifts
    skip[pat.charAt(i)] = m - 1 - i;
}

static List<Integer> horspool(String text, String pat) {
  int[] skip = new int[256];
  skipTable(pat, skip);
  int n = text.length(), m = pat.length();
  List<Integer> hits = new ArrayList<>();
  comparisons = 0;
  nshifts = 0;
  int pos = 0;
  while (pos + m ≤ n) {
    // compare from the tail
    int j = m;
    while (j > 0) {
      comparisons++;
      if (text.charAt(pos + j - 1) ≠ pat.charAt(j - 1))
        break;
      j--;
    }
    if (j == 0) {
      hits.add(pos);
      j = 1; // mismatch-style exit so the shift uses the last char
    }
    char tail = text.charAt(pos + m - 1);
    if (pos + m < n) {
      shifts[nshifts++] = skip[tail];
      pos += skip[tail];
    } else {
      break;
    }
  }
  return hits;
}
```

listing 437: java, the skip table, tail compare, every shift logged, hits keep shifting (source: dsa/samples-java/src/Ch15/Horspool.java, lines 24-61)

```
    /// <summary>Boyer-Moore-Horspool: skip by the bad character table, one table, one pass.</summary>
    public static int Horspool(string text, string pattern)
    {
        if (pattern.Length == 0 || pattern.Length > text.Length)
            return -1;
        var skip = new int[256];
        Array.Fill(skip, pattern.Length);
        for (int i = 0; i < pattern.Length - 1; i++)
            skip[pattern[i]] = pattern.Length - 1 - i;

        int pos = 0;
        while (pos + pattern.Length ≤ text.Length)
        {
            int i = pattern.Length - 1;
            while (i ≥ 0 && text[pos + i] == pattern[i])
                i--;
            if (i < 0)
                return pos;
            pos += skip[text[pos + pattern.Length - 1]];
        }
        return -1;
    }
```

listing 438: c#, one table, one pass, first index back (source: dsa/samples/src/Ch15/Strings.cs, lines 45-66)

```
// shift for each character: rightmost occurrence in pat[0..m-2]
export function badCharTable(pat) {
  const shift = new Map();
  for (let i = 0; i + 1 < pat.length; i++)
    shift.set(pat[i], pat.length - 1 - i);
  return shift;
}

function shiftFor(shift, pat, ch) {
  const s = shift.get(ch);
  return s === undefined ? pat.length : s;
}

// { hits, alignments, compares }: hits at non-overlapping occurrences
export function horspoolStats(text, pat) {
  const shift = badCharTable(pat);
  const hits = [];
  let alignments = 0;
  let compares = 0;
  let pos = 0;
  while (pos + pat.length ≤ text.length) {
    alignments++;
    let j = pat.length;
    while (j > 0) {
      compares++;
      if (text[pos + j - 1] =/= pat[j - 1]) break;
      j--;
    }
    if (j === 0) {
      hits.push(pos);
      pos += pat.length; // a match clears the whole pattern
    } else {
      pos += shiftFor(shift, pat, text[pos + pat.length - 1]);
    }
  }
  return { hits, alignments, compares };
}
```

listing 439: javascript, map-backed table, a match clears the whole pattern (source: dsa/samples-js/src/ch15-horspool.mjs, lines 6-42)

```
def skip_table(pattern):
    # distance from the rightmost occurrence in pattern[:-1] to the tail
    m = len(pattern)
    table = {}
    for i, ch in enumerate(pattern[:-1]):
        table[ch] = m - 1 - i
    return table


def horspool(text, pattern):
    # returns (hits, alignments, comparisons)
    m = len(pattern)
    table = skip_table(pattern)
    align = 0
    hits, alignments, comparisons = [], 0, 0
    while align + m ≤ len(text):
        alignments += 1
        i = m - 1
        while i ≥ 0 and text[align + i] == pattern[i]:  # tail first
            comparisons += 1
            i -= 1
        if i < 0:
            hits.append(align)
        if i ≥ 0:
            comparisons += 1  # the mismatching pair also costs a compare
        tail = text[align + m - 1]
        align += table.get(tail, m)
    return hits, alignments, comparisons
```

listing 440: python, dict table, the mismatching pair also costs a compare (source: dsa/samples-py/src/Ch15/horspool.py, lines 14-41)

```lua
local function skip_table(pat)
  local skip = {}
  for c = 0, 255 do skip[string.char(c)] = #pat end
  for i = 1, #pat - 1 do
    skip[pat:sub(i, i)] = #pat - i
  end
  return skip
end

local function horspool(text, pat)
  local skip = skip_table(pat)
  local hits = {}
  comparisons = 0
  shifts = {}
  local pos = 0 -- 0-based
  while pos + #pat ≤ #text do
    -- compare from the tail
    local j = #pat
    while j > 0 do
      comparisons = comparisons + 1
      if text:byte(pos + j) ~= pat:byte(j) then break end
      j = j - 1
    end
    if j == 0 then
      hits[#hits + 1] = pos
      j = 1 -- mismatch-style exit so the shift uses the last char
    end
    local tail = text:sub(pos + #pat, pos + #pat)
    if pos + #pat < #text then
      shifts[#shifts + 1] = skip[tail]
      pos = pos + skip[tail]
    else
      break
    end
  end
  return hits
end
```

listing 441: lua, table of 256 chars, tail compare leftward (source: dsa/samples-lua/ch15_horspool.lua, lines 9-45)

The suites disagree on purpose about what happens after a hit, and the disagreement is worth reading twice. C, Java, Python, and Lua keep shifting by the tail character's table entry even after a full match, so `abab` in `abababab` reports the overlapping 0, 2, 4 at 12 comparisons, Java through a mismatch-style exit that reuses the tail rule after every hit. Go and JavaScript advance the whole pattern after a hit, the classic textbook advance, so the same fixture yields 0 and 4 and the skip counts pin instead. C# returns the first hit index only. On the miss side all seven agree: an absent tail character jumps the full pattern length, and the pinned skip sequences, like 3 then 2 for `abc` in `abdbcabc`, are the whole point of the table.

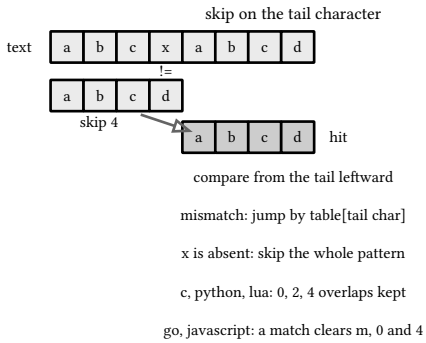


figure 118: horspool skips by the tail character, x is absent so the whole pattern length goes at once

15.4. rabin-karp

Rabin-Karp hashes the pattern and a sliding window with a polynomial rolling hash, subtract the leaving character times the high power, add the entering one, and verifies with a real comparison on hash hits. On its own it is average-linear with hash collision risk, and it is the multi-pattern and two-dimensional workhorse for the same rolling reason.

The dry run: the C# suite pins index agreement with `IndexOf` on its own text, so the walk follows the C and Lua hand-computed anchor, `abab` under the 1e9+7 modulus, every number below pinned.

1. The pattern hash is a polynomial in base 256: ((97 x 256 + 98) x 256 + 97) x 256 + 98 = 1633837410, and one reduction, 1633837410 - 1000000007 = 633837403, pins the pair.
2. Window one `abab` hashes equal, the verification compares four characters, and hit 0 lands.
3. Each slide subtracts the leaving a times 256^3, shifts, adds the entering character, and the suite pins every rolled window equal to a from-scratch hash of the same letters.
4. Only the `abab` windows hash equal across `abababab`: three verifications, hits 0, 2, 4, zero spurious, all pinned.
5. The `aaaa` twin hashes equal at every alignment and still reports 0, 1, 2 with nothing spurious, and the absent `xyz` verifies nothing, both pinned.

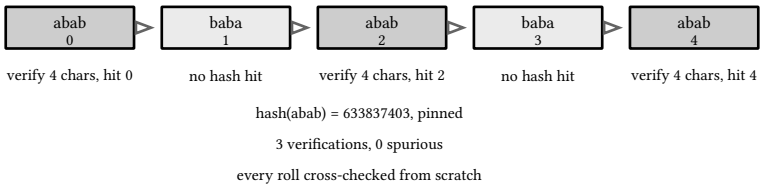


figure 119: the pass over abababab as five windows, the three hash-equal ones shaded and verified, the two others skipped untouched

The 633837403 with three verifications and none spurious is the pinned ledger, and the listings below roll the same window in seven languages.

```c
static int rabinkarp(const char *text, const char *pat, int *hits) {
  size_t tn = strlen(text), pn = strlen(pat);
  if (pn == 0 || pn > tn)
    return 0;
  unsigned long long hp = 0, ht = 0, pw = 1;
  for (size_t i = 0; i < pn; i++) {
    hp = (mulmod(hp, B) + (unsigned char)pat[i]) % M;
    ht = (mulmod(ht, B) + (unsigned char)text[i]) % M;
    if (i)
      pw = mulmod(pw, B);
  }
  hits_found = 0;
  verifications = 0;
  spurious = 0;
  int k = 0;
  for (size_t i = 0;; i++) {
    if (ht == hp) {
      verifications++;
      size_t j = 0;
      while (j < pn && text[i + j] == pat[j])
        j++;
      if (j == pn) {
        hits[k++] = (int)i;
        hits_found++;
      } else {
        spurious++;
      }
    }
    if (i + pn == tn)
      break;
    // roll: drop text[i], shift, add text[i+pn]
    ht = (ht + M - mulmod((unsigned char)text[i], pw)) % M;
    ht = (mulmod(ht, B) + (unsigned char)text[i + pn]) % M;
  }
  return k;
}
```

listing 442: c, modulus 1e9+7, products stay under 2^64, verify on hit (source: dsa/samples-c/src/Ch15/rabinkarp.c, lines 30-65)

```
// MulMod returns a*b mod m for a, b < m, through the 128-bit product
// math/bits.Mul64 hands back. With a, b < m the product is below
// m * 2^64, so the high word is already below m and bits.Div64 is
// within its precondition.
func MulMod(a, b, m uint64) uint64 {
  hi, lo := bits.Mul64(a, b)
  _, r := bits.Div64(hi, lo, m)
  return r
}

// RabinKarpFindAll rolls a polynomial hash across the text: subtract
// the leaving character's weighted contribution, multiply by the
// base, add the entering character. A hash hit triggers a real
// comparison, so collisions can slow it but never fool it.
func RabinKarpFindAll(text, pat string) []int {
  var hits []int
  m := len(pat)
  if m == 0 || m > len(text) {
    return hits
  }
  // pow = base^(m-1) mod m61, the weight of the leaving character
  pow := uint64(1)
  for i := 1; i < m; i++ {
    pow = MulMod(pow, rkBase, rkMod)
  }
  hashWindow := func(s string) uint64 {
    h := uint64(0)
    for i := 0; i < len(s); i++ {
      h = (MulMod(h, rkBase, rkMod) + uint64(s[i])) % rkMod
    }
    return h
  }
  patHash := hashWindow(pat)
  window := hashWindow(text[:m])
  for at := 0; ; at++ {
    if window == patHash && text[at:at+m] == pat {
      hits = append(hits, at)
    }
    if at+m ≥ len(text) {
      return hits
    }
    // subtract-leave, multiply, add-enter
    leaving := MulMod(uint64(text[at]), pow, rkMod)
    window = (window + rkMod - leaving) % rkMod
    window = MulMod(window, rkBase, rkMod)
    window = (window + uint64(text[at+m])) % rkMod
  }
}
```

listing 443: go, mersenne 2^61-1 modulus, mulmod through bits.Mul64 (source: dsa/samples-go/ch15/rabinkarp.go, lines 13-60)

```
  static final long B = 256;
  static final long M = 1_000_000_007L;

  // a, b < M ~ 1e9, so a*b < 1e18 < 2^63: the product fits a signed long
  // with room to spare, no unsigned arithmetic needed on this lane
  static long mulmod(long a, long b) {
    return a * b % M;
  }

  static int verifications;
  static int spurious;

  static List<Integer> rabinkarp(String text, String pat) {
    List<Integer> hits = new ArrayList<>();
    int n = text.length(), m = pat.length();
    if (m == 0 || m > n)
      return hits;
    long hp = 0, ht = 0, pw = 1;
    for (int i = 0; i < m; i++) {
      hp = (mulmod(hp, B) + pat.charAt(i)) % M;
      ht = (mulmod(ht, B) + text.charAt(i)) % M;
      if (i > 0)
        pw = mulmod(pw, B);
    }
    verifications = 0;
    spurious = 0;
    for (int i = 0;; i++) {
      if (ht == hp) {
        verifications++;
        int j = 0;
        while (j < m && text.charAt(i + j) == pat.charAt(j))
          j++;
        if (j == m)
          hits.add(i);
        else
          spurious++;
      }
      if (i + m == n)
        break;
      // roll: drop text[i], shift, add text[i+m]
      ht = (ht + M - mulmod(text.charAt(i), pw)) % M;
      ht = (mulmod(ht, B) + text.charAt(i + m)) % M;
    }
    return hits;
  }

  static long hashOf(String s, int from, int len) {
    long h = 0;
    for (int i = 0; i < len; i++)
      h = (mulmod(h, B) + s.charAt(from + i)) % M;
    return h;
  }
```

listing 444: java, modulus 1e9+7 on plain long, every product under 2^63, prime added back on the roll (source: dsa/samples-java/src/Ch15/Rabinkarp.java, lines 20-71)

```
    /// <summary>Rabin-Karp: rolling polynomial hash, verify on hash hit.</summary>
    public static int RabinKarp(string text, string pattern)
    {
        const long prime = 1_000_000_007L, b = 256;
        int m = pattern.Length, n = text.Length;
        if (m == 0 || m > n)
            return -1;

        long patternHash = 0, windowHash = 0, high = 1;
        for (int i = 0; i < m; i++)
        {
            patternHash = (patternHash * b + pattern[i]) % prime;
            windowHash = (windowHash * b + text[i]) % prime;
            if (i < m - 1)
                high = high * b % prime;
        }

        for (int i = 0; i + m ≤ n; i++)
        {
            if (windowHash == patternHash && text.AsSpan(i, m).SequenceEqual(pattern))
                return i;
            if (i + m < n)
            {
                windowHash = ((windowHash - text[i] * high) * b + text[i + m]) % prime;
                if (windowHash < 0)
                    windowHash += prime;
            }
        }
        return -1;
    }
```

listing 445: c#, the roll on long, negative sums patched back up (source: dsa/samples/src/Ch15/Strings.cs, lines 68-97)

```
// integer boundary: the polynomial value reaches base^m * 256, which
// blows past 2^53 after a handful of characters, so the entire hash runs
// on BigInt and crosses back to Number only for array indices

const BASE = 256n;
const MOD = 1000000007n;

export function hashOf(s) {
  let h = 0n;
  for (const c of s) h = (h * BASE + BigInt(c.charCodeAt(0))) % MOD;
  return h;
}

// { hits, verifications, spurious }
export function rabinKarpStats(text, pat) {
  const m = pat.length;
  const n = text.length;
  if (m === 0 || m > n) return { hits: [], verifications: 0, spurious: 0 };
  const hp = hashOf(pat);
  let ht = 0n;
  let pw = 1n; // base^(m-1), the weight of the leaving character
  for (let i = 0; i < m; i++) {
    ht = (ht * BASE + BigInt(text.charCodeAt(i))) % MOD;
    if (i) pw = (pw * BASE) % MOD;
  }
  const hits = [];
  let verifications = 0;
  let spurious = 0;
  for (let i = 0; ; i++) {
    if (ht === hp) {
      verifications++;
      let j = 0;
      while (j < m && text[i + j] === pat[j]) j++;
      if (j === m) hits.push(i);
      else spurious++;
    }
    if (i + m === n) break;
    // roll: drop text[i], shift, add text[i+m]
    const leave = BigInt(text.charCodeAt(i)) * pw % MOD;
    ht = ((ht + MOD - leave) * BASE + BigInt(text.charCodeAt(i + m))) % MOD;
  }
  return { hits, verifications, spurious };
}
```

listing 446: javascript, the whole hash on BigInt past the 2^53 boundary (source: dsa/samples-js/src/ch15-rabinkarp.mjs, lines 4-46)

```python
BASE = 256
MOD = 1_000_000_007


def rabin_karp(text, pattern):
    m = len(pattern)
    if m == 0 or m > len(text):
        return [], 0, 0
    pat_hash = 0
    for ch in pattern:
        pat_hash = (pat_hash * BASE + ord(ch)) % MOD
    window = 0
    for ch in text[:m]:
        window = (window * BASE + ord(ch)) % MOD
    high = pow(BASE, m - 1, MOD)  # the leaving byte rides this factor out
    hits = hash_hits = 0
    positions = []
    for align in range(len(text) - m + 1):
        if window == pat_hash:
            hash_hits += 1
            if text[align : align + m] == pattern:  # verify: hashes can lie
                hits += 1
                positions.append(align)
        if align + m < len(text):  # roll: subtract leave, add enter
            leave = ord(text[align]) * high % MOD
            window = ((window - leave) * BASE + ord(text[align + m])) % MOD
    return positions, hash_hits, hits
```

listing 447: python, native ints, hash hits and verified hits counted apart (source: dsa/samples-py/src/Ch15/rabinkarp.py, lines 14-40)

```lua
local B = 256
local M = 1000000007

-- fits without wrap: a, b < M ~ 1e9, so a*b < 1e18 < 2^62
local function mulmod(a, b)
  return a * b % M
end

local stats

local function rabinkarp(text, pat)
  local tn, pn = #text, #pat
  if pn == 0 or pn > tn then return {} end
  local hp, ht, pw = 0, 0, 1
  for i = 1, pn do
    hp = (mulmod(hp, B) + pat:byte(i)) % M
    ht = (mulmod(ht, B) + text:byte(i)) % M
    if i > 1 then pw = mulmod(pw, B) end
  end
  stats = {hits = 0, verifications = 0, spurious = 0}
  local hits = {}
  local i = 0 -- 0-based window start
  while true do
    if ht == hp then
      stats.verifications = stats.verifications + 1
      local j = 0
      while j < pn and text:byte(i + j + 1) == pat:byte(j + 1) do j = j + 1 end
      if j == pn then
        hits[#hits + 1] = i
        stats.hits = stats.hits + 1
      else
        stats.spurious = stats.spurious + 1
      end
    end
    if i + pn == tn then break end
    -- roll: drop text[i], shift, add text[i+pn]
    ht = (ht + M - mulmod(text:byte(i + 1), pw)) % M
    ht = (mulmod(ht, B) + text:byte(i + pn + 1)) % M
    i = i + 1
  end
  return hits
end
```

listing 448: lua, small modulus keeps every product below 2^62 (source: dsa/samples-lua/ch15_rabinkarp.lua, lines 8-49)

The rolling hash is the chapter's first real integer-semantics checkpoint, and the seven suites take four different roads through it. C and Lua hold the modulus at 1e9+7, where both operands of every product stay below about 1e9 and the product below 2^62, so a plain multiply and remainder is safe, in Lua specifically because its integers wrap silently at 64 bits. C# and Java run the same modulus on a signed 64-bit long, where the subtraction can leave the window hash negative, so the roll adds the prime back first. Go prefers the Mersenne prime 2^61-1, which needs a genuine 128-bit product from `math/bits.Mul64`. JavaScript puts the entire hash on BigInt with a comment at the boundary, because base 256 to the window length passes 2^53 after a handful of characters. Python needs no ceremony at all. C, Java, and Lua pin the same hand-computed hash, `abab` at 1633837410 reducing to 633837403, and every suite reports 3 verifications and 0 spurious hits on the overlapping fixture.

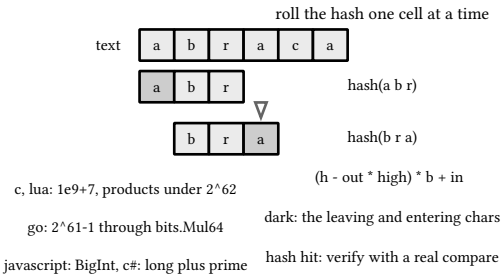


figure 120: rabin karp rolls the hash, the leaving and entering characters carry the update, every hit verified

15.5. aho-corasick

Many patterns, one pass. The machine is a trie of the patterns, chapter 9, plus failure links computed breadth first, which are exactly KMP borders generalized to the trie.

The dry run: the machine is `he`, `she`, `his`, `hers` over `ushershehished`, all eight hits pinned by the C# suite, and the C and Lua twins pin the machine itself, 10 nodes and the four named failure links.

1. The text reads u s h e r s h e h i s h e d, and the walk opens at the root with no output until the first e.
2. Position 3 sits on she and emits twice, she with he riding its failure link, the nested hit, count 2.
3. Position 5 closes hers, count 3, and position 7 brings the second she with its he, count 5.
4. Position 10 fires his after the walk fell back to the root and re-entered through h and hi, count 6.
5. Position 12 closes the third she with its he, count 8, one output per pattern ending, nested included, the pinned total.

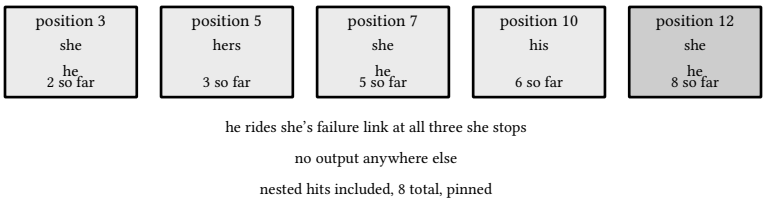


figure 121: the pass as five emission stops, each column the position, the patterns fired there, and the running count

The 8 hits with he inside she at all three of its stops is the pinned pass, and the listings below build the machine in seven languages.

```c
static void build(void) {
  node_n = 0;
  new_node(); // root = 0
  for (int p = 0; p < 4; p++) {
    int cur = 0;
    for (const char *q = PATS[p]; *q; q++) {
      int c = *q - 'a';
      if (nodes[cur].ch[c] < 0)
        nodes[cur].ch[c] = new_node();
      cur = nodes[cur].ch[c];
    }
    nodes[cur].word = p;
  }
  // breadth-first failure links: longest proper suffix that is a prefix
  int qh = 0, qt = 0;
  for (int c = 0; c < 26; c++)
    if (nodes[0].ch[c] > 0)
      bfs[qt++] = nodes[0].ch[c];
  while (qh < qt) {
    int u = bfs[qh++];
    for (int c = 0; c < 26; c++) {
      int v = nodes[u].ch[c];
      if (v < 0)
        continue;
      int f = nodes[u].fail;
      while (f && nodes[f].ch[c] < 0)
        f = nodes[f].fail;
      int nxt = nodes[f].ch[c];
      nodes[v].fail = (nxt ≥ 0 && nxt ≠ v) ? nxt : 0;
      bfs[qt++] = v;
    }
  }
}
```

listing 449: c, the trie, then breadth-first failure links (source: dsa/samples-c/src/Ch15/aho.c, lines 41-73)

```
func BuildAho(patterns []string) *AhoCorasick {
  a := &AhoCorasick{root: &AhoNode{next: map[byte]*AhoNode{}}, patterns: patterns}
  for id, pat := range patterns {
    n := a.root
    for i := 0; i < len(pat); i++ {
      c := pat[i]
      next, ok := n.next[c]
      if !ok {
        next = &AhoNode{next: map[byte]*AhoNode{}}
        n.next[c] = next
      }
      n = next
    }
    n.output = append(n.output, id)
  }
  queue := []*AhoNode{a.root}
  for len(queue) > 0 {
    cur := queue[0]
    queue = queue[1:]
    for c, child := range cur.next {
      fail := cur.fail
      for fail != nil && fail.next[c] == nil {
        fail = fail.fail
      }
      if fail == nil {
        child.fail = a.root // depth 1: nothing shallower fits
      } else {
        child.fail = fail.next[c]
      }
      // fail links always land shallower, so no cycle exists
      child.output = append(child.output, child.fail.output...)
      queue = append(queue, child)
    }
  }
  return a
}
```

listing 450: go, the trie build and map-carrying nodes (source: dsa/samples-go/ch15/aho.go, lines 21-56)

```
  static void build(String[] patterns) {
    pats = patterns;
    nodeN = 0;
    newNode(); // root = 0
    for (int p = 0; p < pats.length; p++) {
      int cur = 0;
      for (int i = 0; i < pats[p].length(); i++) {
        int c = pats[p].charAt(i) - 'a';
        if (ch[cur][c] < 0)
          ch[cur][c] = newNode();
        cur = ch[cur][c];
      }
      word[cur] = p;
    }
    // breadth-first failure links: longest proper suffix that is a prefix
    int qh = 0, qt = 0;
    for (int c = 0; c < 26; c++)
      if (ch[0][c] > 0)
        bfs[qt++] = ch[0][c];
    while (qh < qt) {
      int u = bfs[qh++];
      for (int c = 0; c < 26; c++) {
        int v = ch[u][c];
        if (v < 0)
          continue;
        int f = fail[u];
        while (f > 0 && ch[f][c] < 0)
          f = fail[f];
        int nxt = ch[f][c];
        fail[v] = (nxt ≥ 0 && nxt ≠ v) ? nxt : 0;
        bfs[qt++] = v;
      }
    }
  }

  record Hit(int end, int pat) {}

  // walk the text, reporting every pattern hit with its end index
  static List<Hit> search(String text) {
    List<Hit> hits = new ArrayList<>();
    int cur = 0;
    for (int i = 0; i < text.length(); i++) {
      int c = text.charAt(i) - 'a';
      while (cur > 0 && ch[cur][c] < 0)
        cur = fail[cur];
      if (ch[cur][c] ≥ 0)
        cur = ch[cur][c];
      // output chain: this node and every fail-ancestor ending a pattern
      for (int at = cur; at > 0; at = fail[at])
        if (word[at] ≥ 0)
          hits.add(new Hit(i, word[at]));
    }
    return hits;
  }
```

listing 451: java, the int-array trie, breadth-first fail links, the scan walks the fail chain (source: dsa/samples-java/src/Ch15/Aho.java, lines 37-90)

```
public sealed class AhoCorasick
{
    private sealed class Node
    {
        public readonly Dictionary<char, Node> Children = new();
        public Node? Fail;
        public readonly List<string> Outputs = new();
    }

    private readonly Node _root = new();
    private bool _built;

    public AhoCorasick Add(string pattern)
    {
        if (_built)
            throw new InvalidOperationException("machine already built");
        Node node = _root;
        foreach (char c in pattern)
            node = node.Children.TryGetValue(c, out Node? next)
                ? next
                : node.Children[c] = new Node();
        if (pattern.Length > 0)
            node.Outputs.Add(pattern);
        return this;
    }

    public void Build()
    {
        _built = true;
        var queue = new Queue<Node>();
        _root.Fail = _root;
        foreach (Node child in _root.Children.Values)
        {
            child.Fail = _root;
            queue.Enqueue(child);
        }
        while (queue.Count > 0)
        {
            Node node = queue.Dequeue();
            foreach ((char c, Node child) in node.Children)
            {
                queue.Enqueue(child);
                Node fail = node.Fail!;
                while (fail != _root && !fail.Children.ContainsKey(c))
                    fail = fail.Fail!;
                child.Fail = fail.Children.TryGetValue(c, out Node? next) && next != child
                    ? next
                    : _root;
                child.Outputs.AddRange(child.Fail.Outputs); // suffix outputs count too
            }
        }
    }
```

listing 452: c#, add patterns, then the build that hangs outputs on fail chains (source: dsa/samples/src/Ch15/Strings.cs, lines 101-152)

```
class AhoTrie {
  #nodes = [{ ch: {}, fail: 0, word: -1 }];

  constructor(patterns) {
    patterns.forEach((p, id) => {
      let cur = 0;
      for (const c of p) {
        if (!this.#nodes[cur].ch[c]) {
          this.#nodes[cur].ch[c] = this.#nodes.length;
          this.#nodes.push({ ch: {}, fail: 0, word: -1 });
        }
        cur = this.#nodes[cur].ch[c];
      }
      this.#nodes[cur].word = id;
    });
    this.#bfsLinks();
  }

  // longest proper suffix of the node's string that is a trie prefix
  #bfsLinks() {
    const queue = [];
    for (const c of Object.keys(this.#nodes[0].ch).sort())
      queue.push(this.#nodes[0].ch[c]);
    for (let head = 0; head < queue.length; head++) {
      const u = queue[head];
      for (const [c, v] of Object.entries(this.#nodes[u].ch)) {
        let f = this.#nodes[u].fail;
        while (f !== 0 && !this.#nodes[f].ch[c]) f = this.#nodes[f].fail;
        const nxt = this.#nodes[f].ch[c];
        this.#nodes[v].fail = nxt !== undefined && nxt !== v ? nxt : 0;
        queue.push(v);
      }
    }
  }
```

listing 453: javascript, node array, sorted breadth-first link pass (source: dsa/samples-js/src/ch15-aho.mjs, lines 4-37)

```python
def build(patterns):
    root = AhoNode()
    for pattern in patterns:
        node = root
        for ch in pattern:
            node = node.children.setdefault(ch, AhoNode())
        node.out.append(pattern)
    queue = deque()
    for child in root.children.values():
        child.fail = root
        queue.append(child)
    while queue:  # breadth first: links point strictly shallower
        node = queue.popleft()
        for ch, child in node.children.items():
            queue.append(child)
            back = node.fail
            while back and ch not in back.children:
                back = back.fail
            child.fail = back.children[ch] if back and ch in back.children else root
            child.out = child.out + child.fail.out
    return root


def search(root, text):
    hits = []
    node = root
    for i, ch in enumerate(text):
        while node is not root and ch not in node.children:
            node = node.fail  # fall back, never re-read the text
        node = node.children.get(ch, root)
        for pattern in node.out:
            hits.append((i - len(pattern) + 1, pattern))
    return hits
```

listing 454: python, the build and the single-pass search, deque for the bfs (source: dsa/samples-py/src/Ch15/aho.py, lines 25-57)

```
local function build()
  nodes = {new_node()} -- root is node 1; fail targets use node ids
  for p, pat in ipairs(PATS) do
    local cur = 1
    for i = 1, #pat do
      local c = pat:sub(i, i)
      if not nodes[cur].ch[c] then
        nodes[#nodes + 1] = new_node()
        nodes[cur].ch[c] = #nodes
      end
      cur = nodes[cur].ch[c]
    end
    nodes[cur].word = p
  end
  -- breadth-first failure links: longest proper suffix that is a prefix
  local bfs = {}
  for _, v in pairs(nodes[1].ch) do
    nodes[v].fail = 1 -- depth-one nodes fail to the root
    bfs[#bfs + 1] = v
  end
  local head = 1
  while head ≤ #bfs do
    local u = bfs[head]
    head = head + 1
    for c, v in pairs(nodes[u].ch) do
      local f = nodes[u].fail
      while f > 1 and not nodes[f].ch[c] do f = nodes[f].fail end
      local nxt = nodes[f].ch[c]
      nodes[v].fail = (nxt and nxt ~= v) and nxt or 1
      bfs[#bfs + 1] = v
    end
  end
end

-- walk the text, reporting every pattern hit with its end index
local function search(text)
  local hits = {}
  local cur = 1
  for i = 1, #text do
    local c = text:sub(i, i)
    while cur > 1 and not nodes[cur].ch[c] do cur = nodes[cur].fail end
    if nodes[cur].ch[c] then cur = nodes[cur].ch[c] end
    -- output chain: this node and every fail-ancestor ending a pattern
    local at = cur
    while at > 1 do
      if nodes[at].word then
        hits[#hits + 1] = {endidx = i - 1, pat = nodes[at].word}
      end
      at = nodes[at].fail
    end
  end
  return hits
end
```

listing 455: lua, node table, fail links, the scan with its output chain (source: dsa/samples-lua/ch15_aho.lua, lines 14-66)

Each output node also carries its fail node's outputs, so the scan reports every pattern ending at each position, nested matches included, `he` inside `she` inside `hers`. The test builds the classic four-pattern machine and pins all

eight hits of one text. This is the spam filter, virus scanner, and censorship library algorithm, and it is the last structure the capstone's key index could borrow from, prefix queries over a dictionary of stored keys.

All seven suites build `he`, `she`, `his`, `hers` and pin the same eight hits over `ushershehished`. The output machinery splits two ways, and both are correct: C, Java, JavaScript, and Lua walk the fail chain at search time looking for nodes that end patterns, while C#, Go, and Python fold the fail node's output list into the node during the build, buying a flat scan with one append per link. C, Java, and Lua also pin the machine itself, 10 nodes and the four named failure links, and Python borrows `collections.deque` for the breadth-first queue, the one supporting container in the chapter.

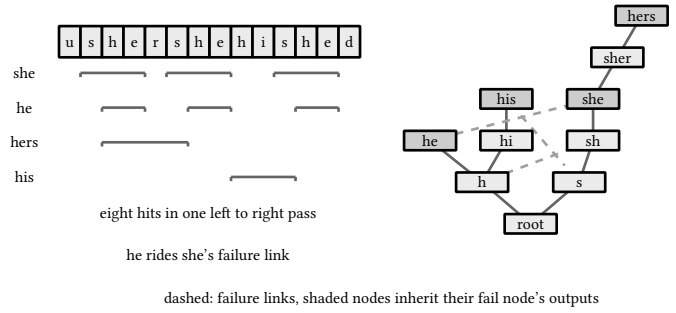


figure 122: aho-corasick, a trie with failure links, one pass reports every pattern including he inside she

15.6. across the seven languages

The build sizes count non-comment source lines over this chapter's five featured files per language, checks included where they share the file, which is the C and Lua convention, and excluded where a separate test project carries them:

language	build SLOC	dependency	boundary note
c	363	libc only	unsigned char casts index the 256-slot skip table, counts in long long, 83 checks ride in main
c#	190	bcl only	the rolling hash runs on long and adds the prime back after a subtract, horspool returns the first index
go	199	math/bits	rabin karp mods the mersenne 2^61-1, mulmod through bits.Mul64, a match advances the whole pattern
java	476	jdk 27 stdlib	the 1e9+7 hash stays on plain long, every product under 2^63, the indexOf referee is non-empty-needles-only because it clamps an empty needle
javascript	182	node stdlib	the hash is BigInt with a comment at the 2^53 boundary, horspool skips overlaps by design
python	274	stdlib only	native ints, str.find is the ground truth, collections.deque is the one borrowed container, for the bfs
lua	339	lib.lua harness	modulus 1e9+7 keeps products under 2^62 so the silent int64 wrap never fires here

sources: learn.microsoft.com, `string.IndexOf` overloads used as ground truth, `MemoryExtensions.SequenceEqual`, `Array.Fill`, accessed 2026-09-08, go.dev/pkg/math/bits for `Mul64`, accessed 2026-09-14, plus the KMP, Boyer-Moore, and Aho-Corasick literature cited in the chapter text. Sample behavior verified by `make verify-csharp`, 11 tests in chapter 15 of the samples suite. The seven-language layer verifies the same way: 5 C programs with 83 embedded checks under `make verify-c`, 5 Ch15 java programs with 83 checks under `run-java-samples`, 17 Go tests, 20 `node --test` cases, 49 Python checks across 5 files, and 21 Lua checks under `run.lua`.

16. number theory and modular arithmetic

The integer toolkit competitive and puzzle problems lean on: gcd and lcm, the sieve, factorization, modular exponentiation and inverses, and the chinese remainder theorem. Every algorithm here is a dozen lines or less, and the chapter's real subject is the boundary each language draws around its integers, because modular arithmetic is exactly where silent overflow turns a right algorithm into a wrong answer. Seven suites, one pinned anchor per family, five different roads through the multiplication problem.

16.1. gcd, lcm, and the binary variant

Euclid's gcd replaces the pair (a, b) with (b, a mod b) until the second slot hits zero, and the first is the answer. The remainder shrinks strictly every step, so it always terminates, and the worst case is consecutive fibonacci numbers. lcm must go through the gcd with the division first, a divided by gcd then times b, because multiplying first can overflow an intermediate that the final answer never needed. Stein's binary variant needs no division at all: strip the common factors of two, keep subtracting the smaller from the larger, an odd minus an odd is even so more twos keep falling out, then shift the saved twos back in.

The dry run: the fixtures are gcd(48, 18), the sign pair, and lcm(4, 6), asserted by the C# suite, Go cross-checks its binary variant row by row, and C and Lua pin the 4 000 000 000 by 3 000 000 000 lcm through the divide-first formula.

1. Euclid stops at 6 on (48, 18), with the zero conventions beside it: gcd(0, 7) = 7 and gcd(0, 0) = 0, both pinned.
2. Signs fold in before the loop: gcd(-24, 36) enters as (24, 36) and leaves 12.
3. The lcm divides first: gcd(4, 6) = 2, then 4 / 2 = 2, and 2 × 6 = 12, so the product in flight never reaches 4 × 6 = 24.
4. The binary variant saves the one common two: 48 >> 4 = 3 stands against 18 >> 1 = 9, both odd.
5. The subtract loop runs 9 - 3 = 6, halves to 3, and 3 - 3 = 0 stops it on the core.
6. The saved two lands back, 3 << 1 = 6, and the sweep agrees with euclid everywhere C checks, both sides 0 through 40.

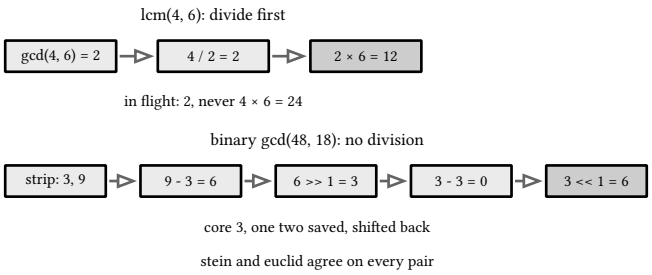


figure 123: the run as two pipelines, the lcm dividing before it multiplies, the binary variant subtracting to the odd core

The 6 and the divide-first 12 are the suite's pins, and the listings below run both variants in seven languages.

```c
static unsigned long long gcd_euclid(unsigned long long a,
                                     unsigned long long b) {
  while (b) {
    unsigned long long t = a % b;
    a = b;
    b = t;
  }
  return a;
}

// stein's binary gcd: strip common twos, then subtract until zero
static unsigned long long gcd_binary(unsigned long long a,
                                     unsigned long long b) {
  if (a == 0)
    return b;
  if (b == 0)
    return a;
  unsigned long long shift = 0;
  while (((a | b) & 1) == 0) {
    a >>= 1;
    b >>= 1;
    shift++;
  }
  while ((a & 1) == 0)
    a >>= 1;
  do {
    while ((b & 1) == 0)
      b >>= 1;
    if (a > b) {
      unsigned long long t = a;
      a = b;
      b = t;
    }
    b -= a;
  } while (b);
  return a << shift;
}

static unsigned long long lcm(unsigned long long a, unsigned long long b) {
  if (a == 0 || b == 0)
    return 0;
  return a / gcd_euclid(a, b) * b;
}
```

listing 456: c, euclid, stein, and the divide-first lcm over unsigned long long (source: dsa/samples-c/src/Ch16/gcdlcm.c, lines 17-59)

```
// GCD is Euclid's algorithm: gcd(a, b) = gcd(b, a mod b).
func GCD(a, b int64) int64 {
  for b ≠ 0 {
    a, b = b, a%b
  }
  if a < 0 {
    return -a
  }
  return a
}

// LCM is |a*b|/gcd(a,b): the gcd divides out first so the
// intermediate product cannot overflow before the answer does.
func LCM(a, b int64) int64 {
  if a == 0 || b == 0 {
    return 0
  }
  g := GCD(a, b)
  l := (a / g) * b
  if l < 0 {
    return -l
  }
  return l
}

// BinaryGCD is Stein's variant: factor out shared twos, then subtract
// (the difference of two odds is even), then restore the twos. No
// division anywhere.
func BinaryGCD(a, b int64) int64 {
  if a == 0 {
    return abs64(b)
  }
  if b == 0 {
    return abs64(a)
  }
  a, b = abs64(a), abs64(b)
  shift := 0
  for a&1 == 0 && b&1 == 0 {
    a >>= 1
    b >>= 1
    shift++
  }
  for a&1 == 0 {
    a >>= 1
  }
  for b ≠ 0 {
    for b&1 == 0 {
      b >>= 1
    }
    if a > b {
      a, b = b, a
    }
    b -= a
  }
  return a << shift
```

listing 457: go, both gcd variants on int64, abs applied at the edges (source: dsa/samples-go/ch16/gcdlcm.go, lines 6-60)

```
static long gcdEuclid(long a, long b) {
  while (b != 0) {
    long t = a % b;
    a = b;
    b = t;
  }
  return a;
}

// stein's binary gcd: strip common twos, then subtract until zero.
// C runs this over unsigned long long, so the swap test compares
// unsigned: Long.compareUnsigned carries that law into java
static long gcdBinary(long a, long b) {
  if (a == 0)
    return b;
  if (b == 0)
    return a;
  int shift = 0;
  while (((a | b) & 1) == 0) {
    a >>>= 1;
    b >>>= 1;
    shift++;
  }
  while ((a & 1) == 0)
    a >>>= 1;
  do {
    while ((b & 1) == 0)
      b >>>= 1;
    if (Long.compareUnsigned(a, b) > 0) {
      long t = a;
      a = b;
      b = t;
    }
    b = b - a;
  } while (b != 0);
  return a << shift;
}

static long lcm(long a, long b) {
  if (a == 0 || b == 0)
    return 0;
  return a / gcdEuclid(a, b) * b;
}
```

listing 458: java, euclid, stein with the swap compared unsigned, divide-first lcm (source: dsa/samples-java/src/Ch16/Gcdlcm.java, lines 16-58)

```
    /// <summary>Euclid gcd, Gcd(0, 0) = 0.</summary>
    public static long Gcd(long a, long b)
    {
        (a, b) = (Math.Abs(a), Math.Abs(b));
        while (b != 0)
            (a, b) = (b, a % b);
        return a;
    }

    public static long Lcm(long a, long b)
    {
        long g = Gcd(a, b);
        return g == 0 ? 0 : Math.Abs(a / g) * Math.Abs(b);
    }

    /// <summary>Binary (Stein) gcd: strip common factors of 2, then subtract. Same results as Euclid.</
summary>
    public static long BinaryGcd(long a, long b)
    {
        (a, b) = (Math.Abs(a), Math.Abs(b));
        if (a == 0)
            return b;
        if (b == 0)
            return a;
        int shift = Math.Min(BitTrailingZeros(a), BitTrailingZeros(b));
        a >>= BitTrailingZeros(a);
        do
        {
            b >>= BitTrailingZeros(b);
            if (a > b)
                (a, b) = (b, a); // keep a ≤ b
            b -= a;
        } while (b != 0);
        return a << shift;
    }
```

listing 459: c#, sign-normalized gcd and lcm, trailing-zero binary gcd (source: dsa/samples/src/Ch16/NumTheory.cs, lines 6-39)

```
export function gcd(a, b) {
  while (b !== 0n) [a, b] = [b, a % b];
  return a;
}

// gcd-first order: divide before multiplying, no premature blowup
export function lcm(a, b) {
  if (a === 0n || b === 0n) return 0n;
  return (a / gcd(a, b)) * b;
}

// stein's binary gcd: strip factors of two, subtract, repeat
export function binaryGcd(a, b) {
  if (a === 0n) return b;
  if (b === 0n) return a;
  let shift = 0n;
  while (((a | b) & 1n) === 0n) {
    a >>= 1n;
    b >>= 1n;
    shift++;
  }
  while ((a & 1n) === 0n) a >>= 1n;
  for (;;) {
    while ((b & 1n) === 0n) b >>= 1n;
    if (a > b) [a, b] = [b, a];
    b -= a;
    if (b === 0n) break;
  }
  return a << shift;
}
```

listing 460: javascript, all BigInt, lcm products cross 2^53 early (source: dsa/samples-js/src/ch16-gcdlcm.mjs, lines 6-35)

```
def gcd_euclid(a, b):
    while b:
        a, b = b, a % b  # the remainder shrinks strictly, always ends at 0
    return a


def lcm(a, b):
    return a // gcd_euclid(a, b) * b  # divide first: the product stays small


def gcd_binary(a, b):
    if a == 0:
        return b
    if b == 0:
        return a
    shift = 0
    while not ((a | b) & 1):  # both even: factor the two out
        a //= 2
        b //= 2
        shift += 1
    while not (a & 1):
        a //= 2
    while not (b & 1):
        b //= 2
    while a != b:
        if a > b:
            a = (a - b) // 2  # odd minus odd is even: halve it
            while not (a & 1):
                a //= 2
        else:
            b = (b - a) // 2
            while not (b & 1):
                b //= 2
    return a << shift
```

listing 461: python, divide-first lcm, halving in the subtract loop (source: dsa/samples-py/src/Ch16/gcdlcm.py, lines 13-46)

```lua
local function gcd_euclid(a, b)
  while b ~= 0 do
    a, b = b, a % b
  end
  return a
end

-- stein's binary gcd: strip common twos, then subtract until zero
local function gcd_binary(a, b)
  if a == 0 then return b end
  if b == 0 then return a end
  local shift = 0
  while (a | b) & 1 == 0 do
    a = a >> 1
    b = b >> 1
    shift = shift + 1
  end
  while a & 1 == 0 do a = a >> 1 end
  repeat
    while b & 1 == 0 do b = b >> 1 end
    if a > b then a, b = b, a end
    b = b - a
  until b == 0
  return a << shift
end

-- divide before multiplying so the lcm never overflows on big inputs
local function lcm(a, b)
  if a == 0 or b == 0 then return 0 end
  return a // gcd_euclid(a, b) * b
end
```

listing 462: lua, integer division and shifts, no overflow at these sizes (source: dsa/samples-lua/ch16_gcdlcm.lua, lines 6-36)

Every suite pins the same fixture family, gcd(48, 18) = 6, gcd(7, 13) = 1 on the coprime pair, and the zero conventions, gcd(0, n) = n and gcd(0, 0) = 0. The binary variant is cross-checked against euclid everywhere, over the shared fixtures and over a full sweep in C, Java, Lua, and Python, Java's sweep also asserting the gcd times lcm equals a times b identity on every pair. Its stein loop runs on `long` but compares the swap through `Long.compareUnsigned`, the unsigned law C gets from its type and every other suite gets from its runtime. C and Lua pin lcm(4 000 000 000, 3 000 000 000) recomputed through the divide-first formula, the case where multiplying first would wrap. JavaScript runs the whole file on BigInt for the same reason in the other direction, its comment pins the boundary: lcm products cross 2^53 long before interesting inputs.

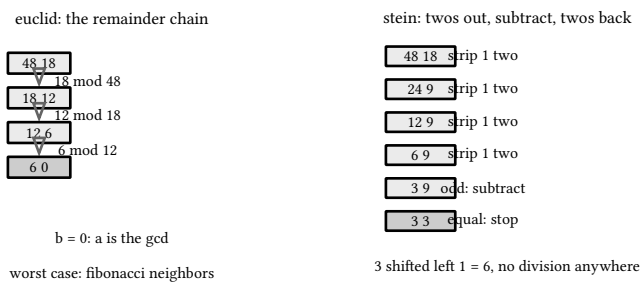


figure 124: euclid walks the remainder chain, stein strips twos and subtracts, both land on 6 for 48 and 18

16.2. the sieve of eratosthenes

Cross off the multiples of every prime as it is found and whatever survives is prime. The loop bound p squared is the whole argument: a composite below p squared has a prime factor below p, so it was already crossed off. Recording which prime does the crossing turns the sieve into a smallest-prime-factor table, and that table is a factorizer, divide it out repeatedly and the number peels into its primes.

The dry run: the fixture is the limit 100, asserted by the C# suite on the count and the spf spot checks, and the siblings split only on where the crossing starts, 2p against p squared.

1. Every flag opens true, then 0 and 1 close, and prime 2 starts crossing at 2 × 2 = 4, stepping by 2 through the evens.
2. The spf table fills in the same pass: 4 meets 2's run first and stamps it, spf[4] = 2, pinned.
3. Prime 3 starts at 3 × 3 = 9: 9 takes spf 3, and the evens keep the 2 they already carry.
4. Prime 5 starts at 5 × 5 = 25, prime 7 at 7 × 7 = 49, and 11 never runs: 11 × 11 = 121 sits past 100, the loop bound.
5. 91 meets its crossing only at 7, as 7 × 13 = 91: composite with spf 7 despite looking prime, both pinned.
6. The survivors count 25, pi(100), and 97 closes the run keeping itself, spf[97] = 97.

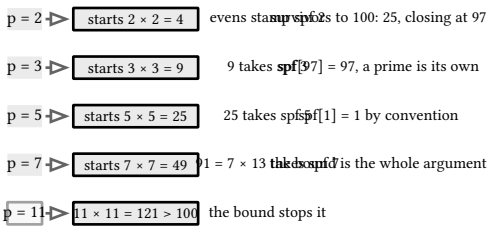


figure 125: the crossing schedule in run order, each prime starting at its square, 11 stopped by the bound, spf stamped as each run passes

The 25 with 91 stamped 7 is the pinned pair, and the listings below sieve in seven languages.

```
#define LIM 100

static unsigned char is_prime[LIM + 1];
static int spf[LIM + 1]; // smallest prime factor, 0 for 0 and 1

static void sieve(void) {
  for (int i = 0; i ≤ LIM; i++) {
    is_prime[i] = 1;
    spf[i] = 0;
  }
  is_prime[0] = is_prime[1] = 0;
  for (int p = 2; p ≤ LIM; p++)
    if (is_prime[p]) {
      spf[p] = p;
      for (int m = 2 * p; m ≤ LIM; m += p) {
        is_prime[m] = 0;
        if (spf[m] == 0)
          spf[m] = p;
      }
    }
}

static int pi(int x) {
  int c = 0;
  for (int i = 2; i ≤ x; i++)
    c += is_prime[i];
  return c;
}
```

listing 463: c, crossing from 2p so the spf table fills in the same pass (source: dsa/samples-c/src/Ch16/sieve.c, lines 16-43)

```go
// Sieve returns isComposite for 0..n by the sieve of Eratosthenes:
// cross off every multiple of each prime, starting at its square.
func Sieve(n int) []bool {
  isComposite := make([]bool, n+1)
  isComposite[0] = true
  if n ≥ 1 {
    isComposite[1] = true
  }
  for p := 2; p*p <= n; p++ {
    if isComposite[p] {
      continue
    }
    for m := p * p; m <= n; m += p {
      isComposite[m] = true
    }
  }
  return isComposite
}

// CountPrimes returns pi(n): how many primes are ≤ n.
func CountPrimes(n int) int {
  count := 0
  for _, comp := range Sieve(n) {
    if !comp {
      count++
    }
  }
  return count
}

// SmallestFactor returns spf, the smallest prime factor of every
// value in 2..n (spf[0] and spf[1] are meaningless). It is the sieve
// recorded differently: every composite gets crossed off by, and
// remembers, its smallest prime.
func SmallestFactor(n int) []int {
  spf := make([]int, n+1)
  for p := 2; p <= n; p++ {
    if spf[p] ≠ 0 {
      continue // already claimed by a smaller prime
    }
    for m := p; m <= n; m += p {
      if spf[m] == 0 {
        spf[m] = p
      }
    }
  }
  return spf
}
```

listing 464: go, the p-squared sieve, then spf as a second recording (source: dsa/samples-go/ch16/sieve.go, lines 3-50)

```
  static void sieve() {
    isPrime = new boolean[LIM + 1];
    spf = new int[LIM + 1];
    for (int i = 2; i ≤ LIM; i++)
      isPrime[i] = true;
    for (int p = 2; p ≤ LIM; p++)
      if (isPrime[p]) {
        spf[p] = p;
        for (int m = 2 * p; m ≤ LIM; m += p) {
          isPrime[m] = false;
          if (spf[m] == 0)
            spf[m] = p;
        }
      }
  }

  static int pi(int x) {
    int c = 0;
    for (int i = 2; i ≤ x; i++)
      if (isPrime[i])
        c++;
    return c;
  }

  // peel the smallest factor until 1, the sieve as a factorizer
  static List<Integer> factorize(int x, int[] table) {
    List<Integer> out = new ArrayList<>();
    while (x > 1) {
      out.add(table[x]);
      x /= table[x];
    }
    return out;
  }

  static int[] spfTable(int limit) {
    int[] t = new int[limit + 1];
    for (int i = 0; i ≤ limit; i++)
      t[i] = i; // default: prime until split
    for (int i = 2; (long) i * i ≤ limit; i++)
      if (t[i] == i)
        for (int m = i * i; m ≤ limit; m += i)
          if (t[m] == m)
            t[m] = i;
    return t;
  }
```

listing 465: java, the 2p crossing stamps spf in the same pass, the p-squared table beside it (source: dsa/samples-java/src/Ch16/Sieve.java, lines 23-67)

```
    /// <summary>Sieve of Eratosthenes up to n inclusive: primality flags plus smallest prime factor per
index.</summary>
    public static (bool[] IsPrime, int[] SmallestFactor) Sieve(int n)
    {
        var isPrime = new bool[n + 1];
        var spf = new int[n + 1];
        Array.Fill(isPrime, true);
        if (n ≥ 0)
            isPrime[0] = false;
        if (n ≥ 1)
            isPrime[1] = false;
        spf[1] = 1; // by convention 1 is its own smallest factor here
        for (int i = 2; (long)i * i ≤ n; i++)
            if (isPrime[i])
                for (long j = (long)i * i; j ≤ n; j += i)
                {
                    isPrime[j] = false;
                    if (spf[j] == 0)
                        spf[j] = i;
                }
        for (int i = 2; i ≤ n; i++)
            if (isPrime[i])
                spf[i] = i;
        return (isPrime, spf);
    }
```

listing 466: c#, flags and spf together, primes stamped last (source: dsa/samples/src/Ch16/NumTheory.cs, lines 52-75)

```
export function sieve(n) {
  const composite = new Array(n + 1).fill(false);
  if (n ≥ 0) composite[0] = true;
  if (n ≥ 1) composite[1] = true;
  for (let i = 2; i * i ≤ n; i++)
    if (!composite[i])
      for (let j = i * i; j ≤ n; j += i) composite[j] = true;
  return composite;
}

export function countPrimes(n) {
  const composite = sieve(n);
  let count = 0;
  for (let i = 2; i ≤ n; i++) if (!composite[i]) count++;
  return count;
}

// spf[v] is the smallest prime factor of v (spf[v] === v for primes)
export function smallestFactor(n) {
  const spf = [...Array(n + 1).keys()];
  for (let i = 2; i * i ≤ n; i++)
    if (spf[i] === i)
      for (let j = i * i; j ≤ n; j += i)
        if (spf[j] === j) spf[j] = i;
  return spf;
}
```

listing 467: javascript, composite flags and the identity-init spf array (source: dsa/samples-js/src/ch16-sieve.mjs, lines 8-33)

```
def sieve(limit):
    is_prime = [True] * (limit + 1)
    is_prime[0] = is_prime[1] = False
    p = 2
    while p * p ≤ limit:  # composites below p*p are already marked
        if is_prime[p]:
            for multiple in range(p * p, limit + 1, p):
                is_prime[multiple] = False
        p += 1
    return is_prime


def primes_up_to(limit):
    is_prime = sieve(limit)
    return [i for i, yes in enumerate(is_prime) if yes]


def smallest_factors(limit):
    spf = list(range(limit + 1))  # spf[i] defaults to i: prime until split
    i = 2
    while i * i ≤ limit:
        if spf[i] == i:  # i never got marked: it is prime
            for multiple in range(i * i, limit + 1, i):
                if spf[multiple] == multiple:
                    spf[multiple] = i
        i += 1
    return spf
```

listing 468: python, the sieve, the prime list, spf defaulting to itself (source: dsa/samples-py/src/Ch16/sieve.py, lines 13-39)

```
local function sieve()
  is_prime, spf = {}, {}
  for i = 0, LIM do
    is_prime[i] = true
    spf[i] = 0
  end
  is_prime[0] = false
  is_prime[1] = false
  for p = 2, LIM do
    if is_prime[p] then
      spf[p] = p
      for m = 2 * p, LIM, p do
        is_prime[m] = false
        if spf[m] == 0 then spf[m] = p end
      end
    end
  end
end

local function pi(x)
  local c = 0
  for i = 2, x do
    if is_prime[i] then c = c + 1 end
  end
  return c
end
```

listing 469: lua, one table pass filling flags and spf together (source: dsa/samples-lua/ch16_sieve.lua, lines 9-34)

All seven suites pin pi(100) = 25, the opening run 2, 3, 5, 7, the close on 97, and that 91 is composite despite looking prime, it is 7 times 13. The crossing start splits by purpose: C, Java, and Lua walk multiples from 2p because the spf table wants to catch every composite with its smallest prime, C# starts at p squared and still fills spf in the same pass since every composite sits at or above its smallest prime's square, while the pure primality sieves in Go, JavaScript, and Python start at p squared and skip the work below it, then record spf in a second pass when they record it at all. Java carries both under one roof, the 2p crossing in its main sieve and a p-squared spf table beside it. Python peels factors straight off its spf table, the sieve as factorizer, and the integer boundary is a non-event here: every value sits at or below the limit, so even JavaScript stays on Number.

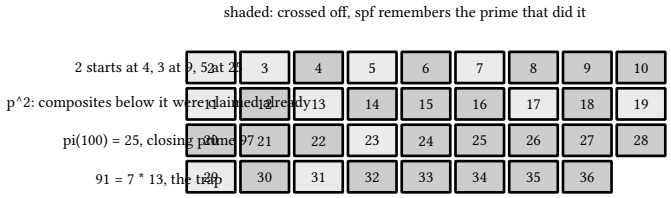


figure 126: the sieve to 36, each prime crosses off from its square, spf remembers who did the crossing

16.3. factorization by trial division

Trial division divides out each candidate as many times as it goes, and the survivor past the square root of what remains is itself prime, so 360 peels 2, 2, 2, 3, 3 and leaves 5.

The dry run: the fixtures are 360, 97, 1, and 1024, asserted by the C# suite, Python adds the survivor 999 983, and every suite holding the expanded list rebuilds the input from it.

1. 360 peels under 2 three times: 360 / 2 = 180, 180 / 2 = 90, 90 / 2 = 45, recording the pair (2, 3).
2. 45 refuses 2, so 3 runs: 45 / 3 = 15 and 15 / 3 = 5, the pair (3, 2).
3. The loop stops before 5 because 3 × 3 = 9 sits past the remainder, so the survivor appends itself as (5, 1).
4. The expanded list reads 2, 2, 2, 3, 3, 5, pinned, and the product rebuilds 360.
5. 1024 peels ten twos into the single pair (2, 10), 97 appends itself, and 1 appends nothing, all pinned.
6. Python's 999 983 rides the frontier to the end: 999 × 999 = 998 001 still inside, 1000 × 1000 = 1 000 000 past it, and the survivor returns as (999 983, 1), a prime proven by exhaustion.

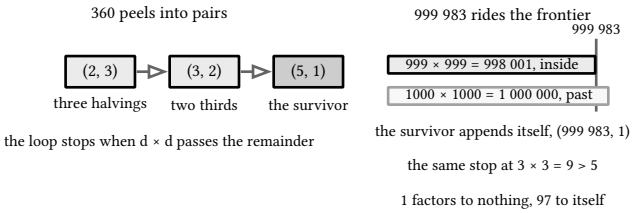


figure 127: the pairs landing in peel order beside the frontier, 360 stops early, 999 983 rides to the bound

The 2, 2, 2, 3, 3, 5 of 360 is the pinned peel, and the listings below divide seven ways.

```c
// factor n into (prime, exponent) pairs, primes ascending; returns count
static int factorize(unsigned long long x, PF *out) {
  int k = 0;
  for (unsigned long long d = 2; d * d ≤ x; d++)
    if (x % d == 0) {
      out[k].p = (int)d;
      out[k].e = 0;
      while (x % d == 0) {
        x /= d;
        out[k].e++;
      }
      k++;
    }
  if (x > 1) {
    out[k].p = (int)x;
    out[k].e = 1;
    k++;
  }
  return k;
}

static int expand(const PF *pf, int k, unsigned long long *out) {
  int m = 0;
  for (int i = 0; i < k; i++)
    for (int j = 0; j < pf[i].e; j++)
      out[m++] = (unsigned long long)pf[i].p;
  return m;
}
```

listing 470: c, the (prime, exponent) pairs, then the expanded list (source: dsa/samples-c/src/Ch16/factors.c, lines 20-47)

```go
// Factorize factors n > 1 by trial division: strip each prime as many
// times as it divides, and stop early once the leftover factor is
// smaller than the square of the next candidate (it must be prime).
func Factorize(n int64) []PrimePower {
  var out []PrimePower
  for p := int64(2); p*p <= n; p++ {
    if n%p != 0 {
      continue
    }
    power := 0
    for n%p == 0 {
      n /= p
      power++
    }
    out = append(out, PrimePower{Prime: p, Power: power})
  }
  if n > 1 {
    out = append(out, PrimePower{Prime: n, Power: 1})
  }
  return out
}

// PrimeListWithMultiplicity flattens the factorization into a sorted
// prime list: 360 becomes [2 2 2 3 3 5].
func PrimeListWithMultiplicity(n int64) []int64 {
  var out []int64
  for _, pp := range Factorize(n) {
    for i := 0; i < pp.Power; i++ {
      out = append(out, pp.Prime)
    }
  }
  return out
}
```

listing 471: go, trial division with the early stop, then the flat list (source: dsa/samples-go/ch16/factors.go, lines 9-41)

```java
record Pf(long p, int e) {}

// factor x into (prime, exponent) pairs, primes ascending
static List<Pf> factorize(long x) {
  List<Pf> out = new ArrayList<>();
  for (long d = 2; d * d ≤ x; d++)
    if (x % d == 0) {
      int e = 0;
      while (x % d == 0) {
        x /= d;
        e++;
      }
      out.add(new Pf(d, e));
    }
  if (x > 1)
    out.add(new Pf(x, 1));
  return out;
}

// the expanded list with each prime repeated by its exponent
static List<Long> expand(List<Pf> pf) {
  List<Long> out = new ArrayList<>();
  for (Pf f : pf)
    for (int j = 0; j < f.e(); j++)
      out.add(f.p());
  return out;
}
```

listing 472: java, a record carries each pair, the peel loop, the expanded list (source: dsa/samples-java/src/Ch16/Factors.java, lines 18-44)

```csharp
/// <summary>Trial division factorization: (prime, exponent) pairs in ascending prime order.</summary>
public static List<(long Prime, int Exp)> Factors(long n)
{
    var parts = new List<(long, int)>();
    for (long p = 2; p * p ≤ n; p++)
        if (n % p == 0)
        {
            int e = 0;
            while (n % p == 0)
            {
                n /= p;
                e++;
            }
            parts.Add((p, e));
        }
    if (n > 1)
        parts.Add((n, 1));
    return parts;
}

/// <summary>Primes of n with multiplicity, sorted ascending, so their product is n.</summary>
public static List<long> PrimeList(long n) =>
    Factors(n).SelectMany(pe => Enumerable.Repeat(pe.Prime, pe.Exp)).ToList();
```

listing 473: c#, strip each prime, flatten to the sorted list (source: dsa/samples/src/Ch16/NumTheory.cs, lines 77-99)

```
export function factorize(n) {
  const out = [];
  let rest = n;
  for (let p = 2; p * p ≤ rest; p++) {
    let e = 0;
    while (rest % p === 0) {
      rest /= p;
      e++;
    }
    if (e) out.push({ p, e });
  }
  if (rest > 1) out.push({ p: rest, e: 1 });
  return out;
}

// e.g. 360 → [2, 2, 2, 3, 3, 5]
export function primeListWithMultiplicity(n) {
  const out = [];
  for (const { p, e } of factorize(n)) for (let i = 0; i < e; i++) out.push(p);
  return out;
}
```

listing 474: javascript, pairs and list, everything under 2^53 (source: dsa/samples-js/src/ch16-factors.mjs, lines 8-28)

```
def prime_factors(x):
    out = []
    d = 2
    while d * d ≤ x:  # any survivor past the loop is itself prime
        while x % d == 0:
            out.append(d)
            x //= d
        d += 1 if d == 2 else 2  # after 2, odd divisors only
    if x > 1:
        out.append(x)
    return out


def factor_powers(x):
    primes = prime_factors(x)
    out = []
    for p in sorted(set(primes)):
        out.append((p, primes.count(p)))
    return out
```

listing 475: python, odd divisors after 2, powers by counting (source: dsa/samples-py/src/Ch16/factors.py, lines 13-31)

```
local function factorize(x)
  local out = {}
  local d = 2
  while d * d ≤ x do
    if x % d == 0 then
      local e = 0
      while x % d == 0 do
        x = x // d
        e = e + 1
      end
      out[#out + 1] = {p = d, e = e}
    end
    d = d + 1
  end
  if x > 1 then out[#out + 1] = {p = x, e = 1} end
  return out
end

-- the expanded prime list with multiplicity
```

listing 476: lua, the peel loop, survivor prime appended (source: dsa/samples-lua/ch16_factors.lua, lines 6-24)

The 360 anchor, three twos, two threes, one five, holds in every suite, as do the boundary cases: a prime factors to itself, 1 factors to nothing, and 1024 is ten twos. The rebuild property, multiplying the expanded list back into the input, is asserted wherever the list exists. Python and Java add the discriminator between the variants, 999 983 surviving trial division as a big prime, and JavaScript's file carries the boundary comment: the loop stops at the square root, so Number stays exact for these fixtures, and reconstructed products in the tests stay below 2^53.

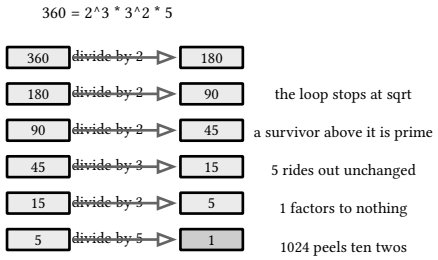


figure 128: 360 peels under trial division, the survivor past the square root is prime

16.4. modular exponentiation

Square-and-multiply computes base to the exponent mod m by reading the exponent in binary: square at every bit, multiply into the result only when the bit is set, reduce after every operation. One hundred multiplications collapse to seven squarings and three multiplies, and the anchor lands everywhere: 2^100 mod 1e9+7 is 976371285 in all seven suites. The chapter's hard problem is hiding inside that innocent sentence, because squaring a residue near 1e9 makes a product near 1e18, and every language draws its own line there.

The dry run: the anchor is 2^100 mod 1e9+7 = 976371285, pinned by all seven suites, the ladder walks C's small pin 3^4 mod 10^6 = 81, and the identity family is the C# suite's.

1. The exponent reads in binary from the bottom: 4 is 100, one set bit at the top, two clear bits below it.
2. Two clear bits mean two bare squarings: the base runs 3 × 3 = 9, then 9 × 9 = 81.
3. The set bit multiplies, 1 × 81 = 81, the pinned answer, then the loop squares once more into 81 × 81 = 6561 and stops.
4. The same machine at 2^100 costs 7 squarings and 3 multiplies, every product reduced in flight, and lands 976371285 everywhere.
5. The identity family: exponent 0 returns 1, exponent 1 the base, and modulus 1 collapses even 5^10 to 0.

6. The wrap story C and Lua share: 10^9 sits at −7 mod the prime, so 6e9 sits at 6 × (-7) = −42, and (-42) × (-42) = 1764, the pinned mulmod.

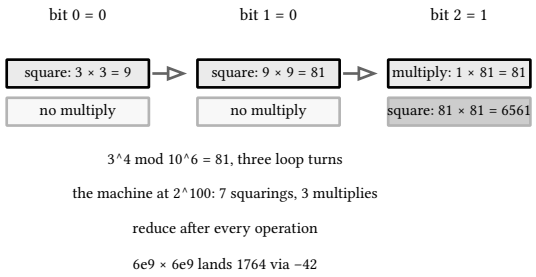


figure 129: the ladder on 3^4, one frame per exponent bit, every frame squares, only the set bit multiplies

The 976371285, pinned seven times over, is the chapter's anchor number, and the listings below climb the ladder seven ways.

```c
// (a * b) % m by binary decomposition of b: add the doubled a into the
// result on set bits, reduce after every addition
static unsigned long long mulmod(unsigned long long a, unsigned long long b,
                                 unsigned long long m) {
  unsigned long long result = 0;
  a %= m;
  while (b) {
    if (b & 1)
      result = (result + a) % m;
    a = (a + a) % m;
    b >>= 1;
  }
  return result;
}

// square-and-multiply, consuming exponent bits from the bottom
static unsigned long long modpow(unsigned long long base,
                                 unsigned long long exp, unsigned long long m) {
  unsigned long long result = 1 % m;
  base %= m;
  while (exp) {
    if (exp & 1)
      result = mulmod(result, base, m);
    base = mulmod(base, base, m);
    exp >>= 1;
  }
  return result;
}
```

listing 477: c, mulmod by add-and-double after the `__uint128` link failure (source: dsa/samples-c/src/Ch16/modpow.c, lines 21-48)

```
// MulMod returns a*b mod m through the 128-bit product math/bits
// hands back. Valid whenever a, b < m: the high word of the product
// then stays below m, inside bits.Div64's precondition.
func MulMod(a, b, m uint64) uint64 {
  hi, lo := bits.Mul64(a, b)
  _, r := bits.Div64(hi, lo, m)
  return r
}

// ModPow computes base^exp mod m by square-and-multiply: square every
// bit, multiply into the result only when the bit is set. O(log exp)
// multiplications, each reduced through MulMod.
func ModPow(base, exp, m uint64) uint64 {
  if m == 1 {
    return 0
  }
  result := uint64(1)
  base %= m
  for exp > 0 {
    if exp&1 == 1 {
      result = MulMod(result, base, m)
    }
    base = MulMod(base, base, m)
    exp >>= 1
  }
  return result
}
```

listing 478: go, the 128-bit product from bits.Mul64, bits.Div64 (source: dsa/samples-go/ch16/modpow.go, lines 5-31)

```java
// (a * b) % m over the unsigned 128-bit product: multiplyHigh holds
// the high word, the loop divides hi:lo by m bit by bit, one compare
// and one subtraction per step because the running remainder stays
// below m so at most one m leaves per doubled value
static long mulmod(long a, long b, long m) {
  long hi = Math.multiplyHigh(a, b);
  long lo = a * b;
  if (hi == 0)
    return Long.remainderUnsigned(lo, m);
  long r = 0;
  for (int i = 127; i ≥ 0; i--) {
    boolean carry = (r & Long.MIN_VALUE) ≠ 0;
    r <<= 1;
    if ((i ≥ 64 ? hi & (1L << (i - 64)) : lo & (1L << i)) ≠ 0)
      r |= 1;
    if (carry || Long.compareUnsigned(r, m) ≥ 0)
      r -= m;
  }
  return r;
}

// square-and-multiply, consuming exponent bits from the bottom
static long modpow(long base, long exp, long m) {
  long result = 1 % m;
  base = Long.remainderUnsigned(base, m);
  while (exp ≠ 0) {
    if ((exp & 1) == 1)
      result = mulmod(result, base, m);
    base = mulmod(base, base, m);
    exp >>>= 1;
  }
  return result;
}

static long ref(long base, long exp, long m) {
  return BigInteger.valueOf(base)
      .modPow(BigInteger.valueOf(exp), BigInteger.valueOf(m))
      .longValue();
}
```

listing 479: java, the real 128-bit product from Math.multiplyHigh, an unsigned shift-subtract remainder, modPow the referee (source: dsa/samples-java/src/Ch16/Modpow.java, lines 23-61)

```
    /// <summary>
    /// Square-and-multiply. Requires 1 &lt;= m &lt;= 3_037_000_499 so residue products fit in a
    /// long, and e &gt;= 0. The base may be negative and is reduced first.
    /// </summary>
    public static long ModPow(long b, long e, long m)
    {
        if (m ≤ 0)
            throw new ArgumentOutOfRangeException(nameof(m), "modulus must be positive");
        if (e < 0)
            throw new ArgumentOutOfRangeException(nameof(e), "exponent must be non-negative");
        long result = 1;
        b %= m;
        if (b < 0)
            b += m;
        while (e > 0)
        {
            if ((e & 1) == 1)
                result = result * b % m;
            b = b * b % m; // one squaring per step, never a tall tower
            e >>= 1;
        }
        return result;
    }
```

listing 480: c#, plain long products, modulus ceiling stated in the contract (source: dsa/samples/src/Ch16/NumTheory.cs, lines 101-123)

```
export function modPow(base, exp, mod) {
  let result = 1n % mod;
  let b = ((base % mod) + mod) % mod;
  let e = exp;
  while (e > 0n) {
    if (e & 1n) result = (result * b) % mod;
    b = (b * b) % mod;
    e >>= 1n;
  }
  return result;
}
```

listing 481: javascript, everything BigInt, exact and unbounded (source: dsa/samples-js/src/ch16-modpow.mjs, lines 6-16)

```
def mod_pow(base, exp, mod):
    result = 1
    base %= mod
    while exp:
        if exp & 1:  # this bit is set: fold in the current square
            result = result * base % mod
        base = base * base % mod  # square for the next bit up
        exp >>= 1
    return result
```

listing 482: python, nine lines, the built-in pow as ground truth (source: dsa/samples-py/src/Ch16/modpow.py, lines 13-21)

```lua
local function wraps_silently()
  return math.maxinteger * 2 == -2
end

-- (a * b) % m by binary decomposition of b: add the doubled a into the
-- result on set bits, reduce after every addition
local function mulmod(a, b, m)
  local result = 0
  a = a % m
  while b > 0 do
    if b & 1 == 1 then result = (result + a) % m end
    a = (a + a) % m
    b = b >> 1
  end
  return result
end

-- square-and-multiply, consuming exponent bits from the bottom
local function modpow(base, exp, m)
  local result = 1 % m
  base = base % m
  while exp > 0 do
    if exp & 1 == 1 then result = mulmod(result, base, m) end
    base = mulmod(base, base, m)
    exp = exp >> 1
  end
  return result
end
```

listing 483: lua, the wrap proof beside the doubling mulmod (source: dsa/samples-lua/ch16_modpow.lua, lines 11-38)

WARNING: five roads through one multiplication

C probed `__uint128_t` first: it compiles, but the link step needs compiler-rt's `__umodti3` helper for a 128-bit remainder and this toolchain does not ship it, so the landed mulmod adds and doubles with a remainder after every step and no product of two 64-bit values ever forms. Go gets the same 128 bits as a first-class operation, `math/bits.Mul64` hands back the high and low words and `bits.Div64` reduces them. Java is the road C could not take, actually run: `Math.multiplyHigh` yields the 128-bit product's high word, the remainder comes from a 128-step unsigned shift-subtract loop because the language has no 128-bit divide, and `BigInteger.modPow` referees every leg. C# states a precondition instead: the modulus stays at or below 3 037 000 499, the square root of the `long` range, so residue products always fit. JavaScript and Lua sit at the extremes, one escapes to BigInt, the other proves `math.maxinteger * 2 == -2` in a test and then never multiplies two big values again.

Every suite also pins the identity family, exponent zero gives 1, exponent one gives the base, modulus 1 collapses everything to 0, and Fermat's little theorem, 2 to the 1e9+6 is 1 mod 1e9+7. C and Lua share one more fixture worth reading: 6e9 times 6e9 wraps any 64-bit product, but since 1e9 is congruent to minus 7 mod 1e9+7, the reduced answer is 36 times 49, which is 1764, and the add-and-double mulmod lands it, Java's 128-bit mulmod landing the same 1764 with the product fully formed. Python cross-checks the whole sweep against the built-in three-argument `pow`, the ground-truth move that language allows, and Java makes the same move with `BigInteger.modPow` over a base, exponent, and modulus grid.

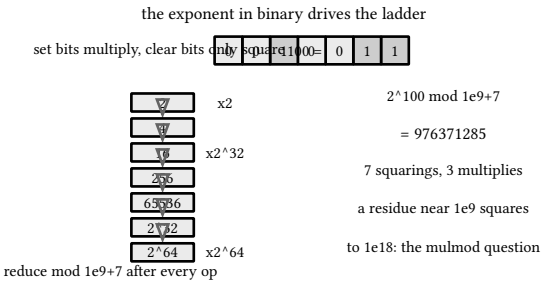


figure 130: square and multiply on 2^100, seven squarings, three multiplies, every product reduced

16.5. modular inverses

Division mod m is multiplication by the inverse, and the inverse exists exactly when a and m share no factor. Two roads compute it: Fermat's little theorem, a to the p minus 2 mod a prime p, and the extended euclidean algorithm, which works for any coprime modulus and hands back the Bezout coefficients as a bonus, a times x plus m times k equals 1 means a times x is 1 mod m.

The dry run: the fixtures are the small inverses asserted by the C# suite, 3 to 4 mod 11, 10 to 12 mod 17, 7 to 15 mod 26, the big inverse 333333336 pinned by Lua, and C pins the bezout bonus on (240, 46).

1. The check is multiply back to 1: 3 × 4 = 12, and 12 - 11 = 1 mod 11.
2. 10 × 12 = 120, and 120 - 119 = 1 mod 17, with 7 × 17 = 119.
3. 7 × 15 = 105, and 105 - 104 = 1 mod 26, with 4 × 26 = 104.
4. The bezout bonus: C reads x = −9 and y = 47 off (240, 46), with 240 × (-9) + 46 × 47 = 2, the gcd.
5. The big inverse: 3 × 333333336 = 1000000008, and 1000000008 - 1000000007 = 1, Lua's pin multiplied back.
6. The guard fires first: gcd(6, 9) = 3, so 6 has no inverse mod 9, and every suite's refusal channel answers before any arithmetic.

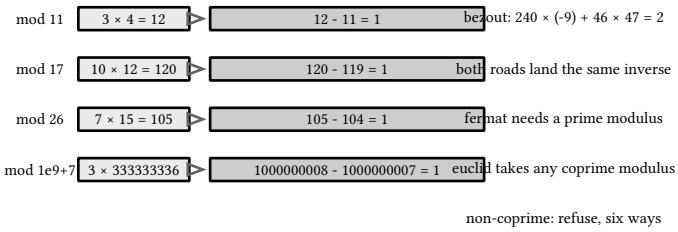


figure 131: four multiply-backs in run order, every inverse times its base landing on 1 over its modulus

The 333333336 multiplied back to 1 is the biggest pin, and the listings below invert seven ways.

```c
// inverse mod a prime: a^(p-2)
static unsigned long long inv_fermat(unsigned long long a,
                                     unsigned long long p) {
  return modpow(a, p - 2, p);
}

// extended euclid over the coefficients; returns gcd, fills x, y with
// a*x + b*y == gcd
static unsigned long long ext_gcd(unsigned long long a, unsigned long long b,
                                  long long *x, long long *y) {
  long long x0 = 1, x1 = 0, y0 = 0, y1 = 1;
  while (b) {
    unsigned long long q = a / b;
    unsigned long long t = a % b;
    a = b;
    b = t;
    long long tx = x0 - (long long)q * x1;
    x0 = x1;
    x1 = tx;
    long long ty = y0 - (long long)q * y1;
    y0 = y1;
    y1 = ty;
  }
  *x = x0;
  *y = y0;
  return a;
}

static unsigned long long gcd_(unsigned long long a, unsigned long long b) {
  while (b) {
    unsigned long long t = a % b;
    a = b;
    b = t;
  }
  return a;
}

// inverse for any modulus; 0 when gcd(a, m) != 1
static unsigned long long inv_ext(unsigned long long a, unsigned long long m) {
  if (gcd_(a, m) != 1)
    return 0;
  long long x, y;
  ext_gcd(a, m, &x, &y);
  long long r = x % (long long)m;
  if (r < 0)
    r += (long long)m;
  return (unsigned long long)r;
}
```

listing 484: c, fermat inverse, extended euclid, zero on non-coprime (source: dsa/samples-c/src/Ch16/modinv.c, lines 46-93)

```
// ModInvPrime inverts a modulo a prime p by Fermat's little theorem:
// a^(p-2) mod p. The guard refuses multiples of p, which have no
// inverse.
func ModInvPrime(a, p uint64) (uint64, error) {
  if p <= 1 || a%p == 0 {
    return 0, fmt.Errorf("%d has no inverse mod %d", a, p)
  }
  return ModPow(a, p-2, p), nil
}

// ExtGCD returns g = gcd(a, b) and the Bezout coefficients with
// a*x + b*y = g.
func ExtGCD(a, b int64) (g, x, y int64) {
  x, y = 1, 0 // invariant: a*x + b*y tracks the leading operand
  rx, ry := int64(0), int64(1)
  for b ≠ 0 {
    q := a / b
    a, b = b, a-q*b
    x, rx = rx, x-q*rx
    y, ry = ry, y-q*ry
  }
  return a, x, y
}

// ModInvEuler inverts a modulo any m through the extended Euclid
// algorithm: a*x + m*k = 1 means a*x = 1 (mod m). The guard refuses
// anything not coprime with m.
func ModInvEuler(a, m int64) (int64, error) {
  if m <= 0 {
    return 0, fmt.Errorf("modulus %d not positive", m)
  }
  g, x, _ := ExtGCD(((a%m)+m)%m, m)
  if g ≠ 1 {
    return 0, fmt.Errorf("%d and %d share factor %d: no inverse", a, m, g)
  }
  return ((x % m) + m) % m, nil
}
```

listing 485: go, both roads, errors carry the refusal (source: dsa/samples-go/ch16/modinv.go, lines 5-41)

```java
// (a * b) % m: with m ~ 1e9 the product fits a signed long, so plain
// multiply-and-reduce is exact here, no 128-bit path needed
static long mulmod(long a, long b, long m) {
  return (a % m) * (b % m) % m;
}

// square-and-multiply over mulmod
static long modpow(long base, long exp, long m) {
  long result = 1 % m;
  base %= m;
  while (exp != 0) {
    if ((exp & 1) == 1)
      result = mulmod(result, base, m);
    base = mulmod(base, base, m);
    exp >>= 1;
  }
  return result;
}

// inverse mod a prime: a^(p-2)
static long invFermat(long a, long p) {
  return modpow(a, p - 2, p);
}

// extended euclid over the coefficients; returns gcd, x and y with
// a*x + b*y == gcd via the returned array
static long[] extGcd(long a, long b) {
  long x0 = 1, x1 = 0, y0 = 0, y1 = 1;
  while (b != 0) {
    long q = a / b;
    long t = a % b;
    a = b;
    b = t;
    long tx = x0 - q * x1;
    x0 = x1;
    x1 = tx;
    long ty = y0 - q * y1;
    y0 = y1;
    y1 = ty;
  }
  return new long[] {a, x0, y0};
}

// inverse for any modulus; 0 when gcd(a, m) != 1
static long invExt(long a, long m) {
  if (gcd(a, m) != 1)
    return 0;
  long[] g = extGcd(a % m, m);
  long r = g[1] % m;
  if (r < 0)
    r += m;
  return r;
}
```

listing 486: java, plain long mulmod with the ceiling in the comment, bezout visible, zero refuses (source: dsa/samples-java/src/Ch16/Modinv.java, lines 27-79)

```
    /// <summary>Extended Euclid: (g, x, y) with a*x + b*y = g = gcd(a, b).</summary>
    public static (long G, long X, long Y) ExtGcd(long a, long b)
    {
        (long oldR, long r) = (a, b);
        (long oldS, long s) = (1, 0);
        (long oldT, long t) = (0, 1);
        while (r != 0)
        {
            long q = oldR / r;
            (oldR, r) = (r, oldR - q * r);
            (oldS, s) = (s, oldS - q * s);
            (oldT, t) = (t, oldT - q * t);
        }
        return (oldR, oldS, oldT);
    }

    /// <summary>Inverse via Fermat's little theorem, modulus must be prime.</summary>
    public static long ModInvFermat(long a, long prime) => ModPow(a, prime - 2, prime);

    /// <summary>Inverse via extended Euclid, works for any coprime modulus, throws otherwise.</summary>
    public static long ModInv(long a, long m)
    {
        (long g, long x, _) = ExtGcd(a % m, m);
        if (g != 1)
            throw new InvalidOperationException($"{a} has no inverse mod {m}, gcd is {g}");
        long inv = x % m;
        return inv < 0 ? inv + m : inv;
    }
```

listing 487: c#, the bezout bookkeeping, throws when no inverse exists (source: dsa/samples/src/Ch16/NumTheory.cs, lines 125-152)

```
export function extGcd(a, b) {
  if (b === 0n) return { g: a, x: 1n, y: 0n };
  const { g, x, y } = extGcd(b, a % b);
  return { g, x: y, y: x - (a / b) * y };
}

// inverse mod a prime p, via a^(p-2); null when a is a multiple of p
export function modInvPrime(a, p) {
  if (gcd(a % p, p) !== 1n) return null;
  return modPow(((a % p) + p) % p, p - 2n, p);
}

// inverse mod any modulus, via extended euclid; null when not coprime
export function modInvExt(a, m) {
  if (m <= 0n) return null;
  const { g, x } = extGcd(((a % m) + m) % m, m);
  if (g !== 1n) return null;
  return ((x % m) + m) % m;
}
```

listing 488: javascript, recursive extGcd on BigInt, null refuses (source: dsa/samples-js/src/ch16-modinv.mjs, lines 11-29)

```python
def inverse_fermat(a, prime):
    # a^(p-2) mod p is the inverse when p is prime and p does not divide a
    if a % prime == 0:
        return None
    return mod_pow(a, prime - 2, prime)


def inverse_euclid(a, m):
    # extended euclid: keep the bezout coefficient of a
    old_r, r = a % m, m
    old_s, s = 1, 0
    while r:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
    if old_r != 1:  # gcd(a, m) != 1: no inverse exists
        return None
    return old_s % m
```

listing 489: python, fermat and euclid, None refuses (source: dsa/samples-py/src/Ch16/modinv.py, lines 25-42)

```lua
-- fermat: a^(p-2) mod p, only valid for a prime modulus
local function modinv_fermat(a, p)
  return modpow(a, p - 2, p)
end

-- extended euclid: returns g, x, y with a*x + b*y = g = gcd(a, b)
local function ext_gcd(a, b)
  if b == 0 then return a, 1, 0 end
  local g, x, y = ext_gcd(b, a % b)
  return g, y, x - (a // b) * y
end

-- inverse of a mod m, or nil when gcd(a, m) ~= 1
local function modinv(a, m)
  local g, x = ext_gcd(a, m)
  if g ~= 1 then return nil end
  return ((x % m) + m) % m
end
```

listing 490: lua, fermat beside euclid, nil refuses (source: dsa/samples-lua/ch16_modinv.lua, lines 30-47)

The fixture families agree on the small primes, 3 inverts to 4 mod 11 and 10 inverts to 12 mod 17, and Lua pins the big one, 3 inverse mod 1e9+7 is 333333336, verified by multiplying back to 1. C, Java, and Python sweep every unit mod 97 and check both roads agree. The refusal channel is the per-language story: C and Java return 0, Go returns an error naming the shared factor, JavaScript and Python return null and None, Lua returns nil, and C# throws `InvalidOperationException`. Six conventions across seven suites, one mathematical fact, a non-coprime pair has no inverse, and the gcd guard fires before any of them is reached. Java's inverse file also drops the chapter's 128-bit mulmod on purpose: with the modulus near 1e9 every residue product stays under 2^63, so a plain multiply-and-reduce is exact, the ceiling stated in the comment the way C# states its contract.

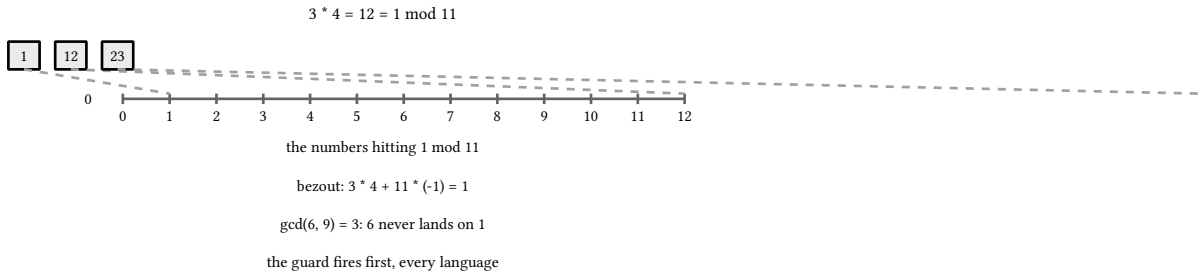


figure 132: the inverse is the bezout coefficient, a times x plus m times k equals 1

16.6. the chinese remainder theorem

Two congruences, x = 2 mod 3 and x = 3 mod 5, describe one class mod 15 when the moduli are coprime, and the theorem folds a whole list pairwise: combine two into one residue over the lcm, then bring in the next. The classic three-congruence fixture, 2 mod 3, 3 mod 5, 2 mod 7, lands on 23 mod 105 in every suite.

The dry run: the fixture is the sun tzu triple, 2 mod 3, 3 mod 5, 2 mod 7, pinned at 23 mod 105 by every suite, the walk follows the C# pairwise fold with each lift spelled as the steps C's search would take, and Python refuses the non-coprime merge the other six accept.

1. Fold 2 mod 3 with 3 mod 5: the moduli share nothing, and the lift steps the first modulus twice, 2 + 3 = 5 then 5 + 3 = 8, where 8 mod 5 = 3 satisfies the second: 8 mod 15.
2. Fold 8 mod 15 with 2 mod 7: the lift needs one step, 8 + 15 = 23, and 23 mod 7 = 2 lands 23 mod 105, the pinned class.
3. The residue check reads 23 mod 3 = 2 and 23 mod 5 = 3, both quoted in the C suite.
4. The non-coprime merge, 1 mod 6 with 3 mod 10: the moduli share 2, the residues differ by 3 - 1 = 2, divisible, so the lift runs 1 + 6 = 7, 7 + 6 = 13, and 13 mod 10 = 3 lands 13 mod 30.
5. The contradiction refuses: 1 mod 6 against 2 mod 10 differs by 1, not divisible by the shared 2, so every suite rejects the system, Go and C# naming the shared factor in the refusal.

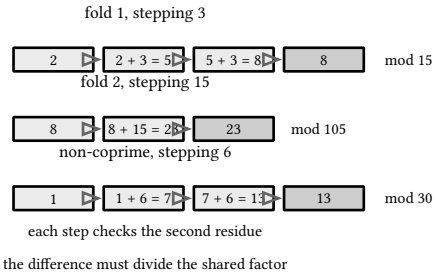


figure 133: the fold as stepped lifts, two coprime folds reaching 23 mod 105, the non-coprime merge reaching 13 mod 30

The 23 mod 105 is the pinned fold, and the listings below combine seven ways.

```c
// combine (r1, m1) and (r2, m2): returns 1 and fills *r, *m on success
static int crt_combine(unsigned long long r1, unsigned long long m1,
                       unsigned long long r2, unsigned long long m2,
                       unsigned long long *r, unsigned long long *m) {
  unsigned long long g = gcd_(m1, m2);
  if ((r2 + m2 - r1 % m1) % m1 % g != 0)
    return 0; // residues disagree modulo the gcd
  unsigned long long l = lcm_(m1, m2);
  // search by stepping the first congruence: fine for the book's sizes
  for (unsigned long long x = r1 % m1; x < l; x += m1)
    if (x % m2 == r2 % m2) {
      *r = x;
      *m = l;
      return 1;
    }
  return 0;
}
```

listing 491: c, the consistency check, then a step search over the lcm (source: dsa/samples-c/src/Ch16/crt.c, lines 31-47)

```go
// CRT combines congruences pairwise. Coprime moduli combine into a
// unique class modulo the product; moduli sharing a factor combine
// only when the residues agree modulo that shared factor, and the
// combined modulus becomes the lcm. Unsatisfiable systems are refused
// with an error.
func CRT(cs []Congruence) (x, combined int64, err error) {
  if len(cs) == 0 {
    return 0, 0, fmt.Errorf("no congruences to combine")
  }
  for _, c := range cs {
    if c.Mod <= 0 {
      return 0, 0, fmt.Errorf("modulus %d not positive", c.Mod)
    }
  }
  x = ((cs[0].Rem % cs[0].Mod) + cs[0].Mod) % cs[0].Mod
  combined = cs[0].Mod
  for _, c := range cs[1:] {
    cRem := ((c.Rem % c.Mod) + c.Mod) % c.Mod
    g, p, _ := ExtGCD(combined, c.Mod)
    diff := cRem - x
    if diff%g != 0 {
      return 0, 0, fmt.Errorf(
        "%d mod %d and %d mod %d disagree on the shared factor %d",
        x, combined, cRem, c.Mod, g)
    }
    lcm := combined / g * c.Mod
    // p inverts combined/g modulo c.Mod/g, so t moves x to the
    // next residue: x + combined*t = cRem (mod c.Mod)
    m2g := c.Mod / g
    t := (p % m2g) * ((diff / g) % m2g) % m2g
    t = (t%m2g + m2g) % m2g
    x = ((x+combined*t)%lcm + lcm) % lcm
    combined = lcm
  }
  return x, combined, nil
}
```

listing 492: go, extGCD drives the lift, the error names the shared factor (source: dsa/samples-go/ch16/crt.go, lines 10-45)

```
record Combined(long r, long m) {}

// combine (r1, m1) and (r2, m2); null when the residues disagree
// modulo the gcd of the moduli
static Combined crtCombine(long r1, long m1, long r2, long m2) {
  long g = gcd(m1, m2);
  if ((r2 + m2 - r1 % m1) % m1 % g != 0)
    return null;
  long l = lcm(m1, m2);
  // search by stepping the first congruence: fine for the book's sizes
  for (long x = r1 % m1; x < l; x += m1)
    if (x % m2 == r2 % m2)
      return new Combined(x, l);
  return null;
}

static Combined crt(List<long[]> congruences) {
  Combined acc = new Combined(0, 1);
  for (long[] c : congruences) {
    acc = crtCombine(acc.r(), acc.m(), c[0], c[1]);
    if (acc == null)
      return null;
  }
  return acc;
}
```

listing 493: java, the step search over the lcm like C, a record pair back, null refuses (source: dsa/samples-java/src/Ch16/Crt.java, lines 32-56)

```
  /// <summary>
  /// Chinese remainder theorem: fold congruences (r, m) pairwise. Consistent non-coprime
  /// moduli are merged via gcd; contradictions throw. Moduli products must fit in a long.
  /// </summary>
  public static (long Remainder, long Modulus) Crt(IEnumerable<(long R, long M)> items)
  {
      IEnumerator<(long R, long M)> it = items.GetEnumerator();
      if (!it.MoveNext())
          throw new ArgumentException("at least one congruence required", nameof(items));
      (long r, long m) = it.Current;
      while (it.MoveNext())
      {
          (long r2, long m2) = it.Current;
          long g = Gcd(m, m2);
          if ((r2 - r) % g != 0)
              throw new InvalidOperationException(
                  $"no solution: {r} mod {m} and {r2} mod {m2} disagree mod {g}");
          long lcm = m / g * m2;
          (_, long p, _) = ExtGcd(m / g, m2 / g); // (m/g)*p + (m2/g)*q = 1
          long t = (r2 - r) / g % (m2 / g) * p % (m2 / g);
          if (t < 0)
              t += m2 / g;
          r = (r + m * t) % lcm;
          m = lcm;
      }
      return (r, m);
  }
```

listing 494: c#, pairwise fold, non-coprime merge, contradictions throw (source: dsa/samples/src/Ch16/NumTheory.cs, lines 154-180)

```javascript
export function crt(system) {
  if (system.length === 0) return null;
  let { r: x, m: mod } = system[0];
  if (mod ≤ 0n) return null;
  for (const { r, m } of system.slice(1)) {
    if (m ≤ 0n) return null;
    const g = gcd(mod, m);
    const diff = r - x;
    if (diff % g =/= 0n) return null; // disagree on the shared factor
    const lcmv = (mod / g) * m;
    // invert mod/g modulo m/g: coprime by construction, so non-coprime
    // systems combine through the lcm instead of the product
    const mOverG = m / g;
    const inv = modInvExt((mod / g) % mOverG, mOverG);
    const t = (((diff / g) % mOverG) * inv) % mOverG;
    x = x + mod * t;
    mod = lcmv;
    x = ((x % mod) + mod) % mod;
  }
  return { x, mod };
}
```

listing 495: javascript, the lift through the inverse mod m/g, all BigInt (source: dsa/samples-js/src/ch16-crt.mjs, lines 12-32)

```python
def crt(congruences):
    # congruences: (remainder, modulus) pairs; returns (x, combined mod)
    x, mod = 0, 1
    for rem, m in congruences:
        if m ≤ 0:
            return None
        inv = inverse_euclid(mod, m)
        if inv is None:
            # mod and m share a factor: coprime crt does not apply
            # (consistent non-coprime systems exist but stay out of scope)
            return None
        t = (rem - x) * inv % m  # lift x into the new modulus
        x = x + mod * t
        mod = mod * m
    return x % mod, mod
```

listing 496: python, the coprime fold, every non-coprime system refused (source: dsa/samples-py/src/Ch16/crt.py, lines 32-46)

```lua
-- merge two congruences; returns r, m or nil, nil when inconsistent
local function merge(r1, m1, r2, m2)
  local g, p = ext_gcd(m1, m2)
  if (r2 - r1) % g ~= 0 then return nil, nil end
  local lcm = m1 // g * m2
  -- (r2 - r1)/g * p is the multiple of m1 to add, taken mod the lcm
  local t = (r2 - r1) // g % (m2 // g) * p % (m2 // g)
  local r = (r1 + m1 * t) % lcm
  return r, lcm
end

-- fold the congruence list; returns r, m or nil when contradictory
local function crt(congs)
  if #congs == 0 then return nil end
  local r, m = congs[1][1], congs[1][2]
  for i = 2, #congs do
    r, m = merge(r, m, congs[i][1], congs[i][2])
    if r == nil then return nil end
  end
  return r, m
end
```

listing 497: lua, the merge via extended euclid, nil on contradiction (source: dsa/samples-lua/ch16_crt.lua, lines 13-33)

PITFALL: the non-coprime split is real, read your library twice

The suites disagree on purpose about moduli that share a factor. Python refuses every non-coprime system, and its own comment says so: consistent non-coprime systems exist, they combine mod the lcm instead of the product, and the implementation stays out of scope. Go, C#, JavaScript, Java, Lua, and C accept them when the residues agree modulo the shared factor, merging mod the lcm, and reject them when they do not, Go and C# with an error that names the shared factor, JavaScript and Java with null, Lua with nil, C with a return code. The 23 mod 105 anchor holds everywhere, but 1 mod 6 with 3 mod 10 is 13 mod 30 in six suites and a refusal in the seventh. This is the exact shape of bug a shared library papers over and a handbook has to name.

The mechanism under the fold: the difference of residues must be divisible by the gcd of the moduli or no x satisfies both, and when it is, the extended euclid inverse of one modulus over the other, taken mod the shared-factor quotient, sizes the jump. C and Java are the honest exceptions, they step the first congruence through the lcm and look for the second, fine at book sizes and free of any inverse subtlety. The residue bookkeeping returns with real workload sizes where bus schedules are congruences and the answer is a chinese remainder in disguise. The contest form is the same test with the merge dropped: the icpc book's 2023 finals, problem F (tilting tiles), reduces its reachability question to linear congruences checked pairwise, one canonical residue per distinct modulus and the gcd of each pair of moduli dividing the residue difference. The moduli are never combined by lcm there, the accumulator was observed overflowing 64 bits at contest sizes, and existence is the whole answer.

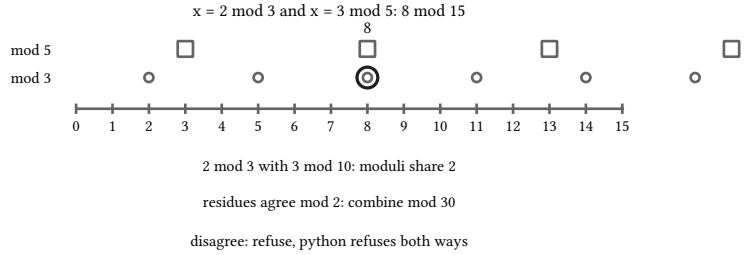


figure 134: two congruence families on one number line, the intersection is one class mod the lcm

16.7. across the seven languages

The build sizes count non-comment source lines over this chapter's six featured files per language, checks included where they share the file:

language	build SLOC	dependency	boundary note
c	407	libc only	`__uint128` compiles but `__umodti3` fails to link, so mulmod adds and doubles with a remainder every step
c#	152	bcl only	plain long products, modulus contract caps at 3 037 000 499 so squares fit, crt throws on contradiction
go	212	math/bits	mulmod through Mul64 and Div64, a real 128-bit product, errors carry the refusals
java	498	jdk 27 stdlib	mulmod runs the true 128-bit product via Math.multiplyHigh with a 128-step unsigned shift-subtract remainder, BigInteger.modPow the referee, modinv drops back to plain long under its 2^63 ceiling
javascript	117	node stdlib	gcd, modpow, modinv, and crt on BigInt, sieve and factors stay on Number under 2^53
python	275	stdlib only	native ints, pow() and math.gcd are ground-truth cross-checks, crt refuses all non-coprime systems
lua	386	lib.lua harness	integers wrap silently, the suite proves math.maxinteger * 2 == −2 then never multiplies big values

sources: learn.microsoft.com, `Math.Abs` and argument-checking guidance for the throwing guards, accessed 2026-09-08, go.dev/pkg/math/bits for `Mul64` and `Div64`, developer.mozilla.org for BigInt arithmetic, accessed 2026-09-14, and the sun tzu remainder problem plus the CLRS number theory treatments cited in the chapter text. Sample behavior verified by `make verify-csharp`, 17 tests in chapter 16 of the samples suite. The seven-language layer verifies the same way: 6 C programs with 119 embedded checks under `make verify-c`, 6 Ch16 java programs with 113 checks under `run-java-samples`, 24 Go tests, 21 `node --test` cases, 63 Python checks across 6 files, and 28 Lua checks under `run.lua`.

17. dynamic programming

Dynamic programming is not a technique so much as a recognition: when a recursive solution recomputes the same subproblems, storing them turns exponential into polynomial. This chapter runs the same recurrences three ways, naive, memoized, tabulated, with call counters making the difference a number, then builds the canonical tables with reconstruction, seven languages deep.

17.1. the three fibonaccis

The naive recursion recomputes subproblems, and the call tree grows like the fibonacci numbers themselves. The meter proves it: fib of 10 makes 177 calls, not 10.

The dry run: the fixture is n = 10 on the C# meter, 177 naive calls for the value 55, with the memo pin 12586269025 at n = 50; C and Python pin 21891 against 39 at n = 20, Go counts only real evaluations at 9, and Lua's memo meter reads 19.

1. The naive meter counts every entry, leaf and interior alike, and the tree at 10 closes at 177 calls for the value 55.
2. The memoized road evaluates each argument once, 0 through 10, and reads the cache after that.
3. The tabulated road runs a pair, one fold per row of the table below, from 0 + 1 = 1 to the register holding 55.
4. All eighty values agree across memo and table, the pin that makes the three roads one function.
5. fib(50) = 12586269025 answers from the memo instantly, the size the naive tree cannot reach.
6. The ladder of 10 rungs is the same recurrence shifted one, and its count is 89.

i	prev	cur	prev + cur
2	0	1	0 + 1 = 1
3	1	1	1 + 1 = 2
4	1	2	1 + 2 = 3
5	2	3	2 + 3 = 5
6	3	5	3 + 5 = 8
7	5	8	5 + 8 = 13
8	8	13	8 + 13 = 21
9	13	21	13 + 21 = 34
10	21	34	21 + 34 = 55

The 177 calls against the pair's ten folds is the recognition in two numbers, and the listings below run all three roads in seven languages.

```c
static long long fib_naive(int k) {
  calls++;
  if (k < 2)
    return k;
  return fib_naive(k - 1) + fib_naive(k - 2);
}

static long long memo_tab[64];
static char memo_seen[64];

static long long fib_memo(int k) {
  calls++;
  if (k < 2)
    return k;
  if (memo_seen[k])
    return memo_tab[k];
  memo_seen[k] = 1;
  return memo_tab[k] = fib_memo(k - 1) + fib_memo(k - 2);
}

static long long fib_table(int k) {
  long long t[64];
  t[0] = 0;
  t[1] = 1;
  for (int i = 2; i <= k; i++)
    t[i] = t[i - 1] + t[i - 2];
  return t[k];
}

// ladder: climb n rungs with steps of 1 or 2, ways(n) = fib(n+1)
static long long ladder(int n) {
  long long a = 1, b = 1; // ways(0) = 1, ways(1) = 1
  for (int i = 2; i <= n; i++) {
    long long c = a + b;
    a = b;
    b = c;
  }
  return b;
}
```

listing 498: c, naive with a call meter, memo arrays, the table, the ladder (source: dsa/samples-c/src/Ch17/memo.c, lines 19-57)

```
// FibNaive recomputes subproblems; calls counts every invocation.
func FibNaive(n int, calls *int64) int64 {
  *calls++
  if n <= 1 {
    return int64(n)
  }
  return FibNaive(n-1, calls) + FibNaive(n-2, calls)
}

// FibMemo recurses with a map memo. Every argument from n down to 0
// is evaluated exactly once, so the evaluation count is linear; cache
// probes are not counted as work.
func FibMemo(n int) (value int64, calls int64) {
  memo := map[int]int64{}
  var walk func(int) int64
  walk = func(k int) int64 {
    if k <= 1 {
      return int64(k)
    }
    if v, ok := memo[k]; ok {
      return v
    }
    calls++ // a real evaluation, not a cache probe
    memo[k] = walk(k-1) + walk(k-2)
    return memo[k]
  }
  return walk(n), calls
}

// FibTable fills the answer left to right: no recursion, no repeats.
func FibTable(n int) int64 {
  if n <= 1 {
    return int64(n)
  }
  table := make([]int64, n+1)
  table[1] = 1
  for k := 2; k <= n; k++ {
    table[k] = table[k-1] + table[k-2]
  }
  return table[n]
}
```

listing 499: go, the meter as a pointer, the memo as a map, real evaluations counted (source: dsa/samples-go/ch17/memo.go, lines 6-46)

```
  static long calls;

  static long fibNaive(int k) {
    calls++;
    if (k < 2)
      return k;
    return fibNaive(k - 1) + fibNaive(k - 2);
  }

  static long[] memoTab = new long[64];
  static boolean[] memoSeen = new boolean[64];

  static long fibMemo(int k) {
    calls++;
    if (k < 2)
      return k;
    if (memoSeen[k])
      return memoTab[k];
    memoSeen[k] = true;
    return memoTab[k] = fibMemo(k - 1) + fibMemo(k - 2);
  }

  static long fibTable(int k) {
    long[] t = new long[64];
    t[0] = 0;
    t[1] = 1;
    for (int i = 2; i ≤ k; i++)
      t[i] = t[i - 1] + t[i - 2];
    return t[k];
  }

  // ladder: climb n rungs with steps of 1 or 2, ways(n) = fib(n+1)
  static long ladder(int n) {
    long a = 1, b = 1; // ways(0) = 1, ways(1) = 1
    for (int i = 2; i ≤ n; i++) {
      long c = a + b;
      a = b;
      b = c;
    }
    return b;
  }
```

listing 500: java, the static meter, seen-flag memo arrays, the table, the ladder (source: dsa/samples-java/src/Ch17/Memo.java, lines 16-56)

```
public static class Dp
{
    public sealed class Meter
    {
        public long Calls { get; set; }
    }

    // naive recursion: calls grow like the fibonacci numbers themselves
    public static long FibNaive(int n, Meter? meter = null)
    {
        if (meter ≠ null)
            meter.Calls++;
        return n ≤ 1 ? n : FibNaive(n - 1, meter) + FibNaive(n - 2, meter);
    }

    // memoized: one call per distinct argument
    public static long FibMemo(int n)
    {
        var memo = new Dictionary<int, long>();
        long Fib(int k)
        {
            if (k ≤ 1)
                return k;
            if (memo.TryGetValue(k, out long v))
                return v;
            return memo[k] = Fib(k - 1) + Fib(k - 2);
        }
        return Fib(n);
    }

    // tabulated: bottom-up, no recursion at all
    public static long FibTab(int n)
    {
        if (n ≤ 1)
            return n;
        long prev = 0, cur = 1;
        for (int i = 2; i ≤ n; i++)
            (prev, cur) = (cur, prev + cur);
        return cur;
    }
```

listing 501: c#, naive with a call meter, memoized, tabulated (source: dsa/samples/src/Ch17/Dp.cs, lines 4-43)

```
export function fibNaive(n, meter) {
  if (meter) meter.calls++;
  if (n < 2) return n;
  return fibNaive(n - 1, meter) + fibNaive(n - 2, meter);
}

export function fibMemo(n, memo = new Map(), meter) {
  if (meter) meter.calls++;
  if (n < 2) return n;
  if (memo.has(n)) return memo.get(n);
  const v = fibMemo(n - 1, memo, meter) + fibMemo(n - 2, memo, meter);
  memo.set(n, v);
  return v;
}

export function fibTab(n) {
  let a = 0;
  let b = 1;
  for (let i = 0; i < n; i++) [a, b] = [b, a + b];
  return a;
}

// the ladder: steps of 1 or 2 rungs, ways(n) = ways(n-1) + ways(n-2),
// which is fibonacci shifted by one
export function ladderTab(n) {
  let a = 1; // ways(0): stand still
  let b = 1; // ways(1)
  for (let i = 2; i ≤ n; i++) [a, b] = [b, a + b];
  return b;
}
```

listing 502: javascript, meter object, Map memo, fib 78 is the exact ceiling (source: dsa/samples-js/src/ch17-memo.mjs, lines 7-36)

```python
def fib_naive(k):
    global naive_calls
    naive_calls += 1
    if k < 2:
        return k
    return fib_naive(k - 1) + fib_naive(k - 2)


def fib_memo(k, table):
    global memo_calls
    memo_calls += 1
    if k < 2:
        return k
    if k not in table:
        table[k] = fib_memo(k - 1, table) + fib_memo(k - 2, table)
    return table[k]


def fib_table(k):
    if k < 2:
        return k
    a, b = 0, 1  # the rolled row: two slots walk up the ladder
    for _ in range(k - 1):
        a, b = b, a + b
    return b


def ladder_ways(rungs):
    # climb 1 or 2 rungs at a time: the same recurrence, new story
    ways = [0] * (rungs + 1)
    ways[0] = 1
    for i in range(1, rungs + 1):
        ways[i] = ways[i - 1] + (ways[i - 2] if i > 1 else 0)
    return ways[rungs]
```

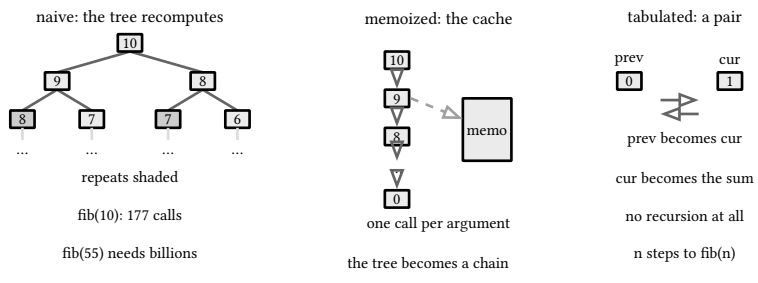
listing 503: python, a dict memo by hand, functools.cache is the named counterpart (source: dsa/samples-py/src/Ch17/memo.py, lines 14-47)

```lua
local function fib_naive(n, meter)
  meter.calls = meter.calls + 1
  if n ≤ 1 then return n end
  return fib_naive(n - 1, meter) + fib_naive(n - 2, meter)
end

local function fib_memo(n, meter)
  local memo = {}
  local function fib(k)
    meter.calls = meter.calls + 1
    if k ≤ 1 then return k end
    if memo[k] then return memo[k] end
    memo[k] = fib(k - 1) + fib(k - 2)
    return memo[k]
  end
  return fib(n)
end

local function fib_tab(n)
  if n ≤ 1 then return n end
  local prev, cur = 0, 1
  for _ = 2, n do
    prev, cur = cur, prev + cur
  end
  return cur
end
```

listing 504: lua, the closure over the memo table, the pair walk (source: dsa/samples-lua/ch17_memo.lua, lines 6-31)

Memoization is top-down with a cache, one call per distinct argument, and the fifty-fifth fibonacci computes instantly where the naive version would need billions of calls. Tabulation is bottom-up, a running pair instead of a table, and the agreement test checks all eighty values match across both. The recognition to carry forward: overlapping subproblems, the cache fixes them, and optimal substructure, the answer composes from subanswers. Both must hold or the memo is just a cache on wrong math.

The call-count anchors line up with one deliberate split in what the meter counts. At n = 10 the naive tree is 177 calls in C#, JavaScript, Go, and Lua, and the memoized run is 19 calls in Lua, every argument touched once. C, Java, and Python pin the bigger instance instead, fib of 20 at 21891 naive calls against 39 memoized, and C and Java add the warm-cache fact, a second memoized run costs exactly 1 call. Go counts only real evaluations, the cache probe is free, so its memo meter reads 9 at n = 10. The ladder recurrence, one or two rungs at a time, is fibonacci shifted by one, and ladder of 10 is 89 wherever it is pinned.



all eighty values agree, the meter is the only difference

figure 135: the same recurrence three ways, the naive tree recomputes, the memo keeps one call per argument, the table keeps a running pair

17.2. grid paths

Grid paths with obstacles is the cleanest 2d recurrence: ways into a cell is ways from above plus ways from the left, walls contribute zero.

The dry run: the fixtures are the open 3 by 3 at 6, the center-wall at 2, and the sealed corner at 0, all asserted by the C# suite; C pins the 4 by 4 double-block table at 4, Lua adds the open 4 by 4 at 20, and Python reruns the count through a memo.

1. The open grid seeds its corner at 1 and the borders follow: row 0 reads 1, 1, 1 and column 0 the same.
2. The interior folds: 1 + 1 = 2 at the center, 2 + 1 = 3 beside it, and the far corner lands 3 + 3 = 6, the choose 4 pick 2 count, pinned.
3. Wall the center: the cell folds in nothing, row 1 reads 1, 0, 1, and the corner recomputes 1 + 1 = 2, the two border routes, pinned.
4. Seal (0, 1) and (1, 0): row 1 reads 1, 0, 0, nothing reaches the far corner, and the answer is 0.
5. The sweep is row-major, one line per cell, so a wall costs nothing extra: it is a cell that contributes zero.

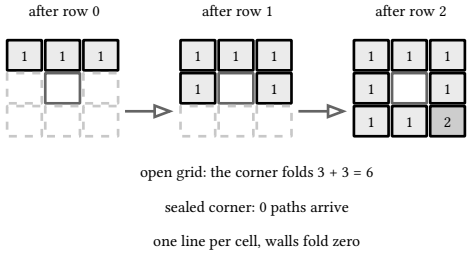


figure 136: the walled 3 by 3 sweeping row by row, the wall contributing zero, the corner landing 2

The 6 against the walled 2 is the pinned pair, and the listings below sweep the grid in seven languages.

```c
static int count_paths(int rows, int cols) {
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++) {
      if (grid[r][c]) {
        dp[r][c] = 0;
        continue;
      }
      if (r == 0 && c == 0) {
        dp[r][c] = 1;
        continue;
      }
      int up = r ? dp[r - 1][c] : 0;
      int left = c ? dp[r][c - 1] : 0;
      dp[r][c] = up + left;
    }
  return dp[rows - 1][cols - 1];
}

static void clear_grid(int rows, int cols) {
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++)
      grid[r][c] = 0;
}
```

listing 505: c, one line per cell, the 4x4 double block table pinned (source: dsa/samples-c/src/Ch17/gridpaths.c, lines 19-41)

```go
// GridPaths counts right-or-down paths from the top-left corner to
// the bottom-right over a grid of blocked cells, mod nothing: the
// numbers fit comfortably in int for the sizes the book cares about.
func GridPaths(blocked [][]bool) int {
  rows := len(blocked)
  if rows == 0 {
    return 0
  }
  cols := len(blocked[0])
  counts := make([][]int, rows)
  for r := range counts {
    counts[r] = make([]int, cols)
  }
  for r := 0; r < rows && !blocked[r][0]; r++ {
    counts[r][0] = 1
  }
  for c := 0; c < cols && !blocked[0][c]; c++ {
    counts[0][c] = 1
  }
  for r := 1; r < rows; r++ {
    for c := 1; c < cols; c++ {
      if blocked[r][c] {
        continue
      }
      counts[r][c] = counts[r-1][c] + counts[r][c-1]
    }
  }
  return counts[rows-1][cols-1]
}
```

listing 506: go, first row and column seeded, then the sweep (source: dsa/samples-go/ch17/gridpaths.go, lines 3-31)

```
static int countPaths(int rows, int cols) {
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++) {
      if (grid[r][c] != 0) {
        dp[r][c] = 0;
        continue;
      }
      if (r == 0 && c == 0) {
        dp[r][c] = 1;
        continue;
      }
      int up = r > 0 ? dp[r - 1][c] : 0;
      int left = c > 0 ? dp[r][c - 1] : 0;
      dp[r][c] = up + left;
    }
  return dp[rows - 1][cols - 1];
}

static void clearGrid(int rows, int cols) {
  for (int r = 0; r < rows; r++)
    for (int c = 0; c < cols; c++)
      grid[r][c] = 0;
}

// the same count through a recursive memo: the table's mirror image,
// the py memo lane
static Map<Long, Integer> table;
static int rowsN, colsN;

static int pathsMemo(int r, int c) {
  if (r == rowsN - 1 && c == colsN - 1)
    return 1;
  long key = (long) r << 32 | c;
  Integer cached = table.get(key);
  if (cached != null)
    return cached;
  int total = 0;
  if (r + 1 < rowsN && grid[r + 1][c] == 0)
    total += pathsMemo(r + 1, c);
  if (c + 1 < colsN && grid[r][c + 1] == 0)
    total += pathsMemo(r, c + 1);
  table.put(key, total);
  return total;
}

static int countPathsMemo(int rows, int cols) {
  rowsN = rows;
  colsN = cols;
  table = new HashMap<>();
  if (grid[0][0] != 0 || grid[rows - 1][cols - 1] != 0)
    return 0;
  return pathsMemo(0, 0);
}
```

listing 507: java, the sweep, then the same count through a HashMap memo keyed on packed r,c (source: dsa/samples-java/src/Ch17/Gridpaths.java, lines 22-74)

```
    /// <summary>Grid paths with obstacles: the classic 2d table.</summary>
    public static long GridPaths(bool[,] blocked)
    {
        int rows = blocked.GetLength(0), cols = blocked.GetLength(1);
        var ways = new long[rows, cols];
        ways[0, 0] = blocked[0, 0] ? 0 : 1;
        for (int r = 0; r < rows; r++)
        {
            for (int c = 0; c < cols; c++)
            {
                if (blocked[r, c] || (r == 0 && c == 0))
                    continue;
                long fromTop = r > 0 ? ways[r - 1, c] : 0;
                long fromLeft = c > 0 ? ways[r, c - 1] : 0;
                ways[r, c] = fromTop + fromLeft;
            }
        }
        return ways[rows - 1, cols - 1];
    }
```

listing 508: c#, grid paths, one line per cell, walls as zeros (source: dsa/samples/src/Ch17/Dp.cs, lines 45-63)

```
export function gridPaths(walls) {
  const rows = walls.length;
  const cols = walls[0].length;
  const dp = Array.from({ length: rows }, () => new Array(cols).fill(0));
  dp[0][0] = walls[0][0] ? 0 : 1;
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < cols; c++) {
      if (walls[r][c]) continue;
      if (r > 0) dp[r][c] += dp[r - 1][c];
      if (c > 0) dp[r][c] += dp[r][c - 1];
    }
  return dp[rows - 1][cols - 1];
}
```

listing 509: javascript, the add-from-above-and-left loop (source: dsa/samples-js/src/ch17-gridpaths.mjs, lines 4-16)

```python
def count_paths(rows, cols, blocked):
    # moves: right or down only
    grid = [[0] * cols for _ in range(rows)]
    grid[0][0] = 1
    for r in range(rows):
        for c in range(cols):
            if (r, c) in blocked:
                grid[r][c] = 0  # an obstacle absorbs every route
                continue
            if r:
                grid[r][c] += grid[r - 1][c]
            if c:
                grid[r][c] += grid[r][c - 1]
    return grid[rows - 1][cols - 1]


def count_paths_memo(rows, cols, blocked):
    # the same count via a recursive memo: the table's mirror image
    table = {}

    def paths(r, c):
        if r == rows - 1 and c == cols - 1:
            return 1
        if (r, c) in table:
            return table[(r, c)]
        total = 0
        if r + 1 < rows and (r + 1, c) not in blocked:
            total += paths(r + 1, c)
        if c + 1 < cols and (r, c + 1) not in blocked:
            total += paths(r, c + 1)
        table[(r, c)] = total
        return total

    if (0, 0) in blocked or (rows - 1, cols - 1) in blocked:
        return 0
    return paths(0, 0)
```

listing 510: python, the table, then the same count through a recursive memo (source: dsa/samples-py/src/Ch17/gridpaths.py, lines 13-48)

```lua
local function grid_paths(rows)
  local r_n, c_n = #rows, #rows[1]
  local t = {}
  for r = 1, r_n do
    t[r] = {}
    for c = 1, c_n do
      t[r][c] = 0
      if rows[r]:sub(c, c) ~= "#" then
        if r == 1 and c == 1 then
          t[r][c] = 1
        else
          t[r][c] = (r > 1 and t[r - 1][c] or 0) + (c > 1 and t[r][c - 1] or 0)
        end
      end
    end
  end
  return t[r_n][c_n]
end
```

listing 511: lua, the grid read as strings, a hash walls the cell (source: dsa/samples-lua/ch17_gridpaths.lua, lines 5-22)

The anchors hold across all seven: the open 3 by 3 counts 6, the choose 4 pick 2 of monotone paths, and blocking the center leaves exactly 2, the two border routes. C and Java pin a full 4 by 4 table with two interior walls and the answer 4, cell for cell, Lua adds the open 4 by 4 at 20 and the sealed corner at 0, and Python and Java run the same count a second way through a recursive memo, the table and the cache agreeing, which is the whole chapter in one fixture, Java's memo a `HashMap` keyed on the two coordinates packed into one long.

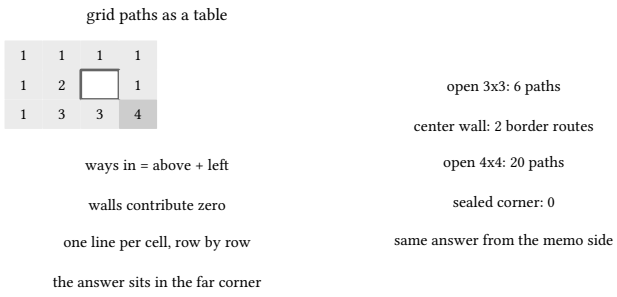


figure 137: grid paths as a table, one line per cell, walls as zeros, the answer in the far corner

17.3. the zero one knapsack

The knapsack is the one interviewers mean when they say dp. The table indexes items against remaining capacity, each cell choosing between skipping and taking the item, and the reconstruction walks the table backwards asking which choice was made.

The dry run: the fixture is weights 2, 3, 4, 5 against values 3, 4, 5, 6 at capacity 5, asserted by the C# suite at 7 with items 0 and 1 rebuilt, and C, Python, and Lua carry the same fixture.

1. Row item 0, weight 2 value 3: capacities 0 and 1 hold 0, and 2 through 5 fill at 0 + 3 = 3.
2. Row item 1, weight 3 value 4: capacity 3 improves to 0 + 4 = 4, and capacity 5 takes 3 + 4 = 7.
3. Row item 2, weight 4 value 5: capacity 4 improves to 0 + 5 = 5, and capacity 5's offer of 0 + 5 = 5 loses to the incumbent 7.
4. Row item 3, weight 5 value 6: the offer of 0 + 6 = 6 loses too, and the corner reads 7, pinned.
5. The backwalk asks each row whether its cell differs from the cell above: items 3 and 2 tie at 7 and drop, item 1 differs, 7 against 3, taken at c = 5 - 3 = 2, and item 0 differs, 3 against 0, taken: items 0 and 1, pinned.
6. The zero-one boundary: three weight-4 items in capacity 8 land 5 + 5 = 10 at exactly two whole items, never a fraction.

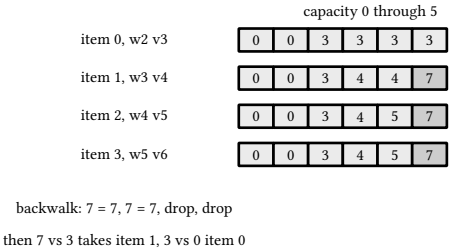


figure 138: the capacity row landing item by item, the corner holding 7, the backwalk dropping through the ties to items 0 and 1

The 7 rebuilt as items 0 and 1 is the pinned pair, and the listings below pack seven ways.

```c
static int solve_table(int items, int cap) {
  for (int c = 0; c ≤ cap; c++)
    dp[0][c] = 0;
  for (int i = 1; i ≤ items; i++)
    for (int c = 0; c ≤ cap; c++) {
      dp[i][c] = dp[i - 1][c];
      if (WT[i - 1] ≤ c && dp[i - 1][c - WT[i - 1]] + VAL[i - 1] > dp[i][c])
        dp[i][c] = dp[i - 1][c - WT[i - 1]] + VAL[i - 1];
    }
  return dp[items][cap];
}

// one row, capacities walked downward so each item is used once
static int solve_rolled(int items, int cap) {
  int row[MAXW + 1] = {0};
  for (int i = 0; i < items; i++)
    for (int c = cap; c ≥ WT[i]; c--)
      if (row[c - WT[i]] + VAL[i] > row[c])
        row[c] = row[c - WT[i]] + VAL[i];
  return row[cap];
}
```

listing 512: c, the full table, then the row rolled right to left (source: dsa/samples-c/src/Ch17/knapsack.c, lines 23-43)

```go
// Knapsack fills the full dp table: dp[i][c] is the optimum using
// items 0..i-1 under capacity c. Reconstruction walks the table
// backwards: an item is taken when the value changed without it.
func Knapsack(capacity int, items []Item) (best int, chosen []int) {
  n := len(items)
  dp := make([][]int, n+1)
  for i := range dp {
    dp[i] = make([]int, capacity+1)
  }
  for i := 1; i <= n; i++ {
    for c := 0; c <= capacity; c++ {
      dp[i][c] = dp[i-1][c] // skip item i-1
      it := items[i-1]
      if it.Weight <= c {
        if take := dp[i-1][c-it.Weight] + it.Value; take > dp[i][c] {
          dp[i][c] = take
        }
      }
    }
  }
  c := capacity
  for i := n; i ≥ 1; i-- {
    if dp[i][c] ≠ dp[i-1][c] {
      chosen = append(chosen, i-1)
      c -= items[i-1].Weight
    }
  }
  for i, j := 0, len(chosen)-1; i < j; i, j = i+1, j-1 {
    chosen[i], chosen[j] = chosen[j], chosen[i]
  }
  return dp[n][capacity], chosen
}

// KnapsackRolled keeps one row: row[c] is the optimum under capacity
// c, updated in place right to left so each item is used at most once.
func KnapsackRolled(capacity int, items []Item) int {
  row := make([]int, capacity+1)
  for _, it := range items {
    for c := capacity; c ≥ it.Weight; c-- {
      if take := row[c-it.Weight] + it.Value; take > row[c] {
        row[c] = take
      }
    }
  }
  return row[capacity]
}
```

listing 513: go, the table with backwalk, the rolled row beside it (source: dsa/samples-go/ch17/knapsack.go, lines 9-54)

```
static int solveTable(int items, int cap) {
  for (int c = 0; c ≤ cap; c++)
    dp[0][c] = 0;
  for (int i = 1; i ≤ items; i++)
    for (int c = 0; c ≤ cap; c++) {
      dp[i][c] = dp[i - 1][c];
      if (WT[i - 1] ≤ c && dp[i - 1][c - WT[i - 1]] + VAL[i - 1] > dp[i][c])
        dp[i][c] = dp[i - 1][c - WT[i - 1]] + VAL[i - 1];
    }
  return dp[items][cap];
}

// one row, capacities walked downward so each item is used once
static int solveRolled(int items, int cap) {
  int[] row = new int[cap + 1];
  for (int i = 0; i < items; i++)
    for (int c = cap; c ≥ WT[i]; c--)
      if (row[c - WT[i]] + VAL[i] > row[c])
        row[c] = row[c - WT[i]] + VAL[i];
  return row[cap];
}

// py lanes: table over arbitrary item sets, rolled row, and the
// backtracked item choice
static int[][] table(int[] w, int[] v, int cap) {
  int[][] best = new int[w.length + 1][cap + 1];
  for (int i = 1; i ≤ w.length; i++)
    for (int c = 0; c ≤ cap; c++) {
      int skip = best[i - 1][c];
      int take = w[i - 1] ≤ c ? best[i - 1][c - w[i - 1]] + v[i - 1] : -1;
      best[i][c] = Math.max(skip, take);
    }
  return best;
}

static int rolled(int[] w, int[] v, int cap) {
  int[] row = new int[cap + 1];
  for (int i = 0; i < w.length; i++)
    for (int c = cap; c ≥ w[i]; c--)
      row[c] = Math.max(row[c], row[c - w[i]] + v[i]);
  return row[cap];
}

static List<Integer> chosen(int[] w, int[] v, int cap) {
  int[][] best = table(w, v, cap);
  List<Integer> items = new ArrayList<>();
  for (int i = w.length, c = cap; i > 0; i--)
    if (best[i][c] ≠ best[i - 1][c]) { // this item was taken
      items.add(i - 1);
      c -= w[i - 1];
    }
  List<Integer> out = new ArrayList<>(items);
  java.util.Collections.reverse(out);
  return out;
}
```

listing 514: java, the table with its final row pinned, the rolled row, the backwalk (source: dsa/samples-java/src/Ch17/Knapsack.java, lines 24-78)

```
    /// <summary>0/1 knapsack, capacity table, with reconstruction of the chosen items.</summary>
    public static (long Value, List<int> Items) Knapsack(int[] weights, long[] values, int capacity)
    {
        int n = weights.Length;
        var table = new long[n + 1, capacity + 1];
        for (int i = 1; i ≤ n; i++)
        {
            for (int c = 0; c ≤ capacity; c++)
            {
                table[i, c] = table[i - 1, c]; // skip item i-1
                if (weights[i - 1] ≤ c)
                {
                    long take = table[i - 1, c - weights[i - 1]] + values[i - 1];
                    if (take > table[i, c])
                        table[i, c] = take;
                }
            }
        }

        var chosen = new List<int>();
        for (int i = n, c = capacity; i > 0; i--)
        {
            if (table[i, c] ≠ table[i - 1, c]) // item i-1 was taken
            {
                chosen.Add(i - 1);
                c -= weights[i - 1];
            }
        }
        chosen.Reverse();
        return (table[n, capacity], chosen);
    }
```

listing 515: c#, zero one knapsack with item reconstruction (source: dsa/samples/src/Ch17/Dp.cs, lines 65-96)

```
export function knapsack(weights, values, cap) {
  const n = weights.length;
  const dp = Array.from({ length: n + 1 }, () => new Array(cap + 1).fill(0));
  for (let i = 1; i ≤ n; i++)
    for (let w = 0; w ≤ cap; w++) {
      dp[i][w] = dp[i - 1][w]; // skip item i-1
      if (weights[i - 1] ≤ w) {
        const take = dp[i - 1][w - weights[i - 1]] + values[i - 1];
        dp[i][w] = Math.max(dp[i][w], take);
      }
    }
  const items = [];
  let w = cap;
  for (let i = n; i ≥ 1; i--)
    if (dp[i][w] =/= dp[i - 1][w]) {
      items.push(i - 1);
      w -= weights[i - 1];
    }
  return { value: dp[n][cap], items: items.reverse() };
}

// one row: dp[w] swept right to left so item i-1 is used at most once
export function knapsackRolled(weights, values, cap) {
  const dp = new Array(cap + 1).fill(0);
  for (let i = 0; i < weights.length; i++)
    for (let w = cap; w ≥ weights[i]; w--)
      dp[w] = Math.max(dp[w], dp[w - weights[i]] + values[i]);
  return dp[cap];
}
```

listing 516: javascript, the table, the backwalk, the rolled row (source: dsa/samples-js/src/ch17-knapsack.mjs, lines 5-33)

```python
def knapsack_table(weights, values, cap):
    # best[i][w]: the optimum using items 0..i-1 under capacity w
    best = [[0] * (cap + 1) for _ in range(len(weights) + 1)]
    for i in range(1, len(weights) + 1):
        for w in range(cap + 1):
            skip = best[i - 1][w]
            take = (
                best[i - 1][w - weights[i - 1]] + values[i - 1]
                if weights[i - 1] ≤ w
                else -1
            )
            best[i][w] = max(skip, take)
    return best


def knapsack_rolled(weights, values, cap):
    row = [0] * (cap + 1)  # one row reused, walked right to left
    for wt, val in zip(weights, values):
        for w in range(cap, wt - 1, -1):  # backwards: each item used once
            row[w] = max(row[w], row[w - wt] + val)
    return row[cap]


def chosen_items(weights, values, cap):
    best = knapsack_table(weights, values, cap)
    items, w = [], cap
    for i in range(len(weights), 0, -1):
        if best[i][w] ≠ best[i - 1][w]:  # this item was taken
            items.append(i - 1)
            w -= weights[i - 1]
    return items[::-1]
```

listing 517: python, the table, the rolled row, the readback (source: dsa/samples-py/src/Ch17/knapsack.py, lines 13-43)

```lua
local function knapsack_table(weights, values, cap)
  local n = #weights
  local t = {}
  for i = 0, n do
    t[i] = {}
    for c = 0, cap do t[i][c] = 0 end
  end
  for i = 1, n do
    for c = 0, cap do
      t[i][c] = t[i - 1][c] -- skip item i
      if weights[i] ≤ c then
        local take = t[i - 1][c - weights[i]] + values[i]
        if take > t[i][c] then t[i][c] = take end
      end
    end
  end
  return t
end

-- one row, capacities walked downward so item i is used at most once
local function knapsack_rolled(weights, values, cap)
  local row = {}
  for c = 0, cap do row[c] = 0 end
  for i = 1, #weights do
    for c = cap, weights[i], -1 do
      local take = row[c - weights[i]] + values[i]
      if take > row[c] then row[c] = take end
    end
  end
  return row[cap]
end
```

listing 518: lua, nested tables, the downward capacity walk (source: dsa/samples-lua/ch17_knapsack.lua, lines 6-36)

The zero-one test matters as a boundary: three identical weight-4 items in capacity 8 yields exactly two whole items, never a fraction, and the fractional comparison in chapter 18 beats it at 240 versus 220 on shared input, the two problem variants genuinely differing.

The shared fixture is weights 2, 3, 4, 5 against values 3, 4, 5, 6 at capacity 5, and the answer 7 with items 0 and 1 taken pins in C, Java, Python, and Lua, Java pinning the whole final table row 0 0 3 4 5 7 and interior cells beside, with Python and Java carrying the second fixture, weights 1, 2, 3 against values 6, 10, 12, landing the classic 22 with items 1 and 2. The rolled row agrees with the full table everywhere it exists, and the downward capacity walk is why: sweeping right to left means an item can never feed its own cell, which is the one-use rule stated as a loop direction. C# keeps its own fixtures and the fractional contrast, Lua brute-forces every mask as a cross-check.

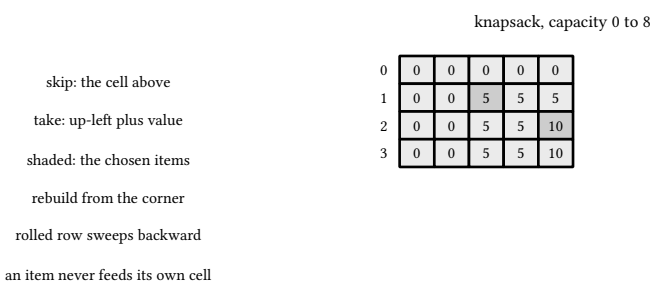


figure 139: the knapsack table as memory, cells choosing skip or take, rebuild from the corner

17.4. sequences and their tables

Longest common subsequence and edit distance are the same table shape wearing different recurrences, match extends the diagonal, otherwise take the better neighbor.

The dry run: the fixtures are ABCBDAB against BDCABA at length 4 with the rebuild checked as a property, and the edit distance family, kitten to sitting 3, flaw to lawn 2, empty against abc 3, same against same 0, all asserted by the C# suite.

1. The lcs corner reads 4 and the rebuild walks (7, 6) backward, emitting only on matches.
2. B against A mismatches and the row above wins, so the walk steps up into the A pair: a match emits A, and the diagonal beneath holds 4 - 1 = 3.
3. D against B mismatches and steps up into the B pair: emit B, diagonal 3 - 1 = 2.
4. C against A steps left into the C pair: emit C, diagonal 2 - 1 = 1.
5. B against D steps left into the B pair: emit B, diagonal 1 - 1 = 0, and the border ends the walk.
6. Reversed, the emission reads BCAB, one of several optimal answers, accepted because both strings contain it at the pinned length 4.

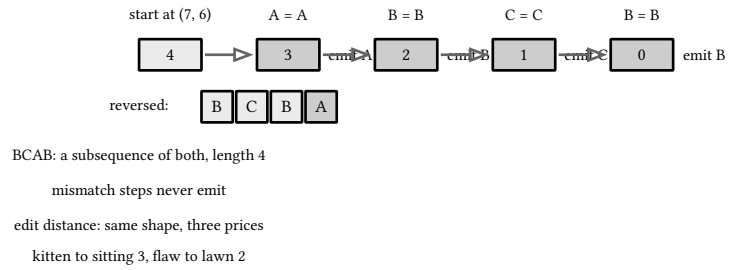


figure 140: the rebuild walking backward, every match emitting one letter and dropping the counter by one, the reversed emission a valid answer

The 4, rebuilt as BCAB and accepted by property, is the pin, and the listings below fill both tables.

```
    /// <summary>Longest common subsequence with reconstruction.</summary>
    public static (int Length, string Sequence) Lcs(string a, string b)
    {
        int m = a.Length, n = b.Length;
        var table = new int[m + 1, n + 1];
        for (int i = 1; i ≤ m; i++)
        {
            for (int j = 1; j ≤ n; j++)
            {
                table[i, j] = a[i - 1] == b[j - 1]
                    ? table[i - 1, j - 1] + 1
                    : Math.Max(table[i - 1, j], table[i, j - 1]);
            }
        }

        var seq = new List<char>();
        for (int i = m, j = n; i > 0 && j > 0;)
        {
            if (a[i - 1] == b[j - 1])
            {
                seq.Add(a[i - 1]);
                i--;
                j--;
            }
            else if (table[i - 1, j] ≥ table[i, j - 1])
                i--;
            else
                j--;
        }
        seq.Reverse();
        return (table[m, n], new string(seq.ToArray()));
    }
```

listing 519: c#, lcs with reconstruction, the chapter 15 fixture family (source: dsa/samples/src/Ch17/Dp.cs, lines 97-128)

LCS reconstruction must be checked as a property, both strings contain the output as a subsequence and its length hits four, because multiple optimal answers exist. Edit distance gets the textbook cases plus symmetry:

```
static int editdist(const char *s, const char *t, int allow_sub) {
  size_t sn = strlen(s), tn = strlen(t);
  for (size_t j = 0; j ≤ tn; j++)
    dp[0][j] = (int)j;
  for (size_t i = 1; i ≤ sn; i++) {
    dp[i][0] = (int)i;
    for (size_t j = 1; j ≤ tn; j++) {
      int del = dp[i - 1][j] + 1;
      int ins = dp[i][j - 1] + 1;
      int sub =
          dp[i - 1][j - 1] + (s[i - 1] == t[j - 1] ? 0 : (allow_sub ? 1 : 2));
      dp[i][j] = del < ins ? (del < sub ? del : sub) : (ins < sub ? ins : sub);
    }
  }
  return dp[sn][tn];
}
```

listing 520: c, the full table, substitution priced 1 or 2 (source: dsa/samples-c/src/Ch17/editdist.c, lines 21-36)

```
// EditDistance is the Levenshtein distance with unit costs: the
// dp[i][j] is the distance between the first i bytes of a and the
// first j bytes of b, built from insert, delete, and substitute.
func EditDistance(a, b string) int {
  m, n := len(a), len(b)
  dp := make([][]int, m+1)
  for i := range dp {
    dp[i] = make([]int, n+1)
    dp[i][0] = i // deleting i characters
  }
  for j := 1; j <= n; j++ {
    dp[0][j] = j // inserting j characters
  }
  for i := 1; i <= m; i++ {
    for j := 1; j <= n; j++ {
      substit := dp[i-1][j-1]
      if a[i-1] ≠ b[j-1] {
        substit++
      }
      dp[i][j] = min(dp[i-1][j]+1, dp[i][j-1]+1, substit)
    }
  }
  return dp[m][n]
}
```

listing 521: go, borders seeded, the built-in min of three (source: dsa/samples-go/ch17/editdist.go, lines 3-26)

```java
static int[][] dp = new int[16][16];

// unit costs, or substitution cost 2 when subCost is 2
static int editdist(String s, String t, int subCost) {
  int sn = s.length(), tn = t.length();
  for (int j = 0; j ≤ tn; j++)
    dp[0][j] = j;
  for (int i = 1; i ≤ sn; i++) {
    dp[i][0] = i;
    for (int j = 1; j ≤ tn; j++) {
      int del = dp[i - 1][j] + 1;
      int ins = dp[i][j - 1] + 1;
      int sub = dp[i - 1][j - 1]
          + (s.charAt(i - 1) == t.charAt(j - 1) ? 0 : subCost);
      dp[i][j] = Math.min(del, Math.min(ins, sub));
    }
  }
  return dp[sn][tn];
}

// the recursive definition as an oracle, per the py brute lane
static int brute(String a, String b, int subCost) {
  if (a.isEmpty())
    return b.length();
  if (b.isEmpty())
    return a.length();
  int drop = brute(a.substring(1), b, subCost) + 1;
  int add = brute(a, b.substring(1), subCost) + 1;
  int sub = brute(a.substring(1), b.substring(1), subCost)
      + (a.charAt(0) == b.charAt(0) ? 0 : subCost);
  return Math.min(drop, Math.min(add, sub));
}
```

listing 522: java, the full table with its final row pinned, subCost a parameter, a recursive brute as oracle (source: dsa/samples-java/src/Ch17/Editdist.java, lines 16-47)

```
    /// <summary>Edit distance: insert, delete, substitute, each weight 1.</summary>
    public static int EditDistance(string a, string b)
    {
        int m = a.Length, n = b.Length;
        var table = new int[m + 1, n + 1];
        for (int i = 0; i ≤ m; i++)
            table[i, 0] = i;
        for (int j = 0; j ≤ n; j++)
            table[0, j] = j;
        for (int i = 1; i ≤ m; i++)
        {
            for (int j = 1; j ≤ n; j++)
            {
                int cost = a[i - 1] == b[j - 1] ? 0 : 1;
                table[i, j] = Math.Min(
                    Math.Min(table[i - 1, j] + 1, table[i, j - 1] + 1),
                    table[i - 1, j - 1] + cost);
            }
        }
        return table[m, n];
    }
```

listing 523: c#, unit costs, the min of three neighbors (source: dsa/samples/src/Ch17/Dp.cs, lines 130-150)

```
export function editDistance(a, b) {
  const m = a.length;
  const n = b.length;
  const dp = Array.from({ length: m + 1 }, (_, i) ⇒
    new Array(n + 1).fill(0).map((_, j) ⇒ i + j),
  );
  for (let i = 1; i ≤ m; i++)
    for (let j = 1; j ≤ n; j++) {
      const sub = a[i - 1] === b[j - 1] ? 0 : 1;
      dp[i][j] = Math.min(
        dp[i - 1][j] + 1, // delete
        dp[i][j - 1] + 1, // insert
        dp[i - 1][j - 1] + sub, // substitute or match
      );
    }
  return dp[m][n];
}
```

listing 524: javascript, the whole table, match free (source: dsa/samples-js/src/ch17-editdist.mjs, lines 4-20)

```python
def edit_distance(a, b, sub_cost=1):
    # dp[i][j]: the distance between a[:i] and b[:j]
    dp = [[0] * (len(b) + 1) for _ in range(len(a) + 1)]
    for i in range(len(a) + 1):
        dp[i][0] = i  # delete everything left in a
    for j in range(len(b) + 1):
        dp[0][j] = j  # insert everything missing from b
    for i in range(1, len(a) + 1):
        for j in range(1, len(b) + 1):
            if a[i - 1] == b[j - 1]:
                dp[i][j] = dp[i - 1][j - 1]  # the letters match: free
                continue
            dp[i][j] = min(
                dp[i - 1][j] + 1,  # delete a's letter
                dp[i][j - 1] + 1,  # insert b's letter
                dp[i - 1][j - 1] + sub_cost,  # substitute
            )
    return dp[len(a)][len(b)]
```

listing 525: python, sub_cost a parameter, a recursive brute cross-check (source: dsa/samples-py/src/Ch17/editdist.py, lines 13-30)

```lua
local function edit_distance(a, b)
  local m, n = #a, #b
  local prev, cur = {}, {}
  for j = 0, n do prev[j] = j end
  for i = 1, m do
    cur[0] = i
    for j = 1, n do
      local cost = a:byte(i) == b:byte(j) and 0 or 1
      cur[j] = math.min(prev[j] + 1, cur[j - 1] + 1, prev[j - 1] + cost)
    end
    prev, cur = cur, prev
  end
  return prev[n]
end
```

listing 526: lua, two rows ping-ponged, O(min) memory (source: dsa/samples-lua/ch17_editdist.lua, lines 5-18)

The kitten to sitting anchor, distance 3, holds in every suite. The variants split by memory and by price: Lua keeps two rows and swaps them, the whole table never exists, C, Java, and Python price substitution at 2 as a parameter and watch the answer rise to 5, insert plus delete arithmetic, and C and Java pin interior cells of the table plus the abcdef to azced case at 3, Java holding the whole final row 6 6 5 4 3 3 2 3 beside. Lua adds the symmetry check on a long pair and the bound by the longer side, and Python and Java cross-check the table against the recursive definition on a sweep of small pairs.

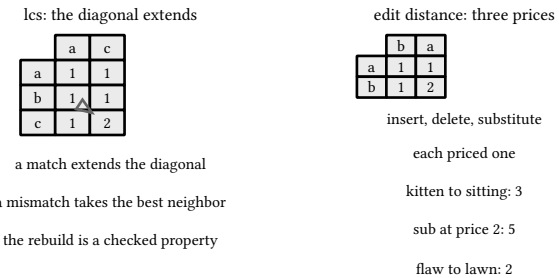


figure 141: the same table shape under two recurrences, lcs extends the diagonal, edit distance prices three moves

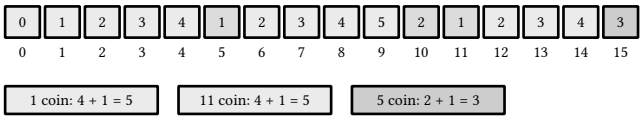
17.5. coin change, the greedy killer

With coins 1, 5, 11 and amount 15, taking the largest coin first strands you at four singles for 5 coins while the table finds three fives.

The dry run: the fixture is coins 1, 5, 11 asserted by the C# suite, amount 10 at 2 and the greedy killer 15 at 3, while C and Python run the 1, 5, 12 family, where 15 costs 3 against greedy's

4.

5. The table seeds at amount 0 with 0 coins, everything past it impossible.
6. The 1 coin climbs the low amounts: table[1] = 0 + 1 = 1 through table[4] = 4.
7. The 5 coin lands at 5 with 0 + 1 = 1, and 10 follows at 1 + 1 = 2, the suite's other pin, two fives.
8. The 11 coin lands at 11 with 0 + 1 = 1.
9. Amount 15 tries all three roads: the 1 coin offers 4 + 1 = 5, the 11 coin offers 4 + 1 = 5, and the 5 coin offers 2 + 1 = 3: the table keeps 3, three fives, pinned.
10. Greedy takes 11 first and strands at 4, paying 1 + 4 = 5 coins, and the unreachable channel shows {2, 4} at 7 reading −1 out.



greedy: 11 then four singles, 5 coins

{2, 4} at 7 never fills, −1 out

figure 142: the min-coin table sweeping amounts 0 to 15, the 5 and 11 coins landing, the three offers at 15 with the cheapest shaded

The 3 against greedy's 5 is the pinned kill, and the listings below count coins seven ways.

```c
static int min_coins(const int *coins, int cn, int amount) {
  int dp[AMAX + 1];
  dp[0] = 0;
  for (int a = 1; a ≤ amount; a++) {
    dp[a] = IMPOSSIBLE;
    for (int i = 0; i < cn; i++)
      if (coins[i] ≤ a && dp[a - coins[i]] + 1 < dp[a])
        dp[a] = dp[a - coins[i]] + 1;
  }
  return dp[amount];
}

// order-independent combinations: outer loop over coins
static long long comb_count(const int *coins, int cn, int amount) {
  long long dp[AMAX + 1] = {0};
  dp[0] = 1;
  for (int i = 0; i < cn; i++)
    for (int a = coins[i]; a ≤ amount; a++)
      dp[a] += dp[a - coins[i]];
  return dp[amount];
}

// greedy for contrast: always take the largest coin that fits
static int greedy(const int *coins, int cn, int amount) {
  int used = 0;
  for (int i = cn - 1; i ≥ 0 && amount > 0; i--)
    while (amount ≥ coins[i]) {
      amount -= coins[i];
      used++;
    }
  return amount == 0 ? used : IMPOSSIBLE;
}
```

listing 527: c, minimum coins, combinations, greedy for contrast (source: dsa/samples-c/src/Ch17/coins.c, lines 20-51)

```go
// MinCoins returns the fewest coins summing to amount, or an error
// when no combination exists. min[0] = 0 seeds the table; every
// amount tries every coin as its last coin.
func MinCoins(amount int, coins []int) (int, error) {
  const impossible = int(1) << 30
  minCoins := make([]int, amount+1)
  for a := 1; a <= amount; a++ {
    minCoins[a] = impossible
    for _, c := range coins {
      if c <= a && minCoins[a-c]+1 < minCoins[a] {
        minCoins[a] = minCoins[a-c] + 1
      }
    }
  }
  if minCoins[amount] == impossible {
    return 0, fmt.Errorf("amount %d unreachable with %v", amount, coins)
  }
  return minCoins[amount], nil
}

// CoinCombinations counts the ways to build amount where order does
// not matter. Processing one coin at a time (outer loop over coins,
// inner over amounts ascending) counts multisets, not sequences: each
// combination is built in exactly one canonical order.
func CoinCombinations(amount int, coins []int) int {
  ways := make([]int, amount+1)
  ways[0] = 1
  for _, c := range coins {
    for a := c; a <= amount; a++ {
      ways[a] += ways[a-c]
    }
  }
  return ways[amount]
}
```

listing 528: go, min coins with an error, combinations one loop per coin (source: dsa/samples-go/ch17/coins.go, lines 5-38)

```java
static final int IMPOSSIBLE = 1_000_000;

static int minCoins(int[] coins, int amount) {
  int[] dp = new int[amount + 1];
  dp[0] = 0;
  for (int a = 1; a ≤ amount; a++) {
    dp[a] = IMPOSSIBLE;
    for (int c : coins)
      if (c ≤ a && dp[a - c] + 1 < dp[a])
        dp[a] = dp[a - c] + 1;
  }
  return dp[amount];
}

// order-independent combinations: outer loop over coins
static long combCount(int[] coins, int amount) {
  long[] dp = new long[amount + 1];
  dp[0] = 1;
  for (int c : coins)
    for (int a = c; a ≤ amount; a++)
      dp[a] += dp[a - c];
  return dp[amount];
}

// greedy for contrast: always take the largest coin that fits
static int greedy(int[] coins, int amount) {
  int used = 0;
  for (int i = coins.length - 1; i ≥ 0 && amount > 0; i--)
    while (amount ≥ coins[i]) {
      amount -= coins[i];
      used++;
    }
  return amount == 0 ? used : IMPOSSIBLE;
}
```

listing 529: java, the one-million sentinel for impossible, the coin-outer count, greedy for contrast (source: dsa/samples-java/src/Ch17/Coins.java, lines 16-49)

```csharp
  /// <summary>Coin change: fewest coins, and whether any amount is reachable.</summary>
  public static int MinCoins(long[] coins, long amount)
  {
      const int impossible = int.MaxValue;
      var table = new long[amount + 1];
      Array.Fill(table, impossible);
      table[0] = 0;
      for (long a = 1; a ≤ amount; a++)
      {
          foreach (long coin in coins)
          {
              if (coin ≤ a && table[a - coin] ≠ impossible)
                  table[a] = Math.Min(table[a], table[a - coin] + 1);
          }
      }
      return table[amount] == impossible ? -1 : (int)table[amount];
  }
```

listing 530: c#, fewest coins, unreachable marked (source: dsa/samples/src/Ch17/Dp.cs, lines 152-168)

```javascript
export function minCoins(coins, amount) {
  const INF = Infinity;
  const dp = new Array(amount + 1).fill(INF);
  dp[0] = 0;
  for (let a = 1; a ≤ amount; a++)
    for (const c of coins)
      if (c ≤ a && dp[a - c] + 1 < dp[a]) dp[a] = dp[a - c] + 1;
  return dp[amount] === INF ? null : dp[amount];
}

// order-insensitive combination count: how many multisets of coins sum
// to the amount
export function comboCount(coins, amount) {
  const dp = new Array(amount + 1).fill(0);
  dp[0] = 1;
  for (const c of coins)
    for (let a = c; a ≤ amount; a++) dp[a] += dp[a - c];
  return dp[amount];
}
```

listing 531: javascript, Infinity for impossible, the coin-outer count (source: dsa/samples-js/src/ch17-coins.mjs, lines 5-23)

```python
def min_coins(coins, amount):
    best = [0] + [None] * amount
    for a in range(1, amount + 1):
        for c in coins:
            if c ≤ a and best[a - c] is not None:
                reach = best[a - c] + 1
                if best[a] is None or reach < best[a]:
                    best[a] = reach
    return best[amount]


def count_combinations(coins, amount):
    # ways[a]: ordered-by-coin-type combinations making exactly a
    ways = [1] + [0] * amount
    for c in coins:  # coin outer: 1+2 and 2+1 count once
        for a in range(c, amount + 1):
            ways[a] += ways[a - c]
    return ways[amount]


def greedy_coins(coins, amount):
    count = 0
    for c in sorted(coins, reverse=True):
        while amount ≥ c:
            amount -= c
            count += 1
    return count if amount == 0 else None
```

listing 532: python, None for unreachable, greedy beside the table (source: dsa/samples-py/src/Ch17/coins.py, lines 13-39)

```lua
local function min_coins(coins, amount)
  local impossible = math.maxinteger
  local t = {}
  for a = 0, amount do t[a] = 0 end
  for a = 1, amount do
    t[a] = impossible
    for _, c in ipairs(coins) do
      if c ≤ a and t[a - c] ~= impossible and t[a - c] + 1 < t[a] then
        t[a] = t[a - c] + 1
      end
    end
  end
  if t[amount] == impossible then return -1 end
  return t[amount]
end

-- distinct combinations: outer loop over coins so order never double
-- counts
local function combinations(coins, amount)
  local t = {}
  for a = 0, amount do t[a] = 0 end
  t[0] = 1
  for _, c in ipairs(coins) do
    for a = c, amount do
      t[a] = t[a] + t[a - c]
    end
  end
  return t[amount]
end
```

listing 533: lua, maxinteger for impossible, −1 back out (source: dsa/samples-lua/ch17_coins.lua, lines 7-35)

The awkward coinage splits into two families, both real: C, Java, and Python use 1, 5, 12, where amount 15 costs 3 against greedy's 4 and amount 16 costs 4 as 5+5+5+1 against 12+1+1+1+1, and C#, JavaScript, and Lua use 1, 5, 11, where 15 costs 3 against greedy's
5. The canonical trap, coins 1, 3, 4 at amount 6, dp says 3+3 while

greedy says 4+1+1, pins in C, Java, and Python. The combination counter puts its coin loop on the outside so order never double counts, and its anchors are 4 ways for 1, 2, 3 at 4 and 5 ways for 1, 5, 12 at 15. The unreachable channel is per language again, minus one in C# and Lua, null in JavaScript, None in Python, an error in Go, and the shared one-million sentinel in C and Java, large enough that no book-size arithmetic ever crosses it.

NOTE: when greedy fails, this is why

Greedy makes one irreversible choice per step. It is optimal when the choice has an exchange argument, chapter 18 builds several that do. Coin change has none: the locally best coin can be globally wrong. The general test of greedy is exactly this, look for a counterexample before trusting it, and reach for the table when one exists.

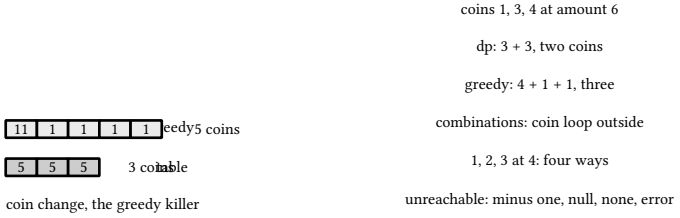


figure 143: coin change kills the greedy take, the table finds three fives where greedy strands on singles

17.6. longest increasing subsequence

Two speeds, one question. The quadratic table lets every element extend the best earlier smaller element and keeps parent links, so it can rebuild an actual subsequence. The patience method keeps tails, the smallest tail that ends a run of each length, and binary searches each arrival into place, an n log n length with no subsequence to show for it unless parents ride along.

The dry run: the fixture is 10, 9, 2, 5, 3, 7, 101, 18, walked by the C# build of chapter 19 with the length pinned at 4 and the rebuild accepted as a property, while Python and Lua pin the run 2, 5, 7, 101 on the same array and C rebuilds 3, 4, 5, 9 on the other fixture.

1. 10 opens the ladder alone, then 9 and 2 each replace it: one rung holding index 2, the value 2.
2. 5 clears the ladder and appends: the tails hold indices 2 and 3 with parent[3] = 2.
3. 3 replaces the second rung with parent[4] = 2: the ladder reads the values 2, 3.
4. 7 and 101 append, parents 4 then 5, and 18 replaces 101 with parent[7] = 5: the final tails hold indices 2, 4, 5, 7 at length 4, pinned.
5. The rebuild walks the parents from the last tail, 7 to 5 to 4 to 2, reading 18, 7, 3, 2 backward: the run 2, 3, 7, 18 forward, a different valid answer from Python and Lua's 2, 5, 7, 101.
6. Both answers pass the property, strictly increasing, correct length, drawn from the input, and the all-equal and falling arrays give 1.

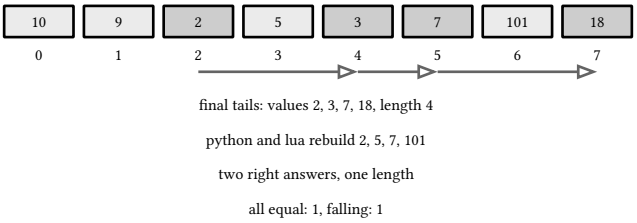


figure 144: the parent chain under the C# walk on 10 9 2 5 3 7 101 18, indices 2, 4, 5, 7 shaded, the rebuild reading 2, 3, 7, 18

The 4, reached by two different valid runs, is the pin, and the listings below climb both roads.

```c
static int lis_n2(const int *a, int len, int *dp, int *prev) {
  if (len == 0)
    return 0;
  int best = 0;
  for (int i = 0; i < len; i++) {
    dp[i] = 1;
    prev[i] = -1;
    for (int j = 0; j < i; j++)
      if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
        dp[i] = dp[j] + 1;
        prev[i] = j;
      }
    if (dp[i] > dp[best])
      best = i;
  }
  return dp[best];
}

// patience: tails[k] = smallest tail of any increasing run of length k+1
static int lis_fast(const int *a, int len) {
  int tails[64], tl = 0;
  for (int i = 0; i < len; i++) {
    // lower bound position in tails
    int lo = 0, hi = tl;
    while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      if (tails[mid] < a[i])
        lo = mid + 1;
      else
        hi = mid;
    }
    tails[lo] = a[i];
    if (lo == tl)
      tl++;
  }
  return tl;
}
```

listing 534: c, the n^2 table with reconstruction, the patience tails (source: dsa/samples-c/src/Ch17/lis.c, lines 18-54)

```go
// LISQuad returns the longest strictly increasing subsequence: the
// quadratic dp where len[i] extends the best len[j] over j < i with
// a[j] < a[i], plus one reconstruction through parent links.
func LISQuad(a []int) (length int, subsequence []int) {
  if len(a) == 0 {
    return 0, nil
  }
  lens := make([]int, len(a))
  parent := make([]int, len(a))
  for i := range lens {
    lens[i] = 1
    parent[i] = -1
  }
  bestEnd := 0
  for i := 1; i < len(a); i++ {
    for j := 0; j < i; j++ {
      if a[j] < a[i] && lens[j]+1 > lens[i] {
        lens[i] = lens[j] + 1
        parent[i] = j
      }
    }
    if lens[i] > lens[bestEnd] {
      bestEnd = i
    }
  }
  for at := bestEnd; at ≠ -1; at = parent[at] {
    subsequence = append(subsequence, a[at])
  }
  slices.Reverse(subsequence)
  return lens[bestEnd], subsequence
}

// LISNLogN keeps tails: tails[k] is the smallest possible tail of an
// increasing subsequence of length k+1. Each element either extends
// tails or replaces the first tail ≥ it (binary search). The tails
// array is not itself a subsequence, but its length is the answer.
func LISNLogN(a []int) int {
  var tails []int
  for _, x := range a {
    at, found := slices.BinarySearch(tails, x)
    if found {
      continue // equal tail already there: strictly increasing
    }
    if at == len(tails) {
      tails = append(tails, x)
    } else {
      tails[at] = x
    }
  }
  return len(tails)
}
```

listing 535: go, the quadratic dp, then tails over slices.BinarySearch (source: dsa/samples-go/ch17/lis.go, lines 5-55)

```
static int lisN2(int[] a) {
  if (a.length == 0)
    return 0;
  dpArr = new int[a.length];
  prevArr = new int[a.length];
  int best = 0;
  for (int i = 0; i < a.length; i++) {
    dpArr[i] = 1;
    prevArr[i] = -1;
    for (int j = 0; j < i; j++)
      if (a[j] < a[i] && dpArr[j] + 1 > dpArr[i]) {
        dpArr[i] = dpArr[j] + 1;
        prevArr[i] = j;
      }
    if (dpArr[i] > dpArr[best])
      best = i;
  }
  return dpArr[best];
}

// reconstruction from the first maximum dp index
static List<Integer> reconstruct(int[] a) {
  int at = 0;
  for (int i = 1; i < a.length; i++)
    if (dpArr[i] > dpArr[at])
      at = i;
  List<Integer> run = new ArrayList<>();
  for (int i = at; i ≥ 0; i = prevArr[i])
    run.add(a[i]);
  List<Integer> out = new ArrayList<>(run);
  java.util.Collections.reverse(out);
  return out;
}

// patience: tails[k] = smallest tail of any increasing run of length k+1
static int lisFast(int[] a) {
  int[] tails = new int[64];
  int tl = 0;
  for (int x : a) {
    // lower bound position in tails
    int lo = 0, hi = tl;
    while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      if (tails[mid] < x)
        lo = mid + 1;
      else
        hi = mid;
    }
    tails[lo] = x;
    if (lo == tl)
      tl++;
  }
  return tl;
}
```

listing 536: java, the quadratic dp with parents, the patience tails by a hand-rolled lower bound (source: dsa/samples-java/src/Ch17/Lis.java, lines 22-75)

```
    /// <summary>Strict lis: tails plus lower bound, parent links rebuild one subsequence.</summary>
    public static int LongestIncreasingSubsequence(long[] a, out int[] indices, Meter? meter = null)
    {
        var tails = new int[a.Length]; // index of the smallest tail of each length
        var parent = new int[a.Length];
        Array.Fill(parent, -1);
        int len = 0;
        for (int i = 0; i < a.Length; i++)
        {
            int lo = 0, hi = len;
            while (lo < hi)
            {
                if (meter != null)
                    meter.Ops++;
                int mid = lo + (hi - lo) / 2;
                if (a[tails[mid]] < a[i])
                    lo = mid + 1;
                else
                    hi = mid;
            }
            if (lo > 0)
                parent[i] = tails[lo - 1];
            if (lo == len)
                tails[len++] = i;
            else
                tails[lo] = i;
        }

        indices = new int[len];
        int cur = len > 0 ? tails[len - 1] : -1;
        for (int k = len - 1; k >= 0; k--)
        {
            indices[k] = cur;
            cur = parent[cur];
        }
        return len;
    }
```

listing 537: c#, the tails ladder with parent links, the advanced chapter's build (source: dsa/samples/src/Ch19/AdvDp.cs, lines 239-275)

```
export function lisN2(a) {
  if (a.length === 0) return { length: 0, seq: [] };
  const dp = new Array(a.length).fill(1);
  const parent = new Array(a.length).fill(-1);
  for (let i = 1; i < a.length; i++)
    for (let j = 0; j < i; j++)
      if (a[j] < a[i] && dp[j] + 1 > dp[i]) {
        dp[i] = dp[j] + 1;
        parent[i] = j;
      }
  let best = 0;
  for (let i = 1; i < a.length; i++) if (dp[i] > dp[best]) best = i;
  const seq = [];
  for (let i = best; i !== -1; i = parent[i]) seq.push(a[i]);
  return { length: dp[best], seq: seq.reverse() };
}

// { length, seq }: tails[k] is the index of the best (smallest) tail of a
// length k+1 subsequence; a replacement keeps its predecessor
export function lisNLogN(a) {
  const tails = [];
  const parent = new Array(a.length).fill(-1);
  for (let i = 0; i < a.length; i++) {
    let lo = 0;
    let hi = tails.length;
    while (lo < hi) {
      const mid = (lo + hi) >> 1;
      if (a[tails[mid]] < a[i]) lo = mid + 1;
      else hi = mid;
    }
    if (lo > 0) parent[i] = tails[lo - 1];
    if (lo === tails.length) tails.push(i);
    else tails[lo] = i;
  }
  const seq = [];
  for (let i = tails[tails.length - 1]; i !== -1; i = parent[i])
    seq.push(a[i]);
  return { length: tails.length, seq: seq.reverse() };
}
```

listing 538: javascript, both roads return a subsequence, different valid ones (source: dsa/samples-js/src/ch17-lis.mjs, lines 7-45)

```python
def lis_quadratic(a):
    # dp[i]: longest increasing run ending exactly at i
    dp = [1] * len(a)
    parent = [-1] * len(a)
    for i in range(len(a)):
        for j in range(i):
            if a[j] < a[i] and dp[j] + 1 > dp[i]:
                dp[i] = dp[j] + 1
                parent[i] = j
    if not a:
        return 0, []
    end = dp.index(max(dp))
    run = []
    while end ≠ -1:
        run.append(a[end])
        end = parent[end]
    return max(dp), run[::-1]


def lis_length_log(a):
    tails = []  # tails[k]: smallest tail of any increasing run of length k+1
    for x in a:
        lo, hi = 0, len(tails)
        while lo < hi:  # first tail ≥ x
            mid = (lo + hi) // 2
            if tails[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        if lo == len(tails):
            tails.append(x)  # x extends the longest run
        else:
            tails[lo] = x  # x is a smaller tail for this length
    return len(tails)
```

listing 539: python, dp with parents, tails by hand-rolled binary search (source: dsa/samples-py/src/Ch17/lis.py, lines 14-47)

```lua
local function lis_quadratic(a)
  local best, parent = {}, {}
  for i = 1, #a do
    best[i], parent[i] = 1, 0
    for j = 1, i - 1 do
      if a[j] < a[i] and best[j] + 1 > best[i] then
        best[i] = best[j] + 1
        parent[i] = j
      end
    end
  end
  local len, at = 0, 0
  for i = 1, #a do
    if best[i] > len then
      len = best[i]
      at = i
    end
  end
  -- rebuild one subsequence by walking parents backward
  local seq = {}
  while at > 0 do
    seq[#seq + 1] = a[at]
    at = parent[at]
  end
  local rev = {}
  for k = #seq, 1, -1 do rev[#rev + 1] = seq[k] end
  return len, rev
end

-- n log n: tails[k] = smallest tail of any increasing run of length k
local function lis_tails(a)
  local tails = {}
  for _, x in ipairs(a) do
    local lo, hi = 1, #tails + 1
    while lo < hi do
      local mid = lo + (hi - lo) // 2
      if tails[mid] < x then lo = mid + 1 else hi = mid end
    end
    tails[lo] = x
  end
  return #tails
end
```

listing 540: lua, the parent walk reversed, tails by integer midpoint (source: dsa/samples-lua/ch17_lis.lua, lines 7-48)

Two fixtures cover the field and every suite pins length 4 on both: 3, 1, 4, 1, 5, 9, 2, 6, where C and Java rebuild 3, 4, 5, 9 from the parents, and 10, 9, 2, 5, 3, 7, 101, 18, where Python, Java, and Lua pin the run 2, 5, 7, 101 and the C# suite of chapter 19 does its parent walk on the same array. JavaScript's file says the interesting part out loud: its two roads return different subsequences, and both are valid answers, which is why reconstruction is checked as a property, strictly increasing, correct length, drawn from the input. Java runs the two roads against each other over a sweep of arrays, the agreement the property stands on. The all-equal array gives length 1 everywhere, strictness is one comparison, and the falling array gives 1 with the first element as the witness.

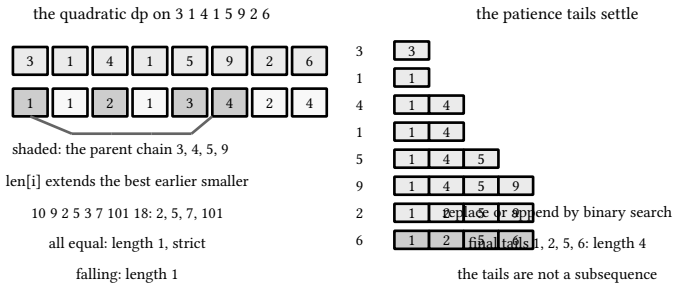


figure 145: two roads to length 4 on the same array, the quadratic parents chain 3 4 5 9, the patience tails settle at 1 2 5 6

17.7. bitmask subsets

When the state is a set of at most about twenty things, the subset becomes an integer bitmask and the table is two to the n. The assignment problem, one worker per task, is the canonical case.

The dry run: the fixtures are identical literals in all seven suites. The 3 by 3 cost matrix reads 1, 100, 100, then 2, 50, 60, then 3, 40, 80, pinned at 101 against the greedy row walk's 131, and the 4 by 4 reads 7, 3, 9, 2, then 4, 8, 3, 6, then 6, 7, 2, 5, then 9, 2, 4, 7, pinned at 10 through the permutation 3, 0, 2, 1. A brute force over permutations agrees in-test on both. The relaxation meter counts every inner (mask, task) pair, 8 rows and 24 attempts at n = 3, Lua's meter bumping only on a free bit for 12 there. Python adds the lone worker at 5 and the 2 by 2 diagonal at 2, Lua its own 2 by 2 at 5.

1. best[000] = 0 seeds the sweep, and popcount of the mask names the worker placing next.
2. Mask 000 calls worker 0: task 0 costs 0 + 1 = 1 while tasks 1 and 2 cost 100 each, so best[001] = 1 and best[010] = best[100] = 100.
3. Mask 001 calls worker 1: task 1 costs 1 + 50 = 51 and task 2 costs 1 + 60 = 61, the cross move greedy will miss.
4. Masks 010 and 100 also call worker 1: their offers, 100 + 2 = 102 and 100 + 50 = 150, lose where they meet the cheaper parents.
5. The full mask fills from the three two-bit states: 51 + 80 = 131, 61 + 40 = 101, 150 + 3 = 153, and best[111] keeps 101.
6. The 101 road is the cross assignment, worker 1 on task 2 and worker 2 on task 1, and the brute-forced 3 by 3 case agrees exactly.

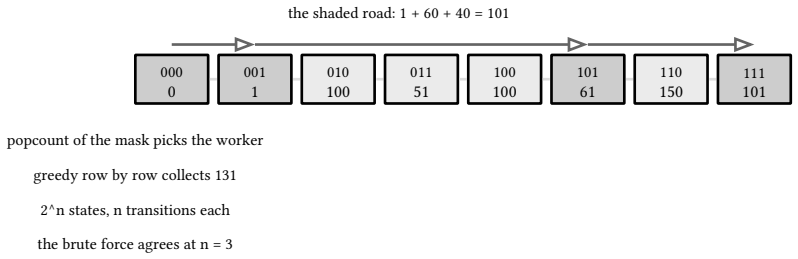


figure 146: the sweep over the eight masks, best costs landing in numeric order, the winning road 000 to 001 to 101 to 111 shaded

The 101 against greedy's 131 is the pinned spread, and the listings below sweep the subsets seven ways.

```
    /// <summary>Bitmask dp over subsets: assignment problem, n workers to n tasks.</summary>
    public static long AssignmentCost(long[,] cost)
    {
        int n = cost.GetLength(0);
        int full = (1 << n) - 1;
        var best = new long[1 << n];
        Array.Fill(best, long.MaxValue);
        best[0] = 0;

        for (int mask = 0; mask <= full; mask++)
        {
            if (best[mask] == long.MaxValue)
                continue;
            int worker = System.Numerics.BitOperations.PopCount((uint)mask);
            for (int task = 0; task < n; task++)
            {
                int bit = 1 << task;
                if ((mask & bit) != 0)
                    continue;
                long candidate = best[mask] + cost[worker, task];
                int next = mask | bit;
                if (candidate < best[next])
                    best[next] = candidate;
            }
        }
        return best[full];
    }
}
```

listing 541: assignment over subsets, popcount picks the worker (source: dsa/samples/src/Ch17/Dp.cs, lines 170-197)

The state is which tasks are done, and popcount of the mask says which worker places next. Twelve workers is four thousand ninety-six states times twelve transitions, while brute force over permutations is four hundred seventy-nine million, and the test brute-forces the three by three case to confirm exact agreement. Exactness ends at the width of the mask: past about twenty tasks the table stops fitting memory, and book 9, ch 42 (practical data structures and algorithms, approximation algorithms) picks up with provable ratios in place of exact answers.

```c
static int popcount(int mask) {
  int bits = 0;
  while (mask) {
    bits += mask & 1;
    mask >>= 1;
  }
  return bits;
}

// best[mask] = cheapest way to assign the popcount(mask) lowest workers
// to exactly the tasks in mask
static long assignment_cost(const long *cost, int nworkers, long *rows,
                            long *attempts) {
  int full = (1 << nworkers) - 1;
  *rows = 0;
  *attempts = 0;
  for (int mask = 0; mask <= full; mask++)
    best[mask] = LONG_MAX;
  best[0] = 0;
  for (int mask = 0; mask <= full; mask++) {
    if (best[mask] == LONG_MAX)
      continue; // unreachable mask, the c# MaxValue guard
    (*rows)++;
    int worker = popcount(mask);
    for (int task = 0; task < nworkers; task++) {
      (*attempts)++;
      int bit = 1 << task;
      if ((mask & bit) != 0)
        continue;
      long candidate = best[mask] + cost[worker * nworkers + task];
      if (candidate < best[mask | bit])
        best[mask | bit] = candidate;
    }
  }
  return best[full];
}
```

listing 542: c, popcount by hand, the mask sweep with rows and attempts metered (source: dsa/samples-c/src/Ch17/assignment.c, lines 22-57)

```
// popcount counts set bits by hand: the worker that moves next is the
// population count of the task mask.
func popcount(mask int) int {
  count := 0
  for mask ≠ 0 {
    count += mask & 1
    mask >>= 1
  }
  return count
}

// AssignmentCost solves the n workers to n tasks assignment problem
// by mask DP: best[mask] is the cheapest way to finish the tasks in
// mask using the first popcount(mask) workers, and each relaxation
// hands one more task to the next worker. Unreachable masks are
// skipped, mirroring the sentinel guard.
func AssignmentCost(cost [][]int64, meter *assignmentMeter) int64 {
  n := len(cost)
  full := 1<<n - 1
  unreachable := int64(1) << 62
  best := make([]int64, 1<<n)
  for mask := range best {
    best[mask] = unreachable
  }
  best[0] = 0
  for mask := 0; mask <= full; mask++ {
    if best[mask] == unreachable {
      continue
    }
    if meter ≠ nil {
      meter.masks++
    }
    worker := popcount(mask)
    for task := 0; task < n; task++ {
      bit := 1 << task
      // an attempted relaxation covers the whole (mask, task)
      // pair space n*2^n; the bit test refuses taken tasks
      if meter ≠ nil {
        meter.relaxations++
      }
      if mask&bit ≠ 0 {
        continue
      }
      candidate := best[mask] + cost[worker][task]
      if candidate < best[mask|bit] {
        best[mask|bit] = candidate
      }
    }
  }
  return best[full]
}
```

listing 543: go, popcount and the sweep, relaxations counted before the bit test (source: dsa/samples-go/ch17/assignment.go, lines 10-60)

```
  // best[mask] = cheapest way to assign the popcount(mask) lowest
  // workers to exactly the tasks in mask
  record Result(long value, long rows, long attempts) {}

  static long[] best = new long[1 << 4];

  static Result assignmentCost(long[][] cost) {
    int workers = cost.length;
    int full = (1 << workers) - 1;
    long rows = 0, attempts = 0;
    java.util.Arrays.fill(best, 0, full + 1, UNREACHED);
    best[0] = 0;
    for (int mask = 0; mask ≤ full; mask++) {
      if (best[mask] == UNREACHED)
        continue; // unreachable mask, the MaxValue guard
      rows++;
      int worker = Integer.bitCount(mask);
      for (int task = 0; task < workers; task++) {
        attempts++;
        int bit = 1 << task;
        if ((mask & bit) ≠ 0)
          continue;
        long candidate = best[mask] + cost[worker][task];
        if (candidate < best[mask | bit])
          best[mask | bit] = candidate;
      }
    }
    return new Result(best[full], rows, attempts);
  }
```

listing 544: java, Integer.bitCount names the worker, rows and attempts returned in a record (source: dsa/samples-java/src/Ch17/Assignment.java, lines 21-49)

```javascript
// ch17: assignment by bitmask dp over task subsets. the popcount of the
// mask picks the worker who moves next, unreachable masks are skipped the
// way the C# MaxValue guard does

export function assignmentCost(cost, meter = null) {
  const n = cost.length;
  const full = (1 << n) - 1;
  const best = new Array(1 << n).fill(Infinity);
  best[0] = 0;

  for (let mask = 0; mask <= full; mask++) {
    if (best[mask] === Infinity) continue;
    if (meter !== null) meter.rowsTouched++;
    let worker = 0;
    for (let b = 0; b < n; b++) if ((mask & (1 << b)) !== 0) worker++;
    for (let task = 0; task < n; task++) {
      if (meter !== null) meter.taskAttempts++;
      const bit = 1 << task;
      if ((mask & bit) !== 0) continue;
      const candidate = best[mask] + cost[worker][task];
      const next = mask | bit;
      if (candidate < best[next]) best[next] = candidate;
    }
  }
  return best[full];
}
```

listing 545: javascript, the worker counted by a bit loop, Infinity for unreachable (source: dsa/samples-js/src/ch17-assignment.mjs, lines 1-26)

```python
def assignment_cost(cost):
    # best[mask]: cheapest way to assign the tasks in mask to the first
    # popcount(mask) workers. unreachable masks never run their task loop
    size = len(cost)
    full = (1 << size) - 1
    best = [None] * (1 << size)
    best[0] = 0
    rows = 0
    attempts = 0
    for mask in range(1 << size):
        if best[mask] is None:
            continue  # the unreachable guard: skip dead masks entirely
        rows += 1
        worker = mask.bit_count()
        for task in range(size):
            attempts += 1
            if mask & (1 << task):
                continue
            candidate = best[mask] + cost[worker][task]
            nxt = mask | (1 << task)
            if best[nxt] is None or candidate < best[nxt]:
                best[nxt] = candidate
    return best[full], rows, attempts
```

listing 546: python, bit_count names the worker, None for unreachable masks (source: dsa/samples-py/src/Ch17/assignment.py, lines 16-38)

```lua
-- popcount by hand: one bit at a time, integers only
local function popcount(mask)
  local n = 0
  while mask > 0 do
    n = n + (mask & 1)
    mask = mask >> 1
  end
  return n
end

-- returns the optimum plus table shape meters: masks visited and
-- relaxations attempted (candidate evaluated)
local function assignment_cost(cost, meter)
  local n = #cost
  local full = (1 << n) - 1
  local best = {[0] = 0}
  for mask = 1, full do best[mask] = nil end -- unreachable until relaxed
  for mask = 0, full do
    if best[mask] ~= nil then
      if meter then meter.rows = meter.rows + 1 end
      local worker = popcount(mask)
      for task = 1, n do
        local bit = 1 << (task - 1)
        if mask & bit == 0 then
          if meter then meter.relax = meter.relax + 1 end
          local candidate = best[mask] + cost[worker + 1][task]
          local nxt = mask | bit
          if best[nxt] == nil or candidate < best[nxt] then
            best[nxt] = candidate
          end
        end
      end
    end
  end
  return best[full], best
end
```

listing 547: lua, popcount by hand, nil marks the unreachable masks (source: dsa/samples-lua/ch17_assignment.lua, lines 6-41)

Measured across the suites: the optimum 101 with greedy at 131 and the 4 by 4 at 10 pin in all seven, every sibling cross-checking the table against a brute-force walk over permutations. The meters agree on the rows, every reachable mask touched, and on the attempts, 24 at n = 3 in C, Go, Java, JavaScript, and Python where the bump lands once per (mask, task) pair, Lua counting only evaluated free bits for 12. Popcount comes from BitOperations in C#, `Integer.bitCount` in Java, bit_count in Python, and a hand-rolled bit loop in the other three.

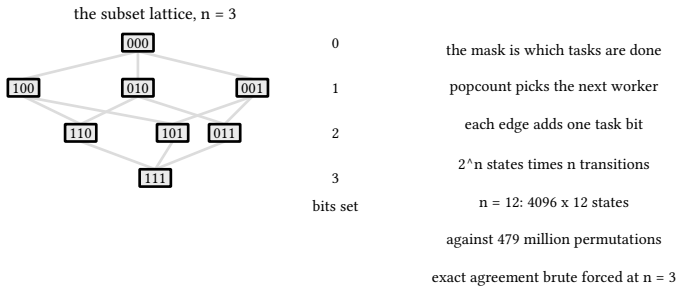


figure 147: the subset lattice as the state space, the mask is which tasks are done, popcount picks the next worker

17.8. across the seven languages

The build sizes count non-comment source lines. The first table covers the chapter's original featured files, the C# row counting `Dp.cs` plus the lis section of the chapter 19 file where its tails build lives:

language	build SLOC	dependency	boundary note
c	343	libc only	memo arrays sized 64, fib(50) in long long, ladder and fib cross-tied
c#	231	bcl only	Dictionary memo, BitOperations.PopCount for the subset dp, unreachable coins read minus one
go	232	slices	slices.BinarySearch inside lis tails, map memo counting real evaluations only
java	517	jdk 27 stdlib	seen-flag long arrays mirror C's memo, the gridpaths memo a HashMap keyed on r and c packed into one long, the coins sentinel shared with C
javascript	132	node stdlib	fib(78) = 8944394323798636 is the last exact Number, stated at the top of the memo file
python	299	stdlib only	dict memo by hand, functools.cache named as the counterpart, brute cross-checks in three files
lua	346	lib.lua harness	two-row edit distance never builds the table, integer midpoint search, mask brute force

The assignment sweep lands as its own file in the six sibling trees, the C# build staying inside the chapter's one Dp.cs:

language	build SLOC	dependency	boundary note
c	115	libc only	popcount by hand, rows and attempts through out parameters, a next-permutation oracle and greedy contrast in the same file
c#	26	bcl only	AssignmentCost inside Dp.cs, BitOperations.PopCount, the MaxValue guard skips dead masks
go	62	slices	the meter behind a pointer, relaxations counted before the bit test, greedy rows beside it
java	113	jdk 27 stdlib	Integer.bitCount names the worker, the unreachable guard skips dead masks, rows and attempts ride a record back
javascript	21	node stdlib	the worker counted by a bit loop, Infinity for unreachable, an optional meter object
python	73	stdlib only	None for unreachable masks, bit_count names the worker, permutation oracle and greedy rows in the checks
lua	111	lib.lua harness	nil marks unreachable, the relax meter bumps on free bits only, a factorial-walk oracle

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` used by the memo, `BitOperations.PopCount`, `Array.Fill`, accessed 2026-09-08, go.dev/pkg/slices for `BinarySearch`, accessed 2026-09-14. Sample behavior verified by `make verify-csharp`, 12 tests in chapter 17 of the samples suite. The seven-language layer verifies the same way: 7 C programs with 97 embedded checks under `make verify-c`, 7 Ch17 java programs with 118 checks under `run-java-samples`, 26 Go tests, 22 `node --test` cases, 76 Python checks across 7 files, and 32 Lua checks under `run.lua`.

18. greedy and backtracking

Greedy algorithms commit to the locally best choice and never look back. Sometimes that is provably optimal, and the proof shape is the interesting part. When no such proof exists, backtracking searches everything with pruning, the honest fallback. This chapter builds both with their meters on, seven languages deep on the greedy side.

18.1. activity selection and rooms

The exchange argument: any optimal schedule can be rearranged to start with the earliest-finishing activity, so taking it loses nothing. Sort by finish, take what fits.

The dry run: the fixtures are the eleven-interval CLRS set and the three rooms cases, asserted by the C# suite, while C, Go, JavaScript, and Lua pin the same CLRS answer and Python runs its own six intervals.

1. Sorted by finish the queue reads 1-4, 3-5, 0-6, 5-7, 3-9, 5-9, 6-10, 8-11, 8-12, 2-14, 12-16, and 1-4 is taken at once, the last finish moving to 4.
2. 3-5 starts at 3 and 0-6 at 0, both inside 4, both skipped.
3. 5-7 starts at 5, past 4, taken, the frontier at 7, and 3-9, 5-9, 6-10 all start below 7 and die.
4. 8-11 starts at 8, taken, the frontier at 11, and 8-12 and 2-14 die the same way.
5. 12-16 starts at 12 and the schedule closes at four picks, 1-4, 5-7, 8-11, 12-16.
6. Rooms on 0-30, 5-10, 15-20: nothing frees before 30, the second meeting piles on, and the third reuses the room freed at 10, a peak of 2.
7. Back-to-back 0-1, 1-2, 2-3 shares one room, and the pile 0-10, 1-9, 2-8 frees nothing, 3 rooms.

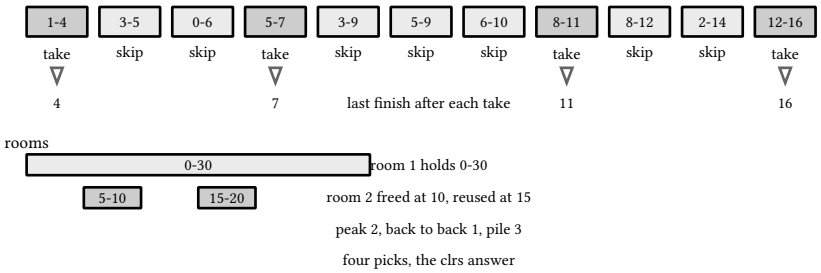


figure 148: the walk in finish-sorted order, taken intervals shaded, the frontier advancing pick by pick, the rooms trio beside it

The four picks and the three-room pile are the pinned landings, and the listings below run the scan in seven languages, rooms in the C# one.

```c
static int cmp_act(const void *x, const void *y) {
  const Act *a = x, *b = y;
  return (a->finish > b->finish) - (a->finish < b->finish);
}

// sorts by finish, keeps every activity that starts after the last pick
static int select_max(Act *acts, int len, Act *chosen) {
  qsort(acts, (size_t)len, sizeof(Act), cmp_act);
  int k = 0, last_finish = -1;
  for (int i = 0; i < len; i++)
    if (acts[i].start >= last_finish) {
      chosen[k++] = acts[i];
      last_finish = acts[i].finish;
    }
  return k;
}
```

listing 548: c, qsort by finish, keep what starts after the last pick (source: dsa/samples-c/src/Ch18/intervals.c, lines 21-36)

```go
// ActivitySelection returns the largest set of non-overlapping
// intervals: sort by end, keep whatever starts after the last chosen
// activity finished. The earliest-finishing choice is always safe,
// which is what makes this a proof rather than a heuristic.
func ActivitySelection(intervals []Interval) []Interval {
  sorted := slices.Clone(intervals)
  slices.SortFunc(sorted, func(a, b Interval) int { return a.End - b.End })
  var chosen []Interval
  var freeFrom int
  for _, iv := range sorted {
    if iv.Start ≥ freeFrom {
      chosen = append(chosen, iv)
      freeFrom = iv.End
    }
  }
  return chosen
}
```

listing 549: go, half-open intervals, sort a clone, keep the compatible (source: dsa/samples-go/ch18/intervals.go, lines 14-30)

```java
  record Act(int start, int finish) {}

  // sorts by finish, keeps every activity that starts after the last pick
  static List<Act> selectMax(Act[] acts) {
    Act[] sorted = acts.clone();
    Arrays.sort(sorted, (x, y) -> Integer.compare(x.finish, y.finish));
    List<Act> chosen = new ArrayList<>();
    int lastFinish = -1;
    for (Act a : sorted)
      if (a.start() ≥ lastFinish) {
        chosen.add(a);
        lastFinish = a.finish();
      }
    return chosen;
  }

  // exhaustive cross-check over every subset, the py brute lane
  static int bruteMax(Act[] acts) {
    int best = 0;
    for (int mask = 1; mask < 1 << acts.length; mask++) {
      Act[] subset = new Act[Integer.bitCount(mask)];
      int k = 0;
      for (int i = 0; i < acts.length; i++)
        if ((mask & (1 << i)) ≠ 0)
          subset[k++] = acts[i];
      Arrays.sort(subset, (x, y) -> Integer.compare(x.finish, y.finish));
      boolean fits = true;
      for (int i = 0; i + 1 < subset.length; i++)
        if (subset[i].finish() > subset[i + 1].start())
          fits = false;
      if (fits && subset.length > best)
        best = subset.length;
    }
    return best;
  }
```

listing 550: java, a record per interval, sort by finish, a bitmask brute cross-check (source: dsa/samples-java/src/Ch18/Intervals.java, lines 20-54)

```
public static class Greedy
{
    public sealed record Interval(int Start, int End);

    /// <summary>Activity selection: sort by finish time, take every compatible one.</summary>
    public static List<Interval> Activities(List<Interval> intervals)
    {
        var chosen = new List<Interval>();
        int lastEnd = int.MinValue;
        foreach (Interval i in intervals.OrderBy(i => i.End))
        {
            if (i.Start ≥ lastEnd)
            {
                chosen.Add(i);
                lastEnd = i.End;
            }
        }
        return chosen;
    }

    /// <summary>Interval partitioning: minimum rooms, heap of end times.</summary>
    public static int Rooms(List<Interval> intervals)
    {
        var byStart = intervals.OrderBy(i => i.Start).ToList();
        var ends = new DsaBook.Samples.Ch08.BinaryHeap<long>();
        int rooms = 0;
        foreach (Interval i in byStart)
        {
            while (ends.Count > 0 && ends.PeekMin() ≤ i.Start)
                ends.ExtractMin(); // a room freed up before this start
            ends.Add(i.End);
            rooms = Math.Max(rooms, ends.Count);
        }
        return rooms;
    }
```

listing 551: c#, activity selection, then interval partitioning through a room heap (source: dsa/samples/src/Ch18/Greedy.cs, lines 6-40)

```
export function activitySelection(intervals) {
  const byFinish = [...intervals].sort((a, b) => a[1] - b[1]);
  const chosen = [];
  let lastFinish = -Infinity;
  for (const [s, e] of byFinish) {
    if (s ≥ lastFinish) {
      chosen.push([s, e]);
      lastFinish = e;
    }
  }
  return chosen;
}
```

listing 552: javascript, stable sort by finish, minus infinity start (source: dsa/samples-js/src/ch18-intervals.mjs, lines 4-15)

```python
def select_activities(intervals):
    # intervals: (start, end) pairs, end exclusive
    chosen = []
    free_at = -1
    for start, end in sorted(intervals, key=lambda iv: iv[1]):
        if start ≥ free_at:  # compatible with everything chosen so far
            chosen.append((start, end))
            free_at = end
    return chosen
```

listing 553: python, sort by end, plus an exhaustive brute cross-check (source: dsa/samples-py/src/Ch18/intervals.py, lines 15-23)

```lua
local function activities(intervals)
  local sorted = {table.unpack(intervals)}
  table.sort(sorted, function(x, y)
    if x[2] ~= y[2] then return x[2] < y[2] end
    return x[1] < y[1]
  end)
  local chosen = {}
  local last_finish = -math.huge
  for _, iv in ipairs(sorted) do
    if iv[1] ≥ last_finish then
      chosen[#chosen + 1] = iv
      last_finish = iv[2]
    end
  end
  return chosen
end
```

listing 554: lua, table.sort by finish then start, the clrs fixture (source: dsa/samples-lua/ch18_intervals.lua, lines 7-22)

The rooms problem is greedy by start time with chapter 8's heap holding room release times, and the maximum simultaneous occupancy is the answer. The tests use the classic eleven-interval CLRS set, which selects exactly four, and the boundary cases: back-to-back meetings share one room, three overlapping meetings need three.

The eleven-interval CLRS fixture and its answer, four picks at 1-4, 5-7, 8-11, 12-16, pin in C, Go, Java, C#, JavaScript, and Lua. Python and Java run their own six-interval set with the pinned choice of three, Python then doing what only it does cheaply, an exhaustive `itertools.combinations` sweep, Java proving no larger compatible subset exists by enumerating every subset mask. Lua checks maximality too, by hand: every discarded interval must overlap an accepted one. Back-to-back compatibility, a start equal to the last finish, is asserted everywhere, and the all-stack fixture pins the earliest finisher surviving alone.

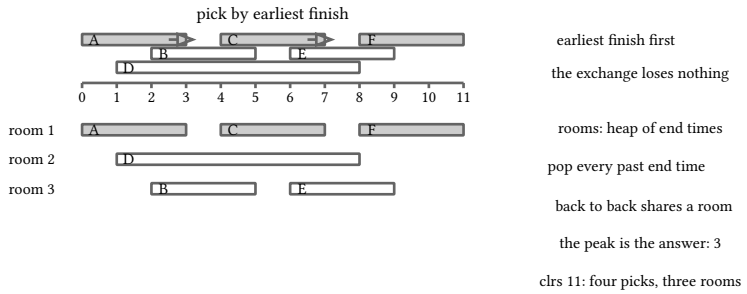


figure 149: intervals on a time axis, earliest finish wins the schedule, the heap of end times counts the rooms

18.2. huffman

Huffman coding is greedy with a proof: repeatedly merge the two lightest symbols, and the resulting prefix-free code minimizes weighted code length. The heap from chapter 8 does the selecting.

The dry run: the fixture is the CLRS weights 45, 13, 12, 16, 9, 5, pinned by the C, Go, Java, JavaScript, and Lua suites at cost 224, while C# asserts the round trip and the one-bit lone symbol and Python and Java run 5, 2, 1, 1 at cost 15.

1. Round 1 marries the two lightest, 5 and 9: 5 + 9 = 14.
2. Round 2: the next pair is 12 and 13: 12 + 13 = 25.
3. Round 3: the 14 meets 16: 14 + 16 = 30.
4. Round 4: 25 + 30 = 55.
5. Round 5 closes the tree: 45 + 55 = 100.
6. The merge weights sum to the cost, 14 + 25 + 30 + 55 + 100 = 224, the same 224 read again as weight times depth with lengths a 1, b, c, d 3, e, f 4.

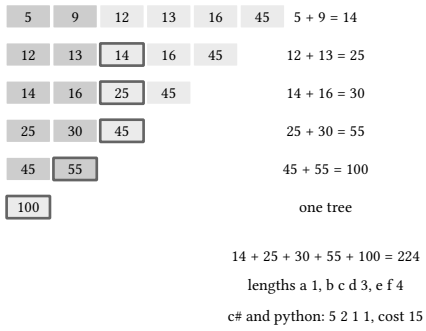


figure 150: the merge loop as one row per round, the two lightest shaded on each row, the fresh merged weight boxed

The 224 lands as both the merge sum and the weighted length, and the listings below build this tree in seven languages.

```c
// repeatedly marry the two lightest nodes; ties break on node index
static unsigned long long build(const unsigned long long *freqs, int count,
                                int *leaf_index) {
  node_n = 0;
  unsigned long long merge_sum = 0;
  // leaves first, in input order
  for (int i = 0; i < count; i++) {
    nodes[node_n].weight = freqs[i];
    nodes[node_n].left = nodes[node_n].right = -1;
    nodes[node_n].parent = -1;
    leaf_index[i] = node_n++;
  }
  for (int round = 0; round < count - 1; round++) {
    // find the two smallest parentless weights
    int a = -1, b = -1;
    for (int i = 0; i < node_n; i++) {
      if (nodes[i].parent >= 0)
        continue;
      if (a < 0 || nodes[i].weight < nodes[a].weight) {
        b = a;
        a = i;
      } else if (b < 0 || nodes[i].weight < nodes[b].weight) {
        b = i;
      }
    }
    HN *p = &nodes[node_n];
    p->weight = nodes[a].weight + nodes[b].weight;
    p->left = a;
    p->right = b;
    p->parent = -1;
    nodes[a].parent = node_n;
    nodes[b].parent = node_n;
    merge_sum += p->weight;
    node_n++;
  }
  // depths by walking parents
  for (int i = 0; i < count; i++) {
    int d = 0;
    for (int j = leaf_index[i]; nodes[j].parent >= 0; j = nodes[j].parent)
      d++;
    nodes[leaf_index[i]].depth = d;
  }
  return merge_sum;
}

static unsigned long long total_cost(const unsigned long long *freqs, int count,
                                     const int *leaf) {
  unsigned long long cost = 0;
  for (int i = 0; i < count; i++)
    cost += freqs[i] * (unsigned long long)nodes[leaf[i]].depth;
  return cost;
}
```

listing 555: c, marry the two lightest, depths by parent walks, the identity (source: dsa/samples-c/src/Ch18/huffman.c, lines 29-80)

```go
// HuffmanCodes builds the prefix code: repeatedly marry the two
// lowest-frequency nodes into a parent whose frequency is the sum.
// Code lengths fall out of the tree depth, and the total cost is the
// sum of freq * depth over every symbol.
func HuffmanCodes(freqs map[byte]int) (lengths map[byte]int, totalCost int) {
  var forest []*huffNode
  for symbol, f := range freqs {
    forest = append(forest, &huffNode{freq: f, symbol: symbol})
  }
  if len(forest) == 0 {
    return map[byte]int{}, 0
  }
  for len(forest) > 1 {
    // two min-scans instead of a heap: the tree is tiny and the
    // selection is the lesson
    first := 0
    for i := range forest {
      if forest[i].freq < forest[first].freq {
        first = i
      }
    }
    a := forest[first]
    forest = append(forest[:first], forest[first+1:]...)
    second := 0
    for i := range forest {
      if forest[i].freq < forest[second].freq {
        second = i
      }
    }
    b := forest[second]
    forest = append(forest[:second], forest[second+1:]...)
    forest = append(forest, &huffNode{freq: a.freq + b.freq, left: a, right: b})
  }
  lengths = make(map[byte]int, len(freqs))
  var assign func(n *huffNode, depth int)
  assign = func(n *huffNode, depth int) {
    if n.left == nil && n.right == nil {
      if depth == 0 { // single-symbol alphabet: still needs a code
        depth = 1
      }
      lengths[n.symbol] = depth
      totalCost += n.freq * depth
      return
    }
    assign(n.left, depth+1)
    assign(n.right, depth+1)
  }
  assign(forest[0], 0)
  return lengths, totalCost
}
```

listing 556: go, two min scans marry the lightest, lengths by depth walk (source: dsa/samples-go/ch18/huffman.go, lines 11-60)

```
  static long build(long[] freqs, int[] leafIndex) {
    int count = freqs.length;
    nodeN = 0;
    long mergeSum = 0;
    // leaves first, in input order
    for (int i = 0; i < count; i++) {
      weight[nodeN] = freqs[i];
      left[nodeN] = right[nodeN] = -1;
      parent[nodeN] = -1;
      leafIndex[i] = nodeN++;
    }
    for (int round = 0; round < count - 1; round++) {
      // find the two smallest parentless weights
      int a = -1, b = -1;
      for (int i = 0; i < nodeN; i++) {
        if (parent[i] ≥ 0)
          continue;
        if (a < 0 || weight[i] < weight[a]) {
          b = a;
          a = i;
        } else if (b < 0 || weight[i] < weight[b]) {
          b = i;
        }
      }
      weight[nodeN] = weight[a] + weight[b];
      left[nodeN] = a;
      right[nodeN] = b;
      parent[nodeN] = -1;
      parent[a] = nodeN;
      parent[b] = nodeN;
      mergeSum += weight[nodeN];
      nodeN++;
    }
    // depths by walking parents
    for (int i = 0; i < count; i++) {
      int d = 0;
      for (int j = leafIndex[i]; parent[j] ≥ 0; j = parent[j])
        d++;
      depth[leafIndex[i]] = d;
    }
    return mergeSum;
  }

  static long totalCost(long[] freqs, int[] leaf) {
    long cost = 0;
    for (int i = 0; i < freqs.length; i++)
      cost += freqs[i] * depth[leaf[i]];
    return cost;
  }
```

listing 557: java, one scan holds both lightest nodes, depths by parent walks, the identity (source: dsa/samples-java/src/Ch18/Huffman.java, lines 26-74)

```
  /// <summary>Huffman coding: prefix-free codes from a heap, chapter 8 at work.</summary>
  public sealed class Huffman
  {
      public sealed record Code(string Symbol, string Bits, long Weight);
```

```
        private sealed class Node : IComparable<Node>
        {
            public readonly string Symbols;
            public readonly long Weight;
            public readonly Node? Left, Right;

            public Node(string symbol, long weight)
            {
                Symbols = symbol;
                Weight = weight;
            }

            public Node(Node left, Node right)
            {
                Left = left;
                Right = right;
                Symbols = left.Symbols + right.Symbols;
                Weight = left.Weight + right.Weight;
            }

            public int CompareTo(Node? other) => Weight.CompareTo(other!.Weight);
        }

        private readonly Dictionary<char, string> _codes = new();
        public readonly long TotalWeight;

        public Huffman(string text)
        {
            var counts = new Dictionary<char, long>();
            foreach (char c in text)
                counts[c] = counts.GetValueOrDefault(c) + 1;
            TotalWeight = text.Length;

            var heap = new DsaBook.Samples.Ch08.BinaryHeap<Node>();
            foreach ((char c, long w) in counts)
                heap.Add(new Node(c.ToString(), w));

            if (heap.Count == 1)
            {
                _codes[text[0]] = "0"; // a single symbol still needs one bit
                return;
            }

            Node root = MergeAll(heap);
            Walk(root, "");
        }

        private static Node MergeAll(DsaBook.Samples.Ch08.BinaryHeap<Node> heap)
        {
            while (heap.Count > 1)
                heap.Add(new Node(heap.ExtractMin(), heap.ExtractMin()));
            return heap.ExtractMin();
        }

        private void Walk(Node node, string prefix)
        {
```

```
            if (node.Left == null)
            {
                _codes[node.Symbols[0]] = prefix;
                return;
            }
            Walk(node.Left, prefix + "0");
            Walk(node.Right!, prefix + "1");
        }

        public string Encode(string text) =>
            string.Concat(text.Select(c => _codes[c]));
```

listing 558: c#, the huffman machine, heap merging, code walking, encode (source: dsa/samples/src/Ch18/Greedy.cs, lines 42-113)

```
        public string Decode(string bits)
        {
            var reverse = _codes.ToDictionary(kv => kv.Value, kv => kv.Key);
            var result = new StringBuilder();
            var current = new StringBuilder();
            foreach (char b in bits)
            {
                current.Append(b);
                if (reverse.TryGetValue(current.ToString(), out char symbol))
                {
                    result.Append(symbol);
                    current.Clear();
                }
            }
            if (current.Length > 0)
                throw new ArgumentException("bits are not a valid encoding");
            return result.ToString();
        }

        /// <summary>Weighted code length, the quantity huffman minimizes.</summary>
        public long Cost(string text)
        {
            var counts = new Dictionary<char, long>();
            foreach (char c in text)
                counts[c] = counts.GetValueOrDefault(c) + 1;
            return counts.Sum(kv => kv.Value * _codes[kv.Key].Length);
        }

        public List<Code> Codes(string text)
        {
            var counts = new Dictionary<char, long>();
            foreach (char c in text)
                counts[c] = counts.GetValueOrDefault(c) + 1;
            return _codes.OrderBy(kv => kv.Key)
                .Select(kv => new Code(kv.Key.ToString(), kv.Value, counts[kv.Key]))
                .ToList();
        }
    }

    /// <summary>Fractional knapsack: greedy is optimal when items can split.</summary>
    public static double FractionalKnapsack((int Weight, int Value)[] items, int capacity)
    {
        double taken = 0;
        int room = capacity;
        foreach ((int w, int v) in items.OrderByDescending(i => (double)i.Value / i.Weight))
        {
            if (room <= 0)
                break;
            int use = Math.Min(w, room);
            taken += (double)v * use / w;
            room -= use;
        }
        return taken;
    }
```

listing 559: c#, decode with the reverse table, weighted cost, the fractional knapsack contrast (source: dsa/samples/src/Ch18/Greedy.cs, lines 114-168)

```
export function huffman(freqs) {
  // local min-heap keyed by (weight, insertion sequence)
  const heap = [];
  let seq = 0;
  const push = (node) => {
    node.seq = seq++;
    heap.push(node);
    let i = heap.length - 1;
    while (i > 0) {
      const p = Math.floor((i - 1) / 2);
      const before =
        heap[p].weight > heap[i].weight ||
        (heap[p].weight === heap[i].weight && heap[p].seq > heap[i].seq);
      if (!before) break;
      [heap[i], heap[p]] = [heap[p], heap[i]];
      i = p;
    }
  };
  const pop = () => {
    const top = heap[0];
    heap[0] = heap[heap.length - 1];
    heap.pop();
    let i = 0;
    for (;;) {
      const l = 2 * i + 1;
      const r = 2 * i + 2;
      let m = i;
      for (const k of [l, r]) {
        if (k < heap.length) {
          const better =
            heap[k].weight < heap[m].weight ||
            (heap[k].weight === heap[m].weight && heap[k].seq < heap[m].seq);
          if (better) m = k;
        }
      }
      if (m === i) break;
      [heap[i], heap[m]] = [heap[m], heap[i]];
      i = m;
    }
    return top;
  };
```

listing 560: javascript, a hand-built heap keyed by weight then insertion order (source: dsa/samples-js/src/ch18-huffman.mjs, lines 16-56)

```
    # freqs: {symbol: weight}; returns (lengths, total cost)
    nodes = [(weight, (sym,)) for sym, weight in freqs.items()]  # (weight, members)
    lengths = {sym: 0 for sym in freqs}
    while len(nodes) > 1:
        nodes.sort()  # a tiny fixture: sort stands in for a priority queue
        (w1, m1), (w2, m2) = nodes[0], nodes[1]
        nodes = nodes[2:] + [(w1 + w2, m1 + m2)]  # merge the two lightest
        for sym in m1 + m2:
            lengths[sym] += 1  # one level deeper for both members
    return lengths, sum(freqs[s] * lengths[s] for s in freqs)


FREQS = {"A": 5, "B": 2, "C": 1, "D": 1}
lengths, cost = huffman_lengths(FREQS)
check(lengths == {"A": 1, "B": 2, "C": 3, "D": 3}, "code lengths 1, 2, 3, 3")
```

listing 561: python, sort as the queue, member lists lift every merged symbol (source: dsa/samples-py/src/Ch18/huffman.py, lines 14-28)

```
local function huffman(freqs)
  -- nodes: {w = weight, ch = symbol or nil, left, right}
  local heap = {}
  for _, f in ipairs(freqs) do
    heap[#heap + 1] = {w = f[2], ch = f[1], seq = #heap + 1}
  end
  local cost = 0
  local seq = #heap
  while #heap > 1 do
    -- two lightest by (weight, insertion order): deterministic
    table.sort(heap, function(x, y)
      if x.w ~= y.w then return x.w < y.w end
      return x.seq < y.seq
    end)
    local a = table.remove(heap, 1)
    local b = table.remove(heap, 1)
    local merged = {w = a.w + b.w, left = a, right = b, seq = seq + 1}
    seq = seq + 1
    cost = cost + merged.w
    heap[#heap + 1] = merged
  end
  return heap[1], cost
end

-- code lengths by traversal; codes as bit strings
```

listing 562: lua, sort by weight then order, the merge sum accumulates (source: dsa/samples-lua/ch18_huffman.lua, lines 7-31)

The tests pin the properties rather than one lucky code: encode and decode round trip, no code prefixes another, frequent symbols get short codes, weighted cost beats fixed-length coding on skewed input, and the degenerate single-symbol alphabet still gets its one bit. The fractional knapsack beside it is the greedy that works precisely because splitting is allowed, and its 240 beats the zero-one table's 220 on the same items.

The CLRS six-symbol fixture, weights 45, 13, 12, 16, 9, 5, pins the same tree in C, Go, Java, JavaScript, and Lua: code lengths 1, 3, 3, 3, 4, 4 and total cost 224, which is also the sum of the merge weights, 14 plus 25 plus 30 plus 55 plus 100, an identity every one of those suites asserts. Python, C#, and Java run the four-symbol 5, 2, 1, 1 fixture with lengths 1, 2, 3, 3 and cost 15, and Python and Java add the Kraft equality, the code lengths summing to exactly 1 as negative powers of two. The selector is the per-language story: C scans for the two smallest, Go does two min

scans, Java finds both lightest nodes in one pass, keeping the two smallest seen so far with the node index as tie-break, Lua sorts the whole forest each round with an insertion-order tie-break, JavaScript hand-builds a binary heap with the same tie-break, and Python lets `sort` stand in for the priority queue because the fixture is tiny.

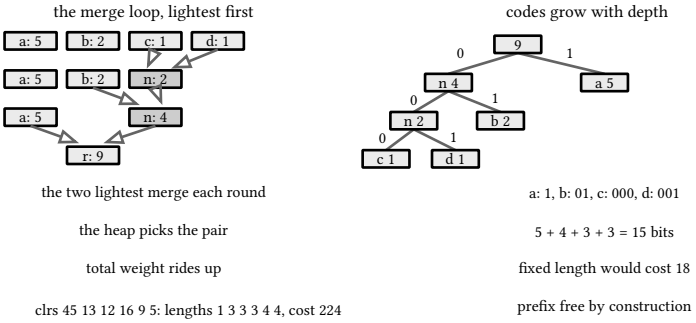


figure 151: huffman, the two lightest merge each round, codes lengthen with depth, prefix freedom holds by construction

18.3. the jump game

Reachability first: scan left to right keeping the farthest index within reach, and the moment the loop index passes that frontier the end is unreachable, no search needed. The minimum jump count is the same scan with level boundaries, the current jump covers a window, the next jump ends wherever the farthest reach inside the window says, and the counter ticks once per window. Both answers are greedy with proofs, the frontier never needs to shrink and the window walk is a breadth-first layering in disguise.

The dry run: the fixtures are 2, 3, 1, 1, 4 at 2 jumps and the trapped 3, 2, 1, 0, 4, asserted by the C# suite and pinned the same way in all six sibling suites.

1. The reach scan over 2, 3, 1, 1, 4 lifts the frontier twice: 0 + 2 = 2, then 1 + 3 = 4, and the frontier already covers the last index.
2. The level walk pays one hop over window 0 to 0, which reaches to 2, then a second over 1 to 2, which reaches 4: 2 jumps.
3. Over 3, 2, 1, 0, 4 the frontier moves once, 0 + 3 = 3, then stalls: 1 + 2 = 3, 2 + 1 = 3, 3 + 0 = 3.
4. Index 4 sits past the stalled frontier, the scan reports unreachable, and the level walk returns minus 1.
5. One step at a time over 1, 1, 1, 1 pays 3 hops, the single 5 over 5, 0, 0 clears in 1, and standing on 0 costs 0.

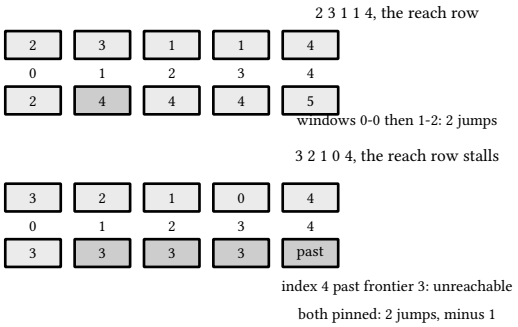


figure 152: two reach traces, the classic lifting its frontier to 4 at index 1, the trapped run stalling at 3 until index 4 passes it

The pair of 2 and minus 1 is the pinned landing, and the listings below run both scans in seven languages.

```c
static int can_reach(const int *a, int len) {
  int far = 0;
  for (int i = 0; i < len; i++) {
    if (i > far)
      return 0;
    if (i + a[i] > far)
      far = i + a[i];
  }
  return 1;
}

// greedy levels: the current jump covers [lo, hi]; the next level ends
// at the farthest reach inside it
static int min_jumps(const int *a, int len) {
  if (len ≤ 1)
    return 0;
  int jumps = 0, lo = 0, hi = 0;
  while (hi < len - 1) {
    int far = 0;
    for (int i = lo; i ≤ hi; i++) {
      if (i + a[i] > far)
        far = i + a[i];
    }
    if (far ≤ hi)
      return IMPOSSIBLE;
    lo = hi + 1;
    hi = far;
    jumps++;
  }
  return jumps;
}
```

listing 563: c, the running frontier, then the level windows (source: dsa/samples-c/src/Ch18/jumpgame.c, lines 19-49)

```go
func CanReach(a []int) bool {
  farthest := 0
  for i := 0; i <= farthest && i < len(a); i++ {
    if i+a[i] > farthest {
      farthest = i + a[i]
    }
  }
  return farthest ≥ len(a)-1
}

// MinJumps returns the fewest jumps from index 0 to the last index
// with the greedy window: the current jump covers [start, end], the
// next jump lands wherever the window's farthest reach allows.
// It returns -1 when the last index is unreachable.
func MinJumps(a []int) int {
  if len(a) <= 1 {
    return 0
  }
  jumps := 0
  end := 0 // the boundary of the current jump
  farthest := 0
  for i := 0; i < len(a)-1; i++ {
    if i+a[i] > farthest {
      farthest = i + a[i]
    }
    if i == end { // the jump is spent: commit another
      jumps++
      end = farthest
      if end ≥ len(a)-1 {
        return jumps
      }
      if end <= i { // stuck: no progress possible
        return -1
      }
    }
  }
  return -1
}
```

listing 564: go, reach by sweep, jumps by window boundary (source: dsa/samples-go/ch18/jumpgame.go, lines 6-43)

```java
static final int IMPOSSIBLE = -1;

static boolean canReach(int[] a) {
  int far = 0;
  for (int i = 0; i < a.length; i++) {
    if (i > far)
      return false; // this index never came into reach
    if (i + a[i] > far)
      far = i + a[i];
  }
  return true;
}

// greedy levels: the current jump covers [lo, hi]; the next level ends
// at the farthest reach inside it
static int minJumps(int[] a) {
  if (a.length ≤ 1)
    return 0;
  int jumps = 0, lo = 0, hi = 0;
  while (hi < a.length - 1) {
    int far = 0;
    for (int i = lo; i ≤ hi; i++)
      if (i + a[i] > far)
        far = i + a[i];
    if (far ≤ hi)
      return IMPOSSIBLE;
    lo = hi + 1;
    hi = far;
    jumps++;
  }
  return jumps;
}
```

listing 565: java, the frontier scan, then the level windows, minus one when stuck (source: dsa/samples-java/src/Ch18/Jumpgame.java, lines 16-47)

```
    {
        int far = 0;
        for (int i = 0; i < a.Length; i++)
        {
            if (i > far)
                return false;
            far = Math.Max(far, i + a[i]);
        }
        return true;
    }

    // greedy levels: the current jump covers [lo, hi], the next level ends
    // at the farthest reach inside it
    public static int MinJumps(ReadOnlySpan<int> a)
    {
        if (a.Length ≤ 1)
            return 0;
        int jumps = 0, lo = 0, hi = 0;
        while (hi < a.Length - 1)
        {
            int far = 0;
            for (int i = lo; i ≤ hi; i++)
                far = Math.Max(far, i + a[i]);
            if (far ≤ hi)
                return Impossible;
            lo = hi + 1;
            hi = far;
            jumps++;
        }
        return jumps;
    }
}
```

listing 566: c#, the running frontier, then the level windows, spans over the array (source: dsa/samples/src/Ch18/JumpGame.cs, lines 13-44)

```javascript
export function canJump(nums) {
  let reach = 0;
  for (let i = 0; i < nums.length; i++) {
    if (i > reach) return false;
    reach = Math.max(reach, i + nums[i]);
  }
  return true;
}

// minimum jumps, current-range greedy: count how many windows the
// frontier needs to cover the last index
export function minJumps(nums) {
  if (nums.length ≤ 1) return 0;
  let jumps = 0;
  let end = 0; // right edge of the current jump's reach
  let far = 0; // furthest reachable from the current window
  for (let i = 0; i < nums.length - 1; i++) {
    far = Math.max(far, i + nums[i]);
    if (i === end) {
      jumps++;
      end = far;
      if (end ≥ nums.length - 1) return jumps;
    }
  }
  return -1; // the frontier never advances: unreachable
}
```

listing 567: javascript, the frontier, then the window commit (source: dsa/samples-js/src/ch18-jumpgame.mjs, lines 4-29)

```python
def can_reach(nums):
    furthest = 0
    for i, jump in enumerate(nums):
        if i > furthest:
            return False  # this index never came into reach
        furthest = max(furthest, i + jump)
    return True


def min_jumps(nums):
    jumps = 0
    end_of_level = 0  # the last index reachable with `jumps` hops
    furthest = 0
    for i in range(len(nums) - 1):
        furthest = max(furthest, i + nums[i])
        if i == end_of_level:  # the level is exhausted: pay one hop
            jumps += 1
            end_of_level = furthest
            if end_of_level ≤ i:
                return None  # stuck: no progress left
    return jumps
```

listing 568: python, furthest reach, end of level pays one hop (source: dsa/samples-py/src/Ch18/jumpgame.py, lines 13-33)

```lua
local function can_reach(a)
  local reach = 0
  for i = 1, #a do
    if i - 1 > reach then return false end
    if i - 1 + a[i] > reach then reach = i - 1 + a[i] end
  end
  return true
end

-- fewest jumps, or -1 when the end is unreachable
local function min_jumps(a)
  local n = #a
  if n == 1 then return 0 end
  local jumps, cur_end, farthest = 0, 0, 0
  for i = 1, n - 1 do
    if i - 1 + a[i] > farthest then farthest = i - 1 + a[i] end
    if i - 1 == cur_end then
      jumps = jumps + 1
      cur_end = farthest
      if cur_end ≥ n - 1 then return jumps end
      if cur_end == i - 1 then return -1 end -- no progress possible
    end
  end
  return -1
end
```

listing 569: lua, 1-based indices shifted down, the same two scans (source: dsa/samples-lua/ch18_jumpgame.lua, lines 6-30)

The classic anchors hold in every suite that carries the topic: 2, 3, 1, 1, 4 is reachable in 2 jumps, 3, 2, 1, 0, 4 is trapped by the zero, one step at a time over four cells costs 3, and a single big first hop costs 1. Python returns None when stuck, the other six return minus one. Lua's consistency check is worth copying: whenever the minimum is finite, the reachability scan must say yes, one fixture set answering both questions.

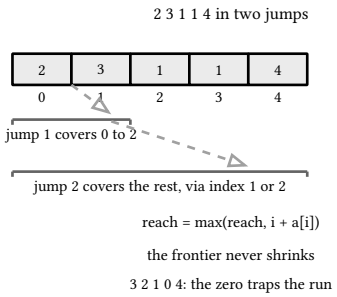


figure 153: the jump game as level windows, each jump covers a window, the next window ends at the farthest reach inside it

18.4. backtracking

N-queens is the canonical backtrack: place row by row, prune on column and both diagonals, recurse, undo. The meter counts placements and prunes.

The dry run: the fixtures are identical literals in all seven suites. Queens pins the triples (solutions, placements, prunes) at n = 1 through 5 and 8: (1, 1, 0), (0, 6, 4), (0, 18, 13), (2, 60, 44), (10, 220, 167), and (92, 15720, 13664), a placement counted per column tried before the conflict test and a prune per rejected column. Subset sum takes 3, 34, 4, 12, 5, 2 to 9 true and 30 false, the single 7 against 7 true, the empty set against 1 false, and 8, 2 against 0 true. Permutations of 4 emit 24 distinct orders, first 0, 1, 2, 3, second 0, 1, 3, 2, third 0, 2, 1, 3, last 3, 2, 1, 0.

1. At n = 1 the lone cell is a solution, count 1, while n = 2 and n = 3 die on the flags, count 0 each.

2. At n = 4 the first live walk tries column 0 at row 0, dies inside row 2, backs out, and column 1 lands the solution 1, 3, 0, 2, every column and diagonal flag fresh at each placement.
3. Undoing to row 0 and trying column 2 lands the mirror 2, 0, 3, 1, and the count closes at 2.
4. n = 5 pins 10 and n = 8 pins 92, with the pruned meter under 100000 placements against the raw tree's 8^8 = 16777216.
5. Subset sum takes 3, 9 - 3 = 6, skips 34, takes 4, 6 - 4 = 2, skips 12 and 5, takes 2, 2 - 2 = 0, true, while the same set against 30 exhausts the tree false.
6. Permutations of 4 enumerate 4 x 3 x 2 x 1 = 24 orders, first 0, 1, 2, 3, last 3, 2, 1, 0.

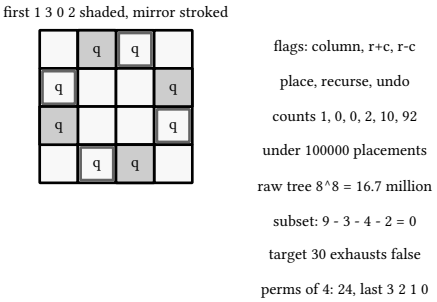


figure 154: the n = 4 board, the first solution shaded column by column, its mirror stroked, the counts and the meter beside it

The count rests at 92 for under a hundred thousand placements, and the listings below carry the whole machine, queens to permutations, seven ways.

```
/// <summary>Backtracking: exhaustive search with pruning, the prune counted.</summary>
public static class Backtrack
{
    public sealed class Meter
    {
        public long Placements { get; set; }

        public long Prunes { get; set; }
    }

    /// <summary>N-queens: all solutions, or just count them.</summary>
    public static long CountQueens(int n, Meter? meter = null)
    {
        var cols = new bool[n];
        var diag1 = new bool[2 * n]; // r + c
        var diag2 = new bool[2 * n]; // r - c + n - 1
        long solutions = 0;

        void Place(int row)
        {
            if (row == n)
            {
                solutions++;
                return;
            }
            for (int c = 0; c < n; c++)
            {
                if (meter != null)
                    meter.Placements++;
                if (cols[c] || diag1[row + c] || diag2[row - c + n - 1])
                {
                    if (meter != null)
```

```
                    meter.Prunes++;
                continue;
            }
            cols[c] = diag1[row + c] = diag2[row - c + n - 1] = true;
            Place(row + 1);
            cols[c] = diag1[row + c] = diag2[row - c + n - 1] = false;
        }
    }

    Place(0);
    return solutions;
}

/// <summary>Subset sum existence: include or exclude, prune on overrun.</summary>
public static bool SubsetSum(int[] values, int target, Meter? meter = null)
{
    bool Search(int i, int remaining)
    {
        if (remaining == 0)
            return true;
        if (i == values.Length || remaining < 0)
            return false;
        if (meter != null)
            meter.Placements++;
        return Search(i + 1, remaining - values[i]) || // take
               Search(i + 1, remaining);               // skip
    }
    return Search(0, target);
}

/// <summary>Permutations lexicographically, next-permutation style.</summary>
public static IEnumerable<int[]> Permutations(int n)
{
    var a = Enumerable.Range(0, n).ToArray();
    while (true)
    {
        yield return (int[])a.Clone();

        int i = n - 2;
        while (i ≥ 0 && a[i] ≥ a[i + 1])
            i--;
        if (i < 0)
            yield break;
        int j = n - 1;
        while (a[j] ≤ a[i])
            j--;
        (a[i], a[j]) = (a[j], a[i]);
        Array.Reverse(a, i + 1, n - i - 1);
    }
}
}
```

listing 570: c#, n-queens with diagonal pruning, subset sum, next permutation (source: dsa/samples/src/Ch18/Greedy.cs, lines 170-254)

The known solution counts are pinned, one, zero, zero, two, ten, through ninety-two at n equal 8, and the meter shows what pruning buys: under a hundred thousand placements against sixteen point seven million for the unpruned tree. The O(1) attack and undo on three flat boolean arrays is the standard trick, no board is ever materialized. Subset sum is the same shape with include and exclude branches, and the permutation enumerator

is backtracking in its amortized next-permutation form. Chapter 20 returns to n-queens with the pruning map around it.

```c
static void place(queens *q, int row) {
  if (row == q->size) {
    q->solutions++;
    return;
  }
  for (int c = 0; c < q->size; c++) {
    q->m->placements++; // one per column tried, before the test
    if (q->cols[c] || q->diag1[row + c] || q->diag2[row - c + q->size - 1]) {
      q->m->prunes++;
      continue;
    }
    q->cols[c] = q->diag1[row + c] = q->diag2[row - c + q->size - 1] = 1;
    place(q, row + 1);
    q->cols[c] = q->diag1[row + c] = q->diag2[row - c + q->size - 1] = 0;
  }
}

// all n-queens solutions with the placement and prune counts, the meter
// zeroed per call so each n reports its own tree
static long count_queens(int size, meter *m) {
  queens q = {0};
  q.size = size;
  m->placements = 0;
  m->prunes = 0;
  q.m = m;
  place(&q, 0);
  return q.solutions;
}
```

listing 571: c, the flag arrays, the metered place and undo (source: dsa/samples-c/src/Ch18/backtrack.c, lines 31-58)

```
// CountQueens counts all n-queens solutions. The used columns and both
// diagonal families make the conflict test O(1); the meter is bumped
// the moment a column is tried, before the test runs.
func CountQueens(n int, meter *BacktrackMeter) int64 {
  cols := make([]bool, n)
  diag1 := make([]bool, 2*n) // r + c
  diag2 := make([]bool, 2*n) // r - c + n - 1
  solutions := int64(0)
  var place func(row int)
  place = func(row int) {
    if row == n {
      solutions++
      return
    }
    for c := 0; c < n; c++ {
      if meter != nil {
        meter.Placements++
      }
      if cols[c] || diag1[row+c] || diag2[row-c+n-1] {
        if meter != nil {
          meter.Prunes++
        }
        continue
      }
      cols[c] = true
      diag1[row+c] = true
      diag2[row-c+n-1] = true
      place(row + 1)
      cols[c] = false
      diag1[row+c] = false
      diag2[row-c+n-1] = false
    }
  }
  place(0)
  return solutions
}
```

listing 572: go, closure recursion, the meter behind a pointer (source: dsa/samples-go/ch18/backtrack.go, lines 11-46)

```
  static void place(int row) {
    if (row == size) {
      solutions++;
      return;
    }
    for (int c = 0; c < size; c++) {
      placements++; // one per column tried, before the test
      if (cols[c] || diag1[row + c] || diag2[row - c + size - 1]) {
        prunes++;
        continue;
      }
      cols[c] = diag1[row + c] = diag2[row - c + size - 1] = true;
      place(row + 1);
      cols[c] = diag1[row + c] = diag2[row - c + size - 1] = false;
    }
  }

  static long countQueens(int n) {
    size = n;
    cols = new boolean[n];
    diag1 = new boolean[2 * n];
    diag2 = new boolean[2 * n];
    placements = 0;
    prunes = 0;
    solutions = 0;
    place(0);
    return solutions;
  }

  // subset sum existence: include or exclude, prune on overrun
  static boolean subsetSum(int[] values, int i, int remaining) {
    if (remaining == 0)
      return true;
    if (i == values.length || remaining < 0)
      return false;
    return subsetSum(values, i + 1, remaining - values[i])
        || subsetSum(values, i + 1, remaining);
  }
```

listing 573: java, static flag arrays and meters, the try counted before the test, include-or-exclude subset sum (source: dsa/samples-java/src/Ch18/Backtrack.java, lines 28-65)

```
// ch18: backtracking with the prune counted. the meters count a placement
// per column tried (incremented before the conflict test) and a prune per
// rejected column. permutations emit in lexicographic order by
// next-permutation

export function countQueens(n, meter = null) {
  const cols = new Array(n).fill(false);
  const diag1 = new Array(2 * n).fill(false); // r + c
  const diag2 = new Array(2 * n).fill(false); // r - c + n - 1
  let solutions = 0;

  const place = (row) => {
    if (row === n) {
      solutions++;
      return;
    }
    for (let c = 0; c < n; c++) {
      if (meter !== null) meter.placements++;
      if (cols[c] || diag1[row + c] || diag2[row - c + n - 1]) {
        if (meter !== null) meter.prunes++;
        continue;
      }
      cols[c] = diag1[row + c] = diag2[row - c + n - 1] = true;
      place(row + 1);
      cols[c] = diag1[row + c] = diag2[row - c + n - 1] = false;
    }
  };

  place(0);
  return solutions;
}
```

listing 574: javascript, an optional meter object, destructured flag writes (source: dsa/samples-js/src/ch18-backtrack.mjs, lines 1-31)

```
def count_queens(size):
    cols = [False] * size
    diag1 = [False] * (2 * size)  # row + column
    diag2 = [False] * (2 * size)  # row - column + size - 1
    stats = [0, 0, 0]  # solutions, placements, prunes

    def place(row):
        if row == size:
            stats[0] += 1
            return
        for col in range(size):
            stats[1] += 1  # counted before the conflict test, per column tried
            if cols[col] or diag1[row + col] or diag2[row - col + size - 1]:
                stats[2] += 1
                continue
            cols[col] = diag1[row + col] = diag2[row - col + size - 1] = True
            place(row + 1)
            cols[col] = diag1[row + col] = diag2[row - col + size - 1] = False

    place(0)
    return tuple(stats)
```

listing 575: python, the stats triple closed over by the recursion (source: dsa/samples-py/src/Ch18/backtrack.py, lines 14-34)

```lua
-- returns solutions plus the meter: placements and prunes
local function count_queens(n, meter)
  local cols, diag1, diag2 = {}, {}, {}
  local solutions = 0
  local function place(row)
    if row == n then
      solutions = solutions + 1
      return
    end
    for c = 0, n - 1 do
      meter.placements = meter.placements + 1 -- per column tried
      local d1, d2 = row + c + 1, row - c + n -- 1-based diag slots
      if cols[c] or diag1[d1] or diag2[d2] then
        meter.prunes = meter.prunes + 1 -- per rejected column
      else
        cols[c], diag1[d1], diag2[d2] = true, true, true
        place(row + 1)
        cols[c], diag1[d1], diag2[d2] = nil, nil, nil
      end
    end
  end
  place(0)
  return solutions
end
```

listing 576: lua, 1-based diagonal slots, nil clears the flags (source: dsa/samples-lua/ch18_backtrack.lua, lines 7-30)

Measured across the suites: all seven pin the same meter triples, from (1, 1, 0) at n = 1 to (92, 15720, 13664) at n = 8, the bump landing the moment a column is tried so the counts stay comparable. Subset sum answers true on 9 and false on 30 with the same take-then-skip branch order, and the permutation enumerators emit the same 24 lexicographic orders with first, second, third, and last pinned. The meter rides a pointer struct in Go, a stats triple in Python, an optional object in JavaScript, a plain struct in C, a keyed table in Lua, static fields on the class in Java, and a class in C#, seven shapes for one contract.

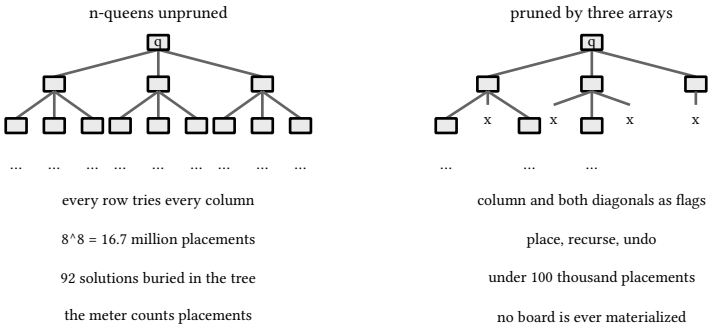


figure 155: the n-queens tree twice, every row against every column unpruned, three flat boolean arrays cut it to under a hundred thousand

WARNING: backtracking is exponential, honestly

Pruning changes constants, not the complexity class. N-queens has no polynomial formula, subset sum is NP-complete, and the meters exist to show the tree is smaller, not that it is small. The engineering line: backtracking for small n or heavy structure, the chapter 17 table when the recurrence has overlapping subproblems, and an approximation or a solver when neither holds. The approximation chapter, book 9, ch 42 (practical data structures and algorithms, approximation algorithms), builds that third road with its ratios measured.

The chapter ends with the selection rule the whole spine argues for: measure the shape of the input, name the invariant you can maintain in the data structure, count the operations honestly, and pick the simplest structure whose counted cost fits the budget. Four more chapters push the table, the range query, the plane, and the probabilistic structure further, then the capstone puts all of it into one running system.

18.5. across the seven languages

The build sizes count non-comment source lines. The first table covers the chapter's original featured files, the C# row carrying its whole `Greedy.cs` plus the jump game build, the backtracking section counted again in the second table:

language	build SLOC	dependency	boundary note
c	174	libc only	qsort with a comparator function, huffman weights in unsigned long long, jumps return minus one
c#	248	bcl only	rooms ride the chapter 8 heap, fractional knapsack at 240 against the table's 220, ReadOnlySpan jump scans
go	104	slices	slices.SortFunc orders intervals, two min scans select the huffman pair, single-symbol codes forced to length 1
java	272	jdk 27 stdlib	a record per interval with a comparator lambda, the subset brute by mask where python uses itertools, huffman finds both lightest nodes in one scan
javascript	110	node stdlib	a hand-built heap keyed by weight then insertion order keeps the fixture tree deterministic
python	115	stdlib only	sort stands in for the priority queue, itertools.combinations brute-checks the schedule, stuck jumps read None
lua	217	lib.lua harness	insertion-order tie-break in every sort, encode and decode round trip abcdef, maximality checked by hand

The backtracking section lands as its own file in the six sibling trees, the C# build staying the Backtrack class inside `Greedy.cs`:

language	build SLOC	dependency	boundary note
c	128	libc only	a meter struct by pointer, three flag arrays, lexicographic order proved by a comparator walk
c#	72	bcl only	the Backtrack class inside Greedy.cs, iterator permutations, a nullable Meter class
go	82	slices	closure recursion, the meter behind a pointer, Permutations materialized as copies
java	137	jdk 27 stdlib	static meter fields and flag arrays, the diagonal shift r - c + n - 1, lexicographic order proved pairwise
javascript	51	node stdlib	generator permutations, an optional meter object, destructured flag writes
python	82	stdlib only	a stats triple closed over by the recursion, slice reversal flips the tail
lua	111	lib.lua harness	1-based diagonal slots, nil clears the flags, table.concat keys prove distinctness

sources: learn.microsoft.com, `BinaryHeap` usage here rides the chapter 8 implementation, `OrderBy` remarks, `StringBuilder`, accessed 2026-09-08, plus the CLRS activity selection and huffman treatments cited in the chapter text. Sample behavior verified by `make verify-csharp`, 13 tests in chapter 18 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 61 embedded checks under `make verify-c`, 4 Ch18 java programs

with 81 checks under `run-java-samples`, 14 Go tests, 13 `node --test` cases, 49 Python checks across 4 files, and 18 Lua checks under `run.lua`.

19. advanced dynamic programming

Chapter 17 paid for its tables in full: every copy of every item got its own scan, every split point of the partition was reweighed from scratch, and longest increasing subsequence was left out entirely. This chapter keeps those recurrences and cuts the waste, counts become binary pieces, then a monotone deque per residue class, the partition becomes lines on a hull, and the longest run becomes tails plus binary search. The meter turns each cut into a pinned number: 15 candidate consults drop to 9 on one small instance, 19900 split points drop to 400 hull touches, and 499500 pairs drop to a ceiling of ten thousand comparisons.

19.1. the bounded knapsack

The vending machine holds three of each coin, the truck carries two of each crate: real knapsacks come with counts. The direct translation of chapter 17′s table weighs every copy count at every cell.

The dry run: the fixture is one item of weight 4, value 5, count 2 against capacity 8, asserted the same way in C, Go, Java, C#, JavaScript, Python, and Lua, with the count 3 variant against 12, fixture A at 31 and fixture B at 25, and the brute walk over every multiplicity tuple riding along as the oracle in each suite.

1. The copy scan starts at k = 0 and consults all 9 cells, every one reading the zero row: capacities 0 to 3 stay 0.
2. k = 1 starts at cell 4, the first its weight fits, and consults 5 cells: 0 + 5 = 5 lands at capacities 4 through 8.
3. k = 2 starts at cell 8 and consults 1: two whole items fill the capacity, 5 + 5 = 10.
4. The meter reads 9 + 5 + 1 = 15 consults and the row closes at 0, 0, 0, 0, 5, 5, 5, 5, 10, the answer 10 at capacity 8.
5. Count 3 against capacity 12 reads the same way, three whole items pay 15, the second pinned answer.
6. The pieces road covers the counts with 0/1 items: 13 splits into pieces 1, 2, 4, 6 with 1 + 2 + 4 + 6 = 13, and the suite checks every count from 1 to 40 reassembles.

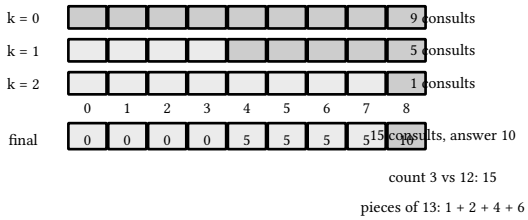


figure 156: the copy scan as one row per count, every cell each k consults shaded, the finished row and its 10 below

The 15 consults close at the pinned 10, and the listings below carry the naive pass, the pieces, and the reused 0/1 walk in seven languages.

```c
// one consult per (copy count, cell) pair, per_k records the split
static long bounded_naive(const int *w, const long *v, const int *c, int items,
                          int capacity, long *consults, long *per_k) {
  long dp[MAX_CAP + 1], next[MAX_CAP + 1];
  *consults = 0;
  for (int cap = 0; cap ≤ capacity; cap++)
    dp[cap] = 0;
  for (int i = 0; i < items; i++) {
    for (int cap = 0; cap ≤ capacity; cap++)
      next[cap] = 0;
    for (int k = 0; k ≤ c[i]; k++) {
      if (per_k)
        per_k[k] = 0;
      for (int cap = k * w[i]; cap ≤ capacity; cap++) {
        (*consults)++;
        if (per_k)
          per_k[k]++;
        long candidate = dp[cap - k * w[i]] + k * v[i];
        if (candidate > next[cap])
          next[cap] = candidate;
      }
    }
    memcpy(dp, next, sizeof dp);
  }
  return dp[capacity];
}

// powers 1, 2, 4, ... then the remainder: subset sums cover every count
static int binary_pieces(int count, int *pieces) {
  int total = 0, power = 1;
  while (power < count) {
    pieces[total++] = power;
    count -= power;
    power *= 2;
  }
  if (count > 0)
    pieces[total++] = count;
  return total;
}
```

listing 577: c, the copy scan with its per-k meter, then the binary pieces (source: dsa/samples-c/src/Ch19/bounded.c, lines 22-60)

```
// BoundedKnapsackNaive solves the bounded knapsack the direct way:
// every copy count k tried at every capacity cell, one consult per
// pair.
func BoundedKnapsackNaive(weights []int, values []int64, counts []int, capacity int, meter *BoundedMeter)
int64 {
  dp := make([]int64, capacity+1)
  next := make([]int64, capacity+1)
  for i := range weights {
    for c := range next {
      next[c] = 0
    }
    for k := 0; k <= counts[i]; k++ {
      for c := k * weights[i]; c <= capacity; c++ {
        if meter != nil {
          meter.Ops++
        }
        candidate := dp[c-k*weights[i]] + int64(k)*values[i]
        if candidate > next[c] {
          next[c] = candidate
        }
      }
    }
    dp, next = next, dp
  }
  return dp[capacity]
}

// BinaryPieces splits a copy count into powers 1, 2, 4, ... then the
// remainder: the piece subset sums cover every count from 0 to count.
func BinaryPieces(count int) []int {
  var pieces []int
  power := 1
  for power < count {
    pieces = append(pieces, power)
    count -= power
    power *= 2
  }
  if count > 0 {
    pieces = append(pieces, count)
  }
  return pieces
}
```

listing 578: go, the naive scan behind a meter struct, the pieces beside it (source: dsa/samples-go/ch19/bounded.go, lines 8-48)

```java
  // one consult per (copy count, cell) pair, per_k records the split
  static long boundedNaive(int[] w, long[] v, int[] c, int capacity, boolean meter) {
    long[] dp = new long[MAX_CAP + 1];
    long[] next = new long[MAX_CAP + 1];
    consults = 0;
    for (int i = 0; i < w.length; i++) {
      java.util.Arrays.fill(next, 0);
      if (meter)
        java.util.Arrays.fill(perK, 0);
      for (int k = 0; k ≤ c[i]; k++) {
        if (meter)
          perK[k] = 0;
        for (int cap = k * w[i]; cap ≤ capacity; cap++) {
          consults++;
          if (meter)
            perK[k]++;
          long candidate = dp[cap - k * w[i]] + k * v[i];
          if (candidate > next[cap])
            next[cap] = candidate;
        }
      }
      long[] t = dp;
      dp = next;
      next = t;
    }
    return dp[capacity];
  }

  // powers 1, 2, 4, ... then the remainder: subset sums cover every count
  static int binaryPieces(int count, int[] pieces) {
    int total = 0, power = 1;
    while (power < count) {
      pieces[total++] = power;
      count -= power;
      power *= 2;
    }
    if (count > 0)
      pieces[total++] = count;
    return total;
  }
```

listing 579: java, the metered copy scan with its per-k split, then the binary pieces (source: dsa/samples-java/src/Ch19/Bounded.java, lines 22-61)

```csharp
    /// <summary>Bounded knapsack the direct way: every copy count k tried at every capacity cell.</
summary>
    public static long BoundedKnapsackNaive(int[] weights, long[] values, int[] counts, int capacity,
Meter? meter = null)
    {
        var dp = new long[capacity + 1];
        var next = new long[capacity + 1];
        for (int i = 0; i < weights.Length; i++)
        {
            Array.Fill(next, 0L);
            for (int k = 0; k ≤ counts[i]; k++)
                for (int c = k * weights[i]; c ≤ capacity; c++)
                {
```

```
                    if (meter != null)
                        meter.Ops++; // one consult per (copy count, cell) pair
                    long candidate = dp[c - k * weights[i]] + k * values[i];
                    if (candidate > next[c])
                        next[c] = candidate;
                }
            (dp, next) = (next, dp);
        }
        return dp[capacity];
    }

    /// <summary>Powers 1, 2, 4, ... then the remainder; subset sums cover every count from 0 to count.</
summary>
    public static IReadOnlyList<int> BinaryPieces(int count)
    {
        var pieces = new List<int>();
        int power = 1;
        while (power < count)
        {
            pieces.Add(power);
            count -= power;
            power *= 2;
        }
        if (count > 0)
            pieces.Add(count);
        return pieces;
    }

    /// <summary>Bounded knapsack by decomposition: pieces become ordinary 0/1 items.</summary>
    public static long BoundedKnapsackBinary(int[] weights, long[] values, int[] counts, int capacity)
    {
        var ws = new List<int>();
        var vs = new List<long>();
        for (int i = 0; i < weights.Length; i++)
            foreach (int piece in BinaryPieces(counts[i]))
            {
                ws.Add(weights[i] * piece);
                vs.Add(values[i] * piece);
            }
        return Knapsack01(ws.ToArray(), vs.ToArray(), capacity);
    }

    // the chapter 15 recurrence, one backward capacity pass per 0/1 item
    private static long Knapsack01(int[] weights, long[] values, int capacity)
    {
        var dp = new long[capacity + 1];
        for (int i = 0; i < weights.Length; i++)
            for (int c = capacity; c ≥ weights[i]; c--)
                dp[c] = Math.Max(dp[c], dp[c - weights[i]] + values[i]);
        return dp[capacity];
    }
```

listing 580: c#, bounded knapsack by copy count, binary pieces, the reused 0/1 pass (source: dsa/samples/src/Ch19/AdvDp.cs, lines 11-71)

```
export function boundedKnapsackNaive(weights, values, counts, capacity, meter = null) {
  let dp = new Array(capacity + 1).fill(0);
  let next = new Array(capacity + 1).fill(0);
  for (let i = 0; i < weights.length; i++) {
    next.fill(0);
    for (let k = 0; k ≤ counts[i]; k++) {
      for (let c = k * weights[i]; c ≤ capacity; c++) {
        if (meter =/= null) meter.ops++; // one consult per (copy count, cell)
        const candidate = dp[c - k * weights[i]] + k * values[i];
        if (candidate > next[c]) next[c] = candidate;
      }
    }
    [dp, next] = [next, dp];
  }
  return dp[capacity];
}

// powers 1, 2, 4, ... then the remainder: subset sums cover every count
// from 0 to count
export function binaryPieces(count) {
  const pieces = [];
  let power = 1;
  while (power < count) {
    pieces.push(power);
    count -= power;
    power *= 2;
  }
  if (count > 0) pieces.push(count);
  return pieces;
}

export function boundedKnapsackBinary(weights, values, counts, capacity) {
  const ws = [];
  const vs = [];
  for (let i = 0; i < weights.length; i++) {
    for (const piece of binaryPieces(counts[i])) {
      ws.push(weights[i] * piece);
      vs.push(values[i] * piece);
    }
  }
  return knapsack01(ws, vs, capacity);
}
```

listing 581: javascript, the metered naive, the pieces, the 0/1 handoff (source: dsa/samples-js/src/ch19-bounded.mjs, lines 5-46)

```
def naive(weights, values, counts, capacity, meter=None):
    dp = [0] * (capacity + 1)
    for i in range(len(weights)):
        nxt = [0] * (capacity + 1)
        for k in range(counts[i] + 1):
            for c in range(k * weights[i], capacity + 1):
                if meter is not None:
                    meter.bump(k)  # one consult per (k, cell) pair
                candidate = dp[c - k * weights[i]] + k * values[i]
                if candidate > nxt[c]:
                    nxt[c] = candidate
        dp = nxt
    return dp[capacity]


def binary_pieces(count):
    pieces = []
    power = 1
    while power < count:
        pieces.append(power)
        count -= power
        power *= 2
    if count > 0:
        pieces.append(count)
    return pieces


def binary_bounded(weights, values, counts, capacity):
    ws, vs = [], []
    for i in range(len(weights)):
        for piece in binary_pieces(counts[i]):
            ws.append(weights[i] * piece)  # pieces scale weight and value
            vs.append(values[i] * piece)
    dp = [0] * (capacity + 1)
    for i in range(len(ws)):  # one backward capacity pass per 0/1 item
        for c in range(capacity, ws[i] - 1, -1):
            if dp[c - ws[i]] + vs[i] > dp[c]:
                dp[c] = dp[c - ws[i]] + vs[i]
    return dp[capacity]
```

listing 582: python, the naive with a per-k meter dict, the pieces, the scaled pass (source: dsa/samples-py/src/Ch19/bounded.py, lines 27-65)

```lua
local function naive(weights, values, counts, capacity, per_k)
  local dp = {}
  for c = 0, capacity do dp[c] = 0 end
  for i = 1, #weights do
    local next_dp = {}
    for c = 0, capacity do next_dp[c] = 0 end
    for k = 0, counts[i] do
      local seen = 0
      for c = k * weights[i], capacity do
        seen = seen + 1
        local candidate = dp[c - k * weights[i]] + k * values[i]
        if candidate > next_dp[c] then next_dp[c] = candidate end
      end
      if per_k then per_k[k] = seen end
    end
    dp = next_dp
  end
  return dp[capacity]
end

-- powers 1, 2, 4, ... then the remainder
local function binary_pieces(count)
  local pieces = {}
  local power = 1
  while power < count do
    pieces[#pieces + 1] = power
    count = count - power
    power = power * 2
  end
  if count > 0 then pieces[#pieces + 1] = count end
  return pieces
end

-- pieces become ordinary 0/1 items, capacity walks downward
local function binary(weights, values, counts, capacity)
  local ws, vs = {}, {}
  for i = 1, #weights do
    for _, piece in ipairs(binary_pieces(counts[i])) do
      ws[#ws + 1] = weights[i] * piece -- weight and value both multiply
      vs[#vs + 1] = values[i] * piece
    end
  end
  local dp = {}
  for c = 0, capacity do dp[c] = 0 end
  for i = 1, #ws do
    for c = capacity, ws[i], -1 do
      if dp[c - ws[i]] + vs[i] > dp[c] then dp[c] = dp[c - ws[i]] + vs[i] end
    end
  end
  return dp[capacity]
end
```

listing 583: lua, the naive noting consults per k, the pieces, the 0/1 walk (source: dsa/samples-lua/ch19_bounded.lua, lines 7-57)

The naive is honest but repetitious. On one item of weight 4, value 5, count 2 against capacity 8 it consults 15 cells, 9 for zero copies, 5 for one, 1 for two, and answers 10, two whole items filling weight 8. Count 3 against capacity 12 answers 15 the same way. The C# suite adds 25 seeded instances of up to 4 items against a brute force that enumerates every multiplicity vector. Binary decomposition buys the same answers cheaper: count 13 splits into

pieces 1, 2, 4, 6 whose subset sums cover every count from 0 to 13, each item collapses to a logarithmic number of 0/1 pieces, and the chapter 17 backward pass does the rest. The pieces always reassemble, the suite checks every count from 1 to 40.

The six new lanes answer from the same literals: 10 and 15 on the single item, 31 on fixture A and 25 on fixture B, the pieces of 13 reading 1, 2, 4, 6 and of 7 reading 1, 2, 4 with the 1 to 40 reassembly lane beside them. The meter granularity splits by tree: C, Java, Python, and Lua record the per-k counts 9, 5, 1 next to the 15 total, Go and JavaScript count the 15 alone, and every tree keeps the multiplicity brute force as its oracle. The scaled pieces multiply weight and value together in all six, and a piece scaled on weight alone would fail the oracle lanes.

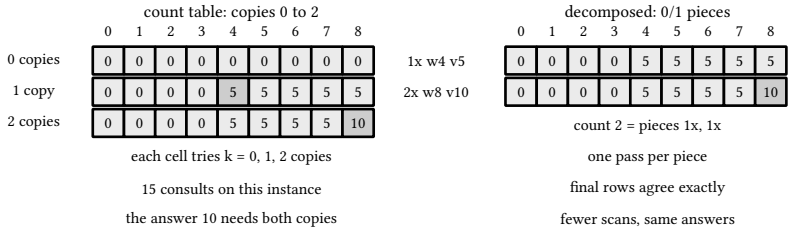


figure 157: the bounded knapsack as memory, copies tried per cell on the left, binary pieces as 0/1 rows on the right, the shading marks where values rise

19.2. the sliding window shortcut

Decomposition removed the counts from the item dimension, but the cell dimension still pays for them, and the naive reweighs candidates that cannot have changed. The structure to notice is vertical: a cell of capacity c only ever reads cells c - k*w for k up to the count, and those are the earlier cells of one residue class of capacity mod w. Number the class by j and the recurrence collapses to a window maximum, subtract j*v from the previous row, keep the max of the last count+1 entries, add j*v back.

The dry run: the fixture is the same single item, weight 4, value 5, count 2 against capacity 8, asserted by all seven suites with both meters attached, 9 window consults against the count scan's 15 on the same answer 10.

1. Capacity mod 4 splits the nine cells into four classes: 0, 4, 8 then 1, 5 then 2, 6 then 3, 7.
2. Class 0 numbers its cells j = 0, 1, 2 and reads the previous row through g = prev - j x 5, landing 0, −5, −10.
3. The deque keeps j = 0 as its front against both arrivals, so the cells pay 0 + 0 = 0, 0 + 5 = 5, 0 + 10 = 10.
4. The three two-cell classes never see a rival candidate, and the row closes 0, 0, 0, 0, 5, 5, 5, 5, 10, the same 10.
5. The window meter reads 3 + 2 + 2 + 2 = 9, one consult per capacity cell, against the count scan's 15.

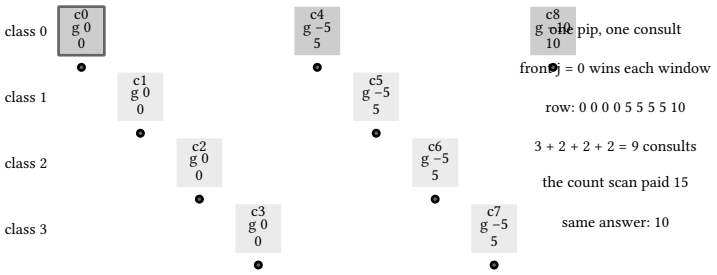


figure 158: the capacity axis split into its four residue classes, one consult pip per cell, the class 0 lane carrying its g values and its winning front

The 9 against 15 on the same answer 10 is the pinned pair, and the listings below run the class walk in seven languages.

```c
// the window value of candidate j in residue class r
static long g_val(const long *prev, int r, int w, long value, int j) {
  return prev[r + j * w] - j * value;
}

// one monotone deque per residue class: dominated candidates pop from
// the back on ≤, the head is evicted when older than j - m
static long window_bounded(const int *w, const long *v, const int *c, int items,
                           int capacity, long *consults) {
  long dp[MAX_CAP + 1], prev[MAX_CAP + 1];
  *consults = 0;
  for (int cap = 0; cap ≤ capacity; cap++)
    dp[cap] = 0;
  for (int i = 0; i < items; i++) {
    memcpy(prev, dp, sizeof dp);
    int weight = w[i], copies = c[i];
    for (int r = 0; r < weight && r ≤ capacity; r++) {
      int cells = (capacity - r) / weight + 1;
      int deque[MAX_CAP + 1], head = 0, tail = 0;
      for (int j = 0; j < cells; j++) {
        while (tail > head && g_val(prev, r, weight, v[i], deque[tail - 1]) ≤
                              g_val(prev, r, weight, v[i], j))
          tail--;
        deque[tail++] = j;
        while (deque[head] < j - copies)
          head++;
        (*consults)++; // exactly one candidate consulted per cell
        dp[r + j * weight] =
            g_val(prev, r, weight, v[i], deque[head]) + j * v[i];
      }
    }
  }
  return dp[capacity];
}
```

listing 584: c, the g value helper, the deque per residue class, one consult per cell (source: dsa/samples-c/src/Ch19/window.c, lines 78-111)

```go
// BoundedKnapsackWindow solves the bounded knapsack in items-times-
// capacity: one monotone deque per residue class of capacity mod
// weight, exactly one candidate consult per capacity cell.
func BoundedKnapsackWindow(weights []int, values []int64, counts []int, capacity int, meter *BoundedMeter)
int64 {
  dp := make([]int64, capacity+1)
  prev := make([]int64, capacity+1)
  for i := range weights {
    w, m := weights[i], counts[i]
    copy(prev, dp)
    for r := 0; r < w && r <= capacity; r++ {
      cells := (capacity-r)/w + 1
      deque := make([]int, cells)
      head, tail := 0, 0
      // the j-th cell of the residue class carries bonus
      // -j*v, so the deque holds plain prev values
      g := func(j int) int64 { return prev[r+j*w] - int64(j)*values[i] }
      for j := 0; j < cells; j++ {
        for tail > head && g(deque[tail-1]) <= g(j) {
          tail-- // a dominated candidate leaves from the back
        }
        deque[tail] = j
        tail++
        for deque[head] < j-m {
          head++ // more than m copies would be needed
        }
        if meter != nil {
          meter.Ops++ // one consult per cell
        }
        dp[r+j*w] = g(deque[head]) + int64(j)*values[i]
      }
    }
  }
  return dp[capacity]
}
```

listing 585: go, one class per residue, the deque holding plain prev indices (source: dsa/samples-go/ch19/window.go, lines 3-36)

```
  // the window value of candidate j in residue class r
  static long gVal(long[] prev, int r, int w, long value, int j) {
    return prev[r + j * w] - j * value;
  }

  // one monotone deque per residue class: dominated candidates pop from
  // the back on ≤, the head is evicted when older than j - m
  static long windowBounded(int[] w, long[] v, int[] c, int capacity) {
    long[] dp = new long[MAX_CAP + 1];
    long[] prev = new long[MAX_CAP + 1];
    consults = 0;
    for (int i = 0; i < w.length; i++) {
      System.arraycopy(dp, 0, prev, 0, capacity + 1);
      int weight = w[i], copies = c[i];
      for (int r = 0; r < weight && r ≤ capacity; r++) {
        int cells = (capacity - r) / weight + 1;
        int[] deque = new int[MAX_CAP + 1];
        int head = 0, tail = 0;
        for (int j = 0; j < cells; j++) {
          while (tail > head
              && gVal(prev, r, weight, v[i], deque[tail - 1]) ≤ gVal(prev, r, weight, v[i], j))
            tail--;
          deque[tail++] = j;
          while (deque[head] < j - copies)
            head++;
          consults++; // exactly one candidate consulted per cell
          dp[r + j * weight] = gVal(prev, r, weight, v[i], deque[head]) + j * v[i];
        }
      }
    }
    return dp[capacity];
  }
```

listing 586: java, the g helper over a copied prev row, one deque per residue class, a consult per cell (source: dsa/samples-java/src/Ch19/Window.java, lines 73-104)

```
    /// <summary>Bounded knapsack in items-times-capacity: a monotone deque per residue class of capacity
mod weight.</summary>
    public static long BoundedKnapsackWindow(int[] weights, long[] values, int[] counts, int capacity,
Meter? meter = null)
    {
        var dp = new long[capacity + 1];
        var prev = new long[capacity + 1];
        for (int i = 0; i < weights.Length; i++)
        {
            int w = weights[i];
            int m = counts[i];
            Array.Copy(dp, prev, dp.Length);
            for (int r = 0; r < w && r ≤ capacity; r++)
            {
                int cells = (capacity - r) / w + 1;
                var deque = new int[cells];
                int head = 0, tail = 0;
                long G(int j) ⇒ prev[r + j * w] - j * values[i];
                for (int j = 0; j < cells; j++)
                {
                    while (tail > head && G(deque[tail - 1]) ≤ G(j))
                        tail--; // a dominated candidate leaves from the back
                    deque[tail++] = j;
                    while (deque[head] < j - m)
                        head++; // more than m copies would be needed
                    if (meter ≠ null)
                        meter.Ops++; // exactly one candidate consulted per cell
                    dp[r + j * w] = G(deque[head]) + j * values[i];
                }
            }
        }
        return dp[capacity];
    }
```

listing 587: c#, bounded knapsack by residue class, a monotone deque serves each window (source: dsa/samples/src/Ch19/AdvDp.cs, lines 73-103)

```
// ch19: the bounded knapsack in items-times-capacity. a monotone deque
// per residue class of capacity mod weight keeps the best candidate for
// every cell, so each cell is consulted exactly once

export function boundedKnapsackWindow(weights, values, counts, capacity, meter = null) {
  let dp = new Array(capacity + 1).fill(0);
  let prev = new Array(capacity + 1).fill(0);
  for (let i = 0; i < weights.length; i++) {
    const w = weights[i];
    const m = counts[i];
    [prev, dp] = [dp, prev];
    // prev now holds the table after items 0..i-1, dp is rewritten below
    for (let r = 0; r < w && r ≤ capacity; r++) {
      const cells = Math.floor((capacity - r) / w) + 1;
      const deque = [];
      const G = (j) ⇒ prev[r + j * w] - j * values[i];
      for (let j = 0; j < cells; j++) {
        // a dominated candidate leaves from the back
        while (deque.length > 0 && G(deque[deque.length - 1]) ≤ G(j)) {
          deque.pop();
        }
        deque.push(j);
        // more than m copies would be needed
        while (deque[0] < j - m) deque.shift();
        if (meter =/= null) meter.ops++; // exactly one consult per cell
        dp[r + j * w] = G(deque[0]) + j * values[i];
      }
    }
  }
  return dp[capacity];
}
```

listing 588: javascript, rows kept by swapping, dominated candidates pop from the back (source: dsa/samples-js/src/ch19-window.mjs, lines 1-31)

```python
def window_bounded(weights, values, counts, capacity, meter=None):
    dp = [0] * (capacity + 1)
    for i in range(len(weights)):
        w, copies, v = weights[i], counts[i], values[i]
        prev = list(dp)
        for r in range(min(w, capacity + 1)):  # one class per residue

            def g(j):
                return prev[r + j * w] - j * v

            cells = (capacity - r) // w + 1
            queue = []
            for j in range(cells):
                while queue and g(queue[-1]) ≤ g(j):
                    queue.pop()  # a dominated candidate leaves from the back
                queue.append(j)
                while queue[0] < j - copies:
                    queue.pop(0)  # too old: more than copies would be needed
                if meter is not None:
                    meter[0] += 1  # exactly one consult per cell
                dp[r + j * w] = g(queue[0]) + j * v
    return dp[capacity]
```

listing 589: python, the class walk, candidates scored by a closure over prev (source: dsa/samples-py/src/Ch19/window.py, lines 15-36)

```lua
-- one deque per residue class: G(j) = prev[r + j*w] - j*v, dominated
-- candidates pop from the back, the head is evicted when older than j - m
local function window(weights, values, counts, capacity, meter)
  local dp = {}
  for c = 0, capacity do dp[c] = 0 end
  for i = 1, #weights do
    local w, m, v = weights[i], counts[i], values[i]
    local prev = {}
    for c = 0, capacity do prev[c] = dp[c] end
    for r = 0, math.min(w - 1, capacity) do
      local cells = (capacity - r) // w + 1
      local deque = {} -- stores j values, head at index 1
      local function G(j) return prev[r + j * w] - j * v end
      for j = 0, cells - 1 do
        while #deque > 0 and G(deque[#deque]) ≤ G(j) do
          deque[#deque] = nil -- dominated from the back
        end
        deque[#deque + 1] = j
        while deque[1] < j - m do
          table.remove(deque, 1) -- too old for m copies
        end
        if meter then meter.ops = meter.ops + 1 end -- one consult per cell
        dp[r + j * w] = G(deque[1]) + j * v
      end
    end
  end
  return dp[capacity]
end
```

listing 590: lua, the G closure over prev, table.remove aging the head (source: dsa/samples-lua/ch19_window.lua, lines 52-79)

A monotone deque serves that max with one consult per cell: on the pinned instance the count table consults 15 candidates and the window consults 9, one per capacity cell, both answering 10. The figure follows residue class

0 of a real two item instance at capacity 18: the row 0, 7, 14, 14, 14, 14, 14 was left by an item of weight 3 and value 7, then the item of weight 3, value 4, count 2 walks the class with its deque. Each index enters the deque once and leaves at most once, from the back when a better candidate dominates it, from the front when it slides past the count, so the total work is items times capacity whatever the counts say, and the C# suite's 25 seeded instances agree with the naive exactly.

The six new lanes pin the same 9 against 15 pair, and the agreement lane widens it: window, count scan, and binary decomposition answer 31 on fixture A and 25 on fixture B, and the window matches the count scan at every capacity from 0 to 12 of fixture A. C and Java score their candidates through a g helper, Go, JavaScript, and Python close over the previous row inside the walk, and JavaScript alone keeps that row by swapping two arrays where the others copy. Lua carries the lane furthest with 39 consults on fixture A, 13 cells for each of 3 items, and a pinned deque trace showing an equal g pops the older candidate, the <= that keeps the window strict.

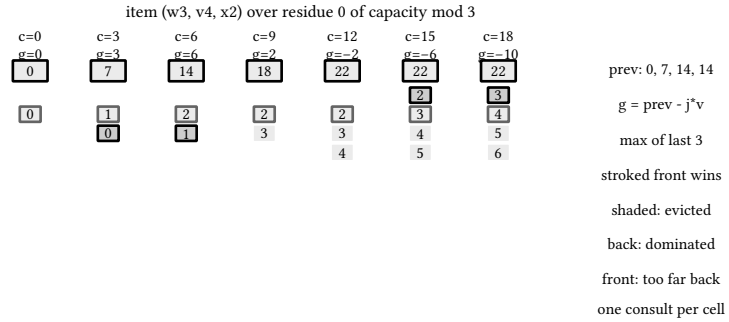


figure 159: one residue class of capacity mod 3 under the second item, the deque under each cell, back evictions shaded below, front evictions above

PITFALL: the window reads the previous row

The deque trick stays bounded only because every candidate comes from `prev`, the row as it stood before this item. Reading the live `dp` array while updating it lets a candidate reuse copies of the current item through cells already written this pass, which is exactly the unbounded knapsack, and the answers drift upward on any item with count 2 or more. The `Array.Copy` snapshot per item is the whole correctness argument, one line of it.

19.3. the grouped knapsack

The other generalization keeps exclusivity and drops counts: the menu has groups, and the knapsack must take exactly one option from each. The recurrence takes a max over options where the 0/1 table chose between skipping and taking, and an unreachable marker carries the budgets no option fits.

The dry run: the fixture is the three-group menu against capacity 10, asserted by all seven suites, the groups reading 2 or 4 at 20 or 40, 3 or 6 at 30 or 60, 5 or 1 at 50 or 10, with the capacity 7 answer of 60 and the infeasible capacities 5 and 3 at −1 pinned beside it.

1. Group 0 alone fills the first row: capacity 2 reads 0 + 20 = 20 and capacity 4 reads 0 + 40 = 40.
2. Group 1 over that row: capacity 5 reads 30 + 20 = 50, capacity 9 reads 60 + 20 = 80, capacity 10 reads 60 + 40 = 100.
3. Group 2 over that: capacity 10 weighs 50 + 50 = 100 against 10 + 80 = 90 and the table closes at 100.
4. The rebuild walks back through weights 2, 3, 5, spending 2 + 3 + 5 = 10 exactly, one option per group, 20 + 30 + 50 = 100.
5. Greedy per group takes 40 then 60, spends 4 + 6 = 10, strands group 2, and serves only 2 of the 3 groups at the same total.

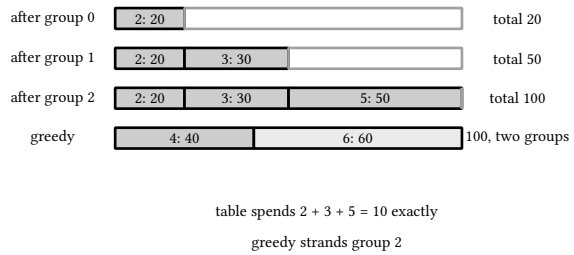


figure 160: the budget bar filling group by group along the table's chosen path, the greedy bar below spending 4 and 6 with nothing left for group 2

The table serves all three groups at 100 with the bar exactly full, and the listings below carry the build and its rebuild in seven languages.

```c
// capacity ascending then options ascending, strictly greater updates:
// that scan order is what makes the readback deterministic
static long grouped_knapsack(const menu_option *groups, const int *nopts,
                             int ngroups, int capacity, int *chosen) {
  long dp[MAX_CAP + 1], next[MAX_CAP + 1];
  static int choice[GROUPS][MAX_CAP + 1];
  for (int cap = 0; cap <= capacity; cap++)
    dp[cap] = cap == 0 ? 0 : UNREACHABLE;
  for (int g = 0; g < ngroups; g++) {
    for (int cap = 0; cap <= capacity; cap++)
      next[cap] = UNREACHABLE;
    for (int cap = 0; cap <= capacity; cap++)
      for (int o = 0; o < nopts[g]; o++) {
        int weight = groups[g * OPTIONS + o].weight;
        long value = groups[g * OPTIONS + o].value;
        if (weight > cap || dp[cap - weight] == UNREACHABLE)
          continue;
        long candidate = dp[cap - weight] + value;
        if (candidate > next[cap]) {
          next[cap] = candidate;
          choice[g][cap] = o + 1; // 0 means no option fit
        }
      }
    for (int cap = 0; cap <= capacity; cap++)
      dp[cap] = next[cap];
  }
  int best = 0;
  for (int cap = 1; cap <= capacity; cap++)
    if (dp[cap] > dp[best])
      best = cap;
  if (dp[best] == UNREACHABLE) {
    for (int g = 0; g < ngroups; g++)
      chosen[g] = -1;
    return -1;
  }
  int cap = best;
  for (int g = ngroups - 1; g >= 0; g--) {
    int o = choice[g][cap] - 1;
    chosen[g] = o;
    cap -= groups[g * OPTIONS + o].weight;
  }
  return dp[best];
}
```

listing 591: c, capacity ascending then options ascending, the choice table read back (source: dsa/samples-c/src/Ch19/grouped.c, lines 30-72)

```
func GroupedKnapsack(groups [][]GroupOption, capacity int) (value int64, chosen []int) {
  unreachable := int64(math.MinInt64 / 4)
  dp := make([]int64, capacity+1)
  for c := range dp {
    dp[c] = unreachable
  }
  dp[0] = 0
  choice := make([][]int, len(groups))
  for g := range choice {
    choice[g] = make([]int, capacity+1)
  }
  for g, options := range groups {
    next := make([]int64, capacity+1)
    for c := range next {
      next[c] = unreachable
    }
    for c := 0; c <= capacity; c++ {
      for o, opt := range options {
        if opt.Weight > c || dp[c-opt.Weight] == unreachable {
          continue
        }
        if candidate := dp[c-opt.Weight] + opt.Value; candidate > next[c] {
          next[c] = candidate
          choice[g][c] = o + 1 // zero means no option fit
        }
      }
    }
    dp = next
  }

  best := 0
  for c := 1; c <= capacity; c++ {
    if dp[c] > dp[best] {
      best = c
    }
  }
  if dp[best] == unreachable {
    return -1, nil
  }

  picks := make([]int, len(groups))
  cap := best
  for g := len(groups) - 1; g ≥ 0; g-- {
    o := choice[g][cap] - 1
    picks[g] = o
    cap -= groups[g][o].Weight
  }
  return dp[best], picks
}
```

listing 592: go, min int over 4 marking unreachable, picks rebuilt from the choice grid (source: dsa/samples-go/ch19/grouped.go, lines 17-65)

```
  record Option(int weight, long value) {}

  static final long UNREACHABLE = Long.MIN_VALUE / 4;

  static long groupedKnapsack(Option[][] groups, int capacity, int[] chosen) {
    int ngroups = groups.length;
    long[] dp = new long[MAX_CAP + 1];
    long[] next = new long[MAX_CAP + 1];
    int[][] choice = new int[GROUPS][MAX_CAP + 1];
    dp[0] = 0;
    for (int cap = 1; cap ≤ capacity; cap++)
      dp[cap] = UNREACHABLE;
    // capacity ascending then options ascending, strictly greater
    // updates: that scan order is what makes the readback deterministic
    for (int g = 0; g < ngroups; g++) {
      java.util.Arrays.fill(next, 0, capacity + 1, UNREACHABLE);
      for (int cap = 0; cap ≤ capacity; cap++)
        for (int o = 0; o < groups[g].length; o++) {
          Option opt = groups[g][o];
          if (opt.weight() > cap || dp[cap - opt.weight()] == UNREACHABLE)
            continue;
          long candidate = dp[cap - opt.weight()] + opt.value();
          if (candidate > next[cap]) {
            next[cap] = candidate;
            choice[g][cap] = o + 1; // 0 means no option fit
          }
        }
      long[] t = dp;
      dp = next;
      next = t;
    }
    int best = 0;
    for (int cap = 1; cap ≤ capacity; cap++)
      if (dp[cap] > dp[best])
        best = cap;
    if (dp[best] == UNREACHABLE) {
      for (int g = 0; g < ngroups; g++)
        chosen[g] = -1;
      return -1;
    }
    int cap = best;
    for (int g = ngroups - 1; g ≥ 0; g--) {
      int o = choice[g][cap] - 1;
      chosen[g] = o;
      cap -= groups[g][o].weight();
    }
    return dp[best];
  }
```

listing 593: java, a record per option, the quarter-of-min unreachable sentinel, the choice table read back (source: dsa/samples-java/src/Ch19/Grouped.java, lines 20-67)

```
    /// <summary>Grouped knapsack: exactly one option from every group, the chosen indices rebuilt.</
summary>
    public static (long Value, int[] Chosen) GroupedKnapsack((int Weight, long Value)[][] groups, int
capacity)
    {
        const long unreachable = long.MinValue / 4;
        var dp = new long[capacity + 1];
        Array.Fill(dp, unreachable);
        dp[0] = 0;
        var choice = new int[groups.Length, capacity + 1];
        for (int g = 0; g < groups.Length; g++)
        {
            var next = new long[capacity + 1];
            Array.Fill(next, unreachable);
            for (int c = 0; c ≤ capacity; c++)
                for (int o = 0; o < groups[g].Length; o++)
                {
                    var (weight, value) = groups[g][o];
                    if (weight > c || dp[c - weight] == unreachable)
                        continue;
                    long candidate = dp[c - weight] + value;
                    if (candidate > next[c])
                    {
                        next[c] = candidate;
                        choice[g, c] = o + 1; // 0 means no option fit
                    }
                }
            dp = next;
        }

        int best = 0;
        for (int c = 1; c ≤ capacity; c++)
            if (dp[c] > dp[best])
                best = c;
        if (dp[best] == unreachable)
            return (-1, Array.Empty<int>());

        var chosen = new int[groups.Length];
        int cap = best;
        for (int g = groups.Length - 1; g ≥ 0; g--)
        {
            int o = choice[g, cap] - 1;
            chosen[g] = o;
            cap -= groups[g][o].Weight;
        }
        return (dp[best], chosen);
    }
```

listing 594: c#, grouped knapsack, one option per group, reconstruction from the choice table (source: dsa/samples/src/Ch19/AdvDp.cs, lines 105-149)

```
export function groupedKnapsack(groups, capacity) {
  let dp = new Array(capacity + 1).fill(UNREACHABLE);
  dp[0] = 0;
  const choice = groups.map(() => new Array(capacity + 1).fill(0));

  for (let g = 0; g < groups.length; g++) {
    const next = new Array(capacity + 1).fill(UNREACHABLE);
    for (let c = 0; c ≤ capacity; c++) {
      for (let o = 0; o < groups[g].length; o++) {
        const option = groups[g][o];
        if (option.weight > c || dp[c - option.weight] === UNREACHABLE) {
          continue;
        }
        const candidate = dp[c - option.weight] + option.value;
        if (candidate > next[c]) {
          next[c] = candidate;
          choice[g][c] = o + 1; // 0 means no option fit
        }
      }
    }
    dp = next;
  }

  let best = 0;
  for (let c = 1; c ≤ capacity; c++) {
    if (dp[c] > dp[best]) best = c;
  }
  if (dp[best] === UNREACHABLE) return { value: -1, chosen: [] };

  const chosen = new Array(groups.length);
  let cap = best;
  for (let g = groups.length - 1; g ≥ 0; g--) {
    const o = choice[g][cap] - 1;
    chosen[g] = o;
    cap -= groups[g][o].weight;
  }
  return { value: dp[best], chosen };
}
```

listing 595: javascript, minus infinity sentinels, value and chosen returned together (source: dsa/samples-js/src/ch19-grouped.mjs, lines 8-45)

```python
def grouped(groups, capacity):
    # unreachable cells carry None: no option of the group so far fits
    dp = [None] * (capacity + 1)
    dp[0] = 0
    choice = [[0] * (capacity + 1) for _ in groups]
    for g, options in enumerate(groups):
        nxt = [None] * (capacity + 1)
        for c in range(capacity + 1):
            for o, (weight, value) in enumerate(options):
                if weight > c or dp[c - weight] is None:
                    continue
                candidate = dp[c - weight] + value
                if nxt[c] is None or candidate > nxt[c]:  # strictly greater
                    nxt[c] = candidate
                    choice[g][c] = o + 1  # 0 means no option fit
        dp = nxt
    best = 0
    for c in range(1, capacity + 1):
        if dp[c] is not None and (dp[best] is None or dp[c] > dp[best]):
            best = c
    if dp[best] is None:
        return -1, []
    chosen = [0] * len(groups)
    cap = best
    for g in range(len(groups) - 1, -1, -1):
        option = choice[g][cap] - 1
        chosen[g] = option
        cap -= groups[g][option][0]
    return dp[best], chosen
```

listing 596: python, none flags unreachable cells, no sentinel arithmetic at all (source: dsa/samples-py/src/Ch19/grouped.py, lines 16-44)

```lua
-- groups: array of arrays of {weight, value}; returns the best value
-- (or -1) and the chosen option index per group (0-based)
local function grouped(groups, capacity)
  local dp = {[0] = 0} -- nil means unreachable
  local choice = {}
  for g = 1, #groups do
    local next_dp = {}
    choice[g] = {}
    for c = 0, capacity do
      for o = 1, #groups[g] do
        local weight, value = groups[g][o][1], groups[g][o][2]
        if weight ≤ c and dp[c - weight] ~= nil then
          local candidate = dp[c - weight] + value
          if next_dp[c] == nil or candidate > next_dp[c] then
            next_dp[c] = candidate
            choice[g][c] = o -- 1-based option, 0 means nothing fit
          end
        end
      end
    end
    dp = next_dp
  end
  local best = 0
  for c = 1, capacity do
    if dp[c] ~= nil and (dp[best] == nil or dp[c] > dp[best]) then best = c end
  end
  if dp[best] == nil then return -1, {} end
  local chosen = {}
  local cap = best
  for g = #groups, 1, -1 do
    local o = choice[g][cap] -- always set on a reachable walk
    chosen[g] = o - 1
    cap = cap - groups[g][o][1]
  end
  return dp[best], chosen
end
```

listing 597: lua, nil for unreachable, one-based options emitted zero-based (source: dsa/samples-lua/ch19_grouped.lua, lines 6-41)

On the instance of three groups against capacity 10 the answer is 100, weights 2, 3 and 5 summing to the budget exactly, and the reconstruction is property checked: the indices are valid, one per group, the weight stays within 10, the values sum back to 100. Greedy per group takes the best option of each, 40 then 60, spends the whole budget and strands the third group with only two groups served, while the table serves all three and still totals 100. Any bundle that must pick one vendor per slot is this shape.

Every lane pins the same three landings: 100 with the picks 0, 0, 0 at capacity 10, 60 with the picks 0, 0, 1 at capacity 7 where the light 1 for 10 option replaces the heavy 5 for 50, and −1 at capacities 5 and 3 where the minimum load of 6 cannot fit. The unreachable marker is where the trees part: C and Java guard a long min over 4, Go a min int over 4, JavaScript minus infinity, Python a none flag that skips the arithmetic, Lua nil. The readback lands the same picks everywhere because the scans run capacity ascending then options ascending and only a strictly greater candidate rewrites a cell, and the brute force over the 8 option tuples confirms 100 and 60 in each suite.

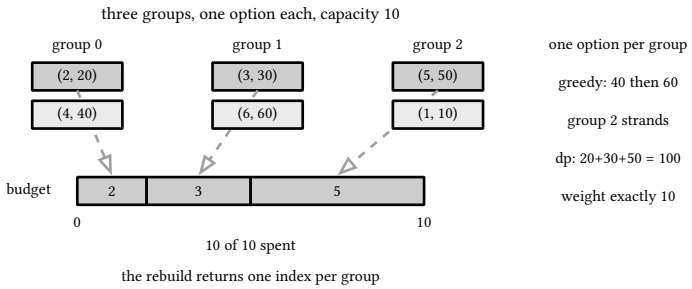


figure 161: the grouped knapsack, one shaded option per group, the budget bar filled exactly by weights 2, 3 and 5

19.4. convex hull trick

Partition costs hide a quadratic in plain sight. Split a sequence into consecutive groups, pay the square of each group sum plus a constant per group, and the recurrence weighs every split point against every prefix: dp of i is the minimum over j below i of dp of j plus the square of the gap between prefix sums plus the constant. Expand that square and the part that still depends on j is a line in the prefix sum S(j), slope minus 2 S(j), intercept dp of j plus S(j) squared, and the query lands at x equal to S(i). Prefix sums of nonnegative values only grow, so the slopes arrive strictly decreasing and the queries only move right, exactly the contract of a pointer hull.

The dry run: the fixture is 1, 2, 3 with constant 10, asserted by all seven suites at 38, with constant 0 at 14, 3, 1, 4, 1, 5 at constant 2 at 62, and 2, 2, 2, 2 at constant 3 at 28, the naive split scan agreeing in every tree, and the C# suite alone running the seeded n = 200 meters.

1. The prefix sums read S = 0, 1, 3, 6, and every candidate split hangs off them.
2. Solo groups 1, 2, 3 cost 1 + 4 + 9 = 14 plus 3 x 10 = 30, a total of 44.
3. The split after the 2 costs 9 + 9 = 18 plus 2 x 10 = 20, a total of 38, the cheapest.
4. The split after the 1 costs 1 + 25 = 26 plus 20, a total of 46, and the whole run costs 36 + 10 = 46.
5. At n = 200 the naive weighs its pinned 19900 split points while the hull adds 200 lines and answers 200 queries, 200 + 200 = 400 touches, the same totals.

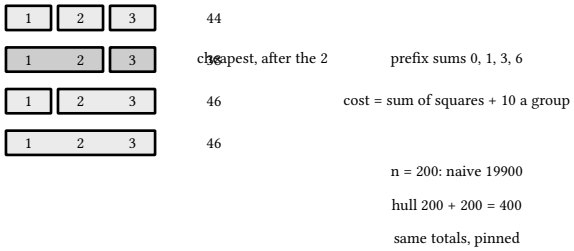


figure 162: every split of 1, 2, 3 weighed, groups boxed with their costs beside, the cheapest after the 2 shaded

The 38 is the small pinned landing and 400 the metered big one, and the listings below drive the recurrence through the hull in seven languages.

```c
// middle line 2 is useless when line 3 overtakes line 1 no later than
// line 2 does, compared by cross multiplication
static int overtaken(const hull *h, int i1, int i2, long long slope3,
                     long long intercept3) {
  return (h->intercepts[i1] - intercept3) * (h->slopes[i2] - h->slopes[i1]) <=
         (h->intercepts[i1] - h->intercepts[i2]) * (slope3 - h->slopes[i1]);
}

static void add_line(hull *h, long long slope, long long intercept) {
  h->lines++;
  while (h->count >= 2 &&
         overtaken(h, h->count - 2, h->count - 1, slope, intercept))
    h->count--;
  h->slopes[h->count] = slope;
  h->intercepts[h->count] = intercept;
  h->count++;
  if (h->ptr >= h->count)
    h->ptr = h->count - 1;
}

static long long query(hull *h, long long x) {
  h->queries++;
  while (h->ptr + 1 < h->count &&
         line_value(h, h->ptr + 1, x) <= line_value(h, h->ptr, x))
    h->ptr++;
  return line_value(h, h->ptr, x);
}

// min of sum of squared group sums plus a constant per group: the line
// (0,0) first, then one line per split point
static long long min_partition_squared(const long long *values, int count,
                                       long long constant, hull *h) {
  long long s[MAX_VALUES + 1], dp[MAX_VALUES + 1];
  s[0] = 0;
  for (int i = 0; i < count; i++)
    s[i + 1] = s[i] + values[i];
  add_line(h, 0, 0);
  for (int i = 1; i <= count; i++) {
    dp[i] = query(h, s[i]) + s[i] * s[i] + constant;
    if (i < count)
      add_line(h, -2 * s[i], dp[i] + s[i] * s[i]);
  }
  return dp[count];
}
```

listing 598: c, the cross-multiplied overtaken test, add, query, the partition loop (source: dsa/samples-c/src/Ch19/cht.c, lines 33-76)

```
// Query evaluates the envelope at x; the pointer only walks forward.
func (h *MonotoneHull) Query(x int64) int64 {
  h.Queries++
  for h.ptr+1 < len(h.slopes) && h.value(h.ptr+1, x) <= h.value(h.ptr, x) {
    h.ptr++
  }
  return h.value(h.ptr, x)
}

// line i2 is useless when the new line 3 overtakes line i1 no later
// than line i2 does
func (h *MonotoneHull) overtaken(i1, i2 int, slope3, intercept3 int64) bool {
  return (h.intercepts[i1]-intercept3)*(h.slopes[i2]-h.slopes[i1]) <=
    (h.intercepts[i1]-h.intercepts[i2])*(slope3-h.slopes[i1])
}

// PartitionMeter counts split-point weighings in the naive scan.
type PartitionMeter struct{ Ops int64 }

// MinPartitionSquaredCost returns the minimum of the sum of squared
// group sums plus a constant per group, via the hull: dp[i] = min over
// j < i of dp[j] + (s[i]-s[j])^2 + constant, the inner minimum being a
// line in s[i] with slope -2*s[j].
func MinPartitionSquaredCost(values []int64, constant int64, hull *MonotoneHull) int64 {
  n := len(values)
  s := make([]int64, n+1)
  for i, v := range values {
    s[i+1] = s[i] + v
  }
  if hull == nil {
    hull = &MonotoneHull{}
  }
  dp := make([]int64, n+1)
  hull.AddLine(0, 0) // j = 0: the whole prefix as one group
  for i := 1; i <= n; i++ {
    dp[i] = hull.Query(s[i]) + s[i]*s[i] + constant
    if i < n { // a line for split point j = i serves every later prefix
      hull.AddLine(-2*s[i], dp[i]+s[i]*s[i])
    }
  }
  return dp[n]
}
```

listing 599: go, the forward pointer, the overtaken test, the recurrence and its naive twin (source: dsa/samples-go/ch19/cht.go, lines 36-77)

```
  long[] intercepts = new long[MAX_LINES];
  int count;
  int ptr;
  long lines;
  long queries;

  long lineValue(int i, long x) {
    return slopes[i] * x + intercepts[i];
  }

  // middle line 2 is useless when line 3 overtakes line 1 no later
  // than line 2 does, compared by cross multiplication
  boolean overtaken(int i1, int i2, long slope3, long intercept3) {
    return (intercepts[i1] - intercept3) * (slopes[i2] - slopes[i1]) ≤
        (intercepts[i1] - intercepts[i2]) * (slope3 - slopes[i1]);
  }

  void addLine(long slope, long intercept) {
    lines++;
    while (count ≥ 2 && overtaken(count - 2, count - 1, slope, intercept))
      count--;
    slopes[count] = slope;
    intercepts[count] = intercept;
    count++;
    if (ptr ≥ count)
      ptr = count - 1;
  }

  long query(long x) {
    queries++;
    while (ptr + 1 < count && lineValue(ptr + 1, x) ≤ lineValue(ptr, x))
      ptr++; // the forward pointer only ever advances
    return lineValue(ptr, x);
  }
}

static final int MAX_VALUES = 16;

// min of sum of squared group sums plus a constant per group: the line
// (0,0) first, then one line per split point
static long minPartitionSquared(long[] values, long constant, Hull h) {
  int count = values.length;
  long[] s = new long[MAX_VALUES + 1];
  long[] dp = new long[MAX_VALUES + 1];
  for (int i = 0; i < count; i++)
    s[i + 1] = s[i] + values[i];
  h.addLine(0, 0);
  for (int i = 1; i ≤ count; i++) {
    dp[i] = h.query(s[i]) + s[i] * s[i] + constant;
    if (i < count)
      h.addLine(-2 * s[i], dp[i] + s[i] * s[i]);
  }
  return dp[count];
}
```

listing 600: java, a nested Hull carrying its own line and query meters, the cross-multiplied test, the recurrence (source: dsa/samples-java/src/Ch19/Cht.java, lines 21-74)

```
    /// <summary>Lower envelope of lines for the convex hull trick: slopes arrive strictly decreasing,
queries at increasing x.</summary>
    public sealed class MonotoneHull
    {
        private long[] slopes = new long[8];
        private long[] intercepts = new long[8];
        private int count;
        private int ptr;

        public long Lines { get; private set; }
        public long Queries { get; private set; }

        /// <summary>Adds one line, popping middle lines that can never hold the minimum again.</summary>
        public void AddLine(long slope, long intercept)
        {
            Lines++;
            while (count ≥ 2 && Overtaken(count - 2, count - 1, slope, intercept))
                count--;
            if (count == slopes.Length)
            {
                Array.Resize(ref slopes, count * 2);
                Array.Resize(ref intercepts, count * 2);
            }
            slopes[count] = slope;
            intercepts[count] = intercept;
            count++;
            if (ptr ≥ count)
                ptr = count - 1;
        }

        /// <summary>Evaluates the envelope at x; the pointer only walks forward.</summary>
        public long Query(long x)
        {
            Queries++;
            while (ptr + 1 < count && Value(ptr + 1, x) ≤ Value(ptr, x))
                ptr++;
            return slopes[ptr] * x + intercepts[ptr];
        }

        private long Value(int i, long x) => slopes[i] * x + intercepts[i];

        // line 2 is useless when line 3 overtakes line 1 no later than line 2 does
        private bool Overtaken(int i1, int i2, long slope3, long intercept3) =>
            (intercepts[i1] - intercept3) * (slopes[i2] - slopes[i1])
                ≤ (intercepts[i1] - intercepts[i2]) * (slope3 - slopes[i1]);
    }

    /// <summary>Minimum of sum of squared group sums plus a constant per group, via the hull.</summary>
    public static long MinPartitionSquaredCost(long[] values, long constant, MonotoneHull? hull = null)
    {
        int n = values.Length;
        var s = new long[n + 1];
        for (int i = 0; i < n; i++)
            s[i + 1] = s[i] + values[i];
        var dp = new long[n + 1];
        hull ??= new MonotoneHull();
        hull.AddLine(0, 0); // j = 0: the whole prefix as one group
```

```
        for (int i = 1; i ≤ n; i++)
        {
            dp[i] = hull.Query(s[i]) + s[i] * s[i] + constant;
            if (i < n) // a line for split point j = i, used by every later prefix
                hull.AddLine(-2 * s[i], dp[i] + s[i] * s[i]);
        }
        return dp[n];
    }
```

listing 601: c#, a monotone hull of lines, then the partition recurrence driven through it (source: dsa/samples/src/Ch19/AdvDp.cs, lines 151-214)

```
export class MonotoneHull {
  #slopes = [];
  #intercepts = [];
  #ptr = 0;
  lines = 0;
  queries = 0;

  #value(i, x) {
    return this.#slopes[i] * x + this.#intercepts[i];
  }

  // line 2 is useless when line 3 overtakes line 1 no later than line 2
  // does, cross-multiplied so everything stays exact integers
  #overtaken(i1, i2, slope3, intercept3) {
    return (
      (this.#intercepts[i1] - intercept3) * (this.#slopes[i2] - this.#slopes[i1]) ≤
      (this.#intercepts[i1] - this.#intercepts[i2]) * (slope3 - this.#slopes[i1])
    );
  }

  addLine(slope, intercept) {
    this.lines++;
    while (
      this.#slopes.length ≥ 2 &&
      this.#overtaken(
        this.#slopes.length - 2,
        this.#slopes.length - 1,
        slope,
        intercept,
      )
    ) {
      this.#slopes.pop();
      this.#intercepts.pop();
    }
    this.#slopes.push(slope);
    this.#intercepts.push(intercept);
    if (this.#ptr ≥ this.#slopes.length) {
      this.#ptr = this.#slopes.length - 1;
    }
  }

  query(x) {
    this.queries++;
    while (
      this.#ptr + 1 < this.#slopes.length &&
      this.#value(this.#ptr + 1, x) ≤ this.#value(this.#ptr, x)
    ) {
      this.#ptr++;
    }
    return this.#value(this.#ptr, x);
  }
}
```

listing 602: javascript, a private-field hull, cross-multiplied overtaken, forward pointer (source: dsa/samples-js/src/ch19-cht.mjs, lines 5-56)

```
    def add_line(self, slope, intercept):
        self.lines += 1
        while len(self.slopes) ≥ 2 and self.overtaken(
            len(self.slopes) - 2, len(self.slopes) - 1, slope, intercept
        ):
            self.slopes.pop()  # a middle line that can never win again
            self.intercepts.pop()
        self.slopes.append(slope)
        self.intercepts.append(intercept)
        if self.ptr ≥ len(self.slopes):
            self.ptr = len(self.slopes) - 1

    def query(self, x):
        self.queries += 1

        def value(i):
            return self.slopes[i] * x + self.intercepts[i]

        while self.ptr + 1 < len(self.slopes):
            if value(self.ptr + 1) > value(self.ptr):
                break  # the next line loses: the pointer stays put
            self.ptr += 1  # the forward pointer only ever advances
        return value(self.ptr)

    def overtaken(self, i1, i2, slope3, intercept3):
        # line 2 is useless when line 3 overtakes line 1 no later than line
        # 2 does, compared by cross multiplication in exact integers
        left = (self.intercepts[i1] - intercept3) * (self.slopes[i2] - self.slopes[i1])
        right = (self.intercepts[i1] - self.intercepts[i2]) * (slope3 - self.slopes[i1])
        return left ≤ right


def hull_partition(values, constant, hull=None):
    # dp[i] = min over j < i of dp[j] + (s[i] - s[j])^2 + constant
    size = len(values)
    s = [0] * (size + 1)
    for i, v in enumerate(values):
        s[i + 1] = s[i] + v
    dp = [0] * (size + 1)
    if hull is None:
        hull = MonotoneHull()
    hull.add_line(0, 0)  # j = 0: the whole prefix as one group
    for i in range(1, size + 1):
        dp[i] = hull.query(s[i]) + s[i] * s[i] + constant
        if i < size:  # a line for split point j = i, used by later prefixes
            hull.add_line(-2 * s[i], dp[i] + s[i] * s[i])
    return dp[size]
```

listing 603: python, the hull then the partition recurrence it serves (source: dsa/samples-py/src/Ch19/cht.py, lines 22-68)

```lua
local function hull()
  local h = {}
  local slopes, intercepts = {}, {}
  h.lines, h.queries = 0, 0
  local function overtaken(i1, i2, s3, b3)
    -- line 2 is useless when line 3 overtakes line 1 no later than line 2
    return (intercepts[i1] - b3) * (slopes[i2] - slopes[i1])
        ≤ (intercepts[i1] - intercepts[i2]) * (s3 - slopes[i1])
  end
  function h.add_line(slope, intercept)
    h.lines = h.lines + 1
    while #slopes ≥ 2
      and overtaken(#slopes - 1, #slopes, slope, intercept) do
      slopes[#slopes] = nil -- the middle line can never hold the min
      intercepts[#intercepts] = nil
    end
    slopes[#slopes + 1] = slope
    intercepts[#intercepts + 1] = intercept
    if h.ptr == nil or h.ptr > #slopes then h.ptr = #slopes end
  end
  function h.query(x)
    h.queries = h.queries + 1
    local function value(i)
      return slopes[i] * x + intercepts[i]
    end
    while h.ptr < #slopes and value(h.ptr + 1) ≤ value(h.ptr) do
      h.ptr = h.ptr + 1 -- the pointer only walks forward
    end
    return slopes[h.ptr] * x + intercepts[h.ptr]
  end
  return h
end

-- min sum of squared group sums plus a constant per group
local function min_partition_squared(values, constant, h)
  local n = #values
  local s = {0}
  for i = 1, n do s[i + 1] = s[i] + values[i] end
  local dp = {0}
  h = h or hull()
  h.add_line(0, 0) -- j = 0: the whole prefix as one group
  for i = 1, n do
    dp[i + 1] = h.query(s[i + 1]) + s[i + 1] * s[i + 1] + constant
    if i < n then -- a line for split point j = i, slope -2*s[i]
      h.add_line(-2 * s[i + 1], dp[i + 1] + s[i + 1] * s[i + 1])
    end
  end
  return dp[n + 1]
end
```

listing 604: lua, closures over the line arrays, the recurrence in one-based prefix sums (source: dsa/samples-lua/ch19_cht.lua, lines 7-55)

The naive weighs n(n-1)/2 split points, 19900 at n = 200, and the C# suite pins that count on the seeded instance. The hull adds one line per split point and answers one query per prefix, 200 and 200, a total of 400 touches, and returns the same totals as the naive there, on the C# suite's 25 seeded arrays up to 60 elements, and on the small case: 1, 2, 3 with constant 10 costs 38, cheapest split after the 2. Dominated lines pop on arrival and the query pointer never walks backward, so each line and each query is touched once, which is what the counters assert.

The six new lanes hold the same envelope. The overtaken test cross-multiplies in 64-bit integers and reads <= in every tree, long long in C, long in Java with every product exact, int64 in Go, Lua integers, Python's native ints, and JavaScript numbers exact at these magnitudes, and the query pointer only walks forward. The counter lane agrees everywhere, 3 values grow 3 hull lines and answer 3 queries, and the naive scan returns 38, 14, 62, and 28 on the four fixtures in each suite. Lua squares the gap by multiplication to stay in integers and pins the naive meter at 10 interior splits for the 5 value fixture, the quadratic count the hull removes.

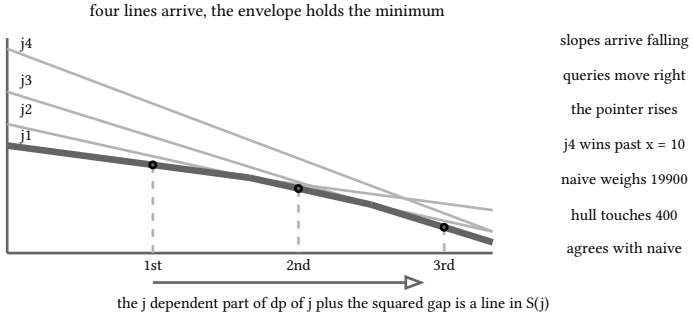


figure 163: the split points as lines, the lower envelope heavier, three queries at successive x positions

WARNING: the pointer hull is conditional

The pointer walk is amortized constant only while the contract holds: slopes arrive strictly decreasing and queries land at nondecreasing x. The partition keeps both because values are nonnegative, so prefix sums only grow, slopes only fall, and query positions only rise. One negative value breaks all three at once. Outside the contract the fix is a binary search inside `Query` or a Li Chao tree, a log factor either way, and the naive stays the cross-check.

19.5. longest increasing subsequence

The last speedup is the same story in miniature: keep less, search what you keep. The array tails holds, for each length, the index of the smallest value that ends an increasing run of that length. The tracked values stay sorted, so every element lower bounds into the array, replaces the first tail it cannot extend and links its parent, or appends and grows the run.

The dry run: the fixture is 10, 9, 2, 5, 3, 7, 101, 18, asserted by all seven suites with the 2, 2, 2 pair and the 1, 2, 2, 3 pair beside it, the n = 1000 meter a C# lane, and the pairs table oracle carried in every tree.

1. 10 opens the tails, 9 and 2 each replace it, and 5 appends: the tails read 2, 5 at length 2.
2. 3 replaces 5, 7 appends, 101 appends: 2, 3, 7, 101 at length 4.
3. 18 replaces 101 by the lower bound and the tails close at 2, 3, 7, 18, length 4.
4. The parents chain back from index 7: 7 to 5 to 4 to 2, and the rebuild reads indices 2, 4, 5, 7 with values 2, 3, 7, 18, climbing in both position and value.
5. On 2, 2, 2 the strict lower bound keeps length 1 while the upper bound variant keeps all three, 3.
6. At n = 1000 the pairs table consults its pinned 499500 pairs while the tails search stays inside 10 x 1000 = 10000 comparisons.

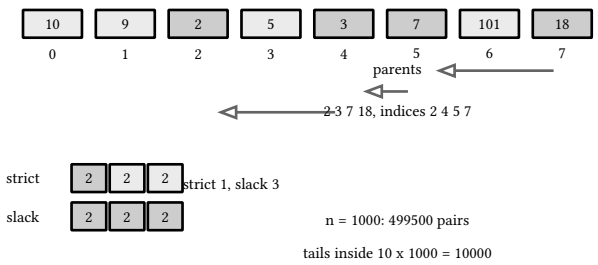


figure 164: the run shaded on the array, the parent chain walking back from index 7, the strict and slack readings of 2, 2, 2 below

The length 4 with its rebuilt indices is the pinned landing, and the listings below carry the tails, the parents, and the slack variant in seven languages.

```c
// strict lis: tails holds the index of the smallest tail of each
// length, parent links rebuild one subsequence in positions
static int lis_strict(const long long *a, int count, int *indices) {
  int tails[MAX_VALUES], parent[MAX_VALUES], len = 0;
  for (int i = 0; i < count; i++) {
    int lo = 0, hi = len;
    while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      if (a[tails[mid]] < a[i]) // lower bound
        lo = mid + 1;
      else
        hi = mid;
    }
    parent[i] = lo > 0 ? tails[lo - 1] : -1;
    if (lo == len)
      tails[len++] = i;
    else
      tails[lo] = i;
  }
  int cur = len > 0 ? tails[len - 1] : -1;
  for (int k = len - 1; k ≥ 0; k--) {
    indices[k] = cur;
    cur = parent[cur];
  }
  return len;
}

// non-decreasing variant: the upper bound lets equals extend the run
static int lis_nondecreasing(const long long *a, int count) {
  long long tails[MAX_VALUES];
  int len = 0;
  for (int i = 0; i < count; i++) {
    int lo = 0, hi = len;
    while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      if (tails[mid] ≤ a[i]) // upper bound
        lo = mid + 1;
      else
        hi = mid;
    }
    if (lo == len)
      tails[len++] = a[i];
    else
      tails[lo] = a[i];
  }
  return len;
}
```

listing 605: c, the strict lower bound with parent links, the upper bound variant (source: dsa/samples-c/src/Ch19/tails.c, lines 19-65)

```go
// LongestIncreasingSubsequence returns the length of the strict LIS
// and the indices of one witness. tails[l] holds the index of the
// smallest tail of any increasing run of length l+1; a lower bound
// (strict compare) places each arrival, parent links rebuild the
// subsequence from the last tail backward.
func LongestIncreasingSubsequence(a []int64, meter *LisMeter) (length int, indices []int) {
  tails := make([]int, len(a))
  parent := make([]int, len(a))
  for i := range parent {
    parent[i] = -1
  }
  length = 0
  for i := 0; i < len(a); i++ {
    lo, hi := 0, length
    for lo < hi {
      if meter ≠ nil {
        meter.Ops++
      }
      mid := lo + (hi-lo)/2
      if a[tails[mid]] < a[i] {
        lo = mid + 1
      } else {
        hi = mid
      }
    }
    if lo > 0 {
      parent[i] = tails[lo-1]
    }
    if lo == length {
      tails[length] = i
      length++
    } else {
      tails[lo] = i
    }
  }

  indices = make([]int, length)
  cur := -1
  if length > 0 {
    cur = tails[length-1]
  }
  for k := length - 1; k ≥ 0; k-- {
    indices[k] = cur
    cur = parent[cur]
  }
  return length, indices
}
```

listing 606: go, the strict walk with its probe meter, parents rebuilt backward (source: dsa/samples-go/ch19/tails.go, lines 6-52)

```
  static final int MAX_VALUES = 16;

  static int[] lastIndices;

  // strict lis: tails holds the index of the smallest tail of each
  // length, parent links rebuild one subsequence in positions
  static int lisStrict(long[] a) {
    int[] tails = new int[MAX_VALUES];
    int[] parent = new int[MAX_VALUES];
    int len = 0;
    for (int i = 0; i < a.length; i++) {
      int lo = 0, hi = len;
      while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (a[tails[mid]] < a[i]) // lower bound: equals replace
          lo = mid + 1;
        else
          hi = mid;
      }
      parent[i] = lo > 0 ? tails[lo - 1] : -1;
      if (lo == len)
        tails[len++] = i;
      else
        tails[lo] = i;
    }
    int cur = len > 0 ? tails[len - 1] : -1;
    int[] indices = new int[len];
    for (int k = len - 1; k ≥ 0; k--) {
      indices[k] = cur;
      cur = parent[cur];
    }
    lastIndices = indices;
    return len;
  }

  // non-decreasing variant: the upper bound lets equals extend the run
  static int lisNondecreasing(long[] a) {
    long[] tails = new long[MAX_VALUES];
    int len = 0;
    for (long x : a) {
      int lo = 0, hi = len;
      while (lo < hi) {
        int mid = lo + (hi - lo) / 2;
        if (tails[mid] ≤ x) // upper bound: equals extend
          lo = mid + 1;
        else
          hi = mid;
      }
      if (lo == len)
        tails[len++] = x;
      else
        tails[lo] = x;
    }
    return len;
  }
```

listing 607: java, the strict lower bound with parent links, the upper bound variant one comparison away (source: dsa/samples-java/src/Ch19/Tails.java, lines 19-73)

```
    /// <summary>Strict lis: tails plus lower bound, parent links rebuild one subsequence.</summary>
    public static int LongestIncreasingSubsequence(long[] a, out int[] indices, Meter? meter = null)
    {
        var tails = new int[a.Length]; // index of the smallest tail of each length
        var parent = new int[a.Length];
        Array.Fill(parent, -1);
        int len = 0;
        for (int i = 0; i < a.Length; i++)
        {
            int lo = 0, hi = len;
            while (lo < hi)
            {
                if (meter != null)
                    meter.Ops++;
                int mid = lo + (hi - lo) / 2;
                if (a[tails[mid]] < a[i])
                    lo = mid + 1;
                else
                    hi = mid;
            }
            if (lo > 0)
                parent[i] = tails[lo - 1];
            if (lo == len)
                tails[len++] = i;
            else
                tails[lo] = i;
        }

        indices = new int[len];
        int cur = len > 0 ? tails[len - 1] : -1;
        for (int k = len - 1; k >= 0; k--)
        {
            indices[k] = cur;
            cur = parent[cur];
        }
        return len;
    }

    /// <summary>Non-decreasing variant: the upper bound lets equals extend the run.</summary>
    public static int LongestNonDecreasing(long[] a)
    {
        var tails = new long[a.Length];
        int len = 0;
        foreach (long v in a)
        {
            int lo = 0, hi = len;
            while (lo < hi)
            {
                int mid = lo + (hi - lo) / 2;
                if (tails[mid] <= v)
                    lo = mid + 1;
                else
                    hi = mid;
            }
            if (lo == len)
                tails[len++] = v;
            else
```

```
                tails[lo] = v;
        }
        return len;
    }
```

listing 608: c#, strict lis with tails and parents, the non-decreasing variant one bound away (source: dsa/samples/src/Ch19/AdvDp.cs, lines 239-299)

```
export function longestIncreasingSubsequence(a) {
  const tails = []; // index of the smallest tail of each length
  const parent = new Array(a.length).fill(-1);
  let len = 0;
  for (let i = 0; i < a.length; i++) {
    let lo = 0;
    let hi = len;
    while (lo < hi) {
      const mid = lo + Math.floor((hi - lo) / 2);
      if (a[tails[mid]] < a[i]) lo = mid + 1;
      else hi = mid;
    }
    if (lo > 0) parent[i] = tails[lo - 1];
    if (lo === len) tails[len++] = i;
    else tails[lo] = i;
  }

  const indices = new Array(len);
  let cur = len > 0 ? tails[len - 1] : -1;
  for (let k = len - 1; k ≥ 0; k--) {
    indices[k] = cur;
    cur = parent[cur];
  }
  return { length: len, indices };
}

export function longestNonDecreasing(a) {
  const tails = [];
  let len = 0;
  for (const v of a) {
    let lo = 0;
    let hi = len;
    while (lo < hi) {
      const mid = lo + Math.floor((hi - lo) / 2);
      if (tails[mid] ≤ v) lo = mid + 1; // equals extend the run
      else hi = mid;
    }
    if (lo === len) tails[len++] = v;
    else tails[lo] = v;
  }
  return len;
}
```

listing 609: javascript, hand-rolled bounds, both variants, indices matching the other lanes (source: dsa/samples-js/src/ch19-tails.mjs, lines 6-47)

```python
def longest_increasing(a):
    tails = [0] * len(a)  # index of the smallest tail of each length
    parent = [-1] * len(a)
    length = 0
    for i in range(len(a)):
        lo, hi = 0, length
        while lo < hi:
            mid = (lo + hi) // 2
            if a[tails[mid]] < a[i]:  # lower bound: equals replace
                lo = mid + 1
            else:
                hi = mid
        if lo > 0:
            parent[i] = tails[lo - 1]
        if lo == length:
            tails[length] = i
            length += 1
        else:
            tails[lo] = i
    indices = [0] * length
    cur = tails[length - 1] if length else -1
    for k in range(length - 1, -1, -1):  # walk parents backward
        indices[k] = cur
        cur = parent[cur]
    return length, indices


def longest_nondecreasing(a):
    tails = []
    for v in a:
        lo, hi = 0, len(tails)
        while lo < hi:
            mid = (lo + hi) // 2
            if tails[mid] ≤ v:  # upper bound: equals extend
                lo = mid + 1
            else:
                hi = mid
        if lo == len(tails):
            tails.append(v)
        else:
            tails[lo] = v
    return len(tails)
```

listing 610: python, the lower bound replaces equals, the upper bound extends them (source: dsa/samples-py/src/Ch19/tails.py, lines 14-55)

```lua
local function lis(a)
  local tails, parent = {}, {}
  local len = 0
  for i = 1, #a do
    local lo, hi = 1, len + 1 -- lower bound over tails indices
    while lo < hi do
      local mid = lo + (hi - lo) // 2
      if a[tails[mid]] < a[i] then lo = mid + 1 else hi = mid end
    end
    if lo > 1 then parent[i] = tails[lo - 1] end
    if lo > len then
      len = lo
      tails[len] = i
    else
      tails[lo] = i
    end
  end
  local indices = {}
  local cur = len > 0 and tails[len] or nil
  for k = len, 1, -1 do -- walk parents backward from the last tail
    indices[k] = cur - 1 -- emit 0-based
    cur = parent[cur]
  end
  local values = {}
  for k = 1, len do values[k] = a[indices[k] + 1] end
  return len, indices, values
end

-- non-decreasing: the upper bound lets equals extend the run
local function non_decreasing(a)
  local tails = {}
  local len = 0
  for _, v in ipairs(a) do
    local lo, hi = 1, len + 1
    while lo < hi do
      local mid = lo + (hi - lo) // 2
      if tails[mid] ≤ v then lo = mid + 1 else hi = mid end
    end
    if lo > len then
      len = lo
      tails[len] = v
    else
      tails[lo] = v
    end
  end
  return len
end
```

listing 611: lua, a one-based search emitting zero-based indices, both bounds (source: dsa/samples-lua/ch19_tails.lua, lines 7-53)

On 10, 9, 2, 5, 3, 7, 101, 18 the run 2, 3, 7, 18 has length 4 and the parent links rebuild its indices, which the test checks strictly increasing in both position and value. Strict versus slack is one comparison swapped, the lower bound for the upper, with a number pinned to it: 2, 2, 2 has strict length 1 and non-decreasing length 3. At n = 1000 the naive pairs table consults 499500 pairs whatever

the data holds, while the tails search stays inside 10 comparisons per element, the ceiling the C# suite pins alongside the agreement of the two lengths.

The new lanes rebuild the same witnesses: indices 2, 4, 5, 7 with values 2, 3, 7, 18 on the classic fixture, the dip fixture 1, 3, 5, 2, 4, 6 rerouting through 0, 3, 4, 5 with values 1, 2, 4, 6, and the 16 element fixture landing length 6 through 0, 4, 6, 9, 13,
15. Strict versus slack stays one bound apart everywhere, the

lower bound replaces an equal tail, the upper bound extends past it, and on 2, 2, 2 that is 1 against 3. The Go lane pins the strict witness index of 2, 2, 2 at [2], the last of the three equals, where the other trees assert the length there, and the n^2 pairs table agrees on length in every suite. JavaScript, Java, and Lua roll the binary search by hand so the emitted indices match the other lanes exactly.

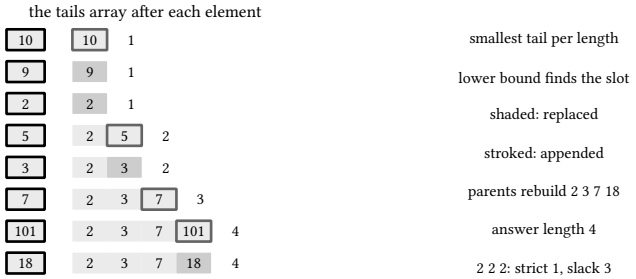


figure 165: longest increasing subsequence on 10 9 2 5 3 7 101 18, the tails after each element, replaced tails shaded, appended tails stroked

19.6. subsets as state, tsp by held-karp

The assignment problem of chapter 17 compressed a set into an integer and paid two to the n times n for it. The traveling salesman is the same mask table with a second coordinate, dp of mask and v is the cheapest path that starts home, visits exactly the cities in the mask, and ends at v, and the tour closes by adding the edge home from every ending city. Brute force over orderings is factorial, held-karp is two to the n times n squared, and the four-city fixture prices the difference in code, six table relaxations against six permutations, but the table keeps winning as cities are added.

The dry run: the fixture is the four-city matrix, 10, 15, 20 against 35, 25, 30, pinned by the C, Go, Java, JavaScript, Python, and Lua suites at tour cost 80, the C# build predating the topic, and the walk reads the C suite's table cells.

1. The seed is mask 0001 at city 0, value 0: home alone.
2. The first hops land three cells: mask 0011 at city 1 reads 10, 0101 at city 2 reads 15, 1001 at city 3 reads 20.
3. Two hops share mask 1011: at city 3 it reads 10 + 25 = 35 by way of 1, at city 1 it reads 20 + 25 = 45 by way of 3.
4. The full mask at city 2 reads 10 + 25 + 30 = 65, the path 0, 1, 3, 2.
5. Closing the loop adds the 15 home: 65 + 15 = 80, and the mirror 0, 2, 3, 1, 0 costs the same.
6. The brute sweep over the six inner orderings agrees at 80, the cross-check C, Go, JavaScript, Python, and Lua all carry.

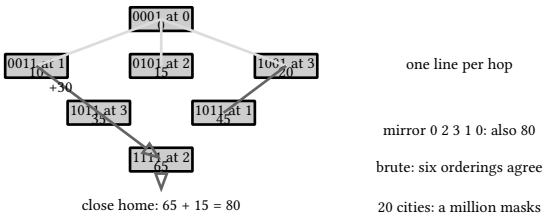


figure 166: the mask table filling by population count, the pinned cells shaded from the seed down to the full mask, the close home at 80

The tour closes at the pinned 80, and the listings below fill this table in six languages plus the C# assignment build it grows from.

```
static int held_karp(void) {
  for (int mask = 0; mask < (1 << V); mask++)
    for (int v = 0; v < V; v++)
      dp[mask][v] = INF;
  dp[1][0] = 0;
  for (int mask = 1; mask < (1 << V); mask++)
    for (int v = 0; v < V; v++) {
      if (!(mask & (1 << v)) || dp[mask][v] == INF)
        continue;
      for (int u = 0; u < V; u++) {
        if (mask & (1 << u))
          continue;
        int nm = mask | (1 << u);
        if (dp[mask][v] + D[v][u] < dp[nm][u])
          dp[nm][u] = dp[mask][v] + D[v][u];
      }
    }
  int full = (1 << V) - 1, best = INF;
  for (int v = 1; v < V; v++)
    if (dp[full][v] + D[v][0] < best)
      best = dp[full][v] + D[v][0];
  return best;
}
```

listing 612: c, the mask table, relax into unvisited cities, close the loop (source: dsa/samples-c/src/Ch19/bitmask.c, lines 26-48)

```
func TSPTour(dist [][]int64) (cost int64, tour []int) {
  n := len(dist)
  if n <= 1 {
    return 0, []int{0}
  }
  const inf = int64(1) << 60
  full := 1<<uint(n) - 1
  dp := make([][]int64, full+1)
  for mask := range dp {
    dp[mask] = make([]int64, n)
    for v := range dp[mask] {
      dp[mask][v] = inf
    }
  }
  dp[1][0] = 0
  for mask := 1; mask <= full; mask++ {
    if mask&1 == 0 {
      continue // every useful path already contains city 0
    }
    for v := 0; v < n; v++ {
      if mask&(1<<uint(v)) == 0 || dp[mask][v] == inf {
        continue
      }
      for w := 0; w < n; w++ {
        if mask&(1<<uint(w)) != 0 {
          continue
        }
        next := mask | 1<<uint(w)
        if candidate := dp[mask][v] + dist[v][w]; candidate < dp[next][w] {
          dp[next][w] = candidate
        }
      }
    }
  }
  best, last := inf, -1
  for v := 1; v < n; v++ {
    if back := dp[full][v] + dist[v][0]; back < best {
      best, last = back, v
    }
  }
```

listing 613: go, the relax loop, then the tour rebuilt by walking the table (source: dsa/samples-go/ch19/bitmask.go, lines 10-49)

```
  // dp[mask][v] = cheapest path from 0 through exactly mask, ending at v
  static int heldKarp() {
    for (int mask = 0; mask < 1 << V; mask++)
      for (int v = 0; v < V; v++)
        dp[mask][v] = INF;
    dp[1][0] = 0;
    for (int mask = 1; mask < 1 << V; mask++)
      for (int v = 0; v < V; v++) {
        if ((mask & (1 << v)) == 0 || dp[mask][v] == INF)
          continue;
        for (int u = 0; u < V; u++) {
          if ((mask & (1 << u)) != 0)
            continue;
          int nm = mask | (1 << u);
          if (dp[mask][v] + D[v][u] < dp[nm][u])
            dp[nm][u] = dp[mask][v] + D[v][u];
        }
      }
    int full = (1 << V) - 1, best = INF;
    for (int v = 1; v < V; v++)
      if (dp[full][v] + D[v][0] < best)
        best = dp[full][v] + D[v][0];
    return best;
  }

  // brute force over the 3! inner permutations
  static int brute() {
    int[] perm = {1, 2, 3};
    int best = INF;
    for (int a = 0; a < 3; a++)
      for (int b = 0; b < 3; b++) {
        if (b == a)
          continue;
        int c = 3 - a - b;
        int cost = D[0][perm[a]] + D[perm[a]][perm[b]] + D[perm[b]][perm[c]]
            + D[perm[c]][0];
        if (cost < best)
          best = cost;
      }
    return best;
  }
```

listing 614: java, the mask table, the permutation sweep beside it, the arbitrary-matrix walk too (source: dsa/samples-java/src/Ch19/Bitmask.java, lines 24-64)

```
    /// <summary>Bitmask dp over subsets: assignment problem, n workers to n tasks.</summary>
    public static long AssignmentCost(long[,] cost)
    {
        int n = cost.GetLength(0);
        int full = (1 << n) - 1;
        var best = new long[1 << n];
        Array.Fill(best, long.MaxValue);
        best[0] = 0;

        for (int mask = 0; mask <= full; mask++)
        {
            if (best[mask] == long.MaxValue)
                continue;
            int worker = System.Numerics.BitOperations.PopCount((uint)mask);
            for (int task = 0; task < n; task++)
            {
                int bit = 1 << task;
                if ((mask & bit) != 0)
                    continue;
                long candidate = best[mask] + cost[worker, task];
                int next = mask | bit;
                if (candidate < best[next])
                    best[next] = candidate;
            }
        }
        return best[full];
    }
}
```

listing 615: c#, the subset dp this chapter builds on, the assignment build (source: dsa/samples/src/Ch17/Dp.cs, lines 170-197)

```
export function tsp(dist) {
  const n = dist.length;
  const FULL = 1 << n;
  const dp = Array.from({ length: FULL }, () => new Array(n).fill(Infinity));
  dp[1][0] = 0; // only city 0 visited, standing at city 0
  for (let mask = 1; mask < FULL; mask++) {
    if (!(mask & 1)) continue; // city 0 starts every route
    for (let v = 0; v < n; v++) {
      if (!(mask & (1 << v)) || dp[mask][v] === Infinity) continue;
      for (let u = 0; u < n; u++) {
        if (mask & (1 << u)) continue;
        const next = mask | (1 << u);
        const cand = dp[mask][v] + dist[v][u];
        if (cand < dp[next][u]) dp[next][u] = cand;
      }
    }
  }
  let best = Infinity;
  for (let v = 1; v < n; v++)
    best = Math.min(best, dp[FULL - 1][v] + dist[v][0]);
  return best;
}
```

listing 616: javascript, infinity for unreachable, one mask per array row (source: dsa/samples-js/src/ch19-bitmask.mjs, lines 5-26)

```
def tsp(dist):
    size = len(dist)
    full = 1 << size
    dp = [[INF] * size for _ in range(full)]
    dp[1][0] = 0  # start at city 0, mask = {0}
    for mask in range(1, full):
        if not mask & 1:
            continue  # every useful mask contains the start
        for v in range(size):
            if dp[mask][v] == INF or not mask & (1 << v):
                continue
            for u in range(size):
                if mask & (1 << u):
                    continue  # u not yet visited: extend into it
                nxt = mask | (1 << u)
                cost = dp[mask][v] + dist[v][u]
                if cost < dp[nxt][u]:
                    dp[nxt][u] = cost
    best = min(dp[full - 1][v] + dist[v][0] for v in range(1, size))
    return best


D = [
    [0, 10, 15, 20],
```

listing 617: python, the same table, every useful mask contains the start (source: dsa/samples-py/src/Ch19/bitmask.py, lines 19-42)

```lua
local function held_karp()
  local n = 4
  local dp = {}
  for mask = 0, (1 << n) - 1 do
    dp[mask] = {}
    for v = 0, n - 1 do dp[mask][v] = math.maxinteger end end
  dp[1][0] = 0 -- mask {0}, at the start city
  for mask = 1, (1 << n) - 1 do
    if mask & 1 == 1 then
      for v = 0, n - 1 do
        if mask & (1 << v) ~= 0 and dp[mask][v] < math.maxinteger then
          for w = 0, n - 1 do
            if mask & (1 << w) == 0 then
              local cand = dp[mask][v] + D[v + 1][w + 1]
              if cand < dp[mask | (1 << w)][w] then
                dp[mask | (1 << w)][w] = cand
              end
            end
          end
        end
      end
    end
  end
  -- close the loop back home
  local full = (1 << n) - 1
  local best, best_end = math.maxinteger, -1
  for v = 1, n - 1 do
    local cand = dp[full][v] + D[v + 1][1]
    if cand < best then
      best = cand
      best_end = v
    end
  end
  return dp, best, best_end
end
```

listing 618: lua, 0-based masks over 1-based tables, maxinteger for infinity (source: dsa/samples-lua/ch19_bitmask.lua, lines 14-48)

The four-city fixture is the classic symmetric matrix, 10, 15, 20 against 35, 25, 30, and the tour cost 80 pins in C, Go, Java, JavaScript, Python, and Lua, with two mirror tours both optimal, 0 to 1 to 3 to 2 and back, 10 plus 25 plus 30 plus 15. Python and Lua accept either mirror in their order checks, Go rebuilds the actual tour by walking the table backwards asking which predecessor still adds up, C and Java brute-force the six inner permutations to confirm the table, and Java carries Python's arbitrary-matrix walk beside it, a two-city pair at 2 and the 3, 4, 5 triangle at 12. The C# suite predates the topic, so its listing is the assignment build the mask idea comes from. At twenty cities the table is a million masks times twenty endpoints and the permutation sweep is twenty factorial, which is the whole pitch.

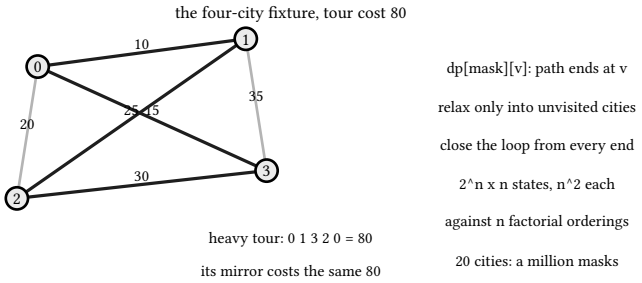


figure 167: held-karp on four cities, the mask table ends at every city, the tour closes from the cheapest

19.7. digit dp, counting under a bound

Counting the numbers up to N with some digit property is a walk over the decimal numeral, not over the numbers. The state is the position, a tight flag saying the prefix still equals N's prefix, a started flag for leading zeros, and whatever the property carries, and once a prefix goes loose every digit is free, which is what collapses a count over N into a table over the digits of N.

The dry run: the fixture is the hand-count staircase under 99, asserted by the C# suite with C, Go, JavaScript, and Lua pinning the same seven-counts, while Python and Java count weakly increasing digits instead, 26 at 30 and 54 at 99.

1. Below 7 nothing qualifies, count 0, and 7 alone makes 1.
2. The units column builds toward 70: 7, 17, through 67, count 7 at N = 69.
3. At 70 the seventies decade joins: 7 + 1 = 8.
4. At 77 the column meets the decade, the 77 shared: 8 + 8 - 1 = 15.
5. At 99 the full column plus the full decade reads 10 + 10 - 1 = 19, which holds at 100.
6. A full thousand misses 9^3 = 729 no-seven strings, so the count lands 1000 - 729 = 271.

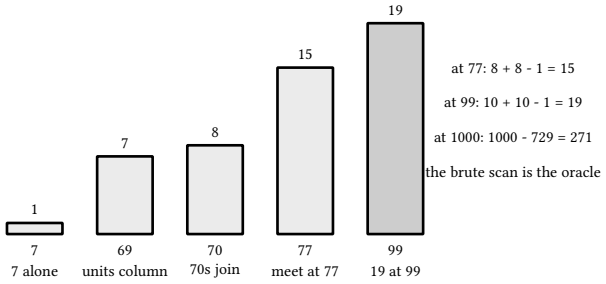


figure 168: the hand-count staircase rising toward 99, the seventies decade joining at 70, the column and decade meeting at 77

The staircase rests at 19 under 99 and 271 at the thousand, and the listings below walk the numeral in seven languages.

```
static long long count_with_seven(unsigned long long N) {
  split(N);
  long long loose[2] = {0, 0};
  long long tight[2] = {1, 0};
  for (int pos = 0; pos < ndigits; pos++) {
    long long nl[2] = {0, 0}, nt[2] = {0, 0};
    for (int seen = 0; seen < 2; seen++) {
      // a loose prefix extends with any digit
      for (int d = 0; d ≤ 9; d++)
        nl[seen || d == 7] += loose[seen];
      // the tight prefix extends up to digits[pos]; matching it stays tight
      for (int d = 0; d ≤ digits[pos]; d++) {
        if (d == digits[pos])
          nt[seen || d == 7] += tight[seen];
        else
          nl[seen || d == 7] += tight[seen];
      }
    }
    loose[0] = nl[0];
    loose[1] = nl[1];
    tight[0] = nt[0];
    tight[1] = nt[1];
  }
  return loose[1] + tight[1];
}
```

listing 619: c, loose and tight rows, the has-7 property, per position (source: dsa/samples-c/src/Ch19/digitdp.c, lines 38-62)

```
func CountWithDigit(n int64, want byte) int64 {
  digits := strconv.FormatInt(n, 10)
  memo := map[[3]int64]int64{}
  var walk func(pos int, tight, started int, state int64) int64
  stateFor := func(prev int64, digit byte) int64 {
    if prev == 1 || digit == want {
      return 1
    }
    return 0
  }
  walk = func(pos int, tight, started int, state int64) int64 {
    if pos == len(digits) {
      if started == 1 && state == 1 {
        return 1
      }
      return 0
    }
    key := [3]int64{int64(pos), int64(tight<<1 | started), state}
    if tight == 0 {
      if cached, ok := memo[key]; ok {
        return cached
      }
    }
    limit := byte('9')
    if tight == 1 {
      limit = digits[pos]
    }
    var total int64
    for d := byte('0'); d <= limit; d++ {
      nextTight := 0
      if tight == 1 && d == limit {
        nextTight = 1
      }
      nextStarted := started
      if d != '0' {
        nextStarted = 1
      }
      nextState := state
      if started == 1 || d != '0' {
        nextState = stateFor(state, d)
      }
      total += walk(pos+1, nextTight, nextStarted, nextState)
    }
    if tight == 0 {
      memo[key] = total
    }
    return total
  }
  return walk(0, 1, 0, 0)
}
```

listing 620: go, the memoized walk keyed by position, flags, state (source: dsa/samples-go/ch19/digitdp.go, lines 12-61)

```
  // count values in [0, N] whose decimal writing contains a 7. loose[]
  // counts prefixes already below N, tight[] the single still-tight prefix
  static long countWithSeven(long n) {
    split(n);
    long[] loose = {0, 0};
    long[] tight = {1, 0};
    for (int pos = 0; pos < ndigits; pos++) {
      long[] nl = {0, 0}, nt = {0, 0};
      for (int seen = 0; seen < 2; seen++) {
        // a loose prefix extends with any digit
        for (int d = 0; d ≤ 9; d++)
          nl[seen == 1 || d == 7 ? 1 : 0] += loose[seen];
        // the tight prefix extends up to digits[pos]; matching it stays tight
        for (int d = 0; d ≤ digits[pos]; d++) {
          if (d == digits[pos])
            nt[seen == 1 || d == 7 ? 1 : 0] += tight[seen];
          else
            nl[seen == 1 || d == 7 ? 1 : 0] += tight[seen];
        }
      }
      loose = nl;
      tight = nt;
    }
    return loose[1] + tight[1];
  }
```

listing 621: java, loose and tight rows over the has-7 property, the py climb lane in the same file (source: dsa/samples-java/src/Ch19/Digitdp.java, lines 36-60)

```
        string s = n.ToString();
        long loose0 = 0, loose1 = 0, tight0 = 1, tight1 = 0;
        foreach (char ch in s)
        {
            int lim = ch - '0';
            long nl0 = 0, nl1 = 0, nt0 = 0, nt1 = 0;
            for (int seen = 0; seen < 2; seen++)
            {
                long loose = seen == 0 ? loose0 : loose1;
                long tight = seen == 0 ? tight0 : tight1;
                for (int d = 0; d ≤ 9; d++)
                {
                    if (seen == 1 || d == 7)
                        nl1 += loose;
                    else
                        nl0 += loose;
                }
                for (int d = 0; d ≤ lim; d++)
                {
                    // matching N's digit stays tight, everything below goes
                    // loose, and the destination row is seen OR this digit
                    bool row = seen == 1 || d == 7;
                    if (d == lim)
                    {
                        if (row)
                            nt1 += tight;
                        else
                            nt0 += tight;
                    }
                    else if (row)
                        nl1 += tight;
                    else
                        nl0 += tight;
                }
            }
            (loose0, loose1, tight0, tight1) = (nl0, nl1, nt0, nt1);
        }
        return loose1 + tight1;
    }
```

listing 622: c#, loose and tight rows, the has-7 property, the destination row is seen or this digit (source: dsa/samples/src/Ch19/DigitDp.cs, lines 13-51)

```javascript
export function countWithDigit(n, d) {
  const s = String(n);
  const memo = new Map();
  const go = (pos, tight, found) => {
    if (pos === s.length) return found ? 1 : 0;
    const key = `${pos},${tight},${found}`;
    if (memo.has(key)) return memo.get(key);
    const limit = tight ? +s[pos] : 9;
    let total = 0;
    for (let dig = 0; dig ≤ limit; dig++)
      total += go(pos + 1, tight && dig === limit, found || dig === d);
    memo.set(key, total);
    return total;
  };
  return go(0, true, false);
}
```

listing 623: javascript, the same walk, the digit a parameter (source: dsa/samples-js/src/ch19-digitdp.mjs, lines 8-24)

```python
def count_nondecreasing(limit):
    # how many k in 1..limit have digits weakly increasing left to right.
    # the walk goes digit by digit: tight while tracking the limit exactly
    if limit < 1:
        return 0
    digits = [int(d) for d in str(limit)]
    memo = {}

    def walk(pos, prev, tight):
        if pos == len(digits):
            return 1  # one full number assembled
        key = (pos, prev, tight)
        if key in memo:
            return memo[key]
        total = 0
        ceiling = digits[pos] if tight else 9
        for d in range(prev, ceiling + 1):  # digits only climb
            total += walk(pos + 1, d, tight and d == ceiling)
        memo[key] = total
        return total

    # leading zeros cover every shorter number; the all-zero walk is the
    # one artifact worth subtracting
    return walk(0, 0, True) - 1
```

listing 624: python, digits only climb, the zero artifact subtracted (source: dsa/samples-py/src/Ch19/digitdp.py, lines 13-38)

```lua
local function count_with_seven(n)
  local ds = digits_of(n)
  local memo = {}
  -- pos, tight, started, has7 -> count
  local function rec(pos, tight, started, has7)
    if pos > #ds then
      return (started and has7) and 1 or 0
    end
    local key = pos .. "|" .. (tight and 1 or 0) .. "|" .. (started and 1 or 0)
      .. "|" .. (has7 and 1 or 0)
    if not tight and memo[key] then return memo[key] end
    local limit = tight and ds[pos] or 9
    local total = 0
    for d = 0, limit do
      local ntight = tight and d == limit
      local nstarted = started or d > 0
      local nhas7 = has7 or d == 7
      total = total + rec(pos + 1, ntight, nstarted, nhas7)
    end
    if not tight then memo[key] = total end
    return total
  end
  return rec(1, true, false, false)
end
```

listing 625: lua, string-keyed memo, the tight prefix never cached (source: dsa/samples-lua/ch19_digitdp.lua, lines 20-43)

The properties split, and both are the real technique: C, Go, Java, C#, JavaScript, and Lua count numbers containing the digit 7, with the anchor 19 numbers up to 99 and the staircase of hand counts below it, 7 alone, 8 at 70, 15 at 77, while Python counts weakly increasing digits and pins 54 up to 99 with the climb from 26 at 30, Java carrying that climb lane beside its has-7 rows. Every suite carries the brute-force oracle, a linear scan over the same range, and the agreement is the test. The caching rule is the subtle part worth reading twice: the tight path is a single chain, so Go and Lua memoize only the loose states, Python's leading zeros cover every shorter number, leaving one all-zero artifact to subtract, and Java's climb memo holds every state keyed on the position, previous digit, and flag packed into one long, harmless because each tight state is visited once anyway.

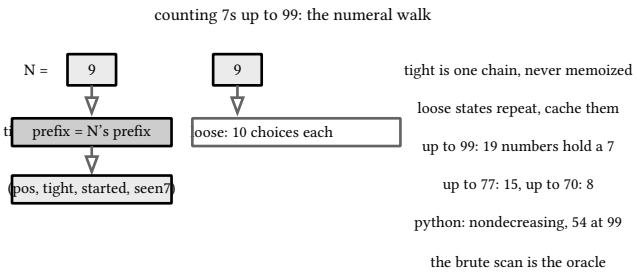


figure 169: digit dp walks the numeral, the tight prefix is one chain, every loose prefix fans out to all digits

19.8. interval dp, palindromes

The last compression makes the interval itself the state. best of i and j is the longest palindromic subsequence of the slice from i to j, the base case is single characters at length one, and an interval is solved from its two immediate shrinkings, ends match and wrap the inside by two, ends differ and drop one end. Because a span reads its strictly shorter spans, filling by increasing length is a valid order and no recursion is needed at all.

The dry run: the fixtures are cbbd at 2 and bbbab at 4 with their interior cells, asserted by the C# suite and pinned the same way in C, Go, Java, JavaScript, Python, and Lua.

1. Span 1 seeds the diagonal: c, b, b, d each score 1.

2. Span 2: cb and bd differ at the ends and keep 1, while bb matches and wraps its empty inside, 0 + 2 = 2.
3. Span 3: cbb reads the max of cb and bb, 1 and 2 = 2, and cbd reads the max of bb and bd, 2 and 1 = 2.
4. Span 4: the ends c and d differ, cbbd reads the max of cbb and bbd, 2 and 2 = 2, the whole-string answer.
5. On bbbab the same order lands 4 at the far corner, with the interior cells bab at 3 and ba at 1.
6. The degenerate family closes it: empty 0, single 1, two equal 2, two different 1.

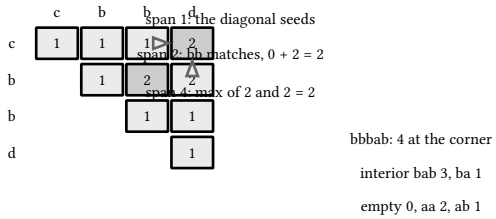


figure 170: the cbbd table as a triangle, span bands rising off the diagonal, the bb cell feeding the corner through both shrinkings

The corner lands at 2 over cbbd and 4 over bbbab, and the listings below fill the table in seven languages.

```c
static int lps(const char *s) {
  size_t len = strlen(s);
  for (size_t i = 0; i < len; i++)
    dp[i][i] = 1;
  for (size_t span = 2; span ≤ len; span++)
    for (size_t i = 0; i + span ≤ len; i++) {
      size_t j = i + span - 1;
      if (s[i] == s[j])
        dp[i][j] = dp[i + 1][j - 1] + 2;
      else
        dp[i][j] = dp[i + 1][j] > dp[i][j - 1] ? dp[i + 1][j] : dp[i][j - 1];
    }
  return len ? dp[0][len - 1] : 0;
}
```

listing 626: c, the table by increasing span, interior cells pinned (source: dsa/samples-c/src/Ch19/intervaldp.c, lines 19-32)

```go
// LongestPalindromicSubsequence returns the length of the longest
// subsequence reading the same both ways. best[i][j] covers the
// substring s[i..j]; intervals grow from length 2 upward, and each
// interval is solved from its two immediate shrinkings.
func LongestPalindromicSubsequence(s string) int {
  n := len(s)
  if n == 0 {
    return 0
  }
  best := make([][]int, n)
  for i := range best {
    best[i] = make([]int, n)
    best[i][i] = 1
  }
  for length := 2; length <= n; length++ {
    for i := 0; i+length-1 < n; i++ {
      j := i + length - 1
      switch {
      case s[i] == s[j]:
        best[i][j] = best[i+1][j-1] + 2 // ends match: wrap them
      default:
        best[i][j] = max(best[i+1][j], best[i][j-1])
      }
    }
  }
  return best[0][n-1]
}
```

listing 627: go, ends match: wrap by two, else the better shrinking (source: dsa/samples-go/ch19/intervaldp.go, lines 3-29)

```
  static int[][] dp = new int[16][16];

  static int lps(String s) {
    int len = s.length();
    for (int i = 0; i < len; i++)
      dp[i][i] = 1;
    for (int span = 2; span ≤ len; span++)
      for (int i = 0; i + span ≤ len; i++) {
        int j = i + span - 1;
        if (s.charAt(i) == s.charAt(j))
          dp[i][j] = dp[i + 1][j - 1] + 2;
        else
          dp[i][j] = Math.max(dp[i + 1][j], dp[i][j - 1]);
      }
    return len > 0 ? dp[0][len - 1] : 0;
  }

  // readback: walk the table from the corners inward, the py lane
  static String lpsSubsequence(String s) {
    StringBuilder left = new StringBuilder();
    StringBuilder right = new StringBuilder();
    int i = 0, j = s.length() - 1;
    while (i < j) {
      if (s.charAt(i) == s.charAt(j)) {
        left.append(s.charAt(i));
        right.append(s.charAt(j));
        i++;
        j--;
      } else if (lps(s.substring(i + 1, j + 1)) ≥ lps(s.substring(i, j))) {
        i++;
      } else {
        j--;
      }
    }
    if (i == j)
      left.append(s.charAt(i));
    return left.append(right.reverse()).toString();
  }
```

listing 628: java, the table by increasing span, then the corner-in readback with its property checks (source: dsa/samples-java/src/Ch19/Intervaldp.java, lines 16-53)

```
    public static int[,] Fill(string s)
    {
        int n = s.Length;
        var dp = new int[n, n];
        for (int i = 0; i < n; i++)
            dp[i, i] = 1;
        for (int span = 2; span ≤ n; span++)
            for (int i = 0; i + span ≤ n; i++)
            {
                int j = i + span - 1;
                // span 2 reads the empty inside, whose default 0 is the
                // second base case: two equal characters score exactly 2
                dp[i, j] = s[i] == s[j]
                    ? dp[i + 1, j - 1] + 2
                    : Math.Max(dp[i + 1, j], dp[i, j - 1]);
            }
        return dp;
    }

    public static int Longest(string s) =>
        s.Length == 0 ? 0 : Fill(s)[0, s.Length - 1];
```

listing 629: c#, the table by increasing span, the zero inside is the second base case (source: dsa/samples/src/Ch19/IntervalDp.cs, lines 12-32)

```
export function lps(s) {
  const n = s.length;
  if (n === 0) return 0;
  const t = Array.from({ length: n }, () => new Array(n).fill(0));
  for (let i = 0; i < n; i++) t[i][i] = 1;
  for (let len = 2; len ≤ n; len++)
    for (let i = 0; i + len ≤ n; i++) {
      const j = i + len - 1;
      if (s[i] === s[j]) t[i][j] = (len === 2 ? 0 : t[i + 1][j - 1]) + 2;
      else t[i][j] = Math.max(t[i + 1][j], t[i][j - 1]);
    }
  return t[0][n - 1];
}
```

listing 630: javascript, the two-base-case trick, length 2 wraps zero (source: dsa/samples-js/src/ch19-intervaldp.mjs, lines 5-18)

```python
def lps_length(s):
    size = len(s)
    best = [[0] * size for _ in range(size)]
    for i in range(size):
        best[i][i] = 1  # every letter alone is a palindrome
    for span in range(2, size + 1):
        for i in range(size - span + 1):
            j = i + span - 1
            if s[i] == s[j]:
                best[i][j] = best[i + 1][j - 1] + 2  # bookends pair up
            else:
                best[i][j] = max(best[i + 1][j], best[i][j - 1])
    return best[0][size - 1] if size else 0


def lps_subsequence(s):
    # readback: walk the table from the corners inward
    i, j = 0, len(s) - 1
    left, right = [], []
    while i < j:
        if s[i] == s[j]:
            left.append(s[i])
            right.append(s[j])
            i += 1
            j -= 1
        elif lps_length(s[i + 1 : j + 1]) >= lps_length(s[i:j]):
            i += 1
        else:
            j -= 1
    if i == j:
        left.append(s[i])
    return "".join(left + right[::-1])
```

listing 631: python, the length, then the readback that rebuilds abdba (source: dsa/samples-py/src/Ch19/intervaldp.py, lines 13-45)

```lua
local function lps(s)
  local n = #s
  if n == 0 then return 0 end
  local best = {}
  for i = 1, n do
    best[i] = {}
    best[i][i] = 1
  end
  for length = 2, n do
    for i = 1, n - length + 1 do
      local j = i + length - 1
      if s:byte(i) == s:byte(j) then
        best[i][j] = (length == 2) and 2 or best[i + 1][j - 1] + 2
      else
        best[i][j] = math.max(best[i + 1][j], best[i][j - 1])
      end
    end
  end
  return best[1][n]
end
```

listing 632: lua, the table plus the brute recursion as oracle (source: dsa/samples-lua/ch19_intervaldp.lua, lines 6-25)

The anchors hold in every suite that carries the topic: `bbbab` gives 4, `agbdba` gives 5 with `abdba` rebuilt in Python and Java, `cbbd` gives 2, and the degenerate family, empty, single, two equal, two different, pins alongside. C, C#, and Java read interior cells out of the table, the `bab` slice of `bbbab` scoring 3 against its own reading, Java property-checks its rebuilt witnesses, a palindrome and a subsequence at the pinned length, and Lua keeps the unmemoized recursion as the oracle the table must agree with.

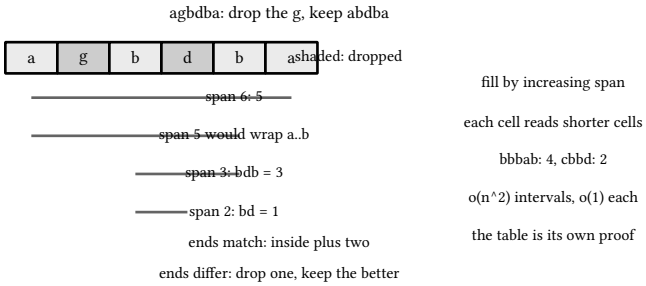


figure 171: interval dp grows by span, ends match and wrap the inside, agbdba rebuilds abdba

19.9. across the seven languages

The build sizes count non-comment source lines over this chapter's 8 featured files per language in the six younger trees, the mask walk, the digit walk, the interval table, bounded, window, grouped, hull, and tails. The C# row is its 3 files, the deepened original carrying the bounded, window, grouped, hull, and tails builds plus the digit and interval walks:

language	build SLOC	dependency	boundary note
c	767	libc only	tsp masks as 1 << v, digit rows carried by hand, per-k consult meters, cross-multiplied overtaken tests
c#	348	bcl only	the deepened original plus the digit and interval walks, lis with parents, no tsp build
go	429	strconv, math	FormatInt splits the numeral, memo keys are arrays, meters as one-field structs, min int over 4 unreachable
java	932	jdk 27 stdlib	every table an exact long, the hull's line and query meters ride the nested Hull class, both digit-dp properties in one file, memo keys packed into a long
javascript	276	node stdlib	infinity sentinels in both knapsack tables, a private-field hull, rows kept by swapping
python	569	stdlib only	digit dp counts a different property, weakly increasing digits, none-flagged unreachable cells, a per-k meter dict
lua	772	lib.lua harness	string-keyed memo tables, maxinteger as infinity, nil sentinels, one-based tables emitting zero-based answers

sources: learn.microsoft.com, `List<T>` behind the pieces and the tails, `Array.Copy` snapshotting the previous item row, `Array.Resize` growing the hull, accessed 2026-09-12. Sample behavior verified by `make verify-csharp`, 19 tests in chapter 19 of the samples suite. The seven-language layer verifies the same way: 8 C programs with 110 embedded checks under `make verify-c`, 8 Ch19 java programs with 133 checks under `run-java-samples`, 28 Go tests, 33 `node --test` cases, 110 Python checks across 8 files, and 42 Lua checks under `run.lua`.

20. pruning and branch and bound

Exponential searches fail for two different reasons, and the fixes are different. Overlapping subproblems call for a memo table, chapter 17. A search space with hopeless branches calls for pruning, and a search with a quality bound calls for branch and bound, where an optimistic estimate of a subtree kills it before it is explored. This chapter puts the three tools side by side on the same fixture, then builds the canonical pruned searches: n-queens, the 0/1 knapsack under a fractional bound, and alpha-beta over a game tree.

20.1. the pruning map, one grid three ways

Count the right-and-down paths across a grid. The naive recursion pays once per path-prefix, the memo table pays once per cell, and a pruned depth-first walk with a visited set pays once per path, which is worse than the memo but survives where the memo cannot, on walks that may revisit cells and would loop forever without the cycle guard.

The dry run: the fixtures are the open 3 by 3 and 4 by 4 grids, asserted by the C# suite, with C carrying the call meter, 19 naive calls against 9 memoized on the 3 by 3.

1. The memo seeds the top row 1, 1, 1, one way into each cell, then fills row by row: row 1 lands 1, 1 + 1 = 2, 2 + 1 = 3, and row 2 lands 1, 1 + 2 = 3, 3 + 3 = 6 at the corner.
2. The open 4 by 4 fills the same way and ends at 20, and with the center blocked the corner drops to 2, both pinned by the suite.
3. Naive recursion pays one call per path prefix, so cell (1,1) is entered twice and the corner six times, summing the 3 by 3 to 1 + 1 + 1 + 1 + 2 + 3 + 1 + 3 + 6 = 19 calls against the memo's 9, one per cell.
4. On the 10 by 10 the same split reads 184755 against exactly 100 calls for the same 48620 paths.
5. The suite adds the walk the memo cannot serve: four-direction simple paths corner to corner count 12 on the open 3 by 3 against the 6 monotone ones.

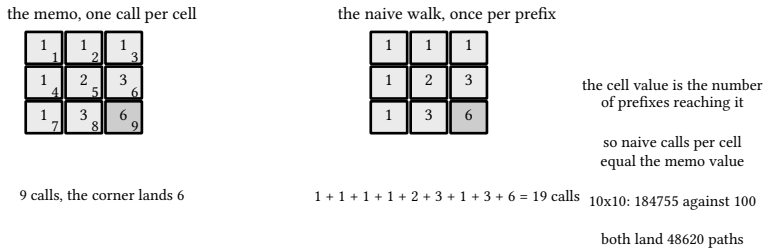


figure 172: the run as two grids of the same nine path counts, the memo computing each cell once, the naive walk paying once per path prefix, 19 calls against 9

The corner lands 6 both ways and the listings below build the memo, the pruned walk, and the simple-path count in seven languages.

```c
static long long naive(int r, int c, int rows, int cols) {
  if (r == rows || c == cols || blocked[r][c])
    return 0;
  calls++;
  if (r == rows - 1 && c == cols - 1)
    return 1;
  return naive(r + 1, c, rows, cols) + naive(r, c + 1, rows, cols);
}

static long long memo_tab[MAXN][MAXN];
static char memo_seen[MAXN][MAXN];

static long long memoized(int r, int c, int rows, int cols) {
  if (r == rows || c == cols || blocked[r][c])
    return 0;
  if (memo_seen[r][c])
    return memo_tab[r][c];
  calls++; // a cache miss: every cell computes exactly once
  if (r == rows - 1 && c == cols - 1) {
    memo_seen[r][c] = 1;
    return memo_tab[r][c] = 1;
  }
  memo_seen[r][c] = 1;
  return memo_tab[r][c] =
             memoized(r + 1, c, rows, cols) + memoized(r, c + 1, rows, cols);
}
```

listing 633: c, naive with a call meter against the memoized walk (source: dsa/samples-c/src/Ch20/memo_vs_prune.c, lines 25-50)

```
}

// GridPathPrune counts the same DAG paths by depth-first search with
// an explicit visited set: the cycle guard a general graph needs, plus
// the wall and boundary pruning. Unmarking on the way out is what
// lets every path reuse the same set.
func GridPathPrune(blocked [][]bool) int {
  rows := len(blocked)
  if rows == 0 {
    return 0
  }
  cols := len(blocked[0])
  visited := make([][]bool, rows)
  for r := range visited {
    visited[r] = make([]bool, cols)
  }
  visited[0][0] = true
  var dfs func(r, c int) int
  dfs = func(r, c int) int {
    if r == rows-1 && c == cols-1 {
      return 1
    }
    total := 0
    for _, d := range [2][2]int{{1, 0}, {0, 1}} { // down, right
      nr, nc := r+d[0], c+d[1]
      if nr ≥ rows || nc ≥ cols || blocked[nr][nc] || visited[nr][nc] {
        continue // pruned: wall, boundary, or already on this path
      }
      visited[nr][nc] = true
      total += dfs(nr, nc)
      visited[nr][nc] = false // backtrack: free the cell
    }
    return total
  }
  return dfs(0, 0)
}
```

listing 634: go, the pruned dfs, unmark on the way out is the whole trick (source: dsa/samples-go/ch20/memo_vs_prune.go, lines 38-73)

```
static final int MAXN = 12;

static int[][] blocked = new int[MAXN][MAXN];
static long calls;

// the call counter tallies real cells only: guards run first, so
// out-of-bounds probes never inflate the count
static long naive(int r, int c, int rows, int cols) {
  if (r == rows || c == cols || blocked[r][c] != 0)
    return 0;
  calls++;
  if (r == rows - 1 && c == cols - 1)
    return 1;
  return naive(r + 1, c, rows, cols) + naive(r, c + 1, rows, cols);
}

static long[][] memoTab = new long[MAXN][MAXN];
static boolean[][] memoSeen = new boolean[MAXN][MAXN];

static long memoized(int r, int c, int rows, int cols) {
  if (r == rows || c == cols || blocked[r][c] != 0)
    return 0;
  if (memoSeen[r][c])
    return memoTab[r][c];
  calls++; // a cache miss: every cell computes exactly once
  if (r == rows - 1 && c == cols - 1) {
    memoSeen[r][c] = true;
    return memoTab[r][c] = 1;
  }
  memoSeen[r][c] = true;
  return memoTab[r][c] =
      memoized(r + 1, c, rows, cols) + memoized(r, c + 1, rows, cols);
}
```

listing 635: java, naive with the meter, the seen-flag memo, guards run before the count (source: dsa/samples-java/src/Ch20/MemoVsPrune.java, lines 20-52)

```
    public static long MemoPaths(string[] grid)
    {
        int rows = grid.Length, cols = grid[0].Length;
        var table = new long[rows, cols];
        table[0, 0] = grid[0][0] == '#' ? 0 : 1;
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
            {
                if (grid[r][c] == '#' || (r == 0 && c == 0))
                    continue;
                long fromUp = r > 0 ? table[r - 1, c] : 0;
                long fromLeft = c > 0 ? table[r, c - 1] : 0;
                table[r, c] = fromUp + fromLeft;
            }
        return table[rows - 1, cols - 1];
    }

    /// <summary>
    /// Same count without a memo: DFS that prunes out-of-bounds and obstacle cells and
    /// abandons a branch the moment it steps past the last row or column. The visited set
    /// is the cycle guard; right/down moves make it vacuous here but SimplePaths below
    /// leans on the same guard for real.
    /// </summary>
    public static long PrunedPaths(string[] grid)
    {
        int rows = grid.Length, cols = grid[0].Length;
        var visited = new HashSet<(int, int)>();
        return Walk(0, 0);

        long Walk(int r, int c)
        {
            if (r ≥ rows || c ≥ cols || grid[r][c] == '#' || !visited.Add((r, c)))
                return 0; // each prune rejects the branch immediately
            long paths = r == rows - 1 && c == cols - 1 ? 1 : Walk(r + 1, c) + Walk(r, c + 1);
            visited.Remove((r, c)); // backtrack
            return paths;
        }
    }
```

listing 636: c#, the memo table, then the same count pruned with a visited set (source: dsa/samples/src/Ch20/Pruning.cs, lines 10-48)

```
export function countMemo(walls) {
  const rows = walls.length;
  const cols = walls[0].length;
  const memo = Array.from({ length: rows }, () => new Array(cols).fill(-1));
  const go = (r, c) => {
    if (r === rows || c === cols || walls[r][c]) return 0;
    if (r === rows - 1 && c === cols - 1) return 1;
    if (memo[r][c] !== -1) return memo[r][c];
    memo[r][c] = go(r + 1, c) + go(r, c + 1);
    return memo[r][c];
  };
  return go(0, 0);
}

export function countPruned(walls) {
  const rows = walls.length;
  const cols = walls[0].length;
  let count = 0;
  const visited = Array.from({ length: rows }, () => new Array(cols).fill(false));
  const dfs = (r, c) => {
    if (r === rows || c === cols || walls[r][c] || visited[r][c]) return;
    visited[r][c] = true; // cycle guard: never revisit a cell
    if (r === rows - 1 && c === cols - 1) count++;
    else {
      dfs(r + 1, c);
      dfs(r, c + 1);
    }
    visited[r][c] = false;
  };
  dfs(0, 0);
```

listing 637: javascript, memo rows of minus one, dfs with the cycle guard (source: dsa/samples-js/src/ch20-memo_vs_prune.mjs, lines 6-35)

```python
def count_memo(rows, cols, blocked):
    table = {}

    def paths(r, c):
        if r == rows or c == cols or (r, c) in blocked:
            return 0
        if r == rows - 1 and c == cols - 1:
            return 1
        if (r, c) not in table:
            table[(r, c)] = paths(r + 1, c) + paths(r, c + 1)
        return table[(r, c)]

    return paths(0, 0)


def count_pruned(rows, cols, blocked):
    visited = set()

    def paths(r, c):
        if r == rows or c == cols or (r, c) in blocked:
            return 0
        if r == rows - 1 and c == cols - 1:
            return 1
        if (r, c) in visited:  # the cycle guard: dead on a monotone walk
            return 0
        visited.add((r, c))
        total = paths(r + 1, c) + paths(r, c + 1)
        visited.discard((r, c))  # backtrack: another lane may reuse the cell
        return total
```

listing 638: python, the memo count and the pruned count as mirror functions (source: dsa/samples-py/src/Ch20/memo_vs_prune.py, lines 15-44)

```
local function memo_paths(rows)
  local rn, cn = #rows, #rows[1]
  local t = {}
  for r = 1, rn do
    t[r] = {}
    for c = 1, cn do
      t[r][c] = 0
      if rows[r]:sub(c, c) ~= "#" then
        if r == 1 and c == 1 then
          t[r][c] = 1
        else
          t[r][c] = (r > 1 and t[r - 1][c] or 0) + (c > 1 and t[r][c - 1] or 0)
        end
      end
    end
  end
  return t[rn][cn]
end

-- no memo: dfs right/down, rejecting out-of-bounds and obstacles on the
-- spot; the visited set is vacuous for right/down walks but the same
-- guard powers simple_paths below
local function pruned_paths(rows)
  local rn, cn = #rows, #rows[1]
  local visited = {}
  local function walk(r, c)
    if r > rn or c > cn or rows[r]:sub(c, c) == "#" or visited[r * cn + c] then
      return 0
    end
    if r == rn and c == cn then return 1 end
    visited[r * cn + c] = true
    local paths = walk(r + 1, c) + walk(r, c + 1)
    visited[r * cn + c] = nil -- backtrack
    return paths
  end
  return walk(1, 1)
end
```

listing 639: lua, the memo table, then the dfs with its guard (source: dsa/samples-lua/ch20_memo_vs_prune.lua, lines 7-44)

The open 3 by 3 grid pins the anchor everywhere: 6 paths. C and Java carry the meter that makes the map a table of numbers, 19 naive calls against 9 memoized on the 3 by 3, and on the 10 by 10 the split is 184755 against exactly 100, one call per cell, for the same 48620 paths, Java adding the walled 3 by 3 to the meter table, 9 naive calls against 8 memoized, the blocked cell returning before the count. With the center blocked every suite still agrees on 2, and the C# suite adds the case the memo cannot serve, four-direction simple paths between two cells, where the visited set is load-bearing because the walk can turn back into itself: 12 simple paths on the open 3 by 3 against the 6 monotone ones.

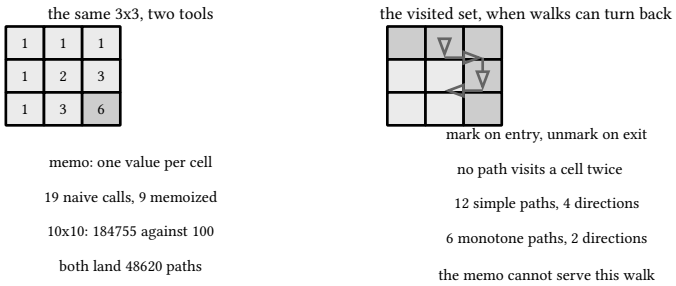


figure 173: the pruning map on one grid, the memo keeps one value per cell, the visited set guards the cycle the memo cannot

20.2. n-queens, pruned by three arrays

Place one queen per row, and before recursing check the column, the diagonal, and the antidiagonal in constant time, row plus column and row minus column are constant along the two diagonal families. The board itself never exists, three flag arrays hold the whole state, and the undo is three assignments.

The dry run: the fixture is n = 4, whose 2 solutions the C# suite pins alongside 10 at n = 5 and 92 at n = 8, and C captures the first placement at columns 1, 3, 0, 2.

1. Row 0 tries column 0 first, its branch dies below, and the first surviving landing is column 1: col 1, diag 0 - 1 = −1, anti 0 + 1 = 1, all fresh.
2. Row 1 rejects column 0 because 1 + 0 = 1 repeats row 0′s anti, column 1 on the column flag, column 2 because 1 - 2 = −1 repeats the diag, and lands column 3: diag −2, anti 4.
3. Row 2 rejects columns 1 and 3 on the column flag and column 2 because 2 + 2 = 4 repeats row 1′s anti, landing column 0: diag 2, anti 2.
4. Row 3 rejects columns 0, 1, and 3 on the column flag and lands column 2: diag 1, anti 5, the captured placement 1, 3, 0, 2.
5. The undo on the way out is three assignments, the scan finishes on the mirror 2, 0, 3, 1, and n = 4 closes at 2.

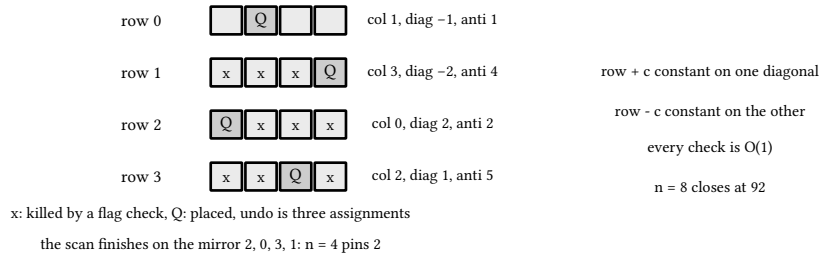


figure 174: the first 4-queens placement row by row, the three flags each queen sets, dead columns marked before every landing

Two at n = 4 is the pinned count and the listings below place, check, and undo in seven languages.

```
static void solve(int row, int size) {
  nodes++;
  if (row == size) {
    if (solutions == 0)
      for (int i = 0; i < size; i++)
        first_sol[i] = sol[i];
    solutions++;
    return;
  }
  for (int c = 0; c < size; c++) {
    int d = row + c, a = row - c + size - 1;
    if (col[c] || diag[d] || anti[a])
      continue;
    col[c] = diag[d] = anti[a] = 1;
    sol[row] = c;
    solve(row + 1, size);
    col[c] = diag[d] = anti[a] = 0;
  }
}

// no pruning: the full permutation tree, for node counting only
static void raw_tree(int row, int size, int used) {
  nodes++;
  if (row == size)
    return;
  for (int c = 0; c < size; c++) {
    if (used & (1 << c))
      continue;
    raw_tree(row + 1, size, used | (1 << c));
  }
}
```

listing 640: c, the three flag arrays, the raw permutation tree for contrast (source: dsa/samples-c/src/Ch20/nqueens.c, lines 24-54)

```
func CountQueens(n int) int {
  col := make([]bool, n)
  diag := make([]bool, 2*n)     // row - col + n
  antidiag := make([]bool, 2*n) // row + col
  solutions := 0
  var place func(row int)
  place = func(row int) {
    if row == n {
      solutions++
      return
    }
    for c := 0; c < n; c++ {
      d, a := row-c+n, row+c
      if col[c] || diag[d] || antidiag[a] {
        continue // pruned: attacked square
      }
      col[c], diag[d], antidiag[a] = true, true, true
      place(row + 1)
      col[c], diag[d], antidiag[a] = false, false, false
    }
  }
  place(0)
  return solutions
}
```

listing 641: go, col, diag, antidiag slices, mark and unmark (source: dsa/samples-go/ch20/nqueens.go, lines 8-31)

```
  static int[] col = new int[8];
  static int[] diag = new int[15];
  static int[] anti = new int[15];
  static int[] sol = new int[8];
  static int[] firstSol = new int[8];
  static long nodes;
  static long solutions;

  static void solve(int row, int size) {
    nodes++;
    if (row == size) {
      if (solutions == 0)
        System.arraycopy(sol, 0, firstSol, 0, size);
      solutions++;
      return;
    }
    for (int c = 0; c < size; c++) {
      int d = row + c, a = row - c + size - 1;
      if (col[c] != 0 || diag[d] != 0 || anti[a] != 0)
        continue;
      col[c] = diag[d] = anti[a] = 1;
      sol[row] = c;
      solve(row + 1, size);
      col[c] = diag[d] = anti[a] = 0;
    }
  }

  // no pruning: the full permutation tree, for node counting only
  static void rawTree(int row, int size, int used) {
    nodes++;
    if (row == size)
      return;
    for (int c = 0; c < size; c++) {
      if ((used & (1 << c)) != 0)
        continue;
      rawTree(row + 1, size, used | (1 << c));
    }
  }
```

listing 642: java, int flag arrays, the first solution captured, the raw permutation tree beside it (source: dsa/samples-java/src/Ch20/Nqueens.java, lines 21-58)

```
    public static int NQueens(int n)
    {
        var cols = new HashSet<int>();
        var diag = new HashSet<int>(); // r - c constant along one diagonal
        var anti = new HashSet<int>(); // r + c constant along the other
        return Place(0);

        int Place(int r)
        {
            if (r == n)
                return 1;
            int solutions = 0;
            for (int c = 0; c < n; c++)
            {
                if (cols.Contains(c) || diag.Contains(r - c) || anti.Contains(r + c))
                    continue;
                cols.Add(c);
                diag.Add(r - c);
                anti.Add(r + c);
                solutions += Place(r + 1);
                cols.Remove(c);
                diag.Remove(r - c);
                anti.Remove(r + c);
            }
            return solutions;
        }
    }
```

listing 643: c#, place row by row, constant-time attack checks (source: dsa/samples/src/Ch20/Pruning.cs, lines 70-96)

```
export function nQueens(n) {
  let count = 0;
  const cols = new Set();
  const diag = new Set(); // r + c constant along one diagonal
  const anti = new Set(); // r - c constant along the other
  const place = (r) => {
    if (r === n) {
      count++;
      return;
    }
    for (let c = 0; c < n; c++) {
      if (cols.has(c) || diag.has(r + c) || anti.has(r - c)) continue;
      cols.add(c);
      diag.add(r + c);
      anti.add(r - c);
      place(r + 1);
      cols.delete(c);
      diag.delete(r + c);
      anti.delete(r - c);
    }
  };
  place(0);
  return count;
}
```

listing 644: javascript, three Sets keyed by column and both diagonals (source: dsa/samples-js/src/ch20-nqueens.mjs, lines 5-28)

```python
def solve_nqueens(size):
    # returns the list of solutions, each a tuple of row -> column
    solutions = []
    cols = set()  # attacked columns
    diags = set()  # attacked r - c diagonals
    antidiags = set()  # attacked r + c diagonals
    board = [0] * size

    def place(row):
        if row == size:
            solutions.append(tuple(board))
            return
        for col in range(size):
            if col in cols or (row - col) in diags or (row + col) in antidiags:
                continue  # pruned: this square is covered
            cols.add(col)
            diags.add(row - col)
            antidiags.add(row + col)
            board[row] = col
            place(row + 1)
            cols.discard(col)  # undo: the backtrack
            diags.discard(row - col)
            antidiags.discard(row + col)

    place(0)
    return solutions
```

listing 645: python, the solver that returns the solutions, validity checked (source: dsa/samples-py/src/Ch20/nqueens.py, lines 13-39)

```lua
local function nqueens(n, meter)
  meter = meter or {placements = 0, prunes = 0}
  local cols, diag, anti = {}, {}, {}
  local function place(r)
    if r > n then return 1 end
    local solutions = 0
    for c = 1, n do
      if cols[c] or diag[r - c] or anti[r + c] then
        meter.prunes = meter.prunes + 1
      else
        cols[c], diag[r - c], anti[r + c] = true, true, true
        meter.placements = meter.placements + 1
        solutions = solutions + place(r + 1)
        cols[c], diag[r - c], anti[r + c] = nil, nil, nil
      end
    end
    return solutions
  end
  return place(1), meter
end
```

listing 646: lua, 0-based diagonals inside, the meter outside (source: dsa/samples-lua/ch20_nqueens.lua, lines 6-25)

The 92 anchor holds in every suite: eight queens have exactly 92 solutions, with the small boards pinned alongside, 1, 0, 0, 2, 10, 4 for n = 1 through 6. Lua reaches past the headline and pins 7 at 40 and 9 at 352, Java pins 7 at 40 too, Python and Java return the actual solutions and check every one peaceful plus duplicate-free, with the lexicographically first pinned, and C and Java keep the first 4-queens placement, columns 1, 3, 0, 2, and measure the prune against the raw permutation tree, 1957 nodes at n = 6 against far fewer pruned.

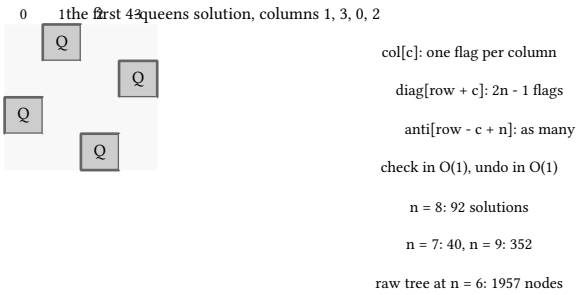


figure 175: n-queens prunes on three arrays, column, diagonal, antidiagonal, all constant time

20.3. branch and bound, the knapsack with a bound

Sort the items by value density, then walk the include-exclude tree with one extra question at every node: what does the fractional knapsack say this subtree could still reach, filling the remaining capacity with slices? If even that optimistic bound cannot beat the best complete solution found so far, the subtree dies unexplored.

The dry run: the fixture is weights 2, 3, 4, 5 against values 3, 4, 5, 6 at capacity 5, asserted by the C# suite at best 7 in exactly 5 nodes, the hand trace.

1. Densities 3/2, 4/3, 5/4, 6/5 leave the identity order 0, 1, 2, 3.
2. The root bound fills the sack exactly, item 0 whole plus item 1 whole, 3 + 4 = 7 with no slice, and 7 beats the incumbent 0.
3. Taking item 0 costs room 5 - 2 = 3 at value 3; the bound there adds item 1 whole, 3 + 4 = 7, above the incumbent 3, so the walk descends again.
4. Taking item 1 too lands room 0 at value 3 + 4 = 7, the new incumbent, and the bound at that node equals 7, so 7 <= 7 stops it.
5. Skipping item 1 holds room 3 at value 3; the bound slices item 2 at 5 x 3 / 4 = 3, landing 3 + 3 = 6, pruned. Skipping item 0 at the root holds room 5 at value 0; its bound takes item 1 whole then slices item 2 at 5 x 2 / 4 = 2, landing 4 + 2 = 6, pruned the same way.

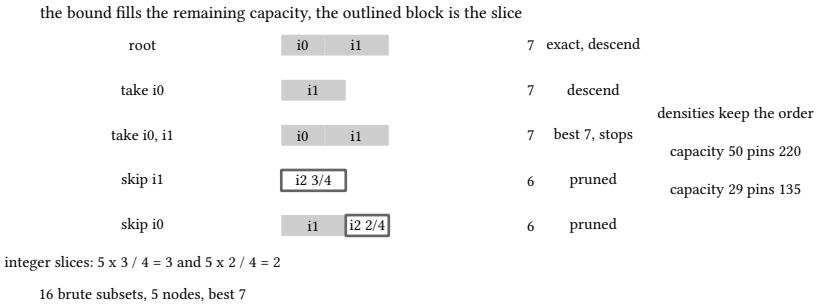


figure 176: the walk as five nodes, each with the fractional fill of its remaining capacity, whole items shaded, the slice outlined

Best 7 in 5 nodes against 16 brute-force subsets, and the listings below sort, bound, and prune in seven languages.

```c
static double upper_bound(int i, int room, int value) {
  if (!use_bound)
    return 1e18; // no bound: explore the whole tree
  double ub = value;
  int r = room;
  for (; i < count_ && r > 0; i++) {
    int take = items[i].w < r ? items[i].w : r;
    ub += (double)items[i].v * take / items[i].w;
    r -= take;
  }
  return ub;
}

static int best;

static void dfs(int i, int room, int value) {
  nodes++;
  if (value > best)
    best = value;
  if (i == count_ || room == 0)
    return;
  if (upper_bound(i, room, value) <= best)
    return; // even the fractional relaxation cannot beat the incumbent
  if (items[i].w <= room)
    dfs(i + 1, room - items[i].w, value + items[i].v);
  dfs(i + 1, room, value);
}

static int branch_and_bound(int k, int capacity, long long *node_count,
                            int bound) {
  count_ = k;
  use_bound = bound;
  best = 0;
  nodes = 0;
  dfs(0, capacity, 0);
  *node_count = nodes;
  return best;
}
```

listing 647: c, the fractional relaxation, the dfs it gates, the dp oracle (source: dsa/samples-c/src/Ch20/bandb.c, lines 38-75)

```go
func BandBKnapsack(capacity int, items []BandBItem) (best int, nodesVisited int64) {
  sorted := make([]BandBItem, len(items))
  copy(sorted, items)
  slices.SortFunc(sorted, func(a, b BandBItem) int {
    da := float64(a.Value) / float64(a.Weight)
    db := float64(b.Value) / float64(b.Weight)
    switch {
    case da > db:
      return -1
    case da < db:
      return 1
    }
    return 0
  })
  var valueOf func(i, room, taken int) int
  valueOf = func(i, room, taken int) int {
    for ; i < len(sorted); i++ {
      if sorted[i].Weight <= room {
        taken += sorted[i].Value
        room -= sorted[i].Weight
      } else {
        // slice the item that does not fit: the relaxation
        taken += sorted[i].Value * room / sorted[i].Weight
        break
      }
    }
    return taken
  }
  var dfs func(i, room, taken int)
  dfs = func(i, room, taken int) {
    nodesVisited++
    if taken > best {
      best = taken
    }
    if i == len(sorted) || room == 0 {
      return
    }
    if valueOf(i, room, taken) <= best {
      return // no completion here can beat the incumbent
    }
    if sorted[i].Weight <= room {
      dfs(i+1, room-sorted[i].Weight, taken+sorted[i].Value)
    }
    dfs(i+1, room, taken) // skip item i
  }
  dfs(0, capacity, 0)
  return best, nodesVisited
}

// EnumerateKnapsackNodeCount is the brute-force contrast: every
// subset of items is one leaf, so the full tree is 2^n nodes.
```

listing 648: go, slice the item that does not fit, nodes counted (source: dsa/samples-go/ch20/bandb.go, lines 17-67)

```
  static void sortByDensity(int k) {
    for (int i = 1; i < k; i++)
      for (int j = i; j > 0 && density[j - 1] < density[j]; j--) {
        int tw = w[j - 1];
        w[j - 1] = w[j];
        w[j] = tw;
        int tv = v[j - 1];
        v[j - 1] = v[j];
        v[j] = tv;
        double td = density[j - 1];
        density[j - 1] = density[j];
        density[j] = td;
      }
  }

  // fractional fill of the remaining capacity: the relaxation every node
  // is measured against
  static double upperBound(int i, int room, int value) {
    if (!useBound)
      return 1e18; // no bound: explore the whole tree
    double ub = value;
    int r = room;
    for (; i < count && r > 0; i++) {
      int take = Math.min(w[i], r);
      ub += (double) v[i] * take / w[i];
      r -= take;
    }
    return ub;
  }

  static void dfs(int i, int room, int value) {
    nodes++;
    if (value > best)
      best = value;
    if (i == count || room == 0)
      return;
    if (upperBound(i, room, value) ≤ best)
      return; // even the fractional relaxation cannot beat the incumbent
    if (w[i] ≤ room)
      dfs(i + 1, room - w[i], value + v[i]);
    dfs(i + 1, room, value);
  }

  record Run(int best, long nodes) {}

  static Run branchAndBound(int k, int capacity, boolean bound) {
    count = k;
    useBound = bound;
    best = 0;
    nodes = 0;
    dfs(0, capacity, 0);
    return new Run(best, nodes);
  }
```

listing 649: java, insertion sort by density, the fractional bound gating the dfs, nodes counted (source: dsa/samples-java/src/Ch20/Bandb.java, lines 26-78)

```
    public static (long Best, long Nodes) Knapsack(int[] weights, long[] values, int capacity)
    {
        int[] order = Enumerable.Range(0, weights.Length)
            .OrderByDescending(i => (double)values[i] / weights[i])
            .ToArray();
        long best = 0, nodes = 0;
        Search(0, capacity, 0);
        return (best, nodes);

        void Search(int k, int room, long value)
        {
            nodes++;
            if (value > best)
                best = value;
            if (k == order.Length)
                return;
            if (Bound(k, room, value) ≤ best)
                return; // even taking fractions of everything left cannot win

            int i = order[k];
            if (weights[i] ≤ room)
                Search(k + 1, room - weights[i], value + values[i]); // take it
            Search(k + 1, room, value); // leave it
        }

        long Bound(int k, int room, long value)
        {
            for (; k < order.Length && room > 0; k++)
            {
                int i = order[k];
                if (weights[i] ≤ room)
                {
                    room -= weights[i];
                    value += values[i];
                }
                else
                {
                    value += values[i] * room / weights[i]; // fraction of the item
                    room = 0;
                }
            }
            return value;
        }
    }
```

listing 650: c#, density sort, fractional bound, incumbent pruning (source: dsa/samples/src/Ch20/Pruning.cs, lines 103-147)

```
export function branchAndBound(weights, values, cap) {
  const n = weights.length;
  const order = [...weights.keys()].sort(
    (a, b) => values[b] / weights[b] - values[a] / weights[a],
  );
  let best = 0;
  let nodes = 0;

  // fractional knapsack value from item i onward, the optimistic ceiling
  const bound = (i, w, v) => {
    let b = v;
    let ww = w;
    for (let k = i; k < n && ww < cap; k++) {
      const idx = order[k];
      const take = Math.min(weights[idx], cap - ww);
      b += (values[idx] / weights[idx]) * take;
      ww += take;
    }
    return b;
  };

  const dfs = (i, w, v) => {
    nodes++;
    if (i === n) {
      best = Math.max(best, v);
      return;
    }
    const idx = order[i];
    if (w + weights[idx] <= cap) dfs(i + 1, w + weights[idx], v + values[idx]);
    if (bound(i + 1, w, v) > best) dfs(i + 1, w, v);
  };
  dfs(0, 0, 0);
  return { best, nodes, bruteLeaves: 2 ** n };
}
```

listing 651: javascript, the bound and the walk, node counts returned (source: dsa/samples-js/src/ch20-bandb.mjs, lines 7-40)

```python
def fractional_bound(level, value, room):
    # fill the remaining room greedily, slicing the last item if needed
    bound = float(value)
    left = room
    for wt, val in ITEMS[level:]:
        if wt ≤ left:
            bound += val
            left -= wt
        else:
            bound += val * left / wt  # the fractional slice
            break
    return bound


def branch_and_bound():
    stats = {"best": 0, "items": [], "nodes": 0}

    def descend(level, value, room, chosen):
        stats["nodes"] += 1
        if level == len(ITEMS):
            if value > stats["best"]:
                stats["best"] = value
                stats["items"] = list(chosen)
            return
        if fractional_bound(level, value, room) ≤ stats["best"]:
            return  # this whole subtree cannot beat the incumbent
        wt, val = ITEMS[level]
        if wt ≤ room:  # the take branch first: incumbents grow fast
            chosen.append(level)
            descend(level + 1, value + val, room - wt, chosen)
            chosen.pop()
        descend(level + 1, value, room, chosen)  # then the skip branch

    descend(0, 0, CAPACITY, [])
    return stats["best"], stats["items"], stats["nodes"]
```

listing 652: python, the fractional bound exact at the root, the descend with choices (source: dsa/samples-py/src/Ch20/bandb.py, lines 18-54)

```lua
local function bandb(weights, values, cap)
  local n = #weights
  -- density order, ties broken by index: fully deterministic
  local order = {}
  for i = 1, n do order[i] = i end
  table.sort(order, function(x, y)
    local dx, dy = values[x] / weights[x], values[y] / weights[y]
    if dx ~= dy then return dx > dy end
    return x < y
  end)
  local best, nodes = 0, 0
  -- the fractional relaxation of items k..n in room, added to value
  local function bound(k, room, value)
    for i = k, n do
      local it = order[i]
      if weights[it] <= room then
        room = room - weights[it]
        value = value + values[it]
      else
        value = value + values[it] * room // weights[it] -- a fraction
        room = 0
      end
      if room == 0 and i < n then break end
    end
    return value
  end
  local function search(k, room, value)
    nodes = nodes + 1
    if value > best then best = value end
    if k > n then return end
    if bound(k, room, value) <= best then return end -- cannot win
    local it = order[k]
    if weights[it] <= room then
      search(k + 1, room - weights[it], value + values[it]) -- take it
    end
    search(k + 1, room, value) -- leave it
  end
  search(1, cap, 0)
  return best, nodes
end
```

listing 653: lua, the same walk, hand-traced node counts pinned (source: dsa/samples-lua/ch20_bandb.lua, lines 8-47)

The optimums pin exactly where the suite traces them by hand. C#, C, and Java hold the classic capacity-50 fixture at 220 with the chapter 17 dynamic program as the cross-check, C's and Java's second fixture, four items at capacity 29, landing 135. Lua hand-traces its trees: weights 2, 3, 4, 5 at capacity 5 answers 7 in exactly 5 nodes, root, two takes, two bound-pruned rejections, and the capacity-50 fixture in 12. JavaScript pins the same shape, optimum 7 in 5 nodes against 16 brute-force leaves, and Python and Java pin 90 in 6 nodes with the bound arithmetic exposed, the root bound exact at 90.0 and the one below it at 88.75. Java also runs its own raw twin with the bound switched off, asserting the pruned tree never costs a node against it. Node counts beyond the hand-traced ones are asserted by inequality only, pruned below raw and raw below the full tree, because capacity trimming makes hand-counts brittle, and every suite keeps the brute-force enumeration as the oracle. When the tree stops finishing inside the budget, book 9, ch 42 (practical data structures and algorithms, approximation algorithms) is the planned exit: a proven ratio on the same instance, bought by giving up the certificate.

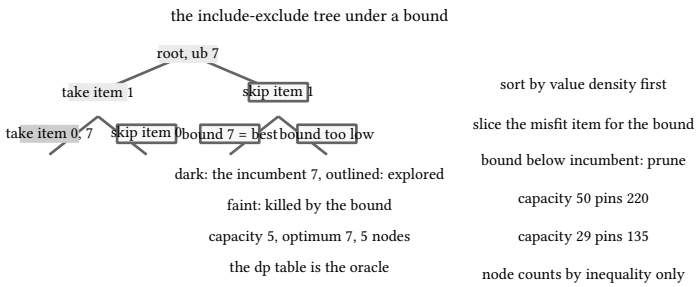


figure 177: branch and bound on the knapsack, the fractional bound kills subtrees the incumbent already beats

20.4. alpha-beta, the game tree pruned by a window

Minimax assumes the opponent plays optimally and takes the value of the game. Alpha-beta computes the identical value while refusing to read leaves that cannot change it: the maximizer at the root carries alpha, the best value it already owns, and the moment a minimizer's running minimum drops to alpha or below, its remaining leaves are dead, the maximizer would never walk into that child.

The dry run: the fixture is the nine leaves 3, 12, 8, 2, 4, 6, 14, 5, 2 at arity 3, asserted by the C# suite at root value 3 with 9 plain leaf reads against 7 under the window.

1. Group 1 reads all three leaves: the running min steps 3, 12, 8 and lands 3, the root maxes to 3, and alpha rises to 3.
2. Group 2 reads one leaf: the 2 drops its running min to 2, beta falls to 2, and beta <= alpha closes the window with leaves 4 and 6 unread.
3. Group 3 reads all three, 14, 5, 2, min 2, and the root keeps 3.
4. Plain minimax pays 3 + 3 + 3 = 9 leaf reads, the window pays 3 + 1 + 3 = 7, at the identical value 3.
5. The depth-four fixture runs the same story bigger: 27 leaves at arity 3 read 27 plain against 21 under the window, 9 + 3 + 9 hand traced, value 7.

leaf	group min now	alpha	beta	what happens
3	3	-inf	+inf	group 1 reads on
12	3	-inf	+inf	group 1 reads on
8	3	3	+inf	group 1 done, root max 3
2	2	3	2	beta <= alpha, 4 and 6 die
14	14	3	+inf	group 3 reads on
5	5	3	+inf	group 3 reads on
2	2	3	+inf	group 3 done, root keeps 3

Seven of nine leaves for the same value 3, and the listings below run both walks with the leaf meter.

```c
static int mm_min(int child) {
  int v = leaves[child][0];
  leaf_evals++;
  for (int l = 1; l < 3; l++) {
    leaf_evals++;
    if (leaves[child][l] < v)
      v = leaves[child][l];
  }
  return v;
}

static int mm_root(void) {
  leaf_evals = 0;
  int best = mm_min(0);
  for (int child = 1; child < 3; child++) {
    int v = mm_min(child);
    if (v > best)
      best = v;
  }
  return best;
}

// alpha-beta: the min side stops scanning once it cannot beat alpha,
// because the max parent would never choose this child
static int ab_min(int child, int alpha) {
  int v = leaves[child][0];
  leaf_evals++;
  for (int l = 1; l < 3 && v > alpha; l++) {
    leaf_evals++;
    if (leaves[child][l] < v)
      v = leaves[child][l];
  }
  return v;
}

static int ab_root(void) {
  leaf_evals = 0;
  int alpha = -1000;
  for (int child = 0; child < 3; child++) {
    int v = ab_min(child, alpha);
    if (v > alpha)
      alpha = v;
  }
  return alpha;
}
```

listing 654: c, plain minimax, then the min side stopping at alpha (source: dsa/samples-c/src/Ch20/alphabeta.c, lines 27-71)

```
      }
    }
    if m > value {
      value = m
    }
  }
  return value, leavesVisited
}

// AlphaBeta carries the root's best guaranteed value alpha into each
// min node. The moment a min node's running minimum drops to alpha,
// max will never enter it, so its remaining leaves go unread. The
// root value is identical to full minimax; only the work shrinks.
func (t AlphaBetaTree) AlphaBeta() (value int, leavesVisited int) {
  value, alpha := abNegInf, abNegInf
  for _, row := range t.Leaves {
    m := abPosInf
    for _, v := range row {
      leavesVisited++
      if v < m {
        m = v
      }
      if m <= alpha {
        break // fail-low: this branch is dead to max
      }
    }
    if m > alpha {
      alpha = m
    }
    if alpha > value {
      value = alpha
    }
  }
  return value, leavesVisited
}
```

listing 655: go, fail-low breaks the row scan, the value unchanged (source: dsa/samples-go/ch20/alphabeta.go, lines 26-61)

```
  static int[][] leaves = new int[3][3];
  static long leafEvals;

  static void setLeaves(int[][] src) {
    for (int i = 0; i < 3; i++)
      for (int j = 0; j < 3; j++)
        leaves[i][j] = src[i][j];
  }

  // plain minimax: every child fully evaluated
  static int mmMin(int child) {
    int v = leaves[child][0];
    leafEvals++;
    for (int l = 1; l < 3; l++) {
      leafEvals++;
      if (leaves[child][l] < v)
        v = leaves[child][l];
    }
    return v;
  }

  static int mmRoot() {
    leafEvals = 0;
    int best = mmMin(0);
    for (int child = 1; child < 3; child++) {
      int v = mmMin(child);
      if (v > best)
        best = v;
    }
    return best;
  }

  // alpha-beta: the min side stops scanning once it cannot beat alpha,
  // because the max parent would never choose this child
  static int abMin(int child, int alpha) {
    int v = leaves[child][0];
    leafEvals++;
    for (int l = 1; l < 3 && v > alpha; l++) {
      leafEvals++;
      if (leaves[child][l] < v)
        v = leaves[child][l];
    }
    return v;
  }

  static int abRoot() {
    leafEvals = 0;
    int alpha = -1000;
    for (int child = 0; child < 3; child++) {
      int v = abMin(child, alpha);
      if (v > alpha)
        alpha = v;
    }
    return alpha;
  }
```

listing 656: java, plain minimax, then the min side stopping at alpha, every leaf metered (source: dsa/samples-java/src/Ch20/Alphabeta.java, lines 19-73)

```
    /// <summary>Same value as minimax, alpha-beta prunes subtrees the window already settles.</summary>
    public static (long Value, long LeafVisits) AlphaBeta(long[] leaves, int arity)
    {
        int depth = TreeDepth(leaves.Length, arity);
        return Walk(0, depth - 1, long.MinValue, long.MaxValue, maximizing: true);

        (long, long) Walk(int index, int level, long alpha, long beta, bool maximizing)
        {
            if (level == 0)
                return (leaves[index], 1);
            long best = maximizing ? long.MinValue : long.MaxValue;
            long visits = 0;
            for (int k = 0; k < arity; k++)
            {
                (long v, long seen) = Walk(index * arity + k, level - 1, alpha, beta, !maximizing);
                visits += seen;
                if (maximizing)
                {
                    best = Math.Max(best, v);
                    alpha = Math.Max(alpha, best);
                }
                else
                {
                    best = Math.Min(best, v);
                    beta = Math.Min(beta, best);
                }
                if (beta ≤ alpha)
                    break; // the window closed, nothing deeper can change the outcome
            }
            return (best, visits);
        }
    }

    private static int TreeDepth(int leaves, int arity)
    {
```

listing 657: c#, the window carried down, leaf visits counted (source: dsa/samples/src/Ch20/Pruning.cs, lines 194-228)

```javascript
export function minimax(node, isMax, meter) {
  if (meter) meter.visits++;
  if (typeof node === "number") return node;
  const vals = node.children.map((c) => minimax(c, !isMax, meter));
  return isMax ? Math.max(...vals) : Math.min(...vals);
}

export function alphaBeta(node, isMax, alpha, beta, meter) {
  if (meter) meter.visits++;
  if (typeof node === "number") return node;
  let v = isMax ? -Infinity : Infinity;
  for (const child of node.children) {
    const got = alphaBeta(child, !isMax, alpha, beta, meter);
    if (isMax) {
      v = Math.max(v, got);
      alpha = Math.max(alpha, v);
    } else {
      v = Math.min(v, got);
      beta = Math.min(beta, v);
    }
    if (beta ≤ alpha) break; // the remaining children cannot matter
  }
  return v;
}
```

listing 658: javascript, minimax and alpha-beta over one node tree (source: dsa/samples-js/src/ch20-alphabeta.mjs, lines 5-28)

```python
def minimax(node, depth, stats):
    if isinstance(node, int):
        stats["leaves"] += 1
        return node
    values = [minimax(child, depth + 1, stats) for child in node]
    return max(values) if depth % 2 == 0 else min(values)


def alphabeta(node, depth, alpha, beta, stats):
    if isinstance(node, int):
        stats["leaves"] += 1
        return node
    maximizing = depth % 2 == 0
    value = float("-inf") if maximizing else float("inf")
    for child in node:
        result = alphabeta(child, depth + 1, alpha, beta, stats)
        if maximizing:
            value = max(value, result)
            alpha = max(alpha, value)
        else:
            value = min(value, result)
            beta = min(beta, value)
        if beta ≤ alpha:
            break  # the winner of this node is already decided
    return value


plain = {"leaves": 0}
root_value = minimax(TREE, 0, plain)
```

listing 659: python, both walks with a stats dict counting leaves (source: dsa/samples-py/src/Ch20/alphabeta.py, lines 17-45)

```
local function minimax(leaves, arity)
  local function walk(index, level, maximizing)
    if level == 0 then return leaves[index], 1 end
    local best = maximizing and -math.maxinteger or math.maxinteger
    local visits = 0
    for k = 0, arity - 1 do
      local v, seen = walk((index - 1) * arity + k + 1, level - 1, not maximizing)
      visits = visits + seen
      if maximizing then
        if v > best then best = v end
      else
        if v < best then best = v end
      end
    end
    return best, visits
  end
  local depth = tree_depth(#leaves, arity)
  return walk(1, depth - 1, true)
end

local function alphabeta(leaves, arity)
  local function walk(index, level, alpha, beta, maximizing)
    if level == 0 then return leaves[index], 1 end
    local best = maximizing and -math.maxinteger or math.maxinteger
    local visits = 0
    for k = 0, arity - 1 do
      local child = (index - 1) * arity + k + 1
      local v, seen = walk(child, level - 1, alpha, beta, not maximizing)
      visits = visits + seen
      if maximizing then
        if v > best then best = v end
        if best > alpha then alpha = best end
      else
        if v < best then best = v end
        if best < beta then beta = best end
      end
      if beta ≤ alpha then break end -- the window closed
    end
    return best, visits
  end
  local depth = tree_depth(#leaves, arity)
  return walk(1, depth - 1, -math.maxinteger, math.maxinteger, true)
end
```

listing 660: lua, the recursive window, beta at or below alpha breaks (source: dsa/samples-lua/ch20_alphabeta.lua, lines 19-61)

The pinned tree is three groups of three leaves, 3, 12, 8, 2, 4, 6, 14, 5, 2, with root value 3, and the meter is the lesson: plain minimax reads all 9 leaves, alpha-beta reads 7, the second group dies after its first leaf because 2 already fails low against the root's alpha of 3. Java adds two orderings beside the pinned tree, a rising one where the window prunes nothing at all and a tied one where five leaf reads suffice, the child-order sensitivity stated as numbers. The deeper fixture agrees everywhere it pins: 27 leaves at arity 3 read 27 plain against 21 with the window, C# and Lua hand-trace it as 9 plus 3 plus 9, and every suite asserts the two algorithms return the same value whatever the tree.

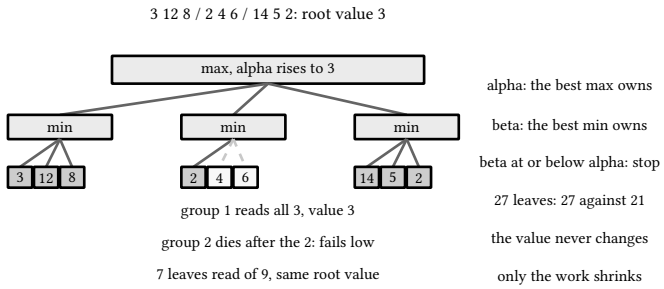


figure 178: alpha-beta on the pinned tree, the second group dies after one leaf, the value is still 3

20.5. across the seven languages

The build sizes count non-comment source lines over this chapter's four featured files per language, checks included where they share the file:

language	build SLOC	dependency	boundary note
c	325	libc only	flag arrays sized by constants, the raw permutation tree kept as the contrast, node counts by inequality
c#	199	bcl only	HashSet for the visited set, the simple-paths case the memo cannot serve, 27 and 21 hand traced
go	198	slices	slices.SortFunc orders by density, closures carry the dfs, the brute enumerates 2^n leaves
java	585	jdk 27 stdlib	the raw tree beside every prune, first solutions captured, the set-based queens solver with a local Walker class, two extra alpha-beta orderings
javascript	109	node stdlib	three Sets for the queens flags, minus-one memo sentinel, the node tree walked recursively
python	198	stdlib only	the solver returns solutions checked peaceful and duplicate-free, set for the visited guard
lua	276	lib.lua harness	maxinteger infinities, hand-traced node counts 5 and 12, 40 and 352 beyond the 92 anchor

sources: learn.microsoft.com, `HashSet<T>` for the visited set, `Array.Sort` comparison overloads, accessed 2026-09-08, plus the CLRS branch-and-bound and the Knuth alpha-beta treatment cited in the chapter text. Sample behavior verified by `make verify-csharp`, 9 tests in chapter 20 of the samples suite. The seven-language layer verifies the same way: 4 C programs with 51 embedded checks under `make verify-c`, 4 Ch20 java programs with 81 checks under `run-java-samples`, 9 Go tests, 8 `node --test` cases, 33 Python checks across 4 files, and 14 Lua checks under `run.lua`.

21. range queries

Range queries ask about slices of an array: the sum of a span, the minimum of a moving window, the nearest larger neighbor of every element. Five tools cover the practical space, and each ships with a meter here, reads per query, nodes touched, pushes and pops, so the logarithmic claims become numbers the tests can check. Prefix sums answer a static array with one subtraction, the fenwick tree handles point updates against prefix sums, the segment tree when the query or the update hits whole ranges, the sparse table when nothing changes at all, and two monotonic scans when the window moves one step at a time. The four matrix structures and the two monotonic scans are built seven languages deep.

21.1. prefix sums

The zeroth tool: never recompute a running total. One pass turns the array into its prefix sums, P of i plus one holds the sum of the first i elements, and every range sum becomes two lookups and a subtraction. The 2d version inverts inclusion-exclusion, each cell of the summed-area table adds the cell above and to the left and subtracts the overlap, and a rectangle query takes four corners.

The dry run: the fixtures are 1, 2, 3, 4, 5 for the 1d pass and the grid 1 through 9 by rows for the table, asserted by the C#, C, Java, and Python suites.

1. The 1d pass seeds P[0] = 0 and folds one element in per step: 0 + 1 = 1, 1 + 2 = 3, 3 + 3 = 6, 6 + 4 = 10, 10 + 5 = 15.
2. Every range is two lookups: the whole span reads 15 - 0 = 15, the interior elements 2, 3, 4 read 10 - 1 = 9.
3. The 2d build starts from a zero border, so the top grid row 1, 2, 3 folds cell plus above plus left minus overlap: 1 + 0 + 0 - 0 = 1, 2 + 0 + 1 - 0 = 3, 3 + 0 + 3 - 0 = 6.
4. Grid row 4, 5, 6 lands 4 + 0 + 1 - 0 = 5, 5 + 3 + 5 - 1 = 12, 6 + 6 + 12 - 3 = 21.
5. Grid row 7, 8, 9 finishes at the far corner: 7 + 0 + 5 - 0 = 12, 8 + 12 + 12 - 5 = 27, 9 + 21 + 27 - 12 = 45.
6. The bottom-right block then costs four corners off the finished table: 45 - 6 - 12 + 1 = 28.

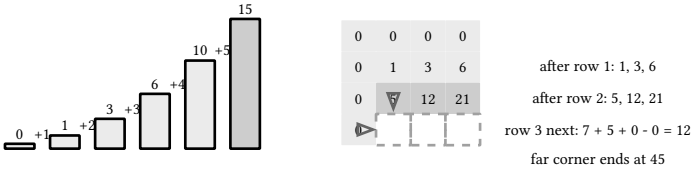


figure 179: the build as a sequence, the totals rising one element at a time, the table landing one row at a time

The 45 and the 28 are asserted by all four suites, and the listings below build these two tables in seven languages.

```c
static void build_1d(void) {
  P[0] = 0;
  for (int i = 0; i < 5; i++)
    P[i + 1] = P[i] + A[i];
}

// inclusive range sum over [l, r]
static int sum1(int l, int r) { return P[r + 1] - P[l]; }

static void build_2d(void) {
  for (int r = 0; r ≤ 3; r++)
    Q[r][0] = 0;
  for (int c = 0; c ≤ 3; c++)
    Q[0][c] = 0;
  for (int r = 0; r < 3; r++)
    for (int c = 0; c < 3; c++)
      Q[r + 1][c + 1] = G[r][c] + Q[r][c + 1] + Q[r + 1][c] - Q[r][c];
}

// inclusive rectangle sum over rows [r1, r2], cols [c1, c2]
static int sum2(int r1, int c1, int r2, int c2) {
  return Q[r2 + 1][c2 + 1] - Q[r1][c2 + 1] - Q[r2 + 1][c1] + Q[r1][c1];
}

int main(void) {
  build_1d();
```

listing 661: c, the 1d pass, then the 2d table and its four-corner query (source: dsa/samples-c/src/Ch21/prefix.c, lines 22-47)

```go
func PrefixSums(a []int64) []int64 {
  p := make([]int64, len(a)+1)
  for i, v := range a {
    p[i+1] = p[i] + v
  }
  return p
}

// RangeSum answers sum(a[lo..hi)) from the prefix table.
func RangeSum(p []int64, lo, hi int) int64 {
  return p[hi] - p[lo]
}

// PrefixSums2D returns ps where ps[r][c] is the sum of the rectangle
// rows [0,r) x cols [0,c): inclusion-exclusion over the three
// overlapping corners.
func PrefixSums2D(grid [][]int64) [][]int64 {
  rows := len(grid) + 1
  cols := 0
  if len(grid) > 0 {
    cols = len(grid[0]) + 1
  }
  ps := make([][]int64, rows)
  for r := range ps {
    ps[r] = make([]int64, cols)
  }
  for r := 1; r < rows; r++ {
    for c := 1; c < cols; c++ {
      ps[r][c] = grid[r-1][c-1] + ps[r-1][c] + ps[r][c-1] - ps[r-1][c-1]
    }
  }
  return ps
}

// RectSum answers the sum of rows [r0,r1) x cols [c0,c1) from the 2D
// prefix table.
func RectSum(ps [][]int64, r0, c0, r1, c1 int) int64 {
  return ps[r1][c1] - ps[r1][c0] - ps[r0][c1] + ps[r0][c0]
}
```

listing 662: go, both builds, the rectangle read from four corners (source: dsa/samples-go/ch21/prefix.go, lines 8-47)

```
static final int[] A = {1, 2, 3, 4, 5};
static int[] P = new int[6];

static final int[][] G = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
static int[][] Q = new int[4][4];

static void build1d() {
  P[0] = 0;
  for (int i = 0; i < 5; i++)
    P[i + 1] = P[i] + A[i];
}

// inclusive range sum over [l, r]
static int sum1(int l, int r) {
  return P[r + 1] - P[l];
}

static void build2d() {
  for (int r = 0; r ≤ 3; r++)
    Q[r][0] = 0;
  for (int c = 0; c ≤ 3; c++)
    Q[0][c] = 0;
  for (int r = 0; r < 3; r++)
    for (int c = 0; c < 3; c++)
      Q[r + 1][c + 1] = G[r][c] + Q[r][c + 1] + Q[r + 1][c] - Q[r][c];
}

// inclusive rectangle sum over rows [r1, r2], cols [c1, c2]
static int sum2(int r1, int c1, int r2, int c2) {
  return Q[r2 + 1][c2 + 1] - Q[r1][c2 + 1] - Q[r2 + 1][c1] + Q[r1][c1];
}
```

listing 663: java, the 1d pass and range subtraction, the 2d corners (source: dsa/samples-java/src/Ch21/Prefix.java, lines 15-45)

```
    {
        var p = new int[a.Length + 1];
        for (int i = 0; i < a.Length; i++)
            p[i + 1] = p[i] + a[i];
        return p;
    }

    /// <summary>Inclusive range sum over [l, r].</summary>
    public static int Sum1(int[] p, int l, int r) => p[r + 1] - p[l];

    public static int[,] Build2d(int[,] g)
    {
        int rows = g.GetLength(0), cols = g.GetLength(1);
        var q = new int[rows + 1, cols + 1];
        for (int r = 0; r < rows; r++)
            for (int c = 0; c < cols; c++)
                q[r + 1, c + 1] = g[r, c] + q[r, c + 1] + q[r + 1, c] - q[r, c];
        return q;
    }

    /// <summary>
    /// Inclusive rectangle over rows [r1, r2] and columns [c1, c2]: add the
    /// far corner, subtract the two near ones, add the overlap back once.
    /// </summary>
    public static int Sum2(int[,] q, int r1, int c1, int r2, int c2) =>
        q[r2 + 1, c2 + 1] - q[r1, c2 + 1] - q[r2 + 1, c1] + q[r1, c1];
```

listing 664: c#, both builds, the rectangle read from four corners (source: dsa/samples/src/Ch21/Prefix.cs, lines 12-37)

```
export function prefixSums(a) {
  const p = new Array(a.length + 1).fill(0);
  for (let i = 0; i < a.length; i++) p[i + 1] = p[i] + a[i];
  return {
    prefix: p,
    rangeSum: (l, r) => p[r + 1] - p[l],
  };
}

// p[r+1][c+1] = sum of grid[0..r][0..c]; region sums inclusive on all
// four corners
export function prefixSums2d(grid) {
  const rows = grid.length;
  const cols = grid[0].length;
  const p = Array.from({ length: rows + 1 }, () => new Array(cols + 1).fill(0));
  for (let r = 0; r < rows; r++)
    for (let c = 0; c < cols; c++)
      p[r + 1][c + 1] =
        p[r][c + 1] + p[r + 1][c] - p[r][c] + grid[r][c];
  return {
    prefix: p,
    regionSum: (r1, c1, r2, c2) =>
      p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1],
  };
}
```

listing 665: javascript, the running total and the summed-area grid (source: dsa/samples-js/src/ch21-prefix.mjs, lines 5-29)

```python
def prefix_1d(a):
    table = [0]
    for x in a:
        table.append(table[-1] + x)
    return table


def range_sum(table, lo, hi):  # inclusive bounds
    return table[hi + 1] - table[lo]


def prefix_2d(grid):
    rows = len(grid) + 1
    cols = len(grid[0]) + 1 if grid else 1
    table = [[0] * cols for _ in range(rows)]
    for r in range(len(grid)):
        for c in range(len(grid[0])):
            table[r + 1][c + 1] = (
                grid[r][c]
                + table[r][c + 1]
                + table[r + 1][c]
                - table[r][c]  # the corner was added twice
            )
    return table


def rect_sum(table, r1, c1, r2, c2):  # inclusive corners
    bottom_right = table[r2 + 1][c2 + 1]
    return bottom_right - table[r1][c2 + 1] - table[r2 + 1][c1] + table[r1][c1]
```

listing 666: python, prefix 1d, range sum, the 2d corners (source: dsa/samples-py/src/Ch21/prefix.py, lines 13-43)

```lua
local function prefix1d(a)
  local p = {0}
  for i = 1, #a do p[i + 1] = p[i] + a[i] end
  return p
end

-- sum of a[lo..hi] inclusive, 0-based
local function range1d(p, lo, hi)
  return p[hi + 2] - p[lo + 1]
end

local GRID = {
  {1, 2, 3, 4},
  {5, 6, 7, 8},
  {9, 10, 11, 12},
}

local function prefix2d(g)
  local rn, cn = #g, #g[1]
  local p = {}
  for r = 0, rn do
    p[r] = {}
    for c = 0, cn do p[r][c] = 0 end
  end
  for r = 1, rn do
    for c = 1, cn do
      p[r][c] = g[r][c] + p[r - 1][c] + p[r][c - 1] - p[r - 1][c - 1]
    end
  end
  return p
end

-- sum of the rectangle rows r1..r2, cols c1..c2, all inclusive 0-based
local function rect(p, r1, c1, r2, c2)
  return p[r2 + 1][c2 + 1] - p[r1][c2 + 1] - p[r2 + 1][c1] + p[r1][c1]
end
```

listing 667: lua, the same two builds, corners by four table reads (source: dsa/samples-lua/ch21_prefix.lua, lines 7-43)

The 1d anchors hold wherever the topic lands: the table 0, 1, 3, 6, 10, 15 over 1 through 5, the whole-range 15, the interior 9, and a single element reading its own slot. The fixture family from the fenwick section below, 3, 1, 4, 1, 5, 9, 2, 6, does double duty in Python and Lua, range 0 to 4 at 14, range 2 to 5 at 19, the whole array at 31. The 2d grid 1 through 9 by rows pins the rectangle arithmetic, 45 for the whole grid, 28 for the bottom-right block, and the four-corner subtraction is the entire trick, add the far corner, subtract the two near ones, add back the doubly-subtracted overlap.

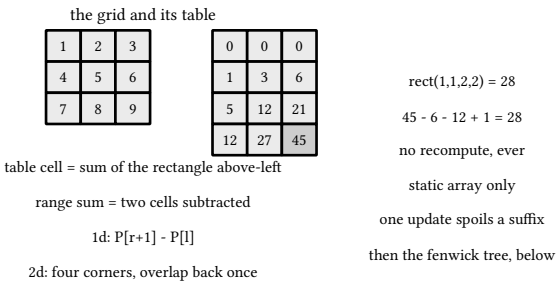


figure 180: the summed-area table, every rectangle is four corners, the overlap added back once

21.2. the fenwick tree

A fenwick tree stores partial sums in the array itself. Internal node j is responsible for the run of cells ending at j, from j - lowbit(j) + 1 through j in 1-based terms, where lowbit is the lowest set bit. A prefix query starts at the endpoint and strips its own lowbit each step, a point update climbs by adding it, and both directions touch one cell per set bit along the way.

The dry run: the fixture is 3, 1, 4, 1, 5, 9, 2, 6, pinned by the C#, JavaScript, Python, and Lua suites, the walk uses the C# labels with 0-based public indices over the 1-based table, so Prefix(4) sums the first five elements, and the build is Go's O(n) pass.

1. Slot 0 stays unused and slots 1 through 8 start as the array itself: 3, 1, 4, 1, 5, 9, 2, 6.
2. Each j pushes its sum up to the parent p = j + lowbit(j): slot 2 lands 1 + 3 = 4, slot 4 takes two pushes to 1 + 4 = 5 then 5 + 4 = 9, and slot 6 lands 9 + 5 = 14.
3. Slot 8 collects three pushes, 6 + 9 = 15, 15 + 14 = 29, 29 + 2 = 31, while j = 8 itself climbs to 16, past the tree.
4. The finished table reads 3, 4, 4, 9, 5, 14, 2, 31: slot 4 owns the first four cells at 9, slot 6 the fifth and sixth at 14, slot 8 all eight at 31.
5. A prefix query strips lowbit: Prefix(4), the first five elements, runs at 1-based j = 5, reads slots 5 and 4: 5 + 9 = 14.
6. A range is two prefixes: through j = 6 it reads 14 + 9 = 23, through j = 2 it reads 4, so 23 - 4 = 19. The whole array, Prefix(7), runs at j = 8 and reads its single slot, 31.

j	lowbit(j)	p	tree[p] += tree[j]	slot p now
1	1	2	1 + 3	4
2	2	4	1 + 4	5
3	1	4	5 + 4	9
4	4	8	6 + 9	15
5	1	6	9 + 5	14
6	2	8	15 + 14	29
7	1	8	29 + 2	31
8	8	16	past the end	

query	walk	arithmetic	lands
Prefix(4)	j = 5 then 4	5 + 9	14
RangeSum(2, 5)	j = 6 then 4, minus j = 2	(14 + 9) - 4	19
Prefix(7)	j = 8 only	31	31

The whole array closes the walk at one read, 31, and the listings below build and query this same table in seven languages.

```
/// <summary>Fenwick tree: point add, prefix and range sum, reads counted per query.</summary>
public sealed class Fenwick
{
    private readonly long[] _tree;
    public int ReadsLastQuery { get; private set; }

    private static int Lowbit(int i) => i & -i;

    public Fenwick(int size) => _tree = new long[size];

    // 0-based in public, 1-based inside: node j covers (j - lowbit(j), j]
    public void Add(int i, long delta)
    {
        for (int j = i + 1; j ≤ _tree.Length; j += Lowbit(j))
            _tree[j - 1] += delta;
    }

    // reads = popcount of the 1-based endpoint, one per set bit
    public long Prefix(int i)
    {
        ReadsLastQuery = 0;
        long sum = 0;
        for (int j = i + 1; j > 0; j -= Lowbit(j))
        {
            sum += _tree[j - 1];
            ReadsLastQuery++;
        }
        return sum;
    }

    public long RangeSum(int lo, int hi) => Prefix(hi) - Prefix(lo - 1);
}
```

listing 668: point add climbs by lowbit, prefix strips it, reads counted per query (source: dsa/samples/src/Ch21/Ranges.cs, lines 3-34)

The tests pin the build 3, 1, 4, 1, 5, 9, 2, 6: Prefix(4) is 14, RangeSum(2,5) is 19, Prefix(7) is 31. The meter carries the real claim: a prefix ending at 1-based i reads exactly one cell per set bit, so the suite asserts ReadsLastQuery equals PopCount(i) for all sixteen endpoints of a 16-tree, where 16 reads one cell and 15 reads four. Two hundred seeded random ops interleaving adds and range sums then match a running array exactly, the cross-check that climbing and stripping agree.

The six sibling builds keep the same contract in their own shape:

```
static void fen_add(int i, int delta) { // 1-indexed
  for (; i ≤ N; i += i & (-i))
    tree[i] += delta;
}

static int fen_prefix(int i) { // sum of entries 1..i
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += tree[i];
  return s;
}

static int fen_get(int i) { return fen_prefix(i) - fen_prefix(i - 1); }
```

listing 669: c, add climbs by lowbit, prefix strips it, a naive twin cross-checks (source: dsa/samples-c/src/Ch21/fenwick.c, lines 21-33)

```go
type Fenwick struct {
  tree []int64 // 1-based; tree[0] unused
}

// NewFenwick builds the tree from initial values in O(n) by pushing
// each partial sum up to its parent.
func NewFenwick(values []int64) *Fenwick {
  f := &Fenwick{tree: append([]int64{0}, values...)}
  for i := 1; i < len(f.tree); i++ {
    parent := i + i&(-i)
    if parent < len(f.tree) {
      f.tree[parent] += f.tree[i]
    }
  }
  return f
}

// Add adds delta at position pos (0-based).
func (f *Fenwick) Add(pos int, delta int64) {
  for i := pos + 1; i < len(f.tree); i += i & (-i) {
    f.tree[i] += delta
  }
}

// Prefix returns the sum of positions [0, pos].
func (f *Fenwick) Prefix(pos int) int64 {
  sum := int64(0)
  for i := pos + 1; i > 0; i -= i & (-i) {
    sum += f.tree[i]
  }
  return sum
}

// Range returns the sum of [lo, hi].
func (f *Fenwick) Range(lo, hi int) int64 {
  if lo == 0 {
    return f.Prefix(hi)
  }
  return f.Prefix(hi) - f.Prefix(lo-1)
```

listing 670: go, the struct, O(n) build from the array, range as two prefixes (source: dsa/samples-go/ch21/fenwick.go, lines 6-44)

```java
static final int N = 8;

static int[] tree = new int[N + 1];

static void fenAdd(int i, int delta) { // 1-indexed
  for (; i ≤ N; i += i & (-i))
    tree[i] += delta;
}

static int fenPrefix(int i) { // sum of entries 1..i
  int s = 0;
  for (; i > 0; i -= i & (-i))
    s += tree[i];
  return s;
}

static int fenGet(int i) {
  return fenPrefix(i) - fenPrefix(i - 1);
}

// the naive twin for cross-checking: true when every prefix agrees
static int[] arr = new int[N + 1];

static boolean checkAll() {
  int s = 0;
  for (int i = 1; i ≤ N; i++) {
    s += arr[i];
    if (fenPrefix(i) ≠ s)
      return false;
  }
  return true;
}
```

listing 671: java, the static tree and its shadow array, add climbs, prefix strips, every prefix cross-checked (source: dsa/samples-java/src/Ch21/Fenwick.java, lines 16-47)

```
export class Fenwick {
  #tree;
  #n;

  constructor(n) {
    this.#n = n;
    this.#tree = new Array(n + 1).fill(0); // 1-based
  }

  // add delta at index i (0-based)
  add(i, delta) {
    for (let x = i + 1; x ≤ this.#n; x += x & -x) this.#tree[x] += delta;
    return this;
  }

  // sum of a[0..i] (0-based, inclusive)
  prefix(i) {
    let sum = 0;
    for (let x = i + 1; x > 0; x -= x & -x) sum += this.#tree[x];
    return sum;
  }

  // sum of a[l..r] inclusive
  rangeSum(l, r) {
    return this.prefix(r) - this.prefix(l - 1);
  }
}
```

listing 672: javascript, the class, prefix and range on the same climb (source: dsa/samples-js/src/ch21-fenwick.mjs, lines 5-31)

```
class Fenwick:
    def __init__(self, values):
        self.tree = [0] * (len(values) + 1)  # 1-indexed, slot 0 unused
        for i, v in enumerate(values, start=1):
            self.add(i, v)

    def add(self, i, delta):
        while i < len(self.tree):  # climb to every covering slot
            self.tree[i] += delta
            i += i & -i  # the lowest set bit climbs one staircase step

    def prefix(self, i):
        total = 0
        while i > 0:
            total += self.tree[i]
            i -= i & -i  # strip the lowest bit: jump to the next span
        return total

    def total(self):
        return self.prefix(len(self.tree) - 1)
```

listing 673: python, the class with a shadow list agreeing after every mutation (source: dsa/samples-py/src/Ch21/fenwick.py, lines 14-33)

```lua
function Fenwick.new(n)
  return {n = n, t = {}}
end

-- add v at 0-based index i
function Fenwick.add(ft, i, v)
  local j = i + 1
  while j ≤ ft.n do
    ft.t[j] = (ft.t[j] or 0) + v
    j = j + (j & -j)
  end
end

-- sum of a[0..i] inclusive
function Fenwick.prefix(ft, i)
  local s = 0
  local j = i + 1
  while j > 0 do
    s = s + (ft.t[j] or 0)
    j = j - (j & -j)
  end
  return s
end

function Fenwick.range(ft, lo, hi)
  return Fenwick.prefix(ft, hi) - Fenwick.prefix(ft, lo - 1)
end
```

listing 674: lua, 0-based outside shifted to 1-based inside, cross-checked against a prefix array (source: dsa/samples-lua/ch21_fenwick.lua, lines 6-33)

The fixture family splits exactly as the history of the book does. C#, JavaScript, Python, and Lua share 3, 1, 4, 1, 5, 9, 2, 6 and pin the same three numbers, prefix 5 at 14, range 2 to 5 at 19, the whole array at 31, and Python watches a shadow list agree after every add. Go builds 2, 4, 1, 6, 3 and pins every prefix, and C and Java load 1 through 8 and pin the triangular prefixes 1, 3, 6, 10,
28. The inside-outside indexing split is worth one read: C, C#,

Go, and Java keep the 1-based indexing inside the structure, Lua and Python shift a 0-based public face down to the 1-based table, and the C and Java suites keep a naive twin answering every prefix, the cross-check that climbing and stripping never drift.

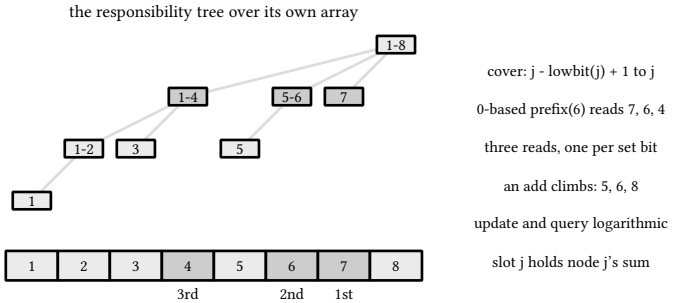


figure 181: the fenwick tree above the flat array it lives in, node j responsible for the run ending at j, one prefix walk of three reads shaded

21.3. the segment tree with lazy propagation

The segment tree halves the array recursively and lets every node answer for its whole segment, so a range query decomposes into a handful of canonical nodes, at most two per level. Lazy propagation gives updates the same deal: a range add parks its delta at the highest nodes that fit inside the range and stops there, and the tag moves down only when a later descent needs those children:

```
/// <summary>Lazy segment tree: range add and range sum, tags pushed only on descent.</summary>
public sealed class LazySeg
{
    private readonly int _n;
    private readonly long[] _sum;
    private readonly long[] _tag;
    public int NodesTouched { get; private set; }

    public LazySeg(long[] values)
    {
        _n = values.Length;
        _sum = new long[4 * _n];
        _tag = new long[4 * _n];
        Build(1, 0, _n - 1, values);
    }

    private void Build(int node, int lo, int hi, long[] values)
    {
        if (lo == hi)
        {
            _sum[node] = values[lo];
            return;
        }
        int mid = (lo + hi) / 2;
        Build(2 * node, lo, mid, values);
        Build(2 * node + 1, mid + 1, hi, values);
        _sum[node] = _sum[2 * node] + _sum[2 * node + 1];
    }

    // the meter counts nodes the traversal enters, push-down tags children it never enters
    public void AddRange(int lo, int hi, long delta)
    {
        NodesTouched = 0;
        Add(1, 0, _n - 1, lo, hi, delta);
    }

    public long SumRange(int lo, int hi)
    {
        NodesTouched = 0;
        return Query(1, 0, _n - 1, lo, hi);
    }

    private void Apply(int node, int lo, int hi, long delta)
    {
        _sum[node] += delta * (hi - lo + 1);
        _tag[node] += delta;
    }

    private void Push(int node, int lo, int mid, int hi)
    {
```

```
        if (_tag[node] == 0)
            return;
        Apply(2 * node, lo, mid, _tag[node]);
        Apply(2 * node + 1, mid + 1, hi, _tag[node]);
        _tag[node] = 0;
    }

    private void Add(int node, int lo, int hi, int l, int r, long delta)
    {
        NodesTouched++;
        if (l ≤ lo && hi ≤ r)
        {
            Apply(node, lo, hi, delta);
            return;
        }
        int mid = (lo + hi) / 2;
        Push(node, lo, mid, hi);
        if (l ≤ mid)
            Add(2 * node, lo, mid, l, r, delta);
        if (r > mid)
            Add(2 * node + 1, mid + 1, hi, l, r, delta);
        _sum[node] = _sum[2 * node] + _sum[2 * node + 1];
    }

    private long Query(int node, int lo, int hi, int l, int r)
    {
        NodesTouched++;
        if (l ≤ lo && hi ≤ r)
            return _sum[node];
        int mid = (lo + hi) / 2;
        Push(node, lo, mid, hi);
        long res = 0;
        if (l ≤ mid)
            res += Query(2 * node, lo, mid, l, r);
        if (r > mid)
            res += Query(2 * node + 1, mid + 1, hi, l, r);
        return res;
    }
}
```

listing 675: range add and range sum, tags pushed only when the descent needs the children (source: dsa/samples/src/Ch21/Ranges.cs, lines 36-124)

On 1, 2, 3, 4, 5 the whole-range sum is 15, and after AddRange(1,3,10) it is 45 while SumRange(0,0) still returns 1: the leaf never saw the tag because nothing descended to it. The meter bounds the traversal, a whole-range add touches exactly one node, and across a hundred seeded random ops on 8 leaves no operation touches more than 9 of the 15 nodes, inside the 4 times ceil(log2 n) = 12 the test allows, with every sum matching brute force. The figure runs the same story on 1 through 8: an AddRange(0,3,10) then SumRange(2,6) lands on covers 2-3, 4-5 and 6-6, which sum to 45.

The six sibling builds drop the lazy tags and answer the matrix contract instead, point update with range sum and range min:

```
// build over [lo, hi] at node t; qlo/qhi clamped query helpers
static void build(int t, int lo, int hi) {
  if (lo == hi) {
    sum_t[t] = arr[lo];
    min_t[t] = arr[lo];
    return;
  }
  int mid = lo + (hi - lo) / 2;
  build(2 * t, lo, mid);
  build(2 * t + 1, mid + 1, hi);
  sum_t[t] = sum_t[2 * t] + sum_t[2 * t + 1];
  min_t[t] = mini(min_t[2 * t], min_t[2 * t + 1]);
}

static int qsum(int t, int lo, int hi, int a, int b) {
  if (b < lo || hi < a)
    return 0;
  if (a ≤ lo && hi ≤ b)
    return sum_t[t];
  int mid = lo + (hi - lo) / 2;
  return qsum(2 * t, lo, mid, a, b) + qsum(2 * t + 1, mid + 1, hi, a, b);
}

static int qmin(int t, int lo, int hi, int a, int b) {
  if (b < lo || hi < a)
    return 1000000;
  if (a ≤ lo && hi ≤ b)
    return min_t[t];
  int mid = lo + (hi - lo) / 2;
  return mini(qmin(2 * t, lo, mid, a, b), qmin(2 * t + 1, mid + 1, hi, a, b));
}

static void update(int t, int lo, int hi, int pos, int val) {
```

listing 676: c, sum and min trees in one build, both queries (source: dsa/samples-c/src/Ch21/segtree.c, lines 22-55)

```
func (st *SegTree) Set(pos int, value int64) {
  i := pos + st.n
  st.sums[i] = value
  st.mins[i] = value
  for i /= 2; i ≥ 1; i /= 2 {
    st.sums[i] = st.sums[2*i] + st.sums[2*i+1]
    st.mins[i] = min(st.mins[2*i], st.mins[2*i+1])
  }
}

// SumRange answers the sum of [lo, hi].
func (st *SegTree) SumRange(lo, hi int) int64 {
  return st.fold(lo, hi, true)
}

// MinRange answers the minimum of [lo, hi].
func (st *SegTree) MinRange(lo, hi int) int64 {
  return st.fold(lo, hi, false)
}

func (st *SegTree) fold(lo, hi int, sum bool) int64 {
  var accSum int64
  accMin := int64(1) << 60
  l, r := lo+st.n, hi+st.n+1
  for l < r {
    if l&1 == 1 {
      if sum {
        accSum += st.sums[l]
      } else {
        accMin = min(accMin, st.mins[l])
      }
      l++
    }
    if r&1 == 1 {
      r--
      if sum {
        accSum += st.sums[r]
      } else {
        accMin = min(accMin, st.mins[r])
      }
    }
    l, r = l/2, r/2
  }
  if sum {
```

listing 677: go, set, sum, min, one fold drives both (source: dsa/samples-go/ch21/segtree.go, lines 40-83)

```
  static final int N = 5;

  static int[] sumT = new int[4 * N];
  static int[] minT = new int[4 * N];
  static int[] arr = new int[N];

  // build over [lo, hi] at node t
  static void build(int t, int lo, int hi) {
    if (lo == hi) {
      sumT[t] = arr[lo];
      minT[t] = arr[lo];
      return;
    }
    int mid = lo + (hi - lo) / 2;
    build(2 * t, lo, mid);
    build(2 * t + 1, mid + 1, hi);
    sumT[t] = sumT[2 * t] + sumT[2 * t + 1];
    minT[t] = Math.min(minT[2 * t], minT[2 * t + 1]);
  }

  static int qsum(int t, int lo, int hi, int a, int b) {
    if (b < lo || hi < a)
      return 0;
    if (a ≤ lo && hi ≤ b)
      return sumT[t];
    int mid = lo + (hi - lo) / 2;
    return qsum(2 * t, lo, mid, a, b) + qsum(2 * t + 1, mid + 1, hi, a, b);
  }

  static int qmin(int t, int lo, int hi, int a, int b) {
    if (b < lo || hi < a)
      return 1_000_000;
    if (a ≤ lo && hi ≤ b)
      return minT[t];
    int mid = lo + (hi - lo) / 2;
    return Math.min(qmin(2 * t, lo, mid, a, b), qmin(2 * t + 1, mid + 1, hi, a, b));
  }

  static void update(int t, int lo, int hi, int pos, int val) {
    if (lo == hi) {
      arr[pos] = val;
      sumT[t] = val;
      minT[t] = val;
      return;
    }
    int mid = lo + (hi - lo) / 2;
    if (pos ≤ mid)
      update(2 * t, lo, mid, pos, val);
    else
      update(2 * t + 1, mid + 1, hi, pos, val);
    sumT[t] = sumT[2 * t] + sumT[2 * t + 1];
    minT[t] = Math.min(minT[2 * t], minT[2 * t + 1]);
  }
```

listing 678: java, the recursive sum and min trees, and the iterative bottom-up shape as a nested class (source: dsa/samples-java/src/Ch21/Segtree.java, lines 15-67)

```
export class SegTree {
  #sum;
  #min;
  #n;

  constructor(a) {
    this.#n = a.length;
    this.#sum = new Array(2 * this.#n).fill(0);
    this.#min = new Array(2 * this.#n).fill(Infinity);
    for (let i = 0; i < this.#n; i++) {
      this.#sum[this.#n + i] = a[i];
      this.#min[this.#n + i] = a[i];
    }
    for (let i = this.#n - 1; i > 0; i--) {
      this.#sum[i] = this.#sum[2 * i] + this.#sum[2 * i + 1];
      this.#min[i] = Math.min(this.#min[2 * i], this.#min[2 * i + 1]);
    }
  }

  // a[i] = v (assignment, not a delta)
  update(i, v) {
    let x = this.#n + i;
    this.#sum[x] = v;
    this.#min[x] = v;
    for (x >>= 1; x ≥ 1; x >>= 1) {
      this.#sum[x] = this.#sum[2 * x] + this.#sum[2 * x + 1];
      this.#min[x] = Math.min(this.#min[2 * x], this.#min[2 * x + 1]);
    }
    return this;
  }

  rangeSum(l, r) {
    let res = 0;
    for (let lo = l + this.#n, hi = r + this.#n + 1; lo < hi; lo >>= 1, hi >>= 1) {
      if (lo & 1) res += this.#sum[lo++];
      if (hi & 1) res += this.#sum[--hi];
    }
    return res;
  }

  rangeMin(l, r) {
    let res = Infinity;
    for (let lo = l + this.#n, hi = r + this.#n + 1; lo < hi; lo >>= 1, hi >>= 1) {
      if (lo & 1) res = Math.min(res, this.#min[lo++]);
      if (hi & 1) res = Math.min(res, this.#min[--hi]);
    }
    return res;
  }
}
```

listing 679: javascript, an iterative bottom-up tree, lo and hi walk to the root (source: dsa/samples-js/src/ch21-segtree.mjs, lines 4-52)

```
class SegTree:
    def __init__(self, values):
        self.size = len(values)
        self.sum = [0] * (2 * self.size)
        self.low = [None] * (2 * self.size)
        for i, v in enumerate(values):
            self.sum[self.size + i] = v
            self.low[self.size + i] = v
        for i in range(self.size - 1, 0, -1):
            self.sum[i] = self.sum[2 * i] + self.sum[2 * i + 1]
            self.low[i] = min(self.low[2 * i], self.low[2 * i + 1])

    def update(self, at, value):
        i = self.size + at
        self.sum[i] = value
        self.low[i] = value
        i //= 2
        while i:
            self.sum[i] = self.sum[2 * i] + self.sum[2 * i + 1]
            self.low[i] = min(self.low[2 * i], self.low[2 * i + 1])
            i //= 2

    def range_sum(self, lo, hi):  # inclusive on both ends
        total = 0
        lo += self.size
        hi += self.size + 1
        while lo < hi:
            if lo & 1:
                total += self.sum[lo]
                lo += 1
            if hi & 1:
                hi -= 1
                total += self.sum[hi]
            lo //= 2
            hi //= 2
        return total

    def range_min(self, lo, hi):
        best = float("inf")
        lo += self.size
        hi += self.size + 1
        while lo < hi:
            if lo & 1:
                best = min(best, self.low[lo])
                lo += 1
            if hi & 1:
                hi -= 1
                best = min(best, self.low[hi])
            lo //= 2
            hi //= 2
        return best


START = [3, 1, 4, 1, 5, 9, 2, 6]
```

listing 680: python, sum and min in one class, brute mirrors both (source: dsa/samples-py/src/Ch21/segtree.py, lines 13-66)

```lua
local function segtree_new(a, op, identity)
  local n = #a
  local t = {n = n, op = op, id = identity}
  local size = 1
  while size < n do size = size * 2 end
  t.size = size
  t.t = {}
  for i = 1, 2 * size do t.t[i] = identity end
  for i = 1, n do t.t[size + i - 1] = a[i] end
  for i = size - 1, 1, -1 do
    t.t[i] = op(t.t[2 * i], t.t[2 * i + 1])
  end
  return t
end

-- set 0-based index i to v
local function segtree_set(st, i, v)
  local p = st.size + i
  st.t[p] = v
  p = p // 2
  while p ≥ 1 do
    st.t[p] = st.op(st.t[2 * p], st.t[2 * p + 1])
    p = p // 2
  end
end

-- query over 0-based [lo, hi] inclusive
local function segtree_query(st, lo, hi)
  local res = st.id
  local l, r = st.size + lo, st.size + hi
  while l ≤ r do
    if l % 2 == 1 then
      res = st.op(res, st.t[l])
      l = l + 1
    end
    if r % 2 == 0 then
      res = st.op(res, st.t[r])
      r = r - 1
    end
    l, r = l // 2, r // 2
  end
  return res
end

local function sum_tree(a)
  return segtree_new(a, function(x, y) return x + y end, 0)
end
local function min_tree(a) return segtree_new(a, math.min, math.maxinteger) end
```

listing 681: lua, the fold and identity as arguments, two trees from one build (source: dsa/samples-lua/ch21_segtree.lua, lines 6-53)

The C# tree above and these five are honestly different structures, and the comparison note is the point: lazy tags exist to update whole ranges in logarithmic time, and once the update is a single point the tag machinery is overhead, so the sibling builds carry sum and min arrays straight down. JavaScript's is the odd shape worth a second look, an iterative tree over doubled indices with no recursion at all, and Lua builds one engine and hands it the sum fold with identity 0 and the min fold with maxinteger. Fixtures differ per suite, C and Java over 2, 1, 5, 3, 4, Go over 2, 4, 1, 6, 3, Python back on 3, 1, 4, 1, 5, 9, 2, 6 with whole-range 31 and range 2 to 5 at 19 again, Java

running that same fixture through the iterative bottom-up shape as a nested class, and every suite cross-checks every query against a linear scan, Python and Java after each of their updates.

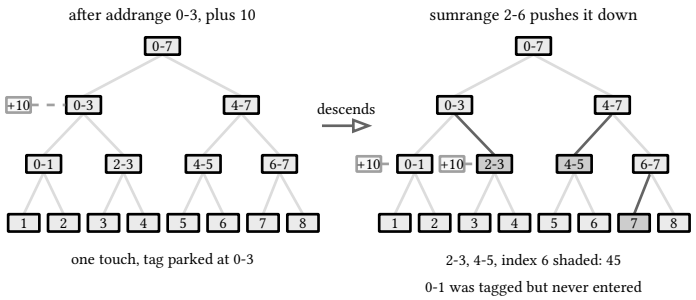


figure 182: a lazy segment tree in two frames, the parked add moving down one level only when the shaded query descends

PITFALL: push before you descend

The lazy tag is a promise the parent's sum already keeps and its children know nothing about. Any traversal that reads or writes below a tagged node must move the tag down first, or it mixes stale children into an updated parent and returns a wrong sum with no crash to announce it. The NodesTouched meter at least keeps the traversal visible while debugging.

21.4. the sparse table

Flip the premise: if the array never changes, precompute every power-of-two cover once. Level k holds the minimum of every 2^k-wide span, and a query takes the two covers of the largest power of two that fits, one anchored at each end. The two overlap in the middle, which is legal because min is idempotent, folding the same element in twice changes nothing:

```
/// <summary>Sparse table for idempotent range min, two power-of-two covers per query.</summary>
public sealed class SparseMin
{
    private readonly long[][] _table;
    public int Levels { get; }

    public SparseMin(long[] values)
    {
        int n = values.Length;
        Levels = System.Numerics.BitOperations.Log2((uint)n) + 1;
        _table = new long[Levels][];
        _table[0] = (long[])values.Clone();
        for (int k = 1; k < Levels; k++)
        {
            _table[k] = new long[n - (1 << k) + 1];
            for (int i = 0; i + (1 << k) <= n; i++)
            {
                int half = 1 << (k - 1);
                _table[k][i] = Math.Min(_table[k - 1][i], _table[k - 1][i + half]);
            }
        }
    }

    // overlap is the trick: min(min(a, b), min(b, c)) = min(a, b, c)
    public long Min(int lo, int hi)
    {
        int k = System.Numerics.BitOperations.Log2((uint)(hi - lo + 1));
        return Math.Min(_table[k][lo], _table[k][hi - (1 << k) + 1]);
    }
}
```

listing 682: two overlapping power-of-two covers per query, overlap legal because min is idempotent (source: dsa/samples/src/Ch21/Ranges.cs, lines 126-155)

Over 5, 2, 8, 1, 9, 3, 7, 4 the tests pin Min(1,5) and Min(0,3) at 1, Min(5,7) at 3, Min(4,4) at 9, and Levels at 4 for n = 8, the top cover spanning the whole array. Min(1,6) deliberately lands on two 4-wide covers anchored at 1 and 3, overlapping at 3 and 4, and every range of seeded arrays up to 64 elements agrees with brute force.

The six sibling builds stack the same doubling levels:

```
static void build(void) {
  for (int i = 0; i < N; i++)
    sp[0][i] = A[i];
  for (int k = 1; k ≤ LOG; k++)
    for (int i = 0; i + (1 << k) ≤ N; i++) {
      int a = sp[k - 1][i], b = sp[k - 1][i + (1 << (k - 1))];
      sp[k][i] = a < b ? a : b;
    }
}

static int floor_log2(int x) {
  int k = -1;
  while (x) {
    x >>= 1;
    k++;
  }
  return k;
}

// index of the minimum in A[a..b] (leftmost on ties)
static int rmq(int a, int b) {
  int k = floor_log2(b - a + 1);
  int len = 1 << k;
  int i1 = sp[k][a], i2 = sp[k][b - len + 1];
  // stored values: compare, then re-find the index through A
  int v = i1 < i2 ? i1 : i2;
  for (int i = a; i ≤ b; i++)
    if (A[i] == v)
      return i;
  return -1;
}

int main(void) {
```

listing 683: c, level k over level k - 1, the two-cover query (source: dsa/samples-c/src/Ch21/sparsetable.c, lines 23-55)

```
type SparseTable struct {
  levels [][]int // levels[k][i] = min(a[i .. i+2^k-1])
}

// NewSparseTable builds the doubling table in O(n log n).
func NewSparseTable(a []int) *SparseTable {
  if len(a) == 0 {
    return &SparseTable{}
  }
  levels := make([][]int, bits.Len(uint(len(a))))
  levels[0] = append([]int(nil), a...)
  for k := 1; k < len(levels); k++ {
    window := 1 << uint(k)
    prev := levels[k-1]
    cur := make([]int, len(a)-window+1)
    for i := range cur {
      cur[i] = min(prev[i], prev[i+window/2])
    }
    levels[k] = cur
  }
  return &SparseTable{levels: levels}
}

// Min returns the minimum of a[lo..hi].
func (st *SparseTable) Min(lo, hi int) int {
  k := bits.Len(uint(hi-lo+1)) - 1
  return min(st.levels[k][lo], st.levels[k][hi-(1<<uint(k))+1])
}
```

listing 684: go, the doubling build, the query by two covers (source: dsa/samples-go/ch21/sparsetable.go, lines 9-37)

```
  static final int N = 8;
  static final int LOG = 4;

  static final int[] A = {5, 2, 4, 1, 3, 7, 6, 8};
  static int[][] sp = new int[LOG + 1][N]; // sp[k][i] = min of A[i .. i+2^k-1]

  static void build() {
    for (int i = 0; i < N; i++)
      sp[0][i] = A[i];
    for (int k = 1; k ≤ LOG; k++)
      for (int i = 0; i + (1 << k) ≤ N; i++)
        sp[k][i] = Math.min(sp[k - 1][i], sp[k - 1][i + (1 << (k - 1))]);
  }

  static int floorLog2(int x) {
    int k = -1;
    while (x ≠ 0) {
      x >>= 1;
      k++;
    }
    return k;
  }

  // index of the minimum in A[a..b] (leftmost on ties)
  static int rmq(int a, int b) {
    int k = floorLog2(b - a + 1);
    int len = 1 << k;
    int v = Math.min(sp[k][a], sp[k][b - len + 1]);
    // stored values: compare, then re-find the index through A
    for (int i = a; i ≤ b; i++)
      if (A[i] == v)
        return i;
    return -1;
  }
```

listing 685: java, the doubling levels, the query answering with the index (source: dsa/samples-java/src/Ch21/Sparsetable.java, lines 16-49)

```
export class SparseTable {
  #table;
  #log;

  constructor(a) {
    const n = a.length;
    this.#table = [a];
    for (let k = 1; 1 << k <= n; k++) {
      const prev = this.#table[k - 1];
      const row = new Array(n - (1 << k) + 1);
      for (let i = 0; i + (1 << k) <= n; i++)
        row[i] = Math.min(prev[i], prev[i + (1 << (k - 1))]);
      this.#table.push(row);
    }
    this.#log = new Array(n + 1).fill(0);
    for (let i = 2; i <= n; i++) this.#log[i] = this.#log[i >> 1] + 1;
  }

  get levels() {
    return this.#table.length;
  }

  // min of a[l..r] inclusive, two overlapping covers
  min(l, r) {
    const k = this.#log[r - l + 1];
    return Math.min(this.#table[k][l], this.#table[k][r - (1 << k) + 1]);
  }
}
```

listing 686: javascript, the class, levels logged at build (source: dsa/samples-js/src/ch21-sparsetable.mjs, lines 5-32)

```python
def log2_floor(x):
    k = 0
    while (1 << (k + 1)) <= x:
        k += 1
    return k


def build(a):
    # table[k][i]: the minimum of the 2^k run starting at i
    table = [a[:]]
    k = 1
    while (1 << k) <= len(a):
        prev = table[k - 1]
        half = 1 << (k - 1)
        run = [min(prev[i], prev[i + half]) for i in range(len(a) - (1 << k) + 1)]
        table.append(run)
        k += 1
    return table


def query(table, lo, hi):  # inclusive, returns the minimum
    k = log2_floor(hi - lo + 1)
    return min(table[k][lo], table[k][hi - (1 << k) + 1])


T = build(A)
check(T[0] == [5, 2, 8, 1, 3, 6, 7, 4], "level 0 is the array itself")
check(T[1] == [2, 2, 1, 1, 3, 6, 4], "level 1 holds pair minima")
check(T[3] == [1], "level 3 covers the single 8-slot run")
check(query(T, 0, 3) == 1, "the front quarter bottoms at index 3")
check(query(T, 4, 7) == 3, "the back half bottoms at index 4")
```

listing 687: python, log2 by hand, levels pinned open (source: dsa/samples-py/src/Ch21/sparsetable.py, lines 16-46)

```lua
local function sparse_new(a)
  local n = #a
  local st = {{table.unpack(a)}}
  local k = 1
  while (1 << k) <= n do
    local prev, cur = st[k], {}
    local span = 1 << k
    for i = 1, n - span + 1 do
      cur[i] = math.min(prev[i], prev[i + span // 2])
    end
    st[k + 1] = cur
    k = k + 1
  end
  st.levels = #st
  return st
end

-- min over 0-based [lo, hi] inclusive
local function sparse_min(st, lo, hi)
  local len = hi - lo + 1
  local k = 0
  while (1 << (k + 1)) <= len do k = k + 1 end
  local base = st[k + 1]
  -- one window at lo, one ending at hi; they overlap freely
  return math.min(base[lo + 1], base[hi - (1 << k) + 2])
end
```

listing 688: lua, the table and the two-cover read, 1-based (source: dsa/samples-lua/ch21_sparsetable.lua, lines 6-31)

The C# and Lua suites share the fixture 5, 2, 8, 1, 9, 3, 7, 4, and both pin queries at 1, 1, 3, and 9 for the single element at index 4. Python and Java open their levels to inspection, level 0 the array itself and level 1 the pair minima, C and Java run 5, 2, 4, 1, 3, 7, 6, 8 and pin rmq(0, 3) at 3 for the value 1, Go runs 5, 2, 4, 1, 3 with seven pinned queries, and JavaScript's table carries the level exponent alongside. Every suite keeps the linear scan as the oracle over seeded ranges.

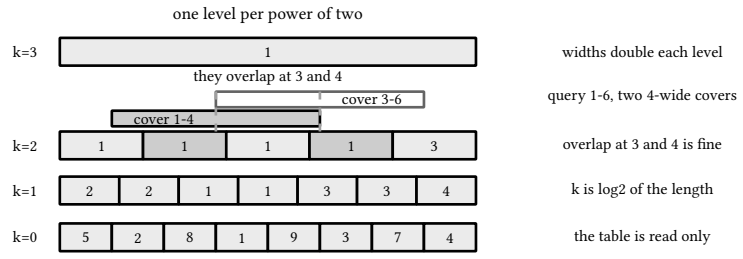


figure 183: the sparse table as stacked doubling levels, query 1-6 answered by two overlapping 4-wide covers

PITFALL: overlap is only legal for idempotent ops

min, max and gcd tolerate the doubled overlap because applying them twice to the same element changes nothing. sum does not, the overlap would be counted twice. The other price is permanence, one changed cell invalidates every cover above it, and at that point this chapter's fenwick tree or a segment tree belongs in the slot instead.

21.5. sliding window maxima

When the window slides one position per step, the running max needs no tree at all. Keep a deque of indices whose values strictly decrease front to back: the front is the current maximum, a new arrival evicts every smaller or equal index off the back, and the front retires by index distance when it slides out.

The dry run: the fixture is 1, 3, −1, −3, 5, 3, 6, 7 with k = 3, the same literals in all seven suites, and the meters count one push per arrival with both eviction kinds summed into the pops.

1. The classic run outputs 3, 3, 5, 5, 6, 7 over 8 pushes and 7 pops, back evictions and front aging counted together.
2. The deque after each output reads 1, 2, then 1, 2, 3, then 4, then 4, 5, then 6, then 7, the maximum always at the front.
3. The ties lane runs 4, 4, 4, 4 with k = 2 and outputs 4, 4, 4 over 4 pushes and 3 pops, the deque after each output holding only the newest index, the older equal value gone.
4. The degenerate windows: k = 1 on 5, 1, 5 echoes 5, 1, 5, and k = n on 2, 9, 4, 1 keeps the single maximum 9.
5. The bound lane: pushes equal n on every fixture, and pushes plus pops stay within 2n, an index enters once and leaves at most once.
6. Per-tree honesty: the C# meter pins the 8 pushes and the 2n ceiling without the 7 pops, the Python and Java suites add a k = 1 meter reading of 2 pops for the front aging out twice, and the deque trace is pinned by the C, Java, JavaScript, Python, and Lua suites.

```
/// <summary>Counter for the monotonic structures: pushes, pops, combined.</summary>
public sealed class Meter
{
    public long Pushes { get; set; }
    public long Pops { get; set; }
    public long Ops { get => Pushes + Pops; }
}

/// <summary>Sliding window maxima from a monotonic deque of indices, the front is the max.</summary>
public static class Windows
{
    // values strictly decrease front to back, an equal arrival evicts the older index
    public static long[] Maxima(long[] a, int k, Meter? meter = null)
    {
        var deque = new LinkedList<int>();
        var result = new long[a.Length - k + 1];
        for (int i = 0; i < a.Length; i++)
        {
            while (deque.Count > 0 && a[deque.Last!.Value] <= a[i])
            {
                deque.RemoveLast();
                if (meter != null)
                    meter.Pops++;
            }
            deque.AddLast(i);
            if (meter != null)
                meter.Pushes++;
            if (i >= k - 1)
            {
                while (deque.First!.Value < i - k + 1)
                {
                    deque.RemoveFirst();
                    if (meter != null)
                        meter.Pops++;
                }
                result[i - k + 1] = a[deque.First!.Value];
            }
        }
        return result;
    }
}
```

listing 689: the meter and the deque, back eviction and front expiry both counted (source: dsa/samples/src/Ch21/Ranges.cs, lines 157-197)

The classic run, 1, 3, −1, −3, 5, 3, 6, 7 with k = 3, produces 3, 3, 5, 5, 6, 7. The meter is the proof of linearity: 8 arrivals make exactly 8 pushes, and since an index leaves the deque at most once, pushes plus pops never reach past 16 for the whole scan. The nested while loop only looks quadratic, every eviction was paid for by the push that preceded it.

The six sibling builds keep the same deque contract over six substrates:

```c
// out receives n - k + 1 maxima, the meters count pushes and pops, and
// the optional trace records the deque contents at every output step
static void maxima(const long long *a, int count, int k, long long *out,
                   long *pushes, long *pops, int trace[][MAX_VALUES + 1]) {
  int deque[MAX_VALUES]; // live window lives in deque[head..tail)
  int head = 0, tail = 0;
  *pushes = 0;
  *pops = 0;
  for (int i = 0; i < count; i++) {
    while (tail > head && a[deque[tail - 1]] ≤ a[i]) {
      tail--; // equal arrival evicts the older index
      (*pops)++;
    }
    deque[tail++] = i;
    (*pushes)++;
    if (i ≥ k - 1) {
      while (deque[head] < i - k + 1) {
        head++; // front evicted by age
        (*pops)++;
      }
      out[i - k + 1] = a[deque[head]];
      if (trace) {
        for (int t = head; t < tail; t++)
          trace[i - k + 1][t - head] = deque[t];
        trace[i - k + 1][tail - head] = -1; // end marker
      }
    }
  }
}
```

listing 690: c, the deque in a fixed array under head and tail cursors, both eviction kinds metered, the trace through an out parameter (source: dsa/samples-c/src/Ch21/maxima.c, lines 19-47)

```go
// Maxima returns the sliding window maxima: a deque of indices whose
// values strictly decrease front to back, so an equal arrival evicts
// the older index, and the front leaves when age passes the window.
func Maxima(a []int64, k int, meter *WindowMeter) []int64 {
  var deque []int
  result := make([]int64, len(a)-k+1)
  for i := 0; i < len(a); i++ {
    for len(deque) > 0 && a[deque[len(deque)-1]] <= a[i] {
      deque = deque[:len(deque)-1]
      if meter ≠ nil {
        meter.Pops++
      }
    }
    deque = append(deque, i)
    if meter ≠ nil {
      meter.Pushes++
    }
    if i ≥ k-1 {
      for deque[0] < i-k+1 {
        deque = deque[1:]
        if meter ≠ nil {
          meter.Pops++
        }
      }
      result[i-k+1] = a[deque[0]]
    }
  }
  return result
}
```

listing 691: go, one slice grown at the back and shortened at both ends, the meter passed beside (source: dsa/samples-go/ch21/maxima.go, lines 13-41)

```
  static final int MAX_VALUES = 8;

  static long pushes;
  static long pops;

  // out receives n - k + 1 maxima; the trace records the deque contents
  // at every output step, or null when unwanted
  static long[] maxima(long[] a, int k, int[][] trace) {
    int[] deque = new int[MAX_VALUES]; // live window in deque[head..tail)
    int head = 0, tail = 0;
    long[] out = new long[a.length - k + 1];
    pushes = 0;
    pops = 0;
    for (int i = 0; i < a.length; i++) {
      while (tail > head && a[deque[tail - 1]] ≤ a[i]) {
        tail--; // equal arrival evicts the older index
        pops++;
      }
      deque[tail++] = i;
      pushes++;
      if (i ≥ k - 1) {
        while (deque[head] < i - k + 1) {
          head++; // front evicted by age
          pops++;
        }
        out[i - k + 1] = a[deque[head]];
        if (trace ≠ null) {
          for (int t = head; t < tail; t++)
            trace[i - k + 1][t - head] = deque[t];
          trace[i - k + 1][tail - head] = -1; // end marker
        }
      }
    }
    return out;
  }

  static boolean dequeMatches(int[] state, int... want) {
    for (int i = 0; i < want.length; i++)
      if (state[i] ≠ want[i])
        return false;
    return state[want.length] == -1;
  }
```

listing 692: java, the fixed-array deque under two cursors, the trace through a parameter, both pop kinds metered, an ArrayDeque lane beside (source: dsa/samples-java/src/Ch21/Maxima.java, lines 21-62)

```
export function maxima(a, k, meter = null) {
  const deque = [];
  const result = new Array(a.length - k + 1);
  for (let i = 0; i < a.length; i++) {
    while (deque.length > 0 && a[deque[deque.length - 1]] ≤ a[i]) {
      deque.pop();
      if (meter =/= null) meter.pops++;
    }
    deque.push(i);
    if (meter =/= null) meter.pushes++;
    if (i ≥ k - 1) {
      while (deque[0] < i - k + 1) {
        deque.shift();
        if (meter =/= null) meter.pops++;
      }
      result[i - k + 1] = a[deque[0]];
    }
  }
  return result;
}
```

listing 693: javascript, pop off the back, shift off the front, both counted as pops (source: dsa/samples-js/src/ch21-maxima.mjs, lines 6-25)

```
def maxima(a, k):
    queue = []  # indices, values strictly decrease front to back
    out = []
    pushes = 0
    pops = 0
    trace = []
    for i in range(len(a)):
        while queue and a[queue[-1]] ≤ a[i]:
            queue.pop()  # back eviction: the arrival is at least as good
            pops += 1
        queue.append(i)
        pushes += 1
        if i ≥ k - 1:
            while queue[0] < i - k + 1:
                queue.pop(0)  # front aging: the index left the window
                pops += 1
            out.append(a[queue[0]])
            trace.append(list(queue))
    return out, pushes, pops, trace
```

listing 694: python, the deque as a list, front aging pops index 0, the trace recorded per output (source: dsa/samples-py/src/Ch21/maxima.py, lines 14-32)

```lua
-- returns the maxima plus meter (pushes, pops) and an optional trace of
-- the deque contents at each output position
local function maxima(a, k, meter, trace)
  local deque = {} -- indices, front at 1
  local result = {}
  for i = 1, #a do
    while #deque > 0 and a[deque[#deque]] ≤ a[i] do
      deque[#deque] = nil -- equal or smaller leaves from the back
      if meter then meter.pops = meter.pops + 1 end
    end
    deque[#deque + 1] = i
    if meter then meter.pushes = meter.pushes + 1 end
    if i ≥ k then
      while deque[1] < i - k + 1 do
        table.remove(deque, 1) -- too old for this window
        if meter then meter.pops = meter.pops + 1 end
      end
      result[i - k + 1] = a[deque[1]]
      if trace then
        local snap = {}
        for d, idx in ipairs(deque) do snap[d] = idx - 1 end
        trace[#trace + 1] = snap
      end
    end
  end
  return result
end
```

listing 695: lua, a back eviction nils the last slot, aging removes position 1, the trace snapped per output (source: dsa/samples-lua/ch21_maxima.lua, lines 7-33)

The deque substrate is where the six builds differ. C and Java never move an element, they walk two cursors through one fixed array and hand the teaching trace out through a parameter, Go grows a slice and shortens it from either end, JavaScript pops the back and shifts the front of a plain array, Python pops index 0 when the front ages, and Lua nils the back slot and removes position 1, Java carrying the ArrayDeque substrate as a second lane beside its cursor build. The meter arithmetic lands the same everywhere, 8 pushes with 7 pops on the classic run, 4 with 3 on the ties lane, and the 2n ceiling on the combined count. The trace lane splits by reach, C, Java, Python, and Lua record it inside the walk, JavaScript keeps a twin walker in the same file for it, and Go checks every output against a brute window maximum instead.

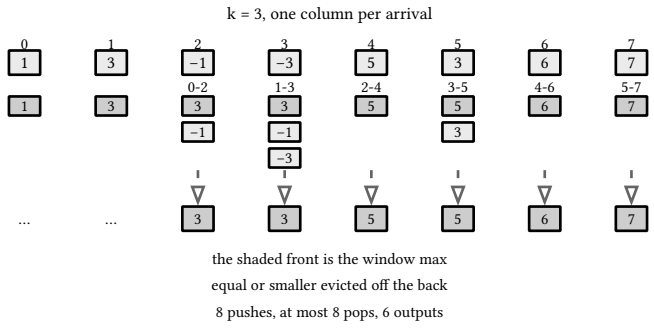


figure 184: sliding window maxima column by column, the deque of survivors under each arrival, the shaded front output as the max

21.6. next greater, both sides

The same monotonic idea with a stack answers the nearest larger neighbor. Scanning left to right, the stack holds indices with strictly decreasing values, and when a bigger value arrives it pops everything smaller: each pop is an

answer, the popper is the poppee's next greater to the right. The right to left pass fills the left array identically, and whatever survives a scan keeps its −1.

The dry run: the answers are indices with −1 when no greater element exists, and only a strictly greater value pops. The six sibling suites assert the family below with identical literals, and the C# suite pins the first row together with its seeded brute agreement.

1. On 2, 1, 3, 2, 5 the right array reads 2, 2, 4, 4, −1 and the left array reads −1, 0, −1, 2, −1.
2. Equals never pop, 7, 7, 7 answers −1 on both sides.
3. The falling run 5, 4, 3, 2, 1 keeps right all −1 and answers left −1, 0, 1, 2, 3, and the rising run 1, 2, 3 answers right 1, 2, −1 with left all −1.
4. The circular lane wraps one pass over 2n - 1 steps with no push after the first lap, so no element answers itself: 3, 1, 4, 0, 2 answers 2, 2, −1, 4, 0, 1, 2, 1 answers 1, −1, 1, and 5, 0, 5 answers −1, 2, −1, where an answer may wrap to a lower index than its query.
5. The oracle lane: a brute scan outward in each direction, and once around the circle, agrees on every fixture in every suite.

```
/// <summary>Next greater element on both sides, one monotonic stack pass per direction.</summary>
public static class NextGreater
{
    // pops are answers: the popper is the poppee's next greater in the scan direction
    public static (int[] Right, int[] Left) Both(long[] a, Meter? meter = null)
    {
        int n = a.Length;
        var right = new int[n];
        Array.Fill(right, -1);
        Scan(a, right, +1, meter);
        var left = new int[n];
        Array.Fill(left, -1);
        Scan(a, left, -1, meter);
        return (right, left);
    }

    private static void Scan(long[] a, int[] answer, int dir, Meter? meter)
    {
        var stack = new Stack<int>();
        for (int step = 0; step < a.Length; step++)
        {
            int i = dir > 0 ? step : a.Length - 1 - step;
            while (stack.Count > 0 && a[stack.Peek()] < a[i])
            {
                answer[stack.Pop()] = i;
                if (meter != null)
                    meter.Pops++;
            }
            stack.Push(i);
            if (meter != null)
                meter.Pushes++;
        }
    }
}
```

listing 696: one pass per direction, pops are answers, survivors stay −1 (source: dsa/samples/src/Ch21/Ranges.cs, lines 199-232)

On 2, 1, 3, 2, 5 the right scan pops 1 and 0 when 3 arrives and 3 and 2 when 5 arrives, giving Right 2, 2, 4, 4, −1, and the left pass gives −1, 0, −1, 2, −1. Brute force agrees with both sides on seeded arrays. This scan is a workhorse beyond its own question: histogram rectangles, stock spans and trapping rainwater all reduce to it.

The six sibling builds carry the two directional scans plus a circular walk the C# ground truth stops short of:

```c
// one pass: dir +1 walks left to right filling right answers, dir -1
// walks right to left filling left answers
static void scan(const long long *a, int count, int *answer, int dir) {
  int stack[MAX_VALUES], top = 0;
  for (int i = 0; i < count; i++)
    answer[i] = -1;
  for (int step = 0; step < count; step++) {
    int i = dir > 0 ? step : count - 1 - step;
    while (top > 0 && a[stack[top - 1]] < a[i])
      answer[stack[--top]] = i; // the popper answers the poppee
    stack[top++] = i;
  }
}

static void both(const long long *a, int count, int *right, int *left) {
  scan(a, count, right, +1);
  scan(a, count, left, -1);
}

// wraparound next greater: one stack pass over 2n - 1 steps, no pushes
// after the first lap so answers never self-reference
static void circular(const long long *a, int count, int *answer) {
  int stack[MAX_VALUES], top = 0;
  for (int i = 0; i < count; i++)
    answer[i] = -1;
  for (int step = 0; step < 2 * count - 1; step++) {
    int i = step % count;
    while (top > 0 && a[stack[top - 1]] < a[i])
      answer[stack[--top]] = i;
    if (step < count)
      stack[top++] = i;
  }
}
```

listing 697: c, one stack pass per direction, the circular walk over 2n - 1 steps (source: dsa/samples-c/src/Ch21/nextgreater.c, lines 19-51)

```
func scanNextGreater(a []int64, answer []int, dir int, meter *WindowMeter) {
  var stack []int
  for step := 0; step < len(a); step++ {
    i := step
    if dir < 0 {
      i = len(a) - 1 - step
    }
    for len(stack) > 0 && a[stack[len(stack)-1]] < a[i] {
      answer[stack[len(stack)-1]] = i
      stack = stack[:len(stack)-1]
      if meter ≠ nil {
        meter.Pops++
      }
    }
    stack = append(stack, i)
    if meter ≠ nil {
      meter.Pushes++
    }
  }
}

// NextGreaterCircular answers next greater with wraparound: one pass
// over 2n-1 steps over the doubled array, pushing indices only during
// the first n. Strict comparison keeps an element from answering
// itself, and answers may wrap to a lower index than the query.
func NextGreaterCircular(a []int64) []int {
  n := len(a)
  answer := make([]int, n)
  for i := range answer {
    answer[i] = -1
  }
  var stack []int
  for step := 0; step < 2*n-1; step++ {
    i := step % n
    for len(stack) > 0 && a[stack[len(stack)-1]] < a[i] {
      answer[stack[len(stack)-1]] = i
      stack = stack[:len(stack)-1]
    }
    if step < n {
      stack = append(stack, i)
    }
  }
  return answer
}
```

listing 698: go, the directional scan and the circular wrap, strict greater pops (source: dsa/samples-go/ch21/nextgreater.go, lines 22-65)

```
  static final int MAX_VALUES = 8;

  // one pass: dir +1 walks left to right filling right answers, dir -1
  // walks right to left filling left answers
  static void scan(long[] a, int[] answer, int dir) {
    int[] stack = new int[MAX_VALUES];
    int top = 0;
    for (int i = 0; i < a.length; i++)
      answer[i] = -1;
    for (int step = 0; step < a.length; step++) {
      int i = dir > 0 ? step : a.length - 1 - step;
      while (top > 0 && a[stack[top - 1]] < a[i])
        answer[stack[--top]] = i; // the popper answers the poppee
      stack[top++] = i;
    }
  }

  static int[] right, left;

  static void both(long[] a) {
    right = new int[a.length];
    left = new int[a.length];
    scan(a, right, +1);
    scan(a, left, -1);
  }

  // wraparound next greater: one stack pass over 2n - 1 steps, no pushes
  // after the first lap so answers never self-reference
  static int[] circular(long[] a) {
    int[] answer = new int[a.length];
    int[] stack = new int[MAX_VALUES];
    int top = 0;
    for (int i = 0; i < a.length; i++)
      answer[i] = -1;
    for (int step = 0; step < 2 * a.length - 1; step++) {
      int i = step % a.length;
      while (top > 0 && a[stack[top - 1]] < a[i])
        answer[stack[--top]] = i;
      if (step < a.length)
        stack[top++] = i;
    }
    return answer;
  }
```

listing 699: java, one dir-flagged stack pass for both directions, the circular walk over 2n - 1 steps (source: dsa/samples-java/src/Ch21/Nextgreater.java, lines 16-58)

```
function scan(a, answer, dir, meter = null) {
  const stack = [];
  for (let step = 0; step < a.length; step++) {
    const i = dir > 0 ? step : a.length - 1 - step;
    while (stack.length > 0 && a[stack[stack.length - 1]] < a[i]) {
      answer[stack.pop()] = i;
      if (meter =/= null) meter.pops++;
    }
    stack.push(i);
    if (meter =/= null) meter.pushes++;
  }
}

export function nextGreaterBoth(a, meter = null) {
  const n = a.length;
  const right = new Array(n).fill(-1);
  scan(a, right, 1, meter);
  const left = new Array(n).fill(-1);
  scan(a, left, -1, meter);
  return { right, left };
}

// wraparound next greater: indices visited 0..n-1 then 0..n-2, pushes
// only on the first lap so no element pops on its own second visit
export function circularNextGreater(a) {
  const n = a.length;
  const answer = new Array(n).fill(-1);
  const stack = [];
  for (let step = 0; step < 2 * n - 1; step++) {
    const i = step % n;
    while (stack.length > 0 && a[stack[stack.length - 1]] < a[i]) {
      answer[stack.pop()] = i;
    }
    if (step < n) stack.push(i);
  }
  return answer;
}
```

listing 700: javascript, the scan, the both sides wrapper, the circular wrap (source: dsa/samples-js/src/ch21-nextgreater.mjs, lines 5-41)

```python
def scan(a, answer, direction):
    stack = []
    for step in range(len(a)):
        i = step if direction > 0 else len(a) - 1 - step
        while stack and a[stack[-1]] < a[i]:  # strict greater pops
            answer[stack.pop()] = i
        stack.append(i)


def both(a):
    right = [-1] * len(a)
    scan(a, right, 1)
    left = [-1] * len(a)
    scan(a, left, -1)  # the same discipline, walked right to left
    return right, left


def circular(a):
    # one pass over 2n - 1 steps; answers may wrap to a lower index
    size = len(a)
    answer = [-1] * size
    stack = []
    for step in range(2 * size - 1):
        i = step % size
        while stack and a[stack[-1]] < a[i]:
            answer[stack.pop()] = i
        if step < size:
            stack.append(i)
    return answer
```

listing 701: python, both directions from one scan, the circular walk beside it (source: dsa/samples-py/src/Ch21/nextgreater.py, lines 14-42)

```
local function scan(a, answer, backward, meter)
  local stack = {} -- indices, top at the end
  local top = 0
  for step = 1, #a do
    local i = backward and #a - step + 1 or step
    while top > 0 and a[stack[top]] < a[i] do
      answer[stack[top]] = i - 1 -- the popper answers the poppee, 0-based
      top = top - 1
      if meter then meter.pops = meter.pops + 1 end
    end
    top = top + 1
    stack[top] = i
    if meter then meter.pushes = meter.pushes + 1 end
  end
end

-- right and left arrays of 0-based indices, -1 when no greater exists
local function both(a, meter)
  local n = #a
  local right, left = {}, {}
  for i = 1, n do right[i], left[i] = -1, -1 end
  scan(a, right, false, meter)
  scan(a, left, true, meter)
  return right, left
end

-- circular wraparound: one pass over 2n - 1 steps, no self reference
local function circular(a)
  local n = #a
  local answer = {}
  for i = 1, n do answer[i] = -1 end
  local stack, top = {}, 0
  for step = 0, 2 * n - 2 do
    local i = step % n + 1
    while top > 0 and a[stack[top]] < a[i] do
      answer[stack[top]] = i - 1
      top = top - 1
    end
    if step < n then
      top = top + 1
      stack[top] = i
    end
  end
  return answer
end
```

listing 702: lua, the scans over 1-based stacks answering in 0-based indices (source: dsa/samples-lua/ch21_nextgreater.lua, lines 6-50)

Every tree runs the same discipline twice, one pass left to right for the right array and one right to left for the left array, and the circular variant repeats the walk over 2n - 1 steps, pushing only during the first lap so a second visit never answers itself. C, Go, Java, JavaScript, and Python pick the visit index from a direction flag, Lua from a backward flag, and the Go window leaves out only the wrapper that fills the two answer arrays, it sits a few lines above the scan in the same file. Lua alone shifts its 1-based stack positions back to 0-based answers, and its meter row pins the push count at one per index per pass, 10 over both directions of the 5 element fixture. The circular lane belongs to the six sibling trees alone, and every brute oracle agrees with every answer they give.

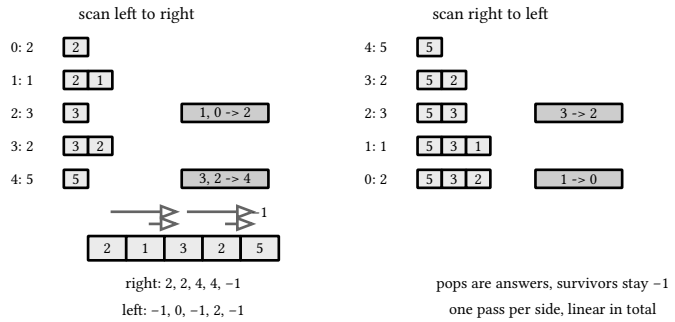


figure 185: next greater both ways, the stack under each arrival, every pop recording one answer

21.7. across the seven languages

The build sizes count non-comment source lines over this chapter's six featured files per language. The C# row is its whole `Ranges.cs`, including the lazy segment tree with range updates the other six replace with point-update builds and the monotonic deque with both scans, plus the prefix sums file:

language	build SLOC	dependency	boundary note
c	471	libc only	static tables sized 4n and log n, a naive twin answers every query, sum and min trees in one build, the deque under two cursors
c#	221	bcl only	the lazy range-add tree is the deep cut, meters count reads and nodes, BitOperations behind the level counts, prefix sums aboard
go	247	none	zero imports, the O(n) fenwick build, one fold drives sum and min queries, the deque resliced off both ends
java	717	jdk 27 stdlib	both segment tree shapes in one file, the recursive 4n arrays beside the iterative nested class, the fenwick shadow array re-checked after every op, rmq answering with indices
javascript	176	node stdlib	the iterative segment tree over doubled indices, no recursion in any of the six builds
python	368	stdlib only	shadow lists agree after every mutation, levels opened to inspection, log2 by hand, the k = 1 pops meter
lua	462	lib.lua harness	the segment tree takes fold and identity as arguments, max-integer the min identity, 1-based tables with a 0-based face

The two monotonic sections measured the same way, with the C# column counting each region of the one shared `Ranges.cs` and the shared Meter booked on the maxima row:

section	c	c#	go	java	javascript	python	lua
sliding window maxima	92	37	32	141	33	60	72
next greater, both sides	113	31	54	119	33	72	105

The next greater spread carries the circular walk, the 6 sibling files ship it and the C# region stops at the two directional scans, which is most of the distance down to 31.

sources: learn.microsoft.com, `LinkedList<int>` as the deque substrate, `Stack<int>` for the scans, `BitOperations.Log2` and `PopCount` behind the level and read counts, `Array.Fill`, accessed 2026-09-12. Sample behavior verified by `make verify-csharp`, 17 tests in chapter 21 of the samples suite. The seven-language layer verifies the same way: 6 C programs with 108 embedded checks under `make verify-c`, 6 Ch21 java programs with 135 checks under `run-java-samples`, 18 Go tests, 18 `node --test` cases, 93 Python checks across 6 files, and 29 Lua checks under `run.lua`.

22. coordinate compression and intervals

Array algorithms want dense indices and the world hands out sparse coordinates. Two ideas close the gap: compress the coordinates to their ranks so a billion-wide domain fits a small array, and do interval algebra on endpoints instead of on the cells between them. This chapter builds both in seven languages, and one convention is pinned before anything else. Intervals here are half-open, start inclusive and end exclusive, so an interval ending at t and one starting at t share the single point t and never merge or stack. The c# and lua suites pin exactly that. The c, go, java, javascript, and python suites pin the touching-fuses reading for merging, go even names the type Closed, so a touching pair becomes one interval there. Every section below names which reading its listings use, and the matrix anchor, (1,3), (2,6), (8,10) merging to two intervals, agrees in all seven languages.

22.1. coordinate compression

Sort the raw values, drop duplicates, then rank every value by binary search into that sorted domain. Three invariants survive the whole construction: order is preserved, a smaller raw value always gets a smaller rank, ranks are dense, the largest rank is one under the distinct count, and the lookup is lossless, domain of rank gives back the original value. The sort pays n log n once and each rank costs log k. It wins whenever coordinates are huge but few, chapter 21′s fenwick tree, this chapter's difference arrays, and sweep arrays in chapter 24 all want indices, not values.

The dry run: the fixture is 100, 10, 100, 40, 10, asserted by the C# suite with lua on the same numbers, while C and java rank 100, −50, 100, 7, −50, 30 to 3, 0, 3, 1, 0, 2.

1. Distinct then order builds the domain: 10, 40, 100, three slots carrying ranks 0, 1, 2.
2. The rank lookup walks the raw list in input order, one search per value: 100 reads slot 2, 10 reads slot 0, 100 slot 2 again, 40 slot 1, 10 slot 0, landing 2, 0, 2, 1, 0.
3. Duplicates never split: both 10s hold rank 0 and both 100s rank 2, one slot per distinct value.
4. The reverse lookup is lossless, domain[2] = 100 and domain[0] = 10, the round trip the suite pins over negatives through 12.
5. The single value 7 compresses to rank 0 over the one-slot domain 7, and the empty list compresses to nothing.

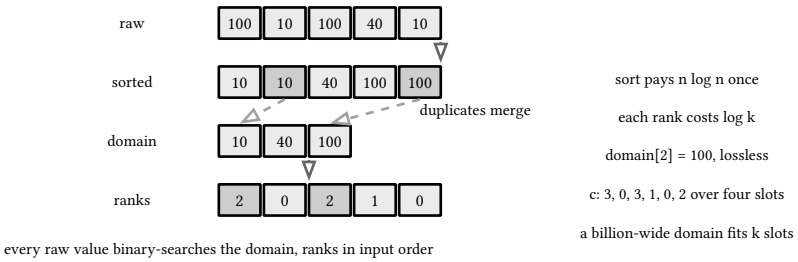


figure 186: the run as a pipeline, raw values, sorted with duplicates, the deduped domain, then every raw value ranked in input order

2, 0, 2, 1, 0 over 10, 40, 100 is the pinned pair and the listings below sort, dedupe, and rank in seven languages.

```c
static int compress(const int *a, int len) {
  int buf[16];
  for (int i = 0; i < len; i++)
    buf[i] = a[i];
  qsort(buf, (size_t)len, sizeof(int), cmp_int);
  uniq_n = 0;
  for (int i = 0; i < len; i++)
    if (uniq_n == 0 || buf[i] != uniq[uniq_n - 1])
      uniq[uniq_n++] = buf[i];
  return uniq_n;
}

// index of v in uniq (v guaranteed present)
static int lower_idx(int v) {
  int lo = 0, hi = uniq_n;
  while (lo < hi) {
    int mid = lo + (hi - lo) / 2;
    if (uniq[mid] < v)
      lo = mid + 1;
    else
      hi = mid;
  }
  return lo;
}
```

listing 703: c, qsort then an in-place unique scan, lower bound ranks each value (source: dsa/samples-c/src/Ch22/coordcompress.c, lines 26-49)

```go
func Compress(values []int64) []int {
  sorted := slices.Clone(values)
  slices.Sort(sorted)
  sorted = slices.Compact(sorted)
  out := make([]int, len(values))
  for i, v := range values {
    out[i], _ = slices.BinarySearch(sorted, v)
  }
  return out
}
```

listing 704: go, slices.sort, slices.compact, and slices.binarysearch are the whole build (source: dsa/samples-go/ch22/coordcompress.go, lines 10-19)

```
  static int[] uniq = new int[16];
  static int uniqCount;

  // build the sorted unique table from the input
  static int compress(int[] a, int len) {
    int[] buf = new int[len];
    System.arraycopy(a, 0, buf, 0, len);
    java.util.Arrays.sort(buf, 0, len);
    uniqCount = 0;
    for (int i = 0; i < len; i++)
      if (uniqCount == 0 || buf[i] != uniq[uniqCount - 1])
        uniq[uniqCount++] = buf[i];
    return uniqCount;
  }

  // index of v in uniq (v guaranteed present)
  static int lowerIdx(int v) {
    int lo = 0, hi = uniqCount;
    while (lo < hi) {
      int mid = lo + (hi - lo) / 2;
      if (uniq[mid] < v)
        lo = mid + 1;
      else
        hi = mid;
    }
    return lo;
  }

  // the py lane: ranks list for arbitrary input, uniques reported fresh
  static List<Integer> pyUniques;

  static List<Integer> compressRanks(int[] values) {
    List<Integer> uniques = new ArrayList<>(java.util.Arrays.stream(values)
        .distinct().sorted().boxed().toList());
    pyUniques = uniques;
    List<Integer> ranks = new ArrayList<>();
    for (int x : values) {
      int lo = 0, hi = uniques.size();
      while (lo < hi) { // first index with uniques[i] ≥ x
        int mid = (lo + hi) / 2;
        if (uniques.get(mid) < x)
          lo = mid + 1;
        else
          hi = mid;
      }
      ranks.add(lo);
    }
    return ranks;
  }
```

listing 705: java, arrays.sort then a dedupe scan, a hand lower bound, a stream-distinct lane beside (source: dsa/samples-java/src/Ch22/Coordcompress.java, lines 19-67)

```
    public static (int[] Compressed, long[] Domain) CoordCompress(IReadOnlyList<long> values)
    {
        long[] domain = values.Distinct().OrderBy(v => v).ToArray();
        var index = new Dictionary<long, int>(domain.Length);
        for (int i = 0; i < domain.Length; i++)
            index[domain[i]] = i;
        var compressed = new int[values.Count];
        for (int i = 0; i < values.Count; i++)
            compressed[i] = index[values[i]];
        return (compressed, domain);
    }
```

listing 706: c#, a distinct ordered domain plus a dictionary from value to rank (source: dsa/samples/src/Ch22/Compression.cs, lines 13-23)

```
export function compress(coords) {
  const unique = [...new Set(coords)].sort((a, b) => a - b);
  // rank of every input coordinate
  const mapped = coords.map((v) => lowerBound(unique, v));
  return { unique, mapped };
}

// rank of an absent value: where it would sit in the unique list
export function lowerBound(a, t) {
  let lo = 0;
  let hi = a.length;
  while (lo < hi) {
    const mid = (lo + hi) >> 1;
    if (a[mid] < t) lo = mid + 1;
    else hi = mid;
  }
  return lo;
}
```

listing 707: javascript, a set for the uniques, a hand lower bound for the ranks (source: dsa/samples-js/src/ch22-coordcompress.mjs, lines 5-22)

```
def compress(values):
    uniques = sorted(set(values))
    ranks = {}

    def rank(x):
        lo, hi = 0, len(uniques)
        while lo < hi:  # first index with uniques[i] ≥ x
            mid = (lo + hi) // 2
            if uniques[mid] < x:
                lo = mid + 1
            else:
                hi = mid
        return lo

    for x in values:
        ranks[x] = rank(x)
    return uniques, [ranks[x] for x in values]
```

listing 708: python, sorted(set(...)) builds the domain, a closure ranks each value (source: dsa/samples-py/src/Ch22/coordcompress.py, lines 14-30)

```lua
local function compress(values)
  local domain = {table.unpack(values)}
  table.sort(domain)
  local uniq = {}
  for _, v in ipairs(domain) do
    if #uniq == 0 or uniq[#uniq] ~= v then uniq[#uniq + 1] = v end
  end
  local ranks = {}
  for i, v in ipairs(values) do
    -- lower bound over the unique domain
    local lo, hi = 1, #uniq + 1
    while lo < hi do
      local mid = lo + (hi - lo) // 2
      if uniq[mid] < v then lo = mid + 1 else hi = mid end
    end
    ranks[i] = lo - 1
  end
  return ranks, uniq
end
```

listing 709: lua, table.sort, a dedupe scan, a 1-based lower bound shifted back to rank 0 (source: dsa/samples-lua/ch22_coordcompress.lua, lines 8-26)

All seven suites pin the same three invariants on their own fixtures. C and java compress 100, −50, 100, 7, −50, 30 into ranks 3, 0, 3, 1, 0, 2 over the domain −50, 7, 30, 100. C# and lua share the fixture 100, 10, 100, 40, 10 landing at 2, 0, 2, 1, 0 over 10, 40, 100, and C# asserts the lossless round trip on negatives −5 through 12. Go uses coordinates up to 100000000, python proves negatives compress too, java's second lane reaches a 1000000 coordinate through `distinct().sorted()` on a stream, and javascript's lowerBound also answers for absent values, the insertion point, which is the same search chapter 14 built. Python's sorted(set(values)) is the whole trick in one expression, and the domain lookup beside it is a dictionary in C#, a rank closure in python, a linear-indexed array in C and java.

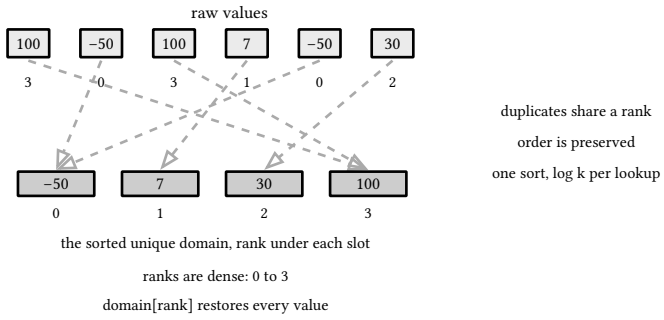


figure 187: compression on the c fixture, six raw values ranked into a four slot domain, duplicates share a rank

22.2. merging intervals

Sort by start, then fold left to right keeping one open cluster. Each interval either extends the cluster's right end or starts a new cluster, so containment and overlap collapse in a single linear pass after the n log n sort. The one decision that changes answers is the tie at a shared endpoint. Under the half-open reading a pair like (1,3) and (3,6) stays two intervals, under the touching-fuses reading it becomes (1,6), and both are defensible as long as the code states which one it means.

The dry run: the fixture is (1,3), (2,6), (8,10), asserted by the C# suite under the strict fold, with the touching pair (1,3), (3,6) pinned as two intervals.

1. Sorting by start leaves the fixture in order, and the scrambled input (8,10), (2,6), (1,3) folds to the same output.
2. Carry (1,3) open; the next start 2 < 3 overlaps, so the end widens to max(3, 6) = 6 and the cluster carries (1,6).
3. The next start 8 fails 8 < 6, so (1,6) is emitted and (8,10) opens anew: two intervals out, the pinned count.

4. The touching pair turns on the same comparison, 3 < 3 is false, no overlap, two intervals stay two under the half-open reading while C, go, java, javascript, and python fuse them.
5. Insert (4,9) into (1,3), (8,10), (15,18): (1,3) ends 3 <= 4 and passes through, (8,10) strictly overlaps and widens the item to (4,10), and (15,18) starts past 10, emitting (1,3), (4,10), (15,18).

interval	test	action	carried
(1,3)	first	open the cluster	(1,3)
(2,6)	2 < 3	widen: max(3, 6) = 6	(1,6)
(8,10)	8 < 6 false	emit, open anew	(8,10)
(3,6)	3 < 3 false	emit, touching stays two	(3,6)

Two intervals out of three is the pinned merge and the listings below fold in seven languages.

```c
static int merge(Iv *iv, int len) {
  qsort(iv, (size_t)len, sizeof(Iv), cmp_iv);
  int k = 0;
  for (int i = 0; i < len; i++) {
    if (k && iv[i].start ≤ iv[k - 1].end) {
      if (iv[i].end > iv[k - 1].end)
        iv[k - 1].end = iv[i].end;
    } else {
      iv[k++] = iv[i];
    }
  }
  return k;
}

// insert one interval into a merged list, merging as needed
static int insert_one(Iv *iv, int len, Iv add) {
  iv[len++] = add;
  return merge(iv, len);
}
```

listing 710: c, the fold after qsort, insert appends one interval and remerges (source: dsa/samples-c/src/Ch22/mergeintervals.c, lines 27-45)

```go
func MergeIntervals(intervals []Closed) []Closed {
  if len(intervals) == 0 {
    return nil
  }
  sorted := slices.Clone(intervals)
  slices.SortFunc(sorted, func(a, b Closed) int { return a.Start - b.Start })
  merged := []Closed{sorted[0]}
  for _, iv := range sorted[1:] {
    last := &merged[len(merged)-1]
    if iv.Start <= last.End { // touching or overlapping: absorb
      if iv.End > last.End {
        last.End = iv.End
      }
      continue
    }
    merged = append(merged, iv)
  }
  return merged
}
```

listing 711: go, closed intervals, touching or overlapping absorbs (source: dsa/samples-go/ch22/mergeintervals.go, lines 13-31)

```
  record Iv(int start, int end) {}

  // merge sorted-or-unsorted input
  static List<Iv> merge(List<Iv> in) {
    List<Iv> sorted = new ArrayList<>(in);
    sorted.sort((a, b) -> Integer.compare(a.start, b.start));
    List<Iv> out = new ArrayList<>();
    for (Iv iv : sorted) {
      if (!out.isEmpty() && iv.start() ≤ out.get(out.size() - 1).end()) {
        Iv last = out.get(out.size() - 1);
        if (iv.end() > last.end())
          out.set(out.size() - 1, new Iv(last.start(), iv.end()));
      } else {
        out.add(iv);
      }
    }
    return out;
  }

  // insert one interval into a merged list, merging as needed
  static List<Iv> insertOne(List<Iv> in, Iv add) {
    List<Iv> combined = new ArrayList<>(in);
    combined.add(add);
    return merge(combined);
  }
```

listing 712: java, a record interval, sort by start then fold, insert appends and remerges (source: dsa/samples-java/src/Ch22/Mergeintervals.java, lines 18-42)

```
    public static List<(long Start, long End)> MergeIntervals(IEnumerable<(long Start, long End)> items)
    {
        var sorted = new List<(long Start, long End)>(items);
        sorted.Sort((a, b) => a.Start ≠ b.Start ? a.Start.CompareTo(b.Start) : a.End.CompareTo(b.End));
        var merged = new List<(long Start, long End)>();
        foreach ((long start, long end) in sorted)
        {
            if (merged.Count > 0 && start < merged[^1].End)
                merged[^1] = (merged[^1].Start, Math.Max(merged[^1].End, end));
            else
                merged.Add((start, end));
        }
        return merged;
    }
```

listing 713: c#, a strict start < end fold, touching pairs stay separate (source: dsa/samples/src/Ch22/Compression.cs, lines 26-39)

```
export function mergeIntervals(intervals) {
  const sorted = [...intervals].sort((a, b) => a[0] - b[0]);
  const out = [];
  for (const [s, e] of sorted) {
    const last = out[out.length - 1];
    if (last && s ≤ last[1]) last[1] = Math.max(last[1], e); // overlap
    else out.push([s, e]);
  }
  return out;
}
```

listing 714: javascript, sort by start then fold pairs in place (source: dsa/samples-js/src/ch22-mergeintervals.mjs, lines 4-13)

```python
def merge(intervals):
    out = []
    for start, end in sorted(intervals):
        if out and start ≤ out[-1][1]:  # touches or overlaps: fuse
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])
    return out


def insert(intervals, new):
    # insert into an already merged list, then re-merge the seam
    combined = merge([*intervals, new])
    return combined
```

listing 715: python, sorted tuples feed the fold, insert is one re-merge (source: dsa/samples-py/src/Ch22/mergeintervals.py, lines 13-26)

```lua
local function merge_intervals(intervals)
  local sorted = sort_by_start(intervals)
  local out = {}
  for _, iv in ipairs(sorted) do
    local last = out[#out]
    if last and iv[1] < last[2] then
      if iv[2] > last[2] then last[2] = iv[2] end
    else
      out[#out + 1] = {iv[1], iv[2]}
    end
  end
  return out
end
```

listing 716: lua, the strict fold, the half-open reading stated in the header (source: dsa/samples-lua/ch22_mergeintervals.lua, lines 15-27)

The anchor fixture agrees everywhere, (1,3), (2,6), (8,10) becomes (1,6) and (8,10) in all seven suites. The touching pair splits them, C, go, java, javascript, and python fuse (1,2) and (2,3) into (1,3) while C# and lua keep two, and go and lua each document the choice at the type or file level. Inserting into an already merged list shows three shapes: C and java append the newcomer and rerun the merge, python and lua rebuild the combined list the same way, and C#, go, and javascript walk the sorted list once, widening the carried interval across every overlap and emitting it before the first interval that starts past it. Go's insert test pins (4,9) into (1,3),(8,10) giving (1,3),(4,10), and C# pins the same seam with a 15,18 tail.

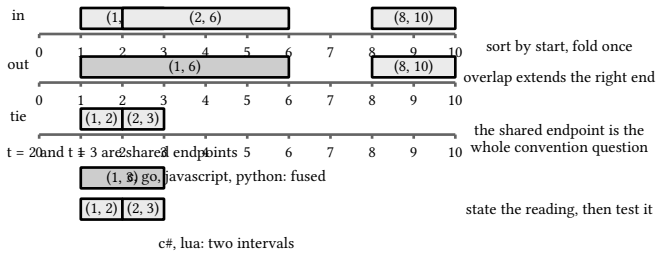


figure 188: the anchor fixture fusing to two intervals, then one touching pair with both readings, fused or separate

22.3. counting overlaps

Two questions about the same intervals. How many cover a point is a linear scan with the same endpoint test as the merge, half-open drops the interval exactly at its end. The maximum simultaneous count is a sweep over events,

plus one at every start and minus one at every end, sorted by coordinate, and the tie rule at equal coordinates is again the whole convention: ends before starts means touching intervals never stack, starts before ends means they do.

The dry run: the fixtures are the point probes and the sweep counts pinned by the C# suite, while C and java trace their own four intervals live at 1, 2, 1, 3, 2, 1, 0.

1. The probe at point 3 over (1,4), (2,6), (5,8), (3,7) counts three covers, 1 <= 3 < 4, 2 <= 3 < 6, 3 <= 3 < 7, and (5,8) has not started, landing 3.
2. At point 4 the first interval drops out because 4 < 4 fails, leaving 2 covers, and at 99 nothing covers.
3. The sweep fixture (1,4), (2,5), (3,6), (7,9) posts eight events: plus one at 1, 2, 3, 7 and minus one at 4, 5, 6, 9.
4. No coordinate is shared, so the tie rule never fires, and the live count steps 1, 2, 3, 2, 1, 0, 1, 0, peaking at 3 when the third start lands.
5. Touching (1,3) and (3,5) is the tie rule's proof: the end at 3 sorts first and live never passes 1.

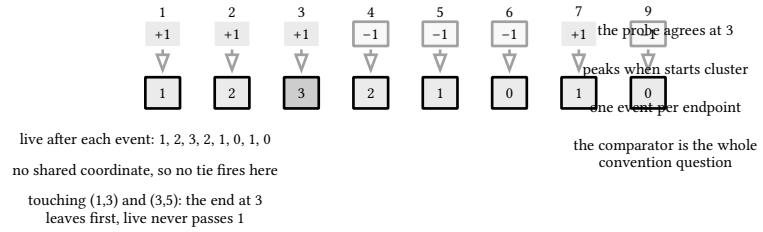


figure 189: the sweep as a rail of eight events, plus one and minus one, the live count under each event, the peak shaded

Peak 3 by probe and by sweep, and the listings below count both questions in seven languages.

```c
static int active_at(int t) {
  int c = 0;
  for (int i = 0; i < NIV; i++)
    if (starts[i] <= t && t < ends[i])
      c++;
  return c;
}

// max simultaneous via sorted events; at equal times ends leave first
static int sweep_max(int *at_time) {
  Event ev[2 * NIV];
  for (int i = 0; i < NIV; i++) {
    ev[2 * i] = (Event){starts[i], +1};
    ev[2 * i + 1] = (Event){ends[i], -1};
  }
  for (int i = 1; i < 2 * NIV; i++)
    for (int j = i; j > 0; j--) {
      int later =
          ev[j - 1].time > ev[j].time ||
          (ev[j - 1].time == ev[j].time && ev[j - 1].delta > ev[j].delta);
      if (!later)
        break;
      Event t = ev[j - 1];
      ev[j - 1] = ev[j];
      ev[j] = t;
    }
  int cur = 0, best = 0;
  *at_time = -1;
  for (int i = 0; i < 2 * NIV; i++) {
    cur += ev[i].delta;
    if (cur > best) {
      best = cur;
      *at_time = ev[i].time;
    }
  }
  return best;
}
```

listing 717: c, the point probe, then events insertion-sorted with ends first (source: dsa/samples-c/src/Ch22/overlaps.c, lines 28-64)

```go
func OverlapsAt(intervals []HalfOpen, at int) int {
  count := 0
  for _, iv := range intervals {
    if iv.Start <= at && at < iv.End {
      count++
    }
  }
  return count
}

// MaxSimultaneous sweeps endpoints: every start increments the live
// count before the simultaneity window closes, every end decrements
// after, so touching intervals never count as simultaneous.
func MaxSimultaneous(intervals []HalfOpen) int {
  type event struct {
    at     int
    effect int
  }
  events := make([]event, 0, 2*len(intervals))
  for _, iv := range intervals {
    events = append(events, event{iv.Start, 1}, event{iv.End, -1})
  }
  slices.SortFunc(events, func(a, b event) int {
    if a.at != b.at {
      return a.at - b.at
    }
    return a.effect - b.effect // ends (-1) before starts (+1)
  })
  live, max := 0, 0
  for _, e := range events {
    live += e.effect
    if live > max {
      max = live
    }
  }
  return max
}
```

listing 718: go, a halfopen type, point count and sweep, ends before starts (source: dsa/samples-go/ch22/overlaps.go, lines 12-48)

```
  static final int NIV = 4;

  static final int[] starts = {1, 2, 5, 5};
  static final int[] ends = {4, 6, 8, 7};

  // how many [s, e) intervals contain the point t
  static int activeAt(int t) {
    int c = 0;
    for (int i = 0; i < NIV; i++)
      if (starts[i] ≤ t && t < ends[i])
        c++;
    return c;
  }

  record Event(int time, int delta) {}

  static int sweepAt = -1;

  // max simultaneous via sorted events; at equal times ends leave first
  static int sweepMax() {
    List<Event> ev = new ArrayList<>();
    for (int i = 0; i < NIV; i++) {
      ev.add(new Event(starts[i], +1));
      ev.add(new Event(ends[i], -1));
    }
    // a later sort key runs later: at equal times -1 sorts before +1
    ev.sort((a, b) -> a.time() ≠ b.time()
        ? Integer.compare(a.time(), b.time())
        : Integer.compare(a.delta(), b.delta()));
    int cur = 0, best = 0;
    sweepAt = -1;
    for (Event e : ev) {
      cur += e.delta();
      if (cur > best) {
        best = cur;
        sweepAt = e.time();
      }
    }
    return best;
  }
```

listing 719: java, the half-open point probe, a record event, the comparator ends first, the peak time kept (source: dsa/samples-java/src/Ch22/Overlaps.java, lines 19-58)

```
    /// <summary>How many intervals cover the point under half-open semantics.</summary>
    public static int CountAt(IEnumerable<(long Start, long End)> items, long point) =>
        items.Count(iv => iv.Start ≤ point && point < iv.End);

    /// <summary>
    /// Maximum simultaneous intervals via a sweep: +1 at every start, -1 at every end,
    /// ends sort before starts at the same coordinate so touching ranges never stack.
    /// </summary>
    public static int MaxSimultaneous(IEnumerable<(long Start, long End)> items)
    {
        var events = new List<(long At, int Delta)>();
        foreach ((long start, long end) in items)
        {
            events.Add((start, 1));
            events.Add((end, -1));
        }
        events.Sort((a, b) => a.At ≠ b.At ? a.At.CompareTo(b.At) : a.Delta.CompareTo(b.Delta));
        int live = 0, best = 0;
        foreach ((_, int delta) in events)
        {
            live += delta;
            best = Math.Max(best, live);
        }
        return best;
    }
```

listing 720: c#, countat under half-open, the sweep orders ties by delta (source: dsa/samples/src/Ch22/Compression.cs, lines 64-88)

```
export function overlapsAt(intervals, x) {
  return intervals.filter(([s, e]) => s ≤ x && x ≤ e).length;
}

// sweep: sort starts and ends independently; the running count is the
// number of intervals covering each event coordinate. starts sort before
// ends at the same coordinate, so touching intervals count as overlapping
export function maxSimultaneous(intervals) {
  const starts = intervals.map(([s]) => s).sort((a, b) => a - b);
  const ends = intervals.map(([, e]) => e).sort((a, b) => a - b);
  let count = 0;
  let max = 0;
  let i = 0;
  let j = 0;
  while (i < starts.length) {
    if (starts[i] ≤ ends[j]) {
      count++;
      max = Math.max(max, count);
      i++;
    } else {
      count--;
      j++;
    }
  }
  return max;
}
```

listing 721: javascript, closed point probe, two sorted arrays merged start-first (source: dsa/samples-js/src/ch22-overlaps.mjs, lines 5-30)

```python
def overlaps_at(intervals, point):
    return sum(1 for lo, hi in intervals if lo ≤ point ≤ hi)


def max_simultaneous(intervals):
    events = []
    for lo, hi in intervals:
        events.append((lo, 1))  # an opening
        events.append((hi, -1))  # a closing: at the same coordinate the
        # closing runs first (sorted tuples put -1 before +1)
    depth = peak = 0
    for _, delta in sorted(events):
        depth += delta
        peak = max(peak, depth)
    return peak
```

listing 722: python, sorted event tuples put −1 ahead of +1 at a tie (source: dsa/samples-py/src/Ch22/overlaps.py, lines 16-30)

```lua
-- how many half-open intervals [s, e) contain the point x
local function count_at(intervals, x)
  local c = 0
  for _, iv in ipairs(intervals) do
    if iv[1] ≤ x and x < iv[2] then c = c + 1 end
  end
  return c
end

-- max simultaneous: sweep endpoints, an end at x closes before a start
-- at x opens (half-open intervals never stack at the shared point)
local function max_simultaneous(intervals)
  local events = {}
  for _, iv in ipairs(intervals) do
    events[#events + 1] = {iv[1], 1} -- start
    events[#events + 1] = {iv[2], -1} -- end
  end
  table.sort(events, function(x, y)
    if x[1] ~= y[1] then return x[1] < y[1] end
    return x[2] < y[2] -- -1 sorts before +1 at the same coordinate
  end)
  local cur, best = 0, 0
  for _, e in ipairs(events) do
    cur = cur + e[2]
    if cur > best then best = cur end
  end
  return best
end
```

listing 723: lua, both passes over half-open pairs, the comparator ends first (source: dsa/samples-lua/ch22_overlaps.lua, lines 6-33)

The c and java suites pin the sweep on the same fixture, (1,4), (2,6), (5,8), (5,7) all half-open: the live count runs 1, 2, 1, 3, 2, 1, 0 and peaks at 3, java naming time 5 as the peak, with a brute-force max over point probes agreeing. Go peaks at 3 on (1,4), (2,5), (3,7), (6,8) and lua pins that touching (1,3) and (3,5) never stack. Python and javascript are the closed-reading outliers in this section, python counts the last closing point 9 as covered while its sweep still ties ends first, and javascript ties starts first, so its (1,3) and (3,5) count as simultaneous, java carrying the closed reading as a second lane beside its half-open one. Same code shape, opposite answer, entirely decided by one comparator line.

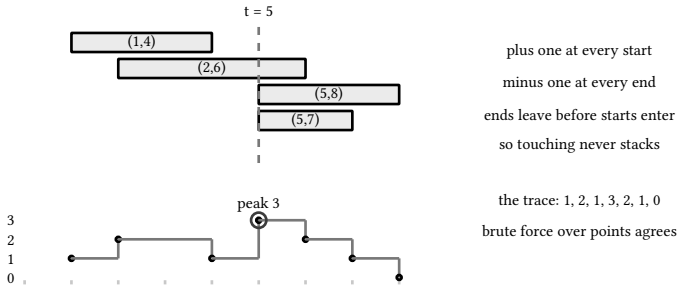


figure 190: the sweep over the c fixture, four half-open bars, the event trace with the peak of 3 at time 5

22.4. the difference array

Flip a stream of range adds into two point writes each. Adding delta over lo through hi writes plus delta at lo and minus delta at hi plus one, and a single prefix-sum pass materializes the final array, so m range adds over n cells cost m plus n instead of m times n. The invariant is the telescoping one, every cell's value is the running sum of writes to its left, and the array carries one extra slot so a range ending at the last cell still has somewhere to subtract.

The dry run: the fixture is a 6-cell array under three range adds, asserted by the C# suite at 2, 5, 5, 2, 2, −1.

1. Add(0, 2, 2) writes two slots: plus 2 at diff[0], minus 2 at diff[3].
2. Add(1, 4, 3) writes plus 3 at diff[1] and minus 3 at diff[5].
3. Add(3, 5, −1) writes minus 1 at diff[3] and plus 1 at diff[6], the extra seventh slot taking the subtract past the end.
4. The backing array now reads 2, 3, 0, −3, 0, −3, 1 over slots 0 through 6.
5. One prefix pass materializes the answer: 2, 2 + 3 = 5, 5, 5 - 3 = 2, 2, 2 - 3 = −1, pinned, and the seventh slot's running sum returns to 0.

op	plus at	minus at	diff now
add 2 over 0 to 2	diff[0] + 2	diff[3] - 2	2, 0, 0, −2, 0, 0, 0
add 3 over 1 to 4	diff[1] + 3	diff[5] - 3	2, 3, 0, −2, 0, −3, 0
add −1 over 3 to 5	diff[3] - 1	diff[6] + 1	2, 3, 0, −3, 0, −3, 1

2, 5, 5, 2, 2, −1 out of six writes and one pass, and the listings below range-add in seven languages.

```c
static void range_add(int lo, int hi, int delta) { // inclusive [lo, hi]
  diff[lo] += delta;
  if (hi + 1 < N)
    diff[hi + 1] -= delta;
}

static void materialize(int *out) {
  int run = 0;
  for (int i = 0; i < N; i++) {
    run += diff[i];
    out[i] = run;
  }
}
```

listing 724: c, two writes per range add, one running sum, inclusive bounds (source: dsa/samples-c/src/Ch22/diffarray.c, lines 20-32)

```
// Add applies delta to every cell in [lo, hi).
func (d *DiffArray) Add(lo, hi int, delta int64) {
  d.diff[lo] += delta
  d.diff[hi] -= delta
}

// Materialize runs the one prefix pass and returns the final array.
func (d *DiffArray) Materialize() []int64 {
  out := make([]int64, len(d.diff)-1)
  running := int64(0)
  for i := 0; i < len(out); i++ {
    running += d.diff[i]
    out[i] = running
  }
  return out
}
```

listing 725: go, add takes the half-open [lo, hi), one prefix pass (source: dsa/samples-go/ch22/diffarray.go, lines 15-30)

```
  static final int N = 8;

  static int[] diff = new int[N];

  static void rangeAdd(int lo, int hi, int delta) { // inclusive [lo, hi]
    diff[lo] += delta;
    if (hi + 1 < N)
      diff[hi + 1] -= delta;
  }

  static int[] materialize() {
    int[] out = new int[N];
    int run = 0;
    for (int i = 0; i < N; i++) {
      run += diff[i];
      out[i] = run;
    }
    return out;
  }

  static boolean eq(int[] got, int... want) {
    if (got.length != want.length)
      return false;
    for (int i = 0; i < want.length; i++)
      if (got[i] != want[i])
        return false;
    return true;
  }

  // the py lane: one extra slot absorbs the +1 edge, any size
  static int[] applyUpdates(int size, int[][] updates) {
    int[] d = new int[size + 1];
    for (int[] u : updates) {
      d[u[0]] += u[2];
      d[u[1] + 1] -= u[2]; // the effect stops after hi
    }
    int[] out = new int[size];
    int running = 0;
    for (int i = 0; i < size; i++) {
      running += d[i];
      out[i] = running;
    }
    return out;
  }
```

listing 726: java, two writes per add with the hi-plus-one guarded, one prefix pass, a naive-sweep lane (source: dsa/samples-java/src/Ch22/Diffarray.java, lines 15-58)

```
        public void Add(int from, int to, long delta) // inclusive range [from, to]
        {
            if (from < 0 || to ≥ Length || from > to)
                throw new ArgumentOutOfRangeException(nameof(from), "bad range");
            _diff[from] += delta;
            _diff[to + 1] -= delta;
        }

        public long[] Materialize()
        {
            var result = new long[Length];
            long running = 0;
            for (int i = 0; i < Length; i++)
            {
                running += _diff[i];
                result[i] = running;
            }
            return result;
        }
```

listing 727: c#, bounds checks throw, a plus-one backing array (source: dsa/samples/src/Ch22/Compression.cs, lines 107-125)

```
export function diffArray(n, ops) {
  const d = new Array(n + 1).fill(0);
  for (const [l, r, delta] of ops) {
    d[l] += delta;
    d[r + 1] -= delta;
  }
  const out = new Array(n);
  let run = 0;
  for (let i = 0; i < n; i++) {
    run += d[i];
    out[i] = run;
  }
  return out;
}
```

listing 728: javascript, one function from ops to the materialized array (source: dsa/samples-js/src/ch22-diffarray.mjs, lines 5-18)

```
class DiffArray:
    def __init__(self, size):
        self.diff = [0] * (size + 1)  # one extra slot absorbs the +1 edge

    def add(self, lo, hi, delta):  # inclusive bounds
        self.diff[lo] += delta
        self.diff[hi + 1] -= delta  # the effect stops after hi

    def materialize(self, size):
        out = []
        running = 0
        for i in range(size):
            running += self.diff[i]
            out.append(running)
        return out
```

listing 729: python, inclusive bounds, hi plus one, class and helper (source: dsa/samples-py/src/Ch22/diffarray.py, lines 13-27)

```lua
function DiffArray.add(da, from_i, to_i, delta)
  if from_i < 0 or to_i ≥ da.n or from_i > to_i then
    error("range out of bounds or reversed", 2)
  end
  da.d[from_i + 1] = (da.d[from_i + 1] or 0) + delta
  da.d[to_i + 2] = (da.d[to_i + 2] or 0) - delta
end

-- one prefix pass rebuilds the array
function DiffArray.materialize(da)
  local out = {}
  local run = 0
  for i = 1, da.n do
    run = run + (da.d[i] or 0)
    out[i] = run
  end
  return out
end
```

listing 730: lua, 1-based cells shift both writes, nil-safe reads in the fold (source: dsa/samples-lua/ch22_diffarray.lua, lines 13-30)

The c and java suites pin the whole pipeline, plus 5 over 0 to 2, plus 3 over 1 to 4, minus 2 over 3 to 6 leaves the difference array 5, 3, 0, −7, 0, −3, 0, 2 and materializes to 5, 8, 8, 1, 1, −2, −2, 0. Python, java, and lua cross-check every single range against a naive cell-by-cell sweep, which is the discipline that catches an off-by-one at either endpoint. The range convention splits by language here, C, C#, java, javascript, python, and lua take inclusive bounds and write at hi plus one, go alone takes the half-open [lo, hi) and writes at hi. Lua refuses reversed or out-of-bounds ranges with error and C# with an exception, the other five trust the caller.

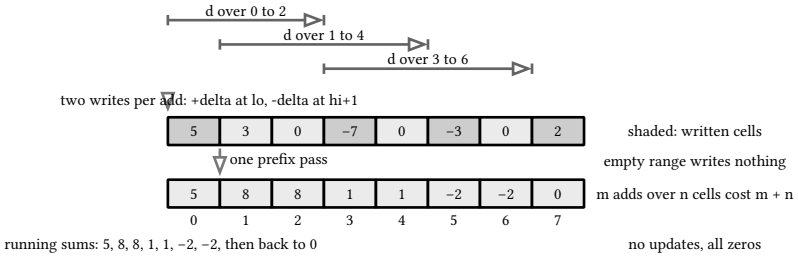


figure 191: three range adds as brackets, the difference array they leave, and the prefix pass that materializes the answer

22.5. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's four sample files, test scripts included where the language embeds them in the same file:

language	build sloc	container dependency	boundary note
c	254	`qsort` + fixed arrays	int coordinates, buffers sized to the fixture, inclusive ranges
c#	99	`List<(long,long)>`, `Dictionary<long,int>`	tests live in a separate project, bounds checks throw
go	123	the `slices` package	`add` takes the half-open [lo, hi), the one range convention that differs
java	397	jdk 27 stdlib	records carry the interval and event, the touching-fuses reading, the sweep keeps its peak time, both overlap readings in one file
javascript	81	`Array` + `Set`	number coordinates, closed overlap reading, starts tie first

language	build sloc	container dependency	boundary note
python	174	`list`, `set`, `dict`	native ints, inclusive ranges, brute-force cross-checks
lua	215	`table` + `table.sort`	1-based cells shift every write, `error()` refuses bad ranges

sources: learn.microsoft.com for tuples and `Dictionary<long,int>`, go.dev for `slices.Sort`, `slices.Compact`, `slices.BinarySearch`, developer.mozilla.org for `Set` and `Array.prototype.sort`, docs.python.org for `sorted` and `set`, lua.org for `table.sort` and `table.unpack`, accessed 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 22: c 4 files and 44 checks, c# 11 tests, go 14 tests, java 4 files and 76 checks, javascript 12 tests, python 4 files and 43 asserts, lua 16 checks, zero skipped.

23. computational geometry

Computational geometry reduces to one integer question asked thousands of times: for a directed segment a to b, which side is c on. The cross product answers exactly, and the convex hull, the closest pair, and the segment sweep are three classic algorithms built on nothing more than its sign. This chapter keeps every coordinate in an integer type and every predicate free of division, and the tests hold each algorithm against brute force at seeded scales.

23.1. the orientation predicate

The cross product of b minus a with c minus a is twice the signed area of the triangle a b c, and its sign is the whole engine: positive when c sits left of the ray a to b, negative right, zero on the line.

The dry run: the fixture is the right triangle (0,0), (4,0), (4,3) with its swap and a collinear triple, plus one billion-scale pair, asserted by the C# suite with lua pinning the billion value again, the other five settling for unit arms and their own segment shapes.

1. Cross of (0,0), (4,0), (4,3) reads 4 × 3 - 0 × 4 = 12, twice the triangle's area, and the turn is counter clockwise.
2. Swapping the last two arguments walks the same triangle from the other side: −12, clockwise.
3. The triple (0,0), (2,2), (4,4) sits on one line: 2 × 4 - 2 × 4 = 0, collinear.
4. At a billion scale the products stay exact, 1,000,000,000 × 1,000,000,000 - 1 × 1 = 999999999999999999, a value no double can represent.

a, b, c	cross	value	turn
(0,0), (4,0), (4,3)	4 × 3 - 0 × 4	12	ccw
(0,0), (4,3), (4,0)	the same, swapped	−12	cw
(0,0), (2,2), (4,4)	2 × 4 - 2 × 4	0	collinear
(0,0), (1e9, 1), (1, 1e9)	1e18 - 1	999999999999999999	ccw

The 999999999999999999, the value a double rounds away, is what the listings below keep exact in seven integer types.

```c
// sign of (a-o) x (b-o): +1 ccw, -1 cw, 0 collinear
static int orient(Pt o, Pt a, Pt b) {
  long long cross = (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
  return (cross > 0) - (cross < 0);
}

static int on_segment(Pt p, Pt q, Pt r) {
  // q collinear with pr, inside the bounding box
  return q.x ≥ (p.x < r.x ? p.x : r.x) && q.x ≤ (p.x > r.x ? p.x : r.x) &&
         q.y ≥ (p.y < r.y ? p.y : r.y) && q.y ≤ (p.y > r.y ? p.y : r.y);
}

static int segments_intersect(Pt p1, Pt p2, Pt p3, Pt p4) {
  int d1 = orient(p3, p4, p1);
  int d2 = orient(p3, p4, p2);
  int d3 = orient(p1, p2, p3);
  int d4 = orient(p1, p2, p4);
  if (d1 ≠ d2 && d3 ≠ d4 && d1 && d2 && d3 && d4)
    return 1; // proper crossing
  if (d1 == 0 && on_segment(p3, p1, p4))
    return 1;
  if (d2 == 0 && on_segment(p3, p2, p4))
    return 1;
  if (d3 == 0 && on_segment(p1, p3, p2))
    return 1;
  if (d4 == 0 && on_segment(p1, p4, p2))
    return 1;
  return 0;
}
```

listing 731: c, the sign of the cross, on-segment by bounding box, the four-orientation predicate (source: dsa/samples-c/src/Ch23/orientation.c, lines 21-49)

```
// onSegment reports whether collinear point p lies inside the bounding
// box of ab.
func onSegment(a, b, p Point) bool {
  return p.X ≥ min(a.X, b.X) && p.X <= max(a.X, b.X) &&
    p.Y ≥ min(a.Y, b.Y) && p.Y <= max(a.Y, b.Y)
}

// SegmentsIntersect reports whether ab and cd share any point,
// including endpoints and full overlap of collinear segments.
func SegmentsIntersect(a, b, c, d Point) bool {
  o1 := Orientation(a, b, c)
  o2 := Orientation(a, b, d)
  o3 := Orientation(c, d, a)
  o4 := Orientation(c, d, b)
  if o1 ≠ o2 && o3 ≠ o4 {
    return true // each segment straddles the other's line
  }
  // collinear special cases: does the stray point lie on the segment?
  if o1 == 0 && onSegment(a, b, c) {
    return true
  }
  if o2 == 0 && onSegment(a, b, d) {
    return true
  }
  if o3 == 0 && onSegment(c, d, a) {
    return true
  }
  if o4 == 0 && onSegment(c, d, b) {
    return true
  }
  return false
}
```

listing 732: go, the bounding-box helper and the straddle predicate over orientation signs (source: dsa/samples-go/ch23/orientation.go, lines 27-58)

```java
record Pt(long x, long y) {}

// sign of (a-o) x (b-o): +1 ccw, -1 cw, 0 collinear
static int orient(Pt o, Pt a, Pt b) {
  long cross = (a.x() - o.x()) * (b.y() - o.y()) - (a.y() - o.y()) * (b.x() - o.x());
  return Long.signum(cross);
}

// twice the signed area of the triangle o-a-b, the raw cross value
static long cross(Pt o, Pt a, Pt b) {
  return (a.x() - o.x()) * (b.y() - o.y()) - (a.y() - o.y()) * (b.x() - o.x());
}

static boolean onSegment(Pt p, Pt q, Pt r) {
  // q collinear with pr, inside the bounding box
  return q.x() ≥ Math.min(p.x(), r.x()) && q.x() ≤ Math.max(p.x(), r.x())
      && q.y() ≥ Math.min(p.y(), r.y()) && q.y() ≤ Math.max(p.y(), r.y());
}

static boolean segmentsIntersect(Pt p1, Pt p2, Pt p3, Pt p4) {
  int d1 = orient(p3, p4, p1);
  int d2 = orient(p3, p4, p2);
  int d3 = orient(p1, p2, p3);
  int d4 = orient(p1, p2, p4);
  if (d1 ≠ d2 && d3 ≠ d4 && d1 ≠ 0 && d2 ≠ 0 && d3 ≠ 0 && d4 ≠ 0)
    return true; // proper crossing
  if (d1 == 0 && onSegment(p3, p1, p4))
    return true;
  if (d2 == 0 && onSegment(p3, p2, p4))
    return true;
  if (d3 == 0 && onSegment(p1, p3, p2))
    return true;
  return d4 == 0 && onSegment(p1, p4, p2);
}
```

listing 733: java, the record point, `Long.signum` off the exact cross, the four-orientation predicate with bounding-box containment (source: dsa/samples-java/src/Ch23/Orientation.java, lines 17-50)

```csharp
    /// <summary>Cross product (b - a) x (c - a), twice the signed area.</summary>
    public static long Cross(Pt a, Pt b, Pt c) ⇒
        (b.X - a.X) * (c.Y - a.Y) - (b.Y - a.Y) * (c.X - a.X);

    /// <summary>Its sign: which way c sits off the ray a -> b.</summary>
    public static Turn Orientation(Pt a, Pt b, Pt c)
    {
        long v = Cross(a, b, c);
        if (v > 0)
            return Turn.CounterClockwise;
        return v < 0 ? Turn.Clockwise : Turn.Collinear;
    }

    /// <summary>Collinear containment: does p lie on segment ab.</summary>
    public static bool OnSegment(Pt a, Pt p, Pt b) ⇒
        Orientation(a, p, b) == Turn.Collinear
        && Math.Min(a.X, b.X) ≤ p.X && p.X ≤ Math.Max(a.X, b.X)
        && Math.Min(a.Y, b.Y) ≤ p.Y && p.Y ≤ Math.Max(a.Y, b.Y);
```

listing 734: c#, cross product, orientation sign, collinear containment (source: dsa/samples/src/Ch23/Geometry.cs, lines 19-36)

```
export function ccw(a, b, c) {
  const cross = (b.x - a.x) * (c.y - a.y) - (b.y - a.y) * (c.x - a.x);
  return cross > 0 ? 1 : cross < 0 ? -1 : 0;
}

// endpoints may touch; collinear overlap requires the bounding-box check
export function segmentsIntersect(p1, p2, p3, p4) {
  const d1 = ccw(p3, p4, p1);
  const d2 = ccw(p3, p4, p2);
  const d3 = ccw(p1, p2, p3);
  const d4 = ccw(p1, p2, p4);
  if (d1 !== d2 && d3 !== d4) return true; // proper crossing
  // collinear cases: any endpoint inside the other segment's bounding box
  const onSeg = (a, b, p) =>
    ccw(a, b, p) === 0 &&
    Math.min(a.x, b.x) ≤ p.x &&
    p.x ≤ Math.max(a.x, b.x) &&
    Math.min(a.y, b.y) ≤ p.y &&
    p.y ≤ Math.max(a.y, b.y);
  return (
    (d1 === 0 && onSeg(p3, p4, p1)) ||
    (d2 === 0 && onSeg(p3, p4, p2)) ||
    (d3 === 0 && onSeg(p1, p2, p3)) ||
    (d4 === 0 && onSeg(p1, p2, p4))
  );
}
```

listing 735: javascript, ccw sign then the same four-case predicate (source: dsa/samples-js/src/ch23-orientation.mjs, lines 4-29)

```python
def cross(o, a, b):
    # twice the signed area of the triangle o-a-b
    return (a[0] - o[0]) * (b[1] - o[1]) - (a[1] - o[1]) * (b[0] - o[0])


def turn(o, a, b):
    c = cross(o, a, b)
    return 1 if c > 0 else (-1 if c < 0 else 0)  # ccw, cw, collinear


def on_segment(p, q, r):
    # q lies within the bounding box of pr, given collinearity
    return min(p[0], r[0]) ≤ q[0] ≤ max(p[0], r[0]) and min(p[1], r[1]) ≤ q[
        1
    ] ≤ max(p[1], r[1])


def segments_intersect(p1, p2, p3, p4):
    d1 = turn(p3, p4, p1)
    d2 = turn(p3, p4, p2)
    d3 = turn(p1, p2, p3)
    d4 = turn(p1, p2, p4)
    if d1 ≠ d2 and d3 ≠ d4:  # each segment straddles the other's line
        return True
    if d1 == 0 and on_segment(p3, p1, p4):
        return True  # an endpoint lies on the other segment
    if d2 == 0 and on_segment(p3, p2, p4):
        return True
    if d3 == 0 and on_segment(p1, p3, p2):
        return True
    if d4 == 0 and on_segment(p1, p4, p2):
        return True
    return False
```

listing 736: python, cross, turn sign, and the straddle predicate (source: dsa/samples-py/src/Ch23/orientation.py, lines 13-45)

```lua
local function cross(a, b, c)
  return (b[1] - a[1]) * (c[2] - a[2]) - (b[2] - a[2]) * (c[1] - a[1])
end

local function orientation(a, b, c)
  local v = cross(a, b, c)
  if v > 0 then return "ccw" end
  if v < 0 then return "cw" end
  return "collinear"
end

-- is point p on segment ab, given collinearity already established
local function on_segment(a, b, p)
  return p[1] ≥ math.min(a[1], b[1]) and p[1] ≤ math.max(a[1], b[1])
    and p[2] ≥ math.min(a[2], b[2]) and p[2] ≤ math.max(a[2], b[2])
end

-- segment intersection predicate, touching endpoints included
local function segments_intersect(p1, p2, p3, p4)
  local d1 = cross(p3, p4, p1)
  local d2 = cross(p3, p4, p2)
  local d3 = cross(p1, p2, p3)
  local d4 = cross(p1, p2, p4)
  if ((d1 > 0 and d2 < 0) or (d1 < 0 and d2 > 0))
    and ((d3 > 0 and d4 < 0) or (d3 < 0 and d4 > 0)) then
    return true -- proper crossing
  end
  if d1 == 0 and on_segment(p3, p4, p1) then return true end
  if d2 == 0 and on_segment(p3, p4, p2) then return true end
  if d3 == 0 and on_segment(p1, p2, p3) then return true end
  if d4 == 0 and on_segment(p1, p2, p4) then return true end
  return false
end
```

listing 737: lua, orientation answers by name, min and max for the box test (source: dsa/samples-lua/ch23_orientation.lua, lines 7-39)

The tests pin all three signs on one shape: cross of (0,0), (4,0), (4,3) is 12, twice the area of legs 4 and 3, swapping the last two arguments flips it to −12, and (0,0), (2,2), (4,4) gives 0. Exactness is the point. Cross of (0,0), (1000000000, 1), (1, 1000000000) is 999999999999999999, and a double rounds 1e18 - 1 up to 1e18 because adjacent doubles at that magnitude sit 128 apart. Worse, a skinny triangle whose true cross is 1 rounds to 0 and the predicate calls it collinear, and every wrong sign propagates into a dropped hull corner or a missed crossing. In longs, coordinates in a billion-wide box keep each product under 1e18 and the cross under 2e18, far inside the long ceiling of 9.22e18.

The seven listings agree on the shape and on the answers. Every suite computes the sign in integer arithmetic, c in long long, go in int64, java in long through `Long.signum`, lua in its native int64 and pins the billion-scale cross at 999999999999999999 with `math.type` reporting integer, the same fixture and the same number as the c# paragraph above. The segment predicate is the same four orientations plus a bounding-box test in all seven languages, and the crossing X, (0,0)-(4,4) against (0,4)-(4,0) meeting at (2,2), is pinned true in c, go, java, javascript, python, and lua, while the c# suite meets its crossings through the sweep section below. The c, go, and java suites all add the T shape with a gap at the crossbar, parallel segments that never meet, and collinear pairs both disjoint and overlapping, java pinning the shared endpoint lane beside them, and lua pins its collinear pair in both directions.

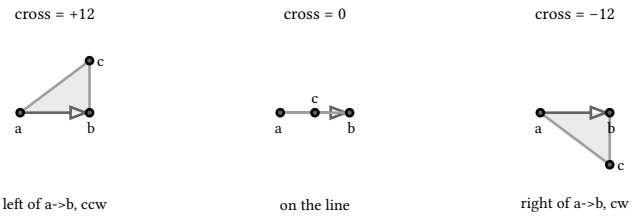


figure 192: the orientation predicate on one fixed segment, c left of a to b gives +12, on the line 0, right gives −12

PITFALL: doubles round the truth away

At billion scale a cross lands near 1e18, where adjacent doubles sit 128 apart, and 1e18 - 1 rounds up to 1e18. A skinny triangle whose true cross is 1 rounds to 0 and the predicate calls it collinear. The fix costs nothing: keep coordinates in longs and every predicate exact, with 9.22e18 of headroom, and take the square root only when a distance, not a decision, is needed.

23.2. the convex hull

The hull is the smallest convex polygon containing the set, and Andrew's monotone chain builds it with the orientation predicate as the only geometric operation. Sort the points lexicographically, sweep left to right, and keep a chain of strict left turns: when the top two chain points plus the candidate fail to turn left, pop. One pass gives the lower chain, the reverse pass the upper, and the concatenation walks the hull counter clockwise from the lexicographically smallest point.

The dry run: the fixture is the seven point set, the 4 by 3 rectangle with (1,2), (2,1), and (3,2) inside, asserted by the C# suite, while C and java run their own squares, go adds an exterior nub at (5,2), python pins a hexagon, and lua checks two properties.

1. Sorted by x then y the sweep enters (0,0), (0,3), (1,2), (2,1), (3,2), (4,0), (4,3), and the lower chain walks first.
2. Candidate (1,2) pops (0,3): the turn (0,0), (0,3), (1,2) reads 0 × 2 - 3 × 1 = −3, not a left turn.
3. Candidate (2,1) pops (1,2) the same way at 1 × 1 - 2 × 2 = −3, and (3,2) survives its test at 2 × 2 - 1 × 3 = 1.
4. Candidate (4,0) pops twice, (3,2) at −3 then (2,1) at −4, leaving (0,0), (4,0), and (4,3) closes the pass at 4 × 3 - 0 × 4 = 12.
5. The upper sweep mirrors back through the same candidates, and on the way down the collinear triple (4,3), (3,2), (2,1) reads 0 exactly, so the <= 0 test pops the middle point.
6. The mirrored pops leave the cycle (0,0), (4,0), (4,3), (0,3), the asserted four corners with all three interior points gone.

candidate	turn	action	chain after
(0,0)	first	push	(0,0)
(0,3)	too short	push	(0,0), (0,3)
(1,2)	−3	pop (0,3), push	(0,0), (1,2)
(2,1)	−3	pop (1,2), push	(0,0), (2,1)
(3,2)	+1	push	(0,0), (2,1), (3,2)
(4,0)	−3, then −4	pop twice, push	(0,0), (4,0)
(4,3)	+12	push, pass closes	(0,0), (4,0), (4,3)

The four corners (0,0), (4,0), (4,3), (0,3) are the pinned cycle, and the listings below build them in seven languages.

```c
// hull in counterclockwise order starting at the leftmost point,
// collinear boundary points excluded
static Pt hull[16];
static int hn;

static void build_hull(void) {
  qsort(pts, (size_t)pn, sizeof(Pt), cmp_pt);
  Pt lower[16], upper[16];
  int ln = 0, un = 0;
  for (int i = 0; i < pn; i++) {
    while (ln ≥ 2 && cross(lower[ln - 2], lower[ln - 1], pts[i]) ≤ 0)
      ln--;
    lower[ln++] = pts[i];
  }
  for (int i = pn - 1; i ≥ 0; i--) {
    while (un ≥ 2 && cross(upper[un - 2], upper[un - 1], pts[i]) ≤ 0)
      un--;
    upper[un++] = pts[i];
  }
  hn = 0;
  for (int i = 0; i < ln - 1; i++)
    hull[hn++] = lower[i];
  for (int i = 0; i < un - 1; i++)
    hull[hn++] = upper[i];
}
```

listing 738: c, two chain sweeps, popping on a non-left turn (source: dsa/samples-c/src/Ch23/hull.c, lines 36-60)

```go
func ConvexHull(points []Point) []Point {
  if len(points) < 3 {
    return slices.Clone(points)
  }
  sorted := slices.Clone(points)
  slices.SortFunc(sorted, func(a, b Point) int {
    if c := cmp.Compare(a.X, b.X); c ≠ 0 {
      return c
    }
    return cmp.Compare(a.Y, b.Y)
  })
  // drop exact duplicates: they would make degenerate turns
  distinct := sorted[:1]
  for _, p := range sorted[1:] {
    if p ≠ distinct[len(distinct)-1] {
      distinct = append(distinct, p)
    }
  }
  var lower []Point
  for _, p := range distinct {
    for len(lower) ≥ 2 && Cross(lower[len(lower)-2], lower[len(lower)-1], p) <= 0 {
      lower = lower[:len(lower)-1] // strictly left turns only
    }
    lower = append(lower, p)
  }
  var upper []Point
  for i := len(distinct) - 1; i ≥ 0; i-- {
    p := distinct[i]
    for len(upper) ≥ 2 && Cross(upper[len(upper)-2], upper[len(upper)-1], p) <= 0 {
      upper = upper[:len(upper)-1]
    }
    upper = append(upper, p)
  }
  // lower ends and upper begin at the same extremes: trim one copy
  return append(lower[:len(lower)-1], upper[:len(upper)-1]...)
}
```

listing 739: go, monotone chain, exact duplicates dropped before the sweep (source: dsa/samples-go/ch23/hull.go, lines 12-47)

```
record Pt(long x, long y) {}

static long cross(Pt o, Pt a, Pt b) {
  return (a.x() - o.x()) * (b.y() - o.y()) - (a.y() - o.y()) * (b.x() - o.x());
}

// hull in counterclockwise order starting at the leftmost point,
// collinear boundary points excluded
static List<Pt> convexHull(List<Pt> points) {
  List<Pt> pts = new ArrayList<>(points);
  pts.sort((a, b) -> a.x() != b.x()
      ? Long.compare(a.x(), b.x()) : Long.compare(a.y(), b.y()));
  if (pts.size() <= 2)
    return pts;
  List<Pt> lower = new ArrayList<>(), upper = new ArrayList<>();
  for (Pt p : pts) {
    while (lower.size() >= 2
        && cross(lower.get(lower.size() - 2), lower.get(lower.size() - 1), p) <= 0)
      lower.remove(lower.size() - 1); // a right turn: the middle is inside
    lower.add(p);
  }
  for (int i = pts.size() - 1; i >= 0; i--) {
    Pt p = pts.get(i);
    while (upper.size() >= 2
        && cross(upper.get(upper.size() - 2), upper.get(upper.size() - 1), p) <= 0)
      upper.remove(upper.size() - 1);
    upper.add(p);
  }
  List<Pt> out = new ArrayList<>(lower.subList(0, lower.size() - 1));
  out.addAll(upper.subList(0, upper.size() - 1));
  return out;
}
```

listing 740: java, the record point, the monotone chain over ArrayLists, cross <= 0 pops (source: dsa/samples-java/src/Ch23/Hull.java, lines 20-51)

```
    /// <summary>Andrew monotone chain, ccw, strict turns.</summary>
    public static List<Pt> ConvexHull(Pt[] points, Meter? meter = null)
    {
        long Turn(Pt a, Pt b, Pt c)
        {
            if (meter ≠ null)
                meter.Ops++;
            return Cross(a, b, c);
        }

        Pt[] pts = points.Distinct().OrderBy(p ⇒ p.X).ThenBy(p ⇒ p.Y).ToArray();
        if (pts.Length ≤ 1)
            return new List<Pt>(pts);

        var hull = new List<Pt>(2 * pts.Length);
        foreach (Pt p in pts) // lower chain, popping non-left turns
        {
            while (hull.Count ≥ 2 && Turn(hull[^2], hull[^1], p) ≤ 0)
                hull.RemoveAt(hull.Count - 1);
            hull.Add(p);
        }
        int lower = hull.Count + 1;
        for (int i = pts.Length - 2; i ≥ 0; i--) // upper chain mirrors back
        {
            Pt p = pts[i];
            while (hull.Count ≥ lower && Turn(hull[^2], hull[^1], p) ≤ 0)
                hull.RemoveAt(hull.Count - 1);
            hull.Add(p);
        }
        hull.RemoveAt(hull.Count - 1); // the start point closes the walk
        return hull;
    }
```

listing 741: c#, monotone chain, two passes, strict turns pop collinear points (source: dsa/samples/src/Ch23/Geometry.cs, lines 38-69)

```
export function convexHull(points) {
  const pts = [...points].sort(
    (a, b) => a.x - b.x || a.y - b.y,
  );
  const cross = (o, a, b) =>
    (a.x - o.x) * (b.y - o.y) - (a.y - o.y) * (b.x - o.x);
  const build = (list) => {
    const hull = [];
    for (const p of list) {
      while (
        hull.length >= 2 &&
        cross(hull[hull.length - 2], hull[hull.length - 1], p) <= 0
      )
        hull.pop(); // clockwise or collinear: not a hull vertex
      hull.push(p);
    }
    return hull;
  };
  const lower = build(pts);
  const upper = build([...pts].reverse());
  // concatenate, dropping each chain's copy of the shared endpoints
  return [...lower.slice(0, -1), ...upper.slice(0, -1)];
}
```

listing 742: javascript, one build helper run forward then reversed (source: dsa/samples-js/src/ch23-hull.mjs, lines 6-28)

```
def convex_hull(points):
    pts = sorted(set(points))
    if len(pts) <= 2:
        return pts
    lower = []
    for p in pts:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], p) <= 0:
            lower.pop()  # a right turn: the middle point is inside
        lower.append(p)
    upper = []
    for p in reversed(pts):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], p) <= 0:
            upper.pop()
        upper.append(p)
    return lower[:-1] + upper[:-1]  # each chain repeats its endpoints
```

listing 743: python, sorted(set(...)) dedupes, two chain loops (source: dsa/samples-py/src/Ch23/hull.py, lines 17-31)

```lua
local function hull(points)
  local pts = {table.unpack(points)}
  table.sort(pts, function(x, y)
    if x[1] ~= y[1] then return x[1] < y[1] end
    return x[2] < y[2]
  end)
  if #pts ≤ 2 then return pts end
  local function chain(seq)
    local out = {}
    for _, p in ipairs(seq) do
      while #out ≥ 2 and cross(out[#out - 1], out[#out], p) ≤ 0 do
        table.remove(out)
      end
      out[#out + 1] = p
    end
    return out
  end
  local lower = chain(pts)
  local rev = {}
  for i = #pts, 1, -1 do rev[#rev + 1] = pts[i] end
  local upper = chain(rev)
  -- drop the last point of each chain: it is the first of the other
  local out = {}
  for i = 1, #lower - 1 do out[#out + 1] = lower[i] end
  for i = 1, #upper - 1 do out[#out + 1] = upper[i] end
  return out
end
```

listing 744: lua, a chain closure over the lexicographically sorted points (source: dsa/samples-lua/ch23_hull.lua, lines 10-36)

A strict left turn fails on collinear too, since the pop tests <= 0, so edge midpoints drop out: the four point set with (2,0) sitting on the bottom edge returns only the triangle corners. The seven point set keeps its four rectangle corners and drops all three interior points. Seeded agreement holds against a brute force that keeps exactly the points no triangle of the others covers, and the meter counts each cross: every point is pushed once and popped at most once per chain, so after the n log n sort the walk is linear.

Every suite pops on cross <= 0, so collinear edge points vanish and the hull cycle carries only corners, and the c# shape above is the same code in the other six languages. The fixtures push in different directions: c's square with three interior points returns its four corners and a fully collinear input degenerates to two endpoints, java carries that square, that collinear degenerate, python's centered square and six vertex hull, and its own pentagon with one interior point, go adds an exterior nub at (5,2) to the square and pins the five vertex cycle, python pins a six vertex hexagon and proves duplicates collapse before the sweep, and lua checks two properties instead of one answer, every cycle turn strictly counter clockwise and every input point inside or on the hull by the half-plane test.

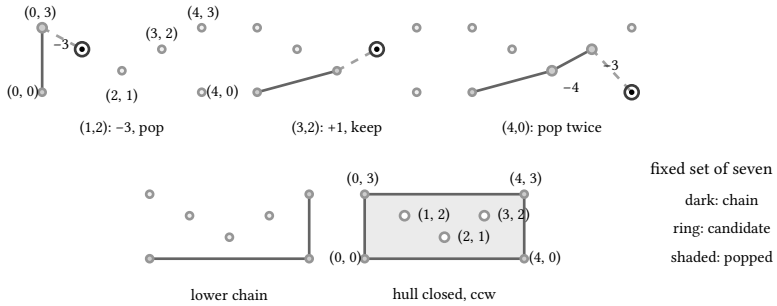


figure 193: the monotone chain on the fixed seven point set, candidates tested against the chain, failing turns shaded, the hull closed

23.3. closest pair

The closest pair splits at the median x, solves both halves, and merges over the vertical strip of width 2δ, where δ is the smaller recursive answer. A pair straddling the split must have both points in that strip, the strip is scanned sorted by y, and each point is compared only while the vertical gap stays under δ, because each δ by δ half of the window holds at most 4 pairwise δ separated points, so at most 7 others can sit above a point inside the window.

The dry run: the fixture is (0,0), (3,4), (10,10), (10,9), (2,2), asserted by the C# suite with lua pinning the same pair, while C, go, and java land diagonal neighbors, javascript floats on hypot to Math.SQRT2, and every suite cross-checks a brute double loop.

1. Sorted by x the array reads (0,0), (2,2), (3,4), (10,9), (10,10), and the top split lands after (3,4), the midpoint at x = 3.
2. The left half bottoms out at two points, (0,0) against (2,2) reading 2 × 2 + 2 × 2 = 8 squared.
3. The right half tries its three pairs: (3,4) against (10,9) reads 49 + 25 = 74, against (10,10) reads 85, and (10,9) against (10,10) reads 0 + 1 = 1, the winner.
4. The smaller answer 1 prices the strip, points with $(x - 3)^2 < 1$: (3,4) survives alone, and (2,2) fails at exactly 1.
5. A strip of one point compares nothing, and the merge hands the winner up unchanged.

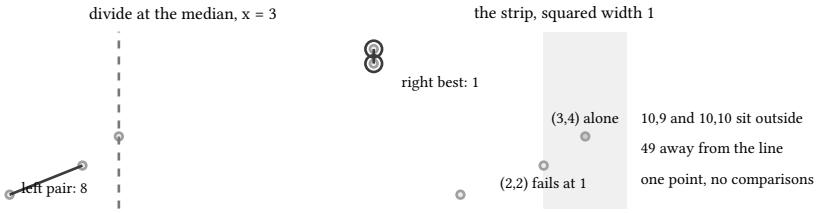


figure 194: the fixture's own run, the split with each half's best squared distance, then the strip holding exactly one point

The squared 1 between (10,9) and (10,10) is the pinned answer, and the listings below find it in seven languages.

```
static void rec(Pt *px, int len) {
  if (len ≤ 3) {
    for (int i = 0; i < len; i++)
      for (int j = i + 1; j < len; j++) {
        long long d = dist2(px[i], px[j]);
        if (d < best2)
          best2 = d;
      }
    return;
  }
  int mid = len / 2;
  long long midx = px[mid].x;
  rec(px, mid);
  rec(px + mid, len - mid);
  // the strip: points within sqrt(best2) of the split line, y-sorted by
  // selection since the fixture is small enough to keep it plain
  Pt strip[16];
  int sn = 0;
  for (int i = 0; i < len; i++) {
    long long dx = px[i].x - midx;
    if (dx * dx < best2)
      strip[sn++] = px[i];
  }
  for (int i = 1; i < sn; i++)
    for (int j = i; j > 0 && strip[j - 1].y > strip[j].y; j--) {
      Pt t = strip[j - 1];
      strip[j - 1] = strip[j];
      strip[j] = t;
    }
  for (int i = 0; i < sn; i++)
    for (int j = i + 1; j < sn; j++) {
      long long dy = strip[j].y - strip[i].y;
      if (dy * dy ≥ best2)
        break;
      long long d = dist2(strip[i], strip[j]);
      if (d < best2)
        best2 = d;
    }
}
```

listing 745: c, the recursion with the strip insertion-sorted by y (source: dsa/samples-c/src/Ch23/closestpair.c, lines 36-74)

```
  recurse = func(pts []Point) []Point {
    if len(pts) <= 3 {
      sortedByY := slices.Clone(pts)
      slices.SortFunc(sortedByY, func(a, b Point) int { return int(a.Y - b.Y) })
      for i := 0; i < len(pts); i++ {
        for j := i + 1; j < len(pts); j++ {
          if d := distSquared(pts[i], pts[j]); d < bestSquared {
            bestSquared, a, b = d, pts[i], pts[j]
          }
        }
      }
      return sortedByY
    }
    mid := len(pts) / 2
    midX := pts[mid].X
    leftY := recurse(pts[:mid])
    rightY := recurse(pts[mid:])
    // merge the halves by y for the strip scan
    merged := make([]Point, 0, len(pts))
    i, j := 0, 0
    for i < len(leftY) && j < len(rightY) {
      if leftY[i].Y <= rightY[j].Y {
        merged = append(merged, leftY[i])
        i++
      } else {
        merged = append(merged, rightY[j])
        j++
      }
    }
    merged = append(merged, leftY[i:]...)
    merged = append(merged, rightY[j:]...)
    // strip: points within sqrt(best) of the split line, in y order
    strip := make([]Point, 0, len(merged))
    for _, p := range merged {
      if math.Abs(float64(p.X-midX)) < math.Sqrt(bestSquared) {
        strip = append(strip, p)
      }
    }
    for i := 0; i < len(strip); i++ {
      for j := i + 1; j < len(strip); j++ {
        dy := strip[j].Y - strip[i].Y
        if float64(dy*dy) ≥ bestSquared {
          break // everything further is too far in y already
        }
        if d := distSquared(strip[i], strip[j]); d < bestSquared {
          bestSquared, a, b = d, strip[i], strip[j]
        }
      }
    }
    return merged
  }
  recurse(px)
  return bestSquared, a, b
}
```

listing 746: go, y order merged through the recursion, the textbook n log n (source: dsa/samples-go/ch23/closestpair.go, lines 25-78)

```
record Pt(long x, long y) {}

static long dist2(Pt a, Pt b) {
  long dx = a.x() - b.x(), dy = a.y() - b.y();
  return dx * dx + dy * dy;
}

static long best2;

static void rec(Pt[] px) {
  int len = px.length;
  if (len ≤ 3) {
    for (int i = 0; i < len; i++)
      for (int j = i + 1; j < len; j++) {
        long d = dist2(px[i], px[j]);
        if (d < best2)
          best2 = d;
      }
    return;
  }
  int mid = len / 2;
  long midx = px[mid].x();
  Pt[] left = new Pt[mid], right = new Pt[len - mid];
  System.arraycopy(px, 0, left, 0, mid);
  System.arraycopy(px, mid, right, 0, len - mid);
  rec(left);
  rec(right);
  // the strip: points within sqrt(best2) of the split line, y-sorted by
  // selection since the fixture is small enough to keep it plain
  List<Pt> strip = new ArrayList<>();
  for (Pt p : px) {
    long dx = p.x() - midx;
    if (dx * dx < best2)
      strip.add(p);
  }
  strip.sort((a, b) -> Long.compare(a.y(), b.y()));
  for (int i = 0; i < strip.size(); i++)
    for (int j = i + 1; j < strip.size(); j++) {
      long dy = strip.get(j).y() - strip.get(i).y();
      if (dy * dy ≥ best2)
        break;
      long d = dist2(strip.get(i), strip.get(j));
      if (d < best2)
        best2 = d;
    }
}
```

listing 747: java, squared long distances, the strip y-sorted by a comparator, the break when dy squared passes the best (source: dsa/samples-java/src/Ch23/Closestpair.java, lines 19-64)

```
/// <summary>Closest pair by divide and conquer over the y-sorted strip.</summary>
public static (long SqDist, Pt A, Pt B) ClosestPair(Pt[] pts, Meter? meter = null)
{
    if (pts.Length < 2)
        throw new ArgumentException("closest pair needs two points", nameof(pts));
    Pt[] px = pts.OrderBy(p => p.X).ThenBy(p => p.Y).ToArray();
    return Closest(px, 0, px.Length, meter);
```

```
    }

    // each point is compared only while the vertical gap stays under the best
    private static (long SqDist, Pt A, Pt B) Closest(
        Pt[] px, int lo, int hi, Meter? meter)
    {
        int n = hi - lo;
        if (n ≤ 3)
        {
            var small = (SqDist: long.MaxValue, A: px[lo], B: px[lo + 1]);
            for (int i = lo; i < hi; i++)
                for (int j = i + 1; j < hi; j++)
                {
                    long d = Sq(px[i], px[j]);
                    if (d < small.SqDist)
                        small = (d, px[i], px[j]);
                }
            return small;
        }

        int mid = lo + n / 2;
        long midX = px[mid].X;
        var left = Closest(px, lo, mid, meter);
        var right = Closest(px, mid, hi, meter);
        var best = left.SqDist ≤ right.SqDist ? left : right;

        var strip = new List<Pt>();
        for (int i = lo; i < hi; i++)
            if ((px[i].X - midX) * (px[i].X - midX) < best.SqDist)
                strip.Add(px[i]);
        strip.Sort((p, q) ⇒ p.Y ≠ q.Y ? p.Y.CompareTo(q.Y) : p.X.CompareTo(q.X));

        for (int i = 0; i < strip.Count; i++)
            for (int j = i + 1; j < strip.Count; j++)
            {
                long dy = strip[j].Y - strip[i].Y;
                if (dy * dy ≥ best.SqDist)
                    break; // packing argument: at most 7 points below this line
                if (meter ≠ null)
                    meter.Ops++;
                long d = Sq(strip[i], strip[j]);
                if (d < best.SqDist)
                    best = (d, strip[i], strip[j]);
            }
        return best;
    }

    private static long Sq(Pt p, Pt q) ⇒
        (p.X - q.X) * (p.X - q.X) + (p.Y - q.Y) * (p.Y - q.Y);
```

listing 748: c#, divide and conquer closest pair, the strip scanned in y with the break (source: dsa/samples/src/Ch23/Geometry.cs, lines 71-126)

```
export function closestPair(points) {
  const px = [...points].sort((a, b) => a.x - b.x || a.y - b.y);
  const py = [...points].sort((a, b) => a.y - b.y || a.x - b.x);

  const solve = (px) => {
    const n = px.length;
    if (n ≤ 3) {
      let best = { dist: Infinity, pair: null };
      for (let i = 0; i < n; i++)
        for (let j = i + 1; j < n; j++) {
          const d = dist(px[i], px[j]);
          if (d < best.dist) best = { dist: d, pair: [px[i], px[j]] };
        }
      return best;
    }
    const mid = n >> 1;
    const midX = px[mid].x;
    const left = solve(px.slice(0, mid));
    const right = solve(px.slice(mid));
    let best = left.dist ≤ right.dist ? left : right;
    // the strip: points within best.dist of the split line, y-sorted
    const strip = py.filter(
      (p) => Math.abs(p.x - midX) < best.dist,
    );
    for (let i = 0; i < strip.length; i++)
      for (let j = i + 1; j < strip.length; j++) {
        if (strip[j].y - strip[i].y ≥ best.dist) break;
        const d = dist(strip[i], strip[j]);
        if (d < best.dist) best = { dist: d, pair: [strip[i], strip[j]] };
      }
    return best;
  };
  return solve(px);
}
```

listing 749: javascript, hypot distances, the strip filtered from a y-sorted copy (source: dsa/samples-js/src/ch23-closestpair.mjs, lines 7-40)

```
def strip_best(strip, best):
    # strip is y-sorted; each point needs at most 7 comparisons ahead
    for i, a in enumerate(strip):
        for b in strip[i + 1 :]:
            if (b[1] - a[1]) ** 2 ≥ best:
                break  # y alone already exceeds the record
            best = min(best, dist_sq(a, b))
    return best


def closest(pts):
    px = sorted(pts)
    py = sorted(pts, key=lambda p: p[1])

    def solve(px, py):
        if len(px) ≤ 3:
            return brute(px)
        mid = len(px) // 2
        midx = px[mid][0]
        left_py = [p for p in py if p < px[mid]]
        right_py = [p for p in py if p ≥ px[mid]]
        best = min(solve(px[:mid], left_py), solve(px[mid:], right_py))
        strip = [p for p in py if (p[0] - midx) ** 2 < best]
        return strip_best(strip, best)

    return solve(px, py)
```

listing 750: python, squared integer distances, the strip kept in y order (source: dsa/samples-py/src/Ch23/closestpair.py, lines 29-54)

```lua
local function rec(pts, ys)
  local n = #pts
  if n ≤ 3 then return brute(pts) end
  local mid = n // 2
  local midx = pts[mid][1]
  local left, right = {}, {}
  for i = 1, mid do left[i] = pts[i] end
  for i = mid + 1, n do right[i - mid] = pts[i] end
  -- split ys by membership (ties on midx must follow the same split)
  local inleft = {}
  for _, p in ipairs(left) do inleft[p] = true end
  local ly, ry = {}, {}
  for _, p in ipairs(ys) do
    if inleft[p] then ly[#ly + 1] = p else ry[#ry + 1] = p end
  end
  local best, pair = rec(left, ly)
  local d2, pair2 = rec(right, ry)
  if d2 < best then best, pair = d2, pair2 end
  -- the strip: points within sqrt(best) of the split line, in y order
  local strip = {}
  for _, p in ipairs(ys) do
    local dx = p[1] - midx
    if dx * dx < best then strip[#strip + 1] = p end
  end
  for i = 1, #strip do
    local j = i + 1
    while j ≤ #strip do
      local dy = strip[j][2] - strip[i][2]
      if dy * dy ≥ best then break end
      local d = dist2(strip[i], strip[j])
      if d < best then best, pair = d, {strip[i], strip[j]} end
      j = j + 1
    end
  end
  return best, pair
end
```

listing 751: lua, ys split by membership, the strip scan with the y break (source: dsa/samples-lua/ch23_closestpair.lua, lines 25-60)

Distances stay squared, no square root until a human reads one. The 5 point test finds the vertical pair (10,9)-(10,10) at squared distance 1, seeded 500 point runs match an O(n^2) scan exactly, and at n = 1000 the strip meter stays under the classical 7 comparisons per point. One honesty note: this version sorts the strip at every merge instead of merging y order through the recursion, so it is n log^2 n against the textbook n log n, and the packing bound survives either way.

The strip handling is where the seven builds differ, and every variant is honest about it. Go merges y order through the recursion and is the textbook n log n, c re-sorts the strip with an insertion sort, java re-sorts it with a plain comparator over squared longs, and python and lua carry a y-sorted list through the split, python by filtering it and lua by membership. The distance type splits the same way: c, java, python, and lua compare squared integers, go keeps the int64 products but accumulates the best as a float64, and javascript is the one float suite, `Math.hypot` distances with the fixture answer asserted as exactly `Math.SQRT2`. Fixtures overlap where it matters: lua pins the same vertical pair (10,9)-(10,10) at squared distance 1 as c#, c, go, and java all land on a squared 2 answer from diagonal neighbors, java adding a unit 5 by 5 grid pinning 1 and a vertical line whose 4 gap squares to 16, and every suite cross-checks the recursion against a brute-force double loop, lua over five deterministic 37-point LCG clouds with no clock and no `math.random`.

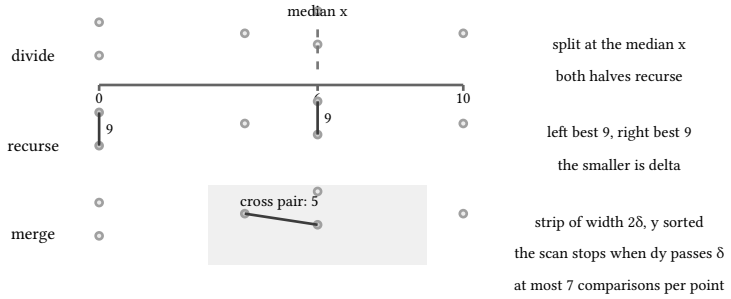


figure 195: closest pair as a pipeline of lanes, divide at the median x, recurse for the two best halves, merge over the strip in y order

23.4. the sweep line

Shamos-Hoey asks whether any two of n segments intersect and answers in n log n by sweeping a vertical line across the endpoints. Events are the endpoints, sorted by x with inserts ahead of erases at a shared point, so segments touching at a corner meet inside the status. The status orders the segments crossing the sweep line by height there, compared exactly by cross multiplication. Two crossing segments must become neighbors in that order, so it suffices to compare an inserted segment against its immediate neighbors and to compare the pair an erasure makes newly adjacent.

The dry run: the clean fixtures are asserted with identical literals by all seven suites, the answers are (min, max) index pairs with none for a disjoint input, and the error lanes reject vertical segments and triple points, each tree picking its own instance of three non-vertical segments through one point.

1. Both segments normalize left to right and emit four events, and the sort reads x first, then y, then inserts ahead of erases at a shared point.
2. The sweep opens at (0,0): s0 enters an empty status, no neighbors exist to test.
3. At (0,4) the insert compares heights by cross multiplication, s1 reading 4 against s0′s 0 at x = 0, so s1 lands above with s0 its only neighbor.
4. The check runs the four orientations and s1′s endpoints read 4 × 4 - 4 × 0 = 16 and 4 × 0 - 4 × 4 = −16 off s0′s line, signs that straddle, so the pair crosses and the sweep returns (0, 1).
5. The touching pair (0,0)-(2,2) and (2,2)-(4,0) shares the corner (2,2): the second insert sorts ahead of the first's erase, they meet as neighbors inside the status, and the answer is (0, 1) again.
6. Disjoint back to back segments erase cleanly with no pair, and the C# suite's 200 seeded segments agree with the O(n²) orientation scan.
7. The later pair fixture holds two safe horizontals in front of a crossing pair at x = 5 and answers (2, 3), and the early pair never touches.
8. The removal adjacency fixture slides a flat middle segment between the crossing pair, and when it erases at x = 6 the check on the newly adjacent neighbors finds the crossing at (10.5, 10.5), answer (0, 1).
9. The error lanes mirror the C# ArgumentException per tree: Go returns an error, JavaScript and Lua raise, Python raises ValueError, C reports a rejected input, Java returns minus one, each with its own valid triple point instance.
10. The oracle lane: the O(n^2) pairwise orientation test agrees on the yes/no answer of every clean fixture, in every suite.
11. Per-tree honesty: the C# suite pins the crossing, the disjoint pair, the touch, and the seeded agreement, and the six sibling suites pin all five clean fixtures plus both error lanes.

event	action	status	neighbor check
(0,0) insert s0	place	s0	none, empty status
(0,4) insert s1	above s0	s1, s0	s0 × s1 straddles: crossing
(4,4) erase s0	never reached		returned (0, 1)
(4,0) erase s1	never reached		

The pair (0, 1) at the crossing X is the pinned answer, and the C# listing below sweeps it in full, the six sibling windows further down picking the walk up at the status.

```
    /// <summary>Shamos-Hoey sweep: a pair if any cross, null when disjoint.</summary>
    public static (int I, int J)? AnyIntersection(Seg[] segments)
    {
        var segs = new Seg[segments.Length];
        for (int i = 0; i < segments.Length; i++)
        {
            Seg s = segments[i];
            if (s.X1 == s.X2)
                throw new ArgumentException(
                    "vertical segments are rejected", nameof(segments));
            segs[i] = s.X1 < s.X2 ? s : new Seg(s.X2, s.Y2, s.X1, s.Y1);
        }
        RejectTriplePoints(segs);

        var events = new List<(long X, long Y, bool Left, int Idx)>();
        for (int i = 0; i < segs.Length; i++)
        {
            events.Add((segs[i].X1, segs[i].Y1, true, i));
            events.Add((segs[i].X2, segs[i].Y2, false, i));
        }
        // inserts sort ahead of erases at a shared point
        events.Sort((e, f) => e.X != f.X ? e.X.CompareTo(f.X)
            : e.Y != f.Y ? e.Y.CompareTo(f.Y)
            : (e.Left ? 0 : 1).CompareTo(f.Left ? 0 : 1));

        (int, int) Pair(int a, int b) => (Math.Min(a, b), Math.Max(a, b));
        var status = new List<int>(); // a list stands in for the balanced tree
        foreach (var (x, _, left, idx) in events)
        {
            Seg cur = segs[idx];
            if (left)
            {
                int at = Insert(segs, status, idx, x);
                if (at > 0 && Intersect(segs[status[at - 1]], cur))
                    return Pair(status[at - 1], idx);
                if (at + 1 < status.Count && Intersect(segs[status[at + 1]], cur))
                    return Pair(status[at + 1], idx);
            }
            else
            {
                int at = status.IndexOf(idx);
                if (at > 0 && Intersect(segs[status[at - 1]], cur))
                    return Pair(status[at - 1], idx); // shared endpoint below
                if (at + 1 < status.Count && Intersect(segs[status[at + 1]], cur))
                    return Pair(status[at + 1], idx); // shared endpoint above
                status.RemoveAt(at);
                int below = at - 1, above = at; // the removal made these adjacent
                if (below >= 0 && above < status.Count
                    && Intersect(segs[status[below]], segs[status[above]]))
                    return Pair(status[below], status[above]);
            }
        }
        return null;
    }
```

listing 752: shamos-hoey sweep, endpoint events, status by height, neighbor checks (source: dsa/samples/src/Ch23/Geometry.cs, lines 128-181)

The preconditions are not decoration. A vertical segment has no single height at the sweep, and three segments through one point make the neighbor order ambiguous, so both are rejected with an exception, the triple point check exact by collecting every crossing as a reduced fraction. Touching endpoints count as intersections and back to back disjoint segments do not, the crossing X returns the pair (0, 1), and 200 seeded segments agree with the O(n^2) orientation test. The status is a plain list, so an insertion costs a shift and the worst case is quadratic, and the discipline the chapter keeps is neighbors only, never all pairs.

The six sibling builds now carry the full sweep themselves. The windows below pick it up at the status walk, and the event sort sits a few lines above in each file:

```
  int status[MAX_SEGS], live = 0;
  for (int e = 0; e < 2 * count; e++) {
    int idx = events[e].idx;
    if (events[e].left) {
      int at = status_insert(segs, status, live, idx, events[e].x);
      live++;
      if (at > 0 && intersect(segs[status[at - 1]], segs[idx])) {
        *pi = status[at - 1] < idx ? status[at - 1] : idx;
        *pj = status[at - 1] < idx ? idx : status[at - 1];
        return 1;
      }
      if (at + 1 < live && intersect(segs[status[at + 1]], segs[idx])) {
        *pi = status[at + 1] < idx ? status[at + 1] : idx;
        *pj = status[at + 1] < idx ? idx : status[at + 1];
        return 1;
      }
    } else {
      int at = 0;
      while (status[at] != idx)
        at++; // a list stands in for the balanced tree
      if (at > 0 && intersect(segs[status[at - 1]], segs[idx])) {
        *pi = status[at - 1] < idx ? status[at - 1] : idx;
        *pj = status[at - 1] < idx ? idx : status[at - 1];
        return 1;
      }
      if (at + 1 < live && intersect(segs[status[at + 1]], segs[idx])) {
        *pi = status[at + 1] < idx ? status[at + 1] : idx;
        *pj = status[at + 1] < idx ? idx : status[at + 1];
        return 1;
      }
      memmove(status + at, status + at + 1,
              (size_t)(live - at - 1) * sizeof(int));
      live--;
      int below = at - 1, above = at; // the removal made these adjacent
      if (below >= 0 && above < live &&
          intersect(segs[status[below]], segs[status[above]])) {
        *pi = status[below] < status[above] ? status[below] : status[above];
        *pj = status[below] < status[above] ? status[above] : status[below];
        return 1;
      }
    }
  }
  return 0;
}
```

listing 753: c, the status walk over a fixed array, binary insert by height, neighbor checks on insert, erase, and removal (source: dsa/samples-c/src/Ch23/sweep.c, lines 185-228)

```
  var status []int // a slice stands in for the balanced tree
  for _, ev := range events {
    cur := segs[ev.idx]
    if ev.left {
      var at int
      status, at = sweepInsert(segs, status, ev.idx, ev.x)
      if at > 0 && sweepIntersect(segs[status[at-1]], cur) {
        return report(status[at-1], ev.idx)
      }
      if at+1 < len(status) && sweepIntersect(segs[status[at+1]], cur) {
        return report(status[at+1], ev.idx)
      }
    } else {
      at := slices.Index(status, ev.idx)
      if at > 0 && sweepIntersect(segs[status[at-1]], cur) {
        return report(status[at-1], ev.idx) // shared endpoint below
      }
      if at+1 < len(status) && sweepIntersect(segs[status[at+1]], cur) {
        return report(status[at+1], ev.idx) // shared endpoint above
      }
      status = slices.Delete(status, at, at+1)
      below, above := at-1, at // the removal made these adjacent
      if below ≥ 0 && above < len(status) &&
        sweepIntersect(segs[status[below]], segs[status[above]]) {
        return report(status[below], status[above])
      }
    }
  }
  return [2]int{}, false, nil
```

listing 754: go, the status slice with insert and delete by height, the removal adjacency check (source: dsa/samples-go/ch23/sweep.go, lines 81-109)

```
    // a list stands in for the balanced tree
    List<Integer> status = new ArrayList<>();
    for (Event e : events) {
      int idx = e.idx();
      if (e.left()) {
        int lo = 0, hi = status.size();
        while (lo < hi) {
          int mid = (lo + hi) / 2;
          if (compareAt(segs[status.get(mid)], segs[idx], e.x()) < 0)
            lo = mid + 1;
          else
            hi = mid;
        }
        status.add(lo, idx);
        if (lo > 0 && intersect(segs[status.get(lo - 1)], segs[idx])) {
          answer(status.get(lo - 1), idx);
          return 1;
        }
        if (lo + 1 < status.size() && intersect(segs[status.get(lo + 1)], segs[idx])) {
          answer(status.get(lo + 1), idx);
          return 1;
        }
      } else {
        int at = status.indexOf(idx);
        if (at > 0 && intersect(segs[status.get(at - 1)], segs[idx])) {
          answer(status.get(at - 1), idx); // shared endpoint below
          return 1;
        }
        if (at + 1 < status.size() && intersect(segs[status.get(at + 1)], segs[idx])) {
          answer(status.get(at + 1), idx); // shared endpoint above
          return 1;
        }
        status.remove(at);
        int below = at - 1, above = at; // the removal made these adjacent
        if (below ≥ 0 && above < status.size()
            && intersect(segs[status.get(below)], segs[status.get(above)])) {
          answer(status.get(below), status.get(above));
          return 1;
        }
      }
    }
```

listing 755: java, the ArrayList status, the binary insert by cross-multiplied height, the three neighbor checks (source: dsa/samples-java/src/Ch23/Sweep.java, lines 163-203)

```
  const status = []; // a list stands in for the balanced tree
  for (const ev of events) {
    const cur = segs[ev.i];
    if (ev.left) {
      let lo = 0;
      let hi = status.length;
      while (lo < hi) {
        const mid = lo + Math.floor((hi - lo) / 2);
        if (compareAt(segs[status[mid]], cur, ev.x) < 0) lo = mid + 1;
        else hi = mid;
      }
      status.splice(lo, 0, ev.i);
      if (lo > 0 && intersect(segs[status[lo - 1]], cur)) {
        return pair(status[lo - 1], ev.i);
      }
      if (lo + 1 < status.length && intersect(segs[status[lo + 1]], cur)) {
        return pair(status[lo + 1], ev.i);
      }
    } else {
      const at = status.indexOf(ev.i);
      if (at > 0 && intersect(segs[status[at - 1]], cur)) {
        return pair(status[at - 1], ev.i); // shared endpoint below
      }
      if (at + 1 < status.length && intersect(segs[status[at + 1]], cur)) {
        return pair(status[at + 1], ev.i); // shared endpoint above
      }
      status.splice(at, 1);
      const below = at - 1;
      const above = at; // the removal made these adjacent
      if (
        below ≥ 0 &&
        above < status.length &&
        intersect(segs[status[below]], segs[status[above]])
      ) {
        return pair(status[below], status[above]);
      }
    }
  }
  return null;
}
```

listing 756: javascript, the status array with a splice insert, the three neighbor checks (source: dsa/samples-js/src/ch23-sweep.mjs, lines 122-161)

```
    status = []  # a list stands in for the balanced tree
    for x, _, left, idx in events:
        cur = segs[idx]
        if left:
            lo, hi = 0, len(status)
            while lo < hi:
                mid = (lo + hi) // 2
                if compare_at(segs[status[mid]], cur, x) < 0:
                    lo = mid + 1
                else:
                    hi = mid
            status.insert(lo, idx)
            if lo > 0 and intersect(segs[status[lo - 1]], cur):
                return pair(status[lo - 1], idx)
            if lo + 1 < len(status) and intersect(segs[status[lo + 1]], cur):
                return pair(status[lo + 1], idx)
        else:
            at = status.index(idx)
            if at > 0 and intersect(segs[status[at - 1]], cur):
                return pair(status[at - 1], idx)  # shared endpoint below
            if at + 1 < len(status) and intersect(segs[status[at + 1]], cur):
                return pair(status[at + 1], idx)  # shared endpoint above
            status.pop(at)
            below, above = at - 1, at  # the removal made these adjacent
            if below ≥ 0 and above < len(status):
                if intersect(segs[status[below]], segs[status[above]]):
                    return pair(status[below], status[above])
    return None
```

listing 757: python, the status list walk with the binary insert inlined (source: dsa/samples-py/src/Ch23/sweep.py, lines 112-139)

```
  local status = {} -- a list stands in for the balanced tree
  for _, ev in ipairs(events) do
    local x, left, idx = ev[1], ev[3], ev[4]
    local cur = segs[idx]
    if left then
      local at = insert_sorted(segs, status, idx, x)
      if at > 1 and intersect(segs[status[at - 1]], cur) then
        return pair(status[at - 1], idx)
      end
      if at < #status and intersect(segs[status[at + 1]], cur) then
        return pair(status[at + 1], idx)
      end
    else
      local at = 0
      for k, s in ipairs(status) do
        if s == idx then at = k end
      end
      if at > 1 and intersect(segs[status[at - 1]], cur) then
        return pair(status[at - 1], idx) -- shared endpoint below
      end
      if at < #status and intersect(segs[status[at + 1]], cur) then
        return pair(status[at + 1], idx) -- shared endpoint above
      end
      table.remove(status, at)
      local below, above = at - 1, at -- the removal made these adjacent
      if below ≥ 1 and above ≤ #status
        and intersect(segs[status[below]], segs[status[above]]) then
        return pair(status[below], status[above])
      end
    end
  end
  return nil
end
```

listing 758: lua, the status walk over 1-based positions, the adjacency check after removal (source: dsa/samples-lua/ch23_sweep.lua, lines 124-156)

The substrate splits along the same lines as the rest of the chapter. C shifts the status with memmove inside a fixed array, Go inserts and deletes through the slices package, Java binary-inserts into an ArrayList of segment indices and finds its erasures by indexOf, JavaScript splices and Python inserts into a list, both with the binary search inlined, and Lua moves table positions. The height compare and the binary insert sit in helper functions just above the window in C, Go, Java, and Lua, and every height compare stays an exact cross multiplication, the same integer predicate the orientation section opened with. The triple point detector needs crossings as reduced fractions so distinct points never collide, integer triple keys in C, Go, and Python, string keys in JavaScript and Lua, a tuple key in the C# original, and in Java the reduced fraction carried as a three-long array compared elementwise. Each tree picked its own three non-vertical segments through one point for that error lane, so the instance differs while the contract does not.

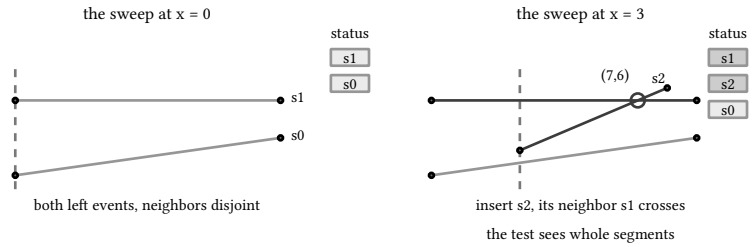


figure 196: the sweep at two consecutive events, the status stack beside each frame, the neighbor comparison at the insert finds the crossing

23.5. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's four matrix topics, the c# file covering all four in one file:

language	build sloc	container dependency	boundary note
c	487	`qsort` + fixed arrays	long long coordinates, every fixture cross-checked by brute force, the sweep over event structs and memmove shifts
c#	228	`List<T>`, `Enumerable.OrderBy`	one file for the whole chapter, seeded 500-point agreement
go	326	`slices`, `cmp`	merges y order through the recursion, int64 products into a float64 best, the sweep on slices insert and delete
java	488	jdk 27 stdlib	records for the points and events, segments as long arrays in the sweep, `Long.signum` off the cross, the strip re-sorted by a comparator, not one double in the chapter
javascript	227	`Array.sort` comparators	float only in the closest pair, the sweep exact in integer doubles
python	296	`sorted`, `set`	tuple points, squared integer distances, the sweep keyed by fraction tuples
lua	413	`table.sort`	native int64 exactness, deterministic LCG clouds, no `math.random`

The sweep line section measured the same way, the c# column counting its region of the shared `Geometry.cs`:

section	c	c#	go	java	javascript	python	lua
the sweep line	240	126	175	218	140	136	190

The sweep is the widest section in every tree, the status machinery, the triple point rejection, and the segment predicate all riding in one file per language, against a region of the shared C# file that leans on the orientation helpers above it.

sources: learn.microsoft.com, `List<T>.Insert` behind the status, `Dictionary.GetValueOrDefault` counting crossings, `Enumerable.OrderBy` sorting the hull input, clrs third edition chapter 33 for the sweep and the strip argument, go.dev for `slices.SortFunc` and `cmp.Compare`, developer.mozilla.org for `Math.hypot`, docs.python.org for `sorted` and `set`, lua.org for `table.sort` and `math.type`, accessed 2026-09-12 and 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 23: c 4 files and 51 checks, c# 11 tests, go 13 tests, java 4 files and 72 checks, javascript 16 tests, python 4 files and 41 asserts, lua 21 checks, zero skipped.

24. lattice geometry and grid algorithms

Integer grids get their own geometry. Polygons with corners on the lattice have exact areas, exact boundary counts, and an exact interior count, all in integer arithmetic, and grid traversal reduces to a handful of neighbor deltas walked with a visited set. Four tools cover the space: the shoelace formula with pick's theorem, even-odd ray casting, flood fill, and the delta tables. They are the working core of grid puzzle solving, the same pair that answers polygon digging problems.

24.1. shoelace area and pick's theorem

Walk the polygon's vertices in order and sum x i y j minus x j y i over each edge. The total is twice the signed area, positive when the cycle runs counter clockwise, and doubling keeps every coordinate product an integer. The boundary count comes from the same walk: an edge between lattice points crosses exactly gcd of dx, dy interior lattice points, so the boundary sum is one gcd per edge. Pick's theorem closes the triangle, area equals interior plus boundary over 2 minus 1, so the interior count falls out of two exact integers.

The dry run: the fixture is the notch, a 6 by 4 block with a 2 by 2 bite out of the top, asserted by the C# and lua suites as the triple 40, 24, 9, while C runs a square and a pentagon, java a square, a legs-6 triangle, and a pentagon, python a 4 by 3 rectangle, and javascript halves the sum early.

1. The walk circles counter clockwise from (0,0) through (6,0), (6,4), (4,4), (4,2), (2,2), (2,4), (0,4), eight edges, one shoelace term and one gcd each.
2. The eight terms sum 0 + 24 + 8 - 8 + 4 + 4 + 8 + 0 = 40, the 6 × 4 block minus the 2 × 2 bite, doubled.
3. The eight gcds sum 6 + 4 + 2 + 2 + 2 + 2 + 2 + 4 = 24, the notch's four short walls adding 2 each.
4. Pick in doubled units lands (40 - 24 + 2) / 2 = 9 interior points.
5. The sibling square reads 32 counter clockwise and −32 reversed, the sign carrying the walk direction, and its pick is the same 9.
6. The brute sweep over the bounding box, boundary excluded, counts the same 9, the cross-check both suites run.

edge	term	gcd
(0,0) to (6,0)	0	6
(6,0) to (6,4)	24	4
(6,4) to (4,4)	8	2
(4,4) to (4,2)	−8	2
(4,2) to (2,2)	4	2
(2,2) to (2,4)	4	2
(2,4) to (0,4)	8	2
(0,4) to (0,0)	0	4

The triple 40, 24, 9 is what both suites assert, and the listings below walk it in seven languages.

```c
// signed, doubled area; positive for counterclockwise vertex order
static long long shoelace2(const Pt *poly, int len) {
  long long s = 0;
  for (int i = 0; i < len; i++) {
    Pt a = poly[i], b = poly[(i + 1) % len];
    s += a.x * b.y - b.x * a.y;
  }
  return s;
}

static long long gcd_ll(long long a, long long b) {
  while (b) {
    long long t = a % b;
    a = b;
    b = t;
  }
  return a;
}

// lattice points on the boundary: sum of gcd(|dx|, |dy|) per edge
static long long boundary_points(const Pt *poly, int len) {
  long long b = 0;
  for (int i = 0; i < len; i++) {
    Pt a = poly[i], b2 = poly[(i + 1) % len];
    b += gcd_ll(b2.x - a.x > 0 ? b2.x - a.x : a.x - b2.x,
                b2.y - a.y > 0 ? b2.y - a.y : a.y - b2.y);
  }
  return b;
}
```

listing 759: c, the doubled signed area, gcd, and the per-edge boundary sum (source: dsa/samples-c/src/Ch24/shoelace.c, lines 21-49)

```
// ShoelaceArea returns twice the signed area of the polygon whose
// vertices are listed in order: positive for counter clockwise, so
// orientation comes free with the area. Doubling keeps integer
// coordinates exact.
func ShoelaceArea(polygon []ch23.Point) int64 {
  twice := int64(0)
  for i := 0; i < len(polygon); i++ {
    a := polygon[i]
    b := polygon[(i+1)%len(polygon)]
    twice += a.X*b.Y - b.X*a.Y
  }
  return twice
}

// BoundaryPoints counts lattice points on the polygon edges: each
// edge from a to b carries gcd(|dx|, |dy|) of them, endpoints
// included once by the wraparound sum.
func BoundaryPoints(polygon []ch23.Point) int64 {
  total := int64(0)
  for i := 0; i < len(polygon); i++ {
    a := polygon[i]
    b := polygon[(i+1)%len(polygon)]
    dx, dy := a.X-b.X, a.Y-b.Y
    if dx < 0 {
      dx = -dx
    }
    if dy < 0 {
      dy = -dy
    }
    total += gcd64(dx, dy)
  }
  return total
}

// InteriorPoints applies Pick's theorem: A = I + B/2 - 1, so the
// interior count is I = A - B/2 + 1. The doubled area keeps the
// arithmetic whole.
func InteriorPoints(polygon []ch23.Point) int64 {
  area2 := ShoelaceArea(polygon)
  if area2 < 0 {
    area2 = -area2
  }
  b := BoundaryPoints(polygon)
  return (area2 - b + 2) / 2
}
```

listing 760: go, the same three functions, points borrowed from chapter 23 (source: dsa/samples-go/ch24/shoelace.go, lines 8-52)

```java
record Pt(long x, long y) {}

// signed, doubled area; positive for counterclockwise vertex order
static long shoelace2(Pt[] poly) {
  long s = 0;
  for (int i = 0; i < poly.length; i++) {
    Pt a = poly[i], b = poly[(i + 1) % poly.length];
    s += a.x() * b.y() - b.x() * a.y();
  }
  return s;
}

static long gcd(long a, long b) {
  while (b != 0) {
    long t = a % b;
    a = b;
    b = t;
  }
  return a;
}

// lattice points on the boundary: sum of gcd(|dx|, |dy|) per edge
static long boundaryPoints(Pt[] poly) {
  long b = 0;
  for (int i = 0; i < poly.length; i++) {
    Pt a = poly[i], b2 = poly[(i + 1) % poly.length];
    b += gcd(Math.abs(b2.x() - a.x()), Math.abs(b2.y() - a.y()));
  }
  return b;
}

// pick's theorem: I = A - B/2 + 1. twice the area and the boundary
// count always share parity, so integer division is exact
static long interiorPoints(Pt[] poly) {
  return shoelace2(poly) / 2 - boundaryPoints(poly) / 2 + 1;
}
```

listing 761: java, the record point, doubled area kept whole, gcd boundary, pick with the parity-exact division (source: dsa/samples-java/src/Ch24/Shoelace.java, lines 16-51)

```
    /// <summary>Twice the signed area, positive counter-clockwise, exact in integers.</summary>
    public static long ShoelaceArea2(IReadOnlyList<(long X, long Y)> polygon)
    {
        long twice = 0;
        for (int i = 0; i < polygon.Count; i++)
        {
            (long xi, long yi) = polygon[i];
            (long xj, long yj) = polygon[(i + 1) % polygon.Count];
            twice += xi * yj - xj * yi;
        }
        return twice;
    }

    /// <summary>Lattice points on the boundary: sum of gcd(|dx|, |dy|) over the edges.</summary>
    public static long BoundaryPoints(IReadOnlyList<(long X, long Y)> polygon)
    {
        long boundary = 0;
        for (int i = 0; i < polygon.Count; i++)
        {
            (long xi, long yi) = polygon[i];
            (long xj, long yj) = polygon[(i + 1) % polygon.Count];
            boundary += Gcd(Math.Abs(xj - xi), Math.Abs(yj - yi));
        }
        return boundary;
    }

    /// <summary>Pick's theorem: I = A - B/2 + 1, written in doubled units so it stays exact.</summary>
    public static long PickInterior(IReadOnlyList<(long X, long Y)> polygon)
    {
        long area2 = ShoelaceArea2(polygon);
        long boundary = BoundaryPoints(polygon);
        return (Math.Abs(area2) - boundary + 2) / 2;
    }
```

listing 762: c#, shoelace, boundary points, pick in doubled units (source: dsa/samples/src/Ch24/Lattice.cs, lines 6-38)

```
export function shoelaceArea(polygon) {
  let sum = 0;
  const n = polygon.length;
  for (let i = 0; i < n; i++) {
    const a = polygon[i];
    const b = polygon[(i + 1) % n];
    sum += a.x * b.y - b.x * a.y;
  }
  return sum / 2;
}

// lattice points on the boundary: sum of gcd(|dx|, |dy|) over the edges
export function boundaryPoints(polygon) {
  const n = polygon.length;
  let count = 0;
  for (let i = 0; i < n; i++) {
    const a = polygon[i];
    const b = polygon[(i + 1) % n];
    count += gcd(Math.abs(b.x - a.x), Math.abs(b.y - a.y));
  }
  return count;
}

// pick: interior count from the area and boundary count
export function interiorPoints(polygon) {
  const area = Math.abs(shoelaceArea(polygon));
  const b = boundaryPoints(polygon);
  return area - b / 2 + 1;
}
```

listing 763: javascript, halves the sum early, pick flows from the half area (source: dsa/samples-js/src/ch24-shoelace.mjs, lines 6-34)

```
def shoelace(poly):
    # twice the signed area: positive when the vertices run ccw
    total = 0
    for i, (x1, y1) in enumerate(poly):
        x2, y2 = poly[(i + 1) % len(poly)]
        total += x1 * y2 - x2 * y1
    return total


def area(poly):
    return shoelace(poly) / 2


def boundary_points(poly):
    # lattice points on the edges: gcd(|dx|, |dy|) per edge
    total = 0
    for i, (x1, y1) in enumerate(poly):
        x2, y2 = poly[(i + 1) % len(poly)]
        total += gcd(abs(x2 - x1), abs(y2 - y1))
    return total


def gcd(a, b):
    while b:
        a, b = b, a % b
    return a


def interior_points(poly):
    # pick's theorem: I = A - B/2 + 1. twice the area and the boundary
    # count always share parity, so integer floor division is exact
    return shoelace(poly) // 2 - boundary_points(poly) // 2 + 1
```

listing 764: python, doubled area, gcd, floor-division pick (source: dsa/samples-py/src/Ch24/shoelace.py, lines 13-44)

```lua
-- signed doubled area: positive when the cycle runs counter-clockwise
local function shoelace_area2(poly)
  local s = 0
  local n = #poly
  for i = 1, n do
    local a, b = poly[i], poly[i % n + 1]
    s = s + a[1] * b[2] - b[1] * a[2]
  end
  return s
end

-- lattice points on the boundary: sum of gcd(|dx|, |dy|) over the edges
local function boundary_points(poly)
  local s = 0
  local n = #poly
  for i = 1, n do
    local a, b = poly[i], poly[i % n + 1]
    s = s + gcd(math.abs(b[1] - a[1]), math.abs(b[2] - a[2]))
  end
  return s
end

-- pick: I = A - B/2 + 1 where A = area2 / 2
local function pick_interior(poly)
  local a2 = shoelace_area2(poly)
  local b = boundary_points(poly)
  return (a2 - b) // 2 + 1
end
```

listing 765: lua, doubled area, boundary gcds, pick with floor division (source: dsa/samples-lua/ch24_shoelace.lua, lines 11-38)

The anchor numbers agree across the fixture families. The 4 by 4 square gives doubled area 32, the reversed walk gives −32, and pick's interior is 9 in c, c#, java, and lua. C# and lua share the notch fixture, a 6 by 4 block with a 2 by 2 bite out of the top, and pin its doubled area 40, boundary 24, interior 9 as a triple. Python uses a 4 by 3 rectangle, doubled 24, boundary 14, interior 6, and cross-checks pick against a direct sweep of the bounding box that counts strictly interior lattice points, as does lua on three shapes. Java runs the square, a legs-6 right triangle at 36, 18, 10, and a pentagon at 40, 16, 13, carries python's rectangle as a lane, and sweeps the box for the same strictly interior count. The one structural difference is javascript, which halves the shoelace sum immediately and lets the half flow into pick, while the other six keep doubled units until the final division.

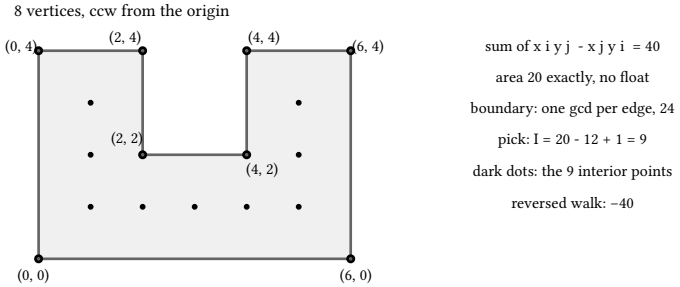


figure 197: the notch fixture with its shoelace terms, boundary gcds, and pick's interior count

24.2. even-odd ray casting

Shoot a horizontal ray east from the probe and count polygon edges that cross it. An odd count means inside. The whole craft is in the edge test: an edge crosses only when exactly one endpoint sits above the ray's height, the half-open rule, which makes vertex hits and horizontal edges count once or never instead of twice. The second

craft question is the boundary. A point exactly on an edge is a case the parity rule does not settle, and the honest answers are an exact on-boundary predicate or a stated convention, never a shrug.

The dry run: the fixture is the notch again, five inside probes and five outside asserted by the C# suite with the boundary cases separated by the exact predicate, while the siblings split over floats, integer division, and a third answer for the edge, java carrying the float and the third-answer lanes in one file.

1. The probe (1,1) meets one straddling edge, the right wall, and its cross-multiplied test reads (1 - 6) × (-4) = 20 against 0: one east crossing, the parity flips once, inside.
2. The probe (3,3), inside the notch hole, straddles four edges, the two outer walls and the two notch walls.
3. The left walls cross west and do not count, while the outer right wall and the notch's right wall cross east: two flips, the parity lands even, the hole reads outside.
4. OnBoundary settles what parity cannot: (3,2) on the notch floor and the vertex (6,4) read true by the exact cross and span test, while (3,4), above the open notch, reads false.
5. The census closes at five trues and five falses, the far away (100,100) among the falses.

edge of the notch	above y = 3	crossing	flip
floor (0,0)-(6,0)	none	-	skip
right wall (6,0)-(6,4)	one	east	yes
top lip (6,4)-(4,4)	both	-	skip
notch wall (4,4)-(4,2)	one	east	yes
notch floor (4,2)-(2,2)	none	-	skip
notch wall (2,2)-(2,4)	one	west	skip
top lip (2,4)-(0,4)	both	-	skip
left wall (0,4)-(0,0)	one	west	skip

The two east flips at (3,3) leave the hole honestly outside, and the listings below cast this ray in seven languages.

```c
// count crossings of the rightward horizontal ray from (px, py) with the
// polygon edges; odd means inside. vertices at the ray height count via
// the (yi > py) != (yj > py) half-open rule, so boundary points can read
// either way and are treated as a stated convention, not a bug
static int inside(Pt *poly, int len, double px, double py) {
  int crossings = 0;
  for (int i = 0; i < len; i++) {
    Pt a = poly[i], b = poly[(i + 1) % len];
    if ((a.y > py) != (b.y > py)) {
      double xint = (b.x - a.x) * (py - a.y) / (b.y - a.y) + a.x;
      if (px < xint)
        crossings++;
    }
  }
  return crossings & 1;
}
```

listing 766: c, the crossing loop with the half-open y rule, float intersection x (source: dsa/samples-c/src/Ch24/raycast.c, lines 21-36)

```
// PointInPolygon answers by even-odd ray casting: shoot the ray right
// from the point and count edge crossings. An odd count means inside.
// Each edge is treated as half-open in y (it includes its lower
// endpoint, excludes the upper), so a ray through a vertex counts it
// exactly once. Points exactly on the boundary are declared inside
// explicitly, before the ray is cast.
func PointInPolygon(polygon []ch23.Point, p ch23.Point) bool {
  if onBoundary(polygon, p) {
    return true // boundary rule, stated up front
  }
  inside := false
  for i := 0; i < len(polygon); i++ {
    a, b := polygon[i], polygon[(i+1)%len(polygon)]
    // does the horizontal ray at p.Y cross edge ab?
    if (a.Y > p.Y) != (b.Y > p.Y) {
      // x coordinate where the edge reaches height p.Y
      crossX := (b.X-a.X)*(p.Y-a.Y)/(b.Y-a.Y) + a.X
      if p.X < crossX {
        inside = !inside
      }
    }
  }
  return inside
}

func onBoundary(polygon []ch23.Point, p ch23.Point) bool {
  for i := 0; i < len(polygon); i++ {
    a, b := polygon[i], polygon[(i+1)%len(polygon)]
    if ch23.Orientation(a, b, p) == 0 {
      if p.X ≥ min64(a.X, b.X) && p.X <= max64(a.X, b.X) &&
        p.Y ≥ min64(a.Y, b.Y) && p.Y <= max64(a.Y, b.Y) {
        return true
      }
    }
  }
  return false
}
```

listing 767: go, boundary declared inside up front, integer division for the crossing x (source: dsa/samples-go/ch24/raycast.go, lines 5-41)

```
  record Pt(double x, double y) {}

  // count crossings of the rightward horizontal ray from (px, py) with
  // the polygon edges; odd means inside. vertices at the ray height count
  // via the (yi > py) ≠ (yj > py) half-open rule, so boundary points can
  // read either way and are treated as a stated convention, not a bug
  static boolean inside(Pt[] poly, double px, double py) {
    int crossings = 0;
    for (int i = 0; i < poly.length; i++) {
      Pt a = poly[i], b = poly[(i + 1) % poly.length];
      if ((a.y() > py) ≠ (b.y() > py)) {
        double xint = (b.x() - a.x()) * (py - a.y()) / (b.y() - a.y()) + a.x();
        if (px < xint)
          crossings++;
      }
    }
    return (crossings & 1) == 1;
  }

  // the py lane: boundary detection first, then the parity cast.
  // null means boundary: neither inside nor outside
  static Boolean insidePy(double px, double py, Pt[] poly) {
    if (onBoundary(px, py, poly))
      return null;
    int crossings = 0;
    for (int i = 0; i < poly.length; i++) {
      Pt a = poly[i], b = poly[(i + 1) % poly.length];
      if ((a.y() > py) ≠ (b.y() > py)) { // the edge straddles the ray
        double xAtY = a.x() + (py - a.y()) * (b.x() - a.x()) / (b.y() - a.y());
        if (xAtY > px) // crossing strictly right of the point
          crossings++;
      }
    }
    return crossings % 2 == 1;
  }

  static boolean onBoundary(double px, double py, Pt[] poly) {
    for (int i = 0; i < poly.length; i++) {
      Pt a = poly[i], b = poly[(i + 1) % poly.length];
      double cross = (b.x() - a.x()) * (py - a.y()) - (b.y() - a.y()) * (px - a.x());
      boolean collinear = cross == 0;
      boolean between = Math.min(a.x(), b.x()) ≤ px && px ≤ Math.max(a.x(), b.x())
          && Math.min(a.y(), b.y()) ≤ py && py ≤ Math.max(a.y(), b.y());
      if (collinear && between)
        return true;
    }
    return false;
  }
```

listing 768: java, the float cast and the py lane in one file, on-boundary returning null before the parity count (source: dsa/samples-java/src/Ch24/Raycast.java, lines 16-63)

```
    /// <summary>
    /// Even-odd ray casting: cast east from the point and count edge crossings. The
    /// comparison is exact integer cross multiplication, no floating point. Points exactly
    /// on the boundary give unspecified answers, callers test OnBoundary first.
    /// </summary>
    public static bool PointInPolygon(IReadOnlyList<(long X, long Y)> polygon, (long X, long Y) p)
    {
        bool inside = false;
        for (int i = 0, j = polygon.Count - 1; i < polygon.Count; j = i++)
        {
            (long xi, long yi) = polygon[i];
            (long xj, long yj) = polygon[j];
            if ((yi > p.Y) == (yj > p.Y))
                continue; // edge entirely on one side of the ray's height
            long dy = yj - yi; // nonzero once the edge straddles
            // p.X < intersection X, cross multiplied to stay in integers
            long lhs = (p.X - xi) * dy;
            long rhs = (xj - xi) * (p.Y - yi);
            if (dy > 0 ? lhs < rhs : lhs > rhs)
                inside = !inside;
        }
        return inside;
    }

    /// <summary>True when the point sits on a polygon edge, collinear and within its span.</summary>
    public static bool OnBoundary(IReadOnlyList<(long X, long Y)> polygon, (long X, long Y) p)
    {
        for (int i = 0; i < polygon.Count; i++)
        {
            (long xi, long yi) = polygon[i];
            (long xj, long yj) = polygon[(i + 1) % polygon.Count];
            long cross = (xj - xi) * (p.Y - yi) - (yj - yi) * (p.X - xi);
            if (cross != 0)
                continue;
            long dot = (p.X - xi) * (xj - xi) + (p.Y - yi) * (yj - yi);
            if (dot >= 0 && dot <= (xj - xi) * (xj - xi) + (yj - yi) * (yj - yi))
                return true;
        }
        return false;
    }
```

listing 769: c#, cross-multiplied comparison, no division anywhere, on-boundary separate (source: dsa/samples/src/Ch24/Lattice.cs, lines 47-86)

```
export function pointInPolygon(polygon, p) {
  let inside = false;
  const n = polygon.length;
  for (let i = 0; i < n; i++) {
    const a = polygon[i];
    const b = polygon[(i + 1) % n];
    const straddles = a.y > p.y !== b.y > p.y;
    if (!straddles) continue;
    // x where the edge reaches the probe's height
    const xAt = a.x + ((p.y - a.y) * (b.x - a.x)) / (b.y - a.y);
    if (p.x < xAt) inside = !inside;
  }
  return inside;
}
```

listing 770: javascript, the parity loop, boundary deliberately unpromised (source: dsa/samples-js/src/ch24-raycast.mjs, lines 8-21)

```
def on_boundary(p, poly):
    def between(a, b):
        return min(a[0], b[0]) ≤ p[0] ≤ max(a[0], b[0]) and min(a[1], b[1]) ≤ p[
            1
        ] ≤ max(a[1], b[1])

    def collinear(a, b):
        cross = (b[0] - a[0]) * (p[1] - a[1]) - (b[1] - a[1]) * (p[0] - a[0])
        return cross == 0

    return any(
        collinear(poly[i], poly[(i + 1) % len(poly)])
        and between(poly[i], poly[(i + 1) % len(poly)])
        for i in range(len(poly))
    )


def inside(p, poly):
    if on_boundary(p, poly):
        return None  # boundary: neither inside nor outside
    x, y = p
    crossings = 0
    for i in range(len(poly)):
        a, b = poly[i], poly[(i + 1) % len(poly)]
        if (a[1] > y) != (b[1] > y):  # the edge straddles the ray's line
            x_at_y = a[0] + (y - a[1]) * (b[0] - a[0]) / (b[1] - a[1])
            if x_at_y > x:  # crossing strictly right of the point
                crossings += 1
    return crossings % 2 == 1
```

listing 771: python, on-boundary returns a third answer, none (source: dsa/samples-py/src/Ch24/raycast.py, lines 14-42)

```lua
local function point_in(p, poly)
  local inside = false
  local n = #poly
  for i = 1, n do
    local a, b = poly[i], poly[i % n + 1]
    -- an edge crosses the eastward ray when it straddles p's y, exactly
    -- one endpoint above; vertices on the ray count half via this test
    if (a[2] > p[2]) ~= (b[2] > p[2]) then
      local x = a[1] + (p[2] - a[2]) / (b[2] - a[2]) * (b[1] - a[1])
      if x > p[1] then inside = not inside end
    end
  end
  return inside
end

local function on_boundary(p, poly)
  local n = #poly
  for i = 1, n do
    local a, b = poly[i], poly[i % n + 1]
    local cross = (b[1] - a[1]) * (p[2] - a[2]) - (b[2] - a[2]) * (p[1] - a[1])
    if cross == 0
      and p[1] ≥ math.min(a[1], b[1]) and p[1] ≤ math.max(a[1], b[1])
      and p[2] ≥ math.min(a[2], b[2]) and p[2] ≤ math.max(a[2], b[2]) then
      return true
    end
  end
  return false
end
```

listing 772: lua, the parity loop plus an exact integer on-boundary predicate (source: dsa/samples-lua/ch24_raycast.lua, lines 9-36)

The exactness spectrum is the story. The c# listing is the canonical form, the crossing test multiplies out to cross-multiplied integers and divides nowhere. Go computes the crossing x with integer division but decides the boundary first. Python and lua both cast the ray through float division and refuse to let that decide the boundary, python's on-boundary test returns a third answer, none, and lua's is an exact integer cross with a min-max box, not a float equality cast. The c suite leans the other way on purpose, floats throughout with the boundary behavior stated as a convention: on the dented pentagon the apex reads inside while a top-edge midpoint reads outside, and the tests pin that asymmetry. Java carries both roads in one file, the same float cast pinning that same asymmetry, apex inside, top-edge midpoint outside, bottom-edge midpoint inside, beside a boundary-first lane whose on-boundary test returns null, and it runs python's L-shape sweep too, 11 half-step hits. Fixtures cover the same shapes, squares and one concave notch or dent each, and python sweeps an L-shape's bounding box on half steps, counting hits against a parity argument.

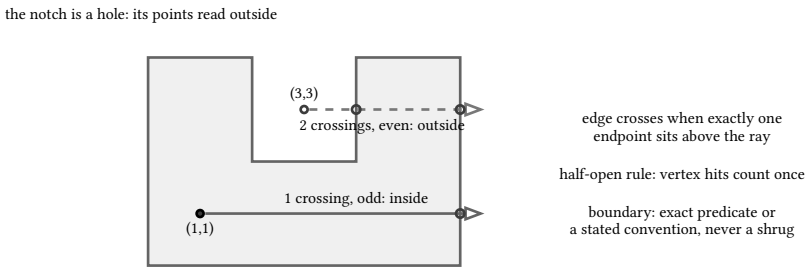


figure 198: two probes through the notch polygon, the eastward ray and its edge crossings, plus one boundary probe

24.3. flood fill

Repainting a connected region is breadth-first or depth-first search wearing work clothes. Seed the start cell, walk neighbors of the same character, mark cells when you enqueue or push them so nothing enters the frontier twice, and paint as you go. The two orders visit the same component and paint the same picture, only the discovery order differs, and every suite pins that equality.

The dry run: the fixture is the 3 by 5 land-water grid, asserted by the C# and lua suites, while python paints a 5 by 4 with two pockets, c and java walk walled 4 by 4s, javascript pins visit order under fixed deltas, and go fills an int grid.

1. The grid holds a water component of 6 with a second water pocket of 1 at (2,3), land components of 3, 4, and 1, and the sizes come back ascending: 1, 1, 3, 4, 6.
2. The fill from (0,0) marks on enqueue: (0,0) enters, (1,0) and (0,1) join behind it, and the queue empties at 3 painted cells, XXWWL, XWWLL, WWLWL.
3. Seeded at (0,2) instead the fill floods the water: (1,2) and (0,3) join first, then (1,1), then (2,1) and (2,0), six cells painting LLXXL, LXXLL, XXLWL.
4. The dfs stack visits the same six in a different order, and the assert is the two grids equal, cell by cell.
5. Refilling with the original character is the identity, every cell back to its own kind.

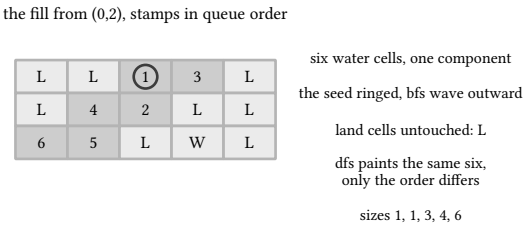


figure 199: the water fill's discovery order, stamps 1 through 6 marking when the queue met each cell

The sizes 1, 1, 3, 4, 6 are the pinned family, and the listings below flood this grid in seven languages.

```c
// bfs flood of open cells connected to (sr, sc), painting them with ch
static int flood_bfs(int sr, int sc, char ch) {
  if (grid[sr][sc] != '.')
    return 0;
  int qr[64], qc[64], qh = 0, qt = 0, cells = 0;
  grid[sr][sc] = ch;
  qr[qt] = sr;
  qc[qt] = sc;
  qt++;
  while (qh < qt) {
    int r = qr[qh], c = qc[qh];
    qh++;
    cells++;
    for (int d = 0; d < 4; d++) {
      int nr = r + DR[d], nc = c + DC[d];
      if (nr >= 0 && nr < R && nc >= 0 && nc < C && grid[nr][nc] == '.') {
        grid[nr][nc] = ch;
        qr[qt] = nr;
        qc[qt] = nc;
        qt++;
      }
    }
  }
  return cells;
}
```

listing 773: c, the bfs fill marking on enqueue, delta arrays above (source: dsa/samples-c/src/Ch24/floodfill.c, lines 33-57)

```go
func (g *Grid) FloodFillBFS(start [2]int, to int) [][2]int {
  from := g.Cells[start[0]][start[1]]
  if from == to {
    return nil
  }
  var order [][2]int
  queue := [][2]int{start}
  g.Cells[start[0]][start[1]] = to
  for len(queue) > 0 {
    cur := queue[0]
    queue = queue[1:]
    order = append(order, cur)
    for _, d := range [4][2]int{{0, 1}, {1, 0}, {0, -1}, {-1, 0}} {
      nr, nc := cur[0]+d[0], cur[1]+d[1]
      if nr < 0 || nr ≥ g.rows() || nc < 0 || nc ≥ g.cols() {
        continue
      }
      if g.Cells[nr][nc] == from {
        g.Cells[nr][nc] = to // mark on enqueue: no duplicates
        queue = append(queue, [2]int{nr, nc})
      }
    }
  }
  return order
}
```

listing 774: go, the fill returns cells in discovery order, marked on enqueue (source: dsa/samples-go/ch24/floodfill.go, lines 14-38)

```
  // bfs flood of open cells connected to (sr, sc), painting them with ch
  static int floodBfs(int sr, int sc, char ch) {
    if (grid[sr][sc] ≠ '.')
      return 0;
    int cells = 0;
    Deque<int[]> queue = new ArrayDeque<>();
    grid[sr][sc] = ch;
    queue.add(new int[] {sr, sc});
    while (!queue.isEmpty()) {
      int[] at = queue.poll();
      cells++;
      for (int d = 0; d < 4; d++) {
        int nr = at[0] + DR[d], nc = at[1] + DC[d];
        if (nr ≥ 0 && nr < R && nc ≥ 0 && nc < C && grid[nr][nc] == '.') {
          grid[nr][nc] = ch;
          queue.add(new int[] {nr, nc});
        }
      }
    }
    return cells;
  }

  // dfs twin with an explicit stack
  static int floodDfs(int sr, int sc, char ch) {
    if (grid[sr][sc] ≠ '.')
      return 0;
    int cells = 0;
    Deque<int[]> stack = new ArrayDeque<>();
    grid[sr][sc] = ch;
    stack.push(new int[] {sr, sc});
    while (!stack.isEmpty()) {
      int[] at = stack.pop();
      cells++;
      for (int d = 0; d < 4; d++) {
        int nr = at[0] + DR[d], nc = at[1] + DC[d];
        if (nr ≥ 0 && nr < R && nc ≥ 0 && nc < C && grid[nr][nc] == '.') {
          grid[nr][nc] = ch;
          stack.push(new int[] {nr, nc});
        }
      }
    }
    return cells;
  }
```

listing 775: java, the ArrayDeque bfs and its dfs twin, one marking discipline, cells painted on enqueue (source: dsa/samples-java/src/Ch24/Floodfill.java, lines 36-78)

```
    /// <summary>Sizes of every 4-connected same-character component, ascending.</summary>
    public static List<int> ComponentSizes(string[] grid)
    {
        var seen = new bool[grid.Length, grid[0].Length];
        var sizes = new List<int>();
        var queue = new Queue<(int R, int C)>();
        for (int r = 0; r < grid.Length; r++)
            for (int c = 0; c < grid[0].Length; c++)
                if (!seen[r, c])
                {
                    char kind = grid[r][c];
                    queue.Enqueue((r, c));
                    seen[r, c] = true;
                    int size = 0;
                    while (queue.Count > 0)
                    {
                        (int cr, int cc) = queue.Dequeue();
                        size++;
                        foreach ((int dr, int dc) in Deltas4)
                        {
                            int nr = cr + dr, nc = cc + dc;
                            if (nr ≥ 0 && nc ≥ 0 && nr < grid.Length && nc < grid[0].Length
                                && !seen[nr, nc] && grid[nr][nc] == kind)
                            {
                                seen[nr, nc] = true;
                                queue.Enqueue((nr, nc));
                            }
                        }
                    }
                    sizes.Add(size);
                }
        sizes.Sort();
        return sizes;
    }
```

listing 776: c#, component sizes for every same-character region, ascending (source: dsa/samples/src/Ch24/Lattice.cs, lines 88-121)

```javascript
function fill(grid, r0, c0, take) {
  const order = [];
  if (grid[r0][c0] !== ".") return order;
  const seen = grid.map((row) => row.map(() => false));
  const frontier = [[r0, c0]];
  seen[r0][c0] = true;
  while (frontier.length) {
    const [r, c] = take(frontier); // shift for bfs, pop for dfs
    order.push([r, c]);
    for (let d = 0; d < 4; d++) {
      const nr = r + DR[d];
      const nc = c + DC[d];
      if (
        nr ≥ 0 &&
        nr < grid.length &&
        nc ≥ 0 &&
        nc < grid[nr].length &&
        !seen[nr][nc] &&
        grid[nr][nc] === "."
      ) {
        seen[nr][nc] = true;
        frontier.push([nr, nc]);
      }
    }
  }
  return order;
}
```

listing 777: javascript, one fill parameterized by take, shift for bfs, pop for dfs (source: dsa/samples-js/src/ch24-floodfill.mjs, lines 8-34)

```python
def fill_bfs(grid, start, paint):
    # queue-driven flood: ripples outward from the start
    grid = clone(grid)
    r0, c0 = start
    target = grid[r0][c0]
    if target == paint:
        return grid, 0
    queue, head = [start], 0
    grid[r0][c0] = paint
    while head < len(queue):
        r, c = queue[head]
        head += 1
        for dr, dc in ((-1, 0), (1, 0), (0, -1), (0, 1)):
            nr, nc = r + dr, c + dc
            if in_bounds(grid, nr, nc) and grid[nr][nc] == target:
                grid[nr][nc] = paint
                queue.append((nr, nc))
    return grid, head
```

listing 778: python, queue plus head index, marks on enqueue (source: dsa/samples-py/src/Ch24/floodfill.py, lines 24-41)

```lua
local function fill_bfs(grid, r, c, x)
  local g = {table.unpack(grid)}
  local want = at(g, r, c)
  local seen = {}
  local queue = {{r, c}}
  local head = 1
  seen[r * 100 + c] = true
  while head ≤ #queue do
    local cur = queue[head]
    head = head + 1
    local cr, cc = cur[1], cur[2]
    g[cr] = g[cr]:sub(1, cc - 1) .. x .. g[cr]:sub(cc + 1)
    for _, d in ipairs({{-1, 0}, {1, 0}, {0, -1}, {0, 1}}) do
      local nr, nc = cr + d[1], cc + d[2]
      if nr ≥ 1 and nr ≤ #g and nc ≥ 1 and nc ≤ #g[1]
        and not seen[nr * 100 + nc] and at(g, nr, nc) == want then
        seen[nr * 100 + nc] = true
        queue[#queue + 1] = {nr, nc}
      end
    end
  end
  return g
end
```

listing 779: lua, strings repainted by splice, seen keyed by r times 100 plus c (source: dsa/samples-lua/ch24_floodfill.lua, lines 14-36)

The fixtures split by family. C# and lua share the 3 by 5 land-water grid and pin its component sizes 1, 1, 3, 4, 6, lua also pinning that the bfs and dfs fills return identical grids. Python's 5 by 4 grid carries two open pockets, 8 cells and 7 cells, and component sizes 7, 8, with the whole painted picture asserted cell by cell and dfs asserting the same. C walks a 4 by 4 with walls, and java walks its own walled 4 by 4 holding one 9-cell region and two isolated single-cell holes, its bfs and dfs twins pinned to the same painted footprint, deltas in parallel up, down, left, right arrays. Javascript pins visit orders under its fixed down, right, up, left delta order, and go fills an int grid and returns the repainted cells in the order the queue found them. The invariant under all seven: mark on enqueue, never on dequeue, or a cell enters the frontier twice and the fill loops or doubles.

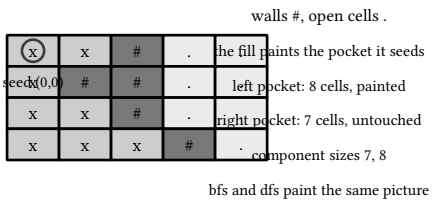


figure 200: python's 5 by 4 fixture, the left pocket of 8 painted by the fill, the right pocket of 7 untouched

24.4. neighbor deltas and 3d walks

Every grid walk in this book reduces to a delta table and a bounds check. The 4-neighborhood is the von Neumann one, the 8-neighborhood adds the diagonals, and in 3d the 6-neighborhood is the cube's faces. The delta order is free to differ between languages because only the visit order depends on it, never the reachable set.

The dry run: the fixture is a 3 by 3 for the delta enumeration and three boxes for the 3d walks, asserted by the C# suite, while C and java price the 5 by 5 edge counts, lua walks a slab, python blocks cells, and javascript walks down first.

1. The corner (0,0) on a 3 by 3 keeps 2 of the 4 deltas, (1,0) then (0,1), the bounds check filtering in listing order.
2. The center (1,1) keeps all four, (0,1), (2,1), (1,0), (1,2), and all 8 with the diagonals.
3. A bfs seeded at (0,0) over the same grid visits 9, every cell, the deltas alone carrying the walk.

4. In 3d the 6 face deltas flood the 2 by 2 by 2 to 8 and the 3 by 3 by 3 to 27.
5. The 1 by 2 by 3 slab still reads 6, the thin axis adds no wall and the box stays one component.

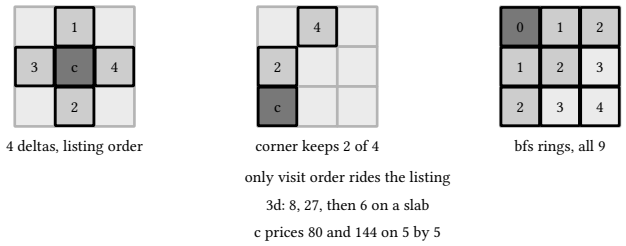


figure 201: delta order as a sequence, the center's four neighbors stamped in listing order, the corner keeping 2 of 4, and the bfs rings covering all 9

The 27 of the open cube is the pinned flood, and the listings below walk these tables in seven languages.

```c
static const int D4[4][2] = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
static const int D8[8][2] = {{-1, 0},  {1, 0},  {0, -1}, {0, 1},
                             {-1, -1}, {-1, 1}, {1, -1}, {1, 1}};

static int rows = 5, cols = 5;

static int in_bounds(int r, int c) {
  return r >= 0 && r < rows && c >= 0 && c < cols;
}

static int count_neighbors(int r, int c, const int *d, int dn) {
  int k = 0;
  for (int i = 0; i < dn; i++)
    if (in_bounds(r + d[2 * i], c + d[2 * i + 1]))
      k++;
  return k;
}

#define D3N 6
static const int D3[D3N][3] = {{1, 0, 0},  {-1, 0, 0}, {0, 1, 0},
                               {0, -1, 0}, {0, 0, 1},  {0, 0, -1}};
static int blocked3[3][3][3];
static int seen3[3][3][3];

static int flood3(int x, int y, int z) {
  if (x < 0 || x > 2 || y < 0 || y > 2 || z < 0 || z > 2)
    return 0;
  if (blocked3[x][y][z] || seen3[x][y][z])
    return 0;
  seen3[x][y][z] = 1;
  int c = 1;
  for (int d = 0; d < D3N; d++)
    c += flood3(x + D3[d][0], y + D3[d][1], z + D3[d][2]);
  return c;
}
```

listing 780: c, both delta tables, in-bounds counting, and the recursive 3d flood (source: dsa/samples-c/src/Ch24/neighbors.c, lines 17-51)

```
// Neighbors4 is the von Neumann neighborhood: up, right, down, left.
var Neighbors4 = [4][2]int{{0, 1}, {1, 0}, {0, -1}, {-1, 0}}

// Neighbors8 adds the diagonals to the 4-neighborhood: the Moore
// neighborhood, king moves on a chessboard.
var Neighbors8 = [8][2]int{
  {0, 1}, {1, 1}, {1, 0}, {1, -1},
  {0, -1}, {-1, -1}, {-1, 0}, {-1, 1},
}

// Neighbors6 is the 3D von Neumann neighborhood: the six face
// neighbors of a cube cell.
var Neighbors6 = [6][3]int{
  {0, 0, 1}, {0, 0, -1}, {0, 1, 0},
  {0, -1, 0}, {1, 0, 0}, {-1, 0, 0},
}

// InBounds2D guards a 2D step against the borders.
func InBounds2D(rows, cols, r, c int) bool {
  return r ≥ 0 && r < rows && c ≥ 0 && c < cols
}
```

listing 781: go, east-first deltas, the bounds guard, walk3d below (source: dsa/samples-go/ch24/neighbors.go, lines 3-23)

```
  static final int[][] D4 = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}};
  static final int[][] D8 = {{-1, 0}, {1, 0}, {0, -1}, {0, 1},
      {-1, -1}, {-1, 1}, {1, -1}, {1, 1}};

  static final int ROWS = 5, COLS = 5;

  static boolean inBounds(int r, int c) {
    return r ≥ 0 && r < ROWS && c ≥ 0 && c < COLS;
  }

  static int countNeighbors(int r, int c, int[][] d) {
    int k = 0;
    for (int[] delta : d)
      if (inBounds(r + delta[0], c + delta[1]))
        k++;
    return k;
  }

  static final int[][] D3 = {{1, 0, 0}, {-1, 0, 0}, {0, 1, 0},
      {0, -1, 0}, {0, 0, 1}, {0, 0, -1}};
  static boolean[][][] blocked3 = new boolean[3][3][3];
  static boolean[][][] seen3 = new boolean[3][3][3];

  static int flood3(int x, int y, int z) {
    if (x < 0 || x > 2 || y < 0 || y > 2 || z < 0 || z > 2)
      return 0;
    if (blocked3[x][y][z] || seen3[x][y][z])
      return 0;
    seen3[x][y][z] = true;
    int c = 1;
    for (int[] d : D3)
      c += flood3(x + d[0], y + d[1], z + d[2]);
    return c;
  }
```

listing 782: java, the two delta tables pinned cell by cell, the recursive 3d flood, walks keyed by packed longs (source: dsa/samples-java/src/Ch24/Neighbors.java, lines 21-54)

```
    /// <summary>4-neighborhood deltas: up, down, left, right.</summary>
    public static readonly (int Dr, int Dc)[] Deltas4 = { (-1, 0), (1, 0), (0, -1), (0, 1) };

    /// <summary>8-neighborhood deltas: the 4 above plus the diagonals.</summary>
    public static readonly (int Dr, int Dc)[] Deltas8 =
    {
        (-1, 0), (1, 0), (0, -1), (0, 1), (-1, -1), (-1, 1), (1, -1), (1, 1),
    };

    /// <summary>3D 6-neighborhood deltas along every axis both ways.</summary>
    public static readonly (int Dr, int Dc, int Dk)[] Deltas6 =
    {
        (-1, 0, 0), (1, 0, 0), (0, -1, 0), (0, 1, 0), (0, 0, -1), (0, 0, 1),
    };

    /// <summary>In-bounds 4-neighbors of a cell, in delta order.</summary>
    public static IEnumerable<(int R, int C)> Neighbors4((int R, int C) cell, int rows, int cols) =>
        Deltas4.Select(d => (cell.R + d.Dr, cell.C + d.Dc))
            .Where(n => n.Item1 ≥ 0 && n.Item2 ≥ 0 && n.Item1 < rows && n.Item2 < cols);

    /// <summary>In-bounds 8-neighbors of a cell, in delta order.</summary>
    public static IEnumerable<(int R, int C)> Neighbors8((int R, int C) cell, int rows, int cols) =>
        Deltas8.Select(d => (cell.R + d.Dr, cell.C + d.Dc))
            .Where(n => n.Item1 ≥ 0 && n.Item2 ≥ 0 && n.Item1 < rows && n.Item2 < cols);
```

listing 783: c#, the three delta arrays, neighbors filtered in delta order (source: dsa/samples/src/Ch24/Lattice.cs, lines 168-191)

```
export const N4 = [
  [-1, 0],
  [1, 0],
  [0, -1],
  [0, 1],
];

export const N8 = [
  [-1, -1],
  [-1, 0],
  [-1, 1],
  [0, -1],
  [0, 1],
  [1, -1],
  [1, 0],
  [1, 1],
];

export const N6_3D = [
  [-1, 0, 0],
  [1, 0, 0],
  [0, -1, 0],
  [0, 1, 0],
  [0, 0, -1],
  [0, 0, 1],
];

export function inBounds(rows, cols, r, c) {
  return r ≥ 0 && r < rows && c ≥ 0 && c < cols;
}

// the deltas that stay inside the grid
export function neighbors(r, c, rows, cols, deltas) {
  return deltas
    .map(([dr, dc]) => [r + dr, c + dc])
    .filter(([nr, nc]) => inBounds(rows, cols, nr, nc));
}
```

listing 784: javascript, three tables, map and filter, a 3d walk over string keys (source: dsa/samples-js/src/ch24-neighbors.mjs, lines 4-40)

```
FOUR = ((-1, 0), (1, 0), (0, -1), (0, 1))
EIGHT = FOUR + ((-1, -1), (-1, 1), (1, -1), (1, 1))
SIX3D = ((-1, 0, 0), (1, 0, 0), (0, -1, 0), (0, 1, 0), (0, 0, -1), (0, 0, 1))


def walk_2d(blocked, start, deltas):
    rows, cols = len(blocked), len(blocked[0])
    seen = {start}
    stack = [start]
    while stack:
        r, c = stack.pop()
        for dr, dc in deltas:
            nr, nc = r + dr, c + dc
            if (
                0 <= nr < rows
                and 0 <= nc < cols
                and not blocked[nr][nc]
                and (nr, nc) not in seen
            ):
                seen.add((nr, nc))
                stack.append((nr, nc))
    return seen
```

listing 785: python, tuple deltas, a stack walk over a seen set (source: dsa/samples-py/src/Ch24/neighbors.py, lines 13-34)

```
local D4 = {{-1, 0}, {1, 0}, {0, -1}, {0, 1}} -- up, down, left, right

local D8 = {
  {-1, -1}, {-1, 0}, {-1, 1},
  {0, -1}, {0, 1},
  {1, -1}, {1, 0}, {1, 1},
}

local function neighbors(r, c, rows, cols, deltas)
  local out = {}
  for _, d in ipairs(deltas) do
    local nr, nc = r + d[1], c + d[2]
    if nr >= 0 and nr < rows and nc >= 0 and nc < cols then
      out[#out + 1] = {nr, nc}
    end
  end
  return out
end

local D6 = {
  {-1, 0, 0}, {1, 0, 0},
  {0, -1, 0}, {0, 1, 0},
  {0, 0, -1}, {0, 0, 1},
}
```

listing 786: lua, the tables and the bounds-checked emitter (source: dsa/samples-lua/ch24_neighbors.lua, lines 6-29)

The c and java suites pin the arithmetic the others skip: over a 5 by 5 grid the 4-neighborhood sums to 80 directed edges and the 8-neighborhood to 144, a corner keeps 2 of 4 and 3 of 8, java also pricing an edge cell at 3 and 5. The 3d walks agree on the volumes: all seven languages flood a box and count the visited cells, c pins 27 for the open 3 by 3 by 3, 26 with the center blocked, and 9 when a full plane splits the cube into halves, java pinning those same three counts and carrying python's 2 by 2 by 2 lanes, the center hole at 7 and the wall slab at 4, lua pins 8, 27,

and 6 for a 1 by 2 by 3 slab, python works a 2 by 2 by 2 with a center hole, 7 of 8 visited, and a wall slab cutting the count to 4. Delta order differs per language, c lists up, down, left, right, go starts east, javascript walks down first, java matching c's up, down, left, right in parallel delta arrays, and the floodfill fixtures above prove the order never changes the region.

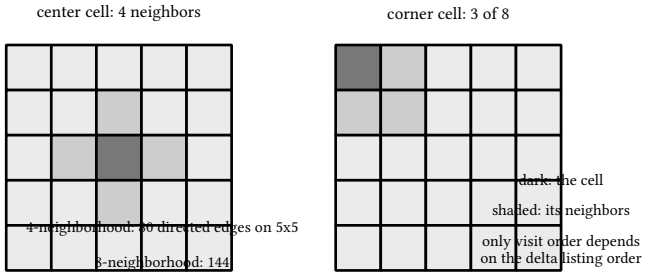


figure 202: one 5 by 5 grid, the center cell's 4 and 8 neighborhoods, and a corner's 2 and 3

24.5. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's four sample files, test scripts included where the language embeds them:

language	build sloc	container dependency	boundary note
c	319	fixed char arrays, int deltas	float ray intersection with boundary conventions stated, long long shoelace
c#	156	tuples, `Queue`, `Stack`	the one fully cross-multiplied ray cast, on-boundary separate
go	191	slices, `ch23.Point` reuse	integer division crossing x, boundary declared inside up front
java	442	jdk 27 stdlib	the float cast and the null boundary lane in one ray file, 3d walks keyed by coordinates packed into a long, component sizes sorted descending
javascript	130	`Array` ops, `Set` keys	halves the shoelace sum early, boundary unpromised
python	239	`list` grids, `set` seen	on-boundary returns a third answer, none
lua	306	`table`, string splice fills	exact integer on-boundary, seen keyed by r times 100 plus c

sources: learn.microsoft.com for `Queue<T>` and `Stack<T>`, go.dev for package layout and `min` and `max` builtins, developer.mozilla .org for `Array.prototype.shift`, docs.python.org for `enumerate` and floor division, lua.org for `table.unpack` and `math.abs`, and the pick's theorem statement as standard lattice geometry, accessed 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 24: c 4 files and 63 checks, c# 12 tests, go 12 tests, java 4 files and 94 checks, javascript 12 tests, python 4 files and 36 asserts, lua 16 checks, zero skipped.

25. practical structures

The book so far has chased exact answers, and production systems chase affordable ones. Redis evicts keys from an approximate lru order, leveldb bolts a bloom filter onto every sstable and keeps its memtable in a skip list, cdns use one to turn one-hit urls away at the door. This chapter builds the four structures behind those moves, each a screen of code with its behavior pinned by tests, and chapter 43′s engine ships 2 of them with the same hash idioms taught here.

25.1. the bloom filter

A set that may report a stranger as present and never reports a member as absent. Adding a key sets k bit positions chosen by hashes, a probe that finds a clear bit answers no for certain, and a probe that finds all k set answers maybe. The maybe rate is bought up front with arithmetic, m = -n ln p / (ln 2)^2 bits for n expected keys at target rate p, then k = m/n ln 2 positions per key. At n = 10,000 and p = 0.01 that is 95,851 bits, 9.6 per key, and 7 positions.

The dry run: the fixture is that design point and its probe census, asserted by the C# suite, while C and java bound lies at 6 of 40, go and lua pair fnv-1a with djb2, javascript runs bigint words, and python's 8-bit arena pins its lies by name, java carrying that 8-bit lane beside its own bounds.

1. Sizing prices the filter first: m = -n ln p / (ln 2)² rounds up to 95,851 bits, and k = 95,851 / 10,000 × ln 2 = 6.64, rounded up to 7 positions per key.
2. The whole arena costs 95,851 / 8 = 11,982 bytes, just under 12 kilobytes to screen 10,000 keys.
3. All 10,000 keys added, every member probe answers maybe, the no-false-negative assert: bits only ever set.
4. Probing 10,000 absent keys then measures 59 lies, 59 / 10,000 = 0.59 percent, inside the asserted bound of 200 and under the 1 percent design.
5. The lies cluster because the 7 positions come from one hash pair, never fully independent, so the realized rate rides either side of the design point.

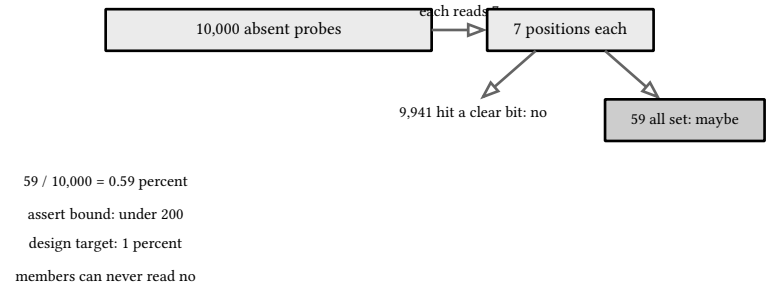


figure 203: the absent-probe census as a run, 10,000 strangers each reading 7 positions, 9,941 stopped by a clear bit

The 59 of 10,000 inside a 95,851-bit arena is the pinned trade, and the listings below buy it in seven languages.

```c
static unsigned long long fnv1a_seeded(const char *s, unsigned long long seed) {
  unsigned long long h = 14695981039346656037ULL ^ seed;
  while (*s) {
    h ^= (unsigned char)*s++;
    h *= 1099511628211ULL;
  }
  return h;
}

#define MAXBITS 256

static unsigned char bits[MAXBITS / 8];
static int nbits;

static void bf_init(int m) {
  nbits = m;
  for (int i = 0; i < MAXBITS / 8; i++)
    bits[i] = 0;
}

static void set_bit(int i) { bits[i / 8] |= (unsigned char)(1 << (i % 8)); }

static int get_bit(int i) { return (bits[i / 8] >> (i % 8)) & 1; }

static unsigned long long djb2(const char *s) {
  unsigned long long h = 5381;
  while (*s)
    h = h * 33 + (unsigned char)*s++;
  return h;
}

// two structurally different mixers: a re-seeded copy of one hash family
// correlates its bit choices and inflates the false positive rate
static int h1(const char *s) { return (int)(fnv1a_seeded(s, 0) % nbits); }

static int h2(const char *s) { return (int)(djb2(s) % nbits); }

static void bf_add(const char *s) {
  set_bit(h1(s));
  set_bit(h2(s));
}

static int bf_maybe(const char *s) { return get_bit(h1(s)) && get_bit(h2(s)); }
```

listing 787: c, fnv-1a against djb2, the second mixer chosen so the feeds stay independent (source: dsa/samples-c/src/Ch25/bloom.c, lines 19-61)

```go
// bitSlots maps a key to its two bit positions, one per hash, by
// double hashing: h1 and h1+h2 give independent-looking indices.
func (b *Bloom) bitSlots(key string) ([2]int, [2]uint) {
  m := uint64(len(b.bits) * 64)
  h1 := fnv1a64(key)
  h2 := djb2(key)
  p1 := h1 % m
  p2 := (h1 + h2) % m
  return [2]int{int(p1 / 64), int(p2 / 64)}, [2]uint{uint(p1 % 64), uint(p2 % 64)}
}

// Add remembers key: both of its bits go high.
func (b *Bloom) Add(key string) {
  words, bits := b.bitSlots(key)
  for i := range words {
    b.bits[words[i]] |= 1 << bits[i]
  }
}

// Contains reports whether key was added, or a false positive.
func (b *Bloom) Contains(key string) bool {
  words, bits := b.bitSlots(key)
  for i := range words {
    if b.bits[words[i]]&(1<<bits[i]) == 0 {
      return false // one clear bit is enough to say no
    }
  }
  return true
}
```

listing 788: go, bit slots by double hashing, h1 and h1 plus h2 into one word array (source: dsa/samples-go/ch25/bloom.go, lines 19-47)

```
static final long FNV_OFFSET = Long.parseUnsignedLong("14695981039346656037");
static final long FNV_PRIME = 1099511628211L;

static long fnv1aSeeded(String s, long seed) {
  long h = FNV_OFFSET ^ seed;
  for (int i = 0; i < s.length(); i++) {
    h ^= s.charAt(i);
    h *= FNV_PRIME;
  }
  return h;
}

static long djb2(String s) {
  long h = 5381;
  for (int i = 0; i < s.length(); i++)
    h = h * 33 + s.charAt(i);
  return h;
}

static final int MAXBITS = 256;

static byte[] bits = new byte[MAXBITS / 8];
static int nbits;

static void bfInit(int m) {
  nbits = m;
  java.util.Arrays.fill(bits, (byte) 0);
}

static void setBit(int i) {
  bits[i / 8] |= (byte) (1 << (i % 8));
}

static int getBit(int i) {
  return bits[i / 8] >> (i % 8) & 1;
}

// two structurally different mixers: a re-seeded copy of one hash
// family correlates its bit choices and inflates the false positive rate
static int h1(String s) {
  return (int) Long.remainderUnsigned(fnv1aSeeded(s, 0), nbits);
}

static int h2(String s) {
  return (int) Long.remainderUnsigned(djb2(s), nbits);
}

static void bfAdd(String s) {
  setBit(h1(s));
  setBit(h2(s));
}

static boolean bfMaybe(String s) {
  return getBit(h1(s)) == 1 && getBit(h2(s)) == 1;
}
```

listing 789: java, fnv-1a with djb2, every hash mod through `Long.remainderUnsigned`, the offset parsed unsigned (source: dsa/samples-java/src/Ch25/Bloom.java, lines 25-79)

```
    public BloomFilter(long expectedKeys, double falsePositiveRate = 0.01)
    {
        if (expectedKeys < 1) throw new ArgumentOutOfRangeException(nameof(expectedKeys));
        if (falsePositiveRate is <= 0 or >= 1) throw new
ArgumentOutOfRangeException(nameof(falsePositiveRate));

        // m = -n ln p / (ln 2)^2, then k = m/n ln 2
        double m = -expectedKeys * Math.Log(falsePositiveRate) / (Math.Log(2) * Math.Log(2));
        _bits = (int)Math.Ceiling(m);
        _hashes = Math.Max(1, (int)Math.Ceiling(_bits / (double)expectedKeys * Math.Log(2)));
        _words = new ulong[(_bits + 63) / 64];
    }

    public int Bits => _bits;
    public int Hashes => _hashes;

    public void Add(string key)
    {
        (ulong h1, ulong h2) = HashPair(key);
        for (int i = 0; i < _hashes; i++)
        {
            // kirsch-mitzenmacher: g_i(x) = h1 + i * h2
            ulong bit = (h1 + (ulong)i * h2) % (ulong)_bits;
            _words[bit / 64] |= 1UL << (int)(bit % 64);
        }
    }

    public bool MightContain(string key)
    {
        (ulong h1, ulong h2) = HashPair(key);
        for (int i = 0; i < _hashes; i++)
        {
            ulong bit = (h1 + (ulong)i * h2) % (ulong)_bits;
            if ((_words[bit / 64] & (1UL << (int)(bit % 64))) == 0)
                return false; // one clear bit is a definite no
        }
        return true; // all set: present, or a one percent lie
    }

    // fnv-1a twice with different seeds, chapter 6's hash at work
    private static (ulong H1, ulong H2) HashPair(string key)
    {
        ulong h1 = 0xcbf29ce484222325UL, h2 = 0x84222325cbf29ce5UL;
        foreach (byte b in Encoding.UTF8.GetBytes(key))
        {
            h1 = (h1 ^ b) * 0x100000001b3UL;
            h2 = (h2 ^ (b ^ 0x5aUL)) * 0x100000001b3UL;
        }
        return (h1, h2 | 1); // odd stride so consecutive probes spread
    }
```

listing 790: c#, sizing in the ctor, kirsch-mitzenmacher probes off one fnv pair, an odd stride (source: dsa/samples/src/Ch25/Practical.cs, lines 12-60)

```
export class Bloom {
  #bits;
  #m;

  constructor(m = 64) {
    this.#m = m;
    this.#bits = new Array(m).fill(false);
  }

  #h1(s) {
    return Number(fnv1a64(s) % BigInt(this.#m));
  }

  #h2(s) {
    return Number(djb2x64(s) % BigInt(this.#m));
  }

  add(s) {
    this.#bits[this.#h1(s)] = true;
    this.#bits[this.#h2(s)] = true;
    return this;
  }

  mightContain(s) {
    return this.#bits[this.#h1(s)] && this.#bits[this.#h2(s)];
  }
}
```

listing 791: javascript, the 64-bit hashes imported from chapter 6 as bigint (source: dsa/samples-js/src/ch25-bloom.mjs, lines 9-35)

```
def h1(key):
    return key % BITS


def h2(key):
    return 3 * key % BITS


class Bloom:
    def __init__(self):
        self.bits = [False] * BITS

    def add(self, key):
        self.bits[h1(key)] = True
        self.bits[h2(key)] = True

    def might_contain(self, key):
        return self.bits[h1(key)] and self.bits[h2(key)]
```

listing 792: python, an 8-bit filter where the false positives pin exactly (source: dsa/samples-py/src/Ch25/bloom.py, lines 17-34)

```lua
local MBITS = 512

local function bloom_new()
  return {bits = {}, n = 0}
end

-- lua's % is floor modulo; both hashes stay non-negative for byte input
local function bloom_add(b, key)
  b.bits[fnv1a(key) % MBITS + 1] = true
  b.bits[djb2(key) % MBITS + 1] = true
  b.n = b.n + 1
end

local function bloom_might_contain(b, key)
  return b.bits[fnv1a(key) % MBITS + 1] and b.bits[djb2(key) % MBITS + 1] or false
end
```

listing 793: lua, fnv-1a and djb2 over 512 bits, floor modulo with a plus one (source: dsa/samples-lua/ch25_bloom.lua, lines 27-42)

The 2 hashes are one fnv-1a pass with 2 seeds, chapter 6's hash run twice with the second seed mixing a byte into every round, and the k positions come from the kirsch-mitzenmacher combination g_i = h_1 + i h_2, one multiply per probe instead of k separate hash functions. The tests pin the design point exactly, 95,851 bits and 7 positions, then add all 10,000 keys and probe each one, and zero report absent because bits only ever set. Probing 10,000 absent keys measured 59 lies, 0.59 percent, under the 1 percent target, and that is expected, the k positions drawn from a single hash pair are never fully independent, so the realized rate can land on either side of the design. The whole filter sits in just under 12 kilobytes, and 1 absent lookup in 100 still pays the disk while the other 99 skip it.

The second hash differs per language, and the choice is measurable. The c listing states the reason in a comment: a re-seeded copy of one hash family correlates its bit choices and inflates the rate, so its second feed is djb2 against fnv-1a, false positives bounded at 6 of 40 probes on 64 bits and 2 of 40 on 256. Go and lua pair the same two hashes, go combining them double-hash style into h1 and h1 plus h2, lua spreading 64 keys over 512 bits with the bound at 10 of 64 and a live-bit count pinned strictly between 64 and 128, proving the two feeds really disagree. Java pairs the same two feeds with the same correlation comment and mirrors C's bounds, 6 of 40 at 64 bits and 2 of 40 at 256, but the road there is its own: the fnv offset 14695981039346656037 sits past 2^63, so the constant parses through `Long.parseUnsignedLong` and every modulo runs through `Long.remainderUnsigned`, signed % disagreeing exactly where a hash crosses 2^63. Java also mirrors a C quirk frozen in the fixture: the query array declares 40 slots and fills 39, so the 40th probe is the empty string, and the java twin hashes that empty string too. Javascript imports both 64-bit words from chapter 6 and runs them on bigint, dropping back to number at the modulo. Python shrinks the arena to 8 bits with key mod 8 and 3 key mod 8 so the lies pin by name, 6, 9, 10, 11, java carrying that 8-bit lane with the same four names. One invariant never bends in any of the seven: members always answer yes, because bits only ever set.

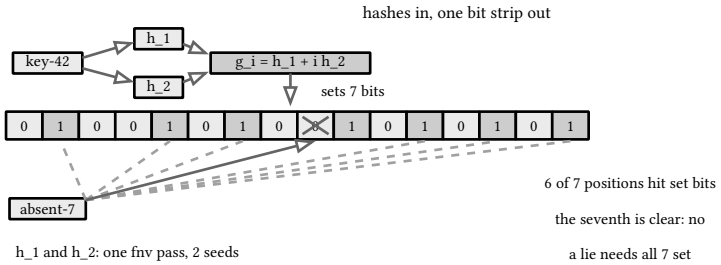


figure 204: one key through the fnv pair into k = 7 positions over a 16 slot toy, the tested filter is 95,851 bits for 10,000 keys, and the absent probe dies on one clear bit

PITFALL: no deletes in a plain bloom filter

> Clearing the k bits of a removed key clears bits that other keys still need, which manufactures false negatives, the one answer the structure promises never to give. Systems that must delete either rebuild the filter from the surviving keys or switch to a counting variant.

25.2. the count-min sketch

The bloom filter's counting sibling answers how many, roughly. The sketch is d rows of w counters, an increment adds its count to one hash chosen cell in every row, and an estimate reads the key's d cells and returns the smallest. Collisions only ever add, so the minimum can only overestimate, and the width prices the error, with w = e over eps counters per row the expected overestimate stays within eps of the total count streamed so far.

The dry run: the fixture is python's 2 by 4 sketch, pinned cell by cell because the C# anchor asserts ranges only, 100 to 102 after a hundred adds of one key, its 2,048-wide run pricing the error instead.

1. The four adds land twice each, one cell per row, row 1 by h1 = key mod 4 and row 2 by h2 = 2 × key mod 4.
2. Row 1 collides keys 1 and 5 into slot 1, 2 + 1 = 3, and pins 0, 3, 1, 3.
3. Row 2 piles keys 1, 3, and 5 into slot 2, 2 + 3 + 1 = 6, and pins 1, 0, 6, 0.
4. Key 1 reads min(3, 6) = 3 against truth 2, and key 5 reads the same 3 against truth 1, overestimates both.
5. Keys 2 and 3 read exact, 1 and 3, the unclobbered case, and the unadded 4 reads 0.
6. Five more of key 2 accumulate in place and its estimate moves only up, to 6.

key	row 1	row 2	min	truth
1	3	6	3	2
5	3	6	3	1
3	3	6	3	3
2	1	1	1	1
4	0	0	0	unadded

The 3 against truths 2 and 1 is the pinned overestimate, and the listings below count in seven languages.

```c
// congruent mod 8 collide in both rows at once. real sketches use
// independent mixed hashes; this fixture makes the behavior legible
static int h(int row, unsigned long long key) {
  return (int)((7 * key + 13ULL * row) % WIDTH);
}

static void cm_update(unsigned long long key, long long count) {
  for (int r = 0; r < ROWS; r++)
    table[r][h(r, key)] += count;
}

static long long cm_estimate(unsigned long long key) {
  long long best = table[0][h(0, key)];
  for (int r = 1; r < ROWS; r++)
    if (table[r][h(r, key)] < best)
      best = table[r][h(r, key)];
  return best;
}
```

listing 794: c, affine hashes picked so the collisions are hand-traceable (source: dsa/samples-c/src/Ch25/countmin.c, lines 24-42)

```go
type CountMin struct {
  rows   [][]int64
  hashes []func(string) uint64
}

// NewCountMin builds a sketch with the given width and hash set.
func NewCountMin(width int, hashes []func(string) uint64) *CountMin {
  cm := &CountMin{hashes: hashes}
  for range hashes {
    cm.rows = append(cm.rows, make([]int64, width))
  }
  return cm
}

// Add records one occurrence of key.
func (cm *CountMin) Add(key string) {
  for r, hash := range cm.hashes {
    cm.rows[r][hash(key)%uint64(len(cm.rows[r]))]++
  }
}

// Estimate reports the minimum counter over the rows.
func (cm *CountMin) Estimate(key string) int64 {
  best := int64(-1)
  for r, hash := range cm.hashes {
    count := cm.rows[r][hash(key)%uint64(len(cm.rows[r]))]
    if best < 0 || count < best {
      best = count
    }
  }
  return best
}
```

listing 795: go, rows injected with hash functions, add and estimate (source: dsa/samples-go/ch25/countmin.go, lines 7-38)

```
  // h_r(k) = (7k + 13r) mod 8: both rows share the multiplier, so keys
  // congruent mod 8 collide in both rows at once. real sketches use
  // independent mixed hashes; this fixture makes the behavior legible
  static int h(int row, long key) {
    return (int) ((7 * key + 13L * row) % WIDTH);
  }

  static void cmUpdate(long key, long count) {
    for (int r = 0; r < ROWS; r++)
      table[r][h(r, key)] += count;
  }

  static long cmEstimate(long key) {
    long best = table[0][h(0, key)];
    for (int r = 1; r < ROWS; r++)
      if (table[r][h(r, key)] < best)
        best = table[r][h(r, key)];
    return best;
  }

  static void reset() {
    for (int r = 0; r < ROWS; r++)
      java.util.Arrays.fill(table[r], 0);
  }

  // the py lane: width 4 with h1 = key % 4 and h2 = 2*key % 4
  static final int PY_W = 4;

  static long[][] pyRows = new long[2][PY_W];

  static int pyH1(long key) {
    return (int) (key % PY_W);
  }

  static int pyH2(long key) {
    return (int) (2 * key % PY_W);
  }

  static void pyAdd(long key, long times) {
    pyRows[0][pyH1(key)] += times;
    pyRows[1][pyH2(key)] += times;
  }

  static long pyEstimate(long key) {
    // collisions only ever add, so the min still overestimates
    return Math.min(pyRows[0][pyH1(key)], pyRows[1][pyH2(key)]);
  }
```

listing 796: java, affine hashes sharing a multiplier so the mod-8 collision is hand-traceable, the py width-4 lane in the same file (source: dsa/samples-java/src/Ch25/Countmin.java, lines 22-68)

```
    public CountMin(int width, int depth, int seed = 0x5eed)
    {
        if (width < 1 || depth < 1)
            throw new ArgumentOutOfRangeException(width < 1 ? nameof(width) : nameof(depth));
        _width = width;
        _seed = (ulong)seed;
        _counters = new long[depth, width];
    }

    public long Total { get; private set; }

    public void Increment(string key, long count = 1)
    {
        byte[] bytes = Encoding.UTF8.GetBytes(key);
        for (int row = 0; row < _counters.GetLength(0); row++)
            _counters[row, RowIndex(row, bytes)] += count;
        Total += count;
    }

    public long Estimate(string key)
    {
        byte[] bytes = Encoding.UTF8.GetBytes(key);
        long best = long.MaxValue;
        for (int row = 0; row < _counters.GetLength(0); row++)
            best = Math.Min(best, _counters[row, RowIndex(row, bytes)]);
        return best;
    }

    // one fnv-1a pass per row, the seed rotated so rows disagree
    private int RowIndex(int row, byte[] bytes)
    {
        ulong h = 0xcbf29ce484222325UL ^ (_seed + (ulong)(uint)row * 0x9e3779b9UL);
        foreach (byte b in bytes)
            h = (h ^ b) * 0x100000001b3UL;
        return (int)(h % (ulong)_width);
    }
```

listing 797: c#, the seed rotates per row so the hashes disagree, the estimate is the row minimum (source: dsa/samples/src/Ch25/Practical.cs, lines 70-105)

```
export class CountMin {
  #table;
  #width;

  constructor(width = 4, depth = 3) {
    this.#width = width;
    this.#table = Array.from({ length: depth }, () =>
      new Array(width).fill(0),
    );
  }

  // row r mixes the two base hashes so rows disagree
  #cell(r, item) {
    const w = BigInt(this.#width);
    const mix =
      (fnv1a64(item) + BigInt(r) * djb2x64(item)) % w;
    return Number(mix);
  }

  add(item, times = 1) {
    for (let r = 0; r < this.#table.length; r++)
      this.#table[r][this.#cell(r, item)] += times;
    return this;
  }

  estimate(item) {
    return Math.min(
      ...this.#table.map((row, r) => row[this.#cell(r, item)]),
    );
  }
}
```

listing 798: javascript, rows mixed from the two bigint hashes per depth (source: dsa/samples-js/src/ch25-countmin.mjs, lines 9-39)

```
def h1(key):
    return key % WIDTH


def h2(key):
    return 2 * key % WIDTH


class CountMin:
    def __init__(self):
        self.rows = [[0] * WIDTH for _ in range(2)]

    def add(self, key, times=1):
        self.rows[0][h1(key)] += times
        self.rows[1][h2(key)] += times

    def estimate(self, key):
        # collisions only ever add, so the min still overestimates
        return min(self.rows[0][h1(key)], self.rows[1][h2(key)])
```

listing 799: python, two rows of width 4, both pinned cell by cell (source: dsa/samples-py/src/Ch25/countmin.py, lines 16-34)

```lua
local function cell_of(s, r)
  if r % 2 == 1 then return fnv1a(s) % W + 1 end
  return djb2(s) % W + 1
end

local function add(s, key, n)
  n = n or 1
  for r = 1, D do
    s.rows[r][cell_of(key, r)] = s.rows[r][cell_of(key, r)] + n
  end
  s.total = s.total + n
end

local function estimate(s, key)
  local best = math.maxinteger
  for r = 1, D do
    if s.rows[r][cell_of(key, r)] < best then best = s.rows[r][cell_of(key, r)] end
  end
  return best
end
```

listing 800: lua, the two hashes alternating across four rows, total tracked beside (source: dsa/samples-lua/ch25_countmin.lua, lines 39-58)

The seed rotates by row so the d hash functions disagree about where a key lands, one fnv pass over the same bytes per row. The never underestimates test streams counts of 1 to 50 over 100 keys into a width 64 sketch, small enough that collisions are guaranteed, and every estimate lands at or above the truth while Total reports the exact sum. The tight test uses the geometry worth remembering, width 2048 and depth 5: after 100 increments of one key and 7 of another, the heavy key reads exactly 100 because at least 1 of 5 rows is collision free, and the light key reads 7 the same way. Five rows of 2,048 long counters is 80 kilobytes to count an unbounded key space, and the expected inflation on that test stream is e over 2048 times the total, under 0.15 counts.

Every build bumps one cell per row and reads the row minimum, so overestimate-only is structural, and the six new suites make the collisions legible instead of pricing them. C pins a 2 by 8 sketch whose affine hashes share a multiplier, so key 9, congruent to key 1 mod 8, collides in both rows at once and reads the full 3 against a truth of 0, and java runs that exact fixture, the same shared multiplier putting key 9 and key 17 in key 1′s cells, with python's pinned rows in the same file. Python's 2 by 4 sketch pins both rows cell by cell, 0, 3, 1, 3 and 1, 0, 6, 0, then reads 3 for keys 1 and 5 against truths 2 and 1. Javascript mixes its two bigint hashes per row, kirsch-mitzenmacher style, and lua alternates the two hashes across 4 rows of width 16 while tracking the exact total beside the estimates. Overestimate-only is asserted everywhere: the truth is a floor, never a ceiling.

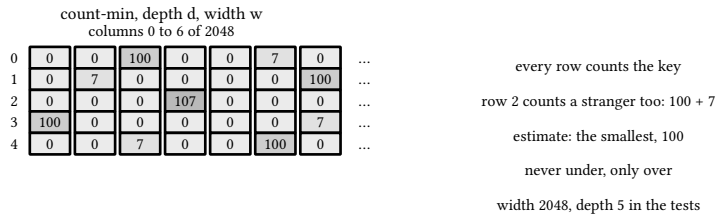


figure 205: the sketch as memory, d rows of w counters, the key lands hot in every row, one row eats a stranger, and the row minimum survives

25.3. lru and lfu caches

Both caches need 2 constant time operations, lookup and eviction, and both get them from a dictionary plus a list the dictionary maintains as a side effect. Lru threads one doubly linked list through the map, newest first, a get unlinks the node and relinks it at the front, and a put into a full cache evicts from the back.

The dry run: the fixtures are the three capacity 2 transcripts asserted by the C# suite, two lru and one lfu, while the ring builders in C, go, java, python, and lua replay the evictions and javascript's ordered map reads its first key.

1. Put a, get a, put b threads the ring b, a front to back, and put c evicts the back: a, with count 2, get b at 2, and get c at 3 pinned after.
2. The second transcript moves the touch later, put a, put b, get a: the get unlinks a and relinks it at the front, the ring reads a, b, and put c now evicts b.
3. The pins read evicted b, get a at 1, get c at 3, one get redirecting the victim.
4. The lfu transcript counts instead: put a, get a, get a leaves a at frequency 3, and put b lands at frequency 1.
5. Put c evicts from the lowest bucket, b goes even though it is the newer key, and the pins read evicted b with a and c answering.

transcript	policy	order before	evicts	pins
a, get a, b, c	lru	b, a	a	count 2, b 2, c 3
a, b, get a, c	lru	a, b	b	a 1, c 3
a, get a, get a, b, c	lfu	freq 3: a, freq 1: b	b	a 1, c 3

The eviction names a, then b, then b across the transcripts, and the listings below thread these rings in seven languages.

```c
static void unlink_(LNode *p) {
  p->prev->next = p->next;
  p->next->prev = p->prev;
}

static void push_front(LNode *p) {
  p->next = head.next;
  p->prev = &head;
  head.next->prev = p;
  head.next = p;
}

static int lru_get(int key, int *out) {
  LNode *p = in_map[key];
  if (!p)
    return 0;
  unlink_(p);
  push_front(p);
  *out = p->val;
  return 1;
}

static void lru_put(int key, int val) {
  LNode *p = in_map[key];
  if (p) {
    p->val = val; // update in place, promote
    unlink_(p);
    push_front(p);
    return;
  }
  if (size_ == CAP) {
    LNode *tail = head.prev;
    evicted[evictions++] = tail->key;
    unlink_(tail);
    in_map[tail->key] = NULL;
    size_--;
  }
  // claim a free slot
  p = &slots[key];
  p->key = key;
  p->val = val;
  in_map[key] = p;
  push_front(p);
  size_++;
}
```

listing 801: c, one ring threaded through a fixed slot array, keys 0 to 7 (source: dsa/samples-c/src/Ch25/lru.c, lines 40-84)

```
// Get returns the value under key and moves the entry to the front.
func (c *LRUCache) Get(key string) (int, bool) {
  n, ok := c.entries[key]
  if !ok {
    return 0, false
  }
  c.unlink(n)
  c.pushFront(n)
  return n.value, true
}

// Put inserts or overwrites key, evicting the least recently used
// entry when the cache is full.
func (c *LRUCache) Put(key string, value int) {
  if n, ok := c.entries[key]; ok {
    n.value = value
    c.unlink(n)
    c.pushFront(n)
    return
  }
  if len(c.entries) ≥ c.capacity {
    victim := c.head.prev
    c.unlink(victim)
    delete(c.entries, victim.key)
  }
  n := &lruNode{key: key, value: value}
  c.entries[key] = n
  c.pushFront(n)
}

func (c *LRUCache) unlink(n *lruNode) {
  n.prev.next = n.next
  n.next.prev = n.prev
}

func (c *LRUCache) pushFront(n *lruNode) {
  n.prev = &c.head
  n.next = c.head.next
  c.head.next.prev = n
  c.head.next = n
```

listing 802: go, a map for lookup, a sentinel ring for order, both splices inline (source: dsa/samples-go/ch25/lru.go, lines 26-66)

```
  static void unlink(LNode p) {
    p.prev.next = p.next;
    p.next.prev = p.prev;
  }

  static void pushFront(LNode p) {
    p.next = head.next;
    p.prev = head;
    head.next.prev = p;
    head.next = p;
  }

  static Integer lruGet(int key) {
    LNode p = inMap.get(key);
    if (p == null)
      return null;
    unlink(p);
    pushFront(p);
    return p.val;
  }

  static void lruPut(int key, int val) {
    LNode p = inMap.get(key);
    if (p != null) {
      p.val = val; // update in place, promote
      unlink(p);
      pushFront(p);
      return;
    }
    if (size == capacity) {
      LNode tail = head.prev;
      evicted.add(tail.key);
      unlink(tail);
      inMap.remove(tail.key);
      size--;
    }
    p = new LNode();
    p.key = key;
    p.val = val;
    inMap.put(key, p);
    pushFront(p);
    size++;
  }
```

listing 803: java, a class node with prev and next, unlink and push front, the sentinel head handing over the victim (source: dsa/samples-java/src/Ch25/Lru.java, lines 41-83)

```
/// <summary>LRU: a dictionary for lookup plus a list that moves the touched key to the front.</summary>
public sealed class LruCache<K, V> where K : notnull
{
    private sealed class Node(K key, V value)
    {
        public K Key = key;
        public V Value = value;
        public Node? Prev, Next;
    }
```

```
    private readonly int _capacity;
    private readonly Dictionary<K, Node> _map = new();
    private readonly Node _front = new(default!, default!); // most recent side
    private readonly Node _back = new(default!, default!); // eviction side

    public LruCache(int capacity)
    {
        if (capacity < 1) throw new ArgumentOutOfRangeException(nameof(capacity));
        _capacity = capacity;
        _front.Prev = _back;
        _back.Next = _front;
    }

    public int Count => _map.Count;
    public K? LastEvicted { get; private set; }

    public V? Get(K key)
    {
        if (!_map.TryGetValue(key, out Node? node))
            return default;
        Unlink(node);
        LinkFront(node);
        return node.Value;
    }

    public void Put(K key, V value)
    {
        if (_map.TryGetValue(key, out Node? node))
        {
            node.Value = value;
            Unlink(node);
            LinkFront(node);
            return;
        }
        if (_map.Count == _capacity)
        {
            Node victim = _back.Next!; // the coldest key sits next to the back sentinel
            Unlink(victim);
            _map.Remove(victim.Key);
            LastEvicted = victim.Key;
        }
        Node fresh = new(key, value);
        LinkFront(fresh);
        _map[key] = fresh;
    }

    private void Unlink(Node node)
    {
        node.Prev!.Next = node.Next;
        node.Next!.Prev = node.Prev;
    }

    private void LinkFront(Node node)
    {
        node.Next = _front;
        node.Prev = _front.Prev;
        _front.Prev!.Next = node;
```

```
        _front.Prev = node;
    }
}
```

listing 804: c#, sentinels at both ends, unlink and relink, the back hands over the victim (source: dsa/samples/src/Ch25/Practical.cs, lines 108-177)

The same c# file carries the lfu twin. Lfu orders by use count instead of touch order, 1 bucket per frequency, each bucket itself recency ordered so ties break the same way lru breaks them, and a minimum frequency pointer that only moves on insert or when a bucket empties:

```
/// <summary>LFU: frequency buckets, LRU order inside a bucket, O(1) on both operations.</summary>
public sealed class LfuCache<K, V> where K : notnull
{
    private sealed class Node(K key, V value)
    {
        public K Key = key;
        public V Value = value;
        public int Freq = 1;
        public Node? Prev, Next;
    }

    private sealed class Lane
    {
        public readonly Node Front = new(default!, default!), Back = new(default!, default!);
        public int Size;
        public Lane() { Front.Prev = Back; Back.Next = Front; }
    }

    private readonly int _capacity;
    private readonly Dictionary<K, Node> _map = new();
    private readonly Dictionary<int, Lane> _lanes = new();
    private int _minFreq;

    public LfuCache(int capacity)
    {
        if (capacity < 1) throw new ArgumentOutOfRangeException(nameof(capacity));
        _capacity = capacity;
    }

    public int Count => _map.Count;
    public K? LastEvicted { get; private set; }

    public V? Get(K key)
    {
        if (!_map.TryGetValue(key, out Node? node))
            return default;
        Bump(node);
        return node.Value;
    }

    public void Put(K key, V value)
    {
        if (_map.TryGetValue(key, out Node? node))
        {
            node.Value = value;
            Bump(node);
```

```
            return;
        }
        if (_map.Count == _capacity)
        {
            Lane coldest = _lanes[_minFreq];
            Node victim = coldest.Back.Next!; // lru order inside the coldest bucket
            Unlink(victim, coldest);
            if (coldest.Size == 0)
                _lanes.Remove(_minFreq);
            _map.Remove(victim.Key);
            LastEvicted = victim.Key;
        }
        Node fresh = new(key, value);
        PushFront(fresh, LaneFor(1));
        _map[key] = fresh;
        _minFreq = 1; // a fresh key is always the new minimum
    }

    private void Bump(Node node)
    {
        Lane old = LaneFor(node.Freq);
        Unlink(node, old);
        if (old.Size == 0)
        {
            _lanes.Remove(node.Freq);
            if (_minFreq == node.Freq)
                _minFreq = node.Freq + 1; // nothing colder survives this lane
        }
        node.Freq++;
        PushFront(node, LaneFor(node.Freq));
    }
```

listing 805: c#, lanes per bucket, a bump per get, the lane helpers below repeat the lru pair (source: dsa/samples/src/Ch25/Practical.cs, lines 179-255)

```
export class LRU {
  #map = new Map();
  #cap;
  evictions = [];

  constructor(cap) {
    this.#cap = cap;
  }

  get size() {
    return this.#map.size;
  }

  get(key) {
    if (!this.#map.has(key)) return undefined;
    const v = this.#map.get(key);
    this.#map.delete(key); // refresh: move to the back
    this.#map.set(key, v);
    return v;
  }

  put(key, value) {
    if (this.#map.has(key)) this.#map.delete(key);
    else if (this.#map.size === this.#cap) {
      const oldest = this.#map.keys().next().value;
      this.#map.delete(oldest);
      this.evictions.push(oldest);
    }
    this.#map.set(key, value);
    return this;
  }
}
```

listing 806: javascript, the insertion-ordered map is the whole cache, refresh by delete and re-set (source: dsa/samples-js/src/ch25-lru.mjs, lines 5-36)

```
class LRUCache:
    def __init__(self, capacity):
        self.capacity = capacity
        self.map = {}
        self.sentinel = RingNode()  # sentinel.next = most recent
        self.evictions = []

    def _unlink(self, node):
        node.prev.next = node.next
        node.next.prev = node.prev

    def _push_front(self, node):
        node.next = self.sentinel.next
        node.prev = self.sentinel
        self.sentinel.next.prev = node
        self.sentinel.next = node

    def get(self, key):
        if key not in self.map:
            return None
        node = self.map[key]
        self._unlink(node)  # a touch refreshes recency
        self._push_front(node)
        return node.value

    def put(self, key, value):
        if key in self.map:
            self._unlink(self.map[key])
        node = RingNode(key, value)
        self.map[key] = node
        self._push_front(node)
        if len(self.map) > self.capacity:
            lru = self.sentinel.prev  # oldest sits left of the sentinel
            self._unlink(lru)
            del self.map[lru.key]
            self.evictions.append(lru.key)
```

listing 807: python, dict plus ring, evictions recorded as they happen (source: dsa/samples-py/src/Ch25/lru.py, lines 24-59)

```lua
-- unlink a node from the ring
local function unlink(node)
  node.prev.next = node.next
  node.next.prev = node.prev
end

-- splice a node right after the sentinel (the front)
local function push_front(l, node)
  local s = l.sentinel
  node.next = s.next
  node.prev = s
  s.next.prev = node
  s.next = node
end

function LRU.put(l, key, value)
  local node = l.map[key]
  if node then
    node.value = value
    unlink(node)
    push_front(l, node)
    return
  end
  if l.map_count ≥ l.cap then
    local victim = l.sentinel.prev -- the least recently used
    unlink(victim)
    l.map[victim.key] = nil
    l.last_evicted = victim.key
    l.map_count = l.map_count - 1
  end
  local fresh = {key = key, value = value}
  l.map[key] = fresh
  l.map_count = l.map_count + 1
  push_front(l, fresh)
end

function LRU.get(l, key)
  local node = l.map[key]
  if not node then return nil end
  unlink(node)
  push_front(l, node)
  return node.value
end
```

listing 808: lua, unlink and push front over a table-keyed ring (source: dsa/samples-lua/ch25_lru.lua, lines 14-56)

The eviction tests read like transcripts. Capacity 2 with put a, get a, put b, put c evicts a, because the get left a newer than b. Capacity 2 with put a, put b, get a, put c evicts b, the get moved a off the cold end. And put a, get a, get a, put b, put c evicts b under lfu, a sits at frequency 3 and the frequency 1 bucket loses its only key. The bcl answer is `MemoryCache`, eviction keyed to memory pressure instead of a strict count, so treat these as the machinery a policy sits on top of. Redis ships both policies for real, allkeys-lru approximated by sampling a few keys and allkeys-lfu over a decayed counter.

Six of the seven builds share the same shape, a hash map for the lookup and a doubly linked ring for the order. Javascript is the exception that proves the map can carry both jobs: its builtin Map keeps insertion order, so a get refreshes by delete and re-set and the first key iterator hands over the victim, no ring at all. The transcripts agree across the ring-builders, touching a saves it and c evicts b at capacity 2 in c#, java, python, and lua. C threads

its ring through a fixed slot array over keys 0 to 7 and pins the longer eviction sequence 2 then 1, java pinning that same sequence over heap nodes, python and java add the capacity 1 churn case and the update that evicts nothing, and go exposes an order dump that tests read front to back.

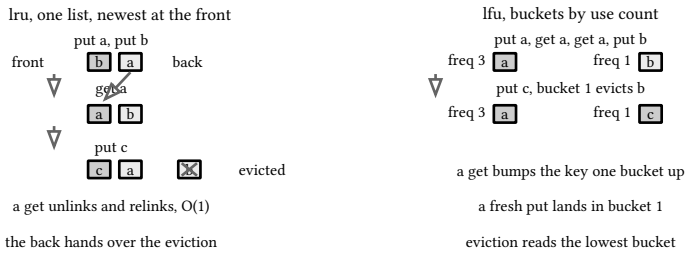


figure 206: lru reorders one list on every touch and evicts from the back, lfu moves keys up a bucket per get and evicts the lowest bucket

25.4. the skip list

Chapter 7′s avl tree keeps a sorted set balanced with rotations, a deterministic guarantee. The skip list buys the same expected log n search with coin flips. Every insert builds a tower, the generator advances once per promotion at p = 1/2, so half the keys reach level 2, a quarter reach level 3, and lane widths halve as they rise. A search starts on the tallest lane and moves right while the next key is smaller, dropping a lane on every overshoot, and it never backtracks.

The dry run: the fixture is the seeded census, a thousand shuffled keys, 4,096 towers, and 1,024 searches, asserted by the C# suite, while the deterministic siblings state their promotion policies, go, java, and python pinning the ruler ladder 1, 2, 1, 3, 1, 2, 1, 4.

1. A thousand keys arrive shuffled and lane 0 enumerates them 0 through 999 sorted, one walk down the bottom lane.
2. Contains answers true for all 1,000 members and false for the 100 probes at 1,000 and up.
3. Of 4,096 towers 2,041 reach level 2, 2,041 / 4,096 = 0.498, inside the asserted band 0.45 to 0.55 around the expected half.
4. The remaining 4,096 - 2,041 = 2,055 stop at level 1, the lane widths halving as they rise.
5. Searching all 1,024 members costs a mean of 16.0 comparisons, under the asserted bound 2 log2 1,024 = 20.
6. The ruler ladder pins the deterministic side: the first eight inserts rise to 1, 2, 1, 3, 1, 2, 1, 4.

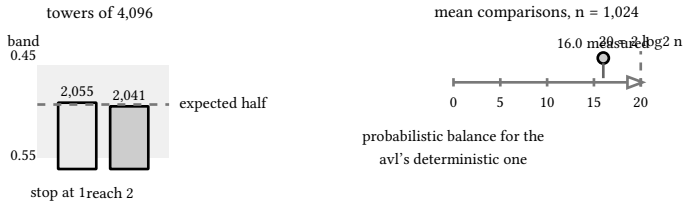


figure 207: the two pinned statistics, the tower census against the expected half and the search mean under its bound

The 0.498 against the half and the 16.0 under 20 are the pinned statistics, and the listings below flip these coins in seven languages.

```c
static int height_for(int insert_index) {
  // insert 1 -> 1, 2 -> 2, 3 -> 1, 4 -> 3, 5 -> 1, 6 -> 2, 7 -> 1, 8 -> 4
  int h = 1;
  while ((insert_index & 1) == 0 && h < MAXL) {
    insert_index >>= 1;
    h++;
  }
  return h;
}

static void sl_init(void) {
  node_n = 0;
  level = 1;
  for (int l = 0; l < MAXL; l++)
    head.next[l] = NULL;
}

static SNode *sl_find(int val) {
  SNode *cur = &head;
  for (int l = level - 1; l ≥ 0; l--)
    while (cur->next[l] && cur->next[l]->val < val)
      cur = cur->next[l];
  cur = cur->next[0];
  return cur && cur->val == val ? cur : NULL;
}

static void sl_insert(int val, int insert_index) {
  SNode *update[MAXL];
  SNode *cur = &head;
  int h = height_for(insert_index);
  int top = h > level ? h : level;
  for (int l = top - 1; l ≥ 0; l--) {
    if (l ≥ level) {
      update[l] = &head; // lanes above the current frontier are empty
      continue;
    }
    while (cur->next[l] && cur->next[l]->val < val)
      cur = cur->next[l];
    update[l] = cur;
  }
  SNode *fresh = &nodes[node_n++];
  fresh->val = val;
  fresh->height = h;
  if (h > level)
    level = h;
  for (int l = 0; l < MAXL; l++)
    fresh->next[l] = NULL;
  for (int l = 0; l < h; l++) {
    fresh->next[l] = update[l]->next[l];
    update[l]->next[l] = fresh;
  }
}
```

listing 809: c, the ruler promotion, splice points collected on the way down (source: dsa/samples-c/src/Ch25/skiplist.c, lines 32-83)

```go
// levelFor is the deterministic promotion: node number i gets 1 plus
// the number of trailing zero bits of i.
func levelFor(i int) int {
  return 1 + bits.TrailingZeros(uint(i))
}

// Insert adds value if absent, promoting per the deterministic policy.
func (s *SkipList) Insert(value int) bool {
  if s.Find(value) {
    return false
  }
  s.size++
  level := levelFor(s.size) // the i-th insertion rises by its bit luck
  for len(s.head.nexts) < level {
    s.head.nexts = append(s.head.nexts, nil)
  }
  fresh := &skipNode{value: value, nexts: make([]*skipNode, level)}
  // walk down from the top, splicing at every level
  updates := s.predecessors(value)
  for l := 0; l < level; l++ {
    fresh.nexts[l] = updates[l].nexts[l]
    updates[l].nexts[l] = fresh
  }
  return true
}
```

listing 810: go, promotion by trailing zeros of the insertion index, math slash bits (source: dsa/samples-go/ch25/skiplist.go, lines 25-49)

```
  static int heightFor(int insertIndex) {
    // insert 1 -> 1, 2 -> 2, 3 -> 1, 4 -> 3, 5 -> 1, 6 -> 2, 7 -> 1, 8 -> 4
    int h = 1;
    while ((insertIndex & 1) == 0 && h < MAXL) {
      insertIndex >>= 1;
      h++;
    }
    return h;
  }

  static void slInit() {
    head = new SNode();
    level = 1;
    heightOf.clear();
  }

  static SNode slFind(int val) {
    SNode cur = head;
    for (int l = level - 1; l ≥ 0; l--)
      while (cur.next[l] ≠ null && cur.next[l].val < val)
        cur = cur.next[l];
    cur = cur.next[0];
    return cur ≠ null && cur.val == val ? cur : null;
  }

  static void slInsert(int val, int insertIndex) {
    SNode[] update = new SNode[MAXL];
    SNode cur = head;
    int h = heightFor(insertIndex);
    int top = Math.max(h, level);
    for (int l = top - 1; l ≥ 0; l--) {
      if (l ≥ level) {
        update[l] = head; // lanes above the current frontier are empty
        continue;
      }
      while (cur.next[l] ≠ null && cur.next[l].val < val)
        cur = cur.next[l];
      update[l] = cur;
    }
    SNode fresh = new SNode();
    fresh.val = val;
    fresh.height = h;
    heightOf.put(val, h);
    if (h > level)
      level = h;
    for (int l = 0; l < h; l++) {
      fresh.next[l] = update[l].next[l];
      update[l].next[l] = fresh;
    }
  }
```

listing 811: java, the ruler policy, find dropping a lane on overshoot, insert collecting its splice points on the descent (source: dsa/samples-java/src/Ch25/Skiplist.java, lines 34-83)

```
/// <summary>Skip list: towers from a seeded xorshift, the sorted map an lsm engine writes from.</summary>
public sealed class SkipList<T> : IEnumerable<T> where T : IComparable<T>
{
```

```
    public sealed class Meter
    {
        public long Ops { get; set; }
    }

    private sealed class Node(T value, int levels)
    {
        public T Value = value;
        public Node?[] Next = new Node?[levels];
    }

    private const int MaxLevel = 16;

    private readonly Node _head = new(default!, MaxLevel);
    private readonly int _seed;
    private int _level = 1;
    private uint _rng;

    public SkipList(int seed = 0x5eed)
    {
        _seed = seed;
        _rng = (uint)seed;
    }

    public int Count { get; private set; }
    public int MaxLevelSeen => _level;

    public bool Add(T value)
    {
        var update = new Node?[MaxLevel];
        Node node = _head;
        for (int i = _level - 1; i ≥ 0; i--)
        {
            while (node.Next[i] is { } next && value.CompareTo(next.Value) > 0)
                node = next;
            update[i] = node;
        }
        if (node.Next[0] is { } candidate && value.CompareTo(candidate.Value) == 0)
            return false; // a sorted set keeps one copy

        int levels = Tower();
        if (levels > _level)
        {
            for (int i = _level; i < levels; i++)
                update[i] = _head;
            _level = levels;
        }
        var fresh = new Node(value, levels);
        for (int i = 0; i < levels; i++)
        {
            fresh.Next[i] = update[i]!.Next[i];
            update[i]!.Next[i] = fresh;
        }
        Count++;
        return true;
    }
```

```
    public bool Contains(T value, Meter? meter = null)
    {
        Node node = _head;
        for (int i = _level - 1; i ≥ 0; i--)
        {
            while (node.Next[i] is { } next)
            {
                if (meter ≠ null)
                    meter.Ops++;
                int c = value.CompareTo(next.Value);
                if (c == 0)
                    return true;
                if (c < 0)
                    break; // this lane overshot, drop down
                node = next;
            }
        }
        return false;
    }
```

listing 812: c#, the descent collects splice points, contains drops a lane on overshoot (source: dsa/samples/src/Ch25/Practical.cs, lines 284-362)

```
    /// <summary>Nodes whose tower reaches the given 1-based level, the lane width.</summary>
    public int CountAtLevel(int level)
    {
        int count = 0;
        for (Node? node = _head.Next[level - 1]; node ≠ null; node = node.Next[level - 1])
            count++;
        return count;
    }

    public IEnumerator<T> GetEnumerator()
    {
        for (Node? node = _head.Next[0]; node ≠ null; node = node.Next[0])
            yield return node.Value;
    }

    System.Collections.IEnumerator System.Collections.IEnumerable.GetEnumerator() ⇒ GetEnumerator();

    // xorshift32: deterministic tower heights, reproducible tests.
    // one fresh flip per promotion, a single flip reused for the whole
    // tower would make every node height 1 or max
    private int Tower()
    {
        int level = 1;
        while (level < MaxLevel)
        {
            _rng ^= _rng << 13;
            _rng ^= _rng >> 17;
            _rng ^= _rng << 5;
            if (_rng == 0)
                _rng = (uint)_seed | 1;
            if ((_rng & 0xFFFF) ≥ 0x8000) // p = 1/2 per promotion
                break;
            level++;
        }
        return level;
    }
```

listing 813: c#, lane widths by count, sorted enumeration, one fresh flip per promotion (source: dsa/samples/src/Ch25/Practical.cs, lines 364-399)

```
export function layersFor(key) {
  const h = fnv1a64(String(key));
  let zeros = 0;
  let bits = h;
  while ((bits & 1n) === 0n && zeros < MAX_LAYERS - 1) {
    zeros++;
    bits >>= 1n;
  }
  return zeros + 1;
}

export class SkipList {
  #head = { key: null, next: [] };
  #count = 0;

  get size() {
    return this.#count;
  }

  get maxLayers() {
    return this.#head.next.length;
  }

  insert(key) {
    if (this.find(key)) return this; // a set: duplicates collapse
    const layers = layersFor(key);
    const tower = { key, next: new Array(layers).fill(null) };
    // walk down from the top, splicing per layer
    let node = this.#head;
    for (let l = Math.max(this.#head.next.length, layers) - 1; l ≥ 0; l--) {
      while (node.next[l] && node.next[l].key < key) node = node.next[l];
      if (l < layers) {
        tower.next[l] = node.next[l] ?? null;
        node.next[l] = tower;
      }
    }
    // grow the head upward only as towers demand it
    while (this.#head.next.length < layers) this.#head.next.push(tower);
    this.#count++;
    return this;
  }
```

listing 814: javascript, the tower is 1 plus trailing zeros of the key's fnv hash (source: dsa/samples-js/src/ch25-skiplist.mjs, lines 14-54)

```python
def promo_level(insert_index):
    # insert_index is 1-based: 1, 2, 3, 4, ... -> 1, 2, 1, 3, 1, 2, 1, 4
    level = 1
    i = insert_index
    while i % 2 == 0 and level < MAX_LEVEL:
        i //= 2
        level += 1
    return level


class SkipNode:
    __slots__ = ("value", "forward")

    def __init__(self, value, level):
        self.value = value
        self.forward = [None] * level


class SkipList:
    def __init__(self):
        self.head = SkipNode(None, MAX_LEVEL)
        self.height = 1

    def insert(self, value, index):
        level = promo_level(index)
        update = [self.head] * MAX_LEVEL
        for lvl in range(self.height - 1, -1, -1):  # descend from the top
            while (nxt := update[lvl].forward[lvl]) and nxt.value < value:
                update[lvl] = nxt
        node = SkipNode(value, level)
        for lvl in range(level):
            node.forward[lvl] = update[lvl].forward[lvl]
            update[lvl].forward[lvl] = node
        self.height = max(self.height, level)
```

listing 815: python, the ruler sequence drives promotion, splice per level (source: dsa/samples-py/src/Ch25/skiplist.py, lines 17-50)

```lua
-- deterministic promotion: level 1 always, one more per test that v
-- passes (divisible by 4, then by 8, then by 16), capped at MAXL
local function level_of(v)
  local lvl = 1
  while lvl < MAXL and v % (1 << (lvl + 1)) == 0 do
    lvl = lvl + 1
  end
  return lvl
end

local function sl_new()
  local head = {val = -math.maxinteger, next = {}, lvl = MAXL}
  for i = 1, MAXL do head.next[i] = nil end
  return {head = head, count = 0}
end

local function sl_insert(sl, v)
  local update = {}
  local at = sl.head
  for lvl = MAXL, 1, -1 do
    while at.next[lvl] and at.next[lvl].val < v do
      at = at.next[lvl]
    end
    update[lvl] = at
  end
  if at.next[1] and at.next[1].val == v then return false end
  local node = {val = v, next = {}, lvl = level_of(v)}
  for lvl = 1, node.lvl do
    node.next[lvl] = update[lvl].next[lvl]
    update[lvl].next[lvl] = node
  end
  sl.count = sl.count + 1
  return true
end
```

listing 816: lua, promotion as a pure function of the value, divisibility by 4, 8, 16 (source: dsa/samples-lua/ch25_skiplist.lua, lines 9-42)

The tests pin the statistics. One thousand inserts in shuffled order enumerate sorted and answer Contains for every member while missing 100 absent probes. Of 4,096 towers, 2,041 reach level 2, 0.498 against the expected one half. And searching all 1,024 members of a seeded list takes a mean of 16.0 comparisons, under the 2 log2 n bound of 20, the probabilistic balance the avl gets deterministically. LevelDB's memtable and redis sorted sets are this list in production.

The other six builds are deterministic, and each states its promotion policy out loud. Go, java, python, and c use the ruler sequence, the i-th insert rises to 1 plus the trailing zeros of i, and go, java, and python all pin the height ladder 1, 2, 1, 3, 1, 2, 1, 4 while python and java read tower heights per value off a map, java also dumping whole lanes, lane 1 skipping the odds and lane 3 holding only the tallest key. Javascript hashes the key itself and counts trailing zero bits of the 64-bit word, so towers depend on the data rather than the insertion order, and lua makes promotion a pure function of divisibility, level 2 exactly when the value is divisible by 4, capped at 16. Whatever the policy, the search spine is identical in all seven: descend from the top lane, walk right while smaller, drop a lane on every overshoot, never backtrack.

PITFALL: one coin per promotion, not one per tower

A tower generator that draws a single random value and tests it in a loop produces heights of exactly 1 or the maximum, half and half. Sortedness still holds and small tests still pass, but the top lane holds every second

key and search degrades toward scanning half the list. The generator must advance between promotions, the measured 16.0 above depends on it.

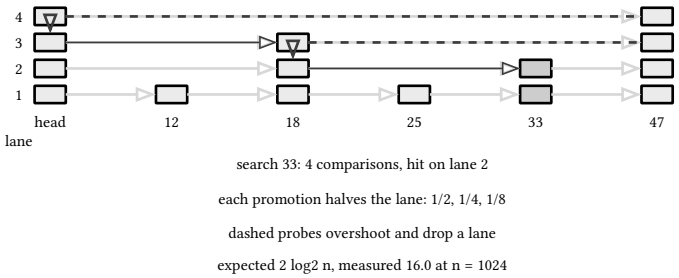


figure 208: towers from coin flips over 5 keys, the shaded search rides the tall lanes and drops where a lane overshoots, never backtracking

25.5. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's four sample files, the c# file also carrying the lfu cache section above, which has no matrix twin in the other six:

language	build sloc	container dependency	boundary note
c	384	byte bit arrays, slot-backed ring	djb2 second hash with the correlation comment, bounded lie counts
c#	341	`Dictionary<TKey,TValue>`, `List<T>`	the only coin-flip skiplist, kirsch-mitzenmacher k of 7, lfu included
go	204	`map[string]*lruNode`, slices	double-hash bloom slots, promotion by insertion index
java	518	jdk 27 stdlib	fnv offset parsed unsigned, every hash mod through `Long.remainderUnsigned`, the c fixture's empty 40th probe mirrored, both count-min fixtures in one file
javascript	125	`Map` in insertion order, `Set`	bigint hash words, the map doubles as the lru ring
python	229	`dict`, list of bools	8-bit bloom with lies pinned by name, ruler promotion
lua	338	`table` with string splice-free keys	fnv offset built from a hex literal, divisibility promotion

sources: learn.microsoft.com, `Dictionary<TKey,TValue>` under both caches, `SortedSet<T>` filling the ordered set role in the bcl, go.dev for `math/bits.TrailingZeros`, developer.mozilla.org for the insertion-ordered `Map` and bigint operators, docs.python.org for `__slots__`, lua.org for `math.maxinteger` and floor modulo, accessed 2026-09-12 and 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 25: c 4 files and 89 checks, c# 11 tests, go 12 tests, java 4 files and 98 checks, javascript 10 tests, python 4 files and 36 asserts, lua 16 checks, zero skipped.

26. parsing and asts

Text becomes numbers through three stages: a lexer chops characters into tokens, a recursive descent parser folds tokens into a tree, and an evaluator walks the tree. Chapter 5 already ran infix to postfix with shunting-yard for a stack machine, and this chapter builds the same arithmetic the way compilers do it, with a grammar that bakes precedence into its own shape. The full-language treatment, keywords, calls, closures, lives in book 5, ch 35 (c# 15, f# 11, and .net 11, capstone: a small language interpreter), and chapter 27 runs the compiled version of these ideas as an opcode machine.

26.1. the lexer

One pass, one cursor. Digit runs accumulate into number tokens, letter runs into identifiers, the four operators and the parentheses pass through as themselves, whitespace separates and vanishes, and anything else is an error reported with its position rather than a silent drop. Every language appends an end marker or relies on running out of tokens, and the token stream for one fixed fixture pins the whole behavior.

The dry run: the fixture is the C# string 12 + foo * (bar - 3), asserted token by token by the C# suite, lua pinning the same string, c and java running x1 + 42 shapes of their own in the same nine-token form.

1. The digit run consumes both characters as one token and carries its parsed value beside its text, 12 with value 12.
2. The letter run folds the same way, f, o, o accumulating into the single identifier foo, the whitespace between tokens vanishing.
3. The one-character tokens pass through as themselves: +, *, (, the bar run, -, the 3, and).
4. The end marker rides after the last token and the stream closes at ten tokens, nine from the source plus the end.
5. Strangers never join: 2 % 3 throws at the % and a#b at the #, each error carrying the character and its index.

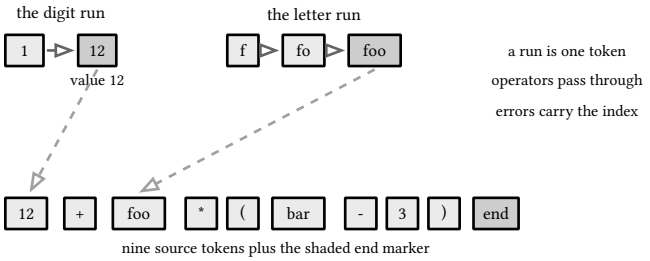


figure 209: the run as a sequence, the two runs growing one character at a time above, the ten tokens closing on the shaded end marker below

The ten-token stream is the pinned assert, and the listings below lex it in seven languages.

```
static void lex(const char *s) {
  tn = 0;
  for (const char *p = s; *p; p++) {
    if (*p == ' ')
      continue;
    Tok t = {0};
    t.text[0] = *p;
    t.text[1] = 0;
    if (*p ≥ '0' && *p ≤ '9') {
      long long v = 0;
      int k = 0;
      while (*p ≥ '0' && *p ≤ '9') {
        v = v * 10 + (*p - '0');
        if (k < 7)
          t.text[k++] = *p;
        p++;
      }
      t.text[k] = 0;
      p--;
      t.kind = NUM;
      t.num = v;
    } else if ((*p ≥ 'a' && *p ≤ 'z') || *p == '_') {
      int k = 0;
      while ((*p ≥ 'a' && *p ≤ 'z') || *p == '_' ||
             (*p ≥ '0' && *p ≤ '9')) {
        if (k < 7)
          t.text[k++] = *p;
        p++;
      }
      t.text[k] = 0;
      p--;
      t.kind = IDENT;
    } else if (*p == '+') {
      t.kind = PLUS;
    } else if (*p == '-') {
      t.kind = MINUS;
    } else if (*p == '*') {
      t.kind = STAR;
    } else if (*p == '/') {
      t.kind = SLASH;
    } else if (*p == '(') {
      t.kind = LPAREN;
    } else if (*p == ')') {
      t.kind = RPAREN;
    } else {
      t.kind = ERR;
    }
    toks[tn++] = t;
  }
  toks[tn].kind = END; // terminator, not counted in tn
}
```

listing 817: c, the cursor loop, digit and letter runs, an err token for strangers (source: dsa/samples-c/src/Ch26/tokens.c, lines 29-79)

```go
// Lex splits source into tokens: digit runs become numbers, letter
// runs become identifiers, + - * / ( ) pass through as themselves,
// whitespace separates and vanishes. Anything else is an error.
func Lex(src string) ([]Token, error) {
  var tokens []Token
  var run strings.Builder
  flush := func() {
    if run.Len() > 0 {
      tokens = append(tokens, Token{Kind: KindNumber, Text: run.String()})
      run.Reset()
    }
  }
  for _, r := range src {
    switch {
    case r ≥ '0' && r <= '9':
      run.WriteRune(r)
    case r == ' ' || r == '\t':
      flush()
    case r == '+' || r == '-' || r == '*' || r == '/':
      flush()
      tokens = append(tokens, Token{Kind: KindOp, Text: string(r)})
    case r == '(':
      flush()
      tokens = append(tokens, Token{Kind: KindLParen, Text: "("})
    case r == ')':
      flush()
      tokens = append(tokens, Token{Kind: KindRParen, Text: ")"})
    case r ≥ 'a' && r <= 'z' || r ≥ 'A' && r <= 'Z':
      flush()
      tokens = append(tokens, Token{Kind: KindIdent, Text: string(r)})
    default:
      return nil, fmt.Errorf("unexpected character %q", r)
    }
  }
  flush()
  return tokens, nil
}
```

listing 818: go, digits buffer into a run, one letter per identifier, errors as values (source: dsa/samples-go/ch26/tokens.go, lines 32-68)

```
  static void lex(String s) {
    tn = 0;
    int i = 0;
    while (i < s.length()) {
      char c = s.charAt(i);
      i++;
      if (c == ' ')
        continue;
      Tok t = toks[tn++];
      if (c ≥ '0' && c ≤ '9') {
        long v = c - '0';
        int st = i - 1;
        while (i < s.length() && s.charAt(i) ≥ '0' && s.charAt(i) ≤ '9') {
          v = v * 10 + (s.charAt(i) - '0');
          i++;
        }
        t.kind = NUM;
        t.num = v;
        t.text = s.substring(st, i);
      } else if (c ≥ 'a' && c ≤ 'z' || c == '_') {
        int st = i - 1;
        while (i < s.length() && (Character.toLowerCase(s.charAt(i)) ≥ 'a'
            && Character.toLowerCase(s.charAt(i)) ≤ 'z'
            || s.charAt(i) == '_' || Character.isDigit(s.charAt(i))))
          i++;
        t.kind = IDENT;
        t.text = s.substring(st, i);
      } else if (c == '+') {
        t.kind = PLUS;
        t.text = "+";
      } else if (c == '-') {
        t.kind = MINUS;
        t.text = "-";
      } else if (c == '*') {
        t.kind = STAR;
        t.text = "*";
      } else if (c == '/') {
        t.kind = SLASH;
        t.text = "/";
      } else if (c == '(') {
        t.kind = LPAREN;
        t.text = "(";
      } else if (c == ')') {
        t.kind = RPAREN;
        t.text = ")";
      } else {
        t.kind = ERR;
        t.text = String.valueOf(c);
      }
    }
    toks[tn].kind = END; // terminator, not counted in tn
    toks[tn].text = "";
    toks[tn].num = 0;
  }
```

listing 819: java, the cursor loop writing into a pre-allocated token array, digit and letter runs, strangers become err tokens (source: dsa/samples-java/src/Ch26/Tokens.java, lines 22-75)

```
public static class Lexer
{
    public static List<Token> Tokenize(string source)
    {
        var tokens = new List<Token>();
        int i = 0;
        while (i < source.Length)
        {
            char c = source[i];
            if (char.IsWhiteSpace(c))
            {
                i++;
                continue;
            }
            if (char.IsDigit(c))
            {
                int start = i;
                while (i < source.Length && char.IsDigit(source[i]))
                    i++;
                string text = source[start..i];
                tokens.Add(new Token(TokenKind.Number, text, long.Parse(text)));
                continue;
            }
            if (char.IsLetter(c) || c == '_')
            {
                int start = i;
                while (i < source.Length && (char.IsLetterOrDigit(source[i]) || source[i] == '_'))
                    i++;
                tokens.Add(new Token(TokenKind.Ident, source[start..i]));
                continue;
            }
            TokenKind kind = c switch
            {
                '+' => TokenKind.Plus,
                '-' => TokenKind.Minus,
                '*' => TokenKind.Star,
                '/' => TokenKind.Slash,
                '(' => TokenKind.LParen,
                ')' => TokenKind.RParen,
                _ => throw new InvalidOperationException($"unexpected character '{c}' at {i}"),
            };
            tokens.Add(new Token(kind, c.ToString()));
            i++;
        }
        tokens.Add(new Token(TokenKind.End, ""));
        return tokens;
    }
```

listing 820: c#, whitespace skip, digit and letter runs, a switch over the operators (source: dsa/samples/src/Ch26/Parsing.cs, lines 19-65)

```
export function tokenize(src) {
  const toks = [];
  let i = 0;
  const isDigit = (c) => c >= "0" && c <= "9";
  const isIdent = (c) => /[a-zA-Z_]/.test(c);
  while (i < src.length) {
    const c = src[i];
    if (c === " ") {
      i++;
      continue;
    }
    if (isDigit(c)) {
      let v = 0;
      const start = i;
      while (i < src.length && isDigit(src[i])) v = v * 10 + +src[i++];
      toks.push({ kind: "num", value: v, pos: start });
      continue;
    }
    if (isIdent(c)) {
      let name = "";
      const start = i;
      while (i < src.length && /[a-zA-Z0-9_]/.test(src[i])) name += src[i++];
      toks.push({ kind: "ident", name, pos: start });
      continue;
    }
    if ("+-*/()".includes(c)) {
      toks.push({ kind: c, pos: i });
      i++;
      continue;
    }
    throw new SyntaxError(`unexpected character '${c}' at ${i}`);
  }
  return toks;
}
```

listing 821: javascript, every token carries its source position (source: dsa/samples-js/src/ch26-tokens.mjs, lines 5-38)

```
def tokenize(source):
    tokens = []
    i = 0
    while i < len(source):
        ch = source[i]
        if ch.isspace():
            i += 1  # skipped entirely
        elif ch.isdigit():
            j = i
            while j < len(source) and source[j].isdigit():
                j += 1
            tokens.append(("num", int(source[i:j])))
            i = j
        elif ch.isalpha() or ch == "_":
            j = i
            while j < len(source) and (source[j].isalnum() or source[j] == "_"):
                j += 1
            tokens.append(("name", source[i:j]))
            i = j
        elif ch in OPERATORS:
            tokens.append(("op", ch))
            i += 1
        elif ch in PARENS:
            tokens.append(("paren", ch))
            i += 1
        else:
            raise SyntaxError(f"unexpected character {ch!r} at {i}")
    return tokens
```

listing 822: python, str methods classify, syntaxerror carries the position (source: dsa/samples-py/src/Ch26/tokens.py, lines 17-44)

```lua
local function tokenize(src)
  local toks = {}
  local i = 1
  while i ≤ #src do
    local c = src:sub(i, i)
    if c == " " or c == "\t" or c == "\r" or c == "\n" then
      i = i + 1
    elseif c:match("%d") then
      local j = i
      while j ≤ #src and src:sub(j, j):match("%d") do j = j + 1 end
      toks[#toks + 1] = {kind = "num", text = src:sub(i, j - 1)}
      i = j
    elseif c:match("%a") then
      local j = i
      while j ≤ #src and src:sub(j, j):match("%w") do j = j + 1 end
      toks[#toks + 1] = {kind = "ident", text = src:sub(i, j - 1)}
      i = j
    elseif c == "+" then
      toks[#toks + 1] = {kind = "plus", text = "+"}; i = i + 1
    elseif c == "-" then
      toks[#toks + 1] = {kind = "minus", text = "-"}; i = i + 1
    elseif c == "*" then
      toks[#toks + 1] = {kind = "star", text = "*"}; i = i + 1
    elseif c == "/" then
      toks[#toks + 1] = {kind = "slash", text = "/"}; i = i + 1
    elseif c == "(" then
      toks[#toks + 1] = {kind = "lparen", text = "("}; i = i + 1
    elseif c == ")" then
      toks[#toks + 1] = {kind = "rparen", text = ")"}; i = i + 1
    else
      error("unexpected character '" .. c .. "'", 2)
    end
  end
  toks[#toks + 1] = {kind = "end", text = ""}
  return toks
end
```

listing 823: lua, sub and match over bytes, an end token appended last (source: dsa/samples-lua/ch26_tokens.lua, lines 6-41)

The stream shape agrees everywhere: identifier, plus, number, star, left paren, identifier, minus, number, right paren, on fixtures like c's x1 + 42 times (foo - 3) and lua's 12 + foo times (bar - 3), and multi-digit numbers lex as one token in all seven. The identifier rule splits by language, and the go listing is the outlier on purpose: it emits one letter per identifier, so ab reads as two names, a choice its tests document. The other six accumulate letters, digits, and underscores, python and java pinning a underscore 9 b as a single name. Error handling splits the same way, c and java keep lexing and return an err token, go returns an error value, javascript, python, and lua raise, and c# throws with the offending character and index in the message.

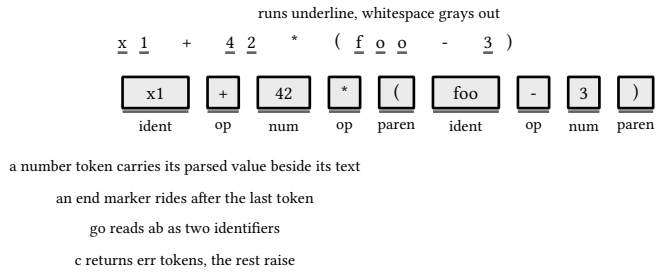


figure 210: the c fixture sliced into tokens, runs underlined, whitespace dropped, kinds labeled

26.2. recursive descent

The grammar is the code. Expr unfolds a term then loops over plus or minus, term unfolds a factor then loops over multiply or divide, and factor is a number, a name, or a parenthesized expr. Precedence comes from the nesting depth, multiply and divide live one level down, left associativity comes from the loop folding each new operator onto everything parsed before it, and parentheses restart the whole climb at the top.

The dry run: the fixture is 2 + 3 * 4 with its parenthesized twin, both trees asserted node for node by the C# suite, the same shapes pinning in the other six.

1. Expr calls Term calls Factor, and the first factor takes the 2, the cursor resting on the +.
2. Term sees no * and returns, Expr's loop matches the + and calls Term again for the right side.
3. The inner Term takes the 3, its own loop matches the * and folds Factor 4 into a multiply node, and the outer fold closes Bin(+, 2, Bin(*, 3, 4)), the asserted tree.
4. The paren twin restarts the climb inside the parens, 2 + 3 folds first, and the tree lifts the plus to the root.
5. The chain 8 - 3 - 2 folds left one loop pass at a time, 8 - 3 = 5 then 5 - 2 = 3, never the 7 of 8 - (3 - 2).
6. The broken inputs die at Expect: 2 + starves the factor, (2 waits for the rparen, and 2 3 leaves input behind at the end check.

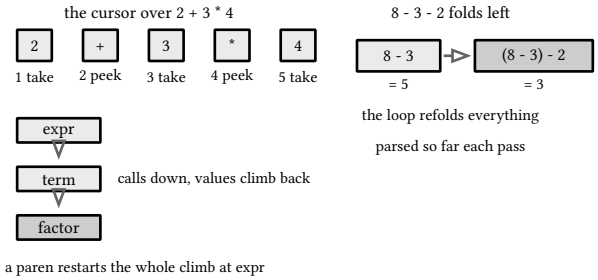


figure 211: the parse as a sequence, the cursor events under the token rail on the left, the left fold of 8 - 3 - 2 building one pass at a time on the right

The pinned trees land the chain at 3, and the listings below parse them in seven languages.

```c
static ANode *term(void) {
  ANode *l = factor();
  while (parse_ok && toks[pos].kind == OP &&
         (toks[pos].op == '*' || toks[pos].op == '/')) {
    char op = toks[pos++].op;
    ANode *p = leaf(op == '*' ? 4 : 5);
    p->l = l;
    p->r = factor();
    l = p;
  }
  return l;
}

static ANode *expr(void) {
  ANode *l = term();
  while (parse_ok && toks[pos].kind == OP &&
         (toks[pos].op == '+' || toks[pos].op == '-')) {
    char op = toks[pos++].op;
    ANode *p = leaf(op == '+' ? 2 : 3);
    p->l = l;
    p->r = term();
    l = p;
  }
  return l;
}
```

listing 824: c, term and expr loops over a shared token cursor, factor below adds unary minus (source: dsa/samples-c/src/Ch26/rdparse.c, lines 118-142)

```
// Parse builds the AST for a full token stream. The grammar layers
// precedence: expr handles + and -, term handles * and /, factor
// handles numbers, variables, parentheses, and the productions that
// recurse back into expr. Left associativity comes from the loop
// shape: each operator folds onto everything parsed before it.
func Parse(tokens []Token) (Expr, error) {
  p := &parser{tokens: tokens}
  e, err := p.expr()
  if err ≠ nil {
    return nil, err
  }
  if p.pos ≠ len(tokens) {
    text := p.tokens[p.pos].Text
    return nil, fmt.Errorf("unexpected token %q after the expression", text)
  }
  return e, nil
}

func (p *parser) peek() (Token, bool) {
  if p.pos ≥ len(p.tokens) {
    return Token{}, false
  }
  return p.tokens[p.pos], true
}

func (p *parser) expr() (Expr, error) {
  left, err := p.term()
  if err ≠ nil {
    return nil, err
  }
  for {
    tok, ok := p.peek()
    if !ok || (tok.Text ≠ "+" && tok.Text ≠ "-") {
      return left, nil
    }
    p.pos++
    right, err := p.term()
    if err ≠ nil {
      return nil, err
    }
    left = Bin{Op: tok.Text, Left: left, Right: right}
  }
}
```

listing 825: go, parse and expr, errors returned not thrown, factor below (source: dsa/samples-go/ch26/rdparse.go, lines 33-75)

```
  static ANode term() {
    ANode l = factor();
    while (parseOk && toks[pos].kind == OP
        && (toks[pos].op == '*' || toks[pos].op == '/')) {
      char op = toks[pos++].op;
      ANode p = leaf(op == '*' ? 4 : 5);
      p.l = l;
      p.r = factor();
      l = p;
    }
    return l;
  }

  static ANode expr() {
    ANode l = term();
    while (parseOk && toks[pos].kind == OP
        && (toks[pos].op == '+' || toks[pos].op == '-')) {
      char op = toks[pos++].op;
      ANode p = leaf(op == '+' ? 2 : 3);
      p.l = l;
      p.r = term();
      l = p;
    }
    return l;
  }

  static ANode parse(String s) {
    lex(s);
    pos = 0;
    parseOk = true;
    ANode root = expr();
    if (toks[pos].kind != END)
      parseOk = false; // trailing junk
    return root;
  }
```

listing 826: java, term and expr loops over the shared cursor, factor below adds unary minus as 0 - x, trees pinned as s-expressions (source: dsa/samples-java/src/Ch26/Rdparse.java, lines 105-139)

```
        public Node Expr()
        {
            Node node = Term();
            while (Peek.Kind is TokenKind.Plus or TokenKind.Minus)
                node = new Bin(Take().Text[0], node, Term());
            return node;
        }

        private Node Term()
        {
            Node node = Factor();
            while (Peek.Kind is TokenKind.Star or TokenKind.Slash)
                node = new Bin(Take().Text[0], node, Factor());
            return node;
        }

        private Node Factor()
        {
            if (Peek.Kind == TokenKind.Number)
                return new Num(Take().Value);
            if (Peek.Kind == TokenKind.Ident)
                return new Ident(Take().Text);
            if (Match(TokenKind.LParen))
            {
                Node inner = Expr();
                Expect(TokenKind.RParen, "')'");
                return inner;
            }
            throw new InvalidOperationException($"expected a factor, got {Describe(Peek)}");
        }

        private static string Describe(Token token) =>
            token.Kind == TokenKind.End ? "end of input" : $"'{token.Text}'";
    }
```

listing 827: c#, expr, term, factor over a peek and take cursor (source: dsa/samples/src/Ch26/Parsing.cs, lines 125-158)

```
export function parse(src) {
  const toks = tokenize(src);
  let pos = 0;
  const peek = () => toks[pos];
  const eat = (kind) => {
    const t = toks[pos];
    if (!t || t.kind !== kind)
      throw new SyntaxError(`expected ${kind} at ${t ? t.pos : "end"}`);
    pos++;
    return t;
  };

  const factor = () => {
    const t = peek();
    if (!t) throw new SyntaxError(`expected a factor at end of input`);
    if (t.kind === "num") return { op: "num", value: eat("num").value };
    if (t.kind === "ident") return { op: "var", name: eat("ident").name };
    eat("(");
    const e = expr();
    eat(")");
    return e;
  };

  const term = () => {
    let node = factor();
    while (peek() && (peek().kind === "*" || peek().kind === "/"))
      node = { op: eat(peek().kind).kind, lhs: node, rhs: factor() };
    return node;
  };

  const expr = () => {
    let node = term();
    while (peek() && (peek().kind === "+" || peek().kind === "-"))
      node = { op: eat(peek().kind).kind, lhs: node, rhs: term() };
    return node;
  };

  const ast = expr();
  if (pos !== toks.length)
    throw new SyntaxError(`trailing input at ${toks[pos].pos}`);
  return ast;
}
```

listing 828: javascript, closures for factor, term, expr, trailing input refused (source: dsa/samples-js/src/ch26-rdparse.mjs, lines 10-51)

```
def parse(source):
    tokens = tokenize(source)
    pos = [0]

    def peek():
        return tokens[pos[0]] if pos[0] < len(tokens) else (None, None)

    def take():
        tok = peek()
        pos[0] += 1
        return tok

    def expr():  # term (('+' | '-') term)*
        node = term()
        while peek() == ("op", "+") or peek() == ("op", "-"):
            op = take()[1]
            node = (op, node, term())
        return node

    def term():  # factor (('*' | '/') factor)*
        node = factor()
        while peek() == ("op", "*") or peek() == ("op", "/"):
            op = take()[1]
            node = (op, node, factor())
        return node

    def factor():  # number or '(' expr ')'
        kind, value = take()
        if kind == "num":
            return ("num", value)
        if (kind, value) == ("paren", "("):
            node = expr()
            assert take() == ("paren", ")"), "expected )"
            return node
        raise SyntaxError(f"expected a factor, got {kind} {value}")

    tree = expr()
    assert pos[0] == len(tokens), "trailing tokens"
    return tree
```

listing 829: python, nested closures over a one-slot cursor list (source: dsa/samples-py/src/Ch26/rdparse.py, lines 38-76)

```lua
local function parse(src)
  local toks = tokenize(src)
  local pos = 1
  local function peek() return toks[pos] end
  local function take() pos = pos + 1; return toks[pos - 1] end

  -- mutual recursion via locals declared up front
  local factor, term_, expr_
  factor = function()
    local t = peek()
    if t.kind == "num" then
      return {tag = "num", v = tonumber(take().text)}
    elseif t.kind == "ident" then
      return {tag = "id", name = take().text}
    elseif t.kind == "lparen" then
      take()
      local e = expr_()
      if peek().kind ~= "rparen" then error("expected ')'", 2) end
      take()
      return e
    end
    error("expected a factor", 2)
  end
  term_ = function()
    local left = factor()
    while peek().kind == "op" and (peek().text == "*" or peek().text == "/") do
      local op = take().text
      left = {tag = "bin", op = op, l = left, r = factor()}
    end
    return left
  end
  expr_ = function()
    local left = term_()
    while peek().kind == "op" and (peek().text == "+" or peek().text == "-") do
      local op = take().text
      left = {tag = "bin", op = op, l = left, r = term_()}
    end
    return left
  end

  local ast = expr_()
  if peek().kind ~= "end" then error("trailing tokens", 2) end
  return ast
```

listing 830: lua, locals declared up front so the mutual recursion binds (source: dsa/samples-lua/ch26_rdparse.lua, lines 39-81)

The shape pins across languages: 2+3 times 4 nests the multiply inside the plus, parentheses lift the plus, and the subtraction chain 8-3-2 parses left associative as (8-3)-2, which is the same fixture chapter 5's shunting-yard settled. Failure modes are where the languages part ways. Go threads errors as values through every level, javascript and python raise SyntaxError, lua and c# throw with the token in the message, and c and java set a parse-ok flag and return a best-effort tree, both parsers accepting a unary minus by rewriting x as 0-x. C's and java's parse files keep single-letter names against a 26-slot symbol table, and go's one-letter identifiers from the previous section flow straight into its factor rule.

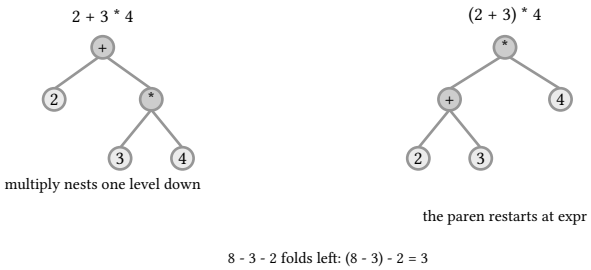


figure 212: two trees from one expression, precedence nesting the multiply, parentheses lifting the plus

26.3. evaluating the ast

Evaluation is a fold over the tree. A number node returns itself, a name node looks up an environment, an operator node evaluates both children and applies. Printing walks the same tree, and the test that keeps printer and parser honest is the round trip: print the tree, parse the print, land on the identical tree.

The dry run: the fixtures are those two trees with the environment x = 5, y = −2, every value asserted by the C# suite, the 14 and 20 pinning in every suite that carries them.

1. The fold bottoms out on the right first: the multiply node folds 3 × 4 = 12, then the root adds 2 + 12 = 14.
2. The paren twin folds inside out the other way, 2 + 3 = 5 then 5 × 4 = 20.
3. The chain 8 - 3 - 2 folds to 5 - 2 = 3, and 7 / 2 truncates to 3.
4. Names read the environment: x × x + 1 folds 5 × 5 = 25 then 25 + 1 = 26, and x × y + 3 folds 5 × −2 = −10 then −10 + 3 = −7.
5. Printing wraps only where precedence demands: 2 + 3 * 4 prints back as itself, 8 - (3 - 2) keeps its pair, and the round trip reparses every print into the identical tree.

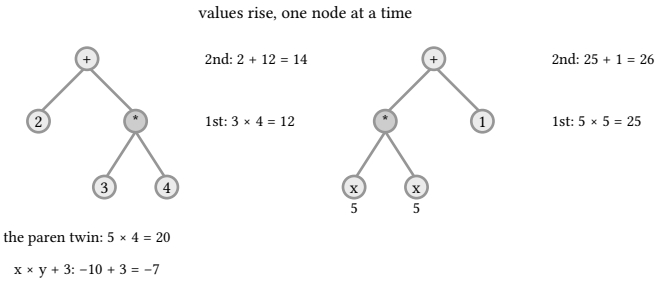


figure 213: the fold rising from the leaves, the multiply closing first at 12 and the root second at 14, the environment lookups feeding the sibling tree

The 14 closes the pinned fold, and the listings below evaluate the same trees in seven languages.

```
static long long eval(ANode *p) {
  switch (p->tag) {
  case 0:
    return p->num;
  case 1:
    return symtab[p->name - 'a'];
  case 2:
    return eval(p->l) + eval(p->r);
  case 3:
    return eval(p->l) - eval(p->r);
  case 4:
    return eval(p->l) * eval(p->r);
  default:
    return eval(p->l) / eval(p->r);
  }
}

// fully parenthesized print
static void print_tree(ANode *p, char *out) {
  if (p->tag <= 1) {
    if (p->tag == 0)
      sprintf(out, "%lld", p->num);
    else
      sprintf(out, "%c", p->name);
    return;
  }
  char opchar = "+-*/"[p->tag - 2];
  char buf[80];
  print_tree(p->l, buf);
  int k = (int)strlen(buf);
  *out++ = '(';
  memcpy(out, buf, (size_t)k);
  out += k;
  *out++ = opchar;
  print_tree(p->r, buf);
  k = (int)strlen(buf);
  memcpy(out, buf, (size_t)k);
  out += k;
  *out++ = ')';
  *out = 0;
}
```

listing 831: c, the fold over node tags, a 26-slot symbol table, print into a char buffer (source: dsa/samples-c/src/Ch26/asteval.c, lines 147-187)

```
func Eval(e Expr, vars map[string]int) (int, error) {
  switch n := e.(type) {
  case Num:
    return n.Value, nil
  case Var:
    v, ok := vars[n.Name]
    if !ok {
      return 0, fmt.Errorf("unbound variable %q", n.Name)
    }
    return v, nil
  case Bin:
    left, err := Eval(n.Left, vars)
    if err ≠ nil {
      return 0, err
    }
    right, err := Eval(n.Right, vars)
    if err ≠ nil {
      return 0, err
    }
    switch n.Op {
    case "+":
      return left + right, nil
    case "-":
      return left - right, nil
    case "*":
      return left * right, nil
    default:
      if right == 0 {
        return 0, fmt.Errorf("division by zero")
      }
      return left / right, nil
    }
  }
  return 0, fmt.Errorf("unknown node %T", e)
}

// PrintParen renders the tree fully parenthesized: every binary node
// wraps itself in parentheses, so printing loses no grouping at all
// and lexing plus parsing the output restores the same shape.
func PrintParen(e Expr) string {
  switch n := e.(type) {
  case Num:
    return fmt.Sprintf("%d", n.Value)
  case Var:
    return n.Name
  case Bin:
    return "(" + PrintParen(n.Left) + n.Op + PrintParen(n.Right) + ")"
  }
  return ""
}

// RoundTrip prints, lexes, parses, and prints again: the second
```

listing 832: go, eval with a vars map, a fully parenthesized printer, roundtrip helper (source: dsa/samples-go/ch26/asteval.go, lines 8-59)

```
  // the symbol table: single-letter names
  static long[] symtab = new long[26];

  static long eval(ANode p) {
    switch (p.tag) {
      case 0:
        return p.num;
      case 1:
        return symtab[p.name - 'a'];
      case 2:
        return eval(p.l) + eval(p.r);
      case 3:
        return eval(p.l) - eval(p.r);
      case 4:
        return eval(p.l) * eval(p.r);
      default:
        return eval(p.l) / eval(p.r);
    }
  }

  // fully parenthesized print
  static void printTree(ANode p, StringBuilder out) {
    if (p.tag ≤ 1) {
      if (p.tag == 0)
        out.append(p.num);
      else
        out.append(p.name);
      return;
    }
    out.append('(');
    printTree(p.l, out);
    out.append("+-*/".charAt(p.tag - 2));
    printTree(p.r, out);
    out.append(')');
  }
```

listing 833: java, the switch fold, a 26-slot symbol table, fully parenthesized printing (source: dsa/samples-java/src/Ch26/Asteval.java, lines 131-165)

```
public static class Ast
{
    public static long Evaluate(Node node, IReadOnlyDictionary<string, long> env) => node switch
    {
        Num n => n.Value,
        Ident id => env.TryGetValue(id.Name, out long value)
            ? value
            : throw new InvalidOperationException($"unknown identifier '{id.Name}'"),
        Bin b => b.Op switch
        {
            '+' => Evaluate(b.Left, env) + Evaluate(b.Right, env),
            '-' => Evaluate(b.Left, env) - Evaluate(b.Right, env),
            '*' => Evaluate(b.Left, env) * Evaluate(b.Right, env),
            '/' => Evaluate(b.Left, env) / Evaluate(b.Right, env), // truncates toward zero
            _ => throw new InvalidOperationException($"unknown operator '{b.Op}'"),
        },
        _ => throw new InvalidOperationException($"cannot evaluate {node.GetType().Name}"),
    };

    /// <summary>Minimal parentheses: only where precedence or left associativity demands.</summary>
    public static string Print(Node node) => Format(node, 0);

    private static string Format(Node node, int parentPrecedence)
    {
        if (node is not Bin b)
            return node switch
            {
                Num n => n.Value.ToString(),
                Ident id => id.Name,
                _ => throw new InvalidOperationException("unknown node"),
            };
        int precedence = b.Op is '*' or '/' ? 2 : 1;
        string text = $"{Format(b.Left, precedence)} {b.Op} {Format(b.Right, precedence + 1)}";
        return precedence < parentPrecedence ? $"({text})" : text;
    }
}
```

listing 834: c#, pattern-switch eval, minimal parentheses by parent precedence (source: dsa/samples/src/Ch26/Parsing.cs, lines 162-197)

```javascript
export function evalAst(ast, env = {}) {
  switch (ast.op) {
    case "num":
      return ast.value;
    case "var":
      if (!(ast.name in env))
        throw new ReferenceError(`unbound name '${ast.name}'`);
      return env[ast.name];
    case "+":
      return evalAst(ast.lhs, env) + evalAst(ast.rhs, env);
    case "-":
      return evalAst(ast.lhs, env) - evalAst(ast.rhs, env);
    case "*":
      return evalAst(ast.lhs, env) * evalAst(ast.rhs, env);
    case "/":
      return Math.trunc(evalAst(ast.lhs, env) / evalAst(ast.rhs, env));
    default:
      throw new Error(`unknown node '${ast.op}'`);
  }
}

// every binary node gets parentheses; leaves render bare
export function printParen(ast) {
  if (ast.op === "num") return String(ast.value);
  if (ast.op === "var") return ast.name;
  return `(${printParen(ast.lhs)}${ast.op}${printParen(ast.rhs)})`;
}
```

listing 835: javascript, truncating division, unbound names raise (source: dsa/samples-js/src/ch26-asteval.mjs, lines 4-30)

```python
def evaluate(tree):
    kind = tree[0]
    if kind == "num":
        return tree[1]
    left, right = evaluate(tree[1]), evaluate(tree[2])
    if kind == "+":
        return left + right
    if kind == "-":
        return left - right
    if kind == "*":
        return left * right
    return left // right  # fixtures keep division exact


def render(tree):
    if tree[0] == "num":
        return str(tree[1])
    return f"({render(tree[1])}{tree[0]}{render(tree[2])})"


def parse_source(source):
    return parse(tokenize(source))
```

listing 836: python, the fold, a fully parenthesized render, the round trip (source: dsa/samples-py/src/Ch26/asteval.py, lines 77-98)

```lua
local function evaluate(ast, env)
  env = env or {}
  if ast.tag == "num" then return ast.v end
  if ast.tag == "id" then
    local v = env[ast.name]
    if v == nil then error("unknown identifier '" .. ast.name .. "'", 2) end
    return v
  end
  local a, b = evaluate(ast.l, env), evaluate(ast.r, env)
  if ast.op == "+" then return a + b end
  if ast.op == "-" then return a - b end
  if ast.op == "*" then return a * b end
  return a // b
end

-- parenthesize a child when its precedence is lower than the parent's
local function print_ast(ast, parent_prec)
  parent_prec = parent_prec or 0
  if ast.tag == "num" then return tostring(ast.v) end
  if ast.tag == "id" then return ast.name end
  local prec = (ast.op == "+" or ast.op == "-") and 1 or 2
  local l, r = print_ast(ast.l, prec), print_ast(ast.r, prec + 1)
  local s = l .. " " .. ast.op .. " " .. r
  if prec < parent_prec then return "(" .. s .. ")" end
  return s
end
```

listing 837: lua, env lookup with a named error, precedence-aware parens (source: dsa/samples-lua/ch26_asteval.lua, lines 76-101)

The values agree where the fixtures overlap: 2+3 times 4 evaluates to 14 and (2+3) times 4 to 20 in every suite that pins them, and the printers split into two honest camps. C, go, java, javascript, and python print fully parenthesized, every binary node wrapped, which loses nothing. C# and lua print minimal parentheses, wrapping a child only when its precedence is lower than its parent's, so 2+3 times 4 prints back as itself. Python pins the strongest property of the section, printing and reparsing its own render yields a tree structurally equal to the original, and java pins the value round trip over five fixtures plus the print fixed point, reprinting the reparsed tree char for char. Division semantics get one sentence each: c#, java, and javascript truncate toward zero, python and lua floor, go and c use integer division on ints, and the fixtures are chosen so the distinction never changes an answer.

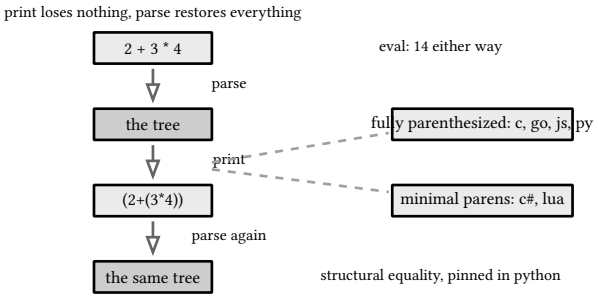


figure 214: the round trip, source to tree to print to tree again, with the two printer camps

26.4. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's three sample files, test scripts included where the language embeds them:

language	build sloc	container dependency	boundary note
c	507	fixed token and node pools	err tokens instead of exceptions, unary minus rewrite, single-letter names in the parser
c#	166	records, pattern switches	minimal-parens printer by parent precedence, positions in errors
go	226	interfaces for node kinds	single-letter identifiers, errors as values through every level
java	512	jdk 27 stdlib	err tokens in the lexer, the parse-ok flag with a best-effort tree, unary minus rewritten as 0 - x, the print fixed point pinned
javascript	96	plain objects, closures	source position on every token, truncating division
python	249	tuples as nodes	floor division on exact fixtures, round-trip equality pinned
lua	291	tables tagged by kind	locals declared ahead for mutual recursion, floor division

sources: learn.microsoft.com for records and pattern matching over them, go.dev for `fmt.Errorf` and interface satisfaction, developer.mozilla.org for `SyntaxError` and `Math.trunc`, docs.python.org for `str.isdigit` and friends, lua.org for `string.match` patterns, accessed 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 26: c 3 files and 47 checks, c# 9 tests, go 14 tests, java 3 files and 47 checks, javascript 11 tests, python 3 files and 27 asserts, lua 12 checks, zero skipped.

27. an opcode machine

The smallest useful interpreter is one array and one index. Memory holds integers, the instruction pointer walks them, and each word names an operation with its operands sitting in the following cells. Chapter 26 parsed expressions into trees and evaluated them recursively, and this chapter compiles the same arithmetic into linear code with jumps, the shape real bytecode interpreters take. It is the same design pushed to networked processors in contest puzzles, a sibling of the state machines in book 14, ch 5 (game systems and architecture, finite state machines), and the loop at the bottom of book 7, ch 12 (python 3.14, how cpython runs your code). The same loop carries the icpc book's 2019 finals, problem I (karel), a tiny robot language compiled to segments and run iteratively with an explicit resume stack, the form the shallowest host call stacks require.

27.1. the instruction pointer loop

Four opcodes carry the whole idea: 1 adds, 2 multiplies, 3 reads one input, 4 writes one output, and 99 halts. Operands are addresses, the result address last, every instruction advances the pointer by its own length, and the machine keeps nothing else, no registers, no stack. Programs are data, memory can be rewritten by the program itself, and cells past the halt are fair storage.

The dry run: the fixture is the C# add program 1,5,6,7,99 with 10 and 20 in its data cells, asserted with its step count by the C# suite, the chained 1,1,1,4,99,5,6,0,99 pinning its 30 in the C, Java, and Python suites.

1. The pointer opens at 0, reads opcode 1, add, and the operand fields 5 and 6 are addresses: the machine reads 10 out of slot 5 and 20 out of slot 6.
2. The destination field 7 sends 10 + 20 = 30 into slot 7, the pointer advances by its own length of 4, and the 99 there closes the run at 2 steps.
3. The multiply twin 2,5,6,7,99 lands 10 × 20 = 200 in the same slot, and the echo program 3,0,4,0,99 reads the input 42 into slot 0 and writes it back out, 3 steps.
4. The chained program rewrites itself: its first add reads slots 1 and 1 and stores 1 + 1 = 2 over the halt at slot 4, so the pointer now stands on a multiply.
5. That multiply reads slots 5 and 6 and stores 5 × 6 = 30 into slot 0, and the relocated halt at slot 8 ends the run, memory closing 30, 1, 1, 4, 2, 5, 6, 0, 99.
6. Cells past the code never move: the machine's clone keeps the original program array untouched while its own memory bends.

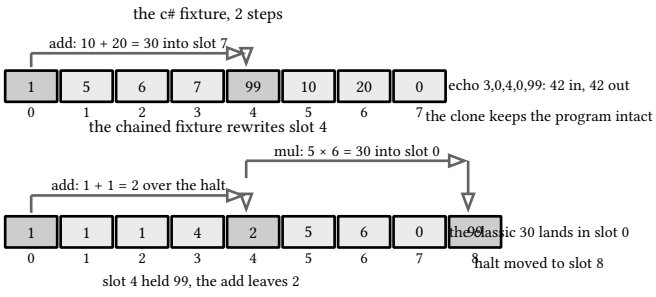


figure 215: the pointer as a sequence over both fixtures, the c# add closing in 2 steps, the chained program relocating its own halt before the multiply lands 30

Both fixtures close at the classic 30, and the listings below run the loop in seven languages.

```c
// run until halt or fault; parameter addressing is positional here
static void run(void) {
  int ip = 0;
  while (ip < MEM) {
    long long op = mem[ip];
    if (op == 99) {
      halted = 1;
      return;
    }
    if (op == 1 || op == 2) {
      long long a = mem[mem[ip + 1]], b = mem[mem[ip + 2]];
      mem[mem[ip + 3]] = op == 1 ? a + b : a * b;
      ip += 4;
    } else if (op == 3) {
      if (in_pos ≥ in_n) {
        error_ = 1; // input exhausted
        return;
      }
      mem[mem[ip + 1]] = input_[in_pos++];
      ip += 2;
    } else if (op == 4) {
      if (out_n < 8)
        output_[out_n++] = mem[mem[ip + 1]];
      ip += 2;
    } else {
      error_ = 1; // unknown opcode
      return;
    }
  }
}
```

listing 838: c, run to halt or fault, positional addressing, error flags over exceptions (source: dsa/samples-c/src/Ch27/opcodes.c, lines 33-62)

```
func (m *Machine) Run() error {
  for m.ip < len(m.memory) {
    op, modes := decode(m.memory[m.ip])
    switch op {
    case 99:
      return nil
    case 1, 2, 7, 8: // add, mul, less-than, equals: three params
      a := m.read(1, modes)
      b := m.read(2, modes)
      at := int(m.memory[m.ip+3])
      switch op {
      case 1:
        m.memory[at] = a + b
      case 2:
        m.memory[at] = a * b
      case 7:
        if a < b {
          m.memory[at] = 1
        } else {
          m.memory[at] = 0
        }
      default:
        if a == b {
          m.memory[at] = 1
        } else {
          m.memory[at] = 0
        }
      }
      m.ip += 4
    case 3: // input: one address parameter
      if len(m.input) == 0 {
        return fmt.Errorf("opcode 3 with empty input queue")
      }
      m.memory[int(m.memory[m.ip+1])] = m.input[0]
      m.input = m.input[1:]
      m.ip += 2
    case 4: // output: one parameter
      m.output = append(m.output, m.read(1, modes))
      m.ip += 2
    case 5, 6: // jump-if-true, jump-if-false: two params
      cond := m.read(1, modes)
      target := m.read(2, modes)
      jump := (op == 5 && cond ≠ 0) || (op == 6 && cond == 0)
      if jump {
        m.ip = int(target)
      } else {
        m.ip += 3
      }
    default:
      return fmt.Errorf("unknown opcode %d at %d", op, m.ip)
    }
  }
  return fmt.Errorf("instruction pointer ran off the end")
}
```

listing 839: go, run drives a machine struct, errors returned at every exit (source: dsa/samples-go/ch27/opcodes.go, lines 39-92)

```
  static void load(long[] prog) {
    for (int i = 0; i < MEM; i++)
      mem[i] = i < prog.length ? prog[i] : 0;
    inPos = 0;
    outN = 0;
    halted = error = false;
  }

  // run until halt or fault; parameter addressing is positional here
  static void run() {
    int ip = 0;
    while (ip < MEM) {
      long op = mem[ip];
      if (op == 99) {
        halted = true;
        return;
      }
      if (op == 1 || op == 2) {
        long a = mem[(int) mem[ip + 1]], b = mem[(int) mem[ip + 2]];
        mem[(int) mem[ip + 3]] = op == 1 ? a + b : a * b;
        ip += 4;
      } else if (op == 3) {
        if (inPos ≥ inN) {
          error = true; // input exhausted
          return;
        }
        mem[(int) mem[ip + 1]] = input[inPos++];
        ip += 2;
      } else if (op == 4) {
        if (outN < 8)
          output[outN++] = mem[(int) mem[ip + 1]];
        ip += 2;
      } else {
        error = true; // unknown opcode
        return;
      }
    }
  }
```

listing 840: java, static long memory, halted and error flags, input exhaustion faults instead of spinning (source: dsa/samples-java/src/Ch27/Opcodes.java, lines 13-50)

```
        int ip = 0, inputIndex = 0, steps = 0;
        while (true)
        {
            steps++;
            long opcode = memory[ip] % 100;
            long Mode(int k) => memory[ip] / TenPow(1 + k) % 10;
            long Read(int k) => Mode(k) == 0 ? memory[memory[ip + k]] : memory[ip + k];

            switch (opcode)
            {
                case 1:
                    memory[memory[ip + 3]] = Read(1) + Read(2);
                    ip += 4;
                    break;
                case 2:
                    memory[memory[ip + 3]] = Read(1) * Read(2);
                    ip += 4;
                    break;
                case 3:
                    if (inputIndex ≥ inputs.Count)
                        throw new InvalidOperationException("machine starved for input");
                    memory[memory[ip + 1]] = inputs[inputIndex++];
                    ip += 2;
                    break;
                case 4:
                    outputs.Add(Read(1));
                    ip += 2;
                    break;
                case 5:
                    if (Read(1) ≠ 0)
                        ip = (int)Read(2);
                    else
                        ip += 3;
                    break;
                case 6:
                    if (Read(1) == 0)
                        ip = (int)Read(2);
                    else
                        ip += 3;
                    break;
                case 7:
                    memory[memory[ip + 3]] = Read(1) < Read(2) ? 1 : 0;
                    ip += 4;
                    break;
                case 8:
                    memory[memory[ip + 3]] = Read(1) == Read(2) ? 1 : 0;
                    ip += 4;
                    break;
                case 99:
                    return new RunResult(memory, outputs, steps);
                default:
                    throw new InvalidOperationException($"unknown opcode {opcode} at address {ip}");
            }
        }
```

listing 841: c#, one switch, mode closures below, long memory, a step counter (source: dsa/samples/src/Ch27/Machine.cs, lines 21-74)

```javascript
export function runProgram(program, inputs = []) {
  const mem = [...program];
  const outputs = [];
  let ip = 0;
  let inPos = 0;
  for (;;) {
    const opcode = mem[ip];
    if (opcode === 99) return { memory: mem, outputs };
    if (opcode === 1) {
      mem[mem[ip + 3]] = mem[mem[ip + 1]] + mem[mem[ip + 2]];
      ip += 4;
    } else if (opcode === 2) {
      mem[mem[ip + 3]] = mem[mem[ip + 1]] * mem[mem[ip + 2]];
      ip += 4;
    } else if (opcode === 3) {
      mem[mem[ip + 1]] = inputs[inPos++];
      ip += 2;
    } else if (opcode === 4) {
      outputs.push(mem[mem[ip + 1]]);
      ip += 2;
    } else {
      throw new Error(`unknown opcode ${opcode} at ${ip}`);
    }
  }
}
```

listing 842: javascript, the base loop, every operand in position mode (source: dsa/samples-js/src/ch27-opcodes.mjs, lines 6-30)

```python
def run(program, inputs):
    mem = list(program)
    ip = 0
    outputs = []
    in_pos = 0
    while True:
        op = mem[ip]
        if op == 99:  # halt
            return mem, outputs
        if op == 1:  # add: mem[c] = mem[a] + mem[b]
            a, b, c = mem[ip + 1 : ip + 4]
            mem[c] = mem[a] + mem[b]
            ip += 4
        elif op == 2:  # multiply: mem[c] = mem[a] * mem[b]
            a, b, c = mem[ip + 1 : ip + 4]
            mem[c] = mem[a] * mem[b]
            ip += 4
        elif op == 3:  # read one input into mem[a]
            a = mem[ip + 1]
            mem[a] = inputs[in_pos]
            in_pos += 1
            ip += 2
        elif op == 4:  # emit mem[a]
            outputs.append(mem[mem[ip + 1]])
            ip += 2
        else:
            raise ValueError(f"unknown opcode {op} at {ip}")
```

listing 843: python, slices pull whole instructions, unknown opcodes raise (source: dsa/samples-py/src/Ch27/opcodes.py, lines 14-40)

```lua
local function run(program, input)
  local mem = {table.unpack(program)}
  local outputs = {}
  local ip = 0 -- 0-based instruction pointer
  local steps = 0
  while true do
    steps = steps + 1
    local op = mem[ip + 1]
    if op == 1 then
      mem[mem[ip + 4] + 1] = mem[mem[ip + 2] + 1] + mem[mem[ip + 3] + 1]
      ip = ip + 4
    elseif op == 2 then
      mem[mem[ip + 4] + 1] = mem[mem[ip + 2] + 1] * mem[mem[ip + 3] + 1]
      ip = ip + 4
    elseif op == 3 then
      if input == nil then error("starved of input", 2) end
      mem[mem[ip + 2] + 1] = input
      ip = ip + 2
    elseif op == 4 then
      outputs[#outputs + 1] = mem[mem[ip + 2] + 1]
      ip = ip + 2
    elseif op == 99 then
      return mem, outputs, steps
    else
      error("unknown opcode " .. tostring(op), 2)
    end
```

listing 844: lua, 1-based cells shift every index, steps counted per instruction (source: dsa/samples-lua/ch27_opcodes.lua, lines 7-32)

The day-2 style fixtures agree across all seven suites: 1,0,0,0,99 writes 1+1 into slot 0, and the chained program 1,1,1,4,99,5,6,0,99 pins the classic 30, the add at slot 0 overwriting the halt that then moves execution forward. Python squares its own input by writing it past the code, java doubling its input the same way with its register past the halt, javascript and java echo an input back out, and the lua suite counts steps, hand-traced instruction executions, alongside every run. Every language refuses unknown opcodes, by exception or error flag, and the c and java loops also stop on exhausted input instead of spinning.

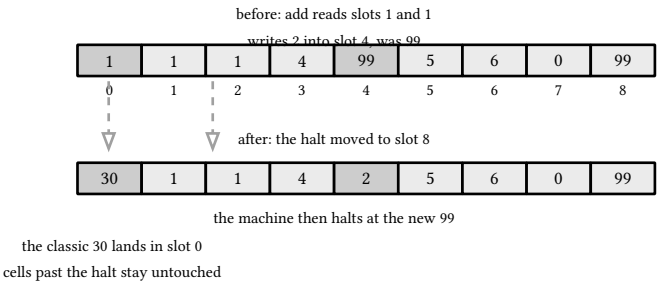


figure 216: the chained fixture executing, the add at slot 0 reading slots 1 and 1, writing 30 over the halt at slot 4

27.2. parameter modes

One instruction word carries more than the opcode. The low two digits name the operation, and each higher decimal digit is the mode of one parameter, least significant first: 0 reads the value at the address, 1 uses the literal in the cell. So 1002 is a multiply with its first operand positional and its second immediate, 1101 adds two literals, and destinations always write through an address, modes never apply to them.

The dry run: the fixture is the C# triplet over one memory shape with 99 and 2 in the data cells, 101, 9, and 104 asserted by the C# suite, go pinning the decoder split of 1002, java the three multiply readings 132, 99, and 12, and python the same word storing

99.

100. One memory, three instruction words: 1, 1101, and 1001 all point their operand fields at 4, 5, 6 with destination 6, and slot 4 holds 99 while slot 5 holds 2.

101. The positional word 1 reads both fields as addresses: 99 + 2 = 101 lands in slot 6.

102. The immediate word 1101 reads both fields as literals: 4 + 5 = 9 in the same slot.

103. The mixed word 1001 reads the first positionally and the second immediately: 99 + 5 = 104.

104. The decoder peels the word from the right: 1002 splits into opcode 2 with modes position then immediate, and python runs that word over 33 in slot 4, storing 33 × 3 = 99.

105. The destination never carries a mode: all three words write through the literal field 6.

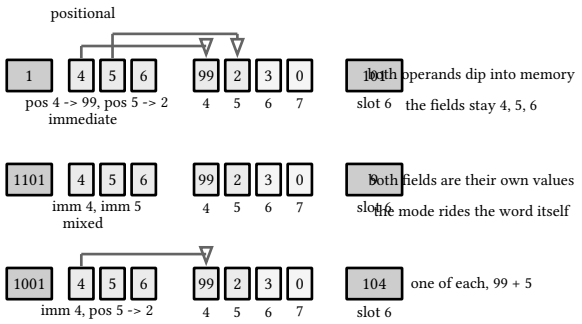


figure 217: the three reads as a sequence over one memory shape, positional arcs dipping into the data cells, immediate labels holding the field itself, the three landings stacked at the right

The triplet 101, 9, 104 closes the reads, and the listings below decode them in seven languages.

```
// operand e: the hundreds digit is param 1's mode, thousands param 2's;
// mode 0 reads through mem, mode 1 uses the literal
static long long operand(int ip, int which) {
  long long mode = mem[ip] / 100;
  for (int k = 0; k < which; k++)
    mode /= 10;
  mode %= 10;
  if (mode == 1)
    return mem[ip + 1 + which];
  return mem[mem[ip + 1 + which]];
}

static void run(void) {
  int ip = 0;
  while (!error_ && ip < MEM) {
    long long op = mem[ip] % 100;
    if (op == 99)
      return;
    if (op == 1 || op == 2) {
      long long a = operand(ip, 0), b = operand(ip, 1);
      mem[mem[ip + 3]] = op == 1 ? a + b : a * b;
      ip += 4;
    } else if (op == 4) {
      if (out_n < 4)
        output_[out_n++] = operand(ip, 0);
      ip += 2;
    } else {
      error_ = 1;
    }
  }
}
```

listing 845: c, operand walks the mode digits one divide and mod per parameter (source: dsa/samples-c/src/Ch27/modes.c, lines 30-60)

```
// DecodeModes is the mode decoder exposed for teaching: the same
// function the machine loop uses, so a test can pin how a full opcode
// word splits.
func DecodeModes(word int64) (op int64, modes uint) { return decode(word) }

// ModeOf reports whether parameter n (1-based) of a word is
// immediate.
func ModeOf(word int64, n int) bool {
  _, modes := decode(word)
  return modes>>(uint(n-1))&1 == 1
}
```

listing 846: go, the decoder exposed for teaching, modes packed as a bitmask (source: dsa/samples-go/ch27/modes.go, lines 3-13)

```
  // operand e: the hundreds digit is param 1's mode, thousands param 2's;
  // mode 0 reads through mem, mode 1 uses the literal
  static long operand(int ip, int which) {
    long mode = mem[ip] / 100;
    for (int k = 0; k < which; k++)
      mode /= 10;
    mode %= 10;
    if (mode == 1)
      return mem[ip + 1 + which];
    return mem[(int) mem[ip + 1 + which]];
  }

  static void run() {
    int ip = 0;
    while (!error && ip < MEM) {
      long op = mem[ip] % 100;
      if (op == 99)
        return;
      if (op == 1 || op == 2) {
        long a = operand(ip, 0), b = operand(ip, 1);
        mem[(int) mem[ip + 3]] = op == 1 ? a + b : a * b;
        ip += 4;
      } else if (op == 4) {
        if (outN < 4)
          output[outN++] = operand(ip, 0);
        ip += 2;
      } else {
        error = true;
      }
    }
  }
```

listing 847: java, the operand helper peeling mode digits one divide and mod per parameter, writes always positional (source: dsa/samples-java/src/Ch27/Modes.java, lines 19-49)

```
        while (true)
        {
            steps++;
            long opcode = memory[ip] % 100;
            long Mode(int k) => memory[ip] / TenPow(1 + k) % 10;
            long Read(int k) => Mode(k) == 0 ? memory[memory[ip + k]] : memory[ip + k];

            switch (opcode)
            {
```

listing 848: c#, the mode and read closures inside the loop, tenpow beside (source: dsa/samples/src/Ch27/Machine.cs, lines 22-30)

```
export function decodeWord(word) {
  return {
    opcode: word % 100,
    modes: [
      Math.floor(word / 100) % 10,
      Math.floor(word / 1000) % 10,
      Math.floor(word / 10000) % 10,
    ],
  };
}
```

listing 849: javascript, decodeword returns opcode plus a modes array (source: dsa/samples-js/src/ch27-modes.mjs, lines 8-17)

```
def decode(instruction):
    op = instruction % 100
    modes = []
    digits = instruction // 100
    for _ in range(2):  # two parameters get modes, low digit first
        modes.append(digits % 10)
        digits //= 10
    return op, modes


def run(program, inputs=()):
    mem = list(program)
    ip = 0
    outputs = []
    in_pos = 0
    while True:
        op, modes = decode(mem[ip])
        if op == 99:
            return mem, outputs

        def param(k):  # position by default, immediate on mode 1
            raw = mem[ip + 1 + k]
            return raw if modes[k] == 1 else mem[raw]

        if op in (1, 2):
            a, b = param(0), param(1)
            out = mem[ip + 3]
            mem[out] = a + b if op == 1 else a * b
            ip += 4
        elif op == 3:
            mem[mem[ip + 1]] = inputs[in_pos]
            in_pos += 1
            ip += 2
        elif op == 4:
            outputs.append(param(0))
            ip += 2
        else:
            raise ValueError(f"unknown opcode {op} at {ip}")
```

listing 850: python, decode peels two digits, a param closure reads either way (source: dsa/samples-py/src/Ch27/modes.py, lines 14-51)

```lua
local function run(program, input)
  local mem = {table.unpack(program)}
  local outputs = {}
  local ip = 0
  local steps = 0
  while true do
    steps = steps + 1
    local instr = mem[ip + 1]
    local op = instr % 100
    local m1 = instr // 100 % 10
    local m2 = instr // 1000 % 10
    local function param(n, mode)
      -- mode 0 reads the cell the operand points at, mode 1 the operand
      if mode == 0 then return mem[mem[ip + n + 1] + 1] end
      return mem[ip + n + 1]
    end
    if op == 1 or op == 2 then
      local a, b = param(1, m1), param(2, m2)
      local out = mem[ip + 4] -- the target address is always positional
      mem[out + 1] = (op == 1) and (a + b) or (a * b)
      ip = ip + 4
    elseif op == 3 then
      mem[mem[ip + 2] + 1] = input
      ip = ip + 2
    elseif op == 4 then
      outputs[#outputs + 1] = param(1, m1)
      ip = ip + 2
    elseif op == 99 then
      return mem, outputs, steps
    else
      error("unknown opcode " .. tostring(op), 2)
```

listing 851: lua, floor div and mod peel the digits, target always positional (source: dsa/samples-lua/ch27_modes.lua, lines 7-37)

The c# suite pins the teaching triplet on one fixture shape, memory 99 and 2 in the data cells: 2 positional gives 101, the sum of two cell values, 1101 immediate gives 9, the sum of two literals, and 1001 mixed gives 104, a cell plus a literal. Go pins the decoder as a unit, 1002 splits into multiply with modes position then immediate and 11107 into a fully immediate comparison, and python pins the same 1002 storing 99, the 33 in slot 4 times the literal

3. Java runs the three multiply readings on one memory shape, 132

all positional, 99 mixed, 12 both immediate, and pins the output modes too, 104 emitting the literal 42 while plain 4 reads through memory. Javascript and lua run the same mixed multiply, and the all-position twin in go and python proves the two readings of one program agree only when the data is arranged for it, which is the whole point of the modes existing.

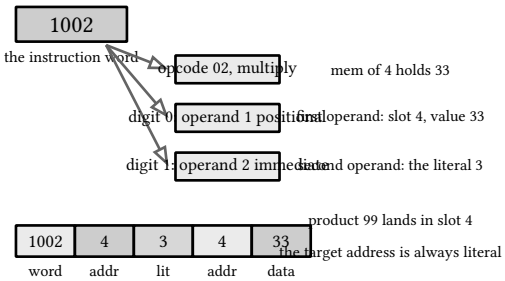


figure 218: one instruction word 1002 splitting into an opcode and two mode digits, each digit steering how its operand is read

27.3. jumps and comparisons

Opcodes 5 and 6 move the pointer: jump if the first parameter is nonzero, or zero, to the address in the second. Opcodes 7 and 8 write a comparison flag, 1 or 0, into a cell. Together they build loops, and the loop program is the chapter's capstone fixture: read n, decrement it, count the passes, jump back while n is nonzero, output the count. Data cells sit past the halt so the loop cannot overwrite its own code, and the jump targets are immediates, a position-mode target would be dereferenced first.

The dry run: the fixture is the countdown program at input 3, its outputs, final cells, and 15 hand-traced steps asserted by the C# suite, lua pinning the same trace.

1. The read at cell 0 stores 3 into data slot 20, 1 step, and the pointer lands on the zero test at 2.
2. Pass one: the test reads 3, nonzero, and falls through, the two immediate adds take n to 2 and the count to 1, and the backward jump returns to 2, 4 steps.
3. Pass two repeats on n = 2: n falls to 1, the count rises to 2, the jump returns, 4 steps.
4. Pass three leaves n at 0 and the count at 3, and now the nonzero test fails, so the pointer falls through to the output at 16, 4 steps.
5. The output writes 3 and the halt at 18 closes the run: 1 + 4 + 4 + 4 + 2 = 15 steps, slot 20 at 0 and slot 21 at 3.
6. Input 0 jumps straight from the test to the output, 4 steps and 0 out, and input 7 loops seven passes to 7.

phase	n	count	steps	pointer walk
read into slot 20	3	0	1	0 -> 2
pass one	3 to 2	0 to 1	4	2 -> 5 -> 9 -> 13 -> 2
pass two	2 to 1	1 to 2	4	2 -> 5 -> 9 -> 13 -> 2
pass three	1 to 0	2 to 3	4	2 -> 5 -> 9 -> 13 -> 16
output then halt	0	3	2	16 -> 18

The 15 steps close at the pinned 3, and the listings below build the loop in seven languages.

```c
// the loop program: registers n at 28, i at 29, acc at 30, flag at 31.
// body: i++; acc += i; flag = (i == n); jump back while flag == 0 (the
// jump target is immediate, address 10); output acc
static const long long SUMPROG[29] = {
    3,    28,         // read n
    1101, 0,  0,  29, // i = 0
    1101, 0,  0,  30, // acc = 0
    1001, 29, 1,  29, // i += 1
    1,    29, 30, 30, // acc += i
    8,    29, 28, 31, // flag = (i == n)
    1006, 31, 10, // while flag == 0 goto 10 (flag positional, target immediate)
    4,    30,     // output acc
    99            // halt
};
```

listing 852: c, the sum program, data registers past the 29 cells of code (source: dsa/samples-c/src/Ch27/jumps.c, lines 87-100)

```go
// CountdownProgram counts an input n down to zero with a jump loop
// and outputs how many passes the loop made. Code occupies cells
// 0..18; the two data cells sit past the end so the loop cannot
// overwrite its own instructions:
//
//  0:  3,19          read n into cell 19
//  2:  1006,19,16    if n is zero, jump to the literal 16
//  5:  1001,19,-1,19 n -= 1
//  9:  1001,20,1,20  pass counter += 1
//  13: 1005,20,2     counter is nonzero: jump to the literal 2
//  16: 4,20          output the pass count
//  18: 99            halt
//
// Both jumps read their condition by address and their target as an
// immediate: in position mode the target operand would be
// dereferenced as an address first.
func CountdownProgram() []int64 {
  return []int64{
    3, 19,
    1006, 19, 16,
    1001, 19, -1, 19,
    1001, 20, 1, 20,
    1005, 20, 2,
    4, 20,
    99,
    0, 0, // cells 19 (n) and 20 (counter)
  }
}
```

listing 853: go, the countdown program as data, the cell map in its comment (source: dsa/samples-go/ch27/jumps.go, lines 3-30)

```
      if (op == 1 || op == 2 || op == 7 || op == 8) {
        long a = operand(ip, 0), b = operand(ip, 1);
        long r = 0;
        if (op == 1)
          r = a + b;
        else if (op == 2)
          r = a * b;
        else if (op == 7)
          r = a < b ? 1 : 0;
        else
          r = a == b ? 1 : 0;
        mem[(int) mem[ip + 3]] = r; // writes are always positional
        ip += 4;
      } else if (op == 5) {
        if (operand(ip, 0) ≠ 0)
          ip = (int) operand(ip, 1);
        else
          ip += 3;
      } else if (op == 6) {
        if (operand(ip, 0) == 0)
          ip = (int) operand(ip, 1);
        else
          ip += 3;
      } else if (op == 3) {
        if (inPos ≥ inN) {
          error = true;
          return;
        }
        mem[(int) mem[ip + 1]] = input[inPos++];
        ip += 2;
      } else if (op == 4) {
        if (outN < 4)
          output[outN++] = operand(ip, 0);
        ip += 2;
      } else {
        error = true;
      }
    }
  }
```

listing 854: java, cases 5 through 8, the pointer moves or advances, never both (source: dsa/samples-java/src/Ch27/Jumps.java, lines 36-74)

```
                case 5:
                    if (Read(1) != 0)
                        ip = (int)Read(2);
                    else
                        ip += 3;
                    break;
                case 6:
                    if (Read(1) == 0)
                        ip = (int)Read(2);
                    else
                        ip += 3;
                    break;
                case 7:
                    memory[memory[ip + 3]] = Read(1) < Read(2) ? 1 : 0;
                    ip += 4;
                    break;
                case 8:
                    memory[memory[ip + 3]] = Read(1) == Read(2) ? 1 : 0;
                    ip += 4;
                    break;
```

listing 855: c#, cases 5 through 8, the pointer moves or advances, never both (source: dsa/samples/src/Ch27/Machine.cs, lines 49-68)

```
export function runJumps(program, inputs = []) {
  const mem = [...program];
  const outputs = [];
  let ip = 0;
  let inPos = 0;
  const read = (i, mode) => (mode === 1 ? mem[i] : mem[mem[i]]);
  for (;;) {
    const { opcode, modes } = decodeWord(mem[ip]);
    if (opcode === 99) return { memory: mem, outputs };
    if (opcode === 5) {
      const cond = read(ip + 1, modes[0]);
      ip = cond !== 0 ? read(ip + 2, modes[1]) : ip + 3;
    } else if (opcode === 6) {
      const cond = read(ip + 1, modes[0]);
      ip = cond === 0 ? read(ip + 2, modes[1]) : ip + 3;
    } else if (opcode === 7 || opcode === 8) {
      const a = read(ip + 1, modes[0]);
      const b = read(ip + 2, modes[1]);
      mem[mem[ip + 3]] = opcode === 7 ? (a < b ? 1 : 0) : (a === b ? 1 : 0);
      ip += 4;
    } else if (opcode === 1 || opcode === 2) {
      const a = read(ip + 1, modes[0]);
      const b = read(ip + 2, modes[1]);
      mem[mem[ip + 3]] = opcode === 1 ? a + b : a * b;
      ip += 4;
    } else if (opcode === 3) {
      mem[mem[ip + 1]] = inputs[inPos++];
      ip += 2;
    } else if (opcode === 4) {
      outputs.push(read(ip + 1, modes[0]));
      ip += 2;
    } else {
      throw new Error(`unsupported opcode ${opcode} at ${ip}`);
    }
  }
}
```

listing 856: javascript, both jump kinds read condition and target under modes (source: dsa/samples-js/src/ch27-jumps.mjs, lines 8-43)

```
# data slots: 22 = n, 23 = sum, 24 = -1, 25 = flag, 26 = zero, 27 = loop
# start (address 2). jump targets are read position-mode, so the target
# address itself lives in a cell
SUM_LOOP = [
    3,
    22,  # n = input
    1,
    23,
    22,
    23,  # sum += n
    1,
    22,
    24,
    22,  # n += -1
    7,
    26,
    22,
    25,  # flag = (0 < n)
    5,
    25,
    27,  # if flag: jump through slot 27 back to the add
    4,
    23,  # output the sum
    99,
    0,
    0,  # 20, 21: padding
    0,
    0,
    -1,
    0,
    0,
    2,  # 22..27: data, slot 27 holds the loop start
]
```

listing 857: python, the loop with its jump target read through slot 27 (source: dsa/samples-py/src/Ch27/jumps.py, lines 55-87)

```
    T.eq(outputs, {1})
    _, outputs = run(below_eight, 8)
    T.eq(outputs, {0})
    _, outputs = run(below_eight, 9)
    T.eq(outputs, {0})
  end},

  {name = "the countdown loop: while n ~= 0 do n--, counter++ end", fn = function()
    -- input n at cell 20, counter at cell 21
    local program = {
      3, 20,           --  0: read n
      1006, 20, 16,    --  2: if n == 0 jump to the output (target immediate)
      101, -1, 20, 20, --  5: n += -1 (immediate)
      101, 1, 21, 21,  --  9: counter += 1
      1005, 20, 2,     -- 13: if n ~= 0 jump back to the test, the loop
      4, 21,           -- 16: write the counter
      99,              -- 18: halt
```

listing 858: lua, the countdown table, 15 hand-traced steps at n equal to 3 (source: dsa/samples-lua/ch27_jumps.lua, lines 62-78)

The countdown shape is shared by c#, go, and lua, identical cell layouts with the two data cells past the code, and the answers pin across the family: input 3 outputs 3, input 0 jumps straight out with 0, input 7 outputs 7, lua

also pinning the full final memory and the hand-traced 15 instructions. The c and java suites sum 1 to n forward instead, registers at cells 28 through 31 past the code, backwards jumps coded as 1006, flag positional and target immediate, java pinning 1, 15, and 55 for n = 1, 5, and 10 and its registers after the run. Python is the deliberate outlier: its loop reads the jump target positionally through slot 27, so the loop-start address is itself data in a cell, and that difference is stated in its comment. Go adds the comparison programs, equals-7 and below-10, as two more data tables, java pinning its comparisons straight as data too, 1107 storing 1 and 1108 storing 0, beside a taken and an untaken 1105.

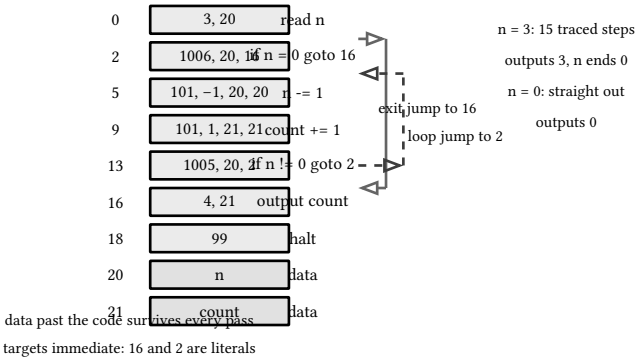


figure 219: the countdown loop as code cells and data cells, the two jumps drawn as arcs, forward exit and backward loop

27.4. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's three sample files, test scripts included where the language embeds them:

language	build sloc	container dependency	boundary note
c	302	static arrays	error flags over exceptions, sum program with registers past the code
c#	72	one file, closures for modes	long memory, step counter, the 101, 9, 104 triplet pinned
go	131	machine struct, int64 slices	decoder exposed as a bitmask, countdown as returned data
java	309	jdk 27 stdlib	halted and error flags, input exhaustion faults, the sum program with registers past the code and its 1006 back-jump
javascript	97	array spread copies	modes as an array, jump targets under full modes
python	230	lists, slices per instruction	jump target read positionally through a cell, the stated outlier
lua	194	tables, 1-based cells	steps counted and hand traced, errors carry level 2

sources: learn.microsoft.com for pattern-switch dispatch, go.dev for multiple return values and `fmt.Errorf`, developer.mozilla.org for spread copies, docs.python.org for slice assignment, lua.org for integer floor division and modulo, accessed 2026-09-14. Sample behavior verified by the seven suite gates scoped to chapter 27: c 3 files and 49 checks, c# 7 tests, go 14 tests, java 3 files and 49 checks, javascript 12 tests, python 3 files and 28 asserts, lua 11 checks, zero skipped.

28. advanced number theory

Chapter 16 built the first toolkit, gcd and the sieve, trial division, modpow, modinv, and the chinese remainder fold, and this chapter is the second course built on top of it, book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) never repeated below. Eight tools make the rung. The linear sieve computes, not just crosses, filling least prime factors plus the phi and mu tables in one pass. Deterministic miller-rabin decides primality for every 64-bit integer and pollard rho factors what it calls composite. Divisor count and divisor sum arrive by accumulation, carrying the phi identities and pillai's gcd-sum. Baby-step giant-step takes discrete logs and finds smallest primitive roots. Garner's algorithm rebuilds an integer from its residues the constructive way, beside factorials mod p and the legendre exponent. Linear diophantine equations solve exactly with no rounding. Gray code and balanced ternary are the two odd numerations worth owning. Continued fractions, the stern-brocot tree, and the farey sequence close the chapter as one structure seen three ways. Every fixture pins the same digits in all seven languages.

28.1. the linear sieve and multiplicative functions

The eratosthenes sieve of chapter 16 crosses a composite once per prime that divides it, which costs O(n log log n) and leaves nothing behind but a boolean. The linear sieve trades the boolean for the least prime factor and gets two things for the trade: a factorization machine, since peeling lp values factors any n in the table, and a linear pass, since every composite is now crossed exactly once, by the product i times lp[i]. The inner loop walks the prime list and stops the moment p would exceed lp[i], because a larger p would cross i times p with a prime that is not its least one. The same loop extends to multiplicative functions for free, and the fork is the whole teaching. When p equals lp[i], the prime is already inside i, so phi of i times p scales by p and mu collapses to 0 on the squared factor. When p is new to i, phi scales by p minus 1 and mu flips sign.

The dry run: the fixture is the C# Build(1000) tables, the prime count, the phi and mu heads, and the lp anchors all asserted by the C# suite, the other six pinning the same rung.

1. i = 2 has no least factor, joins the prime list, and crosses its first composite as 2 × 2 = 4 with lp = 2.
2. i = 3 joins the list and crosses twice: 6 = 3 × 2 gets phi = 2 × 1 = 2 and mu = -(-1) = 1 since 2 is new to 3, then 9 = 3 × 3 gets phi = 2 × 3 = 6 and mu = 0 since 3 already sits inside.
3. i = 5 crosses 10 with 2 new, phi = 4 × 1 = 4 and mu = 1, then 25 with 5 inside, phi = 4 × 5 = 20 and mu = 0.
4. i = 6 meets p = 2 equal to lp[6], so the fork scales phi[12] = 2 × 2 = 4, zeroes mu[12], and the break ends the pass: 12 is never crossed again.
5. The heads land as asserted, phi over 1 to 12 reading 1, 1, 2, 2, 4, 2, 6, 4, 6, 4, 10, 4 and mu zeroing at the squares 4, 8, 9, 12, 16.
6. The pass to 1000 closes at 168 primes with the 168th at 997, and phi(1000) = 400.

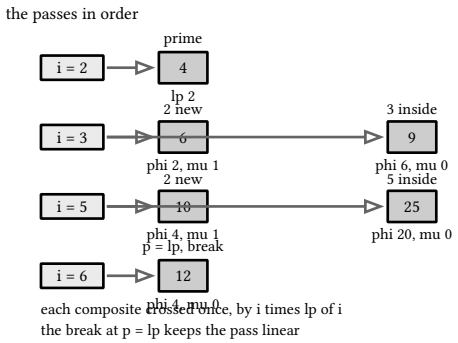


figure 220: the early passes as a sequence, each i crossing its composites left to right, the fork results written inside every target

The 400 closes the table, and the listings below sieve it in seven languages.

```
static void build(void) {
  phi[1] = 1;
  mu[1] = 1;
  for (int i = 2; i ≤ N; i++) {
    if (lp[i] == 0) {
      lp[i] = i;
      primes[pc++] = i;
      phi[i] = i - 1;
      mu[i] = -1;
    }
    for (int j = 0; j < pc; j++) {
      int p = primes[j];
      if (p > lp[i] || p * i > N)
        break;
      lp[p * i] = p;
      if (p == lp[i]) {
        phi[p * i] = phi[i] * p;
        mu[p * i] = 0;
      } else {
        phi[p * i] = phi[i] * (p - 1);
        mu[p * i] = -mu[i];
      }
    }
  }
}
```

listing 859: c, one pass, every composite crossed once by i times lp of i, the phi and mu fork inside (source: dsa/samples-c/src/Ch28/linsieve.c, lines 26-50)

```
func LinSieve(n int) (lp, primes, phi []int32, mu []int8) {
  lp = make([]int32, n+1)
  phi = make([]int32, n+1)
  mu = make([]int8, n+1)
  phi[1], mu[1] = 1, 1 // n = 1 convention
  for i := int32(2); i <= int32(n); i++ {
    if lp[i] == 0 { // i is prime
      lp[i] = i
      primes = append(primes, i)
      phi[i] = i - 1
      mu[i] = -1
    }
    for _, p := range primes {
      ip := i * p
      if ip > int32(n) {
        break
      }
      lp[ip] = p
      if p == lp[i] { // p divides i: the fork closes here
        phi[ip] = phi[i] * p
        mu[ip] = 0
        break
      }
      // p is a new prime factor
      phi[ip] = phi[i] * (p - 1)
      mu[ip] = -mu[i]
    }
  }
  return lp, primes, phi, mu
}
```

listing 860: go, the fork closes the inner loop itself, the hinge that keeps the pass linear (source: dsa/samples-go/ch28/linsieve.go, lines 14-43)

```
  static void build() {
    phi[1] = 1;
    mu[1] = 1;
    for (int i = 2; i ≤ N; i++) {
      if (lp[i] == 0) {
        lp[i] = i;
        primes[pc++] = i;
        phi[i] = i - 1;
        mu[i] = -1;
      }
      for (int j = 0; j < pc; j++) {
        int p = primes[j];
        if (p > lp[i] || p * i > N)
          break;
        lp[p * i] = p;
        if (p == lp[i]) {
          phi[p * i] = phi[i] * p;
          mu[p * i] = 0;
        } else {
          phi[p * i] = phi[i] * (p - 1);
          mu[p * i] = -mu[i];
        }
      }
    }
  }
```

listing 861: java, static lp, phi, mu, and prime arrays, the fork inside, the break tested at the top of the inner loop (source: dsa/samples-java/src/Ch28/Linsieve.java, lines 14-38)

```
    public static (int[] Lp, int[] Primes, int[] Phi, int[] Mu) Build(int n)
    {
        var lp = new int[n + 1];
        var phi = new int[n + 1];
        var mu = new int[n + 1];
        var primes = new List<int>();
        phi[1] = 1;
        mu[1] = 1;
        for (int i = 2; i ≤ n; i++)
        {
            if (lp[i] == 0)
            {
                lp[i] = i;
                phi[i] = i - 1;
                mu[i] = -1;
                primes.Add(i);
            }
            foreach (int p in primes)
            {
                long ip = (long)i * p;
                if (ip > n || p > lp[i])
                    break; // the least prime of i*p would not be p
                lp[(int)ip] = p;
                if (p == lp[i])
                {
                    // p divides i: one more power of the same prime
                    phi[(int)ip] = phi[i] * p;
                    mu[(int)ip] = 0;
                }
                else
                {
                    phi[(int)ip] = phi[i] * (p - 1);
                    mu[(int)ip] = -mu[i];
                }
            }
        }
        return (lp, primes.ToArray(), phi, mu);
    }
```

listing 862: c#, the same build, the break spelled out in a comment (source: dsa/samples/src/Ch28/LinSieve.cs, lines 14-51)

```
export function linearSieve(n) {
  const lp = new Array(n + 1).fill(0);
  const primes = [];
  const phi = new Array(n + 1).fill(0);
  const mu = new Array(n + 1).fill(0);
  phi[1] = 1;
  mu[1] = 1;
  for (let i = 2; i ≤ n; i++) {
    if (lp[i] === 0) {
      lp[i] = i;
      primes.push(i);
      phi[i] = i - 1;
      mu[i] = -1;
    }
    for (const p of primes) {
      if (p > lp[i] || i * p > n) break;
      lp[i * p] = p;
      if (p === lp[i]) {
        // p divides i: the new prime is already in i, phi scales by p
        // and mu collapses on the square
        phi[i * p] = phi[i] * p;
        mu[i * p] = 0;
      } else {
        phi[i * p] = phi[i] * (p - 1);
        mu[i * p] = -mu[i];
      }
    }
  }
  return { lp, primes, phi, mu };
}
```

listing 863: javascript, four 0-based arrays out, plain Number under 1000 (source: dsa/samples-js/src/ch28-linsieve.mjs, lines 8-37)

```python
def linear_sieve(limit):
    # lp[i] = least prime factor (lp[1] = 0 by convention); every
    # composite is crossed exactly once, by i * lp[i]
    lp = [0] * (limit + 1)
    primes = []
    phi = [0] * (limit + 1)
    mu = [0] * (limit + 1)
    phi[1] = 1
    mu[1] = 1
    for i in range(2, limit + 1):
        if lp[i] == 0:
            lp[i] = i
            primes.append(i)
            phi[i] = i - 1
            mu[i] = -1
        for p in primes:
            ip = i * p
            if ip > limit:
                break
            lp[ip] = p
            if p == lp[i]:
                # p divides i: the chain gains no new prime factor
                phi[ip] = phi[i] * p
                mu[ip] = 0
                break
            # p is new to i: both functions lift multiplicatively
            phi[ip] = phi[i] * (p - 1)
            mu[ip] = -mu[i]
    return lp, primes, phi, mu
```

listing 864: python, lp, primes, phi, mu from one loop, the break carried in the branch (source: dsa/samples-py/src/Ch28/linsieve.py, lines 16-44)

```
local lp, phi, mu, primes = {}, {}, {}, {}
lp[1], phi[1], mu[1] = 0, 1, 1

for i = 2, N do
  if lp[i] == nil then
    lp[i] = i
    phi[i] = i - 1
    mu[i] = -1
    primes[#primes + 1] = i
  end
  for _, p in ipairs(primes) do
    if p > lp[i] then break end
    local ip = i * p
    if ip > N then break end
    lp[ip] = p
    if p == lp[i] then
      phi[ip] = phi[i] * p
      mu[ip] = 0
      break
    end
    phi[ip] = phi[i] * (p - 1)
    mu[ip] = -mu[i]
  end
end
```

listing 865: lua, tables indexed by the domain value, so lp of 60 reads lp of 60 (source: dsa/samples-lua/ch28_linsieve.lua, lines 10-33)

The languages split on where the loop stops and it does not matter. C, C#, JavaScript, and Java test p against lp[i] at the top of the inner loop, Go, Python, and Lua break inside the p equals lp[i] branch, and both forms leave the inner loop exactly when the prime reaches the least factor. Fixtures pin the rung above chapter 16′s pi(100) = 25: pi(1000) = 168, the prime list opening 2, 3, 5, 7, 11, 13, the 168th prime 997, and least factors lp(60) = 2, lp(97) = 97, lp(997) = 997, lp(998) = 2, lp(1000) = 2. The function tables read phi over 1 to 12 as 1, 1, 2, 2, 4, 2, 6, 4, 6, 4, 10, 4 and mu over 1 to 16 as 1, −1, −1, 0, −1, 1, −1, 0, 0, 1, −1, 0, −1, 1, 1, 0, with phi(1000) = 400. The n = 1 convention holds everywhere, lp(1) = 0 with phi and mu both 1, phi multiplies over the coprime pair as phi(4) times phi(9) equal to phi(36) equal to 12, mu(30) = −1 on three primes and mu(12) = 0 on the square, and the lp chain peels 998 = 2 times 499 with 499 prime. Python cross-checks all three tables the slow way, lp against trial division everywhere, phi against gcd counting to 200, mu against a factored form over the whole table. No finals problem in the corpus needed this sieve, chapter 16′s carried every year, and the mobius table stays a table here, the inversion identity that uses it lands in chapter 35.

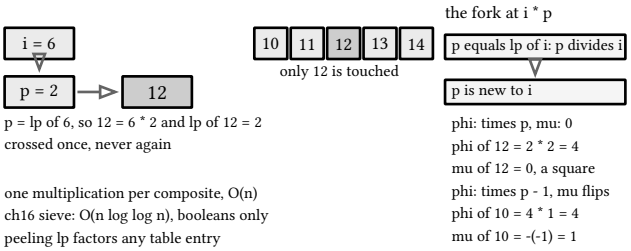


figure 221: composite 12 crossed exactly once, then the fork: the prime already in i scales phi by p and zeroes mu, a new prime scales by p minus 1 and flips mu

28.2. deterministic miller-rabin and pollard rho

Primality becomes a settled question at 64 bits because a fixed witness set settles it. Miller-rabin decomposes n minus 1 as d times 2^s with d odd, then asks each witness a whether a^d or one of its doublings hits 1 or n minus 1. A prime always answers yes, and a composite that answers yes for a particular a is a strong pseudoprime

to base a. The first twelve primes, 2 through 37, are known to expose every composite below 3.317e24, which covers all of uint64, so the test is deterministic with the small divisors pre-checked first. The famous liar at this scale is 3215031751, the product 151 * 751 * 28351, which passes the four bases 2, 3, 5, 7 that settle all of 32 bits and dies on witness 11. Factoring what survives is pollard rho's job: iterate f(x) = x^2 + c mod n with a floyd cycle, one tortoise step against two hare steps, and take the gcd of the gap with n at every step, a nontrivial gcd is a factor. The constants c and every restart draw from the house seeded lcg, x' = 6364136223846793005 * x + 1442695040888963407 mod 2^64 seeded 0x9E3779B97F4A7C15, the same constants the icpc book's chapter 14 pins, so all seven languages walk the identical rho trajectory.

The dry run: the fixtures are the liar 3215031751 asserted both ways and the rho factorizations, carried by the C# suite, the other six walking the same seeded trajectory.

1. The small-divisor gate settles the carmichaels before any witness runs: 5 + 6 + 1 = 12 exposes the 3 in 561, and 41041 opens at 7.
2. The liar decomposes as 3215031751 - 1 = 3215031750 = 2 × 1607515875, so s = 1 and d = 1607515875, one squaring per round.
3. Under the four-base helper {2, 3, 5, 7} every round answers probable prime, the suite asserting true, and under the pinned twelve the first kill lands at witness 11, the suite asserting false, so the public gate reads composite.
4. Rho splits the exposed composites on the shared seed: 561 factors 3, 11, 17 with the product folding back 3 × 11 × 17 = 561, and 41041 factors 7, 11, 13, 41.
5. The semiprime 1000036000099 returns 1000003 and 1000033, the two primes the anchors already declared prime, and the square 2147483647 squared returns its twin halves, rho of 4 returning 2.

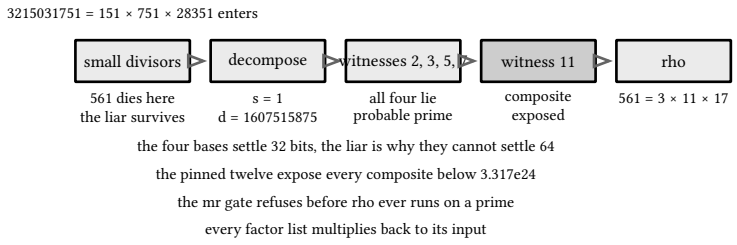


figure 222: the verdict as a pipeline, the liar entering at the left, the pre-check, the s = 1 decomposition, the four lying witnesses, the kill at 11, rho folding the factors back

The two-way liar closes the run, and the listings below carry the gate and rho in seven languages.

```c
// one strong-pseudoprime round: n - 1 = d * 2^s with d odd
static int sprp_round(unsigned long long a, unsigned long long d, int s,
                      unsigned long long n) {
  unsigned long long x = modpow(a, d, n);
  if (x == 1 || x == n - 1)
    return 1;
  for (int r = 1; r < s; r++) {
    x = mulmod(x, x, n);
    if (x == n - 1)
      return 1;
  }
  return 0;
}

static int is_prime_w(const unsigned long long *w, int nw,
                      unsigned long long n) {
  if (n < 2)
    return 0;
  for (int i = 0; i < nw; i++)
    if (n % w[i] == 0)
      return n == w[i];
  unsigned long long d = n - 1;
  int s = 0;
  while (d % 2 == 0) {
    d /= 2;
    s++;
  }
  for (int i = 0; i < nw; i++) {
    unsigned long long a = w[i] % n;
    if (a == 0)
      continue;
    if (!sprp_round(a, d, s, n))
      return 0;
  }
  return 1;
}
```

listing 866: c, one strong-pseudoprime round, then the witness loop over any base set (source: dsa/samples-c/src/Ch28/primality.c, lines 64-99)

```
func IsPrimeWitnesses(n uint64, witnesses []uint64) bool {
  if n < 2 {
    return false
  }
  for _, p := range rhoStrip {
    if n%p == 0 {
      return n == p
    }
    if p*p > n {
      return true
    }
  }
  d, s := n-1, 0
  for d%2 == 0 {
    d /= 2
    s++
  }
  for _, w := range witnesses {
    a := w % n
    if a == 0 {
      continue
    }
    x := ch16.ModPow(a, d, n)
    if x == 1 || x == n-1 {
      continue
    }
    composite := true
    for i := 0; i < s-1; i++ {
      x = ch16.MulMod(x, x, n)
      if x == n-1 {
        composite = false
        break
      }
    }
    if composite {
      return false
    }
  }
  return true
}
```

listing 867: go, small divisors pre-checked, ch16 mulmod and modpow underneath (source: dsa/samples-go/ch28/primality.go, lines 25-64)

```
  // one rho trip: tortoise one step, hare two steps, gcd per step. the
  // lcg wraps negative in java's signed long, so the modulus draws go
  // through Long.remainderUnsigned to match the C unsigned semantics
  static long rho(long n) {
    if (n % 2 == 0)
      return 2;
    if (n % 3 == 0)
      return 3;
    for (int attempt = 0; attempt < 96; attempt++) {
      long c = Long.remainderUnsigned(rngNext(), n - 1) + 1;
      long x = Long.remainderUnsigned(rngNext(), n);
      long y = x;
      for (int step = 0; step < 400000; step++) {
        x = (mulmod(x, x, n) + c) % n;
        y = (mulmod(y, y, n) + c) % n;
        y = (mulmod(y, y, n) + c) % n;
        long diff = x > y ? x - y : y - x;
        if (diff == 0)
          break;
        long d = gcd64(diff, n);
        if (d > 1)
          return d;
      }
    }
    return n;
  }
```

listing 868: java, the seeded lcg draws through `Long.remainderUnsigned`, floyd cycle rho, the add-and-double mulmod above (source: dsa/samples-java/src/Ch28/Primality.java, lines 104-129)

```
    public static bool IsPrimeWith(long n, long[] witnesses)
    {
        if (n < 2)
            return false;
        foreach (long p in SmallPrimes)
        {
            if (n == p)
                return true;
            if (n % p == 0)
                return false;
        }
        long d = n - 1;
        int s = 0;
        while ((d & 1) == 0)
        {
            d >>= 1;
            s++;
        }
        foreach (long w in witnesses)
        {
            long rem = w % n;
            if (rem == 0)
                continue;
            ulong a = PowMod((ulong)rem, (ulong)d, (ulong)n);
            if (a == 1 || a == (ulong)(n - 1))
                continue;
            bool composite = true;
            for (int r = 1; r < s; r++)
            {
                a = MulMod(a, a, (ulong)n);
                if (a == (ulong)(n - 1))
                {
                    composite = false;
                    break;
                }
            }
            if (composite)
                return false;
        }
        return true;
    }
```

listing 869: c#, IsPrimeWith takes the base set, the pinned twelve ride the public gate (source: dsa/samples/src/Ch28/Primality.cs, lines 25-65)

```
// the shared seeded lcg driving c draws and restarts
function makeLcg() {
  let x = 0x9E3779B97F4A7C15n & MASK64;
  return () => {
    x = (6364136223846793005n * x + 1442695040888963407n) & MASK64;
    return x;
  };
}

// one rho run: a nontrivial factor of n, or n itself when the walk dies
function rhoOnce(n, c) {
  const f = (x) => (x * x + c) % n;
  let x = 2n;
  let y = 2n;
  for (;;) {
    x = f(x);
    y = f(f(y)); // floyd cycle: tortoise one step, hare two
    const d = bigGcd(x > y ? x - y : y - x, n);
    if (d !== 1n) return d;
  }
}

function rhoFactor(n) {
  if ((n & 1n) === 0n) return 2n;
  const lcg = makeLcg();
  for (;;) {
    let c = lcg() % n;
    if (c < 2n || c === n - 2n) c = 1n; // skip degenerate f
    const d = rhoOnce(n, c);
    if (d !== n) return d; // else restart on the next c
  }
}
```

listing 870: javascript, the seeded lcg, then rho with the floyd cycle and a restart on a collapsed gcd (source: dsa/samples-js/src/ch28-primality.mjs, lines 60-91)

```
class RhoLcg:
    # the shared generator: x' = (6364136223846793005 * x
    # + 1442695040888963407) mod 2^64, seed 0x9E3779B97F4A7C15
    def __init__(self):
        self.s = 0x9E3779B97F4A7C15

    def draw(self):
        self.s = (6364136223846793005 * self.s + 1442695040888963407) & MASK64
        return self.s


rng = RhoLcg()


def rho(m):
    # f(x) = x^2 + c mod m, floyd cycle, gcd(|x - y|, m) per step
    if m % 2 == 0:
        return 2
    while True:
        c = rng.draw() % m or 1  # c and restarts from the seeded stream
        x = rng.draw() % m
        y = x
        d = 1
        while d == 1:
            x = (x * x + c) % m
            y = (y * y + c) % m
            y = (y * y + c) % m
            d = math.gcd(abs(x - y), m)
        if d != m:
            return d
```

listing 871: python, the rho driver, c and starts drawn from the shared stream (source: dsa/samples-py/src/Ch28/primality.py, lines 49-78)

```lua
local st = 0x9E3779B97F4A7C15 -- the shared seed, wraps like mod 2^64
local function draw()
  st = st * 6364136223846793005 + 1442695040888963407
  return st
end

local function gcd(a, b)
  while b ~= 0 do a, b = b, a % b end
  return a
end

-- one rho attempt: f(x) = x^2 + c mod n, floyd cycle, gcd(|x-y|, n)
local function rho_split(n)
  if n % 2 == 0 then return 2 end
  for _ = 1, 64 do
    local c = draw() % n
    local x = draw() % n
    local y = x
    local d = 1
    while d == 1 do
      x = (mulmod(x, x, n) + c) % n
      y = (mulmod(y, y, n) + c) % n
      y = (mulmod(y, y, n) + c) % n
      d = gcd(math.abs(x - y), n)
    end
    if d ~= n then return d end
  end
  error("rho failed to split " .. n)
end
```

listing 872: lua, the shared seed, then rho splitting with bounded restarts (source: dsa/samples-lua/ch28_primality.lua, lines 61-89)

The integer roads split exactly where chapter 16 left them. Rho on these moduli squares residues at 4.6e18, past 2^63, so C multiplies by add-and-double with a remainder every step because the ch16 link ruling bans 128-bit help, Lua does the same with its silently wrapping int64, Java walks C's add-and-double road too and pushes its rho modulus draws through `Long.remainderUnsigned` because the seeded lcg wraps negative in a signed long, C# rides every product through System.UInt128, Go goes through ch16's Mul64 and Div64, and JavaScript puts the whole computation on BigInt. Fixtures agree digit for digit. The miller-rabin anchors: 2, 3, and 97 prime, 4 not, 1000003 and 1000033 prime, the mersenne prime 2^61 - 1 prime, the carmichael numbers 561 and 41041 exposed, and the liar 3215031751 asserted both ways, true under the four-base helper and false under the pinned twelve. The factorizations: 561 = 3, 11, 17, 41041 = 7, 11, 13, 41, 123456789012345678 = 2, 3, 3, 3, 21491747, 106377431, the semiprime 1000036000099 splitting into 1000003 and 1000033, and 2147483647 squared giving the twin factors again, the perfect-square edge where rho must not loop on equal halves. Edge behavior: 1 is not prime, rho of 4 returns 2, factoring a prime refuses through the miller-rabin gate in every refusal channel, and every factor list multiplies back to its input. Two finals problems live one rung below and one rung above this section: icpc 2025 problem F (book 10, chapter 13) takes the largest prime up to n/3 by trial division, and icpc 2025 problem B (book 10, chapter 13) bipartitions its graph by the parity of the prime factor count, omega read off exactly these factorizations.

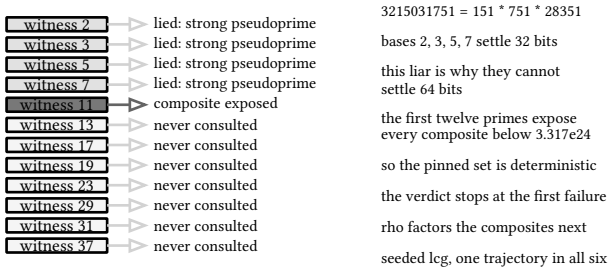


figure 223: the witness ladder: 3215031751 slips past bases 2, 3, 5, 7 and dies on witness 11

28.3. divisor counts, divisor sums, and the phi identities

The multiplicative closed forms for d(n) and sigma(n) need prime factorizations. The accumulation sieve needs nothing but a loop nest: for every d from 1 to n, walk the multiples of d and add 1 to dcnt of each multiple and d itself to sigma of it. Every divisor of m contributes exactly once to each table, the work is the harmonic sum O(n log n), and the tables come out for every m at once. It is the deliberately simple form, the closed forms are the fast road when one n is wanted, and this is the road that fills a table. Three identities ride the tables. The divisor sum of phi closes on n itself, the sum of phi over every divisor of n equals n. The summatory totient sums phi of 1 through n. And pillai's identity prices the gcd-sum, the sum of gcd(k, n) over k from 1 to n equals the sum of d times phi(n/d) over the divisors d of n.

The dry run: the fixtures are the 360 anchors with the smaller pair d(36) = 9 and sigma(36) = 91 asserted beside them by the C# suite, the pillai value brute-counted in-suite.

1. The pass at d = 1 walks every cell, so both tables open at 1 everywhere, and cell 36 starts collecting.
2. Its divisor rail arrives in d order, 1, 2, 3, 4, 6, 9, 12, 18, 36, each arrival adding 1 to dcnt and d itself to sigma.
3. Nine arrivals close dcnt[36] at 9 while sigma climbs the running sums below, ending 55 + 36 = 91.
4. The same rail at 360 collects the pinned 24 contributions and closes d(360) = 24 with sigma(360) = 1170.
5. The phi identity reads the rail: phi summed over those divisors returns exactly 360, and the summatory totient to 100 lands 3044.
6. Pillai prices the gcd-sum of 360 at 3780 both ways, the fold over d times phi(360/d) and the brute gcd count agreeing.

d arrives	dcnt[36]	sigma added	sigma[36] now
1	1	1	1
2	2	2	3
3	3	3	6
4	4	4	10
6	5	6	16
9	6	9	25
12	7	12	37
18	8	18	55
36	9	36	91

The 91 closes the accumulation at the pinned 9 divisors, and the listings below sieve both tables in seven languages.

```
static void build(void) {
  for (int d = 1; d ≤ N; d++)
    for (int m = d; m ≤ N; m += d) {
      dcnt[m]++;
      sig[m] += d;
    }
  // phi by the same linear pass as the linsieve listing
  phi[1] = 1;
  int pc = 0;
  static int primes[168];
  static int lp[N + 1];
  for (int i = 2; i ≤ N; i++) {
    if (lp[i] == 0) {
      lp[i] = i;
      primes[pc++] = i;
      phi[i] = i - 1;
    }
    for (int j = 0; j < pc; j++) {
      int p = primes[j];
      if (p > lp[i] || p * i > N)
        break;
      lp[p * i] = p;
      phi[p * i] = (p == lp[i]) ? phi[i] * p : phi[i] * (p - 1);
    }
  }
```

listing 873: c, the accumulation sieve, then phi by the same linear pass as 28.1 (source: dsa/samples-c/src/Ch28/totdiv.c, lines 23-47)

```
// DivisorSieve fills d(n) and sigma(n) for every n ≤ cap by divisor
// accumulation: every d adds itself to sigma[k*d] and 1 to dcnt[k*d].
// One multiplication per (k, d) pair, O(n log n) total, the simple
// teaching form against the multiplicative closed form.
func DivisorSieve(cap int) (dcnt, sigma []int64) {
  dcnt = make([]int64, cap+1)
  sigma = make([]int64, cap+1)
  for d := int64(1); d <= int64(cap); d++ {
    for k := d; k <= int64(cap); k += d {
      dcnt[k]++
      sigma[k] += d
    }
  }
  return dcnt, sigma
}

// Divisors lists every divisor of n ascending by trial division, the
// ground truth the sieve tables are read against.
func Divisors(n int64) []int64 {
  var small, large []int64
  for d := int64(1); d*d <= n; d++ {
    if n%d != 0 {
      continue
    }
    small = append(small, d)
    if d != n/d {
      large = append(large, n/d) // descending while d grows
    }
  }
  for i, j := 0, len(large)-1; i < j; i, j = i+1, j-1 {
    large[i], large[j] = large[j], large[i]
  }
  return append(small, large...)
}

// PillaiGCDSum evaluates sum_{k=1..n} gcd(k, n) by the identity
// sum_{d|n} d*phi(n/d), using the 28.1 phi table.
func PillaiGCDSum(n int64) int64 {
  _, _, phi, _ := LinSieve(int(n))
  sum := int64(0)
  for _, d := range Divisors(n) {
    sum += d * int64(phi[n/d])
  }
  return sum
}
```

listing 874: go, the sieve, the ground-truth divisor walk, pillai over the 28.1 phi table (source: dsa/samples-go/ch28/totdiv.go, lines 3-47)

```
  static void build() {
    for (int d = 1; d ≤ N; d++)
      for (int m = d; m ≤ N; m += d) {
        dcnt[m]++;
        sig[m] += d;
      }
    // phi by the same linear pass as the linsieve listing
    phi[1] = 1;
    int pc = 0;
    int[] primes = new int[168];
    int[] lp = new int[N + 1];
    for (int i = 2; i ≤ N; i++) {
      if (lp[i] == 0) {
        lp[i] = i;
        primes[pc++] = i;
        phi[i] = i - 1;
      }
      for (int j = 0; j < pc; j++) {
        int p = primes[j];
        if (p > lp[i] || p * i > N)
          break;
        lp[p * i] = p;
        phi[p * i] = (p == lp[i]) ? phi[i] * p : phi[i] * (p - 1);
      }
    }
  }
```

listing 875: java, the accumulation loops, then phi by the same linear pass inlined (source: dsa/samples-java/src/Ch28/Totdiv.java, lines 11-36)

```
    /// <summary>For every d add 1 to dcnt[d] and d to sigma[d]: O(n log n) for both
    /// tables. Every n &gt;= 1 starts with the divisor 1 pre-seeded.</summary>
    public static (int[] D, long[] Sigma) Build(int n)
    {
        var d = new int[n + 1];
        var sigma = new long[n + 1];
        for (int i = 1; i ≤ n; i++)
        {
            d[i] = 1;
            sigma[i] = 1;
        }
        for (int i = 2; i ≤ n; i++)
            for (int j = i; j ≤ n; j += i)
            {
                d[j]++;
                sigma[j] += i;
            }
        return (d, sigma);
    }

    /// <summary>Sorted list of all divisors of n by trial accumulation.</summary>
    public static List<long> Divisors(long n)
    {
        var small = new List<long>();
        var large = new List<long>();
        for (long d = 1; d * d ≤ n; d++)
            if (n % d == 0)
            {
                small.Add(d);
                if (d ≠ n / d)
                    large.Add(n / d);
            }
        large.Reverse();
        small.AddRange(large);
        return small;
    }

    /// <summary>Pillai's identity: sum_{k=1..n} gcd(k, n) = sum_{d|n} d * phi(n/d).</summary>
    public static long PillaiGcdSum(long n, int[] phi)
    {
        long sum = 0;
        foreach (long d in Divisors(n))
            sum += d * phi[n / d];
        return sum;
    }
```

listing 876: c#, both tables, the divisor list, and the pillai fold taking phi as a parameter (source: dsa/samples/src/Ch28/TotDiv.cs, lines 9-53)

```
export function divisorTables(n) {
  const dcnt = new Array(n + 1).fill(0);
  const sigma = new Array(n + 1).fill(0);
  for (let d = 1; d ≤ n; d++) {
    for (let m = d; m ≤ n; m += d) {
      sigma[m] += d;
      dcnt[m] += 1;
    }
  }
  return { dcnt, sigma };
}

export function divisorsOf(n) {
  const out = [];
  for (let d = 1; d * d ≤ n; d++) {
    if (n % d === 0) {
      out.push(d);
      if (d =/= n / d) out.push(n / d);
    }
  }
  return out.sort((a, b) => a - b);
}

function gcd(a, b) {
  while (b) [a, b] = [b, a % b];
  return a;
}

// pillai: sum_{k=1..n} gcd(k, n) = sum_{d|n} d * phi(n/d)
export function gcdSum(n) {
  const { phi } = linearSieve(n);
  let s = 0;
  for (const d of divisorsOf(n)) s += d * phi[n / d];
  return s;
}

// brute cross-check: add gcd(k, n) over k = 1..n directly
export function gcdSumBrute(n) {
  let s = 0;
  for (let k = 1; k ≤ n; k++) s += gcd(k, n);
  return s;
}
```

listing 877: javascript, the tables, the fold, and the brute gcd twin beside it (source: dsa/samples-js/src/ch28-totdiv.mjs, lines 9-50)

```
N = 1000
dcnt = [0] * (N + 1)
dsig = [0] * (N + 1)
for d in range(1, N + 1):  # every d donates itself and one count
    for m in range(d, N + 1, d):
        dsig[m] += d
        dcnt[m] += 1

phi = list(range(N + 1))
for p in range(2, N + 1):
    if phi[p] == p:  # p is prime: sieve the totient below it
        for m in range(p, N + 1, p):
            phi[m] -= phi[m] // p
```

listing 878: python, the accumulation loops with phi sieved in place by subtraction (source: dsa/samples-py/src/Ch28/totdiv.py, lines 16-28)

```
local dcnt, sigma, phi = {}, {}, {}
for i = 1, N do dcnt[i], sigma[i], phi[i] = 0, 0, 0 end
for d = 1, N do
  for m = d, N, d do
    dcnt[m] = dcnt[m] + 1
    sigma[m] = sigma[m] + d
  end
end

-- phi by the linear sieve of this chapter, kept self-contained
local lp, primes = {}, {}
lp[1] = 0
phi[1] = 1
for i = 2, N do
  if lp[i] == nil then
    lp[i] = i
    phi[i] = i - 1
    primes[#primes + 1] = i
  end
  for _, p in ipairs(primes) do
    if p > lp[i] then break end
    local ip = i * p
    if ip > N then break end
    lp[ip] = p
    if p == lp[i] then phi[ip] = phi[i] * p; break end
    phi[ip] = phi[i] * (p - 1)
  end
end
```

listing 879: lua, both tables by accumulation, phi kept self-contained by the linear sieve (source: dsa/samples-lua/ch28_totdiv.lua, lines 8-35)

Python is the one that builds phi differently, and the difference is worth the read: it initializes phi to the identity and subtracts phi of m over p for every prime multiple, the eratosthenes-shaped totient sieve, while the other six reuse the 28.1 linear pass. The anchors: d(360) = 24 and sigma(360) = 1170 with the full 24-entry divisor list pinned 1, 2, 3, 4, 5, 6, 8, 9, 10, 12, 15, 18, 20, 24, 30, 36, 40, 45, 60, 72, 90, 120, 180, 360, and the smaller pair d(36) = 9, sigma(36) = 91. The identities: phi summed over the divisors of 360 returns exactly 360, the summatory totient to 100 is 3044, and the pillai fold values the gcd-sum of 360 at 3780, with every suite brute-counting the gcds as the cross-check. The edges: sigma(28) = 56 and sigma(496) = 992, both perfect, a prime owns exactly 1 and itself with d(997) = 2 and sigma(997) = 998, and d(1) = sigma(1) = 1. Python sweeps both tables against trial division to

200. No finals problem demanded these functions, they are the general technique that keeps showing up under counting problems.

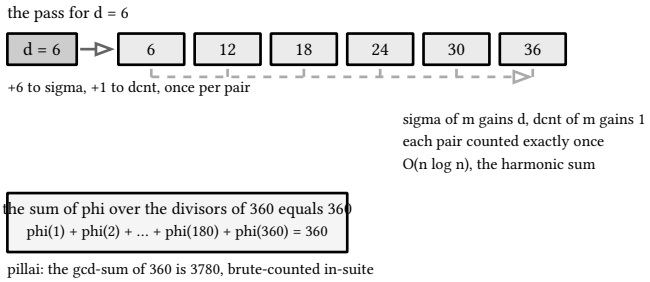


figure 224: the divisor sieve donating 6 to every multiple of 6, beneath the phi identity closing on 360

28.4. discrete logs and primitive roots

Multiplication mod p is a group, and the discrete logarithm asks which power: given a, b, and a prime p, find the smallest x with a^x = b. Baby-step giant-step squares the search away. Build a table of the m baby powers a^0 through a^(m-1) where m is floor of the square root of p, hashed by value to the smallest exponent, then walk giant steps from b multiplying by a^(-m), built by fermat as a^(p-1-m). Any hit gives x = i * m + j, the loop runs at most m + 1 times, and the cost is O(sqrt p) time and space, 31623 table entries at the 1e9+7 anchor. Primitive roots invert the question: g is a primitive root of p when its order is exactly p - 1, and the smallest one is found by factoring p - 1 with the 28.2 machinery, then testing that g^((p-1)/q) differs from 1 for every prime q in that factorization, since the order divides p - 1 and misses it exactly when such a power lands on 1. The order of any a comes off the same walk, shrinking p - 1 by every prime that still yields 1.

The dry run: the fixture is dlog(2, 5, 29) = 22 asserted with its substitution check by the C# suite, the 1e9+7 anchor and the root table pinned beside it.

1. m is the floor of the square root of 29, so the baby table holds j = 0 through 4: the powers 1, 2, 4, 8, 16, one multiplication by 2 each step.
2. The giant factor is a^(-m): 2^5 = 32 = 3 mod 29, and 3 × 10 = 30 = 1, so every giant step multiplies by 10.
3. The walk from b = 5 runs 5 × 10 = 50 = 21, 21 × 10 = 210 = 7, 7 × 10 = 70 = 12, 12 × 10 = 120 = 4.
4. The value 4 already sits in the baby table at j = 2, so x = 4 × 5 + 2 = 22, checked back as 2^22 mod 29 = 5.
5. The same solver at 1e9+7 reads m = 31623 and pins dlog(2, 3) = 385273928, verified by pow.
6. The root side: the smallest root of 29 is 2, the orders of 2 and 3 both read 28, and phi(28) = 12 primitive roots exist.

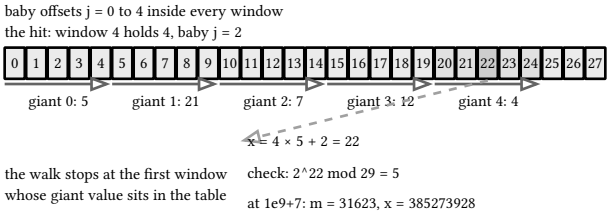


figure 225: the search as a sequence, the exponent line 0 to 27 tiled into giant windows of five, the hit landing in window 4 at offset 2

The 22 closes the meet, and the listings below run both walks in seven languages.

```c
// smallest x ≥ 0 with a^x = b mod p (p prime); -1 refuses
static long long bsgs(unsigned long long a, unsigned long long b,
                      unsigned long long p) {
  if (b == 0)
    return -1; // 0 is never a power mod a prime
  a %= p;
  b %= p;
  unsigned long long m = 0;
  while ((m + 1) * (m + 1) ≤ p)
    m++;
  for (unsigned long long j = 0; j < m; j++) {
    bkey[j] = modpow(a, j, p); // could run-multiply, keep the form simple
    bidx[j] = (int)j;
  }
  qsort(bidx, (size_t)m, sizeof(int), cmp_entry);
  unsigned long long gamma = modpow(a, p - 1 - m, p); // a^(-m) by fermat
  unsigned long long cur = b;
  for (unsigned long long i = 0; i ≤ m; i++) {
    // binary search the smallest exponent j with a^j = cur
    int lo = 0, hi = (int)m - 1, hit = -1;
    while (lo ≤ hi) {
      int mid = (lo + hi) / 2;
      unsigned long long k = bkey[bidx[mid]];
      if (k == cur) {
        hit = mid;
        hi = mid - 1; // keep scanning left for the smallest exponent
      } else if (k < cur)
        lo = mid + 1;
      else
        hi = mid - 1;
    }
    if (hit ≥ 0)
      return (long long)(i * m + (unsigned long long)bidx[hit]);
    cur = cur * gamma % p;
  }
  return -1;
}
```

listing 880: c, the baby table sorted for binary search, giant steps by a^(p-1-m) (source: dsa/samples-c/src/Ch28/dlog.c, lines 46-82)

```
func DiscreteLog(a, b, p uint64) (int64, error) {
  if p < 2 {
    return 0, fmt.Errorf("dlog: modulus %d must be prime ≥ 2", p)
  }
  b %= p
  if b == 0 {
    return 0, fmt.Errorf("dlog: 0 is never a power mod %d", p)
  }
  m := uint64(math.Sqrt(float64(p)))
  for m*m < p {
    m++
  }
  if m*m > p {
    m--
  } // m = floor(sqrt(p))
  baby := make(map[uint64]int64, m)
  cur := uint64(1) % p
  for j := int64(0); j < int64(m); j++ {
    if _, seen := baby[cur]; !seen {
      baby[cur] = j // ascending j keeps the smallest exponent
    }
    cur = ch16.MulMod(cur, a, p)
  }
  invA := ch16.ModPow(a, p-2, p) // p prime: fermat inverse
  step := ch16.ModPow(invA, m, p)
  cur = b
  for i := int64(0); i <= int64(m); i++ {
    if j, ok := baby[cur]; ok {
      return i*int64(m) + j, nil
    }
    cur = ch16.MulMod(cur, step, p)
  }
  return 0, fmt.Errorf("dlog: %d is not a power of %d mod %d", b, a, p)
}
```

listing 881: go, ch16 mulmod and the fermat inverse, errors on refusal (source: dsa/samples-go/ch28/dlog.go, lines 15-48)

```
// smallest x ≥ 0 with a^x = b mod p (p prime); -1 refuses. the baby
// table maps each residue to its smallest exponent, the giant steps
// multiply by a^(-m) so the first hit is the smallest logarithm
static long bsgs(long a, long b, long p) {
  if (b == 0)
    return -1; // 0 is never a power mod a prime
  a %= p;
  b %= p;
  long m = 0;
  while ((m + 1) * (m + 1) ≤ p)
    m++;
  Map<Long, Long> baby = new HashMap<>();
  long cur = 1 % p;
  for (long j = 0; j < m; j++) {
    baby.putIfAbsent(cur, j);
    cur = cur * a % p;
  }
  long gamma = modpow(a, p - 1 - m, p); // a^(-m) by fermat
  cur = b;
  for (long i = 0; i ≤ m; i++) {
    Long j = baby.get(cur);
    if (j ≠ null)
      return i * m + j;
    cur = cur * gamma % p;
  }
  return -1;
}
```

listing 882: java, the HashMap baby table keeping the smallest exponent by `putIfAbsent`, giant factor by fermat, plain long products under 2^63 (source: dsa/samples-java/src/Ch28/Dlog.java, lines 34-60)

```
    /// <summary>Smallest x &gt;= 0 with a^x = b mod p, or throws when b is not a power of a.</summary>
    public static long DiscreteLog(long a, long b, long p)
    {
        if (p ≤ 1)
            throw new ArgumentOutOfRangeException(nameof(p), "modulus must be a prime ≥ 2");
        if (b == 0)
            throw new InvalidOperationException("0 is never a power");
        a %= p;
        b %= p;
        long m = (long)Math.Sqrt(p);
        while (m * m < p)
            m++;
        var baby = new Dictionary<long, long>();
        long cur = 1 % p;
        for (int j = 0; j < m; j++)
        {
            if (!baby.ContainsKey(cur))
                baby[cur] = j; // keep the smallest exponent per value
            cur = cur * a % p;
        }
        long invFactor = NumTheory.ModInv(cur, p); // a^(-m), cur is a^m here
        long gamma = b;
        for (long i = 0; i ≤ m; i++)
        {
            if (baby.TryGetValue(gamma, out long j))
                return i * m + j;
            gamma = gamma * invFactor % p;
        }
        throw new InvalidOperationException($"{b} is not a power of {a} mod {p}");
    }
```

listing 883: c#, a dictionary keyed by value keeps the smallest exponent (source: dsa/samples/src/Ch28/DLog.cs, lines 11-40)

```
// smallest x ≥ 0 with a^x = b (mod p), or null when b is no power of a
export function bsgs(a, b, p) {
  a %= p;
  b %= p;
  if (b === 1n) return 0;
  const m = isqrtBig(p); // baby table of floor(sqrt(p)) entries
  const baby = new Map(); // value -> smallest exponent
  let cur = 1n;
  for (let j = 0n; j < m; j++) {
    if (!baby.has(cur)) baby.set(cur, j);
    cur = (cur * a) % p;
  }
  // giant step multiplies by a^(-m), built as a^(p-1-m) by fermat
  const step = powMod(a, p - 1n - m, p);
  let gamma = b;
  for (let i = 0n; i ≤ m; i++) {
    const j = baby.get(gamma);
    if (j !== undefined) return Number(i * m + j);
    gamma = (gamma * step) % p;
  }
  return null;
}
```

listing 884: javascript, bsgs on BigInt, residues pass 2^53 at 1e9+7 (source: dsa/samples-js/src/ch28-dlog.mjs, lines 26-47)

```python
def bsgs(a, b, p):
    # smallest x ≥ 0 with a^x = b (mod p), or None when b is no power
    if b % p == 0:
        return None  # 0 is never a power
    a %= p
    b %= p
    if b == 1:
        return 0  # a^0 = 1
    m = math.isqrt(p)  # the baby table holds floor(sqrt(p)) entries
    table = {}
    e = 1
    for j in range(m):
        if e not in table:
            table[e] = j  # keep the smallest exponent
        e = e * a % p
    factor = pow(pow(a, p - 2, p), m, p)  # a^(-m)
    gamma = b
    for i in range(m + 1):
        if gamma in table:
            return i * m + table[gamma]
        gamma = gamma * factor % p
    return None
```

listing 885: python, the table as a dict, the giant factor by three-argument pow (source: dsa/samples-py/src/Ch28/dlog.py, lines 16-37)

```lua
-- smallest x ≥ 0 with a^x = b mod prime p, or nil when b is no power
local function dlog(a, b, p)
  if b == 0 then return nil end -- 0 is never a power
  local m = math.floor(math.sqrt(p))
  while (m + 1) * (m + 1) ≤ p do m = m + 1 end
  while m * m > p do m = m - 1 end
  local baby = {}
  local cur = 1 % p
  for j = 0, m - 1 do
    if baby[cur] == nil then baby[cur] = j end -- smallest exponent wins
    cur = cur * a % p
  end
  local gamma = modpow(modpow(a, p - 2, p), m, p) -- a^(-m) mod p
  cur = 1 % p
  for i = 0, m do
    local j = baby[b * cur % p]
    if j ~= nil then return i * m + j end
    cur = cur * gamma % p
  end
  return nil
end
```

listing 886: lua, smallest exponent kept per value, plain products under 2^63 (source: dsa/samples-lua/ch28_dlog.lua, lines 17-37)

The baby table must keep the smallest exponent per value, exponents repeat whenever the order of a is below m, and the suites that skip that check can return a larger x than asked for, java's HashMap holding its table through `putIfAbsent` so the first, smallest exponent survives every repeat. Fixtures pin the 1e9+7 anchor, dlog of 2 to 3 is 385273928, verified back by pow, and the mod 29 triple dlog(2, 5) = 22, dlog(3, 6) = 18, dlog(2, 14) = 13, each verified by substitution. The primitive roots land on the two transform primes of chapter 29, cross-named in every comment: the smallest root of 29 is 2, of 998244353 is 3, of 754974721 is 11, and of 1000000007 is 5. Orders pin at 28 for both 2 and 3 mod 29, and the count of primitive roots mod 29 is phi(28) = 12. Edges: dlog of 2 to 0 refuses

because 0 is never a power, dlog of a to 1 returns 0, and modulus 2 with 1 to 1 returns 0. No finals problem needed a discrete log, this is the general technique.

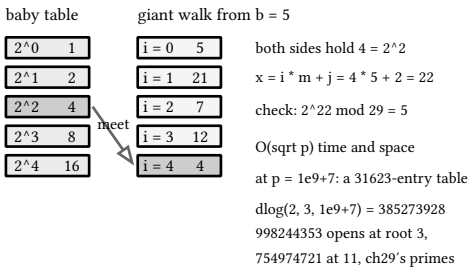


figure 226: the sqrt 29 baby column and the giant walk meeting at 4, the answer 22 = 4 * 5 + 2

28.5. garner's algorithm and factorials mod p

Chapter 16 folds the chinese remainder theorem pairwise, each merge combining two congruences into one over the lcm, and that fold proves existence one step at a time. Garner's algorithm is the constructive rebuild: write the unknown as a mixed-radix number whose digit moduli are the original moduli. Digit x i is the residue r i minus the already-known digits times their moduli, times the inverse of the running product mod m i , and the value folds back from the top as x = ((x k-1) * m k-2 + x k-2) * … . Two properties make it the working form. It never forms the combined modulus as a number, each digit computation stays inside m i , and it accepts any pairwise coprime moduli, primes are not required, which is exactly what the composite moduli below need. The inverse of 998244353 mod 754974721 is 416537774, the pinned INV of chapter 29′s transform fold, cross-named here by the fixture that asserts it. Beside the rebuild sit the factorial facts mod a prime: the table 0 to n multiplies one factor per entry, any n at or past p collapses to 0 exactly when the factor p enters, wilson's pair reads (p-1)! = p - 1 and (p-2)! = 1, and the legendre exponent counts the p-adic valuation of n! as the sum of floor(n / p^k) over k.

The dry run: the fixture is the 2019 K departure time 1736 over the moduli 8, 9, 5, 7, asserted digit by digit by the C# suite, the 12345678987654321 rebuild pinned beside it.

1. Digit x 0 is the residue itself: 0 mod 8.
2. x 1 subtracts the known digits, 8 - 0 = 8, then scales by the inverse of m 0 = 8 mod 9: 8 × 8 = 64 = 1 mod 9, so x 1 = 8 × 8 = 1 mod 9.
3. x 2 subtracts the running value 0 + 1 × 8 = 8, leaving 1 - 8 = −7; m 0 m 1 = 72 = 2 mod 5 with inverse 3 since 2 × 3 = 6 = 1, and −7 × 3 = −21 = 4 mod 5.
4. x 3 subtracts the running value 8 + 4 × 72 = 296, leaving −296; 8 × 9 × 5 = 360 = 3 mod 7 with inverse 5 since 3 × 5 = 15 = 1, and −296 × 5 lands 4 mod 7.
5. The digits read 0, 1, 4, 4, the pin, and the rebuild folds from the top: ((4 × 5 + 4) × 9 + 1) × 8 + 0 = 1736.
6. The big fixture runs the same ladder blind, digits 901234575, 12345678, 0 rebuilding 12345678987654321 exactly, and the factorials ride beside it, 10! mod 11 = 10 with the legendre exponents 97, 498, 505.

i	m i	r i	the subtraction	digit
0	8	0	the residue itself	0
1	9	8	(8 - 0) × 8 mod 9	1
2	5	1	(1 - 8) × 3 mod 5	4
3	7	0	(0 - 296) × 5 mod 7	4

The 1736 closes the rebuild, and the listings below carry the digits in seven languages.

```c
// inverse of a mod m for any coprime m (moduli here include 8 and 9, so
// extended euclid, not fermat)
static unsigned long long modinv(unsigned long long a, unsigned long long m) {
  long long t = 0, newt = 1, r = (long long)m, newr = (long long)(a % m);
  while (newr != 0) {
    long long q = r / newr, tmp;
    tmp = t - q * newt;
    t = newt;
    newt = tmp;
    tmp = r - q * newr;
    r = newr;
    newr = tmp;
  }
  long long inv = t % (long long)m;
  if (inv < 0)
    inv += (long long)m;
  return (unsigned long long)inv;
}

#define GK 4

// digits x_i = (r_i - sum_{j<i} x_j m_j) inv(...) mod m_i, rebuild top down
static unsigned long long garner(const unsigned long long *r,
                                 const unsigned long long *m, int k,
                                 unsigned long long *x) {
  for (int i = 0; i < k; i++) {
    unsigned long long t = r[i] % m[i];
    for (int j = 0; j < i; j++) {
      unsigned long long xj = x[j] % m[i];
      t = (t + m[i] - xj) % m[i];
      t = t * modinv(m[j] % m[i], m[i]) % m[i];
    }
    x[i] = t;
  }
  unsigned long long v = x[k - 1];
  for (int i = k - 2; i >= 0; i--)
    v = v * m[i] + x[i];
  return v;
}
```

listing 887: c, inverse by extended euclid for composite moduli, then the digit loop and the rebuild (source: dsa/samples-c/src/Ch28/garner.c, lines 19-57)

```
// GarnerDigits converts residues over pairwise coprime moduli into the
// mixed-radix digits x_i of the expansion x = x_0 + m_0*x_1 +
// m_0*m_1*x_2 + ..., each digit in [0, m_i). Works for any moduli,
// not just primes, against chapter 16's pairwise crt fold.
func GarnerDigits(residues, moduli []uint64) ([]uint64, error) {
  if len(residues) ≠ len(moduli) {
    return nil, fmt.Errorf("garner: %d residues over %d moduli",
      len(residues), len(moduli))
  }
  r := make([]uint64, len(residues))
  copy(r, residues)
  for i := range r {
    r[i] %= moduli[i]
  }
  x := make([]uint64, len(moduli))
  for i := range moduli {
    for j := 0; j < i; j++ {
      // digits telescope: x_i = ((r_i - x_0)*inv(m_0) - x_1)*...
      inv, err := ch16.ModInvEuler(
        int64(moduli[j]%moduli[i]), int64(moduli[i]))
      if err ≠ nil {
        return nil, err
      }
      diff := (r[i] + moduli[i] - x[j]%moduli[i]) % moduli[i]
      r[i] = ch16.MulMod(diff, uint64(inv), moduli[i])
    }
    x[i] = r[i]
  }
  return x, nil
}
```

listing 888: go, the telescoping digit update, moduli reduced mod m i (source: dsa/samples-go/ch28/garner.go, lines 10-39)

```java
// inverse of a mod m for any coprime m (moduli here include 8 and 9,
// so extended euclid, not fermat)
static long modinv(long a, long m) {
  long t = 0, newt = 1, r = m, newr = a % m;
  while (newr != 0) {
    long q = r / newr;
    long tmp = t - q * newt;
    t = newt;
    newt = tmp;
    tmp = r - q * newr;
    r = newr;
    newr = tmp;
  }
  long inv = t % m;
  if (inv < 0)
    inv += m;
  return inv;
}

// digits x_i = (r_i - sum_{j<i} x_j m_j) inv(...) mod m_i, rebuild top
// down. returns the rebuilt value, fills x with the mixed-radix digits
static long garner(long[] r, long[] m, long[] x) {
  int k = r.length;
  for (int i = 0; i < k; i++) {
    long t = r[i] % m[i];
    for (int j = 0; j < i; j++) {
      long xj = x[j] % m[i];
      t = (t + m[i] - xj) % m[i];
      t = t * modinv(m[j] % m[i], m[i]) % m[i];
    }
    x[i] = t;
  }
  long v = x[k - 1];
  for (int i = k - 2; i >= 0; i--)
    v = v * m[i] + x[i];
  return v;
}
```

listing 889: java, extended euclid inverse for the composite moduli, the telescoping digit update, the rebuild in plain longs (source: dsa/samples-java/src/Ch28/Garner.java, lines 18-54)

```
    /// <summary>Mixed-radix digits x_i of the residues, so x = sum x_i prod_{j&lt;i} m_j.</summary>
    public static long[] Digits(long[] r, long[] m)
    {
        int k = r.Length;
        var digits = new long[k];
        for (int i = 0; i < k; i++)
        {
            long t = r[i] % m[i];
            if (t < 0)
                t += m[i];
            long prefix = 1; // m_0 * ... * m_{i-1} mod m_i, kept reduced so products fit
            for (int j = 0; j < i; j++)
            {
                t = (t - digits[j] % m[i] * prefix) % m[i];
                prefix = prefix * (m[j] % m[i]) % m[i];
            }
            if (t < 0)
                t += m[i];
            digits[i] = t * NumTheory.ModInv(prefix, m[i]) % m[i];
        }
        return digits;
    }

    /// <summary>Rebuild from the top: x = d_0 + m_0 (d_1 + m_1 (d_2 + ...)). The value
    /// must fit in a long; the fixtures' combined moduli products do once d_top is 0.</summary>
    public static long Rebuild(long[] digits, long[] m)
    {
        long x = 0;
        for (int i = digits.Length - 1; i ≥ 0; i--)
            x = checked(x * m[i] + digits[i]);
        return x;
    }
```

listing 890: c#, digits then the top-down rebuild in checked arithmetic (source: dsa/samples/src/Ch28/Garner.cs, lines 11-42)

```
function extGcd(a, b) {
  if (b === 0n) return { g: a, x: 1n, y: 0n };
  const r = extGcd(b, a % b);
  return { g: r.g, x: r.y, y: r.x - (a / b) * r.y };
}

function modInv(a, m) {
  const { g, x } = extGcd(((a % m) + m) % m, m);
  if (g !== 1n) throw new RangeError("moduli must be pairwise coprime");
  return ((x % m) + m) % m;
}

// residues[i] = x mod moduli[i]. returns { digits, value } with
// value = sum_i digits[i] * prod(moduli[0..i-1]); digit i inverts the
// accumulated product mod moduli[i]. the two-prime case over
// (998244353, 754974721) inverts 998244353 mod 754974721 to 416537774,
// exactly chapter 29's pinned INV
export function garner(residues, moduli) {
  const digits = [];
  let value = 0n;
  let prod = 1n; // moduli[0] * ... * moduli[i-1]
  for (let i = 0; i < moduli.length; i++) {
    const m = moduli[i];
    let t = ((residues[i] - value) % m + m) % m;
    t = (t * modInv(prod % m, m)) % m;
    digits.push(t);
    value += t * prod;
    prod *= m;
  }
  return { digits, value };
}
```

listing 891: javascript, extgcd inverse and the fold, BigInt throughout, the combined modulus passes 2^53 (source: dsa/samples-js/src/ch28-garner.mjs, lines 8-38)

```
def garner(residues, moduli):
    # digits x_i = (r_i - sum_{j<i} x_j m_j) * inv(prod_{j<i} m_j) mod m_i,
    # then the rebuild from the top
    k = len(moduli)
    x = [0] * k
    prods = [1] * (k + 1)  # prods[i] = m_0 * ... * m_{i-1}
    for i in range(k):
        prods[i + 1] = prods[i] * moduli[i]
        t = residues[i] % moduli[i]
        for j in range(i):
            t = (t - x[j] * prods[j]) % moduli[i]
        x[i] = t * pow(prods[i] % moduli[i], -1, moduli[i]) % moduli[i]
    total = 0
    for i in reversed(range(k)):
        total = total * moduli[i] + x[i]
    return x, total
```

listing 892: python, the digit loop over running products, pow with −1 for the inverse (source: dsa/samples-py/src/Ch28/garner.py, lines 14-29)

```lua
-- inverse of a mod m for coprime a, m by extended euclid (m need not be
-- prime, so fermat is not available)
local function modinv(a, m)
  local function ext(a, b)
    if b == 0 then return a, 1, 0 end
    local g, x, y = ext(b, a % b)
    return g, y, x - (a // b) * y
  end
  local g, x = ext(a % m, m)
  if g ~= 1 then error("not invertible") end
  return ((x % m) + m) % m
end

-- residues r over pairwise coprime moduli m: mixed-radix digits, then
-- the rebuild from the top. moduli here can be composite, unlike a
-- pairwise fermat fold
local function garner(r, m)
  local k = #m
  local x = {table.unpack(r)}
  for i = 1, k do
    for j = 1, i - 1 do
      x[i] = (x[i] - x[j]) % m[i]
      x[i] = x[i] * modinv(m[j] % m[i], m[i]) % m[i]
    end
  end
  local v = x[k]
  for i = k - 1, 1, -1 do
    v = v * m[i] + x[i] -- every fixture keeps this product under 2^63
  end
  return v, x
end
```

listing 893: lua, extended euclid again, m need not be prime, rebuild under 2^63 (source: dsa/samples-lua/ch28_garner.lua, lines 18-48)

The inverse here must be extended euclid, not fermat, because the 2019 fixture works mod 8 and 9. The pinned rebuild: x = 12345678987654321 over the moduli 1000000007, 998244353, 754974721 has residues 901234575, 760561298, 160985081, digits 901234575, 12345678, 0, and the rebuild returns the input exactly, JavaScript doing it through BigInt because the combined modulus reaches 1.2e16 past 2^53, java rebuilding in plain longs where every intermediate stays under 2^63, and cross-naming the 416537774 INV beside its digits. Factorials: 10! mod 11 = 10 with 9! mod 11 = 1, the wilson pair, 30! mod 1000000007 = 109361473, 100! mod 1000000007 = 437918130, 1000! mod 998244353 = 421678599, and 101! mod 101 = 0. Legendre: v 2 of 100! is 97, v 3 of 1000! is 498, v 5 of 2026! is 505, and 0! = 1! = 1. The contest tie is the third fixture: icpc 2019 problem K (book 10, chapter 10) multiplies survival fractions over residue classes of a departure time mod 2520 = 8 * 9 * 5 * 7, chapter 16 names the crt existence clause, and the departure time 1736 with residues (0, 8, 1, 0) and digits (0, 1, 4, 4) rebuilds through exactly this code, the constructive half of that solution.

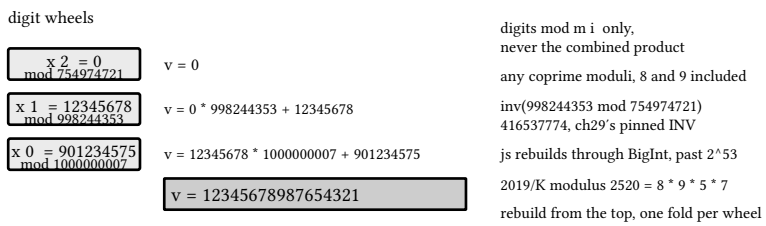


figure 227: the mixed-radix odometer, three digit wheels winding back to 12345678987654321

28.6. linear diophantine equations

The equation ax + by = c over the integers either has infinitely many solutions or none, and gcd decides which: solvable exactly when g = gcd(a, b) divides c. The extended euclid run already produces x and y with ax + by = g, scaling both by c over g gives one particular solution, and the full family steps by (b/g, -a/g), the move that adds ab/g and subtracts ba/g, a net zero. The two practical questions are then floor-and-ceiling arithmetic: shift the particular solution to the family member with the smallest non-negative x by one ceil division, and count the fully non-negative members by the t-window where x stays at or past 0 and y stays at or past 0.

The dry run: the fixture is 39x + 15y = 12 solved end to end with substitution asserted by the C# suite, the count family pinned beside it.

1. The euclid run on 39, 15 leaves g = 3, and 3 divides 12, so the line is solvable, the same test refusing 4x + 6y = 3.
2. Unwinding the quotients gives 39 × 2 + 15 × (-5) = 3, and scaling by 12 / 3 = 4 lands the particular (8, −20), checked 39 × 8 + 15 × (-20) = 312 - 300 = 12.
3. The family steps by (b/g, -a/g) = (5, −13), each move adding 39 × 5 = 195 and subtracting 15 × 13 = 195, a net zero.
4. Walking x down to the smallest non-negative: 8 - 5 = 3 lifts y to −20 + 13 = −7, the pinned (3, −7), and y negative with every step in either direction worse means 0 non-negative pairs.
5. The count family walks up instead: 2x + 3y = 10 lands (2, 2) then (5, 0) for its 2, and 3x + 5y = 30 lands (0, 6), (5, 3), (10, 0) for its 3.

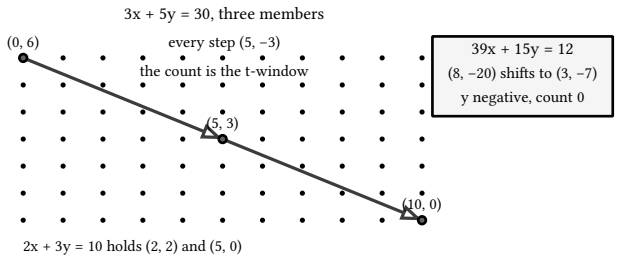


figure 228: the family walk of 3x + 5y = 30 as a sequence, the three non-negative members stepping by (5, −3), the first equation's shift boxed beside it

The three pairs close the count at the pinned 3, and the listings below solve the family in seven languages.

```c
static long long egcd(long long a, long long b, long long *x, long long *y) {
  long long old_r = a, r = b, old_s = 1, s = 0, old_t = 0, t = 1;
  while (r != 0) {
    long long q = old_r / r, tmp;
    tmp = old_r - q * r;
    old_r = r;
    r = tmp;
    tmp = old_s - q * s;
    old_s = s;
    s = tmp;
    tmp = old_t - q * t;
    old_t = t;
    t = tmp;
  }
  *x = old_s;
  *y = old_t;
  return old_r;
}

static long long x0, y0, gg;

// 0 solved, -1 refuses (g does not divide c)
static int solve(long long a, long long b, long long c) {
  gg = egcd(a, b, &x0, &y0);
  if (c % gg != 0)
    return -1;
  x0 *= c / gg;
  y0 *= c / gg;
  return 0;
}

static long long ceil_div(long long a, long long b) { // b > 0
  long long q = a / b, r = a % b;
  return (r > 0) ? q + 1 : q;
}

static long long floor_div(long long a, long long b) { // b > 0
  long long q = a / b, r = a % b;
  return (r < 0) ? q - 1 : q;
}

static long long fx, fy; // the minimal x ≥ 0 member of the family

static void shift_min_x(long long a, long long b) {
  long long bg = b / gg, ag = a / gg; // x = x0 + bg t, y = y0 - ag t
  long long t = ceil_div(-x0, bg);
  fx = x0 + bg * t;
  fy = y0 - ag * t;
}
```

listing 894: c, iterative extgcd, the scaled solve, ceil and floor division, the minimal-x shift (source: dsa/samples-c/src/Ch28/diophantine.c, lines 18-66)

```
// SolveDiophantine returns the particular solution of ax + by = c
// from the extended euclid coefficients scaled by c/g, refusing when
// gcd(a, b) does not divide c.
func SolveDiophantine(a, b, c int64) (int64, int64, error) {
  g, x, y := ch16.ExtGCD(a, b)
  if c%g != 0 {
    return 0, 0, fmt.Errorf(
      "no solution: gcd(%d, %d) = %d does not divide %d", a, b, g, c)
  }
  return x * (c / g), y * (c / g), nil
}

// MinimalX shifts the particular solution along the family
// x = x0 + (b/g)t, y = y0 - (a/g)t to the smallest x ≥ 0.
func MinimalX(a, b, c int64) (int64, int64, error) {
  x0, y0, err := SolveDiophantine(a, b, c)
  if err != nil {
    return 0, 0, err
  }
  g, _, _ := ch16.ExtGCD(a, b)
  step := b / g
  switch {
  case step == 0:
    if x0 < 0 {
      return 0, 0, fmt.Errorf("no solution with x ≥ 0")
    }
    return x0, y0, nil
  case step < 0:
    return 0, 0, fmt.Errorf("negate the equation: b = %d < 0", b)
  }
  // smallest t with x0 + step*t ≥ 0; negative t lowers a positive
  // x0 into the fundamental window [0, step)
  t := ceilDiv(-x0, step)
  return x0 + step*t, y0 - (a/g)*t, nil
}
```

listing 895: go, ch16 extgcd underneath, the negative-x shift into the fundamental window (source: dsa/samples-go/ch28/diophantine.go, lines 9-43)

```
static long egcd(long a, long b) {
  long oldR = a, r = b, oldS = 1, s = 0, oldT = 0, t = 1;
  while (r != 0) {
    long q = oldR / r;
    long tmp = oldR - q * r;
    oldR = r;
    r = tmp;
    tmp = oldS - q * s;
    oldS = s;
    s = tmp;
    tmp = oldT - q * t;
    oldT = t;
    t = tmp;
  }
  x0 = oldS;
  y0 = oldT;
  return oldR;
}

// 0 solved, -1 refuses (g does not divide c)
static int solve(long a, long b, long c) {
  gg = egcd(a, b);
  if (c % gg != 0)
    return -1;
  x0 *= c / gg;
  y0 *= c / gg;
  return 0;
}

static long ceilDiv(long a, long b) { // b > 0
  long q = a / b, r = a % b;
  return (r > 0) ? q + 1 : q;
}

static long floorDiv(long a, long b) { // b > 0
  long q = a / b, r = a % b;
  return (r < 0) ? q - 1 : q;
}

static long fx, fy; // the minimal x ≥ 0 member of the family

static void shiftMinX(long a, long b) {
  long bg = b / gg, ag = a / gg; // x = x0 + bg t, y = y0 - ag t
  long t = ceilDiv(-x0, bg);
  fx = x0 + bg * t;
  fy = y0 - ag * t;
}

// family members with x ≥ 0 and y ≥ 0 (a, b > 0)
static long countNonneg(long a, long b) {
  long tlo = ceilDiv(-x0, b / gg);
  long hi = floorDiv(y0, a / gg);
  return hi - tlo + 1;
}
```

listing 896: java, iterative extgcd, the scaled solve, truncating remainder with explicit ceil and floor corrections, the t-window count (source: dsa/samples-java/src/Ch28/Diophantine.java, lines 19-72)

```
    /// <summary>Particular (x, y) from extended Euclid scaled by c/g, or null when g does not divide c.</
summary>
    public static (long X, long Y)? Solve(long a, long b, long c)
    {
        (long g, long x0, long y0) = NumTheory.ExtGcd(a, b);
        if (c % g != 0)
            return null;
        return (x0 * (c / g), y0 * (c / g));
    }

    /// <summary>The family member with smallest x &gt;= 0; throws when unsolvable.</summary>
    public static (long X, long Y) MinimalNonNegativeX(long a, long b, long c)
    {
        (long x0, long y0)? solved = Solve(a, b, c);
        if (solved is null)
            throw new InvalidOperationException($"gcd({a},{b}) does not divide {c}");
        (long x, long y) = solved.Value;
        if (a == 0)
            return (0, y); // b y = c forces x = 0
        if (b == 0)
            return (x, 0); // a x = c forces y = 0
        long g = NumTheory.Gcd(a, b);
        long step = b / g;
        long t = CeilDiv(-x, step); // smallest t with x + step*t ≥ 0
        return (x + step * t, y - a / g * t);
    }

    /// <summary>Number of solutions with x &gt;= 0 and y &gt;= 0; needs positive coefficients.</summary>
    public static long CountNonNegative(long a, long b, long c)
    {
        if (a ≤ 0 || b ≤ 0)
            throw new ArgumentException("counting needs positive coefficients");
        (long x0, long y0)? solved = Solve(a, b, c);
        if (solved is null)
            return 0;
        (long x0, long y0) = solved.Value;
        long g = NumTheory.Gcd(a, b);
        long lo = CeilDiv(-x0, b / g); // x = x0 + (b/g) t ≥ 0
        long hi = FloorDiv(y0, a / g); // y = y0 - (a/g) t ≥ 0
        return Math.Max(0L, hi - lo + 1);
    }
```

listing 897: c#, solve, minimal non-negative x, and the count from one t-window (source: dsa/samples/src/Ch28/Diophantine.cs, lines 9-48)

```
function extGcd(a, b) {
  if (b === 0) return { g: a, x: 1, y: 0 };
  const r = extGcd(b, a % b);
  return { g: r.g, x: r.y, y: r.x - Math.floor(a / b) * r.y };
}

function gcd(a, b) {
  while (b) [a, b] = [b, a % b];
  return a;
}

// { x, y } with ax + by = c, or null when gcd(a,b) does not divide c
export function solveDiophantine(a, b, c) {
  if (a === 0 && b === 0) return c === 0 ? { x: 0, y: 0 } : null;
  if (a === 0) return c % b === 0 ? { x: 0, y: c / b } : null;
  if (b === 0) return c % a === 0 ? { x: c / a, y: 0 } : null;
  const { g, x, y } = extGcd(a, b);
  if (c % g !== 0) return null;
  return { x: (x * c) / g, y: (y * c) / g };
}

// family member with the smallest x ≥ 0 (y may go negative)
export function minimalNonneg(a, b, c) {
  const s = solveDiophantine(a, b, c);
  if (s === null) return null;
  if (a === 0 || b === 0) return s;
  const step = Math.abs(b / gcd(a, b)); // x-steps of the family
  const x = ((s.x % step) + step) % step;
  const t = (x - s.x) / (b / gcd(a, b));
  return { x, y: s.y - (a / gcd(a, b)) * t };
}
```

listing 898: javascript, recursive extgcd, the modular shift to minimal x (source: dsa/samples-js/src/ch28-diophantine.mjs, lines 6-36)

```
def extgcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x, y = extgcd(b, a % b)
    return g, y, x - (a // b) * y


def solve(a, b, c):
    # particular (x0, y0) with a*x + b*y = c; refuses when g does not
    # divide c; zero coefficients handled by their own branches
    if a == 0 and b == 0:
        raise ValueError("both coefficients vanish")
    if a == 0:
        if c % b:
            raise ValueError("c is not a multiple of b")
        return 0, c // b
    if b == 0:
        if c % a:
            raise ValueError("c is not a multiple of a")
        return c // a, 0
    g, s, _ = extgcd(a, b)
    if c % g:
        raise ValueError(f"gcd {g} does not divide {c}")
    scale = c // g
    return s * scale, (c - a * s * scale) // b


def minimal_x(a, b, c):
    # shift the family to the smallest x ≥ 0 (a, b > 0 in our fixtures)
    x0, y0 = solve(a, b, c)
    g = math.gcd(a, b)
    step = b // g
    t = -(x0 // step)  # ceil(-x0 / step)
    return x0 + t * step, y0 - t * (a // g)


def nonneg_count(a, b, c):
    # solutions with x ≥ 0 and y ≥ 0: the t window of the family
    x0, y0 = solve(a, b, c)
    g = math.gcd(a, b)
    lo = -(x0 // (b // g))  # smallest t with x ≥ 0
    hi = y0 // (a // g)  # largest t with y ≥ 0
    return max(0, hi - lo + 1)
```

listing 899: python, extgcd, the family shift, and the non-negative count (source: dsa/samples-py/src/Ch28/diophantine.py, lines 16-58)

```lua
local function extgcd(a, b)
  if b == 0 then return a, 1, 0 end
  local g, x, y = extgcd(b, a % b)
  return g, y, x - (a // b) * y
end

local function solve(a, b, c) -- one particular (x, y) or nil
  if a == 0 and b == 0 then
    if c == 0 then return 0, 0 end
    return nil
  end
  local g, x, y = extgcd(math.abs(a), math.abs(b))
  if a < 0 then x = -x end
  if b < 0 then y = -y end
  if c % g ~= 0 then return nil end
  return x * (c // g), y * (c // g)
end

local function gcd_of(a, b)
  local g = extgcd(math.abs(a), math.abs(b))
  return g
end

-- shift to the family member with minimal x ≥ 0 (b nonzero)
local function minimal_x(a, b, c)
  local x0, y0 = solve(a, b, c)
  local g = gcd_of(a, b)
  local x = x0 % math.abs(b // g)
  local y = (c - a * x) // b
  return x, y
end

-- count of fully non-negative solutions (a and b nonzero)
local function count_nonneg(a, b, c)
  local g = gcd_of(a, b)
  local _, y = minimal_x(a, b, c)
  if y < 0 then return 0 end
  return y // math.abs(a // g) + 1
end
```

listing 900: lua, sign folds for negative coefficients, x reduced mod the step (source: dsa/samples-lua/ch28_diophantine.lua, lines 7-45)

Fixtures pin three equations end to end, each particular solution verified by substitution in-suite. 39x + 15y = 12 gives (8, −20), 2x + 3y = 10 gives (-10, 10), and 7x + 11y = 100 gives (-300, 200), the extended euclid coefficients landing far from the first quadrant and the shift doing the walking, java keeping its coefficients in static fields with the same explicit ceil and floor corrections C carries for a truncating remainder. Minimal non-negative x: the first lands at (3, −7) with zero fully non-negative solutions, the second at (2, 2) with exactly 2, the pairs (2, 2) and (5, 0), the third at (8, 4) with exactly 1, and 3x + 5y = 30 lands at (0, 6) with 3, the pairs (10, 0), (5, 3), (0, 6). Family spacing is asserted directly, consecutive members differ by exactly (b/g, -a/g), which is (5, −13) on the first equation. Edges: 4x + 6y = 3 refuses because gcd 2 does not divide 3, zero coefficients solve on their own branches with 0x + 5y = 15 at (0, 3) and 5x + 0y = 15 at (3, 0). The contest kin: icpc 2023 problem A (book 10, chapter 12) solves an overdetermined integer 3 by 3 system whose triples have determinant 1 or 2, and the exact two-variable theory here is the same no-rounding backbone. This is general technique beyond that one problem.

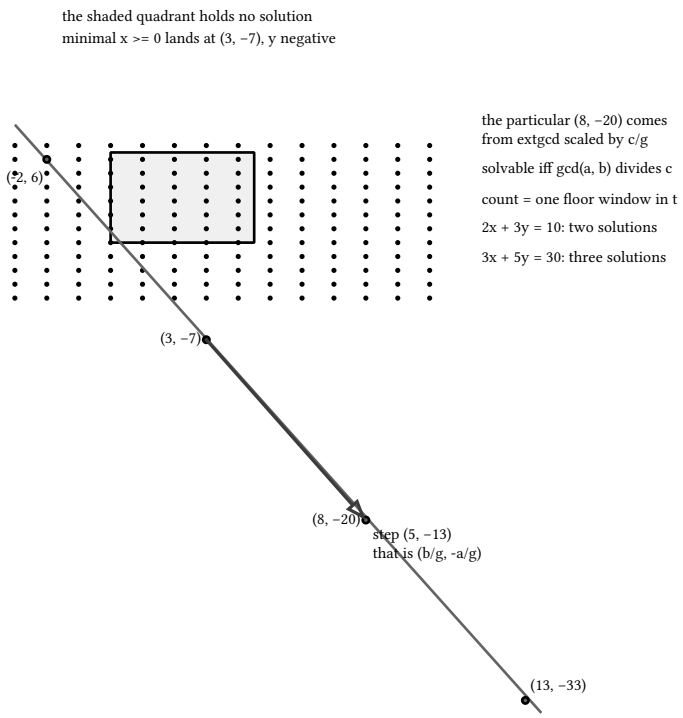


figure 229: the line 39x + 15y = 12 through the lattice, solutions spaced (5, −13), the shaded quadrant holding none

28.7. gray code and balanced ternary

Two numerations earn their keep by what they avoid. Gray code avoids the multi-bit jump: gray(i) = i ^ (i >> 1), and adjacent codes differ in exactly one bit, the bit where the underlying counter carries. The inverse folds prefix xors back, each output bit is the parity of the input bits above it, because the gray map xor-ed with its own right shift returns the original. Balanced ternary avoids the sign bit entirely: write integers in base 3 with digits −1, 0, and 1, printed here as -, 0, and +. The encoder takes remainders and lifts a remainder of 2 to the digit −1 with a carry of +1 into the next division, decode is horner at base 3, negation is flipping every digit, and k trits span exactly the interval from -(3^k - 1)/2 to (3^k - 1)/2.

The dry run: the fixtures are the gray table over 0 to 15 and gray(1234) = 1723 with the balanced ternary strings, asserted by the C# suite and identical in all seven.

1. gray folds one shift: i ^ (i >> 1) walks 0, 1, 3, 2, 6, 7, 5, 4 across 0 to 7, each code one bit from its neighbor.
2. The inverse folds prefix xors back: gray 6 = 110 returns bit 1, then 1 ^ 1 = 0, then 0 ^ 0 = 0, landing 100 = 4.
3. gray(1234) = 1723 with ungray(1723) = 1234 round-trips, and ungray(1789) = 1193.
4. The ternary encoder walks 2026 by remainders, each row one mod and one divide, the lift firing once.
5. That lift is the interesting step: 8 mod 3 = 2 becomes the digit - with a carry, so the next n is (8 + 1) / 3 = 3.
6. The digits close at +0-+000+ and negation flips every sign for −0+-000-, while 1093 = (3^7 - 1) / 2 prints ++++ +++ and 1094 needs the eighth trit.

n	n mod 3	digit	next n
2026	1	+	675
675	0	0	225
225	0	0	75
75	0	0	25
25	1	+	8
8	2	• (carry)	3
3	0	0	1
1	1	+	0

The 1723 round trip closes the numerations, and the listings below build both in seven languages.

```c
static unsigned int gray(unsigned int i) { return i ^ (i >> 1); }

static unsigned int ungray(unsigned int g) {
  unsigned int x = 0;
  for (; g; g >>= 1)
    x ^= g;
  return x;
}

// encode v into out msb-first, no leading zeros, "0" for zero; length
static int bt_encode(long long v, char *out) {
  char tmp[48];
  int len = 0;
  if (v == 0) {
    out[0] = '0';
    out[1] = 0;
    return 1;
  }
  while (v != 0) {
    long long r = v % 3;
    if (r == 2 || r == -1) {
      tmp[len++] = '-';
      v = (v + 1) / 3;
    } else if (r == 1 || r == -2) {
      tmp[len++] = '+';
      v = (v - 1) / 3;
    } else {
      tmp[len++] = '0';
      v = v / 3;
    }
  }
  for (int i = 0; i < len; i++)
    out[i] = tmp[len - 1 - i];
  out[len] = 0;
  return len;
}

static long long bt_decode(const char *s) {
  long long v = 0;
  for (; *s; s++)
    v = v * 3 + (*s == '+' ? 1 : (*s == '-' ? -1 : 0));
  return v;
}
```

listing 901: c, the gray pair, then encode by remainder lift, decode by horner (source: dsa/samples-c/src/Ch28/graytern.c, lines 19-61)

```
// Gray returns the gray code i ^ (i >> 1); adjacent codes differ in
// exactly one bit.
func Gray(i uint64) uint64 { return i ^ (i >> 1) }

// Ungray inverts the gray code by folding the prefix xors down from
// the top bit.
func Ungray(g uint64) uint64 {
  i := g
  for s := g >> 1; s != 0; s >>= 1 {
    i ^= s
  }
  return i
}

func pow3(k int) int64 {
  p := int64(1)
  for i := 0; i < k; i++ {
    p *= 3
  }
  return p
}

// BalancedTernaryDigits returns how many trits encode v: k trits span
// [-(3^k-1)/2, (3^k-1)/2].
func BalancedTernaryDigits(v int64) int {
  k := 1
  for v > (pow3(k)-1)/2 || v < -(pow3(k)-1)/2 {
    k++
  }
  return k
}
```

listing 902: go, the gray pair and the k-trit span walk, the encoder below (source: dsa/samples-go/ch28/graytern.go, lines 3-33)

```
static int gray(int i) {
  return i ^ (i >> 1);
}

static int ungray(int g) {
  int x = 0;
  for (; g != 0; g >>= 1)
    x ^= g;
  return x;
}

// encode v msb-first, no leading zeros, "0" for zero
static String btEncode(long v) {
  if (v == 0)
    return "0";
  StringBuilder tmp = new StringBuilder();
  while (v != 0) {
    long r = v % 3;
    if (r == 2 || r == -1) {
      tmp.append('-');
      v = (v + 1) / 3;
    } else if (r == 1 || r == -2) {
      tmp.append('+');
      v = (v - 1) / 3;
    } else {
      tmp.append('0');
      v = v / 3;
    }
  }
  return tmp.reverse().toString();
}

static long btDecode(String s) {
  long v = 0;
  for (int i = 0; i < s.length(); i++)
    v = v * 3 + (s.charAt(i) == '+' ? 1 : (s.charAt(i) == '-' ? -1 : 0));
  return v;
}
```

listing 903: java, the gray pair with the bit-by-bit inverse fold, encode handling the lifted −1 and −2 (source: dsa/samples-java/src/Ch28/Graytern.java, lines 17-54)

```
    /// <summary>gray(i) = i ^ (i &gt;&gt; 1); adjacent codes differ in exactly one bit.</summary>
    public static long Gray(long i) => i ^ (i >> 1);

    /// <summary>Inverse gray by folding prefix xors over doubling shifts.</summary>
    public static long Ungray(long g)
    {
        long x = g;
        x ^= x >> 1;
        x ^= x >> 2;
        x ^= x >> 4;
        x ^= x >> 8;
        x ^= x >> 16;
        x ^= x >> 32;
        return x;
    }

    /// <summary>Balanced ternary digit string, most significant trit first; a floored
    /// remainder of 2 lifts to the digit -1, and the division stays exact because
    /// v - digit is always divisible by 3 (truncating division would drift on negatives).</summary>
    public static string ToBalancedTernary(long v)
    {
        if (v == 0)
            return "0";
        var sb = new System.Text.StringBuilder();
        while (v != 0)
        {
            long r = ((v % 3) + 3) % 3; // floored remainder in {0, 1, 2}
            long digit = r == 2 ? -1 : r;
            sb.Append(digit == 0 ? '0' : digit > 0 ? '+' : '-');
            v = (v - digit) / 3;
        }
        char[] chars = sb.ToString().ToCharArray();
        Array.Reverse(chars);
        return new string(chars);
    }

    /// <summary>Decode by Horner at base 3 over the {-, 0, +} alphabet.</summary>
    public static long FromBalancedTernary(string s)
    {
        long v = 0;
        foreach (char ch in s)
            v = v * 3 + (ch == '+' ? 1 : ch == '-' ? -1 : 0);
        return v;
    }
```

listing 904: c#, the closed five-shift inverse fold, floored remainder lifted to −1 (source: dsa/samples/src/Ch28/GrayTern.cs, lines 7-50)

```
export const gray = (i) => i ^ (i >> 1);

// fold prefix xors back: each output bit is the parity of the bits
// above it
export function ungray(g) {
  let x = g;
  for (let shift = 1; shift < 32; shift <<= 1) x ^= x >>> shift;
  return x >>> 0;
}

export function popcount(x) {
  let c = 0;
  while (x) {
    x &= x - 1;
    c++;
  }
  return c;
}

// balanced ternary digits, most significant first; "0" for zero
export function btEncode(v) {
  if (v === 0) return "0";
  const out = [];
  let n = v;
  while (n !== 0) {
    const r = n % 3; // js keeps the dividend sign: -2 % 3 = -2
    if (r === 2 || r === -1) {
      out.push("-");
      n = (n + 1) / 3; // r = 2 lifts to -1 with a carry
    } else if (r === 1 || r === -2) {
      out.push("+");
      n = (n - 1) / 3;
    } else {
      out.push("0");
      n /= 3;
    }
  }
  return out.reverse().join("");
}

// horner at base 3
export function btDecode(s) {
  let v = 0;
  for (const ch of s) v = v * 3 + (ch === "+" ? 1 : ch === "-" ? -1 : 0);
  return v;
}
```

listing 905: javascript, shifts inside 32-bit Number territory, js keeps the dividend sign on % (source: dsa/samples-js/src/ch28-graytern.mjs, lines 7-52)

```python
def gray(i):
    return i ^ (i >> 1)


def ungray(g):
    # g_i = b_i ^ b_{i+1}, so fold downward from the copied top bit
    bl = g.bit_length()
    if bl == 0:
        return 0
    b = 1  # the top gray bit is the top binary bit
    x = b << (bl - 1)
    for i in range(bl - 2, -1, -1):
        b = ((g >> i) & 1) ^ b
        x |= b << i
    return x


def popcount(v):
    return bin(v).count("1")


def bt_encode(v):
    # remainder lift: r = 2 becomes -1 with a carry of +1
    if v == 0:
        return "0"
    digits = []
    while v:
        r = v % 3
        v //= 3
        if r == 2:
            r = -1
            v += 1
        digits.append(r)
    digit_char = {-1: "-", 0: "0", 1: "+"}
    return "".join(digit_char[d] for d in reversed(digits))


def bt_decode(s):
    v = 0
    for ch in s:  # horner at base 3
        v = v * 3 + (1 if ch == "+" else -1 if ch == "-" else 0)
    return v


def bt_negate(s):
    return s.translate(str.maketrans("+-", "-+"))
```

listing 906: python, the bit-by-bit inverse fold, encode with the r = 2 lift (source: dsa/samples-py/src/Ch28/graytern.py, lines 13-58)

```lua
local function gray(i) return i ~ (i >> 1) end

local function ungray(g)
  local x = g
  x = x ~ (x >> 1)
  x = x ~ (x >> 2)
  x = x ~ (x >> 4)
  x = x ~ (x >> 8)
  x = x ~ (x >> 16)
  return x
end

local function popcount(v)
  local n = 0
  while v ~= 0 do
    n = n + (v & 1)
    v = v >> 1
  end
  return n
end

-- digits {-1, 0, +1} printed as -, 0, +; least significant trit last
local function bt_encode(v)
  if v == 0 then return "0" end
  local digs = {}
  while v ~= 0 do
    local r = v % 3
    if r == 2 then r = -1 end
    digs[#digs + 1] = (r == 1 and "+" or (r == -1 and "-" or "0"))
    v = (v - r) // 3
  end
  return string.reverse(table.concat(digs))
end

local function bt_decode(s)
  local v = 0
  for i = 1, #s do
    local c = s:sub(i, i)
    local d = c == "+" and 1 or (c == "-" and -1 or 0)
    v = v * 3 + d
  end
  return v
end
```

listing 907: lua, the doubling-shift fold, chunks cut from the right (source: dsa/samples-lua/ch28_graytern.lua, lines 5-47)

The remainder-lift branch structure differs by language because % differs, C, Go, JavaScript, and Java keep the dividend's sign so their encoders handle a lifted −1 and a lifted −2, Python and Lua floor, so only r = 2 ever needs lifting, and C# normalizes into 0, 1, 2 first. All seven pin the same strings. The gray table over 0 to 15 reads 0, 1, 3, 2, 6, 7, 5, 4, 12, 13, 15, 14, 10, 11, 9, 8, gray(1234) = 1723 with ungray(1723) = 1234 and ungray(1789) = 1193, and the sweeps assert one-bit adjacency and round trips across 0 to 999. The balanced ternary encodings, identical digit strings in all six: 2026 is +0-+000+, −2026 is −0+-000-, 121393 is +-0+−−00+, 1093 is +++++++, −1093 is −−−-, 547 is +-+-+-+, and 0, 1, −1 are 0, +, -. The edge is the span: (3^7 - 1)/2 = 1093 is the 7-trit maximum, all plus signs, and 1094 needs 8 trits, with round-trip and negation-by-flip sweeps over −2000 to 2000. No finals problem called for either numeration, they are general technique.

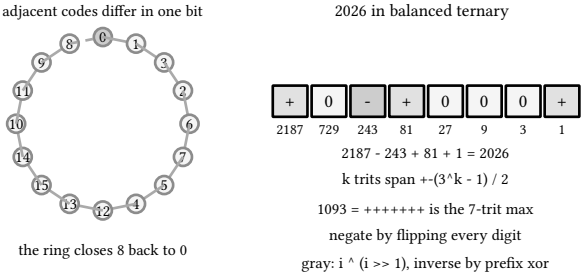


figure 230: the 4-bit gray cycle as a 16-node ring, beside the trit place values stacking 2026

28.8. continued fractions, convergents, and the stern-brocot tree

The euclidean algorithm divides and keeps quotients, and those quotients are a numeration: p over q equals the continued fraction whose terms are the quotients in order. The convergents, the values of each prefix, approximate the fraction from alternating sides and satisfy a two-term recurrence, h i = a i h i-1 + h i-2 and the same for k, off the seeds h = 1, 0 and k = 0, 1, and the last convergent is the fraction itself. Square roots have no finite expansion but a periodic one, computed exactly by the (m, d, a) state machine on integers, no floating point anywhere, closing when a term reaches twice the integer square root. The same tree organizes all positive rationals: the stern-brocot tree starts at 1 over 1 between the bounds 0/1 and 1/0, each step compares the target with the mediant of the bounds and descends left or right, every positive rational appears exactly once, and the path is a finite L/R string. Read the tree's levels in order and the farey sequence falls out, each next term computable from the previous two by one division.

The dry run: the fixture is 415/93 asserted term by term with its convergents by the C# suite, the stern-brocot path to 5/7 pinned beside it.

1. The euclid quotients are the terms: 415 = 4 × 93 + 43, 93 = 2 × 43 + 7, 43 = 6 × 7 + 1, then 7 = 7 × 1, so cf reads 4, 2, 6, 7.
2. The convergent recurrence folds off the seeds: h = 2 × 4 + 1 = 9 with k = 2 × 1 + 0 = 2, then 6 × 9 + 4 = 58 with 6 × 2 + 1 = 13, closing 7 × 58 + 9 = 415 with 7 × 13 + 2 = 93.
3. The four rungs read 4/1, 9/2, 58/13, 415/93, alternating above and below the value, the last the fraction itself.
4. The tree walk to 5/7 opens at the mediant 1/1 and descends left, 5 × 1 < 7 × 1.
5. Against 1/2 the cross products 10 and 7 point right, against 2/3 the pair 15 and 14 points right again, and against 3/4 the pair 20 and 21 points left, the path LRRL.
6. The new mediant of 2/3 and 3/4 is 5/7 itself, the landing, and F5 keeps its 11 terms with every neighbor determinant 1.

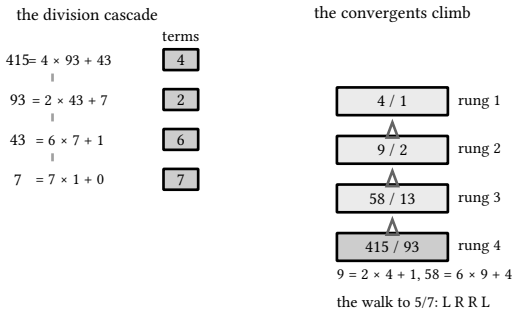


figure 231: the division cascade on the left, each euclid step yielding one term, the convergent ladder climbing on the right, one rung per term

The 415/93 closes as its own last convergent, and the listings below build all three faces in seven languages.

```c
static long long isqrt_ll(long long v) {
  long long r = 0;
  while ((r + 1) * (r + 1) <= v)
    r++;
  return r;
}

// periodic cf of sqrt(n): a0 once then the (m, d, a) machine until
// a = 2*a0; returns the full term count (perfect squares give 1)
static int sqrt_cf(long long v, long long *a) {
  long long a0 = isqrt_ll(v), m = 0, d = 1;
  a[0] = a0;
  int len = 1;
  if (a0 * a0 == v)
    return 1;
  while (a[len - 1] != 2 * a0) {
    long long m2 = d * a[len - 1] - m;
    long long d2 = (v - m2 * m2) / d;
    long long a2 = (a0 + m2) / d2;
    a[len++] = a2;
    m = m2;
    d = d2;
  }
  return len;
}

// stern-brocot walk from 1/1 keeping bounds 0/1 and 1/0, path into out
static int sb_path(long long p, long long q, char *out) {
  long long la = 0, lb = 1, ha = 1, hb = 0;
  int len = 0;
  while (1) {
    long long ma = la + ha, mb = lb + hb;
    if (ma == p && mb == q)
      break;
    if (ma * q > p * mb) { // mediant right of the target: go left
      out[len++] = 'L';
      ha = ma;
      hb = mb;
    } else {
      out[len++] = 'R';
      la = ma;
      lb = mb;
    }
  }
  out[len] = 0;
  return len;
}
```

listing 908: c, the periodic sqrt machine on exact integers, then the stern-brocot walk (source: dsa/samples-c/src/Ch28/cfrac.c, lines 45-91)

```
// Convergents runs the two-term recurrence h_i = a_i h_{i-1} +
// h_{i-2}, k_i likewise, from the seeds h_{-1} = 1, h_{-2} = 0,
// k_{-1} = 0, k_{-2} = 1. Returns the h and k lists in parallel.
func Convergents(cf []int64) (h, k []int64) {
  hPrev, hPrev2 := int64(1), int64(0)
  kPrev, kPrev2 := int64(0), int64(1)
  for _, a := range cf {
    hi := a*hPrev + hPrev2
    ki := a*kPrev + kPrev2
    h = append(h, hi)
    k = append(k, ki)
    hPrev, hPrev2 = hi, hPrev
    kPrev, kPrev2 = ki, kPrev
  }
  return h, k
}

// SqrtCF expands sqrt(n) by the (m, d, a) state machine on exact
// integers and returns the continued fraction [a0; a1, ..., a_period]
// together with the period length. n must not be a perfect square.
func SqrtCF(n int64) ([]int64, int) {
  a0 := int64(math.Sqrt(float64(n)))
  for (a0+1)*(a0+1) <= n {
    a0++
  }
  cf := []int64{a0}
  m, d, a := int64(0), int64(1), a0
  for {
    m = d*a - m
    d = (n - m*m) / d
    a = (a0 + m) / d
    cf = append(cf, a)
    if a == 2*a0 {
      return cf, len(cf) - 1
    }
  }
}
```

listing 909: go, convergents and the sqrt cycle with its period length (source: dsa/samples-go/ch28/cfrac.go, lines 16-52)

```
  // periodic cf of sqrt(n): a0 once then the (m, d, a) machine until
  // a = 2*a0; returns the full term count (perfect squares give 1)
  static int sqrtCf(long v, long[] a) {
    long a0 = isqrtLl(v), m = 0, d = 1;
    a[0] = a0;
    int len = 1;
    if (a0 * a0 == v)
      return 1;
    while (a[len - 1] != 2 * a0) {
      long m2 = d * a[len - 1] - m;
      long d2 = (v - m2 * m2) / d;
      long a2 = (a0 + m2) / d2;
      a[len++] = a2;
      m = m2;
      d = d2;
    }
    return len;
  }

  // stern-brocot walk from 1/1 keeping bounds 0/1 and 1/0
  static String sbPath(long p, long q) {
    long la = 0, lb = 1, ha = 1, hb = 0;
    StringBuilder out = new StringBuilder();
    for (;;) {
      long ma = la + ha, mb = lb + hb;
      if (ma == p && mb == q)
        break;
      if (ma * q > p * mb) { // mediant right of the target: go left
        out.append('L');
        ha = ma;
        hb = mb;
      } else {
        out.append('R');
        la = ma;
        lb = mb;
      }
    }
    return out.toString();
  }
```

listing 910: java, the (m, d, a) sqrt machine on exact longs, then the stern-brocot walk keeping bounds (source: dsa/samples-java/src/Ch28/Cfrac.java, lines 49-87)

```
    /// <summary>Convergents h/k by the two-term recurrence (h_-2 = 0, h_-1 = 1, ...).</summary>
    public static List<(long H, long K)> Convergents(IReadOnlyList<long> a)
    {
        var conv = new List<(long, long)>(a.Count);
        (long hPrev, long kPrev) = (1, 0); // h_{-1}, k_{-1}
        (long hPrevPrev, long kPrevPrev) = (0, 1); // h_{-2}, k_{-2}
        foreach (long ai in a)
        {
            long h = ai * hPrev + hPrevPrev;
            long k = ai * kPrev + kPrevPrev;
            conv.Add((h, k));
            (hPrevPrev, kPrevPrev) = (hPrev, kPrev);
            (hPrev, kPrev) = (h, k);
        }
        return conv;
    }

    /// <summary>Periodic continued fraction of sqrt(n) on exact integers.</summary>
    public static (long[] Head, long[] Period) SqrtCf(long n)
    {
        long a0 = (long)Math.Floor(Math.Sqrt(n));
        var period = new List<long>();
        long m = 0, d = 1, a = a0;
        do
        {
            m = d * a - m;
            d = (n - m * m) / d;
            a = (a0 + m) / d;
            period.Add(a);
        }
        while (a != 2 * a0);
        return (new[] { a0 }, period.ToArray());
    }
```

listing 911: c#, the convergent recurrence and the (m, d, a) cycle as head plus period (source: dsa/samples/src/Ch28/CFrac.cs, lines 19-51)

```
// [a0, ...period] of sqrt(n) by the (m, d, a) state machine; the period
// closes exactly when a = 2*a0
export function sqrtCf(n) {
  const a0 = Math.floor(Math.sqrt(n));
  let m = 0;
  let d = 1;
  let a = a0;
  const out = [a0];
  do {
    m = d * a - m;
    d = (n - m * m) / d;
    a = Math.floor((a0 + m) / d);
    out.push(a);
  } while (a !== 2 * a0);
  return out;
}

// "L"/"R" walk from 1/1 keeping bounds 0/1 and 1/0, mediant steps
export function sternBrocotPath(p, q) {
  let lo = [0, 1];
  let hi = [1, 0];
  let path = "";
  for (;;) {
    const mid = [lo[0] + hi[0], lo[1] + hi[1]];
    if (mid[0] === p && mid[1] === q) return path;
    if (mid[0] * q < p * mid[1]) {
      lo = mid; // mid < p/q: the target sits right
      path += "R";
    } else {
      hi = mid;
      path += "L";
    }
  }
}
```

listing 912: javascript, the sqrt machine and the mediant walk, floor keeps it exact (source: dsa/samples-js/src/ch28-cfrac.mjs, lines 35-68)

```
def convergents(terms):
    # h_{-1} = 1, h_{-2} = 0, k_{-1} = 0, k_{-2} = 1
    h1, h2, k1, k2 = 1, 0, 0, 1
    pairs = []
    for a in terms:
        h1, h2 = a * h1 + h2, h1
        k1, k2 = a * k1 + k2, k1
        pairs.append((h1, k1))
    return pairs


def cf_sqrt(num):
    # the (m, d, a) state machine on exact integers
    a0 = math.isqrt(num)
    m, d, a = 0, 1, a0
    out = [a0]
    while a != 2 * a0:
        m = d * a - m
        d = (num - m * m) // d
        a = (a0 + m) // d
        out.append(a)
    return out


def stern_brocot_path(pn, pd):
    # walk L or R from 1/1 keeping bounds 0/1 and 1/0
    lo, hi = (0, 1), (1, 0)
    out = []
    while True:
        m = (lo[0] + hi[0], lo[1] + hi[1])
        if m == (pn, pd):
            return "".join(out)
        if m[0] * pd < m[1] * pn:  # m < target: the mediant sits left
            out.append("R")
            lo = m
        else:
            out.append("L")
            hi = m
```

listing 913: python, convergents, the sqrt machine, the walk, and farey in four small functions (source: dsa/samples-py/src/Ch28/cfrac.py, lines 25-63)

```
local function convergents(c)
  local h1, h2, k1, k2 = 1, 0, 0, 1
  local out = {}
  for _, a in ipairs(c) do
    local h = a * h1 + h2
    local k = a * k1 + k2
    out[#out + 1] = {h, k}
    h2, h1 = h1, h
    k2, k1 = k1, k
  end
  return out
end

-- periodic cf of sqrt(n) by the (m, d, a) state machine, exact integers
local function cf_sqrt(n)
  local a0 = math.floor(math.sqrt(n))
  if a0 * a0 == n then return {a0}, 0 end
  local m, d, a = 0, 1, a0
  local out = {a0}
  local period = 0
  while a ~= 2 * a0 do
    m = d * a - m
    d = (n - m * m) // d
    a = (a0 + m) // d
    out[#out + 1] = a
    period = period + 1
  end
  return out, period
end

-- stern-brocot walk from 1/1 keeping bounds 0/1 and 1/0, L or R steps
local function sb_path(p, q)
  local path = {}
  local lo, hi = {0, 1}, {1, 0}
  local cur = {1, 1}
  while cur[1] ~= p or cur[2] ~= q do
    if p * cur[2] < q * cur[1] then -- target is left of cur
      hi = cur
      path[#path + 1] = "L"
    else
      lo = cur
      path[#path + 1] = "R"
    end
    cur = {lo[1] + hi[1], lo[2] + hi[2]}
  end
  return table.concat(path)
end
```

listing 914: lua, the walk keeps the current node and compares cross-multiplied (source: dsa/samples-lua/ch28_cfrac.lua, lines 16-62)

Fixtures pin all three faces of the structure. The rational expansions: cf of 415/93 is 4, 2, 6, 7 with convergents 4/1, 9/2, 58/13, 415/93, cf of 10/7 is 1, 2, 3 with convergents 1/1, 3/2, 10/7, and cf of 355/113 is 3, 7, 16, the approximation constant wearing its expansion. The periodic square roots: sqrt(19) gives 4 then the period 2, 1, 3, 1, 2, 8, sqrt(2) gives 1 then 2 with period 1, and sqrt(61) gives 7 then a period of eleven terms closing on 14. The tree: the path to 5/7 is LRRL, to 8/13 is LRLRL, to 3/8 is LLRL, and the root 1/1 walks the empty path. F5 holds its 11 terms 0/1, 1/5, 1/4, 1/3, 2/5, 1/2, 3/5, 2/3, 3/4, 4/5, 1/1, every adjacent pair satisfies the neighbor invariant with determinants exactly 1 in absolute value, and python closes the loop by checking the last convergent of each farey term equals

the term. No finals problem needed these, they are general technique with a long half-life under any approximation question.

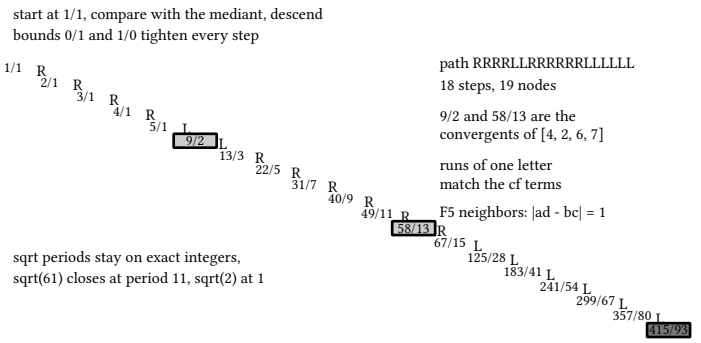


figure 232: 415/93 walked down the stern-brocot tree, path RRRRLLRRRRRLLLLLL, the convergents 9/2 and 58/13 met on the way

28.9. across the seven languages

Build size counts non-blank, non-comment lines over the chapter's eight sample files per language, embedded checks included:

language	build SLOC	dependency	boundary note
c	858	libc only	static tables to 1000 and cap 512 limbs of scratch, add-and-double mulmod with a remainder every step because rho moduli reach 4.6e18
c#	494	bcl, Ch16 NumTheory	UInt128 carries every product past 2^63, NumTheory and the rho lcg shared by import, refusals throw
go	563	dsabook/ch16	MulMod and ModPow from ch16 over bits.Mul64, errors name the cause, one package for all eight files
java	839	jdk 27 stdlib	self-contained files, the ch16 helpers re-carried inline, add-and-double mulmod with rho draws through `Long.remainderUnsigned`, plain long products at the 1e9+7 moduli, rebuild under 2^63
javascript	399	node stdlib	BigInt at each named boundary: rho moduli, dlog modmul at 1e9+7, the garner rebuild past 2^53
python	627	stdlib only	native ints, three-argument pow as the ground-truth cross-check, phi sieved in place by subtraction in 28.3
lua	693	lib.lua harness	1-based tables documented per file, add-and-double mulmod keeps products under 2^63, error() carries refusals

sources: the cp-algorithms articles, all cc by-sa 4.0, rewritten in our own words, accessed 2026-09-20: "Linear Sieve" and "Euler's totient function" for 28.1, "Primality tests" and "Integer factorization" for 28.2, "Number of divisors / sum of divisors" for 28.3, "Discrete Logarithm" and "Primitive Root" for 28.4, "Garner's Algorithm" and "Factorial modulo p" for 28.5, "Linear Diophantine Equations" and "Extended Euclidean Algorithm" for 28.6, "Gray code" and "Balanced Ternary" for 28.7, and "Continued fractions" plus "The Stern-Brocot tree and Farey sequences" for 28.8, at cp-algorithms.com/algebra/prime-sieve-linear.html, phi-function.html, primality_tests.html, factorization.html, divisors.html, discrete-log.html, primitive-root.html, garners-algorithm.html, factorial-modulo.html, linear-diophantine-equation.html, extended-euclid-algorithm.html, gray-code.html, balanced-ternary.html, continued-fractions.html, and cp-algorithms.com/others/stern_brocot_tree_farey_sequences.html. Mobius has no cp-algorithms article, the table treatment follows Hardy and Wright, An Introduction to the Theory of Numbers. The icpc problems named as applications: 2025 F and B, 2019 K, and 2023 A, pinned in the icpc book. Sample behavior verified by the seven suite gates scoped to chapter

28: c 8 files and 142 checks, c# 57 facts, go 26 tests, java 8 files and 228 checks, javascript 26 tests, python 8 files and 113 asserts, lua 32 checks, zero skipped. This chapter carries 56 listings and 8 figures.

29. fast big arithmetic

Chapter 2 multiplied digit strings by the schoolbook grid and chapter 16 squared numbers mod a prime. This chapter builds the whole ladder above those floors, a dynamic bignum core, karatsuba's three-way split, exact convolution through a two-prime number-theoretic transform, complex fft with a measured drift cap, knuth's long division, and polynomials over a transform-friendly prime. Every constant quoted here, the two primes 998244353 and 754974721, the inverse 416537774, the 2^14 fft cap, the per-language karatsuba cuts, flows from the rulings of book 10, ch 15 (the icpc world finals, fast big arithmetic), the measured contest companion whose benches this book does not repeat. The fixtures are fresh and didactic, the work is counted rather than timed, and all seven languages pin the same digits.

29.1. the dynamic bignum core

Everything above this section stands on one representation: sign-magnitude over base 1e9, limbs stored least significant first. Zero is the canonical pair of sign +1 and no limbs, normalize strips high zero limbs and then fixes the sign, and a minus a collapses to exactly that canonical zero instead of a negative empty list. Parse cuts nine-digit chunks from the right of the digit string, accepts a leading minus and nothing else, no plus sign, no leading zeros, no minus zero, and print reverses the cut, the top limb bare and every limb below it zero-padded to nine. Comparison reads sign, then limb count, then limbs top-down, and addition and subtraction route across signs by magnitude, equal signs add magnitudes, mixed signs subtract the smaller from the larger and keep the larger's sign. The schoolbook multiply rides along as the oracle, every later section cross-checks its products against this one.

The dry run: the fixture is the add pair 123456789012345678901234 and 987654321098765432109876, asserted by the C# suite with the sibling suites on the same digits.

1. Parse cuts the 25-digit operand from the right into three limbs, 789012345, 890123456, 1234567 least significant first, and the 24-digit one into 432109876, 321098765, 987654.
2. Limb 0 adds 789012345 + 432109876 = 1221122221, lands 221122221, carries 1.
3. Limb 1 folds the carry in, 890123456 + 321098765 + 1 = 1211222222, lands 211222222, carries 1 again.
4. Limb 2 closes 1234567 + 987654 + 1 = 2222222 with carry 0, and the headroom limb dies in normalize.
5. Print reverses the cut, the top limb bare and the rest padded to nine, reading 2222222, 211222222, 221122221 top down.
6. The carry chain runs the same fold one level up: 999999999 + 1 = 1000000000 lands 0 and carries 1, three times over, finishing at limbs 0, 0, 0, 1.

limb	cur	lands	carry
0	789012345 + 432109876 = 1221122221	221122221	1
1	890123456 + 321098765 + 1 = 1211222222	211222222	1
2	1234567 + 987654 + 1 = 2222222	2222222	0

The pinned sum 222222221122222222211122221 with the 0, 0, 0, 1 chain is what the listings below build in seven languages.

```
static void big_norm(Big *a) {
  while (a->len > 0 && a->limb[a->len - 1] == 0)
    a->len--;
  if (a->len == 0)
    a->sign = 1;
}

// 1 ok, 0 refused: empty, "+5", "-0", "007", any nondigit
static int big_parse(const char *s, Big *a) {
  a->sign = 1;
  a->len = 0;
  if (*s == '-') {
    a->sign = -1;
    s++;
  }
  if (!*s)
    return 0;
  int d = 0;
  for (const char *p = s; *p; p++, d++)
    if (*p < '0' || *p > '9')
      return 0;
  if (d > 1 && s[0] == '0')
    return 0;
  if (a->sign < 0 && d == 1 && s[0] == '0')
    return 0;
  int chunks = (d + 8) / 9;
  if (chunks > CAP)
    return 0;
  for (int c = 0; c < chunks; c++) {
    int hi = d - c * 9, lo = hi - 9;
    if (lo < 0)
      lo = 0;
    unsigned v = 0;
    for (int k = lo; k < hi; k++)
      v = v * 10 + (unsigned)(s[k] - '0');
    a->limb[c] = v;
  }
  a->len = chunks;
  big_norm(a);
  return 1;
}
```

listing 915: c, normalize then parse, chunks cut from the right, static limbs capped at 512 (source: dsa/samples-c/src/Ch29/bigint.c, lines 31-71)

```
func normBig(a Big) Big {
  a.limb = normLimbs(a.limb)
  if len(a.limb) == 0 {
    a.neg = false
  }
  return a
}

// ParseBig accepts a decimal string with an optional leading '-'
// only: no '+', no leading zeros, "0" for zero.
func ParseBig(s string) (Big, error) {
  neg := false
  body := s
  if len(body) > 0 && body[0] == '-' {
    neg = true
    body = body[1:]
  }
  if body == "" {
    return Big{}, fmt.Errorf("parse %q: empty", s)
  }
  for i := 0; i < len(body); i++ {
    if body[i] < '0' || body[i] > '9' {
      return Big{}, fmt.Errorf("parse %q: bad digit %q", s, body[i])
    }
  }
  if len(body) > 1 && body[0] == '0' {
    return Big{}, fmt.Errorf("parse %q: leading zero", s)
  }
  if body == "0" {
    if neg {
      return Big{}, fmt.Errorf("parse %q: negative zero", s)
    }
    return Big{}, nil
  }
  var limb []int64
  for end := len(body); end > 0; end -= 9 { // nine-digit chunks
    start := end - 9
    if start < 0 {
      start = 0
    }
    v := int64(0)
    for i := start; i < end; i++ {
      v = v*10 + int64(body[i]-'0')
    }
    limb = append(limb, v)
  }
  return Big{neg: neg, limb: limb}, nil
}
```

listing 916: go, normalize strips high zeros, ParseBig takes a leading minus only (source: dsa/samples-go/ch29/bigint.go, lines 36-83)

```
  static void bigNorm(Big a) {
    while (a.len > 0 && a.limb[a.len - 1] == 0)
      a.len--;
    if (a.len == 0)
      a.sign = 1;
  }

  // 1 ok, 0 refused: empty, "+5", "-0", "007", any nondigit
  static boolean bigParse(String s0, Big a) {
    a.sign = 1;
    a.len = 0;
    String s = s0;
    if (!s.isEmpty() && s.charAt(0) == '-') {
      a.sign = -1;
      s = s.substring(1);
    }
    if (s.isEmpty())
      return false;
    for (int i = 0; i < s.length(); i++)
      if (s.charAt(i) < '0' || s.charAt(i) > '9')
        return false;
    int d = s.length();
    if (d > 1 && s.charAt(0) == '0')
      return false;
    if (a.sign < 0 && d == 1 && s.charAt(0) == '0')
      return false;
    int chunks = (d + 8) / 9;
    if (chunks > CAP)
      return false;
    for (int c = 0; c < chunks; c++) {
      int hi = d - c * 9, lo = hi - 9;
      if (lo < 0)
        lo = 0;
      int v = 0;
      for (int k = lo; k < hi; k++)
        v = v * 10 + (s.charAt(k) - '0');
      a.limb[c] = v;
    }
    a.len = chunks;
    bigNorm(a);
    return true;
  }
```

listing 917: java, bigNorm then bigParse, chunks cut from the right, int limbs capped at 512 (source: dsa/samples-java/src/Ch29/Bigint.java, lines 19-60)

```
    /// <summary>Parses an optionally negative digit string; rejects "-0", "+5",
    /// leading zeros, and any non-digit.</summary>
    public static BigNum Parse(string s)
    {
        if (s.Length == 0)
            throw new InvalidOperationException("empty numeral");
        int sign = 1;
        int start = 0;
        if (s[0] == '-')
        {
            sign = -1;
            start = 1;
        }
        if (start == s.Length)
            throw new InvalidOperationException($"malformed numeral {s}");
        foreach (char c in s[start..])
            if (c is < '0' or > '9')
                throw new InvalidOperationException($"malformed numeral {s}");
        if (s[start] == '0')
        {
            if (s.Length - start > 1 || sign < 0)
                throw new InvalidOperationException($"malformed numeral {s}"); // "007" or "-0"
            return Zero;
        }
        var limbs = new List<long>();
        for (int end = s.Length; end > start; end -= 9)
        {
            int begin = Math.Max(start, end - 9);
            limbs.Add(long.Parse(s[begin..end]));
        }
        return new BigNum { Sign = sign, Limbs = limbs.ToArray() };
    }
```

listing 918: c#, Parse with the three refusals, limbs least significant first (source: dsa/samples/src/Ch29/BigNum.cs, lines 32-63)

```
export function normalize(x) {
  while (x.limbs.length > 0 && x.limbs[x.limbs.length - 1] === 0) x.limbs.pop();
  if (x.limbs.length === 0) x.sign = 1;
  return x;
}

// leading '-' only: no '+', no leading zeros, "0" for zero
export function parseBig(s) {
  if (!/^-?(0|[1-9][0-9]*)$/.test(s) || s === "-0") {
    throw new RangeError(`unparseable bignum: ${s}`);
  }
  const sign = s.startsWith("-") ? -1 : 1;
  const digits = sign === -1 ? s.slice(1) : s;
  const limbs = [];
  for (let end = digits.length; end > 0; end -= 9) {
    limbs.push(Number(digits.slice(Math.max(0, end - 9), end)));
  }
  return normalize({ sign, limbs });
}

export function bigToString(x) {
  if (x.limbs.length === 0) return "0";
  let s = String(x.limbs[x.limbs.length - 1]);
  for (let i = x.limbs.length - 2; i ≥ 0; i--) {
    s += String(x.limbs[i]).padStart(9, "0");
  }
  return (x.sign === -1 ? "-" : "") + s;
}
```

listing 919: javascript, normalize, parse, print, Number limbs with every exact product on BigInt (source: dsa/samples-js/src/ch29-bigint.mjs, lines 16-43)

```
def parse(s):
    # leading '-' only: no '+', no leading zeros, "0" for zero
    sign = 1
    if s.startswith("-"):
        sign = -1
        s = s[1:]
    if not s or not s.isdigit() or (len(s) > 1 and s[0] == "0"):
        raise ValueError(f"refuses {s!r}: malformed numeral")
    if sign == -1 and s == "0":
        raise ValueError("refuses '-0'")
    limbs = []
    for end in range(len(s), 0, -9):  # nine-digit chunks from the right
        start = max(0, end - 9)
        limbs.append(int(s[start:end]))
    return normalize(sign, limbs)


def to_str(v):
    sign, limbs = v
    if not limbs:
        return "0"
    head = ("-" if sign < 0 else "") + str(limbs[-1])
    return head + "".join(f"{limb:09d}" for limb in limbs[-2::-1])


def cmp_mag(a, b):
    if len(a) != len(b):
        return -1 if len(a) < len(b) else 1
    for x, y in zip(reversed(a), reversed(b)):
        if x != y:
            return -1 if x < y else 1
    return 0


def add_mag(a, b):
    out = []
    carry = 0
    for i in range(max(len(a), len(b))):
        t = carry
        if i < len(a):
            t += a[i]
        if i < len(b):
            t += b[i]
        out.append(t % BASE)
        carry = t // BASE
    if carry:
        out.append(carry)
    return out
```

listing 920: python, parse, print, compare, and the carry add beneath (source: dsa/samples-py/src/Ch29/bigint.py, lines 25-72)

```lua
local function norm(t, s)
  while #t > 0 and t[#t] == 0 do t[#t] = nil end
  if #t == 0 then return 1, {} end
  return s, t
end

local function parse(str)
  local neg = false
  local s = str
  if s:sub(1, 1) == "-" then
    neg = true
    s = s:sub(2)
  elseif s:sub(1, 1) == "+" then
    error("plus sign not accepted: " .. str)
  end
  if #s == 0 or not s:match("^%d+$") then
    error("not a decimal string: " .. str)
  end
  if #s > 1 and s:sub(1, 1) == "0" then
    error("leading zero not accepted: " .. str)
  end
  if neg and s == "0" then error("-0 not accepted: " .. str) end
  -- cut nine-digit chunks from the right
  local t = {}
  local i = #s
  while i > 0 do
    local lo = math.max(1, i - 8)
    t[#t + 1] = tonumber(s:sub(lo, i))
    i = lo - 1
  end
  return norm(t, neg and -1 or 1)
end
```

listing 921: lua, the canonical zero, then parse cutting chunks from the right (source: dsa/samples-lua/ch29_bigint.lua, lines 12-43)

The integer roads: C keeps a static u32 limb array capped at 512 limbs, 4608 digits, with no allocation anywhere. JavaScript stores limbs as Numbers but runs every exact limb product through BigInt, because 999999999 squared passes 2^53. Java keeps int limb arrays with the carry arithmetic in long, every schoolbook product 1e18 inside 2^63, and BigInteger left out on purpose, the limbs are the taught mechanism. Lua keeps integer tables, every schoolbook product 1e18 stays under 2^63. Fixtures pin the round trip first: the 36-digit string 314159265358979323846264338327950288 parses to four limbs reading 327950288, 846264338, 358979323, 314159265 least significant first, and prints back exactly. Then 2^100 = 1267650600228229401496703205376 with limbs 703205376, 229401496, 650600228, 1267, built by a hundred doublings in the languages that do not exponentiate, and reduced mod 1000000007 to 976371285, the same exponent chapter 16 pins from the other side. Arithmetic: 27 nines plus 1 runs the full carry chain into limbs 0, 0, 0, 1, the pair 123456789012345678901234 5 and 987654321098765432109876 adds to 222222221122222222211222 21 and subtracts to 24691356902469135690246 9, a minus a canonicalizes to zero, and the compare ladder on the pair returns 1, −1, 0. The oracle anchor: 987654321 squared is 975461057789971041, the schoolbook anchor chapters 2 and 16 both pinned, re-pinned here as the reference every multiply below must match. Parse refuses “−0”, “007”, and “+5” through each language’s refusal channel, and a zero multiplicand returns zero. One finals solver in the corpus ever needed this ladder, icpc 2018 problem C (book 10, chapter 9), whose cost accumulator peaks at 2.5e17 past 2^53 and carries BigInt in javascript, and the icpc book’s chapter 14 owns that contest-suite twin.

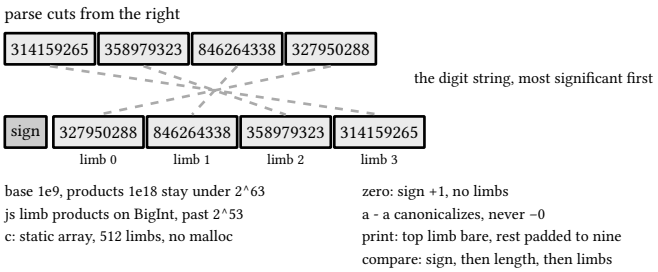


figure 233: the sign cell plus exactly the limbs it needs, the 36-digit fixture cut into four nine-digit chunks from the right

29.2. karatsuba multiplication

Schoolbook spends la times lb limb multiplies because it computes all four partial products of the halves. Karatsuba's move computes three. Split both operands at m, half the larger length floored, into low parts a0, b0 and high parts a1, b1. Then z0 is a0 times b0, z2 is a1 times b1, and the middle comes algebraically, z1 is (a0 + a1) times (b0 + b1) minus z0 minus z2, one multiply replacing two. The three products recombine by limb addition at offsets 0, m, and 2m, the base case is schoolbook below the cut, and the sign applies once at the end. The fresh teaching angle here is counted work: a counter ticks one per scalar limb multiply in the base case, and the recursion always calls (a0, b0), (a1, b1), and the two sums padded to max plus one limb, the plus one being the carry headroom, so the count is a pure function of operand length and cut, identical in every language that shares a cut.

The dry run: the fixture is the pattern operand, limb 123456789 repeated k times, squared at k = 24, 48, and 96 under the 24 cut shared by C, C#, Java, JavaScript, and Python, asserted by the C# suite, while Go walks its measured 48 cut and Lua its 64.

1. At k = 24 the operands sit exactly at the cut: one schoolbook leaf at 24 × 24 = 576, equal to the schoolbook square.
2. At k = 48 the split at m = 24 makes z0 and z2 leaves at 576 each, 576 + 576 = 1152, and the middle recurses on the sums padded to 25 limbs.
3. The 25-limb middle splits at m = 12: 12 × 12 = 144, 13 × 13 = 169, and the 14-limb sums leaf at 14 × 14 = 196, so the middle costs 144 + 169 + 196 = 509 and the node lands 1152 + 509 = 1661 against schoolbook 48 × 48 = 2304.
4. At k = 96 the two 48-limb halves cost 1661 + 1661 = 3322, and the pinned 4941 leaves 4941 - 3322 = 1619 for the 49-limb middle recursion.
5. The counts sit under the squares everywhere past the cut, 1661 against 2304 and 4941 against 96 × 96 = 9216.
6. The algebra checks the arithmetic beside the meter: (10^24 + 7) × (10^24 - 7) = 10^48 - 49, forty-six nines then 51, matching the oracle above.

limbs	the recursion	karatsuba	schoolbook
24	one leaf, 24 × 24	576	576
48	576 + 576 + 509	1661	2304
96	1661 + 1661 + 1619	4941	9216

The 4941 against 9216 at 96 limbs is the counted claim, and the listings below tick the same counter in seven languages.

```c
// karatsuba on magnitudes: split at m = max(la, lb) / 2, z0 and z2 at
// offsets 0 and 2m, z1 = (a0+a1)(b0+b1) - z0 - z2 at offset m
static void kmul(const unsigned int *a, int la, const unsigned int *b, int lb,
                 unsigned int *out) {
  for (int i = 0; i < la + lb; i++)
    out[i] = 0;
  if (la == 0 || lb == 0)
    return;
  if (la ≤ CUT && lb ≤ CUT) {
    school(a, la, b, lb, out);
    return;
  }
  int m = (la > lb ? la : lb) / 2;
  int a0 = la < m ? la : m, a1 = la - a0;
  int b0 = lb < m ? lb : m, b1 = lb - b0;
  int base = ap;
  unsigned int *z0 = &arena[ap];
  ap += a0 + b0;
  unsigned int *z2 = &arena[ap];
  ap += a1 + b1;
  int sa = (a0 > a1 ? a0 : a1) + 1, sb = (b0 > b1 ? b0 : b1) + 1;
  unsigned int *as = &arena[ap];
  ap += sa;
  unsigned int *bs = &arena[ap];
  ap += sb;
  unsigned int *z1 = &arena[ap];
  ap += sa + sb;
  kmul(a, a0, b, b0, z0);
  kmul(a + a0, a1, b + b0, b1, z2);
  add_arr(a, a0, a + a0, a1, as, sa);
  add_arr(b, b0, b + b0, b1, bs, sb);
  kmul(as, sa, bs, sb, z1);
  int z0l = norm_len(z0, a0 + b0), z2l = norm_len(z2, a1 + b1);
  int z1l = sa + sb;
  sub_arr(z1, &z1l, z0, z0l);
  sub_arr(z1, &z1l, z2, z2l);
  add_at(out, la + lb, z0, z0l, 0);
  add_at(out, la + lb, z1, z1l, m);
  add_at(out, la + lb, z2, z2l, 2 * m);
  ap = base;
}
```

listing 922: c, the recursion over a static arena, three multiplies and the offset recombination (source: dsa/samples-c/src/Ch29/karatsuba.c, lines 175-215)

```go
// MulCounted returns the product and the number of scalar limb
// multiplies spent in the base cases. The counting model is the
// canonical one: the recursion calls (a0, b0), (a1, b1), and the two
// sum operands of length max+1 each (the +1 is carry headroom), so
// the count is a pure function of operand length and cut.
func MulCounted(a, b Big) (Big, int) {
  prod, count := mulCounted(normBig(a).limb, normBig(b).limb)
  return normBig(Big{neg: a.neg != b.neg, limb: prod}), count
}

func mulCounted(a, b []int64) ([]int64, int) {
  count := 0
  var rec func(a, b []int64) []int64
  rec = func(a, b []int64) []int64 {
    if len(a) == 0 || len(b) == 0 {
      return nil
    }
    if len(a) <= KaraCut && len(b) <= KaraCut {
      count += len(a) * len(b)
      return mulSchool(a, b) // base case: no normalization gaps
    }
    m := len(a)
    if len(b) > m {
      m = len(b)
    }
    m /= 2 // split at floor(max(la, lb) / 2)
    a0, a1 := splitAt(a, m)
    b0, b1 := splitAt(b, m)
    z0 := rec(a0, b0)
    z2 := rec(a1, b1)
    sa := addMagPadded(a0, a1) // length max(a0, a1) + 1
    sb := addMagPadded(b0, b1)
    z1 := rec(sa, sb)
    z1 = subMag(z1, z0)
    z1 = subMag(z1, z2)
    out := make([]int64, len(a)+len(b))
    addInto(out, z0, 0)
    addInto(out, z1, m)
    addInto(out, z2, 2*m)
    return normLimbs(out)
  }
  return rec(a, b), count
}
```

listing 923: go, the counted recursion, base case at go's measured cut of 48 (source: dsa/samples-go/ch29/karatsuba.go, lines 15-57)

```
  static void kmul(int[] a, int ao, int la, int[] b, int bo, int lb, int[] out, int oo) {
    for (int i = 0; i < la + lb; i++)
      out[oo + i] = 0;
    if (la == 0 || lb == 0)
      return;
    if (la ≤ CUT && lb ≤ CUT) {
      school(a, ao, la, b, bo, lb, out, oo);
      return;
    }
    int m = Math.max(la, lb) / 2;
    int a0 = Math.min(la, m), a1 = la - a0;
    int b0 = Math.min(lb, m), b1 = lb - b0;
    int base = ap;
    int z0p = ap;
    ap += a0 + b0;
    int z2p = ap;
    ap += a1 + b1;
    int sa = Math.max(a0, a1) + 1, sb = Math.max(b0, b1) + 1;
    int asp = ap;
    ap += sa;
    int bsp = ap;
    ap += sb;
    int z1p = ap;
    ap += sa + sb;
    kmul(a, ao, a0, b, bo, b0, arena, z0p);
    kmul(a, ao + a0, a1, b, bo + b0, b1, arena, z2p);
    addArr(a, ao, a0, ao + a0, a1, arena, asp, sa);
    addArr(b, bo, b0, bo + b0, b1, arena, bsp, sb);
    kmul(arena, asp, sa, arena, bsp, sb, arena, z1p);
    int z0l = normLen(arena, z0p, a0 + b0);
    int z2l = normLen(arena, z2p, a1 + b1);
    int[] zlen = {sa + sb};
    subArr(arena, z1p, zlen, arena, z0p, z0l);
    subArr(arena, z1p, zlen, arena, z2p, z2l);
    int z1l = zlen[0];
    addAt(out, oo, la + lb, arena, z0p, z0l, 0);
    addAt(out, oo, la + lb, arena, z1p, z1l, m);
    addAt(out, oo, la + lb, arena, z2p, z2l, 2 * m);
    ap = base;
  }
```

listing 924: java, kmul over the shared arena, windows as array-offset pairs, three multiplies at offsets 0, m, 2m (source: dsa/samples-java/src/Ch29/Karatsuba.java, lines 162-201)

```
    private static long[] Rec(long[] a, long[] b, ref long counter)
    {
        if (Math.Max(a.Length, b.Length) ≤ Cut)
        {
            long[] r = BigNum.SchoolMag(a, b, out long leaves);
            counter += leaves;
            return r;
        }

        int m = Math.Max(a.Length, b.Length) / 2;
        long[] a0 = a[..Math.Min(m, a.Length)], a1 = a[Math.Min(m, a.Length)..];
        long[] b0 = b[..Math.Min(m, b.Length)], b1 = b[Math.Min(m, b.Length)..];
        long[] z0 = Rec(a0, b0, ref counter);
        long[] z2 = Rec(a1, b1, ref counter);
        // the sums keep their +1 carry headroom limb so the counting model stays
        // identical across languages
        long[] sa = Pad(BigNum.AddMag(a0, a1), Math.Max(a0.Length, a1.Length) + 1);
        long[] sb = Pad(BigNum.AddMag(b0, b1), Math.Max(b0.Length, b1.Length) + 1);
        long[] z1 = BigNum.SubMag(BigNum.SubMag(Rec(sa, sb, ref counter), z0), z2);
        return Combine(z0, z1, z2, m, a.Length + b.Length);
    }

    private static long[] Pad(long[] limbs, int width)
    {
        if (limbs.Length ≥ width)
            return limbs;
        long[] padded = new long[width];
        Array.Copy(limbs, padded, limbs.Length);
        return padded;
    }

    /// <summary>r = z2 &lt;&lt; 2m + z1 &lt;&lt; m + z0, assembled by limb addition.</summary>
    private static long[] Combine(long[] z0, long[] z1, long[] z2, int m, int width)
    {
        long[] r = new long[width];
        Accumulate(r, z2, 2 * m);
        Accumulate(r, z1, m);
        Accumulate(r, z0, 0);
        return BigNum.Normalize(r);
    }
```

listing 925: c#, Rec with padded sums, then Combine at offsets 0, m, 2m (source: dsa/samples/src/Ch29/Karatsuba.cs, lines 24-63)

```
function karaRaw(x, y, counter) {
  if (Math.max(x.length, y.length) ≤ KARA_CUT) {
    return schoolbook(x, y, counter);
  }
  const m = Math.floor(Math.max(x.length, y.length) / 2);
  const x0 = x.slice(0, m);
  const x1 = x.slice(m);
  const y0 = y.slice(0, m);
  const y1 = y.slice(m);
  const z0 = karaRaw(x0, y0, counter);
  const z2 = karaRaw(x1, y1, counter);
  const z1 = subRaw(
    subRaw(karaRaw(addRaw(x0, x1), addRaw(y0, y1), counter), z0),
    z2,
  );
  return combine(z0, z1, z2, m);
}

// z2 lands at offset 2m, z1 at m, z0 at 0, by limb addition
function combine(z0, z1, z2, m) {
  const out = [];
  addAt(out, z0, 0);
  addAt(out, z1, m);
  addAt(out, z2, 2 * m);
  while (out.length > 0 && out[out.length - 1] === 0) out.pop();
  return out;
}
```

listing 926: javascript, karaRaw and combine, the counter ticking in the base case (source: dsa/samples-js/src/ch29-karatsuba.mjs, lines 76-102)

```python
def karatsuba(a, b):
    # returns (product, leaf_multiplies)
    if not a or not b:
        return [], 0
    if max(len(a), len(b)) ≤ CUT:
        return school(a, b), len(a) * len(b)
    m = max(len(a), len(b)) // 2  # split both operands at m
    a0, a1 = a[:m], a[m:]
    b0, b1 = b[:m], b[m:]
    z0, c0 = karatsuba(a0, b0)
    z2, c1 = karatsuba(a1, b1)
    # the sums carry one limb of headroom, padded to max + 1 limbs so
    # the count is a pure function of the operand lengths
    width = max(len(a0), len(a1)) + 1
    sum_a = limb_add(a0, a1)
    sum_b = limb_add(b0, b1)
    sa = sum_a + [0] * (width - len(sum_a))
    sb = sum_b + [0] * (width - len(sum_b))
    z1, c2 = karatsuba(sa, sb)
    mid = sub3(z1, z0, z2)  # (a0+a1)(b0+b1) - z0 - z2
    out = []
    add_at(out, z0, 0)
    add_at(out, mid, m)
    add_at(out, z2, 2 * m)
    while out and out[-1] == 0:
        out.pop()
    return out, c0 + c1 + c2
```

listing 927: python, the recursion returning product and leaf-multiplies together (source: dsa/samples-py/src/Ch29/karatsuba.py, lines 86-112)

```
local function split(t, m)
  local lo, hi = {}, {}
  for i = 1, math.min(#t, m) do lo[i] = t[i] end
  for i = m + 1, #t do hi[i - m] = t[i] end
  return lo, hi
end

-- pad a sum operand out to the carry-headroom length max+1 so the
-- metered recursion follows the canonical counting model exactly
local function pad1(t, n)
  local u = {table.unpack(t)}
  while #u < n do u[#u + 1] = 0 end
  return u
end

-- z0 = a0*b0, z2 = a1*b1, z1 = (a0+a1)(b0+b1) - z0 - z2, recombined at
-- offsets 2m, m, 0
local function kmul(a, b, meter)
  if #a == 0 or #b == 0 then return {} end
  if #a ≤ CUT and #b ≤ CUT then
    if meter then meter.leaves = meter.leaves + #a * #b end
    return schoolbook(a, b)
  end
  local m = math.max(#a, #b) // 2
  local a0, a1 = split(a, m)
  local b0, b1 = split(b, m)
  local z0 = kmul(a0, b0, meter)
  local z2 = kmul(a1, b1, meter)
  local la, lb = math.max(#a0, #a1) + 1, math.max(#b0, #b1) + 1
  local z1 = kmul(pad1(magadd(a0, a1), la), pad1(magadd(b0, b1), lb),
                  meter)
  local mid = magsub(z1, magadd(z0, z2))
  local acc = {}
  addshift(acc, z2, 2 * m)
  addshift(acc, mid, m)
  addshift(acc, z0, 0)
  return norm(acc)
end
```

listing 928: lua, split, pad the sums to max plus one, recurse, recombine (source: dsa/samples-lua/ch29_karatsuba.lua, lines 85-122)

The cuts are the icpc book's chapter 14 measured crossovers reused without re-benching, 24 limbs for C, C#, Java, JavaScript, and Python, 48 for Go, 64 for Lua, and each language asserts its own cut's pinned rows. At cut 24 the counts run (24, 576, 576), (48, 1661, 2304), and (96, 4941, 9216) for limbs, karatsuba, schoolbook, go's cut 48 gives (48, 2304, 2304), (96, 6485, 9216), and (144, 12031, 20736), and lua's cut 64 gives (64, 4096, 4096) and (128, 11461, 16384). At the cut karatsuba degenerates to schoolbook, one limb past it the count already drops below the square, and at 96 limbs karatsuba spends 4941 against 9216. The algebraic identities pin the arithmetic: (10^40 - 1)(10^40 + 1) is 10^80 - 1, eighty nines, and (10^24 + 7)(10^24 - 7) is 10^48 - 49. The cut boundary walks the pattern operand, limb 123456789 repeated k times, squared at k one below, at, and one above the cut: the square holds exactly 18k - 1 digits, 413, 431, 449 at the 24 cut, 845, 863, 881 at go's 48, 1133, 1151, 1169 at lua's 64, and every square opens 15241578780673678546 and closes 20515622620750190521. A zero operand returns zero, the asymmetric 144 by 21 limb product equals schoolbook, and all four sign combinations on +-(10^24 + 7) times +-(10^24 - 7) agree with the oracle, the sign applied exactly once.

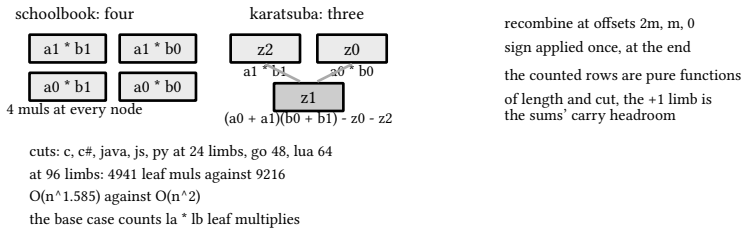


figure 234: the schoolbook grid needs four partial products, karatsuba needs three, the measured cuts beneath

29.3. exact convolution with two primes

Multiplication is a convolution of limbs, and the number-theoretic transform computes a convolution exactly, no floating point anywhere. The operands regroup to base-1e4 limbs through the decimal string, four digits per limb from the right, a linear pass that cannot drift a digit. The transform runs over a prime that admits power-of-two roots, and one prime is not enough: the true column values of a convolution at these limb sizes pass the product of two primes, so the suite transforms twice. P1 is 998244353 = 119 * 2^23 + 1 with primitive root 3, P2 is 754974721 = 45 * 2^24 + 1 with root 11, each column folds by t = (x2 + 2 P2 - x1) times INV mod P2 with INV = 416537774, and the exact value is x1 + P1 t. The wrap-safe ordering, x2 plus twice P2 minus x1, keeps the difference positive before the reduction, the lesson the icpc book's chapter 14 go stream paid for once. The transform itself is the iterative cooley-tukey shape, a bit-reversal permutation then doubling stages, one stage root by fast pow and running powers by repeated multiplication inside the stage, the inverse through inverted roots and the n^(-1) scale.

The dry run: the fixtures are the plain integer convolutions and the stressor limb counts, asserted by the C# suite with Python on the same rows.

1. The walk convolves [1, 2, 3] with [4, 5, 6] column by column: column 0 is 1 × 4 = 4, column 1 is 1 × 5 + 2 × 4 = 5 + 8 = 13.
2. Column 2 folds the full diagonal, 1 × 6 + 2 × 5 + 3 × 4 = 6 + 10 + 12 = 28, and the tail tapers, 2 × 6 + 3 × 5 = 12 + 15 = 27, then 3 × 6 = 18.
3. No column reaches the base, so 4, 13, 28, 27, 18 are the answer themselves, what the transform must land exactly through both primes.
4. The stressor regroups through the decimal string: 10^48 + 7 becomes 13 limbs [7, 0 eleven times, 1], 10^48 - 7 becomes 12 limbs [9993, 9999 eleven times].
5. The transform length is the next power of two past 13 + 12 = 25, which is 32, far under the 2^16 cap.
6. The exact product lands at 10^96 - 49, ninety-four nines then 51, cross-checked against the 29.1 oracle.

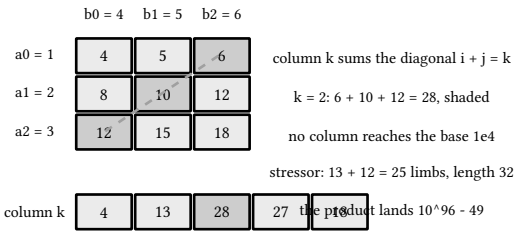


figure 235: the limb-product grid folding along anti-diagonals into the five carry-free columns

10^96 - 49, ninety-four nines then 51, is the pinned product, and the listings below fold it through both primes in seven languages.

```c
static void ntt(unsigned long long *a, int len, int invert,
                unsigned long long p, unsigned long long g) {
  for (int i = 1, j = 0; i < len; i++) {
    int bit = len >> 1;
    for (; j & bit; bit >>= 1)
      j ^= bit;
    j ^= bit;
    if (i < j) {
      unsigned long long t = a[i];
      a[i] = a[j];
      a[j] = t;
    }
  }
  for (int half = 2; half ≤ len; half <<= 1) {
    unsigned long long w = modpow(g, (p - 1) / (unsigned long long)half, p);
    if (invert)
      w = modpow(w, p - 2, p);
    for (int i = 0; i < len; i += half) {
      unsigned long long cur = 1;
      for (int j = 0; j < half / 2; j++) {
        unsigned long long u = a[i + j];
        unsigned long long v = a[i + j + half / 2] * cur % p;
        a[i + j] = (u + v) % p;
        a[i + j + half / 2] = (u + p - v) % p;
        cur = cur * w % p;
      }
    }
  }
  if (invert) {
    unsigned long long ninv = modpow((unsigned long long)len % p, p - 2, p);
    for (int i = 0; i < len; i++)
      a[i] = a[i] * ninv % p;
  }
}
```

listing 929: c, the iterative transform, bit reversal then doubling stages, butterflies inside u64 (source: dsa/samples-c/src/Ch29/ntt.c, lines 43-76)

```
func ntt(a []uint64, p, root uint64, invert bool) {
  n := len(a)
  for i, j := 1, 0; i < n; i++ {
    bit := n >> 1
    for ; j&bit ≠ 0; bit >>= 1 {
      j ^= bit
    }
    j |= bit
    if i < j {
      a[i], a[j] = a[j], a[i]
    }
  }
  for length := 2; length <= n; length <<= 1 {
    w := root
    if invert {
      w = powMod(root, p-2, p)
    }
    wn := powMod(w, (p-1)/uint64(length), p)
    for i := 0; i < n; i += length {
      wCur := uint64(1)
      for j := 0; j < length/2; j++ {
        u := a[i+j]
        v := mulMod(a[i+j+length/2], wCur, p)
        a[i+j] = u + v
        if a[i+j] ≥ p {
          a[i+j] -= p
        }
        a[i+j+length/2] = u + p - v
        if a[i+j+length/2] ≥ p {
          a[i+j+length/2] -= p
        }
        wCur = mulMod(wCur, wn, p)
      }
    }
  }
  if invert {
    nInv := powMod(uint64(n), p-2, p)
    for i := range a {
      a[i] = mulMod(a[i], nInv, p)
    }
  }
}
```

listing 930: go, the transform, add-subtract butterflies staying inside int64 (source: dsa/samples-go/ch29/ntt.go, lines 37-78)

```
  static void ntt(long[] a, int len, boolean invert, long p, long g) {
    for (int i = 1, j = 0; i < len; i++) {
      int bit = len >> 1;
      for (; (j & bit) != 0; bit >>= 1)
        j ^= bit;
      j ^= bit;
      if (i < j) {
        long t = a[i];
        a[i] = a[j];
        a[j] = t;
      }
    }
    for (int half = 2; half ≤ len; half <<= 1) {
      long w = modpow(g, (p - 1) / half, p);
      if (invert)
        w = modpow(w, p - 2, p);
      for (int i = 0; i < len; i += half) {
        long cur = 1;
        for (int j = 0; j < half / 2; j++) {
          long u = a[i + j];
          long v = a[i + j + half / 2] * cur % p;
          a[i + j] = (u + v) % p;
          a[i + j + half / 2] = (u + p - v) % p;
          cur = cur * w % p;
        }
      }
    }
    if (invert) {
      long ninv = modpow(len % p, p - 2, p);
      for (int i = 0; i < len; i++)
        a[i] = a[i] * ninv % p;
    }
  }
```

listing 931: java, the iterative transform, bit reversal then doubling stages, butterflies inside long (source: dsa/samples-java/src/Ch29/Ntt.java, lines 40-72)

```
    public static long[] ConvolveRaw(long[] a, long[] b)
    {
        long[] c1 = ConvolveMod(a, b, P1, 3);
        long[] c2 = ConvolveMod(a, b, P2, 11);
        var r = new long[a.Length + b.Length - 1];
        for (int k = 0; k < r.Length; k++)
        {
            long t = (c2[k] + 2 * P2 - c1[k]) % P2 * InvP1ModP2 % P2; // wrap-safe ordering
            r[k] = c1[k] + P1 * t;
        }
        return r;
    }

    /// <summary>Single-prime convolution over a prime with a power-of-two-friendly root.</summary>
    internal static long[] ConvolveMod(long[] a, long[] b, long p, long root)
    {
        int need = a.Length + b.Length;
        int n = 1;
        while (n < need)
            n <<= 1;
        if (n > Cap)
            throw new InvalidOperationException(
                $"transform length {n} exceeds the 2^16 limb cap");
        var fa = new long[n];
        var fb = new long[n];
        Array.Copy(a, fa, a.Length);
        Array.Copy(b, fb, b.Length);
        Transform(fa, p, root, invert: false);
        Transform(fb, p, root, invert: false);
        for (int i = 0; i < n; i++)
            fa[i] = fa[i] * fb[i] % p;
        Transform(fa, p, root, invert: true);
        return fa[..(need - 1)];
    }
```

listing 932: c#, the crt fold per column, then one prime's convolution driver (source: dsa/samples/src/Ch29/Ntt.cs, lines 52-85)

```
function transform(a, p, root, invert) {
  const n = a.length;
  for (let i = 1, j = 0; i < n; i++) {
    let bit = n >> 1;
    for (; j & bit; bit >>= 1) j ^= bit;
    j |= bit;
    if (i < j) [a[i], a[j]] = [a[j], a[i]];
  }
  for (let len = 2; len ≤ n; len <<= 1) {
    let w = powMod(root, (p - 1n) / BigInt(len), p);
    if (invert) w = powMod(w, p - 2n, p);
    const half = len >> 1;
    for (let i = 0; i < n; i += len) {
      let wp = 1n;
      for (let j = 0; j < half; j++) {
        const u = a[i + j];
        const v = (a[i + j + half] * wp) % p;
        a[i + j] = (u + v) % p;
        a[i + j + half] = (u + p - v) % p;
        wp = (wp * w) % p;
      }
    }
  }
  if (invert) {
    const ninv = powMod(BigInt(n), p - 2n, p);
    for (let i = 0; i < n; i++) a[i] = (a[i] * ninv) % p;
  }
}
```

listing 933: javascript, transform on BigInt, stated openly, one stage root by fast pow (source: dsa/samples-js/src/ch29-ntt.mjs, lines 31-58)

```
def convolve_raw(x, y):
    # base-1e4 limb lists -> true (uncarried) column values
    la, lb = len(x), len(y)
    if la == 0 or lb == 0:
        return []
    need = la + lb - 1  # real column count of the convolution
    size = 1
    while size < need:
        size <<= 1
    if size > CAP:
        raise ValueError("past the 2^16 base-1e4 transform cap")
    cols = []
    for p, root in ((P1, 3), (P2, 11)):
        fa = x + [0] * (size - la)
        fb = y + [0] * (size - lb)
        ntt(fa, root, p, False)
        ntt(fb, root, p, False)
        for k in range(size):
            fa[k] = fa[k] * fb[k] % p
        ntt(fa, root, p, True)
        cols.append(fa[:need])
    out = []
    for c1, c2 in zip(cols[0], cols[1]):  # one convolution per prime
        t = (c2 + 2 * P2 - c1) % P2 * INV % P2  # wrap-safe crt fold
        out.append(c1 + P1 * t)
    return out
```

listing 934: python, one convolution per prime, the wrap-safe crt fold at the bottom (source: dsa/samples-py/src/Ch29/ntt.py, lines 52-77)

```
local function ntt(t, n, p, g, invert)
  local bits = 0
  local nn = n
  while nn > 1 do bits = bits + 1; nn = nn // 2 end
  for i = 1, n do -- bit-reverse permutation, 1-based tables
    local j, x = 0, i - 1
    for _ = 1, bits do
      j = (j << 1) | (x & 1)
      x = x >> 1
    end
    if i - 1 < j then t[i], t[j + 1] = t[j + 1], t[i] end
  end
  local len = 2
  while len ≤ n do
    local wstep = modpow(g, (p - 1) // len, p)
    if invert then wstep = modpow(wstep, p - 2, p) end
    for i = 0, n - 1, len do
      local w = 1
      local half = len // 2
      for j = 0, half - 1 do
        local u = t[i + j + 1]
        local v = t[i + j + half + 1] * w % p
        t[i + j + 1] = (u + v) % p
        t[i + j + half + 1] = (u - v + p) % p
        w = w * wstep % p
      end
    end
    len = len * 2
  end
  if invert then
    local ninv = modpow(n, p - 2, p)
    for i = 1, n do t[i] = t[i] * ninv % p end
  end
end
```

listing 935: lua, the transform over 1-based tables, powers by repeated multiply (source: dsa/samples-lua/ch29_ntt.lua, lines 26-59)

The butterfly arithmetic is where the languages split, and the note is short: a residue product stays under (P-1)^2 < 9.97e17, inside signed 64 bits for C, Go, Java, C#, and Lua, while JavaScript runs the butterflies on BigInt. The ceiling note: P1 times P2 is 753649251896000513, under 2^63, and every true column value the fold rebuilds sits below it. The transform length is the next power of two at or past la plus lb, capped at 2^16 base-1e4 limbs, 262144 decimal digits for the pair, with a loud refusal past the cap in every refusal channel. Fixtures pin carry-free columns first, [1, 2, 3] convolved with [4, 5, 6] gives [4, 13, 28, 27, 18], the delta identity holds, and a single column squares 9999 into 99980001. The bignum stressor: 10^48

1. 7 regroups to 13 limbs reading [7, 0 eleven times, 1], 10^48 - 7 to

12 limbs reading [9993, 9999 eleven times], the product pins 24 limbs [9951, then 9999 twenty-three times], and the value is 10^96 - 49, checked against the 29.1 schoolbook oracle. The INV contract asserts 243269632 times 416537774 equals 1 mod 754974721, the same constant chapter 28's garner fixture cross-names. Zero convolves to zero, all four sign combinations pin, the 29.2 pattern operands at k = 12 and 24 limbs match schoolbook digit for digit, a pair forcing transform length 2^15 is accepted and one forcing 2^17 refused loudly.

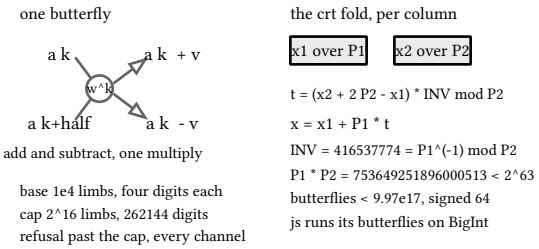


figure 236: one butterfly over w^k beside the two-prime crt fold with INV = 416537774 pinned

29.4. complex fft, plain and split

The complex fft multiplies the same polynomials in floating point, and the engineering is all in the rounding. Two paths run side by side. The plain path transforms both operands forward, multiplies pointwise, and transforms back, three transforms. The split path, the headline, packs both operands into one complex array, P of j equals a of j plus i times b of j, runs one forward transform, and recovers both spectra from it by conjugate reversal, A-hat of k is P-hat of k plus the conjugate of P-hat of n minus k over 2, and B-hat of k is minus i times their difference over 2. Two transforms total. Every coefficient rounds by floor of x plus 0.5, never a nearest-integer builtin, and the guard refuses any value landing 0.25 or more from an integer, the tripwire that turns silent drift into a loud error. The transform cap is 2^14 points, pinned by the icpc book's chapter 14 drift measurement: 0.048 to 0.191 fractional distance at 2^14 against 0.334 to 0.807 at 2^16, so the cap sits about five times inside the guard. The twiddle policy is the drift source in plain sight, one cos and one sin per stage with powers by repeated multiplication.

The dry run: the fixtures are the rounding guard's unit values and the stressor square, asserted by the C# suite on both paths with the sibling suites on the same digits.

1. The guard walks its policy first: exact 5.0 rounds to 5 unchanged, and 2.24 rounds down because floor(2.24 + 0.5) = 2.
2. 2.76 rounds up, floor(2.76 + 0.5) = 3, while 2.25 and 2.75 sit exactly 0.25 from an integer and 0.5 half a unit away, so all three refuse.
3. The stressor squares 10^50 - 3: the identity gives 10^100 - 6 × 10^50 + 9, digits 49 nines, a 4, 49 zeros, a closing 9.
4. The plain path transforms twice forward and once back; the split path packs a + i b into one array, recovers both spectra by conjugate reversal, and inverts once: two transforms.
5. Every coefficient of both paths lands inside the guard, rounds by floor(x + 0.5), and prints the same 100 digits.
6. At the cap exactly, 8192 limbs of 9999 square with 8192 + 8192 = 16384 = 2^14 points, and one doubling past refuses on both paths.

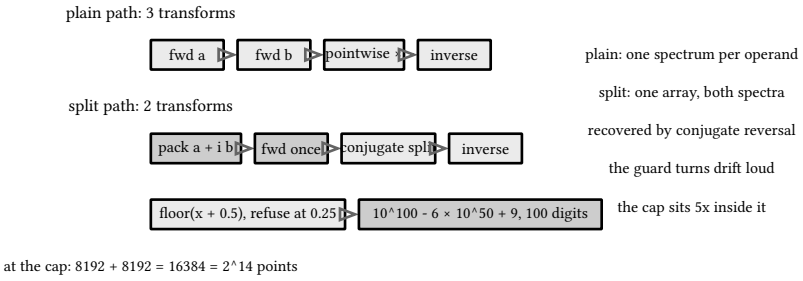


figure 237: the stressor through both paths, three transforms against two, the guard at the shared exit

The same 100 digits off both paths is the pinned stressor, and the listings below run plain and split in seven languages.

```c
static int fft_conv(const unsigned int *a, int la, const unsigned int *b,
                    int lb, double *cols, int split) {
  if (la == 0 || lb == 0)
    return -3;
  int len = 1;
  while (len < la + lb)
    len <<= 1;
  if (len > FFT_CAP)
    return -1;
  if (split) {
    for (int i = 0; i < len; i++) {
      ar[i] = i < la ? (double)a[i] : 0.0;
      ai[i] = i < lb ? (double)b[i] : 0.0;
    }
    fft(ar, ai, len, 0);
    // recover both spectra from the packed transform before overwriting
    // anything: bhat[k] = -i (phat[k] - conj(phat[n-k])) / 2, then
    // ahat[k] = (phat[k] + conj(phat[n-k])) / 2 staged through a temp
    for (int k = 0; k < len; k++) {
      int k2 = (len - k) & (len - 1);
      double dr = ar[k] - ar[k2], di = ai[k] + ai[k2];
      br[k] = di / 2;
      bi[k] = -dr / 2;
      tr[k] = (ar[k] + ar[k2]) / 2;
      ti[k] = (ai[k] - ai[k2]) / 2;
    }
    memcpy(ar, tr, (size_t)len * sizeof(double));
    memcpy(ai, ti, (size_t)len * sizeof(double));
    for (int k = 0; k < len; k++) {
      double cr = ar[k] * br[k] - ai[k] * bi[k];
      double ci = ar[k] * bi[k] + ai[k] * br[k];
      ar[k] = cr;
      ai[k] = ci;
    }
    fft(ar, ai, len, 1);
    for (int i = 0; i < la + lb; i++)
      cols[i] = ar[i];
```

listing 936: c, the split branch, pack, one forward, spectra by conjugate reversal, one inverse (source: dsa/samples-c/src/Ch29/fft.c, lines 83-119)

```
func fftMulLimbs(a, b []int64, plain bool) ([]int64, error) {
  if len(a) == 0 || len(b) == 0 {
    return nil, nil
  }
  size := nextPow2(len(a) + len(b))
  if size > fftCap {
    return nil, fmt.Errorf(
      "fft: transform length %d past the 2^14 cap", size)
  }
  out := make([]int64, len(a)+len(b)-1)
  if plain {
    fa := packComplex(a, size)
    fb := packComplex(b, size)
    dft(fa, false)
    dft(fb, false)
    for i := range fa {
      fa[i] *= fb[i]
    }
    dft(fa, true)
    return roundOut(fa, out)
  }
  p := make([]complex128, size)
  for i := 0; i < len(a) || i < len(b); i++ {
    var av, bv float64
    if i < len(a) {
      av = float64(a[i])
    }
    if i < len(b) {
      bv = float64(b[i])
    }
    p[i] = complex(av, bv)
  }
  dft(p, false)
  // work on a copy: the conjugate indices reread P's spectrum
  phat := make([]complex128, size)
  copy(phat, p)
  for k := 0; k < size; k++ {
    mirror := phat[(size-k)%size]
    conj := complex(real(mirror), -imag(mirror))
    ah := (phat[k] + conj) / 2
    bh := (phat[k] - conj) / complex(0, 2) // -i * (P-conj)/2
    p[k] = ah * bh
  }
  dft(p, true)
  return roundOut(p, out)
}
```

listing 937: go, both paths in one driver, the mirror index spelled out (source: dsa/samples-go/ch29/fft.go, lines 76-121)

```
    if (split) {
      for (int i = 0; i < len; i++) {
        ar[i] = i < la ? a[i] : 0.0;
        ai[i] = i < lb ? b[i] : 0.0;
      }
      fft(ar, ai, len, false);
      // recover both spectra from the packed transform before
      // overwriting anything: bhat[k] = -i (phat[k] - conj(phat[n-k]))/2,
      // then ahat[k] = (phat[k] + conj(phat[n-k]))/2 staged through temp
      for (int k = 0; k < len; k++) {
        int k2 = (len - k) & (len - 1);
        double dr = ar[k] - ar[k2], di = ai[k] + ai[k2];
        br[k] = di / 2;
        bi[k] = -dr / 2;
        tr[k] = (ar[k] + ar[k2]) / 2;
        ti[k] = (ai[k] - ai[k2]) / 2;
      }
      System.arraycopy(tr, 0, ar, 0, len);
      System.arraycopy(ti, 0, ai, 0, len);
      for (int k = 0; k < len; k++) {
        double cr = ar[k] * br[k] - ai[k] * bi[k];
        double ci = ar[k] * bi[k] + ai[k] * br[k];
        ar[k] = cr;
        ai[k] = ci;
      }
      fft(ar, ai, len, true);
      for (int i = 0; i < la + lb; i++)
        cols[i] = ar[i];
    } else {
      for (int i = 0; i < len; i++) {
        ar[i] = i < la ? a[i] : 0.0;
        ai[i] = 0.0;
        br[i] = i < lb ? b[i] : 0.0;
        bi[i] = 0.0;
      }
      fft(ar, ai, len, false);
      fft(br, bi, len, false);
      for (int k = 0; k < len; k++) {
        double cr = ar[k] * br[k] - ai[k] * bi[k];
        ai[k] = ar[k] * bi[k] + ai[k] * br[k];
        ar[k] = cr;
      }
      fft(ar, ai, len, true);
      for (int i = 0; i < la + lb; i++)
        cols[i] = ar[i];
    }
    return 0;
```

listing 938: java, the split branch packs a + i b, spectra by conjugate reversal, plain in the else (source: dsa/samples-java/src/Ch29/Fft.java, lines 88-134)

```
    private static long[] ConvolvePlain(long[] a, long[] b, int n)
    {
        Complex[] fa = ToComplex(a, n);
        Complex[] fb = ToComplex(b, n);
        Transform(fa, invert: false);
        Transform(fb, invert: false);
        for (int i = 0; i < n; i++)
            fa[i] *= fb[i];
        Transform(fa, invert: true);
        return RoundAll(fa, a.Length + b.Length - 1);
    }

    /// <summary>The headline: pack P = a + i b, one forward transform, recover both
    /// spectra by conjugate reversal, pointwise, one inverse.</summary>
    private static long[] ConvolveSplit(long[] a, long[] b, int n)
    {
        var p = new Complex[n];
        for (int i = 0; i < n; i++)
            p[i] = new Complex(i < a.Length ? a[i] : 0, i < b.Length ? b[i] : 0);
        Transform(p, invert: false);
        var c = new Complex[n];
        for (int i = 0; i < n; i++)
        {
            Complex pk = p[i];
            Complex pkc = p[(n - i) % n]; // cyclic conjugate partner
            Complex aHat = (pk + Complex.Conjugate(pkc)) / 2;
            Complex bHat = new Complex(0, -1) * (pk - Complex.Conjugate(pkc)) / 2;
            c[i] = aHat * bHat;
        }
        Transform(c, invert: true);
        return RoundAll(c, a.Length + b.Length - 1);
    }
```

listing 939: c#, plain and split side by side, both spectra from conjugate partners (source: dsa/samples/src/Ch29/Fft.cs, lines 62-93)

```
function convolvePlain(a, b, need) {
  const ar = [...a];
  const ai = new Array(need).fill(0);
  while (ar.length < need) ar.push(0);
  const br = [...b];
  const bi = new Array(need).fill(0);
  while (br.length < need) br.push(0);
  fft(ar, ai, false);
  fft(br, bi, false);
  const cr = new Array(need);
  const ci = new Array(need);
  for (let i = 0; i < need; i++) {
    cr[i] = ar[i] * br[i] - ai[i] * bi[i];
    ci[i] = ar[i] * bi[i] + ai[i] * br[i];
  }
  fft(cr, ci, true);
  return cr;
}

function convolveSplit(a, b, need) {
  const pr = [...a];
  while (pr.length < need) pr.push(0);
  const pi = [...b];
  while (pi.length < need) pi.push(0);
  fft(pr, pi, false);
  const cr = new Array(need);
  const ci = new Array(need);
  for (let k = 0; k < need; k++) {
    const kn = (need - k) % need;
    // Ahat[k] = (Phat[k] + conj(Phat[n-k])) / 2
    const arh = (pr[k] + pr[kn]) / 2;
    const aih = (pi[k] - pi[kn]) / 2;
    // Bhat[k] = -i (Phat[k] - conj(Phat[n-k])) / 2
    const dr = pr[k] - pr[kn];
    const di = pi[k] + pi[kn];
    const brh = di / 2;
    const bih = -dr / 2;
    cr[k] = arh * brh - aih * bih;
    ci[k] = arh * bih + aih * brh;
  }
  fft(cr, ci, true);
  return cr;
}
```

listing 940: javascript, plain then split, the two recovery formulas on doubles (source: dsa/samples-js/src/ch29-fft.mjs, lines 70-112)

```
def mul_plain(x, y):
    # two forward transforms, pointwise, one inverse
    la, lb = len(x), len(y)
    size = transform_size(la + lb - 1)
    fa = [complex(v, 0) for v in x] + [0j] * (size - la)
    fb = [complex(v, 0) for v in y] + [0j] * (size - lb)
    fft(fa, False)
    fft(fb, False)
    for k in range(size):
        fa[k] *= fb[k]
    fft(fa, True)
    return [round_guard(c.real) for c in fa[: la + lb - 1]]


def mul_split(x, y):
    # one forward transform of P[j] = a[j] + i*b[j], spectra by
    # conjugate reversal, pointwise, one inverse
    la, lb = len(x), len(y)
    size = transform_size(la + lb - 1)
    p = [complex(x[i] if i < la else 0, y[i] if i < lb else 0) for i in range(size)]
    fft(p, False)
    ahat = [0j] * size
    bhat = [0j] * size
    for k in range(size):
        pk = p[k]
        pc = p[(size - k) % size].conjugate()
        ahat[k] = (pk + pc) / 2
        bhat[k] = complex(pk.imag - pc.imag, pc.real - pk.real) / 2
    for k in range(size):
        p[k] = ahat[k] * bhat[k]
    fft(p, True)
    return [round_guard(c.real) for c in p[: la + lb - 1]]
```

listing 941: python, mul plain and mul split, guarded rounding at the end (source: dsa/samples-py/src/Ch29/fft.py, lines 83-114)

```lua
-- plain path: two forward transforms, pointwise, one inverse
local function mul_plain(a, b)
  local n = nextpow2(#a + #b)
  if n > CAP then error("fft transform cap 2^14 exceeded") end
  local ra, ia, rb, ib = {}, {}, {}, {}
  for i = 1, n do
    ra[i] = a[i] or 0; ia[i] = 0
    rb[i] = b[i] or 0; ib[i] = 0
  end
  fft(ra, ia, n, false)
  fft(rb, ib, n, false)
  for i = 1, n do
    local r = ra[i] * rb[i] - ia[i] * ib[i]
    local m = ra[i] * ib[i] + ia[i] * rb[i]
    ra[i], ia[i] = r, m
  end
  fft(ra, ia, n, true)
  return ra
end

-- split path: pack a + i*b, one forward, spectra by conjugate reversal.
-- the product spectrum lands in fresh arrays: writing P[k] in place
-- would corrupt P[n-k] before its turn as a partner
local function mul_split(a, b)
  local n = nextpow2(#a + #b)
  if n > CAP then error("fft transform cap 2^14 exceeded") end
  local re, im = {}, {}
  for i = 1, n do re[i] = a[i] or 0; im[i] = b[i] or 0 end
  fft(re, im, n, false)
  local cr, ci = {}, {}
  for k = 0, n - 1 do
    local kk = (n - k) % n -- conjugate partner index
    local pr, pi = re[k + 1], im[k + 1]
    local qr, qi = re[kk + 1], -im[kk + 1]
    local ar = (pr + qr) / 2 -- Ahat[k]
    local ai = (pi + qi) / 2
    local sr = (pr - qr) / 2 -- s = (Phat[k] - conj(Phat[n-k]))/2
    local si = (pi - qi) / 2
    local br = si -- Bhat[k] = -i*s = (si, -sr)
    local bi = -sr
    cr[k + 1] = ar * br - ai * bi
    ci[k + 1] = ar * bi + ai * br
  end
  fft(cr, ci, n, true)
  return cr
end
```

listing 942: lua, plain and split on parallel re and im tables (source: dsa/samples-lua/ch29_fft.lua, lines 73-118)

Fixtures pin the policy numbers as units first: exact integers round unchanged, 2.24 rounds down and 2.76 rounds up, and 0.5, 2.25, and 2.75 all refuse, the last because a quarter-integer distance is exactly the guard. The stressor runs on both paths and they agree digit for digit: (10^50 - 3) squared is 10^100 - 6 times 10^50 + 9, all four sign combinations pin, and zero operands return zero. At the cap exactly, all-9999 base-1e4 operands of 8192 limbs each, 32768 digits with la plus lb equal to 2^14, square to 10^65536 - 2 * 10^32768 + 1 by identity, 65536 digits asserted by count and pattern, and one doubling past the cap refuses loudly on both paths. JavaScript, Java, and Lua ride plain double arrays and tables the whole way, which is the point, the cap and the guard are what make doubles safe here, not a wider type.

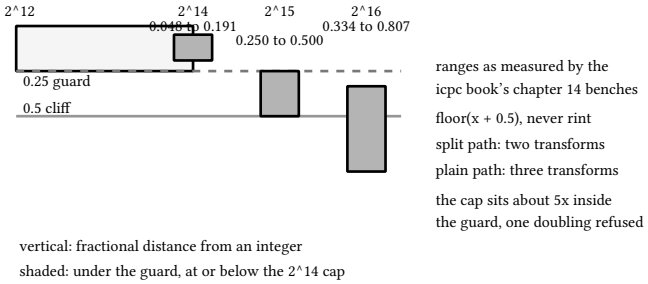


figure 238: measured fractional drift at 2^14, 2^15, 2^16 against the 0.25 guard and the 0.5 rounding cliff, the safe range shaded under the cap

29.5. long division, knuth algorithm d

Division is the one operation where the schoolbook you learned by hand is already the right algorithm, and knuth's algorithm D is that algorithm made careful. A one-limb divisor takes the fast path, a 64-over-32 division walking down the limbs with the remainder carried each step. Otherwise, on magnitudes: normalize both operands by s = B over (v top + 1) so the divisor's top limb reaches B over 2, which bounds the estimate, and give the dividend one sentinel limb of headroom. Each quotient limb comes from the two-limbs-over-one estimate, the top two window limbs over the divisor top, a numerator under 1e18 that fits plain 64-bit division with no 128-bit help anywhere, C bans `__uint128_t` by the icpc book's grep rule. Clamp the estimate to B - 1, refine it against the divisor's second limb at most twice, multiply and subtract down the window, and when the top goes negative add the divisor back once, the branch decades of implementations get wrong. The remainder descales by s, and the signed wrapper truncates toward zero with the remainder carrying the dividend's sign.

The dry run: the fixture is 10^30 over 23 on the one-limb fast path, asserted by the C# suite beside its 2^100 over 97 anchor, the siblings pinning the same quotients.

1. 10^30 parses to four limbs reading 0, 0, 0, 1000 least significant first, the divisor 23 is a single limb, so the fast path walks the limbs top-down carrying the remainder.
2. Top limb: cur = 0 × 1e9 + 1000 = 1000, and 43 × 23 = 989, so the quotient limb is 43 with remainder 1000 - 989 = 11.
3. Next limb: cur = 11 × 1e9 + 0 = 11000000000, and 478260869 × 23 = 10999999987, remainder 13.
4. Next limb: cur = 13 × 1e9 = 13000000000, and 565217391 × 23 = 12999999993, remainder 7.
5. Last limb: cur = 7 × 1e9 = 7000000000, and 304347826 × 23 = 6999999998, remainder 2, the pinned r.
6. The quotient prints top-down, 43 bare then three limbs padded to nine, 43478260869565217391304347826, and q × 23 + 2 rebuilds 10^30, the invariant every sign cell asserts.

limb	cur	q limb	q × 23	r
1000	0 × 1e9 + 1000	43	989	11
0	11000000000	478260869	10999999987	13
0	13000000000	565217391	12999999993	7
0	7000000000	304347826	6999999998	2

43478260869565217391304347826 with remainder 2 is the pinned pair, and the listings below divide in seven languages.

```
    for (int j = m - nn; j ≥ 0; j--) {
      unsigned long long num =
          (unsigned long long)un[j + nn] * BASE + un[j + nn - 1];
      unsigned long long qhat = num / vn[nn - 1];
      if (qhat > BASE - 1)
        qhat = BASE - 1;
      while (qhat * vn[nn - 2] >
             (num - qhat * vn[nn - 1]) * BASE + un[j + nn - 2])
        qhat--;
      // mulsub: window -= qhat * vn
      unsigned long long c2 = 0;
      long long borrow = 0;
      for (int i = 0; i < nn; i++) {
        unsigned long long p = qhat * vn[i] + c2;
        c2 = p / BASE;
        long long sub = (long long)(p % BASE) + borrow;
        long long t = (long long)un[j + i] - sub;
        if (t < 0) {
          t += (long long)BASE;
          borrow = 1;
        } else
          borrow = 0;
        un[j + i] = (unsigned int)t;
      }
      long long top = (long long)un[j + nn] - (long long)c2 - borrow;
      if (top < 0) { // the single add-back
        qhat--;
        unsigned long long c3 = 0;
        for (int i = 0; i < nn; i++) {
          unsigned long long cur = (unsigned long long)un[j + i] + vn[i] + c3;
          c3 = cur / BASE;
          un[j + i] = (unsigned int)(cur % BASE);
        }
        top += (long long)c3 + (long long)BASE;
      }
      un[j + nn] = (unsigned int)top;
      q->limb[j] = (unsigned int)qhat;
    }
```

listing 943: c, the quotient loop, estimate, clamp, refine, mulsub, the rare add-back (source: dsa/samples-c/src/Ch29/divmod.c, lines 200-237)

```
  for j := m; j ≥ 0; j-- {
    num := un[j+n]*bigBase + un[j+n-1]
    qhat := num / vn[n-1]
    if qhat > bigBase-1 {
      qhat = bigBase - 1
    }
    for n ≥ 2 && qhat*vn[n-2] >
      (num-qhat*vn[n-1])*bigBase+un[j+n-2] {
      qhat--
    }
    borrow, mulCarry := int64(0), int64(0)
    for i := 0; i < n; i++ {
      p := qhat*vn[i] + mulCarry
      mulCarry = p / bigBase
      t := un[i+j] - borrow - p%bigBase
      if t < 0 {
        t += bigBase
        borrow = 1
      } else {
        borrow = 0
      }
      un[i+j] = t
    }
    t := un[j+n] - borrow - mulCarry
    if t < 0 {
      t += bigBase
      borrow = 1
    } else {
      borrow = 0
    }
    un[j+n] = t
    q[j] = qhat
    if borrow ≠ 0 { // the rare add-back: qhat was one too big
      q[j]--
      c := int64(0)
      for i := 0; i < n; i++ {
        cur := un[i+j] + vn[i] + c
        un[i+j], c = cur%bigBase, cur/bigBase
      }
      un[j+n] = (un[j+n] + c) % bigBase
    }
  }
  return q, descale(un[:n], s)
```

listing 944: go, the loop with the add-back branch restoring the window (source: dsa/samples-go/ch29/divmod.go, lines 65-107)

```
    for (int j = m - nn; j ≥ 0; j--) {
      long num = un[j + nn] * BASE + un[j + nn - 1];
      long qhat = num / vn[nn - 1];
      if (qhat > BASE - 1)
        qhat = BASE - 1;
      while (qhat * vn[nn - 2] > (num - qhat * vn[nn - 1]) * BASE + un[j + nn - 2])
        qhat--;
      // mulsub: window -= qhat * vn
      long c2 = 0;
      long borrow = 0;
      for (int i = 0; i < nn; i++) {
        long p = qhat * vn[i] + c2;
        c2 = p / BASE;
        long sub = p % BASE + borrow;
        long t = un[j + i] - sub;
        if (t < 0) {
          t += BASE;
          borrow = 1;
        } else
          borrow = 0;
        un[j + i] = (int) t;
      }
      long top = un[j + nn] - c2 - borrow;
      if (top < 0) { // the single add-back
        qhat--;
        long c3 = 0;
        for (int i = 0; i < nn; i++) {
          long cur = (long) un[j + i] + vn[i] + c3;
          c3 = cur / BASE;
          un[j + i] = (int) (cur % BASE);
        }
        top += c3 + BASE;
      }
      un[j + nn] = (int) top;
      q.limb[j] = (int) qhat;
    }
```

listing 945: java, the knuth D loop, estimate, clamp, refine, mulsub, the rare add-back (source: dsa/samples-java/src/Ch29/Divmod.java, lines 187-222)

```
        for (int j = m; j ≥ 0; j--)
        {
            long num = un[j + n] * b + un[j + n - 1]; // < 1e18, no 128-bit needed
            long qhat = num / vn[n - 1];
            if (qhat > b - 1)
                qhat = b - 1;
            long rhat = num - qhat * vn[n - 1];
            // refine against the second limb, at most two decrements ever fire
            while (rhat < b && qhat * vn[n - 2] > rhat * b + un[j + n - 2])
            {
                qhat--;
                rhat += vn[n - 1];
            }

            // mulsub: un[j..j+n) -= qhat * vn, floor-dividing so limbs stay in [0, b)
            long borrow = 0;
            for (int i = 0; i < n; i++)
            {
                long cur = un[i + j] - qhat * vn[i] - borrow;
                long f = cur < 0 ? (cur - b + 1) / b : cur / b; // floor division
                un[i + j] = cur - f * b;
                borrow = -f;
            }
            long top = un[j + n] - borrow;
            if (top < 0)
            {
                qhat--; // the rare add-back: qhat was one too large
                long carry = 0;
                for (int i = 0; i < n; i++)
                {
                    long cur = un[i + j] + vn[i] + carry;
                    un[i + j] = cur % b;
                    carry = cur / b;
                }
                top += carry;
            }
            un[j + n] = top;
            q[j] = qhat;
        }
```

listing 946: c#, the knuth D loop, rhat refinement, floor-dividing mulsub, add-back (source: dsa/samples/src/Ch29/DivMod.cs, lines 49-87)

```
  for (let j = m; j ≥ 0; j--) {
    const num = un[j + n] * BASE + un[j + n - 1];
    let qhat = num / vnTop;
    let rhat = num % vnTop;
    // clamp, then refine against v[n-2] at most twice
    while (
      qhat ≥ BASE ||
      qhat * vn[n - 2] > rhat * BASE + un[j + n - 2]
    ) {
      qhat -= 1n;
      rhat += vnTop;
      if (rhat ≥ BASE) break;
    }
    // multiply and subtract, borrow chain
    let carry = 0n;
    let borrow = 0n;
    for (let i = 0; i < n; i++) {
      const p = qhat * vn[i] + carry;
      carry = p / BASE;
      let cur = un[j + i] - (p % BASE) - borrow;
      if (cur < 0n) {
        cur += BASE;
        borrow = 1n;
      } else {
        borrow = 0n;
      }
      un[j + i] = cur;
    }
    const top = un[j + n] - carry - borrow;
    if (top < 0n) {
      // the rare add-back
      qhat -= 1n;
      let c2 = 0n;
      for (let i = 0; i < n; i++) {
        const t = un[j + i] + vn[i] + c2;
        un[j + i] = t % BASE;
        c2 = t / BASE;
      }
      un[j + n] = top + c2; // lands back in [0, B)
    } else {
      un[j + n] = top;
    }
    q[j] = Number(qhat);
  }
```

listing 947: javascript, estimate and mulsub through BigInt, add-back when the top goes negative (source: dsa/samples-js/src/ch29-divmod.mjs, lines 61-104)

```
    for j in range(m, -1, -1):
        num = un[j + n] * B + un[j + n - 1]  # < 1e18 inside 64 bits
        qhat = min(num // vn[-1], B - 1)
        rhat = num - qhat * vn[-1]
        # refine against v[n-2]: the loop runs at most twice
        while qhat * vn[-2] > rhat * B + un[j + n - 2]:
            qhat -= 1
            rhat += vn[-1]
        borrow = 0  # mulsub down the window
        for i in range(n):
            sub = qhat * vn[i] + borrow
            t = un[i + j] - sub % B
            un[i + j] = t % B
            borrow = sub // B + (1 if t < 0 else 0)
        un[j + n] -= borrow
        if un[j + n] < 0:
            # the rare add-back: qhat was one too large
            un[j + n] += B
            qhat -= 1
            carry = 0
            for i in range(n):
                t = un[i + j] + vn[i] + carry
                un[i + j] = t % B
                carry = t // B
            un[j + n] += carry - B
        q[j] = qhat
    while q and q[-1] == 0:
        q.pop()
    rem = descale(un[:n], s)
    while rem and rem[-1] == 0:
        rem.pop()
    return q, rem
```

listing 948: python, the loop, the add-back, then the descale (source: dsa/samples-py/src/Ch29/divmod.py, lines 89-120)

```
  for j = m, 0, -1 do
    local num = uu[j + n + 1] * B + uu[j + n] -- < 1e18, plain 64/64
    local qhat = num // vv[n]
    local rhat = num % vv[n]
    if qhat ≥ B then qhat = B - 1; rhat = num - qhat * vv[n] end
    while qhat * vv[n - 1] > rhat * B + uu[j + n - 1] do
      qhat = qhat - 1
      rhat = rhat + vv[n]
      if rhat ≥ B then break end -- at most two refinements
    end
    -- mulsub with borrow, then the rare add-back
    local borrow, carry2 = 0, 0
    for i = 1, n do
      local p = qhat * vv[i] + carry2
      carry2 = p // B
      local sub = uu[j + i] - (p % B) - borrow
      if sub < 0 then sub = sub + B; borrow = 1 else borrow = 0 end
      uu[j + i] = sub
    end
    local top = uu[j + n + 1] - carry2 - borrow
    local back = 0
    if top < 0 then -- the add-back branch
      top = top + B
      qhat = qhat - 1
      for i = 1, n do
        local x = uu[j + i] + vv[i] + back
        uu[j + i] = x % B
        back = x // B
      end
    end
    uu[j + n + 1] = (top + back) % B -- the carry cancels the borrow
    q[j + 1] = qhat
  end
```

listing 949: lua, the loop over the sentinel-topped window, add-back last (source: dsa/samples-lua/ch29_divmod.lua, lines 125-157)

Fixtures pin the anchors: 2^100 over 97 gives quotient 1306856288895081857213096088O with remainder 16, and 10^30 over 23 gives quotient 434782608695652173913043478 26 with remainder 2. The sign matrix runs plus and minus 10^30 + 3 against plus and minus 10^15

1. 7, quotient 999999999999993 signed by the product of operand signs,

remainder 52 carrying the dividend’s sign, and every cell asserts the invariants, q times b plus r equals a with the remainder’s magnitude below the divisor’s. The add-back families are the ones the icpc book’s chapter 14 C# stream discovered by exhaustive base-10 scan, cross-named here: 2 B^2 over B^2 + 1 gives quotient 1 with remainder 99999999999999999, and 3 B^2 over B^2 + 1 gives quotient 2 with remainder 99999999999999998, both forcing the branch. A divisor larger in magnitude than the dividend gives quotient 0 and remainder the dividend, a zero dividend divides cleanly, and a zero divisor refuses in every channel.

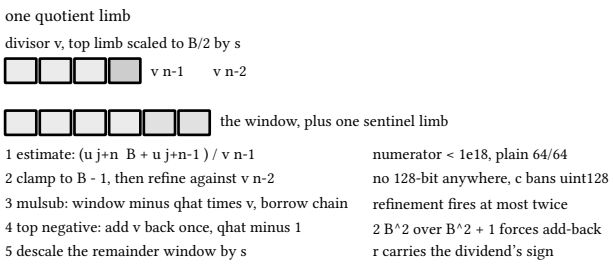


figure 239: one quotient digit of algorithm D, normalize, estimate two-over-one, refine, mulsub down the window, the rare add-back

29.6. polynomials over one prime

Coefficient vectors over F_998244353 close the chapter, because the transform prime makes the polynomial ops ride the machinery above. Multiply picks its road by size, schoolbook for the short and sparse, the 29.3 transform once both operands reach 32 terms, dense and long. Evaluation is horner at an arbitrary point, folding the coefficients from the top. Interpolation is lagrange at n + 1 distinct points, no more than 64 in the fixtures, built from prefix and suffix products, the basis polynomial that vanishes at every node but x i is the product of (x - x j) over j below i times the product over j above i, and each basis scales by the inverse of its denominator, O(n^2) all in. Remainder mod a small polynomial is schoolbook long division, killing the leading term until the degree drops below the divisor's.

The dry run: the fixtures are the eighth binomial row and the quadratic interpolation family, asserted by the C# suite with the sibling suites on the same rows.

1. The row starts at 1 and multiplies (1 + x) eight times, each multiply adding neighbors: the seventh row 1, 7, 21, 35, 35, 21, 7, 1 gives 1 + 7 = 8, 7 + 21 = 28, 21 + 35 = 56, 35 + 35 = 70, mirroring back down.
2. The eighth row reads 1, 8, 28, 56, 70, 56, 28, 8, 1, all below the modulus, exact over the integers.
3. Horner at 10 folds from the top: 1, then 18, 208, 2136, 21430, 214356, 2143588, 21435888, 214358881, which is 11^8.
4. The quadratic family samples x^2 + 3x + 5 at 0 through 4 for 5, 9, 15, 23, 33: first differences 4, 6, 8, 10, second difference 2 throughout.
5. Lagrange over the five points returns the coefficients [5, 3, 1], and horner at 4 checks the round trip, 1 × 4 + 3 = 7 then 7 × 4 + 5 = 33.
6. The remainder edge reads the algebra: x^3 = 1 mod x^2 + x + 1 and 100 = 3 × 33 + 1, so x^100 + 1 reduces to x + 1.

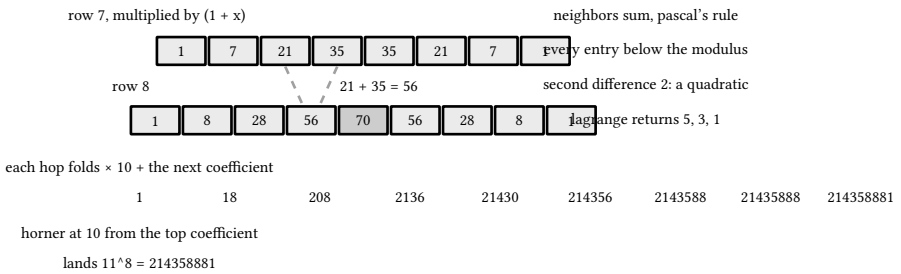


figure 240: the row build adding neighbors into the eighth row, the horner chain at 10 beside it

The eighth row and the 214358881 are the pinned pair, and the listings below fold all of it over one prime in seven languages.

```
static void mul_lin(unsigned long long *p, int *pl, unsigned long long r) {
  static unsigned long long q[PMAX];
  int nl = *pl + 1;
  for (int t = 0; t < nl; t++) {
    unsigned long long v = t > 0 ? p[t - 1] : 0;
    if (t < *pl)
      v = (v + P - r * p[t] % P) % P;
    q[t] = v;
  }
  memcpy(p, q, (size_t)nl * sizeof(unsigned long long));
  *pl = nl;
}

// lagrange at k distinct points by prefix/suffix basis products
static void interp(const unsigned long long *xs, const unsigned long long *ys,
                   int k, unsigned long long *out) {
  static unsigned long long pre[PMAX], suf[PMAX], basis[PMAX];
  int prel, sufl;
  for (int i = 0; i < k; i++)
    out[i] = 0;
  for (int i = 0; i < k; i++) {
    pre[0] = 1;
    prel = 1;
    for (int j = 0; j < i; j++)
      mul_lin(pre, &prel, xs[j]);
    suf[0] = 1;
    sufl = 1;
    for (int j = i + 1; j < k; j++)
      mul_lin(suf, &sufl, xs[j]);
    int bl = pmul_school(pre, prel, suf, sufl, basis);
    unsigned long long den = 1;
    for (int j = 0; j < k; j++)
      if (j != i)
        den = den * ((xs[i] + P - xs[j]) % P) % P;
    unsigned long long w = ys[i] % P * modpow(den, P - 2) % P;
    for (int t = 0; t < bl; t++)
      out[t] = (out[t] + w * basis[t]) % P;
  }
}
```

listing 950: c, mul_lin builds (x - r), interp folds prefix times suffix over the denominator (source: dsa/samples-c/src/Ch29/polyops.c, lines 125-163)

```
func Interpolate(xs, ys []int64) []int64 {
  n := len(xs)
  pre := make([][]int64, n+1)
  suf := make([][]int64, n+1)
  pre[0] = []int64{1}
  for i := 0; i < n; i++ {
    pre[i+1] = polyMulLin(pre[i], xs[i]%polyMod)
  }
  suf[n] = []int64{1}
  for i := n - 1; i ≥ 0; i-- {
    suf[i] = polyMulLin(suf[i+1], xs[i]%polyMod)
  }
  out := make([]int64, n)
  for i := 0; i < n; i++ {
    basis := PolyMul(pre[i], suf[i+1])
    den := int64(1)
    for j := 0; j < n; j++ {
      if j ≠ i {
        den = den * ((xs[i] - xs[j]%polyMod + 2*polyMod) % polyMod) %
          polyMod
      }
    }
    scale := ys[i] % polyMod *
      int64(powMod(uint64(den), uint64(polyMod-2), uint64(polyMod))) %
      polyMod
    for k, c := range basis {
      out[k] = (out[k] + c*scale) % polyMod
    }
  }
  return out
}
```

listing 951: go, Interpolate with polyMulLin and the basis product (source: dsa/samples-go/ch29/polyops.go, lines 59-89)

```
  static void mulLin(long[] p, int[] pl, long r) {
    long[] q = new long[PMAX];
    int nl = pl[0] + 1;
    for (int t = 0; t < nl; t++) {
      long v = t > 0 ? p[t - 1] : 0;
      if (t < pl[0])
        v = (v + P - r * p[t] % P) % P;
      q[t] = v;
    }
    System.arraycopy(q, 0, p, 0, nl);
    pl[0] = nl;
  }

  // lagrange at k distinct points by prefix/suffix basis products
  static void interp(long[] xs, long[] ys, int k, long[] out) {
    long[] pre = new long[PMAX], suf = new long[PMAX], basis = new long[PMAX];
    int[] prel = new int[1], sufl = new int[1];
    for (int i = 0; i < k; i++)
      out[i] = 0;
    for (int i = 0; i < k; i++) {
      pre[0] = 1;
      prel[0] = 1;
      for (int j = 0; j < i; j++)
        mulLin(pre, prel, xs[j]);
      suf[0] = 1;
      sufl[0] = 1;
      for (int j = i + 1; j < k; j++)
        mulLin(suf, sufl, xs[j]);
      int bl = pmulSchool(pre, prel[0], suf, sufl[0], basis);
      long den = 1;
      for (int j = 0; j < k; j++)
        if (j != i)
          den = den * ((xs[i] + P - xs[j]) % P) % P;
      long w = ys[i] % P * modpow(den, P - 2) % P;
      for (int t = 0; t < bl; t++)
        out[t] = (out[t] + w * basis[t]) % P;
    }
  }
```

listing 952: java, mulLin builds (x - r), interp folds prefix times suffix over the denominator inverse (source: dsa/samples-java/src/Ch29/Polyops.java, lines 116-153)

```
    public static long[] Interpolate(long[] xs, long[] ys)
    {
        int n = xs.Length;
        // prefix[i] = prod_{k < i} (x - xs[k]), suffix[i] = prod_{k ≥ i} (x - xs[k])
        var prefix = new long[n + 1][];
        var suffix = new long[n + 1][];
        prefix[0] = new long[] { 1 };
        suffix[n] = new long[] { 1 };
        for (int i = 0; i < n; i++)
            prefix[i + 1] = SchoolMultiply(prefix[i], Lin(xs[i]));
        for (int i = n - 1; i ≥ 0; i--)
            suffix[i] = SchoolMultiply(suffix[i + 1], Lin(xs[i]));
        long[] result = new long[n];
        for (int i = 0; i < n; i++)
        {
            long[] num = SchoolMultiply(prefix[i], suffix[i + 1]); // vanishes at every node but i
            long denom = 1;
            for (int j = 0; j < n; j++)
                if (j ≠ i)
                    denom = denom * ((xs[i] - xs[j]) % P + P) % P;
            long scale = ys[i] % P * NumTheory.ModInv(denom, P) % P;
            for (int k = 0; k < n && k < num.Length; k++)
                result[k] = (result[k] + num[k] * scale) % P;
        }
        return Trim(result);
    }
```

listing 953: c#, Interpolate over prefix and suffix products, the vanishing basis (source: dsa/samples/src/Ch29/PolyOps.cs, lines 39-64)

```
function linMul(poly, x) {
  const xv = BigInt(x) % P;
  const out = new Array(poly.length + 1).fill(0n);
  for (let i = 0; i < poly.length; i++) {
    out[i + 1] = (out[i + 1] + poly[i]) % P;
    out[i] = (out[i] + P - (poly[i] * xv) % P) % P;
  }
  return out;
}

// lagrange at [{ x, y }]: prefix/suffix products of (X - x_j)
export function interpolate(points) {
  const n = points.length;
  const pre = [[1n]];
  for (let i = 0; i < n; i++) pre.push(linMul(pre[i], points[i].x));
  const suf = new Array(n + 1);
  suf[n] = [1n];
  for (let i = n - 1; i ≥ 0; i--) suf[i] = linMul(suf[i + 1], points[i].x);
  const out = new Array(n).fill(0n);
  for (let i = 0; i < n; i++) {
    let denom = 1n;
    for (let j = 0; j < n; j++) {
      if (j === i) continue;
      const d = points[i].x - points[j].x;
      denom = (denom * (((BigInt(d) % P) + P) % P)) % P;
    }
    const w = (BigInt(points[i].y) * inv(denom)) % P;
    const num = polyMul(pre[i], suf[i + 1]);
    for (let k = 0; k < Math.min(num.length, n); k++) {
      out[k] = (out[k] + num[k] * w) % P;
    }
  }
  while (out.length > 0 && out[out.length - 1] === 0n) out.pop();
  return out;
}
```

listing 954: javascript, linMul then the lagrange fold, residues on BigInt (source: dsa/samples-js/src/ch29-polyops.mjs, lines 99-133)

```
def lagrange(xs, ys):
    # n + 1 distinct points; basis i is the prefix-suffix product of
    # (x - x_j) over j ≠ i, scaled by the inverse denominator
    n = len(xs)
    out = [0] * n
    for i in range(n):
        pre = [1]  # prod over j < i of (x - x_j)
        for j in range(i):
            pre = schoolbook(pre, [(-xs[j]) % P, 1])
        suf = [1]  # prod over j > i of (x - x_j)
        for j in range(i + 1, n):
            suf = schoolbook(suf, [(-xs[j]) % P, 1])
        basis = schoolbook(pre, suf)
        den = 1
        for j in range(n):
            if j ≠ i:
                den = den * (xs[i] - xs[j]) % P
        scale = ys[i] * pow(den, P - 2, P) % P
        for d, c in enumerate(basis):
            out[d] = (out[d] + scale * c) % P
    while len(out) > 1 and out[-1] == 0:
        out.pop()
    return out
```

listing 955: python, lagrange by prefix and suffix products, O(n^2) (source: dsa/samples-py/src/Ch29/polyops.py, lines 92-114)

```lua
local function lagrange(xs, ys)
  local n = #xs
  local coef = {}
  for i = 1, n do
    -- numerator (x - x_j) over j ≠ i: the prefix below i times the
    -- suffix above i, each built by incremental multiplication
    local num = {1}
    for j = 1, n do
      if j ~= i then
        local nn = {} -- num *= (x - xs[j]), lowest degree first
        for k = 1, #num + 1 do
          local v = 0
          if k > 1 then v = (v + num[k - 1]) % P end
          if k ≤ #num then v = (v - xs[j] * num[k]) % P end
          nn[k] = v
        end
        num = nn
      end
    end
    local den = 1
    for j = 1, n do
      if j ~= i then den = den * (xs[i] - xs[j]) % P end
    end
    local scale = ys[i] * modpow(den, P - 2) % P
    for k = 1, #num do
      coef[k] = ((coef[k] or 0) + num[k] * scale) % P
    end
  end
  for i = #coef, 1, -1 do
    if coef[i] ~= 0 then break end
    coef[i] = nil
  end
  return coef
end
```

listing 956: lua, the incremental numerator and the scaled accumulation (source: dsa/samples-lua/ch29_polyops.lua, lines 91-124)

The fixtures cross-name book 9, ch 35 (practical data structures and algorithms, combinatorics) for the binomial machinery landing there: multiplying (1 + x) by itself eight times pins the row 1, 8, 28, 56, 70, 56, 28, 8, 1, all below the modulus, and horner at x = 10 returns 11^8 = 214358881. The interpolation fixture: f of x = x^2 + 3x + 5 sampled at 0 through 4 gives the points 5, 9, 15, 23, 33, lagrange recovers the coefficient list [5, 3, 1], and evaluating the result at 4 returns 33. The remainder edge reads off the algebra: x^3 is 1 mod x^2 + x + 1 and 100 = 3 * 33 + 1, so x^100 + 1 reduces to x + 1, coefficients [1, 1], asserted beside the division that produces it. Multiplying by [1] is the identity. The dense check convolves (1..16) with itself, 31 coefficients peaking at 1240, plain integers below the modulus, and the transform path equals schoolbook term for term. Every op cross-checks against schoolbook at small sizes. The finals reality, cross-named from the icpc book's chapter 14 opener, is that no finals submission in the corpus needed this ladder, the education is the point.

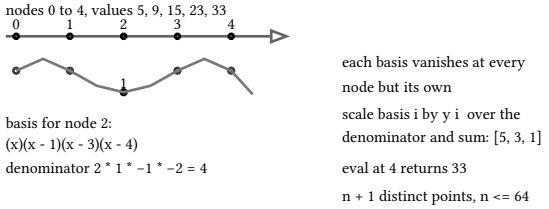


figure 241: lagrange basis polynomials as products of lines, each vanishing at all nodes but one, summing back to the coefficient list 5, 3, 1

29.7. across the seven languages

Build size counts non-blank, non-comment lines over the chapter's seven sample files per language, embedded checks included:

language	build SLOC	dependency	boundary note
c	1603	libc and libm	static arrays throughout, 512-limb caps, no malloc, uint128 banned by the icpc ch14 grep rule, every divmod numerator under 1e18
go	811	stdlib only	one package over six files, errors carry the refusals, add-subtract butterflies stay inside int64
java	1556	jdk 27 stdlib	hand-rolled int limb arrays, BigInteger never invoked, karatsuba arena windows as array-offset pairs, butterflies and the under-1e18 qhat numerator inside long
c#	718	bcl, System.Numerics	long limbs with (p-1)^2 inside signed 64, Complex for the fft, BigNum shared by every later file
javascript	691	node stdlib	BigInt at each named boundary: exact limb products, ntt butterflies, qhat and mulsub, residue products
python	912	stdlib only	native ints, three-argument pow as the ground-truth cross-check, guards raise ValueError
lua	1046	lib.lua harness	integer tables with every product under 2^63, 1-based tables documented per file, error() carries refusals

sources: the cp-algorithms articles, all cc by-sa 4.0, rewritten in our own words, accessed 2026-09-20: "Arbitrary-Precision Arithmetic" at cp-algorithms.com/algebra/big-integer.html for the 29.1 core and the 29.2 multiplication treatment, "Fast Fourier transform" at cp-algorithms.com/algebra/fft.html for its number theoretic transform section behind 29.3 and the complex transform behind 29.4, and "Operations on polynomials and series" at cp-algorithms.com/algebra/polynomial.html for 29.6. Karatsuba cites Karatsuba and Ofman, 1962, and division cites Knuth, The Art of Computer Programming volume 2, section 4.3.1, algorithm D, both exactly as the icpc book's chapter 14 sources row pins them. The measured constants quoted in the text, the two primes, INV, the 2^14 drift cap with its ranges, and the karatsuba cuts, come from the icpc book's chapter 14 benches run 2026-09-19, and icpc 2018 problem C is named as the application. Sample behavior verified by the seven suite gates scoped to chapter 29: c 6 files and 123 checks, go 19 tests, java 6 files and 199 checks under run-java-samples, c# 37 facts, javascript 20 tests, python 6 files and 118 asserts, lua 24 checks, zero skipped. This chapter carries 42 listings and 9 figures.

30. balanced trees and order statistics

Randomized balance buys logarithmic operations without any rotation rules to prove. A treap stores a bst by key and a heap by a random priority, and the two orders together hold the shape near balanced on expectation, so split, merge, insert, erase, and lazy range adds all run in expected logarithmic time. This chapter builds that machine, pairs it with two meldable heaps that lean on the same coin-flip reasoning, and closes with the two order-statistic structures built on top: rank and select over a fenwick tree of frequencies, and the implicit treap that keys nodes by array position instead. The flagship application is icpc world finals 2019 problem F (book 10, chapter 10), a rainfall dp that runs on exactly this treap, and its rebuild-versus-depth-gate decision is the one the sample suites pin.

30.1. the treap

A binary search tree by key, a max-heap by priority. Every node carries a random priority drawn from a counter stream, the splitMix32 sequence incremented once per allocation, and the invariant is two-sided: keys sort left to right inside the tree, priorities heap upward with the maximum at the root. Two recursive functions do all the work. Split walks the key order and tears the tree into keys below the threshold and the rest. Merge joins two trees whose key ranges do not overlap by letting the larger priority become the parent. Insert is split, split again to isolate the one key slot, replace the value in place or allocate, then merge the three pieces back. Erase merges the children of the isolated node. A range operation splits twice around the inclusive key range, reads or mutates the isolated middle as one subtree, and merges back.

The lazy range add is the subtle part. A pending add sits on a node and applies to the whole subtree below it, so the node's stored sum covers itself and its children's relative sums plus its own pending add once per folded child: child sums are relative to the child, the node's lz rides on top of them. Push moves the pending add onto both children before any descent that reads or relinks below the node, the same discipline as the lazy tags of book 9, ch 21 (practical data structures and algorithms, range queries).

The dry run: the fixture inserts keys 5, 2, 8, 1, 4, 7, 9, 3, 6 with values ten times the key, the C# suite pinning the lazy walk with the sibling suites on the same script.

1. The build lands the pinned shape first: inorder 1 through 9 with values 10 through 90, depth 5, zero rebuilds.
2. The whole-range sum reads the root once: 10 × (1 + 2 + … + 9) = 10 × 45 = 450.
3. The lazy add splits twice around keys 2 through 6 and parks +10 on the isolated middle of five keys: 450 + 5 × 10 = 500.
4. The reads split clean: keys 3 to 5 give 40 + 50 + 60 = 150, keys 7 to 9 outside the add give 70 + 80 + 90 = 240, key 4 alone gives 50.
5. Erase(4) drops the 50, 500 - 50 = 450 at size 8, then a −5 add over everything leaves sum(2, 3) at (30 - 5) + (40 - 5) = 60.
6. The gate meter: 31 sequential inserts under the default gate 3 × 5 + 8 = 23 stay at depth 9, zero rebuilds, sum 496, while gate 4 trips 7 rebuilds and lands depth 6.

op	arithmetic	lands
insert 9 keys	10 × 45 = 450	size 9, depth 5, 0 rebuilds
add 10 over 2-6	450 + 5 × 10 = 500	sum 500
reads 3-5, 7-9, 4-4	40 + 50 + 60, 70 + 80 + 90, 50	150, 240, 50
erase 4	500 - 50 = 450	size 8
add −5 over 1-9	(30 - 5) + (40 - 5) = 60	sum(2, 3) = 60
31 inserts, gate 23	3 × 5 + 8 = 23	depth 9, 0 rebuilds, sum 496
31 inserts, gate 4		depth 6, 7 rebuilds

The 500 down to 60 lazy walk with the 7 rebuilds at gate 4 are the pinned rows, and the listings below run them in seven languages.

```c
// split by key: returns the left tree (keys < key), right via pointer
static int split(int x, int key, int *out_right) {
  if (!x) {
    *out_right = 0;
    return 0;
  }
  cur_depth++;
  if (cur_depth > max_depth_seen)
    max_depth_seen = cur_depth;
  push(x);
  if (t[x].k < key) {
    int b;
    int a = split(t[x].r, key, &b);
    t[x].r = a;
    pull(x);
    *out_right = b;
    cur_depth--;
    return x;
  }
  int b;
  int a = split(t[x].l, key, &b);
  t[x].l = b;
  pull(x);
  *out_right = x;
  cur_depth--;
  return a;
}

static int merge(int a, int b) {
  if (!a)
    return b;
  if (!b)
    return a;
  cur_depth++;
  if (cur_depth > max_depth_seen)
    max_depth_seen = cur_depth;
  push(a);
  push(b);
  if (t[a].pri > t[b].pri) {
    t[a].r = merge(t[a].r, b);
    pull(a);
    cur_depth--;
    return a;
  }
  t[b].l = merge(a, t[b].l);
  pull(b);
  cur_depth--;
  return b;
}
```

listing 957: c, split by key and merge by priority, the depth meter bumped on entry and unwound on exit (source: dsa/samples-c/src/Ch30/treap.c, lines 95-143)

```
func (t *Treap) split(x *treapNode, key int64) (*treapNode, *treapNode) {
  if x == nil {
    return nil, nil
  }
  t.curDepth++
  if t.curDepth > t.maxDepth {
    t.maxDepth = t.curDepth
  }
  t.push(x)
  var a, b *treapNode
  if x.key < key {
    a, b = t.split(x.r, key)
    x.r = a
    treapPull(x)
    a = x
  } else {
    a, b = t.split(x.l, key)
    x.l = b
    treapPull(x)
    b = x
  }
  t.curDepth--
  return a, b
}

func (t *Treap) merge(a, b *treapNode) *treapNode {
  if a == nil {
    return b
  }
  if b == nil {
    return a
  }
  t.curDepth++
  if t.curDepth > t.maxDepth {
    t.maxDepth = t.curDepth
  }
  t.push(a)
  t.push(b)
  if a.pri > b.pri {
    a.r = t.merge(a.r, b)
    treapPull(a)
    return a
  }
  b.l = t.merge(a, b.l)
  treapPull(b)
  return b
}
```

listing 958: go, split and merge with the meter in the treap struct, push before each descent (source: dsa/samples-go/ch30/treap.go, lines 104-150)

```
  // split by key: returns the left tree (keys < key), right in
  // splitRight
  static int split(int x, int key) {
    if (x == 0) {
      splitRight = 0;
      return 0;
    }
    curDepth++;
    if (curDepth > maxDepthSeen)
      maxDepthSeen = curDepth;
    push(x);
    if (t[x].k < key) {
      int a = split(t[x].r, key);
      int b = splitRight;
      t[x].r = a;
      pull(x);
      splitRight = b;
      curDepth--;
      return x;
    }
    int a = split(t[x].l, key);
    int b = splitRight;
    t[x].l = b;
    pull(x);
    splitRight = x;
    curDepth--;
    return a;
  }

  static int merge(int a, int b) {
    if (a == 0)
      return b;
    if (b == 0)
      return a;
    curDepth++;
    if (curDepth > maxDepthSeen)
      maxDepthSeen = curDepth;
    push(a);
    push(b);
    if (Integer.compareUnsigned(t[a].pri, t[b].pri) > 0) {
      t[a].r = merge(t[a].r, b);
      pull(a);
      curDepth--;
      return a;
    }
    t[b].l = merge(a, t[b].l);
    pull(b);
    curDepth--;
    return b;
  }
```

listing 959: java, split and merge over the node pool, priorities compared unsigned, the meter unwound on each return (source: dsa/samples-java/src/Ch30/Treap.java, lines 101-150)

```
    private (Node?, Node?) Split(Node? t, long key)
    {
        if (t == null)
            return (null, null);
        _curDepth++;
        _maxDepth = Math.Max(_maxDepth, _curDepth);
        Push(t);
        (Node? a, Node? b) pair;
        if (t.Key < key)
        {
            (Node? a, Node? b) = Split(t.R, key);
            t.R = a;
            Pull(t);
            pair = (t, b);
        }
        else
        {
            (Node? a, Node? b) = Split(t.L, key);
            t.L = b;
            Pull(t);
            pair = (a, t);
        }
        _curDepth--;
        return pair;
    }

    private Node? Merge(Node? a, Node? b)
    {
        if (a == null)
            return b;
        if (b == null)
            return a;
        _curDepth++;
        _maxDepth = Math.Max(_maxDepth, _curDepth);
        Push(a);
        Push(b);
        Node? r;
        if (a.Pri > b.Pri)
        {
            a.R = Merge(a.R, b);
            Pull(a);
            r = a;
        }
        else
        {
            b.L = Merge(a, b.L);
            Pull(b);
            r = b;
        }
        _curDepth--;
        return r;
    }
```

listing 960: c#, the same pair as tuple-returning methods, pull after every relink (source: dsa/samples/src/Ch30/Treap.cs, lines 98-149)

```
  // ——— the split/merge/pull/push core (the chapter listing) ———

  split(t, key) {
    if (t === null) return [null, null];
    this.#curDepth += 1;
    if (this.#curDepth > this.#maxDepth) this.#maxDepth = this.#curDepth;
    push(t);
    let res;
    if (t.k < key) {
      const [a, b] = this.split(t.r, key);
      t.r = a;
      pull(t);
      res = [t, b];
    } else {
      const [a, b] = this.split(t.l, key);
      t.l = b;
      pull(t);
      res = [a, t];
    }
    this.#curDepth -= 1;
    return res;
  }

  merge(a, b) {
    if (a === null) return b;
    if (b === null) return a;
    this.#curDepth += 1;
    if (this.#curDepth > this.#maxDepth) this.#maxDepth = this.#curDepth;
    push(a);
    push(b);
    let res;
    if (a.pri > b.pri) {
      a.r = this.merge(a.r, b);
      pull(a);
      res = a;
    } else {
      b.l = this.merge(a, b.l);
      pull(b);
      res = b;
    }
    this.#curDepth -= 1;
    return res;
  }
```

listing 961: javascript, the split and merge core, private depth fields, destructured pair returns (source: dsa/samples-js/src/ch30-treap.mjs, lines 92-134)

```
    def split(self, t, key):  # left tree holds keys < key
        if t is None:
            return None, None
        self.cur_depth += 1
        self.max_depth_seen = max(self.max_depth_seen, self.cur_depth)
        push(t)
        if t.k < key:
            a, b = self.split(t.r, key)
            t.r = a
            pull(t)
            out = (t, b)
        else:
            a, b = self.split(t.l, key)
            t.l = b
            pull(t)
            out = (a, t)
        self.cur_depth -= 1
        return out

    def merge(self, a, b):
        if a is None:
            return b
        if b is None:
            return a
        self.cur_depth += 1
        self.max_depth_seen = max(self.max_depth_seen, self.cur_depth)
        push(a)
        push(b)
        if a.pri > b.pri:
            a.r = self.merge(a.r, b)
            pull(a)
            return a
        b.l = self.merge(a, b.l)
        pull(b)
        return b
```

listing 962: python, split then merge, push on entry, pull on the way out (source: dsa/samples-py/src/Ch30/treap.py, lines 78-112)

```
function Treap:split(t, key) -- left holds keys < key
  if t == nil then return nil, nil end
  self.cur_depth = self.cur_depth + 1
  self.max_depth_seen = math.max(self.max_depth_seen, self.cur_depth)
  push(t)
  local a, b
  if t.k < key then
    a, b = self:split(t.r, key)
    t.r = a
    pull(t)
    self.cur_depth = self.cur_depth - 1
    return t, b
  end
  a, b = self:split(t.l, key)
  t.l = b
  pull(t)
  self.cur_depth = self.cur_depth - 1
  return a, t
end

function Treap:merge(a, b)
  if a == nil then return b end
  if b == nil then return a end
  self.cur_depth = self.cur_depth + 1
  self.max_depth_seen = math.max(self.max_depth_seen, self.cur_depth)
  push(a)
  push(b)
  if a.pri > b.pri then
    a.r = self:merge(a.r, b)
    pull(a)
    self.cur_depth = self.cur_depth - 1
    return a
  end
  b.l = self:merge(a, b.l)
  pull(b)
  self.cur_depth = self.cur_depth - 1
  return b
end
```

listing 963: lua, the same pair, the meter unwound on both return paths (source: dsa/samples-lua/ch30_treap.lua, lines 60-97)

Every public operation records its maximum split or merge recursion depth and compares it against a gate. The default gate is 3 times ceil(log2(n+1)) plus 8, constructor-overridable. A depth past the gate flags the structure dirty, and the next public operation rebuilds before running: flatten inorder with the lazy adds applied, rebuild by the balanced median recursion, assign priorities 0xffffffff minus depth so the heap order matches the fresh shape exactly, and reset the allocation counter. The rebuild costs O(n), and the amortized argument is that an adversarial burst has to spend operations to deepen the tree before each rebuild can fire. Against book 9, ch 7 (practical data structures and algorithms, trees) the trade is explicit: the avl tree there guarantees worst-case logarithmic depth through rotations, the treap only expects it, but the treap's split and merge compose into range logic the rotation machinery does not give directly.

The fixture families pin identical numbers in all seven suites. The shape family inserts keys 5, 2, 8, 1, 4, 7, 9, 3, 6 with values ten times the key and reads inorder 1 through 9 with values 10 through 90, size 9, depth 5, the preorder 9, 4, 3, 1, 2, 6, 5, 8, 7, and zero rebuilds. Java draws the same splitMix32 priorities as unsigned 32-bit values, so every priority comparison routes through Integer.compareUnsigned, a signed compare sorts the draws past 2^31 negative and grows a different tree. The algebra family splits at 5 into sizes 4 and 5 holding keys 1 through 4 and 5 through 9, splits at 0 and at 10 into 0/9 and 9/0, and merges each back to a preorder identical to the base. The

lazy family sums the whole key range at 450, adds 10 over keys 2 through 6, and reads 500, 150, 240, and 50 over ranges 1-9, 3-5, 7-9, and 4-4, with the inorder values pinning the per-key state. Erasing key 4 returns the total to 450 at size 8, and a −5 add over everything leaves sum(2,3) at 60.

The gate family is the honest one. Over 31 sequential inserts the default gate 23 never trips: depth 9, zero rebuilds, sum 496. A tight gate trips 7 rebuilds, lands at depth 6, keeps inorder 1 through 31 intact, and pins the rebuilt preorder 15, 8, 4, 2, 1, 3, 6, 5, 7, 12, 10, 9, 11, 14, 13, 23, 19, 17, 16, 18, 21, 20, 22, 27, 25, 24, 26, 29, 28, 30, 31. All seven languages reach that shape with gate 4, every meter tracking live recursion, bumped on entry and unwound on each return. The edge family closes the contract: an empty treap sizes 0 and sums 0, inserting key 7 twice replaces the value in place with no allocation, and erasing an absent key is a no-op.

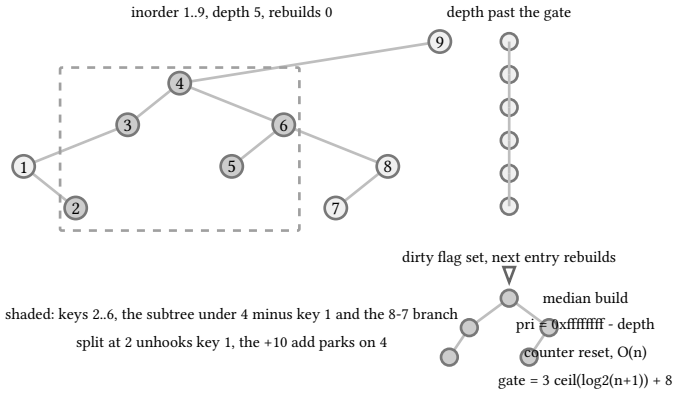


figure 242: the 9-key treap with the 2 to 6 range isolated for a lazy add, and the depth-gated rebuild from a deep chain into the median shape

The application is icpc world finals 2019 problem F (book 10, chapter 10), directing rainfall. The shipped solver's piecewise dp lives on a run treap with lazy adds, subtree min and max, and splits at cell boundaries, and its authors faced exactly this chapter's decision: the go and c solvers rebuild on a fixed every-4096 cadence, the lua solver runs a depth gate of 96, and the measured maximum recursion depth across the judge inputs is 52. The chapter sample is the explicit-key teaching form of that vehicle, and the pf_runs implementation in the icpc book is the contest-grade sibling.

30.2. the splay tree

The splay tree answers the same question without randomness. Every access splays its node to the root through three local shapes: zig, one rotation when the parent is the root, zig-zig for a straight grandparent-parent-node chain, rotated from the top down, and zig-zag for a bent chain, two rotations that lift the node two levels. The amortized O(log n) bound comes from a potential argument, summing logarithms of subtree sizes across the tree and showing each splay pays for itself plus a bounded potential drop, so a sequence of m operations on n keys costs O((m + n) log n) with no probability anywhere. The payoff beyond the bound is working-set locality: recently touched keys sit near the root, so hot workloads run faster than the worst case, and the access-lease structure is what link and cut trees consume downstream.

Positioned against the neighbors: the treap above reaches the same amortized expectation through random priorities and composes through split and merge, while the splay tree reaches it through pointer surgery on every access and composes through joins. The avl tree of book 9, ch 7 (practical data structures and algorithms, trees) keeps a worst-case bound and no locality at all. No samples ship for it: the interesting claim is the potential argument rather than another metered port, and the treap suites already carry the randomized half of the comparison.

The dry run: no suite carries the splay tree, the numbers here are hand-derived, and the run inserts 1 through 5 in order.

1. Every insert lands right of the last, so the tree is a pure right chain under root 1 with node 5 at depth 4.
2. Access 5 meets a straight chain, so one zig-zig rotates 3 under 4 then 4 under 5: node 5 rises to depth 2 with 4 and 3 parked on its left.

3. The chain is straight again, so a second zig-zig rotates 1 under 2 then 2 under 5, and 5 is the root: 2 its left child holding 1 and 4, with 3 under 4, tree depth 3.
4. Access 1 next: the path 5, 2, 1 is a straight left chain, one zig-zig rotates 5 under 2 then 2 under 1, and 1 is the root with the formerly hot 5 sunk to depth 2.
5. Inorder never moves off 1, 2, 3, 4, 5 through any of it, the rotations only ever relink.
6. The two accesses cost 4 + 2 = 6 rotations total, and the second found its key two levels from the root, the locality the bound pays for.

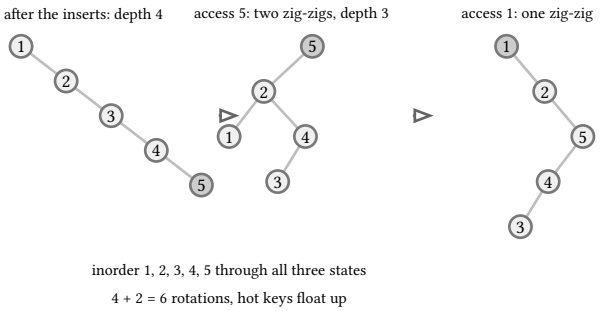


figure 243: the run as three tree states, the right chain, root 5 after the first access, root 1 after the second

Six rotations carry the chain through both accesses and back, and the figure below draws the local shapes the run passes through.

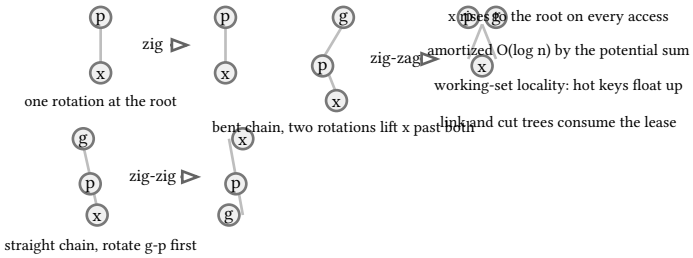


figure 244: the three splay steps as before and after pairs, the accessed x rising to the root each time

30.3. meldable heaps

Two priority queues whose only structural operation is meld, and both fit in one screen. The randomized heap links by the smaller key, then flips a coin to decide which child the losing root descends into: the winner stays the parent, the coin swaps or keeps its two children, and the loser melds into the left link. Each element sits on a randomly chosen root-to-leaf path, so the expected depth is logarithmic and meld runs in expected O(log n). Insert is a meld with a singleton, O(1) to start. Pop melds the two children of the root. The skew heap is the same function with the coin replaced by an unconditional swap: no randomness, no single-operation bound, and an amortized O(log n) over any sequence because the swaps charge against the right paths they shorten. Against the array heap of book 9, ch 8 (practical data structures and algorithms, heaps and priority queues), whose meld would rebuild at O(n), these two trade pointer chasing for the O(1) union.

The dry run: the fixture pushes 5, 1, 8 into heap A and 3, 9, 2 into heap B, melds, and drains, the C# suite pinning the drain order and the comparison meters with the sibling suites on the same script.

1. The builds tick 4: each push past the first melds a singleton at one comparison, 2 for A and 2 for B.
2. The meld ticks 2: the roots compare 1 against 2, the winner's left child meets the arriving 2, and the coin parks it there, meter 6.
3. The drain pops six roots and spends 10 - 6 = 4 more comparisons where both children exist, closing the meter at 10.
4. The skew heap posts the same 4, 2, 10 on the same script, its unconditional swap agreeing with the drawn coins here.
5. Duplicates 2, 2, 1 drain 1, 2, 2 on 2 comparisons, and melding an empty heap compares nothing.

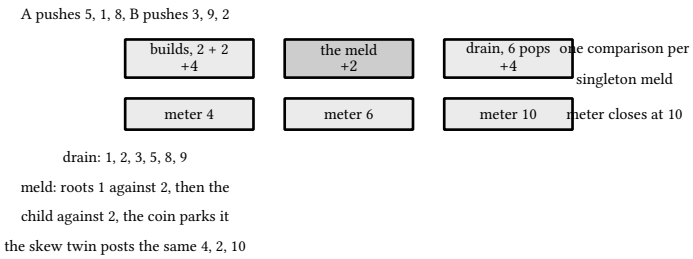


figure 245: the comparison meter through the script, one bar per phase, the running total under each

The meter closing at 10 over the drain 1, 2, 3, 5, 8, 9 is the pinned pair, and the listings below meld both heaps in seven languages.

```c
// the randomized meld: coin decides which child the loser descends into
static int rand_meld(struct Heap *h, int a, int b) {
  if (!a)
    return b;
  if (!b)
    return a;
  h->cmp++;
  if (hp[a].k > hp[b].k) {
    int t = a;
    a = b;
    b = t;
  }
  if (heap_coin(h)) {
    int t = hp[a].l;
    hp[a].l = hp[a].r;
    hp[a].r = t;
  }
  hp[a].l = rand_meld(h, hp[a].l, b);
  return a;
}

// the skew meld: the unconditional swap carries the amortized balance
static int skew_meld(struct Heap *h, int a, int b) {
  if (!a)
    return b;
  if (!b)
    return a;
  h->cmp++;
  if (hp[a].k > hp[b].k) {
    int t = a;
    a = b;
    b = t;
  }
  int t = hp[a].l;
  hp[a].l = hp[a].r;
  hp[a].r = t;
  hp[a].l = skew_meld(h, hp[a].l, b);
  return a;
}
```

listing 964: c, the two melds adjacent, one coin flip against one unconditional swap (source: dsa/samples-c/src/Ch30/mergeheap.c, lines 63-101)

```
func (h *RandHeap) coin() bool {
  h.next++
  return splitMix32(h.next)&1 == 1
}

// Meld links the two heaps and counts the key comparisons.
func (h *RandHeap) Meld(a, b *heapNode) *heapNode {
  if a == nil {
    return b
  }
  if b == nil {
    return a
  }
  h.cmp++
  if a.k > b.k {
    a, b = b, a
  }
  if h.coin() { // swap the children before the descent
    a.l, a.r = a.r, a.l
  }
  a.l = h.Meld(a.l, b)
  return a
}
```

listing 965: go, the coin draw and the randomized meld, comparison meter inside (source: dsa/samples-go/ch30/mergeheap.go, lines 19-41)

```
  // the randomized meld: coin decides which child the loser descends into
  static int randMeld(Heap h, int a, int b) {
    if (a == 0)
      return b;
    if (b == 0)
      return a;
    h.cmp++;
    if (hp[a].k > hp[b].k) {
      int t = a;
      a = b;
      b = t;
    }
    if (heapCoin(h) != 0) {
      int t = hp[a].l;
      hp[a].l = hp[a].r;
      hp[a].r = t;
    }
    hp[a].l = randMeld(h, hp[a].l, b);
    return a;
  }

  // the skew meld: the unconditional swap carries the amortized balance
  static int skewMeld(Heap h, int a, int b) {
    if (a == 0)
      return b;
    if (b == 0)
      return a;
    h.cmp++;
    if (hp[a].k > hp[b].k) {
      int t = a;
      a = b;
      b = t;
    }
    int t = hp[a].l;
    hp[a].l = hp[a].r;
    hp[a].r = t;
    hp[a].l = skewMeld(h, hp[a].l, b);
    return a;
  }
```

listing 966: java, the two melds adjacent, the coin is the splitMix32 low bit, unsigned shifts (source: dsa/samples-java/src/Ch30/Mergeheap.java, lines 68-106)

```
    private Node? Meld(Node? a, Node? b)
    {
        if (a == null)
            return b;
        if (b == null)
            return a;
        Comparisons++;
        if (a.Key > b.Key)
            (a, b) = (b, a);
        if (Coin() == 1)
            (a.L, a.R) = (a.R, a.L);
        a.L = Meld(a.L, b);
        return a;
    }
```

listing 967: c#, the coin meld, the skew twin below swaps unconditionally (source: dsa/samples/src/Ch30/MergeHeap.cs, lines 45-58)

```
  #meld(a, b) {
    if (a === null) return b;
    if (b === null) return a;
    this.cmp += 1;
    if (a.k > b.k) {
      const t = a;
      a = b;
      b = t;
    }
    if (this.#coin()) {
      const t = a.l;
      a.l = a.r;
      a.r = t;
    }
    a.l = this.#meld(a.l, b);
    return a;
  }
```

listing 968: javascript, the randomized meld, the loser descends left after the coin (source: dsa/samples-js/src/ch30-mergeheap.mjs, lines 39-55)

```
    def meld(self, a, b):
        if a is None:
            return b
        if b is None:
            return a
        self.cmp += 1
        if a.k > b.k:
            a, b = b, a
        if self.coin():
            a.l, a.r = a.r, a.l
        a.l = self.meld(a.l, b)
        return a

    def push(self, k):
        self.root = self.meld(self.root, HNode(k))

    def pop(self):
        if self.root is None:
            raise ValueError("pop on an empty heap")
        k = self.root.k
        self.root = self.meld(self.root.l, self.root.r)
        return k


class SkewHeap:
    # the same meld, the unconditional swap, no randomness
    def __init__(self):
        self.root = None
        self.cmp = 0

    def meld(self, a, b):
        if a is None:
            return b
        if b is None:
            return a
        self.cmp += 1
        if a.k > b.k:
            a, b = b, a
        a.l, a.r = a.r, a.l
        a.l = self.meld(a.l, b)
        return a
```

listing 969: python, both melds, the whole contrast in one slice (source: dsa/samples-py/src/Ch30/mergeheap.py, lines 46-86)

```
function RandHeap:meld(a, b)
  if a == nil then return b end
  if b == nil then return a end
  self.cmp = self.cmp + 1
  if a.k > b.k then a, b = b, a end
  if self:coin() == 1 then
    a.l, a.r = a.r, a.l
  end
  a.l = self:meld(a.l, b)
  return a
end

function RandHeap:push(k)
  self.root = self:meld(self.root, {k = k, l = nil, r = nil})
end

function RandHeap:pop()
  local k = self.root.k
  self.root = self:meld(self.root.l, self.root.r)
  return k
end

local SkewHeap = {}
SkewHeap.__index = SkewHeap

function SkewHeap.new()
  return setmetatable({root = nil, cmp = 0}, SkewHeap)
end

function SkewHeap:meld(a, b)
  if a == nil then return b end
  if b == nil then return a end
  self.cmp = self.cmp + 1
  if a.k > b.k then a, b = b, a end
  a.l, a.r = a.r, a.l -- the unconditional swap, no randomness
  a.l = self:meld(a.l, b)
  return a
end
```

listing 970: lua, the coin meld and the skew meld, table heaps (source: dsa/samples-lua/ch30_mergeheap.lua, lines 33-70)

The fixtures run both structures through the same script. Heap A pushes 5, 1, 8, heap B pushes 3, 9, 2, the meld drains 1, 2, 3, 5, 8, 9 from both. The comparison meters post 4 for the builds, 2 for the meld, and 10 after the drain, and the coincidence is honest: on this fixture the coins happened to agree with the unconditional swap, so the skew heap posts the same 4/2/10 while its meter stays its own. Duplicates push 2, 2, 1 and drain 1, 2, 2 on 2 comparisons in both, and melding with an empty heap costs nothing and returns the other heap unchanged. The edges refuse politely per language house style, a sentinel flag in c and java, an exception in c#, go, javascript, and python, a nil-indexing throw in lua, and the python suite pins the self-meld identity on a singleton.

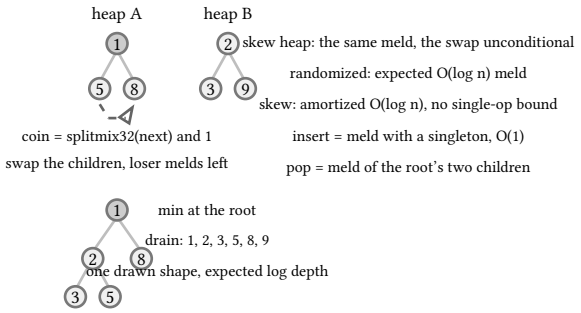


figure 246: two 3-node heaps melding, the coin swapping the winner's children before the descent, the merged heap with the minimum at the root

30.4. order statistics

Two machines for the k-th smallest question. The first is the fenwick tree of book 9, ch 21 (practical data structures and algorithms, range queries) repurposed as a frequency table over a value universe: add(v, plus or minus 1) moves a count, rank(v) is the prefix below v, freq differs two prefixes, and select runs the bit-jump descent. The descent starts at the largest power of two that fits the universe and halves: if the node ahead holds fewer than the remaining k, skip past its whole range and subtract, otherwise stay. Each step fixes one bit of the answer's position, so rank and select both run in O(log U) with no tree walk at all. The second machine is the implicit treap: the same split and merge as the section above, keyed by subtree size instead of by key. Split by index walks until the left subtree is exactly k positions, so insert_at, erase_at, at(k), sum_range, and move_to_front all decompose into cuts and merges over positions, the rope the textbooks promise, with expected O(log n) per operation.

The dry run: the fixture loads 3, 1, 4, 1, 5, 9, 2, 6, 5 as frequencies over a universe of 16 and runs the rope script, the C# suite pinning both with the sibling suites on the same digits.

1. The per-value counts: 1 appears 2 times, 5 appears 2 times, every other loaded value once, so the cumulative over values 1, 2, 3 runs 2, 3, 4.
2. kth(4) descends from the power 16: node 16 holds the total 9, node 8 holds 8, node 4 holds 5, none below the remaining k.
3. Node 2 holds 3, below 4, so the jump lands at pos 2 with k = 4 - 3 = 1, and node 3's 1 is not below 1: the answer is pos + 1 = 3.
4. The rope starts 10, 20, 30, 40, 50 at 10 + 20 + 30 + 40 + 50 = 150, and move_to_front(2, 3) cuts 30, 40 out and merges them ahead for 30, 40, 10, 20, 50.
5. The reads follow the cut: sum(1, 3) gives 40 + 10 + 20 = 70, insert_at(1, 99) lifts the total to 150 + 99 = 249, and erase_at(4) returns the 20 while at(0), at(2) read 30 and 40.

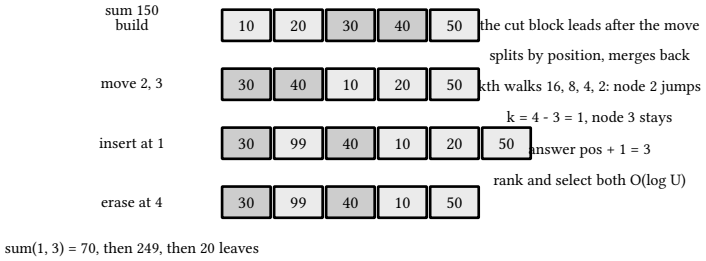


figure 247: the rope run as four list states, the moved block 30, 40 shaded wherever it sits

The kth(4) answer of 3 beside the rope's mid-run 249 are the pinned pair, and the listings below walk both machines in seven languages.

```c
// 1-based k-th value by the bit-jump descent; -1 refuses
static int fen_kth(int k) {
  if (k < 1 || k > fen_prefix(U))
    return -1;
  int pos = 0, kk = k;
  int pw = 1;
  while (pw * 2 <= U)
    pw *= 2;
  for (; pw; pw /= 2) {
    if (pos + pw <= U && fen[pos + pw] < kk) {
      kk -= fen[pos + pw];
      pos += pw;
    }
  }
  return pos + 1;
}

static int fen_rank(int v) { // values < v, clamped to the universe
  if (v < 1)
    return 0;
  if (v > U + 1)
    v = U + 1;
  return fen_prefix(v - 1);
}

static int fen_freq(int v) { return fen_prefix(v) - fen_prefix(v - 1); }
```

listing 971: c, the kth bit-jump descent, rank as a clamped prefix, freq by differencing (source: dsa/samples-c/src/Ch30/orderstats.c, lines 50-75)

```go
// Kth returns the k-th smallest value, 1-based, by walking powers of
// two down and keeping the position where the running count reaches
// k. k = 0 and k past the total refuse.
func (f *FenwickOS) Kth(k int64) (int, error) {
  if k < 1 || k > f.Prefix(f.n) {
    return 0, fmt.Errorf("kth(%d) outside 1..total", k)
  }
  pos := 0
  pw := 1
  for pw*2 <= f.n {
    pw *= 2
  }
  for ; pw > 0; pw /= 2 {
    next := pos + pw
    if next <= f.n && f.tree[next] < k {
      k -= f.tree[next]
      pos = next
    }
  }
  return pos + 1, nil
}
```

listing 972: go, kth walking the powers of two down, refusal outside 1..total (source: dsa/samples-go/ch30/orderstats.go, lines 34-54)

```
// 1-based k-th value by the bit-jump descent; -1 refuses
static int fenKth(int k) {
  if (k < 1 || k > fenPrefix(U))
    return -1;
  int pos = 0, kk = k;
  int pw = 1;
  while (pw * 2 ≤ U)
    pw *= 2;
  for (; pw ≠ 0; pw /= 2) {
    if (pos + pw ≤ U && fen[pos + pw] < kk) {
      kk -= fen[pos + pw];
      pos += pw;
    }
  }
  return pos + 1;
}

static int fenRank(int v) { // values < v, clamped to the universe
  if (v < 1)
    return 0;
  if (v > U + 1)
    v = U + 1;
  return fenPrefix(v - 1);
}

static int fenFreq(int v) {
  return fenPrefix(v) - fenPrefix(v - 1);
}
```

listing 973: java, the kth bit-jump descent, rank as a clamped prefix, freq by differencing (source: dsa/samples-java/src/Ch30/Orderstats.java, lines 49-76)

```
  /// <summary>Splits off the first k positions (left = first k).</summary>
  private (Node?, Node?) Split(Node? t, int k)
  {
      if (t == null)
          return (null, null);
      int ls = t.L?.Sz ?? 0;
      if (ls < k)
      {
          (Node? a, Node? b) = Split(t.R, k - ls - 1);
          t.R = a;
          Pull(t);
          return (t, b);
      }
      {
          (Node? a, Node? b) = Split(t.L, k);
          t.L = b;
          Pull(t);
          return (a, t);
      }
  }
```

listing 974: c#, the implicit split by index, subtree size steering the recursion (source: dsa/samples/src/Ch30/OrderStats.cs, lines 116-135)

```
  // split by index: left tree = the first k positions
  split(t, k) {
    if (t === null) return [null, null];
    const ls = t.l === null ? 0 : t.l.sz;
    if (ls < k) {
      const [a, b] = this.split(t.r, k - ls - 1);
      t.r = a;
      ipull(t);
      return [t, b];
    }
    const [a, b] = this.split(t.l, k);
    t.l = b;
    ipull(t);
    return [a, t];
  }
```

listing 975: javascript, split by index, the left tree keeps the first k positions (source: dsa/samples-js/src/ch30-orderstats.mjs, lines 108-122)

```
        return s

    def kth(self, k):
        if k < 1 or k > self.prefix(self.n):
            raise ValueError("k outside the occupied range")
        pos = 0
        pw = 1
        while pw * 2 ≤ self.n:
            pw *= 2
        while pw:
            if pos + pw ≤ self.n and self.t[pos + pw] < k:
                k -= self.t[pos + pw]
                pos += pw
            pw //= 2
        return pos + 1
```

listing 976: python, the kth descent over the frequency fenwick (source: dsa/samples-py/src/Ch30/orderstats.py, lines 43-57)

```
function ITreap:split(t, k) -- left keeps the first k positions
  if t == nil then return nil, nil end
  local ls = t.l and t.l.sz or 0
  if ls < k then
    local a, b = self:split(t.r, k - ls - 1)
    t.r = a
    ipull(t)
    return t, b
  end
  local a, b = self:split(t.l, k)
  t.l = b
  ipull(t)
  return a, t
end
```

listing 977: lua, the implicit split, 0-based public face over 1-based tables (source: dsa/samples-lua/ch30_orderstats.lua, lines 95-108)

The fenwick family loads the chapter 21 fixture digits 3, 1, 4, 1, 5, 9, 2, 6, 5 as frequencies, nine values in a universe of 16: the total is 9, kth of 1 through 9 reads 1, 1, 2, 3, 4, 5, 5, 6, 9, rank at 1, 2, 5, 7, 10 reads 0, 2, 5, 8, 9, and freq at 1, 5, 6 reads 2, 2, 1. Removing one of the two 4s shifts kth of 4, 5, 6 to 3, 5, 5 and rank(5) to 4. The rope family builds 10, 20, 30, 40, 50, sums 150, moves positions 2 and 3 to the front for 30, 40, 10, 20, 50 with sum(1,3) at 70,

inserts 99 at position 1 for a total of 249, erases position 4 for the value 20, and reads at(0) and at(2) as 30 and 40. Then the interleaved script runs 40 operations off the house lcg with seed 30, insert_at, erase_at, and sum_range mixed, every sum checked against a shadow list, javascript driving the generator through Math.imul so its 32-bit multiplies agree with the other six, java masking the unsigned state to 32 bits before the double widening, and the final list pins at 16, 27, 40, 1, 27. The edges: an empty rope sums 0, a single-element move_to_front is a no-op, kth refuses at 0 and past the total, and rank outside the universe clamps to 0 and to the total.

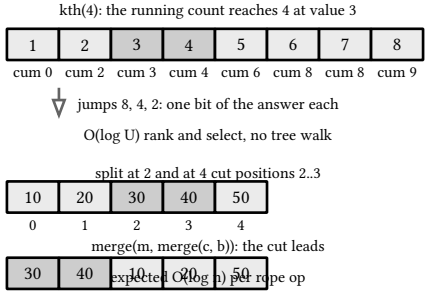


figure 248: the fenwick bit descent selecting kth(4), beside the rope's move_to_front cut of positions 2 and 3

The unranked kth has its own chapter already: the quickselect extension of book 9, ch 14 (practical data structures and algorithms, searching and answer spaces) answers one offline k-th smallest in expected linear time without any structure, and the fenwick here is the online upgrade that answers rank and select repeatedly under inserts and removals.

30.5. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's three sample files per language, embedded test scripts included:

language	build sloc	container dependency	boundary note
c	865	static pool arrays, index 0 the null node	long long sums, uint32 priorities, the empty-pop sentinel flag
go	541	struct with meters, no imports beyond fmt	error returns on empty pops and kth out of range
java	909	jdk 27 stdlib	splitMix32 priorities are uniform uint32 draws, every comparison through Integer.compareUnsigned, the shifts >>> not >>
c#	574	nullable node classes, tuples	default gate from size, gate override in the constructor, exceptions at the boundaries
javascript	451	class fields, private methods	Math.imul on every 32-bit multiply, splitMix32 and the lcg included
python	665	`__slots__` nodes, check harness	system exit on failure, ok-N print, the self-meld identity pinned
lua	630	tables, 1-based inside	64-bit integers keep the priority products exact

sources: cp-algorithms, "Treap (Cartesian tree)", cp-algorithms.com/data_structures/treap.html, covering the explicit and implicit treap sections, "Randomized Heap", cp-algorithms.com/data_structures/randomized_heap.html, "Fenwick Tree", cp-algorithms.com/data_structures/fenwick.html, whose rank/select extension is taught in our own words, and "K-th order statistic in O(N)", cp-algorithms.com/sequences/k-th.html, the quickselect kin, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Application source: icpc world finals 2019 problem F (book 10, chapter 10). Sample behavior verified by the seven suite gates scoped to chapter 30: c 3 files and 82 checks, go 12 test functions, java 3 files and 82 checks under run-java-samples, c# 17 facts, javascript 13 tests and 73 asserts, python 3 files and 57 asserts, lua 14 checks, zero skipped.

31. sqrt decomposition and offline queries

Cut the array into blocks of about sqrt n cells and every range question becomes a few whole blocks plus two partial edges. That one idea, ten minutes to build, updates and queries both in O(sqrt n), and no recursion anywhere, is the baseline every fancier range structure of book 9, ch 21 (practical data structures and algorithms, range queries) has to beat. This chapter builds it, then spends the savings: mo's algorithm sorts offline queries so two pointers walk the whole batch in O((n + q) sqrt n), the minimum queue packages an O(1) window oracle other algorithms embed, the li chao tree answers online line queries by segment descent, the disjoint sparse table buys O(1) range folds for sums and other non-idempotent folds, and rollback dsu plus a segment tree over time answer dynamic connectivity offline. The last engine is the substrate of icpc world finals 2025 problem G (book 10, chapter 13).

31.1. sqrt decomposition

An array of n cells in blocks of b = ceil(isqrt(n)). Each block keeps its sum, a point set updates one cell and one block sum in O(1), and a range sum walks three pieces: the partial edge cells at both ends, scanned directly, and the whole blocks between them, read from the sum array one block at a time. A query spanning fb through lb block ids touches exactly r/b - l/b + 1 blocks, the meter the suites pin alongside every sum.

The dry run: the fixture is the pi digits 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3 over n = 16, asserted by the C# suite with the same sums in the C, Go, and Python suites.

1. The build folds each block once: 3 + 1 + 4 + 1 = 9, 5 + 9 + 2 + 6 = 22, 5 + 3 + 5 + 8 = 21, 9 + 7 + 9 + 3 = 28, four sums over b = 4 and nb = 4.
2. The whole range touches all 4 blocks and reads 80.
3. Query 3 to 8 straddles blocks 0 and 2: the left edge scans cell 3 alone at 1, the middle reads block 1 whole at 22, the right edge scans cell 8 alone at 5, and 1 + 22 + 5 = 28 touching 3.
4. Query 2 to 6 straddles adjacent blocks, so nothing whole sits between: the edges pay (4 + 1) + (5 + 9 + 2) = 21 touching 2.
5. The single cell 4 never leaves its block and scans itself, 5 touching 1.
6. A point set pays twice: Set(4, 100) adds 100 - 5 = 95 onto block 1's 22 for 117, the whole range climbs to 175, and restoring the cell returns 80.

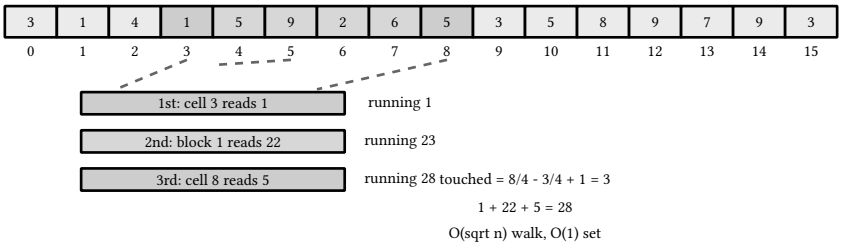


figure 249: the query 3 to 8 as three reads in order, the running total accumulating under each

The 28 at 3 blocks touched closes the walk, and the listings below run it in seven languages.

```c
static long long sum(int l, int r) { // inclusive
  long long s = 0;
  int fb = l / bsz, lb = r / bsz;
  if (fb == lb) {
    for (int i = l; i ≤ r; i++)
      s += a[i];
    return s;
  }
  for (int i = l; i < (fb + 1) * bsz; i++)
    s += a[i];
  for (int k = fb + 1; k < lb; k++)
    s += blk[k];
  for (int i = lb * bsz; i ≤ r; i++)
    s += a[i];
  return s;
}

static int touched(int l, int r) { return r / bsz - l / bsz + 1; }
```

listing 978: c, the three-way sum walk and the touched meter, static arrays (source: dsa/samples-c/src/Ch31/sqrtblocks.c, lines 45-62)

```go
// Sum folds [l, r] inclusive as partial edges plus whole blocks.
func (s *SqrtBlocks) Sum(l, r int) int64 {
  total := int64(0)
  fb, lb := l/s.b, r/s.b
  if fb == lb {
    for i := l; i <= r; i++ {
      total += s.a[i]
    }
    return total
  }
  for i := l; i < (fb+1)*s.b; i++ {
    total += s.a[i]
  }
  for k := fb + 1; k < lb; k++ {
    total += s.blk[k]
  }
  for i := lb * s.b; i <= r; i++ {
    total += s.a[i]
  }
  return total
}

// Touched is the block meter: how many blocks the range spans.
func (s *SqrtBlocks) Touched(l, r int) int {
  return r/s.b - l/s.b + 1
}
```

listing 979: go, sum folds edges then blocks, touched beside (source: dsa/samples-go/ch31/sqrtblocks.go, lines 48-73)

```
static long sum(int l, int r) { // inclusive
  long s = 0;
  int fb = l / bsz, lb = r / bsz;
  if (fb == lb) {
    for (int i = l; i ≤ r; i++)
      s += a[i];
    return s;
  }
  for (int i = l; i < (fb + 1) * bsz; i++)
    s += a[i];
  for (int k = fb + 1; k < lb; k++)
    s += blk[k];
  for (int i = lb * bsz; i ≤ r; i++)
    s += a[i];
  return s;
}

static int touched(int l, int r) {
  return r / bsz - l / bsz + 1;
}
```

listing 980: java, the three-way sum walk and the touched meter, block ids by integer division (source: dsa/samples-java/src/Ch31/Sqrtblocks.java, lines 44-63)

```
  public long Sum(int l, int r) // inclusive
  {
      long s = 0;
      int fb = l / BlockSize;
      int lb = r / BlockSize;
      if (fb == lb)
      {
          for (int i = l; i ≤ r; i++)
              s += _a[i];
          return s;
      }
      for (int i = l; i < (fb + 1) * BlockSize; i++) // left partial edge
          s += _a[i];
      for (int k = fb + 1; k < lb; k++) // whole blocks
          s += _blk[k];
      for (int i = lb * BlockSize; i ≤ r; i++) // right partial edge
          s += _a[i];
      return s;
  }

  public int Touched(int l, int r) ⇒ r / BlockSize - l / BlockSize + 1;
```

listing 981: c#, the same walk, partial edges commented, whole blocks from the sum array (source: dsa/samples/src/Ch31/SqrtBlocks.cs, lines 38-58)

```
  sum(l, r) {
    // inclusive, the three-way block/edge split
    let s = 0;
    const fb = Math.floor(l / this.b);
    const lb = Math.floor(r / this.b);
    if (fb === lb) {
      for (let i = l; i ≤ r; i++) s += this.a[i];
    } else {
      for (let i = l; i < (fb + 1) * this.b; i++) s += this.a[i];
      for (let k = fb + 1; k < lb; k++) s += this.blk[k];
      for (let i = lb * this.b; i ≤ r; i++) s += this.a[i];
    }
    return s;
  }

  touched(l, r) {
    return Math.floor(r / this.b) - Math.floor(l / this.b) + 1;
  }
```

listing 982: javascript, the block or edge split, floor division for block ids (source: dsa/samples-js/src/ch31-sqrtblocks.mjs, lines 31-48)

```
    def sum(self, lo, hi):  # inclusive
        total = 0
        fb, lb = lo // self.b, hi // self.b
        if fb == lb:
            for i in range(lo, hi + 1):
                total += self.a[i]
        else:
            for i in range(lo, (fb + 1) * self.b):  # the left edge
                total += self.a[i]
            for k in range(fb + 1, lb):  # whole blocks
                total += self.blk[k]
            for i in range(lb * self.b, hi + 1):  # the right edge
                total += self.a[i]
        return total

    def touched(self, lo, hi):
        return hi // self.b - lo // self.b + 1
```

listing 983: python, the left edge, whole blocks, right edge (source: dsa/samples-py/src/Ch31/sqrtblocks.py, lines 32-49)

```lua
function SqrtBlocks:sum(l, r) -- inclusive, 0-based
  local s = 0
  local fb, lb = l // self.b, r // self.b -- 0-based block ids
  if fb == lb then
    for i = l, r do s = s + self.a[i + 1] end
  else
    for i = l, (fb + 1) * self.b - 1 do s = s + self.a[i + 1] end
    for k = fb + 2, lb do s = s + self.blk[k] end
    for i = lb * self.b, r do s = s + self.a[i + 1] end
  end
  return s
end

function SqrtBlocks:touched(l, r)
  return r // self.b - l // self.b + 1
end
```

listing 984: lua, 1-based block table behind a 0-based face (source: dsa/samples-lua/ch31_sqrtblocks.lua, lines 37-52)

The fixture family runs n = 16, b = 4, nb = 4 over the pi digits 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5, 8, 9, 7, 9, 3. The whole range sums 80 touching 4 blocks, range 2-6 sums 21 touching 2, the single cell 4 sums 5 touching 1, range 3-8 sums 28 touching 3, range 5-10 sums 30 touching 2, and both endpoint cells read 3 touching 1. Setting cell 4 to 100 lifts the whole range to 175, range 3-8 to 123, and block 1′s sum to 117, and restoring the cell returns 80. The edge family pins n = 1 down to b = 1 with sum 7 touching 1. Against the fenwick tree and the segment tree this loses a factor of sqrt n over log n on every operation, and wins on the minutes count, which is exactly why it is also the substrate of the next section.

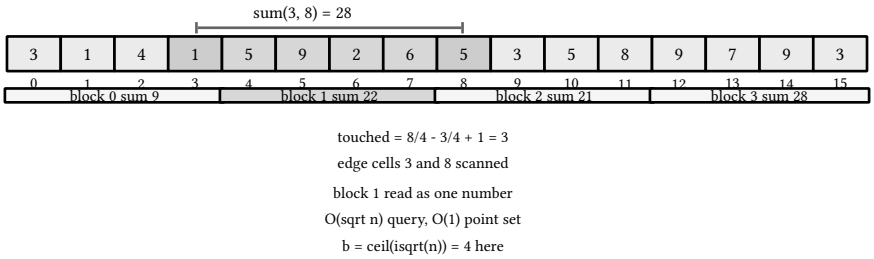


figure 250: 16 cells in 4 blocks, the range 3 to 8 shaded as one partial edge, one whole block, and another partial edge, 3 blocks touched

31.2. mo’s algorithm

Flip the order of work: when the array is static and all queries are known, sort the queries and let one interval answer them all. Sort by the block of the left endpoint, then by the right endpoint, and walk a single current interval from query to query with four while loops, expanding and contracting both ends. Each step moves one pointer by one cell and touches a frequency table: a value’s count crossing zero to one raises the distinct counter, back to zero lowers it, and the counter after the four loops is that query’s answer. Within one left block the right pointer only moves forward, so it travels O(n) per block for O(q) blocks total in one direction, and the left pointer wanders at most b per query: O((n + q) sqrt n) all in, with the travel meter as the evidence. The sort must be a total order stable in the query index, c, go, and java break ties on the index explicitly, javascript leans on its stable sort, so all seven languages process the same order and pin the same meter.

The dry run: the fixture is 1, 2, 1, 3, 2, 1, 4, 1 with b = 3 and the six queries (0,7), (0,3), (2,5), (1,1), (4,7), (2,2), asserted by the C# suite at 20 total pointer moves.

1. The sort by left block then right endpoint lands the order 3, 5, 1, 2, 0, 4: five queries of block 0 by rising r, then q4 alone in block 1.
2. The interval starts empty. q3 (1,1) opens it, the right pointer stepping twice and the left once: the interval 1..1 holds the single value 2 and answers 1 at 3 moves.

3. q5 (2,2) slides a step each way to 2..2 and answers 1 again, 5 moves banked.
4. q1 (0,3) widens to 0..3: the 3 arrives and the returning 1 does not raise the count, answer 3 at 8 moves.
5. q2 (2,5) and q0 (0,7) walk the right edge out: answers 3 then 4, the 4 first counted over 0..7, 16 moves banked.
6. q4 (4,7) rests the right pointer and walks the left 4 cells off the front, dropping the 3 for answer 3, closing at the pinned 20.

run	query	interval	distinct	moves
0	start	0..-1	0	0
1	q3 (1,1)	1..1	1	3
2	q5 (2,2)	2..2	1	5
3	q1 (0,3)	0..3	3	8
4	q2 (2,5)	2..5	3	12
5	q0 (0,7)	0..7	4	16
6	q4 (4,7)	4..7	3	20

The six answers land 4, 3, 3, 1, 3, 1 in input order for the 20 moves, and the listings below run the walk in seven languages.

```
  for (int i = 0; i < qn; i++) {
    proc_order[proc_n++] = order[i].idx;
    while (m.cr < order[i].r) {
      m.cr++;
      m.travel++;
      mo_add(&m, m.cr);
    }
    while (m.cl > order[i].l) {
      m.cl--;
      m.travel++;
      mo_add(&m, m.cl);
    }
    while (m.cr > order[i].r) {
      mo_rem(&m, m.cr);
      m.cr--;
      m.travel++;
    }
    while (m.cl < order[i].l) {
      mo_rem(&m, m.cl);
      m.cl++;
      m.travel++;
    }
    answers[order[i].idx] = m.distinct;
  }
  return m.travel;
```

listing 985: c, the four while loops, travel counted per pointer move (source: dsa/samples-c/src/Ch31/mos.c, lines 91-115)

```
    }
  }
  answers := make([]int64, len(queries))
  cl, cr := 0, -1
  travel := 0
  for _, qi := range order {
    l, r := queries[qi][0], queries[qi][1]
    for cr < r {
      cr++
      travel++
      add(a[cr])
    }
    for cl > l {
      cl--
      travel++
      add(a[cl])
    }
    for cr > r {
      remove(a[cr])
      cr--
      travel++
    }
    for cl < l {
      remove(a[cl])
      cl++
      travel++
    }
```

listing 986: go, the walk with the add and remove closures above (source: dsa/samples-go/ch31/mos.go, lines 51-77)

```
    for (int i = 0; i < qn; i++) {
      procOrder[procN++] = order[i].idx;
      while (m.cr < order[i].r) {
        m.cr++;
        m.travel++;
        m.add(m.cr);
      }
      while (m.cl > order[i].l) {
        m.cl--;
        m.travel++;
        m.add(m.cl);
      }
      while (m.cr > order[i].r) {
        m.rem(m.cr);
        m.cr--;
        m.travel++;
      }
      while (m.cl < order[i].l) {
        m.rem(m.cl);
        m.cl++;
        m.travel++;
      }
      answers[order[i].idx] = m.distinct;
    }
    return m.travel;
  }
```

listing 987: java, the four while loops over the comparator-sorted order, travel per move (source: dsa/samples-java/src/Ch31/Mos.java, lines 78-103)

```
        foreach (int i in order)
        {
            (int l, int r) = queries[i];
            while (cr < r)
            {
                cr++;
                travel++;
                Add(cr);
            }
            while (cl > l)
            {
                cl--;
                travel++;
                Add(cl);
            }
            while (cr > r)
            {
                Remove(cr);
                cr--;
                travel++;
            }
            while (cl < l)
            {
                Remove(cl);
                cl++;
                travel++;
            }
            answers[i] = distinct;
        }
        return (order, answers, travel);
    }
```

listing 988: c#, the same walk over the sorted order, closure add and remove (source: dsa/samples/src/Ch31/Mos.cs, lines 43-73)

```
    if (f + 1 === 1) distinct += 1;
  };
  const rem = (i) => {
    const f = freq.get(a[i]);
    freq.set(a[i], f - 1);
    if (f - 1 === 0) distinct -= 1;
  };

  // the four while loops, the whole algorithm
  for (const i of idx) {
    const [l, r] = queries[i];
    while (cr < r) {
      cr += 1;
      travel += 1;
      add(cr);
    }
    while (cl > l) {
      cl -= 1;
      travel += 1;
      add(cl);
    }
    while (cr > r) {
      rem(cr);
      cr -= 1;
      travel += 1;
    }
    while (cl < l) {
      rem(cl);
      cl += 1;
      travel += 1;
    }
    answers[i] = distinct;
  }
  return { answers, order: idx, travel };
```

listing 989: javascript, the four loops over the stably sorted indices (source: dsa/samples-js/src/ch31-mos.mjs, lines 35-68)

```
    for i in order:
        lo, hi = queries[i]
        while cr < hi:
            cr += 1
            travel += 1
            add(cr)
        while cl > lo:
            cl -= 1
            travel += 1
            add(cl)
        while cr > hi:
            rem(cr)
            cr -= 1
            travel += 1
        while cl < lo:
            rem(cl)
            cl += 1
            travel += 1
        answers[i] = distinct
```

listing 990: python, the walk, answers written in input order (source: dsa/samples-py/src/Ch31/mos.py, lines 42-61)

```
for _, i in ipairs(idx) do
  local l, r = queries[i][1], queries[i][2]
  while cr < r do
    cr = cr + 1
    travel = travel + 1
    add(cr)
  end
  while cl > l do
    cl = cl - 1
    travel = travel + 1
    add(cl)
  end
  while cr > r do
    rem(cr)
    cr = cr - 1
    travel = travel + 1
  end
  while cl < l do
    rem(cl)
    cl = cl + 1
    travel = travel + 1
  end
  answers[i] = distinct
end
return idx, answers, travel
```

listing 991: lua, the walk, 0-based face over 1-based tables (source: dsa/samples-lua/ch31_mos.lua, lines 41-65)

The fixture array is 1, 2, 1, 3, 2, 1, 4, 1 with n = 8 and b = 3. The six queries (0,7), (0,3), (2,5), (1,1), (4,7), (2,2) process in order 3, 5, 1, 2, 0, 4, answer 4, 3, 3, 1, 3, 1 in input order, and total 20 pointer moves, with a brute set-size scan agreeing on every answer. The clustered family 4, 4, 7, 4, 7, 9, 9, 2 forces real distinct counting: its five queries process 1, 2, 0, 4, 3, answer 4, 1, 3, 3, 4, travel 16, brute agreeing again. The edge family repeats identical queries, (3,3) twice with (0,0) and (7,7), and pins the processing order 2, 0, 1, 3 with every answer 1. Offline distinct-count is the canonical vehicle, and no finals problem in the corpus rides it, so it stays a general technique here.

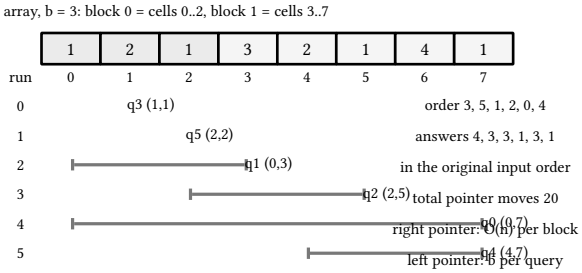


figure 251: the six queries sorted by left block then right endpoint over the 8-cell array, the two pointers walking from query to query

31.3. the minimum stack and queue

The sliding-window scans of book 9, ch 21 (practical data structures and algorithms, range queries) moved a deque of survivors one step at a time. Here the object itself is the deliverable: a minimum oracle other algorithms embed, with every operation O(1) worst case except one amortized step. The min stack stores (value, running minimum) pairs, push takes min with the pair below, and the top pair's second slot answers min in one read. The min queue is two of those stacks back to back: an inbox that receives enqueues and an outbox that serves dequeues. When the outbox empties, the dequeue refills it by popping the whole inbox across, which reverses the order and, for free, recomputes every running minimum for the reversed sequence. The queue minimum is the min of the two stack minima, and the transfer is the amortized seam: every element crosses it at most once per lifetime, so n operations cost O(n) total.

The dry run: the fixture is the push run 5, 2, 7, 1, 4, asserted by the C# suite, the same family in the other five suites.

1. Push 5 opens the stack with the pair 5, 5, value and running minimum agreeing, and min reads 5.
2. Push 2 lands as 2, 2, taking the minimum with the 5 below, and min drops to 2.
3. Push 7 lands as 7, 2: the value rises, the minimum does not, and min stays 2.
4. Push 1 lands as 1, 1 and push 4 as 4, 1: min reads the pinned run 5, 2, 2, 1, 1.
5. Two pops shed 4, 1 and 1, 1, the top is again 7, 2, and min recovers 2.
6. Two more pops shed 7, 2 and 2, 2, and the lone 5, 5 restores min
   5.

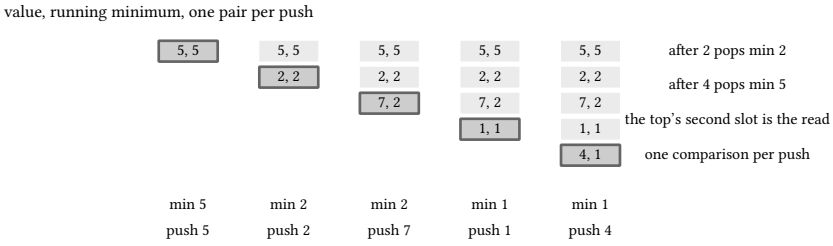


figure 252: the stack one column per push, each pair carrying value and running minimum, the top pair stroked, min under each column

The recovered pair 2 then 5 is the pinned landing, and the listings below build stack and queue in seven languages.

```c
// the amortized seam: refill the outbox reversed, then pop
static void mq_dequeue(struct MinQueue *q) {
  if (!q->out.top) {
    while (q->in.top) {
      int v = ms_pop(&q->in);
      ms_push(&q->out, v);
    }
  }
  ms_pop(&q->out);
}

static int mq_min(struct MinQueue *q, int *ok) {
  if (!q->in.top && !q->out.top) {
    *ok = 0;
    return 0;
  }
  *ok = 1;
  if (!q->in.top)
    return ms_min(&q->out);
  if (!q->out.top)
    return ms_min(&q->in);
  int a = ms_min(&q->in), b = ms_min(&q->out);
  return a < b ? a : b;
}
```

listing 992: c, the dequeue-with-transfer and the two-stack minimum (source: dsa/samples-c/src/Ch31/minstack.c, lines 60-83)

```
// Enqueue pushes onto the inbox.
func (q *MinQueue) Enqueue(v int64) { q.inbox.push(v) }

// Dequeue pops the outbox, transferring the whole inbox when the
// seam is reached.
func (q *MinQueue) Dequeue() error {
  if q.outbox.len() == 0 {
    for q.inbox.len() > 0 {
      q.outbox.push(q.inbox.pop())
    }
  }
  if q.outbox.len() == 0 {
    return fmt.Errorf("dequeue on an empty min queue")
  }
  q.outbox.pop()
  return nil
}

// Min combines the two stack minima, refusing when both are empty.
func (q *MinQueue) Min() (int64, error) {
  o, ook := q.outbox.top()
  i, iok := q.inbox.top()
  switch {
  case ook && iok:
    if o < i {
      return o, nil
    }
    return i, nil
  case ook:
    return o, nil
  case iok:
    return i, nil
  }
  return 0, fmt.Errorf("min of an empty min queue")
}
```

listing 993: go, the transfer loop and the min switch over both tops (source: dsa/samples-go/ch31/minstack.go, lines 63-98)

```java
  // the amortized seam: refill the outbox reversed, then pop
  static void mqDequeue(MinQueue q) {
    if (q.out.top == 0) {
      while (q.in.top != 0) {
        int v = msPop(q.in);
        msPush(q.out, v);
      }
    }
    msPop(q.out);
  }

  // returns the min; ok[0] flags the empty refusal
  static int mqMin(MinQueue q, boolean[] okf) {
    if (q.in.top == 0 && q.out.top == 0) {
      okf[0] = false;
      return 0;
    }
    okf[0] = true;
    if (q.in.top == 0)
      return msMin(q.out);
    if (q.out.top == 0)
      return msMin(q.in);
    return Math.min(msMin(q.in), msMin(q.out));
  }
```

listing 994: java, the dequeue-with-transfer, min over both tops, the empty refusal by out-flag (source: dsa/samples-java/src/Ch31/Minstack.java, lines 53-76)

```
    public void Enqueue(long value) => _inbox.Push(value);

    public long Dequeue()
    {
        if (_outbox.Count == 0)
        {
            if (_inbox.Count == 0)
                throw new InvalidOperationException("dequeue on an empty queue");
            while (_inbox.Count > 0)
                _outbox.Push(_inbox.Pop()); // the amortized transfer at the seam
        }
        return _outbox.Pop();
    }

    public long Min
    {
        get
        {
            if (_inbox.Count == 0 && _outbox.Count == 0)
                throw new InvalidOperationException("min of an empty queue");
            if (_outbox.Count == 0)
                return _inbox.Min;
            if (_inbox.Count == 0)
                return _outbox.Min;
            return Math.Min(_inbox.Min, _outbox.Min);
        }
    }
}
```

listing 995: c#, dequeue refills from the inbox, min combines the two stack minima (source: dsa/samples/src/Ch31/MinStack.cs, lines 46-73)

```
  push(v) {
    this.#inbox.push(v);
    return this;
  }

  shift() {
    if (this.size() === 0) throw new Error("shift on an empty queue");
    if (this.#outbox.size() === 0) this.#transfer();
    return this.#outbox.pop();
  }

  min() {
    if (this.size() === 0) throw new Error("min of an empty queue");
    if (this.#inbox.size() === 0) return this.#outbox.min();
    if (this.#outbox.size() === 0) return this.#inbox.min();
    return Math.min(this.#inbox.min(), this.#outbox.min());
  }

  size() {
```

listing 996: javascript, shift transfers then pops, min reads either side (source: dsa/samples-js/src/ch31-minstack.mjs, lines 44-62)

```
    def enqueue(self, v):
        self.inbox.push(v)

    def _refill(self):
        while self.inbox.vals:
            self.outbox.push(self.inbox.pop())
            self.transfers += 1

    def dequeue(self):
        if not self.outbox.vals:
            self._refill()
        return self.outbox.pop()

    def min(self):
        if not self.inbox.vals and not self.outbox.vals:
            raise ValueError("min of an empty queue")
        if not self.outbox.vals:
            return self.inbox.min()
        if not self.inbox.vals:
            return self.outbox.min()
        return min(self.outbox.min(), self.inbox.min())
```

listing 997: python, refill counted as transfers, dequeue and min (source: dsa/samples-py/src/Ch31/minstack.py, lines 41-61)

```
function MinQueue:push(x)
  self.inbox:push(x)
end

-- the amortized seam: refill the outbox by reversing the inbox
function MinQueue:pop()
  if self.outbox.n == 0 then
    while self.inbox.n > 0 do
      self.outbox:push(self.inbox:pop())
    end
  end
  return self.outbox:pop()
end

function MinQueue:min()
  local got = nil
  if self.outbox.n > 0 then got = self.outbox:min() end
  if self.inbox.n > 0 then
    got = (got == nil) and self.inbox:min()
                       or math.min(got, self.inbox:min())
  end
  if got == nil then error("min of an empty queue") end
  return got
end
```

listing 998: lua, the seam refill reversing the inbox, nil-aware min (source: dsa/samples-lua/ch31_minstack.lua, lines 44-67)

The min stack family pushes 5, 2, 7, 1, 4 and reads the running minima 5, 2, 2, 1, 1, then two pops restore min 2 and four pops restore 5. The window family re-runs the chapter 21 kin array 1, 3, −1, −3, 5, 3, 6, 7 with k = 3 and reads −1, −3, −3, −3, 3, 3, the same array where chapter 21′s maxima scan printed 3, 3, 5, 5, 6, 7, with a brute running-min cross-check and the seam state pinned afterward: inbox 0, outbox 3. The edge family runs k = 1, where every step drains the outbox through a transfer and refills it again and the output is the array itself, and the refusals behave per language: pop on an empty stack and min of an empty queue refuse by flag, exception, or error return.

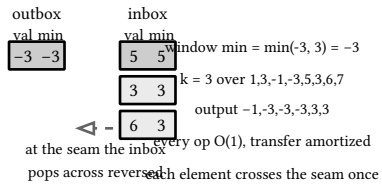


figure 253: the min queue mid-sweep at the transfer seam, the inbox and outbox min columns shaded, the window minimum reading across both

31.4. the li chao tree

Chapter 19′s convex hull trick answered max-over-lines queries after sorting slopes offline. The li chao tree answers the same upper envelope question online, arbitrary insertion order, no sorting: a segment tree over the integer domain where every node keeps one line, the line better at that segment's midpoint. Insertion descends from the root. If the node is empty the line lands there. Otherwise compare the newcomer with the resident at the midpoint: the midpoint winner stays, and the loser can only win on one side of the midpoint, the side where it beats the winner at the segment end, so the recursion follows that half. A line and its host cross at most once, so each level of descent eliminates one candidate honestly, and both insert and query run in O(log C) over a domain of C integers, query taking the max over the root-to-leaf path.

The dry run: the fixture is the domain x in [0, 8) with the four lines (1, 0), (0, 3), (-1, 5), (2, −4), asserted by the C# suite, the domain and lines shared by all seven suites.

1. y = x enters first and the empty root keeps it: the node over 0..8 holds it.
2. y = 3 arrives: at the root's midpoint 4 the resident reads 4 and stays, and the newcomer wins only at the left end, 3 against 0, so it descends and lands over 0..4. The two-line envelope reads 3, 3, 3, 3, 4, 5, 6, 7, pinned.
3. 5 - x arrives: it loses the root's midpoint at 1 against 4, then ties the 0..4 midpoint at 3 = 3 where the resident stays, and lands over 0..2.
4. 2x - 4 arrives: it ties the root's midpoint at 4 = 4, the resident stays, and the newcomer wins only right, landing over 4..8.
5. Query(0) descends root, 0..4, 0..2 reading 0, 3, 5: the path max is 5. Query(4) reads the root's 4 and the 4..8 node's 2 × 4 - 4 = 4, the tie the midpoint rule created.
6. The envelope closes at 5, 4, 3, 3, 4, 6, 8, 10, the brute max-over-lines agreeing at every x.

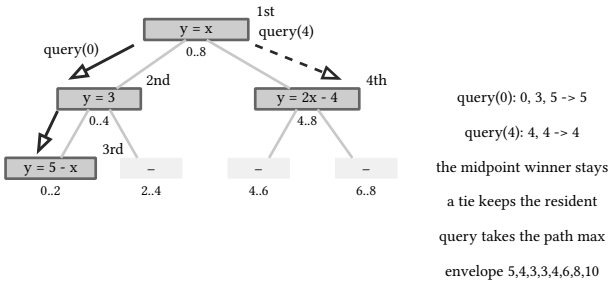


figure 254: the four insertions landing one node each over 0 to 8, order chips beside the winners, the two query paths walked heavy

The envelope 5, 4, 3, 3, 4, 6, 8, 10 is the pinned landing, and the listings below insert these four lines in seven languages.

```c
static void insert_rec(long long m, long long c, int nd, int lo, int hi) {
  if (!has[nd]) {
    has[nd] = 1;
    lm[nd] = m;
    lc[nd] = c;
    return;
  }
  long long mid = (lo + hi) / 2;
  int better_mid = m * mid + c > lm[nd] * mid + lc[nd];
  if (better_mid) { // the newcomer wins the midpoint: swap it in
    long long tm = m, tc = c;
    m = lm[nd];
    c = lc[nd];
    lm[nd] = tm;
    lc[nd] = tc;
  }
  if (hi - lo == 1)
    return;
  // recurse into the half where the loser can still win
  long long xl = m * lo + c, cl = lm[nd] * lo + lc[nd];
  if (xl > cl)
    insert_rec(m, c, 2 * nd, lo, (int)mid);
  else
    insert_rec(m, c, 2 * nd + 1, (int)mid, hi);
}
```

listing 999: c, the insert descent, midpoint winner kept, loser recursed one side (source: dsa/samples-c/src/Ch31/lichao.c, lines 28-52)

```go
func (l *LiChao) insert(node, lo, hi, m, c int64) {
  if !l.has[node] {
    l.has[node], l.m[node], l.c[node] = true, m, c
    return
  }
  cm, cc := l.m[node], l.c[node]
  mid := int64((lo + hi) / 2)
  if lineVal(m, c, mid) > lineVal(cm, cc, mid) {
    m, c, cm, cc = cm, cc, m, c
    l.m[node], l.c[node] = m, c
  }
  if hi-lo == 1 {
    return
  }
  if lineVal(m, c, int64(lo)) > lineVal(cm, cc, int64(lo)) {
    l.insert(2*node, lo, (lo+hi)/2, m, c)
  } else {
    l.insert(2*node+1, (lo+hi)/2, hi, m, c)
  }
}

// Insert adds the line m*x + c; duplicates are tolerated.
func (l *LiChao) Insert(m, c int64) {
  l.insert(1, 0, int64(l.n), m, c)
}
```

listing 1000: go, insert keeps the midpoint winner and follows the loser (source: dsa/samples-go/ch31/lichao.go, lines 22-46)

```
  static void insertRec(long m, long c, int node, int lo, int hi) {
    if (!has[node]) {
      has[node] = true;
      lm[node] = m;
      lc[node] = c;
      return;
    }
    long mid = (lo + hi) / 2;
    boolean betterMid = m * mid + c > lm[node] * mid + lc[node];
    if (betterMid) { // the newcomer wins the midpoint: swap it in
      long tm = m, tc = c;
      m = lm[node];
      c = lc[node];
      lm[node] = tm;
      lc[node] = tc;
    }
    if (hi - lo == 1)
      return;
    // recurse into the half where the loser can still win
    long xl = m * lo + c, cl = lm[node] * lo + lc[node];
    if (xl > cl)
      insertRec(m, c, 2 * node, lo, (int) mid);
    else
      insertRec(m, c, 2 * node + 1, (int) mid, hi);
  }
```

listing 1001: java, the insert descent, midpoint winner kept, loser recursed one side (source: dsa/samples-java/src/Ch31/Lichao.java, lines 29-53)

```
    public void Insert(long m, long c) => Insert(m, c, 1, 0, N);

    private void Insert(long m, long c, int nd, int lo, int hi)
    {
        if (!_hasLine[nd])
        {
            _slope[nd] = m;
            _intercept[nd] = c;
            _hasLine[nd] = true;
            return;
        }
        long cm = _slope[nd];
        long cc = _intercept[nd];
        int mid = (lo + hi) / 2;
        if (Eval(m, c, mid) > Eval(cm, cc, mid))
        {
            (m, c, cm, cc) = (cm, cc, m, c); // the newcomer wins the midpoint: swap
            _slope[nd] = cm;
            _intercept[nd] = cc;
        }
        if (hi - lo == 1)
            return;
        // recurse into the half where the loser can still win
        if (Eval(m, c, lo) > Eval(cm, cc, lo))
            Insert(m, c, 2 * nd, lo, mid);
        else
            Insert(m, c, 2 * nd + 1, mid, hi);
    }
```

listing 1002: c#, the same descent with tuple swaps, nullable query sentinel (source: dsa/samples/src/Ch31/LiChao.cs, lines 26-54)

```
  insert(line, nd = 1, lo = 0, hi = null) {
    if (hi === null) hi = this.#n;
    if (!this.#seg.has(nd)) {
      this.#seg.set(nd, line);
      return this;
    }
    let cur = this.#seg.get(nd);
    const mid = (lo + hi) >> 1;
    if (LiChao.#f(line, mid) > LiChao.#f(cur, mid)) {
      const t = line;
      line = cur;
      cur = t; // the challenger wins the midpoint, takes the node
      this.#seg.set(nd, cur);
    }
    if (hi - lo === 1) return this;
    if (LiChao.#f(line, lo) > LiChao.#f(cur, lo)) {
      this.insert(line, 2 * nd, lo, mid); // loser still wins on the left
    } else {
      this.insert(line, 2 * nd + 1, mid, hi);
    }
    return this;
  }
```

listing 1003: javascript, the descent over a map of segment nodes (source: dsa/samples-js/src/ch31-lichao.mjs, lines 20-42)

```
    def insert(self, line, node=1, lo=0, hi=None):
        if hi is None:
            hi = self.size
        if node not in self.seg:
            self.seg[node] = line
            return
        cur = self.seg[node]
        mid = (lo + hi) // 2
        if self.f(line, mid) > self.f(cur, mid):  # keep the midpoint winner
            line, cur = cur, line
            self.seg[node] = cur
        if hi - lo == 1:
            return
        if self.f(line, lo) > self.f(cur, lo):
            self.insert(line, 2 * node, lo, mid)
        else:
            self.insert(line, 2 * node + 1, mid, hi)
```

listing 1004: python, midpoint swap, left-or-right recursion (source: dsa/samples-py/src/Ch31/lichao.py, lines 25-41)

```
function LiChao:insert(line, nd, lo, hi)
  nd, lo, hi = nd or 1, lo or 0, hi or self.n
  local cur = self.seg[nd]
  if cur == nil then
    self.seg[nd] = line
    return
  end
  local mid = (lo + hi) // 2
  if f(line, mid) > f(cur, mid) then
    line, cur = cur, line -- keep the better line at the midpoint
    self.seg[nd] = cur
  end
  if hi - lo == 1 then return end
  if f(line, lo) > f(cur, lo) then
    self:insert(line, 2 * nd, lo, mid) -- the loser can still win left
  else
    self:insert(line, 2 * nd + 1, mid, hi) -- or only right
  end
end
```

listing 1005: lua, optional-argument defaults thread the segment bounds (source: dsa/samples-lua/ch31_lichao.lua, lines 20-38)

The fixture domain is x in [0, 8). The four lines (m, c) = (1, 0), (0, 3), (-1, 5), (2, −4) answer 5, 4, 3, 3, 4, 6, 8, 10 across the whole domain, brute max-over-lines agreeing at every integer. With only the first two lines the answers are 3, 3, 3, 3, 4, 5, 6, 7, and re-inserting a duplicate line changes nothing, query(0) stays 3. The empty container's query returns the sentinel, null in c# and javascript, a found flag in c and java, a bool beside the value in go, and the edges query at x = 0 and x = 7 once populated, −4 and 10 under the surviving (2, −4) line. The consumer is the dp transition machinery of book 9, ch 19 (practical data structures and algorithms, advanced dynamic programming), which needs exactly this online envelope when the slopes arrive unordered.

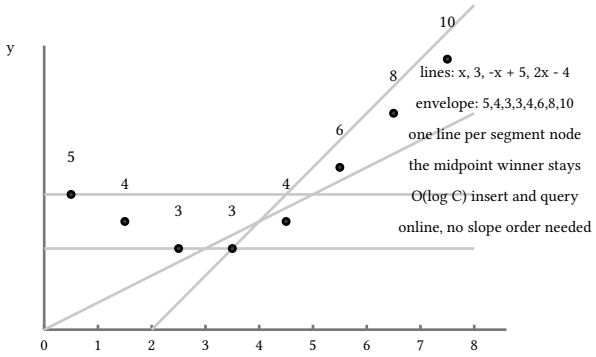


figure 255: the four fixture lines over 0 to 8 with the upper envelope segmented and the winning value labeled at each integer

31.5. the disjoint sparse table

The sparse table of book 9, ch 21 (practical data structures and algorithms, range queries) answers a range query with two overlapping power-of-two covers, legal only because min and max are idempotent. A sum query through that machinery counts the overlap twice. The disjoint sparse table removes the overlap and keeps the O(1) query for any associative fold. Level i cuts the array into aligned blocks of 2^i cells and, at each block midpoint, folds prefixes rightward into the right half and suffixes leftward into the left half. A query (l, r) with l < r reads the level named by the highest differing bit of l and r, where l sits in some left half and r in the matching right half of one block, and answers suffix(l) + prefix(r), two cells, no overlap.

The dry run: the fixture is the array 3, 1, 4, 1, 5, 9, 2, 6, 5, 3 at n = 10 and 4 levels, asserted by the C# suite, the same eight sums in all seven suites.

1. Query 2 to 7 picks its level first: 2 and 7 differ first at bit 2, so level 2, whose aligned blocks run 4 cells with this block's midpoint at 4.
2. The left half folds suffixes away from the midpoint: V of 3 = 1, then 4 + 1 = 5, then 1 + 5 = 6, then 3 + 6 = 9.
3. The right half folds prefixes out of it: U of 4 = 5, then 9 + 5 = 14, then 2 + 14 = 16, then 6 + 16 = 22.
4. The query reads two cells, V of 2 + U of 7 = 5 + 22 = 27, where chapter 21's two covers would pay 19 + 22 = 41 for the same span.
5. The single cell 4 never leaves level 0 and reads itself, 5.

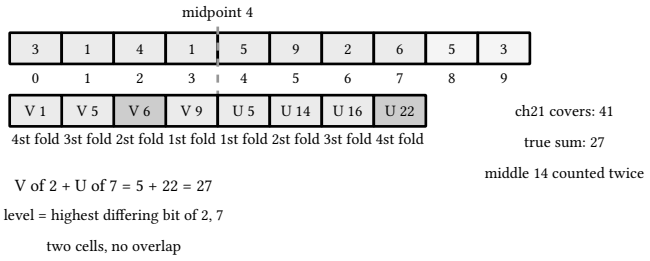


figure 256: the two half-scans running away from midpoint 4 in fill order, suffix cells leftward and prefix cells rightward, the two read cells shaded

The 27 from two cells is the pinned landing, and the listings below build the levels in seven languages.

```c
static void build(const int *src, int len) {
  for (int i = 0; i < len; i++)
    a[i] = src[i];
  levels = 0;
  for (int i = 1; (1 << (i - 1)) < len; i++) { // half widths 1, 2, 4, ...
    int half = 1 << (i - 1);
    for (int base = 0; base < len; base += half * 2) {
      int mid = base + half;
      if (mid >= len)
        break;
      long long acc = 0;
      for (int j = mid; j < mid + half && j < len; j++) {
        acc += a[j];
        U[levels][j] = acc;
      }
      acc = 0;
      for (int j = mid - 1; j >= base; j--) {
        acc += a[j];
        V[levels][j] = acc;
      }
    }
    levels++;
  }
}

static long long range_sum(int l, int r) { // inclusive
  if (l == r)
    return a[l];
  int x = l ^ r;
  int i = 0;
  while ((1 << (i + 1)) <= x)
    i++; // bit_length(x) - 1
  return V[i][l] + U[i][r];
}
```

listing 1006: c, the level build's two half-scans and the two-cell query (source: dsa/samples-c/src/Ch31/disjointsparse.c, lines 28-61)

```
func (d *DisjointSparse) buildLevel(i int) (u, v []int64) {
  half := 1 << (i - 1)
  u = make([]int64, d.n)
  v = make([]int64, d.n)
  for base := 0; base < d.n; base += half * 2 {
    mid := base + half
    if mid ≥ d.n {
      break
    }
    acc := int64(0)
    for j := mid; j < mid+half && j < d.n; j++ {
      acc += d.a[j]
      u[j] = acc
    }
    acc = 0
    for j := mid - 1; j ≥ base; j-- {
      acc += d.a[j]
      v[j] = acc
    }
  }
  return u, v
}

// Levels reports how many half-width levels were built.
func (d *DisjointSparse) Levels() int { return len(d.u) }

// Sum folds [l, r] inclusive in O(1); l == r reads the cell.
func (d *DisjointSparse) Sum(l, r int) int64 {
  if l == r {
    return d.a[l]
  }
  // first differing bit of l and r picks the level
  i := bits.Len(uint(l ^ r))
  return d.v[i-1][l] + d.u[i-1][r]
```

listing 1007: go, buildLevel's half-scans, sum via bits.len (source: dsa/samples-go/ch31/disjoint_sparse.go, lines 33-66)

```
  static void build(int[] src, int len) {
    for (int i = 0; i < len; i++)
      a[i] = src[i];
    levels = 0;
    for (int i = 1; (1 << (i - 1)) < len; i++) { // half widths 1, 2, 4...
      int half = 1 << (i - 1);
      for (int base = 0; base < len; base += half * 2) {
        int mid = base + half;
        if (mid ≥ len)
          break;
        long acc = 0;
        for (int j = mid; j < mid + half && j < len; j++) {
          acc += a[j];
          U[levels][j] = acc;
        }
        acc = 0;
        for (int j = mid - 1; j ≥ base; j--) {
          acc += a[j];
          V[levels][j] = acc;
        }
      }
      levels++;
    }
  }

  static long rangeSum(int l, int r) { // inclusive
    if (l == r)
      return a[l];
    int x = l ^ r;
    int i = 0;
    while ((1 << (i + 1)) ≤ x)
      i++; // bit_length(x) - 1
    return V[i][l] + U[i][r];
  }
```

listing 1008: java, the build's two half-scans, the query level by first differing bit (source: dsa/samples-java/src/Ch31/Disjointsparse.java, lines 27-60)

```
        for (int i = 1; i ≤ levels; i++)
        {
            long[] u = new long[n];
            long[] v = new long[n];
            int half = 1 << (i - 1);
            for (int b = 0; b < n; b += half * 2)
            {
                int mid = b + half;
                if (mid ≥ n)
                    break;
                long acc = 0;
                for (int j = mid; j < Math.Min(mid + half, n); j++)
                {
                    acc += _a[j];
                    u[j] = acc;
                }
                acc = 0;
                for (int j = mid - 1; j ≥ b; j--)
                {
                    acc += _a[j];
                    v[j] = acc;
                }
            }
            _prefix[i - 1] = u;
            _suffix[i - 1] = v;
        }
    }

    public int Levels { get; }

    public long Sum(int l, int r) // inclusive
    {
        if (l == r)
            return _a[l];
        int level = BitLength(l ^ r) - 1; // first differing bit picks the level
        return _suffix[level][l] + _prefix[level][r];
    }
```

listing 1009: c#, the build loop per level, sum by first differing bit (source: dsa/samples/src/Ch31/DisjointSparse.cs, lines 24-60)

```
  }

  #build(i) {
    const half = 1 << (i - 1);
    const U = new Array(this.#n).fill(null); // prefix folds from mid
    const V = new Array(this.#n).fill(null); // suffix folds to mid
    for (let base = 0; base < this.#n; base += half * 2) {
      const mid = base + half;
      if (mid ≥ this.#n) break;
      let acc = 0;
      for (let j = mid; j < Math.min(mid + half, this.#n); j++) {
        acc += this.#a[j];
        U[j] = acc;
      }
      acc = 0;
      for (let j = mid - 1; j ≥ base; j--) {
        acc += this.#a[j];
        V[j] = acc;
      }
    }
    return [U, V];
  }

  sum(l, r) {
    // inclusive
    if (l === r) return this.#a[l];
    const x = l ^ r;
    const i = 32 - Math.clz32(x); // first differing bit picks the level
    const [U, V] = this.#levels[i - 1];
    return V[l] + U[r];
  }
```

listing 1010: javascript, the build and the clz32 level pick (source: dsa/samples-js/src/ch31-disjointsparse.mjs, lines 27-57)

```
    def _build(self, i):
        half = 1 << (i - 1)
        prefix = [None] * self.n  # folds from each block midpoint right
        suffix = [None] * self.n  # folds into each block midpoint left
        for base in range(0, self.n, half * 2):
            mid = base + half
            if mid >= self.n:
                break
            acc = 0
            for j in range(mid, min(mid + half, self.n)):
                acc += self.a[j]
                prefix[j] = acc
            acc = 0
            for j in range(mid - 1, base - 1, -1):
                acc += self.a[j]
                suffix[j] = acc
        return (suffix, prefix)

    def sum(self, lo, hi):  # inclusive, lo < hi takes the disjoint level
        if lo == hi:
            return self.a[lo]
        i = (lo ^ hi).bit_length()  # the first differing bit picks it
        suffix, prefix = self.levels[i - 1]
        return suffix[lo] + prefix[hi]
```

listing 1011: python, prefix and suffix folds per midpoint, the query (source: dsa/samples-py/src/Ch31/disjointsparse.py, lines 25-50)

```
function DisjointSparse:build(i)
  local half = 1 << (i - 1)
  local U, V = {}, {} -- prefix from each midpoint / suffix to it
  for base = 0, self.n - 1, half * 2 do
    local mid = base + half
    if mid >= self.n then break end
    local acc = 0
    for j = mid, math.min(mid + half, self.n) - 1 do -- 0-based walk
      acc = acc + self.a[j + 1]
      U[j] = acc
    end
    acc = 0
    for j = mid - 1, base, -1 do
      acc = acc + self.a[j + 1]
      V[j] = acc
    end
  end
  return {U, V}
end

function DisjointSparse:sum(l, r) -- inclusive, 0-based
  if l == r then return self.a[l + 1] end
  local x = l ~ r
  local bits = 0 -- bit length: the highest differing bit picks level
  while (x >> bits) > 0 do bits = bits + 1 end
  local level = self.levels[bits]
  return level[2][l] + level[1][r]
end
```

listing 1012: lua, the build and a hand-rolled bit length (source: dsa/samples-lua/ch31_disjointsparse.lua, lines 23-50)

The fixture array is 3, 1, 4, 1, 5, 9, 2, 6, 5, 3, n = 10, 4 levels. The eight pinned sums: (0,9) = 39, (2,5) = 19, (4,4) = 5, (3,8) = 28, (7,9) = 14, (1,6) = 22, (0,7) = 31, (2,7) = 27, every one matched by a prefix-sum brute. The overlap demo is the figure's point: query (2,7) answered chapter 21 style as two 4-wide covers reads 19 + 22 = 41 against the true 27, the middle 14 counted twice. The edges read l = r straight from the cell, build n = 1 as a single level, and span the padding boundary at (7,9) where the level's blocks run past the array end. Build is O(n log n), query O(1), and the table is read-only, the same permanence trade as the idempotent original.

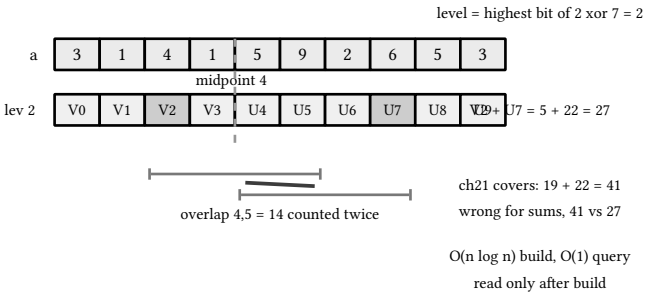


figure 257: two levels of half-block tables with query 2 to 7 shaded as suffix(2) plus prefix(7), the chapter 21 two-cover overlap crossed out

31.6. rollback dsu

The disjoint set union of book 9, ch 12 (practical data structures and algorithms, minimum spanning trees and flows) runs two optimizations, path compression and union by rank or size. Rewinding a union needs the second and forbids the first: compression flattens paths opportunistically and forgets the original parent chain, so there is nothing to undo. Union by size alone keeps find at O(log n), and every successful union becomes invertible by recording it: push the (child, parent) pair onto a stack, and rollback(k) unwinds to stack depth k, restoring the child's parent and the parent's size one pop at a time, O(1) per step. A union of two already-connected vertices pushes nothing, and the current stack depth is the snapshot.

The dry run: the fixture is the 6-vertex time-travel family, asserted by the C# suite, the walk reading the chapter's 1-based labels over the suite's 0-based calls.

1. Four unions (1,2), (3,4), (2,4), (5,6) each join two roots: the component count steps 5, 4, 3, 2 and the pair stack grows to depth 4, one push per union.
2. Union (4,6) joins the last two components: count 1 at depth 5.
3. Rollback to depth 4 unwinds the (4,6) pair, the child's parent and the parent's size return, and count reads 2 with 4 and 6 apart again.
4. The re-union (4,6) succeeds on the rewound state, count 1 at depth 5 again.
5. Rollback to depth 3 leaves 3 components, and rollback to 0 empties the stack and restores 6.
6. A duplicate union of an already connected pair pushes nothing: the stack stays at depth 1.

op	stack depth	components
union (1,2)	1	5
union (3,4)	2	4
union (2,4)	3	3
union (5,6)	4	2
union (4,6)	5	1
rollback to 4	4	2
union (4,6) again	5	1
rollback to 3	3	3
rollback to 0	0	6

The count returns to 6 with the stack empty, and the listings below union and unwind in seven languages.

```c
static int union_ab(int a, int b) {
  a = find(a);
  b = find(b);
  if (a == b)
    return 0;
  if (sz[a] < sz[b]) {
    int t = a;
    a = b;
    b = t;
  }
  parent[b] = a;
  sz[a] += sz[b];
  stk_child[depth] = b;
  stk_parent[depth] = a;
  depth++;
  return 1;
}

static void rollback(int k) {
  while (depth > k) {
    depth--;
    int b = stk_child[depth], a = stk_parent[depth];
    sz[a] -= sz[b];
    parent[b] = b;
  }
}

static int snapshot(void) { return depth; }
```

listing 1013: c, union pushes the pair, rollback unwinds it, snapshot is the depth (source: dsa/samples-c/src/Ch31/rollbackdsu.c, lines 45-72)

```
// connected.
func (d *RollbackDSU) Union(a, b int) bool {
  ra, rb := d.Find(a), d.Find(b)
  if ra == rb {
    return false
  }
  if d.sz[ra] < d.sz[rb] {
    ra, rb = rb, ra
  }
  d.parent[rb] = ra
  d.sz[ra] += d.sz[rb]
  d.stack = append(d.stack, [2]int{rb, ra})
  return true
}

// Rollback unwinds to stack depth k.
func (d *RollbackDSU) Rollback(k int) {
  for len(d.stack) > k {
    child, parent := d.stack[len(d.stack)-1][0],
      d.stack[len(d.stack)-1][1]
    d.stack = d.stack[:len(d.stack)-1]
    d.sz[parent] -= d.sz[child]
    d.parent[child] = child
  }
}

// Snapshot is the current stack depth.
func (d *RollbackDSU) Snapshot() int { return len(d.stack) }
```

listing 1014: go, union by size, rollback slicing the stack (source: dsa/samples-go/ch31/rollbackdsu.go, lines 42-70)

```
static int unionAb(int a, int b) {
  a = find(a);
  b = find(b);
  if (a == b)
    return 0;
  if (sz[a] < sz[b]) {
    int t = a;
    a = b;
    b = t;
  }
  parent[b] = a;
  sz[a] += sz[b];
  stkChild[depth] = b;
  stkParent[depth] = a;
  depth++;
  return 1;
}

static void rollback(int k) {
  while (depth > k) {
    depth--;
    int b = stkChild[depth], a = stkParent[depth];
    sz[a] -= sz[b];
    parent[b] = b;
  }
}

static int snapshot() {
  return depth;
}
```

listing 1015: java, union pushes the pair, rollback unwinds it, snapshot is the depth (source: dsa/samples-java/src/Ch31/Rollbackdsu.java, lines 44-73)

```
  public bool Union(int a, int b)
  {
      a = Find(a);
      b = Find(b);
      if (a == b)
          return false;
      if (_size[a] < _size[b])
          (a, b) = (b, a);
      _parent[b] = a;
      _size[a] += _size[b];
      _stack.Add((b, a));
      return true;
  }

  public void Rollback(int depth)
  {
      while (_stack.Count > depth)
      {
          (int child, int parent) = _stack[^1];
          _stack.RemoveAt(_stack.Count - 1);
          _size[parent] -= _size[child];
          _parent[child] = child;
      }
  }
```

listing 1016: c#, union and rollback over the pair stack (source: dsa/samples/src/Ch31/RollbackDsu.cs, lines 43-67)

```
union(a, b) {
  let ra = this.find(a);
  let rb = this.find(b);
  if (ra === rb) return false;
  if (this.#sz[ra] < this.#sz[rb]) {
    const t = ra;
    ra = rb;
    rb = t;
  }
  this.#p[rb] = ra;
  this.#sz[ra] += this.#sz[rb];
  this.#stack.push([rb, ra]);
  return true;
}

rollback(k) {
  while (this.#stack.length > k) {
    const [b, a] = this.#stack.pop();
    this.#sz[a] -= this.#sz[b];
    this.#p[b] = b;
  }
  return this;
}

snapshot() {
  return this.#stack.length;
```

listing 1017: javascript, union, rollback, snapshot (source: dsa/samples-js/src/ch31-rollbackdsu.mjs, lines 28-53)

```
    def union(self, a, b):
        a, b = self.find(a), self.find(b)
        if a == b:
            return False
        if self.sz[a] < self.sz[b]:
            a, b = b, a
        self.p[b] = a
        self.sz[a] += self.sz[b]
        self.stack.append((b, a))
        return True

    def rollback(self, k):
        while len(self.stack) > k:
            b, a = self.stack.pop()
            self.sz[a] -= self.sz[b]
            self.p[b] = b

    def snapshot(self):
        return len(self.stack)
```

listing 1018: python, the pair stack and the unwind (source: dsa/samples-py/src/Ch31/rollbackdsu.py, lines 27-45)

```
function RollbackDSU:union(a, b)
  a = self:find(a)
  b = self:find(b)
  if a == b then return false end
  if self.sz[a + 1] < self.sz[b + 1] then a, b = b, a end
  self.p[b + 1] = a
  self.sz[a + 1] = self.sz[a + 1] + self.sz[b + 1]
  self.stack[#self.stack + 1] = {b, a}
  return true
end

function RollbackDSU:rollback(k)
  while #self.stack > k do
    local top = table.remove(self.stack)
    local b, a = top[1], top[2]
    self.sz[a + 1] = self.sz[a + 1] - self.sz[b + 1]
    self.p[b + 1] = b
  end
end
```

listing 1019: lua, 1-based tables behind 0-based vertices (source: dsa/samples-lua/ch31_rollbackdsu.lua, lines 29-47)

The time-travel family runs on 6 vertices: the four unions (1,2), (3,4), (2,4), (5,6) walk the component count 5, 4, 3, 2 with snapshot depths 1 through 4, root(1) equals root(4) and differs from root(6). Union (4,6) reaches count 1, rollback to depth 4 restores count 2 and separates them again, the re-union succeeds on the rewound state, rollback to depth 3 gives 3 components, rollback to 0 gives 6, and a duplicate union of a connected pair leaves the stack at depth 1. The hop-meter family builds the binomial merge shape on 8 vertices, the worst union-by-size permits, and find from vertices 4, 6, and 7 each walks exactly 2 hops against the log2 8 = 3 bound. This structure serves icpc world finals 2025 problem G (book 10, chapter 13) through the engine below.

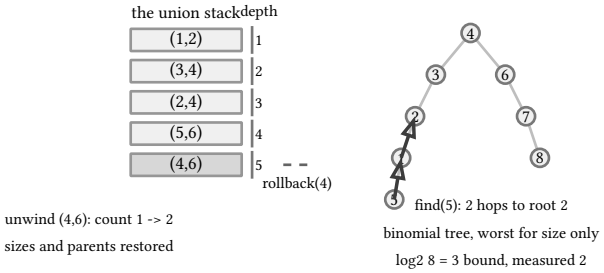


figure 258: the union stack with snapshot ticks and one unwind restoring two components, beside the binomial 8-vertex shape with a 2-hop find path

31.7. offline dynamic connectivity

Edge insertions and deletions defeat plain dsu, but when the whole timeline is known the problem turns static. Give every edge its lifetime as a half-open interval of times, insert it into the O(log T) segment tree nodes covering that interval, then run one dfs over the tree carrying a rollback dsu: on entering a node, union its edges, at a leaf answer the queries stamped with that time, and on leaving, unwind to the snapshot taken on entry. Every edge is unioned along its cover and rolled back on the way out, so the dsu at leaf t holds exactly the edges alive at t, and the total cost is O(E log T log V) with E the sum of interval covers.

The dry run: the fixture is 5 vertices over T = 8 with edges (1,2) alive [0,6), (2,3) on [1,4), (3,4) on [2,8), (1,4) on [5,7), and (4,5) on [7,8), asserted by the C# suite with a brute per-time dsu agreeing cell by cell.

1. The dfs sweeps the leaves t = 0 through 7, unioning on entry, answering at the leaf, unwinding on exit.
2. At t = 0 only (1,2) is alive: {1,2}, 3, 4, 5 read 4 components, and 1 and 3 sit apart.
3. t = 1 adds (2,3): {1,2,3} at 3 components, and conn(1,3) turns true.
4. t = 2 adds (3,4) for 2 components, and t = 3 holds the same 2.
5. t = 4 drops (2,3): {1,2} and {3,4} sit apart at 3 components, conn(1,3) false again.
6. t = 5 adds (1,4) for {1,2,3,4} at 2, true, and t = 6 drops (1,2) leaving {2} beside {1,3,4} at 3, still true.
7. t = 7 swaps (1,4) for (4,5): {3,4,5} leaves 1 alone at 3 components, false.

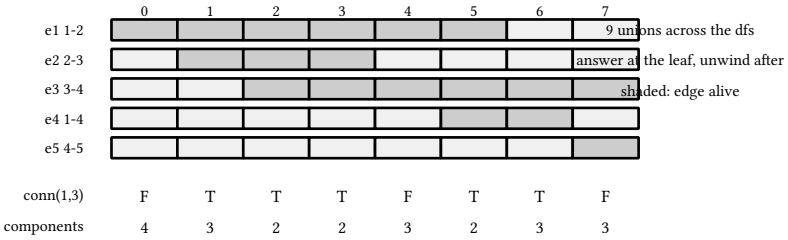


figure 259: the five lifetimes as an on and off strip across the eight times, the two answer rows read at the leaves

The sweep answers all eight times for the pinned 9 unions, and the listings below run the dfs in seven languages.

```c
static void dfs(int nd, int lo, int hi) {
  int mark = depth;
  for (int slot = se_head[nd]; slot; slot = li_next[slot]) {
    int ei = li_e[slot];
    union_ab(edges[ei - 1].u, edges[ei - 1].v); // ei is 1-based
  }
  if (hi - lo == 1) {
    for (int q = 0; q < QN; q++)
      conn_answer[q][lo] = find(queries[q][0]) == find(queries[q][1]);
    int roots = 0;
    for (int i = 0; i < 5; i++)
      if (find(i) == i)
        roots++;
    comp_answer[lo] = roots;
  } else {
    int mid = (lo + hi) / 2;
    dfs(2 * nd, lo, mid);
    dfs(2 * nd + 1, mid, hi);
  }
  rollback(mark);
}
```

listing 1020: c, the dfs: union on entry, answer at the leaf, unwind on exit (source: dsa/samples-c/src/Ch31/offlinedynconn.c, lines 103-123)

```go
  var dfs func(node, lo, hi int)
  dfs = func(node, lo, hi int) {
    mark := dsu.Snapshot()
    for _, e := range seg[node] {
      if dsu.Union(e.U, e.V) {
        res.TotalUnions++
      }
    }
    if hi-lo == 1 {
      for q, pair := range queries {
        res.Connected[q][lo] = dsu.Connected(pair[0], pair[1])
      }
      res.Components[lo] = vertices - dsu.Snapshot()
    } else {
      mid := (lo + hi) / 2
      dfs(2*node, lo, mid)
      dfs(2*node+1, mid, hi)
    }
    dsu.Rollback(mark)
  }
  dfs(1, 0, T)
  return res
}
```

listing 1021: go, the dfs closure, components as vertices minus unions (source: dsa/samples-go/ch31/offlinedynconn.go, lines 50-72)

```
static void dfs(int node, int lo, int hi) {
  int mark = depth;
  for (int slot = seHead[node]; slot != 0; slot = liNext[slot]) {
    int ei = liE[slot];
    unionAb(edges[ei - 1].u, edges[ei - 1].v); // ei is 1-based
  }
  if (hi - lo == 1) {
    for (int q = 0; q < QN; q++)
      connAnswer[q][lo] = find(queries[q][0]) == find(queries[q][1]);
    int roots = 0;
    for (int i = 0; i < 5; i++)
      if (find(i) == i)
        roots++;
    compAnswer[lo] = roots;
  } else {
    int mid = (lo + hi) / 2;
    dfs(2 * node, lo, mid);
    dfs(2 * node + 1, mid, hi);
  }
  rollback(mark);
}
```

listing 1022: java, the dfs over intrusive per-node edge lists, union on entry, unwind on exit (source: dsa/samples-java/src/Ch31/Offlinedynconn.java, lines 107-127)

```
        void Dfs(int nd, int lo, int hi)
        {
            int mark = dsu.SnapshotDepth;
            foreach (int e in seg[nd])
                if (dsu.Union(edges[e].U, edges[e].V))
                    unions++;
            if (hi - lo == 1)
            {
                for (int q = 0; q < queries.Length; q++)
                    conn[q][lo] = dsu.SameRoot(queries[q].U, queries[q].V);
                components[lo] = dsu.ComponentCount;
            }
            else
            {
                int mid = (lo + hi) / 2;
                Dfs(2 * nd, lo, mid);
                Dfs(2 * nd + 1, mid, hi);
            }
            dsu.Rollback(mark);
        }

        return new Result(conn, components, unions);
```

listing 1023: c#, the same dfs as a local function over the node lists (source: dsa/samples/src/Ch31/OfflineDynConn.cs, lines 33-54)

```
  let unions = 0;
  const conn = queryPairs.map(() => new Array(T).fill(false));
  const counts = new Array(T).fill(0);

  // the dfs: add on entry, unwind on exit, answer at the leaves
  const dfs = (nd, lo, hi, dsu) => {
    const mark = dsu.snapshot();
    for (const [u, v] of seg.get(nd) ?? []) {
      if (dsu.union(u, v)) unions += 1;
    }
    if (hi - lo === 1) {
      queryPairs.forEach(([x, y], q) => {
        conn[q][lo] = dsu.find(x) === dsu.find(y);
      });
      const roots = new Set();
      for (let i = 0; i < n; i++) roots.add(dsu.find(i));
      counts[lo] = roots.size;
    } else {
      const mid = (lo + hi) >> 1;
      dfs(2 * nd, lo, mid, dsu);
      dfs(2 * nd + 1, mid, hi, dsu);
```

listing 1024: javascript, the dfs over a node-to-edges map (source: dsa/samples-js/src/ch31-offlinedynconn.mjs, lines 26-47)

```
    def dfs(node, lo, hi):
        mark = dsu.snapshot()
        for u, v, _, _ in seg.get(node, ()):
            if dsu.union(u, v):
                unions[0] += 1
        if hi - lo == 1:
            for x, y in queries:
                answers[(x, y, lo)] = dsu.find(x) == dsu.find(y)
        else:
            mid = (lo + hi) // 2
            dfs(2 * node, lo, mid)
            dfs(2 * node + 1, mid, hi)
        dsu.rollback(mark)

    dfs(1, 0, horizon)
    return answers, unions[0]


T = 8
```

listing 1025: python, the dfs with the snapshot mark and rollback (source: dsa/samples-py/src/Ch31/offlinedynconn.py, lines 66-84)

```
local function dfs(nd, lo, hi)
  local mark = dsu:snapshot()
  for _, e in ipairs(seg[nd] or {}) do
    if dsu:union(e[1], e[2]) then unions = unions + 1 end
  end
  if hi - lo == 1 then
    conn[#conn + 1] = dsu:find(queries[1][1]) == dsu:find(queries[1][2])
    local roots = {}
    for i = 0, n - 1 do roots[dsu:find(i)] = true end
    local c = 0
    for _ in pairs(roots) do c = c + 1 end
    comps[#comps + 1] = c
  else
    local mid = (lo + hi) // 2
    dfs(2 * nd, lo, mid)
    dfs(2 * nd + 1, mid, hi)
  end
  dsu:rollback(mark)
end
dfs(1, 0, T)
```

listing 1026: lua, the dfs, answers appended leaf by leaf (source: dsa/samples-lua/ch31_offlinedynconn.lua, lines 69-88)

The fixture runs 5 vertices over T = 8: edges (1,2) alive [0,6), (2,3) on [1,4), (3,4) on [2,8), (1,4) on [5,7), and (4,5) on [7,8). Connectivity of 1 and 3 reads F, T, T, T, F, T, T, F across the eight times, the component counts read 4, 3, 2, 2, 3, 2, 3, 3, the dfs performs 9 unions total, and a brute per-time dsu agrees on every cell. The negative family holds: 1 and 5 are never connected, vertex 5 joins only at t = 7 when 1 is already isolated. The edges cover the whole horizon, an empty interval, and a query at t = 0 before any edge. Java keeps the timeline tree's per-node edge lists as an intrusive singly linked list over int arrays, slot 0 the terminator, the c arrangement rather than the map the dynamic languages reach for. This engine is icpc world finals 2025 problem G (book 10, chapter 13), lava moat: the level sweep runs exactly this machine over edge lifetimes with per-component length and border flags, at O(n log^2 n).

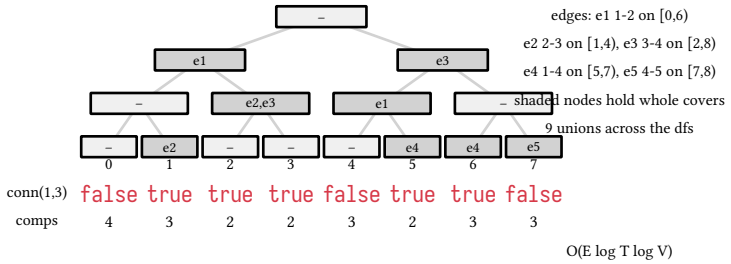


figure 260: the segment tree over times 0 to 7 with the five edge lifetimes as shaded node covers, two query rows answered at the leaves

31.8. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's seven sample files per language, embedded test scripts included:

language	build sloc	container dependency	boundary note
c	792	static arrays, qsort for mo	block ids by integer division, the empty-pop and empty-min sentinel flags
go	419	slices, maps, no imports beyond fmt and bits	components computed as vertices minus unions, snake_case file for the sparse table

language	build sloc	container dependency	boundary note
java	839	jdk 27 stdlib, Arrays.sort comparator for mo	the mo comparator breaks the index tie explicitly, intrusive int-array edge lists for the timeline tree, found-flag li chao sentinel
c#	396	arrays, tuples, dictionaries	nullable long for the li chao sentinel, bitlength by hand and by clz
javascript	324	map-keyed segment stores	stable sort carries the mo tie order, private methods throughout
python	554	lists, dicts, check harness	system exit on failure, ok-N print per file
lua	632	tables, 1-based inside	0-based public faces per the ch21 precedent, hand-rolled bit length

sources: cp-algorithms, “Sqrt Decomposition”, cp-algorithms.com/data_structures/sqrt_decomposition.html, whose mo’s algorithm section is the direct source for 31.2, “Minimum stack / Minimum queue”, cp-algorithms.com/data_structures/stack_queue_modification.html, “Convex hull trick and Li Chao tree”, cp-algorithms.com/geometry/convex_hull_trick.html, the second half of that article, “Sparse Table”, cp-algorithms.com/data_structures/sparse-table.html, the base structure of 31.5 whose disjoint variant is our own extension, and “Disjoint Set Union”, cp-algorithms.com/data_structures/disjoint_set_union.html, the substrate of 31.6 whose rollback variant is likewise ours, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. The offline connectivity engine of 31.7 has no dedicated article there and cites icpc world finals 2025 problem G (book 10, chapter 13) as its application source. Sample behavior verified by the seven suite gates scoped to chapter 31: c 7 files and 136 checks, go 21 test functions, java 7 files and 136 checks under run-java-samples, c# 23 facts, javascript 21 tests and 68 asserts, python 7 files and 122 asserts, lua 21 checks, zero skipped.

32. dynamic programming ii

Five dp shapes that buy asymptotic speedups by exploiting structure the plain table ignores. Divide and conquer dp replaces a quadratic per-layer scan with a recursion whose optimum windows narrow, Knuth's optimization shrinks interval dp root searches to amortized constant, broken profile walks grid columns as bitmasks, slope trick keeps convex cost functions as lazily-shifted heaps of breakpoints, and value iteration solves markov decision processes by repeated sweeps to the fixed point. The chapter's applications run through the icpc book: a 2017 staircase search, a 2022 heap-of-offers sweep, and a 2023 expected-rolls mdp.

32.1. divide and conquer dp

The layered partition dp: cut weights 4, 2, 7, 1, 5, 3 into exactly k non-empty consecutive segments minimizing the sum of squared segment sums plus 20 per segment. Layer k is computed from layer k-1, and the naive transition scans every split point for every position, O(k n^2) total. The cost function, a squared prefix-sum difference plus a constant, satisfies the quadrangle inequality, so the optimum split index is monotone in the position: the best j for position mid lies between the best j of the left half and the best j of the right half. The recursion rec(lo, hi, optl, optr) evaluates candidates for the midpoint only inside that inherited window, writes dp[k][mid], then hands each half its narrowed window.

The dry run: the fixture is the cut weights 4, 2, 7, 1, 5, 3 at 20 per segment, asserted by the C# suite with the same layers and optima pinned in the C, Java, and Python suites.

1. The prefix sums fold 4, 4 + 2 = 6, 6 + 7 = 13, 13 + 1 = 14, 14 + 5 = 19, 19 + 3 = 22.
2. k = 1 takes the whole run as one segment: 22 squared plus 20 = 504.
3. k = 2 splits after the 7: layer 1′s 189 for the 4, 2, 7 plus the tail's 81 plus 20 lands 189 + 81 + 20 = 290 over segments 4, 2, 7 and 1, 5, 3.
4. k = 3 splits 4, 2 | 7, 1 | 5, 3: the segment sums read 6, 8, 8, so 36 + 64 + 64 = 164 plus 3 × 20 = 60 closes at 224.
5. The brute enumeration of cut masks agrees on all three, and the meter reads 39 candidate evaluations against the full scan's 45.

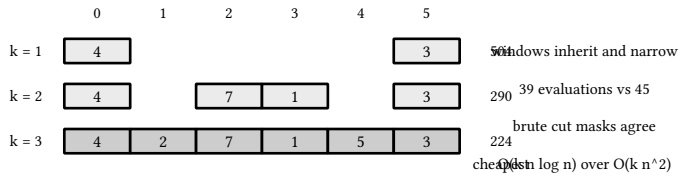


figure 261: the three optimal segmentations one row per k, groups boxed with their totals beside, the k = 3 row shaded cheapest

The 224 closes the ladder at 39 evaluations against 45, and the listings below fill the layers in seven languages.

```c
static void rec(int k, int lo, int hi, int optl, int optr) {
  if (lo > hi)
    return;
  int mid = (lo + hi) / 2;
  long long best = INF;
  int bestj = optl;
  int jhi = optr < mid - 1 ? optr : mid - 1;
  for (int j = optl; j ≤ jhi; j++) { // j = last index of the (k-1)-prefix
    evals++;
    long long s = pre[mid] - pre[j];
    long long c = dp[k - 1][j] + s * s + KCOST;
    if (c < best) {
      best = c;
      bestj = j;
    }
  }
  dp[k][mid] = best;
  rec(k, lo, mid - 1, optl, bestj);
  rec(k, mid + 1, hi, bestj, optr);
}

// layers 1..k filled; returns dp[k][AN]
```

listing 1027: c, the recursion: midpoint scan inside the inherited window, halves narrowed by bestj (source: dsa/samples-c/src/Ch32/dncdp.c, lines 34-55)

```go
    var rec func(lo, hi, optl, optr int)
    rec = func(lo, hi, optl, optr int) {
      if lo > hi {
        return
      }
      mid := (lo + hi) / 2
      best := dncInf
      bestJ := optl
      hiJ := optr
      if mid-1 < hiJ {
        hiJ = mid - 1
      }
      for j := optl; j <= hiJ; j++ {
        evals++
        if layers[k-1][j] ≥ dncInf {
          continue
        }
        s := pre[mid] - pre[j]
        c := layers[k-1][j] + s*s + 20
        if c < best {
          best, bestJ = c, j
        }
      }
      nd[mid] = best
      rec(lo, mid-1, optl, bestJ)
      rec(mid+1, hi, bestJ, optr)
    }
```

listing 1028: go, the closure recursion, infeasible candidates skipped (source: dsa/samples-go/ch32/dncdp.go, lines 42-68)

```
  static void rec(int k, int lo, int hi, int optl, int optr) {
    if (lo > hi)
      return;
    int mid = (lo + hi) / 2;
    long best = INF;
    int bestj = optl;
    int jhi = Math.min(optr, mid - 1);
    for (int j = optl; j ≤ jhi; j++) { // j = last index of the (k-1)-prefix
      evals++;
      long s = pre[mid] - pre[j];
      long c = dp[k - 1][j] + s * s + KCOST;
      if (c < best) {
        best = c;
        bestj = j;
      }
    }
    dp[k][mid] = best;
    rec(k, lo, mid - 1, optl, bestj);
    rec(k, mid + 1, hi, bestj, optr);
  }
```

listing 1029: java, the recursion: midpoint scan inside the inherited window, halves narrowed by bestj (source: dsa/samples-java/src/Ch32/Dncdp.java, lines 32-51)

```
            void Rec(int lo, int hi, int optl, int optr)
            {
                if (lo > hi)
                    return;
                int mid = (lo + hi) / 2;
                long best = inf;
                int bestJ = optl;
                // j = last index of the (k-1)-prefix; segment is j..mid-1
                for (int j = optl; j ≤ Math.Min(optr, mid - 1); j++)
                {
                    evals++;
                    if (prev[j] == inf)
                        continue;
                    long s = pre[mid] - pre[j];
                    long c = prev[j] + s * s + SegmentCost;
                    if (c < best)
                    {
                        best = c;
                        bestJ = j;
                    }
                }
                cur[mid] = best;
                Rec(lo, mid - 1, optl, bestJ);
                Rec(mid + 1, hi, bestJ, optr);
            }
```

listing 1030: c#, the recursion as a local function over the two layer arrays (source: dsa/samples/src/Ch32/DncDp.cs, lines 37-61)

```
  for (let k = 1; k ≤ kmax; k++) {
    const nd = new Array(n + 1).fill(Infinity);
    // dp[k][mid] over j in [optl, min(optr, mid-1)]: the last segment
    // is a[j..mid-1], the rest is layer k-1
    const rec = (lo, hi, optl, optr) => {
      if (lo > hi) return;
      const mid = (lo + hi) >> 1;
      let best = Infinity;
      let bestj = optl;
      for (let j = optl; j ≤ Math.min(optr, mid - 1); j++) {
        evals += 1;
        const s = pre[mid] - pre[j];
        const c = layers[k - 1][j] + s * s + SEG_K;
        if (c < best) {
          best = c;
          bestj = j;
        }
      }
      nd[mid] = best;
      rec(lo, mid - 1, optl, bestj);
      rec(mid + 1, hi, bestj, optr);
    };
    rec(k, n, k - 1, n - 1); // position i ≥ k, j in [k-1, i-1]
    layers[k] = nd;
```

listing 1031: javascript, the recursion, the last segment j to mid-1 (source: dsa/samples-js/src/ch32-dncdp.mjs, lines 67-90)

```
    def rec(lo, hi, optl, optr):
        nonlocal evals
        if lo > hi:
            return
        mid = (lo + hi) // 2
        best = inf
        bestj = optl
        for j in range(optl, min(optr, mid - 1) + 1):
            evals += 1
            seg = pre[mid] - pre[j]
            cost = layers[k - 1][j] + seg * seg + K_COST
            if cost < best:
                best, bestj = cost, j
        nd[mid] = best
        rec(lo, mid - 1, optl, bestj)
        rec(mid + 1, hi, bestj, optr)

    for k in range(1, kmax + 1):
        nd = [inf] * (size + 1)
        rec(k, size, k - 1, size - 1)  # position i ≥ k, j in [k-1, i-1]
        layers[k] = nd
    return layers, evals
```

listing 1032: python, the recursion and the per-layer entry call (source: dsa/samples-py/src/Ch32/dncdp.py, lines 32-53)

```
  for i = 0, n do nd[i] = INF end
  local function rec(lo, hi, optl, optr)
    if lo > hi then return end
    local mid = (lo + hi) // 2
    local best, bestj = INF, optl
    -- j = last index (1-based element count) of the (k-1)-prefix;
    -- the k-th segment covers elements j+1 .. mid
    for j = optl, math.min(optr, mid - 1) do
      evals = evals + 1
      local s = pre[mid] - pre[j]
      local c = layers[k - 1][j] + s * s + K
      if c < best then best, bestj = c, j end
    end
    nd[mid] = best
    rec(lo, mid - 1, optl, bestj)
    rec(mid + 1, hi, bestj, optr)
  end
  rec(k, n, k - 1, n - 1)
```

listing 1033: lua, the recursion, 1-based elements threaded through the window (source: dsa/samples-lua/ch32_dncdp.lua, lines 56-73)

The layer table pins the whole computation: layer 1 reads 36, 56, 189, 216, 381, 504 over the first i elements, layer 2 reads infeasible, 60, 125, 140, 245, 290, layer 3 reads infeasible, infeasible, 129, 144, 181, 224. The optima land at k = 1 cost 504, the whole array as 22 squared plus 20, k = 2 cost 290 with segments 4, 2, 7 and 1, 5, 3, and k = 3 cost 224, every k matched against a brute enumeration of all cut masks. The meter is the point: 39 candidate evaluations across three layers against 45 for the full scan, 15 per layer. The edges cover k = n, all singles, k = 1, and the k past n layers staying infeasible. The application is icpc world finals 2017 problem D (book 10, chapter 8), money for nothing: after pruning producers and consumers to staircases, the best consumer index for the middle producer is found by exactly this monotone-optimum divide and conquer at O((m+n) log n).

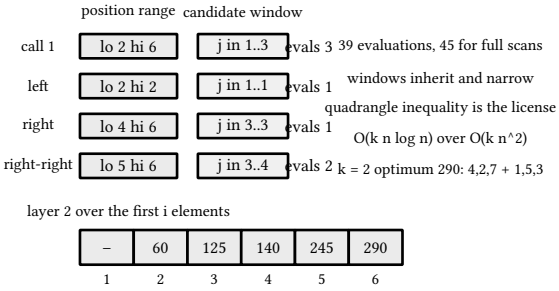


figure 262: layer 2′s divide tree over the six positions with the optimum windows narrowing, 39 candidate evaluations against the full scan's 45

32.2. the knuth optimization

Optimal binary search tree over keys 1..n with access weights 4, 2, 6, 3, 5: dp over every key span, cost the sum of weights times depth plus one, and the naive transition tries every root. Knuth's observation is that the optimal root of a span moves monotonically as the span grows, so the root search for span (i, j) consults only root candidates between root[i][j-1] and root[i+1][j], the optimal roots of the two spans one cell shorter. The conditions behind it are the quadrangle inequality plus the monotonicity of opt, and matrix chain multiplication is the other classic instance of the same shape. The windows shrink the root search so hard that each of the O(n^2) states pays O(1) amortized consultation.

The dry run: the fixture is the access weights 4, 2, 6, 3, 5, asserted by the C# suite with the same 39 and 18 in the C, Java, and Python suites.

1. The length-1 spans root themselves, costs 4, 2, 6, 3, 5 on the diagonal.

2. Span (0,1) inherits the window [0, 1] from the two diagonal roots: root 0 pays 2 + 6 = 8, root 1 pays 4 + 6 = 10, root 0 wins.
3. Span (0,2) inherits [root(0,1), root(1,2)] = [0, 2]: three candidates, root 2 pays 8 + 0 + 12 = 20 and wins.
4. The windows collapse as spans grow: (1,3) inherits [2, 2] and consults one root at 2 + 3 + 11 = 16, and both length-4 spans and the full span consult exactly one root each.
5. The full span (0,4) consults only root 2: left 8 + right 11 + 20 = 39, the pinned cost against 42 brute shapes.
6. The meter: length 2 consults 8 candidates, length 3 consults 7, lengths 4 and 5 consult 2 + 1 = 3, so 8 + 7 + 2 + 1 = 18 against the full scans' 35.

span length	root windows	consults	running
2	0-1, 1-2, 2-3, 3-4	8	8
3	0-2, 2-2, 2-4	7	15
4	2-2, 2-2	2	17
5	2-2	1	18

The full span pays one candidate for the pinned 39, and the listings below fill the windows in seven languages.

```
      root[i][i] = i;
  }
  for (int ln = 2; ln ≤ m; ln++) {
    for (int i = 0; i + ln - 1 < m; i++) {
      int j = i + ln - 1;
      long long W = pre[j + 1] - pre[i];
      long long best = INF;
      int bestr = root ? root[i][j - 1] : i;
      int rlo = root ? root[i][j - 1] : i;
      int rhi = root ? root[i + 1][j] : j;
      for (int r = rlo; r ≤ rhi; r++) {
        cands++;
        long long left = r > i ? dp[i][r - 1] : 0;
        long long right = r < j ? dp[r + 1][j] : 0;
        long long c = left + right + W;
        if (c < best) {
          best = c;
          bestr = r;
        }
      }
      dp[i][j] = best;
      if (root)
        root[i][j] = bestr;
    }
  }
  return dp[0][m - 1];
}
```

listing 1034: c, the length loop with the root window, candidates metered (source: dsa/samples-c/src/Ch32/knuth.c, lines 38-65)

```
    dp[i] = make([]int64, n)
    roots[i] = make([]int, n)
    dp[i][i] = w[i]
    roots[i][i] = i
  }
  for ln := 2; ln <= n; ln++ {
    for i := 0; i+ln-1 < n; i++ {
      j := i + ln - 1
      spanW := pre[j+1] - pre[i]
      best := inf
      bestr := roots[i][j-1]
      for r := roots[i][j-1]; r <= roots[i+1][j]; r++ {
        cands++
        left, right := int64(0), int64(0)
        if r > i {
          left = dp[i][r-1]
        }
        if r < j {
          right = dp[r+1][j]
        }
        if c := left + right + spanW; c < best {
          best, bestr = c, r
```

listing 1035: go, the diagonal and length loops, window from the root table (source: dsa/samples-go/ch32/knuth.go, lines 21-42)

```
    for (int ln = 2; ln ≤ m; ln++) {
      for (int i = 0; i + ln - 1 < m; i++) {
        int j = i + ln - 1;
        long bigW = pre[j + 1] - pre[i];
        long best = INF;
        int bestr = trackRoot ? rt[i][j - 1] : i;
        int rlo = trackRoot ? rt[i][j - 1] : i;
        int rhi = trackRoot ? rt[i + 1][j] : j;
        for (int r = rlo; r ≤ rhi; r++) {
          cands++;
          long left = r > i ? dp[i][r - 1] : 0;
          long right = r < j ? dp[r + 1][j] : 0;
          long c = left + right + bigW;
          if (c < best) {
            best = c;
            bestr = r;
          }
        }
        dp[i][j] = best;
        if (trackRoot)
          rt[i][j] = bestr;
      }
    }
    return dp[0][m - 1];
  }
```

listing 1036: java, the length loop with the root window, candidates metered (source: dsa/samples-java/src/Ch32/Knuth.java, lines 38-62)

```
                long best = long.MaxValue;
                int bestR = roots[i][j - 1];
                // the monotone root window, O(1) amortized consultation
                for (int r = roots[i][j - 1]; r ≤ roots[i + 1][j]; r++)
                {
                    candidates++;
                    long left = r > i ? dp[i, r - 1] : 0;
                    long right = r < j ? dp[r + 1, j] : 0;
                    long c = left + right + w;
                    if (c < best)
                    {
                        best = c;
                        bestR = r;
                    }
                }
                dp[i, j] = best;
                roots[i][j] = bestR;
            }
        return (dp[0, n - 1], roots, candidates);
```

listing 1037: c#, the window loop, the root table filled beside the cost (source: dsa/samples/src/Ch32/Knuth.cs, lines 32-50)

```
  for (let ln = 2; ln ≤ n; ln++) {
    for (let i = 0; i + ln - 1 < n; i++) {
      const j = i + ln - 1;
      const W = pre[j + 1] - pre[i];
      let best = Infinity;
      let bestr = rt[i][j - 1];
      for (let r = rt[i][j - 1]; r ≤ rt[i + 1][j]; r++) {
        cands += 1;
        const left = r > i ? dp[i][r - 1] : 0;
        const right = r < j ? dp[r + 1][j] : 0;
        const c = left + right + W;
        if (c < best) {
          best = c;
          bestr = r;
        }
      }
      dp[i][j] = best;
      rt[i][j] = bestr;
    }
  }
  return { cost: dp[0][n - 1], roots: rt, cands };
}

// brute ground truth: enumerate every bst shape over the span
export function obstBrute(w) {
  const build = (i, j, depth) ⇒ {
    if (i > j) return 0;
    let bc = Infinity;
    for (let r = i; r ≤ j; r++) {
```

listing 1038: javascript, diagonal init then the length loop, r bounded by the two roots (source: dsa/samples-js/src/ch32-knuth.mjs, lines 19-47)

```
        rt[i][i] = i
    for length in range(2, size + 1):
        for i in range(0, size - length + 1):
            j = i + length - 1
            total = pre[j + 1] - pre[i]
            best = inf
            bestr = rt[i][j - 1]
            for r in range(rt[i][j - 1], rt[i + 1][j] + 1):
                cands += 1
                left = dp[i][r - 1] if r > i else 0
                right = dp[r + 1][j] if r < j else 0
                cost = left + right + total
                if cost < best:
                    best, bestr = cost, r
            dp[i][j] = best
            rt[i][j] = bestr
    return dp, rt, cands
```

listing 1039: python, the length loop, the window range from the root table (source: dsa/samples-py/src/Ch32/knuth.py, lines 26-43)

```
  dp[i][i] = w[i + 1]
  rt[i][i] = i
end
local cands = 0
for ln = 2, n do
  for i = 0, n - ln do
    local j = i + ln - 1
    local Wt = pre[j + 1] - pre[i]
    local best, bestr = INF, rt[i][j - 1]
    for r = rt[i][j - 1], rt[i + 1][j] do -- the monotone window
      cands = cands + 1
      local left = (r > i) and dp[i][r - 1] or 0
      local right = (r < j) and dp[r + 1][j] or 0
      local c = left + right + Wt
      if c < best then best, bestr = c, r end
    end
    dp[i][j] = best
    rt[i][j] = bestr
  end
end
```

listing 1040: lua, the length loop, the monotone window in one range (source: dsa/samples-lua/ch32_knuth.lua, lines 34-53)

The fixture weights give optimal cost 39, the weighted path length with the root at depth 0, and the brute ground truth agrees by enumerating all Catalan(5) = 42 shapes. The root table pins as rows 0, 0, 2, 2, 2, then infeasible, 1, 2, 2, 2, then 2, 2, 2, then 3, 4, then 4, all 0-based key indices. The meter reads 18 root candidates consulted against 35 for the full scans. The edges: n = 1 costs its own weight 7, weights 1, 9 cost 11, four 3s cost 24, and 2, 8 cost 12, every one matched against the shape enumeration. No finals problem in the corpus rides Knuth directly, so it stays a general technique carried by its own ground truth.

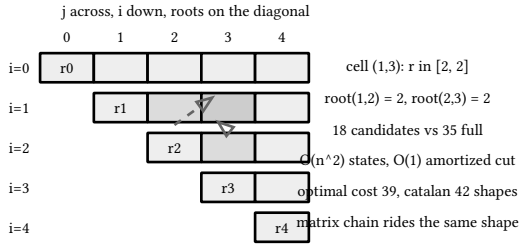


figure 263: the dp grid over key spans with one cell's root window between the roots of the two shorter spans

32.3. broken profile and the largest zero submatrix

Two grid dps in one file. Broken profile counts domino tilings column by column, carrying the mask of cells that protrude from the previous column. The per-column transition fills row by row: a cell already occupied by a protrusion is skipped, a free cell with a free neighbor below takes a vertical domino, and otherwise a horizontal domino protrudes into the next column's mask. Each column is one sweep of this recursion over all reachable masks, and the tiling count is the ways to return to the empty mask after the last column. The largest zero submatrix is the second machine: per row, maintain the histogram of consecutive zero heights, and take the maximum rectangle via the monotonic stack, the next-greater scan of book 9, ch 21 (practical data structures and algorithms, range queries) in its histogram role.

The dry run: the fixture is the 4 by 6 zero grid with rows 010000, 010010, 000010, 010010, asserted by the C# suite at area 8, with the domino ladder 2, 3, 5, 8 pinned beside it.

1. Row 0 reads 0,1,0,0,0,0: every open cell climbs to height 1 and the blocked cell resets, heights 1, 0, 1, 1, 1, 1, best 1 × 4 = 4 across columns 2 to 5.
2. Row 1 reads 0,1,0,0,1,0: heights 2, 0, 2, 2, 0, 2, best 2 × 2 = 4 at columns 2 and 3.
3. Row 2 reads 0,0,0,0,1,0: heights 3, 1, 3, 3, 0, 3, best 3 × 2 = 6 at columns 2 and 3.
4. Row 3 reads 0,1,0,0,1,0: heights 4, 0, 4, 4, 0, 4, best 4 × 2 = 8 at columns 2 and 3, rows 0 through 3.
5. The stack prices each row's histogram in one pass, and the sweep closes at the pinned 8.

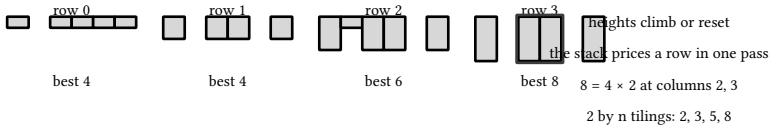


figure 264: the histogram sweep one group per row, heights climbing until a 1 resets, the winning 4 by 2 rectangle outlined on the last group

The sweep closes at the pinned 8 over all four rows, and the listings below carry both machines in seven languages.

```c
// one column's fill: row by row, vertical pairs or horizontal pushes
static void fill(int m, int row, int mask_out, long long ways, int occ) {
  if (row == m) {
    nxt[mask_out] += ways;
    return;
  }
  if (occ & (1 << row)) {
    fill(m, row + 1, mask_out, ways, occ);
    return;
  }
  if (row + 1 < m && !(occ & (1 << (row + 1))))
    fill(m, row + 2, mask_out, ways, occ);            // vertical
  fill(m, row + 1, mask_out | (1 << row), ways, occ); // protrudes right
}

static long long dominoes(int m, int ncols) {
  memset(cur, 0, sizeof(cur));
  cur[0] = 1;
  for (int c = 0; c < ncols; c++) {
    memset(nxt, 0, sizeof(nxt));
    for (int mask = 0; mask < (1 << m); mask++)
      if (cur[mask])
        fill(m, 0, 0, cur[mask], mask);
    memcpy(cur, nxt, sizeof(cur));
  }
  return cur[0];
}

// largest zero submatrix: returns area, reports the canonical rectangle
```

listing 1041: c, the fill recursion, vertical pairs or horizontal protrusions (source: dsa/samples-c/src/Ch32/brokenprofile.c, lines 27-55)

```
func DominoTilings(rows, cols int) int64 {
  dp := map[int]int64{0: 1}
  for c := 0; c < cols; c++ {
    ndp := map[int]int64{}
    var fill func(row, maskOut int, ways int64, occ int)
    fill = func(row, maskOut int, ways int64, occ int) {
      if row == rows {
        ndp[maskOut] += ways
        return
      }
      if occ&(1<<row) != 0 { // already filled from the left
        fill(row+1, maskOut, ways, occ)
        return
      }
      if row+1 < rows && occ&(1<<(row+1)) == 0 { // vertical
        fill(row+2, maskOut, ways, occ)
      }
      fill(row+1, maskOut|1<<row, ways, occ) // horizontal
    }
    for mask, ways := range dp {
      fill(0, 0, ways, mask)
    }
    dp = ndp
  }
  return dp[0]
}

// LargestZeroSubmatrix returns the area and the canonical position
// (c1, c2, r1, r2) of the largest all-zero rectangle: per-row
// histogram of consecutive zero heights, max rectangle via the
// monotonic stack, the chapter 21 next-greater kin.
```

listing 1042: go, the fill recursion inside the column loop (source: dsa/samples-go/ch32/brokenprofile.go, lines 7-37)

```
  // one column's fill: row by row, vertical pairs or horizontal pushes
  static void fill(int m, int row, int maskOut, long ways, int occ) {
    if (row == m) {
      nxt[maskOut] += ways;
      return;
    }
    if ((occ & (1 << row)) != 0) {
      fill(m, row + 1, maskOut, ways, occ);
      return;
    }
    if (row + 1 < m && (occ & (1 << (row + 1))) == 0)
      fill(m, row + 2, maskOut, ways, occ); // vertical
    fill(m, row + 1, maskOut | (1 << row), ways, occ); // protrudes right
  }

  static long dominoes(int m, int ncols) {
    java.util.Arrays.fill(cur, 0);
    cur[0] = 1;
    for (int c = 0; c < ncols; c++) {
      java.util.Arrays.fill(nxt, 0);
      for (int mask = 0; mask < (1 << m); mask++)
        if (cur[mask] != 0)
          fill(m, 0, 0, cur[mask], mask);
      long[] t = cur;
      cur = nxt;
      nxt = t;
    }
    return cur[0];
  }
```

listing 1043: java, the fill recursion over array masks, vertical pairs or horizontal protrusions (source: dsa/samples-java/src/Ch32/Brokenprofile.java, lines 25-53)

```
    public static Rect? LargestZeroSubmatrix(int[][] grid)
    {
        int rows = grid.Length;
        int cols = grid[0].Length;
        var h = new long[cols];
        long best = 0;
        Rect? where = null;
        for (int r = 0; r < rows; r++)
        {
            for (int c = 0; c < cols; c++)
                h[c] = grid[r][c] == 0 ? h[c] + 1 : 0;
            var stack = new List<(int Start, long Height)>(); // rising heights
            for (int c = 0; c ≤ cols; c++)
            {
                long cur = c == cols ? 0 : h[c];
                int start = c;
                while (stack.Count > 0 && stack[^1].Height ≥ cur)
                {
                    (int idx, long ht) = stack[^1];
                    stack.RemoveAt(stack.Count - 1);
                    long area = ht * (c - idx);
                    if (area > best)
                    {
                        best = area;
                        where = new Rect(area, idx, c - 1, r - (int)ht + 1, r);
                    }
                    start = idx;
                }
                stack.Add((start, cur));
            }
        }
        return where;
    }
}
```

listing 1044: c#, the histogram stack half, the fill recursion above it (source: dsa/samples/src/Ch32/BrokenProfile.cs, lines 44-77)

```javascript
export function dominoes(m, ncols) {
  let dp = new Map([[0, 1]]);
  for (let c = 0; c < ncols; c++) {
    const ndp = new Map();
    // fill column c: occ = protrusions from column c-1, mask_out =
    // protrusions into column c+1
    const fill = (row, maskOut, ways, occ) => {
      if (row === m) {
        ndp.set(maskOut, (ndp.get(maskOut) ?? 0) + ways);
        return;
      }
      if (occ & (1 << row)) {
        fill(row + 1, maskOut, ways, occ); // already covered
        return;
      }
      if (row + 1 < m && !(occ & (1 << (row + 1)))) {
        fill(row + 2, maskOut, ways, occ); // vertical domino
      }
      fill(row + 1, maskOut | (1 << row), ways, occ); // horizontal
    };
    for (const [mask, ways] of dp) fill(0, 0, ways, mask);
    dp = ndp;
  }
  return dp.get(0) ?? 0;
}
```

listing 1045: javascript, the fill over map-keyed mask counts (source: dsa/samples-js/src/ch32-brokenprofile.mjs, lines 9-34)

```python
def dominoes(rows, cols):
    # dp over columns; the fill walks the column row by row emitting
    # vertical or horizontal placements into the outgoing mask
    dp = {0: 1}
    for _ in range(cols):
        ndp = {}

        def fill(row, mask_out, ways, occupied):
            if row == rows:
                ndp[mask_out] = ndp.get(mask_out, 0) + ways
                return
            if occupied & (1 << row):  # covered from the previous column
                fill(row + 1, mask_out, ways, occupied)
                return
            if row + 1 < rows and not (occupied & (1 << (row + 1))):
                fill(row + 2, mask_out, ways, occupied)  # vertical domino
            fill(row + 1, mask_out | (1 << row), ways, occupied)  # horizontal

        for mask, ways in dp.items():
            fill(0, 0, ways, mask)
        dp = ndp
    return dp.get(0, 0)
```

listing 1046: python, the fill recursion, dict masks (source: dsa/samples-py/src/Ch32/brokenprofile.py, lines 14-35)

```lua
local function dominoes(m, ncols)
  local dp = {[0] = 1}
  for _ = 1, ncols do
    local ndp = {}
    local function fill(row, mask_out, ways, occ)
      if row == m then
        ndp[mask_out] = (ndp[mask_out] or 0) + ways
        return
      end
      if occ & (1 << row) ~= 0 then
        fill(row + 1, mask_out, ways, occ) -- already protruding
        return
      end
      if row + 1 < m and occ & (1 << (row + 1)) == 0 then
        fill(row + 2, mask_out, ways, occ) -- vertical placement
      end
      fill(row + 1, mask_out | (1 << row), ways, occ) -- horizontal
    end
    for mask, ways in pairs(dp) do
      fill(0, 0, ways, mask)
    end
    dp = ndp
  end
  return dp[0] or 0
end
```

listing 1047: lua, the fill, protrusions carried as bits (source: dsa/samples-lua/ch32_brokenprofile.lua, lines 8-32)

The tiling family pins the doubling: 2 by 2 gives 2, 2 by 3 gives 3, 2 by 4 gives 5, 2 by 5 gives 8, the fibonacci cross-check, 3 by 3 gives 0, 3 by 4 gives 11, 4 by 4 gives 36, and the 1-row grids give 1 and 0. The submatrix family runs the 4 by 6 grid with rows 010000, 010010, 000010, 010010 and finds area 8 at columns 2 and 3, all four rows. The edges: an all-ones 2 by 2 gives 0, an all-zeros 2 by 2 gives the whole grid at 4, a single row 0, 1, 0, 0 gives 2 at columns 2 and 3, and the reference cross-checked 300 random grids against brute-force rectangles. Tiling runs O(n 2^m) and the submatrix O(rows times cols). The counting cousin, the tiling-tiles style sweep of icpc world finals 2023 problem F (book 10, chapter 12), stays in the icpc book.

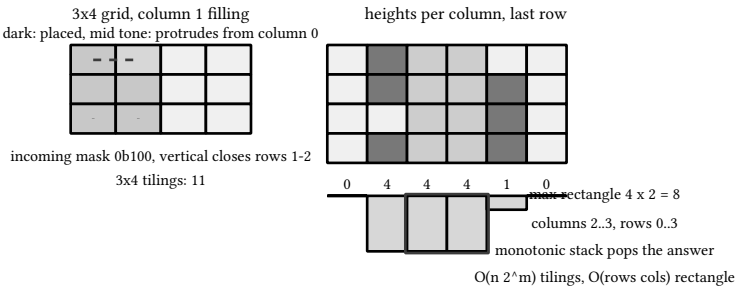


figure 265: the 3 by 4 grid mid-sweep with its protrusion mask, beside the 4 by 6 zero matrix as height bars with the 8-cell rectangle shaded

32.4. slope trick

The convex dp family, solved as heaps. Start with the two-heap median oracle: 3, 1, 4, 1, 5, 9, 2, 6 has lower median 3 and absolute-deviation cost 17. The real machine is the make-non-decreasing dp: keep a max-heap of the left-branch breakpoints of a convex piecewise-linear cost function, push each x, and when the top exceeds x pay top minus x, pop and repush x. The payment is the slope trick: the repush is the lazy right-shift of every breakpoint above x, applied by one heap operation instead of a rebuild. Strictly increasing goes through the index shift z i = a i

- i, a whole-branch translate applied outside the heap.

The dry run: the fixture is 3, 2, 5, 1, asserted by the C# suite at cost 5 with the final multiset 2, 2, 1, 1 and the shift trace 0, 1, 0, 4.

1. x = 3 pushes and the top equals it: no crossing, no payment, running 0.
2. x = 2: the pushed top reads 3, a crossing: pay 3 - 2 = 1, pop it, repush 2, and the heap holds 2, 2 at running 1.
3. x = 5: the top reads 5 and does not cross: heap 5, 2, 2, running still 1.
4. x = 1: the top 5 crosses by 4: pay 5 - 1 = 4, pop, repush 1, and the heap closes at 2, 2, 1, 1, running 5.
5. Every payment is the cost function's minimum rising: the running answers read 0, 1, 1, 5, the pinned trace.
6. Strictly increasing runs the same loop over z = 3, 1, 3, −2 and pays 7.

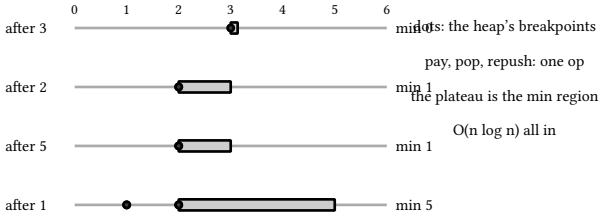


figure 266: the running cost function after each element, the heap's breakpoints as dots, the flat minimum shaded, its value rising 0, 1, 1, 5

The trace closes at the pinned pair, cost 5 and heap 2, 2, 1, 1, and the listings below pay the crossings in seven languages.

```
static long long slope_plain(const int *a, int len, long long *shifts,
                             long long *running) {
  hclear();
  long long ans = 0;
  for (int i = 0; i < len; i++) {
    hpush(a[i]);
    long long moved = 0;
    if (hp[0] > a[i]) {
      moved = hp[0] - a[i];
      ans += moved;
      hpop();
      hpush(a[i]);
    }
    if (shifts)
      shifts[i] = moved;
    if (running)
      running[i] = ans;
  }
  return ans;
}

// (c) strictly increasing through z_i = a_i - i
```

listing 1048: c, the one-heap loop, the payment and the lazy repush (source: dsa/samples-c/src/Ch32/slopetrick.c, lines 84-105)

```
  var h maxHeap
  cost := int64(0)
  for _, x := range a {
    heap.Push(&h, x)
    if h[0] > x {
      cost += h[0] - x
      heap.Pop(&h)
      heap.Push(&h, x)
    }
  }
  return cost, h
}
```

listing 1049: go, slopeRun, the heap.Push and Pop pair at the crossing (source: dsa/samples-go/ch32/slopetrick.go, lines 80-92)

```
  // (b) the one-heap machine; optionally record per-element shifts and
  // the running answer
  static long slopePlain(int[] a, int len, boolean record) {
    hclear();
    long ans = 0;
    for (int i = 0; i < len; i++) {
      hpush(a[i]);
      long moved = 0;
      if (hp[0] > a[i]) {
        moved = hp[0] - a[i];
        ans += moved;
        hpop();
        hpush(a[i]);
      }
      if (record) {
        shiftsOut[i] = moved;
        runningOut[i] = ans;
      }
    }
    return ans;
  }
```

listing 1050: java, the one-heap loop over the hand-rolled long max-heap, payment and lazy repush (source: dsa/samples-java/src/Ch32/Slopetrick.java, lines 87-107)

```
        void Push(long v) => heap.Enqueue(-v, -v);
        long Top() => -heap.Peek();
        long cost = 0;
        foreach (int x in a)
        {
            Push(x);
            if (Top() > x)
            {
                cost += Top() - x; // the shift payment
                heap.Dequeue();
                Push(x); // the lazy right-shift of every breakpoint above x
            }
        }
        var drained = new long[heap.Count];
        for (int i = 0; i < drained.Length; i++)
            drained[i] = -heap.Dequeue(); // max-heap drains descending
        return (cost, drained);
    }
```

listing 1051: c#, the payment loop over a PriorityQueue as a negated max-heap (source: dsa/samples/src/Ch32/SlopeTrick.cs, lines 23-41)

```
export function slopePlain(a) {
  const q = new MaxHeap();
  let ans = 0;
  for (const x of a) {
    q.push(x);
    if (q.peek() > x) {
      ans += q.peek() - x;
      q.pop();
      q.push(x);
    }
  }
  return { ans, heap: q };
}

// per-element trace for the figure: shift paid and running answer
```

listing 1052: javascript, the loop over the MaxHeap class (source: dsa/samples-js/src/ch32-slopetrick.mjs, lines 71-85)

```
def slope_plain(a):
    # the one-heap machine: a step that would cross replaces the top,
    # the lazy right-shift of every breakpoint above x
    heap = []  # max-heap by negation
    ans = 0
    for x in a:
        heapq.heappush(heap, -x)
        if -heap[0] > x:
            ans += -heap[0] - x
            heapq.heappop(heap)
            heapq.heappush(heap, -x)
    return ans, sorted(-v for v in heap)
```

listing 1053: python, the max-heap by negation, the repush after the payment (source: dsa/samples-py/src/Ch32/slopetrick.py, lines 23-35)

```lua
local function slope_plain(a)
  local q = MaxHeap.new()
  local ans = 0
  local trace = {}
  for i, x in ipairs(a) do
    q:push(x)
    local moved = 0
    if q:top() > x then
      moved = q:top() - x
      ans = ans + moved
      q:pop()
      q:push(x)
    end
    trace[i] = {moved, ans}
  end
  return ans, q, trace
end

-- strictly increasing via z_i = a_i - i, the whole-branch translate
```

listing 1054: lua, the loop with the trace table, the shift recorded per element (source: dsa/samples-lua/ch32_slopetrick.lua, lines 58-76)

The fixtures pin the whole ladder. Non-decreasing on 3, 2, 5, 1 costs 5 and leaves the heap descending 1, 1, 2, 2, with the per-element trace paying shifts 0, 1, 0, 4 for running answers 0, 1, 1, 5. The 5, 4, 3, 2, 1 descending run costs 6, already-sorted input costs 0, the single element costs 0, 10, 1, 1 costs 9, and 4, 4, 8, 2 costs 6. Strictly increasing through z maps 3, 2, 5, 1 to 3, 1, 3, −2 and costs 7, holds 5, 5, 5 at 2 through z 5, 4, 3, and 1, 1, 1 at 2. The reference cross-checked 400 random instances against an independent dp ground truth for both variants. The kin application is icpc world finals 2022 problem S (book 10, chapter 11), bridging the gap: its judge-green engine is a pull-based dp row sweep whose offers live in lazily-shifted window heaps, the same lazy-offset heap idiom, kin to the classic objective rather than an instance of it.

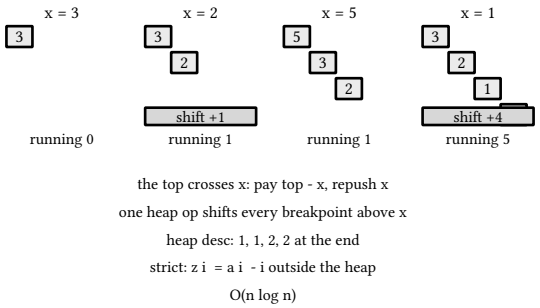


figure 267: the breakpoint multiset growing over the 3, 2, 5, 1 trace with the shift payments shaded, the convex cost accumulating kinks

32.5. value iteration and mdps

A markov decision process on a ladder: states 1 through 5, goal 6 absorbing at cost 0, and two actions per state, walk at cost 1 deterministically to s+1, or gamble at cost 1 with half probability to min(s+2, 6) and half a fall back to state 2. Value iteration fixes the bellman equations by Gauss-Seidel sweeps, updating states in place ascending so each state reads the freshest values below it, until the maximum delta falls under 1e-13 or the iteration cap. The contraction guarantees geometric convergence, and the exact fixed point is confirmed by fraction-arithmetic iteration in the reference.

The dry run: the fixture is the 5-state ladder with goal 6, asserted by the C# suite at V = 4.5, 4, 3, 2, 1.

1. Every value starts at 0. Sweep 1 reads only zeros: each walk costs 1 + 0 = 1, and the ladder lands all 1.
2. Sweep 2 climbs the same way: 2, 2, 2, 2, 1.
3. Sweep 3: 3, 3, 3, 2, 1.

4. Sweep 4: 4, 4, 3, 2, 1, state 1 tying at 1 + 3 = 4.
5. Sweep 5 breaks the tie at state 1: the gamble pays 1 + 0.5 × 3 + 0.5 × 4 = 4.5 under the walk's 1 + 4 = 5, and the sweep closes at 4.5, 4, 3, 2, 1.
6. The next sweep reproduces every value, the fixed point, and the die mini converges to 6 exactly.

sweep	V(1)	V(2)	V(3)	V(4)	V(5)	reads
1	1	1	1	1	1	all walks
2	2	2	2	2	1	
3	3	3	3	2	1	
4	4	4	3	2	1	state 1 ties
5	4.5	4	3	2	1	the gamble wins at 1

The fifth sweep lands the pinned 4.5, 4, 3, 2, 1 and the sixth changes nothing, and the listings below sweep the ladder in seven languages.

```c
// one gauss-seidel sweep, ascending states; returns the max delta
static double sweep(void) {
  double delta = 0;
  for (int s = 1; s <= 5; s++) {
    double walk = 1.0 + V[s + 1];
    int up = s + 2 > 6 ? 6 : s + 2;
    double gamble = 1.0 + 0.5 * V[up] + 0.5 * V[2];
    double nv = walk < gamble ? walk : gamble;
    if (fabs(nv - V[s]) > delta)
      delta = fabs(nv - V[s]);
    V[s] = nv;
  }
  return delta;
}

// dijkstra from the goal over predecessors u of v: u+1 = v or
// min(u+3, 6) = v, cost 1; returns the pop order count written out
```

listing 1055: c, one gauss-seidel sweep ascending, the max delta returned (source: dsa/samples-c/src/Ch32/valueiter.c, lines 24-40)

```
func GaussSeidelLadder() []float64 {
  v := make([]float64, 7) // index 6 is the goal
  for sweeps := 0; sweeps < 100000; sweeps++ {
    delta := 0.0
    for s := 1; s <= 5; s++ {
      walk := 1 + v[s+1]
      up := s + 2
      if up > 6 {
        up = 6
      }
      gamble := 1 + 0.5*v[up] + 0.5*v[2]
      nv := walk
      if gamble < nv {
        nv = gamble
      }
      if d := math.Abs(nv - v[s]); d > delta {
        delta = d
      }
      v[s] = nv
    }
    if delta < 1e-13 {
      break
    }
  }
  return v[1:6]
}
```

listing 1056: go, the ladder sweep, gamble falling to state 2 (source: dsa/samples-go/ch32/valueiter.go, lines 12-37)

```
  // one gauss-seidel sweep, ascending states; returns the max delta
  static double sweep() {
    double delta = 0;
    for (int s = 1; s ≤ 5; s++) {
      double walk = 1.0 + V[s + 1];
      int up = s + 2 > 6 ? 6 : s + 2;
      double gamble = 1.0 + 0.5 * V[up] + 0.5 * V[2];
      double nv = Math.min(walk, gamble);
      if (Math.abs(nv - V[s]) > delta)
        delta = Math.abs(nv - V[s]);
      V[s] = nv;
    }
    return delta;
  }
```

listing 1057: java, one gauss-seidel sweep ascending, the max delta returned (source: dsa/samples-java/src/Ch32/Valueiter.java, lines 22-35)

```
    public static double[] GaussSeidel(
        int stateCount,
        Func<int, List<Action>> actionsOf,
        double tol = 1e-13,
        int maxSweeps = 100_000)
    {
        var v = new double[stateCount + 1];
        for (int sweep = 0; sweep < maxSweeps; sweep++)
        {
            double delta = 0;
            for (int s = 0; s < stateCount; s++)
            {
                List<Action> actions = actionsOf(s);
                if (actions.Count == 0)
                    continue;
                double best = double.MaxValue;
                foreach (Action act in actions)
                {
                    double q = act.Cost;
                    foreach ((double p, int t) in act.Outcomes)
                        q += p * v[t];
                    best = Math.Min(best, q);
                }
                delta = Math.Max(delta, Math.Abs(best - v[s]));
                v[s] = best;
            }
            if (delta < tol)
                break;
        }
        return v[..stateCount];
```

listing 1058: c#, the generic sweep over an action table, walk and gamble supplied as data (source: dsa/samples/src/Ch32/ValueIter.cs, lines 15-44)

```javascript
export function gaussSeidel(table, goals, tol = 1e-13, cap = 100000) {
  const n = table.length;
  const V = new Array(n).fill(0);
  const valueOf = (act) => {
    let s = act.cost;
    for (const [p, t] of act.probs) s += p * V[t];
    return s;
  };
  let sweeps = 0;
  for (;;) {
    let delta = 0;
    for (let i = 0; i < n; i++) {
      if (goals.has(i)) continue;
      const names = Object.keys(table[i] ?? {});
      if (names.length === 0) continue;
      let best = Infinity;
      for (const name of names) best = Math.min(best, valueOf(table[i][name]));
      delta = Math.max(delta, Math.abs(best - V[i]));
      V[i] = best;
    }
    sweeps += 1;
    if (delta < tol || sweeps > cap) break;
  }
  const actions = (i) => {
    const out = {};
    for (const name of Object.keys(table[i] ?? {})) {
      out[name] = valueOf(table[i][name]);
    }
    return out;
  };
  return { V, actions, sweeps };
}
```

listing 1059: javascript, the table-driven sweep returning values and actions (source: dsa/samples-js/src/ch32-valueiter.mjs, lines 57-89)

```python
def gs_ladder():
    # states 1..5, goal 6 absorbing. walk: cost 1 -> s+1. gamble:
    # cost 1, half -> min(s+2, 6), half -> 2
    values = [0.0] * 7
    sweeps = 0
    while True:
        delta = 0.0
        for s in range(1, 6):
            walk = 1.0 + values[s + 1]
            gam = 1.0 + 0.5 * values[min(s + 2, 6)] + 0.5 * values[2]
            new = walk if walk <= gam else gam
            delta = max(delta, abs(new - values[s]))
            values[s] = new
        sweeps += 1
        if delta < 1e-13 or sweeps > 100000:
            break
    return values, sweeps
```

listing 1060: python, the sweep and its delta, states ascending (source: dsa/samples-py/src/Ch32/valueiter.py, lines 18-36)

```lua
local function gs_ladder()
  local V = {}
  for i = 0, 6 do V[i] = 0.0 end
  local sweeps = 0
  while true do
    local delta = 0.0
    for s = 1, 5 do
      local walk = 1.0 + V[s + 1]
      local gam = 1.0 + 0.5 * V[math.min(s + 2, 6)] + 0.5 * V[2]
      local nv = (walk ≤ gam) and walk or gam
      delta = math.max(delta, math.abs(nv - V[s]))
      V[s] = nv
    end
    sweeps = sweeps + 1
    if delta < 1e-13 or sweeps > 100000 then break end
  end
  return V, sweeps
end

-- deterministic variant: walk cost 1 -> s+1, leap cost 1 -> min(s+3,6)
```

listing 1061: lua, the sweep, values kept in a 1-based table (source: dsa/samples-lua/ch32_valueiter.lua, lines 9-28)

The ladder values pin at V(1..5) = 4.5, 4.0, 3.0, 2.0, 1.0 with doubles asserted at 1e-9: gambling is strictly better at state 1, 4.5 against walking's 5, walk and gamble tie at state 2, and walking wins at 3, 4, and 5. The family deliberately pins no policy label at the state-2 tie and no sweep count, both float-noise sensitive across languages, only the values, the strict orderings, and one ordering that is not: the deterministic variant, walk or leap to min(s+3, 6), solved by backward dijkstra over predecessors with (dist, vertex) heap order, gives d(1..5) = 2, 2, 1, 1, 1 and the pop order 6, 3, 4, 5, 1, 2. The geometric member is the die mini, one state rolling toward the goal at one sixth: V converges to exactly
6. The edges cover a goal-only mdp at 0, an unreachable state

keeping its initialized 0 by stated convention, and a state whose every action leads to the goal. The application is icpc world finals 2023 problem K (book 10, chapter 12), alea iacta est: the 7^d expected-rolls mdp solved by value iteration, the approach every contest team used where the editorial ran the backward dijkstra of book 9, ch 11 (practical data structures and algorithms, shortest paths).

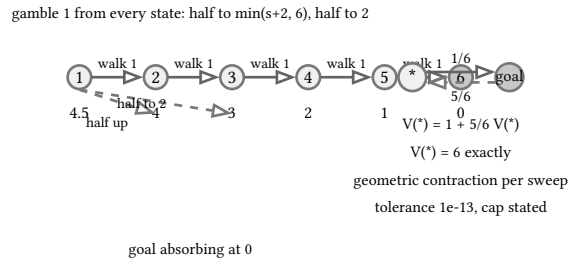


figure 268: the ladder with walk and gamble edges and the fixed-point values, beside the die loop whose geometric series sums to 6

32.6. small-to-large merging

The last amortization is one sentence long and worth its own section. Merging two sets always pours the smaller into the larger. An element that migrates lands in a set at least twice as big as its last home, so any element migrates at most log2 n times and n elements migrate O(n log n) times total, against O(n^2) for merging by always pouring into a fixed side. It is the same amortized-doubling argument as the dynamic array growth of book 9, ch 3 (practical data structures and algorithms, dynamic arrays and amortization), applied to set membership instead of capacity. The consumers are the standard trio: dsu component lists, subtree queries that merge children into the heaviest first, and heavy path bookkeeping in tree decompositions. The kin application is icpc world finals

2025 problem E (book 10, chapter 13), delivery service, whose component-merge accounting runs on exactly this lemma. No samples ship for it: the proof is the doubling ladder below, and the tree structures that consume it belong to later chapters.

The dry run: no suite carries this facet, so the example is hand-derived over 8 elements.

1. Eight singletons merge pairwise under always-smaller-into-larger: 4 merges each pour 1 element, 4 moves.
2. The 2-sets pair into 4-sets, each merge pouring 2: 2 + 2 = 4 moves, and the two 4-sets pour once more for 4: the ladder spends 4 + 4 + 4 = 12 moves.
3. One element's homes went 1, 2, 4, 8: it crossed 3 = log2 8 times.
4. Pour the accumulated set into each new singleton instead and the same 8 elements cost 1 + 2 + 3 + 4 + 5 + 6 + 7 = 28 moves.
5. The bound reads 8 × log2 8 = 24 crossings, the ladder's 12 under it and the fixed side's 28 with no cover.

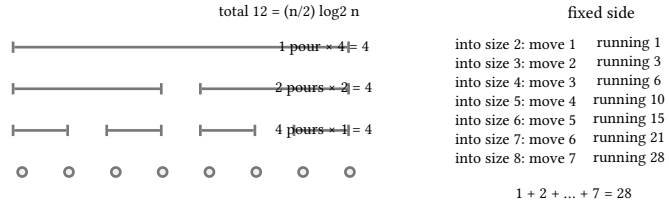


figure 269: the balanced merge tree with per-level pours on the left, the fixed-side spine's move ledger running to 28 on the right

The 12 against 28 is the lemma in numbers, and the doubling ladder below draws the per-element view.

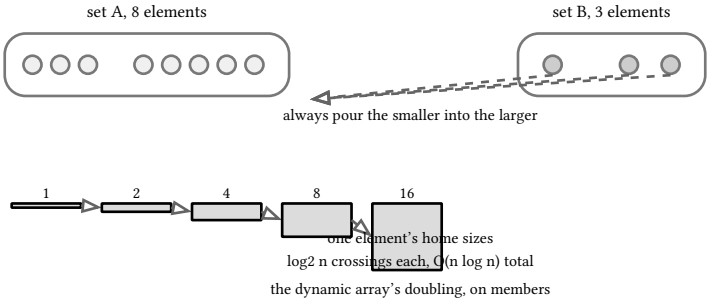


figure 270: two sets with the smaller's elements migrating, and one element's home sizes doubling at every crossing

32.7. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's five sample files per language, embedded test scripts included:

language	build sloc	container dependency	boundary note
c	605	static tables, hand heaps	double and fraction-free paths, fabs for the delta, pop order recorded from the selection loop
go	416	container/heap, math	two heap types over one item shape, the die mini converging under 1e-13
java	630	jdk 27 stdlib	the slope trick max-heap hand-rolled over a long array, PriorityQueue ships min-order only, infeasible cells marked by the 1L << 60 sentinel
c#	301	priorityqueue, tuples, records	the value iterator generic over an action table, walk and gamble supplied as data
javascript	367	map tables, class heaps	actions as plain objects, the sweep returning values and actions together
python	413	heapq by negation, fractions in tests	exact fixed point asserted from fraction iteration, ok-N print

language	build sloc	container dependency	boundary note
lua	504	tables, MaxHeap class	1-based states, the trace tables returned for run.lua

sources: cp-algorithms, “Divide and Conquer DP”, cp-algorithms.com/dynamic_programming/divide-and-conquer-dp.html, “Knuth’s Optimization”, cp-algorithms.com/dynamic_programming/knuth-optimization.html, “Dynamic Programming on Broken Profile. Problem "Parquet"”, cp-algorithms.com/dynamic_programming/profile-dynamics.html, and “Finding the largest zero submatrix”, cp-algorithms.com/dynamic_programming/zero_matrix.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. The slope trick and value iteration sections have no cp-algorithms article and stand as general techniques with their icpc applications as sources: icpc world finals 2017 problem D (book 10, chapter 8), 2022 problem S (book 10, chapter 11), 2023 problem K (book 10, chapter 12), and 2025 problem E (book 10, chapter 13) as the kin clause of the last section. Sample behavior verified by the seven suite gates scoped to chapter 32: c 5 files and 129 checks, go 19 test functions, java 5 files and 129 checks under run-java-samples, c# 23 facts, javascript 18 tests and 66 asserts, python 5 files and 72 asserts, lua 20 checks, zero skipped.

33. string algorithms ii

Chapter 15 built the first matching course, naive scanning, kmp borders, rolling hashes, and aho-corasick, and chapter 9 built the sorted suffix array with kasai lcp and binary-search substring lookup. This chapter owns the linear-time machines that come after them. The z-function reads a string against itself and answers occurrence questions from one array. Manacher finds every palindrome in one pass. The suffix automaton and the suffix tree compress all substrings of a string into structures that answer counting and location queries by walking, and lyndon factorization splits a string into its sorted pieces, which hands over the least rotation and the repetition scan. The chapter 15 machines answer does p occur, the machines here answer how many distinct substrings exist, where is the deepest repeat, and what is the minimal rotation, all with the queries rather than the construction as the teaching point.

33.1. the z-function

For a string s, z[i] is the length of the longest common prefix of s and s[i:], with z[0] = n pinned as the convention and no caller ever reading it. The build keeps a z-box, the interval [l, r) of the most recent position whose prefix match reached furthest right. A new position i inside the box can copy homework: the box is always a substring of the prefix, so the match starting at i - l inside the prefix tells us z[i] is at least min(r - i, z[i - l]), the cap stopping the reuse from reading past r where nothing is known. Beyond the cap the loop extends by direct comparison, and whenever i + z[i] passes r the box re-anchors at i. Each character of s is matched against a character further left only while r advances, so the whole build is O(n).

The dry run: the fixture is abacaba, whose z array 7, 0, 1, 0, 3, 0, 1 is asserted by the C# suite alongside its brute twin, and the other six suites pin the same array.

1. z[0] = 7 is the convention, and i = 1 dies on the first compare, b against a: z[1] = 0 with the box still empty.
2. i = 2 sits outside the empty box and extends by compare, a matches then b stops at c, z[2] = 1, and 2 + 1 = 3 past r re-anchors the box to [2, 3).
3. i = 3 lands outside again: a against c stops at once, z[3] = 0.
4. i = 4 extends three, a, b, a, before c stops it, z[4] = 3, and 4 + 3 = 7 re-anchors the box to [4, 7), the deepest reach of the run.
5. i = 5 copies z[5 - 4] = z[1] = 0 with the cap r - i = 2 slack, and the confirming compare b against a dies: z[5] = 0.
6. i = 6 copies min(r - i = 1, z[2] = 1) = 1, and with i + 1 = 7 = n the cap was exact: z[6] = 1 at zero compares.

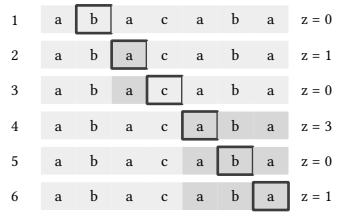


figure 271: the abacaba build as stacked frames, one row per position, the shading the current box, the stroked cell the position being computed

The array lands 7, 0, 1, 0, 3, 0, 1 exactly as pinned, and the listings below build it in seven languages.

```c
static void zfun(const char *s, int len) {
  zv[0] = len;
  int l = 0, r = 0;
  for (int i = 1; i < len; i++) {
    if (i < r) {
      int reuse = r - i;
      zv[i] = zv[i - l] < reuse ? zv[i - l] : reuse;
    } else {
      zv[i] = 0;
    }
    while (i + zv[i] < len && s[zv[i]] == s[i + zv[i]])
      zv[i]++;
    if (i + zv[i] > r) {
      l = i;
      r = i + zv[i];
    }
  }
}

// brute lcp per position, the in-suite cross-check
static int brute_z(const char *s, int len, int i) {
  int k = 0;
  while (i + k < len && s[k] == s[i + k])
    k++;
  return k;
}

static int hits[MAXS];

static int z_search(const char *pat, int plen, const char *text, int tlen) {
  char buf[MAXS];
  memcpy(buf, pat, (size_t)plen);
  buf[plen] = '\x01';
  memcpy(buf + plen + 1, text, (size_t)tlen);
  zfun(buf, plen + 1 + tlen);
  int hn = 0;
  for (int i = 0; i < tlen; i++)
    if (zv[plen + 1 + i] == plen)
      hits[hn++] = i;
  return hn;
}
```

listing 1062: c, the z-box build loop, the brute twin, and the separator search (source: dsa/samples-c/src/Ch33/zfunc.c, lines 26-66)

```
func ZFunction(s string) []int {
  n := len(s)
  z := make([]int, n)
  if n == 0 {
    return z
  }
  z[0] = n
  l, r := 0, 0
  for i := 1; i < n; i++ {
    if i < r {
      z[i] = r - i
      if z[i-l] < z[i] {
        z[i] = z[i-l]
      }
    }
    for i+z[i] < n && s[z[i]] == s[i+z[i]] {
      z[i]++
    }
    if i+z[i] > r {
      l, r = i, i+z[i]
    }
  }
  return z
}

// ZSearch reports every occurrence of pat in text, overlaps
// included, by the separator trick: z over pat + '\x01' + text fires
// exactly where z equals the pattern length. O(n + m).
func ZSearch(pat, text string) []int {
  z := ZFunction(pat + "\x01" + text)
  m := len(pat)
  var hits []int
  for i := range text {
    if z[m+1+i] == m {
      hits = append(hits, i)
    }
  }
  return hits
}
```

listing 1063: go, the build with the min cap spelled out, the search, the rotation test below (source: dsa/samples-go/ch33/zfunc.go, lines 7-45)

```
  static void zfun(String s, int len) {
    zv[0] = len;
    int l = 0, r = 0;
    for (int i = 1; i < len; i++) {
      if (i < r) {
        int reuse = r - i;
        zv[i] = Math.min(zv[i - l], reuse);
      } else {
        zv[i] = 0;
      }
      while (i + zv[i] < len && s.charAt(zv[i]) == s.charAt(i + zv[i]))
        zv[i]++;
      if (i + zv[i] > r) {
        l = i;
        r = i + zv[i];
      }
    }
  }

  // brute lcp per position, the in-suite cross-check
  static int bruteZ(String s, int len, int i) {
    int k = 0;
    while (i + k < len && s.charAt(k) == s.charAt(i + k))
      k++;
    return k;
  }

  static int zSearch(String pat, int plen, String text, int tlen) {
    String buf = pat + SEP + text;
    zfun(buf, plen + 1 + tlen);
    int hn = 0;
    for (int i = 0; i < tlen; i++)
      if (zv[plen + 1 + i] == plen)
        hits[hn++] = i;
    return hn;
  }
```

listing 1064: java, the z-box build, the brute twin, and the separator search over the \x01 join (source: dsa/samples-java/src/Ch33/Zfunc.java, lines 26-61)

```
    public static long[] Z(string s)
    {
        int n = s.Length;
        var z = new long[n];
        if (n == 0)
            return z;
        z[0] = n;
        int l = 0;
        int r = 0; // the rightmost z-box [l, r)
        for (int i = 1; i < n; i++)
        {
            if (i < r)
                z[i] = Math.Min(r - i, z[i - l]); // the box guarantees this much
            while (i + (int)z[i] < n && s[(int)z[i]] == s[i + (int)z[i]])
                z[i]++;
            if (i + z[i] > r)
            {
                l = i;
                r = i + (int)z[i];
            }
        }
        return z;
    }

    public static long[] ZBrute(string s)
    {
        int n = s.Length;
        var z = new long[n];
        for (int i = n == 0 ? 0 : 1; i < n; i++)
            while (i + z[i] < n && s[(int)z[i]] == s[i + (int)z[i]])
                z[i]++;
        if (n > 0)
            z[0] = n;
        return z;
    }

    /// <summary>All occurrence offsets of pat in text (overlaps included), by the
    /// separator search over pat + '\x01' + text.</summary>
    public static List<long> Search(string pat, string text)
    {
        string combined = pat + '\x01' + text;
        long[] z = Z(combined);
        var hits = new List<long>();
        for (int i = 0; i < text.Length; i++)
            if (z[pat.Length + 1 + i] == pat.Length)
                hits.Add(i);
        return hits;
    }
```

listing 1065: c#, the build, the brute cross-check, and the search reading z against the pattern length (source: dsa/samples/src/Ch33/Zfunc.cs, lines 11-58)

```
export function zfun(s) {
  const n = s.length;
  const z = new Array(n).fill(0);
  if (n === 0) return z;
  z[0] = n;
  let l = 0;
  let r = 0;
  for (let i = 1; i < n; i++) {
    if (i < r) z[i] = Math.min(r - i, z[i - l]);
    while (i + z[i] < n && s[z[i]] === s[i + z[i]]) z[i] += 1;
    if (i + z[i] > r) {
      l = i;
      r = i + z[i];
    }
  }
  return z;
}

export function zSearch(pat, text) {
  const z = zfun(pat + "\x01" + text);
  const m = pat.length;
  const out = [];
  for (let i = 0; i < text.length; i++) {
    if (z[m + 1 + i] === m) out.push(i);
  }
  return out;
}
```

listing 1066: javascript, the build and the search, the same two functions (source: dsa/samples-js/src/ch33-zfunc.mjs, lines 9-35)

```python
def zfun(s):
    size = len(s)
    z = [0] * size
    if size == 0:
        return z
    z[0] = size  # pinned convention; no caller reads it
    lft = rgt = 0
    for i in range(1, size):
        if i < rgt:  # inside the box: reuse z[i - lft], capped
            z[i] = min(rgt - i, z[i - lft])
        while i + z[i] < size and s[z[i]] == s[i + z[i]]:
            z[i] += 1
        if i + z[i] > rgt:  # re-anchor the box
            lft, rgt = i, i + z[i]
    return z


def z_brute(s):
    size = len(s)
    out = [0] * size
    if size:
        out[0] = size
    for i in range(1, size):
        k = 0
        while i + k < size and s[k] == s[i + k]:
            k += 1
        out[i] = k
    return out


def z_search(pat, text):
    z = zfun(pat + SEP + text)
    m = len(pat)
    return [i for i in range(len(text)) if z[m + 1 + i] == m]
```

listing 1067: python, the build, the brute lcp per position, the search (source: dsa/samples-py/src/Ch33/zfunc.py, lines 17-50)

```lua
local function zfun(s)
  local n = #s
  local z = {}
  if n == 0 then return z end
  z[1] = n -- z[0] = n by convention, stored at lua index 1
  local l, r = 0, 0 -- the z-box [l, r) in 0-based positions
  for i = 2, n do
    local zi = 0 -- 0-based i-1 is the loop position
    if i - 1 < r then
      zi = math.min(r - (i - 1), z[i - l] or 0)
    end
    while (i - 1) + zi < n
      and s:byte(zi + 1) == s:byte((i - 1) + zi + 1) do
      zi = zi + 1
    end
    z[i] = zi
    if (i - 1) + zi > r then
      l, r = i - 1, (i - 1) + zi
    end
  end
  return z
end

-- naive longest common prefix per position, the house cross-check
local function z_brute(s)
  local n = #s
  local out = {}
  if n == 0 then return out end
  out[1] = n
  for i = 2, n do
    local k = 0
    while (i - 1) + k < n and s:byte(k + 1) == s:byte((i - 1) + k + 1) do
      k = k + 1
    end
    out[i] = k
  end
  return out
end

-- separator search: z over pat + '\x01' + text reports every
-- occurrence, overlaps included
local function z_search(pat, text)
  local z = zfun(pat .. string.char(1) .. text)
  local m = #pat
  local hits = {}
  for i = 0, #text - 1 do
    if z[m + 2 + i] == m then hits[#hits + 1] = i end -- 0-based
  end
  return hits
end
```

listing 1068: lua, 1-based storage over 0-based positions, the build and the search (source: dsa/samples-lua/ch33_zfunc.lua, lines 8-57)

Two queries ride the array. The separator search concatenates pat + '\x01' + text, a separator ordered below every real character so it can never match, and builds z over the combined string: position m

1. 1 + i holds exactly m exactly when the pattern occurs at text

offset i, overlaps included, O(n + m) total, the same contract the kmp of book 9, ch 15 (practical data structures and algorithms, string matching) fills with borders. The rotation test follows: t is a rotation of s exactly when s occurs in t + t, so one separator search over the doubled text decides it.

The fixtures pin the same arrays in all seven languages: z(aaaaa) = 5, 4, 3, 2, 1; z(abacaba) = 7, 0, 1, 0, 3, 0, 1; z(aabaa) = 5, 1, 0, 2, 1; z(abababb) = 6, 0, 4, 0, 2, 0; z(abab) = 4, 0, 2, 0; z(acacacb) = 7, 0, 4, 0, 2, 0, 0. The search family pins ab in abababb at 0, 2, 4, ana in banana at 1, 3, abab in abababab at 0, 2, 4, abc in aabbcc nowhere, and a in aaaa at 0, 1, 2, 3. The rotation family pins (abcab, bcaba), (abab, baba), and (abc, abc) as rotations and (abc, acb) as not. The edges read z of the empty string as empty, z of a as 1, z of aa as 2, 1, and every suite keeps a brute longest-common-prefix-per-position twin agreeing on the whole batch.

The application is icpc world finals 2023 problem F (book 10, chapter 12), tilting tiles, whose per-cycle reasoning matches a pattern against the doubled cycle, exactly the separator search shape, with the chapter 15 kmp as the alternate machine for the same job.

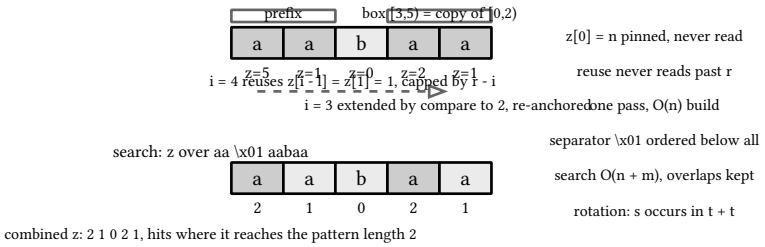


figure 272: the z-box jumping over aabaa, the position-3 box copying the prefix, the reuse arrow at position 4, the hits ledger underneath

33.2. manacher's two passes

Every palindrome has a center, a single character for odd lengths or a gap between characters for even ones, and manacher computes one radius array per kind in a single linear pass each. The odd array d1 counts the odd palindromes centered at each i, so d1[i] is the radius plus one, and the even array d2 counts the even palindromes straddling the i - 1 and i boundary. Both passes keep a mirrored interval [l, r], the furthest-reaching palindrome seen so far. A new center i inside the interval has a mirror at l + r - i, and the palindrome around that mirror is already known, so the radius starts at min(d[mirror], r - i + 1) instead of zero: the mirror's radius is trusted only as far as the interval reaches, and the expansion loop takes over from there. Whenever the palindrome around i reaches past r, the interval re-anchors.

The dry run: the fixture is aabbaa, whose even array 0, 1, 0, 3, 0, 1 and total 11 are asserted by the C# suite, identical in the other six.

1. Center 0 has no left neighbor: d2[0] = 0 without a compare.
2. Center 1 matches the pair a, a and runs out of string: d2[1] = 1, the palindrome aa, and the interval re-anchors to [0, 1].
3. Center 2 sits outside: the boundary a, b mismatches at once, d2[2] = 0.
4. Center 3, outside the small interval, expands on its own: b = b, a = a, a = a climbs the radius to 3, the whole string aabbaa, and the interval jumps to [0, 5].
5. Center 4 mirrors to 0 + 5 - 4 + 1 = 2 and starts at min(d2[2] = 0, 5 - 4 + 1 = 2) = 0, the boundary b, a confirming it: d2[4] = 0.
6. Center 5 mirrors to index 1 and starts at min(d2[1] = 1, 1) = 1: the pair a, a holds and the string ends, so the cap was exact, d2[5] = 1.
7. d2 reads 0, 1, 0, 3, 0, 1 at sum 5, the odd pass contributes 11 - 5 = 6 single characters, and the longest runs 2 × 3 = 6, the string itself.

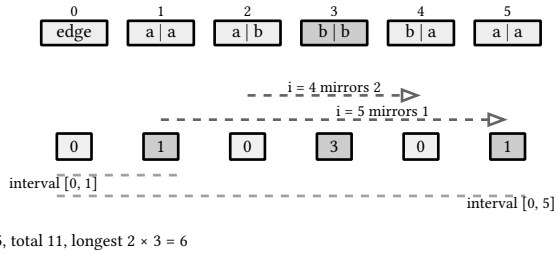


figure 273: the even pass over aabbaa one center at a time, the straddling pair above, the landing d2 below, dashed arrows the mirror reuse, the two intervals spanned underneath

The 0, 1, 0, 3, 0, 1 array and the 11 are pinned across the suites, and the listings below run both passes in seven languages.

```c
static void manacher(const char *s, int len) {
  int l = 0, r = -1;
  for (int i = 0; i < len; i++) { // odd pass
    int k = i > r ? 1 : (d1[l + r - i] < r - i + 1 ? d1[l + r - i] : r - i + 1);
    while (i - k ≥ 0 && i + k < len && s[i - k] == s[i + k])
      k++;
    d1[i] = k;
    if (i + k - 1 > r) {
      l = i - k + 1;
      r = i + k - 1;
    }
  }
  l = 0;
  r = -1;
  for (int i = 0; i < len; i++) { // even pass
    int k =
        i > r ? 0
              : (d2[l + r - i + 1] < r - i + 1 ? d2[l + r - i + 1] : r - i + 1);
    while (i - k - 1 ≥ 0 && i + k < len && s[i - k - 1] == s[i + k])
      k++;
    d2[i] = k;
    if (i + k - 1 > r) {
      l = i - k;
      r = i + k - 1;
    }
  }
}
```

listing 1069: c, the odd pass and the even pass side by side over the same string (source: dsa/samples-c/src/Ch33/manacher.c, lines 26-52)

```
func Manacher(s string) (d1, d2 []int, total int, longest string) {
  n := len(s)
  d1 = make([]int, n)
  d2 = make([]int, n)
  l, r := 0, -1
  for i := 0; i < n; i++ {
    k := 1
    if i > r {
      k = 1
    } else {
      k = d1[l+r-i]
      if r-i+1 < k {
        k = r - i + 1
      }
    }
    for i-k ≥ 0 && i+k < n && s[i-k] == s[i+k] {
      k++
    }
    d1[i] = k
    if i+k-1 > r {
      l, r = i-k+1, i+k-1
    }
  }
  l, r = 0, -1
  for i := 0; i < n; i++ {
    k := 0
    if i <= r {
      k = d2[l+r-i+1]
      if r-i+1 < k {
        k = r - i + 1
      }
    }
    for i-k-1 ≥ 0 && i+k < n && s[i-k-1] == s[i+k] {
      k++
    }
    d2[i] = k
    if i+k-1 > r {
      l, r = i-k, i+k-1
    }
  }
```

listing 1070: go, both passes inside one function, total and longest computed after (source: dsa/samples-go/ch33/manacher.go, lines 9-48)

```
  static void manacher(String s, int len) {
    int l = 0, r = -1;
    for (int i = 0; i < len; i++) { // odd pass
      int k = i > r ? 1 : Math.min(d1[l + r - i], r - i + 1);
      while (i - k ≥ 0 && i + k < len && s.charAt(i - k) == s.charAt(i + k))
        k++;
      d1[i] = k;
      if (i + k - 1 > r) {
        l = i - k + 1;
        r = i + k - 1;
      }
    }
    l = 0;
    r = -1;
    for (int i = 0; i < len; i++) { // even pass
      int k = i > r ? 0 : Math.min(d2[l + r - i + 1], r - i + 1);
      while (i - k - 1 ≥ 0 && i + k < len && s.charAt(i - k - 1) == s.charAt(i + k))
        k++;
      d2[i] = k;
      if (i + k - 1 > r) {
        l = i - k;
        r = i + k - 1;
      }
    }
  }
```

listing 1071: java, the odd pass and the even pass side by side, charAt reads throughout (source: dsa/samples-java/src/Ch33/Manacher.java, lines 24-48)

```
    public static long[] Odd(string s)
    {
        int n = s.Length;
        var d = new long[n];
        int l = 0;
        int r = -1;
        for (int i = 0; i < n; i++)
        {
            long k = i > r ? 1 : Math.Min(d[l + r - i], r - i + 1);
            while (i - (int)k ≥ 0 && i + (int)k < n && s[i - (int)k] == s[i + (int)k])
                k++;
            d[i] = k;
            if (i + (int)k - 1 > r)
            {
                l = i - (int)k + 1;
                r = i + (int)k - 1;
            }
        }
        return d;
    }

    public static long[] Even(string s)
    {
        int n = s.Length;
        var d = new long[n];
        int l = 0;
        int r = -1;
        for (int i = 0; i < n; i++)
        {
            long k = i > r ? 0 : Math.Min(d[l + r - i + 1], r - i + 1);
            while (i - (int)k - 1 ≥ 0 && i + (int)k < n && s[i - (int)k - 1] == s[i + (int)k])
                k++;
            d[i] = k;
            if (i + (int)k - 1 > r)
            {
                l = i - (int)k;
                r = i + (int)k - 1;
            }
        }
        return d;
    }
```

listing 1072: c#, odd and even as twin methods, totals by sum, longest by max (source: dsa/samples/src/Ch33/Manacher.cs, lines 10-50)

```
  // odd pass
  const d1 = new Array(n).fill(0);
  let l = 0;
  let r = -1;
  for (let i = 0; i < n; i++) {
    d1[i] = oddAt(i);
    if (i + d1[i] - 1 > r) {
      l = i - d1[i] + 1;
      r = i + d1[i] - 1;
    }
  }

  // even pass: mirror index is l + r - i + 1
  const d2 = new Array(n).fill(0);
  l = 0;
  r = -1;
  for (let i = 0; i < n; i++) {
    let k = i > r ? 0 : Math.min(d2[l + r - i + 1], r - i + 1);
    while (i - k - 1 ≥ 0 && i + k < n && s[i - k - 1] === s[i + k]) k += 1;
    d2[i] = k;
    if (i + d2[i] - 1 > r) {
      l = i - d2[i];
      r = i + d2[i] - 1;
    }
  }
```

listing 1073: javascript, the two loops with their own l and r, the queries after (source: dsa/samples-js/src/ch33-manacher.mjs, lines 17-41)

```
def manacher_odd(s):
    size = len(s)
    d = [0] * size
    lft, rgt = 0, -1
    for i in range(size):
        k = 1 if i > rgt else min(d[lft + rgt - i], rgt - i + 1)
        while i - k ≥ 0 and i + k < size and s[i - k] == s[i + k]:
            k += 1
        d[i] = k
        if i + k - 1 > rgt:
            lft, rgt = i - k + 1, i + k - 1
    return d


def manacher_even(s):
    size = len(s)
    d = [0] * size
    lft, rgt = 0, -1
    for i in range(size):
        k = 0 if i > rgt else min(d[lft + rgt - i + 1], rgt - i + 1)
        while i - k - 1 ≥ 0 and i + k < size and s[i - k - 1] == s[i + k]:
            k += 1
        d[i] = k
        if i + k - 1 > rgt:
            lft, rgt = i - k, i + k - 1
    return d
```

listing 1074: python, the two passes, the mirror index shifted by one in the even case (source: dsa/samples-py/src/Ch33/manacher.py, lines 14-39)

```lua
local function manacher_odd(s)
  local n = #s
  local d = {}
  local l, r = 1, 0 -- the interval [l, r] in 1-based indices
  for i = 1, n do
    local k = (i > r) and 1
      or math.min(d[l + r - i], r - i + 1)
    while i - k ≥ 1 and i + k ≤ n
      and s:byte(i - k) == s:byte(i + k) do
      k = k + 1
    end
    d[i] = k
    if i + k - 1 > r then
      l, r = i - k + 1, i + k - 1
    end
  end
  return d
end

local function manacher_even(s)
  local n = #s
  local d = {}
  local l, r = 1, 0
  for i = 1, n do
    local k = (i > r) and 0
      or math.min(d[l + r - i + 1], r - i + 1)
    while i - k - 1 ≥ 1 and i + k ≤ n
      and s:byte(i - k - 1) == s:byte(i + k) do
      k = k + 1
    end
    d[i] = k
    if i + k - 1 > r then
      l, r = i - k, i + k - 1
    end
  end
  return d
end
```

listing 1075: lua, both passes in 1-based indices, the same mirrored interval (source: dsa/samples-lua/ch33_manacher.lua, lines 7-43)

The queries fall out of the two arrays by arithmetic. The number of palindromic substrings with multiplicity is sum(d1) + sum(d2), since d1[i] counts the odd palindromes at center i one per radius and d2[i] counts the even ones, and the longest palindromic substring has length max over both arrays of 2 d1[i] - 1 and 2 d2[i]. The fixtures pin abababa with d1 = 1, 2, 3, 4, 3, 2, 1 and d2 all zeros, 16 palindromic substrings, the string itself longest; abacabadabacaba with d1 = 1, 2, 1, 4, 1, 2, 1, 8, 1, 2, 1, 4, 1, 2, 1 and total 32; the even anchors aabaa with d2 = 0, 1, 0, 0, 1 at total 9, abba with d2 = 0, 0, 2, 0 at total 6, aabbaa with d2 = 0, 1, 0, 3, 0, 1 at total 11; and the small case aaa pinning both arrays, d1 = 1, 2, 1 and d2 = 0, 1, 1, at total 6. The edge family reads the empty string at 0, a at 1, and ab at 2 with the first singleton winning ties, and every suite cross-checks the whole batch against a brute all-substring palindrome scan.

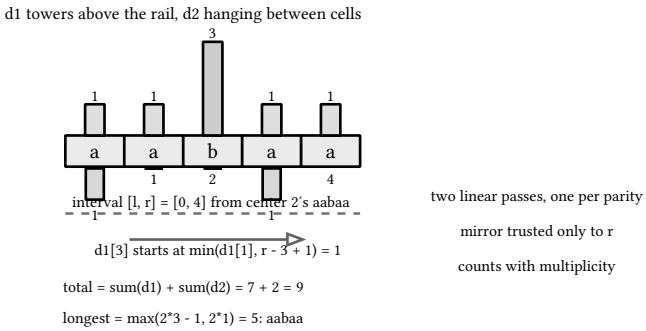


figure 274: aabaa centers on a rail, d1 and d2 as towers, the mirror reuse arrow from center 1 to center 3, even towers hanging between cells

33.3. the suffix automaton

A suffix automaton is the smallest dag whose paths from the start spell exactly the substrings of s, built online one character at a time. Each state is an equivalence class of right-extensions, the endpos set, and carries three fields: len, the longest substring in the class, link, a pointer to the class of the longest proper suffix with a strictly larger endpos set, and a transition map by character. Appending one character walks the suffix links from the last state adding the new transition wherever it is missing. When the walk stops on a state p that already has the character to some q, the split decides the shape: if len[p] + 1 equals len[q] the classes nest cleanly and the new state links to q, otherwise q is too fat, and a clone with len[p] + 1, q's transitions, and q's old link is created, every transition to q along the walk is redirected to the clone, and both q and the new state link to it. The construction is O(n) total and never creates more than 2n - 1 states.

The dry run: the fixture is abab, whose 5 states, 7 distinct substrings, and occurrence counts a 2, b 2, ab 2 and its reversal 1, aba 1, bab 1, abab 1 are asserted by the C# suite, identical in the other six.

1. The first a creates state 1, longest a at len 1, linked to the root.
2. The first b walks state 1 to the root adding the missing transitions and creates state 2, ab at len 2, linked to the root.
3. The second a stops the walk at the root's existing a into state 1, and len 0 + 1 = 1 = len[1] nests cleanly: state 3 is aba at len 3 linked to 1, no clone.
4. The final b stops at state 1's existing b into state 2, len 1 + 1 = 2 = len[2]: state 4 is abab at len 4 linked to 2, 5 states.
5. Distinct substrings telescope down the links, 1 - 0 = 1, 2 - 0 = 2, 3 - 1 = 2, 4 - 2 = 2, and 1 + 2 + 2 + 2 = 7.
6. Occurrences bubble up the same tree: reading ab lands on state 2, its own append plus state 4's pushed 1 counting 1 + 1 = 2, and reading a lands on state 1 for 1 + 1 = 2 the same way.

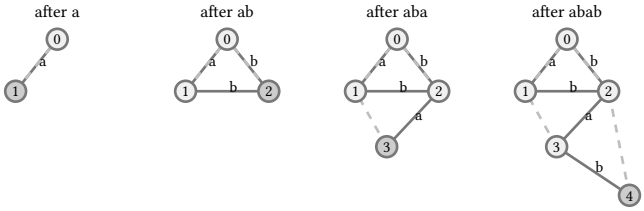


figure 275: the abab automaton growing one append per frame, solid edges the transitions, dashed edges the suffix links, the new state shaded

The 5 states and the 7 close the build exactly as pinned, and the listings below extend the automaton in seven languages.

```c
static void sam_build(const char *s) {
  sc = 0;
  last = new_state(0);
  for (const char *p = s; *p; p++) {
    int ch = *p - 'a', cur = new_state(slen[last] + 1);
    scnt[cur] = 1;
    int q;
    int pp = last;
    while (pp != -1 && !snxt[pp][ch]) {
      snxt[pp][ch] = cur;
      pp = slink[pp];
    }
    if (pp == -1) {
      slink[cur] = 0;
    } else if ((q = snxt[pp][ch]), slen[pp] + 1 == slen[q]) {
      slink[cur] = q;
    } else {
      int clone = new_state(slen[pp] + 1);
      memcpy(snxt[clone], snxt[q], sizeof(snxt[clone]));
      slink[clone] = slink[q];
      scnt[clone] = 0;
      while (pp != -1 && snxt[pp][ch] == q) {
        snxt[pp][ch] = clone;
        pp = slink[pp];
      }
      slink[q] = clone;
      slink[cur] = clone;
    }
    last = cur;
  }
```

listing 1076: c, the build with the clone split, the per-character loop (source: dsa/samples-c/src/Ch33/suffixautomaton.c, lines 38-67)

```
  for i := 0; i < len(s); i++ {
    ch := s[i]
    cur := len(sam.length)
    sam.length = append(sam.length, sam.length[last]+1)
    sam.link = append(sam.link, -1)
    sam.next = append(sam.next, map[byte]int{})
    sam.cnt = append(sam.cnt, 1)
    p, linked := last, false
    for p != -1 {
      if _, ok := sam.next[p][ch]; !ok {
        sam.next[p][ch] = cur
        p = sam.link[p]
        continue
      }
      q := sam.next[p][ch]
      if sam.length[p]+1 == sam.length[q] {
        sam.link[cur] = q
      } else {
        clone := len(sam.length)
        sam.length = append(sam.length, sam.length[p]+1)
        sam.link = append(sam.link, sam.link[q])
        sam.next = append(sam.next,
          map[byte]int{})
        for k, v := range sam.next[q] {
          sam.next[clone][k] = v
        }
        sam.cnt = append(sam.cnt, 0)
        for p != -1 && sam.next[p][ch] == q {
          sam.next[p][ch] = clone
          p = sam.link[p]
        }
        sam.link[q] = clone
        sam.link[cur] = clone
      }
      linked = true
      break
    }
    if !linked {
      sam.link[cur] = 0
    }
    last = cur
  }
```

listing 1077: go, the loop with the clone branch inside (source: dsa/samples-go/ch33/suffixautomaton.go, lines 26-67)

```
  static void samBuild(String s) {
    sc = 0;
    last = newState(0);
    for (int p = 0; p < s.length(); p++) {
      int ch = s.charAt(p) - 'a', cur = newState(slen[last] + 1);
      scnt[cur] = 1;
      int pp = last;
      while (pp != -1 && snxt[pp][ch] == 0) {
        snxt[pp][ch] = cur;
        pp = slink[pp];
      }
      if (pp == -1) {
        slink[cur] = 0;
      } else {
        int q = snxt[pp][ch];
        if (slen[pp] + 1 == slen[q]) {
          slink[cur] = q;
        } else {
          int clone = newState(slen[pp] + 1);
          snxt[clone] = snxt[q].clone();
          slink[clone] = slink[q];
          scnt[clone] = 0;
          while (pp != -1 && snxt[pp][ch] == q) {
            snxt[pp][ch] = clone;
            pp = slink[pp];
          }
          slink[q] = clone;
          slink[cur] = clone;
        }
      }
      last = cur;
    }
```

listing 1078: java, the build with the clone split, the clone copying q's 26-way transition array (source: dsa/samples-java/src/Ch33/Suffixautomaton.java, lines 39-70)

```
        int initial = New(0, -1);
        int last = initial;
        foreach (char ch in s)
        {
            int cur = New(_len[last] + 1, -1);
            _cnt[cur] = 1;
            int p = last;
            while (p != -1 && !_next[p].ContainsKey(ch))
            {
                _next[p][ch] = cur;
                p = _link[p];
            }
            if (p == -1)
            {
                _link[cur] = initial;
            }
            else
            {
                int q = _next[p][ch];
                if (_len[p] + 1 == _len[q])
                    _link[cur] = q;
                else
                {
                    int clone = New(_len[p] + 1, _link[q]);
                    _next[clone] = new Dictionary<char, int>(_next[q]);
                    _cnt[clone] = 0;
                    while (p != -1 && _next[p].GetValueOrDefault(ch) == q)
                    {
                        _next[p][ch] = clone;
                        p = _link[p];
                    }
                    _link[q] = clone;
                    _link[cur] = clone;
                }
            }
            last = cur;
        }
```

listing 1079: c#, the same loop in the constructor, the clone copying q's transitions (source: dsa/samples/src/Ch33/SuffixAutomaton.cs, lines 29-65)

```
  #extend(ch) {
    const cur = this.#len.length;
    this.#len.push(this.#len[this.#last] + 1);
    this.#link.push(-1);
    this.#next.push(new Map());
    this.#cnt.push(1);
    let p = this.#last;
    while (p !== -1 && !this.#next[p].has(ch)) {
      this.#next[p].set(ch, cur);
      p = this.#link[p];
    }
    if (p === -1) {
      this.#link[cur] = 0;
    } else {
      const q = this.#next[p].get(ch);
      if (this.#len[p] + 1 === this.#len[q]) {
        this.#link[cur] = q;
      } else {
        const cl = this.#len.length; // the clone split
        this.#len.push(this.#len[p] + 1);
        this.#link.push(this.#link[q]);
        this.#next.push(new Map(this.#next[q]));
        this.#cnt.push(0);
        while (p !== -1 && this.#next[p].get(ch) === q) {
          this.#next[p].set(ch, cl);
          p = this.#link[p];
        }
        this.#link[q] = cl;
        this.#link[cur] = cl;
      }
    }
    this.#last = cur;
  }
```

listing 1080: javascript, the extend method with the clone split (source: dsa/samples-js/src/ch33-suffixautomaton.mjs, lines 26-58)

```
    def extend(self, last, ch):
        cur = len(self.len)
        self.len.append(self.len[last] + 1)
        self.link.append(-1)
        self.nxt.append({})
        self.cnt.append(1)  # endpos classes seed at one
        p = last
        while p != -1 and ch not in self.nxt[p]:
            self.nxt[p][ch] = cur
            p = self.link[p]
        if p == -1:
            self.link[cur] = 0
            return cur
        q = self.nxt[p][ch]
        if self.len[p] + 1 == self.len[q]:
            self.link[cur] = q
            return cur
        cl = len(self.len)  # the clone split
        self.len.append(self.len[p] + 1)
        self.link.append(self.link[q])
        self.nxt.append(dict(self.nxt[q]))
        self.cnt.append(0)
        while p != -1 and self.nxt[p].get(ch) == q:
            self.nxt[p][ch] = cl
            p = self.link[p]
        self.link[q] = cl
        self.link[cur] = cl
        return cur
```

listing 1081: python, extend as a method, the clone split in the else branch (source: dsa/samples-py/src/Ch33/suffixautomaton.py, lines 29-56)

```lua
function Sam:extend(c)
  local cur = #self.len + 1
  self.len[cur] = self.len[self.last] + 1
  self.link[cur] = -1
  self.next[cur] = {}
  self.cnt[cur] = 1
  local p = self.last
  while p ~= -1 and self.next[p][c] == nil do
    self.next[p][c] = cur
    p = self.link[p]
  end
  if p == -1 then
    self.link[cur] = 1 -- the root
  else
    local q = self.next[p][c]
    if self.len[p] + 1 == self.len[q] then
      self.link[cur] = q
    else
      local cl = #self.len + 1 -- the clone
      self.len[cl] = self.len[p] + 1
      self.link[cl] = self.link[q]
      self.next[cl] = {}
      for k, v in pairs(self.next[q]) do self.next[cl][k] = v end
      self.cnt[cl] = 0
      while p ~= -1 and self.next[p][c] == q do
        self.next[p][c] = cl
        p = self.link[p]
      end
      self.link[q] = cl
      self.link[cur] = cl
    end
  end
  self.last = cur
end
```

listing 1082: lua, the same extend with 1-based state ids (source: dsa/samples-lua/ch33_suffixautomaton.lua, lines 31-64)

Two counts ride the structure. Distinct substrings telescope along the link tree: state v contributes len[v] - len[link[v]] new substrings, the ones whose first occurrence of their class boundary lands in v, so the total is that difference summed over all non-root states. Occurrence counts live on the same tree as subtree sums: each extend state counts one endpos, clones count zero, and processing states in decreasing len order pushes every count into its link parent, after which walking the transitions for any t lands on the state whose count is the number of occurrences, with membership as count above zero. The fixtures pin the canonical machine, identical in all seven languages: abab builds 5 states and 7 distinct substrings with occurrences a 2, b 2, ab 2 and its reversal 1, aba 1, bab 1, abab 1, and abc 0; aab builds 4 states, 5 distinct; banana builds 10 states, 15 distinct, with a 3, an 2, ana 2, na 2, nan 1, ban 1, banana 1, nab 0; aa builds 3 states and 2 distinct; the scale anchors run abababa at 8 states and 13 distinct with aba occurring 3 times, and mississippi at 18 states and 53 distinct with issi, ssi, and iss twice. The edges read the empty string as 1 state, 0 distinct, matching nothing, and a as 2 states, 1 distinct, and every suite cross-checks distinct against a brute set of all substrings and occurrences against a brute sliding count.

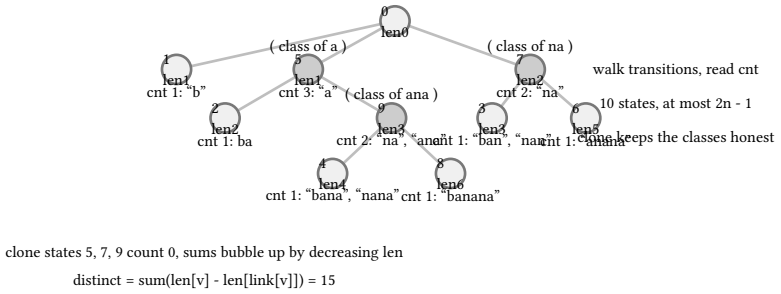


figure 276: the banana link tree with len labels, occurrence counts bubbling up, the endpos classes of a, ana, and na bracketed

33.4. the suffix tree

The suffix tree is the edge-compressed suffix trie: insert every suffix of s + '$' in order, and whenever a walk stops in the middle of an edge label, split the edge at that character and hang both tails off the new node. Nodes hold a child map keyed by the first character of each edge, edges hold (l, r) index pairs into s, and no substring is ever copied, which is what keeps seven implementations of the same insertion order byte-identical in their node counts. The terminal character makes the tree explicit, no suffix is a prefix of another, so every leaf ends a full suffix and the tree has exactly len(s) leaves. The build here is the honest simple one, O(n^2) worst case, the same ruling as the suffix array of book 9, ch 9 (practical data structures and algorithms, tries and suffix structures) at its O(n^2 log n); Ukkonen's algorithm builds the same tree in O(n) and gets the citation rather than the port, because the queries, not the construction, are the teaching point.

The dry run: the fixture is abab$, whose 8 nodes, edge sum 12, 5 leaves, and 7 distinct substrings are asserted by the C# suite, identical in the other six.

1. The suffix abab$ hangs whole off the root, one edge of length 5, and bab$ keys on b as a fresh root child: three nodes.
2. The suffix ab$ walks the a edge and dies at offset 2, $ against a, so the edge splits into [0, 2) plus [2, 5), the mid node gains a $ leaf, and the count reaches five nodes.
3. The suffix b$ dies the same way on the b edge at offset 1, $ against a: [1, 5) splits into [1, 2) plus [2, 5), seven nodes.
4. The lone $ is a third root child, closing at 8 nodes and 5 leaves, with the edge sum 2 + 3 + 1 + 1 + 3 + 1 + 1 = 12.
5. Distinct substrings read 12 - 5 = 7, and the first mid node sits at string depth 2: ab is the longest repeat, occurring 2 times.

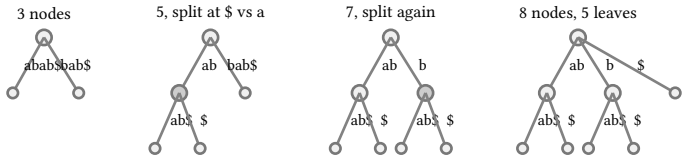


figure 277: the abab$ tree growing one suffix per frame, edges labeled by the strings their index pairs spell, the two split nodes shaded

The 8 nodes, the 12, and the depth-2 ab close the run as pinned, and the listings below insert the suffixes in seven languages.

```c
static void insert_suffix(int i) {
  int cur = 1, j = i;
  while (j < SLEN) {
    int c = cidx(S[j]);
    if (!child[cur][c]) {
      child[cur][c] = new_node(j, SLEN - 1);
      return;
    }
    int nx = child[cur][c];
    int k = el[nx];
    while (k ≤ er[nx] && j < SLEN && S[k] == S[j]) {
      k++;
      j++;
    }
    if (k > er[nx]) {
      cur = nx;
      continue;
    }
    int mid = new_node(el[nx], k - 1); // the shared prefix becomes a node
    child[cur][c] = mid;
    child[mid][cidx(S[k])] = nx;
    el[nx] = k;
    child[mid][cidx(S[j])] = new_node(j, SLEN - 1);
    return;
  }
}
```

listing 1083: c, the suffix insertion walk with the mid-edge split (source: dsa/samples-c/src/Ch33/suffixtree.c, lines 52-77)

```
func (tr *SuffixTree) insertSuffix(i int) {
  cur, j := 0, i
  s := tr.s
  for j < len(s) {
    c := s[j]
    son, ok := tr.child[cur][c]
    if !ok {
      id := tr.newNode(j, len(s)-1)
      tr.child[cur][c] = id
      return
    }
    l, r := tr.l[son], tr.r[son]
    k := l
    for k <= r && j < len(s) && s[k] == s[j] {
      k++
      j++
    }
    if k > r {
      cur = son
      continue
    }
    mid := tr.newNode(l, k-1)
    tr.child[cur][c] = mid
    tr.child[mid][s[k]] = son
    tr.l[son], tr.r[son] = k, r
    id := tr.newNode(j, len(s)-1)
    tr.child[mid][s[j]] = id
    return
  }
}
```

listing 1084: go, insertSuffix with the edge relabel on split (source: dsa/samples-go/ch33/suffixtree.go, lines 36-65)

```
  static void insertSuffix(int i) {
    int cur = 1, j = i;
    while (j < SLEN) {
      int c = cidx(S.charAt(j));
      if (child[cur][c] == 0) {
        child[cur][c] = newNode(j, SLEN - 1);
        return;
      }
      int nx = child[cur][c];
      int k = el[nx];
      while (k ≤ er[nx] && j < SLEN && S.charAt(k) == S.charAt(j)) {
        k++;
        j++;
      }
      if (k > er[nx]) {
        cur = nx;
        continue;
      }
      int mid = newNode(el[nx], k - 1); // the shared prefix becomes a node
      child[cur][c] = mid;
      child[mid][cidx(S.charAt(k))] = nx;
      el[nx] = k;
      child[mid][cidx(S.charAt(j))] = newNode(j, SLEN - 1);
      return;
    }
```

listing 1085: java, insertSuffix with the mid-edge split, child arrays keyed by first character (source: dsa/samples-java/src/Ch33/Suffixtree.java, lines 56-80)

```
    public SuffixTree(string s)
    {
        _s = s;
        NodeCount = 1;
        for (int i = 0; i < s.Length; i++)
        {
            Node cur = _root;
            int j = i;
            while (j < s.Length)
            {
                char c = s[j];
                if (!cur.Children.TryGetValue(c, out (int L, int R, Node Child) edge))
                {
                    cur.Children[c] = (j, s.Length - 1, new Node());
                    NodeCount++;
                    break;
                }
                int k = edge.L;
                while (k ≤ edge.R && j < s.Length && s[k] == s[j])
                {
                    k++;
                    j++;
                }
                if (k > edge.R)
                {
                    cur = edge.Child;
                    continue;
                }
                // the walk stops mid-label: split the edge
                var mid = new Node();
                cur.Children[c] = (edge.L, k - 1, mid);
                mid.Children[s[k]] = (k, edge.R, edge.Child);
                mid.Children[s[j]] = (j, s.Length - 1, new Node());
                NodeCount += 2;
                break;
            }
        }
    }
```

listing 1086: c#, the constructor loop, the split allocating the mid node and two edges (source: dsa/samples/src/Ch33/SuffixTree.cs, lines 18-55)

```
  for (let i = 0; i < s.length; i++) {
    let cur = root;
    let j = i;
    while (j < s.length) {
      const c = s[j];
      if (!cur.children.has(c)) {
        cur.children.set(c, { edge: [j, s.length - 1], node:
          { children: new Map() } });
        nodes += 1;
        break;
      }
      const { edge, node: child } = cur.children.get(c);
      const [l, r] = edge;
      let k = l;
      while (k ≤ r && j < s.length && s[k] === s[j]) {
        k += 1;
        j += 1;
      }
      if (k > r) {
        cur = child;
        continue;
      }
      // the walk stops mid-label: split the edge
      const mid = { children: new Map() };
      cur.children.set(c, { edge: [l, k - 1], node: mid });
      mid.children.set(s[k], { edge: [k, r], node: child });
      mid.children.set(s[j], { edge: [j, s.length - 1], node:
        { children: new Map() } });
      nodes += 2;
      break;
    }
  }
```

listing 1087: javascript, the build loop, edges as index pairs, mid node wired in place (source: dsa/samples-js/src/ch33-suffixtree.mjs, lines 13-44)

```python
def build_suffix_tree(s):
    # s ends in a terminal char that makes the tree explicit
    root = TNode()
    nodes = 1
    for i in range(len(s)):
        cur = root
        j = i
        while j < len(s):
            c = s[j]
            if c not in cur.children:
                cur.children[c] = ((j, len(s) - 1), TNode())
                nodes += 1
                break
            (lft, rgt), child = cur.children[c]
            k = lft
            while k ≤ rgt and j < len(s) and s[k] == s[j]:
                k += 1
                j += 1
            if k > rgt:
                cur = child  # consumed the whole label, keep walking
                continue
            mid = TNode()  # split the edge at k
            cur.children[c] = ((lft, k - 1), mid)
            mid.children[s[k]] = ((k, rgt), child)
            mid.children[s[j]] = ((j, len(s) - 1), TNode())
            nodes += 2
            break
    return root, nodes
```

listing 1088: python, build_suffix_tree, the split at k with two fresh edges (source: dsa/samples-py/src/Ch33/suffixtree.py, lines 22-49)

```
  for i = 1, n do -- insert suffix s[i..n]
    local cur = self.root
    local j = i
    while j ≤ n do
      local c = s:byte(j)
      local edge = cur.children[c]
      if edge == nil then
        cur.children[c] = {{j, n}, newnode()}
        self.nodes = self.nodes + 1
        break
      end
      local l, r = edge[1][1], edge[1][2]
      local child = edge[2]
      local k = l
      while k ≤ r and j ≤ n and s:byte(k) == s:byte(j) do
        k = k + 1
        j = j + 1
      end
      if k > r then
        cur = child -- walked the whole edge, descend
      else
        -- split the edge at k-1 and hang both tails off the mid node
        local mid = newnode()
        cur.children[c] = {{l, k - 1}, mid}
        mid.children[s:byte(k)] = {{k, r}, child}
        mid.children[s:byte(j)] = {{j, n}, newnode()}
        self.nodes = self.nodes + 2
        break
      end
    end
  end
```

listing 1089: lua, the insertion loop inside the constructor, edges as {l, r} pairs (source: dsa/samples-lua/ch33_suffixtree.lua, lines 21-51)

Three queries ride the explicit tree. Distinct substrings are the sum of all edge lengths minus the leaves, because each leaf edge donates exactly the terminal character, which no real substring contains. Occurrences of q are the leaves under the node where the walk for q ends, a walk that stops mid-edge still counts the child's leaves. The longest repeated substring is the label down to the deepest internal node by string depth, since a node with two children is exactly a substring occurring at least twice with different continuations. The fixtures pin banana$ at 11 nodes, edge sum 22, 7 leaves, 15 distinct, and ana as the deepest repeat at depth 3, with occurrences a 3, ana 2, an 2, nan 1, ban 1, banana 1; abab$ at 8 nodes, sum 12, 5 leaves, 7 distinct, ab the repeat; aaa$ at 7 nodes, sum 7, 4 leaves, 3 distinct, aa the repeat; and the scale anchor mississippi$ at 19 nodes, sum 65, 12 leaves, 53 distinct, issi the repeat at depth 4. The lone terminal "$" builds 2 nodes, edge sum 1, 1 leaf, 0 distinct, and every suite asserts the leaves equal len(s) invariant and cross-checks distinct and occurrences against brute.

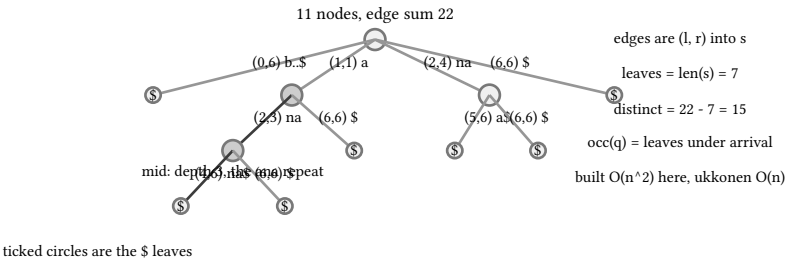


figure 278: the banana$ compressed tree with (l, r) edge labels, the terminal $ leaves ticked, the ana branch shaded as the deepest repeat

33.5. lyndon factorization, least rotation, and repetitions

Three tools share one file because they share one outlook, that a string compares against its own rotations. A lyndon word is a string strictly smaller than every nontrivial rotation of itself, and duval's algorithm factors s into lyndon words in non-increasing order in one linear scan with three pointers: i opens the current factor, j walks ahead, and k tracks the period inside the candidate. On a strict rise s[k] below s[j] the candidate is still lyndon and k resets to i, on an equal pair k advances with j, and when the scan stops at a fall the factor closes with length j - k.

The dry run: the fixture is banana, whose least rotation abanan at start 5 is asserted by the C# suite against the brute minimum over all rotations, identical in the other six. Duval's own walk over abacaba stays with the figure below, this run takes the duel.

1. The duel opens at i = 0, j = 1 over the doubled banana: offset 0 compares b against a, 0 loses at once and skips to 0 + 0 + 1 = 1, colliding with j, so j moves to 2.
2. Candidate 2 falls next, a against n at offset 0: j skips to 2 + 0 + 1 = 3.
3. Candidates 1 and 3 agree through three offsets, a a, n n, a a, and break at offset 3, n against b: 1 loses its whole matched window and lands at 1 + 3 + 1 = 5.
4. Candidates 5 and 3 break at offset 1, b against n: 3 skips to 3 + 1 + 1 = 5, collides with i, and the scan ends with the survivor
   5.
5. The rotation at 5 spells s[5] then s[0..5]: a b a n a n, abanan, the pinned pair.

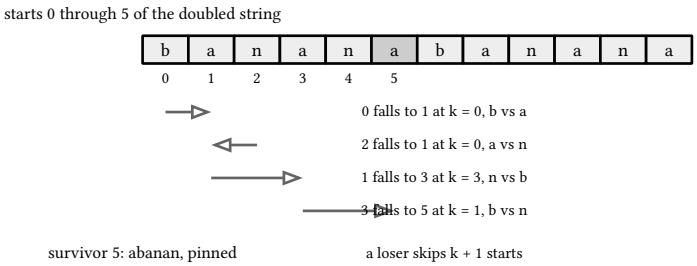


figure 279: the least-rotation duel on doubled banana, four knockouts with the breaking pair named, each loser skipping past its matched window, the survivor shaded

The survivor 5 and its abanan are pinned by every suite, and the listings below carry all three tools in seven languages.

```c
static void duval(const char *s) {
  factcount = 0;
  int len = (int)strlen(s), i = 0;
  while (i < len) {
    int j = i + 1, k = i;
    while (j < len && s[k] ≤ s[j]) {
      if (s[k] < s[j])
        k = i;
      else
        k++;
      j++;
    }
    int ln = j - k;
    memcpy(factors[factcount], s + i, (size_t)ln);
    factors[factcount][ln] = 0;
    factlen[factcount] = ln;
    factcount++;
    i += ln;
  }
}
```

listing 1090: c, duval's three-pointer loop, the factor closing at the fall (source: dsa/samples-c/src/Ch33/lyndon.c, lines 28-47)

```go
func Duval(s string) []string {
  var factors []string
  i, n := 0, len(s)
  for i < n {
    j, k := i+1, i
    for j < n && s[k] <= s[j] {
      if s[k] < s[j] {
        k = i
      } else {
        k++
      }
      j++
    }
    ln := j - k
    factors = append(factors, s[i:i+ln])
    i += ln
  }
  return factors
}
```

listing 1091: go, duval, reset k to i on a strict rise (source: dsa/samples-go/ch33/lyndon.go, lines 8-26)

```
static void duval(String s) {
  factcount = 0;
  int len = s.length(), i = 0;
  while (i < len) {
    int j = i + 1, k = i;
    while (j < len && s.charAt(k) ≤ s.charAt(j)) {
      if (s.charAt(k) < s.charAt(j))
        k = i;
      else
        k++;
      j++;
    }
    int ln = j - k;
    factors[factcount++] = s.substring(i, i + ln);
    i += ln;
  }
}
```

listing 1092: java, duval's three-pointer loop, charAt reads, the factor closing at the fall (source: dsa/samples-java/src/Ch33/Lyndon.java, lines 26-43)

```
  public static string[] Duval(string s)
  {
      var factors = new List<string>();
      int i = 0;
      int n = s.Length;
      while (i < n)
      {
          int j = i + 1;
          int k = i;
          while (j < n && s[k] ≤ s[j])
          {
              if (s[k] < s[j])
                  k = i; // strict rise: reset the period candidate
              else
                  k++; // equal run: walk inside the candidate
              j++;
          }
          int len = j - k;
          factors.Add(s[i..(i + len)]);
          i += len;
      }
      return factors.ToArray();
  }
```

listing 1093: c#, the same scan, the reset and the equal-run walk commented (source: dsa/samples/src/Ch33/Lyndon.cs, lines 11-33)

```javascript
export function duval(s) {
  const factors = [];
  const n = s.length;
  let i = 0;
  while (i < n) {
    let j = i + 1;
    let k = i;
    while (j < n && s[k] ≤ s[j]) {
      if (s[k] < s[j]) {
        k = i; // strict rise: the factor restarts here
      } else {
        k += 1;
      }
      j += 1;
    }
    const ln = j - k;
    factors.push(s.slice(i, i + ln));
    i += ln;
  }
  return factors;
}
```

listing 1094: javascript, the duval loop (source: dsa/samples-js/src/ch33-lyndon.mjs, lines 11-31)

```python
def duval(s):
    factors = []
    i = 0
    size = len(s)
    while i < size:
        j = i + 1
        k = i
        while j < size and s[k] ≤ s[j]:  # reset k on a strict rise
            if s[k] < s[j]:
                k = i
            else:
                k += 1
            j += 1
        length = j - k
        factors.append(s[i : i + length])
        i += length
    return factors
```

listing 1095: python, duval, the two branches inside the rise-or-equal guard (source: dsa/samples-py/src/Ch33/lyndon.py, lines 14-30)

```lua
local function duval(s)
  local factors = {}
  local i = 1 -- 1-based scan, i0 = i - 1 in the prose
  local n = #s
  while i ≤ n do
    local j, k = i + 1, i
    while j ≤ n and s:byte(k) ≤ s:byte(j) do
      if s:byte(k) < s:byte(j) then
        k = i -- strict rise resets the period
      else
        k = k + 1
      end
      j = j + 1
    end
    local ln = j - k
    factors[#factors + 1] = s:sub(i, i + ln - 1)
    i = i + ln
  end
  return factors
end
```

listing 1096: lua, the same loop in 1-based indices (source: dsa/samples-lua/ch33_lyndon.lua, lines 8-27)

The least rotation runs the same three pointers as a duel on s + s: two candidate starts i and j compare character by character at offset k, and whichever loses is skipped past its whole matched window, k + 1 positions, because none of those starts can win either. First index wins ties, and the survivor after the duel is the minimal rotation. The repetition scanner reads equal-pair rails: for each period p, find the maximal segments where s[t] equals s[t + p], and a segment [a, b) with b - a at least p is the maximal repetition (a, b + p - a, p) with length at least 2p. That scan is O(n^2) worst case here, with the main-lorentz algorithm cited for O(n log n), the same cost ruling as the suffix tree above.

The fixtures pin all three pieces identically. Duval factors abababab into ab, ab, ab; abacaba into abac, ab, a; banana into b, an, an, a; acacab into ac, ac, ab; and aab stays whole. The duel returns abanan from banana at 5, ababc from abcab at 3, abacac from acacab at 4, aketeanteaketete from teaketeteaketean at 9, and aaaa at 0. The repetitions read banana as (1, 5, 2), the run anana; abab as (0, 4, 2); aabaab as (0, 2, 1), (0, 6, 3), (3, 2, 1); aaaa as (0, 4, 1) and (0, 4, 2), period 1 and period 2 both maximal; and abcabcabc as (0, 9, 3). The cross-checks are the strictest in the chapter: every emitted factor is verified lyndon against its own rotations, the factor sequence non-increasing, the duel against the brute minimum over all rotations, and the repetition scan against an independent brute enumerator, with the javascript suite adding its own 150 seeded random strings on top. Java runs the brute twins over its named families: every duel checked inside checkRotation against the minimum over all rotations, every repetition table against the independent enumerator.

The application is icpc world finals 2022 problem Y (book 10, chapter 11), compression, where a square-free reachability argument is the contest face of repetition detection; the problem's answer reasoning stays with the icpc book.

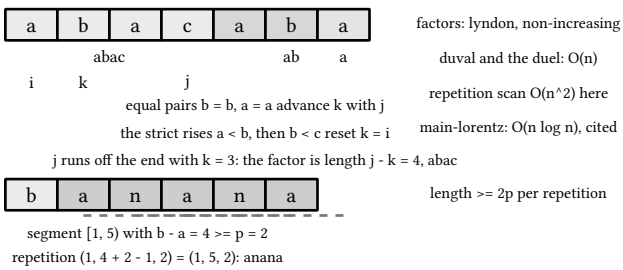


figure 280: duval's three pointers walking abacaba with the three factors boxed, and the p = 2 equal-pair rail of banana highlighting anana as the repetition (1, 5, 2)

33.6. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's five sample files per language, go test files excluded:

language	build sloc	container dependency	boundary note
c	848	static arrays, 26-way transition table	z and manacher over fixed buffers, the sam child array per state, insertion-order suffix tree
go	404	slices, `sort.Slice`	sam link-tree order sorted by len, the duel and scan as free functions
java	805	jdk 27 stdlib, String.charAt walks	per-state int[26] sam transitions cloned by array copy, the \x01 separator as a char constant, insertion-order suffix tree over child arrays
c#	397	`List<Dictionary<char,int>>`, linq	long z arrays, occurrence counts precomputed in the constructor
javascript	304	`Map` transitions, private class fields	plain string compare throughout, the randomized cross-check family seeded by an lcg
python	564	dict transitions, inline asserts	the extend method split out, brute twins per family
lua	654	tables, string.byte keys	1-based storage over 0-based reported positions, the shift documented at the boundary

sources: cp-algorithms, "Z-function", cp-algorithms.com/string/z-function.html, "Manacher's Algorithm - Finding all sub-palindromes in O(N)", cp-algorithms.com/string/manacher.html, "Suffix Automaton", cp-algorithms.com/string/suffix-automaton.html, "Suffix Tree", cp-algorithms.com/string/suffix-tree-ukkonen.html, whose linear-time ukkonen construction is cited in prose while the naive insertion is what ships, "Lyndon factorization", cp-algorithms.com/string/lyndon_factorization.html, covering duval and the least-rotation duel, and "Finding repetitions", cp-algorithms.com/string/main_lorentz.html, the O(n log n) reference for the period scan, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Application sources: icpc world finals 2023 problem F and 2022 problem Y (book 10). Sample behavior verified by the seven suite gates scoped to chapter 33: c 5 files and 147 checks, go 16 test functions, java 5 files and 147 checks under run-java-samples, c# 24 facts, javascript 19 tests and 119 asserts, python 5 files and 333 asserts, lua 19 checks, zero skipped.

34. linear algebra

Matrices earn a chapter in an algorithms book the moment counting walks, solving systems, and pricing determinants stop being hand work. The arithmetic partners are chapter 16′s modpow and modinv, book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic) cross-named once and used throughout, and the exactness ruling is the same one the rest of the book runs on: elimination over the prime field Z p with p = 1e9+7 gives answers all seven languages pin digit for digit, and integer determinants travel fraction-free so no rounding ever enters. Six tools make the chapter: gauss-jordan over a prime field, the bareiss determinant and rank, cramer on the small systems, matrix exponentiation, kirchhoff's spanning tree count, and hensel lifting over prime powers.

34.1. gaussian elimination over a prime field

Dense gauss-jordan over Z p with p = 1e9+7 walks the columns left to right: pick the first nonzero row as the pivot, swap it into place, normalize the whole pivot row by the inverse of the pivot computed as pow(pivot, p - 2), then eliminate the pivot column out of every other row. Because the field is prime, every nonzero element inverts and the algorithm never divides by zero mid-flight, and because the arithmetic is exact, a solved system verifies by plugging back. Three statuses come out of the walk: unique when every column found a pivot, inconsistent when a zero row carries a nonzero right-hand side, and underdetermined when some column never found one. O(n^3).

The dry run: the fixture is the 3x3 system with rows 1, 2, 3, 4, 5, 6, and 7, 8, 10 against 6, 15, 25, whose solution 1, 1, 1 the C# suite asserts together with its plug-back, identical in the other five.

1. Column 0 opens with pivot 1 in place and eliminates 4 and 7 copies of row 0: row 1 lands 0, −3, −6 | −9 with 5 - 4 × 2 = −3, row 2 lands 0, −6, −11 | −17.
2. Column 1 pivots on −3, normalizes row 1 to 0, 1, 2 | 3, then clears the column both ways: row 2 lands 0, 0, 1 | 1 with −11 + 12 = 1 and −17 + 18 = 1, row 0 lands 1, 0, −1 | 0.
3. Column 2 pivots on 1 and clears upward, finishing row 0 at 1, 0, 0 | 1 and row 1 at 0, 1, 0 | 1: the identity with x = 1, 1, 1 sitting in the last column.
4. The plug-back is the suite's own assert: 1 + 2 + 3 = 6, 4 + 5 + 6 = 15, 7 + 8 + 10 = 25.
5. The modular twist solves 2x = 1 in the field, the pivot row normalized by pow(2, p - 2) = 500000004, the pinned half, so the diagonal system answers 500000004, 1.

after	row 0	row 1	row 2
start	1, 2, 3 | 6	4, 5, 6 | 15	7, 8, 10 | 25
column 0	1, 2, 3 | 6	0, −3, −6 | −9	0, −6, −11 | −17
column 1	1, 0, −1 | 0	0, 1, 2 | 3	0, 0, 1 | 1
column 2	1, 0, 0 | 1	0, 1, 0 | 1	0, 0, 1 | 1

The 1, 1, 1 closes the walk as pinned, and the listings below run the same column walk in seven languages.

```
  for (int c = 0; c < size; c++) {
    int piv = -1;
    for (int i = r; i < size; i++)
      if (mm[i][c] % P) {
        piv = i;
        break;
      }
    if (piv < 0)
      continue;
    for (int j = 0; j ≤ size; j++) {
      unsigned long long t = mm[r][j];
      mm[r][j] = mm[piv][j];
      mm[piv][j] = t;
    }
    unsigned long long inv = modpow(mm[r][c], P - 2);
    for (int j = 0; j ≤ size; j++)
      mm[r][j] = mm[r][j] * inv % P;
    for (int i = 0; i < size; i++)
      if (i ≠ r && mm[i][c] % P) {
        unsigned long long f = mm[i][c];
        for (int j = 0; j ≤ size; j++)
          mm[i][j] = (mm[i][j] + P - f * mm[r][j] % P) % P;
      }
    where[c] = r;
    r++;
  }
```

listing 1097: c, the column walk, swap, inverse normalize, elimination, the column-to-row map (source: dsa/samples-c/src/Ch34/gauss.c, lines 55-80)

```
  for c := 0; c < n; c++ {
    piv := -1
    for i := r; i < n; i++ {
      if m[i][c]%p != 0 {
        piv = i
        break
      }
    }
    if piv < 0 {
      continue
    }
    m[r], m[piv] = m[piv], m[r]
    inv := modInv(m[r][c], p)
    for j := range m[r] {
      m[r][j] = m[r][j] * inv % p
    }
    for i := 0; i < n; i++ {
      if i != r && m[i][c]%p != 0 {
        f := m[i][c]
        for j := 0; j <= n; j++ {
          m[i][j] = ((m[i][j]-f*m[r][j])%p + p) % p
        }
      }
    }
    where[c] = r
    r++
  }
```

listing 1098: go, the loop with modinv inline, the where map read at the end (source: dsa/samples-go/ch34/gauss.go, lines 47-73)

```
    int r = 0;
    for (int c = 0; c < size; c++) {
      int piv = -1;
      for (int i = r; i < size; i++)
        if (mm[i][c] % P != 0) {
          piv = i;
          break;
        }
      if (piv < 0)
        continue;
      for (int j = 0; j <= size; j++) {
        long t = mm[r][j];
        mm[r][j] = mm[piv][j];
        mm[piv][j] = t;
      }
      long inv = modpow(mm[r][c], P - 2);
      for (int j = 0; j <= size; j++)
        mm[r][j] = mm[r][j] * inv % P;
      for (int i = 0; i < size; i++)
        if (i != r && mm[i][c] % P != 0) {
          long f = mm[i][c];
          for (int j = 0; j <= size; j++)
            mm[i][j] = (mm[i][j] + P - f * mm[r][j] % P) % P;
        }
      where[c] = r;
      r++;
    }
    for (int i = 0; i < size; i++) {
```

listing 1099: java, the column walk, swap, inverse normalize, elimination, the column-to-row map (source: dsa/samples-java/src/Ch34/Gauss.java, lines 51-78)

```
        for (int c = 0; c < n && r < n; c++)
        {
            int piv = -1;
            for (int i = r; i < n; i++)
                if (m[i][c] % p != 0)
                {
                    piv = i;
                    break;
                }
            if (piv < 0)
                continue; // column without a pivot
            (m[r], m[piv]) = (m[piv], m[r]);
            long inv = NumTheory.ModPow(m[r][c], p - 2, p);
            for (int j = c; j ≤ n; j++)
                m[r][j] = m[r][j] * inv % p;
            for (int i = 0; i < n; i++)
                if (i != r && m[i][c] % p != 0)
                {
                    long f = m[i][c];
                    for (int j = c; j ≤ n; j++)
                        m[i][j] = ((m[i][j] - f * m[r][j]) % p + p) % p;
                }
            where[c] = r;
            r++;
        }
```

listing 1100: c#, the same loop, modpow pulled from the chapter 16 class (source: dsa/samples/src/Ch34/Gauss.cs, lines 37-61)

```
  for (let c = 0; c < n; c++) {
    let piv = -1;
    for (let i = r; i < n; i++) {
      if (M[i][c] % P !== 0n) {
        piv = i;
        break;
      }
    }
    if (piv === -1) continue;
    [M[r], M[piv]] = [M[piv], M[r]];
    const inv = powMod(M[r][c], P - 2n, P);
    M[r] = M[r].map((v) => (v * inv) % P);
    for (let i = 0; i < n; i++) {
      if (i !== r && M[i][c] % P !== 0n) {
        const f = M[i][c];
        M[i] = M[i].map((v, j) => ((((v - f * M[r][j]) % P) + P) % P));
      }
    }
    where[c] = r;
    r += 1;
  }
```

listing 1101: javascript, the same loop, every residue product on BigInt past the 2^53 boundary (source: dsa/samples-js/src/ch34-gauss.mjs, lines 29-49)

```python
def gauss_mod(a, b):
    # returns (status, x); status in unique / inconsistent / under
    rows = len(a)
    m = [row[:] + [b[i]] for i, row in enumerate(a)]
    where = [-1] * rows  # column -> pivot row map
    r = 0
    for col in range(rows):
        piv = next((i for i in range(r, rows) if m[i][col] % P), None)
        if piv is None:
            continue
        m[r], m[piv] = m[piv], m[r]
        inv = pow(m[r][col], P - 2, P)
        m[r] = [v * inv % P for v in m[r]]  # normalize the pivot row
        for i in range(rows):  # eliminate every other row
            if i != r and m[i][col] % P:
                f = m[i][col]
                m[i] = [(m[i][j] - f * m[r][j]) % P for j in range(rows + 1)]
        where[col] = r
        r += 1
    for i in range(rows):  # a zero row with nonzero rhs refuses
        if all(v % P == 0 for v in m[i][:rows]) and m[i][rows] % P:
            return "inconsistent", None
    if any(w == -1 for w in where):
        return "underdetermined", None
    x = [m[where[c]][rows] % P for c in range(rows)]
    return "unique", x
```

listing 1102: python, the whole solver, three statuses, one list comprehension per row op (source: dsa/samples-py/src/Ch34/gauss.py, lines 18-43)

```lua
  for c = 1, n do
    local piv = nil
    for i = r, n do
      if M[i][c] % P ~= 0 then piv = i break end
    end
    if piv == nil then
      -- no pivot in this column
    else
      M[r], M[piv] = M[piv], M[r]
      local inv = modpow(M[r][c], P - 2)
      for j = 1, n + 1 do M[r][j] = M[r][j] * inv % P end
      for i = 1, n do
        if i ~= r and M[i][c] % P ~= 0 then
          local f = M[i][c]
          for j = 1, n + 1 do
            M[i][j] = (M[i][j] - f * M[r][j]) % P
          end
        end
      end
      where[c] = r
      r = r + 1
    end
  end
```

listing 1103: lua, the same loop, plain products with a percent every step (source: dsa/samples-lua/ch34_gauss.lua, lines 31-53)

The fixture families pin the behavior. The 3x3 system with coefficients 1, 2, 3 by rows, 4, 5, 6, and 7, 8, 10 against the right-hand side 6, 15, 25 solves to 1, 1, 1, with the plug-back asserted. The modular twist is the teaching row: the

diagonal system 2, 0, 0, 1 against 1, 1 solves to 500000004 and 1, because one half in this field is the pinned inverse of 2, the same constant chapter 16 carries. The dependent pair 1, 1 and 2, 2 refuses twice, once as inconsistent against 0, 1, the zero-times-x-equals-one row, and once as underdetermined against 2, 4, no solution emitted either way. The edges run the 1x1 system 2x = 4 to x = 2 and the identity returning its right-hand side untouched.

The application is icpc world finals 2022 problem P (book 10, chapter 11), turning red, where lights and buttons form a linear system over Z 3 , each light one equation in the press counts and each press contributing its color's shift. The official solution propagates per connected component because each variable appears in at most two equations, and plain elimination answers the same question: any subset of independent equations solves, and the component structure only changes the bookkeeping, never the solution set.

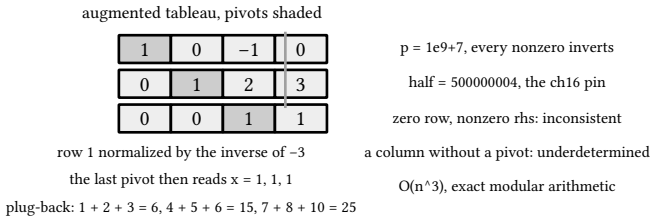


figure 281: the 3x3 fixture mid-elimination, the pivot row normalized by the modular inverse, zeros above and below, the plug-back underneath

34.2. the fraction-free determinant and rank

Elimination over the rationals trades exactness for fractions, and over the reals it trades exactness for rounding. The bareiss variant keeps integers: each step subtracts cross products, the cell times the pivot minus the cell's column mate times the pivot row, and divides exactly by the previous pivot. The division is exact because every intermediate entry is itself an integer minor of the original matrix, which is the whole theorem, and the suites assert the remainder of that division is zero. When the matrix is square and full rank the determinant is the sign times the last pivot, a swap flipping the sign, otherwise 0, and the rank is the pivot count, valid over the rationals for integer matrices.

The dry run: the fixtures are det of 1, 2, 3, 4 at −2 and the singular 1, 2, 3 by 4, 5, 6 by 7, 8, 9 at det 0, rank 2, both asserted by the C# suite, identical in the other six.

1. The 2x2 pivots on 1 with prev = 1, so the exact division divides silently: det = 1 × 4 - 2 × 3 = −2 at rank 2.
2. The 3x3 pivots on 1 again, and the cross-product step computes each cell as the cell times the pivot minus its column mate times the pivot row: row 1 lands −3, −6 with 1 × 5 - 4 × 2 = −3, row 2 lands −6, −12 with 1 × 8 - 7 × 2 = −6.
3. The second pivot is −3 and row 2 divides exactly: (-3 × −12 - −6 × −6) / 1 = 36 - 36 = 0.
4. The third column holds no nonzero pivot, so the walk stops at det 0 with rank 2, the two pivots placed.

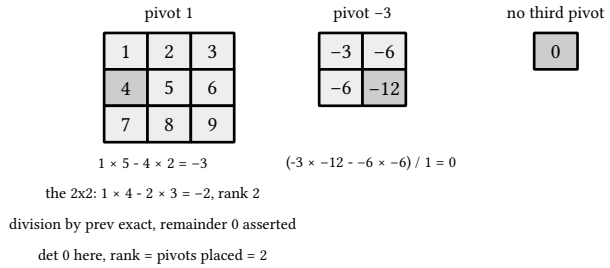


figure 282: the singular 3x3 collapsing one pivot at a time, the cross products of the shaded cell spelled out, the 2x2 corner −2 alongside

The −2 and the rank-2 zero land as pinned, and the listings below run bareiss in seven languages.

```
static void bareiss(const int m[N][N], int rows, int cols, long long *det,
                    int *rank) {
  static long long a[N][N];
  for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
      a[i][j] = m[i][j];
  int sign = 1, prev = 1, r = 0;
  for (int c = 0; c < cols; c++) {
    int piv = -1;
    for (int i = r; i < rows; i++)
      if (a[i][c] != 0) {
        piv = i;
        break;
      }
    if (piv < 0)
      continue;
    if (piv != r) {
      for (int j = 0; j < cols; j++) {
        long long t = a[r][j];
        a[r][j] = a[piv][j];
        a[piv][j] = t;
      }
      sign = -sign;
    }
    for (int i = r + 1; i < rows; i++) {
      for (int j = c + 1; j < cols; j++) {
        a[i][j] = (a[i][j] * a[r][c] - a[i][c] * a[r][j]);
        if (a[i][j] % prev != 0) {
          *det = -999; // the exact division is asserted, never trips here
          return;
        }
        a[i][j] /= prev;
      }
      a[i][c] = 0;
    }
    prev = (int)a[r][c];
    r++;
  }
  *det = (rows == cols && r == rows) ? sign * a[rows - 1][cols - 1] : 0;
  *rank = r;
}
```

listing 1104: c, the bareiss function, cross products divided by the previous pivot, det and rank out (source: dsa/samples-c/src/Ch34/determinant.c, lines 27-67)

```
func Bareiss(mat [][]int64) (int64, int) {
  a := make([][]int64, len(mat))
  for i := range mat {
    a[i] = append([]int64(nil), mat[i]...)
  }
  n := len(a)
  if n == 0 {
    return 0, 0
  }
  m := len(a[0])
  sign := int64(1)
  prev := int64(1)
  r := 0
  for c := 0; c < m; c++ {
    piv := -1
    for i := r; i < n; i++ {
      if a[i][c] != 0 {
        piv = i
        break
      }
    }
    if piv < 0 {
      continue
    }
    if piv != r {
      a[r], a[piv] = a[piv], a[r]
      sign = -sign
    }
    for i := r + 1; i < n; i++ {
      for j := c + 1; j < m; j++ {
        a[i][j] = (a[i][j]*a[r][c] - a[i][c]*a[r][j]) / prev
      }
      a[i][c] = 0
    }
    prev = a[r][c]
    r++
  }
  if n == m && r == n {
    return sign * a[n-1][n-1], r
  }
  return 0, r
}
```

listing 1105: go, bareiss with the pivot search, the permutation expansion below (source: dsa/samples-go/ch34/determinant.go, lines 9-50)

```
  static void bareiss(int[][] m, int rows, int cols) {
    for (int i = 0; i < rows; i++)
      for (int j = 0; j < cols; j++)
        a[i][j] = m[i][j];
    int sign = 1, prev = 1, r = 0;
    for (int c = 0; c < cols; c++) {
      int piv = -1;
      for (int i = r; i < rows; i++)
        if (a[i][c] != 0) {
          piv = i;
          break;
        }
      if (piv < 0)
        continue;
      if (piv != r) {
        for (int j = 0; j < cols; j++) {
          long t = a[r][j];
          a[r][j] = a[piv][j];
          a[piv][j] = t;
        }
        sign = -sign;
      }
      for (int i = r + 1; i < rows; i++) {
        for (int j = c + 1; j < cols; j++) {
          a[i][j] = a[i][j] * a[r][c] - a[i][c] * a[r][j];
          if (a[i][j] % prev != 0) {
            detOut = -999; // the exact division is asserted, never trips
            return;
          }
          a[i][j] /= prev;
        }
        a[i][c] = 0;
      }
      prev = (int) a[r][c];
      r++;
    }
    detOut = (rows == cols && r == rows) ? sign * a[rows - 1][cols - 1] : 0;
    rankOut = r;
  }
```

listing 1106: java, bareiss, cross products divided by the previous pivot, det and rank out (source: dsa/samples-java/src/Ch34/Determinant.java, lines 30-69)

```
    public static (long Det, int Rank) DetRank(long[][] mat)
    {
        int n = mat.Length;
        int cols = mat[0].Length;
        var a = new long[n][];
        for (int i = 0; i < n; i++)
            a[i] = (long[])mat[i].Clone();
        int sign = 1;
        long prev = 1;
        int r = 0;
        for (int c = 0; c < cols && r < n; c++)
        {
            int piv = -1;
            for (int i = r; i < n; i++)
                if (a[i][c] != 0)
                {
                    piv = i;
                    break;
                }
            if (piv < 0)
                continue;
            if (piv != r)
            {
                (a[r], a[piv]) = (a[piv], a[r]);
                sign = -sign;
            }
            for (int i = r + 1; i < n; i++)
            {
                for (int j = c + 1; j < cols; j++)
                    a[i][j] = (a[i][j] * a[r][c] - a[i][c] * a[r][j]) / prev; // exact division
                a[i][c] = 0;
            }
            prev = a[r][c];
            r++;
        }
        long det = n == cols && r == n ? sign * a[n - 1][n - 1] : 0;
        return (det, r);
    }
```

listing 1107: c#, the same function over longs, the exact division on one line (source: dsa/samples/src/Ch34/Determinant.cs, lines 10-47)

```
export function bareiss(mat) {
  const a = mat.map((row) => [...row]);
  const n = a.length;
  const m = a[0].length;
  let sign = 1;
  let prev = 1;
  let r = 0;
  for (let c = 0; c < m; c++) {
    let piv = -1;
    for (let i = r; i < n; i++) {
      if (a[i][c] !== 0) {
        piv = i;
        break;
      }
    }
    if (piv === -1) continue;
    if (piv !== r) {
      [a[r], a[piv]] = [a[piv], a[r]];
      sign = -sign;
    }
    for (let i = r + 1; i < n; i++) {
      for (let j = c + 1; j < m; j++) {
        const v = a[i][j] * a[r][c] - a[i][c] * a[r][j];
        if (v % prev !== 0) throw new Error("bareiss division not exact");
        a[i][j] = v / prev; // asserted exact, an integer minor
      }
      a[i][c] = 0;
    }
    prev = a[r][c];
    r += 1;
  }
  const det = n === m && r === n ? sign * a[n - 1][m - 1] : 0;
  return { det, rank: r };
}
```

listing 1108: javascript, bareiss, the division asserted exact, an integer minor per cell (source: dsa/samples-js/src/ch34-determinant.mjs, lines 10-43)

```
def bareiss(mat):
    # exact integer determinant and rank
    a = [row[:] for row in mat]
    rows = len(a)
    cols = len(a[0])
    sign = 1
    prev = 1
    r = 0
    for col in range(cols):
        piv = next((i for i in range(r, rows) if a[i][col] != 0), None)
        if piv is None:
            continue
        if piv != r:
            a[r], a[piv] = a[piv], a[r]
            sign = -sign
        for i in range(r + 1, rows):
            for j in range(col + 1, cols):
                a[i][j] = a[i][j] * a[r][col] - a[i][col] * a[r][j]
                assert a[i][j] % prev == 0  # the division is exact
                a[i][j] //= prev
            a[i][col] = 0
        prev = a[r][col]
        r += 1
    det = sign * a[rows - 1][rows - 1] if rows == cols and r == rows else 0
    return det, r
```

listing 1109: python, the function with the exactness assert inline (source: dsa/samples-py/src/Ch34/determinant.py, lines 17-41)

```lua
local function bareiss(mat)
  local a = {}
  for i = 1, #mat do a[i] = {table.unpack(mat[i])} end
  local n = #a
  local m = #a[1]
  local sign, prev = 1, 1
  local r = 0 -- pivots placed; the next candidate row is r + 1
  for c = 1, m do
    local piv = nil
    for i = r + 1, n do
      if a[i][c] ~= 0 then piv = i break end
    end
    if piv ~= nil then
      local pr = r + 1
      if piv ~= pr then
        a[pr], a[piv] = a[piv], a[pr]
        sign = -sign
      end
      for i = pr + 1, n do
        for j = c + 1, m do
          a[i][j] = a[i][j] * a[pr][c] - a[i][c] * a[pr][j]
          if a[i][j] % prev ~= 0 then error("division not exact") end
          a[i][j] = a[i][j] // prev
        end
        a[i][c] = 0
      end
      prev = a[pr][c]
      r = r + 1
    end
  end
  local det = (n == m and r == n) and sign * a[n][n] or 0
  return det, r
end
```

listing 1110: lua, the same function, error thrown if the division goes inexact (source: dsa/samples-lua/ch34_determinant.lua, lines 8-40)

The anchors open small, det of 1, 2, 3, 4 is −2 at rank 2 and the identity gives 1, then the singular family: 1, 2, 3 by 4, 5, 6 by 7, 8, 9 has determinant 0 at rank 2, the doubled row ranks 1, the zero matrix ranks 0, and the diagonal 2, 3, 4 gives 24. The 4x4 pair carries the real weight, det 3 and det 12, with their product matrix pinning det(A B) = det(A) det(B) = 36 and every 4x4 answer re-derived by permutation-sign expansion in each suite. The sphinx family rides the same core: the five question rows 1, 0, 0 and 0, 1, 0 and 0, 0, 1 and 1, 1, 1 and 1, 2, 3 give ten triple determinants reading −2, −2, −1, 1, 1, 1, 1, 1, 1, 3 sorted, all nonzero, so every choice of three questions is an independent system. Nine of the ten are plus or minus 1 or 2, and the triple holding the first, second, and fifth rows gives 3, one more than the icpc problem statement's parenthetical claims.

The application is icpc world finals 2023 problem A (book 10, chapter 12), riddle of the sphinx, whose solvability after any single lie is exactly this argument: whichever answer is false, the surviving three rows stay independent, so the system always solves.

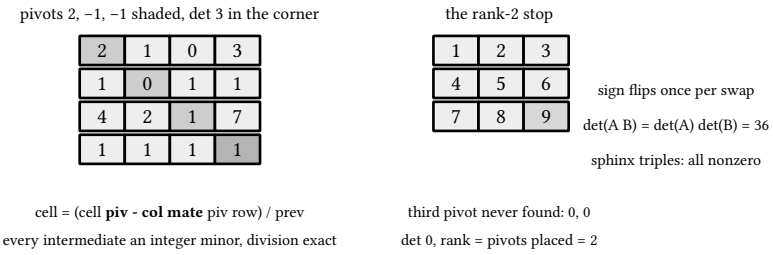


figure 283: the 4x4 bareiss tableau with the exact-division minors shaded and det 3 landing in the corner cell, the zero-pivot rank-2 example beside it

34.3. cramer on the small systems

When the system is 2x2 or 3x3, determinants solve it directly: component i of the solution is the determinant of A with column i replaced by b, divided by det(A). Cramer's rule is not the tool for large systems, O(n!) if generalized naively, but at these sizes the expansions are three products each and the answer stays exact in integers. The cross-language value is the reduced fraction pair: each component travels as a numerator and denominator reduced by their gcd with the denominator kept positive, so a negative determinant flips both signs and all seven languages assert identical pairs. det 0 reports no unique solution.

The dry run: the fixture is the 2x2 system 2, 3, 4, 5 against 7, 9, whose determinant −2 and answers −4 and 5 the C# suite asserts as reduced pairs, identical in the other six.

1. The denominator comes first: det(A) = 2 × 5 - 3 × 4 = −2.
2. Replacing column 0 by b reads 7, 3, 9, 5: det = 7 × 5 - 3 × 9 = 8, so x1 = 8 / −2 = −4, the negative determinant flipping the sign.
3. Replacing column 1 reads 2, 7, 4, 9: det = 2 × 9 - 7 × 4 = −10, so x2 = −10 / −2 = 5.
4. Both answers travel as pairs over a positive denominator, −4 over 1 and 5 over 1, the cross-language shape.
5. The diagonal 2, 0, 0, 4 against 1, 1 goes to fractions the same way: det 2 × 4 = 8, the swapped columns read 4 and 2, and the answers land at 1 over 2 and 1 over 4.
6. The singular pair 1, 2, 2, 4 reads det 1 × 4 - 2 × 2 = 0 and reports no unique solution.

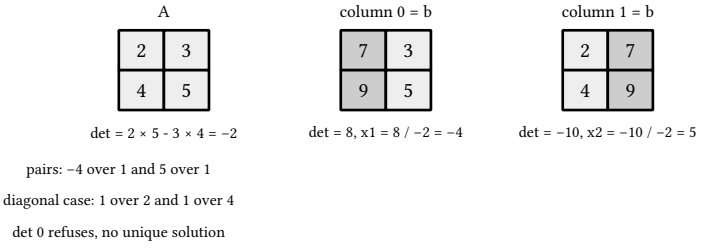


figure 284: the 2x2 fixture with each column swapped for b in turn, the replaced column shaded, each determinant and division written underneath

The −4 and the 5 land as pinned, and the listings below solve both sizes in seven languages.

```
static int det2(const int a[2][2]) {
  return a[0][0] * a[1][1] - a[0][1] * a[1][0];
}

static int det3(const int a[3][3]) {
  return a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1]) -
         a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0]) +
         a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0]);
}

// cramer on 2x2 and 3x3; 1 solved, 0 no unique solution
static int cramer(const int *a, int size, const int *b, Frac *x) {
  int d;
  if (size == 2) {
    int m[2][2] = {{a[0], a[1]}, {a[2], a[3]}};
    d = det2(m);
    if (d == 0)
      return 0;
    x[0] = frac((long long)b[0] * m[1][1] - (long long)m[0][1] * b[1], d);
    x[1] = frac((long long)m[0][0] * b[1] - (long long)b[0] * m[1][0], d);
    return 1;
  }
  int m[3][3];
  for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
      m[i][j] = a[i * 3 + j];
  d = det3(m);
  if (d == 0)
    return 0;
  for (int col = 0; col < 3; col++) {
    int r[3][3];
    for (int i = 0; i < 3; i++) {
      for (int j = 0; j < 3; j++)
        r[i][j] = m[i][j];
      r[i][col] = b[i];
    }
    x[col] = frac(det3(r), d);
  }
  return 1;
}
```

listing 1111: c, the 2x2 and 3x3 minors, the fraction assembly with the gcd reduce (source: dsa/samples-c/src/Ch34/cramer.c, lines 50-89)

```
func reduceFraction(num, den int64) [2]int64 {
  if den < 0 {
    num, den = -num, -den
  }
  g := gcdInt(absInt(num), den)
  if g == 0 {
    g = 1
  }
  return [2]int64{num / g, den / g}
}

// Cramer solves 2x2 and 3x3 systems by determinant ratios carried as
// reduced fraction pairs: component i is det(A with column i
// replaced by b) over det(A). det 0 reports no unique solution.
func Cramer(a [][]int64, b []int64) (int64, [][2]int64, bool) {
  n := len(a)
  var det int64
  var replaceDets []int64
  if n == 2 {
    det = a[0][0]*a[1][1] - a[0][1]*a[1][0]
    replaceDets = []int64{
      b[0]*a[1][1] - a[0][1]*b[1],
      a[0][0]*b[1] - b[0]*a[1][0],
    }
  } else {
    det = detPerm(a)
    replaceDets = make([]int64, n)
    for col := 0; col < n; col++ {
      m := make([][]int64, n)
      for r := range a {
        m[r] = append([]int64(nil), a[r]...)
        m[r][col] = b[r]
      }
      replaceDets[col] = detPerm(m)
    }
  }
  if det == 0 {
    return 0, nil, false
  }
  x := make([][2]int64, n)
  for i := range x {
    x[i] = reduceFraction(replaceDets[i], det)
  }
  return det, x, true
}
```

listing 1112: go, the fraction pair reduce and the cramer driver over both sizes (source: dsa/samples-go/ch34/cramer.go, lines 20-64)

```
  static int det2(int[][] a) {
    return a[0][0] * a[1][1] - a[0][1] * a[1][0];
  }

  static int det3(int[][] a) {
    return a[0][0] * (a[1][1] * a[2][2] - a[1][2] * a[2][1])
        - a[0][1] * (a[1][0] * a[2][2] - a[1][2] * a[2][0])
        + a[0][2] * (a[1][0] * a[2][1] - a[1][1] * a[2][0]);
  }

  static Frac[] xOut = new Frac[3];

  // cramer on 2x2 and 3x3; true solved, false no unique solution
  static boolean cramer(int[] a, int size, int[] b) {
    int d;
    if (size == 2) {
      int[][] m = {{a[0], a[1]}, {a[2], a[3]}};
      d = det2(m);
      if (d == 0)
        return false;
      xOut[0] = frac((long) b[0] * m[1][1] - (long) m[0][1] * b[1], d);
      xOut[1] = frac((long) m[0][0] * b[1] - (long) b[0] * m[1][0], d);
      return true;
    }
    int[][] m = new int[3][3];
    for (int i = 0; i < 3; i++)
      for (int j = 0; j < 3; j++)
        m[i][j] = a[i * 3 + j];
    d = det3(m);
    if (d == 0)
      return false;
    for (int col = 0; col < 3; col++) {
      int[][] r = new int[3][3];
      for (int i = 0; i < 3; i++) {
        for (int j = 0; j < 3; j++)
          r[i][j] = m[i][j];
        r[i][col] = b[i];
      }
      xOut[col] = frac(det3(r), d);
    }
    return true;
  }

  // plug-back on fractions: sum a[i][j] * x[j] == b[i] exactly
```

listing 1113: java, det2, det3, and cramer emitting reduced fraction pairs over a Frac class (source: dsa/samples-java/src/Ch34/Cramer.java, lines 53-96)

```
    public static Solution? Solve(long[][] a, long[] b)
    {
        int n = a.Length;
        long det = n == 2 ? Det2(a) : Det3(a);
        if (det == 0)
            return null; // no unique solution
        var x = new (long, long)[n];
        for (int col = 0; col < n; col++)
        {
            long[][] replaced = a.Select(row => (long[])row.Clone()).ToArray();
            for (int i = 0; i < n; i++)
                replaced[i][col] = b[i];
            long num = n == 2 ? Det2(replaced) : Det3(replaced);
            x[col] = Reduce(num, det);
        }
        return new Solution(det, x);
    }

    private static (long, long) Reduce(long num, long den)
    {
        if (den < 0)
        {
            num = -num;
            den = -den; // keep the denominator positive
        }
        long g = NumTheory.Gcd(num, den);
        if (g == 0)
            g = 1;
        return (num / g, den / g);
    }

    private static long Det2(long[][] m) => m[0][0] * m[1][1] - m[0][1] * m[1][0];

    private static long Det3(long[][] m) =>
        m[0][0] * (m[1][1] * m[2][2] - m[1][2] * m[2][1])
        - m[0][1] * (m[1][0] * m[2][2] - m[1][2] * m[2][0])
        + m[0][2] * (m[1][0] * m[2][1] - m[1][1] * m[2][0]);
```

listing 1114: c#, solve, the reduce keeping the denominator positive, both minor expansions (source: dsa/samples/src/Ch34/Cramer.cs, lines 13-49)

```
const det2 = (A) => A[0][0] * A[1][1] - A[0][1] * A[1][0];

const det3 = (A) =>
  A[0][0] * (A[1][1] * A[2][2] - A[1][2] * A[2][1])
  - A[0][1] * (A[1][0] * A[2][2] - A[1][2] * A[2][0])
  + A[0][2] * (A[1][0] * A[2][1] - A[1][1] * A[2][0]);

export function cramer(A, b) {
  const n = A.length;
  const det = n === 2 ? det2(A) : det3(A);
  if (det === 0) return { det, x: null }; // no unique solution
  const x = [];
  for (let col = 0; col < n; col++) {
    const M = A.map((row, i) => row.map((v, j) => (j === col ? b[i] : v)));
    const d = n === 2 ? det2(M) : det3(M);
    x.push(frac(d, det));
  }
  return { det, x };
}
```

listing 1115: javascript, det2, det3, and the column replacement building each minor (source: dsa/samples-js/src/ch34-cramer.mjs, lines 27-45)

```
def det3(m):
    return (
        m[0][0] * (m[1][1] * m[2][2] - m[1][2] * m[2][1])
        - m[0][1] * (m[1][0] * m[2][2] - m[1][2] * m[2][0])
        + m[0][2] * (m[1][0] * m[2][1] - m[1][1] * m[2][0])
    )


def replace_col(m, col, b):
    out = [row[:] for row in m]
    for r in range(len(m)):
        out[r][col] = b[r]
    return out


def cramer(m, b):
    det = det2(m) if len(m) == 2 else det3(m)
    if det == 0:
        return 0, None  # no unique solution
    xs = []
    for col in range(len(m)):
        replaced = replace_col(m, col, b)
        d = det2(replaced) if len(m) == 2 else det3(replaced)
        xs.append(frac(d, det))
    return det, xs
```

listing 1116: python, det3, the column replacement, cramer emitting fraction pairs (source: dsa/samples-py/src/Ch34/cramer.py, lines 28-52)

```
local function det2(m)
  return m[1][1] * m[2][2] - m[1][2] * m[2][1]
end

local function det3(m)
  return m[1][1] * (m[2][2] * m[3][3] - m[2][3] * m[3][2])
       - m[1][2] * (m[2][1] * m[3][3] - m[2][3] * m[3][1])
       + m[1][3] * (m[2][1] * m[3][2] - m[2][2] * m[3][1])
end

local function cramer(A, b)
  local n = #A
  local d = (n == 2) and det2(A) or det3(A)
  if d == 0 then return 0, nil end -- no unique solution
  local xs = {}
  for col = 1, n do
    local M = {}
    for r = 1, n do
      M[r] = {table.unpack(A[r])}
      M[r][col] = b[r] -- replace column col by b
    end
    local dn = (n == 2) and det2(M) or det3(M)
    xs[col] = reduce(dn, d)
  end
  return d, xs
end
```

listing 1117: lua, the two expansions and the fraction reduce, 1-based columns (source: dsa/samples-lua/ch34_cramer.lua, lines 20-45)

The fixtures pin the four systems. The integer pair 2, 3, 4, 5 against 7, 9 has determinant −2 and solves to −4 and 5, the pair that shows the negative determinant flipping the numerator sign. The diagonal 2, 0, 0, 4 against 1, 1 has determinant 8 and solves to the fraction pairs 1 over 2 and 1 over 4, integers in, fractions out, with no float anywhere. The 3x3 with a zero leading column solves to 0, 2, 1 at determinant −1, and the sphinx-flavored system with rows 1, 1, 1, 1, 2, 3, 2, 3, 1 against 14, 33, 26 has determinant −3 and solves to 2, 5, 7, every suite plugging the fractions back into A x = b exactly. The singular pair 1, 2, 2, 4 against 3, 6 reads det 0 and refuses.

The application is the same icpc world finals 2023 problem A (book 10, chapter 12): once the lie is identified, the surviving 3x3 is solved exactly this way, and the answer's fractions stay honest because no division ever rounds.

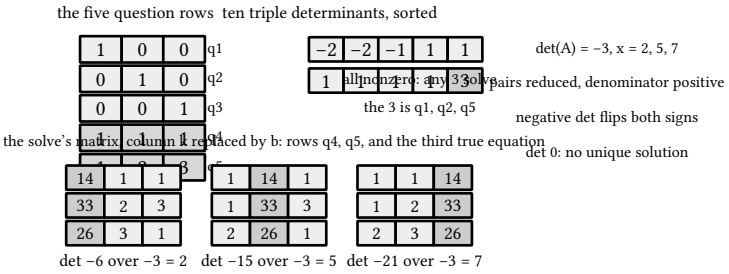


figure 285: the sphinx 5x3 question matrix with the ten triple determinants tallied beside it, the solve's 3x3 shaded, the three replaced-column matrices lined up

34.4. matrix exponentiation

Binary exponentiation lifts from scalars to matrices unchanged: seed the result with the identity, square the base per exponent bit, multiply into the result on the set bits, and with a modulus given, reduce every entry product mod p, O(n^3 log e). The scalar form is chapter 16's, book 9, ch 16 (practical data structures and algorithms, number theory and modular arithmetic), and only the associativity of matrix multiplication is new. Two applications ride

it. Fibonacci: the companion matrix 1, 1, 1, 0 raised to the n has fib(n) in its upper right entry, so one exponentiation replaces n additions, and the pins stay exact up to fib(78) = 8944394323791464, the largest value under the 2^53 ceiling that keeps javascript's Number honest, with fib(100) mod 1e9+7 = 687995182 and fib(10^18) mod 1e9+7 = 209783453 beyond it. Walk counting: the kth power of a digraph's adjacency matrix counts length-k walks, entry i, j holding the walks from i to j, and the sum of all entries counts them everywhere.

The dry run: the fixtures are the walk digraph 0 to 1, 0 to 2, 1 to 2, 2 to 0 with its square and fourth power pinned entry for entry by the C# suite, and fib(50) = 12586269025 off the companion matrix, identical in the other six.

1. The adjacency matrix reads A = 0, 1, 1 over 0, 0, 1 over 1, 0, 0, four 1s, one per directed edge.
2. Squaring folds walks through every middle vertex: A^2 lands 1, 0, 1 over 1, 0, 0 over 0, 1, 1, its five 1s exactly the five 2-walks 0-2-0, 0-1-2, 1-2-0, 2-0-1, 2-0-2, total 5.
3. The fourth power squares again: row 0 of A^2 against its own column 2 gives 1 × 1 + 1 × 1 = 2, the two 4-walks from 0 to 2.
4. A^4 lands 1, 1, 2 over 1, 0, 1 over 1, 1, 1 at total 9, both matrices asserted entry for entry.
5. The fib side splits the exponent, 50 = 32 + 16 + 2, and keeps F^2, F^16, F^32, whose product F^50 reads fib(50) = 12586269025 in the 0, 1 entry, F^2 itself the first squaring with 1 + 1 = 2 in the corner.

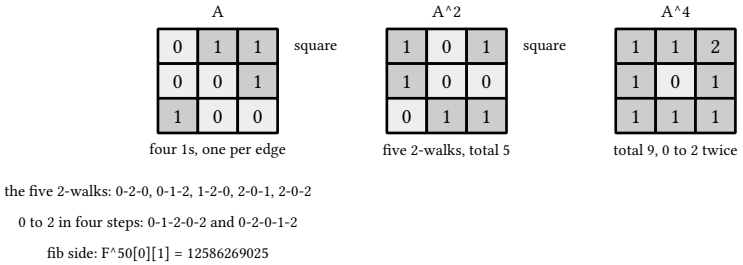


figure 286: the walk digraph's matrix squared twice, A then A^2 then A^4 as grids of walk counts, the totals rising 5 then 9, the two 0 to 2 four-walks spelled out

The 5, the 9, and the 12586269025 land as pinned, and the listings below raise matrices in seven languages.

```c
static void matmul(const Mat a, const Mat b, Mat c, unsigned long long mod,
                   int d) {
  static Mat t;
  memset(t, 0, sizeof(t));
  for (int i = 0; i < d; i++)
    for (int j = 0; j < d; j++) {
      unsigned long long s = 0;
      for (int k = 0; k < d; k++) {
        unsigned long long av = a[i][k], bv = b[k][j];
        if (mod) {
          s = (s + av % mod * (bv % mod)) % mod;
        } else {
          s += av * bv; // exact path, entries small enough
        }
      }
      t[i][j] = s;
    }
  memcpy(c, t, sizeof(t));
}

static void matpow(const Mat a, unsigned long long e, unsigned long long mod,
                   int d, Mat out) {
  Mat r, base;
  for (int i = 0; i < d; i++)
    for (int j = 0; j < d; j++) {
      r[i][j] = i == j;
      base[i][j] = a[i][j];
    }
  while (e) {
    if (e & 1)
      matmul(r, base, r, mod, d);
    matmul(base, base, base, mod, d);
    e >>= 1;
  }
  memcpy(out, r, sizeof(r));
}
```

listing 1118: c, the triple-loop multiply with its optional mod, the square-and-multiply power (source: dsa/samples-c/src/Ch34/matpow.c, lines 27-62)

```
func MatMul(a, b [][]int64, mod int64) [][]int64 {
  n := len(a)
  c := make([][]int64, n)
  for i := range c {
    c[i] = make([]int64, n)
    for j := range c[i] {
      var sum int64
      for k := 0; k < n; k++ {
        sum += a[i][k] * b[k][j]
        if mod > 0 {
          sum %= mod
        }
      }
      c[i][j] = sum
    }
  }
  return c
}

// MatPow raises a square matrix to the exponent by the square-and-
// multiply loop over the exponent bits from the identity. mod 0
// keeps exact integers.
func MatPow(a [][]int64, exp int64, mod int64) [][]int64 {
  n := len(a)
  result := make([][]int64, n)
  for i := range result {
    result[i] = make([]int64, n)
    result[i][i] = 1
  }
  base := make([][]int64, n)
  for i := range base {
    base[i] = append([]int64(nil), a[i]...)
  }
  for exp > 0 {
    if exp&1 == 1 {
      result = MatMul(result, base, mod)
    }
    exp >>= 1
    if exp > 0 { // skip the squaring past the top bit
      base = MatMul(base, base, mod)
    }
  }
  return result
}
```

listing 1119: go, matmul and matpow, the top-bit squaring skip spelled out (source: dsa/samples-go/ch34/matpow.go, lines 5-48)

```
static void matmul(long[][] a, long[][] b, long[][] c, long mod, int d) {
  long[][] t = new long[D][D];
  for (int i = 0; i < d; i++)
    for (int j = 0; j < d; j++) {
      long s = 0;
      for (int k = 0; k < d; k++) {
        long av = a[i][k], bv = b[k][j];
        if (mod != 0)
          s = (s + av % mod * (bv % mod)) % mod;
        else
          s += av * bv; // exact path, entries small enough
      }
      t[i][j] = s;
    }
  for (int i = 0; i < d; i++)
    System.arraycopy(t[i], 0, c[i], 0, d);
}

static void matpow(long[][] a, long e, long mod, int d, long[][] out) {
  long[][] r = new long[D][D], base = new long[D][D];
  for (int i = 0; i < d; i++)
    for (int j = 0; j < d; j++) {
      r[i][j] = i == j ? 1 : 0;
      base[i][j] = a[i][j];
    }
  while (e != 0) {
    if ((e & 1) != 0)
      matmul(r, base, r, mod, d);
    matmul(base, base, base, mod, d);
    e >>= 1;
  }
  for (int i = 0; i < d; i++)
    System.arraycopy(r[i], 0, out[i], 0, d);
}
```

listing 1120: java, matmul with its optional mod, the square-and-multiply power (source: dsa/samples-java/src/Ch34/Matpow.java, lines 23-57)

```
    public static long[][] Pow(long[][] m, long e, long? mod = null)
    {
        int n = m.Length;
        var result = new long[n][];
        var b = new long[n][];
        for (int i = 0; i < n; i++)
        {
            result[i] = new long[n];
            b[i] = (long[])m[i].Clone();
            result[i][i] = 1;
        }
        while (e > 0)
        {
            if ((e & 1) == 1)
                result = Mul(result, b, mod);
            e >>= 1;
            if (e > 0)
                b = Mul(b, b, mod); // square per remaining bit, none wasted past the top
        }
        return result;
    }

    private static long[][] Mul(long[][] x, long[][] y, long? mod)
    {
        int n = x.Length;
        var c = new long[n][];
        for (int i = 0; i < n; i++)
        {
            c[i] = new long[n];
            for (int j = 0; j < n; j++)
            {
                long s = 0;
                for (int k = 0; k < n; k++)
                    s += x[i][k] * y[k][j];
                c[i][j] = mod is long m ? s % m : s;
            }
        }
        return c;
    }
```

listing 1121: c#, pow skipping the last squaring, the multiply reducing per entry (source: dsa/samples/src/Ch34/Matpow.cs, lines 11-49)

```
function mulNumber(A, B) {
  const n = A.length;
  const C = Array.from({ length: n }, () => new Array(n).fill(0));
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      let s = 0;
      for (let k = 0; k < n; k++) s += A[i][k] * B[k][j];
      C[i][j] = s;
    }
  }
  return C;
}

function mulMod(A, B) {
  const n = A.length;
  const C = Array.from({ length: n }, () => new Array(n).fill(0n));
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      let s = 0n;
      for (let k = 0; k < n; k++) {
        // residue products reach ~1e18 past 2^53: BigInt mandatory
        s += BigInt(A[i][k]) * BigInt(B[k][j]);
      }
      C[i][j] = s % P;
    }
  }
  return C;
}

export function matpow(A, e, mod = null) {
  const n = A.length;
  const useMod = mod !== null;
  let R = Array.from({ length: n }, (_, i) =>
    Array.from({ length: n }, (_, j) =>
      (useMod ? BigInt(i === j ? 1 : 0) : i === j ? 1 : 0)));
  let base = A.map((row) => row.map((v) => (useMod ? BigInt(v) : v)));
  let ee = typeof e === "bigint" ? e : BigInt(e);
  while (ee > 0n) {
    if (ee & 1n) R = useMod ? mulMod(R, base) : mulNumber(R, base);
    ee >>= 1n;
    if (ee > 0n) {
      base = useMod ? mulMod(base, base) : mulNumber(base, base);
    }
  }
  return R;
}
```

listing 1122: javascript, the number and bigint multiplies, the same power loop over both (source: dsa/samples-js/src/ch34-matpow.mjs, lines 13-58)

```
def matmul(a, b, mod=None):
    rows, inner, cols = len(a), len(b), len(b[0])
    out = [[0] * cols for _ in range(rows)]
    for i in range(rows):
        for j in range(cols):
            s = 0
            for t in range(inner):
                s += a[i][t] * b[t][j]
            out[i][j] = s % mod if mod else s
    return out


def matpow(base, e, mod=None):
    size = len(base)
    out = [[1 if i == j else 0 for j in range(size)] for i in range(size)]
    base = [row[:] for row in base]
    while e:  # square and multiply over the exponent bits
        if e & 1:
            out = matmul(out, base, mod)
        base = matmul(base, base, mod)
        e >>= 1
    return out
```

listing 1123: python, matmul with an optional mod, matpow over the exponent bits (source: dsa/samples-py/src/Ch34/matpow.py, lines 18-39)

```lua
local function matmul(A, B, mod)
  local n, k, m = #A, #B, #B[1]
  local C = {}
  for i = 1, n do
    C[i] = {}
    for j = 1, m do
      local s = 0
      for t = 1, k do
        local v = A[i][t] * B[t][j]
        s = mod and (s + v) % mod or s + v
      end
      C[i][j] = s
    end
  end
  return C
end

local function identity(n)
  local R = {}
  for i = 1, n do
    R[i] = {}
    for j = 1, n do R[i][j] = (i == j) and 1 or 0 end
  end
  return R
end

local function matpow(A, e, mod)
  local n = #A
  local R = identity(n)
  local base = {}
  for i = 1, n do base[i] = {table.unpack(A[i])} end
  while e > 0 do
    if e & 1 == 1 then R = matmul(R, base, mod) end
    base = matmul(base, base, mod)
    e = e >> 1
  end
  return R
end
```

listing 1124: lua, the multiply, the identity seed, the power loop (source: dsa/samples-lua/ch34_matpow.lua, lines 10-47)

The fixtures pin both applications. The identity anchors read fib(0) through fib(3) as 0, 1, 1, 2 off F^0 and F^1 = F, the exact ladder pins fib(10) = 55, fib(50) = 12586269025, and fib(78) = 8944394323791464 with every value re-derived by plain iterative addition, and the modular pair pins the two residues above. The walk graph runs four edges, 0 to 1, 0 to 2, 1 to 2, and 2 to 0: its square is pinned entry for entry with 5 total 2-walks, and its fourth power holds 2 walks from 0 to 2 with 9 walks total, the modular road matching the exact one on the small entries.

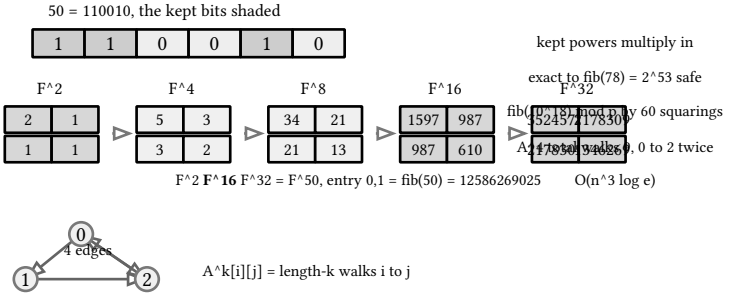


figure 287: the fib 2x2 squaring chain against the bits of 50, the kept powers multiplied in, and the walk graph with its A^4 total

34.5. kirchhoff's matrix-tree count

The spanning trees of an undirected multigraph are counted by a determinant. Build the laplacian, the degree of each vertex on the diagonal and minus the edge multiplicity off it, self loops dropped because they join nothing, delete any one row and the same column, and the determinant of the minor is the tree count. The proof is an elimination argument on the incidence matrix, and the tool here is the bareiss core from the determinant section carried in file, the house standalone pattern, so every language answers from its own small integer determinant.

The dry run: the fixture is the triangle with edge 0-1 doubled, whose count 5 the C# suite asserts against brute enumeration, the plain triangle's 3 alongside, identical in the other six.

1. The doubled edge gives degrees 3, 3, 2, so the laplacian reads 3, −2, −1 over −2, 3, −1 over −1, −1, 2, the doubled edge visible as the off-diagonal −2.
2. Deleting row and column 2 leaves the minor 3, −2 over −2, 3.
3. Its determinant is 3 × 3 - (-2) × (-2) = 9 - 4 = 5, the tree count.
4. The 5 trees are visible in the edge set: each doubled copy pairs with 1-2, each pairs with 2-0, and 1-2 pairs with 2-0, a 2 + 2 + 1 = 5 that matches the determinant.
5. The plain triangle reads the same way, minor 2, −1 over −1, 2 at 4 - 1 = 3.

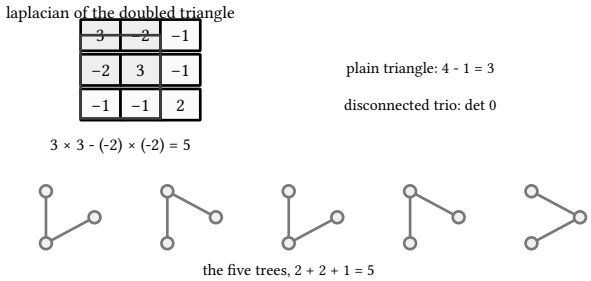


figure 288: the laplacian of the doubled-edge triangle with its framed 2x2 minor at 5, the five spanning trees drawn underneath

The 5 and the 3 land as pinned, and the listings below count spanning trees in seven languages.

```c
// edges as (u, v) pairs, self loops ignored
static long long spanning_trees(int nv, const int edges[][2], int ec) {
  int L[NV][NV];
  memset(L, 0, sizeof(L));
  for (int e = 0; e < ec; e++) {
    int u = edges[e][0], v = edges[e][1];
    if (u == v)
      continue;
    L[u][u]++;
    L[v][v]++;
    L[u][v]--;
    L[v][u]--;
  }
  int minor[NV][NV];
  memset(minor, 0, sizeof(minor));
  for (int i = 0; i < nv - 1; i++)
    for (int j = 0; j < nv - 1; j++)
      minor[i][j] = L[i][j];
  return det_bareiss(minor, nv - 1);
}
```

listing 1125: c, the laplacian assembly and the minor feeding the in-file bareiss det (source: dsa/samples-c/src/Ch34/kirchhoff.c, lines 59-78)

```
func Kirchhoff(n int, edges [][2]int) int64 {
  l := make([][]int64, n)
  for i := range l {
    l[i] = make([]int64, n)
  }
  for _, e := range edges {
    u, v := e[0], e[1]
    if u == v {
      continue
    }
    l[u][u]++
    l[v][v]++
    l[u][v]--
    l[v][u]--
  }
  if n <= 1 {
    return 0
  }
  minor := make([][]int64, n-1)
  for i := range minor {
    minor[i] = l[i][:n-1]
  }
  det, _ := Bareiss(minor)
  return det
}
```

listing 1126: go, the assembly over the edge list, bareiss below (source: dsa/samples-go/ch34/kirchhoff.go, lines 7-31)

```
  static long spanningTrees(int nv, int[][] edges, int ec) {
    int[][] L = new int[NV][NV];
    for (int e = 0; e < ec; e++) {
      int u = edges[e][0], v = edges[e][1];
      if (u == v)
        continue;
      L[u][u]++;
      L[v][v]++;
      L[u][v]--;
      L[v][u]--;
    }
    int[][] minor = new int[NV][NV];
    for (int i = 0; i < nv - 1; i++)
      for (int j = 0; j < nv - 1; j++)
        minor[i][j] = L[i][j];
    return detBareiss(minor, nv - 1);
  }

  // brute: subsets of size n-1 that connect all vertices (union-find);
```

listing 1127: java, the laplacian assembly and the minor feeding the in-file bareiss det (source: dsa/samples-java/src/Ch34/Kirchhoff.java, lines 59-77)

```
    public static long SpanningTrees(int n, (int U, int V)[] edges)
    {
        var lap = new long[n][];
        for (int i = 0; i < n; i++)
            lap[i] = new long[n];
        foreach ((int u, int v) in edges)
        {
            if (u == v)
                continue; // self loops dropped
            lap[u][u]++;
            lap[v][v]++;
            lap[u][v]--;
            lap[v][u]--;
        }
        var minor = new long[n - 1][];
        for (int i = 0; i < n - 1; i++)
        {
            minor[i] = new long[n - 1];
            Array.Copy(lap[i], minor[i], n - 1);
        }
        return Determinant.DetRank(minor).Det;
    }
```

listing 1128: c#, the assembly calling the chapter's determinant class (source: dsa/samples/src/Ch34/Kirchhoff.cs, lines 10-31)

```
function detInt(a) {
  const n = a.length;
  let sign = 1;
  let prev = 1;
  for (let c = 0; c < n; c++) {
    let piv = -1;
    for (let i = c; i < n; i++) {
      if (a[i][c] !== 0) {
        piv = i;
        break;
      }
    }
    if (piv === -1) return 0;
    if (piv !== c) {
      [a[c], a[piv]] = [a[piv], a[c]];
      sign = -sign;
    }
    for (let i = c + 1; i < n; i++) {
      for (let j = c + 1; j < n; j++) {
        a[i][j] = (a[i][j] * a[c][c] - a[i][c] * a[c][j]) / prev;
      }
      a[i][c] = 0;
    }
    prev = a[c][c];
  }
  return sign * a[n - 1][n - 1];
}

export function kirchhoff(n, edges) {
  const L = Array.from({ length: n }, () => new Array(n).fill(0));
  for (const [u, v] of edges) {
    if (u === v) continue; // self loops drop out
    L[u][u] += 1;
    L[v][v] += 1;
    L[u][v] -= 1;
    L[v][u] -= 1;
  }
  const M = L.slice(0, n - 1).map((row) => row.slice(0, n - 1));
  return detInt(M);
}
```

listing 1129: javascript, the in-file integer det and the kirchhoff driver over it (source: dsa/samples-js/src/ch34-kirchhoff.mjs, lines 8-47)

```
def bareiss_det(mat):
    # the same fraction-free core as 34.2, self-contained here
    a = [row[:] for row in mat]
    size = len(a)
    sign = 1
    prev = 1
    r = 0
    for col in range(size):
        piv = next((i for i in range(r, size) if a[i][col] != 0), None)
        if piv is None:
            return 0  # singular laplacian minor: disconnected graph
        if piv != r:
            a[r], a[piv] = a[piv], a[r]
            sign = -sign
        for i in range(r + 1, size):
            for j in range(col + 1, size):
                a[i][j] = a[i][j] * a[r][col] - a[i][col] * a[r][j]
                assert a[i][j] % prev == 0
                a[i][j] //= prev
            a[i][col] = 0
        prev = a[r][col]
        r += 1
    return sign * a[size - 1][size - 1]


def spanning_trees(count, edges):
    lap = [[0] * count for _ in range(count)]
    for u, v in edges:
        if u == v:  # self loops dropped
            continue
        lap[u][u] += 1
        lap[v][v] += 1
        lap[u][v] -= 1
        lap[v][u] -= 1
    minor = [row[: count - 1] for row in lap[: count - 1]]
    return bareiss_det(minor)
```

listing 1130: python, the bareiss core and the assembly, singular minor meaning disconnected (source: dsa/samples-py/src/Ch34/kirchhoff.py, lines 17-52)

```
local function spanning_trees(n, edges)
  local L = {}
  for i = 1, n do
    L[i] = {}
    for j = 1, n do L[i][j] = 0 end
  end
  for _, e in ipairs(edges) do
    local u, v = e[1] + 1, e[2] + 1 -- 0-based fixtures, 1-based tables
    if u ~= v then -- self loops dropped
      L[u][u] = L[u][u] + 1
      L[v][v] = L[v][v] + 1
      L[u][v] = L[u][v] - 1
      L[v][u] = L[v][u] - 1
    end
  end
  local M = {}
  for i = 1, n - 1 do
    M[i] = {}
    for j = 1, n - 1 do M[i][j] = L[i][j] end
  end
  return bareiss_det(M)
end
```

listing 1131: lua, the assembly over 0-based edge fixtures into 1-based tables (source: dsa/samples-lua/ch34_kirchhoff.lua, lines 41-62)

The fixtures are brute-verified in every suite, union-find enumerating all size n - 1 edge subsets that connect everything. The basics: the triangle K3 counts 3, the path P4 counts 1, and the 4-cycle C4 counts 4. Cayley's formula lands as the check, K4 counting 16 = 4^2. The multigraph family is the honest part: a triangle with edge 0-1 doubled counts 5, both parallel edges counted while the 2-cycle they form alone is rejected, the bowtie, two triangles sharing vertex 0, counts 3 times 3 = 9, and the disconnected trio counts 0, the singular minor reading exactly that. The counts stay small exact integers identical across all seven languages, O(n^3):

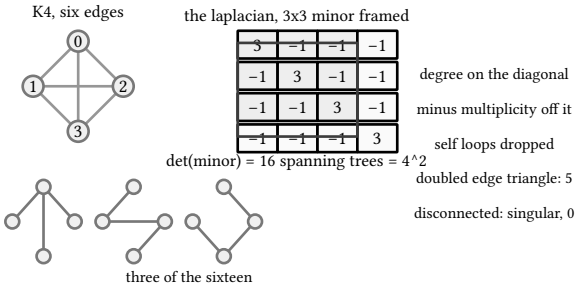


figure 289: K4 with its laplacian grid, the 3x3 minor framed, det 16 annotated as the tree count with three of the sixteen trees drawn

34.6. hensel lifting over prime powers

Linear algebra over Z p ^k^, p prime, solves systems the plain field cannot reach, and the tool is newton's iteration wearing number theory. Solve A x = b mod p with the elimination core, which needs A invertible there. Then lift one power at a time: compute the residual r = b - A x mod p^(m+1), which is exactly divisible by p^m because x already solves mod p^m, divide it down, solve A t = r over p^m mod p, and add p^m t to x. Each step buys one base-p digit of the solution, quadratic convergence in the p-adic metric, and an exact integer solution simply stops moving.

The dry run: the fixture is 2, 1, 1, 1 against 14, 13 over p = 3, whose climb 1, 0 then 1, 3 then 1, 12 the C# suite asserts with its mod 27 plug-back, identical in the other six.

1. Mod 3 the system reads 2x + y = 2 and x + y = 1, since 14 and 13 reduce to 2 and 1: subtracting gives x = 1, then y = 0.

2. The residual is b - A x = 14 - 2, 13 - 1 = 12, 12, divisible by 3 exactly, and divided down reads 4, 4, which is 1, 1 mod 3.
3. The correction solves A t = 1, 1 mod 3 to t = 0, 1, so x climbs by 3 × (0, 1) to 1, 3, solving mod 9.
4. The next residual is 14 - 5, 13 - 4 = 9, 9, divisible by 9, and 1, 1 mod 3 gives t = 0, 1 again: x += 9 × (0, 1) lands at 1, 12, solving mod 27.
5. The plug-back is the suite's own assert: 2 × 1 + 12 = 14 and 1 + 12 = 13, both 0 mod 27.
6. Against 5, 4 the same ladder stops early, 1, 0 then 1, 3 then 1, 3: the residual hits 0, 0 and the exact solution stands still.

solves mod	x	residual b - A x	correction
3	1, 0	12, 12	divide by 3, t = 0, 1
9	1, 3	9, 9	divide by 9, t = 0, 1
27	1, 12	0, 0	exact, stands still

The 1, 12 closes the climb as pinned, and the listings below lift systems in seven languages.

```c
static void hensel(const unsigned long long a[2][2],
                   const unsigned long long *b, int k,
                   unsigned long long x[2][2]) {
  unsigned long long cur[2];
  solve2(a, b, P3, cur);
  x[0][0] = cur[0];
  x[0][1] = cur[1];
  unsigned long long pk = P3;
  for (int s = 1; s < k; s++) {
    pk *= P3;
    unsigned long long res[2];
    for (int i = 0; i < 2; i++) {
      unsigned long long ax = 0;
      for (int j = 0; j < 2; j++)
        ax = (ax + a[i][j] * cur[j]) % pk;
      res[i] = (b[i] + pk - ax) % pk;
    }
    unsigned long long rhs[2], t[2];
    for (int i = 0; i < 2; i++)
      rhs[i] = res[i] / (pk / P3) % P3;
    solve2(a, rhs, P3, t);
    for (int i = 0; i < 2; i++)
      cur[i] = (cur[i] + (pk / P3) * t[i]) % pk;
    x[s][0] = cur[0];
    x[s][1] = cur[1];
  }
}
```

listing 1132: c, the lift loop, residual, divided rhs, correction solve (source: dsa/samples-c/src/Ch34/zpk.c, lines 83-109)

```
func Hensel(a [][]int64, b []int64, p, k int64) ([][]int64, error) {
  status, x, err := GaussMod(a, b, p)
  if err ≠ nil {
    return nil, err
  }
  if status ≠ Unique {
    return nil, fmt.Errorf("hensel: no unique solution mod %d", p)
  }
  n := len(a)
  steps := [][]int64{append([]int64(nil), x...)}
  pk := p
  for step := int64(1); step < k; step++ {
    pk *= p
    res := make([]int64, n)
    for i := range a {
      sum := int64(0)
      for j := range a[i] {
        sum += a[i][j] * x[j]
      }
      res[i] = (((b[i] - sum) % pk) + pk) % pk
    }
    rhs := make([]int64, n)
    for i := range res {
      if res[i]%(pk/p) ≠ 0 {
        return nil, fmt.Errorf(
          "residual not divisible by %d", pk/p)
      }
      rhs[i] = (res[i] / (pk / p)) % p
    }
    st, t, err := GaussMod(a, rhs, p)
    if err ≠ nil || st ≠ Unique {
      return nil, fmt.Errorf("lift step failed: %v %v", st, err)
    }
    for i := range x {
      x[i] = (x[i] + (pk/p)*t[i]) % pk
    }
    steps = append(steps, append([]int64(nil), x...))
  }
  return steps, nil
}
```

listing 1133: go, the loop with the divisibility check made loud (source: dsa/samples-go/ch34/zpk.go, lines 10-49)

```
  static void hensel(long[][] a, long[] b, int k, long[][] x) {
    long[] cur = new long[2];
    solve2(a, b, P3);
    cur[0] = x2[0];
    cur[1] = x2[1];
    x[0][0] = cur[0];
    x[0][1] = cur[1];
    long pk = P3;
    for (int s = 1; s < k; s++) {
      pk *= P3;
      long[] res = new long[2];
      for (int i = 0; i < 2; i++) {
        long ax = 0;
        for (int j = 0; j < 2; j++)
          ax = (ax + a[i][j] * cur[j]) % pk;
        res[i] = (b[i] + pk - ax) % pk;
      }
      long[] rhs = new long[2], t = new long[2];
      for (int i = 0; i < 2; i++)
        rhs[i] = res[i] / (pk / P3) % P3;
      solve2(a, rhs, P3);
      t[0] = x2[0];
      t[1] = x2[1];
      for (int i = 0; i < 2; i++)
        cur[i] = (cur[i] + (pk / P3) * t[i]) % pk;
      x[s][0] = cur[0];
      x[s][1] = cur[1];
    }
  }
```

listing 1134: java, the lift loop, residual, divided rhs, correction solve (source: dsa/samples-java/src/Ch34/Zpk.java, lines 82-111)

```
    public static long[][] HenselLift(long[][] a, long[] b, long p, int k)
    {
        Gauss.Result first = Gauss.Solve(a, b, p);
        if (first.Status != Gauss.Status.Unique)
            throw new InvalidOperationException("hensel lifting needs an invertible mod-p system");
        var steps = new List<long[]> { first.X! };
        long[] x = (long[])first.X!.Clone();
        long pk = p;
        int n = a.Length;
        for (int step = 1; step < k; step++)
        {
            pk *= p;
            var residual = new long[n];
            for (int i = 0; i < n; i++)
            {
                long ax = 0;
                for (int j = 0; j < n; j++)
                    ax += a[i][j] * x[j];
                residual[i] = ((b[i] - ax) % pk + pk) % pk; // divisible by p^(m)
            }
            var rhs = new long[n];
            for (int i = 0; i < n; i++)
                rhs[i] = residual[i] / (pk / p) % p;
            Gauss.Result corr = Gauss.Solve(a, rhs, p);
            for (int i = 0; i < n; i++)
                x[i] = (x[i] + (pk / p) * corr.X![i]) % pk;
            steps.Add((long[])x.Clone());
        }
        return steps.ToArray();
    }
```

listing 1135: c#, the same loop over the gauss class, steps collected (source: dsa/samples/src/Ch34/Zpk.cs, lines 11-40)

```
export function hensel(A, b, p, k) {
  let x = gaussSmall(A, b, p);
  const steps = [[p, [...x]]];
  let pk = p;
  for (let step = 1; step < k; step++) {
    pk *= p;
    const res = A.map((row, i) => {
      let s = 0;
      for (let j = 0; j < row.length; j++) s += row[j] * x[j];
      return (((b[i] - s) % pk) + pk) % pk;
    });
    const rhs = res.map((v) => Math.floor(v / (pk / p)) % p);
    const t = gaussSmall(A, rhs, p);
    x = x.map((v, i) => (v + (pk / p) * t[i]) % pk);
    steps.push([pk, [...x]]);
  }
  return steps;
}
```

listing 1136: javascript, the lift over the small in-file solver, plain Number exact at p = 3 (source: dsa/samples-js/src/ch34-zpk.mjs, lines 50-67)

```
def hensel(a, b, k):
    # solve a x = b mod p exactly, then lift one power per step
    p = P3
    x = gauss_mod3(a, b)
    steps = [x[:]]
    pk = p
    for _ in range(k - 1):
        pk *= p
        res = [
            (b[i] - sum(a[i][j] * x[j] for j in range(len(a)))) % pk
            for i in range(len(a))
        ]
        rhs = [res[i] // (pk // p) % p for i in range(len(a))]
        t = gauss_mod3(a, rhs)
        x = [(x[i] + (pk // p) * t[i]) % pk for i in range(len(a))]
        steps.append(x[:])
    return steps
```

listing 1137: python, the loop with the mod-3 gauss core above it (source: dsa/samples-py/src/Ch34/zpk.py, lines 35-51)

```
local function hensel(A, b, p, k)
  local pk = p
  local x = gauss_mod(A, b, p)
  assert(x ~= nil, "A must be invertible mod p")
  local steps = {{x[1], x[2]}}
  for _ = 1, k - 1 do
    pk = pk * p
    local res = {}
    for i = 1, #A do
      local s = b[i] % pk
      for j = 1, #A do
        s = s - A[i][j] * x[j] % pk
      end
      res[i] = (s % pk + pk) % pk
    end
    local rhs = {}
    for i = 1, #A do rhs[i] = res[i] // (pk // p) % p end
    local t = gauss_mod(A, rhs, p)
    for i = 1, #A do x[i] = (x[i] + (pk // p) * t[i]) % pk end
    steps[#steps + 1] = {x[1], x[2]}
  end
  return steps
end
```

listing 1138: lua, the lift with the in-file gauss riding along (source: dsa/samples-lua/ch34_zpk.lua, lines 69-91)

The second half of the file is the 2022 world finals Z machinery in miniature. Cyclic convolution of count arrays over Z p ^k^ is the multiset-of-sums operator, out at i plus j mod m accumulating a[i] times b[j], and iterating it over a list of levers counts subset sums: each lever doubles the table into itself shifted by its own value. The one-dimensional valuation lemma reads the answer's arithmetic structure: the number of zero levers is the minimum 2-adic valuation of the projected residue counts.

The fixtures run at p = 3. The matrix 2, 1, 1, 1 against 5, 4 lifts 1, 0 then 1, 3 then 1, 3, the exact solution reached and standing still, each step's plug-back asserted mod 27. The same matrix against 14, 13 climbs 1, 0 then 1, 3 then 1, 12, one base-3 digit per step. The convolution family pins the two levers: a zero lever doubles every residue count, 2, 0, 0 against 1, 1, 1 giving 2, 2, 2, while a nonzero lever permutes the counts as a cycle, 1, 1, 0 against 1, 2, 0 giving 1, 3, 2. The levers 1, 1 over Z 3 give subset-sum counts 1, 2, 1, the empty subset included, and the levers

0,0, 1,0, 0,1 over Z 3 ^2^ fill the 3 by 3 grid 2, 2, 0 by 2, 2, 0 by 0, 0, 0, re-derived by direct enumeration of all 8 subsets. The valuation family pins the minima: 1, 1, 2 and 1, 2, 1 at 0, and 2, 4, 6 and 4, 2, 6 at 1.

The application is icpc world finals 2022 problem Z (book 10, chapter 11), archaeological recovery, whose state multisets over Z 3 ^k^, projected residue counts, and subset-sum convolutions are exactly this machinery, with the full recovery algorithm living in the icpc book.

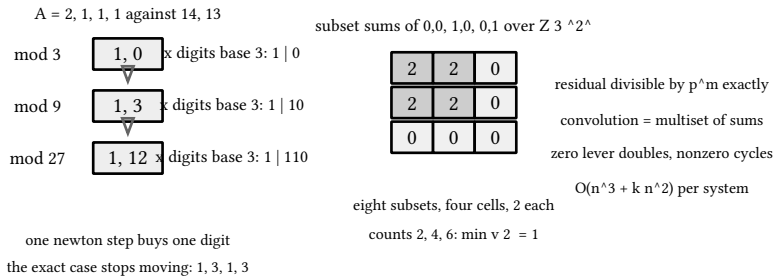


figure 290: the hensel ladder with x climbing mod 3 to 9 to 27 as base-3 digits, beside the 3x3 subset-sum grid with the four occupied cells shaded

34.7. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's six sample files per language, go test files excluded:

language	build sloc	container dependency	boundary note
c	843	static matrices, u64 residues	plain 64-bit products at 1e9+7, add p before subtracting to stay positive, kirchhoff carries its own det
go	385	slices, no imports beyond fmt	hensel refuses loudly when the residual is not divisible, matrix ops as free functions
java	832	jdk 27 stdlib, long residues	p added before every modular subtraction, cramer pairs as a two-field Frac class, the exact fib ladder rides long to fib(78)
c#	305	jagged longs, tuples, linq	reuses the ch16 modpow and the ch34 determinant across files, cramer pairs as value tuples
javascript	301	nested arrays, BigInt at 1e9+7	the zpk file stays on plain Number, p = 3 keeps every product tiny, exact ints to fib(78)
python	479	list rows, inline asserts	one list comprehension per row operation, permutation-sign cross-checks in-suite
lua	644	tables, 1-based rows	fraction pairs as two-slot tables, the kirchhoff fixtures shifting 0-based edges to 1-based

sources: cp-algorithms, "Gauss & System of Linear Equations", cp-algorithms.com/linear_algebra/linear-system-gauss.html, "Kraut & Determinant", cp-algorithms.com/linear_algebra/determinant-kraut.html, the fraction-free variant taught here, "Rank of a matrix", cp-algorithms.com/linear_algebra/rank-matrix.html, "Binary Exponentiation", cp-algorithms.com/algebra/binary-exp.html, the scalar base the matrix version lifts, there being no standalone matrix exponentiation page, and "Kirchhoff Theorem", cp-algorithms.com/graph/kirchhoff-theorem.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. cp-algorithms carries no cramer or hensel article; those sections cite the icpc world finals 2023 problem A and 2022 problem Z solutions as their application sources, and cramer's rule as standard linear algebra. Sample behavior verified by the seven suite gates scoped to chapter 34: c 6 files and 90 checks, go 24 test functions, java 6 files and 90 checks under run-java-samples, c# 28 facts, javascript 24 tests and 69 asserts, python 6 files and 86 asserts, lua 24 checks, zero skipped.

35. combinatorics

Counting is its own skill, separate from finding and optimizing, and this chapter is the counting course. Binomial tables come first because half the sections stand on them, then catalan numbers, stars and bars, and inclusion-exclusion, the four engines. Burnside counts under symmetry, k-subset generation walks objects in order, sperner bounds antichains, the frobenius problem counts what coin systems cannot reach, and josephus counts around a circle. The back half applies the engines: bishops on a chessboard by a per-color dp, balanced brackets by ballot numbers, labeled graphs by an exponential recurrence, and one prose-only stop at the fifteen puzzle, whose solvability test is a single parity predicate with nothing to port. Chapter 16 owns the modular arithmetic underneath, and the mobius table rides in from book 9, ch 28 (practical data structures and algorithms, advanced number theory).

35.1. binomial tables on three roads

One value, three representations, three jobs. The modular road builds factorial and inverse-factorial tables over a prime, one forward pass and one backward pass seeded by a fermat inverse, and then answers C(n, k) mod p in constant time as three table lookups and two multiplies. The exact road runs the product ladder, multiply by n - i and divide by i + 1 at each step, and stays an integer at every step because each prefix divides evenly. The log-space road keeps a cumulative lnfact table and compares binomials by ln of the value, tolerance-based, the only road that ranks C(999, 500) against its neighbors when the integer itself has 299 decimal digits. The p-adic road reads the exponent of a prime in the answer off legendre's sum, the difference of three floor-sums.

The dry run: the fixture is the four-road anchor set of the C# suite, with C and Java pinning the same modular digits and javascript riding BigInt past its 2^53 ceiling.

1. The exact ladder walks C(10, 3) one dividing step at a time: 1 × 10 / 1 = 10, then 10 × 9 / 2 = 45, then 45 × 8 / 3 = 120.
2. The modular road answers from the tables: C(1000, 500) = 159835829 as fact × invfact × invfact, and the row sanity sum over k of C(30, k) = 2^30 with C(30, 15) = 155117520.
3. The ln road subtracts cumulative logs, lnf 1000 - 2 × lnf 500 = 689.467261567851, the table and a direct log sum agreeing to 1e-9.
4. The p-adic road reads legendre's floors: 50 + 25 + 12 + 6 + 3 + 1 = 97 for 100, 25 + 12 + 6 + 3 + 1 = 47 for 50, and 97 - 2 × 47 = 3.
5. So C(100, 50) is 2^3 times an odd part, and that part carries the residue 538992043.
6. The exact ceilings pin the boundary: C(62, 31) = 465428353255261088 holds and C(67, 33) refuses past 2^63.

road	the walk	pinned landing
modular	fact × invfact × invfact	C(1000,500) = 159835829
exact	1 × 10 / 1 = 10, then 45 × 8 / 3 = 120	C(10,3) = 120
ln	lnf1000 - 2 lnf500	689.467261567851
p-adic	97 - 2 × 47 = 3	v2 of C(100,50) = 3

The 120, the 159835829, and the 3 are the asserted landings, and the listings below build all four roads in seven languages.

```
static void build(void) {
  fact[0] = invf[0] = 1;
  for (int i = 1; i < NMAX; i++)
    fact[i] = fact[i - 1] * (unsigned long long)i % P;
  invf[NMAX - 1] = modpow(fact[NMAX - 1], P - 2, P);
  for (int i = NMAX - 1; i > 0; i--)
    invf[i - 1] = invf[i] * (unsigned long long)i % P;
  fact2[0] = invf2[0] = 1;
  for (int i = 1; i < NMAX; i++)
    fact2[i] = fact2[i - 1] * (unsigned long long)i % P2;
  invf2[NMAX - 1] = modpow(fact2[NMAX - 1], P2 - 2, P2);
  for (int i = NMAX - 1; i > 0; i--)
    invf2[i - 1] = invf2[i] * (unsigned long long)i % P2;
  lnf[0] = 0.0;
  for (int i = 1; i ≤ 1000; i++)
    lnf[i] = lnf[i - 1] + log((double)i);
}

static unsigned long long cmod(int a, int b) {
  if (b < 0 || b > a)
    return 0;
  return fact[a] * invf[b] % P * invf[a - b] % P;
}

static unsigned long long cmod2(int a, int b) {
  if (b < 0 || b > a)
    return 0;
  return fact2[a] * invf2[b] % P2 * invf2[a - b] % P2;
}

static long long legendre(long long v, long long p) {
  long long s = 0;
  for (long long pk = p; pk ≤ v; pk *= p)
    s += v / pk;
  return s;
}
```

listing 1139: c, both table builds, the O(1) query, the legendre sum (source: dsa/samples-c/src/Ch35/binom.c, lines 43-78)

```
func binomTable(n int, p int64) (fact, invFact []int64) {
  key := [2]int64{int64(n), p}
  if t, ok := binomTableCache[key]; ok {
    return t[0], t[1]
  }
  fact = make([]int64, n+1)
  invFact = make([]int64, n+1)
  fact[0] = 1
  for i := 1; i <= n; i++ {
    fact[i] = fact[i-1] * int64(i) % p
  }
  invFact[n] = invMod(fact[n], p)
  for i := n; i ≥ 1; i-- {
    invFact[i-1] = invFact[i] * int64(i) % p
  }
  binomTableCache[key] = [2][]int64{fact, invFact}
  return fact, invFact
}

// BinomMod answers C(n,k) mod p in O(1) from the cached tables.
func BinomMod(n, k int, p int64) int64 {
  if k < 0 || k > n {
    return 0
  }
  fact, invFact := binomTable(n, p)
  return fact[n] * invFact[k] % p * invFact[n-k] % p
}
```

listing 1140: go, the cached table pair, the query, the 128-bit exact ladder below (source: dsa/samples-go/ch35/binom.go, lines 28-54)

```
  static void build() {
    fact[0] = invf[0] = 1;
    for (int i = 1; i < NMAX; i++)
      fact[i] = fact[i - 1] * i % P;
    invf[NMAX - 1] = modpow(fact[NMAX - 1], P - 2, P);
    for (int i = NMAX - 1; i > 0; i--)
      invf[i - 1] = invf[i] * i % P;
    fact2[0] = invf2[0] = 1;
    for (int i = 1; i < NMAX; i++)
      fact2[i] = fact2[i - 1] * i % P2;
    invf2[NMAX - 1] = modpow(fact2[NMAX - 1], P2 - 2, P2);
    for (int i = NMAX - 1; i > 0; i--)
      invf2[i - 1] = invf2[i] * i % P2;
    lnf[0] = 0.0;
    for (int i = 1; i ≤ 1000; i++)
      lnf[i] = lnf[i - 1] + Math.log(i);
  }

  static long cmod(int a, int b) {
    if (b < 0 || b > a)
      return 0;
    return fact[a] * invf[b] % P * invf[a - b] % P;
  }

  static long cmod2(int a, int b) {
    if (b < 0 || b > a)
      return 0;
    return fact2[a] * invf2[b] % P2 * invf2[a - b] % P2;
  }

  static long legendre(long v, long p) {
    long s = 0;
    for (long pk = p; pk ≤ v; pk *= p)
      s += v / pk;
    return s;
  }
```

listing 1141: java, both prime-pair table builds, the O(1) query, the legendre sum (source: dsa/samples-java/src/Ch35/Binom.java, lines 42-77)

```
    public Binom(int n, long p)
    {
        _p = p;
        _fact = new long[n + 1];
        _invFact = new long[n + 1];
        _lnFact = new double[n + 1];
        _fact[0] = 1 % p;
        _lnFact[0] = 0;
        for (int i = 1; i ≤ n; i++)
        {
            _fact[i] = _fact[i - 1] * i % p;
            _lnFact[i] = _lnFact[i - 1] + Math.Log(i);
        }
        _invFact[n] = NumTheory.ModPow(_fact[n], p - 2, p);
        for (int i = n; i > 0; i--)
            _invFact[i - 1] = _invFact[i] * i % p;
    }

    public long Choose(int n, int k) ⇒
        k < 0 || k > n ? 0 : _fact[n] * _invFact[k] % _p * _invFact[n - k] % _p;

    /// <summary>C(n,k) mod a second prime using its own tables.</summary>
    public long ChooseMod(int n, int k, long otherPrime)
    {
        var t = new Binom(n, otherPrime);
        return t.Choose(n, k);
    }

    /// <summary>Exact product ladder through UInt128; throws past the 2^63 boundary
    /// where plain-long intermediates would wrap (the multiply peaks above the
    /// final value near mid-ladder).</summary>
    public static long Exact(long n, long k)
    {
        if (k < 0 || k > n)
            return 0;
        k = Math.Min(k, n - k);
        long result = 1;
        for (long i = 0; i < k; i++)
            result = checked((long)((UInt128)result * (ulong)(n - i) / (uint)(i + 1)));
        return result;
    }

    public double LnChoose(int n, int k) ⇒ _lnFact[n] - _lnFact[k] - _lnFact[n - k];
```

listing 1142: c#, the tables and ln table in one constructor, the exact ladder through UInt128 (source: dsa/samples/src/Ch35/Binom.cs, lines 18-60)

```
export class BinomTable {
  constructor(n, p = 1000000007n) {
    this.p = p;
    const fact = [1n];
    for (let i = 1; i ≤ n; i++) {
      fact.push((fact[i - 1] * BigInt(i)) % p);
    }
    const invFact = new Array(n + 1);
    let inv = modPow(fact[n], p - 2n, p); // fermat inverse from the top
    for (let i = n; i ≥ 1; i--) {
      invFact[i] = inv;
      inv = (inv * BigInt(i)) % p;
    }
    invFact[0] = 1n;
    this.fact = fact;
    this.invFact = invFact;
  }

  c(n, k) {
    if (k < 0 || k > n) return 0;
    return Number(this.fact[n] * this.invFact[k] % this.p
      * this.invFact[n - k] % this.p);
  }
}

// the exact ladder: r = r * (n-k+i) / i is an integer at every step
export function binomExact(n, k) {
  if (k < 0 || k > n) return 0n;
  k = Math.min(k, n - k);
  let r = 1n;
  for (let i = 1; i ≤ k; i++) {
    r = (r * BigInt(n - k + i)) / BigInt(i);
  }
  return r;
}
```

listing 1143: javascript, BigInt tables, the fermat inverse from the top, the ladder on BigInt past C(56,28) (source: dsa/samples-js/src/ch35-binom.mjs, lines 22-56)

```
def tables(limit, p):
    fact = [1] * (limit + 1)
    inv = [1] * (limit + 1)
    for i in range(1, limit + 1):
        fact[i] = fact[i - 1] * i % p
    inv[limit] = pow(fact[limit], p - 2, p)
    for i in range(limit, 0, -1):
        inv[i - 1] = inv[i] * i % p
    return fact, inv


def C_mod(n_, k, fact, inv, p):
    if k < 0 or k > n_:
        return 0
    return fact[n_] * inv[k] % p * inv[n_ - k] % p


def C_exact(n_, k):
    k = min(k, n_ - k)
    num = 1
    for i in range(k):  # the product ladder stays exact
        num = num * (n_ - i) // (i + 1)
    return num
```

listing 1144: python, the table build, the query, the exact ladder (source: dsa/samples-py/src/Ch35/binom.py, lines 20-42)

```
local function build_tables(p, n)
  local fact, invf = {}, {}
  fact[1] = 1
  for i = 1, n do fact[i + 1] = fact[i] * i % p end
  invf[n + 1] = modpow(fact[n + 1], p - 2, p)
  for i = n - 1, 0, -1 do invf[i + 1] = invf[i + 2] * (i + 1) % p end
  return fact, invf
end

local fact1, invf1 = build_tables(P1, 1000)
local fact2, invf2 = build_tables(P2, 1000)

local function C_mod(n, k, p)
  local fact = (p == P1) and fact1 or fact2
  local invf = (p == P1) and invf1 or invf2
  if k < 0 or k > n then return 0 end
  return fact[n + 1] * invf[k + 1] % p * invf[n - k + 1] % p
end

-- exact product ladder with overflow detection: each step divides out
-- gcds first so the running value is always the exact C(n, i)
local function C_exact(n, k)
  if k < 0 or k > n then return 0 end
  k = math.min(k, n - k)
  local c = 1
  for i = 1, k do
    local f = n - i + 1
    local g = gcd2(f, i)
    local ff, ii = f // g, i // g
    if c % ii ~= 0 then return nil end -- cannot happen, guarded anyway
    c = c // ii
    if c > math.maxinteger // ff then return nil end -- past 2^63
    c = c * ff
  end
  return c
end
```

listing 1145: lua, the two passes, the query, the overflow-guarded ladder returning nil past 2^63 (source: dsa/samples-lua/ch35_binom.lua, lines 28-63)

The fixtures pin all four roads against each other. Modular: C(10,3) = 120; C(1000,500) mod 1e9+7 = 159835829; the same value mod 998244353, one of the chapter 29 transform primes, = 640488516, a digit the alignment brief originally carried differently and two independent exact roads disproved, so the proven value is what the suites pin; the row sanity sum over k of C(30,k) = 2^30 with C(30,15) = 155117520. Exact and boundary: C(62,31) = 465428353255261088; C(66,33) = 7219428434016265740 is the last central binomial under 2^63 and C(67,33) = 14226520737620288370 crosses, java pinning it through Long.parseUnsignedLong with the identical decimal string and comparing unsigned, with javascript's own ceiling at n = 56, C(56,28) = 7648690600760440, the last under 2^53, its ladder riding BigInt from there. Log and p-adic: ln C(1000,500) = 689.467261567851 within 1e-9, the cumulative table and the lgamma road agreeing to 9e-13 where the language has lgamma, java's stdlib shipping none so its twin runs a nine-coefficient lanczos series and agrees with the summed table to 1e-9; v2 of C(100,50) = 3, read as 97 - 2 times 47 by legendre, and the exact 30-digit value 1008913445455641 93334812497256 pins as 2^3 times an odd number with residue 538992043 mod 1e9+7.

Two finals problems ride this section. Icpc world finals 2018 problem D (book 10, chapter 9), gem island, prices its drop recursion in floats from an lnfact table because the raw binomials near C(999,500) overflow doubles before the division. Icpc world finals 2023 problem B (book 10, chapter 12), schedule, searches for the smallest k with C(k-1, ceil(k/2)) at least n, the modular road with the search loop living in the k-subset section below.

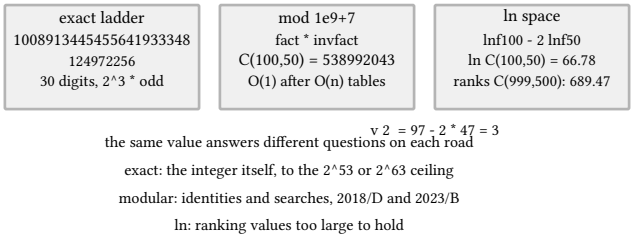


figure 291: one value C(100,50) on three roads: the 30-digit exact integer, its residue, its log

35.2. catalan numbers

The catalan numbers count everything nested: balanced bracket shapes, triangulations, binary trees, mountain profiles. Two roads compute them, and the sweep proving the roads equal is the fixture that matters. The closed form reads C n = C(2n, n) over n + 1 straight off the binomial tables, one modular inverse. The convolution recurrence builds the row bottom up, C n+1 = sum over i of C i C n-i , each number folding from the two halves of everything shorter, O(n^2) against the closed form's O(1) after tables.

The dry run: the fixture is the row C 0 through C 10 with the sweep and the two rungs, asserted by the C# suite; lua lands C 20 by its own exact ladder, C(40, 20) = 137846528820 over 21.

1. The convolution folds from C 0 = 1: C 1 = 1 × 1 = 1, C 2 = 1 × 1
   1. 1 × 1 = 2, C 3 = 1 × 2 + 1 × 1 + 2 × 1 = 5.
2. C 4 pairs every short prefix against its mirror: 1 × 5 + 1 × 2 + 2 × 1 + 5 × 1 = 14, and C 5 reads 14 + 5 + 4 + 5 + 14 = 42.
3. The row runs on to C 10 = 16796, and the sweep asserts recurrence against closed form for every n to 30.
4. The closed form lands the same numbers off the tables: C(8, 4) = 70 and 70 / 5 = 14, C(10, 5) = 252 and 252 / 6 = 42.
5. Exact C 20 = 6564120420 while C 30 mod 1e9+7 = 475387402.

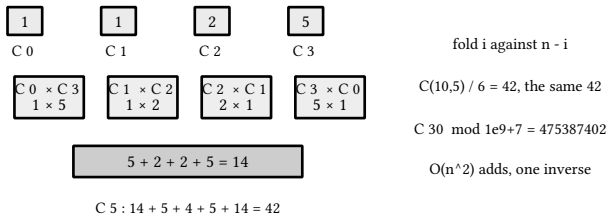


figure 292: the C 4 fold as four mirrored products summing into 14

The 14, the 42, and the 6564120420 are the pinned landings, and the listings below build both roads in seven languages.

```
static unsigned long long catalan_closed(int v) {
  return binom(2 * v, v) % P * modpow(v + 1, P - 2) % P;
}

static unsigned long long cat_rec[64];

int main(void) {
  build();

  // the table C_0..C_10
  unsigned long long want[11] = {1,   1,   2,    5,    14,   42,
                                 132, 429, 1430, 4862, 16796};
  for (int i = 0; i ≤ 10; i++)
    CHECK(catalan_closed(i) == want[i]);

  // modular anchors
  CHECK(catalan_closed(20) == 6564120420ULL % P);
  CHECK(catalan_closed(30) == 475387402ULL);

  // the convolution recurrence, swept against the closed form
  cat_rec[0] = 1;
  for (int v = 0; v < 30; v++) {
    unsigned long long s = 0;
    for (int i = 0; i ≤ v; i++)
      s = (s + cat_rec[i] * cat_rec[v - i]) % P;
    cat_rec[v + 1] = s;
  }
  int agree = 1;
  for (int v = 0; v ≤ 30; v++)
    if (cat_rec[v] ≠ catalan_closed(v))
      agree = 0;
  CHECK(agree);
```

listing 1146: c, the closed form, the table anchors, the recurrence sweep and agreement check (source: dsa/samples-c/src/Ch35/catalan.c, lines 50-81)

```
func CatalanMod(n int, p int64) int64 {
  c := BinomMod(2*n, n, p)
  return c * invMod(int64(n+1), p) % p
}

// CatalanExact carries the closed form on the native integer through
// the 128-bit division.
func CatalanExact(n int) int64 {
  c := BinomExact(2*n, n)
  q, _ := bits.Div64(0, c, uint64(n+1))
  return int64(q)
}

// CatalanRec is the convolution recurrence C_{n+1} = sum_i C_i
// C_{n-i}, the cross-check road.
func CatalanRec(n int, p int64) int64 {
  c := make([]int64, n+1)
  c[0] = 1
  for m := 1; m <= n; m++ {
    sum := int64(0)
    for i := 0; i < m; i++ {
      sum = (sum + c[i]*c[m-1-i]) % p
    }
    c[m] = sum
  }
  return c[n]
}
```

listing 1147: go, closed form, exact form, the recurrence (source: dsa/samples-go/ch35/catalan.go, lines 6-32)

```
  static long binom(int a, int b) {
    if (b < 0 || b > a)
      return 0;
    return fact[a] * invf[b] % P * invf[a - b] % P;
  }

  static long catalanClosed(int v) {
    return binom(2 * v, v) % P * modpow(v + 1, P - 2) % P;
  }

  static long[] catRec = new long[64];
```

listing 1148: java, binom and the closed form, the recurrence array swept in main (source: dsa/samples-java/src/Ch35/Catalan.java, lines 43-53)

```
    public long Number(int n) => _binom.Choose(2 * n, n) * NumTheory.ModInv(n + 1, _p) % _p;

    public long NumberExact(int n) => Binom.Exact(2 * n, n) / (n + 1);

    /// <summary>C_{n+1} = sum_i C_i C_{n-i} mod p, iterated from C_0 = 1.</summary>
    public long ByRecurrence(int n)
    {
        var c = new long[n + 1];
        c[0] = 1;
        for (int i = 1; i ≤ n; i++)
            for (int j = 0; j < i; j++)
                c[i] = (c[i] + c[j] * c[i - 1 - j]) % _p;
        return c[n];
    }
```

listing 1149: c#, the closed form from the binomial class, the recurrence below it (source: dsa/samples/src/Ch35/Catalan.cs, lines 18-31)

```
export function catalanClosed(n) {
  return ((((T.c(2 * n, n) - T.c(2 * n, n + 1)) % PN) + PN) % PN);
}

export function catalanExact(n) {
  return Number(binomExact(2 * n, n) / BigInt(n + 1));
}

export function catalanRecurrence(m) {
  const c = [1]; // C_0, mod p
  for (let n = 0; n < m; n++) {
    let s = 0;
    for (let i = 0; i ≤ n; i++) {
      s = (s + c[i] * c[n - i]) % PN;
    }
    c.push(s);
  }
  return c;
}
```

listing 1150: javascript, the closed form as C(2n,n) - C(2n,n+1), no inverse needed, and the recurrence (source: dsa/samples-js/src/ch35-catalan.mjs, lines 12-30)

```
def catalan_closed(k):
    return C_mod(2 * k, k) * pow(k + 1, MOD - 2, MOD) % MOD


def catalan_recurrence(kmax):
    # C_{n+1} = sum_i C_i C_{n-i}
    out = [1]
    for m in range(kmax):
        total = 0
        for i in range(m + 1):
            total += out[i] * out[m - i]
        out.append(total % MOD)
    return out
```

listing 1151: python, the closed form, the convolution recurrence (source: dsa/samples-py/src/Ch35/catalan.py, lines 40-52)

```lua
local function catalan_closed(n)
  return C_mod(2 * n, n) * modpow(n + 1, P - 2) % P
end

local function catalan_recurrence(nmax)
  local c = {}
  c[1] = 1 -- C_0 at lua index 1
  for n = 0, nmax - 1 do
    local s = 0
    for i = 0, n do s = (s + c[i + 1] * c[n - i + 1]) % P end
    c[n + 2] = s
  end
  return c
end
```

listing 1152: lua, the closed form and the recurrence, C 0 at lua index 1 (source: dsa/samples-lua/ch35_catalan.lua, lines 31-44)

The fixtures pin the row C 0 through C 10 as 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, exact C 20 = 6564120420, C 30 mod 1e9+7 = 475387402, and the sweep asserting the recurrence equals the closed form for every n up to 30. The interpretations read off the row: triangulations of a hexagon = C 4 = 14, binary trees on 5 nodes = C 5 = 42, and C 0 = 1 counting the one empty object. The javascript port computes the closed form as C(2n,n) - C(2n,n+1), which needs no division at all, and lua carries the exact ladder inline, C(40,20) = 137846528820 divided by 21 landing the same
6564120420. This is a general technique section, the counting shape

beneath the bracket section and the nesting facet of icpc world finals 2019 problem D (book 10, chapter 10).

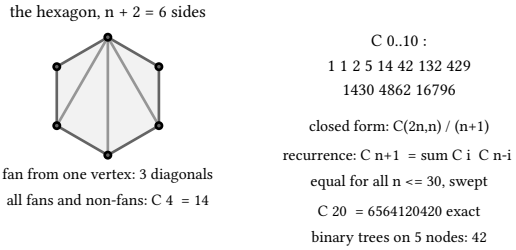


figure 293: triangulations of a convex (n+2)-gon count as C n : the hexagon's 14 (C 4) with one fan drawn

35.3. stars and bars with bounds

Compositions of n into k non-negative parts count as C(n + k - 1, k - 1), the bars choosing their slots among the stars. Requiring positive parts burns one unit per part and lands C(n - 1, k - 1). Upper bounds are where the work is: subtract the arrangements where some part exceeds the cap, by inclusion-exclusion over the set of violated parts, each subset of size s contributing C(k, s) times the shifted count with s caps plus one removed,

sum over s of (-1)^s C(k, s) C(n - s(cap+1) + k - 1, k - 1).

The dry run: the fixtures are both identities and the two bounded counts, asserted by the C# suite with its own nested-loop brute counting the same tuples.

1. Unbounded first: 10 stars cut by 2 bars gives C(12, 2) = 66 arrangements, and positive parts burn 3 units for C(9, 2) = 36.
2. The bounded walk on 20 into 3 with cap 10 keeps a running total: s = 0 adds C(22, 2) = 231, s = 1 subtracts 3 × C(11, 2) = 165, and s = 2 dies at 20 - 22 below zero, landing 231 - 165 = 66.
3. The s = 1 term reads one violated part holding 11 of the 20, 9 stars left, C(11, 2) = 55 arrangements, 3 choices of violator.
4. The 4-part walk on 30 with cap 10 swings negative first: s = 0 adds C(33, 3) = 5456, s = 1 subtracts 4 × 1540 = 6160 for −704, s = 2 adds 6 × 165 = 990, landing 286.

5. The brute twins enumerate every capped tuple and agree, 66 and 286, and the edges hold: k = 1 counts 1 while n fits the cap, x + y = 5 with cap 2 gives 0.

fixture	s	term	running
20 into 3, cap 10	0	1. C(22,2) = +231	231
20 into 3, cap 10	1	• 3 C(11,2) = −165	66
30 into 4, cap 10	0	1. C(33,3) = +5456	5456
30 into 4, cap 10	1	• 4 C(22,3) = −6160	−704
30 into 4, cap 10	2	1. 6 C(11,3) = +990	286

Both bounded answers, 66 and 286, survive their brute twins, and the listings below build the three counters in seven languages.

```
static unsigned long long compositions(int total, int parts) {
  if (parts ≤ 0)
    return total == 0 ? 1 : 0;
  return binom(total + parts - 1, parts - 1);
}

static unsigned long long positive_compositions(int total, int parts) {
  if (total < parts)
    return 0;
  return binom(total - 1, parts - 1);
}

// bounded: each part in [0, cap], IE over parts past the cap
static unsigned long long bounded(int total, int parts, int cap) {
  unsigned long long s = 0;
  for (int i = 0; i ≤ parts; i++)
    if (i & 1)
      s = (s + P -
           binom(parts, i) *
               binom(total - (long long)i * (cap + 1) + parts - 1, parts - 1) %
               P) %
          P;
    else
      s = (s +
           binom(parts, i) *
               binom(total - (long long)i * (cap + 1) + parts - 1, parts - 1) %
               P) %
          P;
  return s;
}
```

listing 1153: c, the two identities, the bounded IE loop in modular arithmetic (source: dsa/samples-c/src/Ch35/starsbars.c, lines 51-80)

```
func Compositions(n, k int, p int64) int64 {
  return BinomMod(n+k-1, k-1, p)
}

// PositiveCompositions counts the k positive parts: C(n-1, k-1).
func PositiveCompositions(n, k int, p int64) int64 {
  return BinomMod(n-1, k-1, p)
}

// BoundedCompositions counts compositions with every part ≤ cap by
// inclusion-exclusion over violated parts:
// sum_s (-1)^s C(k,s) C(n - s(cap+1) + k - 1, k - 1).
func BoundedCompositions(n, k int, cap int64, p int64) int64 {
  total := int64(0)
  for s := 0; s <= k; s++ {
    rem := int64(n) - int64(s)*(cap+1)
    if rem < 0 {
      continue
    }
    term := BinomMod(k, s, p) *
      BinomMod(int(rem)+k-1, k-1, p) % p
    if s%2 == 1 {
      total = (total - term + p) % p
    } else {
      total = (total + term) % p
    }
  }
  return total
}
```

listing 1154: go, the same three over the cached tables (source: dsa/samples-go/ch35/starsbars.go, lines 5-33)

```
  static long compositions(int total, int parts) {
    if (parts ≤ 0)
      return total == 0 ? 1 : 0;
    return binom(total + parts - 1, parts - 1);
  }

  static long positiveCompositions(int total, int parts) {
    if (total < parts)
      return 0;
    return binom(total - 1, parts - 1);
  }

  // bounded: each part in [0, cap], IE over parts past the cap. the
  // shifted top narrows to int exactly as the C long long argument does
  static long bounded(int total, int parts, int cap) {
    long s = 0;
    for (int i = 0; i ≤ parts; i++) {
      long top = total - (long) i * (cap + 1) + parts - 1;
      long term = binom(parts, i) * binom((int) top, parts - 1) % P;
      if ((i & 1) ≠ 0)
        s = (s + P - term) % P;
      else
        s = (s + term) % P;
    }
    return s;
  }
```

listing 1155: java, the two identities, the bounded IE loop with p added before the subtraction (source: dsa/samples-java/src/Ch35/Starsbars.java, lines 50-75)

```
    /// <summary>Non-negative parts: C(n + k - 1, k - 1).</summary>
    public long Count(long n, int k) => _binom.Choose((int)(n + k - 1), k - 1);

    /// <summary>Positive parts: C(n - 1, k - 1).</summary>
    public long PositiveCount(long n, int k) => _binom.Choose((int)(n - 1), k - 1);

    /// <summary>Every part &lt;= cap: sum_s (-1)^s C(k,s) C(n - s(cap+1) + k - 1, k - 1).</summary>
    public long BoundedCount(long n, int k, int cap)
    {
        long total = 0;
        for (int s = 0; s ≤ k; s++)
        {
            long reduced = n - (long)s * (cap + 1);
            if (reduced < 0)
                break;
            long term = _binom.Choose((int)(reduced + k - 1), k - 1) * _binom.Choose(k, s);
            total += s % 2 == 0 ? term : -term;
        }
        return total;
    }
```

listing 1156: c#, the three counters over the binomial class, the IE loop breaking on negative remainder (source: dsa/samples/src/Ch35/StarsBars.cs, lines 13-32)

```javascript
const cSmall = (n, k) => {
  if (k < 0 || k > n) return 0;
  k = Math.min(k, n - k);
  let r = 1;
  for (let i = 1; i ≤ k; i++) r = (r * (n - k + i)) / i;
  return r;
};

export const compositions = (n, k) => cSmall(n + k - 1, k - 1);

export const positiveCompositions = (n, k) => cSmall(n - 1, k - 1);

export function boundedCompositions(n, k, cap) {
  let total = 0;
  for (let s = 0; s ≤ k; s++) {
    const m = n - s * (cap + 1);
    if (m < 0) break;
    total += (s % 2 === 0 ? 1 : -1) * cSmall(k, s) * cSmall(m + k - 1, k - 1);
  }
  return total;
}
```

listing 1157: javascript, the small exact ladder and the three counters on it (source: dsa/samples-js/src/ch35-starsbars.mjs, lines 7-27)

```python
def compositions(total, parts):
    # non-negative parts: C(n + k - 1, k - 1)
    return C_exact(total + parts - 1, parts - 1)


def compositions_positive(total, parts):
    # positive parts: C(n - 1, k - 1)
    return C_exact(total - 1, parts - 1)


def bounded(total, parts, cap):
    # inclusion-exclusion over parts pushed past the cap
    total_count = 0
    for s in range(parts + 1):
        rest = total - s * (cap + 1) + parts - 1
        term = C_exact(parts, s) * C_exact(rest, parts - 1)
        total_count += -term if s % 2 else term
    return total_count
```

listing 1158: python, the three counters, the IE sum over violated parts (source: dsa/samples-py/src/Ch35/starsbars.py, lines 24-41)

```lua
local function compositions(n, k) -- non-negative parts
  return C(n + k - 1, k - 1)
end

local function compositions_positive(n, k)
  if n < k then return 0 end
  return C(n - 1, k - 1)
end

-- inclusion-exclusion over subsets of parts exceeding the cap
local function bounded(n, k, cap)
  local total = 0
  for s = 0, k do
    local m = n - s * (cap + 1)
    if m ≥ 0 then
      local term = C(k, s) * C(m + k - 1, k - 1) % P
      if s % 2 == 0 then
        total = (total + term) % P
      else
        total = (total - term) % P
      end
    end
  end
  return (total % P + P) % P
end
```

listing 1159: lua, the three counters and the IE loop, brute tuples below (source: dsa/samples-lua/ch35_starsbars.lua, lines 32-56)

The fixtures pin both identities and the bounded form. Compositions of 10 into 3 = 66 = C(12,2), positive ones = 36 = C(9,2). Bounded: x + y + z = 20 with each at most 10 gives 66, the IE terms reading C(22,2) - 3 C(11,2) = 231 - 165, and x + y + z + w = 30 with each at most 10 gives 286 as C(33,3) - 4 C(22,3) + 6 C(11,3), both brute-counted in-suite by nested loops. The edges read k = 1 as 1 exactly when n fits the cap, n = 0 into k parts as the one all-zero tuple, and x + y = 5 each at most 2 as 0, the impossible bound. The application is icpc world finals 2018 problem D (book 10, chapter 9) again, whose drop allocations are compositions walking this bounded and unbounded ladder in log space.

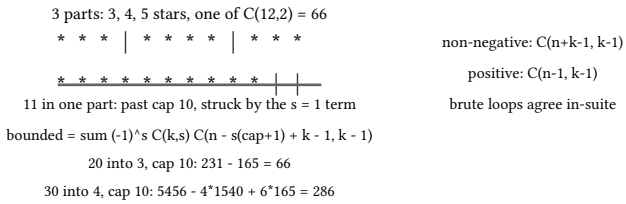


figure 294: 10 stars cut by 2 bars into one of the 66 arrangements, a bounded row struck out by the IE term

35.4. the inclusion-exclusion principle

The union of sets counts as the alternating sum of intersections: add the singles, subtract the pairs, add the triples, and the elements in r sets cancel to exactly one. The principle runs whole families of counting questions once the sets are chosen. Derangements are permutations avoiding every fixed-point set, the subfactorial sum !n = n! times the alternating reciprocal series, computable exactly by the recurrence !n = (n-1) times (!(n-1) + !(n-2)). Coprime counting is legendre's form: integers up to m divisible by none of a prime set, the alternating sum over subset products. Squarefree counting swaps the prime set for the mobius function, the table from book 9, ch 28 (practical data structures and algorithms, advanced number theory)'s linear sieve, summed as mu of d times the floor of n over d squared.

The dry run: the fixtures are the derangement anchors and the coprime ledger, asserted by the C# suite, with the C suite writing the ledger out term by term as its own check.

1. Derangements climb by the recurrence from !0 = 1 and !1 = 0: !2 = 1 × (0 + 1) = 1, !3 = 2 × (1 + 0) = 2, !4 = 3 × (2 + 1) = 9, !5 = 4 × (9 + 2) = 44.
2. The next rung reads !6 = 5 × (44 + 9) = 265, the suite sweeps sum against recurrence for every n to 10, and a brute walk over all 120 permutations of 5 finds exactly 44 fixed-point-free.
3. The coprime ledger over 2, 3, 5 on 1..100 runs 100 - 50 = 50, 50 - 33 = 17, 17 - 20 = −3, then the pairs pull back, −3 + 16 = 13, 13 + 10 = 23, 23 + 6 = 29, and the triple closes 29 - 3 = 26.
4. The running sum dips to −3 midwalk: the singles pull out 103 in all, the pairs push back 32, the triple pulls 3.
5. Squarefree up to 100 reads 61 by the mobius sum, and the exactly-r fork on sizes 20 and 15 sharing 8 gives 20 + 15 - 8 = 27, 12, 7.

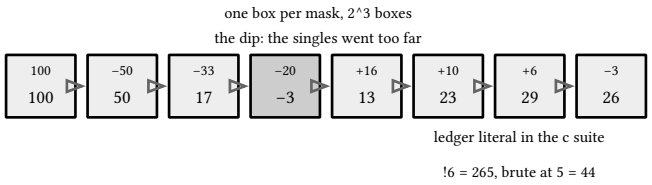


figure 295: the coprime ledger as eight running boxes, the dip below zero shaded, closing at 26

The 44, the 26, and the 61 are the pinned landings, and the listings below run these families in seven languages.

```c
// derangements by the alternating sum !n = sum_k (-1)^k n!/k!
static long long subfactorial_sum(int v) {
  // term_k = n!/k!, computed as the falling product
  long long s = 0;
  for (int k = 0; k ≤ v; k++) {
    long long t = 1;
    for (int j = k + 1; j ≤ v; j++)
      t *= j;
    s += (k & 1) ? -t : t;
  }
  return s;
}

// derangements by the recurrence
static long long subfactorial_rec(int v) {
  long long d0 = 1, d1 = 0;
  if (v == 0)
    return 1;
  if (v == 1)
    return 0;
  for (int i = 2; i ≤ v; i++) {
    long long d2 = (long long)(i - 1) * (d1 + d0);
    d0 = d1;
    d1 = d2;
  }
  return d1;
}
```

listing 1160: c, the subfactorial sum and recurrence, the recurrence's agreement loop in main (source: dsa/samples-c/src/Ch35/incexc.c, lines 46-72)

```
// DerangementRec is the recurrence !n = (n-1)(!(n-1) + !(n-2)).
func DerangementRec(n int) int64 {
  if n == 0 {
    return 1
  }
  prev2, prev1 := int64(1), int64(0) // !0, !1
  for m := 2; m <= n; m++ {
    prev2, prev1 = prev1, int64(m-1)*(prev1+prev2)
  }
  return prev1
}

// CoprimeCount counts integers in 1..limit coprime to every prime
// by the legendre alternating sum over subsets.
func CoprimeCount(limit int64, primes []int64) int64 {
  total := limit
  for mask := 1; mask < 1<<len(primes); mask++ {
    prod := int64(1)
    bitsSet := 0
    overflow := false
    for b, pr := range primes {
      if mask&(1<<b) == 0 {
        continue
      }
      bitsSet++
      prod *= pr
      if prod > limit {
        overflow = true
        break
      }
    }
    if overflow {
      continue
    }
    if bitsSet%2 == 1 {
      total -= limit / prod
    } else {
      total += limit / prod
    }
  }
  return total
}
```

listing 1161: go, the derangement recurrence and the subset-sum coprime count with overflow skip (source: dsa/samples-go/ch35/incexc.go, lines 26-67)

```java
// derangements by the alternating sum !n = sum_k (-1)^k n!/k!
static long subfactorialSum(int v) {
  // term_k = n!/k!, computed as the falling product
  long s = 0;
  for (int k = 0; k ≤ v; k++) {
    long t = 1;
    for (int j = k + 1; j ≤ v; j++)
      t *= j;
    s += (k & 1) ≠ 0 ? -t : t;
  }
  return s;
}

// derangements by the recurrence
static long subfactorialRec(int v) {
  long d0 = 1, d1 = 0;
  if (v == 0)
    return 1;
  if (v == 1)
    return 0;
  for (int i = 2; i ≤ v; i++) {
    long d2 = (long) (i - 1) * (d1 + d0);
    d0 = d1;
    d1 = d2;
  }
  return d1;
}
```

listing 1162: java, the subfactorial sum and recurrence, the falling-product terms (source: dsa/samples-java/src/Ch35/Incexc.java, lines 45-71)

```
    /// <summary>!n = n! sum_k (-1)^k / k!, computed as the integer recurrence sum.</summary>
    public static long Derangement(int n)
    {
        long d0 = 1;
        long d1 = 0; // !0 = 1, !1 = 0
        if (n == 0)
            return d0;
        for (int i = 2; i ≤ n; i++)
        {
            long d2 = (i - 1) * (d1 + d0); // !i = (i-1)(!(i-1) + !(i-2))
            d0 = d1;
            d1 = d2;
        }
        return d1;
    }

    /// <summary>Integers in 1..n coprime to every prime in the set, by the alternating sum.</summary>
    public static long CoprimeCount(long n, long[] primes)
    {
        long total = 0;
        int m = primes.Length;
        for (int mask = 1; mask < 1 << m; mask++)
        {
            long prod = 1;
            int bits = 0;
            for (int i = 0; i < m; i++)
                if ((mask & (1 << i)) ≠ 0)
                {
                    prod *= primes[i];
                    bits++;
                }
            total += bits % 2 == 1 ? n / prod : -n / prod; // singles positive inside the parens
        }
        return n - total;
    }
```

listing 1163: c#, the derangement recurrence and the coprime mask sum over the ch28 mu (source: dsa/samples/src/Ch35/IncExc.cs, lines 12-46)

```
// !n = (n-1)(!(n-1) + !(n-2)), array form with !0 = 1, !1 = 0
export function subfactorialRec(m) {
  const d = [1, 0];
  for (let n = 2; n ≤ m; n++) {
    d.push((n - 1) * (d[n - 1] + d[n - 2]));
  }
  return d;
}

// integers in 1..m coprime to every prime in the set, legendre style
export function coprimeCount(m, primes) {
  let total = 0;
  for (let mask = 0; mask < 1 << primes.length; mask++) {
    let prod = 1;
    let bits = 0;
    for (let i = 0; i < primes.length; i++) {
      if ((mask >> i) & 1) {
        prod *= primes[i];
        bits += 1;
      }
    }
    total += (bits % 2 === 0 ? 1 : -1) * Math.floor(m / prod);
  }
  return total;
}
```

listing 1164: javascript, the recurrence array and the mask walk, mu imported from the ch28 sieve (source: dsa/samples-js/src/ch35-incexc.mjs, lines 30-54)

```
def subfactorial(m):
    # !m = sum_{i=0..m} (-1)^i m!/i!
    facts = [1]
    for i in range(1, m + 1):
        facts.append(facts[-1] * i)
    total = 0
    for i in range(m + 1):
        term = facts[m] // facts[i]
        total += -term if i % 2 else term
    return total


def derange_recurrence(m):
    # !m = (m-1) (!(m-1) + !(m-2))
    d = [1, 0]
    for k in range(2, m + 1):
        d.append((k - 1) * (d[k - 1] + d[k - 2]))
    return d[m]
```

listing 1165: python, the subfactorial sum and the recurrence, brute permutations below (source: dsa/samples-py/src/Ch35/incexc.py, lines 17-34)

```lua
-- derangement recurrence !n = (n-1)(!(n-1) + !(n-2))
local function derangements_rec(nmax)
  local d = {[1] = 1, [2] = 0} -- !0 = 1, !1 = 0
  for k = 2, nmax do
    d[k + 1] = (k - 1) * (d[k] + d[k - 1])
  end
  return d
end

-- the subfactorial sum: !n = n! sum (-1)^k / k, integer arithmetic by
-- alternating partial sums scaled back up
local function derangements_sum(n)
  local acc = 1
  for k = 1, n do
    acc = k * acc + (k % 2 == 0 and 1 or -1)
  end
  return acc
end
```

listing 1166: lua, the recurrence and the scaled alternating sum, the coprime mask below (source: dsa/samples-lua/ch35_incexc.lua, lines 7-24)

The fixtures pin each family. Derangements: !3 = 2, !4 = 9, !5 = 44 with the brute permutation count agreeing at 5, !6 = 265, !10 = 1334961, and the sum and recurrence equal for every n to 10. Coprime: integers in 1..100 sharing no factor with 2, 3, 5 = 26, the terms reading 100 - 50 - 33 - 20 + 16 + 10 + 6 - 3. Squarefree up to 100 = 61 by the mobius sum, brute-verified. The edges cover the empty prime set leaving all 100, a single prime leaving 100 minus 100 over p, and the exactly-r profile on two sets of sizes 20 and 15 sharing 8: union 27, A-only 12, B-only 7. The application is icpc world finals 2017 problem K (book 10, chapter 8), tarot sham boast, which ranks predictions by first-order inclusion-exclusion terms that turn out identical for all strings of one length, the differences coming only from self-overlaps.

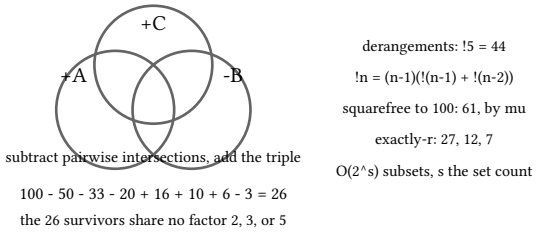


figure 296: three sets with plus and minus signs, above the 26 survivors of {2,3,5} in 1..100

35.5. burnside's lemma and polya enumeration

Counting configurations up to symmetry fails by direct division: dividing k^n colorings by n rotations counts orbits only when every orbit has exactly n members, which runs of a single color break. Burnside's lemma fixes the accounting the other way around: the number of orbits is the average, over the group, of the count of configurations each group element fixes. For necklaces under rotation, the rotation by i positions fixes exactly k to the gcd(n, i) colorings, the string repeating a block of that length, so the necklace count is the sum of k^gcd over all i, divided by n. Bracelets add the dihedral half: reflections through a bead when n is odd, n of them each fixing k^((n+1)/2), and the two axis families when n is even, (n/2) times the sum of k^(n/2+1) and k^(n/2), all divided by 2n. The cube face colorings ride a pinned cycle-index table of the 24 rotations.

The dry run: the fixtures are the necklace and bracelet anchors, asserted by the C# suite with brute canonical forms agreeing; the seven languages share the counts, lua by generating the rotation group itself.

1. The bracelet fork on (6, 2) starts from the rotation half, six rotations fixing 64, 2, 4, 8, 4, 2 colorings for the sum 84.

2. n = 6 splits its 6 reflections into two families: 3 axes through opposite beads hold 2 beads and pair the other 4, fixing 2^4 = 16 colorings each.
3. The 3 axes through opposite edges pair all 6 beads, fixing 2^3 = 8 each, so the reflection half reads 3 × 16 + 3 × 8 = 72.
4. The dihedral average divides by 2n: (84 + 72) / 12 = 13, one fewer than the necklaces' 14, mirror images now counted once.
5. The brute twin canonicalizes all 64 colorings by orbit minimum and counts the same 13.
6. The cube table reads its 24 rotations at k = 3: 1 × 729 + 6 × 27 + 3 × 81 + 8 × 9 + 6 × 27 = 1368, and 1368 / 24 = 57.

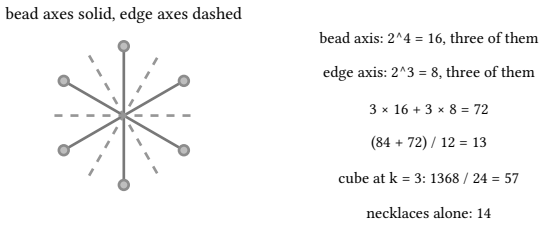


figure 297: the six reflection axes of the 6-bead ring, bead axes solid at 16 fixed each, edge axes dashed at 8

The 13 and the 57 land with their brute twins agreeing, and the listings below average the group in seven languages.

```c
static long long necklaces(int v, int k) {
  long long s = 0;
  for (int i = 0; i < v; i++)
    s += pw(k, (int)gcd_ll(v, i));
  return s / v;
}

static long long bracelets(int v, int k) {
  long long s = 0;
  for (int i = 0; i < v; i++)
    s += pw(k, (int)gcd_ll(v, i));
  if (v & 1)
    s += (long long)v * pw(k, (v + 1) / 2);
  else
    s += (long long)(v / 2) * (pw(k, v / 2 + 1) + pw(k, v / 2));
  return s / (2 * v);
}
```

listing 1167: c, the necklace sum by gcd classes, the bracelet reflection families (source: dsa/samples-c/src/Ch35/burnside.c, lines 37-53)

```
// Necklace is burnside over the rotation group:
// (1/n) sum_i k^gcd(n,i) by rotation classes.
func Necklace(n, k int, p int64) int64 {
  sum := int64(0)
  for i := 0; i < n; i++ {
    sum = (sum + PowMod(int64(k), int64(gcdInt(n, i)), p)) % p
  }
  return sum * invMod(int64(n), p) % p
}

// Bracelet adds the two reflection families to the necklace sum:
// n odd gives n * k^((n+1)/2), n even gives (n/2)(k^(n/2+1) +
// k^(n/2)).
func Bracelet(n, k int, p int64) int64 {
  var refl int64
  if n%2 == 1 {
    refl = int64(n) * PowMod(int64(k), int64((n+1)/2), p) % p
  } else {
    refl = int64(n/2) * (PowMod(int64(k), int64(n/2+1), p) +
      PowMod(int64(k), int64(n/2), p)) % p
  }
  rot := int64(0)
  for i := 0; i < n; i++ {
    rot = (rot + PowMod(int64(k), int64(gcdInt(n, i)), p)) % p
  }
  return (rot + refl) % p * invMod(int64(2*n), p) % p
}
```

listing 1168: go, both sums in modular arithmetic through the chapter tables (source: dsa/samples-go/ch35/burnside.go, lines 3-29)

```
  static long necklaces(int v, int k) {
    long s = 0;
    for (int i = 0; i < v; i++)
      s += pw(k, (int) gcdLl(v, i));
    return s / v;
  }

  static long bracelets(int v, int k) {
    long s = 0;
    for (int i = 0; i < v; i++)
      s += pw(k, (int) gcdLl(v, i));
    if ((v & 1) != 0)
      s += (long) v * pw(k, (v + 1) / 2);
    else
      s += (long) (v / 2) * (pw(k, v / 2 + 1) + pw(k, v / 2));
    return s / (2L * v);
  }
```

listing 1169: java, the necklace sum by gcd classes, the bracelet reflection families (source: dsa/samples-java/src/Ch35/Burnside.java, lines 36-52)

```
    public static long Necklaces(int n, long k)
    {
        var gcdCounts = new Dictionary<long, long>();
        for (int i = 0; i < n; i++)
        {
            long g = NumTheory.Gcd(n, i);
            gcdCounts[g] = gcdCounts.GetValueOrDefault(g) + 1;
        }
        return gcdCounts.Sum(kv => kv.Value * Pow(k, kv.Key)) / n;
    }

    public static long Bracelets(int n, long k)
    {
        long rotations = Necklaces(n, k) * n; // the rotation class sum
        long reflections = n % 2 == 1
            ? n * Pow(k, (n + 1) / 2) // odd: one fixed bead per axis
            : n / 2 * (Pow(k, n / 2) + Pow(k, n / 2 + 1)); // even: the two axis families
        return (rotations + reflections) / (2 * n);
    }
```

listing 1170: c#, the necklace sum folded by gcd counts, the bracelet fork (source: dsa/samples/src/Ch35/Burnside.cs, lines 13-31)

```
export function necklaces(n, k) {
  let sum = 0;
  for (let i = 0; i < n; i++) {
    sum += ipow(k, gcd(n, i));
  }
  return sum / n;
}

export function bracelets(n, k) {
  let rot = 0;
  for (let i = 0; i < n; i++) {
    rot += ipow(k, gcd(n, i));
  }
  const refl = n % 2 === 1
    ? n * ipow(k, (n + 1) / 2)
    : (n / 2) * (ipow(k, n / 2 + 1) + ipow(k, n / 2));
  return (rot + refl) / (2 * n);
}
```

listing 1171: javascript, both sums over plain integer powers (source: dsa/samples-js/src/ch35-burnside.mjs, lines 18-35)

```
def necklaces(beads, colors):
    # (1/n) sum_i k^gcd(n, i)
    total = 0
    for i in range(beads):
        total += colors ** math.gcd(beads, i)
    return total // beads


def bracelets(beads, colors):
    total = necklaces(beads, colors) * beads
    if beads % 2:  # odd: reflections fix one bead
        total += beads * colors ** ((beads + 1) // 2)
    else:  # even: two reflection families
        half = beads // 2
        total += half * (colors ** (half + 1) + colors**half)
    return total // (2 * beads)
```

listing 1172: python, the two sums, the cube table below (source: dsa/samples-py/src/Ch35/burnside.py, lines 17-32)

```
local function necklaces(n, k)
  local total = 0
  for i = 0, n - 1 do
    total = total + intpow(k, gcd(n, i))
  end
  return total // n
end

local function bracelets(n, k)
  local total = 0
  for i = 0, n - 1 do
    total = total + intpow(k, gcd(n, i))
  end
  if n % 2 == 1 then
    total = total + n * intpow(k, (n + 1) // 2) -- reflections on a bead
  else
    total = total + (n // 2) * (intpow(k, n // 2 + 1) + intpow(k, n // 2))
  end
  return total // (2 * n)
end
```

listing 1173: lua, the two sums; the lua suite generates the actual 24-rotation group and counts fixed colorings directly (source: dsa/samples-lua/ch35_burnside.lua, lines 19-38)

The fixtures pin the orbits and brute-check them by canonical form, the minimum over each configuration's orbit. Necklaces: (6,2) = 14, (4,3) = 24, (5,2) = 8, (12,2) = 352, with the canonical-rotation brute agreeing on the first three. Bracelets, reflections included in the orbit: (6,2) = 13, (7,2) = 18, (5,3) = 39, again brute-checked. The cube under its 24 face rotations colors in 10 ways with 2 colors and 57 with 3, every language carrying the same cycle-index table, and lua verifying it the honest way, closing a quarter face turn and a vertex turn under composition until exactly 24 permutations exist, then counting fixed colorings per rotation over all k^6 colorings. The edges read the one-bead necklace as k and (2,2) as 3. General technique, no finals problem in the mined set needed it.

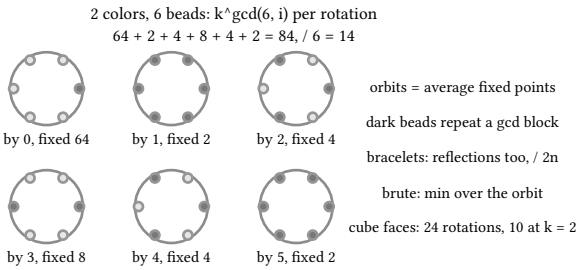


figure 298: the six rotations of a 6-bead ring with fixed counts 64, 2, 4, 8, 4, 2 averaging to 14

35.6. generating k-subsets

Iteration is the quiet half of counting: the solver that proves a count often also needs the objects. Lexicographic k-combinations are sorted 0-based index tuples, and next-combination advances one step: scan from the right for the first index below its maximum, n - k + i, increment it, and reset everything after it to consecutive integers. When no index can advance the walk wraps and the tuple (3,4,5)-style ceiling has been reached. Each step is O(k) and the walk visits all C(n, k) tuples in lexicographic order.

The dry run: the fixture is the 20-tuple walk with its transitions and the k = 6 office strings, asserted by the C# suite.

1. The walk opens (0,1,2), (0,1,3), (0,1,4), (0,1,5): index 2 alone climbs toward its maximum 5 while the frozen prefix holds.
2. (0,2,4) climbs index 2 once more, 4 to 5, landing (0,2,5).
3. (0,2,5) finds index 2 at its maximum, so index 1 climbs 2 to 3 and the tail resets to consecutive: (0,3,4).
4. (0,4,5) is the frozen-prefix extreme: index 0 climbs 0 to 1 and the whole tail resets, (1,2,3).
5. (3,4,5) has no mobile index, and the walk wraps after 20 tuples, C(6,3) = 20.
6. The office-string walk at k = 6 turns (0,1,2) into 011100, ones at bits 1, 2, 3 after the leading 0, and ten strings later (2,3,4) is
   111.
7. The search closes at n = 10: C(5,3) = 10 first reaches it, so the smallest k is 6.

tuple	mobile index	action	next
(0,2,4)	i = 2	4 climbs to 5	(0,2,5)
(0,2,5)	i = 1	2 climbs, tail resets	(0,3,4)
(0,4,5)	i = 0	0 climbs, tail resets	(1,2,3)
(3,4,5)	none	wrap after 20	

The 20, the reset, and the ten strings are the pinned landings, and the listings below run the walk in seven languages.

```c
static int next_comb(int *c, int k, int nn) {
  int i = k - 1;
  while (i ≥ 0 && c[i] == nn - k + i)
    i--;
  if (i < 0)
    return 0;
  c[i]++;
  for (int j = i + 1; j < k; j++)
    c[j] = c[j - 1] + 1;
  return 1;
}

// the 2023/B string: ones at string bits c[j] + 1
static void emit_from_comb(const int *c, int k, int ones, char *out) {
  out[0] = '0';
  for (int i = 1; i < k; i++) {
    int on = 0;
    for (int j = 0; j < ones; j++)
      if (c[j] + 1 == i)
        on = 1;
    out[i] = on ? '1' : '0';
  }
  out[k] = 0;
}
```

listing 1174: c, next-combination, the emit turning a tuple into the office string (source: dsa/samples-c/src/Ch35/ksubset.c, lines 55-78)

```
func NextCombination(cur []int, n int) ([]int, bool) {
  k := len(cur)
  next := append([]int(nil), cur...)
  i := k - 1
  for i ≥ 0 && next[i] == n-k+i {
    i--
  }
  if i < 0 {
    return nil, false
  }
  next[i]++
  for j := i + 1; j < k; j++ {
    next[j] = next[j-1] + 1
  }
  return next, true
}

// OfficeStrings is the 2023/B construction: length-k binary strings
// with first bit 0 and exactly ceil(k/2) ones among the remaining
// k-1 bits, emitted largest-first. Descending string order is
// ascending lexicographic order of the one-position combinations, so
// one walk of NextCombination from the first combination emits the
// strings from the top.
func OfficeStrings(k int) []string {
  c := (k + 1) / 2
  m := k - 1
  cur := make([]int, c)
  for i := range cur {
    cur[i] = i
  }
  var out []string
  for {
    b := []byte("0" + strings.Repeat("0", m))
    for _, p := range cur {
      b[1+p] = '1'
    }
    out = append(out, string(b))
    next, ok := NextCombination(cur, m)
    if !ok {
      return out
    }
    cur = next
  }
}
```

listing 1175: go, the advance and the string walk over it (source: dsa/samples-go/ch35/ksubset.go, lines 9-52)

```java
// advance the sorted tuple; false on wrap
static boolean nextComb(int[] c, int k, int nn) {
  int i = k - 1;
  while (i ≥ 0 && c[i] == nn - k + i)
    i--;
  if (i < 0)
    return false;
  c[i]++;
  for (int j = i + 1; j < k; j++)
    c[j] = c[j - 1] + 1;
  return true;
}

// the 2023/B string: ones at string bits c[j] + 1
static String emitFromComb(int[] c, int k, int ones) {
  StringBuilder out = new StringBuilder();
  out.append('0');
  for (int i = 1; i < k; i++) {
    boolean on = false;
    for (int j = 0; j < ones; j++)
      if (c[j] + 1 == i)
        on = true;
    out.append(on ? '1' : '0');
  }
  return out.toString();
}
```

listing 1176: java, nextComb, the emit turning a tuple into the office string (source: dsa/samples-java/src/Ch35/Ksubset.java, lines 52-77)

```
    public static bool Next(int[] s, int n)
    {
        int k = s.Length;
        if (k == 0)
            return false;
        int i = k - 1;
        while (i ≥ 0 && s[i] == n - k + i)
            i--; // find the rightmost index that can advance
        if (i < 0)
            return false; // wrapped past the last subset
        s[i]++;
        for (int j = i + 1; j < k; j++)
            s[j] = s[j - 1] + 1; // reset the rest to consecutive
        return true;
    }

    /// <summary>Length-k binary strings with first bit 0 and ceil(k/2) ones among the
    /// remaining k-1 bits, emitted in descending lexicographic order exactly as
    /// 2023/B schedules its offices: the ascending subset walk puts ones further
    /// left first, which is the larger string.</summary>
    public static IEnumerable<string> OfficeStrings(int k)
    {
        int ones = (k + 1) / 2;
        int m = k - 1; // choose the positions of the ones after the leading 0
        var cur = Enumerable.Range(0, ones).ToArray();
        while (true)
        {
            var bits = new char[k];
            Array.Fill(bits, '0');
            foreach (int p in cur)
                bits[p + 1] = '1'; // offset by the leading 0
            yield return new string(bits);
            if (!Next(cur, m))
                yield break;
        }
    }
```

listing 1177: c#, next, and the office string generator yielding largest-first (source: dsa/samples/src/Ch35/KSubset.cs, lines 13-48)

```
export function nextCombination(t, n) {
  const k = t.length;
  const next = [...t];
  let i = k - 1;
  while (i ≥ 0 && next[i] === n - k + i) i -= 1;
  if (i < 0) return null; // wrap: the walk is exhausted
  next[i] += 1;
  for (let j = i + 1; j < k; j++) next[j] = next[j - 1] + 1;
  return next;
}

export function combinationStrings2023B(k) {
  const ones = Math.ceil(k / 2);
  let t = firstCombination(ones); // one-positions in the k-1 bits
  const out = [];
  for (;;) {
    const bits = new Array(k - 1).fill("0");
    for (const p of t) bits[p] = "1";
    out.push("0" + bits.join(""));
    const nx = nextCombination(t, k - 1);
    if (nx === null) break;
    t = nx;
  }
  return out;
}
```

listing 1178: javascript, the advance, the 2023/B string family, no backwards walk (source: dsa/samples-js/src/ch35-ksubset.mjs, lines 16-40)

```
def next_comb(cur, n_):
    # advance in place or None on wrap
    k = len(cur)
    out = list(cur)
    i = k - 1
    while i ≥ 0 and out[i] == n_ - k + i:  # rightmost mobile index
        i -= 1
    if i < 0:
        return None
    out[i] += 1
    for j in range(i + 1, k):  # reset the rest to consecutive
        out[j] = out[j - 1] + 1
    return tuple(out)


def walk(cur, n_):
    out = [tuple(cur)]
    while True:
        cur = next_comb(cur, n_)
        if cur is None:
            return out
        out.append(cur)
```

listing 1179: python, next-combination and the walk collecting tuples (source: dsa/samples-py/src/Ch35/ksubset.py, lines 15-36)

```
local function next_comb(c, n)
  local k = #c
  local i = k
  while i ≥ 1 and c[i] == n - k + i - 1 do -- slot already at its max
    i = i - 1
  end
  if i == 0 then return nil end -- wrap signals exhaustion
  c[i] = c[i] + 1
  for j = i + 1, k do c[j] = c[j - 1] + 1 end
  return c
end

-- the 2023/B construction at length k: first bit 0, ceil(k/2) ones
-- among the remaining k-1 bits, emitted in descending lexicographic
-- order: the smallest one-position tuple puts the ones earliest and so
-- yields the largest string, and ascending tuples keep the strings
-- descending, exactly walking next-combination backwards over strings
local function office_strings(k)
  local ones = math.ceil(k / 2)
  local rest = k - 1
  local out = {}
  local c = {}
  for i = 1, ones do c[i] = i - 1 end -- the smallest tuple
  while c do
    local bits = {"0"}
    local set = {}
    for _, v in ipairs(c) do set[v + 1] = true end
    for pos = 0, rest - 1 do
      bits[#bits + 1] = set[pos + 1] and "1" or "0"
    end
    out[#out + 1] = table.concat(bits)
    c = next_comb(c, rest)
  end
  return out
end
```

listing 1180: lua, the advance and the office strings emitted descending (source: dsa/samples-lua/ch35_ksubset.lua, lines 9-43)

The 2023/B construction rides the same walk. Icpc world finals 2023 problem B (book 10, chapter 12), schedule, builds its candidate strings as length-k binary strings with first bit 0 and exactly ceil(k/2) ones among the remaining k - 1 bits, and it needs the n largest compatible strings. The walk over the one-positions emits exactly that: ascending index-tuple order places earlier one-positions first, and earlier ones lex to larger strings, so one forward walk emits the strings largest-first with no backwards traversal. At k = 6 the ten strings pin as 011100, 011010, 011001, 010110, 010101, 010011, 001110, 001101, 001011, 000111.

The walk fixtures pin 3-subsets of 6: 20 in all, opening (0,1,2), (0,1,3), (0,1,4), (0,1,5), (0,2,3), closing at (3,4,5), with the transitions (0,2,4) to (0,2,5) to (0,3,4) showing the reset, and (0,4,5) jumping to (1,2,3). The edges read k = 0 as the one empty subset that wraps immediately, k = n as the full set wrapping the same way, and the search answers the smallest k with C(k-1, ceil(k/2)) at least n: 4, 4, 6 for n = 2, 3, 10, the feasibility threshold 2023/B actually solves for.

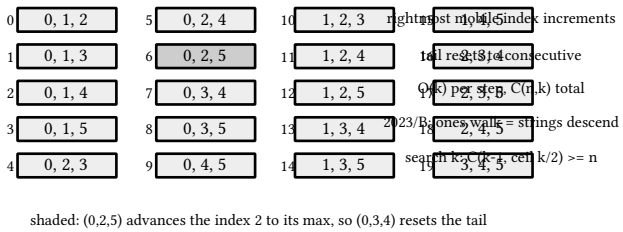


figure 299: the 20 3-subsets of 6 as a lex ladder, the (0,2,5) to (0,3,4) reset step highlighted

35.7. sperner's theorem and antichains

An antichain of subsets is a family where no member contains another. Sperner's theorem, 1928, caps every antichain of the n-set at C(n, floor n/2): the middle layer, all subsets of that one size, is a maximum antichain, since equal sizes are automatically incomparable and nothing larger is possible. The proof the book states without carrying is the LYM inequality, which weights each set by one over C(n, its size) and shows the weights sum to at most 1, a budget the middle layer spends exactly. The dedekind number M(n) flips the question and counts the antichains themselves, bruted here over all 2^(2^n) families with pairwise comparability checks, feasible through n = 4′s 65536 families.

The dry run: the fixtures are the middle-layer row and the dedekind brute to n = 4, asserted by the C# suite; the six families of the n = 2 walk below are hand-derived off the pinned 6.

1. The middle layers read C(n, n/2) for n = 1 through 8: 1, 2, 3, 6, 10, 20, 35, 70, and the n = 4 layer's 6 sets assert pairwise incomparable.
2. The dedekind brute at n = 2 scans all 2^4 = 16 families over the four subsets of a 2-set.
3. Hand-walking the 16: the empty family, the four single-set families, and the pair of singletons {1}, {2} survive, 6 in all.
4. Every other family dies by one containment: the empty set sits inside anything, and {1} or {2} sits inside {1, 2}.
5. n = 3 scans 256 families to 20, n = 4 scans 65536 to 168, so M(0..4) = 2, 3, 6, 20, 168, with M(5) = 7581 quoted as literature.
6. The counterexample chain, the empty set inside {1}, fails the antichain test by exactly one containment.

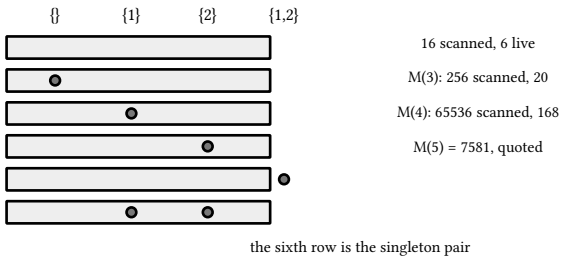


figure 300: the 16 families of a 2-set cut to 6 antichains, one dot per subset held

The 6, the row 2, 3, 6, 20, 168, and the 6-set layer are the pinned landings, and the listings below run the brute in seven languages.

```c
static long long dedekind(int v) {
  int subsets = 1 << v;              // the v-set's powerset members as bitmasks
  long long families = 1 << subsets; // every family is a bitmask over it
  long long count = 0;
  for (long long fam = 0; fam < families; fam++) {
    int ok = 1;
    for (int a = 0; a < subsets && ok; a++)
      if (fam & (1LL << a))
        for (int b = 0; b < subsets && ok; b++)
          if (b != a && (fam & (1LL << b)))
            if ((a & b) == a || (a & b) == b) // one contains the other
              ok = 0;
    if (ok)
      count++;
  }
  return count;
}

// the middle layer of size C(v, v/2)
static long long middle_layer(int v) {
  long long c = 0;
  for (unsigned mask = 0; mask < (1u << v); mask++)
    if (popcount32(mask) == v / 2)
      c++;
  return c;
}

// pairwise incomparability of a list of masks
static int is_antichain(const int *masks, int k) {
  for (int i = 0; i < k; i++)
    for (int j = i + 1; j < k; j++) {
      int inter = masks[i] & masks[j];
      if (inter == masks[i] || inter == masks[j])
        return 0;
    }
  return 1;
}
```

listing 1181: c, the dedekind brute over family bitmasks, the middle layer, the antichain test (source: dsa/samples-c/src/Ch35/sperner.c, lines 32-68)

```
func isAntichain(sets []map[int]bool) bool {
  for i := range sets {
    for j := i + 1; j < len(sets); j++ {
      if subset(sets[i], sets[j]) || subset(sets[j], sets[i]) {
        return false
      }
    }
  }
  return true
}

// Dedekind is M(n), the number of antichains of the n-set, by brute
// over all 2^(2^n) families with pairwise comparability checks,
// feasible to n = 4. M(5) is the stated literature constant.
func Dedekind(n int) int64 {
  subsets := 1 << n
  count := int64(0)
  for family := 0; family < 1<<subsets; family++ {
    var sets []map[int]bool
    for s := 0; s < subsets; s++ {
      if family&(1<<s) == 0 {
        continue
      }
      m := map[int]bool{}
      for b := 0; b < n; b++ {
        if s&(1<<b) != 0 {
          m[b] = true
        }
      }
      sets = append(sets, m)
    }
    if isAntichain(sets) {
      count++
    }
  }
  return count
}
```

listing 1182: go, the antichain test and the dedekind brute over set maps (source: dsa/samples-go/ch35/sperner.go, lines 36-72)

```
// dedekind M(v): count families (subsets of the powerset) that are
// antichains, including the empty family
static long dedekind(int v) {
  int subsets = 1 << v; // the v-set's powerset members as bitmasks
  long families = 1L << subsets; // every family is a bitmask over it
  long count = 0;
  for (long fam = 0; fam < families; fam++) {
    boolean good = true;
    for (int a = 0; a < subsets && good; a++)
      if ((fam & (1L << a)) != 0)
        for (int b = 0; b < subsets && good; b++)
          if (b != a && (fam & (1L << b)) != 0)
            if ((a & b) == a || (a & b) == b) // one contains the other
              good = false;
    if (good)
      count++;
  }
  return count;
}

// the middle layer of size C(v, v/2)
static long middleLayer(int v) {
  long c = 0;
  for (int mask = 0; mask < (1 << v); mask++)
    if (popcount32(mask) == v / 2)
      c++;
  return c;
}

// pairwise incomparability of a list of masks
static boolean isAntichain(int[] masks, int k) {
  for (int i = 0; i < k; i++)
    for (int j = i + 1; j < k; j++) {
      int inter = masks[i] & masks[j];
      if (inter == masks[i] || inter == masks[j])
        return false;
    }
  return true;
}
```

listing 1183: java, the dedekind brute over family bitmasks, the middle layer, the antichain test (source: dsa/samples-java/src/Ch35/Sperner.java, lines 24-62)

```
    public static long MiddleLayerSize(int n) => Binom.Exact(n, n / 2);

    /// <summary>All subsets of size floor(n/2) as sorted element lists.</summary>
    public static long[][] MiddleLayer(int n)
    {
        int k = n / 2;
        var cur = Enumerable.Range(0, k).ToArray();
        var layer = new List<long[]>();
        while (true)
        {
            layer.Add(cur.Select(i => (long)i).ToArray());
            if (!KSubset.Next(cur, n))
                return layer.ToArray();
        }
    }

    /// <summary>Antichain check: no member is a subset of another.</summary>
    public static bool IsAntichain(long[][] family)
    {
        var masks = family.Select(s => s.Aggregate(0L, (acc, x) => acc | 1L << (int)x)).ToArray();
        for (int i = 0; i < masks.Length; i++)
            for (int j = i + 1; j < masks.Length; j++)
                if ((masks[i] & masks[j]) == masks[i] || (masks[i] & masks[j]) == masks[j])
                    return false;
        return true;
    }

    /// <summary>Brute over all 2^(2^n) families with pairwise comparability; n &lt;= 4.</summary>
    public static long Dedekind(int n)
    {
        if (n < 0 || n > 4)
            throw new ArgumentOutOfRangeException(nameof(n), "brute dedekind numbers stop at n = 4");
        int subsets = 1 << n;
        long families = 0;
        for (long fam = 0; fam < 1L << subsets; fam++)
        {
            bool antichain = true;
            for (int a = 0; a < subsets && antichain; a++)
                if ((fam & (1L << a)) != 0)
                    for (int b = 0; b < subsets && antichain; b++)
                        if (b != a && (fam & (1L << b)) != 0 && (a & b) == a && a != b)
                            antichain = false; // a subset of b with a != b
            if (antichain)
                families++;
        }
        return families;
    }
```

listing 1184: c#, the layer walk reusing the k-subset iterator, the dedekind brute refusing past n = 4 (source: dsa/samples/src/Ch35/Sperner.cs, lines 11-57)

```
export function isAntichain(family) {
  for (let i = 0; i < family.length; i++) {
    for (let j = i + 1; j < family.length; j++) {
      const and = family[i] & family[j];
      if (and === family[i] || and === family[j]) return false;
    }
  }
  return true;
}

export function dedekind(n) {
  const count = 1 << n;
  let total = 0;
  for (let fam = 0; fam < 1 << count; fam++) {
    const sets = [];
    for (let s = 0; s < count; s++) {
      if ((fam >> s) & 1) sets.push(s);
    }
    if (isAntichain(sets)) total += 1;
  }
  return total;
}
```

listing 1185: javascript, the bitwise antichain test and the dedekind brute (source: dsa/samples-js/src/ch35-sperner.mjs, lines 13-34)

```
def middle_layer(n_):
    # all subsets of size floor(n/2), as bitmasks
    size = n_ // 2
    return [mask for mask in range(1 << n_) if bin(mask).count("1") == size]


def incomparable(family):
    # pairwise: no member is a proper subset of another
    for a, b in itertools.combinations(family, 2):
        if a & b == a or a & b == b:
            return False
    return True


def dedekind(n_):
    # number of antichains of the n_-set = families of pairwise
    # incomparable subsets, bruted over all 2^(2^n) families
    subsets = list(range(1 << n_))
    count = 0
    for mask in range(1 << (1 << n_)):
        family = [s for i, s in enumerate(subsets) if mask >> i & 1]
        if incomparable(family):
            count += 1
    return count
```

listing 1186: python, the layer, the incomparability test, the dedekind brute (source: dsa/samples-py/src/Ch35/sperner.py, lines 16-39)

```lua
local function middle_layer(n)
  local layer = {}
  local half = n // 2
  for mask = 0, (1 << n) - 1 do
    local bits = 0
    for b = 0, n - 1 do bits = bits + ((mask >> b) & 1) end
    if bits == half then layer[#layer + 1] = mask end
  end
  return layer
end

-- is the family (list of bitmasks) an antichain?
local function is_antichain(family)
  for i = 1, #family do
    for j = i + 1, #family do
      local a, b = family[i], family[j]
      if a & b == a or a & b == b then return false end -- comparable
    end
  end
  return true
end

-- dedekind M(n): count antichain families over all 2^(2^n) subsets of
-- the powerset. each family is a bitmask over the 2^n subsets
local function dedekind(n)
  local npow = 1 << n
  local subsets = {}
  for mask = 0, npow - 1 do subsets[#subsets + 1] = mask end
  local count = 0
  for fam = 0, (1 << npow) - 1 do
    local family = {}
    for i = 0, npow - 1 do
      if (fam >> i) & 1 == 1 then family[#family + 1] = subsets[i + 1] end
    end
    if is_antichain(family) then count = count + 1 end
  end
  return count
end
```

listing 1187: lua, the same three pieces in 1-based masks (source: dsa/samples-lua/ch35_sperner.lua, lines 18-55)

The fixtures pin the layers and the counts. C(n, floor n/2) for n = 1 through 8 reads 1, 2, 3, 6, 10, 20, 35, 70, and the n = 4 middle layer's 6 sets assert pairwise incomparable. Dedekind M(0..4) = 2, 3, 6, 20, 168 by the brute, where M(0) = 2 counts the empty family and the family holding only the empty set and n = 1's singletons lift M(1) to 3. M(5) = 7581 is stated as the literature constant and never asserted as a derived number, the brute being 2^32 families out of sample reach. The pinned counterexample reads the chain of the empty set inside {1} as not an antichain. The application is the optimality half of icpc world finals 2023 problem B (book 10, chapter 12): its compatibility condition makes the strings an antichain, so C(k-1, ceil(k/2)) is not just reachable but the maximum, which is why the search of the previous section is exactly optimal.

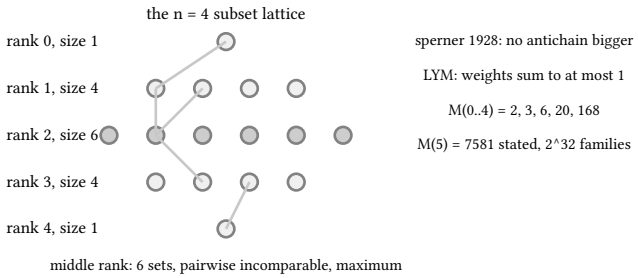


figure 301: the boolean lattice of n = 4 in five rank rows, the middle row of 6 shaded as the maximum antichain

35.8. the frobenius problem

Given coin denominations with gcd 1, which totals cannot be bought. The largest is the frobenius number, and two generators close by formula, F = ab - a - b with genus (a-1)(b-1) / 2 and every larger total reachable. General k has no closed form, and the sample answer is a reachability sieve: mark 0, then walk values ascending, marking v reachable when v minus some generator is, and stop at a cap chosen past the conductor, p max squared plus p max , the classical schur-type bound that settles every value beyond it. The gaps read off as the unmarked values, the largest is F, the conductor is F + 1, and gcd above 1 refuses outright since the gaps are then infinite.

The dry run: the fixture is the (3,5) pair, asserted by the C# suite with the pair formula cross-checked against the sieve.

1. The sieve seeds 0 reachable and walks up from the smallest generator, marking v when v minus some generator is marked.
2. v = 1 and v = 2 find nothing below them marked and stay gaps; v = 3 reads 3 - 3 = 0 and marks.
3. v = 4 reads 4 - 3 = 1, unmarked, and stays a gap; v = 5 reads 5 - 5 = 0 and marks.
4. v = 6 chains off 3, v = 7 reads 7 - 3 = 4 and 7 - 5 = 2, both gaps, and dies as the largest gap.
5. v = 8 reads 8 - 3 = 5 marked and marks, and from there every value chains: 9 - 3 = 6, 10 - 5 = 5, 11 - 3 = 8.
6. The gaps read 1, 2, 4, 7, so F = 7, conductor 8, genus 4, and the pair formula agrees, 15 - 3 - 5 = 7 with (3-1)(5-1)/2 = 4.

v	v - 3	v - 5	verdict
1	-	-	gap
2	-	-	gap
3	0 marked	-	mark
4	1 gap	-	gap
5	2 gap	0 marked	mark
6	3 marked	1 gap	mark
7	4 gap	2 gap	gap
8	5 marked	3 marked	mark

The 7, the four gaps, and the conductor 8 are the pinned landings, and the listings below run the sieve in seven languages.

```
static int semigroup(const int *p, int k, int *gap_count, int *frob,
                     int *conductor) {
  long long g = 0;
  int pmax = 0;
  for (int i = 0; i < k; i++) {
    g = gcd_ll(g, p[i]);
    if (p[i] > pmax)
      pmax = p[i];
  }
  if (g != 1)
    return -1;                  // infinitely many gaps
  int cap = pmax * pmax + pmax; // covers every fixture's conductor
  for (int v = 0; v <= cap; v++)
    reach[v] = 0;
  reach[0] = 1;
  for (int v = 1; v <= cap; v++)
    for (int i = 0; i < k; i++)
      if (p[i] <= v && reach[v - p[i]]) {
        reach[v] = 1;
        break;
      }
  int gc = 0;
  for (int v = 1; v <= cap; v++)
    if (!reach[v])
      gaps[gc++] = v;
  // the conductor: gaps stop before the trailing run of reachables
  *gap_count = gc;
  if (gc == 0) { // generator 1: everything reachable, refuse by -1 report
    *frob = -1;
    *conductor = 0;
  } else {
    *frob = gaps[gc - 1];
    *conductor = *frob + 1;
  }
  return gc;
}
```

listing 1188: c, the sieve with the cap, the gap scan, the conductor (source: dsa/samples-c/src/Ch35/frobenius.c, lines 37-72)

```
func Frobenius(coins []int64) (int64, []int64, error) {
  g := coins[0]
  for _, c := range coins[1:] {
    g = gcdI64(g, c)
  }
  if g > 1 {
    return 0, nil, fmt.Errorf(
      "gcd %d > 1: infinitely many gaps", g)
  }
  pmax := coins[0]
  for _, c := range coins {
    if c > pmax {
      pmax = c
    }
  }
  cap := pmax*pmax + pmax
  reach := make([]bool, cap+1)
  reach[0] = true
  for _, c := range coins {
    for v := c; v <= cap; v++ {
      if reach[v-c] {
        reach[v] = true
      }
    }
  }
  f := int64(-1)
  for v := cap; v ≥ 0; v-- {
    if !reach[v] {
      f = int64(v)
      break
    }
  }
  var gaps []int64
  for v := int64(0); v <= f; v++ {
    if !reach[v] {
      gaps = append(gaps, v)
    }
  }
  return f, gaps, nil
}
```

listing 1189: go, the closed form, the sieve, the gap list (source: dsa/samples-go/ch35/frobenius.go, lines 14-53)

```
  // sieve the semigroup; returns the gap count, -1 when gcd > 1
  static int semigroup(int[] p, int k) {
    long g = 0;
    int pmax = 0;
    for (int i = 0; i < k; i++) {
      g = gcdLl(g, p[i]);
      if (p[i] > pmax)
        pmax = p[i];
    }
    if (g != 1)
      return -1; // infinitely many gaps
    int cap = pmax * pmax + pmax; // covers every fixture's conductor
    for (int v = 0; v ≤ cap; v++)
      reach[v] = 0;
    reach[0] = 1;
    for (int v = 1; v ≤ cap; v++)
      for (int i = 0; i < k; i++)
        if (p[i] ≤ v && reach[v - p[i]] != 0) {
          reach[v] = 1;
          break;
        }
    int gc = 0;
    for (int v = 1; v ≤ cap; v++)
      if (reach[v] == 0)
        gaps[gc++] = v;
    // the conductor: gaps stop before the trailing run of reachables
    gapCount = gc;
    if (gc == 0) { // generator 1: everything reachable, F reported -1
      frob = -1;
      cond = 0;
    } else {
      frob = gaps[gc - 1];
      cond = frob + 1;
    }
    return gc;
  }
```

listing 1190: java, the sieve with the cap, the gap scan, the conductor (source: dsa/samples-java/src/Ch35/Frobenius.java, lines 36-71)

```
    public static Result? Solve(params long[] generators)
    {
        long g = 0;
        foreach (long p in generators)
            g = NumTheory.Gcd(g, p);
        if (g > 1)
            return null; // infinitely many gaps
        long pMax = generators.Max();
        long cap = pMax * pMax + pMax; // every conductor of the family sits under this
        var reach = new bool[cap + 1];
        reach[0] = true;
        for (long v = generators.Min(); v ≤ cap; v++)
            foreach (long p in generators)
                if (v ≥ p && reach[v - p])
                    reach[v] = true;
        var gaps = new List<long>();
        for (long v = 0; v ≤ cap; v++)
            if (!reach[v])
                gaps.Add(v);
        if (generators.Contains(1L) || gaps.Count == 0)
            return new Result(-1, gaps, gaps.Count, gaps.Count == 0 ? 0 : gaps[^1] + 1);
        return new Result(gaps[^1], gaps, gaps.Count, gaps[^1] + 1);
    }
```

listing 1191: c#, the same sieve returning the record (source: dsa/samples/src/Ch35/Frobenius.cs, lines 14-36)

```javascript
export function frobenius(gens) {
  const g = gens.reduce((a, b) => gcd(a, b));
  if (g !== 1) {
    throw new RangeError("gcd > 1: infinitely many gaps");
  }
  if (gens.includes(1)) {
    return { frobenius: -1, gaps: [], gapCount: 0, conductor: 0, genus: 0 };
  }
  const pmax = Math.max(...gens);
  const cap = pmax * pmax + pmax; // covers every fixture's conductor
  const reach = new Array(cap + 1).fill(false);
  reach[0] = true;
  for (let v = 1; v <= cap; v++) {
    for (const p of gens) {
      if (p <= v && reach[v - p]) {
        reach[v] = true;
        break;
      }
    }
  }
  const gaps = [];
  for (let v = 0; v <= cap; v++) {
    if (!reach[v]) gaps.push(v);
  }
  const frob = gaps[gaps.length - 1];
  return {
    frobenius: frob,
    gaps,
    gapCount: gaps.length,
    conductor: frob + 1,
    genus: gaps.length,
  };
}
```

listing 1192: javascript, the sieve with the refuses and the −1 convention (source: dsa/samples-js/src/ch35-frobenius.mjs, lines 15-47)

```python
def semigroup(gens):
    # returns (frobenius, gaps, conductor); gcd > 1 refuses
    if math.gcd(*gens) != 1:
        raise ValueError("gcd > 1: infinitely many gaps")
    if 1 in gens:
        return -1, [], 0  # everything reachable: labeled convention
    pmax = max(gens)
    cap = pmax * pmax + pmax  # the sieve bound, asserted below
    reach = [False] * (cap + 1)
    reach[0] = True
    for v in range(1, cap + 1):  # ascending order: one pass suffices
        for g in gens:
            if v >= g and reach[v - g]:
                reach[v] = True
                break
    gaps = [v for v in range(1, cap + 1) if not reach[v]]
    frob = max(gaps)
    return frob, gaps, frob + 1
```

listing 1193: python, the sieve raising on gcd, the −1 convention for generator 1 (source: dsa/samples-py/src/Ch35/frobenius.py, lines 17-34)

```lua
local function frobenius(gens)
  local g = 0
  for _, x in ipairs(gens) do g = gcd(g, x) end
  if g ~= 1 then error("gcd > 1: infinitely many gaps") end
  if gens[1] == 1 or #gens == 0 then return -1, {}, 0, 0 end
  local pmax = 0
  for _, x in ipairs(gens) do pmax = math.max(pmax, x) end
  local cap = pmax * pmax + pmax -- covers every fixture's conductor
  local reach = {[0] = true}
  for v = 1, cap do
    for _, x in ipairs(gens) do
      if v ≥ x and reach[v - x] then reach[v] = true end
    end
  end
  local gaps = {}
  for v = 1, cap do
    if not reach[v] then gaps[#gaps + 1] = v end
  end
  local frob = gaps[#gaps]
  return frob, gaps, #gaps, frob + 1
end
```

listing 1194: lua, the same sieve, errors on gcd, conductor bound asserted in the suite (source: dsa/samples-lua/ch35_frobenius.lua, lines 14-34)

The fixtures walk the families. Pairs: frobenius(3,5) = 7 = 15 - 8 with gaps 1, 2, 4, 7 and genus 4 = (3-1)(5-1)/2; frobenius(4,7) = 17 with 9 gaps and conductor 18; frobenius(5,8) = 27 by the closed form against the sieve. The mcnugget system (6, 9, 20) pins F = 43, 22 gaps, conductor 44, the gap list itself pinned digit for digit. The edges refuse gcd(4,6) = 2 loudly, report −1 when a generator is 1 and everything is reachable, and assert the bound: every fixture's conductor sits under p max squared plus p max . The application is icpc world finals 2025 problem H (book 10, chapter 13), score values, whose reachable scores are exactly this semigroup capped at the score ceiling, and past the frobenius region, bounded by the square of the largest denomination, exactly the multiples of the gcd survive, which is how that solver splits its work at 1e6.

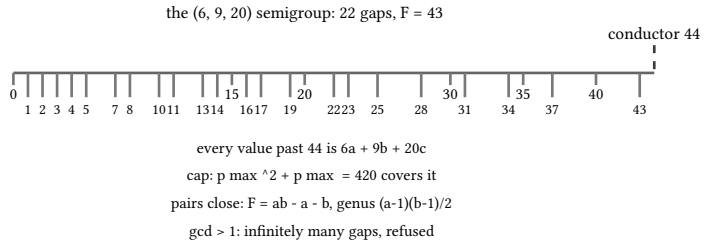


figure 302: the number line to 44 with the 22 mcnugget gaps hatched and the conductor line drawn at 44

35.9. the josephus problem

n people in a circle, every kth leaves, who survives. The recurrence is one line and one index convention: number people 0-based, let j(1) = 0, and j(n) = (j(n-1) + k) mod n, because the circle of n is the circle of n - 1 re-indexed after the first departure. O(n) with no simulation. The k = 2 case closes: with L = n - 2^floor(log2 n), the amount by which n exceeds its largest power of two, the survivor is 2L in the 0-based reading, 2L + 1 in the 1-based one, O(1) after one scan for the top bit. The elimination order needs the actual simulation, index removal on a live list.

The dry run: the fixtures are the recurrence anchors, the closed-form sweep, and the 7-circle order, asserted by the C# suite with C pinning the 10^9 anchor on both fast roads.

1. The recurrence re-indexes upward from j(1) = 0: j(2) = (0 + 2) % 2 = 0, j(3) = (0 + 2) % 3 = 2, j(4) = (2 + 2) % 4 = 0, j(5) = (0 + 2) % 5 = 2, the pinned j(5, 2).

2. The 7-circle order counts triples around the live circle: 0, 1, 2 sends 2 out, then 3, 4, 5 sends 5 out.
3. From 6 the count wraps: 6, 0, 1 sends 1 out, then 3, 4, 6 sends 6 out, then 0, 3, 4 sends 4 out, then 0, 3, 0 sends 0 out.
4. The order reads 2, 5, 1, 6, 4, 0 with 3 standing, and the recurrence agrees, j(7, 3) = 3.
5. The sweep runs the k = 2 closed form against the recurrence for every n to 2000, and both roads answer j(10^9, 2) = 926258176.

counts	leaves	circle after
0, 1, 2	2	0, 1, 3, 4, 5, 6
3, 4, 5	5	0, 1, 3, 4, 6
6, 0, 1	1	0, 3, 4, 6
3, 4, 6	6	0, 3, 4
0, 3, 4	4	0, 3
0, 3, 0	0	3

The standing 3 and the 926258176 are the pinned landings, and the listings below run all three roads in seven languages.

```c
static long long josephus_rec(long long v, long long k) {
  long long r = 0;
  for (long long m = 2; m ≤ v; m++)
    r = (r + k) % m;
  return r;
}

static long long josephus_k2_closed(long long v) {
  long long p = 1;
  while (p * 2 ≤ v)
    p *= 2;           // 2^floor(log2 v)
  return 2 * (v - p); // 0-based survivor
}
```

listing 1195: c, the recurrence loop, the closed form by top-bit scan (source: dsa/samples-c/src/Ch35/josephus.c, lines 19-31)

```go
func Josephus(n, k int64) int64 {
  survivor := int64(0)
  for m := int64(2); m <= n; m++ {
    survivor = (survivor + k) % m
  }
  return survivor
}

// JosephusPow2 is the k = 2 closed form 2L with
// L = n - 2^floor(log2 n), 0-based.
func JosephusPow2(n int64) int64 {
  top := uint64(1) << 63
  for top > uint64(n) {
    top >>= 1
  }
  return 2 * (n - int64(top))
}
```

listing 1196: go, the recurrence, the closed form from the top bit (source: dsa/samples-go/ch35/josephus.go, lines 5-21)

```java
  static long josephusRec(long v, long k) {
    long r = 0;
    for (long m = 2; m ≤ v; m++)
      r = (r + k) % m;
    return r;
  }

  static long josephusK2Closed(long v) {
    long p = 1;
    while (p * 2 ≤ v)
      p *= 2; // 2^floor(log2 v)
    return 2 * (v - p); // 0-based survivor
  }
```

listing 1197: java, the recurrence loop, the closed form by top-bit scan (source: dsa/samples-java/src/Ch35/Josephus.java, lines 18-30)

```c#
    public static long Survivor(long n, long k)
    {
        long j = 0; // j(1) = 0
        for (long m = 2; m ≤ n; m++)
            j = (j + k) % m;
        return j;
    }

    /// <summary>k = 2 closed form: 2L + 1 with L = n - 2^floor(log2 n), 1-based.</summary>
    public static long SurvivorK2(long n)
    {
        long highest = 1L << 62; // the top positive bit; shifting further left goes negative
        while (highest > n)
            highest >>= 1;
        return 2 * (n - highest); // 0-based: 2L
    }
```

listing 1198: c#, the recurrence, the closed form from bit 62 down (source: dsa/samples/src/Ch35/Josephus.cs, lines 9-24)

```javascript
export function josephus(n, k) {
  let r = 0;
  for (let m = 2; m ≤ n; m++) r = (r + k) % m;
  return r;
}

export function josephusClosed2(n) {
  const l = n - 2 ** Math.floor(Math.log2(n));
  return 2 * l; // 2L + 1 in 1-based terms
}

export function eliminationOrder(n, k) {
  const people = Array.from({ length: n }, (_, i) => i);
  const eliminated = [];
  let idx = 0;
  while (people.length > 1) {
    idx = (idx + k - 1) % people.length;
    eliminated.push(people.splice(idx, 1)[0]);
  }
  return { eliminated, survivor: people[0] };
}
```

listing 1199: javascript, recurrence, closed form, the elimination simulation (source: dsa/samples-js/src/ch35-josephus.mjs, lines 6-26)

```python
def josephus(count, k):
    survivor = 0  # j(1) = 0
    for m in range(2, count + 1):
        survivor = (survivor + k) % m
    return survivor


def closed_k2(count):
    # 0-based: 2L where L = n - 2^floor(log2 n)
    highest = 1 << (count.bit_length() - 1)
    return 2 * (count - highest)


def halving_k2(count):
    # j(2m) = 2 j(m), j(2m+1) = 2 j(m) + 2: the O(log n) second road
    if count == 1:
        return 0
    if count % 2 == 0:
        return 2 * halving_k2(count // 2)
    return 2 * halving_k2(count // 2) + 2
```

listing 1200: python, the recurrence, the closed form, the halving recurrence as a third road (source: dsa/samples-py/src/Ch35/josephus.py, lines 15-34)

```lua
local function josephus_rec(n, k)
  local j = 0 -- j(1) = 0
  for m = 2, n do
    j = (j + k) % m
  end
  return j
end

local function josephus_closed_k2(n) -- 0-based: 2L
  local p = 1
  while p * 2 ≤ n do p = p * 2 end -- 2^floor(log2 n)
  return 2 * (n - p)
end
```

listing 1201: lua, the recurrence and the closed form, the order below (source: dsa/samples-lua/ch35_josephus.lua, lines 7-19)

The fixtures pin all three roads. The recurrence anchors: j(5,2) = 2, j(7,3) = 3, j(10,2) = 4, and the classic j(41,3) = 30, person 31 in the 1-based reading. The closed form and a halving recurrence, j(2m) = 2 j(m) and j(2m+1) = 2 j(m) + 2, each sweep-equal to the recurrence for every n to 2000, and both fast roads answer j(10^9, 2) = 926258176. The order family pins (7,3) eliminating 2, 5, 1, 6, 4 first with survivor 3 matching the recurrence. The edges: n = 1 survives at 0 with an empty order, k = 1 leaves the last person as an identity order, k above n wraps, j(3,7) = 2, and k = n gives j(3,3) = 1. General technique.

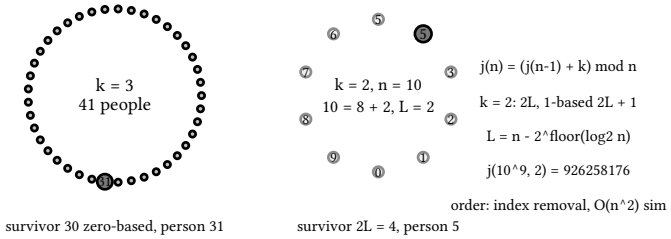


figure 303: the 41-person circle closing on person 31, with the k = 2 doubling ladder marked on a small circle

35.10. fifteen-puzzle solvability

No sample files ship for this section: the content is one parity predicate, a boolean with two additions, and there is no algorithm to port six ways, so the figure and the prose carry it whole. A sliding puzzle move is a transposition of the blank with one tile, and on a board of even width each move also shifts the blank's row by one, so the parity of the tile permutation and the parity of the blank's row distance from its goal row change together, making their sum the invariant. A board is solvable exactly when the invariant matches the solved board's: with width 4, inversions plus the blank's row counted from the bottom must be even. The solved board has 0 inversions and the blank on the first row from the bottom, sum even, solvable; the famous 14-15 swapped board has 1 inversion with the blank in the same place, sum odd, unsolvable, which is the entire 1880 puzzle hoax. The rule generalizes to any n by m board, odd widths needing only the inversion parity because the row shift there preserves parity. General technique, taught by cp-algorithms' "15 Puzzle Game: Existence Of The Solution".

The dry run: no suite carries this facet, so the numbers below are hand-derived, one parity read per board.

1. The solved board reads 0 inversions with the blank on its goal row, sum even, solvable.
2. The 14-15 swap changes exactly one thing, 1 inversion with the blank unmoved, sum odd: unsolvable, the entire 1880 hoax.
3. One legal slide preserves the parity of the sum: slide 12 down and the reading order gains 3 inversions, 13, 14, 15 each passing 12, while the blank moves one row off its goal, 3 + 1 = 4, even again.
4. Every slide is one transposition of the blank with a tile, which flips inversion parity, and on width 4 one row shift, which flips row parity: the two flips cancel every time.
5. Odd widths drop the row term, since the shift there preserves parity and inversions alone decide.

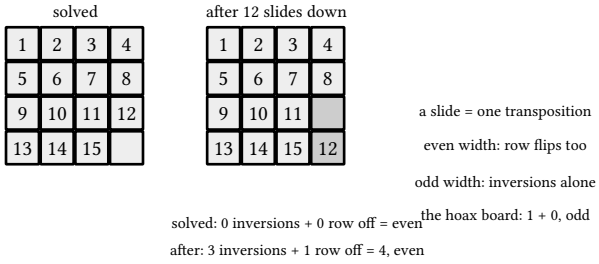


figure 304: the 12-down slide, three inversions bought by one row shift, the parity ledger beneath

Both boards read even and the hoax board reads odd, the whole test in one parity check per board.

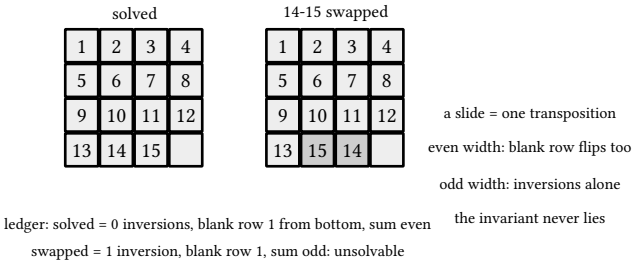


figure 305: the solved board beside the 14-15 swapped board with the parity ledger beneath

35.11. placing bishops on a chessboard

Bishops attack along both diagonals, and the two square colors never interact: a bishop on light squares can never attack one on dark. So the count splits, k bishops over two independent boards, summed across every split of k. Each color runs the same dp over its diagonals of one direction, sorted by non-decreasing length: f[i][j] = f[i-1][j] + f[i-1][j-1] times (len i - (j-1)), place nothing on the new diagonal or place a bishop on it, and each of the j - 1 bishops already on earlier diagonals blocks exactly one cell of it, because the blocking bishop's cross diagonal always meets a not-shorter diagonal, which is what the sort buys. k above 2n - 2 is zero, and the 2n - 2 maximum has exactly 2^n placements, one per diagonal pair choice.

The dry run: the fixture is the n = 4 row, asserted by the C# suite and brute-verified placement by placement; the dp states inside the walk are hand-derived off the recurrence.

1. The two colors of the 4x4 board sort their diagonals to lengths 1, 1, 3, 3 and 2, 2, 4.
2. The even dp seeds f = 1 and folds the first single cell to f = 1, 1; the second single cell lands f = 1, 2, 0, two bishops cannot both sit, 1 - 1 = 0 free cells.
3. The first length-3 diagonal lands f = 1, 5, 4, 0: 2 + 1 × 3 = 5 and 0 + 2 × 2 = 4, each earlier bishop blocking one cell.
4. The second lands f = 1, 8, 14, 4: 5 + 1 × 3 = 8, 4 + 5 × 2 = 14, 0
    1. 4 × 1 = 4.
5. The odd color folds 2, 2, 4 the same way to 1, 8, 14, 4, capped at 3 bishops by its 3 diagonals.
6. The combine reads the splits: k = 1 gives 8 + 8 = 16, k = 2 gives 14 + 64 + 14 = 92, and the ceiling k = 6 gives 4 × 4 = 16, the 2^4 of one diagonal pair each.

after diagonal	f over j = 0..4
seed	1
len 1	1, 1
len 1	1, 2, 0
len 3	1, 5, 4, 0
len 3	1, 8, 14, 4, 0

The full row 1, 16, 92, 232, 260, 112, 16 is the pinned landing, and the listings below run the dp in seven languages.

```c
static long long color_ways(int nn, int color, int k) {
  static long long f[2][32];
  int len[32];
  int cnt = diag_lengths(nn, color, len);
  int cur = 0;
  f[0][0] = 1;
  for (int j = 1; j ≤ k; j++)
    f[0][j] = 0;
  for (int i = 1; i ≤ cnt; i++) {
    int nx = 1 - cur;
    f[nx][0] = 1;
    for (int j = 1; j ≤ k; j++)
      f[nx][j] = f[cur][j] + f[cur][j - 1] * (len[i - 1] - (j - 1));
    cur = nx;
  }
  return f[cur][k];
}

static long long bishops(int nn, int k) {
  if (k > 2 * nn - 2)
    return 0;
  long long total = 0;
  for (int b = 0; b ≤ k; b++)
    total += color_ways(nn, 0, b) * color_ways(nn, 1, k - b);
  return total;
}
```

listing 1202: c, the rolling dp over the length staircase, the color combine (source: dsa/samples-c/src/Ch35/bishops.c, lines 46-71)

```go
// bishopsColorDP counts placements of j bishops on one color over
// its diagonals of one direction sorted by non-decreasing length:
// f[i][j] = f[i-1][j] + f[i-1][j-1] * (len_i - (j-1)); the j-1
// bishops on earlier diagonals each block exactly one cell of
// diagonal i (their cross diagonal always meets a not-shorter
// diagonal).
func bishopsColorDP(lens []int64) []int64 {
  f := make([]int64, len(lens)+1)
  f[0] = 1
  for i := 1; i <= len(lens); i++ {
    next := make([]int64, len(lens)+1)
    next[0] = 1
    for j := 1; j <= i; j++ {
      next[j] = f[j] + f[j-1]*(lens[i-1]-int64(j-1))
    }
    f = next
  }
  return f
}

// colorDiagonals lists the diagonal lengths of one direction for the
// cells of one color, sorted non-decreasing: the ascending order is
// what makes the j-1 blocking argument exact.
func colorDiagonals(n, color int) []int64 {
  var lens []int64
  for d := 0; d < 2*n-1; d++ {
    if d%2 != color {
      continue
    }
    l := d + 1
    if l > n {
      l = 2*n - 1 - d
    }
    if l > 0 {
      lens = append(lens, int64(l))
    }
  }
  sort.Slice(lens, func(i, j int) bool { return lens[i] < lens[j] })
  return lens
}
```

listing 1203: go, the dp over the sorted lengths, the diagonal length frame (source: dsa/samples-go/ch35/bishops.go, lines 5-44)

```
  static long[][] f = new long[2][32];

  static long colorWays(int nn, int color, int k) {
    int[] len = new int[32];
    int cnt = diagLengths(nn, color, len);
    int cur = 0;
    f[0][0] = 1;
    for (int j = 1; j ≤ k; j++)
      f[0][j] = 0;
    for (int i = 1; i ≤ cnt; i++) {
      int nx = 1 - cur;
      f[nx][0] = 1;
      for (int j = 1; j ≤ k; j++)
        f[nx][j] = f[cur][j] + f[cur][j - 1] * (len[i - 1] - (j - 1));
      cur = nx;
    }
    return f[cur][k];
  }

  static long bishops(int nn, int k) {
    if (k > 2 * nn - 2)
      return 0;
    long total = 0;
    for (int b = 0; b ≤ k; b++)
      total += colorWays(nn, 0, b) * colorWays(nn, 1, k - b);
    return total;
  }
```

listing 1204: java, the rolling dp over the length staircase, the color combine (source: dsa/samples-java/src/Ch35/Bishops.java, lines 43-69)

```
    /// <summary>Diagonal lengths of one direction for one color, sorted ascending.</summary>
    private static long[] DiagonalLengths(int n, bool white)
    {
        var lengths = new List<long>();
        for (int d = 0; d < 2 * n - 1; d++)
        {
            int count = 0;
            for (int r = 0; r < n; r++)
            {
                int c = d - r;
                if (c < 0 || c ≥ n)
                    continue;
                if ((r + c) % 2 == (white ? 0 : 1))
                    count++;
            }
            if (count > 0)
                lengths.Add(count);
        }
        lengths.Sort();
        return lengths.ToArray();
    }

    private static long[,] Dp(long[] lengths)
    {
        int m = lengths.Length;
        var f = new long[m + 1, m + 1];
        f[0, 0] = 1;
        for (int i = 1; i ≤ m; i++)
            for (int j = 0; j ≤ i; j++)
            {
                f[i, j] = f[i - 1, j];
                if (j > 0)
                    f[i, j] += f[i - 1, j - 1] * (lengths[i - 1] - (j - 1));
            }
        return f;
    }
```

listing 1205: c#, the per-color length table and the dp, combined in Count (source: dsa/samples/src/Ch35/Bishops.cs, lines 31-66)

```
const colorDiags = (n, color) => {
  // "/" diagonals s = r + c are monochromatic, color = s % 2
  const lens = [];
  for (let s = 0; s <= 2 * n - 2; s++) {
    if (s % 2 !== color) continue;
    lens.push(Math.min(s + 1, 2 * n - 1 - s));
  }
  lens.sort((a, b) => a - b); // non-decreasing
  return lens;
};

const colorCounts = (n, color, kmax) => {
  const lens = colorDiags(n, color);
  let f = new Array(kmax + 1).fill(0);
  f[0] = 1;
  for (const len of lens) {
    const nf = [...f];
    for (let j = 1; j <= kmax; j++) {
      nf[j] = f[j] + f[j - 1] * (len - (j - 1));
    }
    f = nf;
  }
  return f;
};

export function bishops(n, k) {
  if (k > 2 * n - 2) return 0;
  const black = colorCounts(n, 0, k);
  const white = colorCounts(n, 1, k);
  let total = 0;
  for (let b = 0; b <= k; b++) total += black[b] * white[k - b];
  return total;
}
```

listing 1206: javascript, the diagonal lengths, the dp, the split sum (source: dsa/samples-js/src/ch35-bishops.mjs, lines 9-41)

```
def color_diagonals(size, color):
    # the diagonal r - c = d is monochromatic with color d % 2
    lengths = [size - abs(d) for d in range(-(size - 1), size) if d % 2 == color]
    return sorted(lengths)


def per_color_counts(lengths, kmax):
    # f[i][j]: j bishops on the first i diagonals of this color
    f = [[0] * (kmax + 1) for _ in range(len(lengths) + 1)]
    for row in f:
        row[0] = 1
    for i, length in enumerate(lengths, start=1):
        for j in range(1, kmax + 1):
            # each of the j-1 earlier bishops blocks one cell of
            # diagonal i: their cross diagonal always meets it
            f[i][j] = f[i - 1][j] + f[i - 1][j - 1] * (length - (j - 1))
    return f[len(lengths)]


def bishops(size, k):
    if k > 2 * size - 2:
        return 0
    counts = []
    for color in (0, 1):
        lengths = color_diagonals(size, color)
        counts.append(per_color_counts(lengths, k))
    total = 0
    for split in range(k + 1):
        total += counts[0][split] * counts[1][k - split]
    return total
```

listing 1207: python, the three pieces, brute placements below (source: dsa/samples-py/src/Ch35/bishops.py, lines 17-46)

```
local function color_diags(n, color)
  -- diagonals (of one direction) of the given color, lengths sorted
  local lens = {}
  for d = 1, 2 * n - 1 do -- diagonal d covers cells (r, c) with
    local length -- r + c = d + 1 in 1-based, length 1..n
    if d ≤ n then
      length = d
    else
      length = 2 * n - d
    end
    -- cells on diagonal d: (r, d + 1 - r) for r in range; their color
    -- parity: (r + d + 1 - r) = d + 1, so the color is (d + 1) % 2
    if (d + 1) % 2 == color then lens[#lens + 1] = length end
  end
  table.sort(lens) -- non-decreasing
  return lens
end

-- ways to place j bishops on this color's diagonal staircase
local function color_counts(n, color)
  local lens = color_diags(n, color)
  local f = {[0] = 1}
  for i = 1, #lens do
    local nf = {[0] = 1}
    local len = lens[i]
    for j = 1, #lens do
      -- each of the j-1 earlier bishops blocks one cell of this
      -- diagonal: their cross diagonal always meets a not-shorter one
      nf[j] = (f[j] or 0) + (f[j - 1] or 0) * (len - (j - 1))
    end
    f = nf
  end
  return f
end

local function bishops(n, k)
  if k > 2 * n - 2 then return 0 end
  local f1 = color_counts(n, 0)
  local f2 = color_counts(n, 1)
  local total = 0
  for j = 0, k do
    total = total + (f1[j] or 0) * (f2[k - j] or 0)
  end
  return total
end
```

listing 1208: lua, the color staircase, the dp, the combine, brute in the suite (source: dsa/samples-lua/ch35_bishops.lua, lines 8-52)

The fixtures pin whole rows. Bishops(4, k) for k = 0..6 reads 1, 16, 92, 232, 260, 112, 16, brute-verified over all placements at n = 4; bishops(5, k) for k = 0..8 reads 1, 25, 240, 1124, 2728, 3368, 1960, 440, 32. The 8x8 row runs 1, 64, 1736, 26192, 242856, 1444928, 5599888, 14082528, 22522960, 22057472, 12448832, 3672448, 489536, 20224, 256, brute only at the small boards. The edges pin bishops(8,1) = 64 = n squared, the 2n - 2 maximum at 2^n for n = 4, 5, 8 reading 16, 32, 256, bishops(4,7) = 0 past the maximum, and bishops(2,2) = 4, the 2x2 corner. General technique, O(n k).

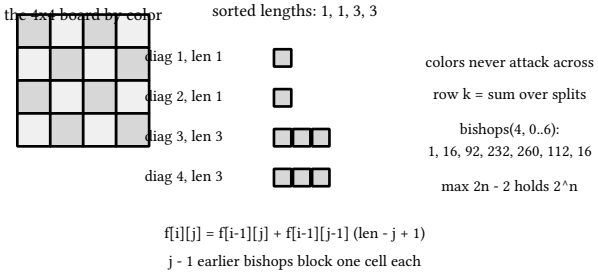


figure 306: the 4x4 board split by color, one color's diagonals as a length-sorted staircase feeding the dp

35.12. balanced bracket sequences

The count of balanced sequences with n pairs is the catalan number again, the nested-shape counter of the earlier section. Generation walks a depth recursion: at each position, emit an open when opens remain, then a close when the depth is positive, and lexicographic order falls out because the open branch always precedes the close branch at the same node. Rank and unrank price that order without walking it: when the string emits a close while an open was legal, every completion in the skipped open branch ranks earlier, and that branch holds exactly ballot(o, c) = C(o+c, o) - C(o+c, o-1) completions with o opens and c closes remaining. Summing those skips ranks any string, and spending the rank against the same ledger unranks.

The dry run: the fixture is the rank and unrank anchors over the 14 four-pair strings, asserted by the C# suite.

1. Unrank(4, 9) prices the open branch before every emit: the first block holds ballot(3, 4) = C(7,3) - C(7,2) = 35 - 21 = 14, and 9 < 14 takes the open.
2. The second block holds ballot(2, 4) = C(6,2) - C(6,1) = 15 - 6 = 9, 9 is not below 9, so the spend empties the rank and the close emits.
3. Rank 0 rides the rest: blocks of 5, 3, and 1 keep taking opens until none remain, then three closes finish ()((())), the pinned unrank of 9.
4. The rank walk sums the same ledger on (()())(): the first close passes a legal open at ballot(1, 4) = 4, two later closes pass at 1 each, 4 + 1 + 1 = 6.
5. The round trip returns every rank 0 through 13 through both roads, and the count itself is the catalan 14.

opens, closes	block	emit	rank
4, 4	ballot(3,4) = 35 - 21 = 14	(	9
3, 4	ballot(2,4) = 15 - 6 = 9	)	0
3, 3	ballot(2,3) = 10 - 5 = 5	(	0
2, 3	ballot(1,3) = 4 - 1 = 3	(	0
1, 3	ballot(0,3) = 1 - 0 = 1	(	0
0, 3	no open left, block 0	) × 3	0

The ()((())) and the 6 are the pinned landings, and the listings below price the order in seven languages.

```
static unsigned long long ballot(int o, int c) {
  if (o > c)
    return 0;
  return (binom(o + c, o) + P - binom(o + c, o - 1)) % P;
}

// rank of a valid sequence among the catalan(v) order
static unsigned long long rank_of(const char *s, int v) {
  unsigned long long r = 0;
  int o = v, c = v;
  for (int i = 0; i < 2 * v; i++) {
    if (s[i] == '(') {
      o--;
    } else {
      // ')' emitted while '(' was legal: add completions after '('
      if (o > 0)
        r = (r + ballot(o - 1, c)) % P;
      c--;
    }
  }
  return r;
}

// unrank: the r-th sequence in the same order
static void unrank(unsigned long long r, int v, char *out) {
  int o = v, c = v;
  for (int i = 0; i < 2 * v; i++) {
    if (o > 0) {
      unsigned long long after_open = ballot(o - 1, c);
      if (r < after_open) {
        out[i] = '(';
        o--;
        continue;
      }
      r -= after_open;
    }
    out[i] = ')';
    c--;
  }
  out[2 * v] = 0;
}
```

listing 1209: c, the ballot difference, rank, unrank over the same ledger (source: dsa/samples-c/src/Ch35/brackets.c, lines 55-95)

```
func ballot(o, c int) int64 {
  if o < 0 || c < 0 || o > c {
    return 0
  }
  return int64(BinomExact(o+c, o) - BinomExact(o+c, o-1))
}

// GenerateBrackets lists every balanced sequence with n pairs in
// lexicographic order by depth recursion.
func GenerateBrackets(n int) []string {
  var out []string
  var walk func(cur []byte, open, close int)
  walk = func(cur []byte, open, close int) {
    if len(cur) == 2*n {
      out = append(out, string(cur))
      return
    }
    if open < n {
      walk(append(cur, '('), open+1, close)
    }
    if close < open {
      walk(append(cur, ')'), open, close+1)
    }
  }
  walk(make([]byte, 0, 2*n), 0, 0)
  return out
}

// BracketRank returns the lexicographic rank among the C_n sequences
// by ballot-number suffix counts: when ')' is legal but '(' is
// emitted, add the completions after '('.
func BracketRank(s string) int64 {
  n := len(s) / 2
  rank := int64(0)
  open, close := n, n
  for i := 0; i < len(s); i++ {
    if s[i] == '(' {
      open--
    } else {
      // a '(' here would leave open-1 opens, close closes
      rank += ballot(open-1, close)
      close--
    }
  }
  return rank
}
```

listing 1210: go, ballot, the generator, and the rank walk (source: dsa/samples-go/ch35/brackets.go, lines 5-50)

```
  static long ballot(int o, int c) {
    if (o > c)
      return 0;
    return (binom(o + c, o) + P - binom(o + c, o - 1)) % P;
  }

  // rank of a valid sequence among the catalan(v) order
  static long rankOf(String s, int v) {
    long r = 0;
    int o = v, c = v;
    for (int i = 0; i < 2 * v; i++) {
      if (s.charAt(i) == '(') {
        o--;
      } else {
        // ')' emitted while '(' was legal: add completions after '('
        if (o > 0)
          r = (r + ballot(o - 1, c)) % P;
        c--;
      }
    }
    return r;
  }

  // unrank: the r-th sequence in the same order
  static String unrank(long r, int v) {
    StringBuilder out = new StringBuilder();
    int o = v, c = v;
    for (int i = 0; i < 2 * v; i++) {
      if (o > 0) {
        long afterOpen = ballot(o - 1, c);
        if (r < afterOpen) {
          out.append('(');
          o--;
          continue;
        }
        r -= afterOpen;
      }
      out.append(')');
      c--;
    }
    return out.toString();
  }
```

listing 1211: java, the ballot difference, rank, unrank over the same ledger (source: dsa/samples-java/src/Ch35/Brackets.java, lines 53-94)

```
    public static long Rank(string s)
    {
        int n = s.Length / 2;
        long rank = 0;
        int open = n;
        int close = n;
        foreach (char c in s)
            if (c == '(')
                open--;
            else
            {
                // ')' is legal here: every completion after '(' ranks before this one
                if (open > 0)
                    rank += Ballot(open - 1, close);
                close--;
            }
        return rank;
    }

    public static string Unrank(int n, long rank)
    {
        var sb = new System.Text.StringBuilder();
        int open = n;
        int close = n;
        while (open + close > 0)
        {
            long block = open > 0 ? Ballot(open - 1, close) : 0;
            if (rank < block)
            {
                sb.Append('(');
                open--;
            }
            else
            {
                rank -= block;
                sb.Append(')');
                close--;
            }
        }
        return sb.ToString();
    }

    private static long Ballot(long o, long c) =>
        o < 0 || c < 0 ? 0 : Binom.Exact(o + c, o) - Binom.Exact(o + c, o - 1);
```

listing 1212: c#, rank, unrank, and the ballot helper beneath them (source: dsa/samples/src/Ch35/Brackets.cs, lines 67-110)

```
export function bracketRank(s) {
  let o = s.length / 2;
  let c = s.length / 2;
  let rank = 0;
  for (const ch of s) {
    if (ch === "(") {
      o -= 1;
    } else {
      if (o ≥ 1) rank += ballot(o - 1, c); // the skipped '(' subtree
      c -= 1;
    }
  }
  return rank;
}

export function bracketUnrank(n, r) {
  let o = n;
  let c = n;
  let out = "";
  while (o + c > 0) {
    const openTree = o ≥ 1 ? ballot(o - 1, c) : 0;
    if (r < openTree) {
      out += "(";
      o -= 1;
    } else {
      r -= openTree;
      out += ")";
      c -= 1;
    }
  }
  return out;
}
```

listing 1213: javascript, rank and unrank over the ballot ledger (source: dsa/samples-js/src/ch35-brackets.mjs, lines 38-69)

```
def rank(s):
    # when ')' is emitted but '(' was legal, the '(' branch precedes
    o = c = s.count("(")
    r = 0
    for ch in s:
        if ch == "(":
            o -= 1
        else:
            if o > 0:  # the '(' branch came first
                r += ballot(o - 1, c)
            c -= 1
    return r


def unrank(pairs, r):
    o = c = pairs
    out = []
    while len(out) < 2 * pairs:
        if o > 0:  # the '(' branch holds ballot(o-1, c) strings
            branch = ballot(o - 1, c)
        else:
            branch = 0
        if r < branch:
            out.append("(")
            o -= 1
        else:
            r -= branch
            out.append(")")
            c -= 1
    return "".join(out)
```

listing 1214: python, rank and unrank, the ballot function above them (source: dsa/samples-py/src/Ch35/brackets.py, lines 68-97)

```lua
-- rank: when ')' is legal but '(' could have been emitted instead, add
-- the completions that would have followed the '('
local function rank(s)
  local n = #s // 2
  local o, c = n, n -- remaining opens and closes
  local r = 0
  for i = 1, #s do
    if s:sub(i, i) == "(" then
      o = o - 1
    else
      if o > 0 then r = r + ballot(o - 1, c) end
      c = c - 1
    end
  end
  return r
end

local function unrank(n, r)
  local s = {}
  local o, c = n, n
  for _ = 1, 2 * n do
    local take_open = 0
    if o > 0 then take_open = ballot(o - 1, c) end
    if o > 0 and r < take_open then
      s[#s + 1] = "("
      o = o - 1
    else
      if o > 0 then r = r - take_open end
      s[#s + 1] = ")"
      c = c - 1
    end
  end
  return table.concat(s)
end
```

listing 1215: lua, rank and unrank over the same counts (source: dsa/samples-lua/ch35_brackets.lua, lines 58-91)

The fixtures pin the order and the arithmetic. Four pairs hold 14 sequences, opening ((((()))), (((()))), (((())())), (((()))()), ((()(()))), closing with ()()(()) and ()()()(). Rank of (((()))() is 3, rank of (()())() is 6, unrank of 9 is ()((())()), and the round trip holds on all 14 in both directions. Validity is the running depth that never dips below zero and ends at zero, the profile of () reading 0, 1, 0, with) (and (() pinned invalid and unrank(0, 0) the empty string. The application is icpc world finals 2019 problem D (book 9, chapter 10), circular dna, which decides per-type nesting under rotation: a type is nested at the cut p exactly when its balance at p - 1 equals its running minimum, the depth rule of this section evaluated around the circle.

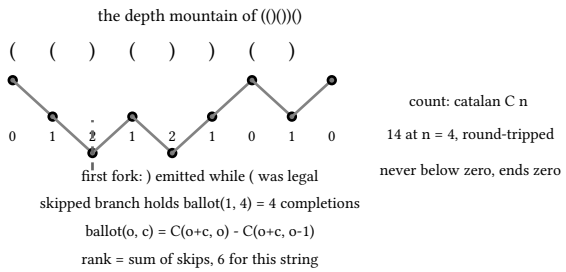


figure 307: the depth profile of (()())() as a mountain, the ballot-number shelf at the first fork

35.13. counting labeled graphs

A simple graph on n labeled vertices chooses each of the C(n,2) potential edges independently, so the total is 2^C(n,2). The connected count needs the exponential recurrence: subtract, from the total, the graphs where vertex 1's component holds exactly k vertices, choosing the k - 1 companions in C(n-1, k-1) ways, connecting them in c k ways, and letting the remaining n - k vertices carry any graph at all, 2^C(n-k,2) ways,

c n = 2^C(n,2) - sum over k of C(n-1, k-1) c k 2^C(n-k,2).

The dry run: the fixture is the exact row to c 8 with the n = 4 brute, asserted by the C# suite; C carries the same row.

1. n = 5 holds 2^C(5,2) = 2^10 = 1024 graphs in all.
2. k = 1 isolates vertex 1: C(4,0) × 1 × 2^C(4,2) = 64.
3. k = 2 and k = 3 follow: C(4,1) × 1 × 2^3 = 32, then C(4,2) × 4 × 2^1 = 48.
4. k = 4 drags the brute-verified c 4 along: C(4,3) × 38 × 2^0 = 152.
5. The subtractand sums to 64 + 32 + 48 + 152 = 296, and c 5 = 1024 - 296 = 728, the fifth rung of the pinned row 1, 1, 4, 38, 728, 26704, 1866256, 251548592.
6. The modular twin answers connected(100) mod 1e9+7 = 686310291 off the same recurrence.

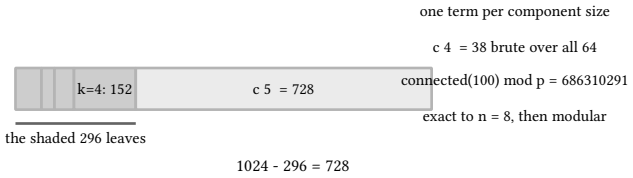


figure 308: the 1024 graphs of n = 5 split by the size of vertex 1's component, the 296 subtractand shaded against the 728

The 728 rung and the 686310291 are the pinned landings, and the listings below run the recurrence in seven languages.

```c
static unsigned long long two_pairs(int v) {
  return modpow(2, (unsigned long long)v * (v - 1) / 2);
}

// connected[v] by the exponential recurrence, mod P
static unsigned long long conn[NMAX];

static void build_connected(int vmax) {
  conn[0] = 0; // c_0 = 0 convention
  if (vmax ≥ 1)
    conn[1] = 1; // the lone vertex is connected
  for (int v = 2; v ≤ vmax; v++) {
    unsigned long long s = 0;
    for (int k = 1; k ≤ v - 1; k++) {
      unsigned long long term = binom(v - 1, k - 1) * conn[k] % P;
      term = term * two_pairs(v - k) % P;
      s = (s + term) % P;
    }
    conn[v] = (two_pairs(v) + P - s) % P;
  }
}
```

listing 1216: c, the power of two over pairs, the recurrence loop modular (source: dsa/samples-c/src/Ch35/labeled.c, lines 54-74)

```
func ConnectedGraphs(n int) int64 {
  return connectedRec(n, 0)
}

// ConnectedGraphsMod is the same recurrence modulo p, n up to 100.
func ConnectedGraphsMod(n int, p int64) int64 {
  return connectedRec(n, p)
}

func connectedRec(n int, p int64) int64 {
  c := make([]int64, n+1)
  c[0] = 0 // convention
  for m := 1; m <= n; m++ {
    total := pow2(m*(m-1)/2, p)
    for k := 1; k < m; k++ {
      term := binomSmall(m-1, k-1, p) * c[k]
      if p > 0 {
        term %= p
      }
      term *= pow2((m-k)*(m-k-1)/2, p)
      if p > 0 {
        term %= p
      }
      total -= term
      if p > 0 {
        total = (total + p) % p
      }
    }
    c[m] = total
  }
  return c[n]
}
```

listing 1217: go, one recurrence parameterized by modulus, zero meaning exact (source: dsa/samples-go/ch35/labeled.go, lines 13-44)

```
  static long twoPairs(int v) {
    return modpow(2, (long) v * (v - 1) / 2);
  }

  // connected[v] by the exponential recurrence, mod P
  static long[] conn = new long[NMAX];

  static void buildConnected(int vmax) {
    conn[0] = 0; // c_0 = 0 convention
    if (vmax ≥ 1)
      conn[1] = 1; // the lone vertex is connected
    for (int v = 2; v ≤ vmax; v++) {
      long s = 0;
      for (int k = 1; k ≤ v - 1; k++) {
        long term = binom(v - 1, k - 1) * conn[k] % P;
        term = term * twoPairs(v - k) % P;
        s = (s + term) % P;
      }
      conn[v] = (twoPairs(v) + P - s) % P;
    }
  }
```

listing 1218: java, the power of two over pairs, the recurrence loop modular (source: dsa/samples-java/src/Ch35/Labeled.java, lines 53-73)

```
    /// <summary>Exact recurrence in plain longs; valid to n = 8 where every term
    /// stays far below 2^63.</summary>
    public static long ConnectedExact(int n)
    {
        if (n == 0)
            return 0; // the c_0 = 0 convention
        var c = new long[n + 1];
        c[1] = 1;
        for (int m = 2; m ≤ n; m++)
        {
            long total = 1L << (m * (m - 1) / 2);
            long subtract = 0;
            for (int k = 1; k < m; k++)
                subtract += Binom.Exact(m - 1, k - 1) * c[k] << ((m - k) * (m - k - 1) / 2);
            c[m] = total - subtract;
        }
        return c[n];
    }

    public static long ConnectedMod(int n, long p)
    {
        if (n == 0)
            return 0;
        var binom = new Binom(n, p);
        var c = new long[n + 1];
        c[1] = 1;
        for (int m = 2; m ≤ n; m++)
        {
            long total = NumTheory.ModPow(2, (long)m * (m - 1) / 2, p);
            long subtract = 0;
            for (int k = 1; k < m; k++)
                subtract = (subtract
                    + binom.Choose(m - 1, k - 1) * c[k] % p
                        * NumTheory.ModPow(2, (long)(m - k) * (m - k - 1) / 2, p)) % p;
            c[m] = ((total - subtract) % p + p) % p;
        }
        return c[n];
    }
```

listing 1219: c#, the exact recurrence in longs, the modular twin (source: dsa/samples/src/Ch35/Labeled.cs, lines 14-51)

```
export function connectedLabeled(n) {
  if (n === 0) return 0;
  const c = [0, 1];
  for (let m = 2; m ≤ n; m++) {
    let s = 2 ** ((m * (m - 1)) / 2);
    for (let k = 1; k ≤ m - 1; k++) {
      s -= cSmall(m - 1, k - 1) * c[k] * 2 ** (((m - k) * (m - k - 1)) / 2);
    }
    c.push(s);
  }
  return c[n];
}
```

listing 1220: javascript, the exact recurrence, the BigInt modular twin below (source: dsa/samples-js/src/ch35-labeled.mjs, lines 20-31)

```
def connected_exact(vertices):
    # c_n = 2^C(n,2) - sum_{k} C(n-1, k-1) c_k 2^C(n-k,2)
    c = [0, 1]  # c_0 = 0 convention, c_1 = 1
    for m in range(2, vertices + 1):
        total = 2 ** C_exact(m, 2)
        sub = 0
        for k in range(1, m):
            sub += C_exact(m - 1, k - 1) * c[k] * 2 ** C_exact(m - k, 2)
        c.append(total - sub)
    return c


def connected_mod(vertices, p):
    c = [0, 1]
    for m in range(2, vertices + 1):
        total = pow(2, C_exact(m, 2), p)
        sub = 0
        for k in range(1, m):
            sub += C_exact(m - 1, k - 1) * c[k] * pow(2, C_exact(m - k, 2), p)
        c.append((total - sub) % p)
    return c
```

listing 1221: python, the exact and modular recurrences, the brute below (source: dsa/samples-py/src/Ch35/labeled.py, lines 30-50)

```
local function connected_mod(n)
  local fact, invf = build(n)
  local pow2 = {[0] = 1} -- 2^C(m,2) for m = 0..n
  for m = 1, n do pow2[m] = pow2[m - 1] * modpow(2, m - 1) % P end
  local c = {[0] = 0}
  for m = 1, n do
    local s = pow2[m]
    for k = 1, m - 1 do
      s = (s - C_mod(m - 1, k - 1, fact, invf) * c[k] % P
               * pow2[m - k]) % P
    end
    c[m] = (s % P + P) % P
  end
  return c, pow2
end
```

listing 1222: lua, the modular recurrence with the pow2 table precomputed (source: dsa/samples-lua/ch35_labeled.lua, lines 35-49)

The fixtures pin the row and the two rungs. Exact: graphs on 5 vertices = 1024, and c 1 through c 8 = 1, 1, 4, 38, 728, 26704, 1866256, 251548592, with c 4 = 38 brute-verified over all 2^6 = 64 graphs by a union-find connectivity test. Modular: C(100,2) = 4950, 2^4950 mod 1e9+7 = 281603733, and connected(100) mod 1e9+7 = 686310291. The edges read n = 1 as one graph, connected, n = 2 as

two graphs with one connected, and the c 0 = 0 convention pinned explicitly. General technique, exact to n = 8 where every term fits the native integer, O(n^2) modular beyond.

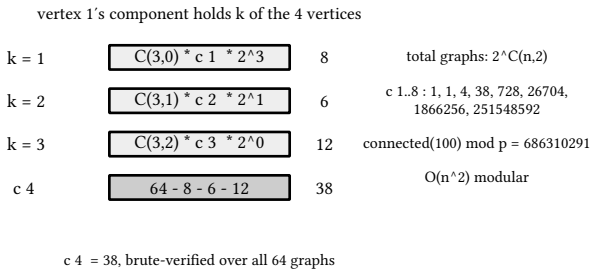


figure 309: n = 4 split by the size k of vertex 1′s component, the blocks multiplying into the 38

35.14. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's twelve sample files per language, go test files excluded:

language	build sloc	container dependency	boundary note
c	1296	static tables, family bitmasks	modular binomials everywhere, dedekind over family bitmasks with long long, exact catalan through u64
go	621	slices, cached table maps	one recurrence parameterized by modulus for labeled graphs, exact binomials through bits.Div64
java	1289	jdk 27 stdlib	C(67,33) pinned through Long.parseUnsignedLong compared unsigned, the lgamma twin as a nine-coefficient lanczos series, sperner's dedekind over long family bitmasks
c#	526	linq, value tuples, class reuse	binom, ksubset, and the ch16 and ch28 classes composed across files, dedekind refusing past n = 4
javascript	450	BigInt tables, plain Number where small	catalan as C(2n,n) - C(2n,n+1) with no division, two test files at the 400 sloc house split
python	853	lists, inline asserts	exact ladders with lgamma cross-checks, brute twins inside every family
lua	1079	tables, 1-based masks	the burnside suite generating the 24-rotation group itself and counting fixed colorings directly, decimal-string binomials past 2^63

sources: cp-algorithms, "Binomial Coefficients", cp-algorithms.com/combinatorics/binomial-coefficients.html, "Catalan Numbers", cp-algorithms.com/combinatorics/catalan-numbers.html, "Stars and bars", cp-algorithms.com/combinatorics/stars_and_bars.html, "The Inclusion-Exclusion Principle", cp-algorithms.com/combinatorics/inclusion-exclusion.html, "Burnside's lemma / Pólya enumeration theorem", cp-algorithms.com/combinatorics/burnside.html, "Generating all K-combinations", cp-algorithms.com/combinatorics/generating_combinations.html, whose subset-of-mask sibling "Enumerating submasks of a bitmask", cp-algorithms.com/algebra/all-submasks.html, is named in prose, "Placing Bishops on a Chessboard", cp-algorithms.com/combinatorics/bishops-on-chessboard.html, "Balanced bracket sequences", cp-algorithms.com/combinatorics/bracket_sequences.html, "Counting labeled graphs", cp-algorithms.com/combinatorics/counting_labeled_graphs.html, "Josephus problem", cp-algorithms.com/others/josephus_problem.html, and "15 Puzzle Game: Existence Of The Solution", cp-algorithms.com/others/15-puzzle.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. cp-algorithms carries no sperner or frobenius article; the sperner section cites Sperner's 1928 theorem and the LYM inequality by name, and the frobenius conductor bound is stated as the classical schur-type result. Application sources: icpc world finals 2018 problem D (book 9, chapter 9), 2023 problem B (book 10, chapter 12), 2017 problem K (book 10, chapter 8), 2025 problem H (book 10, chapter 13), and 2019 problem D (book 10, chapter 10). Sample behavior verified by the seven suite gates scoped to chapter 35: c 12 files and 216 checks, go 39 test functions, java 12 files and 216 checks under run-java-samples, c#

57 facts, javascript 36 tests and 150 asserts across the two-file split, python 12 files and 148 asserts, lua 42 checks, zero skipped.

36. numerical methods

Continuous problems land on a contest board as doubles, and the honest way through is deterministic drivers with pinned stopping rules: every loop count fixed in advance, every assert compared against a pinned constant inside a stated epsilon, never bit equality. This chapter builds the four drivers that carry that discipline, ternary and golden-section search over unimodal valleys, newton iteration for roots and reciprocals, simpson integration in composite and adaptive form, and the linear program duality that turns one world finals flow problem into a one dimensional search. Simulated annealing closes the chapter in prose, the one heuristic in the neighborhood, kept honest by the same seeded determinism the coded sections use. The anchor application is icpc 2025 problem C (book 10, chapter 13), whose reservoir model supplies the duality section end to end.

36.1. ternary search

Two domains, one algorithm. On integers the loop probes m1 = l + (r - l) / 3 and m2 = r - (r - l) / 3 while r - l > 2, keeps the side holding the smaller probe, and on a tie shrinks both ends, the plateau rule, because equal probes say nothing about where a flat stretch ends. A short linear scan closes whatever window of 3 or fewer cells survives. On doubles the same two probes run a fixed 200 iterations and return the midpoint. The fixed count is the whole trick for a seven-language book: no float comparison ever decides control flow differently in c than in lua, so the orbit of intervals is identical everywhere and the pinned outputs hold without any tolerance negotiation. The contract is strictly unimodal input for the clean invariant, minimum inside and both probes strictly ordered off the floor, with the plateau rule documented as the safety net.

The dry run: the fixtures are the valley 12, 9, 7, 3, 2, 5, 8, 11 with its rising, falling, and plateau kin, asserted by the C# suite and pinned at identical counts in all seven.

1. Round 1 on the valley, window 0 through 7: m1 = 0 + 7 / 3 = 2 and m2 = 7 - 7 / 3 = 5, the probes read 7 and 5, 7 > 5 keeps the right side, l = 3, meter at 2.
2. Round 2, window 3 through 7: m1 = 3 + 4 / 3 = 4 and m2 = 7 - 4 / 3 = 6, the probes read 2 and 8, 2 < 8 keeps the left, r = 5, meter at 4.
3. The window 3 through 5 is 3 wide, the loop hands off, and the closing scan pays 2 probes, 2 < 3 moving the best to index 4: the meter lands 6.
4. The plateau 5, 4, 4, 4, 7 ties its round, 4 = 4 at m1 = 1 and m2 = 3, so both ends shrink to 1 through 3 and the strict scan keeps the first minimum, index 1, meter 4.
5. The rising 1, 2, 3, 4, 5 pins the left edge, one round and a two-step scan, meter 4 at argmin 0, and the falling 9, 7, 5, 3 pins the right edge, one round and a one-step scan, meter 3 at argmin 3.

fixture	rounds	round probes	scan probes	total	argmin
12, 9, 7, 3, 2, 5, 8, 11	2	4	2	6	4
1, 2, 3, 4, 5	1	2	2	4	0
9, 7, 5, 3	1	2	1	3	3
5, 4, 4, 4, 7	1, tie	2	2	4	1

The 6 is the meter every suite pins on the valley, and the listings below run the integer loop and the 200-iteration real driver in seven languages.

```
static int ternary_int(const int *a, int len) {
  int l = 0, r = len - 1;
  probes = 0;
  while (r - l > 2) {
    int m1 = l + (r - l) / 3;
    int m2 = r - (r - l) / 3;
    probes += 2;
    if (a[m1] < a[m2])
      r = m2 - 1;
    else if (a[m1] > a[m2])
      l = m1 + 1;
    else {
      l++;
      r--; // plateau: shrink both ends
    }
  }
  int best = l;
  for (int i = l + 1; i ≤ r; i++) {
    probes++;
    if (a[i] < a[best])
      best = i;
  }
  return best;
}

static double ternary_real(double (*f)(double), double lo, double hi) {
  for (int i = 0; i < 200; i++) {
    double m1 = lo + (hi - lo) / 3;
    double m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2))
      hi = m2;
    else
      lo = m1;
  }
  return (lo + hi) / 2;
}
```

listing 1223: c, the integer loop with the tie rule, then the fixed-count real driver (source: dsa/samples-c/src/Ch36/ternary.c, lines 24-59)

```
func TernaryInt(a []int64) (idx, probes int) {
  l, r := 0, len(a)-1
  for r-l > 2 {
    m1 := l + (r-l)/3
    m2 := r - (r-l)/3
    probes += 2
    switch {
    case a[m1] < a[m2]:
      r = m2 - 1
    case a[m1] > a[m2]:
      l = m1 + 1
    default:
      l++
      r--
    }
  }
  best := l
  for i := l + 1; i <= r; i++ {
    probes++
    if a[i] < a[best] {
      best = i
    }
  }
  return best, probes
}

// TernaryReal runs a fixed number of two-probe iterations on doubles
// and returns the interval midpoint: the fixed loop count keeps the
// languages from drifting apart on float comparisons. Strictly
// unimodal contract.
func TernaryReal(f func(float64) float64, lo, hi float64, iters int) float64 {
  for i := 0; i < iters; i++ {
    m1 := lo + (hi-lo)/3
    m2 := hi - (hi-lo)/3
    if f(m1) < f(m2) {
      hi = m2
    } else {
      lo = m1
    }
  }
  return (lo + hi) / 2
}
```

listing 1224: go, the switch form of the three-way probe outcome (source: dsa/samples-go/ch36/ternary.go, lines 7-48)

```
  static int ternaryInt(int[] a) {
    int l = 0, r = a.length - 1;
    probes = 0;
    while (r - l > 2) {
      int m1 = l + (r - l) / 3;
      int m2 = r - (r - l) / 3;
      probes += 2;
      if (a[m1] < a[m2])
        r = m2 - 1;
      else if (a[m1] > a[m2])
        l = m1 + 1;
      else {
        l++;
        r--; // plateau: shrink both ends
      }
    }
    int best = l;
    for (int i = l + 1; i ≤ r; i++) {
      probes++;
      if (a[i] < a[best])
        best = i;
    }
    return best;
  }

  static double ternaryReal(DoubleUnaryOperator f, double lo, double hi) {
    for (int i = 0; i < 200; i++) {
      double m1 = lo + (hi - lo) / 3;
      double m2 = hi - (hi - lo) / 3;
      if (f.applyAsDouble(m1) < f.applyAsDouble(m2))
        hi = m2;
      else
        lo = m1;
    }
    return (lo + hi) / 2;
  }
```

listing 1225: java, the integer loop with the tie rule, then the fixed-count real driver over DoubleUnaryOperator (source: dsa/samples-java/src/Ch36/Ternary.java, lines 21-56)

```
    public static (int Index, int Probes) ArgMin(int[] a)
    {
        int l = 0;
        int r = a.Length - 1;
        int probes = 0;
        while (r - l > 2)
        {
            int m1 = l + (r - l) / 3;
            int m2 = r - (r - l) / 3;
            probes += 2;
            if (a[m1] < a[m2])
                r = m2 - 1;
            else if (a[m1] > a[m2])
                l = m1 + 1;
            else
            {
                l++; // plateau: shrink both ends
                r--;
            }
        }
        int best = l;
        for (int i = l + 1; i ≤ r; i++)
        {
            probes++;
            if (a[i] < a[best])
                best = i; // strict compare keeps the first minimum
        }
        return (best, probes);
    }

    /// <summary>200 fixed iterations of the same two probes on doubles, midpoint out.</summary>
    public static double MinimizeReal(Func<double, double> f, double lo, double hi)
    {
        for (int i = 0; i < 200; i++)
        {
            double m1 = lo + (hi - lo) / 3;
            double m2 = hi - (hi - lo) / 3;
            if (f(m1) < f(m2))
                hi = m2;
            else
                lo = m1;
        }
        return (lo + hi) / 2;
    }
```

listing 1226: c#, the same pair, tuple return carrying the probe meter (source: dsa/samples/src/Ch36/Ternary.cs, lines 11-54)

```
export function ternaryInt(a) {
  let l = 0;
  let r = a.length - 1;
  let probes = 0;
  while (r - l > 2) {
    const m1 = l + Math.floor((r - l) / 3);
    const m2 = r - Math.floor((r - l) / 3);
    probes += 2;
    if (a[m1] < a[m2]) {
      r = m2 - 1;
    } else if (a[m1] > a[m2]) {
      l = m1 + 1;
    } else {
      l += 1; // plateau: shrink both ends
      r -= 1;
    }
  }
  let best = l;
  for (let i = l + 1; i ≤ r; i++) {
    probes += 1;
    if (a[i] < a[best]) best = i;
  }
  return { index: best, value: a[best], probes };
}

export function ternaryReal(f, lo, hi, iters = 200) {
  for (let i = 0; i < iters; i++) {
    const m1 = lo + (hi - lo) / 3;
    const m2 = hi - (hi - lo) / 3;
    if (f(m1) < f(m2)) {
      hi = m2;
    } else {
      lo = m1;
    }
  }
  return (lo + hi) / 2;
}
```

listing 1227: javascript, floor division by hand, the object return (source: dsa/samples-js/src/ch36-ternary.mjs, lines 10-46)

```python
def ternary_int(a):
    lo, hi = 0, len(a) - 1
    probes = 0
    while hi - lo > 2:
        m1 = lo + (hi - lo) // 3
        m2 = hi - (hi - lo) // 3
        probes += 2
        if a[m1] < a[m2]:
            hi = m2 - 1
        elif a[m1] > a[m2]:
            lo = m1 + 1
        else:  # plateau: shrink both ends
            lo += 1
            hi -= 1
    best = lo
    for i in range(lo + 1, hi + 1):  # a short linear scan closes it
        probes += 1
        if a[i] < a[best]:
            best = i
    return best, probes


def ternary_real(f, lo, hi, iters=200):
    for _ in range(iters):  # the loop count is fixed by contract
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        if f(m1) < f(m2):
            hi = m2
        else:
            lo = m1
    return (lo + hi) / 2
```

listing 1228: python, the plateau comment marks the tie rule (source: dsa/samples-py/src/Ch36/ternary.py, lines 15-45)

```lua
local function ternary_int(a)
  local l, r = 0, #a - 1
  local probes = 0
  while r - l > 2 do
    local m1 = l + (r - l) // 3
    local m2 = r - (r - l) // 3
    probes = probes + 2
    if a[m1 + 1] < a[m2 + 1] then
      r = m2 - 1
    elseif a[m1 + 1] > a[m2 + 1] then
      l = m1 + 1
    else
      l = l + 1 -- the plateau rule: shrink both ends
      r = r - 1
    end
  end
  local best = l
  for i = l + 1, r do
    probes = probes + 1
    if a[i + 1] < a[best + 1] then best = i end
  end
  return best, probes
end

local function ternary_real(f, lo, hi, iters)
  iters = iters or 200
  for _ = 1, iters do
    local m1 = lo + (hi - lo) / 3
    local m2 = hi - (hi - lo) / 3
    if f(m1) < f(m2) then
      hi = m2
    else
      lo = m1
    end
  end
  return (lo + hi) / 2
end
```

listing 1229: lua, the 0-based public face over 1-based tables (source: dsa/samples-lua/ch36_ternary.lua, lines 8-44)

The probe meter is deterministic, so the integer fixtures pin identical counts in all seven suites. The valley 12, 9, 7, 3, 2, 5, 8, 11 lands its argmin at index 4 on 6 probes, the rising array 1, 2, 3, 4, 5 puts the minimum at the left edge index 0 on 4 probes, and the falling array 9, 7, 5, 3 hits the right edge index 3 on 3 probes. The plateau family 5, 4, 4, 4, 7 returns index 1, the first minimum, on 4 probes. Every integer answer is cross-checked against a linear scan, and the real drivers pin against the pinned constants: (x - 3)^2 + 0.5 over (0, 10) returns x within 1e-6 of 3, measured 3.0000000074505806, with f(x) = 0.5 to 1e-9, and the kinked valley, parabola right of 2 and a ramp left of it, closes on 2.0 exactly on (0, 6).

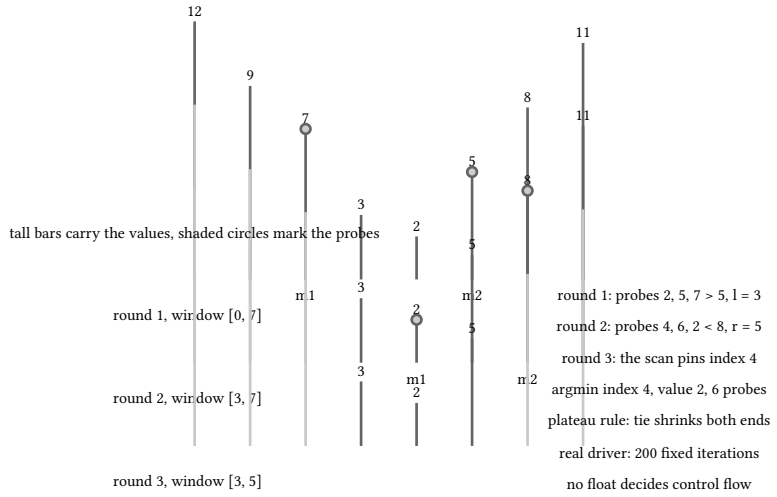


figure 310: the 12, 9, 7, 3, 2, 5, 8, 11 valley over three probe rounds, the kept range shrinking each round, the probe ledger at right

The contest face is icpc 2025 problem C (book 10, chapter 13), where the r = 2 case of the pipe model is exactly this driver over the free reservoir weight and r = 3 nests two of them, and icpc 2022 problem T (book 10, chapter 11), which refines a grid answer with a ternary pass over each corner-bend leg. The monotone cousin, bisection over a boolean answer space, is book 9, ch 14 (practical data structures and algorithms, searching and answer spaces) territory and stays there.

36.2. golden-section search

Ternary spends two evaluations per step because both probe points land in territory the next step cannot reuse. The golden ratio fixes the waste: place c = hi - phi(hi - lo) and d = lo + phi(hi

- lo) with phi = (sqrt(5) - 1) / 2 = 0.6180339887498949, and the

symmetry phi^2 = 1 - phi makes one old interior point land exactly on the next step's opposite probe. Keep that point, evaluate only the new one, and every step after the warm-up pair costs exactly one evaluation. The iteration count is pinned at 100, which shrinks the interval below 1e-19 of its width, far past any useful epsilon, so evaluation counts and final widths are identical doubles in all seven languages. The budget arithmetic is the selling point: 102 total evaluations against 400 for 200 ternary iterations.

The dry run: the fixture is (x - 1)^2 + 7 over (0, 100) with the ternary kink riding along, asserted by the C# suite at exactly 102 evaluations, the width gate, and the phi constants.

1. The warm-up evaluates the first pair, c = 100 - phi × 100 = 38.1966011250105 and d = phi × 100 = 61.8033988749895, 2 evals on the ledger.
2. Both probes sit right of the minimum at 1, fc < fd, so hi takes d and the old c is carried: by phi^2 = 1 - phi it lands exactly on the next step's d, and only the new c costs a call.
3. The width shrinks one phi per step, 100 × phi = 61.8033988749895 then 38.1966011250105 then 23.6067977499790, and after step k the geometry says 100 × phi^k.
4. The constants family pins 100 × phi^100 = 1.2625133380638542e-19 within 1e-12 relative, the honest band, since lua's libm pow differs from the reference in the fifteenth significant digit.
5. Doubles stop shrinking before the geometry does: the measured final width is 2.220446049250313e-16, one double spacing near 1, inside the 1e-15 gate, and the midpoint lands 1.0000000210734243 with f(x) within 1e-12 of 7, the kink landing 2.000000000000001.

step	interval width	evals
warm-up	100	2
1	61.8033988749895	3
2	38.1966011250105	4

step	interval width	evals
k	100 × phi^k	k + 2
100	1.2625133380638542e-19	102

The 102 is what every suite pins, half of ternary's 400 for the same job, and the listings below run the keep-and-replace loop in seven languages.

```c
static double golden(double (*f)(double), double lo, double hi, int iters,
                     double *width_out) {
  double c = hi - PHI * (hi - lo);
  double d = lo + PHI * (hi - lo);
  double fc = f(c), fd = f(d);
  evals = 2;
  for (int i = 0; i < iters; i++) {
    if (fc < fd) {
      hi = d;
      d = c;
      fd = fc;
      c = hi - PHI * (hi - lo);
      fc = f(c);
    } else {
      lo = c;
      c = d;
      fc = fd;
      d = lo + PHI * (hi - lo);
      fd = f(d);
    }
    evals++;
  }
  *width_out = hi - lo;
  return (lo + hi) / 2;
}
```

listing 1230: c, the keep-and-replace loop, one new evaluation per step (source: dsa/samples-c/src/Ch36/golden.c, lines 26-50)

```go
func GoldenSection(f func(float64) float64, lo, hi float64,
  iters int) (float64, int, float64) {
  c := hi - GoldenRatio*(hi-lo)
  d := lo + GoldenRatio*(hi-lo)
  fc, fd := f(c), f(d)
  evals := 2
  for i := 0; i < iters; i++ {
    if fc < fd {
      hi, d, fd = d, c, fc
      c = hi - GoldenRatio*(hi-lo)
      fc = f(c)
    } else {
      lo, c, fc = c, d, fd
      d = lo + GoldenRatio*(hi-lo)
      fd = f(d)
    }
    evals++
  }
  return (lo + hi) / 2, evals, hi - lo
}
```

listing 1231: go, the same multi-assign, evals and width returned (source: dsa/samples-go/ch36/golden.go, lines 12-31)

```
  static double golden(DoubleUnaryOperator f, double lo, double hi, int iters) {
    double c = hi - PHI * (hi - lo);
    double d = lo + PHI * (hi - lo);
    double fc = f.applyAsDouble(c), fd = f.applyAsDouble(d);
    evals = 2;
    for (int i = 0; i < iters; i++) {
      if (fc < fd) {
        hi = d;
        d = c;
        fd = fc;
        c = hi - PHI * (hi - lo);
        fc = f.applyAsDouble(c);
      } else {
        lo = c;
        c = d;
        fc = fd;
        d = lo + PHI * (hi - lo);
        fd = f.applyAsDouble(d);
      }
      evals++;
    }
    widthOut = hi - lo;
    return (lo + hi) / 2;
  }
```

listing 1232: java, the keep-and-replace loop, one new evaluation per step (source: dsa/samples-java/src/Ch36/Golden.java, lines 24-47)

```
    public static (double X, long Evals, double Width) Minimize(
        Func<double, double> f, double lo, double hi)
    {
        double c = hi - Phi * (hi - lo);
        double d = lo + Phi * (hi - lo);
        double fc = f(c);
        double fd = f(d);
        long evals = 2;
        for (int i = 0; i < 100; i++)
        {
            if (fc < fd)
            {
                (hi, d, fd) = (d, c, fc); // keep the interior point
                c = hi - Phi * (hi - lo);
                fc = f(c);
            }
            else
            {
                (lo, c, fc) = (c, d, fd);
                d = lo + Phi * (hi - lo);
                fd = f(d);
            }
            evals++;
        }
        return ((lo + hi) / 2, evals, hi - lo);
    }
```

listing 1233: c#, the tuple swap keeps the interior point in one statement (source: dsa/samples/src/Ch36/Golden.cs, lines 11-36)

```
export function golden(f, lo, hi, iters = 100) {
  let c = hi - PHI * (hi - lo);
  let d = lo + PHI * (hi - lo);
  let fc = f(c);
  let fd = f(d);
  let evals = 2;
  for (let i = 0; i < iters; i++) {
    if (fc < fd) {
      hi = d;
      d = c;
      fd = fc;
      c = hi - PHI * (hi - lo);
      fc = f(c); // the one new evaluation
    } else {
      lo = c;
      c = d;
      fc = fd;
      d = lo + PHI * (hi - lo);
      fd = f(d);
    }
    evals += 1;
  }
  return { x: (lo + hi) / 2, evals, width: hi - lo };
}
```

listing 1234: javascript, the one new evaluation commented where it happens (source: dsa/samples-js/src/ch36-golden.mjs, lines 12-35)

```
PHI = (math.sqrt(5) - 1) / 2


def golden(f, lo, hi, iters=100):
    c = hi - PHI * (hi - lo)
    d = lo + PHI * (hi - lo)
    fc, fd = f(c), f(d)
    evals = 2  # the warm-up pair
    for _ in range(iters):
        if fc < fd:  # keep the interior point d as the new hi
            hi, d, fd = d, c, fc
            c = hi - PHI * (hi - lo)
            fc = f(c)  # one evaluation per step
        else:  # keep c as the new lo
            lo, c, fc = c, d, fd
            d = lo + PHI * (hi - lo)
            fd = f(d)
        evals += 1
    return (lo + hi) / 2, evals, hi - lo
```

listing 1235: python, phi defined from sqrt, the warm-up pair at 2 (source: dsa/samples-py/src/Ch36/golden.py, lines 16-34)

```lua
local function golden(f, lo, hi, iters)
  iters = iters or 100
  local c = hi - PHI * (hi - lo)
  local d = lo + PHI * (hi - lo)
  local fc, fd = f(c), f(d)
  local evals = 2
  for _ = 1, iters do
    if fc < fd then
      hi, d, fd = d, c, fc -- keep the interior point
      c = hi - PHI * (hi - lo)
      fc = f(c)
    else
      lo, c, fc = c, d, fd
      d = lo + PHI * (hi - lo)
      fd = f(d)
    end
    evals = evals + 1
  end
  return (lo + hi) / 2, evals, hi - lo
end
```

listing 1236: lua, the same loop over the shared phi constant (source: dsa/samples-lua/ch36_golden.lua, lines 12-31)

The smooth fixture (x - 1)^2 + 7 over (0, 100) pins the meter exactly: 102 evaluations, x within 1e-6 of 1, measured 1.0000000210734243, f(x) within 1e-12 of 7, and a final interval width of 2.220446049250313e-16, under the 1e-15 gate. The kink function from the ternary section rides the same driver and lands 2.000000000000001 on the same 102 evaluations. The constants family pins phi to its printed form and checks the shrink factor 100 * phi^100 = 1.2625133380638542e-19 within 1e-12 relative. That tolerance is itself a taught decision: 1e-30 relative would demand bit-identical libm pow across languages, below one ulp of a double, and lua's pow genuinely differs from the reference in the fifteenth significant digit, around 1e-15 relative. The python, c#, c, and java suites passed at tighter tolerances locally and stay as committed, the others assert the honest band.

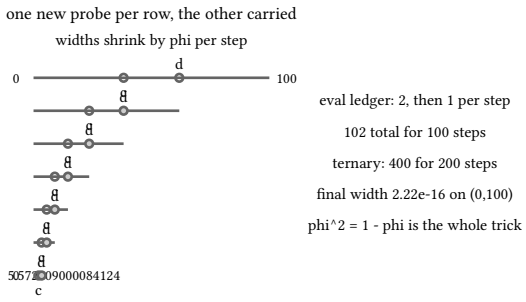


figure 311: the (0, 100) interval cascade, each row a step, the reused interior point shaded, one new probe per line

No finals problem in the six mined years pins this driver, so its row reads general technique. The prose claim worth keeping is the drop-in one: when a dual evaluation is expensive, the 2025/C driver of the last section runs unchanged on this loop and pays half the evaluations.

36.3. newton's method

Root finding by tangent lines. For f(x) = x^2 - 2 the update x <- x - (x^2 - 2) / (2x) divides by the derivative, and from 1.0 the iterates pin in all seven languages: 1, 1.5, 1.416666666666667, 1.414215686274510, 1.414213562374690, 1.414213562373095, five updates to the residual bound 1e-12. The pinned error column shows quadratic convergence, each error roughly the square of the previous: 4.1e-1, 8.6e-2, 2.5e-3, 2.1e-6, 1.6e-12. The second loop computes the reciprocal 1/a by x <- x(2 - ax), which divides nowhere inside the loop, the property that makes it hardware-interesting, and from 0.2 for a = 7 it walks 0.2, 0.12, 0.1392, 0.14276352, 0.142857081500467, 0.142857142857117, five updates to a measured final error of 2.6e-14.

The dry run: the fixtures are the sqrt 2 ladder from 1.0 and the reciprocal of 7 from 0.2, iterates pinned by the C# suite at 9 decimals with the same ladders in all seven.

1. The first update is one fraction: at x = 1 the residual is −1 on a slope of 2, and 1 - (-1) / 2 = 1.5.
2. The next tangent lands 1.5 - 0.25 / 3 = 1.4166666666666667, and three more updates close the ladder, 1.4142156862745099, 1.4142135623746899, 1.4142135623730951, five updates in all under the residual check before each one.
3. The error column squares on the way down, 8.6e-2, 2.5e-3, 2.1e-6, 1.6e-12: quadratic convergence as plain numbers.
4. The reciprocal loop divides nowhere: 0.2 × (2 - 7 × 0.2) = 0.12, then 0.12 × (2 - 7 × 0.12) = 0.1392, then 0.1392 × (2 - 7 × 0.1392) = 0.14276352, five updates to 0.142857142857117.
5. The identity case is free, a start at the exact root passes the first residual check and exits at 0 updates, and from −1.0 the same loop closes on −1.4142135623730951, the negative root.

update	x on x^2 - 2	error	x on 1 / 7
0	1	4.1e-1	0.2
1	1.5	8.6e-2	0.12
2	1.4166666666666667	2.5e-3	0.1392
3	1.4142156862745099	2.1e-6	0.14276352
4	1.4142135623746899	1.6e-12	0.142857081500467
5	1.4142135623730951	under 1e-12	0.142857142857117

Five updates close both ladders, and the listings below run the two loops in seven languages.

```c
static double newton_sqrt2(double x0, double *last, int cap) {
  double x = x0;
  updates = 0;
  for (int i = 0; i < cap; i++) {
    if (fabs(x * x - 2) < 1e-12)
      break;
    x = x - (x * x - 2) / (2 * x);
    updates++;
  }
  *last = x;
  return x;
}

static double newton_recip(double a, double x0, double *last, int cap) {
  double x = x0;
  updates = 0;
  for (int i = 0; i < cap; i++) {
    if (fabs(a * x - 1) < 1e-12)
      break;
    x = x * (2 - a * x);
    updates++;
  }
  *last = x;
  return x;
}
```

listing 1237: c, the two update loops, residual checked before each step (source: dsa/samples-c/src/Ch36/newton.c, lines 24-48)

```
func NewtonSqrt2(x0 float64) ([]float64, int) {
  it := []float64{x0}
  updates := 0
  x := x0
  for step := 0; step < 20; step++ {
    if math.Abs(x*x-2) < 1e-12 {
      break
    }
    x = x - (x*x-2)/(2*x)
    it = append(it, x)
    updates++
  }
  return it, updates
}

// NewtonReciprocal computes 1/a by x ← x*(2 - a*x) from x0: pure
// multiplication and subtraction, no division inside the loop, the
// property that makes it hardware-interesting.
func NewtonReciprocal(a, x0 float64) ([]float64, int) {
  it := []float64{x0}
  updates := 0
  x := x0
  for step := 0; step < 50; step++ {
    if math.Abs(a*x-1) < 1e-12 {
      break
    }
    x = x * (2 - a*x)
    it = append(it, x)
    updates++
  }
  return it, updates
}
```

listing 1238: go, the iterate slice returned whole for the ladder asserts (source: dsa/samples-go/ch36/newton.go, lines 8-39)

```
  static double newtonSqrt2(double x0, int cap) {
    double x = x0;
    updates = 0;
    for (int i = 0; i < cap; i++) {
      if (Math.abs(x * x - 2) < TOL)
        break;
      x = x - (x * x - 2) / (2 * x);
      updates++;
    }
    return x;
  }

  static double newtonRecip(double a, double x0, int cap) {
    double x = x0;
    updates = 0;
    for (int i = 0; i < cap; i++) {
      if (Math.abs(a * x - 1) < TOL)
        break;
      x = x * (2 - a * x);
      updates++;
    }
    return x;
  }
```

listing 1239: java, the two update loops, residual checked before each step (source: dsa/samples-java/src/Ch36/Newton.java, lines 21-43)

```
    public static IEnumerable<double> Sqrt2Iterates(double x0, double tol = 1e-12, int cap = 20)
    {
        double x = x0;
        yield return x;
        for (int i = 0; i < cap; i++)
        {
            if (Math.Abs(x * x - 2) < tol)
                yield break;
            x = x - (x * x - 2) / (2 * x);
            yield return x;
        }
    }

    public static (double X, int Updates) Sqrt2(double x0)
    {
        int updates = 0;
        foreach (double _ in Sqrt2Iterates(x0).Skip(1))
            updates++;
        return (Sqrt2Iterates(x0).Last(), updates);
    }

    public static IEnumerable<double> ReciprocalIterates(double a, double x0, double tol = 1e-12, int cap =
50)
    {
        double x = x0;
        yield return x;
        for (int i = 0; i < cap; i++)
        {
            if (Math.Abs(a * x - 1) < tol)
                yield break;
            x = x * (2 - a * x); // no division anywhere in the loop
            yield return x;
        }
    }
```

listing 1240: c#, both loops as iterate enumerables, the meter outside (source: dsa/samples/src/Ch36/Newton.cs, lines 10-42)

```javascript
export function newtonSqrt2(x0 = 1.0, tol = 1e-12, cap = 20) {
  let x = x0;
  const iterates = [x];
  let updates = 0;
  for (let i = 0; i < cap; i++) {
    if (Math.abs(x * x - 2) < tol) break;
    x = x - (x * x - 2) / (2 * x);
    iterates.push(x);
    updates += 1;
  }
  return { iterates, updates };
}

export function newtonRecip(a, x0, tol = 1e-12, cap = 50) {
  let x = x0;
  const iterates = [x];
  let updates = 0;
  for (let i = 0; i < cap; i++) {
    if (Math.abs(a * x - 1) < tol) break;
    x = x * (2 - a * x);
    iterates.push(x);
    updates += 1;
  }
  return { iterates, updates };
}
```

listing 1241: javascript, iterates collected, updates counted (source: dsa/samples-js/src/ch36-newton.mjs, lines 9-33)

```python
def newton_sqrt2(x0, cap=20):
    x = x0
    iters = [x]
    updates = 0
    for _ in range(cap):
        if abs(x * x - 2) < TOL:  # residual check before each update
            break
        x = x - (x * x - 2) / (2 * x)
        iters.append(x)
        updates += 1
    return iters, updates


def newton_recip(a, x0, cap=50):
    # x <- x(2 - a x) divides nowhere
    x = x0
    iters = [x]
    updates = 0
    for _ in range(cap):
        if abs(a * x - 1) < TOL:
            break
        x = x * (2 - a * x)
        iters.append(x)
        updates += 1
    return iters, updates
```

listing 1242: python, the division-free comment sits on the reciprocal loop (source: dsa/samples-py/src/Ch36/newton.py, lines 19-43)

```lua
local function newton_sqrt2(x0)
  x0 = x0 or 1.0
  local it = {x0}
  local updates = 0
  for _ = 1, 20 do
    if math.abs(x0 * x0 - 2) < 1e-12 then break end
    x0 = x0 - (x0 * x0 - 2) / (2 * x0)
    it[#it + 1] = x0
    updates = updates + 1
  end
  return it, updates
end

local function newton_recip(a, x0)
  local it = {x0}
  local updates = 0
  for _ = 1, 50 do
    if math.abs(a * x0 - 1) < 1e-12 then break end
    x0 = x0 * (2 - a * x0)
    it[#it + 1] = x0
    updates = updates + 1
  end
  return it, updates
end
```

listing 1243: lua, the same pair, prints pinned to 12 decimals (source: dsa/samples-lua/ch36_newton.lua, lines 6-29)

The stopping rule is residual-first: check |x^2 - 2| < 1e-12 before each update, so starting at the exact root exits with zero updates, the identity case every suite pins. The divergence basin is real and pinned too, from −1.0 the same loop converges to −1.414213562373095, the negative root, because the tangent at a negative point never crosses zero. What stays in prose is the double-root slowdown: on f(x) = x^2 the error halves per step instead of squaring, since the derivative vanishes at the root, and the honest fix there is a modified update or a different method entirely. Iterates print in the 12-decimal form %.12f, and every final value is checked against the language's own sqrt and division ground truths, not just against the pinned ladder.

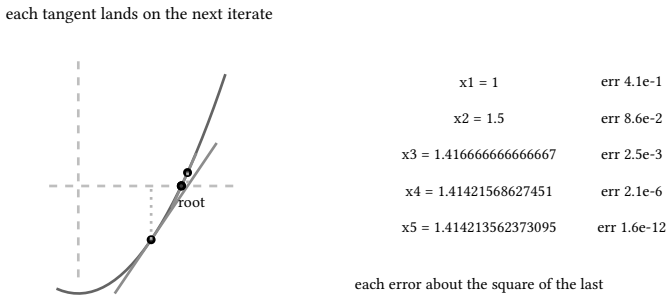


figure 312: the parabola y = x^2 - 2 with the tangent ladder from (1, −1) closing on the root, the error column squaring beside each step

The row's bcl entry is Math.Sqrt, the library call this loop reproduces when the hardware or the language gives you no sqrt and only multiplication is trusted.

36.4. simpson integration

The composite rule fits a parabola through every pair of panels: weights 1, 4, 2, 4, ..., 4, 1 over n even panels, the even-n precondition asserted at the door because the weights pair off. One parabola integrates cubics exactly, so x^2 over (0, 1) on n = 100 returns 1/3 and x^3 over (0, 2) returns 4, both residuals pure rounding, measured 0.3333333333333334 and 4.000000000000001. The adaptive variant compares the whole interval's parabola against its two halves, accepts when the gap is at most 15 eps, and on accept adds back the Richardson correction (left

+ right - whole) / 15, the classical extrapolation of the error estimate. On recursion eps halves and a depth cap of 24 stops runaway splits. Reference values are closed forms where they exist, 1/3, 4, 2 for sin over (0, pi) measured 1.9999999999999991, and the erf-based constant 0.746824132812427 for e^-x^2 over (0, 1): adaptive lands 0.7468241328124992 and composite n = 1000 lands 0.746824132812436, both inside the 1e-9 gate.

The dry run: the fixtures are the exact-through-cubics family, sin over (0, pi), and the e^-x^2 bump against the erf constant, asserted by the C# suite with the printed forms pinned as strings in Python and Java.

1. One panel is the whole kernel: x^2 over (0, 1) at n = 2 reads h / 3 × (0 + 4 × 0.25 + 1) = 0.5 / 3 × 2 = 1 / 3 exactly, the edge the suite pins at 12 decimals.
2. The adaptive rule on e^-x^2 prices the whole panel first, (1 + 4 × 0.7788007830714049 + 0.3678794411714423) / 6 = 0.7471804289095104.
3. Its halves price (1 + 4 × 0.9394130628134758 + 0.7788007830714049) over 12 at 0.4613710861937757, and (0.7788007830714049 + 4 × 0.5697828247309230 + 0.3678794411714423) over 12 at 0.2854842935972116.
4. The gap |0.7468553797909873 - 0.7471804289095104| = 3.25e-4 dwarfs the 15 eps = 1.5e-9 gate, both halves recurse at eps halved, and the recursion accepts its way down to 0.7468241328124992.
5. The composite road at n = 1000 lands 0.746824132812436, both inside 1e-9 of the erf reference 0.746824132812427, sin lands 1.9999999999999991 against 2, and the nine-decimal prints 0.333333333, 4.000000000, 2.000000000, 0.746824133 pin as strings.
6. The edges refuse at the door: a zero-width interval returns 0 and odd panel counts are rejected, exception in C#, assert in Python.

piece	folded sum	value
whole (0, 1)	(1 + 3.1152031322856195 + 0.3678794411714423) / 6	0.7471804289095104
left (0, 0.5)	(1 + 3.7576522512539032 + 0.7788007830714049) / 12	0.4613710861937757
right (0.5, 1)	(0.7788007830714049 + 2.279131298923692 + 0.3678794411714423) / 12	0.2854842935972116
halves	0.4613710861937757 + 0.2854842935972116	0.7468553797909873
gap vs gate	3.25e-4 against 15 × 1e-10	recurse

The erf reference holds both roads inside 1e-9, and the listings below ship the kernel, the composite weights, and the recursion in seven languages.

```c
static double simp(double a, double b, double fa, double fm, double fb) {
  return (b - a) / 6 * (fa + 4 * fm + fb);
}

static double composite(double (*f)(double), double a, double b, int panels) {
  double h = (b - a) / panels;
  double s = f(a) + f(b);
  for (int i = 1; i < panels; i++)
    s += f(a + i * h) * (i % 2 ? 4 : 2);
  return s * h / 3;
}

static double adaptive(double (*f)(double), double a, double b, double eps,
                       int depth) {
  double m = (a + b) / 2;
  double fa = f(a), fm = f(m), fb = f(b);
  double whole = simp(a, b, fa, fm, fb);
  double left = simp(a, m, fa, f((a + m) / 2), fm);
  double right = simp(m, b, fm, f((m + b) / 2), fb);
  if (depth ≥ 24 || fabs(left + right - whole) ≤ 15 * eps)
    return left + right + (left + right - whole) / 15;
  return adaptive(f, a, m, eps / 2, depth + 1) +
         adaptive(f, m, b, eps / 2, depth + 1);
}
```

listing 1244: c, the 1/6 kernel, the composite weights, the adaptive accept-or-recurse (source: dsa/samples-c/src/Ch36/simpson.c, lines 26-49)

```
// SimpsonComposite is the 1/6 rule with weights 1, 4, 2, ..., 4, 1
// over n panels; exact through degree 3.
func SimpsonComposite(f func(float64) float64, a, b float64,
  n int) float64 {
  if err := CheckEvenPanels(n); err ≠ nil {
    panic(err)
  }
  h := (b - a) / float64(n)
  s := f(a) + f(b)
  for i := 1; i < n; i++ {
    w := 2.0
    if i%2 == 1 {
      w = 4.0
    }
    s += f(a+float64(i)*h) * w
  }
  return s * h / 3
}

// simpson3 evaluates the 1/6 rule once over [a, b] from cached ends.
func simpson3(f func(float64) float64, a, b, fa, fm, fb float64) float64 {
  return (b - a) / 6 * (fa + 4*fm + fb)
}

// AdaptiveSimpson compares the whole against left + right halves,
// accepts when the gap ≤ 15 eps, recurses with eps/2 to the depth
// cap of 24, and richardson-corrects by (left+right-whole)/15 on
// accept.
func AdaptiveSimpson(f func(float64) float64, a, b, eps float64) float64 {
  var rec func(a, b, fa, fb, whole, eps float64, depth int) float64
  rec = func(a, b, fa, fb, whole, eps float64, depth int) float64 {
    m := (a + b) / 2
    fm := f(m)
    lm := (a + m) / 2
    rm := (m + b) / 2
    left := simpson3(f, a, m, fa, f(lm), fm)
    right := simpson3(f, m, b, fm, f(rm), fb)
    if depth ≥ 24 || math.Abs(left+right-whole) <= 15*eps {
      return left + right + (left+right-whole)/15
    }
    return rec(a, m, fa, fm, left, eps/2, depth+1) +
      rec(m, b, fm, fb, right, eps/2, depth+1)
  }
  fa, fb := f(a), f(b)
  whole := simpson3(f, a, b, fa, f((a+b)/2), fb)
  return rec(a, b, fa, fb, whole, eps, 0)
}
```

listing 1245: go, the closure recursion passing cached endpoint values down (source: dsa/samples-go/ch36/simpson.go, lines 18-64)

```
static double simp(double a, double b, double fa, double fm, double fb) {
  return (b - a) / 6 * (fa + 4 * fm + fb);
}

static double composite(DoubleUnaryOperator f, double a, double b, int panels) {
  double h = (b - a) / panels;
  double s = f.applyAsDouble(a) + f.applyAsDouble(b);
  for (int i = 1; i < panels; i++)
    s += f.applyAsDouble(a + i * h) * (i % 2 == 0 ? 4 : 2);
  return s * h / 3;
}

static double adaptive(DoubleUnaryOperator f, double a, double b, double eps, int depth) {
  double m = (a + b) / 2;
  double fa = f.applyAsDouble(a), fm = f.applyAsDouble(m), fb = f.applyAsDouble(b);
  double whole = simp(a, b, fa, fm, fb);
  double left = simp(a, m, fa, f.applyAsDouble((a + m) / 2), fm);
  double right = simp(m, b, fm, f.applyAsDouble((m + b) / 2), fb);
  if (depth >= 24 || Math.abs(left + right - whole) <= 15 * eps)
    return left + right + (left + right - whole) / 15;
  return adaptive(f, a, m, eps / 2, depth + 1)
       + adaptive(f, m, b, eps / 2, depth + 1);
}
```

listing 1246: java, the 1/6 kernel, the composite weights, the adaptive accept-or-recurse (source: dsa/samples-java/src/Ch36/Simpson.java, lines 31-53)

```
    public static double Composite(Func<double, double> f, double a, double b, int panels)
    {
        if (panels % 2 != 0)
            throw new ArgumentException("panel count must be even", nameof(panels));
        if (a == b)
            return 0;
        double h = (b - a) / panels;
        double s = f(a) + f(b);
        for (int i = 1; i < panels; i++)
            s += f(a + i * h) * (i % 2 == 1 ? 4 : 2);
        return s * h / 3;
    }

    /// <summary>Accept when whole vs halves differ by at most 15 eps, Richardson-corrected.</summary>
    public static double Adaptive(Func<double, double> f, double a, double b, double eps) =>
        AdaptiveRec(f, a, b, f(a), f(b), eps, 0);

    private static double AdaptiveRec(
        Func<double, double> f, double a, double b, double fa, double fb, double eps, int depth)
    {
        double mid = (a + b) / 2;
        double fm = f(mid);
        double whole = Whole(a, b, fa, fm, fb);
        double lm = (a + mid) / 2;
        double rm = (mid + b) / 2;
        double left = Whole(a, mid, fa, f(lm), fm);
        double right = Whole(mid, b, fm, f(rm), fb);
        if (depth ≥ 24 || Math.Abs(left + right - whole) ≤ 15 * eps)
            return left + right + (left + right - whole) / 15;
        return AdaptiveRec(f, a, mid, fa, fm, eps / 2, depth + 1)
            + AdaptiveRec(f, mid, b, fm, fb, eps / 2, depth + 1);
    }

    private static double Whole(double a, double b, double fa, double fm, double fb) =>
        (b - a) / 6 * (fa + 4 * fm + fb);
```

listing 1247: c#, odd panel counts refused by exception, the recursive accept with correction (source: dsa/samples/src/Ch36/Simpson.cs, lines 9-43)

```
export function simpsonComposite(f, a, b, n) {
  if (n % 2 !== 0) throw new Error("composite simpson needs even n");
  const h = (b - a) / n;
  let s = f(a) + f(b);
  for (let i = 1; i < n; i++) {
    s += f(a + i * h) * (i % 2 === 1 ? 4 : 2);
  }
  return (s * h) / 3;
}

export function adaptiveSimpson(f, a, b, eps, depth = 0, maxDepth = 24) {
  const simp = (lo, hi, flo, fmid, fhi) =>
    ((hi - lo) / 6) * (flo + 4 * fmid + fhi);
  const m = (a + b) / 2;
  const fa = f(a);
  const fm = f(m);
  const fb = f(b);
  const whole = simp(a, b, fa, fm, fb);
  const lm = (a + m) / 2;
  const rm = (m + b) / 2;
  const left = simp(a, m, fa, f(lm), fm);
  const right = simp(m, b, fm, f(rm), fb);
  if (depth ≥ maxDepth || Math.abs(left + right - whole) ≤ 15 * eps) {
    // accept with the richardson correction
    return left + right + (left + right - whole) / 15;
  }
  return adaptiveSimpson(f, a, m, eps / 2, depth + 1, maxDepth)
    + adaptiveSimpson(f, m, b, eps / 2, depth + 1, maxDepth);
}
```

listing 1248: javascript, the kernel as a local arrow function, depth default 24 (source: dsa/samples-js/src/ch36-simpson.mjs, lines 8-36)

```
def simpson_composite(f, a, b, panels):
    assert panels % 2 == 0  # the even-n precondition
    h = (b - a) / panels
    s = f(a) + f(b)
    for i in range(1, panels):
        s += f(a + i * h) * (4 if i % 2 else 2)
    return s * h / 3


def adaptive(f, a, b, eps, depth=0, maxdepth=24):
    def simp(lo, hi, flo, fmid, fhi):
        return (hi - lo) / 6 * (flo + 4 * fmid + fhi)

    mid = (a + b) / 2
    fa, fm, fb = f(a), f(mid), f(b)
    whole = simp(a, b, fa, fm, fb)
    lm = (a + mid) / 2
    rm = (mid + b) / 2
    left = simp(a, mid, fa, f(lm), fm)
    right = simp(mid, b, fm, f(rm), fb)
    if depth ≥ maxdepth or abs(left + right - whole) ≤ 15 * eps:
        # richardson-corrected accept
        return left + right + (left + right - whole) / 15
    return adaptive(f, a, mid, eps / 2, depth + 1, maxdepth) + adaptive(
        f, mid, b, eps / 2, depth + 1, maxdepth
    )
```

listing 1249: python, the even-n assert and the richardson-corrected return (source: dsa/samples-py/src/Ch36/simpson.py, lines 16-41)

```lua
local function simpson_composite(f, a, b, n)
  assert(n % 2 == 0, "panel count must be even")
  local h = (b - a) / n
  local s = f(a) + f(b)
  for i = 1, n - 1 do
    s = s + f(a + i * h) * (i % 2 == 1 and 4 or 2)
  end
  return s * h / 3
end

local function simp(a, b, fa, fm, fb)
  return (b - a) / 6 * (fa + 4 * fm + fb)
end

local function adaptive(f, a, b, eps, depth)
  eps = eps or 1e-10
  depth = depth or 0
  local m = (a + b) / 2
  local fa, fm, fb = f(a), f(m), f(b)
  local whole = simp(a, b, fa, fm, fb)
  local lm, rm = (a + m) / 2, (m + b) / 2
  local left = simp(a, m, fa, f(lm), fm)
  local right = simp(m, b, fm, f(rm), fb)
  if depth ≥ 24 or math.abs(left + right - whole) ≤ 15 * eps then
    return left + right + (left + right - whole) / 15
  end
  return adaptive(f, a, m, eps / 2, depth + 1)
       + adaptive(f, m, b, eps / 2, depth + 1)
end
```

listing 1250: lua, the same recursion, odd panels refused by assert (source: dsa/samples-lua/ch36_simpson.lua, lines 7-35)

The edge family pins the corners: a single panel n = 2 on x^2 is exactly 1/3, a zero-width interval integrates to 0, and the printed nine-decimal forms are pinned as strings, 0.333333333, 4.000000000, 2.000000000, 0.746824133. The printed-form pin is deliberate: it freezes the rounding behavior the prose quotes, so a language whose adaptive path lands one ulp off the composite path still prints the pinned digits. Java formats under Locale.ROOT so the decimal separator never goes locale-dependent, and Math ships no erf, so its reference constant comes from a 60-term Maclaurin series asserted against the adaptive landing to 1e-12.

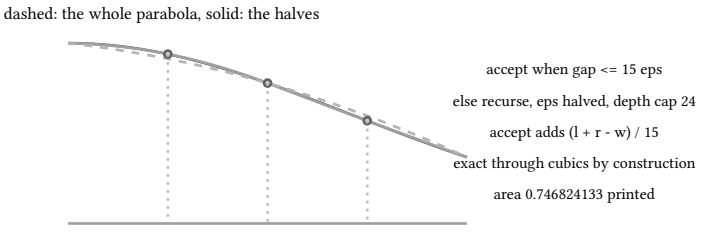


figure 313: e^-x^2 over (0, 1) with the first adaptive parabolas, split points accumulating where the curve bends

36.5. simulated annealing

No samples ship for this section, on purpose: annealing is a randomized heuristic whose every interesting claim is probabilistic, and a deterministic seven-language pin of a float walk would teach the wrong lesson. The idea earns the page. Model the problem as an energy landscape, current solution plus a neighbor move, and walk with the Metropolis rule: a move that lowers energy is always taken, a move raising energy by delta is taken with probability exp(-delta / T), and the temperature T decays geometrically so late steps become a pure local search.

The honest ranking stays: exact algorithms first, local search next, annealing last, when nothing structural is left to exploit.

The full treatment lives in book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics), where the rule runs in an integer form a seeded generator can pin exactly, with counter-exact runs, cooling and reheat schedules, and restart budgets, all asserted by the seven language suites.

36.6. LP duality and dual weights

The 2025/C pipe model is a max-min problem: stations split inflow among ducts, each duct forwards fixed percentages to higher-numbered targets, and the answer is the largest percentage of flow guaranteed to reach the reservoirs. Linear program duality turns that max-min into a one-dimensional minimization. Assign a weight w to every station and reservoir, reservoir weights summing to exactly 1, and require each duct to force w(upstream) >= the sum of percentage-scaled downstream weights, w(station) = max over its ducts. Weak duality says every such weight assignment upper-bounds the primal answer, strong duality says the best bound is tight. Because targets are always higher-numbered, the dual is a dag: fix the free reservoir weight t and the weights propagate bottom-up in one pass over decreasing station id, each station taking the max over its ducts. r = 1 has no free weight and is one pass at t = 1, r = 2 hands the pass to the ternary driver of the first section, 200 fixed iterations over the single free reservoir weight, each dual evaluation O(V), and r = 3 nests two ternary loops, stated in prose because the machinery is identical.

The dry run: the fixtures are F1, one station and two reservoirs, F2 the perfect duct, and F3 the interior kink, asserted by the C# suite and cross-checked by Python's dense grid of 2000001 points.

1. Fix the free weight t and F1 propagates in one pass: w(2) = t, w(3) = 1 - t, and station 1 takes the max of its ducts, 0.5 × t against 0.3 × t + 0.8 × (1 - t) = 0.8 - 0.5 × t.
2. The two duct values cross at 0.5 × t = 0.8 - 0.5 × t, that is t = 0.8: both bind at exactly 0.4, so the dual minimum is 0.4 and the answer prints 100 × 0.4 = 40.0000000000 at weights 0.8, 0.2.
3. F2 at t = 1 is a single pass, w(2) = 1 and station 1 takes max(1.0 × 1, 0.4 × 1) = 1.0, answering 100.0000000000.
4. F3 settles station 2 first, w(2) = w(3) = t, then station 1 takes max(0.5 × t + 0.2 × (1 - t), 1 - t), and the crossing solves 1.3 × t = 0.8, t = 8 / 13, both ducts binding at 5 / 13, the answer 100 × 5 / 13 = 38.4615384615.
5. The dense grid agrees: its F3 minimum reads 0.3846155 at t = 0.6153845, the grid point just under the exact 8 / 13, and its F1 minimum reproduces 0.4.

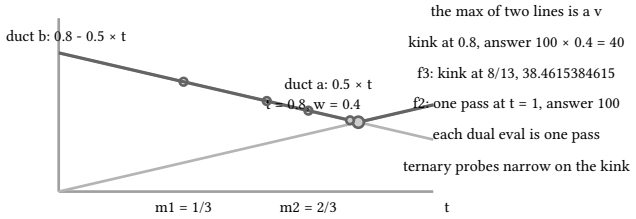


figure 314: F1′s dual as two duct lines over t, the max traced as a v, the ternary probes narrowing onto the kink at 0.8

F1′s 40.0000000000 with both ducts binding is the pinned landing, and the listings below run the bottom-up pass and the drivers in seven languages.

```
static double solve_dual(int s_count, int r_count, const struct Duct *ducts,
                         int dn, double t) {
  double w[NMAX + 2];
  for (int r = 0; r < r_count; r++)
    w[s_count + 1 + r] = r == 0 ? t : 1.0 - t;
  for (int i = s_count; i >= 1; i--) {
    double best = 0.0;
    for (int d = 0; d < dn; d++) {
      if (ducts[d].st != i)
        continue;
      double val = 0.0;
      for (int k = 0; k < ducts[d].nt; k++)
        val += ducts[d].pct[k] / 100.0 * w[ducts[d].to[k]];
      if (val > best)
        best = val;
    }
    w[i] = best;
  }
  return w[1];
}

// 200 fixed ternary iterations over the free weight (36.1's driver)
static double dual_min_ternary(int s_count, int r_count, const struct Duct *d,
                               int dn, double *t_out) {
  double lo = 0.0, hi = 1.0;
  for (int i = 0; i < 200; i++) {
    double m1 = lo + (hi - lo) / 3;
    double m2 = hi - (hi - lo) / 3;
    if (solve_dual(s_count, r_count, d, dn, m1) <
        solve_dual(s_count, r_count, d, dn, m2))
      hi = m2;
    else
      lo = m1;
  }
  double t = (lo + hi) / 2;
  *t_out = t;
  return solve_dual(s_count, r_count, d, dn, t);
}
```

listing 1251: c, the bottom-up dual evaluation feeding the ternary driver (source: dsa/samples-c/src/Ch36/lpdual.c, lines 38-75)

```
func DualEval(sCount, rCount int, ducts []Duct, t float64) float64 {
  w := make(map[int]float64, sCount+rCount)
  for r := 0; r < rCount; r++ {
    id := sCount + 1 + r
    if r == 0 {
      w[id] = t
    } else {
      w[id] = 1 - t
    }
  }
  for i := sCount; i ≥ 1; i-- {
    best := 0.0
    for _, d := range ducts {
      if d.Station ≠ i {
        continue
      }
      val := 0.0
      for _, o := range d.Outs {
        val += o.Pct / 100.0 * w[o.To]
      }
      if val > best {
        best = val
      }
    }
    w[i] = best
  }
  return w[1]
}

// DualOptimum minimizes the dual over the single free reservoir
// weight by the 36.1 ternary driver with 200 fixed iterations;
// r = 1 has no free weight and evaluates at t = 1.
func DualOptimum(sCount, rCount int, ducts []Duct) (float64,
  float64, error) {
  if rCount < 1 {
    return 0, 0, fmt.Errorf("lp dual needs at least one reservoir")
  }
  if rCount == 1 {
    return 1, DualEval(sCount, rCount, ducts, 1), nil
  }
  t := TernaryReal(func(t float64) float64 {
    return DualEval(sCount, rCount, ducts, t)
  }, 0, 1, 200)
  return t, DualEval(sCount, rCount, ducts, t), nil
}
```

listing 1252: go, dual-eval on the duct structs, r = 1 shortcut at t = 1 (source: dsa/samples-go/ch36/lpdual.go, lines 22-66)

```
  static double solveDual(int sCount, int rCount, Duct[] ducts, double t) {
    double[] w = new double[sCount + 2 + rCount];
    for (int r = 0; r < rCount; r++)
      w[sCount + 1 + r] = r == 0 ? t : 1.0 - t;
    for (int i = sCount; i ≥ 1; i--) {
      double best = 0.0;
      for (Duct d : ducts) {
        if (d.st() ≠ i)
          continue;
        double val = 0.0;
        for (int k = 0; k < d.to().length; k++)
          val += d.pct()[k] / 100.0 * w[d.to()[k]];
        if (val > best)
          best = val;
      }
      w[i] = best;
    }
    return w[1];
  }

  static double tOut;

  static double dualMinTernary(int sCount, int rCount, Duct[] d) {
    DoubleUnaryOperator f = t -> solveDual(sCount, rCount, d, t);
    double lo = 0.0, hi = 1.0;
    for (int i = 0; i < 200; i++) {
      double m1 = lo + (hi - lo) / 3;
      double m2 = hi - (hi - lo) / 3;
      if (f.applyAsDouble(m1) < f.applyAsDouble(m2))
        hi = m2;
      else
        lo = m1;
    }
    tOut = (lo + hi) / 2;
    return solveDual(sCount, rCount, d, tOut);
  }
```

listing 1253: java, the bottom-up dual evaluation over record ducts feeding the ternary driver (source: dsa/samples-java/src/Ch36/Lpdual.java, lines 23-58)

```
    public static double SolveDual(
        int stationCount,
        int reservoirCount,
        (int Station, (int To, int Pct)[] Outflows)[] ducts,
        double t)
    {
        var w = new Dictionary<int, double>();
        for (int r = 0; r < reservoirCount; r++)
            w[stationCount + 1 + r] = reservoirCount == 1 ? 1.0 : r == 0 ? t : 1.0 - t;
        for (int i = stationCount; i ≥ 1; i--) // targets always higher: bottom-up pass
        {
            double best = 0.0;
            foreach ((int station, (int to, int pct)[] outflows) in ducts)
                if (station == i)
                    best = Math.Max(best, outflows.Sum(o => o.pct / 100.0 * w[o.to]));
            w[i] = best;
        }
        return w[1];
    }

    /// <summary>200 fixed ternary iterations over t, mirroring the ch36 driver.</summary>
    public static Solution Optimal(
        int stationCount, int reservoirCount, (int Station, (int To, int Pct)[] Outflows)[] ducts)
    {
        double lo = 0.0;
        double hi = 1.0;
        for (int i = 0; i < 200; i++)
        {
            double m1 = lo + (hi - lo) / 3;
            double m2 = hi - (hi - lo) / 3;
            if (SolveDual(stationCount, reservoirCount, ducts, m1) < SolveDual(stationCount,
reservoirCount, ducts, m2))
                hi = m2;
            else
                lo = m1;
        }
        double t = (lo + hi) / 2;
        var weights = new Dictionary<int, double>();
        for (int r = 0; r < reservoirCount; r++)
            weights[stationCount + 1 + r] = reservoirCount == 1 ? 1.0 : r == 0 ? t : 1.0 - t;
        for (int i = stationCount; i ≥ 1; i--)
        {
            double best = 0.0;
            foreach ((int station, (int to, int pct)[] outflows) in ducts)
                if (station == i)
                    best = Math.Max(best, outflows.Sum(o => o.pct / 100.0 * weights[o.to]));
            weights[i] = best;
        }
        double optimum = SolveDual(stationCount, reservoirCount, ducts, t);
        return new Solution(100 * optimum, t, weights);
    }
```

listing 1254: c#, solve-dual over duct tuples, the optimum returning weights and t (source: dsa/samples/src/Ch36/LpDual.cs, lines 14-63)

```
export function solveDual(sCount, rCount, ducts, t) {
  // t is the weight of the first reservoir; the last takes 1 - t
  const w = new Array(sCount + rCount + 1).fill(0);
  for (let r = 0; r < rCount; r++) {
    const rid = sCount + 1 + r;
    w[rid] = r === 0 ? t : 1.0 - t;
  }
  for (let i = sCount; i ≥ 1; i--) {
    let best = 0.0;
    for (const [st, outs] of ducts) {
      if (st =/= i) continue;
      let val = 0.0;
      for (const [o, p] of outs) val += (p / 100.0) * w[o];
      best = Math.max(best, val);
    }
    w[i] = best;
  }
  return w[1];
}

export function dualMinTernary(sCount, rCount, ducts, iters = 200) {
  let lo = 0.0;
  let hi = 1.0;
  for (let i = 0; i < iters; i++) {
    const m1 = lo + (hi - lo) / 3;
    const m2 = hi - (hi - lo) / 3;
    if (solveDual(sCount, rCount, ducts, m1)
      < solveDual(sCount, rCount, ducts, m2)) {
      hi = m2;
    } else {
      lo = m1;
    }
  }
  const t = (lo + hi) / 2;
  return [t, solveDual(sCount, rCount, ducts, t)];
}
```

listing 1255: javascript, weights in a flat array, ducts as pairs (source: dsa/samples-js/src/ch36-lpdual.mjs, lines 12-47)

```python
def solve_dual(s_count, r_count, ducts, t):
    # t is the weight of the first reservoir, the last takes 1 - t
    w = {}
    for idx in range(r_count):
        w[s_count + 1 + idx] = t if idx == 0 else 1.0 - t
    for station in range(s_count, 0, -1):  # targets are higher, so one
        best = 0.0  # descending pass settles every station
        for src, outs in ducts:
            if src != station:
                continue
            val = sum(p / 100.0 * w[o] for o, p in outs)
            best = max(best, val)
        w[station] = best
    return w[1]


def dual_min_ternary(s_count, r_count, ducts, iters=200):
    lo, hi = 0.0, 1.0
    for _ in range(iters):
        m1 = lo + (hi - lo) / 3
        m2 = hi - (hi - lo) / 3
        if solve_dual(s_count, r_count, ducts, m1) < solve_dual(
            s_count, r_count, ducts, m2
        ):
            hi = m2
        else:
            lo = m1
    t = (lo + hi) / 2
    return t, solve_dual(s_count, r_count, ducts, t)
```

listing 1256: python, the descending station pass with the max over ducts (source: dsa/samples-py/src/Ch36/lpdual.py, lines 15-43)

```
local function solve_dual(s_count, r_count, ducts, t)
  local w = {}
  for r = 0, r_count - 1 do
    w[s_count + 1 + r] = (r == 0) and t or 1.0 - t
  end
  for i = s_count, 1, -1 do -- bottom-up over decreasing station id
    local best = 0.0
    for _, d in ipairs(ducts) do
      if d[1] == i then
        local val = 0.0
        for _, out in ipairs(d[2]) do
          val = val + out[2] / 100.0 * w[out[1]]
        end
        best = math.max(best, val)
      end
    end
    w[i] = best
  end
  return w[1]
end

local function dual_min_ternary(s_count, r_count, ducts, iters)
  iters = iters or 200
  local lo, hi = 0.0, 1.0
  for _ = 1, iters do
    local m1 = lo + (hi - lo) / 3
    local m2 = hi - (hi - lo) / 3
    if solve_dual(s_count, r_count, ducts, m1)
       < solve_dual(s_count, r_count, ducts, m2) then
      hi = m2
    else
      lo = m1
    end
  end
  local t = (lo + hi) / 2
  return t, solve_dual(s_count, r_count, ducts, t)
end
```

listing 1257: lua, ducts as nested tables, the same two functions (source: dsa/samples-lua/ch36_lpdual.lua, lines 11-47)

Answers print as percentages in the %.10f form and assert within 1e-6 of the exact rationals. F1, the crafted 2025/C case, is one station and two reservoirs with ducts 50 percent to reservoir 2 and 30 to 2 plus 80 to 3: the answer is 40.0000000000 at weights w(2) = 0.8 and w(3) = 0.2, and complementary slackness is visible, at the optimum both ducts of the station bind at exactly 0.4. F2 is the r = 1 case, a perfect 100 percent duct against a 40 percent one, answering 100.0000000000 in a single pass. F3 is the interior kink, two stations and two reservoirs, answering 38.4615384615 = 100 times 5/13 at w(3) = 8/13 with both of station 1's ducts binding at the crossing. The cross-check is a deterministic dense grid of 2000001 points over the free weight, step 5e-7: it reproduces F1's minimum 0.4 and F3's minimum 0.3846155 at t = 0.6153845, which is a multiple of 1/2000000, not of 1/2000, the grid is genuinely that dense.

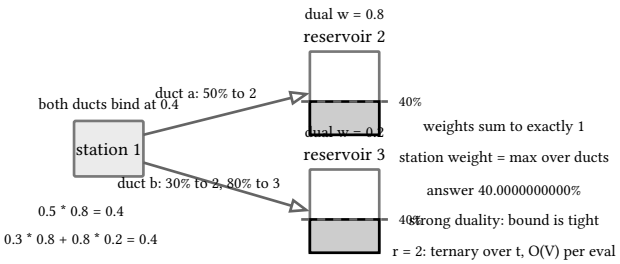


figure 315: F1′s station 1 splitting into two ducts, the percentage arrows, both reservoir tanks filling to the equalized 40 percent line, the dual weights on the tank rims

The application is icpc 2025 problem C (book 10, chapter 13), the anchor for the whole construction: the primal formulation, the dual collapse to bottom-up max propagation on this dag, and the final one-dimensional search are all its solution's shape, and the icpc chapter carries the full recovery algorithm.

36.7. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's five sample files per language, embedded test scripts included where the language embeds them:

language	build sloc	container dependency	boundary note
c	422	libc math only	function pointers for the drivers, the golden pow pin asserted at tighter than 1e-12 relative
go	191	fmt and math	dual-eval errors returned not thrown, ternary real exported for the lpdual driver
java	402	jdk 27 stdlib, java.util.function	DoubleUnaryOperator where c takes function pointers, Math ships no erf so the reference runs a 60-term Maclaurin series, nine-decimal prints under Locale.ROOT, the golden width pinned at the double-ulp floor
c#	205	bcl only, tuples	iterates as enumerables in newton, exceptions refuse odd simpson panels, long eval meter
javascript	146	node stdlib	object returns carry the meters, floor division written by hand
python	311	stdlib math	printed forms pinned as strings, the dense grid cross-check runs 2000001 points in-suite
lua	309	lib.lua harness	in-file approx helper, the pow pin documented at 1e-12 relative with the libm caveat

sources: cp-algorithms, "Ternary Search", cp-algorithms.com/num_methods/ternary_search.html, "Newton's method for finding roots", cp-algorithms.com/num_methods/roots_newton.html, "Integration by Simpson's formula", cp-algorithms.com/num_methods/simpson-integration.html, and "Simulated Annealing", cp-algorithms.com/num_methods/simulated_annealing.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Golden-section search has no cp-algorithms page and is derived here from the ternary article, cited as such. LP duality likewise has no cp-algorithms article. Application sources: icpc 2025 problem C (book 10, chapter 13) and icpc 2022 problem T (book 10, chapter 11). Sample behavior verified by the seven suite gates scoped to chapter 36: c 5 files and 74 checks, go 15 test functions, java 5 files and 74 checks under run-java-samples, c# 21 facts, javascript 19 tests and 56 asserts, python 5 files and 59 asserts, lua 18 checks, zero skipped.

37. computational geometry ii

Chapter 23 built the predicates: the orientation cross, the monotone chain hull, closest pair, and the shamos-hoey boolean sweep. Chapter 24 added the exact lattice arithmetic and ray casting. This chapter takes those as read and adds the layer above them: circles as first-class objects with tangent lines and radical axes, region algebra over half-planes, polygons summed and measured, and the reporting sweep that lists every crossing instead of stopping at the first one. Everything runs in float64 across the seven languages, decision predicates at eps 1e-9, pinned outputs rounded to six decimals and asserted within 1e-6, negative zero normalized to zero. The anchor application is icpc 2018 problem G (book 10, chapter 9), whose panda preserve builds voronoi cells by exactly this chapter's clip.

37.1. line and circle intersections, tangents

Three o(1) constructions carry the section. Line against circle: parametrize the line and project, which turns the intersection into a quadratic in t whose discriminant classifies the answer, negative for a miss, zero for tangency, positive for two hits. Circle against circle: the radical line, project both centers onto the center axis at distance a = (r1^2 - r2^2 + d^2) / (2d) and erect the half-chord height h = sqrt(r1^2 - a^2), giving both points at once and the tangency cases when h collapses. Common tangents: from the base angle atan2(dy, dx), the outer pair sits at acos((r1 - r2) / d) off the axis, the inner pair at acos((r1 + r2) / d), and the count reads off the geometry, 0 when one circle nests inside the other, 1 at the internal touch, 2 while they overlap, 3 at the outer touch, 4 when separate. Tangent lines report normalized so the seven languages pin identical lists: unit normal (a, b), constant c with a times x plus b times y equal to c, the first nonzero of (a, b) positive, the list sorted by (a, b, c). Identical centers report the sentinel −1.

The dry run: the fixtures are the discriminant family on line (0,0)-(4,0), the radical-line pairs around (0,0) r 2, and the tangent quartet at distance 6, asserted by the C# suite and pinned to the same six-decimal values by the six sibling suites, java rounding through a sign-split r6 because Math.round is half-up where c's round is half-away-from-zero.

1. The discriminant classifies before any sqrt runs: center (2,2) r 1 sits 2 away from the line, miss, and (2,1) r 1 sits exactly 1 away, the tangent point (2, 0).
2. Center (2,0.5) r 1 sits 0.5 away, so the half-chord is sqrt(1 - 0.25) = 0.866025 either side of the foot (2,0): the pair (1.133975, 0) and (2.866025, 0).
3. Circle (0,0) r 2 against (3,0) r 1 projects a = (4 - 1 + 9) / 6 = 2 along the center axis with h = sqrt(4 - 4) = 0: the touch (2, 0).
4. Against (2,0) r 2: a = (4 - 4 + 4) / 4 = 1 and h = sqrt(4 - 1) = 1.732051, the pair (1, 1.732051) and (1, −1.732051), while (5,0) r 1 is too far and (1,0) r 1 inside r 3 nests, both empty.
5. Tangents (0,0) r 2 against (6,0) r 1: the outer pair sits acos(1/6) off the axis, normalized (0.166667, 0.986013, 2) and its mirror, and the inner pair acos(3/6) = 60 degrees off, (0.5, 0.866025, 2) and its mirror.
6. The count family: equal circles at distance 4 fold the inner pair into one line, 3 after normalization, the overlap at distance 2 keeps the 2 outer lines, the internal touch at distance 3 lands 3, and concentric input answers the sentinel.

probe	one number	verdict	pinned
line vs (2,2) r 1	distance 2 > 1	miss	empty
line vs (2,1) r 1	distance 1 = 1	tangent	(2, 0)
line vs (2,0.5) r 1	sqrt(0.75) = 0.866025	cut	(1.133975, 0), (2.866025, 0)
circles vs (3,0) r 1	h = sqrt(4 - 4) = 0	touch	(2, 0)
circles vs (2,0) r 2	h = sqrt(4 - 1) = 1.732051	cut	(1, 1.732051) and (1, −1.732051)
circles vs (5,0) r 1	d = 5 > 3	far	empty
circles r 3 vs (1,0) r 1	d = 1 < 2	nested	empty

The quartet of four normalized lines is the pinned landing, and the listings below build all three constructions in seven languages.

```c
static int tangents(double x1, double y1, double r1, double x2, double y2,
                    double r2, double out[4][3]) {
  double dx = x2 - x1, dy = y2 - y1;
  double d = hypot(dx, dy);
  if (d < EPS)
    return -1;
  int cnt = 0;
  double base = atan2(dy, dx);
  if (fabs(r1 - r2) < d - EPS) { // outer pair
    double ang = acos((r1 - r2) / d);
    for (int s = 0; s < 2; s++) {
      double th = base + (s ? -ang : ang);
      double a = cos(th), b = sin(th);
      out[cnt][0] = a;
      out[cnt][1] = b;
      out[cnt][2] = a * x1 + b * y1 + r1;
      cnt++;
    }
  } else if (fabs(fabs(r1 - r2) - d) < EPS) { // outer touch
    double a = cos(base), b = sin(base);
    out[cnt][0] = a;
    out[cnt][1] = b;
    out[cnt][2] = a * x1 + b * y1 + r1;
    cnt++;
  }
  if (r1 + r2 < d - EPS) { // inner pair
    double ang = acos((r1 + r2) / d);
    for (int s = 0; s < 2; s++) {
      double th = base + (s ? -ang : ang);
      double a = cos(th), b = sin(th);
      out[cnt][0] = a;
      out[cnt][1] = b;
      out[cnt][2] = a * x1 + b * y1 + r1;
      cnt++;
    }
  } else if (fabs(r1 + r2 - d) < EPS) { // inner touch
    double a = cos(base), b = sin(base);
    out[cnt][0] = a;
    out[cnt][1] = b;
    out[cnt][2] = a * x1 + b * y1 + r1;
    cnt++;
  }
```

listing 1258: c, the tangent count classifier plus the two angle loops (source: dsa/samples-c/src/Ch37/circles.c, lines 90-131)

```
func Tangents(c1 Point, r1 float64, c2 Point, r2 float64) [][3]float64 {
  dx, dy := c2.X-c1.X, c2.Y-c1.Y
  d := math.Hypot(dx, dy)
  if d < geoEps {
    return nil
  }
  base := math.Atan2(dy, dx)
  var out [][3]float64
  add := func(theta float64) {
    nx, ny := math.Cos(theta), math.Sin(theta)
    out = append(out, [3]float64{nx, ny, nx*c1.X + ny*c1.Y + r1})
  }
  if math.Abs(r1-r2) < d-geoEps { // external pair
    ang := math.Acos((r1 - r2) / d)
    add(base + ang)
    add(base - ang)
  } else if math.Abs(math.Abs(r1-r2)-d) < geoEps {
    add(base) // internal touch
  }
  if r1+r2 < d-geoEps { // internal pair
    ang := math.Acos((r1 + r2) / d)
    add(base + ang)
    add(base - ang)
  } else if math.Abs(r1+r2-d) < geoEps {
    add(base) // external touch
  }
  norm := make([][3]float64, 0, len(out))
  for _, l := range out {
    ln := math.Hypot(l[0], l[1])
    a, b, c := l[0]/ln, l[1]/ln, l[2]/ln
    if a < -geoEps || (math.Abs(a) < geoEps && b < -geoEps) {
      a, b, c = -a, -b, -c
    }
    norm = append(norm, [3]float64{r6(a), r6(b), r6(c)})
  }
  sort.Slice(norm, func(i, j int) bool {
    if norm[i][0] != norm[j][0] {
      return norm[i][0] < norm[j][0]
    }
    if norm[i][1] != norm[j][1] {
      return norm[i][1] < norm[j][1]
    }
    return norm[i][2] < norm[j][2]
  })
  return norm
}
```

listing 1259: go, the add closure, normalized and sorted in place (source: dsa/samples-go/ch37/circles.go, lines 63-108)

```
  static int tangents(double x1, double y1, double r1, double x2, double y2, double r2) {
    double dx = x2 - x1, dy = y2 - y1;
    double d = Math.hypot(dx, dy);
    if (d < EPS)
      return -1;
    int cnt = 0;
    double base = Math.atan2(dy, dx);
    if (Math.abs(r1 - r2) < d - EPS) { // outer pair
      double ang = Math.acos((r1 - r2) / d);
      for (int s = 0; s < 2; s++) {
        double th = base + (s != 0 ? -ang : ang);
        double a = Math.cos(th), b = Math.sin(th);
        lines[cnt][0] = a;
        lines[cnt][1] = b;
        lines[cnt][2] = a * x1 + b * y1 + r1;
        cnt++;
      }
    } else if (Math.abs(Math.abs(r1 - r2) - d) < EPS) { // outer touch
      double a = Math.cos(base), b = Math.sin(base);
      lines[cnt][0] = a;
      lines[cnt][1] = b;
      lines[cnt][2] = a * x1 + b * y1 + r1;
      cnt++;
    }
    if (r1 + r2 < d - EPS) { // inner pair
      double ang = Math.acos((r1 + r2) / d);
      for (int s = 0; s < 2; s++) {
        double th = base + (s != 0 ? -ang : ang);
        double a = Math.cos(th), b = Math.sin(th);
        lines[cnt][0] = a;
        lines[cnt][1] = b;
        lines[cnt][2] = a * x1 + b * y1 + r1;
        cnt++;
      }
    } else if (Math.abs(r1 + r2 - d) < EPS) { // inner touch
      double a = Math.cos(base), b = Math.sin(base);
      lines[cnt][0] = a;
      lines[cnt][1] = b;
      lines[cnt][2] = a * x1 + b * y1 + r1;
      cnt++;
    }
```

listing 1260: java, the tangent count classifier plus the two angle loops, the (a, b, c) order an insertion pass (source: dsa/samples-java/src/Ch37/Circles.java, lines 86-126)

```
    public static List<(double A, double B, double C)>? Tangents(Pt c1, double r1, Pt c2, double r2)
    {
        double dx = c2.X - c1.X;
        double dy = c2.Y - c1.Y;
        double d = Math.Sqrt(dx * dx + dy * dy);
        if (d < Eps)
            return null;
        var outList = new List<(double, double, double)>();
        double baze = Math.Atan2(dy, dx);
        if (Math.Abs(r1 - r2) < d - Eps)
            AddPair(Math.Acos((r1 - r2) / d)); // outer tangents
        else if (Math.Abs(Math.Abs(r1 - r2) - d) < Eps)
            AddOne(baze);
        if (r1 + r2 < d - Eps)
            AddPair(Math.Acos((r1 + r2) / d)); // inner tangents
        else if (Math.Abs(r1 + r2 - d) < Eps)
            AddOne(baze);
        var normalized = outList
            .Select(t =>
            {
                double len = Math.Sqrt(t.Item1 * t.Item1 + t.Item2 * t.Item2);
                double a = t.Item1 / len;
                double b = t.Item2 / len;
                double c = t.Item3 / len;
                if (a < -Eps || (Math.Abs(a) < Eps && b < -Eps))
                    (a, b, c) = (-a, -b, -c); // first nonzero of (a, b) positive
                return (a, b, c);
            })
            .OrderBy(t => Math.Round(t.Item1, 6))
            .ThenBy(t => Math.Round(t.Item2, 6))
            .ThenBy(t => Math.Round(t.Item3, 6))
            .ToList();
        return normalized;

        void AddPair(double ang)
        {
            foreach (double sign in new[] { 1.0, -1.0 })
                AddOne(baze + sign * ang);
        }

        void AddOne(double th)
        {
            double a = Math.Cos(th);
            double b = Math.Sin(th);
            outList.Add((a, b, a * c1.X + b * c1.Y + r1));
        }
    }
```

listing 1261: c#, local functions emit the pair, the normalization a linq pass (source: dsa/samples/src/Ch37/Geometry2.cs, lines 82-128)

```
export function tangents(c1, r1, c2, r2) {
  const dx = c2[0] - c1[0];
  const dy = c2[1] - c1[1];
  const d = Math.hypot(dx, dy);
  if (d < EPS) return -1; // identical centers: infinite or none
  const out = [];
  const base = Math.atan2(dy, dx);
  const push = (th) => {
    const nx = Math.cos(th);
    const ny = Math.sin(th);
    out.push([nx, ny, nx * c1[0] + ny * c1[1] + r1]);
  };
  // external tangents: circles on the same side
  if (Math.abs(r1 - r2) < d - EPS) {
    const ang = Math.acos((r1 - r2) / d);
    push(base + ang);
    push(base - ang);
  } else if (Math.abs(Math.abs(r1 - r2) - d) < EPS) {
    push(base);
  }
  // internal tangents: circles on opposite sides
  if (r1 + r2 < d - EPS) {
    const ang = Math.acos((r1 + r2) / d);
    push(base + ang);
    push(base - ang);
  } else if (Math.abs(r1 + r2 - d) < EPS) {
    push(base);
  }
  return out.map(([a, b, c]) => {
    const ln = Math.hypot(a, b);
    let [na, nb, nc] = [a / ln, b / ln, c / ln];
    if (na < -EPS || (Math.abs(na) < EPS && nb < -EPS)) {
      na = -na;
      nb = -nb;
      nc = -nc;
    }
    return [r6(na), r6(nb), r6(nc)];
  }).sort((u, v) => u[0] - v[0] || u[1] - v[1] || u[2] - v[2]);
}
```

listing 1262: javascript, the map over emitted lines normalizes, the sort chain orders (source: dsa/samples-js/src/ch37-circles.mjs, lines 64-102)

```
def tangents(c1, r1, c2, r2):
    # lines (a, b, c) with a*x + b*y = c; None marks identical centers
    dx, dy = c2[0] - c1[0], c2[1] - c1[1]
    d = math.hypot(dx, dy)
    if d < EPS:
        return None  # the infinite sentinel
    out = []
    base = math.atan2(dy, dx)
    if abs(r1 - r2) < d - EPS:  # outer tangents, circles on one side
        ang = math.acos((r1 - r2) / d)
        for sign in (1, -1):
            th = base + sign * ang
            a, b = math.cos(th), math.sin(th)
            out.append((a, b, a * c1[0] + b * c1[1] + r1))
    elif abs(abs(r1 - r2) - d) < EPS:  # one outer tangent at the touch
        a, b = math.cos(base), math.sin(base)
        out.append((a, b, a * c1[0] + b * c1[1] + r1))
    if r1 + r2 < d - EPS:  # inner tangents, circles on both sides
        ang = math.acos((r1 + r2) / d)
        for sign in (1, -1):
            th = base + sign * ang
            a, b = math.cos(th), math.sin(th)
            out.append((a, b, a * c1[0] + b * c1[1] + r1))
    elif abs(r1 + r2 - d) < EPS:  # one inner tangent at the touch
        a, b = math.cos(base), math.sin(base)
        out.append((a, b, a * c1[0] + b * c1[1] + r1))
    norm = []
    for a, b, c in out:  # unit normal, first nonzero positive, sorted
        length = math.hypot(a, b)
        a, b, c = a / length, b / length, c / length
        if a < -EPS or (abs(a) < EPS and b < -EPS):
            a, b, c = -a, -b, -c
        norm.append((r6(a), r6(b), r6(c)))
    return sorted(norm)
```

listing 1263: python, the four branches with their tangency singles (source: dsa/samples-py/src/Ch37/circles.py, lines 60-93)

```lua
local function tangents(c1, r1, c2, r2)
  local dx, dy = c2[1] - c1[1], c2[2] - c1[2]
  local d = math.sqrt(dx * dx + dy * dy)
  if d < EPS then return -1 end
  local out = {}
  local base = math.atan(dy, dx)
  local function emit(theta)
    local nx, ny = math.cos(theta), math.sin(theta)
    out[#out + 1] = {nx, ny, nx * c1[1] + ny * c1[2] + r1}
  end
  if math.abs(r1 - r2) < d - EPS then -- outer pair
    local ang = math.acos((r1 - r2) / d)
    emit(base + ang)
    emit(base - ang)
  elseif math.abs(math.abs(r1 - r2) - d) < EPS then
    emit(base) -- one outer tangent at the touch
  end
  if r1 + r2 < d - EPS then -- inner pair
    local ang = math.acos((r1 + r2) / d)
    emit(base + ang)
    emit(base - ang)
  elseif math.abs(r1 + r2 - d) < EPS then
    emit(base) -- one inner tangent at the touch
  end
  local norm = {}
  for _, l in ipairs(out) do
    local a, b, c = l[1], l[2], l[3]
    local ln = math.sqrt(a * a + b * b)
    a, b, c = a / ln, b / ln, c / ln
    if a < -EPS or (math.abs(a) < EPS and b < -EPS) then
      a, b, c = -a, -b, -c
    end
    norm[#norm + 1] = {r6(a), r6(b), r6(c)}
  end
  table.sort(norm, function(x, y)
    if x[1] ~= y[1] then return x[1] < y[1] end
    if x[2] ~= y[2] then return x[2] < y[2] end
    return x[3] < y[3]
  end)
  return norm
end
```

listing 1264: lua, emit closure, sign fix, three-key sort (source: dsa/samples-lua/ch37_circles.lua, lines 60-100)

The fixtures pin every family. Line (0,0)-(4,0) against center (2,2) r 1 misses, against (2,1) r 1 touches at (2, 0), against (2,0.5) r 1 cuts at (1.133975, 0) and (2.866025, 0). Circle (0,0) r 2 against (3,0) r 1 touches at (2, 0), against (2,0) r 2 cuts at (1, 1.732051) and (1, −1.732051), against (5,0) r 1 is too far, against (1,0) r 1 nests. The tangent family pins the normalized lists whole: (0,0) r 2 against (6,0) r 1 gives the 4 lines (0.166667, −0.986013, 2), (0.166667, 0.986013, 2), (0.5, −0.866025, 2), (0.5, 0.866025, 2), the equal circles at distance 4 give 3 after normalization, the overlapping pair at distance 2 gives the 2 outer lines, and the internal touch at distance 3 gives 3. Concentric circles report the sentinel and zero intersection points. Javascript carries all the trig in Number with a comment saying so: the pinned six-decimal values sit far from any double ambiguity, and the assert tolerance is the same 1e-6 everywhere.

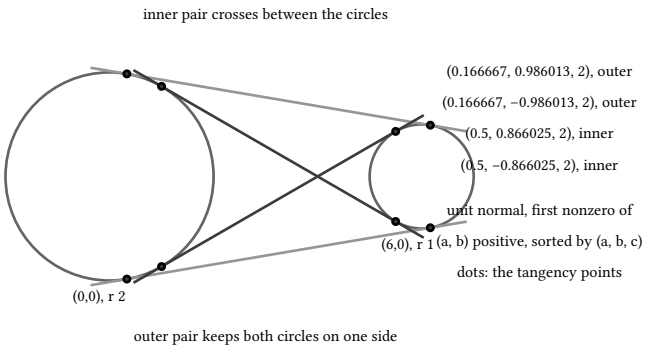


figure 316: two circles at distance 6 with the four common tangent lines labeled by their normalized (a, b, c)

Coverage questions under circle distance are the natural consumers, but no finals problem in the six mined years reduces to these constructions, so the row reads general technique.

37.2. the length of a union of segments

Split every segment into two events, its left end tagged +1 and its right end tagged −1, sort all 2n events by coordinate with opens before closes at a tie, and sweep left to right adding x minus last to the total whenever the coverage counter is positive. The core is fifteen lines and the whole file under 120 sloc. The same length falls out of book 9, ch 22 (practical data structures and algorithms, coordinate compression and intervals)‘s merged intervals, and the cross-name is the point: the event sweep is the stream form, the one that generalizes to weighted coverage counts and repeated sweeps over changing sets.

The dry run: the fixture is (1,4), (2,6), (8,10) with the nested and float families beside, asserted by the C# suite and pinned by the five sibling suites.

1. The six events sort to (1,+1), (2,+1), (4,-1), (6,-1), (8,+1), (10,-1), opens before closes at a tie.
2. The counter is positive across 1 to 2, 2 to 4, and 4 to 6, so the sweep pays 2 - 1 = 1, 4 - 2 = 2, 6 - 4 = 2.
3. The counter is 0 across the gap to 8, which pays nothing, and the last span pays 10 - 8 = 2: 1 + 2 + 2 + 2 = 7.
4. The nested family (0,3), (1,2), (3,5) exercises the tie: the open at 3 sorts before the close, the touching ends merge, and the spans total 1 + 1 + 1 + 2 = 5.
5. The floats (0.5,1.5) and (1.0,2.5) pay 0.5 + 0.5 + 1.0 = 2.0, the overlap counted once.
6. A single point covers 0, empty input covers 0, and the unsorted copy of the first family still totals 7.

event	counter after	added	total
(1, +1)	1	0	0
(2, +1)	2	2 - 1 = 1	1
(4, −1)	1	4 - 2 = 2	3
(6, −1)	0	6 - 4 = 2	5
(8, +1)	1	0	5
(10, −1)	0	10 - 8 = 2	7

The 7 over two covered spans and one gap is the pinned landing, and the listings below run the sweep in seven languages.

```c
static int cmp_ev(const void *pa, const void *pb) {
  int a = *(const int *)pa, b = *(const int *)pb;
  if (ev_x[a] != ev_x[b])
    return ev_x[a] < ev_x[b] ? -1 : 1;
  return ev_d[a] < ev_d[b] ? -1 : 1;
}

static double union_len(const double (*seg)[2], int k) {
  int idx[2 * MAXN];
  int e = 0;
  for (int i = 0; i < k; i++) {
    double lo = seg[i][0] < seg[i][1] ? seg[i][0] : seg[i][1];
    double hi = seg[i][0] < seg[i][1] ? seg[i][1] : seg[i][0];
    ev_x[e] = lo;
    ev_d[e] = 1;
    idx[e] = e;
    e++;
    ev_x[e] = hi;
    ev_d[e] = -1;
    idx[e] = e;
    e++;
  }
  qsort(idx, (size_t)e, sizeof(int), cmp_ev);
  int cov = 0;
  double last = 0.0, total = 0.0;
  for (int i = 0; i < e; i++) {
    if (cov > 0)
      total += ev_x[idx[i]] - last;
    cov += ev_d[idx[i]];
    last = ev_x[idx[i]];
  }
  return total;
}
```

listing 1265: c, the composite qsort comparator and the coverage sweep (source: dsa/samples-c/src/Ch37/segunion.c, lines 25-57)

```
func SegUnion(segs [][2]float64) float64 {
  type event struct {
    x float64
    d int
  }
  events := make([]event, 0, 2*len(segs))
  for _, s := range segs {
    lo, hi := s[0], s[1]
    if lo > hi {
      lo, hi = hi, lo
    }
    events = append(events, event{lo, 1}, event{hi, -1})
  }
  sort.Slice(events, func(i, j int) bool {
    if events[i].x != events[j].x {
      return events[i].x < events[j].x
    }
    return events[i].d < events[j].d
  })
  total, cov, last := 0.0, 0, 0.0
  for _, e := range events {
    if cov > 0 {
      total += e.x - last
    }
    cov += e.d
    last = e.x
  }
  return total
}
```

listing 1266: go, the event struct and one sort call (source: dsa/samples-go/ch37/segunion.go, lines 9-37)

```
  static double unionLen(double[][] seg) {
    int e = seg.length * 2;
    double[] ex = new double[e];
    int[] ed = new int[e];
    int k = 0;
    for (double[] s : seg) {
      double lo = Math.min(s[0], s[1]);
      double hi = Math.max(s[0], s[1]);
      ex[k] = lo;
      ed[k] = 1;
      k++;
      ex[k] = hi;
      ed[k] = -1;
      k++;
    }
    Integer[] idx = new Integer[e];
    for (int i = 0; i < e; i++)
      idx[i] = i;
    Arrays.sort(idx, (a, b) -> {
      if (ex[a] != ex[b])
        return ex[a] < ex[b] ? -1 : 1;
      return Integer.compare(ed[a], ed[b]);
    });
    int cov = 0;
    double last = 0.0, total = 0.0;
    for (int i = 0; i < e; i++) {
      if (cov > 0)
        total += ex[idx[i]] - last;
      cov += ed[idx[i]];
      last = ex[idx[i]];
    }
    return total;
  }
```

listing 1267: java, the endpoint sweep, Arrays.sort over boxed indices the one stdlib sort in the chapter (source: dsa/samples-java/src/Ch37/Segunion.java, lines 19-52)

```
    public static double SegUnion((double Lo, double Hi)[] segments)
    {
        var events = new List<(double X, int Delta)>();
        foreach ((double lo, double hi) in segments)
        {
            events.Add((Math.Min(lo, hi), 1));
            events.Add((Math.Max(lo, hi), -1));
        }
        events.Sort((e1, e2) => e1.X != e2.X ? e1.X.CompareTo(e2.X) : e2.Delta.CompareTo(e1.Delta));
        double total = 0;
        double last = 0;
        int cover = 0;
        foreach ((double x, int delta) in events)
        {
            if (cover > 0)
                total += x - last;
            cover += delta;
            last = x;
        }
        return total;
    }
```

listing 1268: c#, tuple events, the tie rule inside the sort lambda (source: dsa/samples/src/Ch37/Geometry2.cs, lines 131-151)

```
export function unionLength(segs) {
  const ev = [];
  for (const [a, b] of segs) {
    const lo = a <= b ? a : b;
    const hi = a <= b ? b : a;
    ev.push([lo, 1]);
    ev.push([hi, -1]);
  }
  ev.sort((u, v) => u[0] - v[0] || u[1] - v[1]);
  let cov = 0;
  let last = 0;
  let total = 0;
  for (const [x, d] of ev) {
    if (cov > 0) total += x - last;
    cov += d;
    last = x;
  }
  return total;
}
```

listing 1269: javascript, the comparator chain by subtraction (source: dsa/samples-js/src/ch37-segunion.mjs, lines 7-25)

```python
def union_length(segments):
    events = []
    for a, b in segments:  # each segment, oriented or not
        lo, hi = (a, b) if a ≤ b else (b, a)
        events.append((lo, 1))
        events.append((hi, -1))
    events.sort()
    coverage = 0
    last = 0.0
    total = 0.0
    for x, delta in events:
        if coverage > 0:
            total += x - last
        coverage += delta
        last = x
    return total
```

listing 1270: python, tuple sort gives the tie rule for free (source: dsa/samples-py/src/Ch37/segunion.py, lines 14-29)

```lua
local function union_len(segs)
  local ev = {}
  for _, s in ipairs(segs) do
    local lo, hi = s[1], s[2]
    if lo > hi then lo, hi = hi, lo end
    ev[#ev + 1] = {lo, 1}
    ev[#ev + 1] = {hi, -1}
  end
  table.sort(ev, function(a, b)
    if a[1] ~= b[1] then return a[1] < b[1] end
    return a[2] < b[2] -- opens before closes at the same point
  end)
  local cov, last, total = 0, 0.0, 0.0
  for _, e in ipairs(ev) do
    if cov > 0 then total = total + e[1] - last end
    cov = cov + e[2]
    last = e[1]
  end
  return total
end
```

listing 1271: lua, table.sort with the two-key closure (source: dsa/samples-lua/ch37_segunion.lua, lines 8-27)

The fixtures run integers, nesting, floats, and edges. Segments (1,4), (2,6), (8,10) cover 7, two spans and one gap. The nested family (0,3), (1,2), (3,5) totals 5, the touching ends merge because the open event at 3 sorts before the close. Floats sweep the same: (0.5,1.5) and (1.0,2.5) give 2.0. A single point covers nothing, empty input covers nothing, and unsorted or reversed input normalizes to the same totals, the suites pin that invariance. Any comparable coordinate type works, which is why c runs the sweep over doubles with a composite comparator while the python version sorts raw tuples.

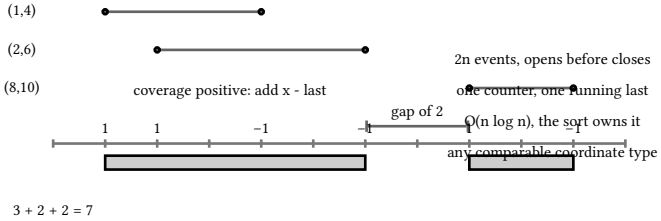


figure 317: three segments on one axis, the covered spans shaded on the ruler below, the gap marked

The application row reads general technique: painted-length questions are common but none of the six mined finals reduces to this exact sweep.

37.3. point in a convex polygon

The fan test. Triangulate the strictly convex ccw polygon from vertex 0: every interior point lands in exactly one wedge p0, pi, pi+1, so binary search the wedge by the cross of p0 to pi against q, then a single triangle orientation decides inside or outside. The boundary is the honest part, and the exact branch earns its lines: collinear cross within eps plus the edge's bounding box, applied to the two fan edges up front, since the wedge search itself cannot see them, and to the found wedge's closing edge. Degenerate polygons of one or two vertices answer outside unless q equals the point or an endpoint. The strict convexity precondition is stated and asserted by the tests, every triple turning left.

The dry run: the fixture is the pentagon (0,0), (4,0), (5,2), (2,5), (-1,2) with the square (0,0), (4,0), (4,4), (0,4) beside, asserted by the C# suite and by the five sibling suites.

1. The two fan edges answer before any search: q = (-1,2) is vertex 4 itself, collinear with edge (0,0)-(-1,2) and inside its bounding box, boundary on the spot.
2. For q = (2,1) the wedge search opens lo = 1, hi = 4 and probes the middle ray: cross((5,2), (2,1)) = 5 - 4 = 1 nonnegative, so lo moves to 2.
3. The next probe reads cross((2,5), (2,1)) = 2 - 10 = −8, hi settles at 3, and the wedge is (0,2,3).
4. The closing edge decides: cross((-3,3), (2,1) - (5,2)) = 3 + 9 = 12 positive, inside.
5. q = (3.5,3.5) lands the same wedge with closing cross exactly 0, on the edge midpoint, boundary. q = (6,0) probes −12 at the middle ray, falls to wedge (0,1,2), and its closing cross of −4 rules it outside.
6. On the square, (2,2) answers inside, (4,2) lands boundary on the closing edge, and (-0.001, 2) trips the fan door: cross((0,4), q) = 0.004, just past eps, outside.

q	cross(p0,p2,q)	cross(p0,p3,q)	wedge	verdict
(2,1)	1	−8	(0,2,3)	inside
(3.5,3.5)	10.5	−10.5	(0,2,3)	boundary
(2,5)	21	0	(0,3,4)	boundary
(6,0)	−12		(0,1,2)	outside
(-1,2)			fan edge	boundary

The inside, boundary, outside split over both polygons is the pinned landing, and the listings below run the fan in seven languages.

```c
static int pinconvex(double qx, double qy) {
  if (pn ≤ 2) { // degenerate: boundary only at the vertices
    for (int i = 0; i < pn; i++)
      if (on_segment(i, i, qx, qy))
        return 2;
    return 1;
  }
  // the closing wedge edge first: boundary handles vertex 0 too
  if (on_segment(pn - 1, 0, qx, qy))
    return 2;
  if (on_segment(0, 1, qx, qy))
    return 2;
  // outside the fan cone between rays 0→1 and 0→n-1
  if (crossq(0, 1, qx, qy) < -EPS || crossq(0, pn - 1, qx, qy) > EPS)
    return 1;
  // binary search the wedge i: cross(0, i, q) ≥ 0, cross(0, i+1, q) < 0
  int lo = 1, hi = pn - 1;
  while (hi - lo > 1) {
    int mid = (lo + hi) / 2;
    if (crossq(0, mid, qx, qy) ≥ 0)
      lo = mid;
    else
      hi = mid;
  }
  if (on_segment(lo, hi, qx, qy))
    return 2;
  // one triangle orientation decides
  double c =
      (px[hi] - px[lo]) * (qy - py[lo]) - (py[hi] - py[lo]) * (qx - px[lo]);
  return c > 0 ? 0 : 1;
}
```

listing 1272: c, the fan edges checked first, then the wedge search and the one-triangle verdict (source: dsa/samples-c/src/Ch37/pinconvex.c, lines 46-76)

```
  // the two fan edges carry the exact boundary check: cross == 0
  // plus the bounding box
  onSeg := func(a, b, q Point) bool {
    return math.Abs(cross(a, b, q)) <= geoEps &&
      q.X ≥ math2min(a.X, b.X)-geoEps &&
      q.X <= math2max(a.X, b.X)+geoEps &&
      q.Y ≥ math2min(a.Y, b.Y)-geoEps &&
      q.Y <= math2max(a.Y, b.Y)+geoEps
  }
  if onSeg(poly[0], poly[1], q) || onSeg(poly[0], poly[n-1], q) {
    return Boundary
  }
  // outside the closing wedge of the fan
  if cross(poly[0], poly[1], q) < -geoEps ||
    cross(poly[0], poly[n-1], q) > geoEps {
    return Outside
  }
  lo, hi := 1, n-1
  for hi-lo > 1 {
    mid := (lo + hi) / 2
    if cross(poly[0], poly[mid], q) ≥ 0 {
      lo = mid
    } else {
      hi = mid
    }
  }
  c := cross(poly[lo], poly[hi], q)
  if math.Abs(c) < geoEps {
    return Boundary
  }
  if c > 0 {
    return Inside
  }
  return Outside
}
```

listing 1273: go, the on-seg closure and the binary search (source: dsa/samples-go/ch37/pinconvex.go, lines 33-67)

```
  static int pinconvex(double qx, double qy) {
    if (vertN ≤ 2) { // degenerate: boundary only at the vertices
      for (int i = 0; i < vertN; i++)
        if (onSegment(i, i, qx, qy))
          return 2;
      return 1;
    }
    // the closing wedge edge first: boundary handles vertex 0 too
    if (onSegment(vertN - 1, 0, qx, qy))
      return 2;
    if (onSegment(0, 1, qx, qy))
      return 2;
    // outside the fan cone between rays 0→1 and 0→n-1
    if (crossq(0, 1, qx, qy) < -EPS || crossq(0, vertN - 1, qx, qy) > EPS)
      return 1;
    // binary search the wedge i: cross(0, i, q) ≥ 0, cross(0, i+1, q) < 0
    int lo = 1, hi = vertN - 1;
    while (hi - lo > 1) {
      int mid = (lo + hi) / 2;
      if (crossq(0, mid, qx, qy) ≥ 0)
        lo = mid;
      else
        hi = mid;
    }
    if (onSegment(lo, hi, qx, qy))
      return 2;
    // one triangle orientation decides
    double c = (px[hi] - px[lo]) * (qy - py[lo]) - (py[hi] - py[lo]) * (qx - px[lo]);
    return c > 0 ? 0 : 1;
  }
```

listing 1274: java, the fan test, the wedge binary search, one triangle orientation deciding (source: dsa/samples-java/src/Ch37/Pinconvex.java, lines 39-69)

```
    public static PolygonPlace Locate(Pt[] poly, Pt q)
    {
        int n = poly.Length;
        if (n == 0)
            return PolygonPlace.Outside;
        if (n ≤ 2)
        {
            for (int i = 0; i < n; i++)
                if (OnSegment(poly[i], poly[Math.Min(i + 1, n - 1)], q))
                    return PolygonPlace.Boundary;
            return PolygonPlace.Outside;
        }
        // boundary check then fan wedge: point must sit in the wedge (p0, p1) .. (p0, p_{n-1})
        if (OnSegment(poly[0], poly[1], q) || OnSegment(poly[0], poly[^1], q))
            return PolygonPlace.Boundary;
        if (Cross(poly[0], poly[1], q) < -Eps || Cross(poly[0], poly[^1], q) > Eps)
            return PolygonPlace.Outside;
        int lo = 1;
        int hi = n - 1;
        while (hi - lo > 1)
        {
            int mid = (lo + hi) / 2;
            if (Cross(poly[0], poly[mid], q) ≥ 0)
                lo = mid;
            else
                hi = mid;
        }
        if (OnSegment(poly[lo], poly[hi], q))
            return PolygonPlace.Boundary;
        return Cross(poly[lo], poly[hi], q) > Eps
            ? PolygonPlace.Inside
            : PolygonPlace.Outside;
    }
```

listing 1275: c#, locate as the enum answer, on-segment a boolean expression (source: dsa/samples/src/Ch37/Geometry2.cs, lines 154-186)

```
export function pointInConvex(poly, q) {
  const n = poly.length;
  if (n === 1) return near(q[0], poly[0][0]) && near(q[1], poly[0][1])
    ? "boundary"
    : "outside";
  if (n === 2) return onSeg(poly[0], poly[1], q) ? "boundary" : "outside";
  const p0 = poly[0];
  if (cross(p0, poly[1], q) < -EPS || cross(p0, poly[n - 1], q) > EPS) {
    return "outside"; // q misses the fan wedge entirely
  }
  if (onSeg(p0, poly[1], q) || onSeg(p0, poly[n - 1], q)) return "boundary";
  let lo = 1;
  let hi = n - 1;
  while (hi - lo > 1) {
    const mid = (lo + hi) >> 1;
    if (cross(p0, poly[mid], q) > 0) lo = mid;
    else hi = mid;
  }
  const c = cross(poly[lo], poly[hi], q);
  if (Math.abs(c) ≤ EPS) {
    return onSeg(poly[lo], poly[hi], q) ? "boundary" : "outside";
  }
  return c > 0 ? "inside" : "outside";
}
```

listing 1276: javascript, string answers, the shift-divide midpoint (source: dsa/samples-js/src/ch37-pinconvex.mjs, lines 22-45)

```
def locate(poly, q):
    # strictly convex ccw polygon, answers inside/outside/boundary
    m = len(poly)
    if m < 3:  # degenerate: boundary only at the points themselves
        for p in poly:
            if abs(p[0] - q[0]) < EPS and abs(p[1] - q[1]) < EPS:
                return "boundary"
        if m == 2 and on_segment(poly[0], poly[1], q):
            return "boundary"
        return "outside"
    p0 = poly[0]
    if on_segment(p0, poly[1], q) or on_segment(p0, poly[m - 1], q):
        return "boundary"  # the closing wedge edges own their spans
    lo, hi = 1, m - 1  # find the wedge i with q inside (p0, pi, pi+1)
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if cross(p0, poly[mid], q) ≥ 0:
            lo = mid
        else:
            hi = mid
    if cross(p0, poly[lo], q) < -EPS or cross(p0, poly[hi], q) > EPS:
        return "outside"  # outside the fan's angular span
    c = cross(poly[lo], poly[hi], q)  # one triangle decides the rest
    if abs(c) < EPS and on_segment(poly[lo], poly[hi], q):
        return "boundary"
    return "inside" if c > 0 else "outside"
```

listing 1277: python, the degenerate family up front, the wedge loop after (source: dsa/samples-py/src/Ch37/pinconvex.py, lines 29-54)

```
  -- the two fan edges get an explicit boundary check: the wedge search
  -- itself cannot see them
  for _, b in ipairs({poly[2], poly[n]}) do
    if math.abs(cross(poly[1], b, q)) < EPS then
      if math.min(poly[1][1], b[1]) - EPS ≤ q[1]
        and q[1] ≤ math.max(poly[1][1], b[1]) + EPS
        and math.min(poly[1][2], b[2]) - EPS ≤ q[2]
        and q[2] ≤ math.max(poly[1][2], b[2]) + EPS then
        return "boundary"
      end
    end
  end
  -- outside the fan's opening wedge
  if cross(poly[1], poly[2], q) < -EPS
     or cross(poly[1], poly[n], q) > EPS then
    return "outside"
  end
  local lo, hi = 2, n -- binary search the wedge [i, i+1]
  while hi - lo > 1 do
    local mid = (lo + hi) // 2
    if cross(poly[1], poly[mid], q) ≥ 0 then
      lo = mid
    else
      hi = mid
    end
  end
  local c = cross(poly[lo], poly[lo + 1], q)
  if math.abs(c) < EPS then
    -- on the line of edge (lo, lo+1): bounding box decides
    local a, b = poly[lo], poly[lo + 1]
    if math.min(a[1], b[1]) - EPS ≤ q[1]
      and q[1] ≤ math.max(a[1], b[1]) + EPS
      and math.min(a[2], b[2]) - EPS ≤ q[2]
      and q[2] ≤ math.max(a[2], b[2]) + EPS then
      return "boundary"
    end
    return "outside"
  end
  if c > 0 then return "inside" end
  return "outside"
end
```

listing 1278: lua, the two fan edges explicit, the bounding box spelled out (source: dsa/samples-lua/ch37_pinconvex.lua, lines 34-74)

The square (0,0), (4,0), (4,4), (0,4) answers inside at (2,2), outside at (5,5) and at (-0.001, 2), boundary at (4,2), at the vertex (0,0), and at the corner (4,4). The pentagon (0,0), (4,0), (5,2), (2,5), (-1,2) answers inside at (2,1), boundary at the vertices (2,5) and (-1,2), at the edge midpoint (3.5,3.5), outside at (6,0). Every suite asserts strict convexity of its fixtures before querying, and the lua suite keeps an all-edges oracle that agrees with the fan on every query, the in-suite cross-check the others get from their own fixture tables. O(log n) per query, no preprocessing beyond having the polygon.

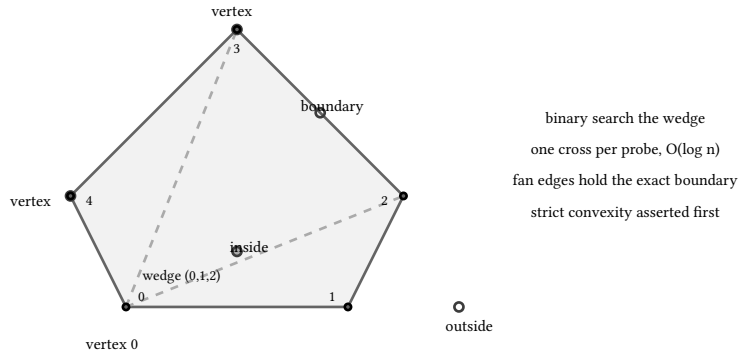


figure 318: the pentagon split into its fan from vertex 0, one query point per wedge shaded by answer

General technique in the topics row: point-in-polygon queries show up everywhere, but the logged sweep and the fan search together cover this book's needs and no mined finals problem pins the fan specifically.

37.4. minkowski sums

The sum of two convex polygons is the polygon swept by adding every point of one to every point of the other, and it is convex with a corner for every pair of corners that touch. The construction rides the edge cycles. Normalize both operands ccw starting at their lexicographic minima, read off both edge lists, and merge them by polar angle exactly like a merge sort, because a convex polygon's edges already wind by angle once around. Parallel edges of the two operands advance together and fold into one step, so collinear-carried vertices drop out and only true corners survive, the same strict-hull convention as book 9, ch 23 (practical data structures and algorithms, computational geometry). The sum starts at the sum of the two minima and closes when the merged edges return there. O(n + m), integer inputs exact.

The dry run: the fixture is the unit square plus the triangle (0,0), (2,0), (0,2), asserted by the C# suite and by the five sibling suites.

1. Both operands normalize ccw from their lexicographic minima, so the square's edges read (1,0), (0,1), (-1,0), (0,-1) and the triangle's (2,0), (-2,2), (0,-2).
2. The merge starts on the shared east direction: (1,0) and (2,0) are parallel, the walk advances both and folds them into the single step (3,0).
3. It then emits the square's (0,1), the triangle's (-2,2), the square's (-1,0), and folds the shared south direction (0,-1) + (0,-2) into (0,-3).
4. Accumulated from the start (0,0) + (0,0): (3,0), (3,1), (1,3), (0,3), and (0,-3) closes the cycle at (0,0), the pinned pentagon.
5. The edge lengths add 3 + 1 + 2 × sqrt(2) + 1 + 3 = 10.828427, the perimeter identity, 4 + 4 + 2 × sqrt(2) on the operand side.
6. The square plus the diamond (0,0), (1,1), (2,0), (1,-1) shares no direction with it, nothing folds, and 8 edges trace the pinned octagon, while a point operand is pure translation.

angle	square edge	triangle edge	merged step	vertex now
0	(1,0)	(2,0)	(3,0), folded	(3,0)
90	(0,1)		(0,1)	(3,1)
135		(-2,2)	(-2,2)	(1,3)
180	(-1,0)		(-1,0)	(0,3)
270	(0,-1)	(0,-2)	(0,-3), folded	(0,0)

The pentagon with its folded pair and the 10.828427 perimeter are the pinned landing, and the listings below merge the cycles in seven languages.

```
static void minkowski(const long long *ax, const long long *ay, int ak,
                      const long long *bx, const long long *by, int bk) {
  long long nx1[MAXP], ny1[MAXP], nx2[MAXP], ny2[MAXP];
  normalize(ax, ay, ak, nx1, ny1);
  normalize(bx, by, bk, nx2, ny2);
  long long ex1[MAXP], ey1[MAXP], ex2[MAXP], ey2[MAXP];
  for (int i = 0; i < ak; i++) {
    ex1[i] = nx1[(i + 1) % ak] - nx1[i];
    ey1[i] = ny1[(i + 1) % ak] - ny1[i];
  }
  for (int i = 0; i < bk; i++) {
    ex2[i] = nx2[(i + 1) % bk] - nx2[i];
    ey2[i] = ny2[(i + 1) % bk] - ny2[i];
  }
  int i1 = 0, i2 = 0;
  mink_n = 0;
  mkx[0] = nx1[0] + nx2[0]; // both walks start at their smallest edge
  mky[0] = ny1[0] + ny2[0];
  mink_n = 1;
  while (i1 < ak || i2 < bk) {
    long long dx = 0, dy = 0;
    if (i1 < ak && i2 < bk && ex1[i1] * ey2[i2] - ey1[i1] * ex2[i2] == 0 &&
        ((ey1[i1] > 0 || (ey1[i1] == 0 && ex1[i1] > 0)) ==
         (ey2[i2] > 0 || (ey2[i2] == 0 && ex2[i2] > 0)))) {
      dx = ex1[i1] + ex2[i2]; // equal directions fold
      dy = ey1[i1] + ey2[i2];
      i1++;
      i2++;
    } else if (i2 >= bk ||
               (i1 < ak && angle_less(ex1[i1], ey1[i1], ex2[i2], ey2[i2]))) {
      dx = ex1[i1];
      dy = ey1[i1];
      i1++;
    } else {
      dx = ex2[i2];
      dy = ey2[i2];
      i2++;
    }
    long long nx = mkx[mink_n - 1] + dx;
    long long ny = mky[mink_n - 1] + dy;
    if (nx == mkx[0] && ny == mky[0])
      break; // the cycle closes
    mkx[mink_n] = nx;
    mky[mink_n] = ny;
    mink_n++;
  }
```

listing 1279: c, normalize, the edge lists, the two-pointer merge with the fold (source: dsa/samples-c/src/Ch37/minkowski.c, lines 87-132)

```
func Minkowski(p, q []Point) []Point {
  p, q = ccwFromMin(p), ccwFromMin(q)
  if len(p) == 0 {
    return q
  }
  if len(q) == 0 {
    return p
  }
  ep, ap := unwrapAngles(p)
  eq, aq := unwrapAngles(q)
  merged := make([]Point, 0, len(ep)+len(eq))
  i, j := 0, 0
  for i < len(ep) || j < len(eq) {
    switch {
    case i ≥ len(ep):
      merged = append(merged, eq[j])
      j++
    case j ≥ len(eq):
      merged = append(merged, ep[i])
      i++
    case aq[j] < ap[i]:
      merged = append(merged, eq[j])
      j++
    default:
      merged = append(merged, ep[i])
      i++
    }
  }
  cur := Point{p[0].X + q[0].X, p[0].Y + q[0].Y}
  var poly []Point
  for _, e := range merged {
    poly = append(poly, cur)
    cur = Point{cur.X + e.X, cur.Y + e.Y}
  }
  var out []Point
  for k := range poly {
    prev := poly[(k+len(poly)-1)%len(poly)]
    next := poly[(k+1)%len(poly)]
    if math.Abs(cross(prev, poly[k], next)) > 1e-9 {
      out = append(out, poly[k])
    }
  }
  return ccwFromMin(out)
}
```

listing 1280: go, unwrapped edge angles for the merge, a strict hull to finish (source: dsa/samples-go/ch37/minkowski.go, lines 64-107)

```
    }
    for (int i = 0; i < bk; i++) {
      ex2[i] = nx2[(i + 1) % bk] - nx2[i];
      ey2[i] = ny2[(i + 1) % bk] - ny2[i];
    }
    int i1 = 0, i2 = 0;
    mkx[0] = nx1[0] + nx2[0]; // both walks start at their smallest edge
    mky[0] = ny1[0] + ny2[0];
    minkN = 1;
    while (i1 < ak || i2 < bk) {
      long dx = 0, dy = 0;
      boolean upper1 = ey1[i1] > 0 || (ey1[i1] == 0 && ex1[i1] > 0);
      boolean upper2 = ey2[i2] > 0 || (ey2[i2] == 0 && ex2[i2] > 0);
      if (i1 < ak && i2 < bk && ex1[i1] * ey2[i2] - ey1[i1] * ex2[i2] == 0
          && upper1 == upper2) {
        dx = ex1[i1] + ex2[i2]; // equal directions fold
        dy = ey1[i1] + ey2[i2];
        i1++;
        i2++;
      } else if (i2 ≥ bk || (i1 < ak && angleLess(ex1[i1], ey1[i1], ex2[i2], ey2[i2]))) {
        dx = ex1[i1];
        dy = ey1[i1];
        i1++;
      } else {
        dx = ex2[i2];
        dy = ey2[i2];
        i2++;
      }
      long nx = mkx[minkN - 1] + dx;
      long ny = mky[minkN - 1] + dy;
      if (nx == mkx[0] && ny == mky[0])
        break; // the cycle closes
      mkx[minkN] = nx;
      mky[minkN] = ny;
      minkN++;
    }
    // canonical cycle: rotate to start at the lexicographic minimum
    int best = 0;
    for (int i = 1; i < minkN; i++)
      if (mkx[i] < mkx[best] || (mkx[i] == mkx[best] && mky[i] < mky[best]))
        best = i;
    long[] tmpx = new long[2 * MAXP], tmpy = new long[2 * MAXP];
    for (int i = 0; i < minkN; i++) {
      tmpx[i] = mkx[(best + i) % minkN];
      tmpy[i] = mky[(best + i) % minkN];
    }
    for (int i = 0; i < minkN; i++) {
      mkx[i] = tmpx[i];
```

listing 1281: java, the edge lists, the two-pointer merge with the fold, exact longs throughout (source: dsa/samples-java/src/Ch37/Minkowski.java, lines 86-133)

```
    public static List<Pt> MinkowskiSum(Pt[] p, Pt[] q)
    {
        if (p.Length == 1)
            return q.Select(v => new Pt(v.X + p[0].X, v.Y + p[0].Y)).ToList();
        if (q.Length == 1)
            return p.Select(v => new Pt(v.X + q[0].X, v.Y + q[0].Y)).ToList();
        List<Pt> pp = RotateToMin(Ccw(p));
        List<Pt> qq = RotateToMin(Ccw(q));
        // two-pointer walk over both edge cycles: both polygons start at their lex
        // min, so their outgoing edges are angle-aligned and the walk emits the
        // sum's corners directly, folding collinear steps by advancing both
        var result = new List<Pt>();
        int i = 0;
        int j = 0;
        while (i < pp.Count || j < qq.Count)
        {
            result.Add(new Pt(pp[i % pp.Count].X + qq[j % qq.Count].X, pp[i % pp.Count].Y + qq[j %
qq.Count].Y));
            Pt peNext = Sub(pp[(i + 1) % pp.Count], pp[i % pp.Count]);
            Pt qeNext = Sub(qq[(j + 1) % qq.Count], qq[j % qq.Count]);
            double turn = Cross(new Pt(0, 0), peNext, qeNext);
            if (i < pp.Count && (j ≥ qq.Count || turn > 0))
                i++;
            else if (j < qq.Count && (i ≥ pp.Count || turn < 0))
                j++;
            else
            {
                i++; // collinear or exhausted partner: fold by advancing both
                j++;
            }
        }
        if (result.Count > 1 && result[^1] == result[0])
            result.RemoveAt(result.Count - 1); // drop the closing repeat of the start
        return result;
    }
```

listing 1282: c#, the turn cross steering i and j, collinear advancing both (source: dsa/samples/src/Ch37/Geometry2.cs, lines 196-229)

```
export function minkowski(P, Q) {
  const pp = reorder(P);
  const qq = reorder(Q);
  const n = pp.length;
  const m = qq.length;
  const edgeP = [];
  for (let i = 0; i ≤ n; i++) {
    const a = pp[i % n];
    const b = pp[(i + 1) % n];
    edgeP.push([b[0] - a[0], b[1] - a[1]]);
  }
  const edgeQ = [];
  for (let j = 0; j ≤ m; j++) {
    const a = qq[j % m];
    const b = qq[(j + 1) % m];
    edgeQ.push([b[0] - a[0], b[1] - a[1]]);
  }
  const out = [];
  let i = 0;
  let j = 0;
  while (i < n || j < m) {
    out.push([pp[i % n][0] + qq[j % m][0], pp[i % n][1] + qq[j % m][1]]);
    const c = edgeP[i][0] * edgeQ[j][1] - edgeP[i][1] * edgeQ[j][0];
    if (j === m || (i < n && c > 0)) {
      i += 1;
    } else if (i === n || c < 0) {
      j += 1;
    } else {
      // collinear edges carried together: the intermediate vertex of
      // the sum drops out, only the corner survives
      i += 1;
      j += 1;
    }
  }
  return out;
}
```

listing 1283: javascript, the reorder helper then the cross-steered walk (source: dsa/samples-js/src/ch37-minkowski.mjs, lines 31-66)

```python
def minkowski(p, q):
    p = ccw_from_min(list(p))
    q = ccw_from_min(list(q))
    ep = [
        (p[(i + 1) % len(p)][0] - p[i][0], p[(i + 1) % len(p)][1] - p[i][1])
        for i in range(len(p))
    ]
    eq = [
        (q[(i + 1) % len(q)][0] - q[i][0], q[(i + 1) % len(q)][1] - q[i][1])
        for i in range(len(q))
    ]

    # two-pointer merge of the cyclically sorted edge lists: take the
    # edge with the smaller polar angle by cross product; parallel
    # edges of both polygons step together (collinear-carried, the
    # final hull drops them)
    i = j = 0
    merged = []
    while i < len(ep) or j < len(eq):
        if j ≥ len(eq) or (i < len(ep) and cross((0, 0), ep[i], eq[j]) > 0):
            merged.append(ep[i])
            i += 1
        elif i ≥ len(ep) or cross((0, 0), eq[j], ep[i]) > 0:
            merged.append(eq[j])
            j += 1
        else:
            merged.append((ep[i][0] + eq[j][0], ep[i][1] + eq[j][1]))
            i += 1
            j += 1
    start = (p[0][0] + q[0][0], p[0][1] + q[0][1])  # min + min
    out = [start]
    for dx, dy in merged:
        out.append((out[-1][0] + dx, out[-1][1] + dy))
    return hull_strict(out)  # drops collinear-carried and zero edges
```

listing 1284: python, the merge with the fold branch, a strict hull to drop carriers (source: dsa/samples-py/src/Ch37/minkowski.py, lines 54-87)

```
  -- two-pointer merge: both edge lists are already sorted by polar
  -- angle starting at the lex minima; cross decides whose edge winds
  -- first, parallel edges merge into one step
  local pts = {{p[1][1] + q[1][1], p[1][2] + q[1][2]}}
  local i, j = 1, 1
  while i ≤ np or j ≤ nq do
    local e
    if i > np then
      e = qe[j]; j = j + 1
    elseif j > nq then
      e = pe[i]; i = i + 1
    else
      local c = pe[i][1] * qe[j][2] - pe[i][2] * qe[j][1]
      if c > 0 then
        e = pe[i]; i = i + 1
      elseif c < 0 then
        e = qe[j]; j = j + 1
      else
        e = {pe[i][1] + qe[j][1], pe[i][2] + qe[j][2]} -- parallel
        i = i + 1
        j = j + 1
      end
    end
    local top = pts[#pts]
    local nx, ny = top[1] + e[1], top[2] + e[2]
    -- drop the collinear carrier as it forms
    if #pts ≥ 2 then
      local prev = pts[#pts - 1]
      if cross(prev, top, {nx, ny}) == 0 then
        pts[#pts] = {nx, ny}
      else
        pts[#pts + 1] = {nx, ny}
      end
    else
      pts[#pts + 1] = {nx, ny}
    end
  end
  -- the walk's last vertex equals the first; close and dedupe collinear
  while #pts > 1 do
    local m = #pts
    if pts[m][1] == pts[1][1] and pts[m][2] == pts[1][2] then
      pts[m] = nil
    elseif #pts ≥ 3
      and cross(pts[m - 1], pts[m], pts[1]) == 0 then
      pts[m] = nil
    else
      break
    end
  end
  return pts
```

listing 1285: lua, the merge walk dropping collinear carriers as they form (source: dsa/samples-lua/ch37_minkowski.lua, lines 61-110)

The fixtures pin canonical cycles starting at the lexicographic minimum. The unit square plus the triangle (0,0), (2,0), (0,2) sums to the pentagon (0,0), (3,0), (3,1), (1,3), (0,3) with perimeter 10.828427, and the fold is visible: the square's east edge (1,0) and the triangle's east edge (2,0) merge into the single step (3,0). Two far squares sum to a square, 3 by 3. The square plus the diamond (0,0), (1,1), (2,0), (1,-1) sums to the octagon (0,0), (1,-1), (2,-1), (3,0), (3,1), (2,2), (1,2), (0,1), no parallel edges to fold. A point plus the square is the square translated. The property fixture pins

the perimeter identity for the first family: the perimeter of the sum equals the sum of the perimeters, 10.828427 exact against the pinned value, and every suite cross-checks the cycle against the strict hull of all pairwise sums, the brute the construction must match.

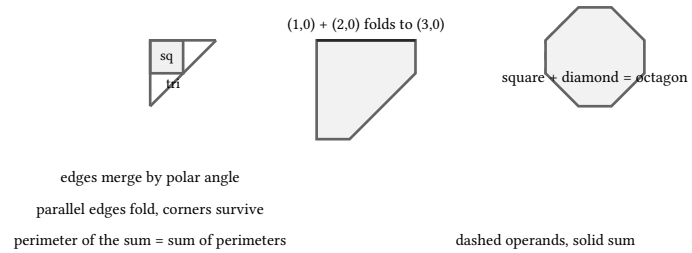


figure 319: left: square plus triangle with the folded edge pair highlighted, right: square plus diamond tracing the octagon

General technique again: minkowski sums answer collision and dilation questions directly, and no mined finals problem pins them.

37.5. rotating calipers

Diameter and width of a convex polygon in one linear walk after the hull. Keep an antipodal index j: for each hull edge i, advance j while the triangle area over that edge keeps growing, the cross of the edge vector with the next edge at j staying positive. Every (vertex, edge) antipodal event is visited exactly once, the diameter is the maximum vertex pair distance over those events, and the width is the minimum over edges of the maximum vertex distance to the edge's supporting line. The hull is the book 9, ch 23 (practical data structures and algorithms, computational geometry) monotone chain rebuilt in file so every run stays standalone.

The dry run: the fixtures are the pentagon (0,0), (4,0), (5,2), (2,5), (-1,2), its own hull, and the rectangle (0,0), (6,0), (6,4), (0,4), asserted by the C# suite and by the five sibling suites.

1. Over the rectangle the antipodal events visit both diagonals: the pair (0,0)-(6,4) reads sqrt(36 + 16) = sqrt(52) = 7.211103, the diameter, and the horizontal edges hold every vertex within 4, the width.
2. The pentagon's diameter peaks at (-1,2)-(5,2), distance 6.0, every other antipodal pair shorter.
3. The width scan measures each edge's supporting line: edge (0,0)-(4,0) holds (2,5) at 5, edge (4,0)-(5,2) holds (-1,2) at 12 / sqrt(5) = 5.366563, and edge (-1,2)-(0,0) holds (5,2) at the same 5.366563.
4. Edge (5,2)-(2,5) on the line x + y = 7 holds (0,0) at 7 / sqrt(2) = 4.949747, and edge (2,5)-(-1,2) holds (4,0) at the same value: the twin minima.
5. The minimum over edges, 4.949747, is the width, and the collinear family (0,0), (3,0), (1,0) degenerates to a 2-point hull: diameter 3.0, width 0.

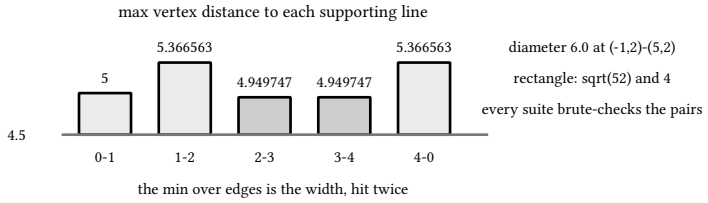


figure 320: the width scan as five bars, one per pentagon edge, the twin minima shaded, the axis truncated at 4.5

The 6.0 and the twice-hit 4.949747 are the pinned landing, and the listings below walk the antipodal loop in seven languages.

```
  double best = 0.0;
  *ia = *ib = 0;
  int j = 1;
  for (int i = 0; i < hn; i++) {
    int i2 = (i + 1) % hn;
    // advance j while the triangle area over edge (i, i2) keeps growing
    while (fabs(cross3(hx[i], hy[i], hx[i2], hy[i2], hx[(j + 1) % hn],
                       hy[(j + 1) % hn])) >
           fabs(cross3(hx[i], hy[i], hx[i2], hy[i2], hx[j], hy[j])))
      j = (j + 1) % hn;
    if (dist(i, j) > best) {
      best = dist(i, j);
      *ia = i;
      *ib = j;
    }
    if (dist(i2, j) > best) {
      best = dist(i2, j);
      *ia = i2;
      *ib = j;
    }
  }
  return best;
}

// width: min over edges of the max vertex distance to the edge line
static double width(void) {
  if (hn <= 2)
    return 0.0;
  double best = 1e18;
  for (int i = 0; i < hn; i++) {
    int i2 = (i + 1) % hn;
    double len = dist(i, i2);
    double md = 0.0;
    for (int v = 0; v < hn; v++) {
      double d = fabs(cross3(hx[i], hy[i], hx[i2], hy[i2], hx[v], hy[v])) / len;
      if (d > md)
        md = d;
    }
    if (md < best)
      best = md;
  }
  return best;
}
```

listing 1286: c, the antipodal walk, the candidate pairs, the width loop (source: dsa/samples-c/src/Ch37/calipers.c, lines 104-146)

```
func Calipers(pts []Point) (d float64, diaA, diaB Point, width float64) {
  hull := strictHull(pts)
  n := len(hull)
  if n == 0 {
    return 0, Point{}, Point{}, 0
  }
  if n == 1 {
    return 0, hull[0], hull[0], 0
  }
  if n == 2 {
    d = math.Hypot(hull[1].X-hull[0].X, hull[1].Y-hull[0].Y)
    return d, hull[0], hull[1], 0
  }
  triArea := func(a, b, c Point) float64 {
    return math.Abs(cross(a, b, c))
  }
  j := 1
  d, width = 0, math.Inf(1)
  for i := 0; i < n; i++ {
    a, b := hull[i], hull[(i+1)%n]
    for triArea(a, b, hull[(j+1)%n]) > triArea(a, b, hull[j]) {
      j = (j + 1) % n
    }
    for _, cand := range [2]Point{hull[j], hull[(j+1)%n]} {
      if dd := math.Hypot(cand.X-a.X, cand.Y-a.Y); dd > d {
        d, diaA, diaB = dd, a, cand
      }
      if dd := math.Hypot(cand.X-b.X, cand.Y-b.Y); dd > d {
        d, diaA, diaB = dd, b, cand
      }
    }
    // width: max perpendicular distance to this edge line
    length := math.Hypot(b.X-a.X, b.Y-a.Y)
    maxDist := 0.0
    for _, p := range hull {
      if dist := math.Abs(cross(a, b, p)) / length; dist > maxDist {
        maxDist = dist
      }
    }
    if maxDist < width {
      width = maxDist
    }
  }
  return d, diaA, diaB, width
}
```

listing 1287: go, the triangle-area advance and the two-candidate diameter update (source: dsa/samples-go/ch37/calipers.go, lines 52-96)

```
  static double diameter() {
    if (hn == 1) {
      iaOut = ibOut = 0;
      return 0.0;
    }
    if (hn == 2) {
      iaOut = 0;
      ibOut = 1;
      return dist(0, 1);
    }
    double best = 0.0;
    iaOut = ibOut = 0;
    int j = 1;
    for (int i = 0; i < hn; i++) {
      int i2 = (i + 1) % hn;
      // advance j while the triangle area over edge (i, i2) keeps growing
      while (Math.abs(cross3(hx[i], hy[i], hx[i2], hy[i2],
          hx[(j + 1) % hn], hy[(j + 1) % hn]))
          > Math.abs(cross3(hx[i], hy[i], hx[i2], hy[i2], hx[j], hy[j])))
        j = (j + 1) % hn;
      if (dist(i, j) > best) {
        best = dist(i, j);
        iaOut = i;
        ibOut = j;
      }
      if (dist(i2, j) > best) {
        best = dist(i2, j);
        iaOut = i2;
        ibOut = j;
      }
    }
    return best;
  }

  static double width() {
    if (hn ≤ 2)
      return 0.0;
    double best = 1e18;
    for (int i = 0; i < hn; i++) {
      int i2 = (i + 1) % hn;
      double len = dist(i, i2);
      double md = 0.0;
      for (int v = 0; v < hn; v++) {
        double d = Math.abs(cross3(hx[i], hy[i], hx[i2], hy[i2], hx[v], hy[v])) / len;
        if (d > md)
          md = d;
      }
      if (md < best)
        best = md;
    }
    return best;
  }
```

listing 1288: java, the antipodal walk, the candidate pairs, the width loop (source: dsa/samples-java/src/Ch37/Calipers.java, lines 93-144)

```
    public static (double Diameter, double Width) Measure(Pt[] points)
    {
        List<Pt> hull = Hull(points);
        int n = hull.Count;
        if (n == 1)
            return (0, 0);
        if (n == 2)
            return (Dist(hull[0], hull[1]), 0); // the collinear degenerate
        double diameter = 0;
        double width = double.MaxValue;
        int k = 1;
        for (int i = 0; i < n; i++)
        {
            Pt e = Sub(hull[(i + 1) % n], hull[i]);
            while (Geometry2.Cross(new Pt(0, 0), e, Sub(hull[(k + 1) % n], hull[k])) > 0)
                k = (k + 1) % n;
            diameter = Math.Max(diameter, Math.Max(Dist(hull[i], hull[k]), Dist(hull[(i + 1) % n],
hull[k])));
            double len = Dist(hull[i], hull[(i + 1) % n]);
            width = Math.Min(width, Math.Abs(Geometry2.Cross(hull[i], hull[(i + 1) % n], hull[k])) / len);
        }
        return (diameter, width);
    }
```

listing 1289: c#, the walk and both measurements in one loop (source: dsa/samples/src/Ch37/Calipers.cs, lines 14-35)

```
export function calipers(points) {
  const P = hullStrict(points);
  const n = P.length;
  if (n === 1) return { diameter: 0, pair: [P[0], P[0]], width: 0 };
  if (n === 2) {
    return {
      diameter: Math.hypot(P[1][0] - P[0][0], P[1][1] - P[0][1]),
      pair: [P[0], P[1]],
      width: 0,
    };
  }

  const dist = (a, b) => Math.hypot(a[0] - b[0], a[1] - b[1]);

  // the antipodal walk for the diameter
  let best = 0;
  let pair = [P[0], P[1]];
  let j = 1;
  for (let i = 0; i < n; i++) {
    const ni = (i + 1) % n;
    for (;;) {
      const nj = (j + 1) % n;
      const e = [P[ni][0] - P[i][0], P[ni][1] - P[i][1]];
      const w = [P[nj][0] - P[j][0], P[nj][1] - P[j][1]];
      if (e[0] * w[1] - e[1] * w[0] > 0) {
        j = nj;
      } else {
        break;
      }
    }
    for (const cand of [[P[i], P[j]], [P[ni], P[j]]]) {
      const d = dist(cand[0], cand[1]);
      if (d > best) {
        best = d;
        pair = cand;
      }
    }
  }

  // the width loop: per edge the farthest vertex, tracked forward
  let width = Infinity;
  let k = 1;
  for (let i = 0; i < n; i++) {
    const a = P[i];
    const b = P[(i + 1) % n];
    const len = Math.hypot(b[0] - a[0], b[1] - a[1]);
    const dOf = (p) => Math.abs(cross(a, b, p)) / len;
    while (dOf(P[(k + 1) % n]) > dOf(P[k])) k = (k + 1) % n;
    width = Math.min(width, dOf(P[k]));
  }
  return { diameter: best, pair, width };
}
```

listing 1290: javascript, the edge-cross advance, the width scan tracked forward (source: dsa/samples-js/src/ch37-calipers.mjs, lines 30-81)

```python
def antipodal_events(hull):
    # (vertex, edge) events: advance j while moving it turns positive
    m = len(hull)
    events = []
    j = 1
    for i in range(m):
        e1 = (hull[(i + 1) % m][0] - hull[i][0], hull[(i + 1) % m][1] - hull[i][1])
        while True:
            nxt_j = hull[(j + 1) % m]
            e2 = (nxt_j[0] - hull[j][0], nxt_j[1] - hull[j][1])
            # cross of consecutive edge vectors turning positive
            if e1[0] * e2[1] - e1[1] * e2[0] > 0:
                j = (j + 1) % m
            else:
                break
        events.append((i, j))
    return events


def calipers(points):
    hull = hull_strict(points)
    if len(hull) == 1:
        return 0.0, None, 0.0
    if len(hull) == 2:
        d = math.dist(hull[0], hull[1])
        return d, (hull[0], hull[1]), 0.0
    best = 0.0
    pair = None
    for i, j in antipodal_events(hull):
        for a in (i, (i + 1) % len(hull)):
            d = math.dist(hull[a], hull[j])
            if d > best:
                best = d
                pair = (hull[a], hull[j])
    width = min(edge_width(hull, i) for i in range(len(hull)))
    return best, pair, width


def edge_width(hull, i):
    # max vertex distance to the edge's supporting line
    a, b = hull[i], hull[(i + 1) % len(hull)]
    length = math.hypot(b[0] - a[0], b[1] - a[1])
    return max(abs(cross(a, b, p)) / length for p in hull)
```

listing 1291: python, antipodal-events as a list, diameter and width read off it (source: dsa/samples-py/src/Ch37/calipers.py, lines 39-81)

```lua
local function diameter(poly)
  local n = #poly
  if n == 1 then return 0.0, poly[1], poly[1] end
  local bestd, ba, bb = 0.0, poly[1], poly[1]
  local j = 2
  for i = 1, n do
    local ni = poly[i % n + 1]
    -- advance j while the triangle area keeps growing
    while true do
      local nj = poly[j % n + 1]
      local cur = math.abs(cross(poly[i], ni, poly[j]))
      local nxt = math.abs(cross(poly[i], ni, nj))
      if nxt > cur + EPS then
        j = j % n + 1
      else
        break
      end
    end
    if dist(poly[i], poly[j]) > bestd then
      bestd, ba, bb = dist(poly[i], poly[j]), poly[i], poly[j]
    end
    local nj = poly[j % n + 1]
    if dist(poly[i], nj) > bestd then
      bestd, ba, bb = dist(poly[i], nj), poly[i], nj
    end
  end
  return bestd, ba, bb
end

-- width: min over edges of the max vertex distance to the edge line
local function width(poly)
  local n = #poly
  if n < 2 then return 0.0 end
  if n == 2 then return 0.0 end
  local best = math.huge
  for i = 1, n do
    local a, b = poly[i], poly[i % n + 1]
    local ln = math.sqrt((b[1] - a[1]) ^ 2 + (b[2] - a[2]) ^ 2)
    local md = 0.0
    for _, p in ipairs(poly) do
      md = math.max(md, math.abs(cross(a, b, p)) / ln)
    end
    best = math.min(best, md)
  end
  return best
end
```

listing 1292: lua, the area-advance loop, then the width function (source: dsa/samples-lua/ch37_calipers.lua, lines 53-98)

The rectangle (0,0), (6,0), (6,4), (0,4) pins diameter 7.211103, exactly sqrt(52), at the pair (0,0)-(6,4), width 4. The pentagon hull pins diameter 6.0 at (-1,2)-(5,2) and width 4.949747, which is 7 over sqrt(2), the max distance from vertex (0,0) to the line through (5,2) and (2,5). The collinear edge case degenerates to a 2-point hull with diameter 3.0 and width 0, and every suite cross-checks the diameter against a brute vertex-pair scan. The honest contest note belongs here: icpc 2017 problem A (book 10, chapter 8) asks for an optimal strip, and its answer equals the point-set diameter only in special shapes. The finals solution enumerates the O(n^2) vertex-pair chords, so this section cross-names that problem as the kin without claiming its solution.

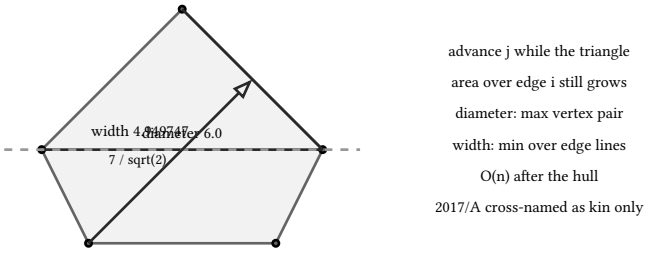


figure 321: the pentagon with both caliper support lines touching the diameter pair, one width measurement drawn

37.6. half-plane intersection

A half-plane is a triple (a, b, c) meaning a times x plus b times y at most c. The book algorithm bootstraps a world box of 1e6, four box half-planes prepended so unbounded regions have edges to cut, normalizes every constraint to a unit normal, sorts by normal angle folding same directions onto the tightest c, and rejects contradictory opposite pairs, n dot p at most c1 together with minus n dot p at most c2, which is empty exactly when c1 + c2 is negative, before any geometry runs. Then the sort-and-incremental deque: for each line in angle order pop the back while the back corner violates the new line, pop the front likewise, push, and after the sweep run both cleanups again because the last lines can invalidate the first corners. The output is the ccw vertex cycle rotated to the lexicographic minimum, plus a box-touch flag reporting true unboundedness, any corner out past the box rim.

The dry run: the fixture is the 2018/G cell, the park box plus three bisector half-planes, asserted by the C# suite and by the five sibling suites.

1. The park box normalizes to x at most 10, x at least 0, y at most 6, y at least 0, and the angle sort folds nothing: four distinct normal directions.
2. The bisector against (10,0) reads 20x at most 100, that is x at most 5, and the one against (0,10) reads 12y at most 36, that is y at most 3.
3. The diagonal bisector reads 20x + 12y at most 136, that is 5x + 3y at most 34, and it passes exactly through the corner (5,3): 25 + 9 = 34, an edge of zero length, dropped by the cleanup.
4. The deque sweep over the angle order leaves the 5 by 3 rectangle (0,0), (5,0), (5,3), (0,3), area 5 × 3 = 15, box flag false.
5. The triangle family clips y at least 1.5 under x + y at most 2 and -x + y at most 2: at height 1.5 the base runs 0.5 - (-0.5) = 1 and the apex sits at (0,2), area 0.25.
6. x at most 0 against x at least 1 is the contradictory pair, rejected before any geometry runs, and x at most 0 with y at most 0 alone reports unbounded, pinned at the world box corner.

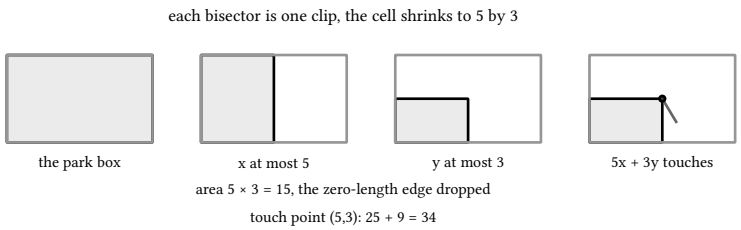


figure 322: the 2018/G cell as a sequence of clips, the 10 by 6 park box narrowing to the 5 by 3 rectangle, the diagonal touching at the corner

The area 15 cell is the pinned landing, and the listings below run the deque sweep in seven languages.

```
  // the sort-and-incremental deque
  HP dq[MAXH];
  int dc = 0;
  for (int i = 0; i < uc; i++) {
    while (dc ≥ 2) {
      double x, y;
      isect(&dq[dc - 2], &dq[dc - 1], &x, &y);
      if (bad(&uniq[i], x, y))
        dc--;
      else
        break;
    }
    while (dc ≥ 2) {
      double x, y;
      isect(&dq[0], &dq[1], &x, &y);
      if (bad(&uniq[i], x, y)) {
        memmove(dq, dq + 1, (size_t)(dc - 1) * sizeof(HP));
        dc--;
      } else
        break;
    }
    dq[dc++] = uniq[i];
  }
  while (dc ≥ 3) {
    double x, y;
    isect(&dq[dc - 2], &dq[dc - 1], &x, &y);
    if (bad(&dq[0], x, y))
      dc--;
    else
      break;
  }
  while (dc ≥ 3) {
    double x, y;
    isect(&dq[0], &dq[1], &x, &y);
    if (bad(&dq[dc - 1], x, y)) {
      memmove(dq, dq + 1, (size_t)(dc - 1) * sizeof(HP));
      dc--;
    } else
      break;
  }
  if (dc < 3) {
    pn = 0;
    box_flag = 0;
    return;
  }
```

listing 1293: c, the deque sweep, the two final cleanups, the corner sweep (source: dsa/samples-c/src/Ch37/halfplane.c, lines 109-153)

```
  var dq []hpLine
  for _, h := range uniq {
    for len(dq) ≥ 2 {
      if p, ok := hpIntersect(dq[len(dq)-2], dq[len(dq)-1]); ok &&
        hpBad(h, p) {
        dq = dq[:len(dq)-1]
        continue
      }
      break
    }
    for len(dq) ≥ 2 {
      if p, ok := hpIntersect(dq[0], dq[1]); ok && hpBad(h, p) {
        dq = dq[1:]
        continue
      }
      break
    }
    dq = append(dq, h)
  }
  for len(dq) ≥ 3 {
    if p, ok := hpIntersect(dq[len(dq)-2], dq[len(dq)-1]); ok &&
      hpBad(dq[0], p) {
      dq = dq[:len(dq)-1]
      continue
    }
    if p, ok := hpIntersect(dq[0], dq[1]); ok &&
      hpBad(dq[len(dq)-1], p) {
      dq = dq[1:]
      continue
    }
    break
  }
  if len(dq) < 3 {
    return nil, false, fmt.Errorf(
      "half-plane intersection is empty")
  }
  poly := make([]Point, 0, len(dq))
  for i := range dq {
    p, ok := hpIntersect(dq[i], dq[(i+1)%len(dq)])
    if !ok {
      return nil, false, fmt.Errorf("parallel lines in the deque")
    }
    poly = append(poly, p)
  }
  for i, h := range dq {
    if hpBad(h, poly[i]) {
      return nil, false, fmt.Errorf(
        "half-plane intersection is empty")
    }
  }
```

listing 1294: go, slice deque popping front and back, the violation sweep at the end (source: dsa/samples-go/ch37/halfplane.go, lines 75-124)

```
    // the sort-and-incremental deque
    double[][] dq = new double[uc][];
    int dc = 0;
    for (int i = 0; i < uc; i++) {
      while (dc ≥ 2) {
        isect(dq[dc - 2], dq[dc - 1], pt);
        if (bad(uniq[i], pt[0], pt[1]))
          dc--;
        else
          break;
      }
      while (dc ≥ 2) {
        isect(dq[0], dq[1], pt);
        if (bad(uniq[i], pt[0], pt[1])) {
          System.arraycopy(dq, 1, dq, 0, dc - 1);
          dc--;
        } else
          break;
      }
      dq[dc++] = uniq[i];
    }
    while (dc ≥ 3) {
      isect(dq[dc - 2], dq[dc - 1], pt);
      if (bad(dq[0], pt[0], pt[1]))
        dc--;
      else
        break;
    }
    while (dc ≥ 3) {
      isect(dq[0], dq[1], pt);
      if (bad(dq[dc - 1], pt[0], pt[1])) {
        System.arraycopy(dq, 1, dq, 0, dc - 1);
        dc--;
      } else
        break;
    }
    if (dc < 3) {
      vertN = 0;
      boxFlag = false;
      return;
    }
    // corners; final violation sweep
    int c2 = 0;
    for (int i = 0; i < dc; i++) {
      isect(dq[i], dq[(i + 1) % dc], pt);
      px[c2] = pt[0];
      py[c2] = pt[1];
      for (int j = 0; j < dc; j++)
        if (bad(dq[j], px[c2], py[c2])) {
          vertN = 0;
          boxFlag = false;
          return;
        }
      c2++;
    }
```

listing 1295: java, the sort-and-incremental deque, the corner sweep with its violation check (source: dsa/samples-java/src/Ch37/Halfplane.java, lines 103-157)

```
        var byDir = uniq.GroupBy(h => Math.Round(Math.Atan2(h.Item2, h.Item1) % Math.PI, 6)).ToList();
        foreach (var group in byDir)
        {
            var members = group.ToList();
            if (members.Count == 2 && members[0].Item3 + members[1].Item3 < -Geometry2.Eps)
                return (Array.Empty<Pt>(), false); // contradictory opposite pair: empty slab
        }

        var deque = new LinkedList<(double, double, double)>();
        foreach (var h in uniq)
        {
            while (deque.Count ≥ 2 && Violates(h, Intersect(deque.Last!.Previous!.Value,
deque.Last.Value)))
                deque.RemoveLast();
            while (deque.Count ≥ 2 && Violates(h, Intersect(deque.First!.Value, deque.First.Next!.Value)))
                deque.RemoveFirst();
            deque.AddLast(h);
        }
        while (deque.Count ≥ 3
               && Violates(deque.First!.Value, Intersect(deque.Last!.Previous!.Value, deque.Last.Value)))
            deque.RemoveLast();
        while (deque.Count ≥ 3 && Violates(deque.Last!.Value, Intersect(deque.First!.Value,
deque.First.Next!.Value)))
            deque.RemoveFirst();
        if (deque.Count < 3)
            return (Array.Empty<Pt>(), false);
        var poly = new List<Pt>();
        foreach (var node in Enumerate(deque))
        {
            var next = node.Next ?? deque.First!;
            Pt? p = Intersect(node.Value, next.Value);
            if (p is null || Violates(node.Value, p) || Violates(next.Value, p))
                return (Array.Empty<Pt>(), false);
            poly.Add(p!.Value);
        }
        bool unbounded = poly.Any(p => Math.Max(Math.Abs(p.X), Math.Abs(p.Y)) > Box - 1.0);
        return (Canonicalize(poly), unbounded);
    }
```

listing 1296: c#, opposite-pair rejection, the linked-list deque, the unbounded flag (source: dsa/samples/src/Ch37/Calipers.cs, lines 93-128)

```
  // opposite pairs: x.n ≤ c1 and -x.n ≤ c2 needs c1 + c2 ≥ 0
  const bydir = new Map();
  for (const h of uniq) {
    const key = r6(((Math.atan2(h[1], h[0]) % Math.PI) + Math.PI)
      % Math.PI);
    if (!bydir.has(key)) bydir.set(key, []);
    bydir.get(key).push(h);
  }
  for (const group of bydir.values()) {
    if (group.length === 2 && group[0][2] + group[1][2] < -EPS) {
      return { poly: [], box: false, area: 0 };
    }
  }
  const dq = [];
  for (const h of uniq) {
    while (dq.length ≥ 2 && bad(h, isect(dq[dq.length - 2],
      dq[dq.length - 1]))) dq.pop();
    while (dq.length ≥ 2 && bad(h, isect(dq[0], dq[1]))) dq.shift();
    dq.push(h);
  }
  while (dq.length ≥ 3 && bad(dq[0], isect(dq[dq.length - 2],
    dq[dq.length - 1]))) dq.pop();
  while (dq.length ≥ 3 && bad(dq[dq.length - 1], isect(dq[0], dq[1]))) {
    dq.shift();
  }
  if (dq.length < 3) return { poly: [], box: false, area: 0 };
  let poly = dq.map((h, i) => isect(h, dq[(i + 1) % dq.length]));
  for (const [h, p] of dq.map((h, i) => [h, poly[i]])) {
    if (bad(h, p)) return { poly: [], box: false, area: 0 };
  }
```

listing 1297: javascript, opposite pairs grouped by direction, the shift-and-pop deque (source: dsa/samples-js/src/ch37-halfplane.mjs, lines 52-81)

```
    bydir = {}
    for h in uniq:  # contradictory opposite pair: the slab is empty
        key = r6(math.atan2(h[1], h[0]) % math.pi)
        bydir.setdefault(key, []).append(h)
    for group in bydir.values():
        if len(group) == 2 and group[0][2] + group[1][2] < -EPS:
            return [], False
    dq = []
    for h in uniq:  # the sort-and-incremental deque
        while len(dq) ≥ 2 and violates(h, intersect(dq[-2], dq[-1])):
            dq.pop()
        while len(dq) ≥ 2 and violates(h, intersect(dq[0], dq[1])):
            dq.pop(0)
        dq.append(h)
    while len(dq) ≥ 3 and violates(dq[0], intersect(dq[-2], dq[-1])):
        dq.pop()
    while len(dq) ≥ 3 and violates(dq[-1], intersect(dq[0], dq[1])):
        dq.pop(0)
    if len(dq) < 3:
        return [], False
    poly = [intersect(dq[i], dq[(i + 1) % len(dq)]) for i in range(len(dq))]
    for h, p in zip(dq, poly):
        if violates(h, p):
            return [], False
    if (
        sum(
            poly[i][0] * poly[(i + 1) % len(poly)][1]
            - poly[(i + 1) % len(poly)][0] * poly[i][1]
            for i in range(len(poly))
        )
        < 0
    ):
        poly.reverse()  # keep ccw
    touches = any(max(abs(x), abs(y)) > BOX - 1.0 for x, y in poly)
    return poly, touches
```

listing 1298: python, the slab test, the deque, the ccw fix and the box flag (source: dsa/samples-py/src/Ch37/halfplane.py, lines 84-118)

```
  -- contradictory opposite pair: x.n ≤ c1 and -x.n ≤ c2 is empty
  -- when c1 + c2 < 0
  for i = 1, #uniq do
    for j = i + 1, #uniq do
      local s, t = uniq[i], uniq[j]
      if math.abs(s[1] + t[1]) < EPS and math.abs(s[2] + t[2]) < EPS then
        if s[3] + t[3] < -EPS then return {}, false end
      end
    end
  end
  local dq = {}
  for _, h in ipairs(uniq) do
    while #dq ≥ 2 and bad(h, isect(dq[#dq - 1], dq[#dq])) do
      table.remove(dq)
    end
    while #dq ≥ 2 and bad(h, isect(dq[1], dq[2])) do
      table.remove(dq, 1)
    end
    dq[#dq + 1] = h
  end
  while #dq ≥ 3 and bad(dq[1], isect(dq[#dq - 1], dq[#dq])) do
    table.remove(dq)
  end
  while #dq ≥ 3 and bad(dq[#dq], isect(dq[1], dq[2])) do
    table.remove(dq, 1)
  end
  if #dq < 3 then return {}, false end
  local poly = {}
  for i = 1, #dq do
    local p = isect(dq[i], dq[i % #dq + 1])
    if bad(dq[i], p) then return {}, false end
    poly[i] = p
  end
  local s = 0.0
  for i = 1, #poly do
    local a, b = poly[i], poly[i % #poly + 1]
    s = s + a[1] * b[2] - b[1] * a[2]
  end
  if s < 0 then
    local rev = {}
    for i = #poly, 1, -1 do rev[#rev + 1] = poly[i] end
    poly = rev
  end
  local touches = false
  for _, p in ipairs(poly) do
    if math.max(math.abs(p[1]), math.abs(p[2])) > BOX - 1.0 then
      touches = true
    end
  end
  return poly, touches
end
```

listing 1299: lua, table.remove at both ends, the orientation sum (source: dsa/samples-lua/ch37_halfplane.lua, lines 46-96)

The square family, x and y each between −1 and 1, returns the cycle (-1,-1), (1,-1), (1,1), (-1,1), area 4, box flag false. The clip family y at least 0, x + y at most 2, minus x plus y at most 2, y at least 1.5 returns the triangle (-0.5,1.5), (0.5,1.5), (0,2), area 0.25. The edges: x at most 0 against x at least 1 is empty, and x at most 0 with y at most 0 alone is unbounded, box flag true with the polygon pinned out at the world box corner (-1000000,-1000000), (0,-1000000),

(0,0), (-1000000,0). The 2018/G anchor is the fourth fixture: the voronoi cell of corner (0,0) among the park corners (0,0), (10,0), (10,6), (0,6), clipped to the park box, is the intersection of the bisector half-planes x at most 5, y at most 3, 20x + 12y at most 136 with the box, and it pins the cycle (0,0), (5,0), (5,3), (0,3), area 15. The 5x + 3y at most 34 bisector passes exactly through (5,3), so the implementation drops the zero-length edge it would create, the consecutive rounded-equal vertex cleanup every suite runs before comparing cycles.

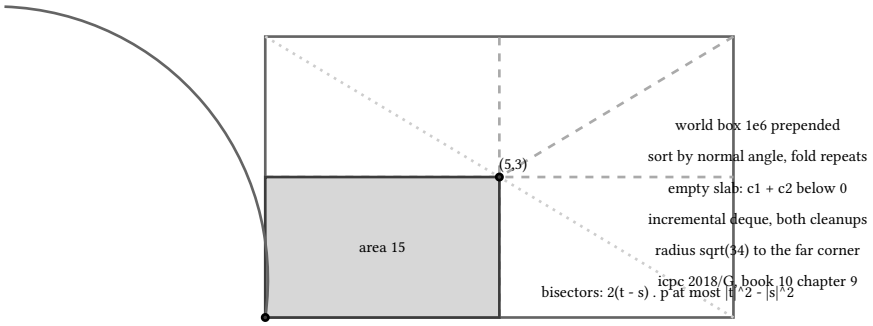


figure 323: the 10 by 6 park split into four voronoi cells meeting at the center, the (0,0) cell shaded, the covering radius arc drawn

The application is icpc 2018 problem G (book 10, chapter 9), the panda preserve: each receiver's voronoi cell is built by exactly this clip, cell against cell, and the covering radius comes from voronoi vertices and edge crossings.

37.7. the minimum enclosing circle

Welzl's move-to-front in its three-nested-loop form, the honest deterministic shell around the randomized argument. Add points in order, keep the current circle, and on a violation reset and re-add the point on the boundary: the outer loop resets to the point alone, the middle loop re-adds a second boundary point and takes the circle on that diameter, the inner loop re-adds a third and takes the circumcenter. The circle from two points is the perpendicular bisector's midpoint, the circle from three is the circumcenter determinant dividing by twice the signed area, and the fixtures never approach the degeneracy where that denominator collapses. Fixtures are at most 6 points with a unique enclosing circle, so input order decides everything deterministically, and the reference itself checked that a seeded shuffle posts the same answers. The textbook note stays in prose: move-to-front runs in expected O(n) under a random order, worst case O(n^3), and the expected bound is why production versions shuffle first.

The dry run: the fixtures are the right triangle (0,0), (4,0), (0,3) and the obtuse family (0,0), (1,0), (0.2,0.1), asserted by the C# suite and by the five sibling suites.

1. The outer loop opens on (0,0) alone, radius 0, and (4,0) lands outside: the reset keeps both as boundary points, and the circle on that diameter is center (2, 0), radius 2.
2. (0,3) checks in at distance sqrt(4 + 9) = sqrt(13) = 3.605551, past radius 2, a violation.
3. The middle loop resets to (0,0) and (4,0), the inner loop re-adds (0,3) as the third boundary point, and the circumcenter of the right triangle is the hypotenuse midpoint (2, 1.5) at half the hypotenuse, 5 / 2 = 2.5.
4. The obtuse family never violates: the diameter circle (0.5, 0) radius 0.5 holds (0.2,0.1) at sqrt(0.09 + 0.01) = 0.316228 inside, so the pair answer stands and the circumcenter, which would fall outside, is never consulted.
5. The square corners pin the circumcircle (0, 0) radius 1.414214, the fourth corner checking in without a reset, and python's interior point (1,1) changes nothing.
6. A single point is its own radius-0 circle, and the pair (0,0), (6,8) meets at (3,4) radius 5.

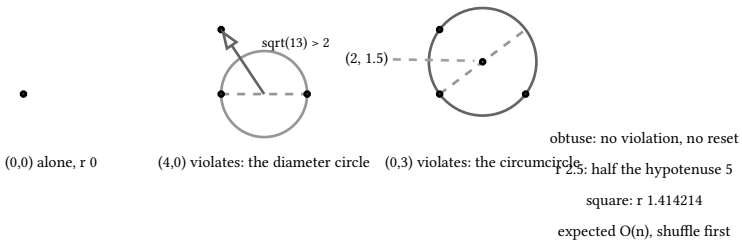


figure 324: the welzl loop on the right triangle as three frames, the diameter circle violated, then the hypotenuse circumcircle closing

The (2, 1.5) radius 2.5 circle is the pinned landing, and the listings below run the triple loop in seven languages.

```
static void circle2(double x1, double y1, double x2, double y2) {
  cx = (x1 + x2) / 2;
  cy = (y1 + y2) / 2;
  cr = hypot(x1 - cx, y1 - cy);
}

static void circle3(double x1, double y1, double x2, double y2, double x3,
                    double y3) {
  double d = 2 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2));
  double s1 = x1 * x1 + y1 * y1, s2 = x2 * x2 + y2 * y2, s3 = x3 * x3 + y3 * y3;
  cx = (s1 * (y2 - y3) + s2 * (y3 - y1) + s3 * (y1 - y2)) / d;
  cy = (s1 * (x3 - x2) + s2 * (x1 - x3) + s3 * (x2 - x1)) / d;
  cr = hypot(x1 - cx, y1 - cy);
}

static void mec(void) {
  have_circle = 0;
  for (int i = 0; i < pc; i++) {
    if (inside(xs[i], ys[i]))
      continue;
    circle1(xs[i], ys[i]);
    for (int j = 0; j < i; j++) {
      if (inside(xs[j], ys[j]))
        continue;
      circle2(xs[i], ys[i], xs[j], ys[j]);
      for (int k = 0; k < j; k++) {
        if (inside(xs[k], ys[k]))
          continue;
        circle3(xs[i], ys[i], xs[j], ys[j], xs[k], ys[k]);
      }
    }
  }
}
```

listing 1300: c, circle2 by the bisector, circle3 by the determinant, the welzl triple loop (source: dsa/samples-c/src/Ch37/mincircle.c, lines 44-76)

```
func MinCircle(pts []Point) (cx, cy, r float64) {
  encloses := func(cx, cy, r float64, q Point) bool {
    return math.Hypot(q.X-cx, q.Y-cy) <= r+1e-7
  }
  circle2 := func(a, b Point) (float64, float64, float64) {
    cx, cy := (a.X+b.X)/2, (a.Y+b.Y)/2
    return cx, cy, math.Hypot(a.X-cx, a.Y-cy)
  }
  circle3 := func(a, b, c Point) (float64, float64, float64, bool) {
    d := 2 * (a.X*(b.Y-c.Y) + b.X*(c.Y-a.Y) + c.X*(a.Y-b.Y))
    if math.Abs(d) < 1e-12 {
      return 0, 0, 0, false
    }
    a2, b2, c2 := a.X*a.X+a.Y*a.Y, b.X*b.X+b.Y*b.Y,
      c.X*c.X+c.Y*c.Y
    ux := (a2*(b.Y-c.Y) + b2*(c.Y-a.Y) + c2*(a.Y-b.Y)) / d
    uy := (a2*(c.X-b.X) + b2*(a.X-c.X) + c2*(b.X-a.X)) / d
    return ux, uy, math.Hypot(a.X-ux, a.Y-uy), true
  }
  cx, cy, r = 0, 0, 0
  for i, q := range pts {
    if encloses(cx, cy, r, q) {
      continue
    }
    cx, cy, r = q.X, q.Y, 0
    for j := 0; j < i; j++ {
      p := pts[j]
      if encloses(cx, cy, r, p) {
        continue
      }
      cx, cy, r = circle2(q, p)
      for k := 0; k < j; k++ {
        s := pts[k]
        if encloses(cx, cy, r, s) {
          continue
        }
        ux, uy, ur, ok := circle3(q, p, s)
        if !ok {
          continue
        }
        cx, cy, r = ux, uy, ur
      }
    }
  }
  return cx, cy, r
}
```

listing 1301: go, the three closures and the triple loop (source: dsa/samples-go/ch37/mincircle.go, lines 10-55)

```
  static void circle2(double x1, double y1, double x2, double y2) {
    cx = (x1 + x2) / 2;
    cy = (y1 + y2) / 2;
    cr = Math.hypot(x1 - cx, y1 - cy);
  }

  static void circle3(double x1, double y1, double x2, double y2, double x3, double y3) {
    double d = 2 * (x1 * (y2 - y3) + x2 * (y3 - y1) + x3 * (y1 - y2));
    double s1 = x1 * x1 + y1 * y1, s2 = x2 * x2 + y2 * y2, s3 = x3 * x3 + y3 * y3;
    cx = (s1 * (y2 - y3) + s2 * (y3 - y1) + s3 * (y1 - y2)) / d;
    cy = (s1 * (x3 - x2) + s2 * (x1 - x3) + s3 * (x2 - x1)) / d;
    cr = Math.hypot(x1 - cx, y1 - cy);
  }

  static void mec() {
    haveCircle = false;
    for (int i = 0; i < pc; i++) {
      if (inside(xs[i], ys[i]))
        continue;
      circle1(xs[i], ys[i]);
      for (int j = 0; j < i; j++) {
        if (inside(xs[j], ys[j]))
          continue;
        circle2(xs[i], ys[i], xs[j], ys[j]);
        for (int k = 0; k < j; k++) {
          if (inside(xs[k], ys[k]))
            continue;
          circle3(xs[i], ys[i], xs[j], ys[j], xs[k], ys[k]);
        }
      }
    }
  }
```

listing 1302: java, circle2 by the bisector, circle3 by the determinant, the welzl triple loop (source: dsa/samples-java/src/Ch37/Mincircle.java, lines 39-70)

```
    public static ((double X, double Y) Center, double Radius) Of(IReadOnlyList<Pt> points)
    {
        var pts = points.ToList();
        (Pt center, double radius) = (pts.Count > 0 ? pts[0] : new Pt(0, 0), 0.0);
        bool Encloses(Pt q) =>
            Math.Sqrt((q.X - center.X) * (q.X - center.X) + (q.Y - center.Y) * (q.Y - center.Y)) ≤ radius
+ 1e-7;
        for (int i = 0; i < pts.Count; i++)
        {
            if (i > 0 && radius > 0 && Encloses(pts[i]))
                continue;
            (center, radius) = (pts[i], 0.0);
            for (int j = 0; j < i; j++)
            {
                if (Encloses(pts[j]))
                    continue;
                (Pt c2, double r2) = Two(pts[i], pts[j]);
                (center, radius) = (c2, r2);
                for (int k = 0; k < j; k++)
                {
                    if (Encloses(pts[k]))
                        continue;
                    (center, radius) = Three(pts[i], pts[j], pts[k]);
                }
            }
        }
        return ((center.X, center.Y), radius);
    }
```

listing 1303: c#, the enclosing local function, the nested resets (source: dsa/samples/src/Ch37/Calipers.cs, lines 186-212)

```
const circle2 = (a, b) => [
  [(a[0] + b[0]) / 2, (a[1] + b[1]) / 2],
  Math.hypot(a[0] - b[0], a[1] - b[1]) / 2,
];

const circle3 = (a, b, c) => {
  const d = 2 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1])
    + c[0] * (a[1] - b[1]));
  if (Math.abs(d) < 1e-12) throw new Error("degenerate circumcircle");
  const a2 = a[0] ** 2 + a[1] ** 2;
  const b2 = b[0] ** 2 + b[1] ** 2;
  const c2 = c[0] ** 2 + c[1] ** 2;
  const ux = (a2 * (b[1] - c[1]) + b2 * (c[1] - a[1])
    + c2 * (a[1] - b[1])) / d;
  const uy = (a2 * (c[0] - b[0]) + b2 * (a[0] - c[0])
    + c2 * (b[0] - a[0])) / d;
  return [[ux, uy], Math.hypot(a[0] - ux, a[1] - uy)];
};

export function minCircle(points) {
  const p = points.map((v) => [...v]);
  let c = null;
  let r = 0.0;
  for (let i = 0; i < p.length; i++) {
    if (enclosing(c, r, p[i])) continue;
    c = [...p[i]];
    r = 0.0;
    for (let j = 0; j < i; j++) {
      if (enclosing(c, r, p[j])) continue;
      [c, r] = circle2(p[i], p[j]);
      for (let k = 0; k < j; k++) {
        if (enclosing(c, r, p[k])) continue;
        [c, r] = circle3(p[i], p[j], p[k]);
      }
    }
  }
  return { center: c, radius: r };
}
```

listing 1304: javascript, destructured reassignments carrying the current circle (source: dsa/samples-js/src/ch37-mincircle.mjs, lines 13-50)

```python
def circle2(a, b):
    return ((a[0] + b[0]) / 2, (a[1] + b[1]) / 2), math.hypot(
        a[0] - b[0], a[1] - b[1]
    ) / 2


def circle3(a, b, c):  # the circumcenter by the determinant form
    d = 2 * (a[0] * (b[1] - c[1]) + b[0] * (c[1] - a[1]) + c[0] * (a[1] - b[1]))
    if abs(d) < 1e-12:
        return None
    a2 = a[0] ** 2 + a[1] ** 2
    b2 = b[0] ** 2 + b[1] ** 2
    c2 = c[0] ** 2 + c[1] ** 2
    ux = (a2 * (b[1] - c[1]) + b2 * (c[1] - a[1]) + c2 * (a[1] - b[1])) / d
    uy = (a2 * (c[0] - b[0]) + b2 * (a[0] - c[0]) + c2 * (b[0] - a[0])) / d
    return (ux, uy), math.hypot(a[0] - ux, a[1] - uy)


def mec(points):
    # input order is deterministic on these unique-circle fixtures
    center, radius = None, 0.0
    for i, q in enumerate(points):
        if enclosing(center, radius, q):
            continue
        center, radius = q, 0.0  # reset, q joins the boundary
        for j in range(i):
            if enclosing(center, radius, points[j]):
                continue
            center, radius = circle2(q, points[j])
            for k in range(j):
                if enclosing(center, radius, points[k]):
                    continue
                center, radius = circle3(q, points[j], points[k])
    return center, radius
```

listing 1305: python, the determinant with its degeneracy guard (source: dsa/samples-py/src/Ch37/mincircle.py, lines 22-55)

```
local function circle2(a, b)
  return {(a[1] + b[1]) / 2, (a[2] + b[2]) / 2},
         math.sqrt((a[1] - b[1]) ^ 2 + (a[2] - b[2]) ^ 2) / 2
end

local function circle3(a, b, c)
  local d = 2 * (a[1] * (b[2] - c[2]) + b[1] * (c[2] - a[2])
                 + c[1] * (a[2] - b[2]))
  if math.abs(d) < 1e-12 then return nil end
  local a2 = a[1] * a[1] + a[2] * a[2]
  local b2 = b[1] * b[1] + b[2] * b[2]
  local c2 = c[1] * c[1] + c[2] * c[2]
  local ux = (a2 * (b[2] - c[2]) + b2 * (c[2] - a[2])
              + c2 * (a[2] - b[2])) / d
  local uy = (a2 * (c[1] - b[1]) + b2 * (a[1] - c[1])
              + c2 * (b[1] - a[1])) / d
  return {ux, uy}, math.sqrt((a[1] - ux) ^ 2 + (a[2] - uy) ^ 2)
end

local function enclosing(c, r, q)
  if c == nil then return false end
  return math.sqrt((q[1] - c[1]) ^ 2 + (q[2] - c[2]) ^ 2) <= r + 1e-7
end

local function mec(pts)
  local p = pts
  local c, r = nil, 0.0
  for i, q in ipairs(p) do
    if not enclosing(c, r, q) then
      c, r = q, 0.0
      for j = 1, i - 1 do
        if not enclosing(c, r, p[j]) then
          c, r = circle2(q, p[j])
          for k = 1, j - 1 do
            if not enclosing(c, r, p[k]) then
              local cc, rr = circle3(q, p[j], p[k])
              if cc then c, r = cc, rr end
            end
          end
        end
      end
    end
  end
  return c, r
end
```

listing 1306: lua, circle2 and circle3 above the same triple loop (source: dsa/samples-lua/ch37_mincircle.lua, lines 9-53)

The right triangle (0,0), (4,0), (0,3) pins center (2, 1.5) radius 2.5, the hypotenuse is the diameter. The obtuse family (0,0), (1,0), (0.2,0.1) pins center (0.5, 0) radius 0.5, the longest side wins because the circumcenter would fall outside. The square corners pin the circumcircle, center (0,0) radius 1.414214. A single point is its own circle of radius 0, the pair (0,0), (6,8) meets midways at (3,4) radius 5, and python adds the fixture that an interior point changes nothing: the square plus (1,1) still pins 1.414214.

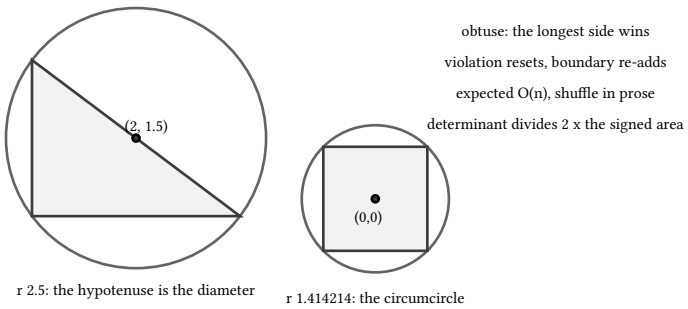


figure 325: the right triangle inside its diameter circle with the midpoint marked, beside the square inside its circumcircle

37.8. voronoi cells and delaunay edges

The cell of site i is the set of points closer to i than to every other site, and each such constraint is one half-plane: the perpendicular bisector of s and t, written 2(t - s) dot p at most |t|^2 - |s|^2, the same form the half-plane section already computes. So build each cell as the world box clipped by one bisector per other site, sutherland-hodgman against each in turn, O(n) clips per cell and O(n^2) total. A delaunay edge exists between two sites exactly when their cells share an edge segment of positive length, adjacency read off rounded cells so clip noise cannot invent or destroy an edge, and duplicate sites empty their cells by rule. This is the 2018/G construction scaled to a family, stated honestly against the literature: no flip algorithm, no divide and conquer, the duality taught on the clipping construction itself.

The dry run: the fixture is the four corners (0,0), (10,0), (10,10), (0,10) over the world box (-10,-10) to (30,30), asserted by the C# suite and by the five sibling suites.

1. Cell 0 opens as the whole 40 by 40 box. The bisector against (10,0) is 20x at most 100, x at most 5, and the bisector against (0,10) reads y at most 5 the same way.
2. The bisector against (10,10) reads x + y at most 10, which only touches the remaining 15 by 15 square at its (5,5) corner: cell 0 closes at area 225.
3. The opposite cell 2 is the 25 by 25 square at 625 and the two side cells run 15 by 25 at 375, and 225 + 375 + 625 + 375 = 1600 tiles the box exactly.
4. The four cells meet only at the single point (5,5), so no shared segment crosses the diagonal: the delaunay list is the rim (0,1), (0,3), (1,2), (2,3), the cocircular degeneracy.
5. Adding the center site (5,5) grows its cell into the diamond (0,5), (5,0), (10,5), (5,10) of area 50 and cuts cell 0 by the new bisector x + y at most 5: 225 - 12.5 = 212.5, the corner triangle gone.
6. The delaunay list grows to the 4 rim edges plus the 4 spokes, 8 total, and duplicate sites empty every copy but one.

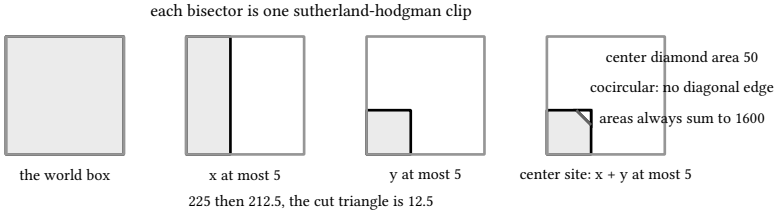


figure 326: cell 0 as a sequence of clips, the 40 by 40 box narrowing to the 15 by 15 corner square, then the center site cutting the last triangle

The 225, 625, 50, and 212.5 areas are the pinned landing, and the listings below clip the cells in seven languages.

```c
static void clip(double a, double b, double c) {
  double nx[MAXV], ny[MAXV];
  int nn = 0;
  for (int i = 0; i < cn; i++) {
    int j = (i + 1) % cn;
    double dc = a * cx[i] + b * cy[i] - c;
    double dn = a * cx[j] + b * cy[j] - c;
    if (dc <= EPS)
      nx[nn] = cx[i], ny[nn] = cy[i], nn++;
    if ((dc < -EPS && dn > EPS) || (dc > EPS && dn < -EPS)) {
      double t = dc / (dc - dn);
      nx[nn] = cx[i] + t * (cx[j] - cx[i]);
      ny[nn] = cy[i] + t * (cy[j] - cy[i]);
      nn++;
    }
  }
  cn = nn;
  memcpy(cx, nx, sizeof(double) * (size_t)nn);
  memcpy(cy, ny, sizeof(double) * (size_t)nn);
}

static void cell_of(int i) {
  // the cell starts as the world box (-10,-10) to (30,30) itself: a
  // rectangle polygon is already the intersection of its four sides
  cn = 4;
  cx[0] = -10;
  cy[0] = -10;
  cx[1] = 30;
  cy[1] = -10;
  cx[2] = 30;
  cy[2] = 30;
  cx[3] = -10;
  cy[3] = 30;
  for (int j = 0; j < sc && cn > 0; j++) {
    if (j == i)
      continue;
    if (sx[j] == sx[i] && sy[j] == sy[i]) { // duplicate site: cell empties
      cn = 0;
      return;
    }
    // |p - s_i|^2 <= |p - s_j|^2  =>  2(s_j - s_i) . p <= |s_j|^2 - |s_i|^2
    double a = 2 * (sx[j] - sx[i]), b = 2 * (sy[j] - sy[i]);
    double c = sx[j] * sx[j] + sy[j] * sy[j] - sx[i] * sx[i] - sy[i] * sy[i];
    clip(a, b, c);
  }
}
```

listing 1307: c, the sutherland-hodgman clip and the cell-of loop (source: dsa/samples-c/src/Ch37/voronoi.c, lines 47-92)

```
// clipHalf keeps a*x + b*y ≤ c by sutherland-hodgman.
func clipHalf(poly []Point, a, b, c float64) []Point {
  var out []Point
  n := len(poly)
  for i := 0; i < n; i++ {
    cur, nxt := poly[i], poly[(i+1)%n]
    dc := a*cur.X + b*cur.Y - c
    dn := a*nxt.X + b*nxt.Y - c
    if dc <= geoEps {
      out = append(out, cur)
    }
    if (dc < -geoEps && dn > geoEps) || (dc > geoEps && dn < -geoEps) {
      t := dc / (dc - dn)
      out = append(out, Point{cur.X + t*(nxt.X-cur.X),
        cur.Y + t*(nxt.Y-cur.Y)})
    }
  }
  return out
}

// VoronoiCells builds each site's cell as the world box clipped by
// one bisector half-plane per other site (the 2018/G construction
// scaled to a family, not a flip algorithm). Duplicate sites empty
// their cells by rule. O(n^2) total.
func VoronoiCells(sites []Point, world []Point) [][]Point {
  cells := make([][]Point, len(sites))
  for i, s := range sites {
    poly := append([]Point(nil), world...)
    dup := false
    for j, t := range sites {
      if i == j {
        continue
      }
      if t == s {
        dup = true
        break
      }
      // |p - s|^2 ≤ |p - t|^2 ⟺ 2(t - s) . p ≤ |t|^2 - |s|^2
      poly = clipHalf(poly, 2*(t.X-s.X), 2*(t.Y-s.Y),
        t.X*t.X+t.Y*t.Y-s.X*s.X-s.Y*s.Y)
      if len(poly) == 0 {
        break
      }
    }
    if dup {
      cells[i] = nil
    } else {
      cells[i] = poly
    }
  }
  return cells
}
```

listing 1308: go, clip-half and the cells loop, duplicates emptying out (source: dsa/samples-go/ch37/voronoi.go, lines 8-59)

```
  static void clip(double a, double b, double c) {
    double[] nx = new double[MAXV], ny = new double[MAXV];
    int nn = 0;
    for (int i = 0; i < cn; i++) {
      int j = (i + 1) % cn;
      double dc = a * cx[i] + b * cy[i] - c;
      double dn = a * cx[j] + b * cy[j] - c;
      if (dc ≤ EPS) {
        nx[nn] = cx[i];
        ny[nn] = cy[i];
        nn++;
      }
      if ((dc < -EPS && dn > EPS) || (dc > EPS && dn < -EPS)) {
        double t = dc / (dc - dn);
        nx[nn] = cx[i] + t * (cx[j] - cx[i]);
        ny[nn] = cy[i] + t * (cy[j] - cy[i]);
        nn++;
      }
    }
    cn = nn;
    System.arraycopy(nx, 0, cx, 0, nn);
    System.arraycopy(ny, 0, cy, 0, nn);
  }

  static void cellOf(int i) {
    // the cell starts as the world box (-10,-10) to (30,30) itself
    cn = 4;
    cx[0] = -10;
    cy[0] = -10;
    cx[1] = 30;
    cy[1] = -10;
    cx[2] = 30;
    cy[2] = 30;
    cx[3] = -10;
    cy[3] = 30;
    for (int j = 0; j < sc && cn > 0; j++) {
      if (j == i)
        continue;
      if (sx[j] == sx[i] && sy[j] == sy[i]) { // duplicate site: cell empties
        cn = 0;
        return;
      }
      // |p - s_i|^2 ≤ |p - s_j|^2  ⇒  2(s_j - s_i) . p ≤ |s_j|^2 - |s_i|^2
      double a = 2 * (sx[j] - sx[i]), b = 2 * (sy[j] - sy[i]);
      double c = sx[j] * sx[j] + sy[j] * sy[j] - sx[i] * sx[i] - sy[i] * sy[i];
      clip(a, b, c);
    }
  }
```

listing 1309: java, the sutherland-hodgman clip and the cell-of loop over the world box (source: dsa/samples-java/src/Ch37/Voronoi.java, lines 39-86)

```
    public static List<List<Pt>> Cells(Pt[] box, IReadOnlyList<Pt> sites)
    {
        var cells = new List<List<Pt>>();
        for (int i = 0; i < sites.Count; i++)
        {
            List<Pt> poly = box.ToList();
            bool dup = false;
            for (int j = 0; j < sites.Count && !dup; j++)
            {
                if (i == j)
                    continue;
                if (sites[j].X == sites[i].X && sites[j].Y == sites[i].Y)
                {
                    dup = true; // duplicate sites empty their cells by rule
                    break;
                }
                // |p - s|^2 ≤ |p - t|^2 becomes 2(t - s) . p ≤ |t|^2 - |s|^2
                double a = 2 * (sites[j].X - sites[i].X);
                double b = 2 * (sites[j].Y - sites[i].Y);
                double c = sites[j].X * sites[j].X + sites[j].Y * sites[j].Y
                    - sites[i].X * sites[i].X - sites[i].Y * sites[i].Y;
                poly = Clip(poly, a, b, c);
                if (poly.Count == 0)
                    break;
            }
            cells.Add(dup ? new List<Pt>() : Halfplane.Canonicalize(poly));
        }
        return cells;
    }

    /// <summary>Sutherland-Hodgman: keep a*x + b*y &lt;= c.</summary>
    private static List<Pt> Clip(List<Pt> poly, double a, double b, double c)
    {
        var output = new List<Pt>();
        for (int i = 0; i < poly.Count; i++)
        {
            Pt cur = poly[i];
            Pt next = poly[(i + 1) % poly.Count];
            double dc = a * cur.X + b * cur.Y - c;
            double dn = a * next.X + b * next.Y - c;
            if (dc ≤ Geometry2.Eps)
                output.Add(cur);
            if ((dc < -Geometry2.Eps && dn > Geometry2.Eps) || (dc > Geometry2.Eps && dn < -Geometry2.Eps))
            {
                double t = dc / (dc - dn);
                output.Add(new Pt(cur.X + t * (next.X - cur.X), cur.Y + t * (next.Y - cur.Y)));
            }
        }
        return output;
    }
```

listing 1310: c#, cells over the box list, the clip with its eps guards (source: dsa/samples/src/Ch37/Voronoi.cs, lines 10-59)

```
// keep a*x + b*y ≤ c, one edge at a time
const clipCell = (poly, a, b, c) => {
  const out = [];
  const n = poly.length;
  for (let i = 0; i < n; i++) {
    const cur = poly[i];
    const nxt = poly[(i + 1) % n];
    const dc = a * cur[0] + b * cur[1] - c;
    const dn = a * nxt[0] + b * nxt[1] - c;
    if (dc ≤ EPS) out.push(cur);
    if ((dc < -EPS && dn > EPS) || (dc > EPS && dn < -EPS)) {
      const t = dc / (dc - dn);
      out.push([cur[0] + t * (nxt[0] - cur[0]),
        cur[1] + t * (nxt[1] - cur[1])]);
    }
  }
  return out;
};

export function voronoiCells(sites) {
  const cells = [];
  for (let i = 0; i < sites.length; i++) {
    const s = sites[i];
    let poly = BOX.map((p) => [...p]);
    let dup = false;
    for (let j = 0; j < sites.length; j++) {
      if (i === j) continue;
      const t = sites[j];
      if (t[0] === s[0] && t[1] === s[1]) {
        dup = true;
        break;
      }
      // |p - s|^2 ≤ |p - t|^2 ⇒ 2(t - s) . p ≤ |t|^2 - |s|^2
      const a = 2 * (t[0] - s[0]);
      const b = 2 * (t[1] - s[1]);
      const c = t[0] ** 2 + t[1] ** 2 - s[0] ** 2 - s[1] ** 2;
      poly = clipCell(poly, a, b, c);
      if (poly.length === 0) break;
    }
    cells.push(dup ? [] : poly);
  }
  return cells;
}
```

listing 1311: javascript, the clip keeping the inside and cutting the crossing (source: dsa/samples-js/src/ch37-voronoi.mjs, lines 45-87)

```
def clip(poly, a, b, c):
    # keep a*x + b*y ≤ c, one edge at a time
    out = []
    for i in range(len(poly)):
        cur, nxt = poly[i], poly[(i + 1) % len(poly)]
        dc = a * cur[0] + b * cur[1] - c
        dn = a * nxt[0] + b * nxt[1] - c
        if dc ≤ EPS:
            out.append(cur)
        if (dc < -EPS and dn > EPS) or (dc > EPS and dn < -EPS):
            t = dc / (dc - dn)
            px = cur[0] + t * (nxt[0] - cur[0])
            py = cur[1] + t * (nxt[1] - cur[1])
            out.append((px, py))
    return out


def voronoi_cells(sites):
    cells = []
    for i, s in enumerate(sites):
        poly = list(BOX)
        dup = False
        for j, t in enumerate(sites):
            if i == j:
                continue
            if t[0] == s[0] and t[1] == s[1]:  # duplicates empty out
                dup = True
                break
            # |p - s|^2 ≤ |p - t|^2 gives 2(t - s) . p ≤ |t|^2 - |s|^2
            a, b = 2 * (t[0] - s[0]), 2 * (t[1] - s[1])
            c = t[0] ** 2 + t[1] ** 2 - s[0] ** 2 - s[1] ** 2
            poly = clip(poly, a, b, c)
            if not poly:
                break
        cells.append([] if dup else poly)
    return cells
```

listing 1312: python, the same pair, the canonical form beside (source: dsa/samples-py/src/Ch37/voronoi.py, lines 47-82)

```
local function clip_cell(poly, a, b, c)
  local out = {}
  local n = #poly
  for i = 1, n do
    local cur, nxt = poly[i], poly[i % n + 1]
    local dc = a * cur[1] + b * cur[2] - c
    local dn = a * nxt[1] + b * nxt[2] - c
    if dc ≤ EPS then out[#out + 1] = cur end
    if (dc < -EPS and dn > EPS) or (dc > EPS and dn < -EPS) then
      local t = dc / (dc - dn)
      out[#out + 1] = {cur[1] + t * (nxt[1] - cur[1]),
                       cur[2] + t * (nxt[2] - cur[2])}
    end
  end
  return out
end

local function voronoi_cells(sites)
  local cells = {}
  for i, s in ipairs(sites) do
    local poly = {}
    for _, p in ipairs(BOX) do poly[#poly + 1] = {p[1], p[2]} end
    local dup = false
    for j, t in ipairs(sites) do
      if i ~= j then
        if t[1] == s[1] and t[2] == s[2] then
          dup = true -- duplicate sites empty their cells by rule
          break
        end
        -- |p - s|^2 ≤ |p - t|^2 becomes 2(t - s) . p ≤ |t|^2 - |s|^2
        local a, b = 2 * (t[1] - s[1]), 2 * (t[2] - s[2])
        local c = t[1] * t[1] + t[2] * t[2] - s[1] * s[1] - s[2] * s[2]
        poly = clip_cell(poly, a, b, c)
        if #poly == 0 then break end
      end
    end
    cells[i] = dup and {} or poly
  end
  return cells
end
```

listing 1313: lua, the clip and cells over the world box table (source: dsa/samples-lua/ch37_voronoi.lua, lines 13-52)

The world box runs (-10,-10) to (30,30) and the fixtures pin canonical cells and areas rounded to six decimals. Four corners (0,0), (10,0), (10,10), (0,10): every cell is a 5-wide quadrant rectangle of the box cut at x = 5 and y = 5, areas 225, 375, 625, 375, summing to the box's 1600. The cocircular degeneracy is visible and pinned: all four cells meet only at (5,5), so the delaunay edge list is exactly the rim (0,1), (0,3), (1,2), (2,3) with no diagonal. Adding the center site (5,5) makes the center cell the diamond (0,5), (5,0), (10,5), (5,10) with area 50, corner cells gain one diagonal corner each, cell 0 cycling (-10,-10), (5,-10), (5,0), (0,5), (-10,5) at area 212.5, and the delaunay list grows to the 4 rim edges plus the 4 spokes (0,4), (1,4), (2,4), (3,4), 8 total. Duplicate sites (2,2), (2,2), (5,1) leave exactly 1 nonempty cell. The property fixture runs the 37.3 fan test: every cell is convex and contains its site, and cell areas always sum to the box area. The application is icpc 2018 problem G (book 10, chapter 9) again, at n = 2000 receivers, this construction verbatim.

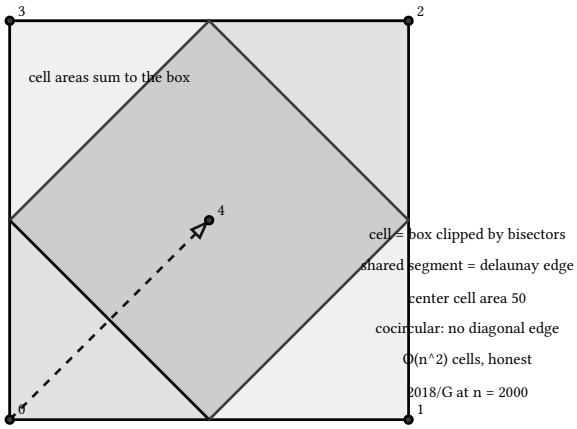


figure 327: the four-corner box with the center site added, cells shaded, one shared edge bolded with its delaunay spoke

37.9. the bentley-ottmann sweep

Report every crossing of a segment set in general position: no vertical segments, no shared endpoints, no crossing shared by two pairs. Events are segment endpoints and discovered crossings in a heap ordered by (x, y). The status is an ordered list of segment ids by height at the sweep x, tie-broken by slope. The discipline is the same one book 9, ch 23 (practical data structures and algorithms, computational geometry)'s shamos-hoey boolean sweep uses, neighbors only, never all pairs: on a left endpoint insert by height and check the two new neighbor pairs, on a right endpoint remove and check the seam, and on a crossing event swap the pair, which the precondition keeps adjacent, then check both new neighbor pairs. Every scheduled crossing lies strictly ahead of the sweep and each pair fires at most once.

The dry run: the fixture is the three segments (0,4)-(10,4), (2,0)-(6,8), (5,9)-(9,0), asserted by the C# suite and by the five sibling suites.

1. The endpoint events open the sweep: s0 enters alone at (0,4), then s1 enters below it at (2,0), and the fresh neighbor pair (s1,s0) schedules its crossing.
2. The heap pops that crossing first, at (4,4): report, swap, and s0 now sits below s1 with no new neighbor pair to check.
3. s2 enters on top at (5,9), the status s0, s1, s2 bottom to top: the pair (s1,s2) schedules (5.705882, 7.411765), while (s0,s1) has already fired and stays quiet.
4. That crossing reports and swaps s2 below s1, and the fresh pair (s0,s2) schedules (7.222222, 4).
5. s1 retires at (6,8) and the seam check finds (s0,s2) already scheduled, then the third crossing reports at (7.222222, 4) and the swap puts s2 below s0.
6. The right endpoints (9,0) and (10,4) drain the status empty: 3 crossings, each pair fired once.

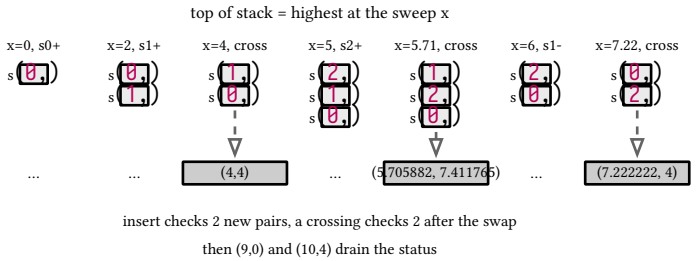


figure 328: the sweep as event columns, the status stack under each event with the highest on top, the three reports shaded

The three pinned crossings in heap order are the landing, and the listings below run the sweep in seven languages.

```
    } else { // crossing: swap a and b (adjacent in the status), recheck
      int pa = -1, pb = -1;
      for (int i = 0; i < stn; i++) {
        if (status[i] == a)
          pa = i;
        if (status[i] == b)
          pb = i;
      }
      if (pa < 0 || pb < 0)
        continue;
      if (pa > pb) {
        int t = pa;
        pa = pb;
        pb = t;
        t = a;
        a = b;
        b = t;
      }
      if (pb != pa + 1)
        continue; // no longer neighbors: stale event
      status[pa] = b;
      status[pb] = a;
      check_pair(pa > 0 ? status[pa - 1] : -1, b, x, y);
      check_pair(a, pb + 1 < stn ? status[pb + 1] : -1, x, y);
      outx[outn] = x;
      outy[outn] = y;
      outn++;
    }
  }
  return outn;
```

listing 1314: c, the crossing event: find, adjacency check, swap, both new pairs (source: dsa/samples-c/src/Ch37/ottmann.c, lines 197-226)

```
    default: // crossing: swap the adjacent pair
      i, j := ev.pair[0], ev.pair[1]
      pi, pj := -1, -1
      for k, v := range status {
        if v == i {
          pi = k
        }
        if v == j {
          pj = k
        }
      }
      if pi < 0 || pj < 0 || pi-pj ≠ 1 && pi-pj ≠ -1 {
        continue // stale: the pair is no longer adjacent
      }
      status[pi], status[pj] = status[pj], status[pi]
      lo, hi := pi, pj
      if lo > hi {
        lo, hi = hi, lo
      }
      if lo > 0 {
        schedule(status[lo-1], status[lo], ev.x)
      }
      if hi+1 < len(status) {
        schedule(status[hi], status[hi+1], ev.x)
      }
      out = append(out, Point{ev.x, ev.y})
    }
```

listing 1315: go, the stale-event guard, the swap, the two schedules (source: dsa/samples-go/ch37/ottmann.go, lines 148-174)

```
      } else { // crossing: swap a and b (adjacent in the status), recheck
        int pa = -1, pb = -1;
        for (int i = 0; i < stn; i++) {
          if (status[i] == a)
            pa = i;
          if (status[i] == b)
            pb = i;
        }
        if (pa < 0 || pb < 0)
          continue;
        if (pa > pb) {
          int t = pa;
          pa = pb;
          pb = t;
          t = a;
          a = b;
          b = t;
        }
        if (pb ≠ pa + 1)
          continue; // no longer neighbors: stale event
        status[pa] = b;
        status[pb] = a;
        checkPair(pa > 0 ? status[pa - 1] : -1, b, x, y);
        checkPair(a, pb + 1 < stn ? status[pb + 1] : -1, x, y);
        outx[outn] = x;
        outy[outn] = y;
        outn++;
      }
    }
```

listing 1316: java, the crossing event: find, adjacency check, swap, both new pairs (source: dsa/samples-java/src/Ch37/Ottmann.java, lines 191-219)

```
        while (events.Count > 0)
        {
            (double x, double y, int kind, int id) = events.Min;
            events.Remove(events.Min);
            if (kind == 0)
            {
                InsertAt(status, segments, id, x + 1e-9);
                CheckNeighbors(status, segments, id, x, y, events, scheduled);
            }
            else if (kind == 2)
            {
                int pos = status.IndexOf(id);
                int left = pos > 0 ? status[pos - 1] : -1;
                int right = pos ≥ 0 && pos + 1 < status.Count ? status[pos + 1] : -1;
                if (pos ≥ 0)
                    status.RemoveAt(pos);
                SchedulePair(left, right, segments, x, y, events, scheduled);
            }
            else
            {
                crossings.Add(new Pt(x, y));
                SwapAtCrossing(status, segments, x, y, out int first, out int second);
                CheckNeighbors(status, segments, first, x, y, events, scheduled);
                CheckNeighbors(status, segments, second, x, y, events, scheduled);
            }
        }
        return crossings
            .Select(p => new Pt(Math.Round(p.X, 6), Math.Round(p.Y, 6)))
            .OrderBy(p => p.X)
            .ThenBy(p => p.Y)
            .ToList();
    }
```

listing 1317: c#, the event loop over the sorted set, insert, remove, swap (source: dsa/samples/src/Ch37/Voronoi.cs, lines 129-160)

```
  while (events.size > 0) {
    const [x, y, kind, data] = events.pop();
    curX = x;
    curY = y;
    if (kind === 0) {
      // insert by height at the sweep x
      let i = 0;
      while (i < status.length && yAt(status[i], x) < y) i += 1;
      status.splice(i, 0, segs[data]);
      checkAround(i);
    } else if (kind === 1) {
      const i = status.indexOf(segs[data]);
      status.splice(i, 1);
      if (i > 0 && i < status.length) schedule(status[i - 1], status[i]);
    } else {
      const [a, b] = data;
      const key = a.i < b.i ? `${a.i}-${b.i}` : `${b.i}-${a.i}`;
      if (crossed.has(key)) continue; // already fired, stale re-schedule
      const ia = status.indexOf(a);
      const ib = status.indexOf(b);
      if (ia === -1 || ib === -1 || Math.abs(ia - ib) !== 1) continue;
      const cp = segInt([a.p, a.q], [b.p, b.q]);
      if (cp === null || Math.hypot(cp[0] - x, cp[1] - y) > 1e-9) continue;
      crossed.add(key);
      // swap the crossed pair in place
      [status[ia], status[ib]] = [status[ib], status[ia]];
      results.push(cp);
      const lo = Math.min(ia, ib);
      const hi = Math.max(ia, ib);
      if (lo > 0) schedule(status[lo - 1], status[lo]);
      if (hi + 1 < status.length) schedule(status[hi], status[hi + 1]);
    }
  }
  return results.sort((u, v) => u[0] - v[0] || u[1] - v[1]);
```

listing 1318: javascript, the three kinds over the heap, results sorted at the end (source: dsa/samples-js/src/ch37-ottmann.mjs, lines 125-158)

```
    while events:
        x, y, kind, payload = heapq.heappop(events)
        if kind == 0:
            i = payload
            ki = key(i, x)
            pos = len(status)
            for k, j in enumerate(status):
                if key(j, x) > ki:
                    pos = k
                    break
            status.insert(pos, i)
            if pos > 0:  # the new neighbors
                check_pair(status[pos - 1], i, x)
            if pos + 1 < len(status):
                check_pair(i, status[pos + 1], x)
        elif kind == 2:
            i = payload
            pos = status.index(i)
            status.pop(pos)
            if 0 < pos < len(status):
                check_pair(status[pos - 1], status[pos], x)
        else:
            a, b = payload  # swap the crossing pair in the status
            if a not in status or b not in status:
                continue  # a stale event
            ia = status.index(a)
            ib = status.index(b)
            if abs(ia - ib) != 1:
                continue  # no longer neighbors
            crossings.append((x, y))
            lo, hi = min(ia, ib), max(ia, ib)
            status[lo], status[hi] = status[hi], status[lo]
            if lo > 0:  # the two fresh neighbor pairs
                check_pair(status[lo - 1], status[lo], x)
            if hi + 1 < len(status):
                check_pair(status[hi], status[hi + 1], x)
    return sorted(crossings)
```

listing 1319: python, check-pair guards the schedule, the crossing branch last (source: dsa/samples-py/src/Ch37/ottmann.py, lines 82-118)

```
function Ottmann:run()
  self.status = {}
  self.sweep_x = -math.huge
  while #self.events > 0 do
    local e = self:pop_event()
    self.sweep_x = e[1]
    local kind = e[3]
    if kind == 0 then -- left endpoint: insert, check new neighbors
      local pos = self:status_insert(e[4], e[1])
      self:check_pair(pos)
      self:check_pair(pos - 1)
    elseif kind == 2 then -- right endpoint: remove, check the seam
      local st = self.status
      for k = 1, #st do
        if st[k] == e[4] then
          table.remove(st, k)
          self:check_pair(k - 1)
          break
        end
      end
    else -- crossing: report, swap, check both new neighbor pairs
      self.out[#self.out + 1] = {e[1], e[2]}
      local st = self.status
      local ka, kb
      for k = 1, #st do
        if st[k] == e[4] then ka = k end
        if st[k] == e[5] then kb = k end
      end
      if ka and kb then
        st[ka], st[kb] = st[kb], st[ka]
        local lo = math.min(ka, kb)
        self:check_pair(lo - 1)
        self:check_pair(lo + 1)
      end
    end
  end
  table.sort(self.out, function(a, b)
    if a[1] ~= b[1] then return a[1] < b[1] end
    return a[2] < b[2]
  end)
  return self.out
end
```

listing 1320: lua, the run loop with all three event kinds (source: dsa/samples-lua/ch37_ottmann.lua, lines 145-186)

The plain-list status keeps inserts linear, so the honest worst case is O((n + k) n), with the O((n + k) log n) balanced-tree claim stated in prose, the discipline matching chapter 23. Every suite cross-checks its sweep against the all-pairs orientation test on the same fixtures. The three-segment family (0,4)-(10,4), (2,0)-(6,8), (5,9)-(9,0) reports 3 crossings (4, 4), (5.705882, 7.411765), (7.222222, 4). The four-segment family reports 5: (2, 0), (4.25, 2.75), (4.4, 2.4), (4.5, 2.5), (7, 0). Three mutually disjoint segments terminate after their 6 endpoint events with nothing scheduled. The seeded family draws 12 segments from the pinned lcg, seed 10, endpoints as 4 values mod 101 with vertical and degenerate draws rejected, and the suite pins exactly 11 crossings, first (25.207353, 55.124721), last (79.084648, 79.760402).

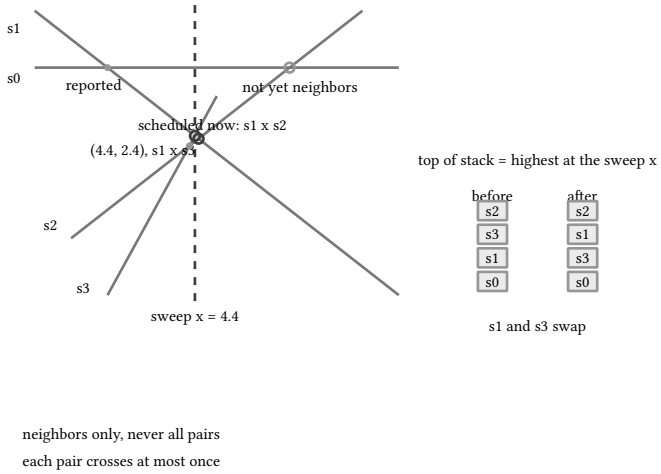


figure 329: the sweep frozen at a crossing event, the status stack beside it, the swap making two new neighbor pairs, one new crossing scheduled

37.10. planar faces and point location

No samples ship for this section: the interesting machinery is the arrangement bookkeeping, not another metered port, and the ottmann suite already carries the sweep it would build on. Euler's formula closes the face count of a connected planar embedding, V - E + F = 2: run the ottmann sweep, split every segment at its crossings, count vertices as endpoints plus crossings and edges as the split pieces, and the faces fall out by subtraction. The faces themselves are the cycles of the arrangement graph walked by the next-half-edge rule, at each vertex take the outgoing edge clockwise-adjacent to the one you arrived on, and every edge is bordered by exactly two faces walked in opposite directions. Point location then answers which face holds a query: the slab method sorts the segments into vertical slabs and binary searches each slab's ordered list in O(log n), and persistence structures answer arbitrary planar subdivisions the same way. Ties back into this chapter twice: the 2018/G boundary walk is a face traversal on the voronoi arrangement, and the voronoi section's cell-containment check is point location run cell by cell.

The dry run: no suite carries this section, the numbers are hand-derived, and they stand on the ottmann suite's pinned crossings for the four-segment family.

1. The family (0,0)-(10,0), (0,-2)-(10,8), (1,6)-(9,-2), (2,8)- (5,1) pins 5 crossings, so the arrangement holds V = 8 + 5 = 13 vertices, endpoints plus crossings.
2. Splitting at the crossings: s0 collects (2,0) and (7,0) into 3 pieces, s1 its three into 4, s2 its three into 4, s3 its two into 3.
3. E = 3 + 4 + 4 + 3 = 14 split edges, and the embedding is connected, every segment crossing at least one other.
4. Euler closes the count: F = 2 - V + E = 2 - 13 + 14 = 3, two bounded faces plus the outer one.
5. The bounded pair reads straight off the pinned crossings: the large cycle through (2,0), (4.5,2.5), (7,0) and the sliver (4.25,2.75), (4.5,2.5), (4.4,2.4).
6. Point location on the same arrangement is the slab method: sort the split edges into vertical slabs, binary search the slab, compare against its ordered edges.

segment	crossings on it	pieces
s0 (0,0)-(10,0)	(2,0), (7,0)	3
s1 (0,-2)-(10,8)	(2,0), (4.4,2.4), (4.5,2.5)	4
s2 (1,6)-(9,-2)	(4.25,2.75), (4.5,2.5), (7,0)	4
s3 (2,8)-(5,1)	(4.25,2.75), (4.4,2.4)	3

The face count 3 out of V - E + F = 2 is the landing, and the figure below shades the two bounded faces it counts.

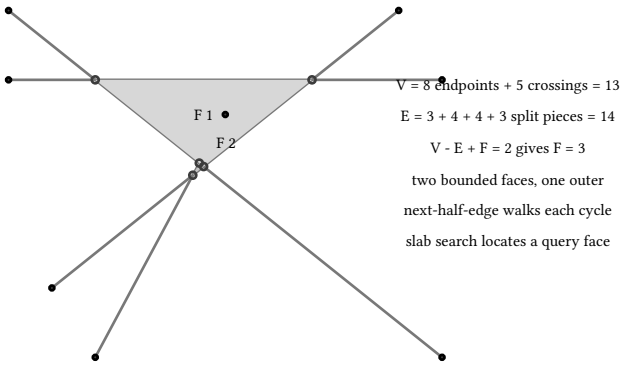


figure 330: a 4-segment arrangement with its bounded faces shaded and counted against V - E + F

37.11. the manhattan transform

Map (x, y) to (u, v) = (x + y, x - y). The L1 distance of two points equals the chebyshev distance of their images, |dx| + |dy| against max(|du|, |dv|), because the transform is a scaling and 45-degree rotation: the L1 ball is a diamond, the chebyshev ball is the square it rotates into. The L1 diameter of a point set is then read off the four extreme images, minimum u, maximum u, minimum v, maximum v, a linear scan after the transformation, with the O(n^2) pairwise scan kept in every suite as the oracle. Integer inputs keep everything exact in all seven languages.

The dry run: the fixture is (0,0), (1,5), (4,2), (6,6), asserted by the C# suite and by the five sibling suites.

1. The transform lands the images (u, v) = (x + y, x - y) at (0,0), (6,-4), (6,2), (12,0).
2. The u extremes are 0 at (0,0) and 12 at (6,6), spread 12 - 0 = 12, and the v extremes are −4 at (1,5) and 2 at (4,2), spread 2 - (-4) = 6.
3. The chebyshev diameter is the max of the spreads, 12, and the oracle agrees: the L1 distance of (0,0)-(6,6) is 6 + 6 = 12.
4. Every other pair sits at exactly 6 in both frames, so the four extreme images carry the whole answer.
5. The property family draws 20 seeded points and asserts the equality pairwise, and the edge family pins a single point at diameter 0 with the whole x = y line mapping to v = 0.

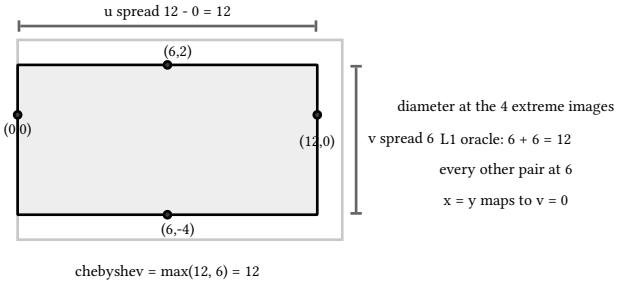


figure 331: the four uv images, the u spread of 12 shaded as the wide band, the v spread of 6 as the narrow one, the max wins

The 12, the same number in both frames, is the pinned landing, and the listings below transform and scan in seven languages.

```c
// the L1 diameter from the four extreme images
static long long l1_diameter(int *ia, int *ib) {
  long long minu = 0, maxu = 0, minv = 0, maxv = 0;
  int iminu = 0, imaxu = 0, iminv = 0, imaxv = 0;
  for (int i = 0; i < pc; i++) {
    long long u = px[i] + py[i], v = px[i] - py[i];
    if (i == 0 || u < minu) {
      minu = u;
      iminu = i;
    }
    if (i == 0 || u > maxu) {
      maxu = u;
      imaxu = i;
    }
    if (i == 0 || v < minv) {
      minv = v;
      iminv = i;
    }
    if (i == 0 || v > maxv) {
      maxv = v;
      imaxv = i;
    }
  }
  long long best = -1;
  int cands[8] = {iminu, imaxu, iminv, imaxv, iminu, imaxv, iminv, imaxu};
  for (int i = 0; i < 8; i++)
    for (int j = i + 1; j < 8; j++) {
      long long d = l1(px[cands[i]], py[cands[i]], px[cands[j]], py[cands[j]]);
      if (d > best) {
        best = d;
        *ia = cands[i];
        *ib = cands[j];
      }
    }
  return best;
}
```

listing 1321: c, the transform inline, the four extremes, the candidate pairs (source: dsa/samples-c/src/Ch37/manhattan.c, lines 36-71)

```
func L1Diameter(pts []Point) (float64, Point, Point) {
  if len(pts) == 0 {
    return 0, Point{}, Point{}
  }
  type uv struct {
    u, v float64
    p    Point
  }
  images := make([]uv, len(pts))
  for i, p := range pts {
    images[i] = uv{p.X + p.Y, p.X - p.Y, p}
  }
  pick := func(better func(a, b uv) bool) uv {
    best := images[0]
    for _, x := range images[1:] {
      if better(x, best) {
        best = x
      }
    }
    return best
  }
  minU := pick(func(a, b uv) bool { return a.u < b.u })
  maxU := pick(func(a, b uv) bool { return a.u > b.u })
  minV := pick(func(a, b uv) bool { return a.v < b.v })
  maxV := pick(func(a, b uv) bool { return a.v > b.v })
  d := 0.0
  a, b := minU.p, maxU.p
  check := func(x, y uv) {
    if c := math2max(math.Abs(x.u-y.u), math.Abs(x.v-y.v)); c > d {
      d, a, b = c, x.p, y.p
    }
  }
  check(minU, maxU)
  check(minV, maxV)
  return d, a, b
}
```

listing 1322: go, the pick helper scanning for each extreme (source: dsa/samples-go/ch37/manhattan.go, lines 9-44)

```
  static long l1Diameter() {
    long minu = 0, maxu = 0, minv = 0, maxv = 0;
    int iminu = 0, imaxu = 0, iminv = 0, imaxv = 0;
    for (int i = 0; i < pc; i++) {
      long u = px[i] + py[i], v = px[i] - py[i];
      if (i == 0 || u < minu) {
        minu = u;
        iminu = i;
      }
      if (i == 0 || u > maxu) {
        maxu = u;
        imaxu = i;
      }
      if (i == 0 || v < minv) {
        minv = v;
        iminv = i;
      }
      if (i == 0 || v > maxv) {
        maxv = v;
        imaxv = i;
      }
    }
    long best = -1;
    int[] cands = {iminu, imaxu, iminv, imaxv, iminu, imaxv, iminv, imaxu};
    for (int i = 0; i < 8; i++)
      for (int j = i + 1; j < 8; j++) {
        long d = l1(px[cands[i]], py[cands[i]], px[cands[j]], py[cands[j]]);
        if (d > best) {
          best = d;
          iaOut = cands[i];
          ibOut = cands[j];
        }
      }
    return best;
  }
```

listing 1323: java, the transform inline, the four extremes, the candidate pairs (source: dsa/samples-java/src/Ch37/Manhattan.java, lines 34-69)

```
/// <summary>(x, y) to (u, v) = (x + y, x - y): L1 distance becomes chebyshev distance.</summary>
public static class Manhattan
{
    public static (long U, long V) Transform(Pt p) => ((long)(p.X + p.Y), (long)(p.X - p.Y));

    /// <summary>L1 diameter at the four extreme images, a linear scan after transform.</summary>
    public static long L1Diameter(IReadOnlyList<Pt> points)
    {
        if (points.Count ≤ 1)
            return 0;
        var images = points.Select(Transform).ToList();
        long best = 0;
        foreach ((long u1, long v1) in images)
            foreach ((long u2, long v2) in images)
                best = Math.Max(best, Math.Max(Math.Abs(u1 - u2), Math.Abs(v1 - v2)));
        return best;
    }
```

listing 1324: c#, transform as a tuple, the diameter over image pairs (source: dsa/samples/src/Ch37/Calipers.cs, lines 230-246)

```
export const toChebyshev = (points) =>
  points.map(([x, y]) => [x + y, x - y]);

export function manhattanDiameter(points) {
  if (points.length === 0) return { d: 0, pair: null };
  const uv = toChebyshev(points);
  let minU = 0;
  let maxU = 0;
  let minV = 0;
  let maxV = 0;
  for (let i = 1; i < uv.length; i++) {
    if (uv[i][0] < uv[minU][0]) minU = i;
    if (uv[i][0] > uv[maxU][0]) maxU = i;
    if (uv[i][1] < uv[minV][1]) minV = i;
    if (uv[i][1] > uv[maxV][1]) maxV = i;
  }
  const du = uv[maxU][0] - uv[minU][0];
  const dv = uv[maxV][1] - uv[minV][1];
  if (du ≥ dv) {
    return { d: du, pair: [points[minU], points[maxU]] };
  }
  return { d: dv, pair: [points[minV], points[maxV]] };
}
```

listing 1325: javascript, the index extremes, the u or v decision (source: dsa/samples-js/src/ch37-manhattan.mjs, lines 8-30)

```
def cheb_diameter(points):
    images = [transform(p) for p in points]
    best = 0
    pair = None
    for i in range(len(images)):
        for j in range(i + 1, len(images)):
            d = cheb(images[i], images[j])
            if d > best:
                best = d
                pair = (images[i], images[j])
    return best, pair


def extremes_diameter(points):
    # the four extreme images: a linear scan after transformation
    images = [transform(p) for p in points]
    corners = [
        min(images),
        max(images),
        min(images, key=lambda t: (t[1], t[0])),
        max(images, key=lambda t: (t[1], t[0])),
    ]
    best = 0
    for a in corners:
        for b in corners:
            best = max(best, cheb(a, b))
    return best
```

listing 1326: python, the brute oracle and the four-corners scan side by side (source: dsa/samples-py/src/Ch37/manhattan.py, lines 39-65)

```lua
local function to_uv(p)
  return {p[1] + p[2], p[1] - p[2]}
end

local function l1(a, b)
  return math.abs(a[1] - b[1]) + math.abs(a[2] - b[2])
end

local function cheb(a, b)
  return math.max(math.abs(a[1] - b[1]), math.abs(a[2] - b[2]))
end

-- the L1 diameter by the four extreme images
local function l1_diameter(pts)
  local uv = {}
  for i, p in ipairs(pts) do uv[i] = to_uv(p) end
  local minu, maxu, minv, maxv = 1, 1, 1, 1
  for i = 2, #uv do
    if uv[i][1] < uv[minu][1] then minu = i end
    if uv[i][1] > uv[maxu][1] then maxu = i end
    if uv[i][2] < uv[minv][2] then minv = i end
    if uv[i][2] > uv[maxv][2] then maxv = i end
  end
  local best, pair = 0, nil
  for _, pairidx in ipairs({{minu, maxu}, {minv, maxv}, {minu, maxv},
                            {minv, maxu}}) do
    local d = l1(pts[pairidx[1]], pts[pairidx[2]])
    if d > best then best, pair = d, {pts[pairidx[1]], pts[pairidx[2]]} end
  end
  return best, pair
end
```

listing 1327: lua, to-uv and the four-extremes candidates (source: dsa/samples-lua/ch37_manhattan.lua, lines 8-38)

The fixture points (0,0), (1,5), (4,2), (6,6) pin an L1 diameter of 12 at the pair (0,0)-(6,6), and the chebyshev distance of the images is also 12. The property family draws 20 seeded lcg points and asserts the equality pairwise, every L1 distance in xy equal to the chebyshev distance in uv, plus the four-extremes answer against the full scan. The edges: a single point has diameter 0, and every point on the x = y line maps to v = 0, pinned as a family.

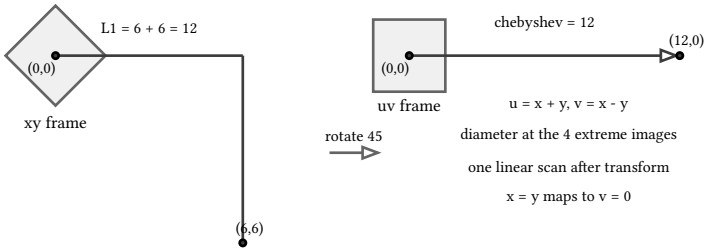


figure 332: the L1 ball, a diamond, rotating into the chebyshev ball, a square, with one distance pair drawn in both frames

37.12. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's sample files per language, the c# family files carrying their whole facet groups:

language	build sloc	container dependency	boundary note
c	1601	static arrays, one file per facet	qsort with composite comparators, memmove deque pops, all pins rounded 6 and asserted at 1e-6

language	build sloc	container dependency	boundary note
go	908	math and sort only	errors returned from halfplane, geo.go holds the shared point helpers
java	1543	jdk 27 stdlib	r6 hand-rolled by sign, Math.round is half-up where c rounds half-away-from-zero, angle orders as insertion sorts, segunion's boxed Arrays.sort the one stdlib sort
c#	662	records, tuples, linq	three family files, geometry2, calipers, voronoi, nullable tangents carry the sentinel
javascript	637	node stdlib	all trig in number with the safety comment, two per-chapter test files at 101 asserts
python	956	stdlib math	check harness with the ok-n print, brute oracles beside every construction
lua	1249	lib.lua harness	1-based tables throughout, string keys for delaunay segments, negative zero normalized in r6

sources: cp-algorithms, "Circle-Line Intersection", cp-algorithms.com/geometry/circle-line-intersection.html, "Circle-Circle Intersection", cp-algorithms.com/geometry/circle-circle-intersection.html, "Finding common tangents to two circles", cp-algorithms.com/geometry/tangents-to-two-circles.html, "Length of the union of segments", cp-algorithms.com/geometry/length-of-segments-union.html, "Check if point belongs to the convex polygon in O(log N)", cp-algorithms.com/geometry/point-in-convex-polygon.html, "Minkowski sum of convex polygons", cp-algorithms.com/geometry/minkowski.html, "Convex hull construction", cp-algorithms.com/geometry/convex-hull.html, "Half-plane intersection - S&I Algorithm in O(N log N)", cp-algorithms.com/geometry/halfplane-intersection.html, "Minimum Enclosing Circle", cp-algorithms.com/geometry/enclosing-circle.html, "Delaunay triangulation and Voronoi diagram", cp-algorithms.com/geometry/delaunay.html, "Search for a pair of intersecting segments", cp-algorithms.com/geometry/intersecting_segments.html, "Finding faces of a planar graph", cp-algorithms.com/geometry/planar.html, "Point location in O(log n)", cp-algorithms.com/geometry/point-location.html, and "Manhattan Distance", cp-algorithms.com/geometry/manhattan-distance.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Rotating calipers and the bentley-ottmann sweep have no dedicated cp-algorithms article: the calipers section cites "Convex hull construction" as the base plus clrs third edition chapter 33 for the antipodal-pair argument, and the ottmann section cites "Search for a pair of intersecting segments" for the neighbor-ordering groundwork plus the same clrs chapter for the full sweep. Application sources: icpc 2018 problem G (book 10, chapter 9) for the half-plane, voronoi, and planar-faces sections, and icpc 2017 problem A (book 10, chapter 8) cross-named as the calipers kin. Sample behavior verified by the seven suite gates scoped to chapter 37: c 10 files and 113 checks, go 31 test functions, java 10 files and 113 checks under run-java-samples, c# 25 facts, javascript 35 tests and 101 asserts across the two per-chapter files, python 10 files and 108 asserts, lua 38 checks, zero skipped.

38. network flows ii

Chapter 12 built the residual graph and ran edmonds-karp over it, one shortest augmenting path at a time. This chapter keeps the residual graph and changes the search. Dinic stratifies it by levels and pushes blocking flows, push-relabel abandons paths entirely and ships excess downhill by height labels, and both answer in numbers the fixture families pin. Then the family around the flow idea: minimum-cost flow by successive shortest paths, lower bounds through a super-source transform, the stoer-wagner global cut, and the matching trio of kuhn, hungarian, and hall's theorem, each with its own contest face. The icpc applications run through 2017 problems C and J, 2018 problem C, 2022 problem X, and 2025 problems B and F.

38.1. dinic's algorithm

Same residual graph as the edmonds-karp baseline of book 9, ch 12 (practical data structures and algorithms, minimum spanning trees and flows), smarter search. A bfs from the source labels every vertex with its level, the shortest-path distance over arcs that still carry capacity. The dfs then pushes flow only along level-plus-one arcs, so every augmenting path is shortest, and it keeps going until the level graph is blocked, no path remains. The per-vertex iteration pointer is what makes a phase cheap: when a recursive push through an arc fails, the arc is abandoned for the rest of the whole phase, never retried, so the blocking flow costs one pass of pointer advances. Phase count is O(V) in general and O(sqrt(E)) on unit-capacity graphs, for the O(V^2 E) total. Arcs live in one flat list with paired residuals, forward at even indices, the backward twin at the odd neighbor, so xor with 1 always finds the reverse arc.

The dry run: the fixture is the pinch network, source 0, sink 6, its arcs and answers pinned word for word by all seven suites, and the walk replays the phases on those capacities.

1. The first bfs levels the graph: 1 and 2 land level 1, 3 and 4 land level 2, the sink 6 lands level 3, and every push must climb exactly one level per arc.
2. The first path runs 0, 1, 3, 6 at bottleneck min(10, 4, 10) = 4 and fills the 4-wide 1 to 3 arc exactly.
3. The pointer at 1 steps past its spent arc, so the second path turns over 0, 1, 4, 6 at min(6, 8, 3) = 3: the 3-wide pinch saturates.
4. Vertex 4 now has no arc left into level 3, its pointer runs off the end of the list, and the third path crosses 0, 2, 3, 6 at min(10, 9, 6) = 6, closing 3 to 6 at 4 + 6 = 10 of 10.
5. The blocking flow totals 4 + 3 + 6 = 13 with both sink arcs spent, and the second bfs strands the sink at level minus 1: the run returns 13.
6. The residual walk from 0 keeps exactly {0, 1, 2, 3, 4}, the cut side of value 13, and the parallel-arc fixture folds the same machine to 5 with its sinkless arc carrying nothing.

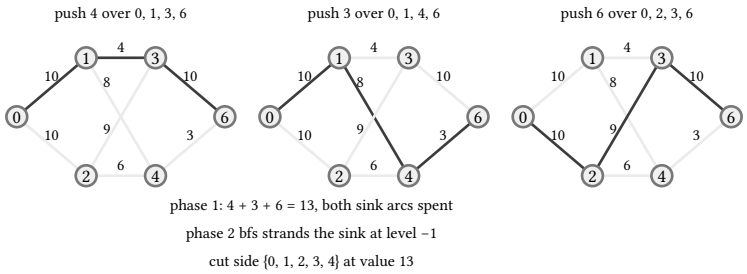


figure 333: the pinch network three times, one augmenting path bold per frame, the pinch and the sink arc saturating in sequence

The 13 with the cut side {0, 1, 2, 3, 4} is the seven-suite pin, and the listings below run these phases in seven languages.

```c
static int bfs(int s, int t) {
  memset(lvl, -1, sizeof(int) * (size_t)nv);
  int qh = 0, qt = 0;
  lvl[s] = 0;
  queue[qt++] = s;
  while (qh < qt) {
    int u = queue[qh++];
    for (int e = head[u]; e >= 0; e = nxt[e])
      if (cap[e] > 0 && lvl[to[e]] < 0) {
        lvl[to[e]] = lvl[u] + 1;
        queue[qt++] = to[e];
      }
  }
  return lvl[t] >= 0;
}

static long long dfs(int u, int t, long long f) {
  if (u == t)
    return f;
  for (int *e = &it[u]; *e >= 0; *e = nxt[*e]) {
    int v = to[*e];
    if (cap[*e] > 0 && lvl[v] == lvl[u] + 1) {
      long long d = dfs(v, t, f < cap[*e] ? f : cap[*e]);
      if (d > 0) {
        cap[*e] -= d;
        cap[*e ^ 1] += d;
        return d;
      }
    }
  }
  return 0;
}

static long long maxflow(int s, int t) {
  long long flow = 0;
  while (bfs(s, t)) {
    for (int i = 0; i < nv; i++)
      it[i] = head[i];
    for (;;) {
      long long f = dfs(s, t, INF);
      if (!f)
        break;
      flow += f;
    }
  }
  return flow;
}
```

listing 1328: c, the level bfs, the blocking dfs with the pointer, the phase loop (source: dsa/samples-c/src/Ch38/dinic.c, lines 47-93)

```
func (d *Dinic) MaxFlow(s, t int) int64 {
  const inf = int64(1) << 60
  flow := int64(0)
  for {
    lvl := make([]int, d.n)
    for i := range lvl {
      lvl[i] = -1
    }
    lvl[s] = 0
    queue := []int{s}
    for len(queue) > 0 {
      u := queue[0]
      queue = queue[1:]
      for _, e := range d.adj[u] {
        if d.cap[e] > 0 && lvl[d.to[e]] < 0 {
          lvl[d.to[e]] = lvl[u] + 1
          queue = append(queue, d.to[e])
        }
      }
    }
    if lvl[t] < 0 {
      return flow
    }
    it := make([]int, d.n)
    var dfs func(u int, f int64) int64
    dfs = func(u int, f int64) int64 {
      if u == t {
        return f
      }
      for it[u] < len(d.adj[u]) {
        e := d.adj[u][it[u]]
        v := d.to[e]
        if d.cap[e] > 0 && lvl[v] == lvl[u]+1 {
          pushed := dfs(v, minI64(f, d.cap[e]))
          if pushed > 0 {
            d.cap[e] -= pushed
            d.cap[e^1] += pushed
            return pushed
          }
        }
        it[u]++
      }
      return 0
    }
    for {
      f := dfs(s, inf)
      if f == 0 {
        break
      }
      flow += f
    }
  }
}
```

listing 1329: go, levels by bfs, blocking flow by recursion over the level graph (source: dsa/samples-go/ch38/dinic.go, lines 30-82)

```
  static boolean bfs(int s, int t) {
    java.util.Arrays.fill(lvl, 0, nv, -1);
    int qh = 0, qt = 0;
    lvl[s] = 0;
    queue[qt++] = s;
    while (qh < qt) {
      int u = queue[qh++];
      for (int e = head[u]; e ≥ 0; e = nxt[e])
        if (cap[e] > 0 && lvl[to[e]] < 0) {
          lvl[to[e]] = lvl[u] + 1;
          queue[qt++] = to[e];
        }
    }
    return lvl[t] ≥ 0;
  }

  static long dfs(int u, int t, long f) {
    if (u == t)
      return f;
    for (; it[u] ≥ 0; it[u] = nxt[it[u]]) {
      int e = it[u];
      int v = to[e];
      if (cap[e] > 0 && lvl[v] == lvl[u] + 1) {
        long d = dfs(v, t, Math.min(f, cap[e]));
        if (d > 0) {
          cap[e] -= d;
          cap[e ^ 1] += d;
          return d;
        }
      }
    }
    return 0;
  }

  static long maxflow(int s, int t) {
    long flow = 0;
    while (bfs(s, t)) {
      for (int i = 0; i < nv; i++)
        it[i] = head[i];
      for (;;) {
        long f = dfs(s, t, INF);
        if (f == 0)
          break;
        flow += f;
      }
    }
    return flow;
  }
```

listing 1330: java, the level bfs, the blocking dfs with the pointer, the phase loop (source: dsa/samples-java/src/Ch38/Dinic.java, lines 45-93)

```
    public long MaxFlow(int s, int t)
    {
        long flow = 0;
        while (true)
        {
            var level = new int[_n];
            Array.Fill(level, -1);
            level[s] = 0;
            var queue = new Queue<int>();
            queue.Enqueue(s);
            while (queue.Count > 0)
            {
                int u = queue.Dequeue();
                foreach (int e in _graph[u])
                    if (_cap[e] > 0 && level[_to[e]] < 0)
                    {
                        level[_to[e]] = level[u] + 1;
                        queue.Enqueue(_to[e]);
                    }
            }
            if (level[t] < 0)
                return flow;
            var iter = new int[_n];

            long Dfs(int u, long limit)
            {
                if (u == t)
                    return limit;
                while (iter[u] < _graph[u].Count)
                {
                    int e = _graph[u][iter[u]];
                    int v = _to[e];
                    if (_cap[e] > 0 && level[v] == level[u] + 1)
                    {
                        long d = Dfs(v, Math.Min(limit, _cap[e]));
                        if (d > 0)
                        {
                            _cap[e] -= d;
                            _cap[e ^ 1] += d;
                            return d;
                        }
                    }
                    iter[u]++; // every arc is abandoned once per phase
                }
                return 0;
            }

            long pushed;
            while ((pushed = Dfs(s, long.MaxValue / 2)) > 0)
                flow += pushed;
        }
    }
```

listing 1331: c#, the same phase loop, the nested dfs abandons each dead arc once (source: dsa/samples/src/Ch38/Flow2.cs, lines 31-82)

```
  #dfs(level, iter, u, t, f) {
    if (u === t) return f;
    const g = this.#g;
    while (iter[u] < g[u].length) {
      const e = g[u][iter[u]];
      const v = this.#to[e];
      if (this.#cap[e] > 0 && level[v] === level[u] + 1) {
        const d = this.#dfs(level, iter, v, t, Math.min(f, this.#cap[e]));
        if (d > 0) {
          this.#cap[e] -= d;
          this.#cap[e ^ 1] += d;
          return d;
        }
      }
      iter[u] += 1; // abandon this edge for the whole phase
    }
    return 0;
  }

  maxflow(s, t) {
    let flow = 0;
    for (;;) {
      const level = new Array(this.#n).fill(-1);
      level[s] = 0;
      const q = [s];
      while (q.length > 0) {
        const u = q.shift();
        for (const e of this.#g[u]) {
          if (this.#cap[e] > 0 && level[this.#to[e]] < 0) {
            level[this.#to[e]] = level[u] + 1;
            q.push(this.#to[e]);
          }
        }
      }
      if (level[t] < 0) return flow;
      const iter = new Array(this.#n).fill(0);
      for (;;) {
        const f = this.#dfs(level, iter, s, t, INF);
        if (f === 0) break;
        flow += f;
      }
    }
  }
```

listing 1332: javascript, the private dfs and the phase driver, exact on number below 2^53 (source: dsa/samples-js/src/ch38-dinic.mjs, lines 41-83)

```
    def maxflow(self, s, t):
        flow = 0
        while True:
            level = [-1] * self.n  # bfs builds the level graph
            level[s] = 0
            q = deque([s])
            while q:
                u = q.popleft()
                for e in self.adj[u]:
                    if self.cap[e] > 0 and level[self.to[e]] < 0:
                        level[self.to[e]] = level[u] + 1
                        q.append(self.to[e])
            if level[t] < 0:
                return flow
            it = [0] * self.n  # the per-vertex iteration pointer

            def dfs(u, limit):  # blocking flow by level
                if u == t:
                    return limit
                while it[u] < len(self.adj[u]):
                    e = self.adj[u][it[u]]
                    v = self.to[e]
                    if self.cap[e] > 0 and level[v] == level[u] + 1:
                        got = dfs(v, min(limit, self.cap[e]))
                        if got > 0:
                            self.cap[e] -= got
                            self.cap[e ^ 1] += got
                            return got
                    it[u] += 1  # abandon this arc for the phase
                return 0

            while True:
                pushed = dfs(s, float("inf"))
                if pushed == 0:
                    break
                flow += pushed

    def mincut_side(self, s):
        seen = [False] * self.n
        seen[s] = True
        q = deque([s])
        while q:
            u = q.popleft()
            for e in self.adj[u]:
                if self.cap[e] > 0 and not seen[self.to[e]]:
                    seen[self.to[e]] = True
                    q.append(self.to[e])
        return [i for i in range(self.n) if seen[i]]
```

listing 1333: python, the nested dfs with the pointer, the cut side walk below (source: dsa/samples-py/src/Ch38/dinic.py, lines 32-79)

```
function Dinic:maxflow(s, t)
  local flow = 0
  while true do
    local lvl = {}
    for i = 1, self.n do lvl[i] = -1 end
    lvl[s + 1] = 0
    local q = {s + 1}
    local head = 1
    while head ≤ #q do
      local u = q[head]
      head = head + 1
      for _, e in ipairs(self.g[u] or {}) do
        if self.cap[e + 1] > 0 and lvl[self.to[e + 1] + 1] < 0 then
          lvl[self.to[e + 1] + 1] = lvl[u] + 1
          q[#q + 1] = self.to[e + 1] + 1
        end
      end
    end
    if lvl[t + 1] < 0 then return flow end
    local it = {}
    for i = 1, self.n do it[i] = 1 end
    local function dfs(u, f) -- blocking flow, iterative-friendly size
      if u == t + 1 then return f end
      local adj = self.g[u] or {}
      while it[u] ≤ #adj do
        local e = adj[it[u]]
        local v = self.to[e + 1]
        if self.cap[e + 1] > 0 and lvl[v + 1] == lvl[u] + 1 then
          local d = dfs(v + 1, math.min(f, self.cap[e + 1]))
          if d > 0 then
            self.cap[e + 1] = self.cap[e + 1] - d
            self.cap[(e ~ 1) + 1] = self.cap[(e ~ 1) + 1] + d
            return d
          end
        end
        it[u] = it[u] + 1
      end
      return 0
    end
    while true do
      local f = dfs(s + 1, math.maxinteger // 2)
      if f == 0 then break end
      flow = flow + f
    end
  end
end
```

listing 1334: lua, the same phases, 1-based tables over the 0-based public face (source: dsa/samples-lua/ch38_dinic.lua, lines 28-73)

The min cut falls out of the last residual graph: the set of vertices still reachable from the source is exactly the source side of a minimum cut, and every solver here reports it.

The fixture families pin identical numbers in all seven languages. The pinch network, source 0, sink 6, arcs (0,1,10), (0,2,10), (1,3,4), (1,4,8), (2,3,9), (2,4,6), (3,6,10), (4,6,3), flows 13 with the cut side {0,1,2,3,4}: the arc 4 to 6 of capacity 3 is the pinch. Parallel arcs (0,1,3) and (0,1,4) against a 5 bottleneck flow 5, a dead arc to a sinkless vertex carrying nothing. The 2017 problem J mini network in water-equivalent units answers all three of its questions from the one solver: a super source feeding stations 1 and 2 at 100 each, pipes 1-3 (4), 2-3 (4), 1-4 (2), 3-4 (2) installed as paired arcs both ways, sink 3, gives Z = 10, and rerunning without the super source gives Fmax = 6 from station

1 alone and Wmax = 4 from station 2. The seeded family draws 40 arcs over 20 plus 20 nodes from the pinned lcg, and the unit-capacity flow of 18 equals the kuhn matching size of the same graph, the cross-check the kuhn section below leans on. Java's copy of that generator keeps the lcg state in a signed int, so the draw routes through Integer.remainderUnsigned, a plain % goes negative once the state crosses 2^31 and the arc list drifts.

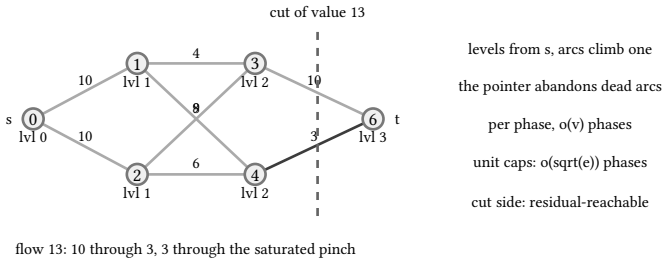


figure 334: the pinch network with the saturated 4 to 6 arc, level numbers on vertices, the minimum cut drawn between the two sides

The application is icpc world finals 2017 problem J (book 10, chapter 8), son of pipe stream, whose water-equivalent network is answered by four dinic runs over exactly this residual machinery, and the F3 fixture above is its crafted sample.

38.2. push-relabel

Drop the augmenting path entirely. A preflow violates conservation on purpose: the source arcs are saturated at the start, every vertex holds an excess, and the algorithm moves excess around instead of building paths. Each vertex carries an integer height label, the source at n, and an arc is admissible when it carries capacity and drops exactly one height level. The discharge loop pushes excess along admissible arcs, and a vertex that cannot push anywhere is relabeled to one above its lowest reachable neighbor, so the excess always has a way out. The fifo queue keeps the discharges moving, the sink absorbs everything that arrives, and the excess at the sink is the flow value.

The dry run: the fixture is the parallel network, twin arcs (0,1,3) and (0,1,4) folding onto one dense cell of 7 against the 5 bottleneck, pinned at 5 by all seven suites; the edge family splits after it, python carrying the unreachable sink at 0 and the single arc of capacity 7.

1. Saturation pushes the whole folded 7 onto vertex 1 and leaves the source arc at 0, a preflow on purpose: vertex 1 holds excess 7.
2. Vertex 1 starts at height 0 among neighbors at 0, 0, and 4, stuck, so it relabels to 1 + min(4, 0, 0) = 1.
3. Admissible now: the drop to the sink carries min(7, 5) = 5 and the sink absorbs it, then the drop to the sinkless vertex 3 takes the remaining min(2, 9) = 2.
4. Vertex 3 has one exit, back to 1: it lifts to 1 + 1 = 2 and returns the 2, then 1 lifts to 1 + min(4, 2) = 3 and sends them back, then 3 lifts to 1 + 3 = 4 and returns them again.
5. Vertex 1 finally lifts to 1 + 4 = 5, exactly one above the source, and the last 2 drains home through the residual: conservation restored everywhere but the sink.
6. The sink's excess reads 5, the pinned value, and the same machine answers 13 on the pinch network and 10 on the 2017 problem J graph.

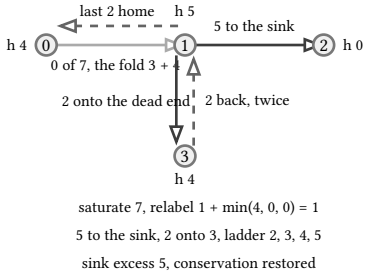


figure 335: the parallel network under push-relabel, the source saturated, the sink filled, the stranded 2 climbing the height ladder home

The 5 with the stranded units walked home is the seven-suite pin, and the listings below discharge the same fixtures in seven languages.

```
  for (int v = 0; v < nv; v++)
    if (cap[s][v] > 0) {
      long long f = cap[s][v];
      cap[s][v] -= f;
      cap[v][s] += f;
      excess[v] += f;
      if (v != t && !inq[v]) {
        queue[qt] = v;
        inq[v] = 1;
        qt = (qt + 1) % MAXV;
      }
    }
  while (qh != qt) {
    int u = queue[qh];
    inq[u] = 0;
    qh = (qh + 1) % MAXV;
    if (u == s || u == t)
      continue;
    int pushed = 1;
    while (excess[u] > 0 && pushed) {
      pushed = 0;
      for (int v = 0; v < nv; v++)
        if (excess[u] > 0 && cap[u][v] > 0 && height[u] == height[v] + 1) {
          long long f = excess[u] < cap[u][v] ? excess[u] : cap[u][v];
          cap[u][v] -= f;
          cap[v][u] += f;
          excess[u] -= f;
          excess[v] += f;
          if (v != s && v != t && !inq[v]) {
            queue[qt] = v;
            inq[v] = 1;
            qt = (qt + 1) % MAXV;
          }
          pushed = 1;
        }
      if (excess[u] > 0 && !pushed) {
        long long minh = -1;
        for (int v = 0; v < nv; v++)
          if (cap[u][v] > 0 && (minh < 0 || height[v] < minh))
            minh = height[v];
        height[u] = (int)minh + 1;
      }
    }
    if (excess[u] > 0) { // still stuck: requeue
      queue[qt] = u;
      inq[u] = 1;
      qt = (qt + 1) % MAXV;
    }
  }
  return excess[t];
}
```

listing 1335: c, saturate the source, the fifo discharge loop, relabel to 1 + the min neighbor height (source: dsa/samples-c/src/Ch38/pushrelabel.c, lines 46-96)

```
  for v := 0; v < n; v++ {
    if cap[s][v] > 0 {
      f := cap[s][v]
      cap[s][v] -= f
      cap[v][s] += f
      ex[v] += f
      if v != t {
        queue = append(queue, v)
      }
    }
  }
  for len(queue) > 0 {
    u := queue[0]
    queue = queue[1:]
    if u == s || u == t {
      continue
    }
    pushed := true
    for ex[u] > 0 && pushed {
      pushed = false
      for v := 0; v < n; v++ {
        if ex[u] > 0 && cap[u][v] > 0 && h[u] == h[v]+1 {
          f := minI64(ex[u], cap[u][v])
          cap[u][v] -= f
          cap[v][u] += f
          ex[u] -= f
          ex[v] += f
          if v != s && v != t && ex[v] == f {
            queue = append(queue, v)
          }
          pushed = true
        }
      }
      if ex[u] > 0 && !pushed {
        minH := 1 << 30
        for v := 0; v < n; v++ {
          if cap[u][v] > 0 && h[v] < minH {
            minH = h[v]
          }
        }
        h[u] = minH + 1
      }
    }
    if ex[u] > 0 {
      queue = append(queue, u)
    }
  }
  return ex[t]
}
```

listing 1336: go, saturate, discharge, relabel, the dense capacity matrix folding parallel arcs (source: dsa/samples-go/ch38/pushrelabel.go, lines 22-70)

```
    height[s] = nv;
    int qh = 0, qt = 0;
    for (int v = 0; v < nv; v++)
      if (cap[s][v] > 0) {
        long f = cap[s][v];
        cap[s][v] -= f;
        cap[v][s] += f;
        excess[v] += f;
        if (v != t && !inq[v]) {
          queue[qt] = v;
          inq[v] = true;
          qt = (qt + 1) % MAXV;
        }
      }
    while (qh != qt) {
      int u = queue[qh];
      inq[u] = false;
      qh = (qh + 1) % MAXV;
      if (u == s || u == t)
        continue;
      boolean pushed = true;
      while (excess[u] > 0 && pushed) {
        pushed = false;
        for (int v = 0; v < nv; v++)
          if (excess[u] > 0 && cap[u][v] > 0 && height[u] == height[v] + 1) {
            long f = Math.min(excess[u], cap[u][v]);
            cap[u][v] -= f;
            cap[v][u] += f;
            excess[u] -= f;
            excess[v] += f;
            if (v != s && v != t && !inq[v]) {
              queue[qt] = v;
              inq[v] = true;
              qt = (qt + 1) % MAXV;
            }
            pushed = true;
          }
        if (excess[u] > 0 && !pushed) {
          int minh = -1;
          for (int v = 0; v < nv; v++)
            if (cap[u][v] > 0 && (minh < 0 || height[v] < minh))
              minh = height[v];
          height[u] = minh + 1;
        }
      }
      if (excess[u] > 0) { // still stuck: requeue
        queue[qt] = u;
        inq[u] = true;
        qt = (qt + 1) % MAXV;
      }
    }
    return excess[t];
  }
```

listing 1337: java, saturate the source, the fifo discharge loop, relabel to 1 + the min neighbor height (source: dsa/samples-java/src/Ch38/Pushrelabel.java, lines 44-96)

```
        while (queue.Count > 0)
        {
            int u = queue.Dequeue();
            if (u == s || u == t)
                continue;
            bool pushed = true;
            while (excess[u] > 0 && pushed)
            {
                pushed = false;
                for (int v = 0; v < n; v++)
                    if (excess[u] > 0 && cap[u][v] > 0 && height[u] == height[v] + 1)
                    {
                        long f = Math.Min(excess[u], cap[u][v]);
                        cap[u][v] -= f;
                        cap[v][u] += f;
                        excess[u] -= f;
                        excess[v] += f;
                        if (v != s && v != t && excess[v] == f)
                            queue.Enqueue(v); // first-time excess enqueues
                        pushed = true;
                    }
                if (excess[u] > 0 && !pushed)
                    height[u] = 1 + Enumerable.Range(0, n)
                        .Where(v => cap[u][v] > 0)
                        .Min(v => height[v]); // relabel to one above the min neighbor
            }
            if (excess[u] > 0)
                queue.Enqueue(u);
        }
        return excess[t];
```

listing 1338: c#, the same discharge loop, the relabel as a linq minimum (source: dsa/samples/src/Ch38/Flow2.cs, lines 143-172)

```
  for (let v = 0; v < n; v++) {
    if (cap[s][v] > 0) {
      const f = cap[s][v];
      cap[s][v] -= f;
      cap[v][s] += f;
      ex[v] += f;
      if (v !== t) q.push(v);
    }
  }
  while (q.length > 0) {
    const u = q.shift();
    if (u === s || u === t) continue;
    let pushed = true;
    while (ex[u] > 0 && pushed) {
      pushed = false;
      for (let v = 0; v < n; v++) {
        if (ex[u] > 0 && cap[u][v] > 0 && h[u] === h[v] + 1) {
          const f = Math.min(ex[u], cap[u][v]);
          cap[u][v] -= f;
          cap[v][u] += f;
          ex[u] -= f;
          ex[v] += f;
          if (v !== s && v !== t && ex[v] === f) q.push(v);
          pushed = true;
        }
      }
      if (ex[u] > 0 && !pushed) {
        // the discharge loop: relabel to 1 + min reachable height
        let minH = Number.MAX_SAFE_INTEGER;
        for (let v = 0; v < n; v++) {
          if (cap[u][v] > 0) minH = Math.min(minH, h[v]);
        }
        h[u] = 1 + minH;
      }
    }
    if (ex[u] > 0) q.push(u);
  }
  return ex[t];
```

listing 1339: javascript, the same machine, heights integer, excess exact on number (source: dsa/samples-js/src/ch38-pushrelabel.mjs, lines 17-54)

```python
def push_relabel(count, arcs, s, t):
    cap = [[0] * count for _ in range(count)]
    for u, v, c in arcs:
        cap[u][v] += c
    if s == t:
        return 0
    height = [0] * count
    height[s] = count
    excess = [0] * count
    q = deque()
    for v in range(count):
        if cap[s][v] > 0:  # saturate every source arc
            f = cap[s][v]
            cap[s][v] -= f
            cap[v][s] += f
            excess[v] += f
            if v != t:
                q.append(v)
    while q:
        u = q.popleft()
        if u == s or u == t:
            continue
        pushed = True
        while excess[u] > 0 and pushed:
            pushed = False
            for v in range(count):  # admissible: downhill by exactly 1
                if excess[u] > 0 and cap[u][v] > 0 and height[u] == height[v] + 1:
                    f = min(excess[u], cap[u][v])
                    cap[u][v] -= f
                    cap[v][u] += f
                    excess[u] -= f
                    excess[v] += f
                    if v not in (s, t) and excess[v] == f:
                        q.append(v)
                    pushed = True
            if excess[u] > 0 and not pushed:  # stuck: lift the vertex
                reach = [height[v] for v in range(count) if cap[u][v] > 0]
                height[u] = 1 + min(reach)  # lift past the neighborhood
        if excess[u] > 0:
            q.append(u)
    return excess[t]
```

listing 1340: python, the whole solver, restart-proof guard at s = t (source: dsa/samples-py/src/Ch38/pushrelabel.py, lines 17-57)

```
  local q = {}
  for v = 1, n do
    if cap[s][v] > 0 then
      local f = cap[s][v]
      cap[s][v] = cap[s][v] - f
      cap[v][s] = cap[v][s] + f
      ex[v] = ex[v] + f
      if v ~= t then q[#q + 1] = v end
    end
  end
  local head = 1
  while head ≤ #q do
    local u = q[head]
    head = head + 1
    if u ~= s and u ~= t then
      local pushed = true
      while ex[u] > 0 and pushed do
        pushed = false
        for v = 1, n do
          if ex[u] > 0 and cap[u][v] > 0 and h[u] == h[v] + 1 then
            local f = math.min(ex[u], cap[u][v])
            cap[u][v] = cap[u][v] - f
            cap[v][u] = cap[v][u] + f
            ex[u] = ex[u] - f
            ex[v] = ex[v] + f
            if v ~= s and v ~= t and ex[v] == f then q[#q + 1] = v end
            pushed = true
          end
        end
        if ex[u] > 0 and not pushed then
          local minh = math.maxinteger
          for v = 1, n do
            if cap[u][v] > 0 then minh = math.min(minh, h[v]) end
          end
          h[u] = 1 + minh -- relabel to just above the cheapest neighbor
        end
      end
    end
    if ex[u] > 0 and u ~= s and u ~= t then
      q[#q + 1] = u -- still has excess, come back later
    end
  end
  return ex[t]
```

listing 1341: lua, discharge and relabel, still-stuck vertices requeue (source: dsa/samples-lua/ch38_pushrelabel.lua, lines 26-68)

The O(V^3) bound comes from the height labels: each relabel lifts a vertex, heights stay bounded, and the total lifting is finite. The improved variant with the gap heuristic is named in the literature and stays prose here, the fixture scale never opens a gap. Capacities ride a dense matrix, so parallel arcs fold into one cell, which is flow-safe and keeps the fixture graphs tiny.

Every fixture asserts equality with the dinic value on the same graph: the pinch network 13, the parallel network 5, the 2017 problem J network 10. The edge family closes it: s equals t returns 0, a vertex with no entering arc simply never fills, an unreachable sink drains 0, and a single arc of capacity 7 pushes all 7 through.

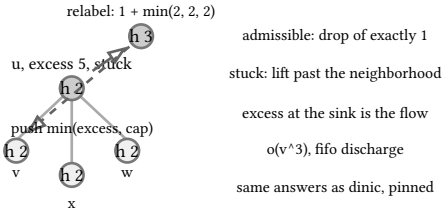


figure 336: one relabel step, the vertex with excess stuck at height 2 among neighbors at 2, lifted to 3 with the push arrow

The application is general technique, the second max-flow engine in the toolbox, and the choice when the level graph fights back.

38.3. minimum-cost maximum flow

Augment the cheapest path first and the maximum flow arrives at minimum cost. Each round finds a shortest path by cost over the residual graph, pushes the bottleneck along the parent-arc chain, and accumulates flow times path cost. The shortest-path engine is spfa, bellman-ford with a queue: negative arcs are fine, which matters because every residual arc negates the cost of its forward twin. On these fixtures the residuals never close a negative cycle, the condition spfa needs to terminate. The potentials form, johnson distances reused round to round so dijkstra replaces spfa, is the production upgrade and stays prose.

The dry run: the fixture is the layered network, arcs (0,1,3,1), (0,2,2,1), (1,3,3,1), (2,3,2,2), (3,4,4,1), (1,4,1,4), pinned at (5, 19) by all seven suites.

1. Round 1 prices the route 0, 1, 3, 4 at 1 + 1 + 1 = 3, the bottleneck min(3, 3, 4) = 3 fills 0 to 1 and 1 to 3 exactly, and the round books 3 × 3 = 9.
2. Round 2 rides 0, 2, 3, 4 at 1 + 2 + 1 = 4, where the 3 to 4 arc is down to 4 - 3 = 1, so the push caps at 1 and the books read 9 + 4 = 13.
3. Round 3 finds 3 to 4 spent and takes the residual 3 to 1 at cost 0 - 1 = −1: the route 0, 2, 3, 1, 4 totals 1 + 2 - 1 + 4 = 6, pushes 1, and 13 + 6 = 19.
4. Round 4 sees both source arcs at 0, spfa prices the sink at infinity, and the run closes at (5, 19), the fifth unit forced through the cost-4 arc.
5. Drop the 1 to 4 arc and only rounds 1 and 2 exist: (4, 13). The negative family books 2 × (-3 + 1) = −4 then 2 × 5 = 10 for (4, 6), and the 2018 problem C tree pushes 2, 1, 1 along routes of 3, 4, and 4: 6 + 4 + 4 = 14.

round	route	bottleneck	route cost	cost so far
1	0-1-3-4	3	3	9
2	0-2-3-4	1	4	13
3	0-2-3-1-4	1	6	19

The (5, 19) with its forced expensive unit is the seven-suite pin, and the listings below run these rounds in seven languages.

```
static void mcmf(int s, int t, long long *flow_out, long long *cost_out) {
  long long flow = 0, cst = 0;
  for (;;) {
    for (int i = 0; i < nv; i++) {
      dist[i] = INF;
      pe[i] = -1;
      inq[i] = 0;
    }
    dist[s] = 0;
    int qh = 0, qt = 0;
    queue[qt] = s;
    inq[s] = 1;
    qt++;
    while (qh != qt) {
      int u = queue[qh];
      inq[u] = 0;
      qh = (qh + 1) % MAXV;
      for (int e = head[u]; e >= 0; e = nxt[e]) {
        int v = to[e];
        if (cap[e] > 0 && dist[u] + cost[e] < dist[v]) {
          dist[v] = dist[u] + cost[e];
          pe[v] = e;
          if (!inq[v]) {
            queue[qt] = v;
            inq[v] = 1;
            qt = (qt + 1) % MAXV;
          }
        }
      }
    }
    if (dist[t] >= INF)
      break;
    long long f = INF;
    for (int v = t; v != s; v = to[pe[v] ^ 1])
      if (cap[pe[v]] < f)
        f = cap[pe[v]];
    for (int v = t; v != s; v = to[pe[v] ^ 1]) {
      cap[pe[v]] -= f;
      cap[pe[v] ^ 1] += f;
    }
    flow += f;
    cst += f * dist[t];
  }
  *flow_out = flow;
  *cost_out = cst;
}
```

listing 1342: c, the spfa loop and the augmentation walk, flow times path cost accumulated (source: dsa/samples-c/src/Ch38/mcmf.c, lines 53-98)

```
func (m *Mcmf) Solve(s, t int) (int64, int64) {
  const inf = int64(1) << 60
  flow, cst := int64(0), int64(0)
  for {
    dist := make([]int64, m.n)
    for i := range dist {
      dist[i] = inf
    }
    inq := make([]bool, m.n)
    pe := make([]int, m.n)
    dist[s] = 0
    queue := []int{s}
    inq[s] = true
    for len(queue) > 0 {
      u := queue[0]
      queue = queue[1:]
      inq[u] = false
      for _, e := range m.adj[u] {
        v := m.to[e]
        if m.cap[e] > 0 && dist[u]+m.cost[e] < dist[v] {
          dist[v] = dist[u] + m.cost[e]
          pe[v] = e
          if !inq[v] {
            inq[v] = true
            queue = append(queue, v)
          }
        }
      }
    }
    if dist[t] ≥ inf {
      return flow, cst
    }
    f := inf
    for v := t; v ≠ s; {
      e := pe[v]
      if m.cap[e] < f {
        f = m.cap[e]
      }
      v = m.to[e^1]
    }
    for v := t; v ≠ s; {
      e := pe[v]
      m.cap[e] -= f
      m.cap[e^1] += f
      v = m.to[e^1]
    }
    flow += f
    cst += f * dist[t]
  }
}
```

listing 1343: go, spfa over the residual, the two walks that push and account (source: dsa/samples-go/ch38/mcmf.go, lines 35-84)

```
  static void mcmf(int s, int t) {
    long flow = 0, cst = 0;
    for (;;) {
      java.util.Arrays.fill(dist, 0, nv, INF);
      java.util.Arrays.fill(pe, 0, nv, -1);
      java.util.Arrays.fill(inq, 0, nv, false);
      dist[s] = 0;
      int qh = 0, qt = 0;
      queue[qt] = s;
      inq[s] = true;
      qt++;
      while (qh != qt) {
        int u = queue[qh];
        inq[u] = false;
        qh = (qh + 1) % MAXV;
        for (int e = head[u]; e >= 0; e = nxt[e]) {
          int v = to[e];
          if (cap[e] > 0 && dist[u] + cost[e] < dist[v]) {
            dist[v] = dist[u] + cost[e];
            pe[v] = e;
            if (!inq[v]) {
              queue[qt] = v;
              inq[v] = true;
              qt = (qt + 1) % MAXV;
            }
          }
        }
      }
      if (dist[t] >= INF)
        break;
      long f = INF;
      for (int v = t; v != s; v = to[pe[v] ^ 1])
        if (cap[pe[v]] < f)
          f = cap[pe[v]];
      for (int v = t; v != s; v = to[pe[v] ^ 1]) {
        cap[pe[v]] -= f;
        cap[pe[v] ^ 1] += f;
      }
      flow += f;
      cst += f * dist[t];
    }
    flowOut = flow;
    costOut = cst;
  }
```

listing 1344: java, the spfa loop and the augmentation walk, flow times path cost accumulated (source: dsa/samples-java/src/Ch38/Mcmf.java, lines 53-96)

```
    public (long Flow, long Cost) Solve(int s, int t)
    {
        const long inf = long.MaxValue / 4;
        long flow = 0;
        long cost = 0;
        while (true)
        {
            var dist = new long[_n];
            Array.Fill(dist, inf);
            var inQueue = new bool[_n];
            var prevEdge = new int[_n];
            Array.Fill(prevEdge, -1);
            dist[s] = 0;
            var queue = new Queue<int>();
            queue.Enqueue(s);
            inQueue[s] = true;
            while (queue.Count > 0)
            {
                int u = queue.Dequeue();
                inQueue[u] = false;
                foreach (int e in _graph[u])
                {
                    int v = _to[e];
                    if (_cap[e] > 0 && dist[u] + _cost[e] < dist[v])
                    {
                        dist[v] = dist[u] + _cost[e];
                        prevEdge[v] = e;
                        if (!inQueue[v])
                        {
                            inQueue[v] = true;
                            queue.Enqueue(v);
                        }
                    }
                }
            }
            if (dist[t] ≥ inf)
                return (flow, cost); // unreachable sink: (0, 0) on a fresh network
            long bottleneck = inf;
            for (int v = t; v ≠ s; v = _to[prevEdge[v] ^ 1])
                bottleneck = Math.Min(bottleneck, _cap[prevEdge[v]]);
            for (int v = t; v ≠ s; v = _to[prevEdge[v] ^ 1])
            {
                _cap[prevEdge[v]] -= bottleneck;
                _cap[prevEdge[v] ^ 1] += bottleneck;
            }
            flow += bottleneck;
            cost += bottleneck * dist[t];
        }
    }
```

listing 1345: c#, the same rounds, the bottleneck pushed along the parent-arc chain (source: dsa/samples/src/Ch38/Costs2.cs, lines 34-82)

```
  let flow = 0;
  let cst = 0;
  for (;;) {
    const dist = new Array(n).fill(INF);
    const inq = new Array(n).fill(false);
    const pe = new Array(n).fill(-1);
    dist[s] = 0;
    const q = [s];
    inq[s] = true;
    while (q.length > 0) {
      const u = q.shift();
      inq[u] = false;
      for (const e of g[u]) {
        const v = to[e];
        if (cap[e] > 0 && dist[u] + cost[e] < dist[v]) {
          dist[v] = dist[u] + cost[e];
          pe[v] = e;
          if (!inq[v]) {
            inq[v] = true;
            q.push(v);
          }
        }
      }
    }
    if (dist[t] ≥ INF) return [flow, cst];
    let f = INF;
    for (let v = t; v =/= s; v = to[pe[v] ^ 1]) f = Math.min(f, cap[pe[v]]);
    for (let v = t; v =/= s; v = to[pe[v] ^ 1]) {
      cap[pe[v]] -= f;
      cap[pe[v] ^ 1] += f;
    }
    flow += f;
    cst += f * dist[t];
  }
}
```

listing 1346: javascript, the round loop, costs exact on number below 2^53 (source: dsa/samples-js/src/ch38-mcmf.mjs, lines 25-59)

```
    flow = spent = 0
    while True:
        dist = [INF] * count  # spfa over the residual graph
        inq = [False] * count
        pred = [-1] * count
        dist[s] = 0
        q = deque([s])
        inq[s] = True
        while q:
            u = q.popleft()
            inq[u] = False
            for e in adj[u]:
                v = to[e]
                if cap[e] > 0 and dist[u] + cost[e] < dist[v]:
                    dist[v] = dist[u] + cost[e]
                    pred[v] = e
                    if not inq[v]:
                        inq[v] = True
                        q.append(v)
        if dist[t] ≥ INF:
            return flow, spent
        bottleneck = INF  # walk the parent-arc chain
        v = t
        while v ≠ s:
            e = pred[v]
            bottleneck = min(bottleneck, cap[e])
            v = to[e ^ 1]
        v = t
        while v ≠ s:
            e = pred[v]
            cap[e] -= bottleneck
            cap[e ^ 1] += bottleneck
            v = to[e ^ 1]
        flow += bottleneck
        spent += bottleneck * dist[t]
```

listing 1347: python, spfa rounds, the residual riding the negative cost back (source: dsa/samples-py/src/Ch38/mcmf.py, lines 31-65)

```
  local flow, cst = 0, 0
  while true do
    local dist, inq, pe = {}, {}, {}
    for i = 1, n do
      dist[i] = INF
      inq[i] = false
    end
    dist[s] = 0
    local q = {s}
    inq[s] = true
    local head = 1
    while head ≤ #q do
      local u = q[head]
      head = head + 1
      inq[u] = false
      for _, e in ipairs(g[u]) do
        local v = to[e + 1]
        if cap[e + 1] > 0 and dist[u] + cost[e + 1] < dist[v] then
          dist[v] = dist[u] + cost[e + 1]
          pe[v] = e
          if not inq[v] then
            inq[v] = true
            q[#q + 1] = v
          end
        end
      end
    end
    if dist[t] ≥ INF then return flow, cst end
    local f = INF
    local v = t
    while v ~= s do
      local e = pe[v]
      f = math.min(f, cap[e + 1])
      v = to[(e ~ 1) + 1] -- the paired arc points back at the tail
    end
    v = t
    while v ~= s do
      local e = pe[v]
      cap[e + 1] = cap[e + 1] - f
      cap[(e ~ 1) + 1] = cap[(e ~ 1) + 1] + f
      v = to[(e ~ 1) + 1]
    end
    flow = flow + f
    cst = cst + f * dist[t]
  end
```

listing 1348: lua, the same rounds in 1-based tables (source: dsa/samples-lua/ch38_mcmf.lua, lines 24-68)

The fixtures pin (flow, cost) pairs. The layered network with arcs (0,1,3,1), (0,2,2,1), (1,3,3,1), (2,3,2,2), (3,4,4,1), (1,4,1,4) gives (5, 19): the 3 to 4 arc of capacity 4 bottlenecks the cheap paths, so the expensive 1 to 4 arc of cost 4 is forced for the last unit. Drop that arc and the answer is (4, 13). The 2018 problem C mini tree, both-way tree arcs of costs 1, 2, 3, 4 by the spec's shape, a super source feeding node 1 with 4 and node 4 with 1, and nodes 2, 3, 5 draining 2, 1, 1, lands (4, 14), the sample answer printed in the problem statement. A negative arc, (0,1,2,-3), (1,2,2,1), (0,2,2,5), rides to (4, 6) through spfa. The unreachable sink returns (0, 0), and a demand exceeding supply is detected by flow falling short, the case the demands transform below handles honestly.

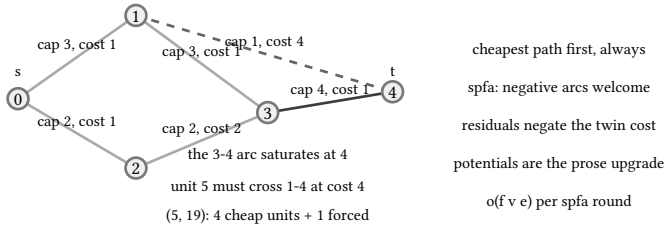


figure 337: the layered cost network with the bottleneck arc shaded and the forced expensive 1 to 4 arc dashed

The application is icpc world finals 2018 problem C (book 10, chapter 9), conquer the world, a min-cost flow on a tree whose finals solution runs a convex dp with heaps. This section pins the direct flow formulation the dp replaces.

38.4. flows with demands

Lower bounds break the plain residual graph because zero flow on an arc is no longer legal. The transform fixes it in one pass: an arc with bounds [l, u] becomes an arc of capacity u - l, and the bound l itself moves into a per-vertex ledger, d(v) as inflow lower bounds minus outflow lower bounds. A vertex with positive d received promises it must honor, so a super source feeds it exactly d. A vertex with negative d promised outflow it cannot guarantee, so it drains to a super sink. A free return arc from t to s at infinity closes the circulation, and the whole system is feasible exactly when every super-source arc saturates, one dinic run on the transformed graph. Recovery reads each arc's true flow off its backward twin plus l.

The dry run: the fixture is the three-arc chain with bounds [1,2], [1,3], [2,2] from source 0 to sink 3, feasible with every arc forced to 2, asserted by all seven suites.

1. The lower bounds strip out of the capacities first: 2 - 1 = 1, 3 - 1 = 2, 2 - 2 = 0, the last arc now fixed at its floor.
2. The ledger d(v) = inflow bounds minus outflow bounds reads d(0) = 0 - 1 = −1, d(1) = 1 - 1 = 0, d(2) = 1 - 2 = −1, d(3) = 2 - 0 = +2.
3. The super source feeds vertex 3 exactly 2, vertices 0 and 2 drain 1 each to the super sink, and the free return 3 to 0 closes the circulation.
4. The two units route S, 3, 0, T and S, 3, 0, 1, 2, T: need 2, got 2, every super-source arc saturated, feasible.
5. Recovery reads the backward twins at 1, 1, and 0 and adds the floors back: 1 + 1 = 2, 1 + 1 = 2, 2 + 0 = 2, the forced flow.
6. Tighten the first arc to [2,2] against a [1,1] middle: every transformed capacity drops to 0, the return arc delivers 1 against a need of 1 + 1 = 2, and the super source refuses to saturate.

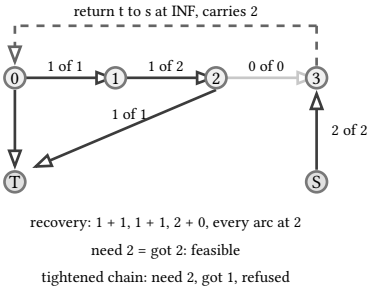


figure 338: the chain after the transform, the super source feeding 2, both units crossing the return arc, recovery adding the floors back

The forced 2 on every arc is the seven-suite pin, and the listings below run this transform in seven languages.

```c
static int with_demands(int vn, const int (*arcs)[4], int ac, int s, int t,
                        int (*out)[3], int *oc) {
  init(vn + 2);
  int S = vn, T = vn + 1;
  long long deficit[MAXV];
  memset(deficit, 0, sizeof(deficit));
  int flow_arc[MAXE / 2], fc = 0;
  for (int i = 0; i < ac; i++) {
    int u = arcs[i][0], v = arcs[i][1], lo = arcs[i][2], hi = arcs[i][3];
    add_arc(u, v, hi - lo);
    arc_lo[fc] = lo;
    flow_arc[fc] = ec - 2; // the forward arc id
    fc++;
    deficit[u] -= lo;
    deficit[v] += lo;
  }
  add_arc(t, s, INF); // the free return
  long long need = 0;
  for (int v = 0; v < vn; v++) {
    if (deficit[v] > 0) {
      add_arc(S, v, deficit[v]);
      need += deficit[v];
    } else if (deficit[v] < 0)
      add_arc(v, T, -deficit[v]);
  }
  long long got = maxflow(S, T);
  if (got != need)
    return 0;
  // every super-source arc saturated: recover the flows
  *oc = 0;
  for (int i = 0; i < fc; i++) {
    out[*oc][0] = arcs[i][0];
    out[*oc][1] = arcs[i][1];
    out[*oc][2] = arc_lo[i] + (int)cap[flow_arc[i] ^ 1];
    (*oc)++;
  }
  return 1;
}
```

listing 1349: c, the transform, the deficit ledger, the feasibility check and flow recovery (source: dsa/samples-c/src/Ch38/demands.c, lines 100-137)

```
  excess := make([]int64, n)
  var flowArcs []struct {
    eidx int
    u, v int
    lo   int64
  }
  for _, a := range arcs {
    add(a.U, a.V, a.Hi-a.Lo)
    excess[a.U] -= a.Lo
    excess[a.V] += a.Lo
    flowArcs = append(flowArcs, struct {
      eidx int
      u, v int
      lo   int64
    }{len(to) - 2, a.U, a.V, a.Lo})
  }
  add(t, s, inf)
  for v := 0; v < n; v++ {
    if excess[v] > 0 {
      add(S, v, excess[v])
    } else if excess[v] < 0 {
      add(v, T, -excess[v])
    }
  }
  // run dinic on the transformed graph by transplanting the arcs
  d := &Dinic{n: big, to: to, cap: cap, adj: g}
  d.MaxFlow(S, T)
  for _, e := range g[S] {
    if cap[e] > 0 {
      return false, nil
    }
  }
  flows := make([][3]int, 0, len(flowArcs))
  for _, fa := range flowArcs {
    flows = append(flows, [3]int{fa.u, fa.v,
      int(fa.lo + cap[fa.eidx^1])})
  }
  return true, flows
}
```

listing 1350: go, excess bookkeeping, the arc transplant into dinic, recovery by the residual (source: dsa/samples-go/ch38/demands.go, lines 30-68)

```
  static boolean withDemands(int vn, int[][] arcs, int s, int t, int[][] flows) {
    init(vn + 2);
    int superS = vn, superT = vn + 1;
    long[] deficit = new long[MAXV];
    int[] flowArc = new int[MAXE / 2];
    int fc = 0;
    for (int[] arc : arcs) {
      addArc(arc[0], arc[1], arc[3] - arc[2]);
      arcLo[fc] = arc[2];
      flowArc[fc] = ec - 2; // the forward arc id
      fc++;
      deficit[arc[0]] -= arc[2];
      deficit[arc[1]] += arc[2];
    }
    addArc(t, s, INF); // the free return
    long need = 0;
    for (int v = 0; v < vn; v++) {
      if (deficit[v] > 0) {
        addArc(superS, v, deficit[v]);
        need += deficit[v];
      } else if (deficit[v] < 0)
        addArc(v, superT, -deficit[v]);
    }
    long got = maxflow(superS, superT);
    if (got != need)
      return false;
    // every super-source arc saturated: recover the flows
    flowCount = 0;
    for (int i = 0; i < fc; i++) {
      flows[flowCount][0] = arcs[i][0];
      flows[flowCount][1] = arcs[i][1];
      flows[flowCount][2] = arcLo[i] + (int) cap[flowArc[i] ^ 1];
      flowCount++;
    }
    return true;
  }
```

listing 1351: java, the transform, the deficit ledger, the feasibility check and flow recovery (source: dsa/samples-java/src/Ch38/Demands.java, lines 101-136)

```
    public static (bool Feasible, List<(int U, int V, long Flow)>? Flows) Solve(
        int n, (int U, int V, long Lo, long Hi)[] arcs, int s, int t)
    {
        var dinic = new Dinic(n + 2);
        const long inf = long.MaxValue / 4;
        var excess = new long[n];
        var edgeIndex = new List<int>();
        foreach ((int u, int v, long lo, long hi) in arcs)
        {
            edgeIndex.Add(dinic.EdgeCount);
            dinic.AddEdge(u, v, hi - lo);
            excess[u] -= lo;
            excess[v] += lo;
        }
        dinic.AddEdge(t, s, inf); // the free return arc
        for (int v = 0; v < n; v++)
            if (excess[v] > 0)
                dinic.AddEdge(n, v, excess[v]); // surplus drains from the super source
            else if (excess[v] < 0)
                dinic.AddEdge(v, n + 1, -excess[v]);
        dinic.MaxFlow(n, n + 1);
        if (!dinic.SaturatedFromSuperSource(n))
            return (false, null);
        var flows = new List<(int, int, long)>();
        for (int i = 0; i < arcs.Length; i++)
        {
            long transformed = dinic.FlowOnEdge(edgeIndex[i]);
            flows.Add((arcs[i].U, arcs[i].V, arcs[i].Lo + transformed));
        }
        return (true, flows);
    }
}
```

listing 1352: c#, the same transform over the chapter's dinic, lower bounds added back at the end (source: dsa/samples/src/Ch38/Costs2.cs, lines 92-123)

```
export function flowWithDemands(n, arcs, s, t) {
  // arcs: [u, v, lo, hi]
  const N = n + 2;
  const S = n;
  const T = n + 1;
  const to = [];
  const cap = [];
  const gg = Array.from({ length: N }, () => []);
  const add = (u, v, c) => {
    gg[u].push(to.length);
    to.push(v);
    cap.push(c);
    gg[v].push(to.length);
    to.push(u);
    cap.push(0);
  };
  const excess = new Array(n).fill(0);
  const flowArcs = [];
  for (const [u, v, lo, hi] of arcs) {
    add(u, v, hi - lo);
    excess[u] -= lo;
    excess[v] += lo;
    flowArcs.push([to.length - 2, u, v, lo]);
  }
  add(t, s, INF); // the free return arc
  for (let v = 0; v < n; v++) {
    if (excess[v] > 0) add(S, v, excess[v]);
    else if (excess[v] < 0) add(v, T, -excess[v]);
  }
  const d = Dinic.fromArcList(N, to, cap, gg);
  d.maxflow(S, T);
  for (const e of gg[S]) {
    if (cap[e] > 0) return { feasible: false, flows: null };
  }
  const flows = flowArcs.map(([eidx, u, v, lo]) => [u, v, lo
    + cap[eidx ^ 1]]);
  return { feasible: true, flows };
}
```

listing 1353: javascript, the whole transform, dinic adopted over prebuilt arc arrays (source: dsa/samples-js/src/ch38-demands.mjs, lines 13-50)

```
def flow_with_demands(count, arcs, s, t):
    # arcs: (u, v, lo, hi); returns (feasible, [(u, v, flow)])
    total = count + 2
    src, snk = count, count + 1
    net = Dinic(total)
    excess = [0] * count
    flow_arcs = []
    for u, v, lo, hi in arcs:
        net.add(u, v, hi - lo)  # the free capacity
        excess[u] -= lo  # outflow lower bounds debit u
        excess[v] += lo  # and credit v
        flow_arcs.append((len(net.to) - 2, u, v, lo))
    net.add(t, s, INF)  # the free return arc closes the circulation
    for v in range(count):
        if excess[v] > 0:
            net.add(src, v, excess[v])  # surplus fed from the super src
        elif excess[v] < 0:
            net.add(v, snk, -excess[v])  # deficits drain to the super snk
    net.maxflow(src, snk)
    for e in net.adj[src]:
        if net.cap[e] > 0:
            return False, None  # a super-source arc failed to saturate
    flows = []
    for eidx, u, v, lo in flow_arcs:
        flows.append((u, v, lo + net.cap[eidx ^ 1]))  # add the lower
    return True, flows
```

listing 1354: python, the ledger, the super arcs, the saturation test (source: dsa/samples-py/src/Ch38/demands.py, lines 74-99)

```
local function flow_with_demands(n, arcs, s, t)
  local N = n + 2
  local S, Tt = n, n + 1 -- super source and sink (0-based)
  local d = Dinic.new(N)
  local excess = {}
  for i = 0, n - 1 do excess[i + 1] = 0 end
  local flow_arcs = {}
  for _, a in ipairs(arcs) do
    d:add(a[1], a[2], a[4] - a[3]) -- capacity u - l
    excess[a[1] + 1] = excess[a[1] + 1] - a[3]
    excess[a[2] + 1] = excess[a[2] + 1] + a[3]
    flow_arcs[#flow_arcs + 1] = {#d.cap - 2, a[1], a[2], a[3]}
  end
  d:add(t, s, INFLOW) -- the free return arc
  for v = 0, n - 1 do
    if excess[v + 1] > 0 then
      d:add(S, v, excess[v + 1])
    elseif excess[v + 1] < 0 then
      d:add(v, Tt, -excess[v + 1])
    end
  end
  d:maxflow(S, Tt)
  for _, e in ipairs(d.g[S + 1] or {}) do
    if d.cap[e + 1] > 0 then return false, nil end -- unsaturated
  end
  local flows = {}
  for _, fa in ipairs(flow_arcs) do
    flows[#flows + 1] = {fa[2], fa[3], fa[4] + d.cap[fa[1] + 1 + 1]}
  end
  return true, flows
end
```

listing 1355: lua, the same transform, flows read off the paired arcs (source: dsa/samples-lua/ch38_demands.lua, lines 78-108)

The chain s = 0 to t = 3 with arcs (0,1,[1,2]), (1,2,[1,3]), (2,3,[2,2]) is feasible and forced: every arc carries 2. Tighten the first arc to (0,1,[2,2]) against (1,2,[1,1]) and the middle chain cannot carry 2, infeasible, the super source refuses to saturate. A circulation over a 3-cycle with every arc [1,2] is feasible with the per-arc flows only asserted to lie inside the bounds and conserve, the split between 1 and 2 is genuinely not unique.

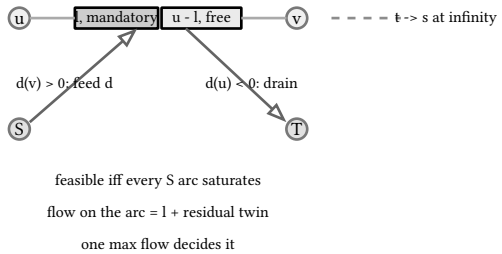


figure 339: one arc split into its mandatory l block and free u minus l block, the super source and sink feeding the deficits

The application is general technique, the standard bridge from a constraint with a floor on it to a plain max flow.

38.5. global minimum cut, stoer-wagner

The min cut of chapter 12 asks for the best split naming two terminals. The global cut asks for the best split of any shape. The stoer-wagner answer runs n - 1 phases on the shrinking graph: build a maximum adjacency ordering, repeatedly move the vertex most tightly connected to the already-moved set, and the cut of the phase is the total weight incident to the last vertex of that order. Then merge the last two vertices of the order into one, folding their

weights, and repeat. Some phase cut is the global minimum. The bookkeeping that makes the answer readable is the merged group: each live vertex knows which original vertices it absorbed, so the winning phase reports the side that contains vertex 0, complemented when the phase cut named the other side.

The dry run: the fixture is the two weight-10 triangles joined by weight-2 bridges, cut 4 on the side {0,1,2}, asserted by all seven suites.

1. Phase 1 pulls triangle members after triangle members, the ordering runs 0, 1, 2, 3, 4, 5, and the cut at the last vertex 5 is 10 + 10 = 20. The merge folds 5 into 4, so the 3 to 5 rim reads 10 + 10 = 20 into the merged cell.
2. Phase 2 orders 0, 1, 2, 3, 4 and cuts at 0 + 2 + 0 + 20 = 22, then folds 4 into 3: the group {3,4,5} is now one live vertex.
3. Phase 3 orders 0, 1, 2, 3, and the last vertex is the merged triangle: its crossing weight is exactly the two bridges, 2 + 2 = 4, the new best.
4. Phases 4 and 5 cut 10 + 12 = 22 and 20 against the folded groups, so the best stays 4.
5. The winning side is the group {3,4,5}, which misses vertex 0, so the report complements it to {0,1,2}, and the cycle-with-diagonals and single-edge fixtures pin their 7s the same way.

phase	ordering	cut at the last	best so far
1	0, 1, 2, 3, 4, 5	20	20
2	0, 1, 2, 3, 4	22	20
3	0, 1, 2, 3	4	4
4	0, 1, 2	22	4
5	0, 1	20	4

The 4 on {0,1,2} is the seven-suite pin, and the listings below run these phases in seven languages.

```
  while (ln > 1) {
    // maximum adjacency order over the active vertices
    int order[NV], on = 0;
    int rest[NV], rn = ln;
    memcpy(rest, live, sizeof(int) * (size_t)ln);
    long long key[NV];
    memset(key, 0, sizeof(key));
    int used[NV];
    memset(used, 0, sizeof(used));
    while (rn > 0) {
      int bi = -1;
      for (int i = 0; i < rn; i++)
        if (bi < 0 || key[rest[i]] > key[rest[bi]])
          bi = i;
      int v = rest[bi];
      memmove(rest + bi, rest + bi + 1, sizeof(int) * (size_t)(rn - bi - 1));
      rn--;
      used[v] = 1;
      order[on++] = v;
      for (int i = 0; i < rn; i++)
        key[rest[i]] += w[v][rest[i]];
    }
    // phase cut: weight incident to the last vertex of the order
    int last = order[on - 1];
    long long cut = 0;
    for (int i = 0; i < ln; i++)
      if (live[i] != last)
        cut += w[live[i]][last];
    if (cut < best) {
      best = cut;
      bestsn = gsz[last];
      memcpy(bestside, group[last], sizeof(int) * (size_t)gsz[last]);
    }
    // merge the last two of the order into the second-to-last
    int t2 = order[on - 2], t1 = last;
    for (int i = 0; i < gsz[t1]; i++)
      group[t2][gsz[t2]++] = group[t1][i];
    for (int i = 0; i < ln; i++)
      if (live[i] != t1 && live[i] != t2) {
        w[t2][live[i]] += w[t1][live[i]];
        w[live[i]][t2] = w[t2][live[i]];
      }
    int pos = 0;
    while (live[pos] != t1)
      pos++;
    memmove(live + pos, live + pos + 1, sizeof(int) * (size_t)(ln - pos - 1));
    ln--;
  }
```

listing 1356: c, the maximum adjacency ordering, the phase cut, the merge of the last two (source: dsa/samples-c/src/Ch38/stoerwagner.c, lines 43-90)

```
  for len(verts) > 1 {
    a := []int{verts[0]}
    inA := make([]bool, n)
    inA[verts[0]] = true
    rest := append([]int(nil), verts[1:]...)
    for len(rest) > 0 {
      // ascending scan with strict >: the smallest vertex id
      // wins ties, keeping the phase order deterministic
      bestIdx, bestW := -1, int64(-1)
      for idx, v := range rest {
        var sum int64
        for _, u := range a {
          sum += w[u][v]
        }
        if sum > bestW {
          bestW, bestIdx = sum, idx
        }
      }
      bestV := rest[bestIdx]
      rest = append(rest[:bestIdx], rest[bestIdx+1:]...)
      a = append(a, bestV)
      inA[bestV] = true
    }
    last := a[len(a)-1]
    var cut int64
    for _, v := range verts {
      if v != last {
        cut += w[v][last]
      }
    }
    if cut < best {
      side := append([]int(nil), groups[last]...)
      if !containsInt(side, 0) {
        side = nil
        for v := 0; v < n; v++ {
          if !containsInt(groups[last], v) {
            side = append(side, v)
          }
        }
      }
      sortInts(side)
      best, bestSide = cut, side
    }
    // merge the last two vertices of the order
    t2, t1 := a[len(a)-2], a[len(a)-1]
    groups[t2] = append(groups[t2], groups[t1]...)
    for _, v := range verts {
      if v != t1 && v != t2 {
        w[t2][v] += w[t1][v]
        w[v][t2] = w[t2][v]
      }
    }
  }
```

listing 1357: go, the ascending scan keeps the order deterministic, the merge folds weights (source: dsa/samples-go/ch38/stoerwagner.go, lines 28-79)

```
      int[] order = new int[NV];
      int on = 0;
      int[] rest = new int[NV];
      int rn = ln;
      System.arraycopy(live, 0, rest, 0, ln);
      long[] key = new long[NV];
      boolean[] used = new boolean[NV];
      while (rn > 0) {
        int bi = -1;
        for (int i = 0; i < rn; i++)
          if (bi < 0 || key[rest[i]] > key[rest[bi]])
            bi = i;
        int v = rest[bi];
        System.arraycopy(rest, bi + 1, rest, bi, rn - bi - 1);
        rn--;
        used[v] = true;
        order[on++] = v;
        for (int i = 0; i < rn; i++)
          key[rest[i]] += w[v][rest[i]];
      }
      // phase cut: weight incident to the last vertex of the order
      int last = order[on - 1];
      long cut = 0;
      for (int i = 0; i < ln; i++)
        if (live[i] != last)
          cut += w[live[i]][last];
      if (cut < best) {
        best = cut;
        bestsn = gsz[last];
        System.arraycopy(group[last], 0, bestside, 0, gsz[last]);
      }
      // merge the last two of the order into the second-to-last
      int t2 = order[on - 2], t1 = last;
      for (int i = 0; i < gsz[t1]; i++)
        group[t2][gsz[t2]++] = group[t1][i];
      for (int i = 0; i < ln; i++)
        if (live[i] != t1 && live[i] != t2) {
          w[t2][live[i]] += w[t1][live[i]];
          w[live[i]][t2] = w[t2][live[i]];
        }
      int pos = 0;
      while (live[pos] != t1)
        pos++;
      System.arraycopy(live, pos + 1, live, pos, ln - pos - 1);
```

listing 1358: java, the maximum adjacency ordering, the phase cut, the merge of the last two (source: dsa/samples-java/src/Ch38/Stoerwagner.java, lines 46-89)

```
        while (verts.Count > 1)
        {
            // maximum adjacency ordering over the active vertices
            var order = new List<int> { verts[0] };
            var rest = new HashSet<int>(verts.Skip(1));
            while (rest.Count > 0)
            {
                int bestV = -1;
                long bestW = -1;
                foreach (int v in rest)
                {
                    long incident = order.Sum(u => w[u, v]);
                    if (incident > bestW)
                    {
                        bestW = incident;
                        bestV = v;
                    }
                }
                order.Add(bestV);
                rest.Remove(bestV);
            }
            int last = order[^1];
            long cut = verts.Where(v => v != last).Sum(v => w[v, last]);
            if (cut < best)
            {
                int[] side = groups[last].OrderBy(v => v).ToArray();
                if (!side.Contains(0))
                    side = Enumerable.Range(0, n).Where(v => !side.Contains(v)).ToArray(); // complement
                best = cut;
                bestSide = side;
            }
            int mergeInto = order[^2]; // fold the last vertex into its predecessor
            groups[mergeInto].AddRange(groups[last]);
            foreach (int v in verts)
                if (v != last && v != mergeInto)
                {
                    w[mergeInto, v] += w[last, v];
                    w[v, mergeInto] = w[mergeInto, v];
                }
            verts.Remove(last);
        }
        return (best, bestSide);
```

listing 1359: c#, the same phase loop, linq summing the incident weight (source: dsa/samples/src/Ch38/Cuts.cs, lines 22-63)

```
  while (verts.length > 1) {
    // maximum adjacency order over the active vertices
    const a = [verts[0]];
    const rest = new Set(verts.slice(1));
    while (rest.size > 0) {
      let bestv = -1;
      let bestw = -1;
      for (const v of rest) {
        let sum = 0;
        for (const u of a) sum += w[u][v];
        if (sum > bestw) {
          bestw = sum;
          bestv = v;
        }
      }
      a.push(bestv);
      rest.delete(bestv);
    }
    let cut = 0;
    for (const u of verts) {
      if (u !== a[a.length - 1]) cut += w[u][a[a.length - 1]];
    }
    if (cut < best) {
      side = [...groups[a[a.length - 1]]].sort((u, v) => u - v);
      if (!side.includes(0)) {
        side = Array.from({ length: n }, (_, i) => i)
          .filter((v) => !side.includes(v));
      }
      best = cut;
    }
    // merge the last two vertices of the order
    const t2 = a[a.length - 2];
    const t1 = a[a.length - 1];
    groups[t2] = groups[t2].concat(groups[t1]);
    for (const v of verts) {
      if (v !== t1 && v !== t2) {
        w[t2][v] += w[t1][v];
        w[v][t2] = w[t2][v];
      }
    }
    verts = verts.filter((v) => v !== t1);
  }
```

listing 1360: javascript, the ordering, the cut, the merge, sets for the frontier (source: dsa/samples-js/src/ch38-stoerwagner.mjs, lines 21-62)

```
    while len(verts) > 1:
        order = [verts[0]]  # maximum adjacency ordering
        rest = set(verts[1:])
        while rest:
            pick, weight = None, -1
            for v in rest:  # the most-tightly-connected vertex joins
                total = sum(w[u][v] for u in order)
                if total > weight:
                    weight, pick = total, v
            order.append(pick)
            rest.discard(pick)
        cut = sum(w[u][order[-1]] for u in verts if u != order[-1])
        if cut < best[0]:  # the cut of the phase
            side = sorted(groups[order[-1]])
            if 0 not in side:  # report the side holding vertex 0
                side = [v for v in range(count) if v not in side]
            best = (cut, side)
        merged, gone = order[-2], order[-1]  # merge the last two
        groups[merged] = groups[merged] + groups[gone]
        for v in verts:
            if v not in (merged, gone):
                w[merged][v] += w[gone][v]
                w[v][merged] = w[merged][v]
        verts = [v for v in verts if v != gone]
    return best
```

listing 1361: python, the whole phase loop in 25 lines (source: dsa/samples-py/src/Ch38/stoerwagner.py, lines 27-51)

```
    local a = {verts[1]}
    local rest = {}
    local inrest = {}
    for i = 2, #verts do
      rest[#rest + 1] = verts[i]
      inrest[verts[i] + 1] = true
    end
    while #rest > 0 do
      local bestv, bestw, bestk = nil, -1, nil
      for k, v in ipairs(rest) do
        local sum = 0
        for _, u in ipairs(a) do
          sum = sum + w[v + 1][u + 1]
        end
        if sum > bestw then
          bestw, bestv, bestk = sum, v, k
        end
      end
      a[#a + 1] = bestv
      table.remove(rest, bestk)
      inrest[bestv + 1] = nil
    end
    local last = a[#a]
    local cut = 0
    for _, v in ipairs(verts) do
      if v ~= last then cut = cut + w[last + 1][v + 1] end
    end
    if cut < best then
      local side = {}
      for _, v in ipairs(groups[last + 1]) do side[#side + 1] = v end
      table.sort(side)
      if side[1] ~= 0 then -- report the side containing vertex 0
        local comp = {}
        local inside = {}
        for _, v in ipairs(side) do inside[v + 1] = true end
        for v = 0, n - 1 do
          if not inside[v + 1] then comp[#comp + 1] = v end
        end
        side = comp
      end
      best, bestside = cut, side
    end
    -- merge the last two vertices of the order
    local t2, t1 = a[#a - 1], a[#a]
    local gt = groups[t2 + 1]
    for _, v in ipairs(groups[t1 + 1]) do gt[#gt + 1] = v end
    for _, v in ipairs(verts) do
      if v ~= t1 and v ~= t2 then
        w[t2 + 1][v + 1] = w[t2 + 1][v + 1] + w[t1 + 1][v + 1]
        w[v + 1][t2 + 1] = w[t2 + 1][v + 1]
      end
    end
```

listing 1362: lua, the same phases over the dense matrix (source: dsa/samples-lua/ch38_stoerwagner.lua, lines 29-80)

The dense form is O(n^3) per the matrix scan, the heap form O(V E + V^2 log V), and fixture graphs stay at n of at most 6. Four vertices with a cycle of weight 3 plus diagonals of weight 1 cut at 7 on the side {0,1,2}. Two triangles

of weight 10 joined by two bridges of weight 2 cut at 4, again {0,1,2}: the bridges are the weak seam, and no s-t machinery would have known to look there. Two vertices with one weight-7 edge cut at 7 on {0}.

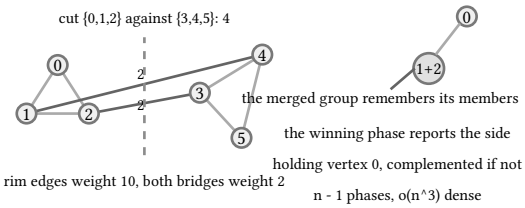


figure 340: two weight-10 triangles joined by weight-2 bridges, one merge folding a triangle, the phase cut, then the true 4-cut

The application is general technique, the cut tool when no terminal pair is given, undirected and weighted.

38.6. kuhn's bipartite matching

Walk the left vertices in index order and let each one try its neighbors in index order. A free neighbor is taken at once. An owned neighbor triggers the recursive question, can the current owner move somewhere else, and the reassignment ripples down an alternating path until some left vertex settles into a free slot or the attempt fails. Visited stamps reset per left vertex so a right vertex is contested at most once per attempt. The rule is deterministic, first-free-or- reassign in index order, so the matching itself pins wherever it is unique.

The dry run: the fixture is the 2017 problem C crate layer, rows {0,1} against columns {0,2}, the one place the suites pin the owner array itself, and all seven agree on (1, −1, 0) at size 2.

1. Row 0 opens on column 0, finds it free, and takes it: one match.
2. Row 1 opens on column 0 too: owned, so the current owner is asked whether it can move, the recursive question at the heart of the method.
3. Row 0′s scan steps past the visited column 0 and finds column 2 free: row 0 relocates, row 1 settles at column 0.
4. The owner array reads (1, −1, 0) with column 1 idle, and the height-1 layer with its empty row side matches 0.
5. The 2025 problem B bank at n = 8 walks left 1, 4, 6 in index order, the alternating path reaching two owners deep, and closes at size 3: erasing 2 or 8 holds the 3, erasing 4 or 6 drops it to 2, so the first move is 2.
6. The denial shape matches 3 of 4 with card 3 forbidden and 4 once the fourth player may take it, the empty right side pins 0, the complete 3 × 3 pins 3.

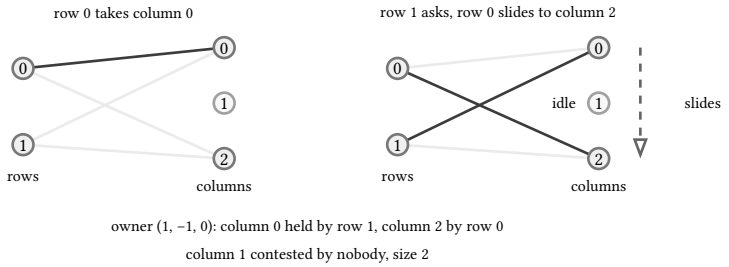


figure 341: the crate layer before and after row 1 arrives, the ask rippling row 0 over to column 2, column 1 idle throughout

The owner array beside the size 2 is the seven-suite pin, and the listings below ripple these asks in seven languages.

```c
static int try_k(int u) {
  for (int i = 0; i < deg[u]; i++) {
    int v = adj[u][i];
    if (vis[v])
      continue;
    vis[v] = 1;
    if (owner[v] < 0 || try_k(owner[v])) {
      owner[v] = u;
      return 1;
    }
  }
  return 0;
}

static int kuhn(void) {
  int size = 0;
  for (int u = 0; u < nl; u++) {
    memset(vis, 0, sizeof(int) * (size_t)nr);
    if (try_k(u))
      size++;
  }
  return size;
}
```

listing 1363: c, the recursive try and the driver, the deterministic owner array (source: dsa/samples-c/src/Ch38/kuhn.c, lines 39-61)

```go
func KuhnFull(nl, nr int, adj [][]int) (int, []int) {
  matchR := make([]int, nr)
  for i := range matchR {
    matchR[i] = -1
  }
  var try func(u int, vis []bool) bool
  try = func(u int, vis []bool) bool {
    for _, v := range adj[u] {
      if !vis[v] {
        vis[v] = true
        if matchR[v] < 0 || try(matchR[v], vis) {
          matchR[v] = u
          return true
        }
      }
    }
    return false
  }
  size := 0
  for u := 0; u < nl; u++ {
    if try(u, make([]bool, nr)) {
      size++
    }
  }
  return size, matchR
}
```

listing 1364: go, kuhn full, the right-side owner array returned for pinning (source: dsa/samples-go/ch38/kuhn.go, lines 12-37)

```
  static boolean tryK(int u) {
    for (int i = 0; i < deg[u]; i++) {
      int v = adj[u][i];
      if (vis[v])
        continue;
      vis[v] = true;
      if (owner[v] < 0 || tryK(owner[v])) {
        owner[v] = u;
        return true;
      }
    }
    return false;
  }

  static int kuhn() {
    int size = 0;
    for (int u = 0; u < nl; u++) {
      java.util.Arrays.fill(vis, false);
      if (tryK(u))
        size++;
    }
    return size;
  }
```

listing 1365: java, the recursive try and the driver, the deterministic owner array (source: dsa/samples-java/src/Ch38/Kuhn.java, lines 40-63)

```
    public static (int Size, int[] RightOwner) Match(int leftCount, int rightCount, int[][] adjacency)
    {
        var owner = new int[rightCount];
        Array.Fill(owner, -1);
        bool Try(int u, bool[] visited)
        {
            foreach (int v in adjacency[u])
                if (!visited[v])
                {
                    visited[v] = true;
                    if (owner[v] < 0 || Try(owner[v], visited))
                    {
                        owner[v] = u; // first-free-or-reassign
                        return true;
                    }
                }
            return false;
        }

        int size = 0;
        for (int u = 0; u < leftCount; u++)
            if (Try(u, new bool[rightCount]))
                size++;
        return (size, owner);
    }
```

listing 1366: c#, the local try function, first-free-or-reassign (source: dsa/samples/src/Ch38/Match.cs, lines 10-34)

```javascript
export function kuhn(nl, nr, adj) {
  const matchR = new Array(nr).fill(-1);
  const tryK = (u, vis) => {
    for (const v of adj[u]) {
      if (vis[v]) continue;
      vis[v] = true;
      if (matchR[v] < 0 || tryK(matchR[v], vis)) {
        matchR[v] = u;
        return true;
      }
    }
    return false;
  };
  let size = 0;
  for (let u = 0; u < nl; u++) {
    if (tryK(u, new Array(nr).fill(false))) size += 1;
  }
  return { size, matchR };
}
```

listing 1367: javascript, the whole matcher, size and match r out (source: dsa/samples-js/src/ch38-kuhn.mjs, lines 7-25)

```python
def kuhn(adjacency, left, right):
    match = [-1] * right  # right-side owner

    def try_k(u, seen):
        for v in adjacency[u]:
            if not seen[v]:
                seen[v] = True
                if match[v] < 0 or try_k(match[v], seen):
                    match[v] = u
                    return True
        return False

    size = 0
    for u in range(left):
        if try_k(u, [False] * right):
            size += 1
    return size, match
```

listing 1368: python, the try recursion and the driver (source: dsa/samples-py/src/Ch38/kuhn.py, lines 14-30)

```lua
local function kuhn(nl, nr, adj)
  local matchr = {}
  for i = 0, nr - 1 do matchr[i + 1] = -1 end
  local function try_k(u, vis)
    for _, v in ipairs(adj[u + 1] or {}) do
      if not vis[v + 1] then
        vis[v + 1] = true
        if matchr[v + 1] < 0 or try_k(matchr[v + 1], vis) then
          matchr[v + 1] = u
          return true
        end
      end
    end
    return false
  end
  local size = 0
  for u = 0, nl - 1 do
    local vis = {}
    for i = 0, nr - 1 do vis[i + 1] = false end
    if try_k(u, vis) then size = size + 1 end
  end
  return size, matchr
end
```

listing 1369: lua, the same recursion over 1-based tables (source: dsa/samples-lua/ch38_kuhn.lua, lines 8-30)

The bound is O(V E), one depth-first try per left vertex over the whole edge list, and hopcroft-karp's O(sqrt(V) E) is the named upgrade that stays prose. Matching is unit max flow, and the seeded family of the dinic section above runs both engines on the same graph to the same 18.

The fixtures are three contest shapes. The 2017 problem C crate layer, rows {0,1} against columns {0,2} as a complete bipartite graph, matches 2, and the deterministic rule pins the owner array (1, −1, 0): row 0 takes column 0 first, row 1 reassigns to column 2. The height-1 layer with an empty row side matches 0. The 2022 problem X denial shape, 4 slots against cards {0,1,2,3} with card 3 forbidden to every player, matches 3, and 3 under 4 flags the infeasibility, while opening card 3 to the fourth player completes the matching at 4. The 2025 problem B prime-multiple graph splits the numbers 1 through n by omega parity, the count of prime factors with multiplicity, even bank against odd bank, edges where one divides the other. At n = 6 the matching is 3 and deleting any even vertex drops it to 2, so every opening loses, second wins. At n = 8 the matching is 3, deleting 2 or 8 keeps 3 while deleting 4 or 6 drops to 2, so the first player opens by erasing 2. The edges close with an empty right side at 0 and a complete 3 by 3 at 3.

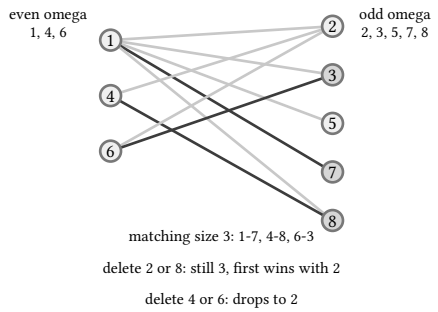


figure 342: the n = 8 prime-multiple graph, the matching 1-7, 4-8, 3-6 bold, vertex 2 unmatched as the winning opening

The applications are icpc world finals 2017 problem C (book 10, chapter 8), mission improbable, one matching per distinct crate height with m + n minus match piles left standing, icpc world finals 2022 problem X (book 10, chapter 11), quartets, re-testing the 32-slot matching after every action and reporting the first failure, and icpc

world finals 2025 problem B (book 10, chapter 13), blackboard game, whose openings are decided by whether some maximum matching avoids the erased vertex.

38.7. the hungarian algorithm

The assignment problem asks for the cheapest perfect matching on a square cost matrix, and the hungarian method solves it in O(n^3) with potentials. Each row carries a potential u and each column a potential v, and a cell (i, j) is tight when u of i plus v of j equals the cost there. One row at a time, an alternating tree grows over the tight cells: the slack of a column is how far its cost sits above the potentials, the tree extends through the smallest-slack column, and when every reachable column has been visited the potentials adjust by the common delta, tightening at least one more cell without loosening any visited one. The tree reaches a free column, the augmenting path commits along the parent links, and the matching grows by one.

The dry run: the fixture is the 3 × 3 matrix 4, 1, 3 over 2, 0, 5 over 3, 2, 2, cost 5 on the unique assignment (1, 0, 2), asserted by all seven suites.

1. Row 0 grows over slacks 4, 1, 3: the smallest sits at column 1, the delta is 1, and the free column commits at once.
2. Row 1 finds column 1 tight at slack 0 but owned, so the tree extends through row 0′s cells: the remaining slacks run 2 and 2, the common delta is 2, and after u(0) += 2 and u(1) += 2 against v(1) -= 2 column 0 comes tight and free, the commit lands there.
3. Row 2 prices its cells at 3, 4, 2 against the standing potentials: delta 2, column 2 free, the third commit closes the matching.
4. The potentials settle at u = 3, 2, 2 and v = 0, −2, 0, and their sum 3 + 2 + 2 + 0 - 2 + 0 = 5 equals the assignment cost 1 + 2 + 2 = 5, the dual pair the method maintains.
5. The 4 × 4 answers 13 against the permutation brute, the 1 × 1 of 7 answers 7, and the profit matrix maximizes at 21 by negating on the way in and the way out.

entering row	deltas paid	commits	cell cost
0	1	column 1, free at once	1
1	0, then 2	column 0, freed by the shift	2
2	2	column 2, free at once	2

The cost 5 on the unique assignment (1, 0, 2) is the seven-suite pin, and the listings below grow these trees in seven languages.

```
  for (int i = 1; i ≤ nn; i++) {
    p[0] = i;
    int j0 = 0;
    for (int j = 0; j ≤ nn; j++) {
      minv[j] = BIG;
      used[j] = 0;
    }
    do {
      used[j0] = 1;
      int i0 = (int)p[j0], j1 = -1;
      long long delta = BIG;
      for (int j = 1; j ≤ nn; j++)
        if (!used[j]) {
          long long cur = cost[(i0 - 1) * nn + (j - 1)] - u[i0] - v[j];
          if (cur < minv[j]) {
            minv[j] = cur;
            way[j] = j0;
          }
          if (minv[j] < delta) {
            delta = minv[j];
            j1 = j;
          }
        }
      for (int j = 0; j ≤ nn; j++) {
        if (used[j]) {
          u[(int)p[j]] += delta;
          v[j] -= delta;
        } else {
          minv[j] -= delta;
        }
      }
      j0 = j1;
    } while (p[j0] ≠ 0);
    do {
      int j1 = (int)way[j0];
      p[j0] = p[j1];
      j0 = j1;
    } while (j0);
  }
```

listing 1370: c, one row's tree growth, the delta adjustment, the augmenting commit (source: dsa/samples-c/src/Ch38/hungarian.c, lines 35-73)

```
  for i := 1; i <= n; i++ {
    p[0] = i
    j0 := 0
    minv := make([]int64, n+1)
    for k := range minv {
      minv[k] = inf
    }
    used := make([]bool, n+1)
    for {
      used[j0] = true
      i0 := p[j0]
      delta := inf
      j1 := -1
      for j := 1; j <= n; j++ {
        if used[j] {
          continue
        }
        cur := cost[i0-1][j-1] - u[i0] - v[j]
        if cur < minv[j] {
          minv[j] = cur
          way[j] = j0
        }
        if minv[j] < delta {
          delta = minv[j]
          j1 = j
        }
      }
      for j := 0; j <= n; j++ {
        if used[j] {
          u[p[j]] += delta
          v[j] -= delta
        } else {
          minv[j] -= delta
        }
      }
      j0 = j1
      if p[j0] == 0 {
        break
      }
    }
    for {
      j1 := way[j0]
      p[j0] = p[j1]
      j0 = j1
      if j0 == 0 {
        break
      }
    }
  }
```

listing 1371: go, the shortest-slack column, the delta, the commit (source: dsa/samples-go/ch38/hungarian.go, lines 14-62)

```
      p[0] = i;
      int j0 = 0;
      for (int j = 0; j ≤ nn; j++) {
        minv[j] = BIG;
        used[j] = false;
      }
      do {
        used[j0] = true;
        int i0 = (int) p[j0], j1 = -1;
        long delta = BIG;
        for (int j = 1; j ≤ nn; j++)
          if (!used[j]) {
            long cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
            if (cur < minv[j]) {
              minv[j] = cur;
              way[j] = j0;
            }
            if (minv[j] < delta) {
              delta = minv[j];
              j1 = j;
            }
          }
        for (int j = 0; j ≤ nn; j++) {
          if (used[j]) {
            u[(int) p[j]] += delta;
            v[j] -= delta;
          } else {
            minv[j] -= delta;
          }
        }
        j0 = j1;
      } while (p[j0] ≠ 0);
      do {
        int j1 = (int) way[j0];
        p[j0] = p[j1];
        j0 = j1;
      } while (j0 ≠ 0);
    }
    long total = 0;
```

listing 1372: java, one row's tree growth, the delta adjustment, the augmenting commit (source: dsa/samples-java/src/Ch38/Hungarian.java, lines 36-74)

```
        for (int i = 1; i ≤ n; i++)
        {
            p[0] = i;
            int j0 = 0;
            var minv = new long[n + 1];
            Array.Fill(minv, inf);
            var used = new bool[n + 1];
            do
            {
                used[j0] = true;
                int i0 = p[j0];
                long delta = inf;
                int j1 = -1;
                for (int j = 1; j ≤ n; j++)
                    if (!used[j])
                    {
                        long cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
                        if (cur < minv[j])
                        {
                            minv[j] = cur; // grow the tree on the shortest slack
                            way[j] = j0;
                        }
                        if (minv[j] < delta)
                        {
                            delta = minv[j];
                            j1 = j;
                        }
                    }
                for (int j = 0; j ≤ n; j++)
                    if (used[j])
                    {
                        u[p[j]] += delta; // the potential adjustment
                        v[j] -= delta;
                    }
                    else
                        minv[j] -= delta;
                j0 = j1;
            }
            while (p[j0] ≠ 0);
            do
            {
                int j1 = way[j0]; // commit the augmenting path
                p[j0] = p[j1];
                j0 = j1;
            }
            while (j0 ≠ 0);
        }
```

listing 1373: c#, the same row loop, way links the parents, the commit walk (source: dsa/samples/src/Ch38/Hungarian.cs, lines 18-64)

```
  for (let i = 1; i ≤ n; i++) {
    p[0] = i;
    let j0 = 0;
    const minv = new Array(n + 1).fill(INF);
    const used = new Array(n + 1).fill(false);
    for (;;) {
      used[j0] = true;
      const i0 = p[j0];
      let delta = INF;
      let j1 = -1;
      for (let j = 1; j ≤ n; j++) {
        if (used[j]) continue;
        const cur = cost[i0 - 1][j - 1] - u[i0] - v[j];
        if (cur < minv[j]) {
          minv[j] = cur;
          way[j] = j0;
        }
        if (minv[j] < delta) {
          delta = minv[j];
          j1 = j;
        }
      }
      for (let j = 0; j ≤ n; j++) {
        if (used[j]) {
          u[p[j]] += delta;
          v[j] -= delta;
        } else {
          minv[j] -= delta;
        }
      }
      j0 = j1;
      if (p[j0] === 0) break;
    }
    for (;;) {
      const j1 = way[j0];
      p[j0] = p[j1];
      j0 = j1;
      if (j0 === 0) break;
    }
  }
```

listing 1374: javascript, the same row loop, values exact on number (source: dsa/samples-js/src/ch38-hungarian.mjs, lines 13-52)

```
    for i in range(1, size + 1):
        match[0] = i
        j0 = 0
        minv = [BIG] * (size + 1)  # the shortest slack per column
        used = [False] * (size + 1)
        while True:
            used[j0] = True
            i0 = match[j0]
            delta = BIG
            j1 = -1
            for j in range(1, size + 1):
                if not used[j]:
                    cur = cost[i0 - 1][j - 1] - u[i0] - v[j]
                    if cur < minv[j]:
                        minv[j] = cur
                        way[j] = j0
                    if minv[j] < delta:  # the tightest column grows next
                        delta = minv[j]
                        j1 = j
            for j in range(size + 1):  # potentials adjust by delta
                if used[j]:
                    u[match[j]] += delta
                    v[j] -= delta
                else:
                    minv[j] -= delta
            j0 = j1
            if match[j0] == 0:  # a free column ends the tree
                break
        while True:  # walk the parent links, commit the augmentation
            j1 = way[j0]
            match[j0] = match[j1]
            j0 = j1
            if j0 == 0:
                break
```

listing 1375: python, the tree growth with the tightest column, the commit walk (source: dsa/samples-py/src/Ch38/hungarian.py, lines 26-59)

```
  for i = 1, n do
    p[1] = i
    local j0 = 0
    local minv = {}
    local used = {}
    for j = 0, n do
      minv[j + 1] = BIG
      used[j + 1] = false
    end
    repeat
      used[j0 + 1] = true
      local i0 = p[j0 + 1]
      local delta, j1 = BIG, -1
      for j = 1, n do
        if not used[j + 1] then
          local cur = cost[i0][j] - u[i0 + 1] - v[j + 1]
          if cur < minv[j + 1] then
            minv[j + 1] = cur
            way[j + 1] = j0
          end
          if minv[j + 1] < delta then
            delta = minv[j + 1]
            j1 = j
          end
        end
      end
      for j = 0, n do
        if used[j + 1] then
          u[p[j + 1] + 1] = u[p[j + 1] + 1] + delta
          v[j + 1] = v[j + 1] - delta
        else
          minv[j + 1] = minv[j + 1] - delta
        end
      end
      j0 = j1
    until p[j0 + 1] == 0
    repeat
      local j1 = way[j0 + 1]
      p[j0 + 1] = p[j1 + 1]
      j0 = j1
    until j0 == 0
  end
```

listing 1376: lua, the potentials loop in 1-based columns (source: dsa/samples-lua/ch38_hungarian.lua, lines 19-60)

The solver minimizes, and maximization negates the matrix and negates the answer. Big entries model forbidden pairs, the trick the next section builds on. The 3 by 3 matrix 4, 1, 3 over 2, 0, 5 over 3, 2, 2 costs 5 on the unique assignment (1, 0, 2), row i takes column a of i, cross-checked against the permutation brute. The 4 by 4 costs 13 with the assignment asserted a valid permutation of exactly that cost. The 1 by 1 of 7 answers 7, and the profit matrix 3, 8, 4 over 9, 1, 6 over 7, 5, 2 maximizes at 21 by negation.

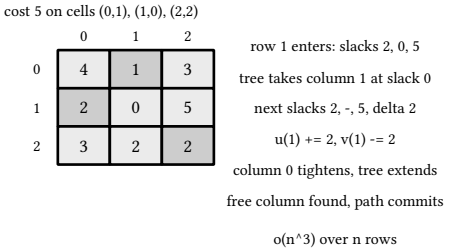


figure 343: the 3 by 3 cost grid with the three chosen cells shaded and one row of potential adjustments written beside it

The application is general technique, the exact assignment engine whenever the matrix is small and the costs are real.

38.8. assignment problems

The modeling facet around the hungarian solver: how the messy question becomes the square matrix. Forbidden pairs, worker w cannot do task t, become an entry of 1e9, far past any real cost, so a feasible assignment never touches one and a total at or past the same threshold proves no feasible assignment exists. A rectangular n by m problem with m above n pads zero-cost dummy columns up to the square, and which real tasks get done falls out of the solve, the dummies absorbing the idle rows. Maximization negates going in and coming out.

The dry run: the fixtures are the forbidden matrix, 4, 1e9, 6 over 7, 3, 5 over 6, 5, 4 with worker 0 barred from task 1, and the 3 × 5 rectangle, the unique cost 11 and the 9 over real tasks {0, 1, 2}, asserted by all seven suites.

1. The 1e9 at cell (0, 1) never turns tight: it prices past every delta the tree ever pays, so the solver walks around it and the diagonal commits at 4 + 3 + 4 = 11.
2. The rectangle pads columns 5 and 6 at zero, a 3 × 7 square, and the solve picks (0, 1), (1, 0), (2, 2) at 4 + 3 + 2 = 9 with both dummies idle.
3. All three rows keep real tasks, the pinned read {0, 1, 2}, and in general it is the dummies that absorb whatever rows sit out.
4. The 1 × 1 pair pins its single cost, 7 in the C# suite and 5 in C, and a full-forbidden row drives the total to the threshold itself: the same number that models the ban reports the infeasibility.

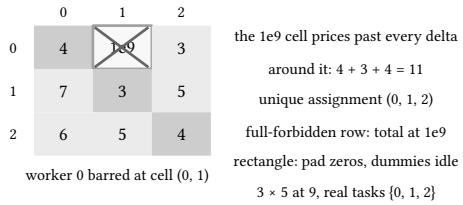


figure 344: the forbidden matrix with the big cell struck through, the walk around it shaded on the diagonal, the threshold read beside

The 11 around the ban and the 9 over real tasks are the seven-suite pins, and the listings below model these matrices in seven languages.

```c
// rectangular n workers x m tasks (m ≥ n) padded with zero columns
static long long assign_rect(const long long *cost, int rows, int cols,
                             int *out) {
  long long sq[HN * HN];
  memset(sq, 0, sizeof(sq));
  for (int i = 0; i < rows; i++)
    for (int j = 0; j < cols; j++)
      sq[i * cols + j] = cost[i * cols + j];
  return hungarian(sq, cols, out);
}
```

listing 1377: c, the rectangle squared by zero padding, forbidden pairs enter as the 1e9 constant (source: dsa/samples-c/src/Ch38/assign.c, lines 83-92)

```
// assignINF models forbidden pairs; costs at or past it flag
// infeasibility.
const assignINF = int64(1e9)

// AssignSquare solves a square assignment with forbidden pairs as
// INF entries.
func AssignSquare(cost [][]int64) (int64, []int) {
  return Hungarian(cost)
}

// AssignRect pads a rectangular n x m (m > n) matrix with zero-cost
// dummy columns: which tasks get done falls out of the solve.
func AssignRect(rect [][]int64) (int64, []int) {
  rows := len(rect)
  cols := len(rect[0])
  sq := make([][]int64, rows)
  for i := range sq {
    sq[i] = append(append([]int64(nil), rect[i]...),
      make([]int64, cols-rows)...)
  }
  return Hungarian(sq)
}
```

listing 1378: go, the forbidden constant, the square and rectangle wrappers (source: dsa/samples-go/ch38/assign.go, lines 3-24)

```
  static long assignRect(long[][] cost, int rows, int cols, int[] out) {
    long[][] sq = new long[HN][HN];
    for (int i = 0; i < rows; i++)
      for (int j = 0; j < cols; j++)
        sq[i][j] = cost[i][j];
    return hungarian(sq, cols, out);
  }
```

listing 1379: java, the rectangle squared by zero padding, forbidden pairs enter as the 1e9 constant (source: dsa/samples-java/src/Ch38/Assign.java, lines 85-92)

```
public static class Assign
{
    public const long Inf = 1_000_000_000L; // big values model forbidden pairs

    public static (long Cost, int[] Assignment) SolveRect(long[][] cost)
    {
        int rows = cost.Length;
        int cols = cost[0].Length;
        int side = Math.Max(rows, cols);
        var square = new long[side][];
        for (int i = 0; i < side; i++)
        {
            square[i] = new long[side];
            for (int j = 0; j < side; j++)
                square[i][j] = i < rows && j < cols ? cost[i][j] : 0; // dummy padding
        }
        return Hungarian.Solve(square);
    }
}
```

listing 1380: c#, the assign facet over the solver, dummy cells at zero (source: dsa/samples/src/Ch38/Hungarian.cs, lines 87-105)

```javascript
export const ASSIGN_BIG = 1e9; // models a forbidden pair

export function solveAssign(matrix) {
  const rows = matrix.length;
  const cols = matrix[0].length;
  const side = Math.max(rows, cols);
  const sq = Array.from({ length: side }, (_, i) =>
    Array.from({ length: side }, (_, j) =>
      (i < rows && j < cols ? matrix[i][j] : 0)));
  return hungarian(sq);
}
```

listing 1381: javascript, the solver imported and called, padding inline (source: dsa/samples-js/src/ch38-assign.mjs, lines 9-19)

```python
def solve(cost):
    # square up, run the solver, strip the padding from the report
    rows = len(cost)
    cols = len(cost[0])
    if cols < rows:  # pad with zero-cost dummy columns
        cost = [row + [0] * (rows - cols) for row in cost]
    total, assign = hungarian(cost)
    used = assign[:cols]
    done = sorted(set(used) & set(range(cols)))
    return total, used, done
```

listing 1382: python, square up, solve, strip the padding from the report (source: dsa/samples-py/src/Ch38/assign.py, lines 68-77)

```lua
  {name = "F2: rectangular 3x5 padded to a square, cost 9", fn = function()
    local rect = {{6, 4, 5, 8, 7}, {3, 6, 4, 5, 6}, {8, 3, 2, 4, 5}}
    local sq = {}
    for i, row in ipairs(rect) do
      sq[i] = {row[1], row[2], row[3], row[4], row[5], 0, 0} -- pad
    end
    local total, a = hungarian(sq)
    T.eq(total, 9)
    local used = {}
    for _, j in ipairs(a) do
      if j ≤ 5 then used[j] = true end -- dummy columns do nothing
    end
    local real = 0
    for j in pairs(used) do real = real + 1 end
    T.eq(real, 3)
    -- the optimal real-task choice: brute over 5-permutations
```

listing 1383: lua, the padded square in the fixture, dummy columns doing nothing (source: dsa/samples-lua/ch38_assign.lua, lines 79-94)

Each language calls the solver of the previous section rather than recoding it, in-file where the house form keeps samples standalone. The forbidden fixture, 4, inf, 6 over 7, 3, 5 over 6, 5, 4 with worker 0 barred from task 1, costs 11 on the unique assignment (0, 1, 2). The 3 by 5 rectangle 6, 4, 5, 8, 7 over 3, 6, 4, 5, 6 over 8, 3, 2, 4, 5 pads two zero columns and costs 9, rows taking tasks (1, 0, 2), the real tasks used exactly {0, 1, 2}. The 1 by 1 forced pair pins, and a full-forbidden row reports a cost at the threshold, infeasibility by the same number that models it.

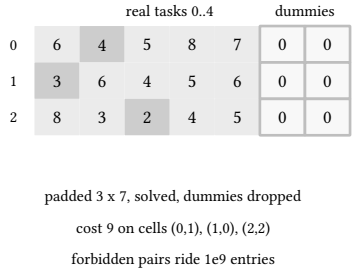


figure 345: the 3 by 5 rectangle padded to a square, the two dummy columns shaded, the chosen cells marked

The application is general technique, and the same modeling carries to any solver of the assignment shape.

38.9. hall's theorem

A bipartite graph with left side L saturates L exactly when every subset S of L has at least as many neighbors, |S| at most |N(S)|. The theorem converts a matching question into a counting question, and this section ships two checkers. The general one enumerates all 2^n left subsets for n up to 20, unions the neighbor bitmasks, and fails on the first subset that outgrows its neighborhood. The prefix counting form is the useful special case for lower-bounded slot families: each item i demands at least L of i slots among the first ones, and the condition collapses to, for every prefix length q, at least q items carry L at most q, checked by sorting the bounds and scanning, with the first violated q reported.

The dry run: the fixtures are the four 2025 problem F bound families plus the seeded agreement family, asserted by all seven suites, and the walk runs the prefix form.

1. Sorted, the feasible family 2, 2, 1, 1 reads 1, 1, 2, 2: the prefix counts run 2, 4, 4, 4 against the needs 1, 2, 3, 4, every fence covered.
2. The family 3, 2, 3 fails at the first fence: sorted 2, 3, 3 counts 0 items at or below q = 1, no variety tolerates a single slot.
3. The family 1, 2, 2 counts 1, 3, 3 against 1, 2, 3 and holds, while 2, 2, 2 counts 0, 3, 3 and fails at q = 1 again.
4. The seeded agreement family draws five bipartite graphs where hall holds exactly when kuhn saturates all ten lefts: the pins read (fails, 9), (fails, 9), (holds, 10), (fails, 9), (fails, 8).
5. The degenerates: a lone left vertex with no neighbors has |S| = 1 against |N(S)| = 0 and fails both checkers, and the empty left side holds with nothing to check.

bounds	m	sorted	counts at q = 1 to m	verdict
2, 2, 1, 1	4	1, 1, 2, 2	2, 4, 4, 4	holds
3, 2, 3	3	2, 3, 3	0, 1, 3	fails at q = 1
1, 2, 2	3	1, 2, 2	1, 3, 3	holds
2, 2, 2	3	2, 2, 2	0, 3, 3	fails at q = 1

The two holds and the two q = 1 failures are the seven-suite pins, and the listings below sweep subsets and prefixes in seven languages.

```c
// general checker: every nonempty left subset has enough neighbors
static int hall_holds(void) {
  for (int mask = 1; mask < (1 << nl); mask++) {
    long long nb = 0;
    for (int i = 0; i < nl; i++)
      if (mask & (1 << i))
        for (int j = 0; j < deg[i]; j++)
          nb |= 1LL << adj[i][j];
    int lc = __builtin_popcount((unsigned)mask);
    int rc = __builtin_popcountll(nb);
    if (lc > rc)
      return 0;
  }
  return 1;
}

// prefix counting form: returns the first violated q, -1 when feasible
static int hall_prefix(const int *L, int k, int m, int sortbuf[MAXN]) {
  memcpy(sortbuf, L, sizeof(int) * (size_t)k);
  for (int i = 1; i < k; i++) // insertion sort
    for (int j = i; j > 0 && sortbuf[j - 1] > sortbuf[j]; j--) {
      int t = sortbuf[j];
      sortbuf[j] = sortbuf[j - 1];
      sortbuf[j - 1] = t;
    }
  for (int q = 1; q <= m; q++) {
    int cnt = 0;
    for (int i = 0; i < k; i++)
      if (sortbuf[i] <= q)
        cnt++;
    if (cnt < q)
      return q;
  }
  return -1;
}
```

listing 1384: c, the bitmask sweep and the prefix scan, first violated q out (source: dsa/samples-c/src/Ch38/hall.c, lines 64-98)

```
func HallHolds(nl int, adj [][]int) bool {
  for mask := 1; mask < 1<<nl; mask++ {
    nb := 0
    for i := 0; i < nl; i++ {
      if mask&(1<<i) == 0 {
        continue
      }
      for _, v := range adj[i] {
        nb |= 1 << v
      }
    }
    if bitCount(mask) > bitCount(nb) {
      return false
    }
  }
  return true
}

func bitCount(x int) int {
  c := 0
  for x > 0 {
    c += x & 1
    x >>= 1
  }
  return c
}

// HallPrefix is the counting form for lower-bounded slot families:
// sort the L bounds, for every prefix q require at least q items
// with L ≤ q, reporting the first violated q (-1 when feasible).
func HallPrefix(bounds []int64, m int) (bool, int) {
  for q := 1; q <= m; q++ {
    cnt := 0
    for _, l := range bounds {
      if l <= int64(q) {
        cnt++
      }
    }
    if cnt < q {
      return false, q
    }
  }
  return true, -1
}
```

listing 1385: go, the subset form, bitcount, the prefix form (source: dsa/samples-go/ch38/hall.go, lines 5-48)

```
  static boolean hallHolds() {
    for (int mask = 1; mask < (1 << nl); mask++) {
      long nb = 0;
      for (int i = 0; i < nl; i++)
        if ((mask & (1 << i)) != 0)
          for (int j = 0; j < deg[i]; j++)
            nb |= 1L << adj[i][j];
      int lc = Integer.bitCount(mask);
      int rc = Long.bitCount(nb);
      if (lc > rc)
        return false;
    }
    return true;
  }

  // prefix counting form: returns the first violated q, -1 when feasible
  static int hallPrefix(int[] bounds, int m) {
    int[] sorted = bounds.clone();
    for (int i = 1; i < sorted.length; i++) // insertion sort
      for (int j = i; j > 0 && sorted[j - 1] > sorted[j]; j--) {
        int t = sorted[j];
        sorted[j] = sorted[j - 1];
        sorted[j - 1] = t;
      }
    for (int q = 1; q ≤ m; q++) {
      int cnt = 0;
      for (int k : sorted)
        if (k ≤ q)
          cnt++;
      if (cnt < q)
        return q;
    }
    return -1;
  }
```

listing 1386: java, the bitmask sweep and the prefix scan, first violated q out, bitCount from the stdlib (source: dsa/samples-java/src/Ch38/Hall.java, lines 64-98)

```
    public static bool Holds(int leftCount, int[][] adjacency)
    {
        for (int mask = 1; mask < 1 << leftCount; mask++)
        {
            long neighbors = 0;
            for (int i = 0; i < leftCount; i++)
                if ((mask & (1 << i)) != 0)
                    foreach (int v in adjacency[i])
                        neighbors |= 1L << v;
            if (System.Numerics.BitOperations.PopCount((uint)mask) >
                System.Numerics.BitOperations.PopCount((ulong)neighbors))
                return false;
        }
        return true;
    }

    /// <summary>For every prefix q, at least q varieties need L &lt;= q; the second
    /// value names the first violated q or -1.</summary>
    public static (bool Holds, int FirstViolation) PrefixHolds(long[] lowerBounds, int m)
    {
        var sorted = lowerBounds.OrderBy(v => v).ToArray();
        for (int q = 1; q <= m; q++)
        {
            int count = sorted.Count(l => l <= q);
            if (count < q)
                return (false, q);
        }
        return (true, -1);
    }
```

listing 1387: c#, both checkers, popcount on the unioned neighbors (source: dsa/samples/src/Ch38/Match.cs, lines 43-71)

```javascript
const popcount = (x) => {
  let c = 0;
  while (x > 0) {
    c += x & 1;
    x >>= 1;
  }
  return c;
};

export function hallHolds(nl, adj) {
  for (let mask = 1; mask < 1 << nl; mask++) {
    let nb = 0;
    for (let i = 0; i < nl; i++) {
      if ((mask >> i) & 1) {
        for (const v of adj[i]) nb |= 1 << v;
      }
    }
    if (popcount(mask) > popcount(nb)) return false;
  }
  return true;
}

export function hallPrefix(L, m) {
  for (let q = 1; q ≤ m; q++) {
    let cnt = 0;
    for (const x of L) {
      if (x ≤ q) cnt += 1;
    }
    if (cnt < q) return [false, q];
  }
  return [true, -1];
}
```

listing 1388: javascript, popcount, the subset sweep, the prefix scan (source: dsa/samples-js/src/ch38-hall.mjs, lines 7-38)

```python
def hall_holds(adjacency, left, right):
    for mask in range(1, 1 << left):  # every nonempty left subset
        neighbors = 0
        for i in range(left):
            if mask >> i & 1:
                for v in adjacency[i]:
                    neighbors |= 1 << v
        if bin(mask).count("1") > bin(neighbors).count("1"):
            return False  # |S| outgrew |N(S)|
    return True


def hall_prefix(lower, slots):
    # first violated q, or -1 when the family is feasible
    ordered = sorted(lower)
    for q in range(1, slots + 1):
        with_room = sum(1 for x in ordered if x ≤ q)
        if with_room < q:  # fewer than q varieties fit q slots
            return False, q
    return True, -1
```

listing 1389: python, the subset sweep, the prefix form with the kuhn partner below (source: dsa/samples-py/src/Ch38/hall.py, lines 15-34)

```lua
local function hall_holds(nl, adj, nr)
  for mask = 1, (1 << nl) - 1 do
    local nb = 0
    for i = 0, nl - 1 do
      if (mask >> i) & 1 == 1 then
        for _, v in ipairs(adj[i + 1] or {}) do
          nb = nb | (1 << v)
        end
      end
    end
    local ssize, nbsize = 0, 0
    local m = mask
    while m > 0 do
      ssize = ssize + (m & 1)
      m = m >> 1
    end
    m = nb
    while m > 0 do
      nbsize = nbsize + (m & 1)
      m = m >> 1
    end
    if ssize > nbsize then return false end
  end
  return true
end

-- 2025/F prefix form: returns feasible, first violated q (-1 if none)
local function hall_prefix(L, m)
  local sl = {table.unpack(L)}
  table.sort(sl)
  for q = 1, m do
    local cnt = 0
    for _, x in ipairs(sl) do
      if x ≤ q then cnt = cnt + 1 end
    end
    if cnt < q then return false, q end
  end
  return true, -1
end
```

listing 1390: lua, both forms, hand-rolled bit counts (source: dsa/samples-lua/ch38_hall.lua, lines 9-47)

The agreement family runs five seeded bipartite graphs beside the kuhn matcher of the matching section above: hall holds exactly when the kuhn size is the full left side, and the pins read (fails, 9), (fails, 9), (holds, 10), (fails, 9), (fails, 8). The edge family: a single left vertex with no neighbors fails hall and matches 0, and the empty left side holds trivially.

The prefix form carries the 2025 problem F fixtures: lower bounds 2, 2, 1, 1 over m = 4 slots are feasible, 3, 2, 3 over m = 3 fails first at q = 1, no variety tolerates a single slot, 1, 2, 2 over m = 3 holds, and 2, 2, 2 over m = 3 fails at q = 1 again.

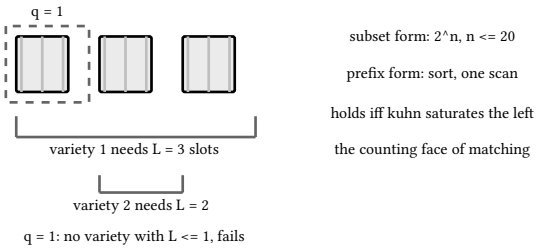


figure 346: the pot row with the lower-bound brackets of the failing family, the violated q = 1 prefix shaded

The application is icpc world finals 2025 problem F (book 10, chapter 13), herding cats, where variety lower bounds L(z) and the prefix sufficiency check are the whole solution. Hall has no dedicated cp-algorithms article, and the sources below cite the kuhn article where the theorem section lives.

38.10. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's nine sample files per language, c# spread over its five family files, go test files excluded:

language	build sloc	container dependency	boundary note
c	1118	static arc arrays, paired residuals	long long capacities, xor-1 twins, per-file mains printing ok N
go	563	slices, no imports beyond fmt and sort	shared helpers per package, the deterministic stoer-wagner scan documented
java	1156	jdk 27 stdlib	cap[e ^ 1] paired arcs port directly on int arc ids, the seeded lcg draws through Integer.remainderUnsigned, each file standalone like c
c#	465	lists, linq, tuples	five family files, the demands solver reuses the chapter dinic through accessors
javascript	385	classes with private fields	number capacities with the below-2^53 note, es modules importing dinic for demands
python	763	stdlib only, inline asserts	float("inf") as the flow infinity, brute twins on hungarian and the bipartite family
lua	970	tables, 1-based inside	0-based public faces, integer division for the infinity sentinels, check tables for run.lua

sources: cp-algorithms, "Maximum flow - Dinic's algorithm", cp-algorithms.com/graph/dinic.html, "Maximum flow - Push-relabel algorithm", cp-algorithms.com/graph/push-relabel.html, whose improved variant is named in prose only, "Minimum-cost flow", cp-algorithms.com/graph/min_cost_flow.html, "Flows with demands", cp-algorithms.com/graph/flow_with_demands.html, "Minimum cut - Stoer-Wagner algorithm", cp-algorithms.com/graph/stoer_wagner_mincut.html, "Kuhn's Algorithm - Maximum Bipartite Matching", cp-algorithms.com/graph/kuhn_maximum_bipartite_matching.html, where the hall's theorem section also lives, that article being the closest cp-algorithms carries to a dedicated hall page, "Hungarian algorithm", cp-algorithms.com/graph/hungarian-algorithm.html, and "Assignment problem", cp-algorithms.com/graph/Assignment-problem-min-flow.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Application sources: icpc world finals 2017 problems C and J, 2018 problem C, 2022 problem X, and 2025 problems B and F (book 10). Sample behavior verified by the seven suite gates scoped to chapter 38: c 9 files and 83 checks, go 26 test functions, java 9 files and 83 checks under run-java-samples, c# 28 facts, javascript 29 tests and 66 asserts across its two split files, python 9 files and 83 asserts, lua 33 checks, zero skipped.

39. graph connectivity and decomposition

The dfs of book 9, ch 10 (practical data structures and algorithms, graphs) answers reachability and counts components. This chapter asks the sharper questions the same walk cannot answer alone: which vertices are mutually reachable, what holds a graph together, what can be split by removing one vertex or one edge, and how a tree decomposes into structures that make path questions cheap. Kosaraju's two passes label the strongly connected components, disc and low values expose bridges and articulation points, degree parity decides eulerian walks, and four tree decompositions, ancestors by lifting, heavy paths, centroid layers, and the rings of functional graphs, turn hard path queries into short walks. The chapter closes with prufer codes, the second-best spanning tree, and robbins' orientation theorem in prose.

39.1. strongly connected components

Two passes, no recursion anywhere, the iterative house form of the chapter 10 dfs. The first pass walks the graph forward on an explicit stack of vertex-and-iterator frames and records vertices by finished time: a vertex is appended only when its entire subtree is done. The second pass consumes that order backwards over the reversed graph, flooding a fresh component label from each unfinished vertex. The reversal is the whole argument: within a strongly connected group, finish order survives the reversal, so the backward flood from the latest-finishing member captures exactly the group and nothing past it. Canonical labels rename each component by its smallest vertex index, condensation arcs are deduplicated and sorted, and the condensation itself is a dag.

The dry run: the fixture is the 8-vertex chain of cycles, arcs 0 to 1, 1 to 2, 2 back to 0, 2 to 3, 3 to 4, 4 to 5, 5 back to 3, 5 to 6, 6 to 7, 7 back to 6, asserted by all seven suites.

1. The first pass runs from 0 and pushes one spine, 0, 1, 2, 3, 4, 5, 6, 7: every arc leads onward, and the two closers, 2 to 0 and 5 to 3, both find their targets already seen.
2. Nothing new is reachable, so the unwind appends finished vertices deepest first: the order reads 7, 6, 5, 4, 3, 2, 1, 0.
3. The second pass consumes that order backwards over the reversed arcs, opening at 0: the flood pulls in 2 and then 1, raw component 0 claiming {0, 1, 2}.
4. The next unclaimed vertex is 3, whose flood pulls 5 and 4 for raw component 1, and 6 opens the last flood with 7 for raw 2.
5. Renaming by smallest member turns the raw ids into labels 0, 3, 6, and the two cross arcs dedupe to (0, 3) and (3, 6), a two-arc dag.

step	action	state
push	spine 0 through 7, closers seen	no arc left untried
unwind	append on finish, deepest first	order 7, 6, 5, 4, 3, 2, 1, 0
flood 0	reversed arcs pull 2, 1	raw 0 = {0, 1, 2}
flood 3	reversed arcs pull 5, 4	raw 1 = {3, 4, 5}
flood 6	reversed arcs pull 7	raw 2 = {6, 7}
relabel	smallest member per raw id	labels 0, 3, 6

The labels 0, 0, 0, 3, 3, 3, 6, 6 over three components with the arcs (0, 3) and (3, 6) are the pinned row, and the listings below run both passes in seven languages.

```c
static void kosaraju(int *count_out, int (*out_arcs)[2], int *oc) {
  int on = 0;
  memset(seen, 0, sizeof(int) * (size_t)nv);
  for (int s = 0; s < nv; s++) {
    if (seen[s])
      continue;
    int stk[MAXV], iter[MAXV], top = 0;
    stk[top] = s;
    iter[top] = ehead[s];
    top++;
    seen[s] = 1;
    while (top) {
      int u = stk[top - 1];
      if (iter[top - 1] >= 0) {
        int e = iter[top - 1];
        iter[top - 1] = enext[e];
        int v = eto[e];
        if (!seen[v]) {
          seen[v] = 1;
          stk[top] = v;
          iter[top] = ehead[v];
          top++;
        }
      } else {
        order[on++] = u;
        top--;
      }
    }
  }
  memset(comp, -1, sizeof(int) * (size_t)nv);
  int cid = 0;
  for (int i = on - 1; i >= 0; i--) {
    int s = order[i];
    if (comp[s] >= 0)
      continue;
    int queue[MAXV], qh = 0, qt = 0;
    comp[s] = cid;
    queue[qt++] = s;
    while (qh < qt) {
      int u = queue[qh++];
      for (int e = rhead[u]; e >= 0; e = rnext[e])
        if (comp[rto[e]] < 0) {
          comp[rto[e]] = cid;
          queue[qt++] = rto[e];
        }
    }
    cid++;
  }
```

listing 1391: c, the finish-order pass on explicit frames, the reversed-graph flood (source: dsa/samples-c/src/Ch39/scc.c, lines 50-97)

```
  for s := 0; s < n; s++ {
    if seen[s] {
      continue
    }
    type frame struct{ v, i int }
    stack := []frame{{s, 0}}
    seen[s] = true
    for len(stack) > 0 {
      fr := &stack[len(stack)-1]
      if fr.i < len(g[fr.v]) {
        v := g[fr.v][fr.i]
        fr.i++
        if !seen[v] {
          seen[v] = true
          stack = append(stack, frame{v, 0})
        }
      } else {
        order = append(order, fr.v)
        stack = stack[:len(stack)-1]
      }
    }
  }
  comp := make([]int, n)
  for i := range comp {
    comp[i] = -1
  }
  cid := 0
  for idx := len(order) - 1; idx ≥ 0; idx-- {
    s := order[idx]
    if comp[s] ≥ 0 {
      continue
    }
    queue := []int{s}
    comp[s] = cid
    for len(queue) > 0 {
      u := queue[0]
      queue = queue[1:]
      for _, v := range rg[u] {
        if comp[v] < 0 {
          comp[v] = cid
          queue = append(queue, v)
        }
      }
    }
    cid++
  }
```

listing 1392: go, the two passes over frame structs, component ids from the flood (source: dsa/samples-go/ch39/scc.go, lines 20-65)

```
  static void kosaraju() {
    int on = 0;
    java.util.Arrays.fill(seen, 0, nv, false);
    for (int s = 0; s < nv; s++) {
      if (seen[s])
        continue;
      int[] stk = new int[MAXV], iter = new int[MAXV];
      int top = 0;
      stk[top] = s;
      iter[top] = ehead[s];
      top++;
      seen[s] = true;
      while (top > 0) {
        int u = stk[top - 1];
        if (iter[top - 1] ≥ 0) {
          int e = iter[top - 1];
          iter[top - 1] = enext[e];
          int v = edgeTo[e];
          if (!seen[v]) {
            seen[v] = true;
            stk[top] = v;
            iter[top] = ehead[v];
            top++;
          }
        } else {
          order[on++] = u;
          top--;
        }
      }
    }
    java.util.Arrays.fill(comp, 0, nv, -1);
    int cid = 0;
    for (int i = on - 1; i ≥ 0; i--) {
      int s = order[i];
      if (comp[s] ≥ 0)
        continue;
      int[] queue = new int[MAXV];
      int qh = 0, qt = 0;
      comp[s] = cid;
      queue[qt++] = s;
      while (qh < qt) {
        int u = queue[qh++];
        for (int e = rhead[u]; e ≥ 0; e = rnext[e])
          if (comp[revTo[e]] < 0) {
            comp[revTo[e]] = cid;
            queue[qt++] = revTo[e];
          }
      }
      cid++;
    }
```

listing 1393: java, the finish-order pass on explicit frames, the reversed-graph flood, a line-for-line port of the c stack (source: dsa/samples-java/src/Ch39/Scc.java, lines 51-100)

```
        for (int s = 0; s < n; s++)
        {
            if (seen[s])
                continue;
            var stack = new Stack<(int Vertex, int Next)>();
            stack.Push((s, 0));
            seen[s] = true;
            while (stack.Count > 0)
            {
                (int u, int i) = stack.Pop();
                if (i < graph[u].Count)
                {
                    stack.Push((u, i + 1));
                    int v = graph[u][i];
                    if (!seen[v])
                    {
                        seen[v] = true;
                        stack.Push((v, 0));
                    }
                }
                else
                    order.Add(u); // finished
            }
        }
        var comp = new int[n];
        Array.Fill(comp, -1);
        int next = 0;
        for (int idx = order.Count - 1; idx ≥ 0; idx--)
        {
            int s = order[idx];
            if (comp[s] ≥ 0)
                continue;
            var queue = new Queue<int>();
            queue.Enqueue(s);
            comp[s] = next;
            while (queue.Count > 0)
            {
                int u = queue.Dequeue();
                foreach (int v in reversed[u])
                    if (comp[v] < 0)
                    {
                        comp[v] = next;
                        queue.Enqueue(v);
                    }
            }
            next++;
        }
        return comp;
    }
```

listing 1394: c#, raw components in condensation order, relabeled by smallest member above (source: dsa/samples/src/Ch39/Strong.cs, lines 43-91)

```
export function sccKosaraju(n, edges) {
  const g = Array.from({ length: n }, () => []);
  const rg = Array.from({ length: n }, () => []);
  for (const [u, v] of edges) {
    g[u].push(v);
    rg[v].push(u);
  }
  const order = [];
  const seen = new Array(n).fill(false);
  for (let s = 0; s < n; s++) {
    if (seen[s]) continue;
    const stack = [[s, 0]];
    seen[s] = true;
    while (stack.length > 0) {
      const [u, i] = stack[stack.length - 1];
      if (i < g[u].length) {
        stack[stack.length - 1][1] += 1;
        const v = g[u][i];
        if (!seen[v]) {
          seen[v] = true;
          stack.push([v, 0]);
        }
      } else {
        order.push(u);
        stack.pop();
      }
    }
  }
  const comp = new Array(n).fill(-1);
  let cid = 0;
  for (let i = order.length - 1; i ≥ 0; i--) {
    const s = order[i];
    if (comp[s] ≥ 0) continue;
    const q = [s];
    comp[s] = cid;
    while (q.length > 0) {
      const u = q.shift();
      for (const v of rg[u]) {
        if (comp[v] < 0) {
          comp[v] = cid;
          q.push(v);
        }
      }
    }
    cid += 1;
  }
```

listing 1395: javascript, the frame stack, the backward flood, raw ids (source: dsa/samples-js/src/ch39-scc.mjs, lines 8-53)

```python
def kosaraju(count, edges):
    graph = [[] for _ in range(count)]
    reversed_graph = [[] for _ in range(count)]
    for u, v in edges:
        graph[u].append(v)
        reversed_graph[v].append(u)
    order = []
    seen = [False] * count
    for start in range(count):  # pass 1: finish order, no recursion
        if seen[start]:
            continue
        seen[start] = True
        stack = [(start, 0)]
        while stack:
            u, i = stack.pop()
            if i < len(graph[u]):
                stack.append((u, i + 1))
                v = graph[u][i]
                if not seen[v]:
                    seen[v] = True
                    stack.append((v, 0))
            else:
                order.append(u)
    comp = [-1] * count
    comp_count = 0
    for start in reversed(order):  # pass 2: reversed graph
        if comp[start] ≥ 0:
            continue
        comp[start] = comp_count
        q = deque([start])
        while q:
            u = q.popleft()
            for v in reversed_graph[u]:
                if comp[v] < 0:
                    comp[v] = comp_count
                    q.append(v)
        comp_count += 1
    return comp, comp_count


def canon(count, edges):
    comp, comp_count = kosaraju(count, edges)
    members = {}
    for v, c in enumerate(comp):
        members.setdefault(c, []).append(v)
    remap = {c: min(members[c]) for c in range(comp_count)}  # smallest
    labels = [remap[c] for c in comp]
    arcs = {(labels[u], labels[v]) for u, v in edges if labels[u] ≠ labels[v]}
    return labels, len(set(labels)), sorted(arcs)
```

listing 1396: python, the two passes then the canonical relabel and condensation (source: dsa/samples-py/src/Ch39/scc.py, lines 17-65)

```
  -- first pass: order by finished time, explicit stack
  local order = {}
  local seen = {}
  for s0 = 0, n - 1 do
    if not seen[s0 + 1] then
      seen[s0 + 1] = true
      local stack = {{s0, 1}}
      while #stack > 0 do
        local top = stack[#stack]
        local u, i = top[1], top[2]
        local adj = g[u + 1]
        if i ≤ #adj then
          top[2] = i + 1
          local v = adj[i]
          if not seen[v + 1] then
            seen[v + 1] = true
            stack[#stack + 1] = {v, 1}
          end
        else
          order[#order + 1] = u
          stack[#stack] = nil
        end
      end
    end
  end
  -- second pass over the reversed graph
  local comp = {}
  for i = 0, n - 1 do comp[i + 1] = -1 end
  local cid = 0
  for oi = #order, 1, -1 do
    local s0 = order[oi]
    if comp[s0 + 1] < 0 then
      local q = {s0}
      local head = 1
      comp[s0 + 1] = cid
      while head ≤ #q do
        local u = q[head]
        head = head + 1
        for _, v in ipairs(rg[u + 1]) do
          if comp[v + 1] < 0 then
            comp[v + 1] = cid
            q[#q + 1] = v
          end
        end
      end
      cid = cid + 1
    end
  end
  return comp, cid
end
```

listing 1397: lua, the same two passes in 1-based tables (source: dsa/samples-lua/ch39_scc.lua, lines 20-69)

Tarjan's one-pass alternative with a live stack carries the same answer in one sweep and is named here for completeness, kosaraju is what ships because two simple passes beat one subtle one.

The fixture family runs one graph through all of it: 8 vertices with arcs 0 to 1, 1 to 2, 2 back to 0, 2 to 3, 3 to 4, 4 to 5, 5 back to 3, 5 to 6, 6 to 7, 7 back to 6. Labels land 0, 0, 0, 3, 3, 3, 6, 6, three components, condensation arcs

(0, 3) and (3, 6). A pure dag stays split, 0, 1, 2, 3 with the dag itself as its condensation, a bare 3-cycle collapses to one component with no condensation arcs, a lone vertex is one component, and two vertices with arcs both ways form one component. Java's two passes are a line-for-line port of the c frame stack, the iter arrays walking the same orbit, so the finish order and the labels pin without translation.

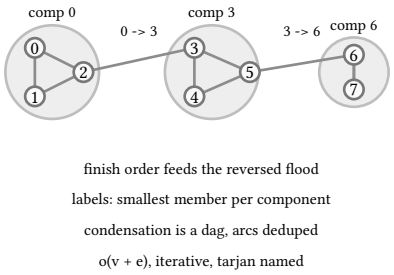


figure 347: the fixture graph with its three components as shaded blobs chained by the two condensation arcs

The application is general technique, the substrate under 2-sat below and every condensation-shaped dp.

39.2. 2-sat

A clause (a or b) is two promises: if a is false then b, and if b is false then a. Literal x lives at node 2v and its negation at 2v + 1, uniform across the seven languages, and each clause adds the two implication arcs, from the negation of one side into the other. One kosaraju run over that implication graph decides satisfiability: a variable whose two literals share a component is contradictory, the component forces x and not x together. Otherwise every variable takes the literal whose component sits later in the condensation order, the side with no outgoing path into its negation.

The dry run: the fixture is the forced formula (a) and (not a or not b) and (b or c), asserted by all seven suites, each brute-checking that a true, b false, c true is the only satisfying row.

1. Literals sit at 2v and 2v + 1: a at 0, not a at 1, b at 2, not b at 3, c at 4, not c at 5.
2. The clauses write five distinct arcs: (a) gives not a to a, (not a or not b) gives a to not b and b to not a, and (b or c) gives not b to c and not c to b.
3. The embedded kosaraju finishes c, not b, a, not a, b, not c in that order: the pass from 0 dies at c, and every later seed finds only visited targets.
4. Consumed backwards, each flood dies at its seed, so the raw components read a 3, not a 2, b 1, not b 4, c 5, not c 0, six singletons in condensation order.
5. No literal shares a component with its negation, so the formula is satisfiable, and later wins: 3 > 2 makes a true, 1 > 4 leaves b false, 5 > 0 makes c true.

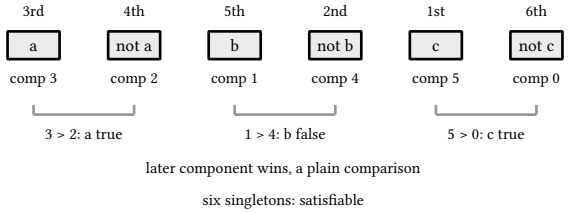


figure 348: the forced formula's run, finish rank above each literal, raw component below, the three comparisons deciding the assignment

The unique a true, b false, c true is the pinned row, and the listings below build the implication graph and read the assignment out in seven languages.

```
// -1 unsatisfiable; otherwise writes the assignment (1 = true)
static int twosat(int vars, int *val) {
  kosaraju();
  for (int v = 0; v < vars; v++)
    if (comp[2 * v] == comp[2 * v + 1])
      return -1;
  for (int v = 0; v < vars; v++)
    val[v] = comp[2 * v] > comp[2 * v + 1];
  return 1;
}

// clause evaluation for the brute check
static int sat_at(const int (*cl)[4], int k, const int *val) {
  for (int i = 0; i < k; i++) {
    int a = val[cl[i][0]] ^ cl[i][1];
    int b = val[cl[i][2]] ^ cl[i][3];
    if (!(a || b))
      return 0;
  }
  return 1;
}
```

listing 1398: c, the contradiction test and the later-component rule, the clause builder two arcs above (source: dsa/samples-c/src/Ch39/twosat.c, lines 105-125)

```
func TwoSat(nvars int, clauses []tsClause) (bool, []bool) {
  lit := func(v int, neg bool) int {
    if neg {
      return 2*v + 1
    }
    return 2 * v
  }
  n := 2 * nvars
  g := make([][]int, n)
  rg := make([][]int, n)
  var addEdge func(a, b int)
  addEdge = func(a, b int) {
    g[a] = append(g[a], b)
    rg[b] = append(rg[b], a)
  }
  for _, c := range clauses {
    a, b := lit(c.av, c.an), lit(c.bv, c.bn)
    addEdge(a^1, b)
    addEdge(b^1, a)
  }
  comp := kosarajuComponents(n, g, rg)
  for v := 0; v < nvars; v++ {
    if comp[2*v] == comp[2*v+1] {
      return false, nil
    }
  }
  val := make([]bool, nvars)
  for v := range val {
    val[v] = comp[2*v] > comp[2*v+1]
  }
  return true, val
}
```

listing 1399: go, literal encoding, the embedded kosaraju twin, the extraction (source: dsa/samples-go/ch39/twosat.go, lines 10-41)

```
  static int[] valOut = new int[8];

  // -1 unsatisfiable; otherwise writes the assignment (1 = true)
  static int twosat(int vars) {
    kosaraju();
    for (int v = 0; v < vars; v++)
      if (comp[2 * v] == comp[2 * v + 1])
        return -1;
    for (int v = 0; v < vars; v++)
      valOut[v] = comp[2 * v] > comp[2 * v + 1] ? 1 : 0;
    return 1;
  }

  // clause evaluation for the brute check
  static boolean satAt(int[][] cl, int[] val) {
    for (int[] c : cl) {
      int a = val[c[0]] ^ c[1];
      int b = val[c[2]] ^ c[3];
      if ((a | b) == 0)
        return false;
    }
    return true;
  }
```

listing 1400: java, the contradiction test and the later-component rule, the clause evaluator for the brute twin (source: dsa/samples-java/src/Ch39/Twosat.java, lines 107-129)

```
    public static Result Solve(
        int variables, ((int Var, bool Neg) A, (int Var, bool Neg) B)[] clauses)
    {
        var arcs = new List<(int, int)>();
        foreach (((int av, bool an), (int bv, bool bn)) in clauses)
        {
            int a = 2 * av + (an ? 1 : 0);
            int b = 2 * bv + (bn ? 1 : 0);
            arcs.Add((a ^ 1, b)); // not-a -> b
            arcs.Add((b ^ 1, a)); // not-b -> a
        }
        int[] comp = Scc.RawComponents(2 * variables, arcs.ToArray());
        var values = new bool[variables];
        for (int v = 0; v < variables; v++)
        {
            if (comp[2 * v] == comp[2 * v + 1])
                return new Result(false, null);
            values[v] = comp[2 * v] > comp[2 * v + 1]; // later component wins
        }
        return new Result(true, values);
    }
}
```

listing 1401: c#, the clause-to-arcs translation, the value extraction over raw ids (source: dsa/samples/src/Ch39/Strong.cs, lines 104-125)

```
export function twoSat(nvars, clauses) {
  // clauses as [varA, negatedA, varB, negatedB]
  const lit = (v, neg) => 2 * v + (neg ? 1 : 0);
  const edges = [];
  for (const [av, an, bv, bn] of clauses) {
    edges.push([lit(av, an) ^ 1, lit(bv, bn)]);
    edges.push([lit(bv, bn) ^ 1, lit(av, an)]);
  }
  const comp = kosaraju(2 * nvars, edges);
  for (let v = 0; v < nvars; v++) {
    if (comp[2 * v] === comp[2 * v + 1]) return { sat: false, values: null };
  }
  const values = Array.from({ length: nvars }, (_, v) =>
    comp[2 * v] > comp[2 * v + 1]);
  return { sat: true, values };
}
```

listing 1402: javascript, the clause walk, the share test, the values (source: dsa/samples-js/src/ch39-twosat.mjs, lines 55-70)

```
def twosat(clauses, variables):
    # clauses as (var_a, neg_a, var_b, neg_b) literal tuples
    def lit(var, neg):
        return 2 * var + (1 if neg else 0)

    edges = []
    for av, an, bv, bn in clauses:
        a, b = lit(av, an), lit(bv, bn)  # (a or b) twice implicated
        edges.append((a ^ 1, b))
        edges.append((b ^ 1, a))
    comp, _ = kosaraju(2 * variables, edges)
    values = []
    for v in range(variables):
        if comp[2 * v] == comp[2 * v + 1]:
            return False, None  # x shares a component with not-x
        # pick the side whose component finishes earlier
        values.append(comp[2 * v] > comp[2 * v + 1])
    return True, values


def sat_check(clauses, values):
    for av, an, bv, bn in clauses:
        if not ((values[av] ^ an) or (values[bv] ^ bn)):
            return False
    return True
```

listing 1403: python, the implication build, the extraction, the evaluator below (source: dsa/samples-py/src/Ch39/twosat.py, lines 57-81)

```lua
-- clauses as {var, negated, var, negated}
local function twosat(nvars, clauses)
  local lit = function(v, neg) return 2 * v + (neg and 1 or 0) end
  local arcs = {}
  for _, c in ipairs(clauses) do
    local a, b = lit(c[1], c[2]), lit(c[3], c[4])
    arcs[#arcs + 1] = {a ~ 1, b} -- not-a -> b
    arcs[#arcs + 1] = {b ~ 1, a} -- not-b -> a
  end
  local comp = kosaraju(2 * nvars, arcs)
  for v = 0, nvars - 1 do
    if comp[2 * v + 1] == comp[2 * v + 2] then return false, nil end
  end
  local val = {}
  for v = 0, nvars - 1 do
    val[v + 1] = comp[2 * v + 1] > comp[2 * v + 2]
  end
  return true, val
end

local function sat_check(clauses, val)
  for _, c in ipairs(clauses) do
    local a = (val[c[1] + 1] and 1 or 0) ~ (c[2] and 1 or 0)
    local b = (val[c[3] + 1] and 1 or 0) ~ (c[4] and 1 or 0)
    if a == 0 and b == 0 then return false end
  end
  return true
end
```

listing 1404: lua, the same build and extraction, the checker beneath (source: dsa/samples-lua/ch39_twosat.lua, lines 65-92)

Each standalone file embeds its own small kosaraju with raw component ids in condensation order, the trick that makes “later component wins” a plain integer comparison.

The four sign combinations of (a or b) together are unsatisfiable, every assignment falsifies one of them. The chain, (a) and (not a or b) and (not b or c), forces all true. The forced fixture, (a) and (not a or not b) and (b or c), pins the unique satisfying assignment a true, b false, c true, and the suites verify the uniqueness by brute enumeration before pinning it. The edges: (x or x) forces x true, and (x) with (not x) refuses.

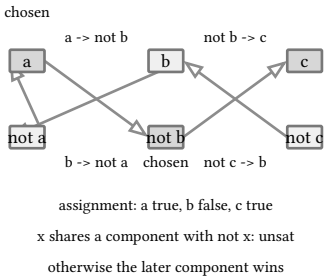


figure 349: the forced fixture’s implication graph with the component split and the truth choice per variable

The application is general technique, the solver behind scheduling with conflicts, seating arrangements, and every yes-or-no-per-variable question whose constraints come in pairs.

39.3. bridges and articulation points

One dfs carries two numbers per vertex: the discovery time disc, the tick when the vertex was first reached, and the low-link, the earliest disc reachable from the vertex’s subtree using at most one back edge. The classification happens on the unwind: a child whose low stays above its parent’s disc cannot reach behind the parent, so the

tree edge between them is a bridge, and a non-root vertex whose child's low reaches only as far as the parent's disc sits at the articulation hinge. The parent edge is skipped by edge id, not by endpoint, so a true parallel edge would count as a second way around, the fixtures carry none. The root is its own trap, it articulates exactly when it has two or more dfs children, the case the single-edge fixture pins.

The dry run: the fixture is two triangles joined by the single edge 2 to 3, asserted by all seven suites.

1. The timer opens at 1 on root 0 and the spine runs 0, 1, 2, 3, 4, 5, one discovery each: disc 1 through 6, low starting equal.
2. The closers are back edges: 2 to 0 drops low of 2 to min(3, 1) = 1, and 5 to 3 drops low of 5 to min(6, 4) = 4.
3. The unwind merges child lows into parents: low of 4 takes min(5, 4) = 4, low of 3 holds 4, and low of 2, 1, and 0 settle at 1.
4. Classifying as frames pop: low of 4 reads 4 >= disc of 3, so 3 articulates, and low of 3 reads 4 > disc of 2, so edge (2, 3) is the bridge and 2 articulates with it.
5. The root keeps one dfs child and stays out: the verdicts land bridge (2, 3) with articulation vertices 2 and 3.

vertex	disc	low	verdict
0	1	1	root, one dfs child, safe
1	2	1	plain
2	3	1	articulation
3	4	4	articulation
4	5	4	plain
5	6	4	plain

The lone bridge with both endpoints hinging is the pinned pair, and the listings below classify in seven languages.

```c
  for (int root = 0; root < nv; root++) {
    if (visited[root])
      continue;
    int rootchildren = 0;
    int stk[MAXV], pe[MAXV], it[MAXV], top = 0;
    stk[top] = root;
    pe[top] = -1;
    it[top] = ehead[root];
    top++;
    visited[root] = 1;
    disc[root] = low[root] = timer++;
    while (top) {
      int u = stk[top - 1];
      int adv = 0;
      for (; it[top - 1] >= 0; it[top - 1] = enext[it[top - 1]]) {
        int e = it[top - 1];
        int v = eto[e];
        if (e == pe[top - 1] || e == (pe[top - 1] ^ 1))
          continue; // the parent edge and its reverse
        if (!visited[v]) {
          if (u == root)
            rootchildren++;
          visited[v] = 1;
          disc[v] = low[v] = timer++;
          stk[top] = v;
          pe[top] = e;
          it[top] = ehead[v];
          top++;
          adv = 1;
          break;
        } else if (disc[v] < low[u])
          low[u] = disc[v];
      }
      if (adv)
        continue;
      // unwind: merge low into the parent, classify
      top--;
      if (top) {
        int p = stk[top - 1];
        if (low[u] < low[p])
          low[p] = low[u];
        if (low[u] > disc[p])
          bridge[pe[top]] = bridge[pe[top] ^ 1] = 1;
        if (p != root && low[u] >= disc[p])
          is_art[p] = 1;
      }
    }
    if (rootchildren >= 2)
      is_art[root] = 1;
  }
```

listing 1405: c, the iterative unwind merging low and classifying bridge and articulation (source: dsa/samples-c/src/Ch39/bridges.c, lines 56-105)

```
  for root := 0; root < n; root++ {
    if visited[root] {
      continue
    }
    rootChildren := 0
    stack := []frame{{root, -1, 0}}
    visited[root] = true
    disc[root], low[root] = timer, timer
    timer++
    for len(stack) > 0 {
      fr := &stack[len(stack)-1]
      if fr.i < len(g[fr.v]) {
        arc := g[fr.v][fr.i]
        fr.i++
        v := other(arc)
        if fr.parentArc ≥ 0 && arc/2 == fr.parentArc/2 {
          continue // the tree edge into this frame
        }
        if !visited[v] {
          if fr.v == root {
            rootChildren++
          }
          visited[v] = true
          disc[v], low[v] = timer, timer
          timer++
          stack = append(stack, frame{v, arc, 0})
        } else if disc[v] < low[fr.v] {
          low[fr.v] = disc[v]
        }
      } else {
        stack = stack[:len(stack)-1]
        if len(stack) > 0 {
          p := stack[len(stack)-1].v
          if low[fr.v] < low[p] {
            low[p] = low[fr.v]
          }
          if low[fr.v] > disc[p] {
            isBridge[fr.parentArc/2] = true
          }
          if p ≠ root && low[fr.v] ≥ disc[p] {
            isArt[p] = true
          }
        }
      }
    }
    if rootChildren ≥ 2 {
      isArt[root] = true
    }
  }
```

listing 1406: go, paired arcs per edge, the unwind classifying on the parent frame (source: dsa/samples-go/ch39/bridges.go, lines 34-82)

```
    for (int root = 0; root < nv; root++) {
      if (visited[root])
        continue;
      int rootchildren = 0;
      int[] stk = new int[MAXV], pe = new int[MAXV], it = new int[MAXV];
      int top = 0;
      stk[top] = root;
      pe[top] = -1;
      it[top] = ehead[root];
      top++;
      visited[root] = true;
      disc[root] = low[root] = timer++;
      while (top > 0) {
        int u = stk[top - 1];
        boolean adv = false;
        for (; it[top - 1] ≥ 0; it[top - 1] = enext[it[top - 1]]) {
          int e = it[top - 1];
          int v = edgeTo[e];
          if (e == pe[top - 1] || e == (pe[top - 1] ^ 1))
            continue; // the parent edge and its reverse
          if (!visited[v]) {
            if (u == root)
              rootchildren++;
            visited[v] = true;
            disc[v] = low[v] = timer++;
            stk[top] = v;
            pe[top] = e;
            it[top] = ehead[v];
            top++;
            adv = true;
            break;
          } else if (disc[v] < low[u])
            low[u] = disc[v];
        }
        if (adv)
          continue;
        // unwind: merge low into the parent, classify
        top--;
        if (top > 0) {
          int p = stk[top - 1];
          if (low[u] < low[p])
            low[p] = low[u];
          if (low[u] > disc[p]) {
            bridge[pe[top]] = true;
            bridge[pe[top] ^ 1] = true;
          }
          if (p ≠ root && low[u] ≥ disc[p])
            isArt[p] = 1;
        }
      }
      if (rootchildren ≥ 2)
        isArt[root] = 1;
    }
```

listing 1407: java, the iterative unwind merging low and classifying bridge and articulation (source: dsa/samples-java/src/Ch39/Bridges.java, lines 56-108)

```
        int timer = 1;
        for (int root = 0; root < n; root++)
        {
            if (visited[root])
                continue;
            int rootChildren = 0;
            var stack = new Stack<(int Vertex, int ParentEdge, List<(int V, int Id)>.Enumerator It)>();
            visited[root] = true;
            disc[root] = low[root] = timer++;
            stack.Push((root, -1, graph[root].GetEnumerator()));
            while (stack.Count > 0)
            {
                (int u, int parentEdge, var it) = stack.Peek();
                bool advanced = false;
                while (it.MoveNext())
                {
                    (int v, int id) = it.Current;
                    if (id == parentEdge)
                        continue; // skip the edge we arrived on
                    if (!visited[v])
                    {
                        if (u == root)
                            rootChildren++; // the classic root trap
                        visited[v] = true;
                        disc[v] = low[v] = timer++;
                        stack.Push((v, id, graph[v].GetEnumerator()));
                        advanced = true;
                        break;
                    }
                    low[u] = Math.Min(low[u], disc[v]); // back edge
                }
                if (advanced)
                    continue;
                (int popped, int poppedEdge, _) = stack.Pop(); // unwind and classify
                if (stack.Count > 0)
                {
                    (int p, _, _) = stack.Peek();
                    low[p] = Math.Min(low[p], low[popped]);
                    if (low[popped] > disc[p])
                        isBridge[poppedEdge] = true; // the edge the popped frame arrived on
                    if (p != root && low[popped] ≥ disc[p])
                        isArticulation[p] = true;
                }
            }
            if (rootChildren ≥ 2)
                isArticulation[root] = true;
        }
```

listing 1408: c#, the same unwind, enumerators on the frame stack, the root child count (source: dsa/samples/src/Ch39/Edges.cs, lines 24-70)

```
  for (let root = 0; root < n; root++) {
    if (visited[root]) continue;
    let rootChildren = 0;
    const stack = [[root, -1, 0]];
    visited[root] = true;
    disc[root] = low[root] = timer;
    timer += 1;
    while (stack.length > 0) {
      const frame = stack[stack.length - 1];
      const [u, pe] = frame;
      let advanced = false;
      while (frame[2] < g[u].length) {
        const [v, ei] = g[u][frame[2]];
        frame[2] += 1;
        if (ei === pe) continue; // the parent edge itself
        if (!visited[v]) {
          if (u === root) rootChildren += 1;
          visited[v] = true;
          disc[v] = low[v] = timer;
          timer += 1;
          stack.push([v, ei, 0]);
          advanced = true;
          break;
        }
        low[u] = Math.min(low[u], disc[v]);
      }
      if (!advanced) {
        stack.pop();
        if (stack.length > 0) {
          const p = stack[stack.length - 1][0];
          low[p] = Math.min(low[p], low[u]);
          if (low[u] > disc[p]) bridge[pe] = true;
          if (p !== root && low[u] ≥ disc[p]) isArt[p] = true;
        }
      }
    }
    if (rootChildren ≥ 2) isArt[root] = true;
  }
```

listing 1409: javascript, the back-edge min, the unwind, the root rule (source: dsa/samples-js/src/ch39-bridges.mjs, lines 20-57)

```
    for root in range(count):
        if visited[root]:
            continue
        root_children = 0
        visited[root] = True
        disc[root] = low[root] = timer
        timer += 1
        stack = [(root, -1, 0)]  # vertex, parent edge, next index
        while stack:
            u, parent_edge, i = stack[-1]
            advanced = False
            while i < len(graph[u]):
                v, edge_id = graph[u][i]
                i += 1
                if edge_id == parent_edge:  # skip the tree edge up
                    continue
                if not visited[v]:
                    if u == root:
                        root_children += 1
                    visited[v] = True
                    disc[v] = low[v] = timer
                    timer += 1
                    stack[-1] = (u, parent_edge, i)
                    stack.append((v, edge_id, 0))
                    advanced = True
                    break
                low[u] = min(low[u], disc[v])  # back edge
            if advanced:
                continue
            stack.pop()  # unwind: merge low, classify
            if stack:
                p = stack[-1][0]
                low[p] = min(low[p], low[u])
                if low[u] > disc[p]:
                    bridge[parent_edge] = True
                if p != root and low[u] >= disc[p]:
                    is_artic[p] = True
        if root_children >= 2:  # the classic root trap
            is_artic[root] = True
```

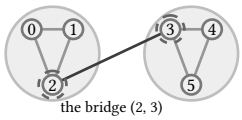
listing 1410: python, frames as tuples, the unwind with both classifications (source: dsa/samples-py/src/Ch39/bridges.py, lines 27-65)

```
  for root = 0, n - 1 do
    if not visited[root + 1] then
      local rootchildren = 0
      visited[root + 1] = true
      disc[root + 1] = timer
      low[root + 1] = timer
      timer = timer + 1
      -- stack entries: vertex, parent edge id, next adjacency index
      local stack = {{root, -1, 1}}
      while #stack > 0 do
        local top = stack[#stack]
        local u, pe, i = top[1], top[2], top[3]
        local adj = g[u + 1]
        local adv = false
        while i ≤ #adj do
          local nb = adj[i]
          local v, ei = nb[1], nb[2]
          i = i + 1
          if ei ~= pe then
            if not visited[v + 1] then
              if u == root then rootchildren = rootchildren + 1 end
              visited[v + 1] = true
              disc[v + 1] = timer
              low[v + 1] = timer
              timer = timer + 1
              top[3] = i
              stack[#stack + 1] = {v, ei, 1}
              adv = true
              break
            else
              if disc[v + 1] < low[u + 1] then
                low[u + 1] = disc[v + 1]
              end
            end
          end
        end
        if not adv then
          top[3] = i
          stack[#stack] = nil
          if #stack > 0 then -- unwind: merge low, classify
            local p = stack[#stack][1]
            if low[u + 1] < low[p + 1] then low[p + 1] = low[u + 1] end
            if low[u + 1] > disc[p + 1] then bridge[pe] = true end
            if p ~= root and low[u + 1] ≥ disc[p + 1] then
              is_art[p + 1] = true
            end
          end
        end
      end
      if rootchildren ≥ 2 then is_art[root + 1] = true end
    end
  end
```

listing 1411: lua, the same unwind over table frames (source: dsa/samples-lua/ch39_bridges.lua, lines 27-78)

Two triangles sharing vertex 0 have no bridges and exactly one articulation vertex, 0. Two triangles joined by the single bridge 2 to 3 report the bridge and both of its endpoints as articulation points. A path of four vertices makes

every edge a bridge and both interior vertices articulation points, and a single edge is a bridge with no articulation vertex at all, the root trap: the root has one child.



the bridge (2, 3)

bridge: low child > disc parent

articulation: low child >= disc parent

root: two or more dfs children

the ringed 2 appears on both sides

figure 350: two triangles joined by the bridge 2 to 3, the bridge cut, the two biconnected blobs, both articulation vertices ringed

The application is icpc world finals 2022 problem R (book 10, chapter 11), zoo management, which drops every bridge and solves each 2-edge-connected component alone. The online variant, bridges under edge insertion via link-cut machinery, is one prose paragraph in the sources and stays out of the code.

39.4. eulerian paths

Degree parity is the whole decision. A connected multigraph with exactly two odd-degree vertices has an open eulerian path between them, zero odd vertices has a closed circuit, and anything else has no walk at all, konigsberg's four lands with degrees 5, 3, 3, 3 included. Hierholzer's construction walks greedily from the canonical start, the smallest odd vertex or else the smallest non-isolated one, always taking the smallest-indexed unused edge by multigraph edge id, and appends the current vertex to the walk when stuck. The stack order reversed is the walk, and the smallest-next rule makes it identical in all seven languages.

The dry run: the fixture is the triangle 0, 1, 2 plus the tail 2 to 3, 3 to 4, asserted by all seven suites.

1. Degrees read 2, 2, 3, 2, 1: exactly two odd vertices, 2 and 4, so an open path exists and the canonical start is the smaller, 2.
2. The stack opens at 2 and the smallest-unused rule walks 0, 1, back to 2, then 3, then 4: the stack reads 2, 0, 1, 2, 3, 4.
3. Vertex 4 has spent its only edge and is stuck, and so is every frame beneath it once control returns: the pops append 4, 3, 2, 1, 0, 2.
4. Reversing the append order reads the walk 2, 0, 1, 2, 3, 4, five edges, each spent exactly once.

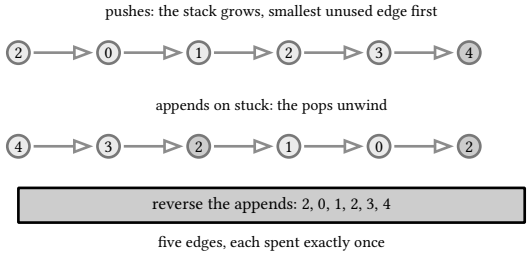


figure 351: the run as two sequences, the stack growing under the smallest-unused rule above, the stuck-append unwind below, the reverse landing the walk

The walk 2, 0, 1, 2, 3, 4 is the pinned row, and the listings below trace it in seven languages.

```
static int eulerian(void) {
  nodd = 0;
  for (int v = 0; v < nv; v++)
    if (deg[v] % 2 == 1)
      odd[nodd++] = v;
  if (nodd != 0 && nodd != 2)
    return 0;
  walkn = 0;
  if (ec == 0) {
    walk[walkn++] = 0; // a lone vertex walks itself
    return 1;
  }
  int start = nodd ? odd[0] : 0;
  while (deg[start] == 0)
    start++;
  memset(used, 0, sizeof(int) * (size_t)ec);
  int stk[MAXE + 1], top = 0;
  stk[top++] = start;
  while (top) {
    int u = stk[top - 1];
    int found = -1, bestv = nv;
    for (int e = adj_head[u]; e ≥ 0; e = adj_next[e]) {
      int id = adj_id[e];
      if (used[id])
        continue;
      int v = eu[id] == u ? ev[id] : eu[id];
      if (v < bestv) { // the smallest-indexed unused neighbor
        bestv = v;
        found = id;
      }
    }
    if (found < 0) {
      walk[walkn++] = u;
      top--;
    } else {
      used[found] = 1;
      stk[top++] = bestv;
    }
  }
  for (int i = 0, j = walkn - 1; i < j; i++, j--) {
    int t = walk[i];
    walk[i] = walk[j];
    walk[j] = t;
  }
  return 1;
}
```

listing 1412: c, parity and start, the stack loop taking the smallest unused edge, the reverse (source: dsa/samples-c/src/Ch39/euler.c, lines 54-99)

```
  other := func(arc int) int {
    if arc%2 == 0 {
      return edges[arc/2][1]
    }
    return edges[arc/2][0]
  }
  used := make([]bool, len(edges))
  walk := []int{}
  stack := []int{start}
  for len(stack) > 0 {
    u := stack[len(stack)-1]
    arcs := append([]int(nil), adj[u]...)
    sort.Slice(arcs, func(i, j int) bool {
      if other(arcs[i]) != other(arcs[j]) {
        return other(arcs[i]) < other(arcs[j])
      }
      return arcs[i] < arcs[j]
    })
    moved := false
    for _, arc := range arcs {
      if !used[arc/2] {
        used[arc/2] = true
        stack = append(stack, other(arc))
        moved = true
        break
      }
    }
    if !moved {
      walk = append(walk, u)
      stack = stack[:len(stack)-1]
    }
  }
  for i, j := 0, len(walk)-1; i < j; i, j = i+1, j-1 {
    walk[i], walk[j] = walk[j], walk[i]
  }
  return walk, true
```

listing 1413: go, the paired arc ids, the sorted adjacency, the loop and reverse (source: dsa/samples-go/ch39/euler.go, lines 44-79)

```
  static boolean eulerian() {
    nodd = 0;
    for (int v = 0; v < nv; v++)
      if (deg[v] % 2 == 1)
        odd[nodd++] = v;
    if (nodd != 0 && nodd != 2)
      return false;
    walkN = 0;
    if (ec == 0) {
      walk[walkN++] = 0; // a lone vertex walks itself
      return true;
    }
    int start = nodd > 0 ? odd[0] : 0;
    while (deg[start] == 0)
      start++;
    java.util.Arrays.fill(used, 0, ec, false);
    int[] stk = new int[MAXE + 1];
    int top = 0;
    stk[top++] = start;
    while (top > 0) {
      int u = stk[top - 1];
      int found = -1, bestv = nv;
      for (int e = adjHead[u]; e ≥ 0; e = adjNext[e]) {
        int id = adjId[e];
        if (used[id])
          continue;
        int v = eu[id] == u ? ev[id] : eu[id];
        if (v < bestv) { // the smallest-indexed unused neighbor
          bestv = v;
          found = id;
        }
      }
      if (found < 0) {
        walk[walkN++] = u;
        top--;
      } else {
        used[found] = true;
        stk[top++] = bestv;
      }
    }
    for (int i = 0, j = walkN - 1; i < j; i++, j--) {
      int t = walk[i];
      walk[i] = walk[j];
      walk[j] = t;
    }
    return true;
  }
```

listing 1414: java, parity and start, the stack loop taking the smallest unused edge, the reverse (source: dsa/samples-java/src/Ch39/Euler.java, lines 55-101)

```
        var odd = Enumerable.Range(0, n).Where(v => degree[v] % 2 == 1).ToList();
        if (odd.Count is not 0 and not 2)
            return null; // konigsberg: no walk
        if (edges.Length == 0)
            return new List<int> { 0 }; // the lone vertex walks alone
        int start = odd.Count == 2
            ? odd.Min()
            : Enumerable.Range(0, n).First(v => degree[v] > 0);
        var used = new bool[edges.Length];
        var walk = new List<int>();
        var stack = new Stack<int>();
        stack.Push(start);
        while (stack.Count > 0)
        {
            int u = stack.Peek();
            (int V, int Id)? next = null;
            foreach ((int v, int id) in adjacency[u].OrderBy(t => t.V))
                if (!used[id])
                {
                    next = (v, id); // smallest-indexed unused edge
                    break;
                }
            if (next is null)
            {
                walk.Add(u); // stuck: append and pop
                stack.Pop();
            }
            else
            {
                used[next.Value.Id] = true;
                stack.Push(next.Value.V);
            }
        }
        walk.Reverse();
        return walk;
    }
```

listing 1415: c#, the odd count, the canonical start, the stuck-append loop (source: dsa/samples/src/Ch39/Edges.cs, lines 99-134)

```
  const odd = Array.from({ length: n }, (_, v) => v)
    .filter((v) => deg[v] % 2 === 1);
  if (odd.length !== 0 && odd.length !== 2) {
    return { walk: null, odd };
  }
  if (edges.length === 0) return { walk: [0], odd };
  let start = odd.length > 0 ? odd[0] : 0;
  if (odd.length === 0) {
    while (deg[start] === 0) start += 1;
  }
  const used = new Array(edges.length).fill(false);
  const walk = [];
  const stack = [start];
  while (stack.length > 0) {
    const u = stack[stack.length - 1];
    let found = false;
    for (const [v, ei] of adj[u]) {
      if (!used[ei]) {
        used[ei] = true;
        stack.push(v);
        found = true;
        break;
      }
    }
    if (!found) {
      walk.push(u);
      stack.pop();
    }
  }
  walk.reverse();
  return { walk, odd };
```

listing 1416: javascript, parity, start, the pre-sorted neighbors, the reverse (source: dsa/samples-js/src/ch39-euler.mjs, lines 20-50)

```python
def eulerian(count, edges):
    degree = [0] * count
    graph = [[] for _ in range(count)]
    for i, (u, v) in enumerate(edges):
        graph[u].append((v, i))
        graph[v].append((u, i))
        degree[u] += 1
        degree[v] += 1
    odd = [v for v in range(count) if degree[v] % 2 == 1]
    if len(odd) not in (0, 2):
        return None, odd
    if not edges:
        return [0], odd
    # canonical start: smallest odd vertex, else smallest non-isolated
    start = odd[0] if odd else min(v for v in range(count) if degree[v])
    used = [False] * len(edges)
    walk = []
    stack = [start]
    while stack:
        u = stack[-1]
        found = False
        for v, edge_id in sorted(graph[u]):  # smallest next, canonical
            if not used[edge_id]:
                used[edge_id] = True
                stack.append(v)
                found = True
                break
        if not found:  # stuck: this vertex closes the walk
            walk.append(u)
            stack.pop()
    walk.reverse()
    return walk, odd
```

listing 1417: python, the whole walk, sorted next-hops, append on stuck (source: dsa/samples-py/src/Ch39/euler.py, lines 15-46)

```
  local odd = {}
  for v = 0, n - 1 do
    if deg[v + 1] % 2 == 1 then odd[#odd + 1] = v end
  end
  if #odd ~= 0 and #odd ~= 2 then return nil, odd, deg end
  if #edges == 0 then return {0}, odd, deg end
  local start = odd[1]
  if start == nil then
    for v = 0, n - 1 do
      if deg[v + 1] > 0 then start = v break end
    end
  end
  local used = {}
  for i = 1, #edges do used[i] = false end
  for i = 0, n - 1 do
    table.sort(adj[i + 1], function(a, b)
      if a[1] ~= b[1] then return a[1] < b[1] end
      return a[2] < b[2]
    end)
  end
  local walk, stack = {}, {start}
  local ptr = {}
  for i = 0, n - 1 do ptr[i + 1] = 1 end
  while #stack > 0 do
    local u = stack[#stack]
    local adj_u = adj[u + 1]
    local found = false
    while ptr[u + 1] ≤ #adj_u do
      local nb = adj_u[ptr[u + 1]]
      if not used[nb[2]] then
        used[nb[2]] = true
        stack[#stack + 1] = nb[1]
        found = true
        break -- smallest-indexed unused edge first
      end
      ptr[u + 1] = ptr[u + 1] + 1
    end
    if not found then
      walk[#walk + 1] = u
      stack[#stack] = nil
    end
  end
  -- reversed, the stuck-order append becomes the walk
  for i = 1, #walk // 2 do
    walk[i], walk[#walk - i + 1] = walk[#walk - i + 1], walk[i]
  end
  return walk, odd, deg
```

listing 1418: lua, parity and start, per-vertex pointers, the reverse (source: dsa/samples-lua/ch39_euler.lua, lines 22-68)

Konigsberg refuses with all four lands odd. The triangle 0, 1, 2 plus the tail 2 to 3, 3 to 4 has odd vertices 2 and 4, and the canonical walk reads 2, 0, 1, 2, 3, 4. The bare triangle closes the circuit 0, 1, 2, 0 from vertex 0. The edges: a lone vertex with no edges walks itself as the single vertex 0, parallel edges make a two-cycle circuit, and an isolated vertex stays out of the walk entirely.

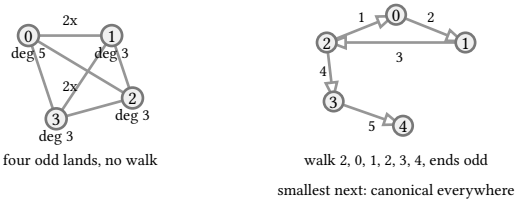


figure 352: the konigsberg graph with its four odd lands beside the triangle-plus-tail walk numbered by step

The application is general technique, the mail-carrier and circuit-tracing family, and the parity gate is always checked before any walking starts.

39.5. lowest common ancestors

Two solvers share one file. Binary lifting builds a parent table by bfs, children visited in ascending index order so every derived array is identical across languages, then doubles it: level k of the table holds the 2^k-th ancestor, each level a plain lookup into the one below. A query lifts the deeper node by the bits of the depth difference, then walks both nodes down from the top level while their ancestors differ, converging one step above the answer. Tarjan's offline solver is the other half: a dfs with a dsu, where each vertex merges into its parent on return and stamps the component's ancestor, and a query whose other endpoint is already finished resolves at that moment through find. Both assert all nine fixtures of the pinned tree, and the rmq equivalence, euler tour plus the sparse table of book 9, ch 21 (practical data structures and algorithms, range queries), is the prose third way.

The dry run: the fixture is the pinned 9-vertex tree, asserted by all seven suites; the walk uses the 0-based labels the suites store, one below the book's reading.

1. The bfs visits 0 through 8 with children ascending: parent of 7 is 4, depth of 7 and 8 is 3, the pinned structural row.
2. Level 0 of the table is the parent row 0, 0, 0, 1, 1, 2, 2, 4, 4 with the root lifting to itself, and one doubling moves only 7 and 8, to 1: level 1 reads 0, 0, 0, 0, 0, 0, 0, 1, 1.
3. The query (6, 9) of the fixture family is (5, 8) here at depths 2 and 3: the deeper 8 aligns by one level-0 lift onto 4.
4. Now 4 and 5 sit level: the 2-jumps agree, both 0, and only the single steps differ, 1 against 2, so both lift to the root's children.
5. The meeting point is their parent: 0 here, vertex 1 in the book's labels, the pinned answer.

step	action	lands
bfs	children ascending from root 0	parent 4 for 7, depths 3 and 3
level 0	parent row, root self-lift	0, 0, 0, 1, 1, 2, 2, 4, 4
level 1	one doubling	0, 0, 0, 0, 0, 0, 0, 1, 1
align	8 lifts one, depth 3 to 2	8 lands on 4
diverge	2-jumps agree, singles differ	1 against 2
answer	parent of the pair	0, book vertex 1

The meeting at 1 is one of nine rows both solvers pin, and the listings below lift and walk offline in seven languages.

```c
static int lca_lift(int u, int v) {
  if (depthv[u] < depthv[v]) {
    int t = u;
    u = v;
    v = t;
  }
  int diff = depthv[u] - depthv[v];
  for (int k = 0; k < LOG; k++)
    if (diff & (1 << k))
      u = up[k][u];
  if (u == v)
    return u;
  for (int k = LOG - 1; k >= 0; k--)
    if (up[k][u] != up[k][v]) {
      u = up[k][u];
      v = up[k][v];
    }
  return parent[u];
}

// tarjan offline with a dsu: queries resolved on dfs return
static int dsu_p[MAXN], anc[MAXN];
static int q_u[16], q_v[16], q_a[16], qn;
static int visited2[MAXN];

static int find(int i) {
  return dsu_p[i] == i ? i : (dsu_p[i] = find(dsu_p[i]));
}

static void tarjan(int u) {
  dsu_p[u] = u;
  anc[u] = u;
  for (int e = adjh[u]; e >= 0; e = adjn[e]) {
    int v = adjv[e];
    if (v == parent[u])
      continue;
    tarjan(v);
    dsu_p[find(v)] = u;
    anc[find(u)] = u;
  }
  visited2[u] = 1;
  for (int i = 0; i < qn; i++) {
    if (q_u[i] == u && visited2[q_v[i]])
      q_a[i] = anc[find(q_v[i])];
    if (q_v[i] == u && visited2[q_u[i]])
      q_a[i] = anc[find(q_u[i])];
  }
}
```

listing 1419: c, the lift query, then the tarjan pass with its dsu (source: dsa/samples-c/src/Ch39/lca.c, lines 90-137)

```
// LCA answers for 1-indexed vertices: lift the deeper node then
// binary search the split.
func (l *LiftLCA) LCA(u, v int) int {
  u, v = u-1, v-1
  if l.depth[u] < l.depth[v] {
    u, v = v, u
  }
  diff := l.depth[u] - l.depth[v]
  for k := 0; diff > 0; k++ {
    if diff&1 == 1 {
      u = l.up[k][u]
    }
    diff >>= 1
  }
  if u == v {
    return u + 1
  }
  for k := len(l.up) - 1; k ≥ 0; k-- {
    if l.up[k][u] ≠ l.up[k][v] {
      u = l.up[k][u]
      v = l.up[k][v]
    }
  }
  return l.parent[u] + 1
}
```

listing 1420: go, the lift query, the tarjan twin lives below in the same file (source: dsa/samples-go/ch39/lca.go, lines 81-105)

```
static int lcaLift(int u, int v) {
  if (depthv[u] < depthv[v]) {
    int t = u;
    u = v;
    v = t;
  }
  int diff = depthv[u] - depthv[v];
  for (int k = 0; k < LOG; k++)
    if ((diff & (1 << k)) != 0)
      u = up[k][u];
  if (u == v)
    return u;
  for (int k = LOG - 1; k >= 0; k--)
    if (up[k][u] != up[k][v]) {
      u = up[k][u];
      v = up[k][v];
    }
  return parent[u];
}

// tarjan offline with a dsu: queries resolved on dfs return
static int[] dsuP = new int[MAXN], ancestorOf = new int[MAXN];
static int[] qu = new int[16], qv = new int[16], qa = new int[16];
static int qn;
static boolean[] visited2 = new boolean[MAXN];

static int find(int i) {
  if (dsuP[i] != i)
    dsuP[i] = find(dsuP[i]);
  return dsuP[i];
}

static void tarjan(int u) {
  dsuP[u] = u;
  ancestorOf[u] = u;
  for (int e = adjh[u]; e >= 0; e = adjn[e]) {
    int v = adjv[e];
    if (v == parent[u])
      continue;
    tarjan(v);
    dsuP[find(v)] = u;
    ancestorOf[find(u)] = u;
  }
  visited2[u] = true;
  for (int i = 0; i < qn; i++) {
    if (qu[i] == u && visited2[qv[i]])
      qa[i] = ancestorOf[find(qv[i])];
    if (qv[i] == u && visited2[qu[i]])
      qa[i] = ancestorOf[find(qu[i])];
  }
}
```

listing 1421: java, the lift query, then the tarjan pass with its dsu (source: dsa/samples-java/src/Ch39/Lca.java, lines 85-135)

```
        _maxLog = 1;
        while (1 << _maxLog < n)
            _maxLog++;
        _up = new int[_maxLog + 1][];
        _up[0] = Parent.ToArray();
        for (int u = 0; u < n; u++)
            if (_up[0][u] < 0)
                _up[0][u] = u; // the root lifts to itself
        for (int k = 1; k ≤ _maxLog; k++)
        {
            _up[k] = new int[n];
            for (int v = 0; v < n; v++)
                _up[k][v] = _up[k - 1][_up[k - 1][v]];
        }
    }

    public int Lift(int u, int v)
    {
        if (Depth[u] < Depth[v])
            (u, v) = (v, u);
        for (int k = _maxLog; k ≥ 0; k--)
            if (Depth[u] - (1 << k) ≥ Depth[v])
                u = _up[k][u];
        if (u == v)
            return u;
        for (int k = _maxLog; k ≥ 0; k--)
            if (_up[k][u] ≠ _up[k][v])
            {
                u = _up[k][u];
                v = _up[k][v];
            }
        return Parent[u];
    }
```

listing 1422: c#, the doubling table above, the lift query, depths and split search (source: dsa/samples/src/Ch39/Paths2.cs, lines 49-81)

```
export function buildLca(n, tree, root) {
  const { g, parent, depth, order } = buildGraph(n, tree, root);
  const LOG = 32 - Math.clz32(n) || 1;
  const up = Array.from({ length: LOG }, () => new Array(n + 1).fill(0));
  up[0] = [...parent];
  for (let k = 1; k < LOG; k++) {
    for (let v = 1; v ≤ n; v++) {
      up[k][v] = up[k - 1][up[k - 1][v]];
    }
  }
  const lca = (a, b) => {
    let u = a;
    let v = b;
    if (depth[u] < depth[v]) [u, v] = [v, u];
    const diff = depth[u] - depth[v];
    for (let k = 0; k < LOG; k++) {
      if ((diff >> k) & 1) u = up[k][u];
    }
    if (u === v) return u;
    for (let k = LOG - 1; k ≥ 0; k--) {
      if (up[k][u] !== up[k][v]) {
        u = up[k][u];
        v = up[k][v];
      }
    }
    return parent[u];
  };
  return { lca, parent, depth, order, graph: g };
}
```

listing 1423: javascript, the doubling build and the lift closure (source: dsa/samples-js/src/ch39-lca.mjs, lines 39-67)

```
parent, depth, order = build()
LOG = max(1, max(depth).bit_length())
up = [parent[:]]  # up[k][v]: 2^k-th ancestor
for k in range(1, LOG):
    prev = up[k - 1]
    up.append([prev[prev[v]] if prev[v] ≥ 0 else -1 for v in range(N)])


def lca_lifting(u, v):
    if depth[u] < depth[v]:
        u, v = v, u
    diff = depth[u] - depth[v]  # lift the deeper node level
    for k in range(LOG):
        if diff >> k & 1:
            u = up[k][u]
    if u == v:
        return u
    for k in range(LOG - 1, -1, -1):  # binary search the split
        if up[k][u] != up[k][v]:
            u = up[k][u]
            v = up[k][v]
    return parent[u]
```

listing 1424: python, the table, the lift, the split search (source: dsa/samples-py/src/Ch39/lca.py, lines 55-76)

```
local function make_lifting()
  local par, depth = build()
  local LOG = 1
  while (1 << LOG) < N do LOG = LOG + 1 end
  local up = {}
  up[0] = par
  for k = 1, LOG do
    up[k] = {}
    for v = 1, N do
      local p = up[k - 1][v]
      up[k][v] = (p == -1) and -1 or up[k - 1][p]
    end
  end
  return function(u, v)
    if depth[u] < depth[v] then u, v = v, u end
    local diff = depth[u] - depth[v]
    local k = 0
    while diff > 0 do
      if diff & 1 == 1 then u = up[k][u] end
      diff = diff >> 1
      k = k + 1
    end
    if u == v then return u end
    for k = LOG, 0, -1 do
      if up[k][u] ~= up[k][v] then
        u = up[k][u]
        v = up[k][v]
      end
    end
    return par[u]
  end
end
```

listing 1425: lua, make lifting returns the query closure, doubling inside (source: dsa/samples-lua/ch39_lca.lua, lines 45-76)

The pinned tree is 9 vertices rooted at 1 with edges 1-2 (5), 1-3 (7), 2-4 (3), 2-5 (9), 3-6 (2), 3-7 (8), 5-8 (4), 5-9 (6). The structural family pins the bfs order 1 through 9, the parent of 8 as 5, and the depth of 8 and 9 as 3. The query family, both solvers on every row: (4,8) meets at 2, (6,9) at 1, (8,8) at itself, (4,1) at 1, (5,7) at 1, (8,9) at 5, (4,6) at 1, (2,5) at 2, (7,9) at 1.

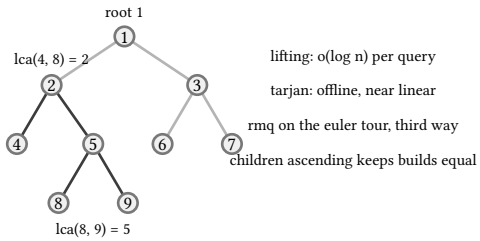


figure 353: the pinned 9-node tree with the 4 to 8 and 8 to 9 paths shaded up to their meeting points

The application is general technique, the meeting point under every distance-on-tree question, and the speedup the next two sections assume.

39.6. heavy-light decomposition

Split the tree into vertical chains. One dfs computes subtree sizes and marks each vertex's heavy child, the child with the largest subtree, ties to the smaller index. A second walk assigns every vertex a chain head and a base position, heavy child first, so each chain occupies one contiguous run of the base array. Any root-to-vertex path

crosses at most one light edge per chain change, and the chains it rides are contiguous, so a path query decomposes into O(log n) contiguous base ranges, each answered by a segment tree, here the point-update max shape of book 9, ch 21 (practical data structures and algorithms, range queries) rebuilt in file. The walk goes head to head, the deeper head first, taking the max edge weight over each crossed chain's run.

The dry run: the fixture is the same pinned tree in the 0-based labels the suites store, asserted by all seven suites.

1. The size pass picks heavy children: 1 at the root with subtree 5 against 2′s 3, 4 at 1 with 3 against 3′s 1, and the size-1 ties at 2 and 4 resolve to the smaller index, 5 and 7.
2. The second walk goes heavy child first: the root chain 0, 1, 4, 7 takes positions 0, 1, 2, 3, vertex 8 opens its own lane at 4, 3 at 5, and the 2, 5 chain closes at 6, 7 with 6 at 8.
3. Heads read 0, 0, 2, 3, 0, 2, 6, 0, 8 and the base visit order 0, 1, 4, 7, 8, 3, 2, 5, 6, both arrays pinned.
4. The query (7, 9) of the fixture family is (6, 8) here: it rides four runs, 8′s own edge at 6, 6′s at 8, 2′s at 7, then the shared lane's max(5, 9).
5. The best over the runs is 9, and the walk never touches a fifth range.

chain head	vertices	positions	parent-edge weights on the lane
0	0, 1, 4, 7	0, 1, 2, 3	5, 9, 4
8	8	4	6
3	3	5	3
2	2, 5	6, 7	7, 2
6	6	8	8

The 9 over four contiguous runs is the pinned row, and the listings below decompose and query in seven languages.

```c
static int seg_query(int l, int r) { // inclusive over base positions
  int best = -1;
  for (l += 16, r += 17; l < r; l >>= 1, r >>= 1) {
    if (l & 1)
      best = best > seg[l++] ? best : seg[l - 1];
    if (r & 1)
      best = best > seg[--r] ? best : seg[r];
  }
  return best;
}

// max edge weight on the tree path, -1 when u == v (no edges)
static int path_max(int u, int v) {
  int best = -1;
  while (headv[u] != headv[v]) {
    if (depthv[headv[u]] < depthv[headv[v]]) {
      int t = u;
      u = v;
      v = t;
    }
    int q = seg_query(pos[headv[u]] - 1, pos[u] - 1);
    if (q > best)
      best = q;
    u = parent[headv[u]];
  }
  if (u == v)
    return best;
  if (pos[u] > pos[v]) {
    int t = u;
    u = v;
    v = t;
  }
  int q = seg_query(pos[u], pos[v] - 1); // skip u's own edge
  return q > best ? q : best;
}
```

listing 1426: c, the segment query and the path walk, deeper head first (source: dsa/samples-c/src/Ch39/hld.c, lines 151-185)

```
func (h *HLD) rangeMax(lo, hi int) int64 { // inclusive
  best := int64(-1)
  l, r := lo+h.segN, hi+h.segN+1
  for l < r {
    if l&1 == 1 {
      if h.seg[l] > best {
        best = h.seg[l]
      }
      l++
    }
    if r&1 == 1 {
      r--
      if h.seg[r] > best {
        best = h.seg[r]
      }
    }
    l >>= 1
    r >>= 1
  }
  return best
}

// PathMax returns the heaviest edge weight on the u-v path for
// 1-indexed vertices, -1 when the path has no edges.
func (h *HLD) PathMax(u, v int) int64 {
  u, v = u-1, v-1
  best := int64(-1)
  for h.head[u] != h.head[v] {
    if h.posOf[h.head[u]] < h.posOf[h.head[v]] {
      u, v = v, u
    }
    if m := h.rangeMax(h.pos[h.head[u]], h.pos[u]); m > best {
      best = m
    }
    u = h.climbToParent(u)
  }
  if u == v {
    return best
  }
  if h.pos[u] > h.pos[v] {
    u, v = v, u
  }
  if m := h.rangeMax(h.pos[u]+1, h.pos[v]); m > best {
    best = m
  }
  return best
}

// climbToParent steps from a chain head to its parent, crossing
// into the next chain.
func (h *HLD) climbToParent(u int) int { return h.parent[h.head[u]] }
```

listing 1427: go, the range max tree and the head-to-head walk (source: dsa/samples-go/ch39/hld.go, lines 110-160)

```
  static int segQuery(int l, int r) { // inclusive over base positions
    int best = -1;
    for (l += 16, r += 17; l < r; l >>= 1, r >>= 1) {
      if ((l & 1) != 0)
        best = best > seg[l++] ? best : seg[l - 1];
      if ((r & 1) != 0)
        best = best > seg[--r] ? best : seg[r];
    }
    return best;
  }

  // max edge weight on the tree path, -1 when u == v (no edges)
  static int pathMax(int u, int v) {
    int best = -1;
    while (headv[u] != headv[v]) {
      if (depthv[headv[u]] < depthv[headv[v]]) {
        int t = u;
        u = v;
        v = t;
      }
      int q = segQuery(pos[headv[u]] - 1, pos[u] - 1);
      if (q > best)
        best = q;
      u = parent[headv[u]];
    }
    if (u == v)
      return best;
    if (pos[u] > pos[v]) {
      int t = u;
      u = v;
      v = t;
    }
    int q = segQuery(pos[u], pos[v] - 1); // skip u's own edge
    return q > best ? q : best;
  }
```

listing 1428: java, the flat-tree range max, the path walk, deeper head first (source: dsa/samples-java/src/Ch39/Hld.java, lines 152-186)

```
    private long RangeMax(int l, int r) // inclusive over base positions
    {
        long best = long.MinValue;
        int size = NextPow2(_n);
        l += size;
        r += size + 1;
        while (l < r)
        {
            if ((l & 1) == 1)
                best = Math.Max(best, _tree[l++]);
            if ((r & 1) == 1)
                best = Math.Max(best, _tree[--r]);
            l >>= 1;
            r >>= 1;
        }
        return best;
    }

    /// <summary>Max edge weight on the tree path; -1 when u == v (no edges).</summary>
    public long PathMax(int u, int v)
    {
        long best = -1;
        while (_head[u] != _head[v])
        {
            if (_depth[_head[u]] < _depth[_head[v]])
                (u, v) = (v, u); // walk the deeper head first
            // the head's own parent edge is on the path, so its position is included
            best = Math.Max(best, RangeMax(_pos[_head[u]], _pos[u]));
            u = _parent[_head[u]];
        }
        if (u == v)
            return best;
        if (_depth[u] < _depth[v])
            (u, v) = (v, u);
        best = Math.Max(best, RangeMax(_pos[v] + 1, _pos[u])); // skip v's own parent edge
        return best;
    }
```

listing 1429: c#, the range max and the path walk over chains (source: dsa/samples/src/Ch39/Paths2.cs, lines 238-274)

```
  pathMax(a, b) {
    // 1-indexed vertices, max edge weight on the path
    let u = a;
    let v = b;
    let best = -1;
    while (this.#head[u] !== this.#head[v]) {
      if (this.#depth[this.#head[u]] < this.#depth[this.#head[v]]) {
        [u, v] = [v, u];
      }
      best = Math.max(best,
        this.#seg.range(this.#pos[this.#head[u]], this.#pos[u]));
      u = this.#parent[this.#head[u]];
    }
    if (u !== v) {
      if (this.#depth[u] > this.#depth[v]) [u, v] = [v, u];
      best = Math.max(best, this.#seg.range(this.#pos[u] + 1, this.#pos[v]));
    }
    return best;
  }
```

listing 1430: javascript, the path walk over the private segment tree (source: dsa/samples-js/src/ch39-hld.mjs, lines 113-131)

```
def seg_query(node, lo, hi, ql, qh):
    if qh ≤ lo or hi ≤ ql:
        return -1
    if ql ≤ lo and hi ≤ qh:
        return SEG[node]
    mid = (lo + hi) // 2
    left = seg_query(2 * node, lo, mid, ql, qh)
    right = seg_query(2 * node + 1, mid, hi, ql, qh)
    return max(left, right)


seg_build(1, 0, N)


def path_max(u, v):
    best = -1
    while HEAD[u] ≠ HEAD[v]:  # the deeper head climbs first
        if depth_of[HEAD[u]] < depth_of[HEAD[v]]:
            u, v = v, u
        span = seg_query(1, 0, N, POS[HEAD[u]], POS[u] + 1)
        best = max(best, span)
        u = PARENT[HEAD[u]]
    if POS[u] > POS[v]:
        u, v = v, u
    best = max(best, seg_query(1, 0, N, POS[u] + 1, POS[v] + 1))
    return best
```

listing 1431: python, the recursive range query and the walk (source: dsa/samples-py/src/Ch39/hld.py, lines 110-135)

```lua
local function path_max(u, v)
  local par, depth, hhead, pos, base, g = decompose()
  local w = {}
  for _, e in ipairs(EDGES) do
    w[e[2]] = e[3] -- e[2] is the child of e[1] in this rooted tree
  end
  local val = {}
  for i = 1, N do
    local u0 = base[i]
    val[i] = (par[u0] == -1) and -1 or w[u0]
  end
  local function qmax(l, r) -- inclusive 1-based range
    local best = -1
    for i = l, r do best = math.max(best, val[i]) end
    return best
  end
  local best = -1
  while hhead[u] ~= hhead[v] do
    if depth[hhead[u]] < depth[hhead[v]] then u, v = v, u end
    best = math.max(best, qmax(pos[hhead[u]], pos[u]))
    u = par[hhead[u]]
  end
  if u ~= v then
    if depth[u] < depth[v] then u, v = v, u end
    best = math.max(best, qmax(pos[v] + 1, pos[u]))
  end
  return best
end
```

listing 1432: lua, the range scan and the walk, −1 for no edges (source: dsa/samples-lua/ch39_hld.lua, lines 90-117)

On the pinned tree of the ancestor section the decomposition arrays pin exactly: heads 0, 0, 2, 3, 0, 2, 6, 0, 8 and positions 0, 1, 6, 5, 2, 7, 8, 3, 4 with base visit order 0, 1, 4, 7, 8, 3, 2, 5, 6, all 0-indexed. The nine path maxima over the weighted edges: (4,8) hits 9, (6,9) hits 9, (8,8) reports −1 for no edges, (4,1) hits 5, (5,7) hits 9, (8,9) hits 6, (4,6) hits 7, (2,5) hits 9, (7,9) hits 9.

The star fixture is the honest one, corrected against its own first draft: a center with leaves 1 through 5 has all leaf subtrees tied at size 1, the tie rule gives the center's heavy child to leaf 1, so the center and leaf 1 share a chain and the other four leaves hang as singletons, heads 0, 0, 2, 3, 4, 5. Leaf-to-center queries cross two chains except through leaf 1's shared edge, where they ride one.

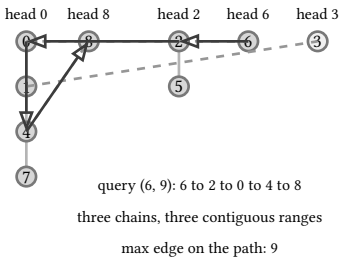


figure 354: the pinned tree drawn as chains in vertical lanes, the 6 to 9 query shaded as it crosses three of them

The application is general technique, path updates and queries on trees, the machinery under every heaviest-edge and path-sum question.

39.7. centroid decomposition

Peel the tree from the top. The centroid of a component is the vertex whose largest remaining piece after removal is smallest, found by walking from any vertex toward the subtree holding the majority and stopping when no side does, ties resolved to the smaller index. Removing it splits the component into pieces of at most half the size, so the recursion is O(log n) deep and every original vertex hangs under O(log n) centroids. Hop distances from each centroid over its own component are bfs-precomputed at build, and the query machinery follows: marking a vertex updates its distance at every centroid ancestor, and the closest marked vertex to a query walks the same ancestor chain, because the path from the query to any marked vertex passes through their common centroid ancestor.

The dry run: the fixture is the same pinned tree in 0-based labels, asserted by all seven suites.

1. The majority walk from 0 steps into child 1, whose subtree of 5 outweighs the parent side's 9 - 5 = 4, and no piece of 1 beats half of 9: vertex 1 is the root centroid.
2. Removing 1 leaves {3}, {4, 7, 8}, and {0, 2, 5, 6}: the big piece centers on 2, whose largest side is one leaf, the middle on 4, and the lone 3 centers on itself, so the parent array reads 2, −1, 1, 1, 1, 2, 2, 4, 4.
3. The script opens mark 5: vertex 3 and the mark meet only at the root centroid, dist 1 + 3 = 4, the row the section's figure draws.
4. Marking 7 too tightens it to dist 1 + 2 = 3, unmarking 7 restores the 4, and marking 0 answers query 8 at dist 2 + 1 = 3, the query two hops and the mark one hop off the centroid root.

action	ancestor chains	answer
mark 5	3, 1 against 5, 2, 1	1 + 3 = 4
mark 7 too	3, 1 against 7, 4, 1	1 + 2 = 3
unmark 7	only 5 remains marked	4
mark 0, query 8	8, 4, 1 against 0, 2, 1	2 + 1 = 3

The 4, 3, 4, 3 script is the pinned row, and the listings below build and query in seven languages.

```c
static void rec(int u, int p) {
  getsize(u, -1);
  int c = find_c(u, -1, subsz[u]);
  removed[c] = 1;
  cpar[c] = p;
  // hop distances from c over its remaining subtree by bfs
  int queue[MAXN], qpar[MAXN], qh = 0, qt = 0;
  for (int v = 0; v < nv; v++)
    dist[c][v] = -1;
  queue[qt] = c;
  qpar[qt] = -1;
  dist[c][c] = 0;
  qt++;
  while (qh < qt) {
    int v = queue[qh];
    int kids[MAXN] = {0};
    int kc = orderkids(v, qpar[qh], kids);
    for (int k = 0; k < kc; k++) {
      queue[qt] = kids[k];
      qpar[qt] = v;
      dist[c][kids[k]] = dist[c][v] + 1;
      qt++;
    }
    qh++;
  }
  int kids[MAXN] = {0};
  int kc = orderkids(c, -1, kids);
  for (int k = 0; k < kc; k++)
    rec(kids[k], c);
}

static int closest_marked(int q) {
  int best = -1;
  for (int c = q; c != -1; c = cpar[c])
    for (int v = 0; v < nv; v++)
      if (mark[v] && dist[c][q] >= 0 && dist[c][v] >= 0) {
        int d = dist[c][q] + dist[c][v]; // the path through c
        if (best < 0 || d < best)
          best = d;
      }
  return best;
}
```

listing 1433: c, the recursive build with its bfs tables, the ancestor-walking query (source: dsa/samples-c/src/Ch39/centroid.c, lines 124-165)

```go
// Mark toggles a vertex on, relaxing every centroid ancestor.
func (q *ClosestMarked) Mark(v int) *ClosestMarked {
  q.marked[v] = true
  for u := v; u != -1; u = q.cpar[u] {
    if d := q.cdist[u][v]; d < q.best[u] {
      q.best[u] = d
    }
  }
  return q
}

// Unmark toggles a vertex off, recomputing the touched ancestors
// from the surviving marks (fixture scale).
func (q *ClosestMarked) Unmark(v int) *ClosestMarked {
  q.marked[v] = false
  for u := v; u != -1; u = q.cpar[u] {
    best := 1 << 30
    for w, on := range q.marked {
      if on && q.cdist[u][w] >= 0 && q.cdist[u][w] < best {
        best = q.cdist[u][w]
      }
    }
    q.best[u] = best
  }
  return q
}

// Query walks the centroid ancestors of v: the answer for any
// marked w runs through the common centroid ancestor.
func (q *ClosestMarked) Query(v int) int {
  best := 1 << 30
  for u := v; u != -1; u = q.cpar[u] {
    if q.best[u] < 1<<30 {
      if d := q.best[u] + q.cdist[u][v]; d < best {
        best = d
      }
    }
  }
  return best
}
```

listing 1434: go, mark relaxing ancestors, the query walking them (source: dsa/samples-go/ch39/centroid.go, lines 123-162)

```
  static void rec(int u, int p) {
    getSize(u, -1);
    int c = findC(u, -1, subsz[u]);
    removed[c] = true;
    cpar[c] = p;
    // hop distances from c over its remaining subtree by bfs
    int[] queue = new int[MAXN], qpar = new int[MAXN];
    int qh = 0, qt = 0;
    for (int v = 0; v < nv; v++)
      dist[c][v] = -1;
    queue[qt] = c;
    qpar[qt] = -1;
    dist[c][c] = 0;
    qt++;
    while (qh < qt) {
      int v = queue[qh];
      int[] kids = new int[MAXN];
      int kc = orderkids(v, qpar[qh], kids);
      for (int k = 0; k < kc; k++) {
        queue[qt] = kids[k];
        qpar[qt] = v;
        dist[c][kids[k]] = dist[c][v] + 1;
        qt++;
      }
      qh++;
    }
    int[] kids = new int[MAXN];
    int kc = orderkids(c, -1, kids);
    for (int k = 0; k < kc; k++)
      rec(kids[k], c);
  }

  static int closestMarked(int q) {
    int best = -1;
    for (int c = q; c != -1; c = cpar[c])
      for (int v = 0; v < nv; v++)
        if (mark[v] && dist[c][q] >= 0 && dist[c][v] >= 0) {
          int d = dist[c][q] + dist[c][v]; // the path through c
          if (best < 0 || d < best)
            best = d;
        }
    return best;
  }
```

listing 1435: java, the recursive build with its bfs tables, the ancestor-walking query (source: dsa/samples-java/src/Ch39/Centroid.java, lines 124-166)

```
    public void Mark(int v) => _marked.Add(v);

    public void Unmark(int v) => _marked.Remove(v);

    /// <summary>Closest marked vertex by walking the centroid ancestors.</summary>
    public int ClosestMarked(int q)
    {
        int best = int.MaxValue;
        for (int c = q; c >= 0; c = CentroidParent[c])
            if (_dist[c].TryGetValue(q, out int toCentroid))
                foreach (int m in _marked)
                    if (_dist[c].TryGetValue(m, out int fromCentroid))
                        best = Math.Min(best, toCentroid + fromCentroid);
        return best;
    }
```

listing 1436: c#, mark, unmark, and closest by the ancestor walk, tables built above (source: dsa/samples/src/Ch39/Centers.cs, lines 79-93)

```
mark(v) {
  this.#eachAncestor(v, (cid) => {
    const d = this.#dist[cid].get(v);
    const list = this.#marked[cid];
    let i = 0;
    while (i < list.length && list[i] < d) i += 1;
    list.splice(i, 0, d);
  });
  return this;
}

unmark(v) {
  this.#eachAncestor(v, (cid) => {
    const i = this.#marked[cid].indexOf(this.#dist[cid].get(v));
    if (i >= 0) this.#marked[cid].splice(i, 1);
  });
  return this;
}

closestMarked(q) {
  let best = Infinity;
  this.#eachAncestor(q, (cid) => {
    if (this.#marked[cid].length > 0) {
      best = Math.min(best,
        this.#marked[cid][0] + this.#dist[cid].get(q));
    }
  });
  return best === Infinity ? -1 : best;
}
```

listing 1437: javascript, mark and unmark keep sorted distance lists per centroid (source: dsa/samples-js/src/ch39-centroid.mjs, lines 86-114)

```
def dist_between(a, b):  # the owning centroid joins the two walks
    if a == b:
        return 0
    for centroid, table in TABLES.items():
        if a in table and b in table:
            return table[a] + table[b]
    return None


# family 1: the centroid parents pin
check(CPAR == [2, -1, 1, 1, 1, 2, 2, 4, 4], "centroid parents pin, root centroid 1")

# family 2: the query script (0-indexed)
marked = {5}
closest = min(dist_between(3, v) for v in marked)
check(closest == 4, "mark 5, query 3 answers 4")
marked.add(7)
closest = min(dist_between(3, v) for v in marked)
check(closest == 3, "mark 7, query 3 answers 3")
marked.discard(7)
closest = min(dist_between(3, v) for v in marked)
check(closest == 4, "unmark 7, query 3 answers 4 again")
marked.add(0)
check(min(dist_between(8, v) for v in marked) == 3, "mark 0, query 8 answers 3")
```

listing 1438: python, the owning-centroid join and the pinned query script (source: dsa/samples-py/src/Ch39/centroid.py, lines 102-125)

```
function Centroid:mark(u)
  local c = u
  while c ~= -1 do -- walk the centroid ancestors
    local d = self.dist[c][u + 1]
    local m = self.marked[c]
    m[d] = (m[d] or 0) + 1
    c = self.cpar[c + 1]
  end
end

function Centroid:unmark(u)
  local c = u
  while c ~= -1 do
    local d = self.dist[c][u + 1]
    local m = self.marked[c]
    m[d] = m[d] - 1
    c = self.cpar[c + 1]
  end
end

function Centroid:closest_marked(u) -- nil when nothing is marked
  local best = nil
  local c = u
  while c ~= -1 do
    local d = self.dist[c][u + 1]
    local m = self.marked[c]
    for k, cnt in pairs(m) do
      if cnt > 0 then
        local tot = d + k
        if best == nil or tot < best then best = tot end
      end
    end
    c = self.cpar[c + 1]
  end
  return best
end
```

listing 1439: lua, mark, unmark, closest, distance counts per centroid (source: dsa/samples-lua/ch39_centroid.lua, lines 83-118)

On the pinned tree the centroid parents land 2, −1, 1, 1, 1, 2, 2, 4, 4 with root centroid 1, all 0-indexed, and the query script runs verbatim: mark 5 and query 3 for 4, mark 7 and the same query drops to 3, unmark 7 and it returns to 4, mark 0 and query 8 for 3. A path graph of five vertices centers on the exact middle with parents 1, 2, −1, 2, 3, and a lone marked vertex queried at itself answers 0.

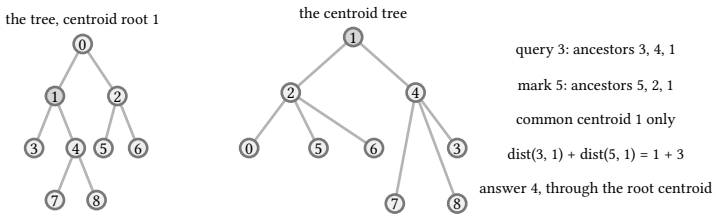


figure 355: the pinned tree, its centroid tree beside it, one query walking up the centroid ancestors with distances written

The application is general technique, closest-marked-vertex and path-counting questions on trees at O(n log n) build and O(log n) per query.

39.8. functional graphs

Every vertex has exactly one successor, so the graph is a set of rings with trees hanging off them, and the 2019 problem H question over it, for every station, how many distinct stations' journeys of at most k legs reach it, splits along that seam. The indegree peel, leaves first, children before parents, splits ring nodes from tree nodes. Tree nodes answer bottom-up in peel order: each hanging tree counts its own subtree starts within k legs, exact-distance bookkeeping folding the overcount out. Ring nodes share a difference array: every node whose ring entry is reached within k casts an arc of length min(k - d + 1, ring length) starting at its entry, wraps split into a head and tail contribution, and one prefix sum turns the arcs into per-station counts. The walk is deterministic, so counting distinct stations is counting starts whose first arrival lands inside the budget, and a brute that walks k plus 1 steps per start cross-checks every fixture.

The dry run: the fixture is the 2019 problem H sample, successors 2, 3, 2, 5, 4, 5, 6 in the statement's reading with k = 2, asserted by all seven suites against a walking brute.

1. The indegree peel splits two rings, 1 with 2 and 3 with 4: station 0 hangs one leg off 1, and the chain 6, 5 enters the second ring at 4.
2. A ring node's budget covers its whole pair, the arc length min(2 + 1, 2) = 2, so the ring's own starts add 2 to both stations of each pair.
3. The trees fold in bottom-up: 0's two legs reach 1 and 2, lifting that pair to 2 + 1 = 3, while 5 adds itself, 3, and 4, and 6 adds itself, 5, and 4.
4. Station 4 collects the most, ring 2 plus 5 plus 6 = 4, and station 6 keeps only itself at 1.

start	stations reached within 2 legs
0	0, 1, 2
1	1, 2
2	1, 2
3	3, 4
4	3, 4
5	3, 4, 5
6	4, 5, 6

Counting the starts per station reads the answer row 1, 3, 3, 3, 4, 2, 1, and the listings below peel and count in seven languages.

```
    } while (cur != r0);
    // collect contributions: every node whose entry is on this ring
    int diff[2 * MAXN];
    memset(diff, 0, sizeof(diff));
    for (int w = 0; w < nv; w++) {
      int entry = -1, dw = 0;
      if (onring[w]) {
        entry = w;
        dw = 0;
      } else {
        // walk to the ring entry
        int c2 = succ[w], d2 = 1;
        while (!onring[c2]) {
          c2 = succ[c2];
          d2++;
        }
        entry = c2;
        dw = d2;
      }
      if (done[entry] != 1 || pos[entry] >= L || pos[entry] < 0)
        continue; // belongs to another ring
      if (dw > k)
        continue;
      int len = k - dw + 1;
      if (len > L)
        len = L;
      int p = pos[entry];
      diff[p] += 1;
      diff[p + len] -= 1;
    }
    for (int i = 0; i < 2 * L; i++) {
      if (i)
        diff[i] += diff[i - 1];
      cnt[ring[i % L]] += diff[i] > 0 ? diff[i] : 0;
    }
    // mark this ring consumed so later rings skip its contributions
    for (int i = 0; i < L; i++)
      done[ring[i]] = 2;
    // the entries on this ring must still accept later walks... they
    // belong to this ring only, so the done flag is final
  }
```

listing 1440: c, the ring walk, the difference-array arcs, the prefix fold (source: dsa/samples-c/src/Ch39/functional.c, lines 96-136)

```
    ring := []int{s}
    visited[s] = true
    for v := next[s]; v ≠ s; v = next[v] {
      ring = append(ring, v)
      visited[v] = true
    }
    L := len(ring)
    diff := make([]int64, L+1)
    addArc := func(start, length int) {
      if length ≥ L {
        diff[0]++
        diff[L]--
        return
      }
      if start+length <= L {
        diff[start]++
        diff[start+length]--
      } else {
        diff[start]++
        diff[L]--
        diff[0]++
        diff[start+length-L]--
      }
    }
    for i := range ring { // the ring vertices themselves
      addArc(i, k+1)
    }
```

listing 1441: go, the ring collected in walk order, addarc, the ring's own arcs (source: dsa/samples-go/ch39/functional.go, lines 58-84)

```
    // ring part: per-ring difference array over the unrolled 2L axis.
    // a node w at distance dw from entry e contributes to arc
    // e .. e + min(k - dw, L - 1)
    int[] done = new int[MAXN];
    java.util.Arrays.fill(done, 0, nv, 0);
    for (int r0 = 0; r0 < nv; r0++) {
      if (!onring[r0] || done[r0] != 0)
        continue;
      int[] ring = new int[MAXN], pos = new int[MAXN];
      int len = 0, cur = r0;
      do {
        ring[len] = cur;
        pos[cur] = len;
        done[cur] = 1;
        len++;
        cur = succ[cur];
      } while (cur != r0);
      // collect contributions: every node whose entry is on this ring
      int[] diff = new int[2 * MAXN];
      for (int w = 0; w < nv; w++) {
        int entry, dw;
        if (onring[w]) {
          entry = w;
          dw = 0;
        } else {
          // walk to the ring entry
          int c2 = succ[w];
          int d2 = 1;
          while (!onring[c2]) {
            c2 = succ[c2];
            d2++;
          }
          entry = c2;
          dw = d2;
        }
        if (done[entry] != 1 || pos[entry] >= len || pos[entry] < 0)
          continue; // belongs to another ring
        if (dw > k)
          continue;
        int reach = k - dw + 1;
        if (reach > len)
          reach = len;
        int p = pos[entry];
        diff[p] += 1;
        diff[p + reach] -= 1;
      }
      for (int i = 0; i < 2 * len; i++) {
        if (i > 0)
          diff[i] += diff[i - 1];
        cnt[ring[i % len]] += diff[i] > 0 ? diff[i] : 0;
      }
      // mark this ring consumed so later rings skip its contributions
      for (int i = 0; i < len; i++)
        done[ring[i]] = 2;
    }
```

listing 1442: java, the ring walk, the difference-array arcs, the doubled prefix fold (source: dsa/samples-java/src/Ch39/Functional.java, lines 76-130)

```
    private static void FoldRing(List<int> ring, int[] successor, bool[] onRing, long k, long[] counts)
    {
        int len = ring.Count;
        var position = new Dictionary<int, int>();
        for (int i = 0; i < len; i++)
            position[ring[i]] = i;
        var diff = new long[len + 1]; // the extra slot catches arcs ending at len
        void Cast(int start, long length)
        {
            length = Math.Min(length, len);
            if (length ≥ len)
            {
                diff[0]++; // covers the whole ring, no decrement needed
                return;
            }
            diff[start]++;
            int end = start + (int)length;
            if (end ≤ len)
                diff[end]--;
            else
            {
                diff[0]++; // the wrapped head covers from zero
                diff[end - len]--;
            }
        }

        for (int i = 0; i < len; i++)
            Cast(i, k + 1); // ring node: budget k from its own position
        foreach (int v in Enumerable.Range(0, onRing.Length).Where(v => !onRing[v]))
        {
            int entry = v; // walk forward until the ring: the first ring vertex
            long d = 0;
            while (!onRing[entry])
            {
                entry = successor[entry];
                d++;
            }
            if (k - d ≥ 0 && position.ContainsKey(entry))
                Cast(position[entry], k - d + 1); // budget shrinks by the entry distance
        }
        long run = 0;
        for (int i = 0; i < len; i++)
        {
            run += diff[i];
            counts[ring[i]] += run;
        }
    }
```

listing 1443: c#, the cast helper with its wraps, the entry walk, the fold (source: dsa/samples/src/Ch39/Centers.cs, lines 170-216)

```
  // ring arcs: every start adds to an arc of its ring
  const ans = new Array(n).fill(0);
  const treeByRing = rings.map(() => []);
  for (let v = 0; v < n; v++) {
    if (!isRing[v]) treeByRing[ringOf[v]].push(v);
  }
  rings.forEach((order, rid) => {
    const L = order.length;
    const diff = new Array(2 * L + 1).fill(0);
    const addArc = (s, m) => {
      diff[s] += 1;
      diff[s + m + 1] -= 1;
    };
    for (let p = 0; p < L; p++) {
      addArc(p, Math.min(k, L - 1)); // ring starts cover themselves
    }
    for (const v of treeByRing[rid]) {
      const d = dep[v];
      if (d ≤ k) addArc(posInRing[v], Math.min(k - d, L - 1));
    }
    const pref = [0];
    for (let i = 0; i < 2 * L; i++) pref.push(pref[i] + diff[i]);
    order.forEach((v, p) => {
      ans[v] = pref[p + 1] + pref[p + 1 + L];
    });
  });
```

listing 1444: javascript, the ring arcs and the doubled prefix, the tree part above (source: dsa/samples-js/src/ch39-functional.mjs, lines 138-163)

```
    # ring part: walk each ring once, arc of length min(k - dist + 1, L)
    visited = [False] * count
    for v in range(count):
        if not on_ring[v] or visited[v]:
            continue
        ring = [v]
        visited[v] = True
        cur = successor[v]
        while cur ≠ v:
            ring.append(cur)
            visited[cur] = True
            cur = successor[cur]
        length = len(ring)
        diff = [0] * (length + 1)  # a plain prefix-sum span, wraps split
        for idx, node in enumerate(ring):
            for cand in range(count):  # every start entering this ring
                if entry[cand] ≠ node:
                    continue
                budget = k - dist_to_ring[cand]
                if budget < 0:
                    continue
                arc = min(budget + 1, length)
                if arc ≥ length:  # the arc covers the whole ring
                    diff[0] += 1
                    diff[length] -= 1
                elif idx + arc > length:  # wraps: split tail and head
                    diff[idx] += 1
                    diff[length] -= 1
                    diff[0] += 1
                    diff[(idx + arc) % length] -= 1
                else:
                    diff[idx] += 1
                    diff[idx + arc] -= 1
        running = 0
        for idx in range(length):
            running += diff[idx]
            counts[ring[idx]] = running
    return counts
```

listing 1445: python, the ring walk, split arcs, the running sum (source: dsa/samples-py/src/Ch39/functional.py, lines 66-103)

```
    local L = #ring
    local idx = {}
    for i, x in ipairs(ring) do idx[x + 1] = i end
    -- the ring difference array over 2L linear slots: an arc
    -- (start, len) with len < L covers start .. start + len - 1;
    -- len == L covers everything and is added flat instead
    local d = {}
    for i = 1, 2 * L do d[i] = 0 end
    local flat = 0
    local function addarc(start, len, w)
      if len ≥ L then
        flat = flat + w
      else
        d[start] = d[start] + w
        d[start + len] = d[start + len] - w
      end
    end
    for _, x in ipairs(ring) do -- ring starts cover min(k+1, L)
      addarc(idx[x + 1], math.min(k + 1, L), 1)
    end
    for pi = 1, #peel do -- tree starts, by distance below the entry
      local u = peel[pi]
      -- only direct hangers fire: their bydd already aggregates
      -- every start in their subtree, deeper nodes would double count
      if ringentry[u + 1] == 1 then
        local du = bydd[u + 1]
        local e = f[u + 1] -- the ring entry, one leg down
        if idx[e + 1] ~= nil then -- only trees entering THIS ring
          for dd = 1, math.min(#du, k + 1) do
            local l = dd -- legs: one to the entry plus dd-1 below u
            if l ≤ k then
              addarc(idx[e + 1], math.min(k - l + 1, L), du[dd])
            end
          end
        end
      end
    end
    -- prefix once over 2L: real position i is covered at linear i
    -- and at linear i + L (the lap wrap), at most one per short arc
    local pre = {}
    pre[0] = 0
    for i = 1, 2 * L do pre[i] = pre[i - 1] + d[i] end
    for i = 1, L do
      cnt[ring[i] + 1] = cnt[ring[i] + 1] + pre[i] + pre[i + L] + flat
    end
```

listing 1446: lua, addarc over 2l slots, the doubled prefix, flat whole-ring arcs (source: dsa/samples-lua/ch39_functional.lua, lines 72-116)

The 2019 problem H sample is the anchor: successors 2, 3, 2, 5, 4, 5, 6 in the statement's 1-indexed reading, k = 2, answers 1, 3, 3, 3, 4, 2, 1, exactly the spec's crafted case. A pure ring of 3 with k = 1 counts 2 for every station. A chain into a ring reads 1, 3, 2 at k = 1 and 1, 3, 3 at k = 2, the ring saturating as the budget grows.

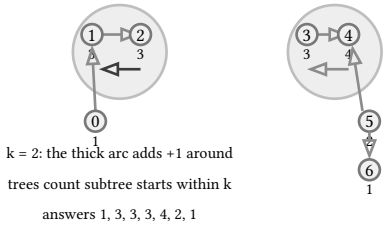


figure 356: the 2019 problem H fixture as two rings with trees hanging, every station labeled with its answer, one arc drawn thick on a ring

The application is icpc world finals 2019 problem H (book 10, chapter 10), hobson's trains, exactly this structure at n of 5e5 with k-th-ancestor counts and ring difference arrays, and the chapter sample is the contest's own crafted case.

39.9. the 2-core

The maximal subgraph of minimum degree 2 is one queue away. Every vertex of degree at most 1 leaves, its removal decrements each neighbor, and anything that drops to degree 1 joins the queue, the same degree-array idiom the graph chapter of book 9, ch 10 (practical data structures and algorithms, graphs) uses for component counting. What survives the peel is the 2-core, and the edges between a survivor and a peeled vertex form the boundary, the exact seam the dead-end sign question cares about.

The dry run: the fixture is the 2019 problem E sample graph, edges 1-2, 2-3, 3-1, 3-4, 4-5, 6-7 in the statement's reading, asserted by all seven suites.

1. Degrees start 3, 2, 2, 2, 1, 1, 1 over the 0-based labels: vertices 4, 5, 6 open the queue.
2. Peeling 4 decrements 3 from 2 to 1, and 3 joins the queue; 5 and 6 peel each other down to 0.
3. Peeling 3 decrements 0 from 3 to 2: nothing drops back to 1, and the queue empties.
4. The survivors are 0, 1, 2, the triangle, and the one edge from a survivor to a peeled vertex is (0, 3), the boundary.

step	queue after	degrees after
open	4, 5, 6	3, 2, 2, 2, 1, 1, 1
peel 4	5, 6, 3	3, 2, 2, 1, 0, 1, 1
peel 5, 6	3	3, 2, 2, 1, 0, 0, 0
peel 3	empty	2, 2, 2, 0, 0, 0, 0
survivors	none left	core 0, 1, 2, boundary (0, 3)

The core 0, 1, 2 with the single boundary edge (0, 3) is the pinned pair, and the listings below peel in seven languages.

```
static void peel(void) {
  memset(peeled, 0, sizeof(int) * (size_t)nv);
  int queue[MAXV], qh = 0, qt = 0;
  for (int v = 0; v < nv; v++)
    if (deg[v] ≤ 1)
      queue[qt++] = v;
  while (qh < qt) {
    int u = queue[qh++];
    if (peeled[u] || deg[u] > 1)
      continue;
    peeled[u] = 1;
    for (int e = adjh[u]; e ≥ 0; e = adjn[e]) {
      int v = adjv[e];
      if (!peeled[v] && --deg[v] == 1)
        queue[qt++] = v;
    }
  }
  coren = 0;
  for (int v = 0; v < nv; v++)
    if (!peeled[v])
      core[coren++] = v;
  bndn = 0;
  for (int e = 0; e < ec; e++)
    if (peeled[eu[e]] ≠ peeled[ev[e]]) {
      int a = eu[e], b = ev[e];
      if (a > b) {
        int t = a;
        a = b;
        b = t;
      }
      bnd[bndn][0] = a;
      bnd[bndn][1] = b;
      bndn++;
    }
}
```

listing 1447: c, the peel queue, the survivor collection, the boundary edges (source: dsa/samples-c/src/Ch39/twocore.c, lines 51-85)

```
  removed := make([]bool, n)
  queue := make([]int, 0, n)
  for v := 0; v < n; v++ {
    if deg[v] <= 1 {
      queue = append(queue, v)
    }
  }
  for len(queue) > 0 {
    u := queue[0]
    queue = queue[1:]
    if removed[u] || deg[u] > 1 {
      continue
    }
    removed[u] = true
    for _, v := range adj[u] {
      if !removed[v] {
        deg[v]--
        if deg[v] == 1 {
          queue = append(queue, v)
        }
      }
    }
  }
  for v := 0; v < n; v++ {
    if !removed[v] {
      core = append(core, v)
    }
  }
  for _, e := range edges {
    if removed[e[0]] != removed[e[1]] {
      a, b := e[0], e[1]
      if a > b {
        a, b = b, a
      }
      boundary = append(boundary, [2]int{a, b})
    }
  }
  sort.Slice(boundary, func(i, j int) bool {
    if boundary[i][0] != boundary[j][0] {
      return boundary[i][0] < boundary[j][0]
    }
    return boundary[i][1] < boundary[j][1]
  })
  return core, boundary
```

listing 1448: go, the queue, the decrement, the boundary collection and sort (source: dsa/samples-go/ch39/twocore.go, lines 17-60)

```
  static void peel() {
    java.util.Arrays.fill(peeled, 0, nv, false);
    int[] queue = new int[MAXV];
    int qh = 0, qt = 0;
    for (int v = 0; v < nv; v++)
      if (deg[v] ≤ 1)
        queue[qt++] = v;
    while (qh < qt) {
      int u = queue[qh++];
      if (peeled[u] || deg[u] > 1)
        continue;
      peeled[u] = true;
      for (int e = adjh[u]; e ≥ 0; e = adjn[e]) {
        int v = adjv[e];
        if (!peeled[v] && --deg[v] == 1)
          queue[qt++] = v;
      }
    }
    coren = 0;
    for (int v = 0; v < nv; v++)
      if (!peeled[v])
        core[coren++] = v;
    bndn = 0;
    for (int e = 0; e < ec; e++)
      if (peeled[eu[e]] ≠ peeled[ev[e]]) {
        int a = Math.min(eu[e], ev[e]);
        int b = Math.max(eu[e], ev[e]);
        bnd[bndn][0] = a;
        bnd[bndn][1] = b;
        bndn++;
      }
  }
```

listing 1449: java, the peel queue, the survivor collection, the boundary edges (source: dsa/samples-java/src/Ch39/Twocore.java, lines 52-83)

```
public static (int[] Core, (int, int)[] Boundary) Peel(int n, (int U, int V)[] edges)
{
    var degree = new int[n];
    var adjacency = Enumerable.Range(0, n).Select(_ => new List<int>()).ToList();
    foreach ((int u, int v) in edges)
    {
        adjacency[u].Add(v);
        adjacency[v].Add(u);
        degree[u]++;
        degree[v]++;
    }
    var removed = new bool[n];
    var queue = new Queue<int>(Enumerable.Range(0, n).Where(v => degree[v] ≤ 1));
    while (queue.Count > 0)
    {
        int u = queue.Dequeue();
        if (removed[u] || degree[u] > 1)
            continue;
        removed[u] = true;
        foreach (int v in adjacency[u])
            if (!removed[v] && --degree[v] == 1)
                queue.Enqueue(v);
    }
    int[] core = Enumerable.Range(0, n).Where(v => !removed[v]).ToArray();
    (int, int)[] boundary = edges
        .Where(e => removed[e.U] ≠ removed[e.V])
        .Select(e => (Math.Min(e.U, e.V), Math.Max(e.U, e.V)))
        .OrderBy(e => e.Item1)
        .ThenBy(e => e.Item2)
        .ToArray();
    return (core, boundary);
}
```

listing 1450: c#, the same peel, core and boundary as linq reads (source: dsa/samples/src/Ch39/Centers.cs, lines 226-257)

```
  const removed = new Array(n).fill(false);
  const q = [];
  for (let v = 0; v < n; v++) {
    if (deg[v] ≤ 1) q.push(v);
  }
  while (q.length > 0) {
    const u = q.shift();
    if (removed[u] || deg[u] > 1) continue;
    removed[u] = true;
    for (const v of adj[u]) {
      if (!removed[v]) {
        deg[v] -= 1;
        if (deg[v] === 1) q.push(v);
      }
    }
  }
  const core = Array.from({ length: n }, (_, v) => v)
    .filter((v) => !removed[v]);
  const boundary = edges
    .filter(([u, v]) => removed[u] =/= removed[v])
    .map(([u, v]) => [Math.min(u, v), Math.max(u, v)])
    .sort((a, b) => a[0] - b[0] || a[1] - b[1]);
  return { core, boundary };
```

listing 1451: javascript, the peel and both output lists (source: dsa/samples-js/src/ch39-twocore.mjs, lines 16-38)

```
def twocore(count, edges):
    degree = [0] * count
    graph = [[] for _ in range(count)]
    for u, v in edges:
        graph[u].append(v)
        graph[v].append(u)
        degree[u] += 1
        degree[v] += 1
    removed = [False] * count
    q = deque(v for v in range(count) if degree[v] ≤ 1)
    while q:  # peel leaves and isolated vertices
        u = q.popleft()
        if removed[u] or degree[u] > 1:
            continue
        removed[u] = True
        for v in graph[u]:
            if not removed[v]:
                degree[v] -= 1
                if degree[v] == 1:
                    q.append(v)
    core = [v for v in range(count) if not removed[v]]
    boundary = sorted(
        tuple(sorted((u, v))) for u, v in edges if removed[u] =/= removed[v]
    )
    return core, boundary
```

listing 1452: python, the whole peel with the boundary read (source: dsa/samples-py/src/Ch39/twocore.py, lines 16-40)

```
  local removed = {}
  local q = {}
  for v = 0, n - 1 do
    if deg[v + 1] ≤ 1 then q[#q + 1] = v end
  end
  local head = 1
  while head ≤ #q do
    local u = q[head]
    head = head + 1
    if not removed[u + 1] and deg[u + 1] ≤ 1 then
      removed[u + 1] = true
      for _, v in ipairs(adj[u + 1]) do
        if not removed[v + 1] then
          deg[v + 1] = deg[v + 1] - 1
          if deg[v + 1] == 1 then q[#q + 1] = v end
        end
      end
    end
  end
  local core = {}
  for v = 0, n - 1 do
    if not removed[v + 1] then core[#core + 1] = v end
  end
  local boundary = {}
  for _, e in ipairs(edges) do
    if removed[e[1] + 1] ~= removed[e[2] + 1] then
      boundary[#boundary + 1] = {math.min(e[1], e[2]),
                                 math.max(e[1], e[2])}
    end
  end
  table.sort(boundary, function(x, y)
    if x[1] ~= y[1] then return x[1] < y[1] end
    return x[2] < y[2]
  end)
  return core, boundary
end
```

listing 1453: lua, the queue peel in 1-based tables (source: dsa/samples-lua/ch39_twocore.lua, lines 20-55)

The 2019 problem E sample graph, edges 1-2, 2-3, 3-1, 3-4, 4-5, 6-7 in its 1-indexed reading, keeps the core {0, 1, 2} with boundary edge (0, 3), and the {5, 6} component peels entirely, its leaf entrances keeping signs on both ends, the redundancy rule that problem's statement adds on top. A bare 4-cycle keeps everyone with no boundary, a path peels to nothing, a single edge peels to nothing, isolated vertices peel immediately, and parallel edges hold degree 2 on both sides and survive.

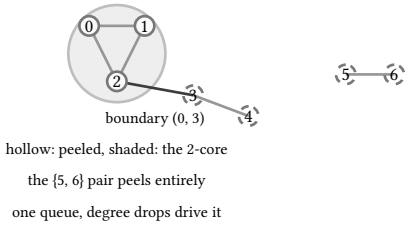


figure 357: the 2019 problem E fixture with peeled vertices hollow, the core shaded, the boundary arrow at vertex 3

The application is icpc world finals 2019 problem E (book 10, chapter 10), dead-end detector, this peel plus a sign-placement rule on the boundary edges, and the fixture above is its sample graph.

39.10. prufer codes and the second-best mst

A labeled tree of n vertices compresses to n - 2 numbers: repeatedly remove the smallest-labeled leaf and record its unique neighbor. The decode inverts it by degree bookkeeping, each code entry lifts one degree, the smallest current leaf pairs with it, and the last two leaves close the tree. Prufer labels are 1-indexed by convention and the lua port asserts the list unchanged. The same file answers the second-best spanning tree: kruskal from book 9, ch 12 (practical data structures and algorithms, minimum spanning trees and flows) builds the base tree, then every non-tree edge scores a swap, its weight minus the heaviest edge on the tree path between its endpoints, and the best positive swap over the minimum total is the runner-up. The path maximum is an honest O(n) parent walk per edge here, with the lifting speedup of the ancestor section above cross-named for the logarithmic form.

The dry run: the fixtures are the 6-vertex tree and the 7-edge second-best graph, asserted by all seven suites.

1. The encode strips smallest leaves: 1 goes recording 3, 2 goes recording 3, 3 goes recording 4, 5 goes recording 4, and 4, 6 close, the pinned code 3, 3, 4, 4 that decode rebuilds exactly.
2. Kruskal lines the 7 edges up by weight, 1, 1, 2, 2, 3, 4, 5: the first four unions span all 5 vertices at 1 + 1 + 2 + 2 = 6, the mst.
3. The three non-tree edges each score a swap against the heaviest edge on their tree path: (3,4) at 3 meets a 2, (4,5) at 4 the same 2, and (5,1) at 5 the same 2 again.
4. The cheapest wins: adding (3,4) closes the cycle through 3, 2, 4 and drops the path's heaviest edge (2,3), landing the runner-up at 6 + 1 = 7.

candidate	heaviest on tree path	gain	total
mst picks	(1,2), (2,4), (2,3), (3,5)		6
(3,4) at 3	(2,3) at 2	3 - 2 = 1	7
(4,5) at 4	(2,3) at 2	4 - 2 = 2	8
(5,1) at 5	(2,3) or (3,5) at 2	5 - 2 = 3	9

The mst 6 and the runner-up 7 by the (2,3) for (3,4) swap are the pinned pair, and the listings below encode, decode, and scan in seven languages.

```c
static void prufer_encode(int vn, const int (*edges)[2], int ecnt, int *len) {
  int deg[MAXN] = {0};
  int alive[MAXN];
  int adj[MAXN][MAXN], acnt[MAXN];
  memset(acnt, 0, sizeof(acnt));
  for (int e = 0; e < ecnt; e++) {
    int u = edges[e][0], v = edges[e][1];
    deg[u]++;
    deg[v]++;
    adj[u][acnt[u]++] = v;
    adj[v][acnt[v]++] = u;
  }
  for (int v = 0; v < vn; v++)
    alive[v] = 1;
  int cn = 0;
  while (cn < vn - 2) {
    int best = -1; // the smallest live leaf
    for (int v = 0; v < vn; v++)
      if (alive[v] && deg[v] == 1) {
        best = v;
        break;
      }
    alive[best] = 0;
    int nbr = -1;
    for (int i = 0; i < acnt[best]; i++)
      if (alive[adj[best][i]])
        nbr = adj[best][i];
    code[cn++] = nbr + 1; // 1-indexed by convention
    deg[nbr]--;
  }
  *len = cn;
}
```

listing 1454: c, the encode loop removing the smallest leaf, decode below (source: dsa/samples-c/src/Ch39/prufer.c, lines 30-61)

```
  var seq []int64
  alive := make([]bool, n)
  for i := range alive {
    alive[i] = true
  }
  leaf := -1
  pickLeaf := func() int {
    best := -1
    for v := 0; v < n; v++ {
      if alive[v] && deg[v] == 1 && (best < 0 || v < best) {
        best = v
      }
    }
    return best
  }
  for len(seq) < n-2 {
    if leaf < 0 || !alive[leaf] || deg[leaf] != 1 {
      leaf = pickLeaf()
    }
    var nbr int
    for u := range adjSet[leaf] {
      nbr = u
    }
    seq = append(seq, int64(nbr+1))
    alive[leaf] = false
    delete(adjSet[nbr], leaf)
    deg[nbr]--
    leaf = -1
    for v := 0; v < n; v++ {
      if alive[v] && deg[v] == 1 {
        leaf = v
        break
      }
    }
  }
  return seq
}
```

listing 1455: go, the leaf pick and removal, the neighbor recorded 1-indexed (source: dsa/samples-go/ch39/prufer.go, lines 23-59)

```
  static int pruferEncode(int vn, int[][] edges) {
    int[] deg = new int[MAXN], alive = new int[MAXN];
    int[][] adj = new int[MAXN][MAXN];
    int[] acnt = new int[MAXN];
    for (int[] e : edges) {
      deg[e[0]]++;
      deg[e[1]]++;
      adj[e[0]][acnt[e[0]]++] = e[1];
      adj[e[1]][acnt[e[1]]++] = e[0];
    }
    for (int v = 0; v < vn; v++)
      alive[v] = 1;
    int cn = 0;
    while (cn < vn - 2) {
      int best = -1; // the smallest live leaf
      for (int v = 0; v < vn; v++)
        if (alive[v] == 1 && deg[v] == 1) {
          best = v;
          break;
        }
      alive[best] = 0;
      int nbr = -1;
      for (int i = 0; i < acnt[best]; i++)
        if (alive[adj[best][i]] == 1)
          nbr = adj[best][i];
      code[cn++] = nbr + 1; // 1-indexed by convention
      deg[nbr]--;
    }
    return cn;
  }
```

listing 1456: java, the encode loop removing the smallest leaf over an inline adjacency matrix, decode below (source: dsa/samples-java/src/Ch39/Prufer.java, lines 26-55)

```
    public static int[] Encode(int n, (int U, int V)[] edges)
    {
        var degree = new int[n];
        var adjacency = Enumerable.Range(0, n).Select(_ => new HashSet<int>()).ToList();
        foreach ((int u, int v) in edges)
        {
            adjacency[u].Add(v);
            adjacency[v].Add(u);
            degree[u]++;
            degree[v]++;
        }
        var leaves = new PriorityQueue<int, int>();
        for (int v = 0; v < n; v++)
            if (degree[v] == 1)
                leaves.Enqueue(v, v);
        var code = new List<int>();
        var alive = new bool[n];
        Array.Fill(alive, true);
        while (code.Count < n - 2)
        {
            int leaf = leaves.Dequeue();
            alive[leaf] = false;
            int neighbor = adjacency[leaf].First(v => alive[v]);
            code.Add(neighbor + 1); // 1-indexed prufer labels
            adjacency[neighbor].Remove(leaf);
            if (--degree[neighbor] == 1)
                leaves.Enqueue(neighbor, neighbor);
        }
        return code.ToArray();
    }
```

listing 1457: c#, the priority-queue encode, the degree decode below (source: dsa/samples/src/Ch39/Prufer.cs, lines 11-40)

```
export function pruferEncode(n, edges0) {
  const adj = Array.from({ length: n }, () => new Set());
  const deg = new Array(n).fill(0);
  for (const [u, v] of edges0) {
    adj[u].add(v);
    adj[v].add(u);
    deg[u] += 1;
    deg[v] += 1;
  }
  const seq = [];
  const alive = new Array(n).fill(true);
  while (seq.length < n - 2) {
    let leaf = 0;
    while (!alive[leaf] || deg[leaf] !== 1) leaf += 1;
    alive[leaf] = false;
    let nbr = -1;
    for (const v of adj[leaf]) {
      if (alive[v]) {
        nbr = v;
        break;
      }
    }
    adj[nbr].delete(leaf);
    deg[nbr] -= 1;
    seq.push(nbr + 1); // 1-indexed labels
  }
  return seq;
}
```

listing 1458: javascript, the encode loop over adjacency sets (source: dsa/samples-js/src/ch39-prufer.mjs, lines 35-62)

```
def prufer_encode(count, edges):
    # edges 0-indexed in, prufer numbers 1-indexed out
    degree = [0] * count
    adj = [set() for _ in range(count)]
    for u, v in edges:
        adj[u].add(v)
        adj[v].add(u)
        degree[u] += 1
        degree[v] += 1
    leaves = [v for v in range(count) if degree[v] == 1]
    heapq.heapify(leaves)
    alive = [True] * count
    code = []
    while len(code) < count - 2:  # remove the smallest leaf each step
        leaf = heapq.heappop(leaves)
        alive[leaf] = False
        neighbor = next(v for v in adj[leaf] if alive[v])
        code.append(neighbor + 1)
        adj[neighbor].discard(leaf)
        degree[neighbor] -= 1
        if degree[neighbor] == 1:
            heapq.heappush(leaves, neighbor)
    return code
```

listing 1459: python, the heap-driven encode, the code 1-indexed out (source: dsa/samples-py/src/Ch39/prufer.py, lines 18-40)

```
  local seq = {}
  while #seq < n - 2 do
    -- the smallest leaf: scan, n is small and the rule is canonical
    local leaf = -1
    for v = 1, n do
      if alive[v] and deg[v] == 1 then
        leaf = v
        break
      end
    end
    alive[leaf] = false
    local nbr = -1
    for w in pairs(adj[leaf]) do
      if alive[w] then nbr = w end
    end
    adj[leaf][nbr] = nil
    adj[nbr][leaf] = nil
    deg[nbr] = deg[nbr] - 1
    seq[#seq + 1] = nbr -- 1-indexed prufer numbers
  end
  return seq
end
```

listing 1460: lua, the scan for the smallest leaf, the removal (source: dsa/samples-lua/ch39_prufer.lua, lines 26-47)

The tree with edges 1-3, 2-3, 3-4, 4-5, 4-6 encodes to 3, 3, 4, 4 and decodes back identically. Five seeded trees from the pinned lcg round-trip, the first a star on 5 vertices coding 1, 1, 1. The java copy of the seeded family draws its trees through Integer.remainderUnsigned and hands pruferEncode an Arrays.copyOf(edges, vn - 1) slice, because its generator buffer keeps all 8 rows whatever vn the lcg picks. The second-best fixtures: the 7-edge graph over 5 vertices has its mst at weight 6 on edges (1,2), (2,3), (2,4), (3,5), and the runner-up at 7 by swapping (2,3) for (3,4), both matched against spanning-tree enumeration. The complete graph on 4 vertices with weights 1 through 6 around runs mst 6 and second-best 8.

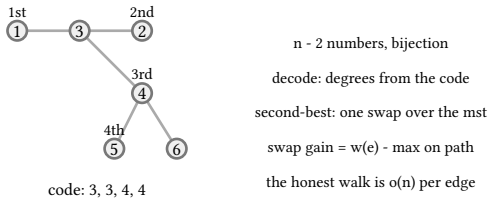


figure 358: the 6-vertex tree with the removal order numbered on leaves, the code 3, 3, 4, 4 written below

The application is general technique, the counting license for labeled trees and the runner-up question whenever the mst itself is not enough.

39.11. strong orientation

An undirected graph admits a strongly connected orientation, an assignment of directions making every vertex reachable from every other, exactly when it is connected and bridgeless. That is robbins' theorem, and both halves read off the machinery above: connectivity is the baseline walk of book 9, ch 10 (practical data structures and algorithms, graphs), and bridgeless is the disc and low classification of the bridges section, since a bridge in the underlying graph is a one-way bottleneck under any assignment of directions, whatever crosses it can never come back. When the graph qualifies, the constructive rule is a dfs orientation: every tree edge points away from the root along the dfs, every back edge points toward the ancestor, and every cycle of the graph becomes a directed cycle, which is exactly the local strong connectivity the flood needs. No samples ship for this section, the decision

is two existing tools composed, and the theorem is the prose payoff. Icpc world finals 2022 problem R (book 10, chapter 11) is the reachability kin, its 2-edge-connected components being the maximal pieces that qualify.

The dry run: no suite carries this section, the numbers are hand-derived over the shared-vertex triangles of the bridges fixture.

1. The gate reads twice: the triangles form one component and carry zero bridges, so robbins promises a strong orientation.
2. A dfs from 0 orients the tree edges away, 0 to 1, 1 to 2 and 0 to 3, 3 to 4, and the closers 2 to 0 and 4 to 0 point back at the ancestor.
3. Each triangle is now a directed cycle, 0, 1, 2 and 0, 3, 4, sharing vertex 0: every vertex rides its own cycle to 0, then the other cycle anywhere.
4. The bridged cousin of the same fixture fails the second read: whichever way the bridge 2 to 3 points, the far side can never route anything back, so no orientation exists at all.

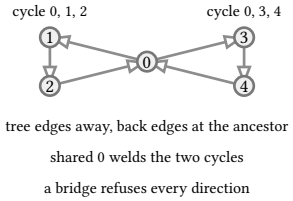


figure 359: the shared-vertex triangles oriented by the dfs rule, two directed cycles welded at vertex 0

Both reads cost one existing pass each, and the figure below draws the orientable and the bridged side by side.

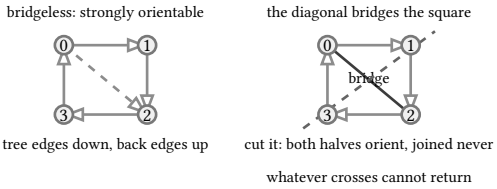


figure 360: a bridgeless graph oriented by the dfs rule, beside the same graph with its bridge cut and no strong orientation possible

39.12. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's ten sample files per language, c# spread over its five family files, go test files excluded:

language	build sloc	container dependency	boundary note
c	1563	static arc arrays, per-file mains	the shared 9-node tree inlined per file, 1-based fixtures translated at the boundary
go	1134	slices, sort	the treeadj helper shared across lca, hld, centroid, functional ports fully iterative
java	1607	jdk 27 stdlib	the frame-stack dfs ports verbatim from c, the prufer lcg draws through Integer.remainderUnsigned and trims its 8-row buffer by Arrays.copyOf, no stdlib sort in the chapter
c#	815	lists, linq, tuples	five family files, the 2-sat solver reuses the scc raw pass, priority queues in prufer
javascript	837	classes with private fields	1-indexed public faces over 0-based internals in lca and hld, the fenwick inside functional
python	956	stdlib only, inline asserts	brute twins per section, recursion where it reads best, iterative dfs throughout

language	build sloc	container dependency	boundary note
lua	1245	tables, 1-based inside	0-based reported positions, integer division for the lcg, check tables for run.lua

sources: cp-algorithms, “Strongly Connected Components and Condensation Graph”, cp-algorithms.com/graph/strongly-connected-components.html, “2-SAT”, cp-algorithms.com/graph/2SAT.html, “Finding Bridges in O(N+M)”, cp-algorithms.com/graph/bridge-searching.html, “Finding Articulation Points in O(N+M)”, cp-algorithms.com/graph/cutpoints.html, “Finding Bridges Online”, cp-algorithms.com/graph/bridge-searching-online.html, for the prose note on the link-cut variant, “Eulerian Path”, cp-algorithms.com/graph/euler_path.html, “Lowest Common Ancestor”, cp-algorithms.com/graph/lca.html, “Lowest Common Ancestor - Binary Lifting”, cp-algorithms.com/graph/lca_binary_lifting.html, “Lowest Common Ancestor - Tarjan’s off-line algorithm”, cp-algorithms.com/graph/lca_tarjan.html, “Solve RMQ by finding LCA”, cp-algorithms.com/graph/rmq_linear.html, for the prose equivalence, “Heavy-light decomposition”, cp-algorithms.com/graph/hld.html, “Centroid decomposition”, cp-algorithms.com/graph/centroid_decomposition.html, “Prüfer code”, cp-algorithms.com/graph/pruefer_code.html, “Second best Minimum Spanning Tree - Using Kruskal and Lowest Common Ancestor”, cp-algorithms.com/graph/second_best_mst.html, and “Strong Orientation”, cp-algorithms.com/graph/strong-orientation.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Functional graphs and the 2-core have no dedicated cp-algorithms article, both taught as general technique with the icpc world finals 2019 problem H and problem E solutions (book 10, chapter 10) as the application sources, alongside icpc world finals 2022 problem R (book 9, chapter 11). Sample behavior verified by the seven suite gates scoped to chapter 39: c 10 files and 208 checks, go 32 test functions, java 10 files and 210 checks under run-java-samples, c# 32 facts, javascript 33 tests and 79 asserts across its two split files, python 10 files and 80 asserts, lua 33 checks, zero skipped.

40. game theory and scheduling

Two players, perfect information, no chance: the question is always who wins, and the answer is a number. The sprague-grundy theorem turns every impartial game into a nim pile, so the chapter opens with mex tables and nim sums and the winning move that falls out of an xor. The second section moves the token onto an arbitrary directed graph, where win, lose, and draw are decided by one retrograde sweep from the terminals, and the matching characterization behind the 2025 blackboard game is stated in prose. The third section reframes the same minimax thinking as one player against time, four scheduling machines from the shortest-job-first baseline to the 2017 problem H window solver, and the chapter's fixtures reproduce that problem's judge-verified answers byte for byte.

40.1. sprague-grundy and nim

Every position of an impartial game earns a grundy number: the mex of the grundy numbers of its options, the minimal excluded value, so a position with options 0 and 2 earns 1 and a position with no options earns 0. The subtraction game with moves {1, 3, 4} tabulates in one line per position, and its table over n = 0 through 15 reads 0, 1, 0, 1, 2, 3, 2, 0, 1, 0, 1, 2, 3, 2, 0, 1, periodic with period 7 from n = 2 onward. The theorem's payload is that grundy numbers add by xor: a sum of independent games is equivalent to the nim pile whose size is the xor of the parts, and the first player wins exactly when that xor is nonzero. Nim itself is the pure case, pile sizes grundy numbers outright, and the winning move recomputes itself, the pile whose xor with the total drops.

The dry run: the fixtures are the subtraction table for moves 1, 3, 4 over n = 0 through 15 and the nim family, asserted by the C# suite with the same values pinned in C.

1. n = 0 has no move, its option set is empty, and the mex of nothing is 0.
2. n = 1 reaches only n = 0, options 0, mex 1, and n = 2 reaches only n = 1, options 1, mex 0.
3. n = 3 reaches 2 and 0, both grundy 0, options 0, mex 1, and n = 4 reaches 3, 1, 0, options 1, 1, 0, mex 2.
4. n = 5 reaches 4, 2, 1, options 2, 0, 1, mex 3, and n = 7 reaches 6, 4, 3, options 2, 2, 1, mex 0, one of the two zeros in the period-7 block that starts at n = 2.
5. Nim adds by xor: 3 xor 4 = 7 and 7 xor 5 = 2, nonzero, so first wins by cutting pile 3 to 3 xor 2 = 1, while 1 xor 2 xor 3 = 0 hands (1, 2, 3) to the second player.
6. The sums agree with the table: g(5) xor g(6) = 3 xor 2 = 1, first, and g(7) xor g(9) = 0 xor 0 = 0, second.

n	options reach grundy	g(n)
0	none	0
1	0	1
2	1	0
3	0, 0	1
4	1, 1, 0	2
5	2, 0, 1	3
6	3, 1, 0	2
7	2, 2, 1	0
8	0, 3, 2	1

The period-7 block and the xor verdicts land identically in every suite, and the listings below build the mex table and the nim analysis in seven languages.

```
static void build_subtraction(const int *moves, int mc, int nmax) {
  for (int x = 0; x ≤ nmax; x++) {
    g[x] = 0;
    // options: g[x - m] for each legal move
    int opts[8], oc = 0;
    for (int i = 0; i < mc; i++)
      if (moves[i] ≤ x)
        opts[oc++] = g[x - moves[i]];
    int seen[8] = {0};
    for (int i = 0; i < oc; i++)
      seen[opts[i]] = 1;
    int m = 0;
    while (seen[m])
      m++;
    g[x] = m;
  }
}

// the xor-total first/second verdict and the winning move pile/target
static int nim_move(const int *piles, int pc, int *pile_out, int *to_out) {
  int x = 0;
  for (int i = 0; i < pc; i++)
    x ^= piles[i];
  if (!x) {
    *pile_out = -1;
    *to_out = -1;
    return 0; // second wins
  }
  for (int i = 0; i < pc; i++)
    if ((piles[i] ^ x) < piles[i]) {
      *pile_out = i;
      *to_out = piles[i] ^ x;
      return 1;
    }
  return 1;
}
```

listing 1461: c, the mex table build and the nim verdict with its winning move (source: dsa/samples-c/src/Ch40/grundy.c, lines 24-59)

```
// GrundyTable builds the mex table for the subtraction game over
// the move set: g(n) = mex{g(n - m) : m in moves, m ≤ n}.
func GrundyTable(moves []int64, nmax int) []int64 {
  g := make([]int64, nmax+1)
  sorted := append([]int64(nil), moves...)
  sort.Slice(sorted, func(i, j int) bool { return sorted[i] < sorted[j] })
  for n := int64(1); n <= int64(nmax); n++ {
    seen := map[int64]bool{}
    for _, m := range sorted {
      if m > n {
        break
      }
      seen[g[n-m]] = true
    }
    mex := int64(0)
    for seen[mex] {
      mex++
    }
    g[n] = mex
  }
  return g
}

// NimMove evaluates the nim sum xor of pile sizes and computes the
// winning move: the pile whose xor with the total drops it.
// Returns (first player wins, [pile, from, to], move exists).
func NimMove(piles []int64) (bool, [3]int, bool) {
  xor := int64(0)
  for _, p := range piles {
    xor ^= p
  }
  if xor == 0 {
    return false, [3]int{}, false
  }
  for i, p := range piles {
    if target := p ^ xor; target < p {
      return true, [3]int{i, int(p), int(target)}, true
    }
  }
  return true, [3]int{}, false
}
```

listing 1462: go, the table over sorted moves, the nim analysis (source: dsa/samples-go/ch40/grundy.go, lines 5-45)

```
  static void buildSubtraction(int[] moves, int nmax) {
    for (int x = 0; x ≤ nmax; x++) {
      g[x] = 0;
      int[] opts = new int[8];
      int oc = 0;
      for (int move : moves)
        if (move ≤ x)
          opts[oc++] = g[x - move];
      boolean[] seen = new boolean[8];
      for (int i = 0; i < oc; i++)
        seen[opts[i]] = true;
      int m = 0;
      while (seen[m])
        m++;
      g[x] = m;
    }
  }

  static int pileOut, toOut;

  // the xor-total verdict and the winning move pile/target
  static int nimMove(int[] piles) {
    int x = 0;
    for (int p : piles)
      x ^= p;
    if (x == 0) {
      pileOut = -1;
      toOut = -1;
      return 0; // second wins
    }
    for (int i = 0; i < piles.length; i++)
      if ((piles[i] ^ x) < piles[i]) {
        pileOut = i;
        toOut = piles[i] ^ x;
        return 1;
      }
    return 1;
  }
```

listing 1463: java, the mex table build over a seen array, the nim verdict with its winning move (source: dsa/samples-java/src/Ch40/Grundy.java, lines 21-58)

```
    public static int[] SubtractionTable(int[] moves, int maxN)
    {
        var g = new int[maxN + 1];
        for (int n = 1; n ≤ maxN; n++)
        {
            var options = new HashSet<int>();
            foreach (int m in moves)
                if (m ≤ n)
                    options.Add(g[n - m]);
            int mex = 0;
            while (options.Contains(mex))
                mex++;
            g[n] = mex;
        }
        return g;
    }

    /// <summary>First player wins iff the xor is nonzero; the move recomputes the
    /// winning pile (the first whose value drops when xored with the total).</summary>
    public static (bool First, (int Pile, long To)? Move) Nim(long[] piles)
    {
        long total = piles.Aggregate(0L, (acc, p) => acc ^ p);
        if (total == 0)
            return (false, null); // second wins
        for (int i = 0; i < piles.Length; i++)
            if ((piles[i] ^ total) < piles[i])
                return (true, (i, piles[i] ^ total));
        return (true, null); // unreachable for a nonzero xor
    }
```

listing 1464: c#, the option set and its mex, the move from the pile that drops (source: dsa/samples/src/Ch40/Grundy.cs, lines 9-37)

```javascript
export function grundyTable(sub, nmax) {
  const g = new Array(nmax + 1).fill(0);
  for (let n = 1; n ≤ nmax; n++) {
    const reach = new Set();
    for (const m of sub) {
      if (m ≤ n) reach.add(g[n - m]);
    }
    let m = 0;
    while (reach.has(m)) m += 1;
    g[n] = m; // mex of the option set
  }
  return g;
}

export function nimAnalysis(piles) {
  let x = 0;
  for (const p of piles) x ^= p;
  let move = null;
  if (x =/= 0) {
    for (let i = 0; i < piles.length; i++) {
      if ((piles[i] ^ x) < piles[i]) {
        move = [i, piles[i], piles[i] ^ x];
        break;
      }
    }
  }
  return { xor: x, first: x =/= 0, move };
}
```

listing 1465: javascript, the mex loop and the nim analysis, xor on number (source: dsa/samples-js/src/ch40-grundy.mjs, lines 8-35)

```python
def grundy_table(moves, limit):
    table = [0] * (limit + 1)
    for m in range(1, limit + 1):
        reach = {table[m - move] for move in moves if move ≤ m}
        table[m] = mex(reach)
    return table


def mex(options):
    m = 0
    while m in options:
        m += 1
    return m


def nim_move(piles):
    total = 0
    for p in piles:
        total ^= p
    if total == 0:
        return None  # second wins, no move
    for i, p in enumerate(piles):  # the pile whose xor with total drops
        if (p ^ total) < p:
            return i, p, p ^ total
    return None
```

listing 1466: python, mex as a helper, the table, the move (source: dsa/samples-py/src/Ch40/grundy.py, lines 14-38)

```lua
local function mex(s)
  local m = 0
  while s[m] do m = m + 1 end
  return m
end

local function grundy_table(sub, nmax)
  local g = {[0] = 0}
  for n = 1, nmax do
    local reach = {}
    for _, m in ipairs(sub) do
      if m ≤ n then reach[g[n - m]] = true end
    end
    g[n] = mex(reach)
  end
  return g
end

-- returns first player wins, and {pile, from, to} for the winning move
local function nim(piles)
  local x = 0
  for _, p in ipairs(piles) do x = x ~ p end
  if x == 0 then return false, nil end
  for i, p in ipairs(piles) do
    if (p ~ x) < p then return true, {i, p, p ~ x} end
  end
  error("unreachable")
end
```

listing 1467: lua, the same pair, xor as the operator (source: dsa/samples-lua/ch40_grundy.lua, lines 8-35)

The misere caveat stays prose: when the loser is the player who takes the last object instead of the one who cannot move, only the endgame flips, and for nim specifically the strategy changes exactly when every pile is of size 1, where the parity of the pile count decides. Everything before that point plays identically.

The fixture families pin identical values in all seven languages. The table above and its period. The nim positions: (3, 4, 5) xors to 2, first wins by cutting the pile of 3 down to 1, (1, 2, 3) xors to 0 and the second player wins, a lone pile of 7 moves to 0, the twin piles (5, 5) cancel for the second player, and (1, 1, 2) wins by emptying the third pile. The sum table over the subtraction game pins g(5) xor g(5) = 0, g(5) xor g(6) = 1, g(5) xor g(10) = 2, g(7) xor g(9) = 0, both grundy numbers being 0 there, and g(9) xor g(9) = 0, first exactly when the cell is nonzero. The edges close it: the empty position xors to 0 and loses by convention, and a single pile of 1 moves to 0. The c suite asserts the 3, 4, 5 verdict twice in a row, a literal duplicate, and the java port drops the second copy, 33 grundy checks where c counts 34.

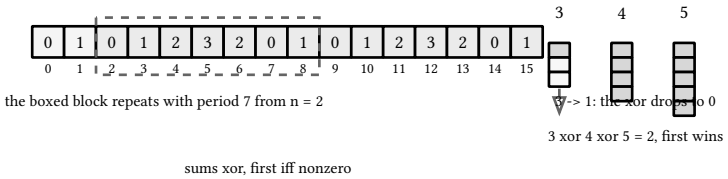


figure 361: the subtraction grundy ladder with the period-7 block boxed, beside the 3, 4, 5 nim piles and the 3 to 1 winning move

The application is general technique. The corpus holds no nim-sum finals problem, and the 2025 blackboard game is a graph game decided by matchings, the next section's business.

40.2. games on arbitrary graphs

Move the token onto a directed graph and let the rules be minimal: the player to move picks an out-edge and slides the token, and the player with no out-edge loses. Retrograde analysis labels every vertex from the terminals inward. A vertex with no out-edge loses for the mover. A vertex with a move into a losing vertex wins. A vertex whose every successor wins, loses. Whatever the sweep cannot reach draws, the cycles nobody can force either way. The implementation is degree counting with a queue over the reverse arcs, one pass, O(V + E).

The dry run: the fixture is the 8-vertex graph with its two mutations, asserted by the C# suite and pinned the same way in C.

1. Vertices 5 and 6 have no out-edge, the mover loses, so the queue seeds two L verdicts.
2. The wave runs over reverse arcs: vertex 4 points at 5 and 6, both L, so 4 is W through terminal 5.
3. Vertex 3 points at 1 and 4, one successor W and one undetermined, so no rule fires and its degree never drains, it stays pending.
4. The 3-cycle 1, 2, 3 and the 2-cycle 7, 8 never touch the wave, and the sweep leaves them D: D, D, D, W, L, L, D, D.
5. Deleting arc 3 to 4 changes nothing, 4 still wins through 5, and adding arc 6 to 5 gives 6 a move into L, flipping it to W: D, D, D, W, L, W, D, D.
6. The edges agree: a lone vertex is L, a self-loop only draws, and a vertex with an edge into a terminal is W.

vertex	verdict	why
5	L	no out-edge, terminal
6	L	no out-edge, terminal
4	W	a move into 5, an L vertex
3	D	successor 1 never decided
1	D	the 3-cycle 1, 2, 3
2	D	the 3-cycle 1, 2, 3
7	D	the 2-cycle 7, 8
8	D	the 2-cycle 7, 8

D, D, D, W, L, L, D, D is the pinned string, and the listings below run the sweep in seven languages.

```
static void solve(void) {
  memcpy(deg, outdeg, sizeof(int) * (size_t)nv);
  for (int v = 0; v < nv; v++)
    res[v] = 0;
  int queue[MAXV], qh = 0, qt = 0;
  for (int v = 0; v < nv; v++)
    if (outdeg[v] == 0) {
      res[v] = -1;
      queue[qt++] = v;
    }
  while (qh < qt) {
    int v = queue[qh++];
    for (int e = rhead[v]; e >= 0; e = rnext[e]) {
      int u = radj[e]; // u -> v
      if (res[u])
        continue;
      if (res[v] == -1) {
        res[u] = 1;
        queue[qt++] = u;
      } else {
        deg[u]--;
        if (deg[u] == 0) {
          res[u] = -1;
          queue[qt++] = u;
        }
      }
    }
  }
}
```

listing 1468: c, the retrograde loop, reverse arcs, degree counting (source: dsa/samples-c/src/Ch40/graphgames.c, lines 47-75)

```
func Retrograde(n int, edges [][2]int) []Outcome {
  reverse := make([][]int, n)
  outdeg := make([]int, n)
  for _, e := range edges {
    reverse[e[1]] = append(reverse[e[1]], e[0])
    outdeg[e[0]]++
  }
  res := make([]Outcome, n) // Draw until decided otherwise
  queue := make([]int, 0, n)
  for v := 0; v < n; v++ {
    if outdeg[v] == 0 {
      res[v] = Lose
      queue = append(queue, v)
    }
  }
  for len(queue) > 0 {
    v := queue[0]
    queue = queue[1:]
    for _, u := range reverse[v] {
      if res[u] != Draw {
        continue
      }
      if res[v] == Lose {
        res[u] = Win // u moves into a losing position
        queue = append(queue, u)
      } else if outdeg[u]--; outdeg[u] == 0 {
        res[u] = Lose // every successor wins: u loses
        queue = append(queue, u)
      }
    }
  }
  return res
}
```

listing 1469: go, the outcome enum, terminals seeded, predecessors decided (source: dsa/samples-go/ch40/graphgames.go, lines 20-52)

```
  static void solve() {
    System.arraycopy(outdeg, 0, deg, 0, nv);
    for (int v = 0; v < nv; v++)
      res[v] = 0;
    int[] queue = new int[MAXV];
    int qh = 0, qt = 0;
    for (int v = 0; v < nv; v++)
      if (outdeg[v] == 0) {
        res[v] = -1;
        queue[qt++] = v;
      }
    while (qh < qt) {
      int v = queue[qh++];
      for (int e = rhead[v]; e ≥ 0; e = rnext[e]) {
        int u = radj[e]; // u -> v
        if (res[u] ≠ 0)
          continue;
        if (res[v] == -1) {
          res[u] = 1;
          queue[qt++] = u;
        } else {
          deg[u]--;
          if (deg[u] == 0) {
            res[u] = -1;
            queue[qt++] = u;
          }
        }
      }
    }
  }
```

listing 1470: java, the retrograde loop, reverse arcs, degree counting (source: dsa/samples-java/src/Ch40/Graphgames.java, lines 46-75)

```
    public static char[] Solve(int n, (int U, int V)[] edges)
    {
        var adjacency = Enumerable.Range(0, n).Select(_ => new List<int>()).ToList();
        var outDegree = new int[n];
        foreach ((int u, int v) in edges)
        {
            adjacency[u].Add(v);
            outDegree[u]++;
        }
        var outcomes = new char[n];
        Array.Fill(outcomes, '?');
        var degree = (int[])outDegree.Clone();
        var queue = new Queue<int>();
        for (int v = 0; v < n; v++)
            if (outDegree[v] == 0)
            {
                outcomes[v] = 'L'; // the mover with no out-edge loses
                queue.Enqueue(v);
            }
        var reverse = Enumerable.Range(0, n).Select(_ => new List<int>()).ToList();
        foreach ((int u, int v) in edges)
            reverse[v].Add(u);
        while (queue.Count > 0)
        {
            int v = queue.Dequeue();
            foreach (int u in reverse[v])
                if (outcomes[u] == '?')
                {
                    if (outcomes[v] == 'L')
                    {
                        outcomes[u] = 'W'; // u moves into a losing position
                        queue.Enqueue(u);
                    }
                    else if (--degree[u] == 0)
                    {
                        outcomes[u] = 'L'; // every successor wins: u loses
                        queue.Enqueue(u);
                    }
                }
        }
        for (int v = 0; v < n; v++)
            if (outcomes[v] == '?')
                outcomes[v] = 'D'; // cycles without forcing draw
        return outcomes;
    }
```

listing 1471: c#, the same sweep, the draw assignment at the end (source: dsa/samples/src/Ch40/GraphGames.cs, lines 9-53)

```
export function solveGraph(n, edges) {
  const adj = Array.from({ length: n }, () => []);
  const radj = Array.from({ length: n }, () => []);
  const outdeg = new Array(n).fill(0);
  for (const [u, v] of edges) {
    adj[u].push(v);
    radj[v].push(u);
    outdeg[u] += 1;
  }
  const res = new Array(n).fill("?");
  const deg = [...outdeg];
  const q = [];
  for (let v = 0; v < n; v++) {
    if (outdeg[v] === 0) {
      res[v] = "L"; // terminals lose for the mover
      q.push(v);
    }
  }
  while (q.length > 0) {
    const v = q.shift();
    for (const u of radj[v]) {
      if (res[u] !== "?") continue;
      if (res[v] === "L") {
        res[u] = "W"; // u can move into a loss
        q.push(u);
      } else {
        deg[u] -= 1; // one successor decided against u
        if (deg[u] === 0) {
          res[u] = "L";
          q.push(u);
        }
      }
    }
  }
  for (let v = 0; v < n; v++) {
    if (res[v] === "?") res[v] = "D"; // cycles without forcing
  }
  return res;
}
```

listing 1472: javascript, the wave off the terminals, the undecided stay draws (source: dsa/samples-js/src/ch40-graphgames.mjs, lines 9-47)

```
def solve(count, edges):
    # outcome per vertex in {W, L, D}
    graph = [[] for _ in range(count)]
    outdegree = [0] * count
    for u, v in edges:
        graph[u].append(v)
        outdegree[u] += 1
    preds = [[] for _ in range(count)]
    for u, v in edges:
        preds[v].append(u)
    outcome = ["?"] * count
    remaining = outdegree[:]
    queue = deque()
    for v in range(count):
        if outdegree[v] == 0:  # terminals lose immediately
            outcome[v] = "L"
            queue.append(v)
    while queue:  # the retrograde wave
        v = queue.popleft()
        for u in preds[v]:  # reverse arcs, counted by degree
            if outcome[u] != "?":
                continue
            if outcome[v] == "L":  # u can move into a loss: u wins
                outcome[u] = "W"
                queue.append(u)
            else:  # one successor decided against u
                remaining[u] -= 1
                if remaining[u] == 0:  # every move loses: u loses
                    outcome[u] = "L"
                    queue.append(u)
    for v in range(count):
        if outcome[v] == "?":
            outcome[v] = "D"  # cycles never forced either way
    return outcome
```

listing 1473: python, the sweep and the draw default (source: dsa/samples-py/src/Ch40/graphgames.py, lines 17-50)

```
local function solve_graph(n, edges)
  local radj, outdeg = {}, {}
  for i = 0, n - 1 do
    radj[i + 1] = {}
    outdeg[i + 1] = 0
  end
  for _, e in ipairs(edges) do
    radj[e[2] + 1][#radj[e[2] + 1] + 1] = e[1]
    outdeg[e[1] + 1] = outdeg[e[1] + 1] + 1
  end
  local res = {}
  for i = 0, n - 1 do res[i + 1] = "?" end
  local deg = {table.unpack(outdeg)}
  local q = {}
  for v = 0, n - 1 do
    if outdeg[v + 1] == 0 then
      res[v + 1] = "L" -- no out-edge: the mover loses
      q[#q + 1] = v
    end
  end
  local head = 1
  while head ≤ #q do
    local v = q[head]
    head = head + 1
    for _, u in ipairs(radj[v + 1]) do -- predecessors decide off v
      if res[u + 1] == "?" then
        if res[v + 1] == "L" then
          res[u + 1] = "W" -- u can move into a loss
          q[#q + 1] = u
        else -- v is W: one successor decided against u
          deg[u + 1] = deg[u + 1] - 1
          if deg[u + 1] == 0 then
            res[u + 1] = "L" -- every move wins for the opponent
            q[#q + 1] = u
          end
        end
      end
    end
  end
  for v = 0, n - 1 do
    if res[v + 1] == "?" then res[v + 1] = "D" end
  end
  return res
end
```

listing 1474: lua, the same loop over reverse adjacency (source: dsa/samples-lua/ch40_graphgames.lua, lines 10-53)

The fixture graph runs 8 vertices with arcs 1 to 2, 2 to 3, 3 back to 1, 3 to 4, 4 to 5, 4 to 6, 7 to 8, 8 back to 7, and vertices 5 and 6 terminal. Outcomes read D, D, D, W, L, L, D, D: the 3-cycle draws, vertex 4 wins through terminal 5, both terminals lose, and the 2-cycle draws. The mutation family probes the labeling: deleting edge 3 to 4 changes nothing, 4 still wins through 5, and adding edge 6 to 5 flips vertex 6 from losing to winning, its move now reaching a losing position. The edges: a lone vertex loses, a self-loop only draws, and a vertex with an edge into a terminal wins.

Against the alpha-beta search of book 9, ch 20 (practical data structures and algorithms, pruning and branch and bound), this is the same question at graph scale: alpha-beta walks a small game tree once, retrograde analysis memoizes the whole position graph and answers every start in one sweep.

The matching characterization stays prose but carries the contest weight: for the slide-a-token family where moves never repeat a vertex, the second player wins exactly when a maximum matching covers the token's starting component, because the matched edge gives the second player an answer to every first move, and an unmatched start lets the first player walk the matching forever. That is how the 2025 blackboard game decides its openings, and the matching machinery itself is the kuhn section of book 9, ch 38 (practical data structures and algorithms, network flows ii).

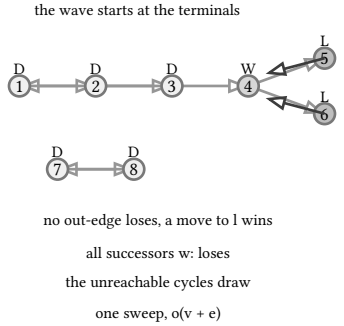


figure 362: the 8-vertex fixture colored w, l, d, the retrograde wave arrows out of the terminals 5 and 6

The application is icpc world finals 2025 problem B (book 10, chapter 13), blackboard game, the prime-multiple graph whose openings are decided by the matching characterization stated above, with the kuhn solver of chapter 38 doing the matching.

40.3. scheduling

One player against time is still a game, lose only when the deadline arrives, and the winning moves are orders. Four machines ship in one file. On a single machine minimizing total completion time, the shortest-processing-time order wins, every swap of a longer job in front of a shorter one delays everyone behind. On two machines in series, johnson's rule wins: jobs with a under b sorted by a ascending, then the rest by b descending, and the makespan falls out of the max-cascade, machine 2 waiting for machine 1 and for its own previous job. Unit jobs with deadlines are feasible exactly under the earliest-deadline-first order, slot k meeting the k-th tightest deadline or nothing will. The fourth machine is the 2017 problem H window solver, photos of common duration t inside per-photo windows [a, b], decided by the two-phase garey-johnson-simons-tarjan construction: phase 1 sweeps release times descending, stacks each suffix as late as possible pushed out of forbidden start regions, and marks [C-t+1, s-1] forbidden when the chain leaves C - s under t, phase 2 runs the earliest-deadline greedy over candidate starts, the current time, release times, and region ends, never starting inside a region.

The dry run: the fixtures are the spt durations 5, 2, 8, 1, the johnson five-job set, the edf deadline families, and the 2017 problem H windows, asserted by the C# suite against the judge answers.

1. Spt sorts 5, 2, 8, 1 into durations 1, 2, 5, 8: the clock reads 1, 1 + 2 = 3, 3 + 5 = 8, 8 + 8 = 16, total 28 against the input order's 43.
2. Johnson splits (3, 5), (1, 2), (4, 1), (2, 6), (5, 4): jobs with a under b lead sorted by a, 1, 2, 3, the rest trail by b descending, 4, 1, so the order reads 1, 3, 0, 4, 2.
3. The cascade: machine 1 finishes 1, 3, 6, 11, 15 while machine 2 pays max(t2, t1) + b, landing 3, 9, 14, 18, 19, the makespan 19 the 120-order brute confirms.
4. Edf checks slot k against the k-th tightest deadline: 3, 3, 3, 7, 7, 7 passes at 1, 2, 3, 4, 5, 6, and 3, 1, 2, 3 stalls at slot 4, deadline 3, infeasible.
5. The window mini, three photos of t = 2 in (0,8), (1,3), (4,6): phase 1 stacks the release-4 suffix to start 6 - 2 = 4 and, zero slack above the release, marks 4 - 2 + 1 = 3 forbidden, then the release-1 suffix lands (1,3) at 3 - 2 = 1 and marks start 0.
6. Phase 2 then starts (1,3) at 1, (4,6) at 4, (0,8) at 6, ending 3, 6, 8, the verdict yes.

step	job (a, b)	t1 after	t2 = max + b
1	(1, 2)	1	max(0, 1) + 2 = 3
2	(2, 6)	3	max(3, 3) + 6 = 9
3	(3, 5)	6	max(9, 6) + 5 = 14
4	(5, 4)	11	max(14, 11) + 4 = 18
5	(4, 1)	15	max(18, 15) + 1 = 19

The 19 equals the brute optimum and the window verdicts reproduce 2017 problem H, and the listings below ship the four machines in seven languages.

```
  for (int si = 0; si < sn; si++) {
    int s = starts[si];
    // the batch: photos with release ≥ s, latest deadline first
    int batch[MAXJ], bn = 0;
    for (int i = 0; i < jc; i++)
      if (ph[i][0] ≥ s)
        batch[bn++] = i;
    for (int i = 1; i < bn; i++)
      for (int j = i; j > 0 && (ph[batch[j - 1]][1] < ph[batch[j]][1] ||
                                (ph[batch[j - 1]][1] == ph[batch[j]][1] &&
                                 ph[batch[j - 1]][0] < ph[batch[j]][0]));
           j--) {
        int tt = batch[j];
        batch[j] = batch[j - 1];
        batch[j - 1] = tt;
      }
    long long cur = -1, C = -1;
    for (int i = 0; i < bn; i++) {
      long long end = cur < 0 ? ph[batch[i]][1]
                              : (ph[batch[i]][1] < cur ? ph[batch[i]][1] : cur);
      int st = adjust_down(end - t);
      C = st;
      cur = st;
    }
    if (C < s)
      return 0;
    if (C - s < t) {
      int lo = C - t + 1, hi = s - 1;
      if (lo ≤ hi) {
        int merged = 0;
        for (int r = 0; r < nreg; r++)
          if (!(regions[r][1] + 1 < lo || hi + 1 < regions[r][0])) {
            if (lo < regions[r][0])
              regions[r][0] = lo;
            if (hi > regions[r][1])
              regions[r][1] = hi;
            merged = 1;
            break;
          }
        if (!merged) {
          regions[nreg][0] = lo;
          regions[nreg][1] = hi;
          nreg++;
        }
      }
    }
  }
```

listing 1475: c, the phase 1 sweep, the as-late chain pushed out of regions, the forbidden marking (source: dsa/samples-c/src/Ch40/scheduling.c, lines 149-195)

```
  for _, s := range rel {
    batch := [][2]int64{}
    for _, p := range photos {
      if p[0] ≥ s {
        batch = append(batch, p)
      }
    }
    sort.Slice(batch, func(x, y int) bool {
      if batch[x][1] ≠ batch[y][1] {
        return batch[x][1] > batch[y][1] // tightest deadline
      }
      return batch[x][0] > batch[y][0]
    })
    var chainEnd int64 // the batch always holds s's own photo
    for k, p := range batch {
      latest := p[1]
      if k > 0 && chainEnd < latest {
        latest = chainEnd
      }
      chainEnd = adjustDown(latest - t)
    }
    if chainEnd < s {
      return false
    }
    if chainEnd-s < t {
      lo, hi := chainEnd-t+1, s-1
      if lo <= hi {
        merged := false
        for i := range regions {
          r := &regions[i]
          if !(r[1]+1 < lo || hi+1 < r[0]) {
            if lo < r[0] {
              r[0] = lo
            }
            if hi > r[1] {
              r[1] = hi
            }
            merged = true
            break
          }
        }
        if !merged {
          regions = append(regions, [2]int64{lo, hi})
        }
      }
    }
  }
```

listing 1476: go, the release sweep, the tightest-deadline batch, the marking (source: dsa/samples-go/ch40/scheduling.go, lines 163-209)

```
    for (int si = 0; si < sn; si++) {
      int s = starts[si];
      // the batch: photos with release ≥ s, latest deadline first
      int[] batch = new int[MAXJ];
      int bn = 0;
      for (int i = 0; i < jc; i++)
        if (ph[i][0] ≥ s)
          batch[bn++] = i;
      for (int i = 1; i < bn; i++)
        for (int j = i; j > 0 && (ph[batch[j - 1]][1] < ph[batch[j]][1]
            || (ph[batch[j - 1]][1] == ph[batch[j]][1]
                && ph[batch[j - 1]][0] < ph[batch[j]][0])); j--) {
          int tt = batch[j];
          batch[j] = batch[j - 1];
          batch[j - 1] = tt;
        }
      long cur = -1, chainEnd = -1;
      for (int i = 0; i < bn; i++) {
        long end = cur < 0 ? ph[batch[i]][1]
            : (ph[batch[i]][1] < cur ? ph[batch[i]][1] : cur);
        int st = adjustDown(end - t);
        chainEnd = st;
        cur = st;
      }
      if (chainEnd < s)
        return false;
      if (chainEnd - s < t) {
        int lo = (int) chainEnd - t + 1, hi = s - 1;
        if (lo ≤ hi) {
          boolean merged = false;
          for (int r = 0; r < nreg; r++)
            if (!(regions[r][1] + 1 < lo || hi + 1 < regions[r][0])) {
              if (lo < regions[r][0])
                regions[r][0] = lo;
              if (hi > regions[r][1])
                regions[r][1] = hi;
              merged = true;
              break;
            }
          if (!merged) {
            regions[nreg][0] = lo;
            regions[nreg][1] = hi;
            nreg++;
          }
        }
      }
    }
```

listing 1477: java, the phase 1 sweep, the as-late chain pushed out of regions, the forbidden marking, a byte-for-byte port of the c loop (source: dsa/samples-java/src/Ch40/Scheduling.java, lines 139-185)

```
        var starts = photos.Select(p => p.Lo).Distinct().OrderByDescending(v => v).ToList();
        foreach (long s in starts)
        {
            var batch = photos
                .Where(p => p.Lo >= s)
                .OrderByDescending(p => p.Hi)
                .ThenByDescending(p => p.Lo)
                .ToList();
            long chain = 0;
            long cur = 0;
            foreach ((_, long hi) in batch)
            {
                long end = chain == 0 ? hi : Math.Min(hi, cur);
                long start = AdjustDown(end - t);
                cur = start;
                chain = start;
            }
            if (chain < s)
                return false;
            if (chain - s < t)
            {
                long lo = chain - t + 1;
                long hi = s - 1;
                if (lo <= hi)
                {
                    bool merged = false;
                    for (int r = 0; r < regions.Count; r++)
                        if (regions[r].Hi + 1 >= lo && hi + 1 >= regions[r].Lo)
                        {
                            regions[r] = (Math.Min(regions[r].Lo, lo), Math.Max(regions[r].Hi, hi));
                            merged = true;
                            break;
                        }
                    if (!merged)
                        regions.Add((lo, hi));
                }
            }
        }
```

listing 1478: c#, the same sweep, adjust down, the region merge, the greedy below (source: dsa/samples/src/Ch40/Scheduling.cs, lines 83-120)

```
  const starts = [...new Set(photos.map((p) => p[0]))].sort((a, b) => b - a);
  for (const s of starts) {
    const batch = photos.filter((p) => p[0] >= s)
      .sort((x, y) => y[1] - x[1] || y[0] - x[0]);
    let cur = null;
    let C = null;
    for (const [a, b] of batch) {
      const end = cur === null ? b : Math.min(b, cur);
      const st = adjustDown(end - t);
      C = st;
      cur = st;
    }
    if (C < s) return false;
    if (C - s < t) {
      const lo = C - t + 1;
      const hi = s - 1;
      if (lo <= hi) {
        let merged = false;
        for (const r of regions) {
          if (!(r[1] + 1 < lo || hi + 1 < r[0])) {
            r[0] = Math.min(r[0], lo);
            r[1] = Math.max(r[1], hi);
            merged = true;
            break;
          }
        }
        if (!merged) regions.push([lo, hi]);
      }
    }
  }
```

listing 1479: javascript, phase 1 whole, the region bookkeeping (source: dsa/samples-js/src/ch40-scheduling.mjs, lines 76-105)

```
    releases = sorted({p[0] for p in photos}, reverse=True)
    for s in releases:
        batch = sorted(
            (p for p in photos if p[0] ≥ s), key=lambda p: (-p[1], -p[0])
        )  # tightest deadline
        chain_end = None
        for a, b in batch:
            latest = b if chain_end is None else min(b, chain_end)
            start = adjust_down(latest - duration)
            chain_end = start
        if chain_end < s:
            return False
        if chain_end - s < duration:  # too tight: forbid the region
            lo, hi = chain_end - duration + 1, s - 1
            if lo ≤ hi:
                merged = False
                for region in regions:
                    if not (region[1] + 1 < lo or hi + 1 < region[0]):
                        region[0] = min(region[0], lo)
                        region[1] = max(region[1], hi)
                        merged = True
                        break
                if not merged:
                    regions.append([lo, hi])
```

listing 1480: python, releases descending, the chain, the too-tight region (source: dsa/samples-py/src/Ch40/scheduling.py, lines 90-113)

```
  local starts = {}
  for _, p in ipairs(photos) do starts[p[1]] = true end
  local sorted_starts = {}
  for s in pairs(starts) do sorted_starts[#sorted_starts + 1] = s end
  table.sort(sorted_starts, function(a, b) return a > b end)
  for _, s in ipairs(sorted_starts) do
    -- the suffix of photos with release ≥ s, tightest deadline first
    local batch = {}
    for _, p in ipairs(photos) do
      if p[1] ≥ s then batch[#batch + 1] = {p[1], p[2]} end
    end
    table.sort(batch, function(x, y)
      if x[2] ~= y[2] then return x[2] > y[2] end
      return x[1] > y[1]
    end)
    local cur, C = nil, nil
    for _, p in ipairs(batch) do
      local last = (cur == nil) and p[2] or math.min(p[2], cur)
      local st = adjust_down(last - t)
      C = st
      cur = st
    end
    if C < s then return false end
    if C - s < t then
      local lo, hi = C - t + 1, s - 1
      if lo ≤ hi then
        local merged = false
        for _, r in ipairs(regions) do
          if not (r[2] + 1 < lo or hi + 1 < r[1]) then
            r[1] = math.min(r[1], lo)
            r[2] = math.max(r[2], hi)
            merged = true
            break
          end
        end
        if not merged then regions[#regions + 1] = {lo, hi} end
      end
    end
  end
```

listing 1481: lua, the same sweep and marking, phase 2 below (source: dsa/samples-lua/ch40_scheduling.lua, lines 98-136)

The simplified teaching form is O(n^2) against the shipped solver's sorts, with the semantics mirrored exactly, and the reference reproduces all six judge-verified answers of the 2017 problem H statement.

The spt fixture runs durations 5, 2, 8, 1: order the jobs 1, 2, 8, 5 by index, completions 1, 3, 8, 16, total 28 against the input order's
43. Johnson on (3,5), (1,2), (4,1), (2,6), (5,4) pins the canonical

order and makespan 19, equal to the brute optimum over all 120 orders, the brute's own lexicographic optimum differing at the tail and the canonical output being the pinned one, and the all-ties instance of three (2,3) jobs keeps input order at makespan 11. The edf family: deadlines 1, 2, 3 feasible in order, 3, 3, 3, 7, 7, 7 feasible, 3, 1, 2, 3 infeasible because the greedy stalls at slot 4, and 1, 1, 2 infeasible outright. The window machine: the six judge-verified cases of 2017 problem H answer yes, no, yes, yes, no, yes, and the crafted pair, a single photo in a generous window and four photos fighting for overlapping windows, answer yes and no. Java's window machine ports the c phase 1 loop statement for statement, adjust-down and the region merge identical, so the six verdicts land on

the same orbit. The edges: a single photo is feasible whenever its window fits t, and two identical windows decide by 2t against the span, both fitting at a span of 4 with t = 2, neither at a span of 3.

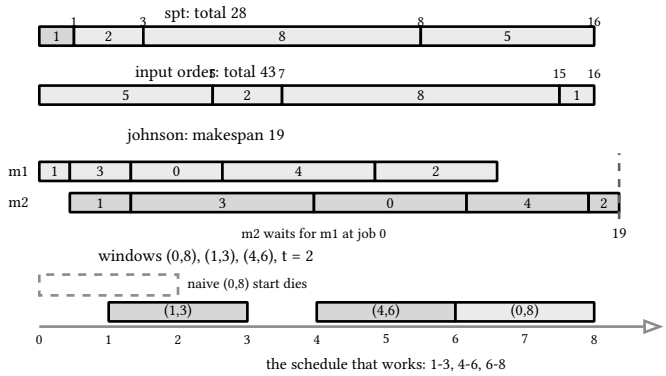


figure 363: the spt timeline against the input-order one, the johnson gantt with its makespan cascade, and the forbidden region blocking the naive start in the 2017 problem H mini

The application is icpc world finals 2017 problem H (book 10, chapter 8), scenery, where the garey-johnson-simons-tarjan two-phase machine is exactly the fourth solver, and the shipped seven-language solvers of the icpc book are its contest-grade siblings. When an instance outruns every exact solver, the next tier over surrenders on purpose and keeps a number: book 9, ch 42 (practical data structures and algorithms, approximation algorithms) and its proven ratios.

40.4. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's three sample files per language, go test files excluded:

language	build sloc	container dependency	boundary note
c	440	static arrays, insertion sorts	all four machines in one file, the judge-verified windows answers pinned, 65 checks
go	328	slices, sort	the johnson brute as ground truth, the outcome enum for retrograde
java	447	jdk 27 stdlib	the phase 1 window machine a byte-for-byte port of the c loop, the grundy port drops the c suite's one duplicated verdict check, insertion sorts hand-rolled
c#	225	linq ordering throughout	the window machine as one method, phase comments mirroring the shipped solver
javascript	179	node stdlib	tightest file of the wave, the window machine under 70 lines
python	263	stdlib only, inline asserts	itertools permutations as the johnson oracle, four families pinned
lua	338	tables, lib.lua harness	1-based jobs and deadlines shifted at the boundary, check tables for run.lua

sources: cp-algorithms, "Sprague-Grundy theorem. Nim", cp-algorithms.com/game_theory/sprague-grundy-nim.html, "Games on arbitrary graphs", cp-algorithms.com/game_theory/games_on_graphs.html, "Scheduling jobs on one machine", cp-algorithms.com/schedules/schedule_one_machine.html, "Scheduling jobs on two machines", cp-algorithms.com/schedules/schedule_two_machines.html, and "Optimal schedule of jobs given their deadlines and durations", cp-algorithms.com/schedules/schedule-with-completion-duration.html, all accessed 2026-09-20, cc by-sa 4.0, our own words and code throughout. Application sources: icpc world finals 2025 problem B and 2017 problem H (book 10, chapters 13 and 8). Sample behavior verified by the seven suite gates scoped to

chapter 40: c 3 files and 65 checks, go 12 test functions, java 3 files and 64 checks under run-java-samples, c# 12 facts, javascript 12 tests and 47 asserts, python 3 files and 41 asserts, lua 12 checks, zero skipped.

41. local search and metaheuristics

Exact methods buy guarantees and pay in scale: held-karp dies around 20 cities, branch and bound dies on the wrong instance shape. This chapter is about what runs when the exact tool stops scaling, under three house rules: keep the objective an integer, keep the search reproducible from a seed, and know exactly what the guarantee became. Every algorithm here is a seeded deterministic program with its accepted moves, counters, and finals pinned by test, in all seven languages, never a coin flip. The tour runs from hill climbing through 2-opt, simulated annealing with a rational Metropolis rule, tabu search with a move-expiry memory, genetic algorithms, ant colony optimization with integer pheromones, and restart portfolios, all on small integer fixtures whose optima a brute force confirms in-test.

41.1. local search: the frame

Three questions define every algorithm of the chapter. What is the state, here a city permutation or a bitstring. What is the neighborhood, the set of states one move away. What is the objective, an integer energy to minimize. Given the three, the driver is one paragraph of code: scan the neighborhood in a fixed order, take the first strictly better move, restart the scan from the beginning after every accept, stop when a full scan finds nothing. The accepted-move sequence is then a pure function of the start state, the energy, and the scan order, which is what makes the exact lane assertable byte for byte across seven languages.

The stochastic algorithms later only swap the acceptance rule for draws from a seeded generator, so the generator is the chapter's meter stick and gets pinned first. The state x is an unsigned 64-bit integer, each step computes x = (A x + C) mod 2^64 with A = 6364136223846793005 and C = 1442695040888963407, and the output is the high 32 bits u read as unsigned. A bounded draw below(k) = floor(u k / 2^32) consumes exactly one u, so draw counts are meters too. The constants split into exact 32-bit limbs, A = 0x5851F42D4C957F2D and C = 0x14057B7EF767814F, and JavaScript carries the state as two 32-bit halves multiplied through 16-bit limbs, because a single decimal literal past 2^53 is an inexact Number and BigInt stays out. Java sits at the opposite pole: long is 64-bit and wraps mod 2^64, so the multiply-add is one statement with zero limb work, and below(k) masks both operands to 32 bits, multiplies into a long, and shifts >>> 32.

The dry run: the seven-way contract pins identical literals and expected values in every suite, the lcg vectors first, then the ring fixture with its brute-force optimum.

1. Seed 1 opens 1817669548, 2187888307, 2784682393, 1644385741 and seed 42 opens 2440530669, 968358053: 24 integers, 8 per seed across seeds 1, 2, 42, asserted as the first test of each tree's localsearch file.
2. below(k) consumes exactly one output, the draw meter asserted in python, c#, and javascript: lua pins below(65536) = u >> 16 as an identity, javascript and c# recompute the fold against a bigint oracle across k, below(8) = u >> 29 is the derivation behind the tournament draws of section 8, and below(1) is always 0, asserted in python and c.
3. The ring F1 is 8 cities on a 4x4 grid perimeter under the Manhattan metric: d[0][4] = 8, d[3][5] = 4, and the identity tour 0..7 costs 16.
4. The brute-force oracle over the 5040 permutations with city 0 fixed finds 16, c pinning the first winner as the ring order itself.
5. The swap neighborhood at n = 8 holds 28 moves, counted in c and python, and the insert neighborhood holds 56, n(n-1), counted in python.
6. The driver on the identity start accepts nothing, trace [16] pinned in python and go, and the start with positions 0 and 1 exchanged repairs in one move, trace 20, 16, pinned in python.

The frame lands its first two numbers, 16 and 5040, and the listings below ship the generator and the fixture in seven languages.

```c
// x = A * x + C wraps mod 2^64, output is the high 32 bits
static uint64_t lcg_state;

static void lcg_seed(uint64_t s) { lcg_state = s; }

static uint32_t lcg_next(void) {
  lcg_state = 6364136223846793005ULL * lcg_state + 1442695040888963407ULL;
  return (uint32_t)(lcg_state >> 32);
}

static uint32_t below(uint32_t k) {
  return (uint32_t)(((uint64_t)lcg_next() * k) >> 32);
}

// the ring: 8 cities on a 4x4 grid perimeter, manhattan metric
static const int ring[8][8] = {
    {0, 2, 4, 6, 8, 6, 4, 2}, {2, 0, 2, 4, 6, 4, 6, 4},
    {4, 2, 0, 2, 4, 6, 8, 6}, {6, 4, 2, 0, 2, 4, 6, 4},
    {8, 6, 4, 2, 0, 2, 4, 6}, {6, 4, 6, 4, 2, 0, 2, 4},
    {4, 6, 8, 6, 4, 2, 0, 2}, {2, 4, 6, 4, 6, 4, 2, 0},
};
```

listing 1482: c, the wrapped multiply-add generator, its high-32 output, the bounded draw, the ring matrix as data (source: dsa/samples-c/src/Ch41/localsearch.c, lines 18-38)

```go
type Lcg struct {
  x     uint64
  Draws int64 // bounded draws consumed, for the pinned meter checks
}

// NewLcg seeds the generator. Seeds 1, 2 and 42 carry the pinned
// witness vectors asserted in the tests.
func NewLcg(seed uint64) *Lcg {
  return &Lcg{x: seed}
}

// Next advances the state once and returns the high 32 bits read as
// an unsigned integer.
func (g *Lcg) Next() uint32 {
  // A = 0x5851F42D4C957F2D, C = 0x14057B7EF767814F, uint64 wraps.
  g.x = 0x5851F42D4C957F2D*g.x + 0x14057B7EF767814F
  return uint32(g.x >> 32)
}

// Below maps one fresh draw onto 0..k-1 with no floats: the product
// u*k stays under 2^48 for k ≤ 65536, and its high half is exactly
// floor(u*k / 2^32).
func (g *Lcg) Below(k uint32) uint32 {
  g.Draws++
  return uint32((uint64(g.Next()) * uint64(k)) >> 32)
}
```

listing 1483: go, the generator struct, the high-32 next, the floor draw with its meter (source: dsa/samples-go/ch41/localsearch.go, lines 9-34)

```
  static long lcgState;

  static void lcgSeed(long s) {
    lcgState = s;
  }

  static int lcgNext() {
    lcgState = 6364136223846793005L * lcgState + 1442695040888963407L;
    return (int) (lcgState >>> 32);
  }

  static int below(int k) {
    return (int) (((lcgNext() & 0xFFFFFFFFL) * (k & 0xFFFFFFFFL)) >>> 32);
  }

  // the ring: 8 cities on a 4x4 grid perimeter, manhattan metric
  static final int[][] ring = {
      {0, 2, 4, 6, 8, 6, 4, 2}, {2, 0, 2, 4, 6, 4, 6, 4},
      {4, 2, 0, 2, 4, 6, 8, 6}, {6, 4, 2, 0, 2, 4, 6, 4},
      {8, 6, 4, 2, 0, 2, 4, 6}, {6, 4, 6, 4, 2, 0, 2, 4},
      {4, 6, 8, 6, 4, 2, 0, 2}, {2, 4, 6, 4, 6, 4, 2, 0}};
```

listing 1484: java, the multiply-add generator straight on long, the >>> 32 output, the bounded draw, the ring matrix as data (source: dsa/samples-java/src/Ch41/Localsearch.java, lines 18-38)

```
public sealed class Lcg
{
    private const ulong A = 0x5851F42D4C957F2DUL; // 6364136223846793005
    private const ulong C = 0x14057B7EF767814FUL; // 1442695040888963407

    private ulong _x;

    public int Draws { get; private set; }

    public Lcg(ulong seed) => _x = seed;

    public uint Next()
    {
        unchecked
        {
            _x = A * _x + C;
        }
        return (uint)(_x >> 32);
    }

    /// <summary>Bounded draw floor(u * k / 2^32): one output consumed per
    /// call, draws never skip.</summary>
    public int Below(uint k)
    {
        uint u = Next();
        Draws++;
        return (int)(((ulong)u * k) >> 32);
    }
}
```

listing 1485: c#, the lcg as one unchecked multiply-add over ulong, the bounded draw metered (source: dsa/samples/src/Ch41/Localsearch.cs, lines 9-37)

```
const AH = 0x5851f42d; // 1481765933
const AL = 0x4c957f2d; // 1284865837
const CH = 0x14057b7e; // 335903614
const CL = 0xf767814f; // 4150755663

// (x*y) mod 2^32 plus the carry into bit 32, both operands < 2^32,
// every partial product < 2^33 so nothing leaves the exact range
function mul32lo(x, y) {
  const xh = Math.floor(x / 65536);
  const xl = x % 65536;
  const yh = Math.floor(y / 65536);
  const yl = y % 65536;
  const ll = xl * yl;
  const mid = xh * yl + xl * yh;
  const midLo = (mid % 65536) * 65536;
  const midHi = Math.floor(mid / 65536);
  const hh = xh * yh;
  const raw = midLo + ll;
  return {
    lo: raw % 4294967296,
    carry: hh + midHi + (raw ≥ 4294967296 ? 1 : 0),
  };
}

export function makeLcg(seed) {
  let xh = 0;
  let xl = seed >>> 0;
  let draws = 0;
  const next = () => {
    const low = mul32lo(AL, xl);
    const rawLow = low.lo + CL;
    const newXL = rawLow % 4294967296;
    const carryLow = low.carry + (rawLow ≥ 4294967296 ? 1 : 0);
    const h1 = mul32lo(AL, xh);
    const h2 = mul32lo(AH, xl);
    xh = (h1.lo + h2.lo + CH + carryLow) % 4294967296;
    xl = newXL;
    return xh;
  };
```

listing 1486: javascript, the state as 32-bit halves multiplied through 16-bit limbs, no bigint (source: dsa/samples-js/src/ch41-localsearch.mjs, lines 10-48)

```
MASK = (1 << 64) - 1
A = 6364136223846793005
C = 1442695040888963407


class Lcg:
    # state x is an unsigned 64-bit integer, step x = (A*x + C) mod 2^64,
    # the output is the high 32 bits, below(k) = floor(u * k / 2^32)

    def __init__(self, seed):
        self.x = seed & MASK
        self.draws = 0

    def next(self):
        self.x = (A * self.x + C) & MASK
        return self.x >> 32

    def below(self, k):
        u = self.next()
        self.draws += 1
        return (u * k) >> 32
```

listing 1487: python, the mask-and-multiply generator, the same 24 integers asserted first (source: dsa/samples-py/src/Ch41/localsearch.py, lines 17-37)

```
local A = 6364136223846793005
local C = 1442695040888963407

-- x is the unsigned 64-bit state: the multiply-add wraps mod 2^64 in
-- lua integer arithmetic (no float ever touches the state), the output
-- is the high 32 bits through the logical shift
local function lcg(seed)
  local x = seed
  local function nextu()
    x = x * A + C
    return x >> 32
  end
  local function below(k) return (nextu() * k) >> 32 end
  return { next = nextu, below = below }
end
```

listing 1488: lua, integer arithmetic wrapping mod 2^64, the logical shift output (source: dsa/samples-lua/ch41_localsearch.lua, lines 8-22)

The fixture family behind the section: the 24 witness integers, the below laws with one draw per call, the pinned 8x8 matrix with its perimeter rows, the 5040-permutation oracle returning 16 with the ring order as first winner, the 28 and 56 neighborhood counts, and the two driver checks, zero moves from the optimum and one swap repair from [1, 0, 2, 3, 4, 5, 6, 7]. The convex hex F2 rides along as the second fixture: 6 lattice points in convex position, Manhattan matrix pinned the same way, optimum 20 achieved by the hull order 0..5 and confirmed by a brute force over the 120 permutations.

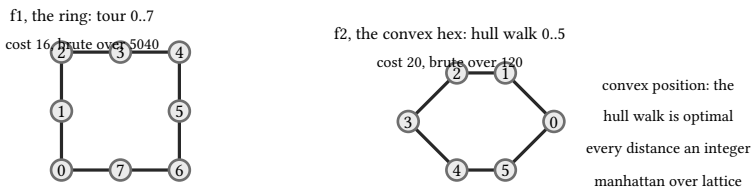


figure 364: the ring f1 and the convex hex f2 with their optimal tours, 16 and 20, every edge a manhattan lattice distance

The generator has one more teaching point worth the ink: the multiplier and increment are Knuth's recommendations for 64-bit congruential generators, and every language reaches the same stream through its own native wrap, unsigned overflow in C, unchecked ulong in C#, uint64 in Go, a mask in Python, integer arithmetic in Lua, and the limb dance in JavaScript. When a fixture looks wrong in one tree, the 24 integers say which side drifted.

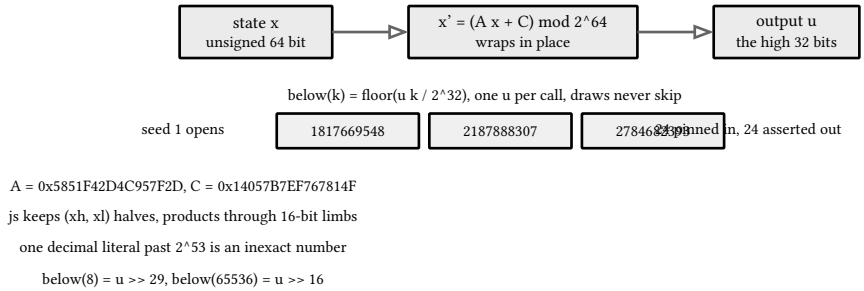


figure 365: the seeded lcg as the chapter's meter stick: the wrapped multiply-add, the high 32 bits, the bounded fold, the limb spelling

41.2. hill climbing and plateaus

First improvement takes the first strictly better neighbor in scan order and restarts the scan after the accept, eager and cheap per round. Best improvement scans the whole neighborhood and applies the single minimum, first move winning ties, more work per round for a steeper step. On the same energy surface the two rules walk different accepted-move sequences, and both are deterministic: pin the scan order, ascending position pairs (i, j) with i < j, and the traces follow. Strict descent has one blind spot, the plateau: an equal-energy move is never taken, so a run that steps onto level ground stops there even when the path across it leads down.

The dry run: the seven-way contract pins the two swap traces and the plateau boundary, byte-identical energy sequences in all seven suites.

1. The swap climb from [0, 4, 1, 5, 2, 6, 3, 7] walks 44, 36, 32, 28, 24, 20, 16 and finishes on the ring order 0..7, the optimum.
2. The swap climb from [0, 2, 4, 6, 1, 3, 5, 7] walks 32, 28 and stops at [0, 6, 4, 2, 1, 3, 5, 7], a different local optimum.
3. Best improvement from the first start walks 44, 28, 24, the trace pinned in python and lua with lua carrying the final tour, go, javascript, and c# checking the strictly decreasing and swap-local invariants, c shipping no best variant: two full scans, two accepted moves, a third scan finds nothing.
4. At [0, 1, 2, 4, 5, 6, 3, 7], energy 24, no swap neighbor is strictly better and equal-energy swaps exist: the plateau the strict rule refuses to cross.
5. Every trace is strictly decreasing and every final tour is a fixed point, a second climb accepts nothing.

rule	start	trace	final	verdict
first	[0,4,1,5,2,6,3,7]	44, 36, 32, 28, 24, 20, 16	[0..7]	optimum
first	[0,2,4,6,1,3,5,7]	32, 28	[0,6,4,2,1,3,5,7]	local
best	[0,4,1,5,2,6,3,7]	44, 28, 24	[3,2,1,5,4,6,0,7]	local, py and lua

The 16 against the 28 is the section's pair of numbers, and the listings below run both rules in seven languages.

```c
// swap hill climb: scan positions (i, j) ascending, restart after a move
static int climb(const int *start, int *trace, int *tn, int *final) {
  int t[8];
  for (int i = 0; i < 8; i++)
    t[i] = start[i];
  trace[0] = tour_len(t);
  *tn = 1;
  for (;;) {
    int moved = 0;
    for (int i = 0; i < 7 && !moved; i++) {
      for (int j = i + 1; j < 8 && !moved; j++) {
        int cand[8];
        for (int k = 0; k < 8; k++)
          cand[k] = t[k];
        cand[i] = t[j];
        cand[j] = t[i];
        int len = tour_len(cand);
        if (len < trace[*tn - 1]) {
          for (int k = 0; k < 8; k++)
            t[k] = cand[k];
          trace[(*tn)++] = len;
          moved = 1;
        }
      }
    }
    if (!moved) {
      for (int k = 0; k < 8; k++)
        final[k] = t[k];
      return trace[*tn - 1];
    }
  }
}
```

listing 1489: c, the swap climb, ascending scan, restart after every accept (source: dsa/samples-c/src/Ch41/hill.c, lines 31-62)

```go
// HillClimb descends over the swap neighborhood: first improvement,
// strictly better moves only, scan order (i, j) ascending with a
// restart after each accept. Plateaus stop it, because an
// equal-energy swap is never taken.
func HillClimb(d [][]int, tour []int) ([]int, []int) {
  energy := func(s []int) int { return TourLength(d, s) }
  return Descend(energy, ApplySwap, MovePairs(len(tour)), tour)
}

// BestImprovement scans the whole neighborhood each round and takes
// the single best strictly-improving move, the first move winning
// ties. Same landscape, different accepted-move sequence.
func BestImprovement(energy func([]int) int, apply func([]int, Move) []int, moves []Move, start []int)
([]int, []int) {
  s := slices.Clone(start)
  cur := energy(s)
  trace := []int{cur}
  for {
    var bestCand []int
    bestE := cur
    for _, m := range moves {
      cand := apply(s, m)
      if e := energy(cand); e < bestE {
        bestCand, bestE = cand, e
      }
    }
    if bestCand == nil {
      return s, trace
    }
    s, cur = bestCand, bestE
    trace = append(trace, cur)
  }
}
```

listing 1490: go, the swap climb delegating to the shared driver, the best-improvement variant (source: dsa/samples-go/ch41/hill.go, lines 5-36)

```
  static int climb(int[] start, int[] finalOut) {
    int[] t = start.clone();
    trace[0] = tourLen(t);
    traceN = 1;
    for (;;) {
      boolean moved = false;
      for (int i = 0; i < 7 && !moved; i++) {
        for (int j = i + 1; j < 8 && !moved; j++) {
          int[] cand = t.clone();
          cand[i] = t[j];
          cand[j] = t[i];
          int len = tourLen(cand);
          if (len < trace[traceN - 1]) {
            t = cand;
            trace[traceN++] = len;
            moved = true;
          }
        }
      }
      if (!moved) {
        System.arraycopy(t, 0, finalOut, 0, 8);
        return trace[traceN - 1];
      }
    }
  }
```

listing 1491: java, the swap climb, ascending scan, restart after every accept (source: dsa/samples-java/src/Ch41/Hill.java, lines 34-58)

```
    public static (int[] Final, int[] Trace) Climb(int[][] d, int[] tour)
    {
        var t = (int[])tour.Clone();
        int cur = Localsearch.TourLen(d, t);
        var trace = new List<int> { cur };
        bool moved = true;
        while (moved)
        {
            moved = false;
            for (int i = 0; i + 1 < t.Length && !moved; i++)
            {
                for (int j = i + 1; j < t.Length; j++)
                {
                    (t[i], t[j]) = (t[j], t[i]);
                    int len = Localsearch.TourLen(d, t);
                    if (len < cur)
                    {
                        cur = len;
                        trace.Add(len);
                        moved = true;
                        break; // restart the scan from (0, 1)
                    }
                    (t[i], t[j]) = (t[j], t[i]);
                }
            }
        }
        return (t, trace.ToArray());
    }
```

listing 1492: c#, first improvement with the swap applied and undone, the restart comment (source: dsa/samples/src/Ch41/Hill.cs, lines 12-39)

```
export function hillClimb(d, tour) {
  const n = tour.length;
  let t = [...tour];
  const trace = [tourLen(d, t)];
  for (;;) {
    let moved = false;
    for (let i = 0; i + 1 < n && !moved; i++) {
      for (let j = i + 1; j < n; j++) {
        const cand = [...t];
        [cand[i], cand[j]] = [cand[j], cand[i]];
        const len = tourLen(d, cand);
        if (len < trace[trace.length - 1]) {
          t = cand;
          trace.push(len);
          moved = true;
          break;
        }
      }
    }
    if (!moved) return { final: t, trace };
  }
}
```

listing 1493: javascript, the eager loop, the first better neighbor wins (source: dsa/samples-js/src/ch41-hill.mjs, lines 9-30)

```python
def hill_first(t):
    # first improvement: take the first strictly better neighbor in
    # scan order, then restart the scan from (0, 1) after the accept
    t = list(t)
    trace = [tour_len(t)]
    while True:
        for c in swaps(t):
            e = tour_len(c)
            if e < trace[-1]:
                t = c
                trace.append(e)
                break
        else:
            return t, trace


def hill_best(t):
    # best improvement: scan the whole neighborhood, take the minimum
    t = list(t)
    trace = [tour_len(t)]
    while True:
        best, best_e = None, None
        for c in swaps(t):
            e = tour_len(c)
            if best_e is None or e < best_e:
                best, best_e = c, e
        if best_e ≥ trace[-1]:
            return t, trace
        t = best
        trace.append(best_e)
```

listing 1494: python, first and best improvement side by side (source: dsa/samples-py/src/Ch41/hill.py, lines 36-65)

```lua
local function hill_climb(d, tour)
  local n = #tour
  local t = {table.unpack(tour)}
  local trace = {tour_len(d, t)}
  while true do
    local improved = false
    for i = 1, n - 1 do
      for j = i + 1, n do
        local cand = {table.unpack(t)}
        cand[i], cand[j] = cand[j], cand[i]
        local l = tour_len(d, cand)
        if l < trace[#trace] then
          t = cand
          trace[#trace + 1] = l
          improved = true
          break
        end
      end
      if improved then break end
    end
    if not improved then return t, trace end
  end
end
```

listing 1495: lua, the same pair over 1-based positions (source: dsa/samples-lua/ch41_hill.lua, lines 28-50)

The full fixture family: the two pinned swap traces with their finals, the best-improvement trace 44, 28, 24 with the final tour carried by lua, go, javascript, and c# asserting the strictly decreasing trace and the swap-local stop while c ships no best variant, the plateau facts at energy 24, no strictly better swap and sideways swaps present, and the fixed-point re-climb. The 24 plateau is not incidental scenery: it is exactly where the 2-opt run of the next section dies, and the tabu run of section 6 walks across it with sideways moves.

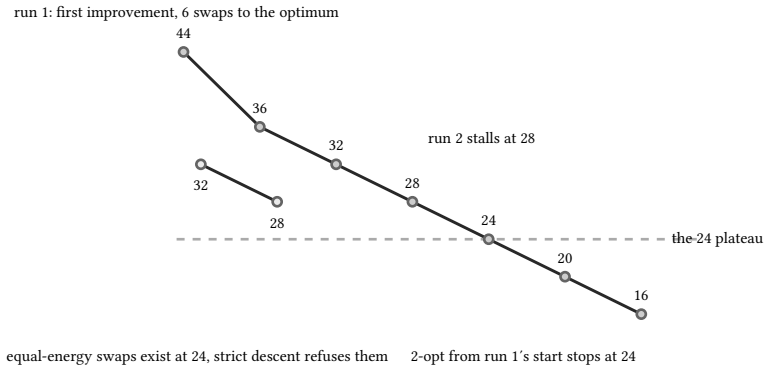


figure 366: the two swap traces as staircases, the first run walking 44 down to 16, the second stopping at 28, the 24 plateau marked where sideways swaps live

41.3. tsp neighborhoods: swap, insert, 2-opt

Three move sets dominate the traveling salesman literature. Swap exchanges the cities at two positions, 28 moves at n = 8. Insert pulls a city out and slides it into another position, 56 moves, n(n-1). 2-opt cuts the tour at two positions and reverses the segment between, which is the move that keeps the object a tour while swapping exactly two edges, and it shares the pair count with swap, 28. The economic argument for 2-opt is incremental evaluation: reversing t[i..j] removes the edges (a, b) and (c, e) and adds (a, c) and (b, e), where a is the cyclic predecessor of position i, b the city at i, c the city at j, and e the cyclic successor of j, four matrix lookups instead of a full rescan. The whole-tour reversal maps the cycle onto itself and prices at delta 0, an honest non-move.

The dry run: the seven-way contract pins the three 2-opt traces, one accepted-move sequence per start, identical in all seven suites, and the delta identity, priced against full recomputation in go, javascript, python, lua, and c#, with c asserting the stall half alone, no candidate in negative delta at its stopped tour.

1. 2-opt from [0, 4, 1, 5, 2, 6, 3, 7] walks 44, 36, 32, 24 and stops at [0, 1, 2, 4, 5, 6, 3, 7], a local optimum 8 over the global 16.
2. The first move prices itself exactly: reversing positions 1..2 removes edges (0, 4) of length 8 and (1, 5) of length 4, adds (0, 1) of length 2 and (4, 5) of length 2, delta −8, and 44 lands on 36.
3. 2-opt from [0, 2, 4, 6, 1, 3, 5, 7] walks 32, 28 and stops at [6, 4, 2, 0, 1, 3, 5, 7].
4. On the convex hex from [0, 3, 1, 4, 2, 5] the walk is 32, 28, 24, 20, the hull order, and 20 is the brute-force optimum over 120.
5. The delta equals full recomputation on every one of the 28 candidates of the pinned start, and the delta-priced driver accepts the identical move sequence.
6. At the stalled 24 tour no candidate has negative delta: the stop is genuine, and the swap neighborhood from the same start reaches 16, while on the second start both neighborhoods stop at 28. Neither dominates.

The worked delta, −8 for the first accepted move, is the number to remember, and the listings below ship the reversal, the delta, and the descent in seven languages.

```c
// reversing t[i..j] removes edges (t[i-1], t[i]) and (t[j], t[j+1]) and
// adds (t[i-1], t[j]) and (t[i], t[j+1]), all indices cyclic. the pair
// (0, nn-1) reverses the whole cycle, the same tour up to rotation: the
// two removed edges coincide and the honest delta is 0
static int twoopt_delta(const int d[8][8], int nn, const int *t, int i, int j) {
  if (i == 0 && j == nn - 1)
    return 0;
  int a = t[(i + nn - 1) % nn], b = t[i], c = t[j], e = t[(j + 1) % nn];
  return d[a][c] + d[b][e] - d[a][b] - d[c][e];
}

static int twoopt(const int d[8][8], int nn, const int *start, int *trace,
                  int *tn, int *final) {
  int t[8];
  for (int k = 0; k < nn; k++)
    t[k] = start[k];
  int cur = tlen(d, nn, t);
  trace[0] = cur;
  *tn = 1;
  for (;;) {
    int moved = 0;
    for (int i = 0; i < nn - 1 && !moved; i++) {
      for (int j = i + 1; j < nn && !moved; j++) {
        int delta = twoopt_delta(d, nn, t, i, j);
        if (delta < 0) {
          for (int l = i, r = j; l < r; l++, r--) {
            int tmp = t[l];
            t[l] = t[r];
            t[r] = tmp;
          }
          cur += delta;
          trace[(*tn)++] = cur;
          moved = 1;
        }
      }
    }
    if (!moved) {
      for (int k = 0; k < nn; k++)
        final[k] = t[k];
      return cur;
    }
  }
}
```

listing 1496: c, the four-edge delta with the whole-cycle guard, the reversal in place, the descent (source: dsa/samples-c/src/Ch41/twoopt.c, lines 39-81)

```go
// TwoOpt descends over 2-opt reversals: first improvement over the
// pairs (i, j) ascending lexicographic, strictly better moves only,
// scan restart after each accept. The reversal replaces the two tour
// edges touching the segment, so the search reads one delta per
// candidate instead of a full rescan.
func TwoOpt(d [][]int, tour []int) ([]int, []int) {
  energy := func(s []int) int { return TourLength(d, s) }
  return Descend(energy, ApplyReverse, MovePairs(len(tour)), tour)
}

// TwoOptDelta is the incremental evaluation of reversing t[i..j]:
// only the edges (prev, t[i]) and (t[j], next) leave and the edges
// (prev, t[j]) and (t[i], next) enter, four matrix lookups. The full
// reversal (i == 0, j == n-1) maps the tour onto itself and reports
// delta 0.
func TwoOptDelta(d [][]int, t []int, m Move) int {
  n := len(t)
  if m.I == 0 && m.J == n-1 {
    return 0
  }
  prev := t[(m.I-1+n)%n]
  a := t[m.I]
  b := t[m.J]
  next := t[(m.J+1)%n]
  return d[prev][b] + d[a][next] - d[prev][a] - d[b][next]
}
```

listing 1497: go, the reversal move over the shared driver, the same four lookups (source: dsa/samples-go/ch41/twoopt.go, lines 3-28)

```
static int twooptDelta(int[][] d, int nn, int[] t, int i, int j) {
  if (i == 0 && j == nn - 1)
    return 0;
  int a = t[(i + nn - 1) % nn], b = t[i], c = t[j], e = t[(j + 1) % nn];
  return d[a][c] + d[b][e] - d[a][b] - d[c][e];
}

static int traceN;
static int[] trace = new int[64];

static int twoopt(int[][] d, int nn, int[] start, int[] finalOut) {
  int[] t = new int[8];
  System.arraycopy(start, 0, t, 0, nn);
  int cur = tlen(d, nn, t);
  trace[0] = cur;
  traceN = 1;
  for (;;) {
    boolean moved = false;
    for (int i = 0; i < nn - 1 && !moved; i++) {
      for (int j = i + 1; j < nn && !moved; j++) {
        int delta = twooptDelta(d, nn, t, i, j);
        if (delta < 0) {
          for (int l = i, r = j; l < r; l++, r--) {
            int tmp = t[l];
            t[l] = t[r];
            t[r] = tmp;
          }
          cur += delta;
          trace[traceN++] = cur;
          moved = true;
        }
      }
    }
    if (!moved) {
      System.arraycopy(t, 0, finalOut, 0, nn);
      return cur;
    }
  }
}
```

listing 1498: java, the four-edge delta with the whole-cycle guard, the reversal in place, the descent (source: dsa/samples-java/src/Ch41/Twoopt.java, lines 38-76)

```
    public static int Delta(int[][] d, int[] t, int i, int j)
    {
        int n = t.Length;
        if (i == 0 && j == n - 1)
            return 0; // full reversal: the same cycle
        int prev = t[(i - 1 + n) % n];
        int next = t[(j + 1) % n];
        return d[prev][t[j]] + d[t[i]][next] - d[prev][t[i]] - d[t[j]][next];
    }

    public static (int[] Final, int[] Trace) FirstImprovement(int[][] d, int[] tour)
    {
        var t = (int[])tour.Clone();
        int cur = Localsearch.TourLen(d, t);
        var trace = new List<int> { cur };
        bool moved = true;
        while (moved)
        {
            moved = false;
            for (int i = 0; i + 1 < t.Length && !moved; i++)
            {
                for (int j = i + 1; j < t.Length; j++)
                {
                    int delta = Delta(d, t, i, j);
                    if (delta < 0)
                    {
                        t = ReverseSeg(t, i, j);
                        cur += delta;
                        trace.Add(cur);
                        moved = true;
                        break; // restart the scan from (0, 1)
                    }
                }
            }
        }
        return (t, trace.ToArray());
    }
```

listing 1499: c#, the delta and the first-improvement loop over the pairs (source: dsa/samples/src/Ch41/TwoOpt.cs, lines 24-60)

```
export function twoOptDelta(d, t, i, j) {
  const n = t.length;
  if (i === 0 && j === n - 1) return 0; // full reversal: the same cycle
  const prev = t[(i - 1 + n) % n];
  const next = t[(j + 1) % n];
  return d[prev][t[j]] + d[t[i]][next] - d[prev][t[i]] - d[t[j]][next];
}

export function twoOpt(d, tour) {
  const n = tour.length;
  let t = [...tour];
  let cur = tourLen(d, t);
  const trace = [cur];
  for (;;) {
    let moved = false;
    for (let i = 0; i + 1 < n; i++) {
      for (let j = i + 1; j < n; j++) {
        const delta = twoOptDelta(d, t, i, j);
        if (delta < 0) {
          t = reverseSeg(t, i, j);
          cur += delta;
          trace.push(cur);
          moved = true;
          break;
        }
      }
      if (moved) break;
    }
    if (!moved) return { final: t, trace };
  }
}
```

listing 1500: javascript, the delta function and the eager loop (source: dsa/samples-js/src/ch41-twoopt.mjs, lines 19-49)

```
def two_opt_delta(d, t, i, j):
    # reversing t[i..j] swaps the two boundary edges, the inner edges
    # keep their lengths; a whole-tour reversal changes nothing
    if i == 0 and j == len(t) - 1:
        return 0
    a = t[i - 1]
    b = t[(j + 1) % len(t)]
    return d[a][t[j]] + d[t[i]][b] - d[a][t[i]] - d[t[j]][b]


START = [0, 4, 1, 5, 2, 6, 3, 7]
e0 = tour_len(DR, START)
check(
    all(
        tour_len(DR, reversed_segment(START, i, j))
        == e0 + two_opt_delta(DR, START, i, j)
        for i in range(7)
        for j in range(i + 1, 8)
    ),
    "the four-cell delta prices every candidate exactly",
)


def two_opt_fast(d, t):
    # the same accepted-move sequence, priced by deltas alone
    t = list(t)
    e = tour_len(d, t)
    trace = [e]
    while True:
        improved = False
        for i in range(len(t) - 1):
            for j in range(i + 1, len(t)):
                delta = two_opt_delta(d, t, i, j)
                if delta < 0:
                    t = reversed_segment(t, i, j)
                    e += delta
                    trace.append(e)
                    improved = True
                    break
            if improved:
                break
        if not improved:
            return t, trace
```

listing 1501: python, the delta and the fast driver that accepts the identical sequence (source: dsa/samples-py/src/Ch41/twoopt.py, lines 104-146)

```lua
local function two_opt(d, tour)
  local n = #tour
  local t = {table.unpack(tour)}
  local trace = {tour_len(d, t)}
  while true do
    local improved = false
    for i = 1, n - 1 do
      for j = i + 1, n do
        local cand = reversed(t, i, j)
        local l = tour_len(d, cand)
        if l < trace[#trace] then
          t = cand
          trace[#trace + 1] = l
          improved = true
          break
        end
      end
      if improved then break end
    end
    if not improved then return t, trace end
  end
end

-- the four affected edges only: a-b and c-e leave, a-c and b-e enter,
-- with a the cyclic predecessor of i and e the cyclic successor of j
local function delta(d, t, i, j)
  local n = #t
  local a, b, c, e = t[i - 1], t[i], t[j], t[j + 1]
  if i == 1 then a = t[n] end
  if j == n then e = t[1] end
  return d[a + 1][c + 1] + d[b + 1][e + 1] - d[a + 1][b + 1] - d[c + 1][e + 1]
end
```

listing 1502: lua, the descent and the four-edge delta with cyclic wrap (source: dsa/samples-lua/ch41_twoopt.lua, lines 44-75)

The fixture family: the ring traces 44, 36, 32, 24 and 32, 28 with their final tours, the hex trace 32, 28, 24, 20 onto the hull order, the hex matrix reproduced from its points before use, the brute optima 16 and 20, the delta identity over all candidates, and the cross-neighborhood verdict, swaps reach 16 where 2-opt holds 24, both hold 28 on the second start. For points in convex position the hull walk is the optimal tour, the shoelace fact, and through Manhattan edges its length is exactly computable, which is why F2 is a fair witness for any tour builder.

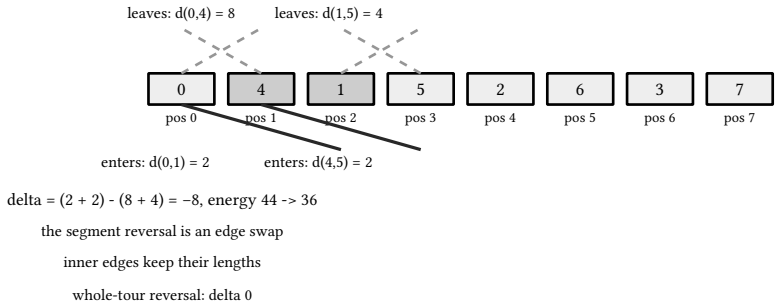


figure 367: the 2-opt move on the pinned start: positions 1..2 reversed, edges (0,4) and (1,5) leaving dashed, edges (0,1) and (4,5) entering heavy, delta −8

41.4. simulated annealing and the metropolis rule

Strict descent dies on the first plateau. The Metropolis rule lets the walk climb: a downhill or level move is always taken, an uphill move with energy increase delta is taken with a probability that decreases in delta and

relaxes as the temperature T falls. The textbook form is exp(-delta / T), a floating transcendental, and this chapter makes a stated engineering substitution: accept with probability T / (T + delta), the rational sibling with the same decreasing shape in delta, realized exactly as the integer test below(T + delta) < T. One draw, integer arithmetic, and zero cross-language libm divergence, so all seven suites assert the same accept and reject counts. The draw happens only on the uphill branch, which keeps the meter honest.

The landscape is F4, a 12-bit QUBO: energy(x) = c + sum a_i x_i + sum b_ij x_i x_j with c = −7, the coefficient vector 3, −4, 2, 5, −6, 1, −2, 4, 3, −5, 2, −3, and b = −6 on exactly 14 index pairs. Every coefficient is an integer, so all 4096 states price exactly, and the brute-force oracle runs in-test. The contract: 200 iterations, start all-zero, T_i = max(1, 64 >> (i / 25)), move = flip bit below(12).

The dry run: the seven-way contract pins the three counter triples and the finals, exact equality, with the best asserted against the brute-force optimum at ratio 1.0.

1. Brute force over the 4096 states finds the minimum −91 at x = 4095, all bits on, and prices the all-zero state at −7.
2. Seed 1: 106 accepted, 94 rejected, 47 of the accepts uphill, best −91, final state 4095.
3. Seed 2: 106, 94, 50, best −91, final state 4095.
4. Seed 42: 123, 77, 58, best −91, final state 4094, one bit short of the optimum on the final state while the best bank holds 4095′s energy.
5. accepted + rejected = 200 in every run: every iteration lands on one side of the rule.

seed	accepted	rejected	uphill	best	final x
1	106	94	47	−91	4095
2	106	94	50	−91	4095
42	123	77	58	−91	4094

The −91 on every seed is the pinned landing, and the listings below run the rule in seven languages.

```
static void sa_run(uint64_t seed, int reheat_on, SaOut *out) {
  lcg_seed(seed);
  int x = 0, cur = qenergy(x), best = cur, bx = x;
  int t0 = 64, streak = 0, s = 0;
  out->acc = out->rej = out->accworse = out->reheats = out->logn = 0;
  for (int it = 0; it < 200; it++) {
    int t = t0 >> (s / 25);
    if (t < 1)
      t = 1;
    int i = (int)below(12);
    int nx = x ^ (1 << i);
    int ne = qenergy(nx);
    int delta = ne - cur;
    if (delta <= 0) {
      x = nx;
      cur = ne;
      out->acc++;
      streak = 0;
    } else {
      if (below((uint32_t)(t + delta)) < (uint32_t)t) {
        x = nx;
        cur = ne;
        out->acc++;
        out->accworse++;
        streak = 0;
```

listing 1503: c, the schedule, the bit flip, downhill and the rational uphill test (source: dsa/samples-c/src/Ch41/anneal.c, lines 57-81)

```
func anneal(seed uint64, iters, t0 int, reheat bool, streakLimit int) SaResult {
  g := NewLcg(seed)
  x := 0
  cur := QuboEnergy(x)
  best, bx := cur, x
  var res SaResult
  streak := 0
  s := 0 // cooling clock: iterations since the last reheat
  for it := 0; it < iters; it++ {
    t := t0 >> (s / 25)
    if t < 1 {
      t = 1
    }
    nx := x ^ (1 << int(g.Below(12)))
    ne := QuboEnergy(nx)
    delta := ne - cur
    if delta <= 0 {
      x, cur = nx, ne
      res.Meter.Accepted++
      streak = 0
    } else if int(g.Below(uint32(t+delta))) < t {
      x, cur = nx, ne
      res.Meter.Accepted++
      res.Meter.UphillAccepted++
```

listing 1504: go, the shared engine through the uphill accept (source: dsa/samples-go/ch41/anneal.go, lines 73-96)

```
  static void saRun(long seed, boolean reheatOn) {
    lcgSeed(seed);
    int x = 0, cur = qenergy(x), bestE = cur, bx = x;
    int t0 = 64, streak = 0, s = 0;
    acc = rej = accworse = reheats = logn = 0;
    for (int it = 0; it < 200; it++) {
      int t = t0 >> (s / 25);
      if (t < 1)
        t = 1;
      int i = below(12);
      int nx = x ^ (1 << i);
      int ne = qenergy(nx);
      int delta = ne - cur;
      if (delta ≤ 0) {
        x = nx;
        cur = ne;
        acc++;
        streak = 0;
      } else {
        if (below(t + delta) < t) {
          x = nx;
          cur = ne;
          acc++;
          accworse++;
          streak = 0;
```

listing 1505: java, the schedule, the bit flip, downhill and the rational uphill test (source: dsa/samples-java/src/Ch41/Anneal.java, lines 56-80)

```
    public static (int Accepted, int Rejected, int UphillAccepted, int Best, int X) Sa(
        int seed, int iters = 200, int t0 = 64)
    {
        var g = new Lcg((ulong)seed);
        int x = 0;
        int cur = QuboEnergy(x);
        int best = cur;
        int accepted = 0;
        int rejected = 0;
        int uphillAccepted = 0;
        for (int it = 0; it < iters; it++)
        {
            int t = CoolT(t0, it);
            int bit = g.Below(12);
            int nx = x ^ (1 << bit);
            int delta = QuboEnergy(nx) - cur;
            if (delta <= 0)
            {
                x = nx;
                cur += delta;
                accepted++;
            }
            else if (g.Below((uint)(t + delta)) < t) // the rational rule
            {
                x = nx;
                cur += delta;
                accepted++;
                uphillAccepted++;
            }
            else
            {
                rejected++;
            }
            if (cur < best)
                best = cur;
        }
        return (accepted, rejected, uphillAccepted, best, x);
    }
```

listing 1506: c#, the same run, the uphill draw on the else branch (source: dsa/samples/src/Ch41/Anneal.cs, lines 44-81)

```
export function saQubo(seed, iters = 200, t0 = 64) {
  const g = makeLcg(seed);
  let x = 0;
  let cur = quboEnergy(x);
  let best = cur;
  let accepted = 0;
  let rejected = 0;
  let uphillAccepted = 0;
  for (let it = 0; it < iters; it++) {
    const t = coolT(t0, it);
    const bit = g.below(12);
    const nx = x ^ (1 << bit);
    const delta = quboEnergy(nx) - cur;
    if (delta ≤ 0) {
      x = nx;
      cur += delta;
      accepted += 1;
    } else if (g.below(t + delta) < t) {
      x = nx;
      cur += delta;
      accepted += 1;
      uphillAccepted += 1;
    } else {
      rejected += 1;
    }
    if (cur < best) best = cur;
  }
  return { accepted, rejected, uphillAccepted, best, x };
}
```

listing 1507: javascript, coolT and the run loop, the integer rule (source: dsa/samples-js/src/ch41-anneal.mjs, lines 32-60)

```
# the integer metropolis rule: an uphill move with increase delta at
# temperature t is accepted with probability t / (t + delta), realized
# exactly as below(t + delta) < t, the draw happens only uphill
def sa(seed, iters=200, t0=64):
    g = Lcg(seed)
    x = 0
    cur = qenergy(x)
    best = cur
    acc = rej = up = 0
    for it in range(iters):
        t = max(1, t0 >> (it // 25))
        nxt = x ^ (1 << g.below(12))
        e = qenergy(nxt)
        if e ≤ cur:
            x, cur = nxt, e
            acc += 1
        elif g.below(t + e - cur) < t:
            x, cur = nxt, e
            acc += 1
            up += 1
        else:
            rej += 1
        if cur < best:
            best = cur
    return acc, rej, up, best, x
```

listing 1508: python, the metropolis comment and the sa driver (source: dsa/samples-py/src/Ch41/anneal.py, lines 70-94)

```
-- 200 iterations, start all-zero, t_i = max(1, 64 >> (i // 25)), move =
-- flip bit below(12), the metropolis draw happens only on the uphill
-- branch. returns accepted, rejected, uphill-accepted, best, final x
local function sa(seed)
  local g = lcg(seed)
  local x = 0
  local cur = qenergy(x)
  local best = cur
  local acc, rej, accw = 0, 0, 0
  for it = 0, 199 do
    local t = math.max(1, 64 >> (it // 25))
    local i = g.below(12)
    local nx = x ~ (1 << i)
    local delta = qenergy(nx) - cur
    if delta ≤ 0 then
      x, cur = nx, qenergy(nx)
      acc = acc + 1
    elseif g.below(t + delta) < t then
      x, cur = nx, qenergy(nx)
      acc = acc + 1
      accw = accw + 1
    else
      rej = rej + 1
    end
    if cur < best then best = cur end
  end
  return acc, rej, accw, best, x
end
```

listing 1509: lua, the same loop, max and floor shifts by hand (source: dsa/samples-lua/ch41_anneal.lua, lines 50-77)

The fixture family: the brute oracle, −91 at 4095 and −7 at 0, the three counter rows with their finals, the 200 accounting identity, and the tolerance lane stated as ratio 1.0 against the oracle. The compact prose treatment in book 9, ch 36 (practical data structures and algorithms, numerical methods) keeps the exp form and points here for the integer rule and the pinned runs.

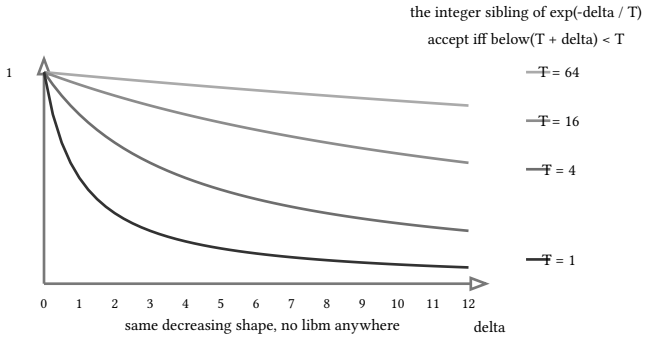


figure 368: the rational metropolis rule T / (T + delta) at four temperatures over delta up to 12, high T accepting most climbs, T = 1 nearly greedy

41.5. cooling schedules, restarts, reheating

The temperature program is where annealing earns its tuning. The chapter's schedule is geometric and integral: T_i = max(1, T0 >> (i / 25)) halves every 25 iterations, so 200 iterations from T0 = 64 read 64, 32, 16, 8, 4, 2, then 1 forever, a ladder with no float in it. Restarting and reheating answer the failure mode where the walk freezes in a bad basin: after 20 consecutive non-accepted moves the state resets to the best-so-far, T0 doubles, and the cooling clock restarts, so T indexes iterations since the last reheat. The reheat iteration itself does not advance the clock, one off-by-one that changes counters and is pinned on purpose.

PITFALL: the clock indexes reheats, not iterations

After a reheat the schedule does not resume at the global iteration count. `s` restarts at 0, so iteration 182 of the seed 1 run reads `T = 128 >> (0 / 25) = 128`, a full-hot temperature late in the run. Forgetting the reset, or advancing `s` on the reheat iteration itself, shifts every later draw and both counter columns drift. The pinned reheat logs exist to catch exactly this.

The dry run: the seven-way contract pins the three reheat runs with their logs, counters exact in all seven suites, and the ladder values, asserted in python and lua.

1. The ladder reads 64 through iteration 24, 32 at 25, and 1 from iteration 150 on: 64, 32, 16, 8, 4, 2, 1, 1 sampled every 25.
2. Seed 1: 124 accepted, 76 rejected, 59 uphill, 1 reheat at iteration 181 doubling T0 to 128, best −91, final state 2614.
3. Seed 2: 111, 89, 54, 1 reheat at 193 to T0 128, best −91, final state 2030.
4. Seed 42: 123, 77, 58, 0 reheats, best −91, final state 4094: the accept rate never dies 20 draws in a row, itself a teaching row.
5. Seed 42 never reheats, so its plain and reheat drivers agree counter for counter.

seed	acc	rej	uphill	reheats	best	reheat log
1	124	76	59	1	−91	(181, 128)
2	111	89	54	1	−91	(193, 128)
42	123	77	58	0	−91	

The 181 and the 193 are the section's pinned pair, and the listings below ship the reheat contract in seven languages.

```
      } else {
        out->rej++;
        streak++;
        if (reheat_on && streak >= 20) {
          x = bx;
          cur = best;
          t0 *= 2;
          out->reheats++;
          streak = 0;
          s = 0;
          out->log_it[out->logn] = it;
          out->log_t0[out->logn] = t0;
          out->logn++;
          continue;
        }
      }
    }
    if (cur < best) {
      best = cur;
      bx = x;
    }
    s++;
  }
  out->best = best;
  out->x = x;
}
```

listing 1510: c, the rejection branch, the streak, the reheat with its log, the clock (source: dsa/samples-c/src/Ch41/anneal.c, lines 82-107)

```
    } else {
      res.Meter.Rejected++
      streak++
      if reheat && streak >= streakLimit {
        x, cur = bx, best
        t0 *= 2
        res.Meter.Reheats++
        streak = 0
        s = 0
        res.ReheatLog = append(res.ReheatLog, ReheatEvent{Iter: it, NewT0: t0})
        continue
      }
    }
    if cur < best {
      best, bx = cur, x
    }
    s++
  }
  res.Best, res.Final = best, x
  return res
}
```

listing 1511: go, the reject branch and the reheat event recorded (source: dsa/samples-go/ch41/anneal.go, lines 98-118)

```
        } else {
          rej++;
          streak++;
          if (reheatOn && streak ≥ 20) {
            x = bx;
            cur = bestE;
            t0 *= 2;
            reheats++;
            streak = 0;
            s = 0;
            logIt[logn] = it;
            logT0[logn] = t0;
            logn++;
            continue;
          }
        }
      }
      if (cur < bestE) {
        bestE = cur;
        bx = x;
      }
      s++;
    }
    best = bestE;
    xEnd = x;
  }
```

listing 1512: java, the rejection branch, the streak, the reheat with its log, the clock and best bank (source: dsa/samples-java/src/Ch41/Anneal.java, lines 81-106)

```
        int s = 0; // cooling clock: iterations since the last reheat
        var reheatLog = new List<(int, int)>();
        for (int it = 0; it < iters; it++)
        {
            int t = CoolT(baseT0, s);
            int bit = g.Below(12);
            int nx = x ^ (1 << bit);
            int delta = QuboEnergy(nx) - cur;
            if (delta ≤ 0)
            {
                x = nx;
                cur += delta;
                accepted++;
                streak = 0;
            }
            else if (g.Below((uint)(t + delta)) < t)
            {
                x = nx;
                cur += delta;
                accepted++;
                uphillAccepted++;
                streak = 0;
            }
            else
            {
                rejected++;
                streak++;
                if (streak ≥ streakLimit)
                {
                    x = bestX; // reset to the best-so-far, double T0
                    cur = best;
                    baseT0 *= 2;
                    reheats++;
                    streak = 0;
                    s = 0;
                    reheatLog.Add((it, baseT0));
                    continue; // the reheat iteration is not a cooling step
                }
            }
            if (cur < best)
            {
                best = cur;
                bestX = x;
            }
            s++;
        }
        return (accepted, rejected, uphillAccepted, reheats, best, x, reheatLog.ToArray());
```

listing 1513: c#, the reheat driver, reset to best, double T0, restart the clock (source: dsa/samples/src/Ch41/Anneal.cs, lines 99-145)

```
export function saReheat(seed, iters = 200, t0 = 64, streakLimit = 20) {
  const g = makeLcg(seed);
  let x = 0;
  let cur = quboEnergy(x);
  let best = cur;
  let bestX = x;
  let baseT0 = t0;
  let accepted = 0;
  let rejected = 0;
  let uphillAccepted = 0;
  let reheats = 0;
  let streak = 0;
  let s = 0;
  const reheatLog = [];
  for (let it = 0; it < iters; it++) {
    const t = coolT(baseT0, s);
    const bit = g.below(12);
    const nx = x ^ (1 << bit);
    const delta = quboEnergy(nx) - cur;
    if (delta <= 0) {
      x = nx;
      cur += delta;
      accepted += 1;
      streak = 0;
    } else if (g.below(t + delta) < t) {
      x = nx;
      cur += delta;
      accepted += 1;
      uphillAccepted += 1;
      streak = 0;
    } else {
      rejected += 1;
      streak += 1;
      if (streak >= streakLimit) {
        x = bestX;
        cur = best;
        baseT0 *= 2;
        reheats += 1;
        streak = 0;
        s = 0;
        reheatLog.push([it, baseT0]);
        continue;
      }
    }
    if (cur < best) {
      best = cur;
      bestX = x;
    }
    s += 1;
  }
```

listing 1514: javascript, saReheat whole, the log array and the clock reset (source: dsa/samples-js/src/ch41-anneal.mjs, lines 62-111)

```
# reheat from best: after 20 consecutive non-accepted moves the state
# resets to the best-so-far, t0 doubles, and the cooling clock restarts
def sa_reheat(seed, iters=200, t0=64, streak_limit=20):
    g = Lcg(seed)
    x = 0
    cur = qenergy(x)
    best, bx = cur, x
    acc = rej = up = reheats = 0
    streak = 0
    clock = 0
    log = []
    for it in range(iters):
        t = max(1, t0 >> (clock // 25))
        nxt = x ^ (1 << g.below(12))
        e = qenergy(nxt)
        delta = e - cur
        if delta <= 0:
            x, cur = nxt, e
            acc += 1
            streak = 0
        elif g.below(t + delta) < t:
            x, cur = nxt, e
            acc += 1
            up += 1
            streak = 0
        else:
            rej += 1
            streak += 1
            if streak >= streak_limit:
                x, cur = bx, best
                t0 *= 2
                reheats += 1
                streak = 0
                clock = 0
                log.append((it, t0))
                continue
        if cur < best:
            best, bx = cur, x
        clock += 1
    return acc, rej, up, reheats, best, x, log
```

listing 1515: python, reheat from best with the pinned tuple returned (source: dsa/samples-py/src/Ch41/anneal.py, lines 113-152)

```lua
-- same loop with reheat: after 20 consecutive non-accepted moves the
-- state resets to the best-so-far, t0 doubles, the cooling clock
-- restarts (s counts iterations since the last reheat, and the reheat
-- iteration itself does not advance it)
local function sa_reheat(seed)
  local g = lcg(seed)
  local x, bx = 0, 0
  local cur = qenergy(x)
  local best = cur
  local acc, rej, accw, reheats = 0, 0, 0, 0
  local streak, s, t0 = 0, 0, 64
  local log = {}
  for it = 0, 199 do
    local t = math.max(1, t0 >> (s // 25))
    local i = g.below(12)
    local nx = x ~ (1 << i)
    local delta = qenergy(nx) - cur
    local reheated = false
    if delta ≤ 0 then
      x, cur = nx, qenergy(nx)
      acc = acc + 1
      streak = 0
    elseif g.below(t + delta) < t then
      x, cur = nx, qenergy(nx)
      acc = acc + 1
      accw = accw + 1
      streak = 0
    else
      rej = rej + 1
      streak = streak + 1
      if streak ≥ 20 then
        x, cur = bx, best
        t0 = t0 * 2
        reheats = reheats + 1
        streak, s = 0, 0
        log[#log + 1] = {it, t0}
        reheated = true
      end
    end
    if not reheated then
      if cur < best then best, bx = cur, x end
      s = s + 1
    end
  end
  return acc, rej, accw, reheats, best, x, log
end
```

listing 1516: lua, the same loop, the reheated flag keeping the clock honest (source: dsa/samples-lua/ch41_anneal.lua, lines 79-124)

The fixture family: the ladder values, the three reheat rows with counter triples, best, final state, and log, the 200 identity again, and the seed 42 agreement between drivers. A reheat is a restart with a memory: the state comes back from the bank, the temperature comes back from the schedule, and the walk tries a second descent from the best it owns.

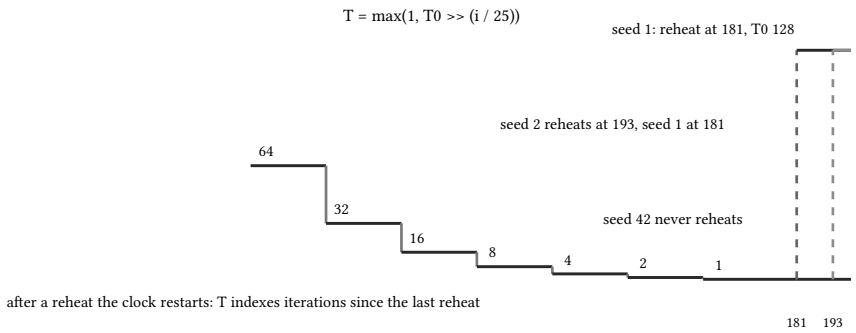


figure 369: the cooling ladder halving every 25 iterations, the seed 1 reheat at 181 and seed 2 at 193 jumping back to T0 128, seed 42 never reheating

41.6. tabu search and tenure

Hill climbing's memory is one state. Tabu search adds a recency memory over moves: a move taken at iteration it is banned for tenure iterations, realized as an expiry stamp it + 3 that stays ahead of the clock, and the ban is what legalizes sideways and uphill moves, the thing strict descent refused. Each iteration picks the minimum-length candidate scanning (i, j) ascending with the first move winning ties, applies it, and stamps it tabu. The run can therefore walk across plateaus and climb, because the moves it just used are the ones it cannot immediately reuse.

The dry run: the seven-way contract pins the 12-iteration log move by move and energy by energy, identical in all seven suites.

1. Iterations 0, 1, 2 improve 44 to 36 to 28 to 24 with moves (1, 2), (2, 4), (3, 4).
2. Iterations 3 through 5 hold 24 with sideways moves (0, 5), (0, 6), (0, 1) while earlier bans expire: this is the plateau crossing.
3. Iteration 6 lifts to 20 with (1, 2), legal again after its ban from iteration 0 expired, and iteration 7 reaches 16 with (2, 3), the optimum.
4. Iterations 8 through 11 hold 16 with (0, 6), (0, 7), (1, 7), (0, 6), and the run ends on the ring order.
5. The best bank reads 16 where plain 2-opt from the same start died at 24: the tenure memory carried the search off the plateau.

iteration	move	energy	tabu until
0	[1,2]	36	3
1	[2,4]	28	4
2	[3,4]	24	5
3	[0,5]	24	6
4	[0,6]	24	7
5	[0,1]	24	8
6	[1,2]	20	9
7	[2,3]	16	10
8	[0,6]	16	11
9	[0,7]	16	12
10	[1,7]	16	13
11	[0,6]	16	14

The 16 at iteration 7 is the pinned landing, and the listings below run the memory in seven languages.

```
        if ((!have || len < blen) && !tabu) {
          bi = i;
          bj = j;
          blen = len;
          have = 1;
          for (int k = 0; k < 8; k++)
            final[k] = cand[k];
        }
      }
    }
    if (!have) {
      log_e[it] = cur;
      log_mi[it] = -1;
      log_mj[it] = -1;
      return it + 1;
    }
    for (int k = 0; k < 8; k++)
      t[k] = final[k];
    cur = blen;
    int w = 0;
    for (int b = 0; b < nbans; b++)
      if (bans[b].exp > it)
        bans[w++] = bans[b];
    nbans = w;
    bans[nbans].i = bi;
    bans[nbans].j = bj;
    bans[nbans].exp = it + 3;
    nbans++;
    if (cur < best) {
      best = cur;
      for (int k = 0; k < 8; k++)
        bt[k] = final[k];
    }
    log_mi[it] = bi;
    log_mj[it] = bj;
    log_e[it] = cur;
  }
  return iters;
}
```

listing 1517: c, the pick, the stall case, the ban store with expiry, the log (source: dsa/samples-c/src/Ch41/tabu.c, lines 79-117)

```
func tabuRun(d [][]int, tour []int, iterations, tenure int, candidateList bool) TabuResult {
  n := len(tour)
  t := slices.Clone(tour)
  cur := TourLength(d, t)
  best, bestTour := cur, slices.Clone(t)
  expiry := make(map[Move]int)
  lifts := 0
  var res TabuResult
  for it := 0; it < iterations; it++ {
    p := -1
    if candidateList {
      p = slices.Index(t, 0)
    }
    var bestCand []int
    var bestMove Move
    bestLen := -1
    for i := 0; i < n-1; i++ {
      for j := i + 1; j < n; j++ {
        if candidateList && i != p && j != p {
          continue
        }
        mv := Move{I: i, J: j}
        cand := ApplyReverse(t, mv)
        l := TourLength(d, cand)
        tabu := expiry[mv] > it
        if tabu && l < best {
          tabu = false // aspiration beats the global best
          lifts++
        }
        if (bestLen < 0 || l < bestLen) && !tabu {
          bestCand, bestMove, bestLen = cand, mv, l
        }
      }
    }
    if bestCand == nil {
      break
    }
    t, cur = bestCand, bestLen
    expiry[bestMove] = it + tenure
    if cur < best {
      best, bestTour = cur, slices.Clone(t)
    }
    res.Log = append(res.Log, TabuEvent{Iter: it, Move: bestMove, Energy: cur})
  }
  res.Best, res.BestTour, res.Final, res.Lifts = best, bestTour, t, lifts
  return res
}
```

listing 1518: go, the whole engine, expiry map, first-wins ties (source: dsa/samples-go/ch41/tabu.go, lines 41-87)

```
      if (!have) {
        logE[it] = cur;
        logMi[it] = -1;
        logMj[it] = -1;
        return it + 1;
      }
      System.arraycopy(bestcand, 0, t, 0, 8);
      System.arraycopy(bestcand, 0, finalTour, 0, 8);
      cur = blen;
      int w = 0;
      for (int b = 0; b < nbans; b++)
        if (bans[b].exp() > it)
          bans[w++] = bans[b];
      nbans = w;
      bans[nbans++] = new Ban(bi, bj, it + 3);
      if (cur < best) {
        best = cur;
        System.arraycopy(t, 0, bestTour, 0, 8);
      }
      logMi[it] = bi;
      logMj[it] = bj;
      logE[it] = cur;
    }
    return iters;
  }
```

listing 1519: java, the stall case, the move commit, the ban store pruned by expiry and extended, the log (source: dsa/samples-java/src/Ch41/Tabu.java, lines 87-111)

```
        for (int it = 0; it < iterations; it++)
        {
            int p = candidateList ? Array.IndexOf(t, 0) : -1;
            int bestLen = int.MaxValue;
            int bi = -1;
            int bj = -1;
            int[]? bestCand = null;
            for (int i = 0; i + 1 < n; i++)
            {
                for (int j = i + 1; j < n; j++)
                {
                    if (candidateList && i ≠ p && j ≠ p)
                        continue;
                    int[] cand = TwoOpt.ReverseSeg(t, i, j);
                    int len = Localsearch.TourLen(d, cand);
                    bool isTabu = tabuUntil.TryGetValue((i, j), out int until)
                        && until > it;
                    if (isTabu && len < best)
                    {
                        isTabu = false; // aspiration beats the global best
                        lifts++;
                    }
                    if (len < bestLen && !isTabu)
                    {
                        bestCand = cand;
                        bi = i;
                        bj = j;
                        bestLen = len;
                    }
                }
```

listing 1520: c#, the scan with the expiry dictionary, aspiration inside (source: dsa/samples/src/Ch41/Tabu.cs, lines 37-66)

```
function tabuCore(d, tour, iterations, tenure, candidateList) {
  const n = tour.length;
  let t = [...tour];
  let cur = tourLen(d, t);
  let best = cur;
  let bestTour = [...t];
  let tabu = [];
  const log = [];
  let lifts = 0;
  for (let it = 0; it < iterations; it++) {
    const p = candidateList ? t.indexOf(0) : -1;
    let bestCand = null;
    let bestMove = null;
    let bestLen = null;
    for (let i = 0; i + 1 < n; i++) {
      for (let j = i + 1; j < n; j++) {
        if (candidateList && i !== p && j !== p) continue;
        const cand = reverseSeg(t, i, j);
        const len = tourLen(d, cand);
        let isTabu = tabu.some(([mi, mj, exp]) => mi === i && mj === j
          && exp > it);
        if (isTabu && len < best) {
          isTabu = false; // aspiration beats the global best
          lifts++;
        }
        if ((bestLen === null || len < bestLen) && !isTabu) {
          bestCand = cand;
          bestMove = [i, j];
          bestLen = len;
        }
      }
    }
    if (bestCand === null) {
      log.push([it, null, cur]);
      break;
    }
    t = bestCand;
    cur = bestLen;
    tabu = tabu.filter(([, , exp]) => exp > it);
    tabu.push([bestMove[0], bestMove[1], it + tenure]);
    if (cur < best) {
      best = cur;
      bestTour = [...t];
    }
    log.push([it, bestMove, cur]);
  }
  return { log, best, bestTour, final: t, lifts };
}
```

listing 1521: javascript, the core with the expiry list pruned on commit (source: dsa/samples-js/src/ch41-tabu.mjs, lines 13-60)

```python
def tabu(t, iterations, tenure=3, candidate_list=False):
    # per iteration pick the minimum-length candidate scanning (i, j)
    # ascending, first wins ties. a move stays tabu while its expiry
    # iteration is still ahead; aspiration lifts the ban for candidates
    # that beat the global best
    t = list(t)
    cur = tour_len(t)
    best, bt = cur, list(t)
    banned = []  # (frozenset move, expiry iteration)
    log = []
    lifts = 0
    for it in range(iterations):
        p = t.index(0) if candidate_list else None
        pick, pick_move, pick_len = None, None, None
        for i in range(7):
            for j in range(i + 1, 8):
                if candidate_list and i != p and j != p:
                    continue
                cand = reversed_segment(t, i, j)
                e = tour_len(cand)
                move = frozenset((i, j))
                is_tabu = any(m == move and exp > it for m, exp in banned)
                if is_tabu and e < best:
                    is_tabu = False
                    lifts += 1
                if not is_tabu and (pick_len is None or e < pick_len):
                    pick, pick_move, pick_len = cand, move, e
        if pick is None:
            break
        t, cur = pick, pick_len
        banned = [(m, exp) for m, exp in banned if exp > it]
        banned.append((pick_move, it + tenure))
        if cur < best:
            best, bt = cur, list(t)
        log.append((it, sorted(pick_move), cur))
    return log, best, bt, lifts, t
```

listing 1522: python, the engine with frozenset moves and the log (source: dsa/samples-py/src/Ch41/tabu.py, lines 35-70)

```
local function tabu(tour, iterations, tenure, cand_only)
  local n = #tour
  local t = {table.unpack(tour)}
  local cur = tour_len(RING, t)
  local best, bt = cur, {table.unpack(t)}
  local expiry = {}
  local log = {}
  local lifts = 0
  for it = 0, iterations - 1 do
    local p = 0
    if cand_only then
      for i = 1, n do
        if t[i] == 0 then p = i end
      end
    end
    local bestcand, besti, bestj, bestlen
    for i = 1, n - 1 do
      for j = i + 1, n do
        if not cand_only or i == p or j == p then
          local cand = reversed(t, i, j)
          local l = tour_len(RING, cand)
          local is_tabu = (expiry[i .. ":" .. j] or -1) > it
          if is_tabu and l < best then
            is_tabu = false
            lifts = lifts + 1
          end
          if (bestlen == nil or l < bestlen) and not is_tabu then
            bestcand, besti, bestj, bestlen = cand, i, j, l
          end
        end
      end
    end
    if bestcand == nil then break end
    t, cur = bestcand, bestlen
    expiry[besti .. ":" .. bestj] = it + tenure
    if cur < best then
      best = cur
      bt = {table.unpack(t)}
    end
    log[#log + 1] = {it, {besti - 1, bestj - 1}, cur}
  end
  return log, best, bt, lifts, t
end
```

listing 1523: lua, the same engine over string-keyed expiry (source: dsa/samples-lua/ch41_tabu.lua, lines 41-83)

The fixture family: the full 12-row log above, tenure 3 with the expiry column, best 16 with the final tour the ring order, and the contrast row, plain 2-opt stalls at 24 on the identical start. The memory is small, a dictionary of move pairs, and it is the whole difference between dead at 24 and done at 16.

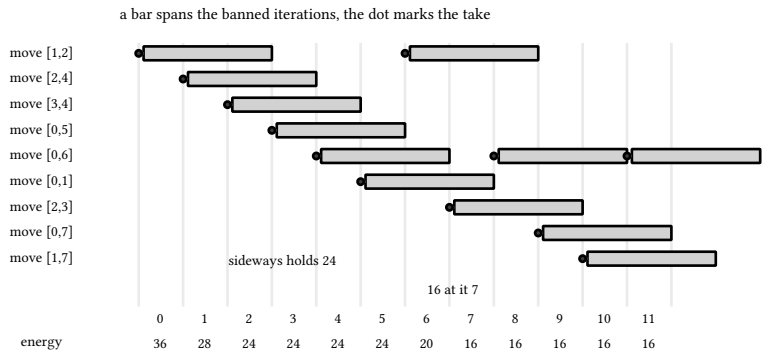


figure 370: the tenure timeline: each accepted move bans itself for 3 iterations, lanes per move, the plateau crossing and the 16 arrival marked

41.7. aspiration and candidate lists

Two refinements polish the memory. Aspiration lifts a ban when the tabu candidate beats the global best, on the theory that a banned move good enough to improve the incumbent should never be blocked by bookkeeping. Candidate lists shrink the scan: instead of all 28 pairs, restrict each iteration to the 2-opt moves (i, j) with i == p or j == p, where p is the current position of city 0, n - 1 = 7 candidates per iteration, a quarter of the work.

The pinned logs keep both honest. The base run's global best reaches the optimum 16 at iteration 7, and no later tabu candidate can beat it, so the aspiration branch never fires: the machinery stays in the code and every suite asserts lifts == 0 as a fact about this instance, a dormant branch is not a dead branch. The candidate list pays for its speed the other way.

The dry run: the seven-way contract pins the candidate-list log, its best tour, its end state, and the zero lift counts.

1. lifts == 0 on both the full-scan and the candidate-list runs, asserted with the lift branch present in the code.
2. The candidate-list log reads energies 44, 36, 32, 32, 32, 28, 28, 24, 24, 24, 24, 28 over moves [0,1], [1,3], [0,3], [0,4], [1,4], [0,1], [0,5], [1,5], [1,6], [0,6], [0,5], [1,5].
3. The best tour banks 24 as [6, 0, 2, 1, 5, 4, 3, 7].
4. The end state after the 12th move is [3, 0, 2, 1, 5, 4, 6, 7] at energy 28: the run leaves its best behind, best tour and end state are different objects.
5. The restricted scan never reaches the 16 the full scan found: 7 candidates per iteration buy speed and lose the optimum.

iteration	move	energy
0	[0,1]	44
1	[1,3]	36
2	[0,3]	32
3	[0,4]	32
4	[1,4]	32
5	[0,1]	28
6	[0,5]	28
7	[1,5]	24
8	[1,6]	24
9	[0,6]	24
10	[0,5]	24
11	[1,5]	28

The 24 against the full scan's 16 is the closing contrast, and the listings below ship both refinements in seven languages.

```c
  for (int it = 0; it < iters; it++) {
    int p = 0;
    if (cand_list)
      for (int k = 0; k < 8; k++)
        if (t[k] == 0)
          p = k;
    int bi = -1, bj = -1, blen = 0, have = 0;
    int cand[8];
    for (int i = 0; i < 7; i++) {
      for (int j = i + 1; j < 8; j++) {
        if (cand_list && i != p && j != p)
          continue;
        for (int k = 0; k < 8; k++)
          cand[k] = t[k];
        for (int l = i, r = j; l < r; l++, r--) {
          int tmp = cand[l];
          cand[l] = cand[r];
          cand[r] = tmp;
        }
        int len = tour_len(cand);
        int tabu = 0;
        for (int b = 0; b < nbans; b++)
          if (bans[b].i == i && bans[b].j == j && bans[b].exp > it)
            tabu = 1;
        if (tabu && len < best) {
          tabu = 0;
          (*lifts)++;
        }
```

listing 1524: c, the city 0 position guard, the candidate filter, the ban test with aspiration and the lift counter (source: dsa/samples-c/src/Ch41/tabu.c, lines 51-78)

```go
// TabuTwoOpt runs tabu search with 2-opt moves: per iteration it
// picks the minimum-length candidate scanning (i, j) ascending,
// first move winning ties. A move is tabu for tenure iterations
// after it is taken, and aspiration lifts the ban when the candidate
// beats the global best, so sideways and uphill moves stay legal
// where plain 2-opt would stall.
func TabuTwoOpt(d [][]int, tour []int, iterations, tenure int) TabuResult {
  return tabuRun(d, tour, iterations, tenure, false)
}

// TabuTwoOptCandidateList restricts the per-iteration scan to the
// 2-opt moves (i, j) with i == p or j == p, where p is the current
// position of city 0: n-1 candidates instead of n(n-1)/2, at the
// price of a weaker neighborhood.
func TabuTwoOptCandidateList(d [][]int, tour []int, iterations, tenure int) TabuResult {
  return tabuRun(d, tour, iterations, tenure, true)
}
```

listing 1525: go, the full scan and the candidate-list wrappers (source: dsa/samples-go/ch41/tabu.go, lines 23-39)

```
    for (int it = 0; it < iters; it++) {
      int p = 0;
      if (candList)
        for (int k = 0; k < 8; k++)
          if (t[k] == 0)
            p = k;
      int bi = -1, bj = -1, blen = 0;
      boolean have = false;
      int[] cand = new int[8];
      int[] bestcand = new int[8];
      for (int i = 0; i < 7; i++) {
        for (int j = i + 1; j < 8; j++) {
          if (candList && i != p && j != p)
            continue;
          System.arraycopy(t, 0, cand, 0, 8);
          for (int l = i, r = j; l < r; l++, r--) {
            int tmp = cand[l];
            cand[l] = cand[r];
            cand[r] = tmp;
          }
          int len = tourLen(cand);
          boolean tabu = false;
          for (int b = 0; b < nbans; b++)
            if (bans[b].i() == i && bans[b].j() == j && bans[b].exp() > it)
              tabu = true;
          if (tabu && len < best) {
            tabu = false;
            lifts++;
          }
```

listing 1526: java, the city 0 position guard, the candidate filter, the ban test with aspiration and the lift counter (source: dsa/samples-java/src/Ch41/Tabu.java, lines 49-77)

```
{
    public static ((int Iteration, int I, int J, int Energy)[] Log, int Best,
        int[] BestTour, int[] Final, int Lifts)
        TwoOptSearch(int[][] d, int[] tour, int iterations, int tenure = 3)
        => Core(d, tour, iterations, tenure, false);

    public static ((int Iteration, int I, int J, int Energy)[] Log, int Best,
        int[] BestTour, int[] Final, int Lifts)
        Candidate(int[][] d, int[] tour, int iterations, int tenure = 3)
        => Core(d, tour, iterations, tenure, true);

    private static ((int Iteration, int I, int J, int Energy)[] Log, int Best,
        int[] BestTour, int[] Final, int Lifts)
        Core(int[][] d, int[] tour, int iterations, int tenure, bool candidateList)
    {
        int n = tour.Length;
        var t = (int[])tour.Clone();
        int cur = Localsearch.TourLen(d, t);
        int best = cur;
        var bestTour = (int[])t.Clone();
        var tabuUntil = new Dictionary<(int, int), int>(); // move -> expiry
```

listing 1527: c#, the two entry points and the expiry dictionary (source: dsa/samples/src/Ch41/Tabu.cs, lines 14-34)

```
  for (let it = 0; it < iterations; it++) {
    const p = candidateList ? t.indexOf(0) : -1;
    let bestCand = null;
    let bestMove = null;
    let bestLen = null;
    for (let i = 0; i + 1 < n; i++) {
      for (let j = i + 1; j < n; j++) {
        if (candidateList && i !== p && j !== p) continue;
        const cand = reverseSeg(t, i, j);
        const len = tourLen(d, cand);
        let isTabu = tabu.some(([mi, mj, exp]) => mi === i && mj === j
          && exp > it);
        if (isTabu && len < best) {
          isTabu = false; // aspiration beats the global best
          lifts++;
        }
        if ((bestLen === null || len < bestLen) && !isTabu) {
          bestCand = cand;
          bestMove = [i, j];
          bestLen = len;
        }
```

listing 1528: javascript, the position guard, the expiry test, the aspiration line (source: dsa/samples-js/src/ch41-tabu.mjs, lines 22-42)

```
    for it in range(iterations):
        p = t.index(0) if candidate_list else None
        pick, pick_move, pick_len = None, None, None
        for i in range(7):
            for j in range(i + 1, 8):
                if candidate_list and i != p and j != p:
                    continue
                cand = reversed_segment(t, i, j)
                e = tour_len(cand)
                move = frozenset((i, j))
                is_tabu = any(m == move and exp > it for m, exp in banned)
                if is_tabu and e < best:
                    is_tabu = False
                    lifts += 1
                if not is_tabu and (pick_len is None or e < pick_len):
                    pick, pick_move, pick_len = cand, move, e
```

listing 1529: python, the candidate guard and the aspiration inside the scan (source: dsa/samples-py/src/Ch41/tabu.py, lines 46-61)

```
  for it = 0, iterations - 1 do
    local p = 0
    if cand_only then
      for i = 1, n do
        if t[i] == 0 then p = i end
      end
    end
    local bestcand, besti, bestj, bestlen
    for i = 1, n - 1 do
      for j = i + 1, n do
        if not cand_only or i == p or j == p then
          local cand = reversed(t, i, j)
          local l = tour_len(RING, cand)
          local is_tabu = (expiry[i .. ":" .. j] or -1) > it
          if is_tabu and l < best then
            is_tabu = false
            lifts = lifts + 1
          end
          if (bestlen == nil or l < bestlen) and not is_tabu then
            bestcand, besti, bestj, bestlen = cand, i, j, l
          end
        end
      end
    end
```

listing 1530: lua, the cand_only filter with the lift counter (source: dsa/samples-lua/ch41_tabu.lua, lines 49-71)

The fixture family: the 12-row candidate log, best 24 with its tour, end state [3, 0, 2, 1, 5, 4, 6, 7] at 28, the zero lift counts on both runs, and the headline trade, 7 candidates against 28 bought a local optimum 8 worse. Best tour and end state diverge here, which is why the result type carries both.

41.8. genetic algorithms: encoding and selection

A genetic algorithm searches a population instead of a point. The encoding decides everything downstream: a bitstring for the knapsack, where bit i means item i is packed, or a permutation for the tour, where crossover must respect the one-city-each shape. Selection is the pressure knob. Roulette is fitness-proportional: sort the population, sum the fitness, draw one below(total), walk the cumulative sums until the draw is passed. Tournament is simpler and stiffer: two below(n) draws, the larger index wins, no sorting and no floats, and the pinned mini-run shows the mechanism, seed 1 draws (3, 4) and keeps 4, seed 2 draws (6, 7) and keeps 7, seed 42 draws (4, 1) and keeps 4, each draw being below(8) = u >> 29.

The fixture is F5: 10 items with weights 7, 4, 9, 3, 5, 8, 2, 6, 10, 3 and values 12, 6, 18, 5, 10, 17, 3, 11, 20, 7 against capacity 18. The in-test oracle is the chapter 17 dynamic program, optimum 37, and the brute force agrees, best bitstring 288, items 5 and 8, weights 8 + 10 and values 17 + 20.

The dry run: the seven-way contract pins the oracle, the tournament draws, and the three run rows with exact counters.

1. The dynamic program lands 37 at capacity 18 and the brute force over all 1024 bitstrings lands 37 at x = 288.
2. Tournament pins: seed 1 (3, 4) to 4, seed 2 (6, 7) to 7, seed 42 (4, 1) to 4.
3. Seed 1: best fitness 31, 140 crossovers, 57 mutation flips, best individual 266.
4. Seed 2: best fitness 0, 140 crossovers, 49 flips.
5. Seed 42: best fitness 34, 140 crossovers, 52 flips, best individual 22.
6. 140 = 7 children x 20 generations, exact on every seed, and seeds 1 and 42 stay within ratio 0.75 of the optimum while seed 2 asserts its 0.

seed	best fitness	crossovers	flips	best x	verdict
1	31	140	57	266	ratio 31/37, within 0.75
2	0	140	49		converged onto infeasible
42	34	140	52	22	ratio 34/37, within 0.75

The cautionary row is seed 2, and the listings below run the whole contract in seven languages.

```
static void ga_run(uint64_t seed, GaOut *out) {
  lcg_seed(seed);
  int pop[8];
  for (int i = 0; i < 8; i++)
    pop[i] = (int)below(1024);
  out->crossovers = 0;
  out->flips = 0;
  for (int gen = 0; gen < 20; gen++) {
    Scored sc[8];
    for (int i = 0; i < 8; i++) {
      sc[i].f = fitness(pop[i]);
      sc[i].x = pop[i];
      sc[i].idx = i;
    }
    sort_scored(sc, 8);
    int newpop[8], np = 0;
    newpop[np++] = sc[0].x;
    while (np < 8) {
      int p1 = roulette_pick(sc);
      int p2 = roulette_pick(sc);
      int cut = (int)below(9) + 1;
      out->crossovers++;
      int mask = (1 << cut) - 1;
      int child = (p1 & mask) | (p2 & ~mask);
      for (int b = 0; b < 10; b++)
        if (below(100) < 4) {
          child ^= 1 << b;
          out->flips++;
        }
      newpop[np++] = child;
    }
    for (int i = 0; i < 8; i++)
      pop[i] = newpop[i];
  }
  out->fb = fitness(pop[0]);
  out->bestx = pop[0];
  for (int i = 1; i < 8; i++)
    if (fitness(pop[i]) > out->fb) {
      out->fb = fitness(pop[i]);
      out->bestx = pop[i];
    }
}
```

listing 1531: c, the run whole: seeded population, insertion sort, roulette walks, children (source: dsa/samples-c/src/Ch41/ga.c, lines 91-132)

```
func GaKnapsack(seed uint64, popSize, gens int) GaResult {
  g := NewLcg(seed)
  pop := make([]int, popSize)
  for i := range pop {
    pop[i] = int(g.Below(1024))
  }
  var res GaResult
  for gen := 0; gen < gens; gen++ {
    scored := gaScored(pop)
    newPop := make([]int, 1, popSize)
    newPop[0] = scored[0].x
    for len(newPop) < popSize {
      p1 := roulettePick(g, scored)
      p2 := roulettePick(g, scored)
      cut := int(g.Below(9)) + 1
      res.Meter.Crossovers++
      mask := 1<<cut - 1
      child := p1&mask | p2&^mask
      for b := 0; b < 10; b++ {
        if g.Below(100) < 4 {
          child ^= 1 << b
          res.Meter.MutationFlips++
        }
      }
      newPop = append(newPop, child)
    }
    pop = newPop
  }
  res.Population = pop
  for _, x := range pop {
    if f := KnapsackFitness(x); f > res.BestFitness {
      res.BestFitness, res.BestIndividual = f, x
    }
  }
  return res
}
```

listing 1532: go, the metered run, the scored sort, two draws per child (source: dsa/samples-go/ch41/ga.go, lines 85-120)

```
  static void gaRun(long seed) {
    lcgSeed(seed);
    int[] pop = new int[8];
    for (int i = 0; i < 8; i++)
      pop[i] = below(1024);
    crossovers = 0;
    flips = 0;
    for (int gen = 0; gen < 20; gen++) {
      Scored[] sc = new Scored[8];
      for (int i = 0; i < 8; i++)
        sc[i] = new Scored(fitness(pop[i]), pop[i], i);
      // descending fitness, ties by smaller bitstring, then position
      for (int i = 1; i < 8; i++) {
        Scored v = sc[i];
        int j = i - 1;
        while (j ≥ 0
            && (v.f() > sc[j].f()
                || (v.f() == sc[j].f()
                    && (v.x() < sc[j].x()
                        || (v.x() == sc[j].x() && v.idx() < sc[j].idx()))))) {
          sc[j + 1] = sc[j];
          j--;
        }
        sc[j + 1] = v;
      }
      int[] newpop = new int[8];
      int np = 0;
      newpop[np++] = sc[0].x(); // elitism 1
      while (np < 8) {
        int p1 = roulettePick(sc);
        int p2 = roulettePick(sc);
        int cut = below(9) + 1;
        crossovers++;
        int mask = (1 << cut) - 1;
        int child = (p1 & mask) | (p2 & ~mask);
        for (int b = 0; b < 10; b++)
          if (below(100) < 4) {
            child ^= 1 << b;
            flips++;
          }
        newpop[np++] = child;
      }
      pop = newpop;
    }
    fb = fitness(pop[0]);
    bestx = pop[0];
    for (int i = 1; i < 8; i++)
      if (fitness(pop[i]) > fb) {
        fb = fitness(pop[i]);
        bestx = pop[i];
      }
  }
```

listing 1533: java, the run whole: seeded population, insertion sort with the tie chain, roulette walks, children (source: dsa/samples-java/src/Ch41/Ga.java, lines 67-118)

```
        var g = new Lcg((ulong)seed);
        var pop = new int[popn];
        for (int i = 0; i < popn; i++)
            pop[i] = g.Below(1024);
        int crossovers = 0;
        int mutationFlips = 0;
        for (int gen = 0; gen < gens; gen++)
        {
            var scored = new (int Fitness, int X)[popn];
            for (int i = 0; i < popn; i++)
                scored[i] = (KnapFitness(pop[i]), pop[i]);
            SortByFitness(scored); // fitness desc, ties by smaller bitstring
            var newPop = new int[popn];
            newPop[0] = scored[0].X; // elitism keeps the best individual
            int count = 1;

            int Pick()
            {
                int total = 0;
                for (int i = 0; i < popn; i++)
                    total += scored[i].Fitness;
                int point = total > 0 ? g.Below((uint)total) : 0;
                int run = 0;
                for (int i = 0; i < popn; i++)
                {
                    run += scored[i].Fitness;
                    if (run > point)
                        return scored[i].X;
                }
                return scored[popn - 1].X;
            }

            while (count < popn)
            {
                int p1 = Pick();
                int p2 = Pick();
                int cut = g.Below(9) + 1;
                crossovers++;
                int mask = (1 << cut) - 1;
                int child = (p1 & mask) | (p2 & (1023 & ~mask));
                for (int b = 0; b < 10; b++)
                {
                    if (g.Below(100) < 4)
                    {
                        child ^= 1 << b;
                        mutationFlips++;
                    }
                }
                newPop[count++] = child;
            }
```

listing 1534: c#, the run body with the roulette closure over the scored array (source: dsa/samples/src/Ch41/Ga.cs, lines 47-96)

```
export function gaRun(seed, popn = 8, gens = 20) {
  const g = makeLcg(seed);
  let pop = Array.from({ length: popn }, () => g.below(1024));
  let crossovers = 0;
  let mutationFlips = 0;
  for (let gen = 0; gen < gens; gen++) {
    // highest fitness first, ties by smaller bitstring: sort by
    // (fitness, -x) descending, stable on full ties
    const scored = pop.map((x) => [knapFitness(x), -x, x])
      .sort((a, b) => b[0] - a[0] || b[1] - a[1]);
    const newPop = [scored[0][2]];
    const pick = () => {
      let total = 0;
      for (const s of scored) total += s[0];
      const point = total > 0 ? g.below(total) : 0;
      let run = 0;
      for (const s of scored) {
        run += s[0];
        if (run > point) return s[2];
      }
      return scored[scored.length - 1][2];
    };
    while (newPop.length < popn) {
      const p1 = pick();
      const p2 = pick();
      const cut = g.below(9) + 1;
      crossovers += 1;
      const mask = (1 << cut) - 1;
      let child = (p1 & mask) | (p2 & (1023 & ~mask));
      for (let b = 0; b < 10; b++) {
        if (g.below(100) < 4) {
          child ^= 1 << b;
          mutationFlips += 1;
        }
      }
      newPop.push(child);
    }
    pop = newPop;
  }
  let bestX = pop[0];
  let bestFitness = knapFitness(pop[0]);
  for (const x of pop) {
    if (knapFitness(x) > bestFitness) {
      bestFitness = knapFitness(x);
      bestX = x;
    }
  }
  return { population: pop, bestFitness, crossovers, mutationFlips, bestX };
}
```

listing 1535: javascript, the run with the stable scored sort and the cumulative pick (source: dsa/samples-js/src/ch41-ga.mjs, lines 34-82)

```
def ga(seed, popn=8, gens=20, mut_pct=4):
    g = Lcg(seed)
    pop = [g.below(1024) for _ in range(popn)]
    flips = 0
    crosses = 0
    for _ in range(gens):
        # elitism 1: the highest fitness, ties to the smaller bitstring
        scored = sorted(((fitness(x), -x, x) for x in pop), reverse=True)
        newpop = [scored[0][2]]

        def pick():
            total = sum(f for f, _, _ in scored)
            point = g.below(total) if total > 0 else 0
            run = 0
            for f, _, x in scored:
                run += f
                if run > point:
                    return x
            return scored[-1][2]

        while len(newpop) < popn:
            p1, p2 = pick(), pick()
            cut = g.below(9) + 1
            crosses += 1
            child = (p1 & ((1 << cut) - 1)) | (p2 & ~((1 << cut) - 1))
            for b in range(10):
                if g.below(100) < mut_pct:
                    child ^= 1 << b
                    flips += 1
            newpop.append(child)
        pop = newpop
    return pop, max(fitness(x) for x in pop), crosses, flips
```

listing 1536: python, the generator, elitism 1, the pick closure, the child loop (source: dsa/samples-py/src/Ch41/ga.py, lines 64-95)

```lua
-- the contract: population sorted fitness-descending then bitstring
-- ascending, one below(total) draw per roulette parent, cut =
-- below(9) + 1, the child keeps parent 1's low cut bits, mutation
-- draws below(100) per bit ascending and flips under 4
local function ga(seed)
  local g = lcg(seed)
  local pop = {}
  for i = 1, 8 do pop[i] = g.below(1024) end
  local mut, cross = 0, 0
  for _ = 1, 20 do
    local scored = {}
    for i = 1, 8 do scored[i] = {pop[i], fitness(pop[i])} end
    table.sort(scored, function(a, b)
      if a[2] ~= b[2] then return a[2] > b[2] end
      return a[1] < b[1]
    end)
    local function pick()
      local total = 0
      for i = 1, 8 do total = total + scored[i][2] end
      local point = 0
      if total > 0 then point = g.below(total) end
      local run = 0
      for i = 1, 8 do
        run = run + scored[i][2]
        if run > point then return scored[i][1] end
      end
      return scored[8][1]
    end
    local newpop = {scored[1][1]}
    while #newpop < 8 do
      local p1, p2 = pick(), pick()
      local cut = g.below(9) + 1
      cross = cross + 1
      local lowmask = (1 << cut) - 1
      local child = (p1 & lowmask) | (p2 & ~lowmask)
      for b = 0, 9 do
        if g.below(100) < 4 then
          child = child ~ (1 << b)
          mut = mut + 1
        end
      end
      newpop[#newpop + 1] = child
    end
    pop = newpop
  end
  local fb, bx = 0, nil
  for i = 1, 8 do
    local f = fitness(pop[i])
    if f > fb then fb, bx = f, pop[i] end
  end
  return fb, bx, cross, mut
end
```

listing 1537: lua, the contract comment, the comparator, pick, and children (source: dsa/samples-lua/ch41_ga.lua, lines 49-100)

WARNING: selection pressure can kill the run

> Seed 2′s population converged onto individuals that overflow the capacity, every fitness 0, and roulette over a total of 0 returns the last individual of the sorted array: the population is frozen and mutation at 4 percent per bit never digs all 8 individuals out within 20 generations. The assert states the 0. The honest readings are a smaller mutation grid, a repair operator, or tournament selection whose pressure does not depend on fitness sums.

The fixture family: the oracle pair 37 and 288, the three tournament pins, the three run rows with 140 crossovers exact and flip counts 57, 49, 52, the ratio lane at 0.75, and the seed 2 anatomy, all 8 final individuals at fitness 0. Flip counts sit near their expectation, 4 percent of 1400 mutation draws is 56, and the three seeds land 57, 49, and 52 around it.

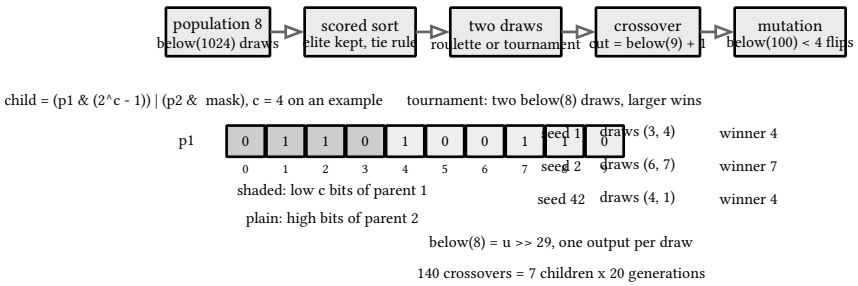


figure 371: the generation pipeline: scored sort, elite kept, two draws, the mask crossover, per-bit mutation, and the pinned tournament mini-run

41.9. crossover, mutation, ox

On bitstrings the operators are masks. Single-point crossover draws a cut below(9) + 1 between 1 and 9 and builds the child as the low cut bits of parent 1 glued to the high bits of parent 2, one mask and one or per child. Mutation walks the 10 bits in ascending order, draws below(100) for each, and flips when the draw lands under 4, a 4 percent per-bit rate realized without a float. Elitism carries the best individual of the sorted population forward untouched, 7 children fill the rest, which is why the crossover meter reads exactly 140.

Permutations cannot be masked, a slice of one parent pasted into the other duplicates cities. Order crossover, OX, is the repair: the child keeps positions cut1..cut2-1 of parent 1, then fills the remaining positions cycling from cut2 with the genes of parent 2 in order, skipping the kept ones. It is deterministic, no rng anywhere, and both worked examples pin.

The dry run: the seven-way contract pins the two OX children and the exact mutation ledger.

1. The worked pair: ox([1, 2, 3, 4, 5], [3, 1, 2, 5, 4], 1, 3) returns [4, 2, 3, 1, 5]: positions 1..2 keep [2, 3], the fill list is parent 2′s genes minus the kept pair, 1, 5, 4, and the slots cycle 3, 4, 0, landing 1, 5, 4 into positions 3, 4, 0.
2. The second pin: ox([5, 4, 3, 2, 1], [2, 1, 4, 3, 5], 1, 4) returns [5, 4, 3, 2, 1], the case where OX reproduces parent 1 exactly, 4 of 5 genes kept and the fill order cooperating.
3. The child is a permutation in both cases, checked, and the flip ledger reads 57, 49, 52 against the expectation of 56, 4 percent of the 1400 mutation draws of a run.

The fill cycling is the whole algorithm, and the listings below ship OX in seven languages.

```c
// order crossover: keep p1[cut1..cut2-1], fill the rest cycling with the
// genes of p2 in order, skipping the kept ones
static void ox(const int *p1, const int *p2, int cut1, int cut2, int *child) {
  int used[6] = {0};
  for (int i = cut1; i < cut2; i++) {
    child[i] = p1[i];
    used[p1[i]] = 1;
  }
  int k = 0;
  for (int phase = 0; phase < 2; phase++)
    for (int i = phase == 0 ? cut2 : 0; i < (phase == 0 ? 5 : cut1); i++) {
      while (used[p2[k]])
        k++;
      child[i] = p2[k];
      used[p2[k]] = 1;
      k++;
    }
}
```

listing 1538: c, order crossover keeping the cut segment, the two-phase cycling fill (source: dsa/samples-c/src/Ch41/ga.c, lines 134-151)

```go
// OX is order crossover on permutations: the child keeps positions
// cut1..cut2-1 of parent 1, then fills the remaining positions
// cycling from cut2 with the genes of parent 2 in order, skipping
// the kept ones. Deterministic, no RNG.
func OX(p1, p2 []int, cut1, cut2 int) []int {
  n := len(p1)
  child := make([]int, n)
  kept := make(map[int]bool, n)
  for i := cut1; i < cut2; i++ {
    child[i] = p1[i]
    kept[p1[i]] = true
  }
  var fill []int
  for _, gene := range p2 {
    if !kept[gene] {
      fill = append(fill, gene)
    }
  }
  k := 0
  for i := cut2; i < n; i++ {
    child[i] = fill[k]
    k++
  }
  for i := 0; i < cut1; i++ {
    child[i] = fill[k]
    k++
  }
  return child
}
```

listing 1539: go, the kept map and the wrapped slot walk (source: dsa/samples-go/ch41/ga.go, lines 133-161)

```
static void ox(int[] p1, int[] p2, int cut1, int cut2, int[] child) {
  boolean[] used = new boolean[6];
  for (int i = cut1; i < cut2; i++) {
    child[i] = p1[i];
    used[p1[i]] = true;
  }
  int k = 0;
  for (int phase = 0; phase < 2; phase++)
    for (int i = phase == 0 ? cut2 : 0; i < (phase == 0 ? 5 : cut1); i++) {
      while (used[p2[k]])
        k++;
      child[i] = p2[k];
      used[p2[k]] = true;
      k++;
    }
}
```

listing 1540: java, order crossover keeping the cut segment, the two-phase cycling fill (source: dsa/samples-java/src/Ch41/Ga.java, lines 122-137)

```
/// <summary>Order crossover: positions cut1..cut2-1 keep parent 1, the
/// rest fills cycling from cut2 with parent 2's genes in order.</summary>
public static int[] Ox(int[] p1, int[] p2, int cut1, int cut2)
{
    int n = p1.Length;
    var child = new int[n];
    var kept = new HashSet<int>();
    for (int i = cut1; i < cut2; i++)
    {
        child[i] = p1[i];
        kept.Add(p1[i]);
    }
    var fill = new List<int>();
    for (int i = 0; i < n; i++)
    {
        if (!kept.Contains(p2[i]))
            fill.Add(p2[i]);
    }
    int k = 0;
    for (int i = cut2; i < n; i++)
        child[i] = fill[k++];
    for (int i = 0; i < cut1; i++)
        child[i] = fill[k++];
    return child;
}
```

listing 1541: c#, ox with the kept set and the fill list (source: dsa/samples/src/Ch41/Ga.cs, lines 113-137)

```
export function ox(p1, p2, cut1, cut2) {
  const n = p1.length;
  const child = new Array(n).fill(null);
  for (let i = cut1; i < cut2; i++) child[i] = p1[i];
  const fill = p2.filter((gene) => !child.includes(gene));
  const order = [];
  for (let i = cut2; i < n; i++) order.push(i);
  for (let i = 0; i < cut1; i++) order.push(i);
  let k = 0;
  for (const i of order) {
    child[i] = fill[k];
    k += 1;
  }
  return child;
}
```

listing 1542: javascript, ox through filter and the cycling slot order (source: dsa/samples-js/src/ch41-ga.mjs, lines 84-98)

```
# order crossover: keep positions cut1..cut2-1 of parent 1, fill the
# rest cycling from cut2 with the genes of parent 2 in order
def ox(p1, p2, cut1, cut2):
    child = [None] * len(p1)
    for i in range(cut1, cut2):
        child[i] = p1[i]
    fill = [g for g in p2 if g not in child]
    k = 0
    for i in list(range(cut2, len(p1))) + list(range(0, cut1)):
        child[i] = fill[k]
        k += 1
    return child


child = ox([1, 2, 3, 4, 5], [3, 1, 2, 5, 4], 1, 3)
check(child == [4, 2, 3, 1, 5], "ox keeps [2, 3] and fills cycling")
check(sorted(child) == [1, 2, 3, 4, 5], "the child is a permutation")
child = ox([5, 4, 3, 2, 1], [2, 1, 4, 3, 5], 1, 4)
check(child == [5, 4, 3, 2, 1], "ox can reproduce parent 1 exactly")
```

listing 1543: python, the fill list and the wrapped slot order, both pins beside it (source: dsa/samples-py/src/Ch41/ga.py, lines 141-159)

```lua
-- order crossover, deterministic, no rng: keep positions cut1+1..cut2 of
-- parent 1, fill the rest cycling from cut2+1 with parent 2's genes in
-- order, skipping the kept ones
local function ox(p1, p2, cut1, cut2)
  local n = #p1
  local child = {}
  local keep = {}
  for i = cut1 + 1, cut2 do
    child[i] = p1[i]
    keep[p1[i]] = true
  end
  local fill = {}
  for i = 1, n do
    if not keep[p2[i]] then fill[#fill + 1] = p2[i] end
  end
  local slots = {}
  for i = cut2 + 1, n do slots[#slots + 1] = i end
  for i = 1, cut1 do slots[#slots + 1] = i end
  for k, i in ipairs(slots) do child[i] = fill[k] end
  return child
end
```

listing 1544: lua, the 1-based cut arithmetic mirroring the contract (source: dsa/samples-lua/ch41_ga.lua, lines 102-122)

The fixture family: the two OX children, the permutation checks, the 140 crossover identity, and the flip counts with their expectation. The mask operators and OX are the two encodings' halves of the same idea, mix two parents without leaving the representable set, a bitstring any bitstring, a permutation any permutation.

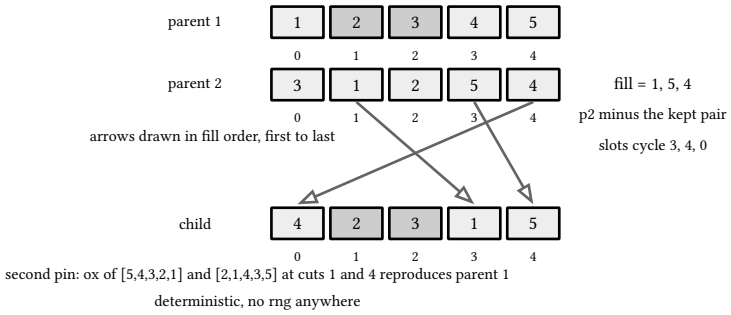


figure 372: the ox worked example: positions 1..2 kept from parent 1, the fill list 1, 5, 4 cycling into slots 3, 4, 0, the child assembled

41.10. ant colony optimization and the pheromone model

The ant moves the memory outside the search. A pheromone value tau sits on every directed edge, all initialized to tau0 = 100, and each of 4 ants builds a tour greedily but randomly: ant k starts at city k mod 8, and each next city is drawn among the unvisited with integer weight w = tau[cur][j] times (10000 / d), one below(total) draw then a cumulative walk. After the tours, every off-diagonal pheromone evaporates, tau = tau times 9 / 10 with integer division, and every ant deposits 10000 / L along both directions of its tour edges, so short tours lay stronger trails. Every rational step is a floor, never a rounded float, and the whole model stays in integers.

The weights make the preference explicit: at equal tau the weight ratio is the inverse distance ratio, so an edge of length 2 carries 5000 per unit tau, 10000 over 2, against 1250 for length 8, four to one. The deposit arithmetic prices an optimal tour at 625 per edge, 10000 over 16, and evaporation floors, 137 times 9 over 10 = 123.

The dry run: the seven-way contract pins the three base runs, their best tours, and the exact draw count.

1. Seed 1: best 16, tour [2, 1, 0, 7, 6, 5, 4, 3].
2. Seed 2: best 16, tour [0, 1, 2, 3, 4, 5, 6, 7].
3. Seed 42: best 16, tour [2, 3, 4, 5, 6, 7, 0, 1].

4. 280 draws = 4 ants x 7 choices x 10 iterations, exact on every seed, and best = 16 = OPT, ratio 1.0 stated as the tolerance.
5. The integer facts, asserted in lua: 10000

seed	best	best tour	draws
1	16	[2,1,0,7,6,5,4,3]	280
2	16	[0,1,2,3,4,5,6,7]	280
42	16	[2,3,4,5,6,7,0,1]	280

Every seed reaches the optimum, and the listings below run the model in seven languages.

```
    for (int k = 0; k < 4; k++) {
      int startc = k % 8, vis[8] = {0}, tn = 1, cur = startc;
      vis[startc] = 1;
      tours[k][0] = startc;
      for (int step = 0; step < 7; step++) {
        int cand[8];
        long long ws[8], total = 0;
        int nc = 0;
        for (int j = 0; j < 8; j++)
          if (!vis[j]) {
            long long w = tau[cur][j] * (10000 / ring[cur][j]);
            cand[nc] = j;
            ws[nc] = w;
            nc++;
            total += w;
          }
        uint32_t point = below((uint32_t)total);
        out->draws++;
        long long run = 0;
        int pick = cand[nc - 1];
        for (int c = 0; c < nc; c++) {
          run += ws[c];
          if (run > (long long)point) {
            pick = cand[c];
            break;
          }
        }
        vis[pick] = 1;
        tours[k][tn++] = pick;
        cur = pick;
      }
      lens[k] = tour_len(tours[k]);
    }
    if (variant == 0) {
      // base: evaporate, deposit every ant, then note the best
      for (int i = 0; i < 8; i++)
        for (int j = 0; j < 8; j++)
          if (i != j)
            tau[i][j] = tau[i][j] * 9 / 10;
      for (int k = 0; k < 4; k++)
        for (int i = 0; i < 8; i++) {
          int a = tours[k][i], b = tours[k][(i + 1) % 8];
          tau[a][b] += 10000 / lens[k];
          tau[b][a] += 10000 / lens[k];
        }
      for (int k = 0; k < 4; k++)
        if (out->bestl < 0 || lens[k] < out->bestl) {
          out->bestl = lens[k];
          for (int i = 0; i < 8; i++)
            out->bestt[i] = tours[k][i];
        }
```

listing 1545: c, the ant walk with integer weights, the draw and cumulative pick, the base update (source: dsa/samples-c/src/Ch41/aco.c, lines 64-114)

```
    for k := 0; k < ants; k++ {
      vis := make([]bool, n)
      vis[k%n] = true
      tour := []int{k % n}
      cur := k % n
      for step := 1; step < n; step++ {
        total := 0
        for j := 0; j < n; j++ {
          if !vis[j] {
            total += tau[cur][j] * (10000 / d[cur][j])
          }
        }
        point := int(g.Below(uint32(total)))
        run := 0
        next := -1
        for j := 0; j < n; j++ {
          if vis[j] {
            continue
          }
          run += tau[cur][j] * (10000 / d[cur][j])
          if run > point {
            next = j
            break
          }
        }
        vis[next] = true
        tour = append(tour, next)
        cur = next
      }
      tours = append(tours, antTour{TourLength(d, tour), tour})
    }
    for i := range tours {
      if bestLen < 0 || tours[i].length < bestLen {
        bestLen, bestTour = tours[i].length, tours[i].tour
      }
    }
    for i := 0; i < n; i++ {
      for j := 0; j < n; j++ {
        if i != j {
          tau[i][j] = tau[i][j] * 9 / 10
        }
      }
    }
```

listing 1546: go, the ant loop, the best bank, the evaporation sweep (source: dsa/samples-go/ch41/aco.go, lines 57-99)

```
      for (int k = 0; k < 4; k++) {
        int startc = k % 8;
        boolean[] vis = new boolean[8];
        int tn = 1, cur = startc;
        vis[startc] = true;
        tours[k][0] = startc;
        for (int step = 0; step < 7; step++) {
          int[] cand = new int[8];
          long[] ws = new long[8];
          long total = 0;
          int nc = 0;
          for (int j = 0; j < 8; j++)
            if (!vis[j]) {
              long w = tau[cur][j] * (10000 / ring[cur][j]);
              cand[nc] = j;
              ws[nc] = w;
              nc++;
              total += w;
            }
          int point = below((int) total);
          draws++;
          long run = 0;
          int pick = cand[nc - 1];
          for (int c = 0; c < nc; c++) {
            run += ws[c];
            if (run > point) {
              pick = cand[c];
              break;
            }
          }
          vis[pick] = true;
          tours[k][tn++] = pick;
          cur = pick;
        }
        lens[k] = tourLen(tours[k]);
      }
      if (variant == 0) {
        // base: evaporate, deposit every ant, then note the best
        for (int i = 0; i < 8; i++)
          for (int j = 0; j < 8; j++)
            if (i != j)
              tau[i][j] = tau[i][j] * 9 / 10;
        for (int k = 0; k < 4; k++)
          for (int i = 0; i < 8; i++) {
            int a = tours[k][i], b = tours[k][(i + 1) % 8];
            tau[a][b] += 10000 / lens[k];
            tau[b][a] += 10000 / lens[k];
          }
        for (int k = 0; k < 4; k++)
          if (bestLen < 0 || lens[k] < bestLen) {
            bestLen = lens[k];
            System.arraycopy(tours[k], 0, bestTour, 0, 8);
          }
```

listing 1547: java, the ant walk with long weights, the draw and cumulative pick, the base update (source: dsa/samples-java/src/Ch41/Aco.java, lines 62-114)

```
            for (int k = 0; k < ants; k++)
            {
                int start = k % n;
                var vis = new bool[n];
                vis[start] = true;
                var t = new List<int> { start };
                int cur = start;
                for (int step = 1; step < n; step++)
                {
                    var cand = new List<int>();
                    var ws = new List<long>();
                    for (int j = 0; j < n; j++)
                    {
                        if (!vis[j])
                        {
                            cand.Add(j);
                            ws.Add(tau[cur][j] * (10000 / d[cur][j]));
                        }
                    }
                    long total = 0;
                    foreach (long w in ws)
                        total += w;
                    int point = g.Below((uint)total);
                    int chosen = cand[cand.Count - 1];
                    long run = 0;
                    for (int idx = 0; idx < cand.Count; idx++)
                    {
                        run += ws[idx];
                        if (run > point)
                        {
                            chosen = cand[idx];
                            break;
                        }
                    }
                    vis[chosen] = true;
                    t.Add(chosen);
                    cur = chosen;
                }
                var arr = t.ToArray();
                tours.Add((Localsearch.TourLen(d, arr), arr));
            }
```

listing 1548: c#, the ant build loop and the weighted choice (source: dsa/samples/src/Ch41/Aco.cs, lines 46-86)

```
  for (let k = 0; k < ants; k++) {
    const start = k % n;
    const vis = new Array(n).fill(false);
    vis[start] = true;
    const t = [start];
    let cur = start;
    for (let step = 1; step < n; step++) {
      const cand = [];
      const ws = [];
      for (let j = 0; j < n; j++) {
        if (!vis[j]) {
          cand.push(j);
          ws.push(tau[cur][j] * Math.floor(10000 / d[cur][j]));
        }
      }
      let total = 0;
      for (const w of ws) total += w;
      const point = g.below(total);
      let chosen = cand[cand.length - 1];
      let run = 0;
      for (let idx = 0; idx < cand.length; idx++) {
        run += ws[idx];
        if (run > point) {
          chosen = cand[idx];
          break;
        }
      }
      vis[chosen] = true;
      t.push(chosen);
      cur = chosen;
    }
    tours.push([tourLen(d, t), t]);
  }
  // best-so-far is banked before the deposits so the elitist extra
  // deposit is defined from iteration 0
  for (const [l, t] of tours) {
    if (bestL === null || l < bestL) {
      bestL = l;
      bestT = t;
    }
  }
  for (let i = 0; i < n; i++) {
    for (let j = 0; j < n; j++) {
      if (i =/= j) tau[i][j] = Math.floor((tau[i][j] * 9) / 10);
    }
  }
```

listing 1549: javascript, the walk, the banked best, the evaporation (source: dsa/samples-js/src/ch41-aco.mjs, lines 22-67)

```
def aco(seed, variant="base", ants=4, iters=10, tau0=100):
    n = 8
    tau = [[tau0] * n for _ in range(n)]
    g = Lcg(seed)
    draws = 0
    best_len, best_tour = None, None
    for _ in range(iters):
        tours = []
        for k in range(ants):
            cur = k % n
            vis = [False] * n
            vis[cur] = True
            t = [cur]
            for _ in range(n - 1):
                cand = [j for j in range(n) if not vis[j]]
                ws = [tau[cur][j] * (10000 // D[cur][j]) for j in cand]
                point = g.below(sum(ws))
                draws += 1
                run = 0
                for j, w in zip(cand, ws):
                    run += w
                    if run > point:
                        break
                vis[j] = True
                t.append(j)
                cur = j
            tours.append((tour_len(t), t))
        # best-so-far updates before the deposits, so the elitist extra
        # deposit is defined from iteration 0
        for ln, t in tours:
            if best_len is None or ln < best_len:
                best_len, best_tour = ln, t
        for i in range(n):
            for j in range(n):
                if i != j:
                    tau[i][j] = tau[i][j] * 9 // 10
        if variant in ("base", "elitist"):
            for ln, t in tours:
                for i in range(n):
                    a, b = t[i], t[(i + 1) % n]
                    tau[a][b] += 10000 // ln
                    tau[b][a] += 10000 // ln
```

listing 1550: python, the model whole, weights, walk, evaporate, deposit (source: dsa/samples-py/src/Ch41/aco.py, lines 47-88)

```
  for _ = 1, 10 do
    local tours = {}
    for k = 1, 4 do
      local startc = (k - 1) % n
      local vis = {[startc] = true}
      local t = {startc}
      local cur = startc
      for _ = 1, n - 1 do
        local cand, ws = {}, {}
        for j = 0, n - 1 do
          if not vis[j] then
            cand[#cand + 1] = j
            ws[#ws + 1] = tau[cur][j] * (10000 // RING[cur + 1][j + 1])
          end
        end
        local total = 0
        for _, w in ipairs(ws) do total = total + w end
        local point = g.below(total)
        draws = draws + 1
        local run, chosen = 0, cand[#cand]
        for idx, j in ipairs(cand) do
          run = run + ws[idx]
          if run > point then chosen = j break end
        end
        vis[chosen] = true
        t[#t + 1] = chosen
        cur = chosen
      end
      tours[#tours + 1] = {len = tour_len(t), tour = t}
    end
    if variant ~= nil then
      -- variants rank before depositing, so iteration 0 already feeds
      -- the elitist extra deposit
      for _, e in ipairs(tours) do
        if bestl == nil or e.len < bestl then bestl, bestt = e.len, e.tour end
      end
    end
    for i = 0, n - 1 do
      for j = 0, n - 1 do
        if i ~= j then tau[i][j] = tau[i][j] * 9 // 10 end
      end
    end
```

listing 1551: lua, the same loop with floor division throughout (source: dsa/samples-lua/ch41_aco.lua, lines 55-96)

The fixture family: the three tours, the 280 draw identity, the deposit and evaporation floors, and the optimum reached on all seeds. The trail is the algorithm's state and it is shared: one matrix of integers, updated once per iteration, read by every ant of the next one.

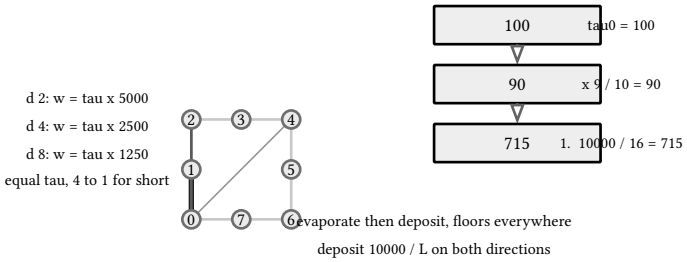


figure 373: the weight rule on the ring: short edges carry four times the weight of long ones at equal tau, and the tau arithmetic strip: 100, evaporate to 90, deposit 625

41.11. aco variants

The variants change only the update rule, never ant movement, so every run consumes the same 280 draws and only the trail differs. Elitist adds an extra deposit of 10000 / L along the best-so-far tour after every ant has deposited, and the best is banked before the deposits each iteration so the extra is defined from iteration 0. Iteration-best lets only the round's best ant deposit, ties going to the lower ant index, concentrating the trail on one tour per round. MMAS, max-min ant system, keeps the iteration-best deposit and clamps every off-diagonal tau into [5, 200] after the update, bounding how strong any trail can grow and how far an unused edge can fade.

The dry run: the seven-way contract pins the variant tours on seeds 1 and 2, all at best 16 and 280 draws.

1. Elitist: seed 1 pins [3, 4, 5, 6, 7, 0, 1, 2], seed 2 pins [2, 3, 4, 5, 6, 7, 0, 1].
2. Iteration-best: seed 1 pins [0, 7, 6, 5, 4, 3, 2, 1], seed 2 pins [2, 3, 4, 5, 6, 7, 0, 1].
3. MMAS: seed 1 pins [1, 0, 7, 6, 5, 4, 3, 2], seed 2 pins [0, 1, 2, 3, 4, 5, 6, 7].
4. Every variant run answers 16, the optimum, ratio 1.0, on 280 draws exact: the clamp and the deposit changes never touch the draw sequence.

variant	seed 1 tour	seed 2 tour	draws
base	[2,1,0,7,6,5,4,3]	[0,1,2,3,4,5,6,7]	280
elitist	[3,4,5,6,7,0,1,2]	[2,3,4,5,6,7,0,1]	280
iterbest	[0,7,6,5,4,3,2,1]	[2,3,4,5,6,7,0,1]	280
mmas	[1,0,7,6,5,4,3,2]	[0,1,2,3,4,5,6,7]	280

Same reach, different tours: the update rule shapes the trail, and the listings below ship the four rules in seven languages.

```
    } else {
      // variants: best-so-far is noted before the deposits, so the elitist
      // extra deposit is defined from iteration 0
      for (int k = 0; k < 4; k++)
        if (out->bestl < 0 || lens[k] < out->bestl) {
          out->bestl = lens[k];
          for (int i = 0; i < 8; i++)
            out->bestt[i] = tours[k][i];
        }
      for (int i = 0; i < 8; i++)
        for (int j = 0; j < 8; j++)
          if (i != j)
            tau[i][j] = tau[i][j] * 9 / 10;
      if (variant == 1) {
        for (int k = 0; k < 4; k++)
          for (int i = 0; i < 8; i++) {
            int a = tours[k][i], b = tours[k][(i + 1) % 8];
            tau[a][b] += 10000 / lens[k];
            tau[b][a] += 10000 / lens[k];
          }
        for (int i = 0; i < 8; i++) {
          int a = out->bestt[i], b = out->bestt[(i + 1) % 8];
          tau[a][b] += 10000 / out->bestl;
          tau[b][a] += 10000 / out->bestl;
        }
      } else {
        int bk = 0;
        for (int k = 1; k < 4; k++)
          if (lens[k] < lens[bk])
            bk = k;
        for (int i = 0; i < 8; i++) {
          int a = tours[bk][i], b = tours[bk][(i + 1) % 8];
          tau[a][b] += 10000 / lens[bk];
          tau[b][a] += 10000 / lens[bk];
        }
        if (variant == 3)
          for (int i = 0; i < 8; i++)
            for (int j = 0; j < 8; j++)
              if (i != j) {
                if (tau[i][j] < 5)
                  tau[i][j] = 5;
                if (tau[i][j] > 200)
                  tau[i][j] = 200;
              }
```

listing 1552: c, the variant updates: rank, evaporate, deposits, the elitist extra, the mmas clamp (source: dsa/samples-c/src/Ch41/aco.c, lines 115-158)

```
    switch kind {
    case AcoPlain, AcoElitist:
      for _, at := range tours {
        acoDeposit(tau, at.tour, 10000/at.length)
      }
      if kind == AcoElitist {
        acoDeposit(tau, bestTour, 10000/bestLen)
      }
    case AcoIterBest, AcoMmas:
      bi := 0
      for i := range tours {
        if tours[i].length < tours[bi].length {
          bi = i
        }
      }
      acoDeposit(tau, tours[bi].tour, 10000/tours[bi].length)
    }
    if kind == AcoMmas {
      for i := 0; i < n; i++ {
        for j := 0; j < n; j++ {
          if i ≠ j {
            tau[i][j] = min(200, max(5, tau[i][j]))
          }
        }
      }
    }
```

listing 1553: go, the variant switch and the mmas min-max clamp (source: dsa/samples-go/ch41/aco.go, lines 100-125)

```
      } else {
        // variants: best-so-far is noted before the deposits, so the
        // elitist extra deposit is defined from iteration 0
        for (int k = 0; k < 4; k++)
          if (bestLen < 0 || lens[k] < bestLen) {
            bestLen = lens[k];
            System.arraycopy(tours[k], 0, bestTour, 0, 8);
          }
        for (int i = 0; i < 8; i++)
          for (int j = 0; j < 8; j++)
            if (i != j)
              tau[i][j] = tau[i][j] * 9 / 10;
        if (variant == 1) {
          for (int k = 0; k < 4; k++)
            for (int i = 0; i < 8; i++) {
              int a = tours[k][i], b = tours[k][(i + 1) % 8];
              tau[a][b] += 10000 / lens[k];
              tau[b][a] += 10000 / lens[k];
            }
          for (int i = 0; i < 8; i++) {
            int a = bestTour[i], b = bestTour[(i + 1) % 8];
            tau[a][b] += 10000 / bestLen;
            tau[b][a] += 10000 / bestLen;
          }
        } else {
          int bk = 0;
          for (int k = 1; k < 4; k++)
            if (lens[k] < lens[bk])
              bk = k;
          for (int i = 0; i < 8; i++) {
            int a = tours[bk][i], b = tours[bk][(i + 1) % 8];
            tau[a][b] += 10000 / lens[bk];
            tau[b][a] += 10000 / lens[bk];
          }
          if (variant == 3)
            for (int i = 0; i < 8; i++)
              for (int j = 0; j < 8; j++)
                if (i != j) {
                  if (tau[i][j] < 5)
                    tau[i][j] = 5;
                  if (tau[i][j] > 200)
                    tau[i][j] = 200;
                }
        }
      }
```

listing 1554: java, the variant updates: the banked best first, evaporate, the elitist extra, iteration-best, the mmas clamp (source: dsa/samples-java/src/Ch41/Aco.java, lines 115-159)

```
            // best-so-far is banked before the deposits so the elitist
            // extra deposit is defined from iteration 0
            foreach (var (l, tt) in tours)
            {
                if (bestT == null || l < bestL)
                {
                    bestL = l;
                    bestT = tt;
                }
            }
            for (int i = 0; i < n; i++)
            {
                for (int j = 0; j < n; j++)
                {
                    if (i != j)
                        tau[i][j] = tau[i][j] * 9 / 10;
                }
            }
            if (variant == AcoVariant.Base || variant == AcoVariant.Elitist)
            {
                foreach (var (l, tt) in tours)
                    Deposit(tau, tt, 10000 / l);
                if (variant == AcoVariant.Elitist)
                    Deposit(tau, bestT!, 10000 / bestL);
            }
            else
            {
                int iterL = tours[0].Len;
                int[] iterT = tours[0].Tour;
                foreach (var (l, tt) in tours)
                {
                    if (l < iterL)
                    {
                        iterL = l;
                        iterT = tt;
                    }
                }
                Deposit(tau, iterT, 10000 / iterL);
                if (variant == AcoVariant.Mmas)
                {
                    for (int i = 0; i < n; i++)
                    {
                        for (int j = 0; j < n; j++)
                        {
                            if (i != j)
                                tau[i][j] = Math.Clamp(tau[i][j], 5, 200);
                        }
                    }
                }
            }
```

listing 1555: c#, the banked best, the evaporation, the variant branches with Math.Clamp (source: dsa/samples/src/Ch41/Aco.cs, lines 87-136)

```
    if (variant === "base" || variant === "elitist") {
      for (const [l, t] of tours) {
        for (let i = 0; i < n; i++) {
          const a = t[i];
          const b = t[(i + 1) % n];
          tau[a][b] += Math.floor(10000 / l);
          tau[b][a] += Math.floor(10000 / l);
        }
      }
      if (variant === "elitist") {
        for (let i = 0; i < n; i++) {
          const a = bestT[i];
          const b = bestT[(i + 1) % n];
          tau[a][b] += Math.floor(10000 / bestL);
          tau[b][a] += Math.floor(10000 / bestL);
        }
      }
    } else {
      let iterL = tours[0][0];
      let iterT = tours[0][1];
      for (const [l, t] of tours) {
        if (l < iterL) {
          iterL = l;
          iterT = t;
        }
      }
      for (let i = 0; i < n; i++) {
        const a = iterT[i];
        const b = iterT[(i + 1) % n];
        tau[a][b] += Math.floor(10000 / iterL);
        tau[b][a] += Math.floor(10000 / iterL);
      }
      if (variant === "mmas") {
        for (let i = 0; i < n; i++) {
          for (let j = 0; j < n; j++) {
            if (i !== j) {
              tau[i][j] = Math.min(200, Math.max(5, tau[i][j]));
            }
          }
        }
      }
    }
```

listing 1556: javascript, the deposit branches and the clamp (source: dsa/samples-js/src/ch41-aco.mjs, lines 68-109)

```
            if variant == "elitist":
                for i in range(n):
                    a, b = best_tour[i], best_tour[(i + 1) % n]
                    tau[a][b] += 10000 // best_len
                    tau[b][a] += 10000 // best_len
        else:
            ln, t = min(tours, key=lambda row: row[0])
            for i in range(n):
                a, b = t[i], t[(i + 1) % n]
                tau[a][b] += 10000 // ln
                tau[b][a] += 10000 // ln
        if variant == "mmas":
            for i in range(n):
                for j in range(n):
                    if i != j:
                        tau[i][j] = max(5, min(200, tau[i][j]))
    return best_len, best_tour, draws
```

listing 1557: python, the variant branches, elitist extra, iterbest pick, clamp (source: dsa/samples-py/src/Ch41/aco.py, lines 89-105)

```
local depositors = tours
if variant == "iterbest" or variant == "mmas" then
  local bi = 1
  for idx = 2, #tours do
    if tours[idx].len < tours[bi].len then bi = idx end
  end
  depositors = {tours[bi]}
end
for _, e in ipairs(depositors) do
  local t = e.tour
  for i = 1, n do
    local p, q = t[i], t[i % n + 1]
    tau[p][q] = tau[p][q] + 10000 // e.len
    tau[q][p] = tau[q][p] + 10000 // e.len
  end
end
if variant == "elitist" then
  local t = bestt
  for i = 1, n do
    local p, q = t[i], t[i % n + 1]
    tau[p][q] = tau[p][q] + 10000 // bestl
    tau[q][p] = tau[q][p] + 10000 // bestl
  end
end
if variant == "mmas" then
  for i = 0, n - 1 do
    for j = 0, n - 1 do
      if i ~= j then
        tau[i][j] = math.min(200, math.max(5, tau[i][j]))
      end
    end
  end
end
```

listing 1558: lua, depositors, the elitist extra, the mmas band (source: dsa/samples-lua/ch41_aco.lua, lines 97-129)

The fixture family: the six variant tours of the table, the base tours for context, the 280 identity holding across all twelve runs, and the clamp bounds 5 and 200 asserted directly in lua, math.min(200, math.max(5, 3)) = 5 and the

same at 999 landing 200. On this fixture the variants are a wash at the optimum and differ only in which optimal rotation they bank, which is the honest reading: at n = 8 the instance is easy, the variants exist for the sizes where the trail shape decides convergence.

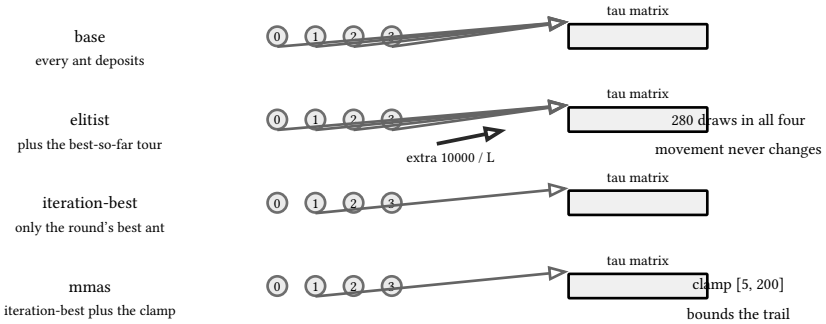


figure 374: the four update rules: base deposits from all ants, elitist adds the best-so-far, iteration-best from one ant, mmas from one ant plus the clamp band

41.12. stopping: portfolios and restart budgets

The last question is when to stop, and the honest answer is a budget. Sequential restarts of the swap hill climb give the cheapest portfolio: restart r seeds the generator with 1000 + r, the start tour is a Fisher-Yates shuffle over [0..7] drawing j = below(i + 1) for i from 7 down to 1, 7 draws per shuffle, and the exact-lane climb runs to its local optimum. The portfolio banks the cumulative best, so the anytime answer only improves: every completed restart either lowers the bank or leaves it, and a late restart stuck on a plateau never erases what earlier restarts banked. A stopping rule can therefore be an acceptance threshold on the bank, a wall-clock budget, or a fixed restart count, and the run answers sensibly under all three.

The dry run: the seven-way contract pins the five starts, traces, finals, and the bank after each restart.

1. r0 starts [2, 1, 6, 4, 7, 3, 5, 0], walks 36, 32, 28, 24, 20, 16, finishes at [0, 7, 6, 5, 4, 3, 2, 1], the bank opens at 16.
2. r1 starts [6, 7, 5, 2, 0, 4, 1, 3], walks 40, 36, 28, 24, 20, 16, finishes at [4, 5, 6, 7, 0, 1, 2, 3].
3. r2 starts [2, 1, 3, 5, 0, 7, 4, 6] and r3 starts [3, 2, 6, 1, 5, 4, 7, 0], both walk 36, 32, 28, 24, 20, 16, finishing at [4, 3, 2, 1, 0, 7, 6, 5] and [1, 2, 3, 4, 5, 6, 7, 0].
4. r4 starts [0, 1, 2, 6, 5, 4, 7, 3], walks 32, 24, and stops at [2, 1, 0, 6, 5, 4, 7, 3], a local optimum of 24: the anytime row.
5. The bank reads 16 from restart 0 onward, so a budget satisfied at 16 stops after one restart, and the fifth plateau cannot touch it.

restart	start	trace	final	local opt	bank
r0	[2,1,6,4,7,3,5,0]	36,32,28,24,20,16	[0,7,6,5,4,3,2,1]	16	16
r1	[6,7,5,2,0,4,1,3]	40,36,28,24,20,16	[4,5,6,7,0,1,2,3]	16	16
r2	[2,1,3,5,0,7,4,6]	36,32,28,24,20,16	[4,3,2,1,0,7,6,5]	16	16
r3	[3,2,6,1,5,4,7,0]	36,32,28,24,20,16	[1,2,3,4,5,6,7,0]	16	16
r4	[0,1,2,6,5,4,7,3]	32,24	[2,1,0,6,5,4,7,3]	24	16

Four of five restarts bank the optimum and the fifth plateaus at 24, and the listings below run the portfolio in seven languages.

```
// fisher-yates over [0..7]: i runs 7 down to 1, j = below(i + 1)
static void seeded_shuffle(uint64_t seed, int *t) {
  lcg_seed(seed);
  for (int i = 0; i < 8; i++)
    t[i] = i;
  for (int i = 7; i ≥ 1; i--) {
    int j = (int)below((uint32_t)(i + 1));
    int tmp = t[i];
    t[i] = t[j];
    t[j] = tmp;
  }
}

// the swap hill climb from the hill chapter, first improvement
static int climb(const int *start, int *trace, int *tn, int *final) {
  int t[8];
  for (int i = 0; i < 8; i++)
    t[i] = start[i];
  trace[0] = tour_len(t);
  *tn = 1;
  for (;;) {
    int moved = 0;
    for (int i = 0; i < 7 && !moved; i++)
      for (int j = i + 1; j < 8 && !moved; j++) {
        int cand[8];
        for (int k = 0; k < 8; k++)
          cand[k] = t[k];
        cand[i] = t[j];
        cand[j] = t[i];
        int len = tour_len(cand);
        if (len < trace[*tn - 1]) {
          for (int k = 0; k < 8; k++)
            t[k] = cand[k];
          trace[(*tn)++] = len;
          moved = 1;
        }
      }
    if (!moved) {
      for (int k = 0; k < 8; k++)
        final[k] = t[k];
      return trace[*tn - 1];
    }
  }
}
```

listing 1559: c, the fisher-yates start generator and the restart file's own climb (source: dsa/samples-c/src/Ch41/restarts.c, lines 46-89)

```go
// RestartPortfolio runs sequential restarts of the swap hill climb,
// restart r seeded 1000 + r, banking the best local optimum seen.
// Anytime semantics: every completed restart only improves the
// cumulative best, so the budget can stop as soon as it is good
// enough.
func RestartPortfolio(d [][]int, restarts int) []RestartRun {
  runs := make([]RestartRun, 0, restarts)
  cum := 0
  for r := 0; r < restarts; r++ {
    start := SeededShuffle(len(d), NewLcg(uint64(1000+r)))
    final, trace := HillClimb(d, start)
    v := trace[len(trace)-1]
    if r == 0 || v < cum {
      cum = v
    }
    runs = append(runs, RestartRun{
      Restart:  r,
      Seed:     uint64(1000 + r),
      Start:    start,
      Final:    final,
      Trace:    trace,
      LocalOpt: v,
      CumBest:  cum,
    })
  }
  return runs
}
```

listing 1560: go, the portfolio banking the best local optimum (source: dsa/samples-go/ch41/restarts.go, lines 30-56)

```
  static void seededShuffle(long seed, int[] t) {
    lcgSeed(seed);
    for (int i = 0; i < 8; i++)
      t[i] = i;
    for (int i = 7; i ≥ 1; i--) {
      int j = below(i + 1);
      int tmp = t[i];
      t[i] = t[j];
      t[j] = tmp;
    }
  }

  static int traceN;
  static int[] trace = new int[64];

  // the swap hill climb from the hill chapter, first improvement
  static int climb(int[] start, int[] finalOut) {
    int[] t = start.clone();
    trace[0] = tourLen(t);
    traceN = 1;
    for (;;) {
      boolean moved = false;
      for (int i = 0; i < 7 && !moved; i++)
        for (int j = i + 1; j < 8 && !moved; j++) {
          int[] cand = t.clone();
          cand[i] = t[j];
          cand[j] = t[i];
          int len = tourLen(cand);
          if (len < trace[traceN - 1]) {
            t = cand;
            trace[traceN++] = len;
            moved = true;
          }
        }
      if (!moved) {
        System.arraycopy(t, 0, finalOut, 0, 8);
        return trace[traceN - 1];
      }
    }
  }
```

listing 1561: java, the fisher-yates start generator over the shared lcg, the restart file's own climb (source: dsa/samples-java/src/Ch41/Restarts.java, lines 46-86)

```
    public static int[] SeededShuffle(int n, Lcg g)
    {
        var t = new int[n];
        for (int i = 0; i < n; i++)
            t[i] = i;
        for (int i = n - 1; i ≥ 1; i--)
        {
            int j = g.Below((uint)(i + 1));
            (t[i], t[j]) = (t[j], t[i]);
        }
        return t;
    }

    public static
        (int Restart, int[] Start, int[] Trace, int[] Final, int Cumulative)[]
        Portfolio(int[][] d, int runs = 5)
    {
        int cumulative = int.MaxValue;
        var rows = new (int, int[], int[], int[], int)[runs];
        for (int r = 0; r < runs; r++)
        {
            var g = new Lcg((ulong)(1000 + r));
            int[] start = SeededShuffle(d.Length, g);
            var (final, trace) = Hill.Climb(d, start);
            int localOpt = trace[^1];
            cumulative = Math.Min(cumulative, localOpt);
            rows[r] = (r, start, trace, final, cumulative);
        }
        return rows;
    }
```

listing 1562: c#, the seeded shuffle and the portfolio rows with the bank (source: dsa/samples/src/Ch41/Restarts.cs, lines 14-43)

```
export function seededShuffle(n, g) {
  const t = Array.from({ length: n }, (_, i) => i);
  for (let i = n - 1; i ≥ 1; i--) {
    const j = g.below(i + 1);
    [t[i], t[j]] = [t[j], t[i]];
  }
  return t;
}

export function restartPortfolio(d, runs = 5) {
  let cumulative = null;
  const rows = [];
  for (let r = 0; r < runs; r++) {
    const g = makeLcg(1000 + r);
    const start = seededShuffle(d.length, g);
    const { final, trace } = hillClimb(d, start);
    const localOpt = trace[trace.length - 1];
    cumulative = cumulative === null ? localOpt : Math.min(cumulative,
      localOpt);
    rows.push({ restart: r, start, trace, final, cumulative });
  }
  return rows;
}
```

listing 1563: javascript, shuffle and portfolio over the imported climb (source: dsa/samples-js/src/ch41-restarts.mjs, lines 11-33)

```
def seeded_shuffle(g):
    # fisher-yates over 0..7: j = below(i + 1) for i = 7 down to 1
    t = list(range(8))
    for i in range(7, 0, -1):
        j = g.below(i + 1)
        t[i], t[j] = t[j], t[i]
    return t


PORTFOLIO = [
    ([2, 1, 6, 4, 7, 3, 5, 0], [36, 32, 28, 24, 20, 16], [0, 7, 6, 5, 4, 3, 2, 1]),
    ([6, 7, 5, 2, 0, 4, 1, 3], [40, 36, 28, 24, 20, 16], [4, 5, 6, 7, 0, 1, 2, 3]),
    ([2, 1, 3, 5, 0, 7, 4, 6], [36, 32, 28, 24, 20, 16], [4, 3, 2, 1, 0, 7, 6, 5]),
    ([3, 2, 6, 1, 5, 4, 7, 0], [36, 32, 28, 24, 20, 16], [1, 2, 3, 4, 5, 6, 7, 0]),
    ([0, 1, 2, 6, 5, 4, 7, 3], [32, 24], [2, 1, 0, 6, 5, 4, 7, 3]),
]
cum = None
for r, (start, trace, final) in enumerate(PORTFOLIO):
    g = Lcg(1000 + r)
    st = seeded_shuffle(g)
    check(st == start, f"restart {r} fisher-yates start pins")
    check(g.draws == 7, f"restart {r} spent 7 draws on the shuffle")
    t, tr = hill(st)
    check(tr == trace, f"restart {r} descent trace pins")
    check(t == final, f"restart {r} final tour pins")
    cum = tr[-1] if cum is None else min(cum, tr[-1])
    check(cum == 16, f"restart {r} leaves the banked best at 16")
check(PORTFOLIO[4][1] == [32, 24], "the anytime row r4 plateaus at 24, not 16")
check(cum == 16, "four restarts banked the optimum, the budget stops")
```

listing 1564: python, the shuffle, the pinned portfolio, the bank checks (source: dsa/samples-py/src/Ch41/restarts.py, lines 72-100)

```
local function hill_climb(tour)
  local n = #tour
  local t = {table.unpack(tour)}
  local trace = {tour_len(t)}
  while true do
    local improved = false
    for i = 1, n - 1 do
      for j = i + 1, n do
        local cand = {table.unpack(t)}
        cand[i], cand[j] = cand[j], cand[i]
        local l = tour_len(cand)
        if l < trace[#trace] then
          t = cand
          trace[#trace + 1] = l
          improved = true
          break
        end
      end
      if improved then break end
    end
    if not improved then return t, trace end
  end
end

local function seeded_shuffle(g)
  local t = {}
  for i = 0, 7 do t[i + 1] = i end
  for i = 8, 2, -1 do -- fisher-yates off the shared lcg
    local j = g.below(i) + 1
    t[i], t[j] = t[j], t[i]
  end
  return t
end

local function restart(r)
  local g = lcg(1000 + r)
  return seeded_shuffle(g)
end
```

listing 1565: lua, the climb and the fisher-yates over the shared lcg (source: dsa/samples-lua/ch41_restarts.lua, lines 41-78)

The fixture family: the five starts, the four identical 16 traces and r1′s 40 lead, the r4 row 32, 24 at a different final tour, the bank constant at 16, and the shuffle meter, 7 draws per restart. The same seed discipline closes the chapter the way it opened: a restart portfolio is just the lcg walked twice, once for the shuffle and once inside the climb it feeds.

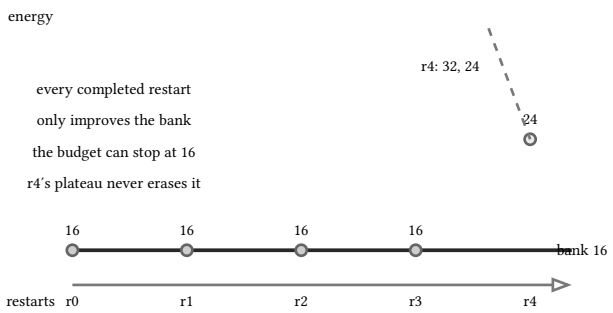


figure 375: the anytime curve: cumulative best flat at 16 from restart 0, the five local optima as dots, r4 plateauing at 24 above the bank

41.13. across the seven languages

Featured build size counted as non-blank, non-comment lines of the chapter's eight sample files per language, go test files excluded, the c column dropping its preprocessor lines:

stem	c	go	java	c#	javascript	python	lua
localsearch	103	108	104	61	75	118	127
hill	96	26	84	64	48	66	91
twoopt	109	16	99	51	40	118	126
anneal	135	88	136	131	108	136	160
tabu	146	67	131	69	56	88	107
ga	180	132	170	132	82	118	156
aco	176	114	167	133	105	106	176
restarts	108	43	103	33	24	71	109
total	1053	594	994	674	538	821	1052

The small numbers are sharing, not absence. Go's twoopt is 16 lines because the descent itself is the shared Descend of the localsearch file and twoopt only contributes the reversal move and the delta, and go's hill and restarts lean on the same helpers the same way. The C# tree keeps Lcg and the fixture data in the first file and references them everywhere, javascript imports the limb generator and the hill climb across stems, and restarts is thin in three trees because it reuses the climb instead of restating it. C and python restate the generator and matrix per file, 8 times each, which is why their totals run high, and lua restates the same way with the 1-based boundary shifts and the run.lua check rows added, its total one line under c. Java restates per stem too, the ring matrix in six files and the lcg in the five that draw, each generator one multiply-add on long with the >>> 32 output and no limb work, and java.util.Random never appears in the tree.

The corpus placement is honest silence: the icpc book's mined finals problems all yielded to exact algorithms, so no chapter there cites a metaheuristic, and this chapter's contest relevance is the negative space, the methods that run when the exact tool the icpc chapters teach does not finish. The nearest neighbors in this book are the pruning discipline of book 9, ch 20 (practical data structures and algorithms, pruning and branch and bound), the same determinism contract over a search tree, and the compact annealing prose of book 9, ch 36 (practical data structures and algorithms, numerical methods) that this chapter's section 4 replaces at full depth.

sources: E. Aarts and J. K. Lenstra, editors, "Local Search in Combinatorial Optimization", Wiley, 1997, the survey that named the frame of section 1. S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by Simulated Annealing", Science 220, 1983, sections 4 and 5. F. Glover, "Tabu Search: Part I", ORSA Journal on Computing 1, 1989, sections 6 and 7, tenure and aspiration as introduced. D. E. Goldberg, "Genetic Algorithms in Search, Optimization, and Machine Learning", Addison-Wesley, 1989, sections 8 and 9. M. Dorigo, V. Maniezzo, and A. Colorni, "Ant System: Optimization by a Colony of Cooperating Agents", IEEE Transactions on Systems, Man, and Cybernetics, 1996, sections 10 and 11, and T. Stutzle and H. Hoos, "MAX-MIN Ant System", Future Generation Computer Systems, 2000, the clamp of section 11. The generator constants follow D. Knuth, "The Art of Computer Programming", volume 2, seminumerical algorithms, the 64-bit congruential recommendations. Sample behavior verified by the seven suite gates scoped to chapter 41, zero skipped, all pins from the witness ledger of the wave brief. Java's gate runs the same pins under run-java-samples, 8 files and 359 checks.

42. approximation algorithms

An approximation algorithm is a contract with 2 clauses: return a feasible answer, and prove how far from the optimum it can sit. This chapter treats the contract as code. Every test in every tree asserts the 2 halves separately, the witness is checked against the problem's own feasibility rule and its value is divided into an exact optimum computed inside the same test, by brute force or by dynamic programming at teaching sizes. The chapter walks the proven ratios in order of machinery: the matching cover at 2, greedy set cover at ln n + 1, the double tree and christofides for metric tsp at 2 and 1.5, the knapsack ladder from a greedy 1/2 through a ptas to a full fptas, the bin packing heuristics, and the single-flip discipline of book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics) coming back with a guarantee attached. Two prose sections close it, the barriers that make some ratios impossible and the decision table that picks the right surrender for a given instance size.

42.1. np-hardness as an engineering boundary

Chapter 18 named the wall: some problems carry no known polynomial exact algorithm, and the theory says that is expected. The exact lanes of the corpus all pay exponentially for their certainty, the mask tables of chapter 17 and book 9, ch 19 (practical data structures and algorithms, advanced dynamic programming) grow as 2^n x n, the permutation oracle of the tsp fixtures walks (n - 1)! tours, and the branch and bound of book 9, ch 20 (practical data structures and algorithms, pruning and branch and bound) prunes an exponential tree with no word on when it stops. Past a few dozen vertices the honest question changes from what the best answer is to what can be proven about the answer inside the time left. That question has an engineering answer, and it starts with the exact optimum at small sizes, because a ratio with nothing to divide is just a number.

The dry run: the fixtures are the 2 exact lanes whose optima every ratio in this chapter divides, the 3-disjoint-edge vertex cover and the capacity 18 knapsack, asserted by all 6 suites with the oracles built in-test.

1. The cover lane holds nodes a through f with edges (a, b), (c, d), (e, f). Vertex subsets arrive in size order, and at k = 3 the subset a, c, e touches every edge, so the brute force answers 3.
2. Nothing smaller works: k = 2 covers at most 2 disjoint edges, the third edge stands uncovered, and the empty set misses everything.
3. The knapsack lane holds 10 items against capacity 18. The dp table fills by capacity descending and closes at 1300, items 5 and 8, weights 8 + 10, values 600 + 700.
4. The mask enumeration over all 1024 subsets agrees at 1300, the same oracle written the slow way.
5. Every one of these oracles is exponential in something: 64 vertex subsets, 1024 masks, and the 5040 tours of the 8 city fixture in section 5, which is exactly why they live in the test and not in the algorithm under test.

The oracles pin the denominators, cover 3 and knapsack 1300, and the listings below build the cover lane in 7 languages.

```c
// three disjoint edges on nodes 0..5
static const int E3[3][2] = {{0, 1}, {2, 3}, {4, 5}};

static int is_cover(const int *cover, int cover_n) {
  for (int e = 0; e < 3; e++) {
    int hit = 0;
    for (int v = 0; v < cover_n; v++)
      if (cover[v] == E3[e][0] || cover[v] == E3[e][1])
        hit = 1;
    if (!hit)
      return 0;
  }
  return 1;
}

static int min_vc_brute(int m) {
  for (int k = 0; k ≤ m; k++) {
    int c[6];
    for (int i = 0; i < k; i++)
      c[i] = i;
    do {
      if (is_cover(c, k))
        return k;
    } while (comb_next(c, k, m));
  }
  return m;
}
```

listing 1566: c, the 3 disjoint edges, the cover test, subsets in size order (source: dsa/samples-c/src/Ch42/ratio.c, lines 35-61)

```go
// MinVertexCoverBrute is the exact oracle at teaching sizes: vertex
// subsets by increasing size, first cover wins. Chapter graphs stay
// under a dozen vertices.
func MinVertexCoverBrute(edges [][2]int, n int) (int, []int) {
  for k := 0; k <= n; k++ {
    for mask := 0; mask < 1<<n; mask++ {
      if bits.OnesCount(uint(mask)) ≠ k {
        continue
      }
      cover := make([]bool, n)
      for v := 0; v < n; v++ {
        cover[v] = mask>>v&1 == 1
      }
      if Covers(edges, cover) {
        out := make([]int, 0, k)
        for v := 0; v < n; v++ {
          if cover[v] {
            out = append(out, v)
          }
        }
        return k, out
      }
    }
  }
  return n, nil // unreachable: the full vertex set covers
}
```

listing 1567: go, the mask oracle, k ascending, ones count filtering each size (source: dsa/samples-go/ch42/ratio.go, lines 17-42)

```
// three disjoint edges on nodes 0..5
static final int[][] E3 = {{0, 1}, {2, 3}, {4, 5}};

static boolean isCover(int[] cover) {
  for (int[] e : E3) {
    boolean hit = false;
    for (int v : cover)
      if (v == e[0] || v == e[1])
        hit = true;
    if (!hit)
      return false;
  }
  return true;
}

static int minVcBrute() {
  for (int k = 0; k ≤ 6; k++) {
    int[] c = new int[k];
    for (int i = 0; i < k; i++)
      c[i] = i;
    do {
      if (isCover(c))
        return k;
    } while (combNext(c, 6));
  }
  return 6;
}
```

listing 1568: java, the 3 disjoint edges, the cover test, subsets in size order (source: dsa/samples-java/src/Ch42/Ratio.java, lines 35-62)

```
  private static (int Size, int[] Cover)? Search(
      (int A, int B)[] edges, int n, int k, int start, List<int> cur)
  {
      if (cur.Count == k)
      {
          var inSet = new bool[n];
          foreach (int v in cur)
              inSet[v] = true;
          foreach (var (a, b) in edges)
          {
              if (!inSet[a] && !inSet[b])
                  return null;
          }
          return (k, cur.ToArray());
      }
      for (int v = start; v + (k - cur.Count) ≤ n; v++)
      {
          cur.Add(v);
          var hit = Search(edges, n, k, v + 1, cur);
          cur.RemoveAt(cur.Count - 1);
          if (hit ≠ null)
              return hit;
      }
      return null;
  }
```

listing 1569: c#, the subset search, size then lexicographic, first cover wins (source: dsa/samples/src/Ch42/Ratio.cs, lines 56-80)

```
export function bruteVertexCover(n, edges) {
  const covers = (s) => {
    const inSet = new Array(n).fill(false);
    for (const v of s) inSet[v] = true;
    return edges.every(([a, b]) => inSet[a] || inSet[b]);
  };
  for (let k = 0; k <= n; k++) {
    const cur = [];
    const rec = (start) => {
      if (cur.length === k) {
        if (covers(cur)) return [...cur];
        return null;
      }
      for (let v = start; v + (k - cur.length) <= n; v++) {
        cur.push(v);
        const hit = rec(v + 1);
        cur.pop();
        if (hit !== null) return hit;
      }
      return null;
    };
    const hit = rec(0);
    if (hit !== null) return { size: k, cover: hit };
  }
  return { size: n, cover: [] };
}
```

listing 1570: javascript, the recursive subset walk with the cover test inside (source: dsa/samples-js/src/ch42-ratio.mjs, lines 10-35)

```
def covers(edges, s):
    return all(a in s or b in s for a, b in edges)


def vc_opt(edges, nodes):
    for k in range(len(nodes) + 1):
        for comb in itertools.combinations(nodes, k):
            if covers(edges, set(comb)):
                return k
    return None


EDGES = [("a", "b"), ("c", "d"), ("e", "f")]
cover, matching = vc_matching(EDGES)
check(matching == EDGES, "on disjoint edges the greedy matching takes all three")
check(covers(EDGES, cover), "the cover is feasible, every edge covered")
check(len(cover) == 6, "its witness value is 6")
check(vc_opt(EDGES, "abcdef") == 3, "brute force over vertex subsets finds opt 3")
check(len(cover) == 2 * 3, "the ratio is exactly 2, the tight family")
check(
    len(cover) <= 2 * vc_opt(EDGES, "abcdef"), "the rho-approx contract holds at rho 2"
)
```

listing 1571: python, combinations as the oracle, the pinned cover 3 beside it (source: dsa/samples-py/src/Ch42/ratio.py, lines 32-53)

```lua
local function is_cover(edges, s)
  for _, e in ipairs(edges) do
    if not s[e[1]] and not s[e[2]] then return false end
  end
  return true
end

-- smallest k first, combinations in node order
local function min_vc_brute(edges, n)
  for k = 0, n do
    local comb = {}
    local function rec(depth, first)
      if depth == k then
        local s = {}
        for _, v in ipairs(comb) do s[v] = true end
        if is_cover(edges, s) then return #comb end
        return nil
      end
      for v = first, n do
        comb[depth + 1] = v
        local r = rec(depth + 1, v + 1)
        if r then return r end
      end
      return nil
    end
    local r = rec(0, 1)
    if r then return r end
  end
  error("unreachable")
end
```

listing 1572: lua, the same combinations recursion over 1-based nodes (source: dsa/samples-lua/ch42_ratio.lua, lines 11-40)

The same 2 numbers return in every section. The matching cover of section 3 doubles the 3, the greedy knapsack of section 7 lands 10 under the 1300, and the fptas of section 8 closes the 10 exactly. One ratio file serves this section and the next, the oracle half above and the contract half below.

42.2. the approximation ratio

A rho-approximation for a minimization problem returns a feasible answer of value at most rho x opt, and for a maximization problem a feasible answer of value at least opt / rho, with rho at least 1 in both directions. The ratio is a worst case contract over every instance, and an instance family is tight when the algorithm sits at rho x opt on it forever, which is the difference between a bound that holds and a bound that is the whole story. In the tests the ratio is never computed as a division: the minimization check reads alg less than or equal rho x opt and the maximization check reads alg x rho greater than or equal opt, cross-multiplied integers, so no floating point value ever sits inside an assert.

The dry run: the fixtures are the tight family and the near miss, asserted by all 6 suites as feasibility first and ratio second.

1. On the 3 disjoint edges the greedy maximal matching takes all 3 edges, the cover is both endpoints of each, 6 nodes, and the oracle says 3. Feasibility holds, every edge covered, and 6 = 2 x 3, the ratio is exactly 2, tight.
2. On the knapsack the density greedy of section 7 fills items for 1290 against the oracle 1300. Feasibility holds, the fill weighs exactly the capacity 18, and 2 x 1290 exceeds 1300, so the greedy clears its rho = 2 contract with room.

3. The maximization check for that lane reads value x 2 greater than or equal opt, the cross-multiplied form, and the better of the fill and the best single item is what the guarantee needs: the fill 1290 already beats the best single item 700.
4. Tightness is a property of the family, never of one instance: k disjoint edges push the matching cover to 2k against an optimum of k at every k.

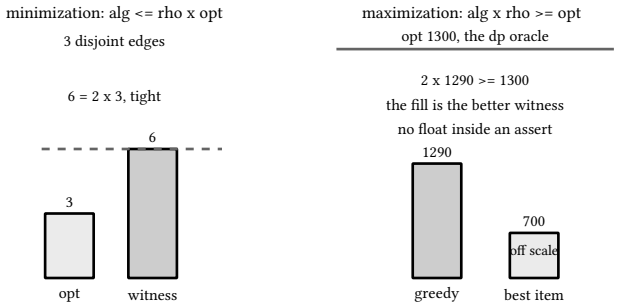


figure 376: the two ratio shapes: the cover witness sitting on exactly twice its optimum, the knapsack greedy one step under its optimum

The 6 over 3 and the 1290 under 1300 are the 2 pinned shapes, and the listings below carry the exact lanes and the contract checks in 7 languages.

```c
// greedy maximal matching in input edge order, cover = both endpoints
static int matching_cover(int *cover) {
  int covered[6] = {0}, cn = 0, matched = 0;
  for (int e = 0; e < 3; e++) {
    int a = E3[e][0], b = E3[e][1];
    if (!covered[a] && !covered[b]) {
      covered[a] = covered[b] = 1;
      cover[cn++] = a;
      cover[cn++] = b;
      matched++;
    }
  }
  return matched;
}

int main(void) {
  // exact lane 1: the greedy cover on the disjoint edges is feasible and
  // has size 6, the brute force minimum is 3
  int cover[6];
  CHECK(matching_cover(cover) == 3);
  CHECK(is_cover(cover, 6));
  CHECK(min_vc_brute(6) == 3);

  // rho-approximation for minimization: alg ≤ rho * opt, and here the
  // bound is tight, alg = 2 * opt exactly
  int alg = 6, opt = 3;
  CHECK(alg ≤ 2 * opt);
  CHECK(alg == 2 * opt);
```

listing 1573: c, the matching cover, feasibility, then 6 against 2 x 3 exactly (source: dsa/samples-c/src/Ch42/ratio.c, lines 63-90)

```go
// KnapsackDP is the exact lane for the maximization ratios: 0/1
// knapsack by capacity, integral weights and values.
func KnapsackDP(weights, values []int, cap int) int {
  dp := make([]int, cap+1)
  for i := range weights {
    for c := cap; c ≥ weights[i]; c-- {
      if v := dp[c-weights[i]] + values[i]; v > dp[c] {
        dp[c] = v
      }
    }
  }
  return dp[cap]
}
```

listing 1574: go, the dp knapsack lane, 1300 in 13 lines (source: dsa/samples-go/ch42/ratio.go, lines 44-56)

```java
  static int coverCount;

  // greedy maximal matching in input edge order, cover = both endpoints
  static int matchingCover() {
    boolean[] covered = new boolean[6];
    int[] cover = new int[6];
    int cn = 0, matched = 0;
    for (int[] e : E3) {
      if (!covered[e[0]] && !covered[e[1]]) {
        covered[e[0]] = true;
        covered[e[1]] = true;
        cover[cn++] = e[0];
        cover[cn++] = e[1];
        matched++;
      }
    }
    coverCount = cn;
    return matched;
  }

  public static void main(String[] args) {
    // exact lane 1: the greedy cover on the disjoint edges is feasible
    // and has size 6, the brute force minimum is 3
    ok(matchingCover() == 3, "the greedy matching takes all 3 edges");
    ok(coverCount == 6 && isCover(new int[] {0, 1, 2, 3, 4, 5}),
        "the cover of both endpoints is feasible at 6");
    ok(minVcBrute() == 3, "the brute minimum cover is 3");

    // rho-approximation for minimization: alg ≤ rho * opt, and here
    // the bound is tight, alg = 2 * opt exactly
    int alg = 6, opt = 3;
    ok(alg ≤ 2 * opt, "alg ≤ 2 * opt holds");
    ok(alg == 2 * opt, "the bound is tight at exactly 2x");
```

listing 1575: java, the matching cover, feasibility, then 6 against 2 x 3 exactly (source: dsa/samples-java/src/Ch42/Ratio.java, lines 63-95)

```
    public static int BruteKnap(int[] w, int[] v, int cap)
    {
        int n = w.Length;
        int best = 0;
        for (int m = 0; m < 1 << n; m++)
        {
            int tw = 0;
            int tv = 0;
            for (int i = 0; i < n; i++)
            {
                if ((m >> i & 1) == 1)
                {
                    tw += w[i];
                    tv += v[i];
                }
            }
            if (tw <= cap && tv > best)
                best = tv;
        }
        return best;
    }

    public static bool WithinMin(int apx, int opt, int rho) => apx <= rho * opt;

    /// <summary>apx at least (num / den) * opt, cross-multiplied so the
    /// ratio never becomes a float.</summary>
    public static bool WithinMax(int apx, int opt, int num, int den) =>
        (long)apx * den >= (long)num * opt;
```

listing 1576: c#, the mask knapsack oracle, withinMin, the cross-multiplied withinMax (source: dsa/samples/src/Ch42/Ratio.cs, lines 27-54)

```javascript
export function bruteKnap(w, v, cap) {
  const n = w.length;
  let best = 0;
  for (let m = 0; m < 1 << n; m++) {
    let tw = 0;
    let tv = 0;
    for (let i = 0; i < n; i++) {
      if ((m >> i) & 1) {
        tw += w[i];
        tv += v[i];
      }
    }
    if (tw <= cap && tv > best) best = tv;
  }
  return best;
}

export function withinMin(apx, opt, rho) {
  return apx <= rho * opt;
}

export function withinMax(apx, opt, num, den) {
  // apx >= (num / den) * opt, cross-multiplied: apx * den >= num * opt
  return apx * den >= num * opt;
}
```

listing 1577: javascript, the mask oracle with the 2 contract predicates (source: dsa/samples-js/src/ch42-ratio.mjs, lines 37-61)

```python
GW = [7, 4, 9, 3, 5, 8, 2, 6, 10, 3]
GV = [420, 210, 630, 170, 350, 600, 100, 380, 700, 240]
CAP = 18


def knap_dp(w, v, cap):
    dp = [0] * (cap + 1)
    for i in range(len(w)):
        for c in range(cap, w[i] - 1, -1):
            if dp[c - w[i]] + v[i] > dp[c]:
                dp[c] = dp[c - w[i]] + v[i]
    return dp[cap]


def knap_brute(w, v, cap):
    best = 0
    for mask in range(1 << len(w)):
        tw = sum(w[i] for i in range(len(w)) if mask >> i & 1)
        if tw <= cap:
            best = max(best, sum(v[i] for i in range(len(w)) if mask >> i & 1))
    return best


check(knap_dp(GW, GV, CAP) == 1300, "the knapsack dp optimum is 1300")
check(knap_brute(GW, GV, CAP) == 1300, "brute force over 1024 subsets agrees")
```

listing 1578: python, dp and mask oracles side by side, both landing 1300 (source: dsa/samples-py/src/Ch42/ratio.py, lines 55-79)

```lua
local GW = {7, 4, 9, 3, 5, 8, 2, 6, 10, 3}
local GV = {420, 210, 630, 170, 350, 600, 100, 380, 700, 240}
local CAP = 18

local function dp_opt()
  local dp = {}
  for c = 0, CAP do dp[c] = 0 end
  for i = 1, 10 do
    for c = CAP, GW[i], -1 do
      local cand = dp[c - GW[i]] + GV[i]
      if cand > dp[c] then dp[c] = cand end
    end
  end
  return dp[CAP]
end

-- density order by cross-multiplied products, v_i * w_j versus v_j * w_i,
-- ties by lower index, then the best-single-item refinement
local function greedy_density()
  local order = {}
  for i = 1, 10 do order[i] = i end
  table.sort(order, function(i, j)
    if GV[i] * GW[j] ~= GV[j] * GW[i] then
      return GV[i] * GW[j] > GV[j] * GW[i]
    end
    return i < j
  end)
  local tw, tv, picks = 0, 0, {}
  for _, i in ipairs(order) do
    if tw + GW[i] ≤ CAP then
      tw = tw + GW[i]
      tv = tv + GV[i]
      picks[#picks + 1] = i
    end
  end
  local bv, bi = 0, nil
  for i = 1, 10 do
    if GW[i] ≤ CAP and GV[i] > bv then bv, bi = GV[i], i end
  end
  if bv > tv then return bv, {bi} end
  return tv, picks
end
```

listing 1579: lua, the dp optimum then the density greedy at 1290, cross-multiplied (source: dsa/samples-lua/ch42_ratio.lua, lines 42-83)

The triangle already says something kind: on 3 nodes wired as a cycle the matching cover is optimal, the ratio is 1, and the reason is structural, a matching in a triangle never exceeds one edge. The next section collects the whole fixture family and separates the instances where the cover is lucky from the family where it is exactly as bad as the theorem allows.

42.3. vertex cover by maximal matching

Scan the edges in input order, take an edge when both endpoints are still unmatched, and put both endpoints into the cover. The taken edges form a maximal matching, maximal in the sense that no remaining edge can extend it, and the cover is feasible exactly because of that: an uncovered edge would have 2 free endpoints and the scan would have taken it. The cost argument counts. Any vertex cover must touch every edge of a matching, and

matching edges share no endpoints, so the optimum is at least the matching size, and the returned cover is exactly 2 matching edges' worth of vertices, 2 per edge, hence at most twice the optimum.

The dry run: the fixtures are the 4 pinned graphs, asserted by all 6 suites with feasibility, the brute oracle, and the ratio as separate checks.

1. 3 disjoint edges: the matching takes all 3, the cover is 6, the oracle says 3, ratio exactly 2, the tight family.
2. The triangle a, b, c: the matching takes the first edge only, the cover a, b has size 2, the oracle says 2, ratio 1. A matching in a triangle never exceeds one edge, every 2 of the 3 nodes cover it.
3. The path a, b, c, d with the middle edge first, (b, c), (a, b), (c, d): the greedy takes the middle edge, both endpoints now matched, and stops at 1 edge. The cover b, c has size 2 and equals the optimum, ratio 1, while a maximum matching of the same path holds 2 edges. Maximal is not maximum.
4. The same path in input order, (a, b), (b, c), (c, d): the greedy takes a, b then c, d, 2 edges, the cover is all 4 nodes against an optimum of 2, ratio 2 again. Edge order alone doubled the answer.

fixture	matching	cover	opt	ratio
3 disjoint edges	3	6	3	2, tight
triangle	1	2	2	1
path, middle first	1	2	2	1
path, input order	2	4	2	2

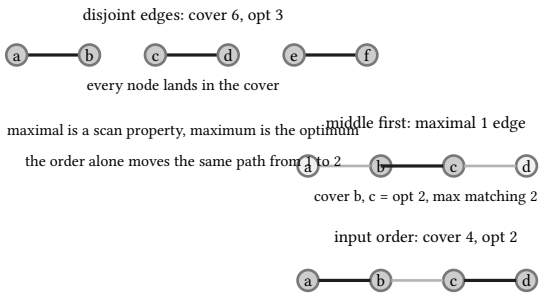


figure 377: the matching cover on the 2 tight shapes against the lucky ones: disjoint edges taking every node, the middle-first path stopping at 1 edge, the same path in input order taking all 4

The table and the figure agree with every suite, and the listings below run the scan in 7 languages.

```c
int main(void) {
  for (int t = 0; t < 4; t++) {
    const Case *f = &CASES[t];
    int covered[6] = {0}, cover[8], cn = 0, matched = 0;
    for (int e = 0; e < f->en; e++) {
      int a = f->edges[e][0], b = f->edges[e][1];
      if (!covered[a] && !covered[b]) {
        covered[a] = covered[b] = 1;
        cover[cn++] = a;
        cover[cn++] = b;
        matched++;
      }
    }
    CHECK(matched == f->want_matched);
    CHECK(cn == f->want_cover);
    CHECK(is_cover(f, cover, cn));
    CHECK(min_vc_brute(f) == f->want_opt);
    CHECK(cn <= 2 * f->want_opt);
    if (t == 0)
      CHECK(cn == 2 * f->want_opt);
    if (t == 2)
      CHECK(cover[0] == 1 && cover[1] == 2);
  }
```

listing 1580: c, the scan over the 4 pinned cases, feasibility, oracle, ratio (source: dsa/samples-c/src/Ch42/vc.c, lines 68-90)

```go
// VertexCoverMatching builds a greedy maximal matching in input edge
// order and returns the cover of both endpoints of every matched
// edge. Maximal is not maximum: the edge order alone decides how far
// the cover overshoots the optimum, and twice the matching size is
// always at most twice the optimum.
func VertexCoverMatching(edges [][2]int) (cover []bool, matched [][2]int) {
  n := 0
  for _, e := range edges {
    n = max(n, e[0]+1, e[1]+1)
  }
  cover = make([]bool, n)
  for _, e := range edges {
    if !cover[e[0]] && !cover[e[1]] {
      cover[e[0]], cover[e[1]] = true, true
      matched = append(matched, e)
    }
  }
  return cover, matched
}
```

listing 1581: go, the whole engine, cover as a bool slice, matched edges beside it (source: dsa/samples-go/ch42/vc.go, lines 3-21)

```
    for (int t = 0; t < 4; t++) {
      Case f = CASES[t];
      boolean[] covered = new boolean[6];
      int[] cover = new int[8];
      int cn = 0, matched = 0;
      for (int[] e : f.edges()) {
        if (!covered[e[0]] && !covered[e[1]]) {
          covered[e[0]] = true;
          covered[e[1]] = true;
          cover[cn++] = e[0];
          cover[cn++] = e[1];
          matched++;
        }
      }
      ok(matched == f.wantMatched(), "case " + t + " matches " + f.wantMatched());
      ok(cn == f.wantCover(), "case " + t + " covers at " + f.wantCover());
      ok(isCover(f, java.util.Arrays.copyOf(cover, cn)),
          "case " + t + " cover is feasible");
      ok(minVcBrute(f) == f.wantOpt(), "case " + t + " brute optimum pins");
      ok(cn ≤ 2 * f.wantOpt(), "case " + t + " holds the 2x bound");
      if (t == 0)
        ok(cn == 2 * f.wantOpt(), "the disjoint edges hit the bound exactly");
      if (t == 2)
        ok(cover[0] == 1 && cover[1] == 2, "the middle-first order pins the cover");
    }
```

listing 1582: java, the scan over the 4 pinned cases, feasibility, oracle, ratio (source: dsa/samples-java/src/Ch42/Vc.java, lines 66-90)

```
    public static ((int A, int B)[] Matching, int[] Cover) Matching(
        int n, (int A, int B)[] edges)
    {
        var covered = new bool[n];
        var matching = new List<(int, int)>();
        var cover = new List<int>();
        foreach (var (a, b) in edges)
        {
            if (!covered[a] && !covered[b])
            {
                covered[a] = true;
                covered[b] = true;
                matching.Add((a, b));
                cover.Add(a);
                cover.Add(b);
            }
        }
        var sorted = cover.ToArray();
        Array.Sort(sorted);
        return (matching.ToArray(), sorted);
    }
```

listing 1583: c#, the greedy maximal matching returning cover and matching (source: dsa/samples/src/Ch42/Vc.cs, lines 14-34)

```javascript
export function vcMatching(n, edges) {
  const covered = new Array(n).fill(false);
  const cover = [];
  const matching = [];
  for (const [a, b] of edges) {
    if (!covered[a] && !covered[b]) {
      covered[a] = true;
      covered[b] = true;
      matching.push([a, b]);
      cover.push(a, b);
    }
  }
  cover.sort((x, y) => x - y);
  return { cover, matching };
}

export function isCover(n, edges, subset) {
  const inSet = new Array(n).fill(false);
  for (const v of subset) inSet[v] = true;
  return edges.every(([a, b]) => inSet[a] || inSet[b]);
}
```

listing 1584: javascript, the scan and the cover test, first-fit edge order (source: dsa/samples-js/src/ch42-vc.mjs, lines 8-28)

```python
def vc_matching(edges):
    # input edge order drives the greedy maximal matching
    covered = set()
    cover = set()
    matching = []
    for a, b in edges:
        if a not in covered and b not in covered:
            covered.update((a, b))
            matching.append((a, b))
            cover.update((a, b))
    return cover, matching


def covers(edges, s):
    return all(a in s or b in s for a, b in edges)


def vc_opt(edges, nodes):
    for k in range(len(nodes) + 1):
        for comb in itertools.combinations(nodes, k):
            if covers(edges, set(comb)):
                return k
    return None
```

listing 1585: python, the matching, the cover test, the combinations oracle (source: dsa/samples-py/src/Ch42/vc.py, lines 18-41)

```
local function vc_matching(edges)
  local covered, cover, matched = {}, {}, {}
  for _, e in ipairs(edges) do -- input order drives the matching
    if not covered[e[1]] and not covered[e[2]] then
      covered[e[1]], covered[e[2]] = true, true
      matched[#matched + 1] = e
      cover[#cover + 1] = e[1]
      cover[#cover + 1] = e[2]
    end
  end
  return cover, matched
end

local function min_vc_brute(edges, n)
  for k = 0, n do
    local comb = {}
    local function rec(depth, first)
      if depth == k then
        local s = {}
        for _, v in ipairs(comb) do s[v] = true end
        if is_cover(edges, s) then return #comb, {table.unpack(comb)} end
        return nil
      end
      for v = first, n do
        comb[depth + 1] = v
        local r, w = rec(depth + 1, v + 1)
        if r then return r, w end
      end
      return nil
    end
    local r, w = rec(0, 1)
    if r then return r, w end
  end
  error("unreachable")
end
```

listing 1586: lua, the scan and the brute oracle, plus the maximum-matching contrast (source: dsa/samples-lua/ch42_vc.lua, lines 15-49)

The correction worth recording: the 2 bound is not tight on disjoint triangles, there the ratio is 1 because a triangle caps its own matching at one edge and 2 endpoints cover it. The tight families are k disjoint edges and the in-order path, both shown above, and the second one matters more than the first because it needs only 4 nodes and an unlucky order. The maximum matching counterweight is its own lane in the python and lua suites, the middle-first path pinning maximal 1 against maximum 2 there, while the other 4 trees name the contrast and pin the maximal side alone. The same word pair returns in section 10 where the locally best partition is not the best partition.

42.4. set cover by greedy ln n

The ground set has 15 elements, the family has 6 sets, and greedy takes the set covering the most uncovered ground at every step, ties to the earliest set in input order. The pinned family is built to make the harmonic bound visible: a chain G1 through G4 of shrinking sets, 8, 4, 2, 1 elements, eats the ground from the front while 2 wide sets A and B wait behind it, and A with B cover everything in 2 picks.

The dry run: the fixture is the telescoping family, asserted by all 6 suites with the per-step coverage table, the union of the picks as feasibility, and the brute-force optimum over family subsets.

1. Step 1, 15 uncovered: G1 covers 8, G2 4, G3 2, G4 1, A 8, B 7. G1 and A tie at 8 and input order gives G1 the pick.
2. Step 2, 7 uncovered: G2 covers 4, A 3, B 4, the rest less. G2 and B tie at 4 and G2 wins the same way.
3. Step 3, 3 uncovered: G3 covers 2, B 2, A 1. The tie goes to G3.
4. Step 4, 1 uncovered: G4 covers 1, A 1. The tie goes to G4, the chain is complete, 4 sets.
5. The oracle walks family subsets in size order and answers 2, the pair A and B, whose union is the whole ground. The union of the 4 greedy picks is also the whole ground, feasibility holds, and 4 is exactly 2 x 2, ratio 2 at n = 15.

step	left	G1	G2	G3	G4	A	B	pick
1	15	8	4	2	1	8	7	G1 over A, tie
2	7	0	4	2	1	3	4	G2 over B, tie
3	3	0	0	2	1	1	2	G3 over B, tie
4	1	0	0	0	1	1	0	G4 over A, tie

PITFALL: the tie break is part of the algorithm

Every tie in the table above resolves by input order, and the pinned chain exists only because G1 arrives before A. A stable ordering is not a cosmetic choice here: an unstable sort in the family scan moves the first pick to A, the second to B, and greedy answers 2 on the instance built to cost it 4. The tests pin the whole pick sequence, the count alone would let a moved tie pass, and the full pin fails loudly in all 6 trees at once.

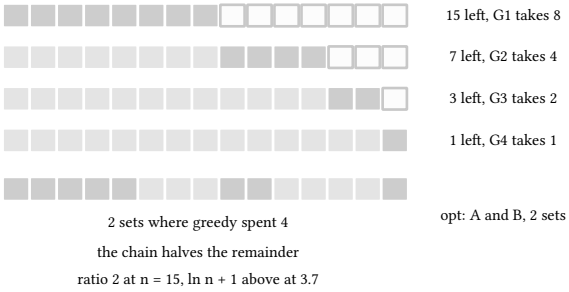


figure 378: the greedy telescoping walk over the 15 element ground, one row per step, the newly covered cells shaded, the two waiting 8-sets covering everything in 2 picks below

The 4 against 2 is the pinned landing, and the listings below run the greedy loop and the subset oracle in 7 languages.

```c
  // greedy: max new coverage per step, first-in-family wins ties. the
  // pinned chain picks G1, G2, G3, G4 with coverages 8, 4, 2, 1
  uint16_t unc = GROUND;
  int picks[8], pn = 0;
  static const int want_cov[4] = {8, 4, 2, 1};
  while (unc) {
    int bi = -1, bc = 0;
    for (int i = 0; i < 6; i++) {
      int c = popcnt((uint16_t)(FAM[i] & unc));
      if (c > bc) {
        bc = c;
        bi = i;
      }
    }
    CHECK(bi ≥ 0);
    CHECK(bc == want_cov[pn]);
    picks[pn++] = bi;
    unc = (uint16_t)(unc & ~FAM[bi]);
  }
  CHECK(pn == 4);
  static const int want_picks[4] = {0, 1, 2, 3};
  for (int i = 0; i < 4; i++)
    CHECK(picks[i] == want_picks[i]);
```

listing 1587: c, the greedy loop over bitmasks, the pinned coverages 8, 4, 2, 1 (source: dsa/samples-c/src/Ch42/setcover.c, lines 82-104)

```go
func GreedySetCover(ground []int, family [][]int) []int {
  masks := SetCoverMasks(ground, family)
  uncovered := uint32(1)<<len(ground) - 1
  var picks []int
  for uncovered ≠ 0 {
    best, bestGain := -1, -1
    for si, m := range masks {
      if gain := bits.OnesCount32(m & uncovered); gain > bestGain {
        best, bestGain = si, gain
      }
    }
    if bestGain <= 0 {
      return nil // an element sits in no set
    }
    picks = append(picks, best)
    uncovered &^= masks[best]
  }
  return picks
}
```

listing 1588: go, coverage counts as popcounts over the uncovered mask (source: dsa/samples-go/ch42/setcover.go, lines 24-42)

```
    // greedy: max new coverage per step, first-in-family wins ties. the
    // pinned chain picks G1, G2, G3, G4 with coverages 8, 4, 2, 1
    int unc = GROUND;
    int[] picks = new int[8];
    int pickCount = 0;
    int[] wantCov = {8, 4, 2, 1};
    while (unc != 0) {
      int bi = -1, bc = 0;
      for (int i = 0; i < 6; i++) {
        int c = popcnt(FAM[i] & unc);
        if (c > bc) {
          bc = c;
          bi = i;
        }
      }
      ok(bi >= 0, "a best set always exists while uncovered ground remains");
      ok(bc == wantCov[pickCount], "pick " + pickCount + " covers " + wantCov[pickCount]);
      picks[pickCount++] = bi;
      unc = unc & ~FAM[bi];
    }
    ok(pickCount == 4, "greedy takes 4 sets");
    int[] wantPicks = {0, 1, 2, 3};
    for (int i = 0; i < 4; i++)
      ok(picks[i] == wantPicks[i], "pick " + i + " is G" + (i + 1));
```

listing 1589: java, the greedy loop over bitmasks, the pinned coverages 8, 4, 2, 1 (source: dsa/samples-java/src/Ch42/Setcover.java, lines 76-99)

```
    public static (int[] Picks, (int Chosen, int[] Counts, int Remaining)[] Steps) Greedy(
        int groundN, int[][] family)
    {
        var uncovered = new HashSet<int>();
        for (int e = 1; e ≤ groundN; e++)
            uncovered.Add(e);
        var picks = new List<int>();
        var steps = new List<(int, int[], int)>();
        while (uncovered.Count > 0)
        {
            int best = -1;
            int bestCov = 0;
            for (int s = 0; s < family.Length; s++)
            {
                int c = 0;
                foreach (int e in family[s])
                {
                    if (uncovered.Contains(e))
                        c++;
                }
                if (c > bestCov)
                {
                    bestCov = c;
                    best = s;
                }
            }
            if (bestCov == 0)
                throw new InvalidOperationException("family cannot cover the ground");
            var counts = new int[family.Length];
            for (int s = 0; s < family.Length; s++)
            {
                foreach (int e in family[s])
                {
                    if (uncovered.Contains(e))
                        counts[s]++;
                }
            }
            steps.Add((best, counts, uncovered.Count));
            picks.Add(best);
            foreach (int e in family[best])
                uncovered.Remove(e);
        }
        return (picks.ToArray(), steps.ToArray());
    }
```

listing 1590: c#, greedy with the per-step coverage table recorded beside the picks (source: dsa/samples/src/Ch42/Setcover.cs, lines 13-56)

```javascript
export function greedySetCover(groundN, family) {
  const uncovered = new Set();
  for (let e = 1; e ≤ groundN; e++) uncovered.add(e);
  const picks = [];
  const steps = [];
  while (uncovered.size > 0) {
    let best = -1;
    let bestCov = 0;
    for (let s = 0; s < family.length; s++) {
      let c = 0;
      for (const e of family[s]) {
        if (uncovered.has(e)) c += 1;
      }
      if (c > bestCov) {
        bestCov = c;
        best = s;
      }
    }
    if (bestCov === 0) return null;
    const counts = family.map((f) => f.filter((e) => uncovered.has(e)).length);
    steps.push({ chosen: best, counts, remaining: uncovered.size });
    picks.push(best);
    for (const e of family[best]) uncovered.delete(e);
  }
  return { picks, steps };
}
```

listing 1591: javascript, the same greedy, steps and counts returned together (source: dsa/samples-js/src/ch42-setcover.mjs, lines 8-33)

```python
def greedy(ground, sets):
    # max new coverage per step, the earliest set in input order wins
    # ties, tracked through the per-step coverage table
    unc = set(ground)
    picks = []
    table = []
    while unc:
        covs = [len(s & unc) for s in sets]
        table.append(covs)
        best = max(range(len(sets)), key=lambda i: covs[i])
        if covs[best] == 0:
            return None, table
        picks.append(best)
        unc -= sets[best]
    return picks, table


def opt_cover(ground, sets):
    for k in range(1, len(sets) + 1):
        for comb in itertools.combinations(range(len(sets)), k):
            union = set()
            for i in comb:
                union |= sets[i]
            if union == set(ground):
                return k, comb
    return None
```

listing 1592: python, the greedy and the combinations oracle landing 4 against 2 (source: dsa/samples-py/src/Ch42/setcover.py, lines 29-54)

```lua
local function greedy(uncovered)
  local names = {}
  local remaining = 0
  for v = 1, 15 do
    if uncovered[v] then remaining = remaining + 1 end
  end
  while remaining > 0 do
    local bi, best = nil, -1
    for i, f in ipairs(FAMILY) do
      local c = coverage(f[2], uncovered)
      if c > best then bi, best = i, c end
    end
    if best ≤ 0 then error("family cannot cover the ground") end
    for _, v in ipairs(FAMILY[bi][2]) do
      if uncovered[v] then
        uncovered[v] = nil
        remaining = remaining - 1
      end
    end
    names[#names + 1] = FAMILY[bi][1]
  end
  return names
end
```

listing 1593: lua, the same loop, ties to the earliest set in family order (source: dsa/samples-lua/ch42_setcover.lua, lines 26-48)

Why the bound has a logarithm in it: after any greedy pick the uncovered count r shrinks by at least r / opt, because the opt solution's sets partition the remaining r elements among opt sets and the widest of them covers at least r / opt, and greedy takes at least the widest. Shrinking by that factor for k steps leaves n x (1 - 1/opt)^k, which is under n over e to the k over opt, and that drops below 1 once k passes opt x ln n. Counting the last partial pick gives the textbook bound, greedy uses at most ln n + 1 times the optimum, here 3.7 against the observed 2. The bound is nearly the truth in both directions: Slavik's tight analysis pins the greedy worst case at exactly ln n - ln ln n + theta(1), and the families reaching it are his construction, prose here because the fixtures above only reach ratio 2. Section 11 returns to the other direction, that doing meaningfully better than the logarithm would collapse complexity classes.

42.5. metric tsp by double-tree

Give the tsp a metric, symmetric distances that obey the triangle inequality, and 2 proven facts fall out. Deleting an edge from an optimal tour leaves a spanning tree, so the minimum spanning tree weighs at most the optimum. Doubling every tree edge makes all degrees even, and a connected graph with all even degrees carries an euler circuit, so a closed walk of weight exactly 2 x mst exists and costs at most 2 x opt. Shortcutting that walk, keeping the first visit of every vertex, never lengthens it under the triangle inequality, because a jump across a skipped stretch costs at most the stretch itself. The implementation is direct: prim with the scan order pinned, tree vertices ascending then candidates ascending, strict improvement so the lowest index pair wins ties, hierholzer from vertex 0 always taking the first unused arc, then the shortcut.

The dry run: the fixtures are the 3 metric instances, asserted by all 6 suites with the mst, the permutation check, the recomputed tour length, and the ratio against the permutation oracle. The euler walk weights 28, 32, and 88 are pinned by 5 of the 6 trees, and c carries them as comments beside the pinned lengths.

1. The ring, 8 cities on a 4 by 4 perimeter: mst 14, a path through all 8, doubled walk 28, euler circuit 15 visits long, shortcut back to the ring order, tour 16, the oracle itself, ratio 1.0.
2. The convex hex: mst 16, walk 32, and the shortcut tour reads 0, 1, 2, 3, 5, 4 for length 24 against the oracle 20, ratio 1.2 exactly, 5 x 24 = 6 x 20. This is the non-trivial pin.

3. The hex walk itself reads 0, 1, 2, 3, 2, 1, 0, 5, 4, 5, 0: the doubled tree walks out to vertex 3, backtracks 3, 2, 1, 0, then crosses to 5 and bounces 5, 4, 5. The shortcut jumps 3 straight to 5, paying 6 for the 14 the backtrack spent, and the bounce costs the same either way, 32 down to 24.
4. The two cluster instance, 2 tight 2 by 2 blocks 20 apart: mst 44, a path inside each block joined by one long edge, walk 88, shortcut tour 0, 1, 3, 4, 5, 7, 6, 2 for length 84, the oracle itself, ratio 1.0.

instance	mst	walk	tour length	opt	ratio
ring8	14	28	16	16	1.0
hex	16	32	24	20	1.2
clusters	44	88	84	84	1.0

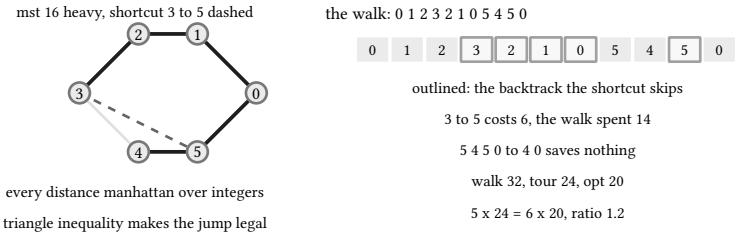


figure 379: the hex double tree: the mst heavy, the euler walk with its backtrack, the 3 to 5 shortcut chord, the arithmetic 32 against 24

The ring collapses to its own optimum, the hex pays 1.2, and the listings below double the tree and walk it in 7 languages.

```c
typedef struct {
  int nbr, eid;
} Half;

// doubled multigraph: each mst edge appears twice with its own edge id
static void build_adj(int nn, const int edges[7][3], Half adj[8][20],
                      int adjn[8]) {
  for (int v = 0; v < nn; v++)
    adjn[v] = 0;
  for (int k = 0; k < nn - 1; k++) {
    int a = edges[k][1], b = edges[k][2];
    adj[a][adjn[a]].nbr = b;
    adj[a][adjn[a]++].eid = 2 * k;
    adj[a][adjn[a]].nbr = b;
    adj[a][adjn[a]++].eid = 2 * k + 1;
    adj[b][adjn[b]].nbr = a;
    adj[b][adjn[b]++].eid = 2 * k;
    adj[b][adjn[b]].nbr = a;
    adj[b][adjn[b]++].eid = 2 * k + 1;
  }
}

// hierholzer from vertex 0, then shortcut by first occurrence
static int euler_shortcut(const Half adj[8][20], const int adjn[8], int *tour) {
  int stack[40], sp = 0, out[40], on = 0;
  char used[16] = {0}, seen[8] = {0};
  stack[sp++] = 0;
  while (sp > 0) {
    int v = stack[sp - 1], advanced = 0;
    for (int k = 0; k < adjn[v]; k++)
      if (!used[adj[v][k].eid]) {
        used[adj[v][k].eid] = 1;
        stack[sp++] = adj[v][k].nbr;
        advanced = 1;
        break;
      }
    if (!advanced)
      out[on++] = stack[--sp];
  }
  int tn = 0;
  for (int i = on - 1; i ≥ 0; i--)
    if (!seen[out[i]]) {
      seen[out[i]] = 1;
      tour[tn++] = out[i];
    }
  return tn;
}
```

listing 1594: c, the doubled multigraph with parallel edge ids, hierholzer, the shortcut (source: dsa/samples-c/src/Ch42/tsp2x.c, lines 111-157)

```go
type graphArc struct {
  to, id int
}

// EulerTour walks the multigraph from vertex 0 by Hierholzer, always
// taking the first unused arc in adjacency order, and returns the
// Euler circuit.
func EulerTour(n int, edges [][2]int) []int {
  adj := make([][]graphArc, n)
  for id, e := range edges {
    adj[e[0]] = append(adj[e[0]], graphArc{e[1], id})
    adj[e[1]] = append(adj[e[1]], graphArc{e[0], id})
  }
  used := make([]bool, len(edges))
  stack := []int{0}
  var out []int
  for len(stack) > 0 {
    v := stack[len(stack)-1]
    moved := false
    for _, arc := range adj[v] {
      if !used[arc.id] {
        used[arc.id] = true
        stack = append(stack, arc.to)
        moved = true
        break
      }
    }
    if !moved {
      out = append(out, v)
      stack = stack[:len(stack)-1]
    }
  }
  slices.Reverse(out)
  return out
}

// Shortcut drops repeats from a closed walk. The triangle inequality
// is what makes the shortcut tour no longer than the walk.
func Shortcut(walk []int, n int) []int {
  seen := make([]bool, n)
  tour := make([]int, 0, n)
  for _, v := range walk {
    if !seen[v] {
      seen[v] = true
      tour = append(tour, v)
    }
  }
  return tour
}
```

listing 1595: go, the arc list euler tour and the shortcut over first visits (source: dsa/samples-go/ch42/tsp2x.go, lines 55-103)

```java
// doubled multigraph: each mst edge appears twice with its own edge id
static int[][][] adj = new int[8][20][];
static int[] adjn = new int[8];

static void buildAdj(int nn, int[][] edges) {
  for (int v = 0; v < nn; v++)
    adjn[v] = 0;
  for (int k = 0; k < nn - 1; k++) {
    int a = edges[k][1], b = edges[k][2];
    adj[a][adjn[a]++] = new int[] {b, 2 * k};
    adj[a][adjn[a]++] = new int[] {b, 2 * k + 1};
    adj[b][adjn[b]++] = new int[] {a, 2 * k};
    adj[b][adjn[b]++] = new int[] {a, 2 * k + 1};
  }
}

// hierholzer from vertex 0, then shortcut by first occurrence
static int eulerShortcut(int[] tour) {
  int[] stack = new int[40], out = new int[40];
  int sp = 0, on = 0;
  boolean[] used = new boolean[16], seen = new boolean[8];
  stack[sp++] = 0;
  while (sp > 0) {
    int v = stack[sp - 1];
    boolean advanced = false;
    for (int k = 0; k < adjn[v]; k++)
      if (!used[adj[v][k][1]]) {
        used[adj[v][k][1]] = true;
        stack[sp++] = adj[v][k][0];
        advanced = true;
        break;
      }
    if (!advanced)
      out[on++] = stack[--sp];
  }
  int tn = 0;
  for (int i = on - 1; i ≥ 0; i--)
    if (!seen[out[i]]) {
      seen[out[i]] = true;
      tour[tn++] = out[i];
    }
  return tn;
}
```

listing 1596: java, the doubled multigraph with parallel edge ids, hierholzer, the shortcut (source: dsa/samples-java/src/Ch42/Tsp2x.java, lines 111-153)

```
    public static int[] EulerWalk(List<(int V, int Id)>[] adj)
    {
        var stack = new Stack<int>();
        stack.Push(0);
        var outs = new List<int>();
        var used = new HashSet<int>();
        while (stack.Count > 0)
        {
            int v = stack.Peek();
            bool advanced = false;
            foreach (var (nbr, eid) in adj[v])
            {
                if (!used.Contains(eid))
                {
                    used.Add(eid);
                    stack.Push(nbr);
                    advanced = true;
                    break;
                }
            }
            if (!advanced)
                outs.Add(stack.Pop());
        }
        outs.Reverse();
        return outs.ToArray();
    }

    public static int[] Shortcut(int[] walk)
    {
        int n = 0;
        foreach (int v in walk)
            n = Math.Max(n, v + 1);
        var seen = new bool[n];
        var t = new List<int>();
        foreach (int v in walk)
        {
            if (!seen[v])
            {
                seen[v] = true;
                t.Add(v);
            }
        }
        return t.ToArray();
    }
```

listing 1597: c#, the euler walk on a stack of arcs, the shortcut keeping first visits (source: dsa/samples/src/Ch42/Tsp2x.cs, lines 123-166)

```
export function eulerWalk(adj) {
  const stack = [0];
  const out = [];
  const used = new Set();
  while (stack.length > 0) {
    const v = stack[stack.length - 1];
    let advanced = false;
    for (const [nbr, eid] of adj[v]) {
      if (!used.has(eid)) {
        used.add(eid);
        stack.push(nbr);
        advanced = true;
        break;
      }
    }
    if (!advanced) out.push(stack.pop());
  }
  out.reverse();
  return out;
}

export function shortcut(walk) {
  const seen = new Set();
  const t = [];
  for (const v of walk) {
    if (!seen.has(v)) {
      seen.add(v);
      t.push(v);
    }
  }
  return t;
}

export function buildAdj(n, edges) {
  // edges: [a, b] pairs, each entry doubled with its own parallel id
  const adj = Array.from({ length: n }, () => []);
  edges.forEach(([a, b], eid) => {
    adj[a].push([b, eid]);
    adj[b].push([a, eid]);
    adj[a].push([b, edges.length + eid]);
    adj[b].push([a, edges.length + eid]);
  });
  return adj;
}
```

listing 1598: javascript, the walk, the shortcut, the adjacency build with doubled ids (source: dsa/samples-js/src/ch42-tsp2x.mjs, lines 80-123)

```
def euler(adj):
    # hierholzer from vertex 0 over the multigraph
    stack = [0]
    out = []
    used = set()
    while stack:
        v = stack[-1]
        moved = False
        for nbr, eid in adj[v]:
            if eid not in used:
                used.add(eid)
                stack.append(nbr)
                moved = True
                break
        if not moved:
            out.append(stack.pop())
    return out[::-1]


def shortcut(walk):
    seen = set()
    tour = []
    for v in walk:
        if v not in seen:
            seen.add(v)
            tour.append(v)
    return tour


def double_tree(d):
    n = len(d)
    edges = mst(d)
    adj = [[] for _ in range(n)]
    for eid, (w, a, b) in enumerate(edges):
        # each tree edge twice, its own parallel id
        adj[a].append((b, eid))
        adj[b].append((a, eid))
        adj[a].append((b, n + eid))
        adj[b].append((a, n + eid))
    walk = euler(adj)
    tour = shortcut(walk)
    w = sum(e[0] for e in edges)
    return w, 2 * w, walk, tour, tour_len(d, tour)
```

listing 1599: python, euler by stack, shortcut, the double tree assembly (source: dsa/samples-py/src/Ch42/tsp2x.py, lines 59-101)

```
local function euler_walk(adj)
  local stack = {0}
  local out = {}
  local used = {}
  while #stack > 0 do
    local v = stack[#stack]
    local advanced = false
    for _, e in ipairs(adj[v]) do
      if not used[e[2]] then
        used[e[2]] = true
        stack[#stack + 1] = e[1]
        advanced = true
        break
      end
    end
    if not advanced then
      out[#out + 1] = stack[#stack]
      stack[#stack] = nil
    end
  end
  for i = 1, #out // 2 do
    out[i], out[#out - i + 1] = out[#out - i + 1], out[i]
  end
  return out
end

local function shortcut(walk)
  local seen, t = {}, {}
  for _, v in ipairs(walk) do
    if not seen[v] then
      seen[v] = true
      t[#t + 1] = v
    end
  end
  return t
end

local function double_tree(d, n)
  local mst = mst_prim(d, n)
  local w = 0
  for _, e in ipairs(mst) do w = w + e[1] end
  local adj = {}
  for v = 0, n - 1 do adj[v] = {} end
  for eid, e in ipairs(mst) do
    adj[e[2]][#adj[e[2]] + 1] = {e[3], eid}
    adj[e[3]][#adj[e[3]] + 1] = {e[2], eid}
    adj[e[2]][#adj[e[2]] + 1] = {e[3], 100 + eid}
    adj[e[3]][#adj[e[3]] + 1] = {e[2], 100 + eid}
  end
  local t = shortcut(euler_walk(adj))
  return w, 2 * w, t, tour_len(d, t)
end
```

listing 1600: lua, the same walk and shortcut, the tour length recomputed (source: dsa/samples-lua/ch42_tsp2x.lua, lines 66-117)

The ring and the hex come from book 9, ch 41 (practical data structures and algorithms, local search and meta-heuristics) on purpose: chapter 41 searches that metric pair with 2-opt descents, tabu tenure, and ant cycles, and the same point lists and manhattan matrices return here so one instance family carries both stories, the heuristic

that improves with no promise attached and the algorithm that promises 1.5 on its first pass. The clusters instance joins from the same manhattan shelf, 2 dense blocks 20 apart, its odd set straddling the divide so section 6 has a crossing to price. Reading the 2 chapters against the same hex makes the difference concrete: chapter 41's 2-opt descent reaches 20 by a chain of improving reversals, the double tree reaches 24 on the first pass and stops, and section 6 buys the missing 4 back with a matching.

42.6. christofides 1.5x

The double tree wastes half its budget fixing degrees it broke itself. Christofides fixes only the ones that need fixing: find the odd degree vertices of the mst, which are always an even number because degrees sum to twice the edge count, add a minimum weight perfect matching on exactly that set, and the union is connected with all even degrees. Its euler circuit weighs mst plus matching, the shortcut costs no more, and the ratio is 1.5. The matching half of the argument: walk an optimal tour and shortcut it to a cycle on just the odd vertices, legal by the triangle inequality and no heavier than the tour, then that even cycle's edges alternate into 2 perfect matchings, so the cheaper of the 2 costs at most half the cycle, and the minimum matching costs no more than that. Mst at most opt plus matching at most opt over 2 is 1.5 x opt.

Computing that matching is a blossom algorithm in general, and the prose defers to the matching chapters for the real machinery. At teaching sizes the honest implementation is exhaustive: the odd set is at most 6 vertices in these fixtures, permutations of it pair up as consecutive elements, the lexicographically first minimum wins, and 720 orderings are nothing.

The dry run: the fixtures are the same 3 metric instances, asserted by all 6 suites with the odd set, the matching weight, the tour, and the ratio as 2 x length less than or equal 3 x opt.

1. The ring: mst 14, a path with endpoints 6 and 7 odd, matching weight 2 on the pair, euler over tree plus matching, shortcut back to the ring order, tour 16, the oracle, ratio 1.0.
2. The hex: mst 16, odd set 3 and 4, matching weight 4, tour the hull order 0, 1, 2, 3, 4, 5 at 20, the oracle, ratio 1.0. The double tree's 1.2 came from one bad shortcut, and the matching removed the degree that caused it.
3. The clusters: mst 44, odd set 2, 4, 6, 7, one vertex of the near block and 3 of the far block's 4 corners. The minimum matching pairs 2 with 4 across the divide for 39 and 6 with 7 inside the far block for 1, weight 40, and the shortcut tour reads 0, 1, 3, 4, 5, 7, 6, 2 at 84, the oracle, ratio 1.0.
4. The euler graph check rides along: the walk uses exactly n - 1 tree edges plus the pairs, 7 plus 2 arcs of matching here, and the python suite counts that arc total while the other 5 trees assert the walk's outcomes.

instance	mst	odd set	matching	tour length	opt
ring8	14	{6, 7}	2	16	16
hex	16	{3, 4}	4	20	20
clusters	44	{2, 4, 6, 7}	40	84	84

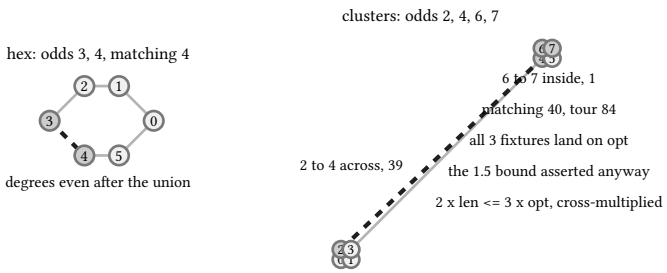


figure 380: the odd set matchings: the hex pair 3 and 4 closed for 4, the cluster instance crossing the divide for 39 and closing inside the far block for 1

All 3 fixtures land on their optima here, which is the honest reading of 1.5 at these sizes, and the listings below build the matching and the union in 7 languages.

```c
// minimum perfect matching on the odd set, first permutation wins ties
static int min_matching(const int d[8][8], const int *odds, int on,
                        int pairs[3][2]) {
  int perm[6], best = 0x7fffffff, bp[6];
  for (int i = 0; i < on; i++)
    perm[i] = odds[i];
  do {
    int w = 0;
    for (int i = 0; i < on; i += 2)
      w += d[perm[i]][perm[i + 1]];
    if (w < best) {
      best = w;
      for (int i = 0; i < on; i++)
        bp[i] = perm[i];
    }
  } while (perm_next(perm, on));
  for (int i = 0; i < on; i += 2) {
    pairs[i / 2][0] = bp[i];
    pairs[i / 2][1] = bp[i + 1];
  }
  return best;
}
```

listing 1601: c, the minimum perfect matching over odd-set permutations (source: dsa/samples-c/src/Ch42/christofides.c, lines 109-130)

```go
func MinPerfectMatching(d [][]int, odds []int) (weight int, pairs [][2]int) {
  rem := slices.Clone(odds)
  slices.Sort(rem)
  best := -1
  var bestPairs [][2]int
  var pick func(rem []int, acc int, pairs [][2]int)
  pick = func(rem []int, acc int, pairs [][2]int) {
    if len(rem) == 0 {
      if best < 0 || acc < best {
        best, bestPairs = acc, pairs
      }
      return
    }
    a := rem[0]
    for k := 1; k < len(rem); k++ {
      next := make([]int, 0, len(rem)-2)
      for j := 1; j < len(rem); j++ {
        if j != k {
          next = append(next, rem[j])
        }
      }
      np := make([][2]int, len(pairs)+1)
      copy(np, pairs)
      np[len(pairs)] = [2]int{a, rem[k]}
      pick(next, acc+d[a][rem[k]], np)
    }
  }
  pick(rem, 0, nil)
  return best, bestPairs
}
```

listing 1602: go, fix the smallest odd vertex, try every partner ascending (source: dsa/samples-go/ch42/christofides.go, lines 12-41)

```
  static int minMatching(int[][] d, int[] odds, int[][] pairs) {
    int on = odds.length;
    int[] perm = new int[6], bp = new int[6];
    int best = Integer.MAX_VALUE;
    for (int i = 0; i < on; i++)
      perm[i] = odds[i];
    do {
      int w = 0;
      for (int i = 0; i < on; i += 2)
        w += d[perm[i]][perm[i + 1]];
      if (w < best) {
        best = w;
        System.arraycopy(perm, 0, bp, 0, on);
      }
    } while (permNext(perm));
    for (int i = 0; i < on; i += 2) {
      pairs[i / 2][0] = bp[i];
      pairs[i / 2][1] = bp[i + 1];
    }
    return best;
  }
```

listing 1603: java, the minimum perfect matching over odd-set permutations (source: dsa/samples-java/src/Ch42/Christofides.java, lines 109-130)

```
    public static (int Weight, (int A, int B)[] Pairs) MinPerfectMatching(
        int[][] d, int[] odds)
    {
        var o = (int[])odds.Clone();
        Array.Sort(o);
        int m = o.Length;
        int bestW = int.MaxValue;
        var bestPairs = Array.Empty<(int, int)>();
        var perm = new int[m];
        var used = new bool[m];
        void Rec(int depth)
        {
            if (depth == m)
            {
                int w = 0;
                for (int i = 0; i + 1 < m; i += 2)
                    w += d[perm[i]][perm[i + 1]];
                if (w < bestW)
                {
                    bestW = w;
                    var pairs = new (int, int)[m / 2];
                    for (int i = 0; i + 1 < m; i += 2)
                        pairs[i / 2] = (perm[i], perm[i + 1]);
                    bestPairs = pairs;
                }
                return;
            }
            for (int i = 0; i < m; i++)
            {
                if (used[i])
                    continue;
                used[i] = true;
                perm[depth] = o[i];
                Rec(depth + 1);
                used[i] = false;
            }
        }
        Rec(0);
        return (bestW, bestPairs);
    }
```

listing 1604: c#, the matching recursion, pairs off consecutive elements of the permutation (source: dsa/samples/src/Ch42/Christofides.cs, lines 16-55)

```
export function minPerfectMatching(d, odds) {
  const o = [...odds].sort((x, y) => x - y);
  const m = o.length;
  let best = null;
  const perm = new Array(m);
  const used = new Array(m).fill(false);
  const rec = (depth) => {
    if (depth === m) {
      const pairs = [];
      for (let i = 0; i < m; i += 2) pairs.push([perm[i], perm[i + 1]]);
      let w = 0;
      for (const [a, b] of pairs) w += d[a][b];
      if (best === null || w < best.weight) {
        best = { weight: w, pairs: pairs.map((p) => [...p]) };
      }
      return;
    }
    for (let i = 0; i < m; i++) {
      if (used[i]) continue;
      used[i] = true;
      perm[depth] = o[i];
      rec(depth + 1);
      used[i] = false;
    }
  };
  rec(0);
  return best;
}
```

listing 1605: javascript, the same permutation minimum, first strict winner (source: dsa/samples-js/src/ch42-christofides.mjs, lines 11-38)

```
def mst(d):
    n = len(d)
    inside = [0]
    outside = list(range(1, n))
    edges = []
    while outside:
        best = None
        for a in sorted(inside):
            for b in outside:
                if best is None or d[a][b] < best[0]:
                    best = (d[a][b], a, b)
        w, a, b = best
        edges.append((w, a, b))
        inside.append(b)
        outside.remove(b)
    return edges


def min_perfect_matching(d, odds):
    # pair up the sorted odd set along every permutation, keep the
    # first minimum, exactly what |odds| ≤ 6 affords
    best = None
    for perm in itertools.permutations(sorted(odds)):
        pairs = [(perm[i], perm[i + 1]) for i in range(0, len(perm), 2)]
        w = sum(d[a][b] for a, b in pairs)
        if best is None or w < best[0]:
            best = (w, pairs)
    return best
```

listing 1606: python, prim again, the itertools permutations matching (source: dsa/samples-py/src/Ch42/christofides.py, lines 39-66)

```
local function min_matching(d, odds)
  local m = #odds
  local used, perm = {}, {}
  local best, bpairs
  local function rec(depth)
    if depth > m then
      local w = 0
      local pairs = {}
      for i = 1, m, 2 do
        pairs[#pairs + 1] = {perm[i], perm[i + 1]}
        w = w + d[perm[i]][perm[i + 1]]
      end
      if best == nil or w < best then
        best, bpairs = w, pairs
      end
      return
    end
    for _, v in ipairs(odds) do
      if not used[v] then
        used[v] = true
        perm[depth] = v
        rec(depth + 1)
        used[v] = nil
      end
    end
  end
  rec(1)
  return best, bpairs
end
```

listing 1607: lua, the matching recursion over the odd set, its own euler walk and shortcut (source: dsa/samples-lua/ch42_christofides.lua, lines 66-94)

The tightness of 1.5 is real and stays prose: families exist where christofides returns exactly 1.5 x opt, improving the constant stood open for 4 decades, and the only known improvement, from 2020, shaves a vanishingly small margin off it. What the fixtures teach instead is the mechanism, that the matching erases exactly the parity the double tree created, and on the hex that recovers the full 1.2 to 1.0.

42.7. knapsack greedy 1/2 and the ptas

The greedy lane first: sort items by value density, value over weight, and fill the sack in that order. Densities are compared as cross-multiplied products, v_i x w_j against v_j x w_i, ties by lower index, so the ordering is an exact rational comparison with no float anywhere in the decision. The plain fill can be terrible on its own, so the algorithm keeps the better of the fill and the best single item, and that refinement is what the 1/2 needs: the fill plus the first item it rejected together outweigh the optimum, one of the 2 must hold at least half of it, and the returned witness is at least that one.

The dry run: the fixture is the capacity 18 knapsack against the dp oracle 1300. All 6 suites assert the cross-multiplied bound, the density order array is pinned by the c#, javascript, and python trees, the picks by every tree but c, and c pins the values and the k = 2 seed mask.

1. The density order pins as 9, 5, 2, 4, 8, 7, 0, 3, 1, 6, the 80 of item 9, the 75 of item 5, and the tied 70s by index 2, 4, 8.
2. The fill takes 9 at weight 3, 5 at weight 8, skips 2 and 8, takes 4 at weight 5, and closes with 6 at weight 2: weights 3 + 8 + 5 + 2 = 18 exactly, values 240 + 600 + 350 + 100 = 1290.

3. The best single item is 8 at value 700, the fill wins, and the greedy answers 1290 with 2 x 1290 greater than or equal 1300.
4. The ptas enumerates every k-subset of items that fits and density-fills the remainder over the unused items. At k = 1 it is the greedy plus the single-item refinement, 1290. At k = 2 the pair 5 and 8, weights 8 + 10 = 18, values 600 + 700, is among the enumerated seeds and the answer is the optimum 1300. At k = 3 the answer falls back to 1290: every fitting 3-subset leaves a remainder whose fill scores at most 1290.
5. Monotonicity in k fails at the instance level, 1290, 1300, 1290, and that is the teaching row. The guarantee is asymptotic, at least 1 - 1 over (k + 1) of the optimum in the worst case, and a larger k buys the bound, never a monotone score.

k	value	vs opt	what the enumeration found
1	1290	0.992	the greedy fill, items 9, 5, 4, 6
2	1300	1.0	the seed pair 5 and 8, weight 18 exactly
3	1290	0.992	no fitting triple seeds anything better

The 1290 under 1300 and the k = 2 recovery are pinned in every tree, and the listings below build the density order, the fill, and the enumeration in 7 languages.

```c
// density order: v_i / w_i descending, ties by lower index
static void density_order(const int *w, const int *v, int m, int *order) {
  for (int i = 0; i < m; i++)
    order[i] = i;
  for (int i = 1; i < m; i++) {
    int x = order[i], j = i - 1;
    while (j ≥ 0 &&
           (long long)v[x] * w[order[j]] > (long long)v[order[j]] * w[x]) {
      order[j + 1] = order[j];
      j--;
    }
    order[j + 1] = x;
  }
}

static int greedy_fill(int subset, int *tv_out) {
  int order[10], tw = 0, tv = 0;
  // the fill continues from the subset's weight, the capacity is shared
  for (int i = 0; i < 10; i++)
    if ((subset >> i) & 1)
      tw += GW[i];
  density_order(GW, GV, 10, order);
  for (int k = 0; k < 10; k++) {
    int i = order[k];
    if ((subset >> i) & 1)
      continue;
    if (tw + GW[i] ≤ CAP) {
      tw += GW[i];
      tv += GV[i];
    }
  }
  *tv_out = tv;
  return tw;
}
```

listing 1608: c, the cross-multiplied insertion sort, the fill from a seeded subset (source: dsa/samples-c/src/Ch42/ptas.c, lines 44-77)

```go
func densityOrder(weights, values []int) []int {
  idx := make([]int, len(weights))
  for i := range idx {
    idx[i] = i
  }
  slices.SortFunc(idx, func(a, b int) int {
    left := values[a] * weights[b]
    right := values[b] * weights[a]
    if left ≠ right {
      if left > right {
        return -1
      }
      return 1
    }
    return a - b
  })
  return idx
}

// KnapsackGreedy fills by density and keeps the better of the fill
// and the best single item (value ties to the later index), the
// 1/2-approximation for maximization knapsack.
func KnapsackGreedy(weights, values []int, cap int) (int, []int) {
  tw, tv := 0, 0
  var picks []int
  for _, i := range densityOrder(weights, values) {
    if tw+weights[i] <= cap {
      tw += weights[i]
      tv += values[i]
      picks = append(picks, i)
    }
  }
  bi := -1
  for i := range weights {
    if weights[i] > cap {
      continue
    }
    if bi < 0 || values[i] ≥ values[bi] {
      bi = i
    }
  }
  if bi ≥ 0 && values[bi] > tv {
    return values[bi], []int{bi}
  }
  return tv, picks
}
```

listing 1609: go, the exact comparator and the greedy with its refinement (source: dsa/samples-go/ch42/ptas.go, lines 12-57)

```
  // density order: v_i / w_i descending, ties by lower index
  static void densityOrder(int[] w, int[] v, int[] order) {
    int m = w.length;
    for (int i = 0; i < m; i++)
      order[i] = i;
    for (int i = 1; i < m; i++) {
      int x = order[i], j = i - 1;
      while (j ≥ 0 && (long) v[x] * w[order[j]] > (long) v[order[j]] * w[x]) {
        order[j + 1] = order[j];
        j--;
      }
      order[j + 1] = x;
    }
  }

  static int fillVOut;

  static int greedyFill(int subset) {
    int[] order = new int[10];
    int tw = 0, tv = 0;
    // the fill continues from the subset's weight, the capacity is shared
    for (int i = 0; i < 10; i++)
      if ((subset >> i & 1) ≠ 0)
        tw += GW[i];
    densityOrder(GW, GV, order);
    for (int k = 0; k < 10; k++) {
      int i = order[k];
      if ((subset >> i & 1) ≠ 0)
        continue;
      if (tw + GW[i] ≤ CAP) {
        tw += GW[i];
        tv += GV[i];
      }
    }
    fillVOut = tv;
    return tw;
  }
```

listing 1610: java, the cross-multiplied insertion sort in long, the fill from a seeded subset (source: dsa/samples-java/src/Ch42/Ptas.java, lines 44-80)

```
    public static int[] DensityOrder(int[] w, int[] v, HashSet<int>? exclude = null)
    {
        var idx = new List<int>();
        for (int i = 0; i < w.Length; i++)
        {
            if (exclude == null || !exclude.Contains(i))
                idx.Add(i);
        }
        var a = idx.ToArray();
        for (int i = 1; i < a.Length; i++) // insertion sort: stable ties
        {
            int key = a[i];
            int j = i - 1;
            while (j ≥ 0 && (long)v[key] * w[a[j]] > (long)v[a[j]] * w[key])
            {
                a[j + 1] = a[j];
                j--;
            }
            a[j + 1] = key;
        }
        return a;
    }

    public static (int Value, int[] Picks) GreedyDensity(int[] w, int[] v, int cap)
    {
        int tw = 0;
        int tv = 0;
        var picks = new List<int>();
        foreach (int i in DensityOrder(w, v))
        {
            if (tw + w[i] ≤ cap)
            {
                tw += w[i];
                tv += v[i];
                picks.Add(i);
            }
        }
        int bestSingle = -1;
        int bestIdx = -1;
        for (int i = 0; i < w.Length; i++)
        {
            if (w[i] ≤ cap && v[i] ≥ bestSingle)
            {
                bestSingle = v[i];
                bestIdx = i;
            }
        }
        if (bestSingle > tv)
            return (bestSingle, new[] { bestIdx });
        return (tv, picks.ToArray());
    }
```

listing 1611: c#, density order by insertion, the fill, the best-single refinement (source: dsa/samples/src/Ch42/Ptas.cs, lines 19-69)

```
export function densityOrder(w, v, exclude) {
  const idx = [];
  for (let i = 0; i < w.length; i++) {
    if (!exclude || !exclude.has(i)) idx.push(i);
  }
  return idx.sort((a, b) => v[b] * w[a] - v[a] * w[b]);
}

export function greedyDensity(w, v, cap) {
  let tw = 0;
  let tv = 0;
  const picks = [];
  for (const i of densityOrder(w, v)) {
    if (tw + w[i] ≤ cap) {
      tw += w[i];
      tv += v[i];
      picks.push(i);
    }
  }
  let bestSingle = -1;
  let bestIdx = -1;
  for (let i = 0; i < w.length; i++) {
    if (w[i] ≤ cap && v[i] ≥ bestSingle) {
      bestSingle = v[i];
      bestIdx = i;
    }
  }
  if (bestSingle > tv) return { value: bestSingle, picks: [bestIdx] };
  return { value: tv, picks };
}
```

listing 1612: javascript, the comparator as cross-multiplied products, the fill (source: dsa/samples-js/src/ch42-ptas.mjs, lines 13-42)

```
def density_order(w, v):
    # v/w descending, compared as v_i * w_j versus v_j * w_i so no
    # float ever enters the decision, ties keep the lower index
    rest = list(range(len(w)))
    order = []
    while rest:
        best = rest[0]
        for i in rest[1:]:
            if v[i] * w[best] > v[best] * w[i]:
                best = i
        rest.remove(best)
        order.append(best)
    return order


check(
    density_order(GW, GV) == [9, 5, 2, 4, 8, 7, 0, 3, 1, 6],
    "the density order pins, ties by lower index",
)


def fill(order, w, v, cap, taken=()):
    # greedily add still-fitting items in the given order
    skip = set(taken)
    tw = tv = 0
    picks = []
    for i in order:
        if i not in skip and tw + w[i] ≤ cap:
            tw += w[i]
            tv += v[i]
            picks.append(i)
    return tv, picks


order = density_order(GW, GV)
tv, picks = fill(order, GW, GV, CAP)
check(picks == [9, 5, 4, 6], "the density fill takes 9, 5, 4, 6")
check(tv == 1290, "for value 1290")
check(2 * tv ≥ 1300, "density greedy with the best-single refinement stays above half")
check(max(GV) ≤ tv, "the fill already beats every single item here")
```

listing 1613: python, the selection-order density walk, the fill, the pinned 1290 (source: dsa/samples-py/src/Ch42/ptas.py, lines 36-75)

```lua
local function by_density(items)
  local order = {table.unpack(items)}
  table.sort(order, function(i, j)
    if GV[i] * GW[j] ~= GV[j] * GW[i] then
      return GV[i] * GW[j] > GV[j] * GW[i]
    end
    return i < j
  end)
  return order
end

local function greedy_density()
  local tw, tv, picks = 0, 0, {}
  for _, i in ipairs(by_density({1, 2, 3, 4, 5, 6, 7, 8, 9, 10})) do
    if tw + GW[i] ≤ CAP then
      tw = tw + GW[i]
      tv = tv + GV[i]
      picks[#picks + 1] = i
    end
  end
  local bv, bi = 0, nil
  for i = 1, 10 do
    if GW[i] ≤ CAP and GV[i] > bv then bv, bi = GV[i], i end
  end
  if bv > tv then return bv, {bi} end
  return tv, picks
end
```

listing 1614: lua, the sort comparator and the greedy, picks as 1-based indices (source: dsa/samples-lua/ch42_ptas.lua, lines 26-52)

One ptas file serves this section and the next, the greedy and the enumeration above, the scaled table below. The enumeration cost is n^k subsets times a linear fill, polynomial for every fixed k and rising with it, which is the whole meaning of a ptas: pick the accuracy first, pay a polynomial whose degree depends on it.

42.8. fptas by profit scaling

The ptas pays n^k for its accuracy. The fptas pays n squared over eps. Scale every profit v_i down to v_i / k with k = max(1, eps x vmax / n), rounded to integers toward zero, run the minimum weight dynamic program over scaled profits, dp2 at s holding the least weight reaching scaled profit s, answer the largest s whose weight fits, and report the true value of the reconstructed set. The arithmetic of the bound is 3 lines: each item loses under k in scaling, n items lose under n x k together, n x k is at most eps x vmax, and vmax is at most the optimum, so the scaled problem still contains a set worth at least 1 - eps of the optimum, and the program finds the best scaled one. The running time is n times the scaled profit sum, which is n squared vmax over k, which is n cubed over eps: polynomial in n and in 1 over eps together, the definition of an fptas. Epsilon travels as the integer pair num over den, so the bound check, value x den at least (den - num) x opt, stays integer too.

The dry run: the fixtures are the big instance at 4 epsilons and the near-tie instance at the same 4, asserted by all 6 suites with k, the value, feasibility, and the bound per eps.

1. On the big instance every eps is exact: eps 1 scales by k = 70, eps 1/2 by 35, eps 1/4 by 17, eps 1/10 by 7, and all 4 runs return items 5 and 8 at 1300 with weight 18. The bound value x den at least (den - num) x 1300 holds everywhere, with room.
2. The near-tie instance holds 4 items of weights 3, 3, 2, 2 and values 25, 25, 24, 24 against capacity 6. The optimum is 50, the 2 heavy items together.
3. At eps 1, k = 6: 25 / 6 and 24 / 6 both round to 4, the scaled problem cannot tell the heavy pair from the light pair, and the program returns the light pair, true value 48. At eps 1/2, k = 3: both round to 8 again, 48 once more.

4. At eps 1/4, k = 1: the profits survive untouched, the answer is the heavy pair, 50. At eps 1/10 the floor lands at 0 and the clamp holds k at 1, 50 again.
5. The error is visible exactly when the bound allows it: 48 against 50 is 0.96, above 1 - eps at eps 1 and eps 1/2, and the suite asserts the bound at every step, the value only where it pins.

eps	big k	big value	near-tie k	near-tie value	bound
1	70	1300	6	48	48 >= 0 x 50
1/2	35	1300	3	48	96 >= 50
1/4	17	1300	1	50	200 >= 150
1/10	7	1300	1	50	500 >= 450

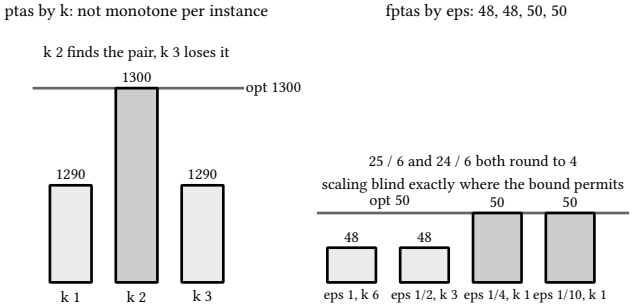


figure 381: the 2 error ladders: the ptas k values 1290, 1300, 1290 against the optimum line, the fptas eps values 48, 48, 50, 50 closing to it

The ladders are the pinned values of both instances, and the listings below run the scaled table in 7 languages.

```c
// fptas: v'_i = v_i / k with k = max(1, floor(eps * vmax / n)), eps as the
// fraction num / den so every step stays integer
static int fptas(const int *w, const int *v, int m, int cap, int num, int den,
                 int *k_out, int *items, int *items_n) {
  int vmax = 0;
  for (int i = 0; i < m; i++)
    if (v[i] > vmax)
      vmax = v[i];
  int k = (num * vmax) / (den * m);
  if (k < 1)
    k = 1;
  *k_out = k;
  static int dp2[MAXS + 1];
  static unsigned char take[10][MAXS + 1];
  int sv[10], svmax = 0;
  for (int i = 0; i < m; i++) {
    sv[i] = v[i] / k;
    svmax += sv[i];
  }
  for (int s = 0; s <= svmax; s++) {
    dp2[s] = s == 0 ? 0 : INF;
    for (int i = 0; i < m; i++)
      take[i][s] = 0;
  }
  for (int i = 0; i < m; i++)
    for (int s = svmax; s >= sv[i]; s--)
      if (dp2[s - sv[i]] + w[i] < dp2[s]) {
        dp2[s] = dp2[s - sv[i]] + w[i];
        take[i][s] = 1;
      }
  int best_s = 0;
  for (int s = svmax; s >= 0; s--)
    if (dp2[s] <= cap) {
      best_s = s;
      break;
    }
  int cnt = 0, s = best_s, val = 0, wt = 0;
  for (int i = m - 1; i >= 0; i--)
    if (take[i][s]) {
      items[cnt++] = i;
      s -= sv[i];
      val += v[i];
      wt += w[i];
    }
  *items_n = cnt;
  CHECK(wt <= cap);
  return val;
}
```

listing 1615: c, the fptas whole: k from the eps pair, scaled profits, dp2, rebuild (source: dsa/samples-c/src/Ch42/ptas.c, lines 110-157)

```
  k = epsNum * vmax / (epsDen * n)
  if k < 1 {
    k = 1
  }
  sv := make([]int, n)
  svmax := 0
  for i := range values {
    sv[i] = values[i] / k
    svmax += sv[i]
  }
  const inf = math.MaxInt / 4
  dp2 := make([]int, svmax+1)
  for s := range dp2 {
    dp2[s] = inf
  }
  dp2[0] = 0
  take := make([][]bool, n)
  for i := range take {
    take[i] = make([]bool, svmax+1)
  }
  for i := 0; i < n; i++ {
    for s := svmax; s ≥ sv[i]; s-- {
      if dp2[s-sv[i]] < inf && dp2[s-sv[i]]+weights[i] < dp2[s] {
        dp2[s] = dp2[s-sv[i]] + weights[i]
        take[i][s] = true
      }
    }
  }
  scaledS = 0
  for s := svmax; s ≥ 0; s-- {
    if dp2[s] <= cap {
      scaledS = s
      break
    }
  }
  items = []int{}
```

listing 1616: go, the scale factor, the fill, the largest feasible s (source: dsa/samples-go/ch42/ptas.go, lines 125-160)

```
  // fptas: v'_i = v_i / k with k = max(1, floor(eps * vmax / n)), eps
  // as the fraction num / den so every step stays integer
  static int fptas(int[] w, int[] v, int cap, int num, int den) {
    int m = w.length;
    int vmax = 0;
    for (int i = 0; i < m; i++)
      if (v[i] > vmax)
        vmax = v[i];
    int k = (num * vmax) / (den * m);
    if (k < 1)
      k = 1;
    kOut = k;
    int[] sv = new int[m];
    int svmax = 0;
    for (int i = 0; i < m; i++) {
      sv[i] = v[i] / k;
      svmax += sv[i];
    }
    for (int s = 0; s ≤ svmax; s++) {
      dp2[s] = s == 0 ? 0 : INF;
      for (int i = 0; i < m; i++)
        take[i][s] = false;
    }
    for (int i = 0; i < m; i++)
      for (int s = svmax; s ≥ sv[i]; s--)
        if (dp2[s - sv[i]] + w[i] < dp2[s]) {
          dp2[s] = dp2[s - sv[i]] + w[i];
          take[i][s] = true;
        }
    int bestS = 0;
    for (int s = svmax; s ≥ 0; s--)
      if (dp2[s] ≤ cap) {
        bestS = s;
        break;
      }
    int cnt = 0, s = bestS, val = 0, wt = 0;
    for (int i = m - 1; i ≥ 0; i--)
      if (take[i][s]) {
        items[cnt++] = i;
        s -= sv[i];
        val += v[i];
        wt += w[i];
      }
    itemsN = cnt;
    ok(wt ≤ cap, "the recovered knapsack respects the capacity");
    return val;
  }
```

listing 1617: java, the fptas whole: k from the eps pair as num over den, scaled profits, dp2, the rebuild (source: dsa/samples-java/src/Ch42/Ptas.java, lines 118-164)

```
        const int Inf = int.MaxValue / 2; // unreachable weight sentinel
        var dp2 = new int[svmax + 1];
        Array.Fill(dp2, Inf);
        dp2[0] = 0;
        var take = new bool[n][];
        for (int i = 0; i < n; i++)
            take[i] = new bool[svmax + 1];
        for (int i = 0; i < n; i++)
        {
            for (int s = svmax; s ≥ sv[i]; s--)
            {
                if (dp2[s - sv[i]] + w[i] < dp2[s])
                {
                    dp2[s] = dp2[s - sv[i]] + w[i];
                    take[i][s] = true;
                }
            }
        }
        int bestS = 0;
        for (int s = svmax; s ≥ 0; s--)
        {
            if (dp2[s] ≤ cap)
            {
                bestS = s;
                break;
            }
        }
        var items = new List<int>();
        int s2 = bestS;
        for (int i = n - 1; i ≥ 0; i--)
        {
            if (take[i][s2])
            {
                items.Add(i);
                s2 -= sv[i];
            }
        }
        items.Reverse();
        int value = 0;
        int weight = 0;
        foreach (int i in items)
        {
            value += v[i];
            weight += w[i];
        }
        return (k, bestS, items.ToArray(), value, weight);
    }
```

listing 1618: c#, the min-weight table over scaled profit, the backward rebuild (source: dsa/samples/src/Ch42/Ptas.cs, lines 144-190)

```
export function fptas(w, v, cap, num, den) {
  const n = w.length;
  let vmax = 0;
  for (const x of v) {
    if (x > vmax) vmax = x;
  }
  const K = Math.max(1, Math.floor((num * vmax) / (den * n)));
  const sv = v.map((x) => Math.floor(x / K));
  let svmax = 0;
  for (const x of sv) svmax += x;
  const dp2 = new Array(svmax + 1).fill(Infinity);
  dp2[0] = 0;
  const take = Array.from({ length: n }, () => new Array(svmax + 1)
    .fill(false));
  for (let i = 0; i < n; i++) {
    for (let s = svmax; s ≥ sv[i]; s--) {
      if (dp2[s - sv[i]] + w[i] < dp2[s]) {
        dp2[s] = dp2[s - sv[i]] + w[i];
        take[i][s] = true;
      }
    }
  }
  let bestS = 0;
  for (let s = svmax; s ≥ 0; s--) {
    if (dp2[s] ≤ cap) {
      bestS = s;
      break;
    }
  }
  const items = [];
  let s = bestS;
  for (let i = n - 1; i ≥ 0; i--) {
    if (take[i][s]) {
      items.push(i);
      s -= sv[i];
    }
  }
  items.reverse();
  let value = 0;
  let weight = 0;
  for (const i of items) {
    value += v[i];
    weight += w[i];
  }
  return { K, scaledProfit: bestS, items, value, weight };
}
```

listing 1619: javascript, scaled dp with the take table, value recomputed at the end (source: dsa/samples-js/src/ch42-ptas.mjs, lines 85-130)

```python
def fptas(w, v, cap, pn, pd):
    # eps = pn / pd, K = max(1, floor(eps * vmax / n)), scaled profits
    # v'_i = v_i // K, dp2[s] = minimum weight reaching scaled profit s
    n_items = len(w)
    vmax = max(v)
    k = max(1, (pn * vmax) // (pd * n_items))
    sv = [x // k for x in v]
    smax = sum(sv)
    dp2 = [BIG] * (smax + 1)
    dp2[0] = 0
    take = [[False] * (smax + 1) for _ in range(n_items)]
    for i in range(n_items):
        for s in range(smax, sv[i] - 1, -1):
            if dp2[s - sv[i]] + w[i] < dp2[s]:
                dp2[s] = dp2[s - sv[i]] + w[i]
                take[i][s] = True
    best_s = 0
    for s in range(smax, -1, -1):
        if dp2[s] ≤ cap:
            best_s = s
            break
    items = []
    s = best_s
    for i in range(n_items - 1, -1, -1):
        if take[i][s]:
            items.append(i)
            s -= sv[i]
    items.reverse()
    value = sum(v[i] for i in items)
    return value, items, k, sum(w[i] for i in items)
```

listing 1620: python, the same table, integer division on profits, both instances pinned (source: dsa/samples-py/src/Ch42/ptas.py, lines 107-136)

```lua
local function fptas(w, v, cap, num, den)
  local n = #w
  local vmax = 0
  for i = 1, n do
    if v[i] > vmax then vmax = v[i] end
  end
  local k = (num * vmax) // (den * n)
  if k < 1 then k = 1 end
  local sv = {}
  local svmax = 0
  for i = 1, n do
    sv[i] = v[i] // k
    svmax = svmax + sv[i]
  end
  local inf = 1 << 30
  local dp2, take = {}, {}
  for s = 0, svmax do dp2[s] = inf end
  dp2[0] = 0
  for i = 1, n do
    take[i] = {}
    for s = svmax, sv[i], -1 do
      if dp2[s - sv[i]] + w[i] < dp2[s] then
        dp2[s] = dp2[s - sv[i]] + w[i]
        take[i][s] = true
      end
    end
  end
  local best_s = 0
  for s = svmax, 0, -1 do
    if dp2[s] ≤ cap then
      best_s = s
      break
    end
  end
  local items = {}
  local s = best_s
  for i = n, 1, -1 do
    if take[i][s] then
      items[#items + 1] = i
      s = s - sv[i]
    end
  end
  local val = 0
  for _, i in ipairs(items) do val = val + v[i] end
  return val, items, k, best_s
end
```

listing 1621: lua, the scaled dp, 1 << 30 as the sentinel, k pinned per eps (source: dsa/samples-lua/ch42_ptas.lua, lines 103-148)

The near-tie instance is what an fptas at work honestly looks like. It does not gently approach the optimum as eps shrinks, it is blind below a threshold and exact above it, the threshold sits where the scaling step is as wide as the value differences, and the bound is what the blindness costs, never more. Section 11 records the other half: knapsack is as far as this comfort reaches.

42.9. bin packing first fit

3 orderings of one rule: first fit scans the open bins in creation order and takes the first with room, else opens a new bin. First fit decreasing sorts the items non-increasing first, stable so equal sizes keep input order, then runs the same scan. Next fit keeps a single open bin and closes it the moment an item does not fit. The oracle is branch and bound over item to bin assignments, each item trying the existing bins by load with equal loads pruned as symmetric and the fresh bin last, exact at n up to 9. The bounds are prose: next fit never doubles the optimum, pairing each closed bin with the item that killed it fills 2 bins past the cap for every pair, and first fit decreasing lands within 11/9 of the optimum plus 2/3, Johnson's ratio with Dosa's tight constant.

The dry run: the fixtures are the 5 pinned instances, asserted by all 6 suites as bin counts. The c tree adds the pinned ffd loads and the factor 2 bounds, and the other 5 check that loads conserve the items and stay under the cap on every bin.

1. The first instance, 5, 5, 4, 4, 3, 3, 3, 3 at cap 10: ff 4, ffd 4, nf 4, opt 3, the classic shape where every heuristic sits one bin above. The ffd loads pin as 10, 8, 9, 3, the first bin exactly full and 3 stranded units in the last.
2. The second, 6, 6, 4, 4 at cap 10: ff 2, ffd 2, nf 3, opt 2. Next fit closes the first bin at 6 when the second 6 arrives, pairs 6 + 4 and 6 + 4 fit, but the closed bin stays closed, 3 against 2.
3. The third, 7, 6, 3, 4, 5, 4, 7, 3 at cap 10: 4, 4, 5, 4.
4. The fourth, 8, 5, 5, 8, 6, 4, 4, 6 at cap 13: 4, 4, 4, 4, nothing to prove, everything lands on the optimum.
5. The fifth, 4, 4, 4, 6, 6, 6, 5, 5, 5 at cap 10: ff 6, ffd 5, nf 6, opt 5. Input order alone costs first fit a bin, its loads read 8, 10, 6, 6, 10, 5 while the sorted pass reads 10, 10, 10, 10, 5.

items	cap	ff	ffd	nf	opt
5, 5, 4, 4, 3, 3, 3, 3	10	4	4	4	3
6, 6, 4, 4	10	2	2	3	2
7, 6, 3, 4, 5, 4, 7, 3	10	4	4	5	4
8, 5, 5, 8, 6, 4, 4, 6	13	4	4	4	4
4, 4, 4, 6, 6, 6, 5, 5, 5	10	6	5	6	5

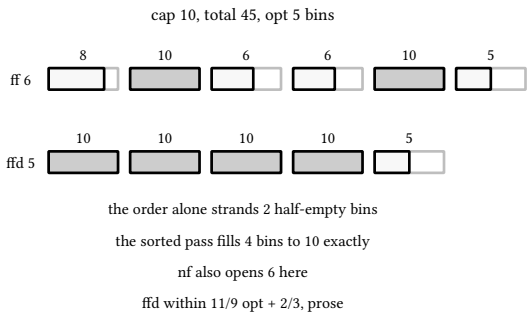


figure 382: the fifth instance at cap 10: first fit opening 6 bins in input order against the sorted pass closing in 5, the slack shaded

The 6 against 5 and the pinned loads carry the section, and the listings below ship the 3 heuristics and the oracle in 7 languages.

```c
static int ff_bins(const int *items, int m, int cap, int *loads) {
  int bn = 0;
  for (int i = 0; i < m; i++) {
    int placed = 0;
    for (int b = 0; b < bn; b++)
      if (loads[b] + items[i] ≤ cap) {
        loads[b] += items[i];
        placed = 1;
        break;
      }
    if (!placed)
      loads[bn++] = items[i];
  }
  return bn;
}

static int ffd_bins(const int *items, int m, int cap, int *loads) {
  int idx[9], src[9];
  for (int i = 0; i < m; i++)
    idx[i] = i;
  for (int i = 1; i < m; i++) {
    int v = idx[i], j = i - 1;
    while (j ≥ 0 && items[idx[j]] < items[v]) {
      idx[j + 1] = idx[j];
      j--;
    }
    idx[j + 1] = v;
  }
  for (int i = 0; i < m; i++)
    src[i] = items[idx[i]];
  return ff_bins(src, m, cap, loads);
}

static int nf_bins(const int *items, int m, int cap) {
  int bn = 0, open = 0, any = 0;
  for (int i = 0; i < m; i++) {
    if (!any || open + items[i] > cap) {
      open = items[i];
      any = 1;
      bn++;
    } else {
      open += items[i];
    }
  }
  return bn;
}
```

listing 1622: c, first fit, the stable decreasing sort, next fit (source: dsa/samples-c/src/Ch42/binpack.c, lines 30-75)

```
func FirstFit(items []int, cap int) (count int, loads []int) {
  for _, it := range items {
    placed := false
    for b := range loads {
      if loads[b]+it <= cap {
        loads[b] += it
        placed = true
        break
      }
    }
    if !placed {
      loads = append(loads, it)
    }
  }
  return len(loads), loads
}

// FirstFitDecreasing sorts items non-increasing first, stable so
// ties keep input order, then runs first fit on the sorted list.
func FirstFitDecreasing(items []int, cap int) (int, []int) {
  sorted := slices.Clone(items)
  slices.SortStableFunc(sorted, func(a, b int) int { return b - a })
  return FirstFit(sorted, cap)
}

// NextFit keeps exactly one open bin and closes it whenever the
// next item does not fit.
func NextFit(items []int, cap int) (count int, loads []int) {
  for _, it := range items {
    if len(loads) == 0 || loads[len(loads)-1]+it > cap {
      loads = append(loads, it)
    } else {
      loads[len(loads)-1] += it
    }
  }
  return len(loads), loads
}
```

listing 1623: go, the scans and the stable non-increasing sort (source: dsa/samples-go/ch42/binpack.go, lines 7-43)

```
static int ffBins(int[] items, int cap, int[] loads) {
  int m = items.length;
  int bn = 0;
  for (int i = 0; i < m; i++) {
    boolean placed = false;
    for (int b = 0; b < bn; b++)
      if (loads[b] + items[i] ≤ cap) {
        loads[b] += items[i];
        placed = true;
        break;
      }
    if (!placed)
      loads[bn++] = items[i];
  }
  return bn;
}

static int ffdBins(int[] items, int cap, int[] loads) {
  int m = items.length;
  Integer[] idx = new Integer[m];
  for (int i = 0; i < m; i++)
    idx[i] = i;
  for (int i = 1; i < m; i++) {
    Integer v = idx[i];
    int j = i - 1;
    while (j ≥ 0 && items[idx[j]] < items[v]) {
      idx[j + 1] = idx[j];
      j--;
    }
    idx[j + 1] = v;
  }
  int[] src = new int[m];
  for (int i = 0; i < m; i++)
    src[i] = items[idx[i]];
  return ffBins(src, cap, loads);
}

static int nfBins(int[] items, int cap) {
  int bn = 0, open = 0;
  boolean any = false;
  for (int i : items) {
    if (!any || open + i > cap) {
      open = i;
      any = true;
      bn++;
    } else {
      open += i;
    }
  }
  return bn;
}
```

listing 1624: java, first fit, the stable decreasing sort, next fit (source: dsa/samples-java/src/Ch42/Binpack.java, lines 27-77)

```
    public static (int Count, int[] Bins) FirstFit(int[] items, int cap)
    {
        var bins = new List<int>();
        foreach (int it in items)
        {
            bool placed = false;
            for (int b = 0; b < bins.Count; b++)
            {
                if (bins[b] + it ≤ cap)
                {
                    bins[b] += it;
                    placed = true;
                    break;
                }
            }
            if (!placed)
                bins.Add(it);
        }
        return (bins.Count, bins.ToArray());
    }

    public static (int Count, int[] Bins) FirstFitDec(int[] items, int cap)
    {
        var sorted = (int[])items.Clone();
        Array.Sort(sorted, (a, b) => b.CompareTo(a));
        return FirstFit(sorted, cap);
    }

    public static (int Count, int[] Bins) NextFit(int[] items, int cap)
    {
        var bins = new List<int>();
        foreach (int it in items)
        {
            if (bins.Count == 0 || bins[^1] + it > cap)
                bins.Add(it);
            else
                bins[^1] += it;
        }
        return (bins.Count, bins.ToArray());
    }
```

listing 1625: c#, the 3 scans over a growing bin list (source: dsa/samples/src/Ch42/Binpack.cs, lines 15-54)

```
export function firstFit(items, cap) {
  const bins = [];
  for (const it of items) {
    let placed = false;
    for (let b = 0; b < bins.length; b++) {
      if (bins[b] + it ≤ cap) {
        bins[b] += it;
        placed = true;
        break;
      }
    }
    if (!placed) bins.push(it);
  }
  return { count: bins.length, bins };
}

export function firstFitDec(items, cap) {
  return firstFit([...items].sort((a, b) => b - a), cap);
}

export function nextFit(items, cap) {
  const bins = [];
  for (const it of items) {
    if (bins.length === 0 || bins[bins.length - 1] + it > cap) {
      bins.push(it);
    } else {
      bins[bins.length - 1] += it;
    }
  }
  return { count: bins.length, bins };
}
```

listing 1626: javascript, the same 3 rules over plain arrays (source: dsa/samples-js/src/ch42-binpack.mjs, lines 9-39)

```python
def first_fit(items, cap, decreasing=False):
    # scan existing bins in creation order, take the first fit, else
    # open a new bin. FFD sorts non-increasing first, stable
    src = sorted(items, reverse=True) if decreasing else list(items)
    bins = []
    for it in src:
        for b in range(len(bins)):
            if bins[b] + it ≤ cap:
                bins[b] += it
                break
        else:
            bins.append(it)
    return bins


def next_fit(items, cap):
    # one open bin, closed when the next item does not fit
    bins = []
    for it in items:
        if not bins or bins[-1] + it > cap:
            bins.append(it)
        else:
            bins[-1] += it
    return bins
```

listing 1627: python, the fill with the for-else open, next fit, one flag for ffd (source: dsa/samples-py/src/Ch42/binpack.py, lines 14-37)

```lua
local function first_fit(items, cap, decreasing)
  local src = {table.unpack(items)}
  if decreasing then
    for i = 2, #src do -- stable non-increasing
      local it = src[i]
      local j = i - 1
      while j ≥ 1 and src[j] < it do
        src[j + 1] = src[j]
        j = j - 1
      end
      src[j + 1] = it
    end
  end
  local bins = {}
  for _, it in ipairs(src) do
    local placed = false
    for b = 1, #bins do
      if bins[b] + it ≤ cap then
        bins[b] = bins[b] + it
        placed = true
        break
      end
    end
    if not placed then bins[#bins + 1] = it end
  end
  return #bins, bins
end

local function next_fit(items, cap)
  local bins = {}
  for _, it in ipairs(items) do
    if #bins == 0 or bins[#bins] + it > cap then
      bins[#bins + 1] = it
    else
      bins[#bins] = bins[#bins] + it
    end
  end
  return #bins, bins
end
```

listing 1628: lua, the insertion-sorted decreasing pass, the 2 scans (source: dsa/samples-lua/ch42_binpack.lua, lines 10-48)

The oracle earns a note of its own: it is the chapter 20 machinery in miniature, its bound the count of open bins in place of a value, pruning symmetric branches the heuristics never see, and it runs in microseconds at n = 9 and in years at n = 20, which is the whole reason the heuristics exist.

42.10. local search as an approximation engine

Chapter 41 improved its states one move at a time and promised nothing about where the moves stop. The single-flip driver here, strictly improving flips from a fixed vertex order, inherits a theorem: at a local optimum every vertex has at least as many crossing edges as same-side edges, a flipping vertex would gain the difference, so summing over vertices counts each crossing edge twice on the crossing ledger and each internal edge twice on the other, giving cut at least half the edges. The guarantee is rho = 2 for maximization, cut x 2 at least m, and it needs nothing but the stop condition, no order, no restarts, no schedule.

The dry run: the fixture is k5 minus the edges a to d and d to e, 8 edges over 5 vertices, asserted by all 6 suites with the flip count, the side, the oracle over all 32 assignments, and the bound.

1. All 5 vertices start on one side, cut 0, and the first pass flips a, then b: 2 flips, both strictly improving, side a, b against c, d, e.
2. No single flip helps from there: flipping a drops the 2 crossing edges a to c and a to e and lifts only a to b, flipping d trades b to d for c to d, the partition is flip-stable and the suite checks every vertex.
3. The local optimum cuts 5 edges, the brute force over the 32 sides finds 6, and 2 x 5 is at least 8, the bound holds with room, the achieved ratio is 5/6.
4. The optimum itself splits a, d, e against b, c: 6 crossing edges, and the 2 left inside are a to e and b to c, the cheapest pair of edges that settles all 5 triangles hiding in the fixture.

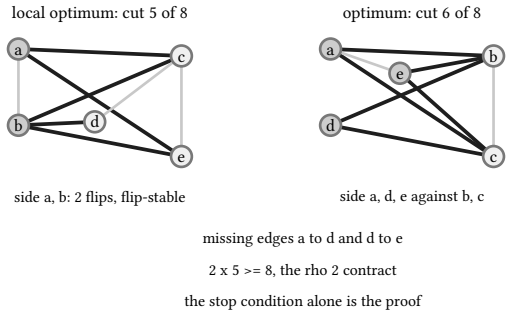


figure 383: the flip search stopping at 5 of 8 edges against the optimal split at 6, cut edges heavy in both panels

The 5 against 6 with the bound intact is the pinned landing, and the listings below run the flip driver in 7 languages.

```c
int main(void) {
  // brute force optimum over all 2^5 assignments
  int opt = 0;
  for (int bits = 0; bits < 32; bits++) {
    char side[5];
    for (int v = 0; v < 5; v++)
      side[v] = (char)((bits >> v) & 1);
    if (cut(side) > opt)
      opt = cut(side);
  }
  CHECK(opt == 6);

  // flip local search: passes over the fixed order a..e, keep a flip only
  // when it strictly improves the cut
  char side[5] = {0};
  int flips = 0, guard = 0;
  for (;;) {
    int improved = 0;
    guard++;
    CHECK(guard ≤ 100);
    for (int v = 0; v < 5; v++) {
      side[v] ^= 1;
      int newc = cut(side);
      side[v] ^= 1;
      int oldc = cut(side);
      if (newc > oldc) {
        side[v] ^= 1;
        flips++;
        improved = 1;
      }
    }
    if (!improved)
      break;
  }
  CHECK(flips == 2);
  CHECK(cut(side) == 5);
  CHECK(side[0] == 1 && side[1] == 1);
  CHECK(side[2] == 0 && side[3] == 0 && side[4] == 0);

  // the 2 approximation, cross-multiplied: 2 * 5 ≥ 8
  CHECK(2 * cut(side) ≥ 8);
  CHECK(cut(side) < opt);

  printf("ok %d maxcut\n", n);
```

listing 1629: c, the oracle over 32 sides, the flip loop with its guard, the checks (source: dsa/samples-c/src/Ch42/maxcut.c, lines 30-73)

```
func MaxCutLocal(edges [][2]int, n int, order []int) (cut int, side []int, flips int) {
  side = make([]int, n)
  cutValue := func() int {
    c := 0
    for _, e := range edges {
      if side[e[0]] != side[e[1]] {
        c++
      }
    }
    return c
  }
  improved := true
  for improved {
    improved = false
    for _, v := range order {
      before := cutValue()
      side[v] ^= 1
      if after := cutValue(); after > before {
        flips++
        improved = true
      } else {
        side[v] ^= 1
      }
    }
  }
  return cutValue(), side, flips
}
```

listing 1630: go, the same driver, the order a parameter, the brute beside it (source: dsa/samples-go/ch42/maxcut.go, lines 7-33)

```
  public static void main(String[] args) {
    // brute force optimum over all 2^5 assignments
    int opt = 0;
    for (int bits = 0; bits < 32; bits++) {
      int[] side = new int[5];
      for (int v = 0; v < 5; v++)
        side[v] = (bits >> v) & 1;
      if (cut(side) > opt)
        opt = cut(side);
    }
    ok(opt == 6, "the brute optimum cuts 6");

    // flip local search: passes over the fixed order a..e, keep a flip
    // only when it strictly improves the cut
    int[] side = new int[5];
    int flips = 0, guard = 0;
    for (;;) {
      boolean improved = false;
      guard++;
      ok(guard ≤ 100, "pass " + guard + " stays under the guard");
      for (int v = 0; v < 5; v++) {
        side[v] ^= 1;
        int newc = cut(side);
        side[v] ^= 1;
        int oldc = cut(side);
        if (newc > oldc) {
          side[v] ^= 1;
          flips++;
          improved = true;
        }
      }
      if (!improved)
        break;
    }
    ok(flips == 2, "two flips carry the search to its stall");
    ok(cut(side) == 5, "the stall cuts 5");
    ok(side[0] == 1 && side[1] == 1, "a and b moved sides");
    ok(side[2] == 0 && side[3] == 0 && side[4] == 0, "c, d, e stayed");

    // the 2 approximation, cross-multiplied: 2 * 5 ≥ 8
    ok(2 * cut(side) ≥ 8, "2 * 5 ≥ |E| holds");
    ok(cut(side) < opt, "the stall sits strictly below the optimum");
  }
```

listing 1631: java, the oracle over 32 sides, the flip loop with its guard, the checks (source: dsa/samples-java/src/Ch42/Maxcut.java, lines 28-70)

```
    public static (int Cut, int[] Side, int Flips) Local(int n, (int A, int B)[] edges)
    {
        var side = new int[n];
        int flips = 0;
        bool improved = true;
        while (improved)
        {
            improved = false;
            for (int v = 0; v < n; v++)
            {
                side[v] ^= 1;
                int after = CutValue(edges, side);
                side[v] ^= 1;
                if (after > CutValue(edges, side))
                {
                    side[v] ^= 1; // keep the flip
                    flips++;
                    improved = true;
                }
            }
        }
        return (CutValue(edges, side), side, flips);
    }
```

listing 1632: c#, the pass over the fixed order, keep the flip only when it gains (source: dsa/samples/src/Ch42/Maxcut.cs, lines 25-47)

```
export function maxcutLocal(n, edges) {
  const side = new Array(n).fill(0);
  let flips = 0;
  let improved = true;
  while (improved) {
    improved = false;
    for (let v = 0; v < n; v++) {
      side[v] ^= 1;
      const after = cutValue(n, edges, side);
      side[v] ^= 1;
      if (after > cutValue(n, edges, side)) {
        side[v] ^= 1; // keep the flip
        flips += 1;
        improved = true;
      }
    }
  }
  return { cut: cutValue(n, edges, side), side, flips };
}
```

listing 1633: javascript, the flip pass and the reverted probe (source: dsa/samples-js/src/ch42-maxcut.mjs, lines 16-34)

```
def cut(side):
    return sum(1 for a, b in EDGES if side[a] != side[b])


def local_search():
    # start all on one side, flip vertices in fixed order a..e, keep a
    # flip only when it strictly improves the cut
    side = {v: 0 for v in NODES}
    flips = 0
    while True:
        improved = False
        for v in NODES:
            base = cut(side)
            side[v] ^= 1
            if cut(side) > base:
                flips += 1
                improved = True
            else:
                side[v] ^= 1
        if not improved:
            return side, flips
```

listing 1634: python, the cut, the local search, the flip-stability check below (source: dsa/samples-py/src/Ch42/maxcut.py, lines 30-50)

```
local function maxcut_local()
  local side = {0, 0, 0, 0, 0}
  local flips = 0
  local improved = true
  while improved do
    improved = false
    for v = 1, 5 do
      side[v] = side[v] ~ 1
      local reverted = {table.unpack(side)}
      reverted[v] = reverted[v] ~ 1
      if cut(side) > cut(reverted) then
        flips = flips + 1
        improved = true
      else
        side[v] = side[v] ~ 1
      end
    end
  end
  return cut(side), side, flips
end
```

listing 1635: lua, the driver mirrored from chapter 41, the revert path (source: dsa/samples-lua/ch42_maxcut.lua, lines 25-44)

The bridge back to book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics) is the point of the section. There a single flip earns annealing schedules, tabu tenure, and restarts to escape exactly this kind of stop, and none of it carries a number. Here the stop is the theorem, the number is 2, and the fixtures show the slack, 5 of a possible 6. Between them sits a refinement the corpus states without sampling: on cubic graphs the locally optimal cut reaches 2/3 of the optimum, and this fixture is not cubic, degrees run 2 through 4, so 5/6 clearing 2/3 is luck of the instance and nothing stronger.

42.11. inapproximability: a reader's map

Everything above the line costs a ratio. The map below says where the line is, and it is written as prose because the content is reduction sketches, not code the fixtures could pin.

The first barrier is the oldest one. If a polynomial algorithm approximated an NP-complete problem with a ratio close enough to 1 to separate a reduction's yes and no cases, a separation reduction turns it into an exact polynomial algorithm: build an instance where the yes and no cases sit a known factor apart, run the approximation, and read the answer off the side it lands on. That would solve an NP-complete problem in polynomial time, so unless P = NP the exact version is out of reach and every ratio in this chapter is a negotiation with the barrier, priced individually. The statement is not that approximation is useless, the sections above are the counterexample.

The second barrier is the metric itself. Drop the triangle inequality from the tsp and no finite ratio survives: encode any hamiltonian cycle question as a complete graph with edge weights 1 and w, w huge, the weight at which a tour exists and a tour does not differ by more than any fixed rho, so a rho-approximation would decide hamiltonicity. The metric instances of sections 5 and 6 are not a convenience of the fixtures, the hypothesis carries the theorem, and the manhattan matrices of the chapter keep it witnessed in integers.

The third barrier prices the logarithm. Set cover admits the ln n + 1 of section 4 and nothing polynomial meaningfully better: no algorithm reaches (1 - eps) x ln n for any eps unless problems in NP fall to quasi-polynomial time, the conditional hardness of Feige's reduction, and Slavik's analysis shows the greedy sits exactly at ln n - ln ln n + theta(1), so the classic algorithm and the barrier nearly touch. Vertex cover sits in the opposite situation, its 2 is ancient, no polynomial 1.9x is known, and the known hardness stops well short of forbidding one, the widest open spread in the field.

The fourth barrier is the quiet one, and it is good news. Knapsack carries an fptas, section 8, and that is the best class of guarantee an NP-hard problem can hold: any strongly NP-hard problem, hard even when every number is polynomially bounded, admits no fptas unless P = NP, because epsilon small enough makes the running time polynomial and the error zero at once. Knapsack is only weakly hard, its numbers are the enemy and scaling disarms them, and the fptas is the exact shape of that weakness. When a new problem arrives, the first structural question after the exact lanes fail is which side of this line its hardness lives on.

42.12. choosing the right surrender

The corpus now holds every tier of the trade, and the choice is a table lookup once the instance size and the required guarantee are on the table.

1. Under a few million states, exact wins outright: the mask and interval tables of chapters 17 and 19, the assignment and flow solvers of chapter 38, all polynomial or singly exponential in one small parameter, all returning the optimum with a witness.
2. When the states explode but the structure prunes, chapter 20′s branch and bound buys the same exactness on luck, the fractional bound deciding how much luck.
3. When exactness leaves the budget and a proven ratio suffices, this chapter: the 2 of the matching cover, the ln n + 1 of greedy set cover, the 1.5 of christofides, the bin packing heuristics, each with its feasibility check and its oracle in the test.
4. When the ratio itself must be chosen, knapsack's ptas and fptas, sections 7 and 8, accuracy as an input, polynomial time in n and 1 over eps.
5. When no ratio is known or the instance is simply too large, book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics), anytime improvement with no promise, run against a clock and kept if it beats what is on hand.

tool	guarantee	instance size	where
exact dp	optimal, exponential in one parameter	under a few million states	chapters

tool	guarantee	instance size	where
			17 and 19, masks, intervals, tables
branch and bound	optimal, bound dependent	unpredictable, seconds to hours	chapter 20, the fractional knapsack bound
flows and matching	optimal, polynomial	tens of thousands of edges	chapter 38, kuhn and the flow solvers
proven ratio	2, ln n + 1, 1.5, 11/9	any size the scan fits	this chapter, sections 3 through 6 and 9
ptas and fptas	1 - eps, polynomial in n and 1 over eps	knapsack shapes at any size	this chapter, sections 7 and 8
book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics)	none, anytime	anything, budget the clock	chapter 41, anneal-

tool	guarantee	instance size	where
			ing, tabu, genetic

The rows read as a pipeline, and the corpus chapters cross-link along it: chapter 17 points forward to here when its tables stop fitting memory, chapter 20 points here when its trees stop finishing, and section 10 points back at book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics) for the driver it borrowed. The last rule of the table is the one the whole chapter argues: whatever the row, the answer it returns is a witness with a check attached, feasible by construction and priced against an oracle in the same test, and a result that cannot say both of those things about itself is not an approximation, it is a guess.

42.13. across the seven languages

Featured build size counted as non-blank, non-comment source lines of the chapter's 8 sample files per language in the c, go, java, javascript, python, and lua trees, the c# row counting non-blank lines of its 8 files, go test files excluded:

stem	c	go	java	c#	javascript	python	lua	note
ratio	109	44	112	76	48	55	90	serves sections 1 and 2, oracle half and contract half
vc	80	15	80	46	20	59	105	go leans on ratio.go for the oracle and feasibility
setcover	102	62	94	89	43	57	113	bitmask ground in c and go, sets elsewhere
tsp2x	182	97	174	172	121	106	181	matrices restated per file in c, python, lua
christofides	218	54	208	92	56	115	218	c# and js import the tsp2x and ch41 helpers
ptas	182	152	183	187	115	127	183	serves sections 7 and 8, greedy and both tables
binpack	115	63	119	85	56	72	121	the oracle is branch and bound at n up to 9
maxcut	58	43	56	64	39	57	73	the thinnest stem, one driver and one oracle

The small numbers are sharing. Go's vc is 15 source lines because the oracle and the feasibility predicate live in ratio.go and the matching engine is all that is left to state, and go's christofides at 54 composes the prim, euler, and shortcut helpers of tsp2x.go instead of restating them. The c# tree keeps the fixtures and the shared walkers in tsp2x.cs and the chapter 41 localsearch class, so christofides.cs is 92 lines of matching and assembly, and javascript imports eulerWalk, mstPrim, and shortcut across the stem boundary the same way. C, python, and lua restate the matrices and the walkers per file, 3 trees times 8 files, which is why their columns run high, lua adding the 1-based index adjustments and the run.lua check rows on top. Java restates the same way, every stem standalone, prim and the euler walk rebuilt inside tsp2x and christofides rather than imported, and its density and ratio comparisons ride long products cross-multiplied, never doubles.

The corpus placement is honest silence: the icpc book's mined finals problems all yielded to exact algorithms, so no chapter there cites an approximation technique, and this chapter's contest relevance is the negative space, the tier that takes over when the exact tool the icpc chapters teach does not finish. The nearest neighbors in this book are the held-karp mask tables of book 9, ch 19 (practical data structures and algorithms, advanced dynamic programming) that hold the optima section 5′s permutation oracles walk the slow way, the pruning discipline of book 9, ch 20 (practical data structures and algorithms, pruning and branch and bound) that section 9′s oracle miniaturizes, and the single-flip discipline of book 9, ch 41 (practical data structures and algorithms, local search and metaheuristics) that section 10 returns with a theorem.

sources: V. V. Vazirani, "Approximation Algorithms", Springer, 2001, the vertex cover and set cover chapters behind sections 3 and 4. D. P. Williamson and D. B. Shmoys, "The Design of Approximation Algorithms", Cambridge University Press, 2011, the greedy, tsp, and knapsack sections behind sections 4 through 8. T. H. Cormen, C. E.

Leiserson, R. L. Rivest, and C. Stein, “Introduction to Algorithms”, third edition, MIT Press, 2009, chapter 35, sections 35.1 through 35.5, the set cover, tsp, and subset sum fptas treatments. P. Slavik, “A tight analysis of the greedy algorithm for set cover”, Journal of Algorithms 25, 1997, the exact greedy worst case named in section 4. The bin packing bounds follow D. S. Johnson’s 1973 thesis and G. Dosa’s 2007 tightening of the additive constant. Sample behavior verified by the 7 suite gates scoped to chapter 42: c 8 files and 8 checks, go 17 test functions, java 8 files and 224 checks under run-java-samples, c# 24 facts, javascript 24 tests across 8 suites, python 8 files and 8 checks, lua 37 checks, zero skipped.

43. capstone: storage engine toolkit

The capstone is a log-structured storage engine with a live dashboard: puts and deletes, crash recovery, prefix scans, compaction, and an htmx 4 web face over it. Every piece is a chapter of this book doing its job under a real workload, and every file sits under 260 lines.

43.1. the integrity layer

Every byte written to disk gets a CRC-32 frame, the table-driven reflected polynomial, so bit rot and torn writes are detected rather than silently parsed:

```
public static class Crc32
{
    public const uint Poly = 0xEDB88320;

    private static readonly uint[] Table = BuildTable();

    private static uint[] BuildTable()
    {
        var table = new uint[256];
        for (uint i = 0; i < 256; i++)
        {
            uint c = i;
            for (int k = 0; k < 8; k++)
                c = (c & 1) != 0 ? Poly ^ (c >> 1) : c >> 1;
            table[i] = c;
        }
        return table;
    }

    public static uint Compute(ReadOnlySpan<byte> bytes)
    {
        uint crc = 0xFFFFFFFF;
        foreach (byte b in bytes)
            crc = Table[(crc ^ b) & 0xFF] ^ (crc >> 8);
        return ~crc;
    }

    /// <summary>Continue a checksum across frames, as the record reader does.</summary>
    public static uint Extend(uint crc, ReadOnlySpan<byte> bytes)
    {
        uint c = ~crc;
        foreach (byte b in bytes)
            c = Table[(c ^ b) & 0xFF] ^ (c >> 8);
        return ~c;
    }
}
```

listing 1636: crc 32 with the extension form for multi frame checksums (source: dsa/capstone/src/Engine/Crc32.cs, lines 4-39)

Above it sits the bloom filter. Chapter 25 teaches the structure from scratch, double hashing with two FNV-1a seeds through the kirsch-mitzenmacher sum, so this chapter keeps only the engine-grade half: the sizing math that picks bits and hashes from the target false positive rate, about one percent here, and the byte serialization that carries the filter inside every segment, header plus raw words. No false negatives ever is the property the read path leans on, one memory probe replacing an index search:

```
    public Bloom(long expectedKeys, double falsePositiveRate = 0.01)
    {
        if (expectedKeys < 1)
            throw new ArgumentOutOfRangeException(nameof(expectedKeys));
        if (falsePositiveRate is <= 0 or >= 1)
            throw new ArgumentOutOfRangeException(nameof(falsePositiveRate));

        // m = -n ln p / (ln 2)^2, k = m/n ln 2
        double m = -expectedKeys * Math.Log(falsePositiveRate) / (Math.Log(2) * Math.Log(2));
        _bitCount = Math.Max(64, (int)Math.Ceiling(m));
        _hashes = Math.Max(1, (int)Math.Ceiling(_bitCount / (double)expectedKeys * Math.Log(2)));
        _words = new ulong[(_bitCount + 63) / 64];
    }

    private Bloom(int bitCount, int hashes, ulong[] words)
    {
        _bitCount = bitCount;
        _hashes = hashes;
        _words = words;
    }

    public int Bits => _bitCount;

    public int Hashes => _hashes;

    /// <summary>Serialize as header (bit count, hash count) plus the raw words.</summary>
    public byte[] ToBytes()
    {
        var bytes = new byte[8 + _words.Length * 8];
        BitConverter.TryWriteBytes(bytes.AsSpan(0, 4), _bitCount);
        BitConverter.TryWriteBytes(bytes.AsSpan(4, 4), _hashes);
        MemoryMarshal.AsBytes<ulong>(_words).CopyTo(bytes.AsSpan(8));
        return bytes;
    }

    public static Bloom FromBytes(ReadOnlySpan<byte> bytes)
    {
        int bitCount = BitConverter.ToInt32(bytes[..4]);
        int hashes = BitConverter.ToInt32(bytes.Slice(4, 4));
        var words = new ulong[(bitCount + 63) / 64];
        MemoryMarshal.Cast<byte, ulong>(bytes.Slice(8, words.Length * 8)).CopyTo(words);
        return new Bloom(bitCount, hashes, words);
    }
```

listing 1637: sizing from the false positive rate, serialization to bytes and back (source: dsa/capstone/src/Engine/Bloom.cs, lines 12-54)

The tests pin the reference vectors, walk every single-bit flip of a sample buffer, verify no false negative over ten thousand keys, and measure the realized false positive rate against the design point.

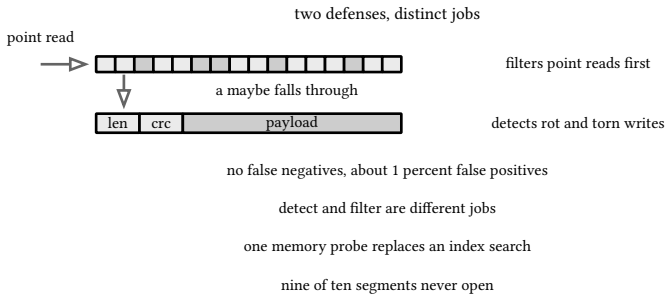


figure 384: the two defenses stacked, the bloom filters point reads, the crc detects what actually opens

43.2. the write-ahead log

Writes append framed records to a log before anything else happens: `[len][crc][payload]`, payload carrying the operation, the key, and the value. The flush call is `FileStream.Flush(flushToDisk: true)`, the durability flag, not the buffer-only flush:

```
public enum WalOp : byte
{
    Put = 1,
    Delete = 2,
}

/// <summary>A write-ahead-log record: op, key, optional value, framed with a crc.</summary>
public readonly record struct WalRecord(WalOp Op, byte[] Key, byte[]? Value)
{
    public int FramedSize => 8 + 1 + 4 + Key.Length + (Value is null ? 0 : 4 + Value.Length);

    public static WalRecord Put(byte[] key, byte[] value) => new(WalOp.Put, key, value);

    public static WalRecord Delete(byte[] key) => new(WalOp.Delete, key, null);
}

/// <summary>Appends framed records to a stream and reads them back with truncation tolerance.</summary>
public sealed class Wal : IDisposable
{
    private readonly FileStream _file;
```

listing 1638: the record and the op vocabulary (source: dsa/capstone/src/Engine/Wal.cs, lines 5-24)

```
    public void Append(WalRecord record)
    {
        var payload = EncodePayload(record);
        var frame = new byte[8 + payload.Length];
        BinaryPrimitives.WriteUInt32LittleEndian(frame.AsSpan(0, 4), (uint)payload.Length);
        BinaryPrimitives.WriteUInt32LittleEndian(frame.AsSpan(4, 4), Crc32.Compute(payload));
        payload.CopyTo(frame, 8);
        _file.Position = _file.Length;
        _file.Write(frame);
        _file.Flush(flushToDisk: true); // the durability flag, not the buffer flush alone
        RecordCount++;
    }

    /// <summary>Read every intact record in order. Stops at the first torn or corrupt frame.</summary>
    public static List<WalRecord> ReadAll(string path)
    {
        var records = new List<WalRecord>();
        if (!File.Exists(path))
            return records;

        using var file = new FileStream(path, FileMode.Open, FileAccess.Read, FileShare.ReadWrite);
        var body = new byte[file.Length];
        file.ReadExactly(body);

        int pos = 0;
        while (pos + 8 ≤ body.Length)
        {
            uint len = BinaryPrimitives.ReadUInt32LittleEndian(body.AsSpan(pos, 4));
            uint crc = BinaryPrimitives.ReadUInt32LittleEndian(body.AsSpan(pos + 4, 4));
            if (pos + 8 + len > body.Length)
                break; // torn tail from a crash mid append
            var payload = body.AsSpan(pos + 8, (int)len);
            if (Crc32.Compute(payload) ≠ crc)
                break; // corruption: everything after is untrusted
            records.Add(DecodePayload(payload));
            pos += 8 + (int)len;
        }
        return records;
    }
```

listing 1639: append one durable frame, read back with truncation tolerance (source: dsa/capstone/src/Engine/Wal.cs, lines 48-86)

The reader is where crash semantics live. A frame whose length runs past the end of the file is a torn tail, a crash mid append, and it is ignored. A frame whose crc fails is corruption, and everything after it is untrusted. The tests simulate both by truncating and flipping bytes in a written log, and recovery keeps exactly the records that were intact.

The flush path obeys the same discipline in the other direction: `WriteRun` fsyncs a finished segment before the engine truncates the wal that covers it, because a reset wal over a lost segment loses writes with no error anywhere. The fsync itself is not unit-assertable from user code, so the test pins the invariant it protects: it deletes the wal by hand after a flush and reopens, and every flushed key still reads.

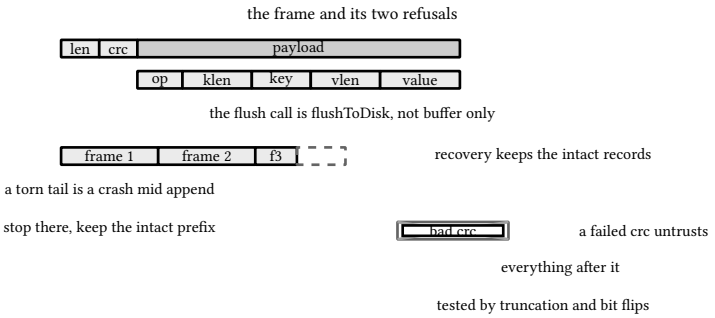


figure 385: the wal frame anatomy and the reader's two refusals, a torn tail stops the read, a bad crc untrusts the rest

43.3. the memtable

Live writes land in a skip list keyed by raw bytes. Chapter 25 teaches the structure itself, the tower heights and the search walk, so this chapter keeps the engine-grade half: a dictionary alongside the spine answers latest-write queries in constant time, size accounting tracks the bytes a flush will emit so the 256 KiB rollover is honest, and `Sorted()` plus `SortedFrom()` stream the run in byte order for flushes and for scans that seek:

```
    public void Put(byte[] key, byte[]? value, long sequence)
    {
        if (!_latest.TryGetValue(key, out Entry? existing))
        {
            SizeBytes += 16 + key.Length + (value?.Length ?? 0);
            InsertNode(key);
        }
        else
        {
            SizeBytes -= 16 + existing.Key.Length + (existing.Value?.Length ?? 0);
            SizeBytes += 16 + key.Length + (value?.Length ?? 0);
        }
        _latest[key] = new Entry(key, value, sequence);
    }

    public Entry? Get(byte[] key) => _latest.GetValueOrDefault(key);

    /// <summary>Sorted walk, ascending by unsigned byte order.</summary>
    public IEnumerable<Entry> Sorted()
    {
        Node? node = _head.Next[0];
        while (node != null)
        {
            yield return _latest[node.Key];
            node = node.Next[0];
        }
    }

    /// <summary>Sorted walk starting at the first key at or above the argument.</summary>
    public IEnumerable<Entry> SortedFrom(byte[] fromKeyInclusive)
    {
        Node? node = _head.Next[0];
        // a level 0 walk skips to the lower bound, then streams as Sorted does
        while (node != null && ByteArrayComparer.Compare(node.Key, fromKeyInclusive) < 0)
            node = node.Next[0];
        while (node != null)
        {
            yield return _latest[node.Key];
            node = node.Next[0];
        }
    }
```

listing 1640: put with size accounting, get from the dictionary, sorted walks from a lower bound (source: dsa/capstone/src/Engine/MemTable.cs, lines 42-82)

The towers carry a war story. `RandomLevel` drew one xorshift value and reused it for the whole climb, so every tower was height one or max, half and half, while its own comment claimed p = 1/2 per promotion. Chapter 25′s teaching twin measured the lane widths, caught the degeneracy, and the fix is the idiom that chapter teaches, a fresh flip per promotion, now pinned here the same way: level 2 holds about half the nodes, level 5 about a sixteenth.

The unsigned byte-order comparer at the bottom of the file is the one total order a binary key space has, prefix before longer, exactly what chapter 9′s trie assumed too.

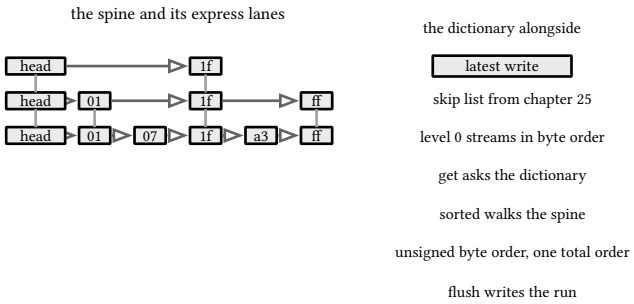


figure 386: the skip list spine with towers from a seeded xorshift, the level 0 chain streams sorted bytes

43.4. the sorted run

A flush writes the memtable as an immutable sstable: crc framed entries, the bloom bytes, a sparse index block with every sixteenth key pointing at its offset, and a twenty byte footer holding the index offset, bloom length, and a magic. Opening reads only that tail, footer first then bloom and index backward from it; the data region stays on disk and is fetched block by block:

```
    /// <summary>Open reads only the footer, the bloom, and the index; the data
    /// region stays on disk and is fetched block by block.</summary>
    public static SSTable Open(string path, BlockCache? cache = null)
    {
        using var probe = new FileStream(path, FileMode.Open, FileAccess.Read, FileShare.Read);
        if (probe.Length < FooterBytes)
            throw new InvalidDataException($"{path} is too small to be an sstable");

        probe.Seek(-FooterBytes, SeekOrigin.End);
        var footer = new byte[FooterBytes];
        probe.ReadExactly(footer);
        uint magic = BinaryPrimitives.ReadUInt32LittleEndian(footer.AsSpan(^4..));
        if (magic != Magic)
            throw new InvalidDataException($"{path} footer magic {magic:X8} is not ours");
        long indexOffset = BinaryPrimitives.ReadInt64LittleEndian(footer.AsSpan(^FooterBytes..^12));
        long bloomLength = BinaryPrimitives.ReadInt64LittleEndian(footer.AsSpan(^12..^4));

        var bloomBytes = new byte[bloomLength];
        probe.Seek(indexOffset - bloomLength, SeekOrigin.Begin);
        probe.ReadExactly(bloomBytes);
        var bloom = Bloom.FromBytes(bloomBytes);

        var index = new List<(byte[], long)>();
        var head = new byte[4];
        probe.Seek(indexOffset, SeekOrigin.Begin);
        while (probe.Position < probe.Length - FooterBytes)
        {
            probe.ReadExactly(head);
            int klen = BinaryPrimitives.ReadInt32LittleEndian(head);
            var rec = new byte[klen + 8];
            probe.ReadExactly(rec);
            index.Add((rec[..klen],
                BinaryPrimitives.ReadInt64LittleEndian(rec.AsSpan(klen..))));
        }
        if (index.Count == 0)
            throw new InvalidDataException($"{path} has an empty index block");

        return new SSTable(path, cache, indexOffset, bloomLength, index, bloom);
    }
```

listing 1641: open reads only the tail: footer, bloom, and sparse index (source: dsa/capstone/src/Engine/SSTable.cs, lines 49-87)

The stride 16 index already defines the blocks: block i spans `[Index[i].Offset, Index[i+1].Offset)`, the last ends where the bloom begins, so blocks needed no format change, only a different unit of reading. A point read is the book in miniature, the bloom answers first, the index is binary searched with chapter 14's lower bound, and the bracketing block is decoded and walked until sorted order says the key was passed. A range scan takes the same lower bound with no bloom gate, because a point filter cannot answer a range, chapter 21's query shape streaming over chapter 14's bound:

```
    /// <summary>Every entry in order, streamed block by block without the cache:
    /// compaction scans are sequential and would only evict useful blocks.</summary>
    public IEnumerable<MemTable.Entry> ReadAll()
    {
        for (int b = 0; b < Index.Count; b++)
            foreach (MemTable.Entry e in DecodeBlock(ReadBlockBytes(b)))
                yield return e;
```

```
    }

    /// <summary>Point read: bloom first, then the sparse index block, then a block walk.</summary>
    public MemTable.Entry? Get(byte[] key)
    {
        if (!Bloom.MightContain(key))
            return null; // no false negatives, so a miss here is final

        for (int b = LowerBlock(key); b < Index.Count; b++)
        {
            foreach (MemTable.Entry e in ReadBlock(b))
            {
                int c = ByteArrayComparer.Compare(e.Key, key);
                if (c == 0)
                    return e;
                if (c > 0)
                    return null; // sorted order says we passed it
            }
        }
        return null;
    }

    /// <summary>
    /// Stream every entry at or above the key, block aligned. No bloom gate:
    /// a range cannot be answered by a point filter, chapter 14's lower
    /// bound streaming instead.
    /// </summary>
    public IEnumerable<MemTable.Entry> Seek(byte[] fromKeyInclusive)
    {
        for (int b = LowerBlock(fromKeyInclusive); b < Index.Count; b++)
            foreach (MemTable.Entry e in ReadBlock(b))
                if (ByteArrayComparer.Compare(e.Key, fromKeyInclusive) >= 0)
                    yield return e;
    }

    /// <summary>The block a key would live in: the greatest index point strictly
    /// below it, chapter 14's lower bound. Blocks tile the data region with no
    /// gaps between index points, so walking forward from the bracketing block
    /// covers every candidate entry and the old boundary extension dissolves.</summary>
    private int LowerBlock(byte[] key)
    {
        int lo = 0, hi = Index.Count;
        while (lo < hi)
        {
            int mid = (lo + hi) / 2;
            if (ByteArrayComparer.Compare(Index[mid].Key, key) < 0)
                lo = mid + 1;
            else
                hi = mid;
        }
```

listing 1642: read all uncached, get and seek over blocks, the lower bound (source: dsa/capstone/src/Engine/SSTable.cs, lines 89-145)

Reading one block is three moves: seek to its index point, read the span up to the next one, decode the frames into entries, routed through the cache when one is attached and around it for `ReadAll`, because a compaction scan is sequential and would only evict useful blocks:

```
    }

    private long DataEnd => _indexOffset - _bloomLength;

    /// <summary>Read one block's bytes: the span between its index point and the next.</summary>
    private byte[] ReadBlockBytes(int block)
    {
        long start = Index[block].Offset;
        long end = block + 1 < Index.Count ? Index[block + 1].Offset : DataEnd;
        var bytes = new byte[end - start];
        // point reads share one file stream, the lock keeps each seek plus
        // read pair atomic under the engine's single writer
        lock (_file)
        {
            _file.Seek(start, SeekOrigin.Begin);
            _file.ReadExactly(bytes);
        }
        return bytes;
    }

    /// <summary>Decode a block's bytes into its entries, frame by frame.</summary>
    private static List<MemTable.Entry> DecodeBlock(byte[] bytes)
    {
        var entries = new List<MemTable.Entry>();
        int pos = 0;
        while (pos < bytes.Length)
        {
            (MemTable.Entry entry, int next) = EntryCodec.ReadFrame(bytes, pos);
            entries.Add(entry);
            pos = next;
        }
        return entries;
    }

    /// <summary>One index aligned block through the cache when one is attached.</summary>
    private List<MemTable.Entry> ReadBlock(int block)
    {
        List<MemTable.Entry>? cached = _cache?.Get(Path, block);
        if (cached is not null)
            return cached;
        var entries = DecodeBlock(ReadBlockBytes(block));
        _cache?.Put(Path, block, entries);
        return entries;
    }

    private int CountEntries()
    {
        int count = 0;
        for (int b = 0; b < Index.Count; b++)
            count += DecodeBlock(ReadBlockBytes(b)).Count;
        return count;
    }
```

listing 1643: the block plumbing: span read, frame decode, cache routing (source: dsa/capstone/src/Engine/SSTable.cs, lines 147-198)

The frames themselves come from `EntryCodec`, shared by the writer and every reader so the bytes have exactly one definition:

```
    /// <summary>Append one framed entry to a stream.</summary>
    public static void WriteFrame(Stream writer, MemTable.Entry e)
    {
        byte[] payload = Encode(e);
        var frame = new byte[8 + payload.Length];
        BinaryPrimitives.WriteUInt32LittleEndian(frame.AsSpan(0, 4), (uint)payload.Length);
        BinaryPrimitives.WriteUInt32LittleEndian(frame.AsSpan(4, 4), Crc32.Compute(payload));
        payload.CopyTo(frame, 8);
        writer.Write(frame);
    }

    /// <summary>Decode the frame at pos, returning the entry and the next frame position.</summary>
    public static (MemTable.Entry Entry, int Next) ReadFrame(byte[] body, int pos)
    {
        uint len = BinaryPrimitives.ReadUInt32LittleEndian(body.AsSpan(pos, 4));
        uint crc = BinaryPrimitives.ReadUInt32LittleEndian(body.AsSpan(pos + 4, 4));
        var payload = body.AsSpan(pos + 8, (int)len);
        if (Crc32.Compute(payload) != crc)
            throw new InvalidDataException($"entry at offset {pos} failed its crc");
        return (Decode(payload), pos + 8 + (int)len);
    }
```

listing 1644: one entry as len, crc, payload, the codec both sides share (source: dsa/capstone/src/Engine/EntryCodec.cs, lines 11-31)

The cache is chapter 25's LRU in its teaching shape, a dictionary for the hit test and a doubly linked list for recency: a hit moves the node to the head, an insert past capacity evicts from the tail. The engine owns one cache across every segment and rebuilds it when compaction retires the old paths:

```
    /// <summary>The decoded block, or null on a miss. A hit refreshes recency.</summary>
    public List<MemTable.Entry>? Get(string sstPath, int blockIndex)
    {
        if (!_nodes.TryGetValue((sstPath, blockIndex), out Node? node))
        {
            Misses++;
            return null;
        }
        Hits++;
        Unlink(node);
        LinkFront(node);
        return node.Entries;
    }

    /// <summary>Insert a decoded block. Blocks are immutable, so inserting a
    /// key that is already resident only refreshes its recency.</summary>
    public void Put(string sstPath, int blockIndex, List<MemTable.Entry> entries)
    {
        if (_nodes.TryGetValue((sstPath, blockIndex), out Node? node))
        {
            Unlink(node);
            LinkFront(node);
            return;
        }
        var fresh = new Node(sstPath, blockIndex, entries);
        _nodes[(sstPath, blockIndex)] = fresh;
        LinkFront(fresh);
        if (_nodes.Count > _capacity)
            Evict();
    }

    private void Evict()
    {
        Node victim = _tail.Prev!;
        Unlink(victim);
        _nodes.Remove((victim.Path, victim.Block));
```

listing 1645: get and put, move to front, evict from the tail (source: dsa/capstone/src/Engine/BlockCache.cs, lines 37-73)

The tests grew the sparse-index logic honestly: an early version excluded the boundary entry at the next index point and failed on `key016` of a hundred-key run, the off-by-one chapter 14 warned about, caught on real bytes. Blocks brought one documented behavior change: a bit flipped inside an entry frame used to be refused at `Open`, which walked every entry to count them, and is now refused when the block holding it is read. `Open` trusts the footer, the bloom, and the index; the data region earns its trust per block, crc first.

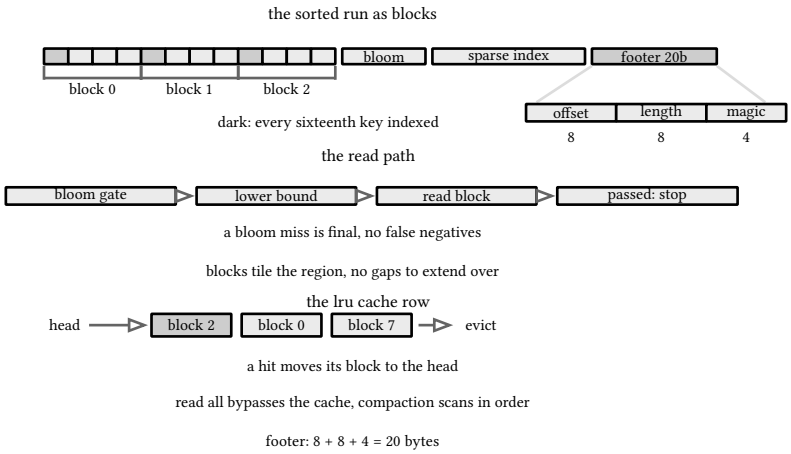


figure 387: the sorted run as one byte map, entries bracketed into blocks by the index points, an lru cache row under the read path

43.5. compaction

Segments accumulate, so compaction merges them: the cursor-array k-way merge from chapter 8, runs given oldest first so the newest version of a key wins, tombstones dropped only when every run is being merged because only then is nothing older below:

```
public static class Compaction
{
    /// <summary>
    /// Merge runs given oldest first into one sorted output. When a key
    /// appears in several runs the version from the newest run, the highest
    /// index, survives. Tombstones survive unless this is a full compaction
    /// of every run, where nothing older exists below them.
    /// </summary>
    public static List<MemTable.Entry> Merge(
        IEnumerable<IEnumerable<MemTable.Entry>> runsOldestFirst, bool dropTombstones)
    {
        var cursors = runsOldestFirst.Select(r => r.GetEnumerator()).ToList();
        try
        {
            var heads = new MemTable.Entry?[cursors.Count];
            for (int i = 0; i < cursors.Count; i++)
                heads[i] = cursors[i].MoveNext() ? cursors[i].Current : null;

            var output = new List<MemTable.Entry>();
            while (true)
            {
                int best = -1;
                for (int i = 0; i < heads.Length; i++)
                {
                    if (heads[i] is null)
                        continue;
                    if (best < 0)
                    {
                        best = i;
                        continue;
                    }
                    int c = ByteArrayComparer.Compare(heads[i]!.Key, heads[best]!.Key);
                    if (c < 0 || c == 0)
                        best = i; // equal keys: the later, newer run wins
```

```
                }
                if (best < 0)
                    break;

                MemTable.Entry winner = heads[best]!;
                byte[] key = winner.Key;
                if (!(dropTombstones && winner.IsTombstone))
                    output.Add(winner);

                // advance every cursor sitting on this key
                for (int i = 0; i < heads.Length; i++)
                {
                    if (heads[i] != null && ByteArrayComparer.Compare(heads[i]!.Key, key) == 0)
                        heads[i] = cursors[i].MoveNext() ? cursors[i].Current : null;
                }
            }
            return output;
        }
        finally
        {
            foreach (IEnumerator<MemTable.Entry> cursor in cursors)
                cursor.Dispose();
        }
    }
}
```

listing 1646: the full merge, heads array, equal-key advancement (source: dsa/capstone/src/Engine/Compaction.cs, lines 4-62)

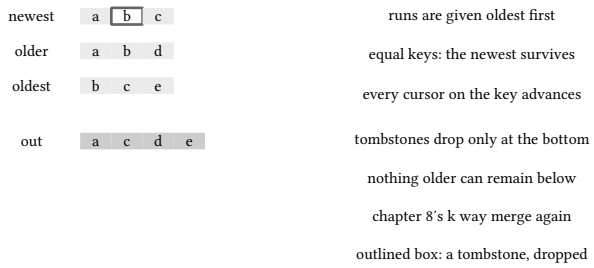


figure 388: compaction, the cursor merge oldest first, equal keys advance every cursor and the newest run wins

The merge itself is only half of compaction. Writing the merged run and then deleting the old segments is a two-step mutation of the directory, and a crash between the steps leaves the tombstone-dropped merge coexisting with stale runs, which glob-order opening reads as newest: a deleted key resurrects. The fix is a manifest, a json list of the live segment files swapped atomically:

```
    public static string PathFor(string dir) => Path.Combine(dir, FileName);

    /// <summary>
    /// The live segment file names, sorted. A missing manifest falls back to
    /// the legacy glob of every .sst, so directories written before the
    /// manifest existed, and standalone sstable tests with no manifest at
    /// all, keep working.
    /// </summary>
    public static List<string> Load(string dir)
    {
        string path = PathFor(dir);
        if (!File.Exists(path))
            return Directory.GetFiles(dir, "*.sst")
                .Select(Path.GetFileName).Order().ToList();

        using JsonDocument doc = JsonDocument.Parse(File.ReadAllBytes(path));
        return doc.RootElement.EnumerateArray()
            .Select(e => e.GetString()!).ToList();
    }

    /// <summary>
    /// Atomically replace the live set: serialize into a tmp file, fsync it,
    /// rename over the real name, then sync the directory entry.
    /// </summary>
    public static void Save(string dir, IEnumerable<string> files)
    {
        string tmp = PathFor(dir) + ".tmp";
        using (var writer = new FileStream(
            tmp, FileMode.Create, FileAccess.Write, FileShare.Read))
        {
            JsonSerializer.Serialize(writer, files.ToList());
            writer.Flush(flushToDisk: true);
        }
        File.Move(tmp, PathFor(dir), overwrite: true);
        DirSync(dir);
    }
```

listing 1647: load with the legacy glob fallback, save as a fsynced tmp plus rename (source: dsa/capstone/src/Engine/Manifest.cs, lines 15-50)

Compaction writes the merged run durably, saves the manifest as a tmp file that is fsynced and renamed over the old one, and only then deletes the retired segments. A crash anywhere in the sequence leaves at worst ignored garbage, never two competing truths. The regression test plants a pre-compaction segment back under a stale name and asserts the deleted key stays deleted.

43.6. the engine

The orchestrator wires the pieces into the lsm shape: writes hit the wal then the memtable, flushes cut segments, reads walk the memtable then segments newest first, and reopening replays the wal. Opening reads segments through the manifest, so any `.sst` outside the live list is garbage from an interrupted swap, never a run to merge. Three knobs shape the machine, the memtable flush size at 256 KiB, the compaction threshold at 4 segments, the block cache at 64 blocks, and the threshold bounds the segment count with no caller discipline: a flush that brings the count to the threshold merges inside the same call:

```
public sealed class Engine : IDisposable
{
```

```
    private readonly string _dir;
    private readonly Wal _wal;
    private readonly int _memtableFlushBytes;
    private readonly int _compactionThreshold;
    private readonly int _blockCacheBlocks;
    private readonly List<SSTable> _segments = new(); // oldest first
    private BlockCache _cache;
    private MemTable _memtable = new();
    private long _sequence;
    private int _nextSegmentId = 1;

    public Engine(string dir, int memtableFlushBytes = 256 * 1024,
        int compactionThreshold = 4, int blockCacheBlocks = 64)
    {
        _dir = dir;
        _memtableFlushBytes = memtableFlushBytes;
        _compactionThreshold = compactionThreshold;
        _blockCacheBlocks = blockCacheBlocks;
        _cache = new BlockCache(blockCacheBlocks);
        System.IO.Directory.CreateDirectory(dir);

        // the manifest names the live set; any .sst outside it is garbage
        // from an interrupted swap, never a segment to read
        foreach (string file in Manifest.Load(dir))
        {
            _segments.Add(SSTable.Open(Path.Combine(dir, file), _cache));
            _nextSegmentId = Math.Max(_nextSegmentId, SegmentId(file) + 1);
        }

        _wal = Wal.Open(Path.Combine(dir, "wal.log"));
        foreach (WalRecord record in Wal.ReadAll(_wal.Path))
        {
            Apply(record);
        }
    }

    public int SegmentCount => _segments.Count;

    public long WalRecords => _wal.RecordCount;

    public long MemTableBytes => _memtable.SizeBytes;

    public long MemTableCount => _memtable.Count;

    public string DirectoryPath => _dir;

    public string ManifestPath => Manifest.PathFor(_dir);

    public void Put(byte[] key, byte[] value)
    {
        _wal.Append(WalRecord.Put(key, value));
        _memtable.Put(key, value, ++_sequence);
        if (_memtable.SizeBytes >= _memtableFlushBytes)
            Flush();
    }

    public void Delete(byte[] key)
```

```
    {
        _wal.Append(WalRecord.Delete(key));
        _memtable.Put(key, null, ++_sequence);
    }

    public byte[]? Get(byte[] key)
    {
        MemTable.Entry? live = _memtable.Get(key);
        if (live != null)
            return live.IsTombstone ? null : live.Value;

        // newest segments are last in the list, walk backward
        for (int i = _segments.Count - 1; i >= 0; i--)
        {
            MemTable.Entry? hit = _segments[i].Get(key);
            if (hit != null)
                return hit.IsTombstone ? null : hit.Value;
        }
        return null;
    }

    /// <summary>All live keys starting with the prefix, sorted, newest value for each.</summary>
    public IEnumerable<(byte[] Key, byte[] Value)> Scan(byte[] prefix)
    {
        // every run seeks to the prefix, chapter 14's lower bound streaming,
        // so nothing below the range is read; merge order stays oldest first
        // with the memtable last for its versions to win
        var runs = new List<IEnumerable<MemTable.Entry>>(_segments.Select(s => s.Seek(prefix)))
        {
            _memtable.SortedFrom(prefix),
        };
        foreach (MemTable.Entry e in Compaction.Merge(runs, dropTombstones: false))
        {
            if (!e.Key.AsSpan().StartsWith(prefix))
                break; // the merged stream is sorted and past the prefix end
            if (!e.IsTombstone)
                yield return (e.Key, e.Value!);
        }
    }
```

listing 1648: open with recovery, put and get through the level stack, prefix scan (source: dsa/capstone/src/Engine/Engine.cs, lines 4-101)

```
    /// <summary>Move the memtable into a fresh sstable and truncate the wal.</summary>
    public void Flush()
    {
        if (_memtable.Count == 0)
            return;
        _segments.Add(SSTable.Create(SegmentPath(_nextSegmentId++), _memtable.Sorted(), _cache));
        // the manifest names the widened live set before the wal resets, so
        // the worst crash here leaves a wal replaying onto its own segment
        Manifest.Save(_dir, _segments.Select(s => Path.GetFileName(s.Path)));
        _memtable = new MemTable();
        _wal.Reset();
        if (_segments.Count >= _compactionThreshold)
            Compact(); // the threshold bounds the count with no caller discipline
    }
```

```
    /// <summary>Full compaction: one merge over every segment, tombstones dropped,
    /// swapped in atomically through the manifest.</summary>
    public void Compact()
    {
        if (_segments.Count < 2)
            return;

        List<MemTable.Entry> merged = Compaction.Merge(
            _segments.Select(s => s.ReadAll()), dropTombstones: true);

        string compacted = SegmentPath(_nextSegmentId++);
        // durable via WriteRun's flushToDisk; the throwaway handle is closed
        // here, the table is reopened below with the fresh cache
        SSTable.Create(compacted, merged).Dispose();

        // the swap: from here the merged run is the only truth. a crash before
        // the deletes leaves ignored garbage behind, and a stale run planted
        // back cannot resurrect what the merge dropped
        Manifest.Save(_dir, new[] { Path.GetFileName(compacted) });

        foreach (SSTable old in _segments)
        {
            old.Dispose();
            File.Delete(old.Path);
        }
        Manifest.DirSync(_dir);
        _segments.Clear();
        _cache = new BlockCache(_blockCacheBlocks); // the retired paths are gone
        _segments.Add(SSTable.Open(compacted, _cache));
    }

    private string SegmentPath(int id) => Path.Combine(_dir, $"{id:D6}.sst");

    private static int SegmentId(string path) =>
        int.TryParse(Path.GetFileNameWithoutExtension(path).AsSpan(0, 6), out int id) ? id : 0;

    private void Apply(WalRecord record)
    {
        _sequence++;
        _memtable.Put(record.Key, record.Value, _sequence);
    }

    public void Dispose()
    {
        foreach (SSTable segment in _segments)
            segment.Dispose();
        _wal.Dispose();
    }
}
```

listing 1649: flush cuts a segment and widens the manifest, the threshold merge swaps it atomically (source: dsa/capstone/src/Engine/Engine.cs, lines 103-166)

The scan merges every level oldest first with the memtable last, so fresh writes and deletes win, a run-order bug the tests caught when a deleted key resurrected from an older segment. Every run now seeks to the prefix first, so a scan reads nothing below the range and stops at the first key past it.

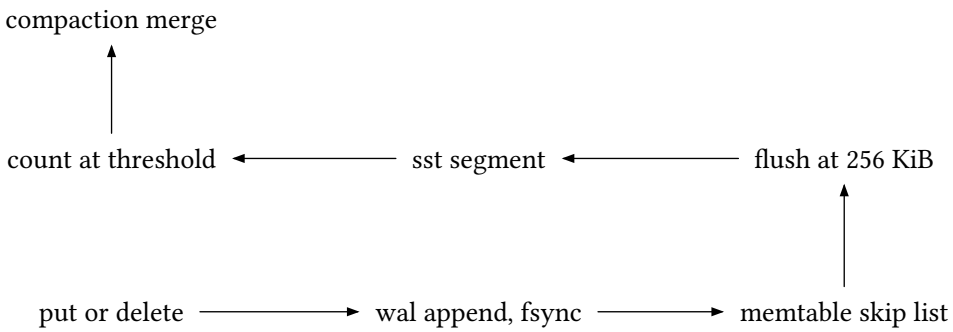


figure 389: the write path through the engine, a segment count at the threshold folds back into a merge

43.7. cross-examined against sqlite

A suite written against the engine alone can share the engine's blind spots, so a comparator suite answers every question twice: 512 keys and values from pure functions of the index load into both this engine and sqlite 3.53.4 through `Microsoft.Data.Sqlite` with the `SQLitePCLRaw.bundle_e_sqlite3` pin, and every answer the two stores can disagree on is asserted identical, point reads over the full keyspace, negative lookups on both sides, deletes and overwrites, prefix scans against an ordered select over blob keys, and a full reopen of both. The engine version itself is asserted, and the per-side clocks are printed to the test output, never asserted, because timings describe the machine more than the code:

```
    /// <summary>Replays the deterministic op stream through both engines: every
    /// deleteEvery-th key is a delete, the rest puts, orderKeys other-prefix keys
    /// added on top, one engine flush right after row flushAfter. Returns load
    /// milliseconds per side, informational only.</summary>
    private (long LsmMs, long SqlMs) LoadBoth(
        Engine engine, SqliteConnection conn, int count,
        int deleteEvery = 0, int flushAfter = -1, int orderKeys = 0)
    {
        const string upsert =
            "INSERT INTO kv(k, v) VALUES(@k, @v) ON CONFLICT(k) DO UPDATE SET v = excluded.v";
        var sw = Stopwatch.StartNew();
        for (int i = 0; i < count; i++)
        {
            if (deleteEvery > 0 && i % deleteEvery == 0)
                engine.Delete(Key(i));
            else
                engine.Put(Key(i), Value(i));
            if (i < orderKeys)
                engine.Put(OrderKey(i), Value(count + i));
            if (i == flushAfter)
                engine.Flush();
        }
        sw.Stop();
        long lsmMs = sw.ElapsedMilliseconds;

        sw.Restart();
        for (int i = 0; i < count; i++)
        {
            if (deleteEvery > 0 && i % deleteEvery == 0)
                SqlExec(conn, "DELETE FROM kv WHERE k = @k", ("@k", Key(i)));
            else
                SqlExec(conn, upsert, ("@k", Key(i)), ("@v", Value(i)));
            if (i < orderKeys)
                SqlExec(conn, upsert, ("@k", OrderKey(i)), ("@v", Value(count + i)));
        }
        sw.Stop();
        return (lsmMs, sw.ElapsedMilliseconds);
    }
```

listing 1650: one deterministic op stream replayed through both engines, per-side clocks (source: dsa/capstone/tests/Engine.Tests/SqliteComparatorTests.cs, lines 134-171)

Determinism can only walk the paths its author imagined, so a fuzz suite adds the paths nobody would: three seeded random op streams, 400 ops over a 64 key space each, weighted puts and deletes with a flush every twenty-five ops, a compaction every hundred, and a dispose-and-reopen every hundred fifty, mirrored op for op into the file-backed sqlite. Every fifty ops the whole keyspace is probed on both sides and a full engine scan is compared against an ordered select over blob keys. A disagreeing seed is a reproducer, the whole point of seeding:

```
    private void Run(int seed)
    {
        Directory.CreateDirectory(_dir);
        var rng = new Random(seed);
        using SqliteConnection mirror = OpenSqlite();

        var engine = new Engine(_dir);
        for (int op = 1; op ≤ Ops; op++)
        {
            byte[] key = Key(rng.Next(Keys));
            if (rng.Next(100) < 75)
            {
                byte[] value = Value(rng.Next(1_000_000) + op);
                engine.Put(key, value);
                SqlExec(mirror, Upsert, ("@k", key), ("@v", value));
            }
            else
            {
                engine.Delete(key);
                SqlExec(mirror, "DELETE FROM kv WHERE k = @k", ("@k", key));
            }

            if (op % 25 == 0)
                engine.Flush(); // segments pile up between the checkpoints
            if (op % 100 == 0)
                engine.Compact();
            if (op % 150 == 0)
            {
                engine.Dispose();
                engine = new Engine(_dir); // crash and reopen: recovery must agree too
            }

            if (op % 50 == 0)
            {
                AssertPointsAgree(engine, mirror);
                AssertScanAgrees(engine, mirror);
            }
        }
        engine.Dispose();
    }
```

listing 1651: one seeded op stream, flushes, compactions, reopens, checkpoints every fifty ops (source: dsa/capstone/tests/Engine.Tests/SqliteFuzzTests.cs, lines 33-72)

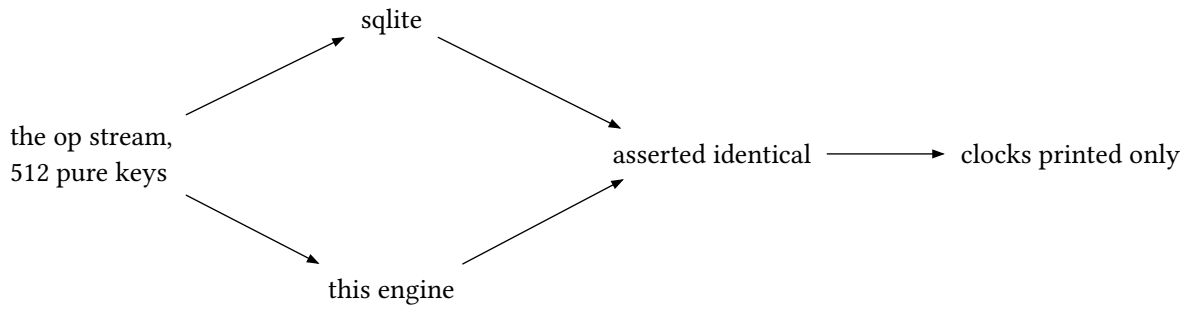


figure 390: the comparator, one stream through two engines, answers asserted identical, clocks only printed

43.8. htmx, taught before it is used

The dashboard needs interactivity, and the tool chosen for it is htmx 4, pinned at 4.0.0 from the npm registry and vendored into `wwwroot/htmx.min.js`. The vendored file's SHA-384 base64 digest is `BvJpBiO8Kh31EqtJe5DRIeWrHWnCGkwytKs9NKFi86Hhw96dEqdEMzZDeK9iEGTc`, byte identical to the package's published CDN integrity value, so the local copy is the published artifact, not a transcription. The npm `latest` tag still points at 2.0.10, 4.0.0 rides the `next` tag, which is worth knowing before pinning.

The whole idea fits in five attributes. `hx-get` and `hx-post` issue an ajax request to a url:

```
<button hx-post="/compact" hx-swap="none">compact segments</button>
```

fragment, illustrative, not executed

`hx-target` picks where the response goes, a css selector, and `hx-swap` picks how it lands, `innerHTML` the default, and `outerHTML`, `afterbegin`, `beforebegin`, `beforeend`, `afterend`, `delete`, or `none`. `hx-trigger` replaces the default event, click for buttons and submit for forms, and its modifiers carry real logic: `every 2s` polls, `delay:`, `throttle:`, `changed`, `once`:

```
<section id="stats" hx-get="/stats" hx-trigger="every 2s" hx-swap="innerHTML">
  <!-- the server re-renders these cards every two seconds -->
</section>
```

fragment, illustrative, not executed

The last piece is `hx-swap-oob`, out of band swaps. A response can carry elements that update targets elsewhere on the page: htmx takes any element with `hx-swap-oob` and swaps it into the element with the matching id, while the rest of the response goes to the original target. That is how the compact button refreshes the stats without touching anything else.

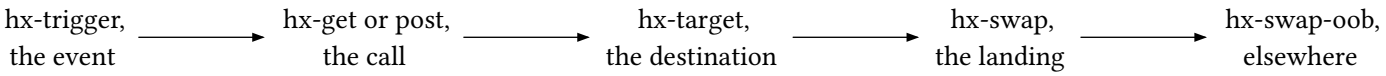


figure 391: the five htmx attributes as one request machine

One language note from building the page: the html lives in interpolated raw strings, and a single-`$` raw string rejects two consecutive braces as content, so a page holding a css block needs the `$$` form, literal braces plain and interpolations doubled. The compile error is `CS9006`, found the honest way.

43.9. the dashboard

An ASP.NET Core minimal api serves fragments, one endpoint per interaction, every body built by the `Ui` class:

```
app.MapGet("/", () => Results.Content(
    Ui.Page("storage engine dashboard", hosted.Recent()), "text/html"));

app.MapGet("/stats", () => Results.Content(StatsFragment(), "text/html"));

app.MapGet("/get", (string key) => Results.Content(
    Ui.Lookup(key, hosted.Get(key)), "text/html"));

app.MapPost("/put", async (HttpRequest request) =>
{
    IFormCollection form = await request.ReadFormAsync();
    string key = form["key"].ToString();
    string value = form["value"].ToString();
    if (key.Length == 0)
        return Results.BadRequest("key is required");
    hosted.Put(key, value);
    return Results.Content(Ui.TailRow("put", key), "text/html");
});

app.MapPost("/delete", async (HttpRequest request) =>
{
    IFormCollection form = await request.ReadFormAsync();
    string key = form["key"].ToString();
    if (key.Length == 0)
        return Results.BadRequest("key is required");
    hosted.Delete(key);
    return Results.Content(Ui.TailRow("delete", key), "text/html");
});

// compaction returns the fresh stats as an out-of-band swap: the caller's
// button swaps nothing itself, the #stats element in the response updates
app.MapPost("/compact", () =>
{
    hosted.Store.Compact();
    return Results.Content(
        $"""<div id="stats" hx-swap-oob="innerHTML">{StatsFragment()}</div>""",
        "text/html");
});

app.Run();

public partial class Program { }
```

listing 1652: the endpoints: stats poll, lookup, put and delete forms, oob compact (source: dsa/capstone/src/Dashboard/Program.cs, lines 35-76)

```
    /// <summary>The stats cards. Served inside the page and polled every 2 seconds.</summary>
    public static string Stats(string dir, int segments, long walRecords, long memBytes) => $"""
        <div class="stat"><b>{segments}</b>segments</div>
        <div class="stat"><b>{walRecords}</b>wal records</div>
        <div class="stat"><b>{memBytes / 1024.0:F1}</b>KiB memtable</div>
        <div class="stat"><b>{Escape(dir)}</b>data dir</div>
        """;

    /// <summary>One tail row. The put and delete forms target #tail-row with afterbegin.</summary>
    public static string TailRow(string kind, string key) => $"""
        <tr><td>now</td><td>{Escape(kind)}</td><td>{Escape(key)}</td></tr>
        """;

    /// <summary>The server-rendered recent rows of the initial page, oldest first.</summary>
    public static string RecentRows(IEnumerable<EngineHosted.Receipt> receipts)
    {
        var rows = new StringBuilder();
        foreach (EngineHosted.Receipt r in receipts)
            rows.Append(
                $"<tr class=\"recent\"><td>{r.Sequence}</td><td>{Escape(r.Kind)}</td><td>{Escape(r.Key)}</
td></tr>\n");
        return rows.ToString();
    }

    /// <summary>Lookup answer for the get form.</summary>
    public static string Lookup(string key, string? value) => value is null
        ? $"<code>{Escape(key)}</code> is not present"
        : $"<code>{Escape(key)}</code> = <code>{Escape(value)}</code>";

    public static string Escape(string s) => HtmlEncoder.Default.Encode(s);
```

listing 1653: the fragment builders: stats cards, tail row, server-rendered recent rows, lookup answer (source: dsa/capstone/src/Dashboard/EngineHosted.cs, lines 128-157)

The put and delete forms return a table row targeted at `#tail-row` with `afterbegin`, so the write log grows in place. The stats section polls every two seconds. The compact button posts and swaps `none` for itself while the response's out-of-band stats block updates the cards.

The recent-writes ring earned its keep the honest way: chapter 5's `Ring<T>` sat in the host as dead state for a whole draft, enqueued on every write and read by nothing, until the overhaul made `GET /` render the last sixteen receipts server-side so the page shows real history before any htmx call fires. The ring grows past its capacity rather than wrapping, so the host enforces the bound itself, evicting the oldest receipt before the newest lands, and because the ring exposes only enqueue and dequeue, the snapshot rotates it once, dequeuing sixteen and re-enqueueing the same sixteen, to read without destroying.

endpoint	verb	returns	target	swap
/stats	get	cards	#stats	innerHTML
/get	get	answer	its form	innerHTML
/put	post	a row	#tail-row	afterbegin
/delete	post	a row	#tail-row	afterbegin
/compact	post	oob stats	none	oob

stats polls every 2s, writes grow the tail row in place

compact swaps none itself, its oob block updates stats

each body is a fragment, the ring is chapter 5's

figure 392: the endpoint wiring as a matrix, what each route returns, where it lands, and how

Integration tests run the real host through `WebApplicationFactory`, per-test data directories via the `DSA_DATA_DIR` setting, and assert what htmx will actually see: the page carries the script tag and the wired attributes, the dist file serves with `var htmx=` at its start and its bytes hash to the published sha384 digest, form posts return row fragments with the right content type, values are html escaped, a put is visible to a later get, deletes remove keys, and data survives a full host restart, and the recent-writes ring renders its sixteen rows server-side and wraps at seventeen. Fourteen dashboard tests over sixty-one engine tests, zero skipped.

The honest boundaries, stated rather than hidden: one engine instance per data directory, single threaded, the file-level locking is advisory. Reads are block oriented through the lru cache now, no longer whole files in memory, though a block-decode is still a full copy, right for the sizes this engine teaches at. Every wal append fsyncs, and every flush fsyncs its segment before the wal resets, durable and deliberately slow, milliseconds a write on the dev machine. Compaction is crash safe through the manifest and threshold triggered, but still full, not leveled, and nothing is compressed anywhere. Every one of those lines is where a production engine, rocksdb and its kin, spends its complexity budget, and each is now a labeled door rather than a mystery.

sources: htmx.org docs pages for attributes, swapping, triggers, and the reference, the npm registry entry for `htmx.org@4.0.0` with its published integrity digest, learn.microsoft.com minimal apis fundamentals, `WebApplicationFactory`, `FileStream.Flush(Boolean)`, `System.Text.Encodings.Web`, accessed 2026-09-08, learn.microsoft.com `Microsoft.Data.Sqlite` parameters, `SqliteDataReader`, and `ExecuteScalar`, accessed 2026-09-10. Verified by `make verify-csharp`, 75 capstone tests plus the sample suite.

44. appendices: topic matrix and sources

The topic matrix is this book's audit trail, and the overhaul made it seven audit trails. Every structure and algorithm the chapters build now exists once per language, built from scratch behind the same fixtures and the same anchors, and every chapter listing reads a real file that the pinned toolchain of its language compiles and runs. The totals: c 222 files and 4140 checks, go 724 tests, java 222 files and 4269 checks, c# 784 tests, javascript 755 tests, python 222 files and 3007 asserts, lua 936 checks, and the c# capstone of chapter 43 stays single-language inside the same suite. Zero skipped anywhere.

44.1. topic coverage

Every topic maps to a chapter: 0 unmapped. The bcl column carries the honest boundary of the original c# wave, several rows say `n/a` because the .NET base class library genuinely ships no counterpart, no deque, no skip list, no dijkstra, and knowing that is as practical as knowing the ones it does ship.

topic	chapter	cost	bcl counterpart
op counting and allocation metering	ch 1	n/a instrument	BenchmarkDotNet
binary search probe bound	ch 1	O(log n)	Array.BinarySearch
row-major layout, cache line model	ch 2	O(1) index	Span<T>
bounds-checked access, failure modes	ch 2	O(1)	IndexOutOfRangeException
dynamic array, doubling growth	ch 3	O(1) amortized add	List<T>
front insertion cost	ch 3	O(n)	List<T>.Insert
singly linked list, in-place reversal	ch 4	O(n) time O(1) space	LinkedList<T>
floyd cycle detection	ch 4	O(n) time O(1) space	n/a
sentinel doubly linked ring	ch 4	O(1) remove by node	LinkedList<T>
array stack	ch 5	O(1) push pop	Stack<T>
circular buffer queue	ch 5	O(1) enqueue dequeue	Queue<T>
deque via ring or sentinels	ch 5	O(1) both ends	none in bcl
bracket matching, shunting yard	ch 5	O(n)	n/a
fnv-1a hash	ch 6	O(k) per key	XxHash3
open addressing, linear probing, tombstones	ch 6	O(1) avg	Dictionary<K,V>
separate chaining	ch 6	O(1) avg	n/a in bcl
hash set over the probe core	ch 6	O(1) avg	HashSet<T>
binary search tree, three-case delete	ch 7	O(h)	SortedDictionary<K,V>
avl rotations	ch 7	O(log n)	SortedSet<T>
inorder, preorder, postorder, level order	ch 7	O(n)	n/a
binary heap, sift up and down	ch 8	O(log n)	PriorityQueue<E,P>
heapsort	ch 8	O(n log n)	Array.Sort path
k-way merge	ch 8	O(n log k)	n/a
decrease-key emulation	ch 8	O(n) scan	PriorityQueue.Remove
top-k with a bounded heap	ch 8	O(n log k)	n/a
byte trie, prefix queries	ch 9	O(k)	n/a
suffix array, kasai lcp	ch 9	build O(n^2 log n) here	n/a
substring search via suffix array	ch 9	O(m log n)	IndexOf
adjacency list graph	ch 10	O(V+E) storage	n/a
bfs with distances, dfs with stamps	ch 10	O(V+E)	n/a

topic	chapter	cost	bcl counterpart
kahn topological sort	ch 10	O(V+E)	n/a
back edge cycle detection	ch 10	O(V+E)	n/a
undirected components	ch 10	O(V+E)	n/a
dijkstra with indexed heap	ch 11	O(E log V)	n/a
bellman-ford, negative cycle check	ch 11	O(V E)	n/a
floyd-warshall	ch 11	O(V^3)	n/a
a star with admissible heuristic	ch 11	O(E log V)	n/a
0-1 bfs, deque as the sorted frontier	ch 11	O(V+E)	n/a
d'esopo-pape worklist refinement	ch 11	exponential worst, usually fast	n/a
union-find, compression and rank	ch 12	O(alpha) amortized	n/a
kruskal, prim	ch 12	O(E log E) / O(E log V)	n/a
edmonds-karp max flow	ch 12	O(V E^2)	n/a
max-flow min-cut	ch 12	verified by test	n/a
insertion, binary insertion	ch 13	O(n^2) / adaptive	Array.Sort finisher
merge sort with inversion count	ch 13	O(n log n) stable	OrderBy
quicksort, median of three, hoare	ch 13	O(n log n) avg	Array.Sort
counting sort on small keys	ch 13	O(n + k)	n/a
stability under equal keys	ch 13	O(n log n) stable	OrderBy
quickselect, median of three	ch 13	expected O(n)	n/a
lower and upper bound	ch 14	O(log n)	Array.BinarySearch
rotated array search	ch 14	O(log n)	n/a
binary search on the answer	ch 14	O(n log range)	n/a
kadane maximum subarray, index recovery	ch 14	O(n)	n/a
mex, minimal excluded value and prefix table	ch 14	O(n)	n/a
naive matcher with comparison meter	ch 15	O(n m)	n/a
kmp with failure table	ch 15	O(n + m)	n/a
boyer-moore-horspool	ch 15	O(n m) worst, sublinear avg	n/a
rabin-karp rolling hash	ch 15	O(n + m) avg	n/a
aho-corasick multi pattern	ch 15	O(n + hits)	n/a
gcd and lcm, euclid and stein	ch 16	O(log min)	n/a
sieve of eratosthenes, p squared start	ch 16	O(n log log n)	n/a
trial division factorization	ch 16	O(sqrt n)	n/a
modular exponentiation, square and multiply	ch 16	O(log e) mults	BigInteger.ModPow
modular inverse via bezout	ch 16	O(log m)	n/a
chinese remainder theorem	ch 16	O(k log m)	BigInteger
memoization and tabulation	ch 17	calls metered	n/a
grid paths with obstacles	ch 17	O(rows cols)	n/a
0/1 knapsack with reconstruction	ch 17	O(n W)	n/a
lcs, edit distance	ch 17	O(n m)	n/a
coin change	ch 17	O(n amount)	n/a
longest increasing subsequence, parents and tails	ch 17	O(n^2) / O(n log n)	n/a
bitmask assignment	ch 17	O(2^n n)	n/a

topic	chapter	cost	bcl counterpart
activity selection, rooms	ch 18	O(n log n)	n/a
huffman coding	ch 18	O(n log n)	n/a
fractional knapsack	ch 18	O(n log n)	n/a
jump game, greedy reach	ch 18	O(n)	n/a
n-queens, subset sum backtracking	ch 18	exponential, pruned	n/a
bounded knapsack, binary decomposition and the monotone window	ch 19	O(n W), one consult per cell	n/a
grouped knapsack, one option per group	ch 19	O(groups x options x W)	n/a
convex hull trick transitions	ch 19	amortized O(1) per line	n/a
longest increasing subsequence, tails	ch 19	O(n log n)	n/a
digit dp with tight flags	ch 19	O(digits states)	n/a
interval dp on spans	ch 19	O(n^2) states, O(n) cut	n/a
memoize versus prune, the pruning map	ch 20	problem shaped	n/a
n-queens with flag arrays	ch 20	exponential, pruned	n/a
branch and bound knapsack	ch 20	bound kills branches	n/a
alpha-beta pruning	ch 20	O(b^d) worst, pruned avg	n/a
prefix sums and summed area	ch 21	O(1) range query	n/a
fenwick tree, point add and prefix sum	ch 21	O(log n), reads = popcount	n/a
lazy segment tree, range add and sum	ch 21	O(log n) per op	n/a
sparse table, idempotent range min	ch 21	O(n log n) build, O(1) query	n/a
sliding window maxima, monotonic deque	ch 21	O(n)	LinkedList as the deque
next greater both sides, monotonic stack	ch 21	O(n)	n/a
coordinate compression	ch 22	O(n log n)	n/a
merge intervals, half-open	ch 22	O(n log n)	n/a
overlap counting by sweep	ch 22	O(n log n)	n/a
difference array, range add point read	ch 22	O(1) add, O(n) settle	n/a
orientation predicate, exact integer cross	ch 23	O(1)	n/a
andrew monotone chain hull	ch 23	O(n log n)	n/a
closest pair, strip merge in y	ch 23	O(n log^2 n) as built	n/a
shamos-hoey sweep, neighbor checks	ch 23	O(n log n) with a tree	n/a
shoelace area with pick interior	ch 24	O(b) boundary walk	n/a
ray casting point in polygon	ch 24	O(b) exact integer	n/a
flood fill components	ch 24	O(cells)	n/a
grid neighborhoods, 4 and 8 way	ch 24	O(1) per neighbor	n/a
bloom filter fundamentals	ch 25	O(k) probes, 1 percent design	n/a
count-min sketch	ch 25	O(d) update and query	n/a
lru and lfu caches	ch 25	O(1) get and put	MemoryCache approximate, GetOrCreate
skip list fundamentals	ch 25	O(log n) expected	SortedSet for the ordered set role
lexer token stream	ch 26	O(n)	n/a

topic	chapter	cost	bcl counterpart
recursive descent parser	ch 26	O(n)	n/a
ast print and evaluate	ch 26	O(n) per pass	n/a
opcode dispatch loop	ch 27	O(program)	n/a
parameter modes, position and immediate	ch 27	O(1) decode	n/a
jumps and the instruction pointer	ch 27	O(1) retarget	n/a
linear sieve, least prime factor, phi and mu tables	ch 28	O(n) once	n/a
deterministic miller-rabin, pollard rho factorization	ch 28	O(w log^2 n), rho ~O(n^0.25) expected	n/a
divisor count and sum sieve, phi identities, pillai gcd-sum	ch 28	O(n log n) table	n/a
baby-step giant-step, smallest primitive root	ch 28	O(sqrt p)	n/a
garner mixed-radix crt, factorial mod p, legendre exponent	ch 28	O(k^2 + k log m) rebuild, O(n) table	BigInteger
linear diophantine solve, parametric family, non-negative count	ch 28	O(log min(a,b)) solve	n/a
gray code and inverse, balanced ternary signed digits	ch 28	O(1) / O(log v) digits	n/a
continued fractions, convergents, periodic sqrt, stern-brocot path, farey order	ch 28	O(log q) terms	n/a
dynamic bignum, sign-magnitude 1e9 limbs, schoolbook oracle	ch 29	O(n) add, O(n^2) mul	System.Numerics.BigInteger
karatsuba three-way split, counted leaf work, per-language cuts	ch 29	O(n^1.585)	BigInteger multiply path
dual-prime ntt convolution, crt fold, 2^16 cap	ch 29	O(n log n)	n/a
complex fft plain and packed-split, 2^14 drift cap, 0.25 guard	ch 29	O(n log n)	System.Numerics.Complex
knuth algorithm d division, one-limb fast path, add-back branch	ch 29	O(n * m) limbs	BigInteger.DivRem
polynomial multiply, horner eval, lagrange interpolation, mod remainder	ch 29	O(n log n) / O(n) / O(n^2) / O(n m)	n/a
treap split and merge, lazy add, subtree aggregates, depth-gated rebuild	ch 30	expected O(log n) per op	SortedSet<T> for the ordered role
splay trees, zig zig-zig zig-zag, amortized potential	ch 30	amortized O(log n)	n/a
meldable heaps, randomized coin and skew swap	ch 30	expected/amortized O(log n) meld	n/a
order statistics, fenwick rank/select and implicit treap	ch 30	O(log U) rank, O(log n) select	n/a
sqrt decomposition, block sums and touched meter	ch 31	O(sqrt n) per op	n/a
mo's algorithm, offline distinct count with travel meter	ch 31	O((n+q) sqrt n)	n/a

topic	chapter	cost	bcl counterpart
minimum stack and queue, o(1) oracle at the seam	ch 31	O(1) push pop min	none in bcl
li chao tree, online line container	ch 31	O(log C) insert and query	n/a
disjoint sparse table, o(1) non-idempotent range fold	ch 31	O(n log n) build, O(1) query	n/a
rollback dsu, union by size, unwind stack	ch 31	O(log n) union, O(1) unwind	n/a
offline dynamic connectivity, segment tree over time with rollback dsu	ch 31	O(E log T log V)	n/a
divide and conquer dp, monotone opt windows	ch 32	O(k n log n)	n/a
knuth optimization, interval dp root windows	ch 32	O(n^2) states, O(1) amortized cut	n/a
broken profile domino tilings, largest zero submatrix	ch 32	O(n 2^m), O(rows cols)	n/a
slope trick, lazily-shifted breakpoint heaps	ch 32	O(n log n)	PriorityQueue<TElement,TPriority> as the substrate
value iteration mdp, gauss-seidel vs backward dijkstra	ch 32	geometric contraction per sweep	n/a
small-to-large merging, amortized doubling	ch 32	O(n log n) total moves	n/a
z-function and the separator search	ch 33	O(n) build, O(n+m) search	n/a
manacher, odd and even palindromes	ch 33	O(n)	n/a
suffix automaton, distinct and occurrence queries	ch 33	O(n) build	n/a
suffix tree, edge-compressed	ch 33	O(n^2) build here	n/a
lyndon factorization, least rotation, repetitions	ch 33	duval O(n), repetitions O(n^2) here	n/a
gaussian elimination mod a prime	ch 34	O(n^3)	n/a
fraction-free determinant and rank	ch 34	O(n^3) integer	Matrix4x4.GetDeterminant
cramer on 2x2 and 3x3	ch 34	O(n) per minor	n/a
matrix exponentiation, fibonacci and walk counts	ch 34	O(n^3 log e)	n/a
kirchhoff spanning tree count	ch 34	O(n^3)	n/a
hensel lifting over Z_p^k	ch 34	O(n^3 + k n^2)	n/a
binomial tables: exact, modular, ln-space, p-adic valuation	ch 35	O(n) tables, O(1) query	n/a (BigInteger for exact)
catalan numbers, closed form and convolution recurrence	ch 35	O(1) from tables	n/a
compositions and bounded stars and bars by inclusion-exclusion	ch 35	O(2^k) bound subsets	n/a
inclusion-exclusion: unions, exactly-r, derangements, coprime and squarefree counts	ch 35	O(2^s) subsets	n/a
burnside orbit counting, necklaces, bracelets, cube rotations	ch 35	O(divisors(n)) per necklace	n/a

topic	chapter	cost	bcl counterpart
lexicographic k-combination iteration, next-combination	ch 35	O(k) per step	n/a
sperner maximum antichain, middle layer, dedekind counts	ch 35	O(2^n) layers, brute 2^(2^n) at n <= 4	n/a
frobenius number, semigroup sieve, conductor bound	ch 35	O(p_max^2) sieve	n/a
josephus recurrence, k=2 closed form, elimination order	ch 35	O(n), O(1) at k=2	n/a
sliding puzzle permutation parity	ch 35	O(n^2) inversions	n/a
non-attacking bishops, per-color diagonal dp	ch 35	O(n k)	n/a
bracket sequence count, lexicographic rank and unrank	ch 35	O(n) per query	n/a
labeled graphs total and connected by exponential recurrence	ch 35	O(n^2) modular	n/a
ternary search, integer and real	ch 36	O(log range) probe pairs	n/a
golden-section search, one eval per step	ch 36	O(log range) evals	n/a
newton for roots and reciprocals	ch 36	quadratic convergence	Math.Sqrt
simpson, composite and adaptive	ch 36	adaptive, eps-driven	n/a
simulated annealing	ch 36	prose only	n/a
LP duality and dual weights	ch 36	O(V) per dual eval	n/a
line-circle and circle-circle intersection, common tangents	ch 37	O(1) each	n/a
length of the union of segments	ch 37	O(n log n)	n/a
point in convex polygon, binary fan search	ch 37	O(log n)	n/a
minkowski sum of convex polygons	ch 37	O(n + m)	n/a
rotating calipers, diameter and width	ch 37	O(n) after hull	n/a
half-plane intersection, sort and incremental deque	ch 37	O(n log n)	n/a
minimum enclosing circle, welzl move to front	ch 37	expected O(n)	n/a
voronoi cells by clipping, delaunay edges	ch 37	O(n^2) cells	n/a
bentley-ottmann crossing sweep	ch 37	O((n + k) log n) with a tree	n/a
planar faces and point location	ch 37	O(n log n), prose	n/a
manhattan to chebyshev transform	ch 37	O(n)	n/a
dinic level graph blocking flow	ch 38	O(V^2 E)	n/a
push-relabel with fifo discharge	ch 38	O(V^3)	n/a
minimum-cost flow, successive shortest paths	ch 38	O(F * V * E) SPFA	n/a
flows with lower bounds and demands	ch 38	one max flow	n/a
stoer-wagner global minimum cut	ch 38	O(V^3) / O(V E + V^2 log V)	n/a
kuhn augmenting-path bipartite matching	ch 38	O(V * E)	n/a
hungarian O(n^3) assignment solver	ch 38	O(n^3)	n/a

topic	chapter	cost	bcl counterpart
assignment modeling, forbidden pairs and padding	ch 38	O(n^3)	n/a
hall's theorem, subset and prefix forms	ch 38	O(2^n) / O(n log n) prefix	n/a
strongly connected components and condensation	ch 39	O(V + E)	n/a
2-sat via implication graph scc	ch 39	O(V + E)	n/a
bridges and articulation points, disc and low	ch 39	O(V + E)	n/a
eulerian path and circuit, hierholzer	ch 39	O(V + E)	n/a
lca by binary lifting and offline tarjan	ch 39	O(log n) query, offline linear	n/a
heavy-light decomposition, path max on chains	ch 39	O(log^2 n) query	n/a
centroid decomposition, closest marked queries	ch 39	O(n log n) build, O(log n) query	n/a
functional graphs, rings with hanging trees	ch 39	O(n)	n/a
2-core peel, survivors and boundary	ch 39	O(V + E)	n/a
prufer codes, second-best mst swap	ch 39	O(n log n) build, swap scan	n/a
strong orientation, robbins theorem	ch 39	O(V + E), prose	n/a
local search frame, state neighborhood objective, first improvement	ch 41	O(1) per scanned move	n/a
hill climbing, plateaus, first versus best improvement	ch 41	O(n^2) per scan	n/a
tsp neighborhoods, swap insert 2-opt, delta evaluation	ch 41	O(1) delta per 2-opt move	n/a
simulated annealing, integer metropolis rule, seeded lcg	ch 41	O(iterations)	n/a
cooling schedules, restarts, reheating from best	ch 41	O(iterations)	n/a
tabu search, tenure and move expiry	ch 41	O(n^2) per iteration	n/a
aspiration overrides and candidate lists	ch 41	O(candidates) per iteration	n/a
genetic algorithms, bitstring encoding, roulette and tournament	ch 41	O(pop x len) per generation	n/a
crossover, mutation, order crossover ox	ch 41	O(len) per child	n/a
ant colony optimization, integer pheromone model	ch 41	O(ants x n^2) per iteration	n/a
aco variants, elitist, iteration-best, mmas clamp	ch 41	O(ants x n^2) per iteration	n/a
restart portfolios, anytime budgets, seeded shuffles	ch 41	O(restarts x descent)	n/a
approximation ratio, feasibility plus proven bound, tight families	ch 42	oracle-checked	n/a
vertex cover by greedy maximal matching, 2-approx	ch 42	O(V + E)	n/a
greedy set cover, ln n + 1 harmonic bound	ch 42	O(sets x ground)	n/a

topic	chapter	cost	bcl counterpart
metric tsp double-tree, 2-approx euler shortcut	ch 42	O(n^2)	n/a
christofides 1.5x, odd-set minimum matching	ch 42	matching-dominated	n/a
knapsack density greedy 1/2 and the ptas	ch 42	O(n log n + n^k)	n/a
fptas by profit scaling, dp on scaled sums	ch 42	O(n^2 / eps)	n/a
bin packing, first fit, decreasing, next fit	ch 42	O(n^2) / O(n log n)	n/a
max-cut flip local search, m/2 guarantee	ch 42	O(m x flips)	n/a
inapproximability barriers, reader's map	ch 42	prose	n/a
choosing the surrender, decision table	ch 42	prose	n/a
crc-32 framing	ch 43	O(n)	System.IO.Hashing.Crc32
bloom filter, serialized per run	ch 43	O(k) probes	System.IO.Hashing n/a
write-ahead log, crash recovery	ch 43	O(1) append	n/a
skip list memtable	ch 43	O(log n)	n/a
sstable, index-aligned blocks and lru cache	ch 43	O(log n + block)	n/a
engine, seek scan, threshold compaction	ch 43	read O(levels + block)	n/a
manifest, atomic compaction swap	ch 43	crash safe	n/a
sqlite differential fuzz	ch 43	three seeds	Microsoft.Data.Sqlite
htmx fragments, minimal api	ch 43	n/a	ASP.NET Core

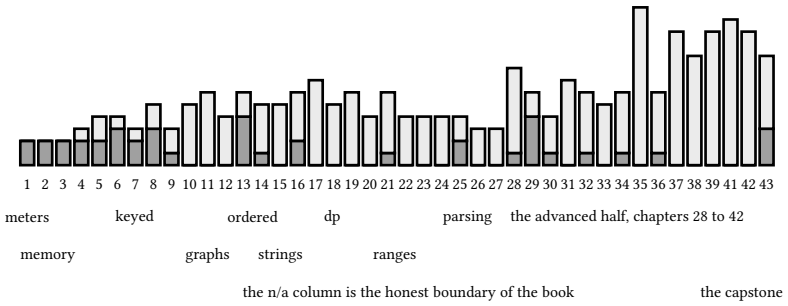


figure 393: the matrix as one strip, one bar per chapter that carries matrix rows, height is topic count, dark is shipped

44.2. the seven sample trees

One row per sample program, 222 of them across chapters 1 to 42, and the file that carries it in each language. The roots, relative to the book directory: c in `dsa/samples-c/src/ChNN`, go in `dsa/samples-go/chNN`, java in `dsa/samples-java/src/ChNN`, c# in `dsa/samples/src/ChNN`, javascript in `dsa/samples-js/src`, python in `dsa/samples-py/src/ChNN`, lua in `dsa/samples-lua`. The c# tree splits most chapters across a few files and groups sibling topics inside one file, so that column sometimes repeats the grouping file, go spells one ch31 file `disjoint_sparse.go` where the other trees say `disjointsparse`, go's ch37 tree shares one `geo.go` helper beside its ten samples, java spells every stem in pascal case so `memo_vs_prune.c` is `MemoVsPrune.java`, and chapter 15's naive matcher lives in its own file beside the frozen original.

topic	ch	c	go	java	c#	javascript	python	lua
allocs	ch 1	allocs.c	allocs.go	Allocs.java	Analysis.cs	ch01-allocs.mjs	allocs.py	ch01_allocs.lua

topic	ch	c	go	java	c#	javascript	python	lua
countops	ch 1	countops.c	countops.go	Countops.java	Analysis.cs	ch01-countops.mjs	countops.py	ch01_countops.lua
probebin	ch 1	probebin.c	probebin.go	Probebin.java	Analysis.cs	ch01-probebin.mjs	probebin.py	ch01_probebin.lua
bounds	ch 2	bounds.c	bounds.go	Bounds.java	Bounds.cs	ch02-bounds.mjs	bounds.py	ch02_bounds.lua
cache	ch 2	cache.c	cache.go	Cache.java	Arrays.cs	ch02-cache.mjs	cache.py	ch02_cache.lua
rowmajor	ch 2	rowmajor.c	rowmajor.go	Rowmajor.java	Arrays.cs	ch02-rowmajor.mjs	rowmajor.py	ch02_rowmajor.lua
spans	ch 2	spans.c	spans.go	Spans.java	Arrays.cs	ch02-spans.mjs	spans.py	ch02_spans.lua
growth	ch 3	growth.c	growth.go	Growth.java	Dynamic.cs	ch03-growth.mjs	growth.py	ch03_growth.lua
vector	ch 3	vector.c	vector.go	Vector.java	Dynamic.cs	ch03-vector.mjs	vector.py	ch03_vector.lua
ring	ch 4	ring.c	ring.go	Ring.java	Linked.cs	ch04-ring.mjs	ring.py	ch04_ring.lua
slist	ch 4	slist.c	slist.go	Slist.java	Linked.cs	ch04-slist.mjs	slist.py	ch04_slist.lua
deque	ch 5	deque.c	deque.go	Deque.java	Deque.cs	ch05-deque.mjs	deque.py	ch05_deque.lua
ringqueue	ch 5	ringqueue.c	ringqueue.go	Ringqueue.java	Buffers.cs	ch05-ringqueue.mjs	ringqueue.py	ch05_ringqueue.lua
shunting	ch 5	shunting.c	shunting.go	Shunting.java	Buffers.cs	ch05-shunting.mjs	shunting.py	ch05_shunting.lua
stack	ch 5	stack.c	stack.go	Stack.java	Buffers.cs	ch05-stack.mjs	stack.py	ch05_stack.lua
chainmap	ch 6	chainmap.c	chainmap.go	Chainmap.java	Hashing.cs	ch06-chainmap.mjs	chainmap.py	ch06_chainmap.lua
fnv	ch 6	fnv.c	fnv.go	Fnv.java	Hashing.cs	ch06-fnv.mjs	fnv.py	ch06_fnv.lua
hashset	ch 6	hashset.c	hashset.go	Hashset.java	HashSet.cs	ch06-hashset.mjs	hashset.py	ch06_hashset.lua
openaddr	ch 6	openaddr.c	openaddr.go	Openaddr.java	Hashing.cs	ch06-openaddr.mjs	openaddr.py	ch06_openaddr.lua
balance	ch 7	balance.c	balance.go	Balance.java	Trees.cs	ch07-balance.mjs	balance.py	ch07_balance.lua
bst	ch 7	bst.c	bst.go	Bst.java	Trees.cs	ch07-bst.mjs	bst.py	ch07_bst.lua
traverse	ch 7	traverse.c	traverse.go	Traverse.java	Trees.cs	ch07-traverse.mjs	traverse.py	ch07_traverse.lua
heap	ch 8	heap.c	heap.go	Heap.java	Heaps.cs	ch08-heap.mjs	heap.py	ch08_heap.lua
heapsort	ch 8	heapsort.c	heapsort.go	Heapsort.java	Heaps.cs	ch08-heapsort.mjs	heapsort.py	ch08_heapsort.lua
kway	ch 8	kway.c	kway.go	Kway.java	Heaps.cs	ch08-kway.mjs	kway.py	ch08_kway.lua
topk	ch 8	topk.c	topk.go	Topk.java	TopK.cs	ch08-topk.mjs	topk.py	ch08_topk.lua
prefixscan	ch 9	prefixscan.c	prefixscan.go	Prefixscan.java	Tries.cs	ch09-prefixscan.mjs	prefixscan.py	ch09_prefixscan.lua
suffixarray	ch 9	suffixarray.c	suffixarray.go	Suffixarray.java	Tries.cs	ch09-suffixarray.mjs	suffixarray.py	ch09_suffixarray.lua
suffixsearch	ch 9	suffixsearch.c	suffixsearch.go	Suffixsearch.java	Tries.cs	ch09-suffixsearch.mjs	suffixsearch.py	ch09_suffixsearch.lua
trie	ch 9	trie.c	trie.go	Trie.java	Tries.cs	ch09-trie.mjs	trie.py	ch09_trie.lua
bfs	ch 10	bfs.c	bfs.go	Bfs.java	Graphs.cs	ch10-bfs.mjs	bfs.py	ch10_bfs.lua
build	ch 10	build.c	build.go	Build.java	Graphs.cs	ch10-build.mjs	build.py	ch10_build.lua
components	ch 10	components.c	components.go	Components.java	Graphs.cs	ch10-components.mjs	components.py	ch10_components.lua
dfs	ch 10	dfs.c	dfs.go	Dfs.java	Graphs.cs	ch10-dfs.mjs	dfs.py	ch10_dfs.lua
topo	ch 10	topo.c	topo.go	Topo.java	Graphs.cs	ch10-topo.mjs	topo.py	ch10_topo.lua
astar	ch 11	astar.c	astar.go	Astar.java	Paths.cs	ch11-astar.mjs	astar.py	ch11_astar.lua
bellman	ch 11	bellman.c	bellman.go	Bellman.java	Paths.cs	ch11-bellman.mjs	bellman.py	ch11_bellman.lua
dijkstra	ch 11	dijkstra.c	dijkstra.go	Dijkstra.java	Paths.cs	ch11-dijkstra.mjs	dijkstra.py	ch11_dijkstra.lua
floyd	ch 11	floyd.c	floyd.go	Floyd.java	Paths.cs	ch11-floyd.mjs	floyd.py	ch11_floyd.lua
dsu	ch 12	dsu.c	dsu.go	Dsu.java	Flows.cs	ch12-dsu.mjs	dsu.py	ch12_dsu.lua
kruskal	ch 12	kruskal.c	kruskal.go	Kruskal.java	Flows.cs	ch12-kruskal.mjs	kruskal.py	ch12_kruskal.lua
maxflow	ch 12	maxflow.c	maxflow.go	Maxflow.java	Flows.cs	ch12-maxflow.mjs	maxflow.py	ch12_maxflow.lua
mincut	ch 12	mincut.c	mincut.go	Mincut.java	Flows.cs	ch12-mincut.mjs	mincut.py	ch12_mincut.lua
prim	ch 12	prim.c	prim.go	Prim.java	Flows.cs	ch12-prim.mjs	prim.py	ch12_prim.lua
counting	ch 13	counting.c	counting.go	Counting.java	Sorting.cs	ch13-counting.mjs	counting.py	ch13_counting.lua
insertion	ch 13	insertion.c	insertion.go	Insertion.java	Sorting.cs	ch13-insertion.mjs	insertion.py	ch13_insertion.lua
mergesort	ch 13	mergesort.c	mergesort.go	Mergesort.java	Sorting.cs	ch13-mergesort.mjs	mergesort.py	ch13_mergesort.lua
quickselect	ch 13	quickselect.c	quickselect.go	Quickselect.java	Sorting.cs	ch13-quickselect.mjs	quickselect.py	ch13_quickselect.lua
quicksort	ch 13	quicksort.c	quicksort.go	Quicksort.java	Sorting.cs	ch13-quicksort.mjs	quicksort.py	ch13_quicksort.lua
stability	ch 13	stability.c	stability.go	Stability.java	Sorting.cs	ch13-stability.mjs	stability.py	ch13_stability.lua
answerspace	ch 14	answerspace.c	answerspace.go	Answerspace.java	Searching.cs	ch14-answerspace.mjs	answerspace.py	ch14_answerspace.lua
bounds	ch 14	bounds.c	bounds.go	Bounds.java	Searching.cs	ch14-bounds.mjs	bounds.py	ch14_bounds.lua
kadane	ch 14	kadane.c	kadane.go	Kadane.java	Searching.cs	ch14-kadane.mjs	kadane.py	ch14_kadane.lua

topic	ch	c	go	java	c#	javascript	python	lua
mex	ch 14	mex.c	mex.go	Mex.java	Searching.cs	ch14-mex.mjs	mex.py	ch14_mex.lua
rotated	ch 14	rotated.c	rotated.go	Rotated.java	Searching.cs	ch14-rotated.mjs	rotated.py	ch14_rotated.lua
aho	ch 15	aho.c	aho.go	Aho.java	Strings.cs	ch15-aho.mjs	aho.py	ch15_aho.lua
horspool	ch 15	horspool.c	horspool.go	Horspool.java	Strings.cs	ch15-horspool.mjs	horspool.py	ch15_horspool.lua
kmp	ch 15	kmp.c	kmp.go	Kmp.java	Strings.cs	ch15-kmp.mjs	kmp.py	ch15_kmp.lua
naive	ch 15	naive.c	naive.go	Naive.java	Naive.cs	ch15-naive.mjs	naive.py	ch15_naive.lua
rabinkarp	ch 15	rabinkarp.c	rabinkarp.go	Rabinkarp.java	Strings.cs	ch15-rabinkarp.mjs	rabinkarp.py	ch15_rabinkarp.lua
crt	ch 16	crt.c	crt.go	Crt.java	NumTheory.cs	ch16-crt.mjs	crt.py	ch16_crt.lua
factors	ch 16	factors.c	factors.go	Factors.java	NumTheory.cs	ch16-factors.mjs	factors.py	ch16_factors.lua
gcdlcm	ch 16	gcdlcm.c	gcdlcm.go	Gcdlcm.java	NumTheory.cs	ch16-gcdlcm.mjs	gcdlcm.py	ch16_gcdlcm.lua
modinv	ch 16	modinv.c	modinv.go	Modinv.java	NumTheory.cs	ch16-modinv.mjs	modinv.py	ch16_modinv.lua
modpow	ch 16	modpow.c	modpow.go	Modpow.java	NumTheory.cs	ch16-modpow.mjs	modpow.py	ch16_modpow.lua
sieve	ch 16	sieve.c	sieve.go	Sieve.java	NumTheory.cs	ch16-sieve.mjs	sieve.py	ch16_sieve.lua
assignment	ch 17	assignment.c	assignment.go	Assignment.java	Dp.cs	ch17-assignment.mjs	assignment.py	ch17_assignment.lua
coins	ch 17	coins.c	coins.go	Coins.java	Dp.cs	ch17-coins.mjs	coins.py	ch17_coins.lua
editdist	ch 17	editdist.c	editdist.go	Editdist.java	Dp.cs	ch17-editdist.mjs	editdist.py	ch17_editdist.lua
gridpaths	ch 17	gridpaths.c	gridpaths.go	Gridpaths.java	Dp.cs	ch17-gridpaths.mjs	gridpaths.py	ch17_gridpaths.lua
knapsack	ch 17	knapsack.c	knapsack.go	Knapsack.java	Dp.cs	ch17-knapsack.mjs	knapsack.py	ch17_knapsack.lua
lis	ch 17	lis.c	lis.go	Lis.java	Dp.cs	ch17-lis.mjs	lis.py	ch17_lis.lua
memo	ch 17	memo.c	memo.go	Memo.java	Dp.cs	ch17-memo.mjs	memo.py	ch17_memo.lua
backtrack	ch 18	backtrack.c	backtrack.go	Backtrack.java	Greedy.cs	ch18-backtrack.mjs	backtrack.py	ch18_backtrack.lua
huffman	ch 18	huffman.c	huffman.go	Huffman.java	Greedy.cs	ch18-huffman.mjs	huffman.py	ch18_huffman.lua
intervals	ch 18	intervals.c	intervals.go	Intervals.java	Greedy.cs	ch18-intervals.mjs	intervals.py	ch18_intervals.lua
jumpgame	ch 18	jumpgame.c	jumpgame.go	Jumpgame.java	JumpGame.cs	ch18-jumpgame.mjs	jumpgame.py	ch18_jumpgame.lua
bitmask	ch 19	bitmask.c	bitmask.go	Bitmask.java	AdvDp.cs	ch19-bitmask.mjs	bitmask.py	ch19_bitmask.lua
bounded	ch 19	bounded.c	bounded.go	Bounded.java	AdvDp.cs	ch19-bounded.mjs	bounded.py	ch19_bounded.lua
cht	ch 19	cht.c	cht.go	Cht.java	AdvDp.cs	ch19-cht.mjs	cht.py	ch19_cht.lua
digitdp	ch 19	digitdp.c	digitdp.go	Digitdp.java	DigitDp.cs	ch19-digitdp.mjs	digitdp.py	ch19_digitdp.lua
grouped	ch 19	grouped.c	grouped.go	Grouped.java	AdvDp.cs	ch19-grouped.mjs	grouped.py	ch19_grouped.lua
intervaldp	ch 19	intervaldp.c	intervaldp.go	Intervaldp.java	IntervalDp.cs	ch19-intervaldp.mjs	intervaldp.py	ch19_intervaldp.lua
tails	ch 19	tails.c	tails.go	Tails.java	AdvDp.cs	ch19-tails.mjs	tails.py	ch19_tails.lua
window	ch 19	window.c	window.go	Window.java	AdvDp.cs	ch19-window.mjs	window.py	ch19_window.lua
alphabeta	ch 20	alphabeta.c	alphabeta.go	Alphabeta.java	Pruning.cs	ch20-alphabeta.mjs	alphabeta.py	ch20_alphabeta.lua
bandb	ch 20	bandb.c	bandb.go	Bandb.java	Pruning.cs	ch20-bandb.mjs	bandb.py	ch20_bandb.lua
memo_vs_prune	ch 20	memo_vs_prune.c	memo_vs_prune.go	MemoVsPrune.java	Pruning.cs	ch20-memo_vs_prune.mjs	memo_vs_prune.py	ch20_memo_vs_prune.lua
nqueens	ch 20	nqueens.c	nqueens.go	Nqueens.java	Pruning.cs	ch20-nqueens.mjs	nqueens.py	ch20_nqueens.lua
fenwick	ch 21	fenwick.c	fenwick.go	Fenwick.java	Ranges.cs	ch21-fenwick.mjs	fenwick.py	ch21_fenwick.lua
maxima	ch 21	maxima.c	maxima.go	Maxima.java	Ranges.cs	ch21-maxima.mjs	maxima.py	ch21_maxima.lua
nextgreater	ch 21	nextgreater.c	nextgreater.go	Nextgreater.java	Ranges.cs	ch21-nextgreater.mjs	nextgreater.py	ch21_nextgreater.lua
prefix	ch 21	prefix.c	prefix.go	Prefix.java	Prefix.cs	ch21-prefix.mjs	prefix.py	ch21_prefix.lua
segtree	ch 21	segtree.c	segtree.go	Segtree.java	Ranges.cs	ch21-segtree.mjs	segtree.py	ch21_segtree.lua
sparsetable	ch 21	sparsetable.c	sparsetable.go	Sparsetable.java	Ranges.cs	ch21-sparsetable.mjs	sparsetable.py	ch21_sparsetable.lua
coordcompress	ch 22	coordcompress.c	coordcompress.go	Coordcompress.java	Compression.cs	ch22-coordcompress.mjs	coordcompress.py	ch22_coordcompress.lua
diffarray	ch 22	diffarray.c	diffarray.go	Diffarray.java	Compression.cs	ch22-diffarray.mjs	diffarray.py	ch22_diffarray.lua
mergeintervals	ch 22	mergeintervals.c	mergeintervals.go	Mergeintervals.java	Compression.cs	ch22-mergeintervals.mjs	mergeintervals.py	ch22_mergeintervals.lua
overlaps	ch 22	overlaps.c	overlaps.go	Overlaps.java	Compression.cs	ch22-overlaps.mjs	overlaps.py	ch22_overlaps.lua
closestpair	ch 23	closestpair.c	closestpair.go	Closestpair.java	Geometry.cs	ch23-closestpair.mjs	closestpair.py	ch23_closestpair.lua
hull	ch 23	hull.c	hull.go	Hull.java	Geometry.cs	ch23-hull.mjs	hull.py	ch23_hull.lua
orientation	ch 23	orientation.c	orientation.go	Orientation.java	Geometry.cs	ch23-orientation.mjs	orientation.py	ch23_orientation.lua
sweep	ch 23	sweep.c	sweep.go	Sweep.java	Geometry.cs	ch23-sweep.mjs	sweep.py	ch23_sweep.lua

topic	ch	c	go	java	c#	javascript	python	lua
floodfill	ch 24	floodfill.c	floodfill.go	Floodfill.java	Lattice.cs	ch24-floodfill.mjs	floodfill.py	ch24_floodfill.lua
neighbors	ch 24	neighbors.c	neighbors.go	Neighbors.java	Lattice.cs	ch24-neighbors.mjs	neighbors.py	ch24_neighbors.lua
raycast	ch 24	raycast.c	raycast.go	Raycast.java	Lattice.cs	ch24-raycast.mjs	raycast.py	ch24_raycast.lua
shoelace	ch 24	shoelace.c	shoelace.go	Shoelace.java	Lattice.cs	ch24-shoelace.mjs	shoelace.py	ch24_shoelace.lua
bloom	ch 25	bloom.c	bloom.go	Bloom.java	Practical.cs	ch25-bloom.mjs	bloom.py	ch25_bloom.lua
countmin	ch 25	countmin.c	countmin.go	Countmin.java	Practical.cs	ch25-countmin.mjs	countmin.py	ch25_countmin.lua
lru	ch 25	lru.c	lru.go	Lru.java	Practical.cs	ch25-lru.mjs	lru.py	ch25_lru.lua
skiplist	ch 25	skiplist.c	skiplist.go	Skiplist.java	Practical.cs	ch25-skiplist.mjs	skiplist.py	ch25_skiplist.lua
asteval	ch 26	asteval.c	asteval.go	Asteval.java	Parsing.cs	ch26-asteval.mjs	asteval.py	ch26_asteval.lua
rdparse	ch 26	rdparse.c	rdparse.go	Rdparse.java	Parsing.cs	ch26-rdparse.mjs	rdparse.py	ch26_rdparse.lua
tokens	ch 26	tokens.c	tokens.go	Tokens.java	Parsing.cs	ch26-tokens.mjs	tokens.py	ch26_tokens.lua
jumps	ch 27	jumps.c	jumps.go	Jumps.java	Machine.cs	ch27-jumps.mjs	jumps.py	ch27_jumps.lua
modes	ch 27	modes.c	modes.go	Modes.java	Machine.cs	ch27-modes.mjs	modes.py	ch27_modes.lua
opcodes	ch 27	opcodes.c	opcodes.go	Opcodes.java	Machine.cs	ch27-opcodes.mjs	opcodes.py	ch27_opcodes.lua
cfrac	ch 28	cfrac.c	cfrac.go	Cfrac.java	CFrac.cs	ch28-cfrac.mjs	cfrac.py	ch28_cfrac.lua
diophantine	ch 28	diophantine.c	diophantine.go	Diophantine.java	Diophantine.cs	ch28-diophantine.mjs	diophantine.py	ch28_diophantine.lua
dlog	ch 28	dlog.c	dlog.go	Dlog.java	DLog.cs	ch28-dlog.mjs	dlog.py	ch28_dlog.lua
garner	ch 28	garner.c	garner.go	Garner.java	Garner.cs	ch28-garner.mjs	garner.py	ch28_garner.lua
graytern	ch 28	graytern.c	graytern.go	Graytern.java	GrayTern.cs	ch28-graytern.mjs	graytern.py	ch28_graytern.lua
linsieve	ch 28	linsieve.c	linsieve.go	Linsieve.java	LinSieve.cs	ch28-linsieve.mjs	linsieve.py	ch28_linsieve.lua
primality	ch 28	primality.c	primality.go	Primality.java	Primality.cs	ch28-primality.mjs	primality.py	ch28_primality.lua
totdiv	ch 28	totdiv.c	totdiv.go	Totdiv.java	TotDiv.cs	ch28-totdiv.mjs	totdiv.py	ch28_totdiv.lua
bigint	ch 29	bigint.c	bigint.go	Bigint.java	BigNum.cs	ch29-bigint.mjs	bigint.py	ch29_bigint.lua
divmod	ch 29	divmod.c	divmod.go	Divmod.java	DivMod.cs	ch29-divmod.mjs	divmod.py	ch29_divmod.lua
fft	ch 29	fft.c	fft.go	Fft.java	Fft.cs	ch29-fft.mjs	fft.py	ch29_fft.lua
karatsuba	ch 29	karatsuba.c	karatsuba.go	Karatsuba.java	Karatsuba.cs	ch29-karatsuba.mjs	karatsuba.py	ch29_karatsuba.lua
ntt	ch 29	ntt.c	ntt.go	Ntt.java	Ntt.cs	ch29-ntt.mjs	ntt.py	ch29_ntt.lua
polyops	ch 29	polyops.c	polyops.go	Polyops.java	PolyOps.cs	ch29-polyops.mjs	polyops.py	ch29_polyops.lua
mergeheap	ch 30	mergeheap.c	mergeheap.go	Mergeheap.java	MergeHeap.cs	ch30-mergeheap.mjs	mergeheap.py	ch30_mergeheap.lua
orderstats	ch 30	orderstats.c	orderstats.go	Orderstats.java	OrderStats.cs	ch30-orderstats.mjs	orderstats.py	ch30_orderstats.lua
treap	ch 30	treap.c	treap.go	Treap.java	Treap.cs	ch30-treap.mjs	treap.py	ch30_treap.lua
disjointsparse	ch 31	disjointsparse.c	disjoint_sparse.go	DisjointSparse.java	DisjointSparse.cs	ch31-disjointsparse.mjs	disjointsparse.py	ch31_disjointsparse.lua
lichao	ch 31	lichao.c	lichao.go	Lichao.java	LiChao.cs	ch31-lichao.mjs	lichao.py	ch31_lichao.lua
minstack	ch 31	minstack.c	minstack.go	Minstack.java	MinStack.cs	ch31-minstack.mjs	minstack.py	ch31_minstack.lua
mos	ch 31	mos.c	mos.go	Mos.java	Mos.cs	ch31-mos.mjs	mos.py	ch31_mos.lua
offlinedynconn	ch 31	offlinedynconn.c	offlinedynconn.go	Offlinedynconn.java	OfflineDynConn.cs	ch31-offlinedynconn.mjs	offlinedynconn.py	ch31_offlinedynconn.lua
rollbackdsu	ch 31	rollbackdsu.c	rollbackdsu.go	Rollbackdsu.java	RollbackDsu.cs	ch31-rollbackdsu.mjs	rollbackdsu.py	ch31_rollbackdsu.lua
sqrtblocks	ch 31	sqrtblocks.c	sqrtblocks.go	Sqrtblocks.java	SqrtBlocks.cs	ch31-sqrtblocks.mjs	sqrtblocks.py	ch31_sqrtblocks.lua
brokenprofile	ch 32	brokenprofile.c	brokenprofile.go	Brokenprofile.java	BrokenProfile.cs	ch32-brokenprofile.mjs	brokenprofile.py	ch32_brokenprofile.lua
dncdp	ch 32	dncdp.c	dncdp.go	Dncdp.java	DncDp.cs	ch32-dncdp.mjs	dncdp.py	ch32_dncdp.lua
knuth	ch 32	knuth.c	knuth.go	Knuth.java	Knuth.cs	ch32-knuth.mjs	knuth.py	ch32_knuth.lua
slopetrick	ch 32	slopetrick.c	slopetrick.go	Slopetrick.java	SlopeTrick.cs	ch32-slopetrick.mjs	slopetrick.py	ch32_slopetrick.lua
valueiter	ch 32	valueiter.c	valueiter.go	Valueiter.java	ValueIter.cs	ch32-valueiter.mjs	valueiter.py	ch32_valueiter.lua
lyndon	ch 33	lyndon.c	lyndon.go	Lyndon.java	Lyndon.cs	ch33-lyndon.mjs	lyndon.py	ch33_lyndon.lua
manacher	ch 33	manacher.c	manacher.go	Manacher.java	Manacher.cs	ch33-manacher.mjs	manacher.py	ch33_manacher.lua
suffixautomaton	ch 33	suffixautomaton.c	suffixautomaton.go	Suffixautomaton.java	SuffixAutomaton.cs	ch33-suffixautomaton.mjs	suffixautomaton.py	ch33_suffixautomaton.lua
suffixtree	ch 33	suffixtree.c	suffixtree.go	Suffixtree.java	SuffixTree.cs	ch33-suffixtree.mjs	suffixtree.py	ch33_suffixtree.lua
zfunc	ch 33	zfunc.c	zfunc.go	Zfunc.java	Zfunc.cs	ch33-zfunc.mjs	zfunc.py	ch33_zfunc.lua
cramer	ch 34	cramer.c	cramer.go	Cramer.java	Cramer.cs	ch34-cramer.mjs	cramer.py	ch34_cramer.lua

topic	ch	c	go	java	c#	javascript	python	lua
determinant	ch 34	determinant.c	determinant.go	Determinant.java	Determinant.cs	ch34-determinant.mjs	determinant.py	ch34_determinant.lua
gauss	ch 34	gauss.c	gauss.go	Gauss.java	Gauss.cs	ch34-gauss.mjs	gauss.py	ch34_gauss.lua
kirchhoff	ch 34	kirchhoff.c	kirchhoff.go	Kirchhoff.java	Kirchhoff.cs	ch34-kirchhoff.mjs	kirchhoff.py	ch34_kirchhoff.lua
matpow	ch 34	matpow.c	matpow.go	Matpow.java	Matpow.cs	ch34-matpow.mjs	matpow.py	ch34_matpow.lua
zpk	ch 34	zpk.c	zpk.go	Zpk.java	Zpk.cs	ch34-zpk.mjs	zpk.py	ch34_zpk.lua
binom	ch 35	binom.c	binom.go	Binom.java	Binom.cs	ch35-binom.mjs	binom.py	ch35_binom.lua
bishops	ch 35	bishops.c	bishops.go	Bishops.java	Bishops.cs	ch35-bishops.mjs	bishops.py	ch35_bishops.lua
brackets	ch 35	brackets.c	brackets.go	Brackets.java	Brackets.cs	ch35-brackets.mjs	brackets.py	ch35_brackets.lua
burnside	ch 35	burnside.c	burnside.go	Burnside.java	Burnside.cs	ch35-burnside.mjs	burnside.py	ch35_burnside.lua
catalan	ch 35	catalan.c	catalan.go	Catalan.java	Catalan.cs	ch35-catalan.mjs	catalan.py	ch35_catalan.lua
frobenius	ch 35	frobenius.c	frobenius.go	Frobenius.java	Frobenius.cs	ch35-frobenius.mjs	frobenius.py	ch35_frobenius.lua
incexc	ch 35	incexc.c	incexc.go	Incexc.java	IncExc.cs	ch35-incexc.mjs	incexc.py	ch35_incexc.lua
josephus	ch 35	josephus.c	josephus.go	Josephus.java	Josephus.cs	ch35-josephus.mjs	josephus.py	ch35_josephus.lua
ksubset	ch 35	ksubset.c	ksubset.go	Ksubset.java	KSubset.cs	ch35-ksubset.mjs	ksubset.py	ch35_ksubset.lua
labeled	ch 35	labeled.c	labeled.go	Labeled.java	Labeled.cs	ch35-labeled.mjs	labeled.py	ch35_labeled.lua
sperner	ch 35	sperner.c	sperner.go	Sperner.java	Sperner.cs	ch35-sperner.mjs	sperner.py	ch35_sperner.lua
starsbars	ch 35	starsbars.c	starsbars.go	Starsbars.java	StarsBars.cs	ch35-starsbars.mjs	starsbars.py	ch35_starsbars.lua
golden	ch 36	golden.c	golden.go	Golden.java	Golden.cs	ch36-golden.mjs	golden.py	ch36_golden.lua
lpdual	ch 36	lpdual.c	lpdual.go	Lpdual.java	LpDual.cs	ch36-lpdual.mjs	lpdual.py	ch36_lpdual.lua
newton	ch 36	newton.c	newton.go	Newton.java	Newton.cs	ch36-newton.mjs	newton.py	ch36_newton.lua
simpson	ch 36	simpson.c	simpson.go	Simpson.java	Simpson.cs	ch36-simpson.mjs	simpson.py	ch36_simpson.lua
ternary	ch 36	ternary.c	ternary.go	Ternary.java	Ternary.cs	ch36-ternary.mjs	ternary.py	ch36_ternary.lua
calipers	ch 37	calipers.c	calipers.go	Calipers.java	Calipers.cs	ch37-calipers.mjs	calipers.py	ch37_calipers.lua
circles	ch 37	circles.c	circles.go	Circles.java	Geometry2.cs	ch37-circles.mjs	circles.py	ch37_circles.lua
halfplane	ch 37	halfplane.c	halfplane.go	Halfplane.java	Calipers.cs	ch37-halfplane.mjs	halfplane.py	ch37_halfplane.lua
manhattan	ch 37	manhattan.c	manhattan.go	Manhattan.java	Calipers.cs	ch37-manhattan.mjs	manhattan.py	ch37_manhattan.lua
mincircle	ch 37	mincircle.c	mincircle.go	Mincircle.java	Calipers.cs	ch37-mincircle.mjs	mincircle.py	ch37_mincircle.lua
minkowski	ch 37	minkowski.c	minkowski.go	Minkowski.java	Geometry2.cs	ch37-minkowski.mjs	minkowski.py	ch37_minkowski.lua
ottmann	ch 37	ottmann.c	ottmann.go	Ottmann.java	Voronoi.cs	ch37-ottmann.mjs	ottmann.py	ch37_ottmann.lua
pinconvex	ch 37	pinconvex.c	pinconvex.go	Pinconvex.java	Geometry2.cs	ch37-pinconvex.mjs	pinconvex.py	ch37_pinconvex.lua
segunion	ch 37	segunion.c	segunion.go	Segunion.java	Geometry2.cs	ch37-segunion.mjs	segunion.py	ch37_segunion.lua
voronoi	ch 37	voronoi.c	voronoi.go	Voronoi.java	Voronoi.cs	ch37-voronoi.mjs	voronoi.py	ch37_voronoi.lua
assign	ch 38	assign.c	assign.go	Assign.java	Hungarian.cs	ch38-assign.mjs	assign.py	ch38_assign.lua
demands	ch 38	demands.c	demands.go	Demands.java	Costs2.cs	ch38-demands.mjs	demands.py	ch38_demands.lua
dinic	ch 38	dinic.c	dinic.go	Dinic.java	Flow2.cs	ch38-dinic.mjs	dinic.py	ch38_dinic.lua
hall	ch 38	hall.c	hall.go	Hall.java	Match.cs	ch38-hall.mjs	hall.py	ch38_hall.lua
hungarian	ch 38	hungarian.c	hungarian.go	Hungarian.java	Hungarian.cs	ch38-hungarian.mjs	hungarian.py	ch38_hungarian.lua
kuhn	ch 38	kuhn.c	kuhn.go	Kuhn.java	Match.cs	ch38-kuhn.mjs	kuhn.py	ch38_kuhn.lua
mcmf	ch 38	mcmf.c	mcmf.go	Mcmf.java	Costs2.cs	ch38-mcmf.mjs	mcmf.py	ch38_mcmf.lua
pushrelabel	ch 38	pushrelabel.c	pushrelabel.go	Pushrelabel.java	Flow2.cs	ch38-pushrelabel.mjs	pushrelabel.py	ch38_pushrelabel.lua
stoerwagner	ch 38	stoerwagner.c	stoerwagner.go	Stoerwagner.java	Cuts.cs	ch38-stoerwagner.mjs	stoerwagner.py	ch38_stoerwagner.lua
bridges	ch 39	bridges.c	bridges.go	Bridges.java	Edges.cs	ch39-bridges.mjs	bridges.py	ch39_bridges.lua
centroid	ch 39	centroid.c	centroid.go	Centroid.java	Centers.cs	ch39-centroid.mjs	centroid.py	ch39_centroid.lua
euler	ch 39	euler.c	euler.go	Euler.java	Edges.cs	ch39-euler.mjs	euler.py	ch39_euler.lua
functional	ch 39	functional.c	functional.go	Functional.java	Centers.cs	ch39-functional.mjs	functional.py	ch39_functional.lua
hld	ch 39	hld.c	hld.go	Hld.java	Paths2.cs	ch39-hld.mjs	hld.py	ch39_hld.lua
lca	ch 39	lca.c	lca.go	Lca.java	Paths2.cs	ch39-lca.mjs	lca.py	ch39_lca.lua
prufer	ch 39	prufer.c	prufer.go	Prufer.java	Prufer.cs	ch39-prufer.mjs	prufer.py	ch39_prufer.lua
scc	ch 39	scc.c	scc.go	Scc.java	Strong.cs	ch39-scc.mjs	scc.py	ch39_scc.lua
twocore	ch 39	twocore.c	twocore.go	Twocore.java	Centers.cs	ch39-twocore.mjs	twocore.py	ch39_twocore.lua
twosat	ch 39	twosat.c	twosat.go	Twosat.java	Strong.cs	ch39-twosat.mjs	twosat.py	ch39_twosat.lua
graphgames	ch 40	graphgames.c	graphgames.go	Graphgames.java	GraphGames.cs	ch40-graphgames.mjs	graphgames.py	ch40_graphgames.lua

topic	ch	c	go	java	c#	javascript	python	lua
grundy	ch 40	grundy.c	grundy.go	Grundy.java	Grundy.cs	ch40-grundy.mjs	grundy.py	ch40_grundy.lua
scheduling	ch 40	scheduling.c	scheduling.go	Scheduling.java	Scheduling.cs	ch40-scheduling.mjs	scheduling.py	ch40_scheduling.lua
aco	ch 41	aco.c	aco.go	Aco.java	Aco.cs	ch41-aco.mjs	aco.py	ch41_aco.lua
anneal	ch 41	anneal.c	anneal.go	Anneal.java	Anneal.cs	ch41-anneal.mjs	anneal.py	ch41_anneal.lua
ga	ch 41	ga.c	ga.go	Ga.java	Ga.cs	ch41-ga.mjs	ga.py	ch41_ga.lua
hill	ch 41	hill.c	hill.go	Hill.java	Hill.cs	ch41-hill.mjs	hill.py	ch41_hill.lua
localsearch	ch 41	localsearch.c	localsearch.go	Localsearch.java	Localsearch.cs	ch41-localsearch.mjs	localsearch.py	ch41_localsearch.lua
restarts	ch 41	restarts.c	restarts.go	Restarts.java	Restarts.cs	ch41-restarts.mjs	restarts.py	ch41_restarts.lua
tabu	ch 41	tabu.c	tabu.go	Tabu.java	Tabu.cs	ch41-tabu.mjs	tabu.py	ch41_tabu.lua
twoopt	ch 41	twoopt.c	twoopt.go	Twoopt.java	TwoOpt.cs	ch41-twoopt.mjs	twoopt.py	ch41_twoopt.lua
binpack	ch 42	binpack.c	binpack.go	Binpack.java	Binpack.cs	ch42-binpack.mjs	binpack.py	ch42_binpack.lua
christofides	ch 42	christofides.c	christofides.go	Christofides.java	Christofides.cs	ch42-christofides.mjs	christofides.py	ch42_christofides.lua
maxcut	ch 42	maxcut.c	maxcut.go	Maxcut.java	Maxcut.cs	ch42-maxcut.mjs	maxcut.py	ch42_maxcut.lua
ptas	ch 42	ptas.c	ptas.go	Ptas.java	Ptas.cs	ch42-ptas.mjs	ptas.py	ch42_ptas.lua
ratio	ch 42	ratio.c	ratio.go	Ratio.java	Ratio.cs	ch42-ratio.mjs	ratio.py	ch42_ratio.lua
setcover	ch 42	setcover.c	setcover.go	Setcover.java	Setcover.cs	ch42-setcover.mjs	setcover.py	ch42_setcover.lua
tsp2x	ch 42	tsp2x.c	tsp2x.go	Tsp2x.java	Tsp2x.cs	ch42-tsp2x.mjs	tsp2x.py	ch42_tsp2x.lua
vc	ch 42	vc.c	vc.go	Vc.java	Vc.cs	ch42-vc.mjs	vc.py	ch42_vc.lua

language	suite size	unit	gate
c	222 files, 4140	checks, one ok line per file	`make verify-c` scope `books/dsa/samples-c/src`
go	724	tests, go test	`make verify-go`
java	222 files, 4269	checks, one ok line per file	`make verify-java` scope `books/dsa/samples-java/src`
c#	784	tests, dotnet test	`make verify-csharp`
javascript	755	tests, node test runner	`node --test` over `test/*.test.mjs`
python	222 files, 3007	asserts, one ok line per file	`make verify-py` scope `books/dsa/samples-py/src`
lua	936	checks, the run.lua runner	`make verify-lua`

The icpc toolbox suites ride beside these in book 10 and share the same six toolchains, 26 c# tests, 36 javascript tests, 67 python asserts, and 45 lua checks over 6 to 10 files per language.

44.3. pinned sources

Every chapter ends with its own sources line naming the official pages consulted while writing it. The book level pins are:

topic	url	accessed
collections and spans	https://learn.microsoft.com/en-us/dotnet/standard/collections/	2026-09-08
priorityqueue with remove and fused ops	https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic.priorityqueue-2?view=net-10.0	2026-09-08
what is new in .net 9 libraries, decrease-key	https://learn.microsoft.com/en-us/dotnet/core/whats-new/dotnet-9/libraries	2026-09-08
memory and span guidance	https://learn.microsoft.com/en-us/dotnet/standard/memory-and-spans/	2026-09-08
minimal apis fundamentals	https://learn.microsoft.com/en-us/aspnet/core/fundamentals/minimal-apis	2026-09-08

topic	url	accessed
webapplicationfactory integration tests	https://learn.microsoft.com/en-us/aspnet/core/test/integration-tests	2026-09-08
htmx attribute reference and docs	https://htmx.org/docs/	2026-09-08
htmx.org 4.0.0 package and integrity digest	https://www.npmjs.com/package/htmx.org/v/4.0.0	2026-09-08
benchmarkdotnet	https://benchmarkdotnet.org/	2026-09-08
runtime probes on net 10.0.400	dotnet 10.0.400 sdk, comparer and raw string probes run locally	2026-09-08
0-1 bfs	https://cp-algorithms.com/graph/01_bfs.html	2026-09-20
d'esopo-pape algorithm	https://cp-algorithms.com/graph/desopo_pape.html	2026-09-20
treap (cartesian tree)	https://cp-algorithms.com/data_structures/treap.html	2026-09-20
randomized heap	https://cp-algorithms.com/data_structures/randomized_heap.html	2026-09-20
fenwick tree	https://cp-algorithms.com/data_structures/fenwick.html	2026-09-20
k-th order statistic in o(n)	https://cp-algorithms.com/sequences/k-th.html	2026-09-20
sqrt decomposition	https://cp-algorithms.com/data_structures/sqrt_decomposition.html	2026-09-20
minimum stack / minimum queue	https://cp-algorithms.com/data_structures/stack_queue_modification.html	2026-09-20
convex hull trick and li chao tree	https://cp-algorithms.com/geometry/convex_hull_trick.html	2026-09-20
sparse table	https://cp-algorithms.com/data_structures/sparse-table.html	2026-09-20
disjoint set union	https://cp-algorithms.com/data_structures/disjoint_set_union.html	2026-09-20
divide and conquer dp	https://cp-algorithms.com/dynamic_programming/divide-and-conquer-dp.html	2026-09-20
knuth's optimization	https://cp-algorithms.com/dynamic_programming/knuth-optimization.html	2026-09-20
dynamic programming on broken profile, problem parquet	https://cp-algorithms.com/dynamic_programming/profile-dynamics.html	2026-09-20
finding the largest zero submatrix	https://cp-algorithms.com/dynamic_programming/zero_matrix.html	2026-09-20
z-function	https://cp-algorithms.com/string/z-function.html	2026-09-20
manacher's algorithm, finding all sub-palindromes in o(n)	https://cp-algorithms.com/string/manacher.html	2026-09-20
suffix automaton	https://cp-algorithms.com/string/suffix-automaton.html	2026-09-20
suffix tree	https://cp-algorithms.com/string/suffix-tree-ukkonen.html	2026-09-20
lyndon factorization	https://cp-algorithms.com/string/lyndon_factorization.html	2026-09-20
finding repetitions	https://cp-algorithms.com/string/main_lorentz.html	2026-09-20
gauss and system of linear equations	https://cp-algorithms.com/linear_algebra/linear-system-gauss.html	2026-09-20

topic	url	accessed
kraut and determinant	https://cp-algorithms.com/linear_algebra/determinant-kraut.html	2026-09-20
rank of a matrix	https://cp-algorithms.com/linear_algebra/rank-matrix.html	2026-09-20
binary exponentiation	https://cp-algorithms.com/algebra/binary-exp.html	2026-09-20
kirchhoff theorem	https://cp-algorithms.com/graph/kirchhoff-theorem.html	2026-09-20
binomial coefficients	https://cp-algorithms.com/combinatorics/binomial-coefficients.html	2026-09-20
catalan numbers	https://cp-algorithms.com/combinatorics/catalan-numbers.html	2026-09-20
stars and bars	https://cp-algorithms.com/combinatorics/stars_and_bars.html	2026-09-20
the inclusion-exclusion principle	https://cp-algorithms.com/combinatorics/inclusion-exclusion.html	2026-09-20
burnside's lemma / polya enumeration theorem	https://cp-algorithms.com/combinatorics/burnside.html	2026-09-20
generating all k-combinations	https://cp-algorithms.com/combinatorics/generating_combinations.html	2026-09-20
enumerating submasks of a bitmask	https://cp-algorithms.com/algebra/all-submasks.html	2026-09-20
placing bishops on a chessboard	https://cp-algorithms.com/combinatorics/bishops-on-chessboard.html	2026-09-20
balanced bracket sequences	https://cp-algorithms.com/combinatorics/bracket_sequences.html	2026-09-20
counting labeled graphs	https://cp-algorithms.com/combinatorics/counting_labeled_graphs.html	2026-09-20
josephus problem	https://cp-algorithms.com/others/josephus_problem.html	2026-09-20
15 puzzle game, existence of the solution	https://cp-algorithms.com/others/15-puzzle.html	2026-09-20
ternary search	https://cp-algorithms.com/num_methods/ternary_search.html	2026-09-20
newton's method for finding roots	https://cp-algorithms.com/num_methods/roots_newton.html	2026-09-20
integration by simpson's formula	https://cp-algorithms.com/num_methods/simpson-integration.html	2026-09-20
simulated annealing	https://cp-algorithms.com/num_methods/simulated_annealing.html	2026-09-20
lp duality for icpc 2025 c	icpc 2025 solutions.pdf, local cache ref/icpc/2025	2026-09-20
circle-line intersection	https://cp-algorithms.com/geometry/circle-line-intersection.html	2026-09-20
circle-circle intersection	https://cp-algorithms.com/geometry/circle-circle-intersection.html	2026-09-20
finding common tangents to two circles	https://cp-algorithms.com/geometry/tangents-to-two-circles.html	2026-09-20
length of the union of segments	https://cp-algorithms.com/geometry/length-of-segments-union.html	2026-09-20

topic	url	accessed
check if point belongs to the convex polygon in o(log n)	https://cp-algorithms.com/geometry/point-in-convex-polygon.html	2026-09-20
minkowski sum of convex polygons	https://cp-algorithms.com/geometry/minkowski.html	2026-09-20
convex hull construction	https://cp-algorithms.com/geometry/convex-hull.html	2026-09-20
half-plane intersection, s and i algorithm in o(n log n)	https://cp-algorithms.com/geometry/halfplane-intersection.html	2026-09-20
minimum enclosing circle	https://cp-algorithms.com/geometry/enclosing-circle.html	2026-09-20
delaunay triangulation and voronoi diagram	https://cp-algorithms.com/geometry/delaunay.html	2026-09-20
search for a pair of intersecting segments	https://cp-algorithms.com/geometry/intersecting_segments.html	2026-09-20
finding faces of a planar graph	https://cp-algorithms.com/geometry/planar.html	2026-09-20
point location in o(log n)	https://cp-algorithms.com/geometry/point-location.html	2026-09-20
manhattan distance	https://cp-algorithms.com/geometry/manhattan-distance.html	2026-09-20
clrs third edition chapter 33, calipers and ottmann	cormen, leiserson, rivest, stein, introduction to algorithms 3e, ch 33	2026-09-20
maximum flow, dinic's algorithm	https://cp-algorithms.com/graph/dinic.html	2026-09-20
maximum flow, push-relabel algorithm	https://cp-algorithms.com/graph/push-relabel.html	2026-09-20
minimum-cost flow	https://cp-algorithms.com/graph/min_cost_flow.html	2026-09-20
flows with demands	https://cp-algorithms.com/graph/flow_with_demands.html	2026-09-20
minimum cut, stoer-wagner algorithm	https://cp-algorithms.com/graph/stoer_wagner_mincut.html	2026-09-20
kuhn's algorithm, maximum bipartite matching	https://cp-algorithms.com/graph/kuhn_maximum_bipartite_matching.html	2026-09-20
hungarian algorithm	https://cp-algorithms.com/graph/hungarian-algorithm.html	2026-09-20
assignment problem	https://cp-algorithms.com/graph/Assignment-problem-min-flow.html	2026-09-20
strongly connected components and condensation graph	https://cp-algorithms.com/graph/strongly-connected-components.html	2026-09-20
2-sat	https://cp-algorithms.com/graph/2SAT.html	2026-09-20
finding bridges in o(n+m)	https://cp-algorithms.com/graph/bridge-searching.html	2026-09-20
finding articulation points in o(n+m)	https://cp-algorithms.com/graph/cutpoints.html	2026-09-20
finding bridges online	https://cp-algorithms.com/graph/bridge-searching-online.html	2026-09-20
eulerian path	https://cp-algorithms.com/graph/euler_path.html	2026-09-20
lowest common ancestor	https://cp-algorithms.com/graph/lca.html	2026-09-20
lowest common ancestor, binary lifting	https://cp-algorithms.com/graph/lca_binary_lifting.html	2026-09-20
lowest common ancestor, tarjan's off-line algorithm	https://cp-algorithms.com/graph/lca_tarjan.html	2026-09-20
solve rmq by finding lca	https://cp-algorithms.com/graph/rmq_linear.html	2026-09-20
heavy-light decomposition	https://cp-algorithms.com/graph/hld.html	2026-09-20

topic	url	accessed
centroid decomposition	https://cp-algorithms.com/graph/centroid_decomposition.html	2026-09-20
prufer code	https://cp-algorithms.com/graph/pruefer_code.html	2026-09-20
second best minimum spanning tree	https://cp-algorithms.com/graph/second_best_mst.html	2026-09-20
strong orientation	https://cp-algorithms.com/graph/strong-orientation.html	2026-09-20
sprague-grundy theorem, nim	https://cp-algorithms.com/game_theory/sprague-grundy-nim.html	2026-09-20
games on arbitrary graphs	https://cp-algorithms.com/game_theory/games_on_graphs.html	2026-09-20
scheduling jobs on one machine	https://cp-algorithms.com/schedules/schedule_one_machine.html	2026-09-20
scheduling jobs on two machines	https://cp-algorithms.com/schedules/schedule_two_machines.html	2026-09-20
optimal schedule of jobs given their deadlines and durations	https://cp-algorithms.com/schedules/schedule-with-completion-duration.html	2026-09-20
search the subsegment with the maximum or minimum sum	https://cp-algorithms.com/others/maximum_average_segment.html	2026-09-20
mex task, minimal excluded element in an array	https://cp-algorithms.com/sequences/mex.html	2026-09-20
linear sieve	https://cp-algorithms.com/algebra/prime-sieve-linear.html	2026-09-20
euler's totient function	https://cp-algorithms.com/algebra/phi-function.html	2026-09-20
primality tests	https://cp-algorithms.com/algebra/primality_tests.html	2026-09-20
integer factorization	https://cp-algorithms.com/algebra/factorization.html	2026-09-20
number of divisors and sum of divisors	https://cp-algorithms.com/algebra/divisors.html	2026-09-20
discrete logarithm	https://cp-algorithms.com/algebra/discrete-log.html	2026-09-20
primitive root	https://cp-algorithms.com/algebra/primitive-root.html	2026-09-20
garner's algorithm	https://cp-algorithms.com/algebra/garners-algorithm.html	2026-09-20
factorial modulo p	https://cp-algorithms.com/algebra/factorial-modulo.html	2026-09-20
linear diophantine equation	https://cp-algorithms.com/algebra/linear-diophantine-equation.html	2026-09-20
extended Euclidean algorithm	https://cp-algorithms.com/algebra/extended-euclid-algorithm.html	2026-09-20
gray code	https://cp-algorithms.com/algebra/gray-code.html	2026-09-20
balanced ternary	https://cp-algorithms.com/algebra/balanced-ternary.html	2026-09-20
continued fractions	https://cp-algorithms.com/algebra/continued-fractions.html	2026-09-20
the stern-brocot tree and farey sequences	https://cp-algorithms.com/others/stern_brocot_tree_farey_sequences.html	2026-09-20
hardy and wright for mobius	hardy and wright, an introduction to the theory of numbers, mu cited by name	2026-09-20
arbitrary-precision arithmetic	https://cp-algorithms.com/algebra/big-integer.html	2026-09-20

topic	url	accessed
fast fourier transform	https://cp-algorithms.com/algebra/fft.html	2026-09-20
operations on polynomials and series	https://cp-algorithms.com/algebra/polynomial.html	2026-09-20
karatsuba and ofman 1962	karatsuba and ofman, multiplication of multidigit numbers on automata, 1962	2026-09-20
knuth taocp v2 4.3.1 algorithm d	knuth, the art of computer programming v2, 4.3.1 algorithm d	2026-09-20
aarts and lenstra, local search in combinatorial optimization	aarts and lenstra (eds), local search in combinatorial optimization, wiley 1997	2026-09-23
kirkpatrick gelatt vecchi, optimization by simulated annealing	kirkpatrick, gelatt, and vecchi, optimization by simulated annealing, science 220, 1983	2026-09-23
glover, tabu search part i	glover, tabu search: part i, orsa journal on computing 1, 1989	2026-09-23
goldberg, genetic algorithms	goldberg, genetic algorithms in search, optimization, and machine learning, addison-wesley 1989	2026-09-23
dorigo maniezzo colorni, ant system	dorigo, maniezzo, and colorni, ant system: optimization by a colony of cooperating agents, ieee tsmc 1996	2026-09-23
stutzle and hoos, max-min ant system	stutzle and hoos, max-min ant system, future generation computer systems 16, 2000	2026-09-23
knuth taocp v2, linear congruential constants	knuth, the art of computer programming v2, seminumerical algorithms, 64-bit lcg recommendations	2026-09-23
vazirani, approximation algorithms	vazirani, approximation algorithms, springer 2001	2026-09-23
williamson and shmoys, the design of approximation algorithms	williamson and shmoys, the design of approximation algorithms, cambridge 2011	2026-09-23
clrs 3e chapter 35, approximation algorithms	cormen, leiserson, rivest, and stein, introduction to algorithms 3e, ch 35.1-35.5	2026-09-23
slavik, greedy set cover tightness	slavik, a tight analysis of the greedy algorithm for set cover, journal of algorithms 25, 1997	2026-09-23
johnson 1973 and dosa 2007, bin packing bounds	johnson, near-optimal bin packing algorithms, mit thesis 1973; dosa, the tight bound of first fit decreasing, 2007	2026-09-23

The six-language wave pinned six toolchains, verified 2026-09-14 against the gates above, and the java wave adds the seventh, verified 2026-10-06:

topic	pin	url	accessed
c	clang 23, `-std=c23`, msvc headers, lld	releases.llvm.org	2026-09-14
go	go 1.27	go.dev/dl	2026-09-14
java	oracle jdk 27, build 27+35-2325	download.oracle.com/java/27	2026-10-06
c#	dotnet sdk 10.0.401, net10.0	dotnet.microsoft.com	2026-09-14
javascript	node 26.3.0, esm	nodejs.org	2026-09-14
python	cpython 3.14.7	python.org	2026-09-14
lua	lua 5.5, built as tools/lua55	lua.org	2026-09-14

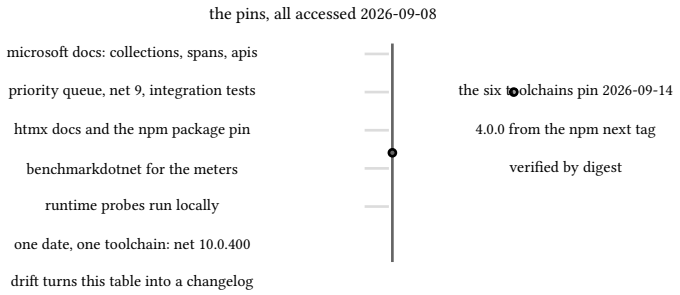


figure 394: the pin set on its access timeline, ten pins in four groups, all on one date

NOTE: version drift rule

These pins were verified 2026-09-08 against .NET 10.0.400 and the language version it ships, with htmx 4.0.0 taken from the npm `next` tag and vendored by digest. Findings that contradicted documentation, the absent BCL deque, the silent tolerance of inconsistent comparers, the single-dollar raw string brace rule, the floor-mid probe asymmetry, are recorded in their chapters as verified behavior of this toolchain, and this table is the changelog if a future release moves any of them.

NOTE: cross-language pins from the six-language wave

Four findings are pinned at book level because they differ by language on purpose. C's mulmod runs add-and-double with a modulo every step because the 128-bit remainder helper fails to link on this toolchain, recorded in the chapter 16 listing header. The chinese remainder implementations split policy, python refuses every non-coprime system while go merges the consistent ones, chapter 16 names the difference. The skip list promotions are deterministic in six languages with four stated policies, java joining the ruler sequence, chapter 25. And chapter 22's interval convention is half-open where c# and lua pin it, touching-fuses in the other five.